

Totem

User Manual

Software Release 2019R2

Manual Version: Production

Copyright © March 24, 2020

ANSYS, Inc.

Copyright Notice and Proprietary Information

No part of this document may be reproduced or transmitted in any form or by any means, electronic, or mechanical, for any purpose, without the express written permission of ANSYS Inc. This manual and the program described in it are owned by ANSYS Inc. and may be used only as authorized in the license agreement controlling such use, and may not be copied except in accordance with the terms of this agreement.

Copyright © 2013, 2014, 2015, 2016; ANSYS INC (Semiconductor Business Unit). All rights reserved.

Disclaimer

ANSYS INC makes no warranty of any kind, expressed or implied, with respect to software or documentation, its quality, or performance. The information in this document is subject to change without notice and does not represent a commitment on the part of ANSYS INC.

Trademarks

ANSYS and any and all ANSYS INC brand, product, service and feature names, logos and slogans are registered trademarks or trademarks of ANSYS, INC. or its subsidiaries in the United States or other countries. All other trademarks referred to are the property of their registered owners.

ANSYS, Inc.

2645 Zanker Road

San Jose, CA 95134

Tel:(408) 457-2000

Fax: (408) 428-9569

apache_sales@apache-da.com

support_center@apache-da.com

www.apache-da.com

TABLE OF CONTENTS

Chapter 1 - Introduction

Silicon Integrity Analysis 1-1
Using Totem in the Design Flow 1-2
Early Design Analysis 1-3
Design Verification 1-4
Summary 1-4

Chapter 2 - General Setup and Flow

Introduction 2-5
Support of Multiple Design Styles 2-5
Totem Flow Overview 2-6
Introduction 2-6
Static and Dynamic Analysis Design Input Data Requirements 2-7
Totem Data Preparation 2-7
Probe Generation 2-8
Running Spice Simulation 2-8
Generating Electrical models in APLMMX 2-9
Preparing the physical models 2-9
Totem Flow Description 2-9
GDS-based Setup 2-9
Layout only (Grid-Check, Static) 2-9
Netlist-based Setup 2-10
Layout + DSPF/Layout netlist (Static) 2-10
Layout + Schematic netlist + DSPF/Layout netlist (Static/Dynamic) 2-10
With Layout netlist for Decap co-ordinates 2-10
LVS-based Flows 2-11
Hercules-based flow 2-11
APLMMX Data Preparation 2-12
Requirements for Spice Netlists 2-12
Probe generation from the netlist 2-13
Requirements for DSPF file (Star/Calibre/Assura) 2-13
Coordinate Requirements in DSPF 2-14
Sample DSPF format 2-14
Device mapping between DSPF and the schematic netlist 2-15
Simulation Requirements and Settings 2-16
Current polarity requirements 2-16
HSpice Simulator 2-17
HSIM Simulator 2-17
XA Simulator 2-18
UltraSim Simulator 2-18
Spectre Simulator 2-18
Eldo Simulator 2-18
NanoSim Simulator 2-19

Finesim Simulator 2-19
GDS Processing Setup 2-19
Introduction 2-19
GDS2DB Configuration File 2-20
Overview 2-20
Nets definition 2-20
GDS2DB Layer Map file 2-21
Mini-LVS Functions 2-23
Layer extraction and device recognition 2-23
Logical Layer Operations 2-24
Support of LDMOS marker layer 2-24
Tap Cell Detection 2-25
Pin Snapping 2-25
General Notes on GDS Handling 2-25
Special GDS2DB Steps 2-25
PLOC Automation Procedure 2-25
OASIS Processing 2-25
Introduction 2-26
Input Data Preparation 2-26
GSR keywords 2-26

Chapter 3 - Characterization and Modeling

Introduction 3-28
Modeling Method 3-28
Characterization: Creating Electrical APLMMX Models 3-29
Running APLMMX Characterization 3-29
Static Analysis Characterization Setup 3-29
Dynamic Analysis Characterization Setup - Outside Totem 3-30
APLMMX New Device Name Mapping Engine 3-30
Improved finger handling 3-31
TSR Flow Setup Inside Totem 3-32
APLMMX Configuration File Keywords 3-35
Required Keywords 3-35
Optional APLMMX Configuration File Keywords 3-38
Sample APLMMX Configuration File 3-88
Modeling MOS intentional device capacitance 3-89
Using ‘m-factor' to split subcircuit X-instances 3-89
Support for XA as a Third Party Simulator 3-89
Special APLMMX Applications 3-90
Multi-state Analysis Support 3-90
Static IR analysis using DC-OP information 3-95
Voltage Derating 3-96
Characterization of Leakage Current 3-97
Characterization of Designs with Power Switches 3-97
Automatic modelling of Output port current for regulated nets 3-98
Checking and Debugging APLMMX Characterization Results 3-99
Resistor Lists 3-99

Other Checks 3-99
GDSII Processing: Creating Physical Models 3-101
GDS2DB Configuration File Keywords 3-102
GDSII Files 3-103
Top Cell(s) Definitions 3-103
Nets Definition 3-105
Layer Map Definition 3-112
Input LEF 3-113
Geometry Extraction 3-114
Mini LVS and Device Handling 3-119
Selective Cell Hierarchy Handling 3-132
Auto Pad Location Generation 3-136
Automated Switch Cell Handling 3-138
ESD Analysis 3-142
Other Keywords 3-145
Checking and Debugging GDS2DB Modeling Results 3-149
Debugging GDS-related issues in Totem 3-149
GDS debugging - Shorts 3-149
General GDS debugging tools 3-151

Chapter 4 - Static and Dynamic Analysis

Introduction 4-152
Power Calculation 4-152
Power Calculation Procedure and Results Evaluation 4-153
Power Grid Resistance Extraction 4-155
Metal Density Calculation 4-155
TCL Command R Extraction 4-155
GUI Extraction 4-155
Examining Power/Ground Grid Weakness 4-156
Defining Pad and Package Parameters 4-159
Command Line Procedure for Package Modeling 4-159
GUI Procedure for Package Modeling 4-159
Package Compiler Utility 4-160
Setting up and Running Static Analysis 4-160
GSR File 4-160
Other Files 4-161
TCL Command File for Static Analysis 4-161
Running Static Analysis 4-161
Setup Details for Static-based Analysis 4-162
Flat Static Analysis using Block Power for Scaling 4-162
Region-based Static Analysis 4-162
Netlist-Driven Hierarchical Static Analysis 4-164
P/G grid integrity checking 4-166
Setting Up and Running Totem Dynamic Analysis 4-166
GSR File for Dynamic Analysis 4-166
Other Files 4-167
TCL Command File for Dynamic Analysis 4-167

DvD Backannotation to Timing Flow 4-167
Running Dynamic Analysis 4-168
Netlist-Driven Hierarchical Dynamic Analysis 4-168
Mixed Mode VCD and Vectorless Analysis 4-169
Setup for Mixed-Mode Custom Design Flow 4-169
Power/Ground EM Analysis 4-171
Static EM Analysis Overview 4-171
Dynamic EM Analysis Overview 4-171
Setting Up EM Limits 4-172
Temperature Setting 4-173
Running Power EM Analysis 4-174
Analyzing Power EM Analysis Results 4-174
Current direction and EM violations 4-177
Fixing Power EM Violations 4-178

Chapter 5 - Design-Specific Setup

Introduction 5-179
Embedded Memory Macros 5-179
Analog Designs 5-180
Diode Devices 5-180
Marker layer 5-180
PIN snapping 5-180
BJT Devices 5-181
Marker layer 5-181
Poly Resistor Devices 5-182
Marker Layer 5-182
Technology file changes for poly resistors 5-183
Power-gated Designs 5-184
Overview 5-184
On-state Analysis 5-187
Power Gating Flow 5-188
General data requirements 5-188
Spice netlist data requirements 5-188
GDS switch data requirements 5-189
Switch model creation and characterization 5-189
APLMMX electrical model processing 5-196
GDS2DB physical model processing 5-197
Define power gating On-Off switching with the GSC file 5-202
Defining and Importing switches using the GSR file 5-202
Running Totem Low Power Analysis 5-204
Mixed Mode Low Power Analysis 5-204
Power Gating Results 5-204
Reports on switches in the design 5-204
switch_static. rpt File 5-204
switch_dynamic.rpt File 5-205
LDO Low Power Designs 5-205
Overview 5-205

LDO design modeling 5-206
Controlling LDO switching state using GSC file 5-207
Outputs generated in LDO-based analysis 5-208
LDO Modeling with APLDO 5-208
LDO DC model example configuration file 5-209
APLDO example configuration file for load regulation dynamic model 5-210
Generating the DC LDO model 5-212
Testing LDO models 5-212
Other practical LDO applications 5-212
Stand-along power-gate analysis 5-213
Full Custom Hierarchical Designs 5-215
DRAM/Flash memories 5-215
FLASH design operating modes 5-215
Handling large design sizes 5-216
Mixed Signal Designs 5-217
Setup for Mixed-Mode Custom Design Flow 5-218
3DIC setup for EMIR 5-219
I/O Devices 5-220

Chapter 6 - Results Evaluation

Introduction 6-221
P/G Grid Integrity Checking 6-221
Static IR Drop Analysis 6-222
EM Analysis 6-223
List of Worst EM display 6-223
Other results display 6-224
Dynamic Voltage Drop Analysis 6-224
MMX Lists and Colormaps of Critical Conditions 6-224
Histograms 6-226
Decap Results 6-226
MMX Design Summary Report 6-227
Mixed-Mode Custom Design Analysis 6-230
“What-if” Analysis 6-231
CMM Reports 6-231
CMM Statistics 6-231
Wire Voltage Drops Report 6-232
Constraint Violation Summary 6-232
Dynamic Analysis Constraint Summary 6-233
Static Analysis Constraint Summary 6-233

Chapter 7 - Package and Board Modeling

Introduction 7-235
Integrated CPA Flow 7-235
Simple Package RLC Model 7-236
Distributed RLCK Package and Board Subcircuit Model 7-237
Support for Package K-parameters 7-239

Linear Current- and Voltage-Controlled Source Models 7-240
Linear Voltage-Controlled Voltage Source (E) 7-241
Linear Current-Controlled Current Source (F) 7-241
Linear Voltage-Controlled Current Source (G) 7-241
Linear Current-Controlled Voltage Source (H) 7-241
Mapping Package Port Names to Die Pad Names in the PLOC File 7-242
Chip-Die Mapping Using Package Compiler 7-242
Overview 7-243
Inputs 7-243
Command Syntax 7-244
Outputs 7-245
Known Restrictions 7-246

Chapter 8 - Chip Power Models (CPM)

Introduction 8-247
Design Flow 8-248
Totem Modeling of Chip Power Delivery Network 8-248
Modeling Choices Based on Number of Pads 8-249
CPM for Flip Chip Designs 8-249
CPM for Wirebond Designs 8-250
Modeling Choices Based on Analysis Speed and Accuracy 8-250
High speed modeling 8-250
High Accuracy Modeling 8-251
CPM Simulation Procedure 8-251
Setup and Preparation 8-251
Running CPM 8-251
Setting up for power integrity analysis 8-256
Setting up for EMI modeling 8-257
Modeling leakage resistance using arbitrary partitioning 8-257
iCPM- Internal Node Probing 8-258
Resonance frequency-aware mode 8-258
Power Transient Mode (Mixed Signal /Digital P&R Designs) 8-259
User-configurable mode (Mixed Signal /Digital P&R Designs) 8-261
CPM LDO analysis support 8-263
CPM Outputs 8-264
CPM Model Files 8-264
get_cdie.sp File 8-265
*.cdie File 8-265
Using the Chip Power Model 8-267
Validating the Model 8-267

Chapter 9 - Fixing and Optimizing Grid and Power Performance

Introduction 9-269
Manual Power Grid Modification 9-270
Changing a Metal Layer or Via Resistivity 9-270
Adding a Power/Ground Pad 9-270

Adding a Set of Power/Ground Pads over a Specified Area 9-271
Deleting a Power/Ground Pad 9-271
Adding a Via 9-271
Deleting a Via 9-271
Adding One or Multiple Power Straps 9-271
Editing/Deleting a Power Strap 9-273
Adding a Decap Cell 9-274
Adding Metal Layers and Vias or Via Arrays 9-276
Undo and Redo 9-277
Automated Grid Optimization and Fixing Procedures 9-277
Fixing and Optimization Flow for Static IR Drop Improvement 9-277
FAO Procedure for Multiple Vdd Designs 9-278
Grid Optimization 9-278
Mesh Commands 9-278
GSR Keywords for Region-based Grid Width Sizing (mesh optimize) 9-279
GSR Keywords for Hot-spot Based Grid Width Fixing (mesh fix) 9-279
Examples of Grid Optimization 9-280
Example A - Full Chip Optimization - Relaxing Static IR Drop 9-280
Example B - Full Chip Grid Fixing - Reducing Static IR Drop 9-281
Example C - Partial Chip Optimization, Relaxing Static IR Drop 9-282
Example D - Partial Chip Fixing, Reducing Static IR Drop 9-284
Example E - Mesh Optimize, -taper option, to reduce regional static IR drop 9-286
Example F - Mesh Fix, -taper option, to reduce hot spot static IR drop 9-287
Fixing and Optimization (FAO) for DvD and Timing in Digital P & R Blocks 9-287
Overview 9-287
Preparation for Decap FAO 9-289
Decap Modification Operations 9-289
Decoupling Capacitance Modification Commands 9-290
Decap Modification Constraints and Interfaces with Other Programs 9-290
GSR Keywords Controlling Decap Modification 9-290
Additional Tools for Fixing High Voltage Drop Areas 9-291
Supplemental Power Routing with ‘route fix’ 9-291
Cell Moving or Cell Swapping of “Hot Instances” 9-291
Examples of FAO for DvD 9-294
Example G - Full Chip DvD Reduction - Non-overlap Decap and Grid Fix 9-294
Example H - Full Chip DvD Reduction - Decap Overlap 9-296
Reviewing and Implementing ECOs in a layout editor 9-297
Saving Design Changes with the ECO Command 9-297
Writing an ECO File 9-297
Reading an ECO File 9-297
ECO File Format Definition 9-298
Fixing and Optimization Command Reference 9-299
Descriptions of Mesh Optimization Commands 9-299
Mesh and ring commands - FAO functions to optimize and fix grids. 9-299
Descriptions of Decap Modification Commands 9-305
Supplementary Voltage Drop Fixing Commands 9-315
GSR Keywords Supporting FAO Functionality 9-315

Command and GSR Keyword Syntax Conventions 9-315

Chapter 10 - Custom Macro Models

Introduction 10-316
CMM Interfaces 10-316
Tech files 10-316
Defining CMM Pins 10-316
Embedding Signatures in a CMM 10-317
Keywords for including data in CMM models - analog 10-317
Keywords for including data in CMM models - digital place and route 10-318
ESD-aware CMM Creation 10-319
Keywords for excluding data from CMMs 10-319
Choosing appropriate hierarchy levels for CMMs 10-319
CMM database compatibility across RedHawk and Totem releases 10-320
Preparing APLMMX and GDSMMX Model Input 10-320
No APLMMX or GDSMMX model optimization 10-320
GSR Example 1 - no APLMMX or GDSMMX optimization 10-320
APLMMX and GDSMMX model optimization 10-321
GSR Example 2 10-321
GSR Example 3 10-322
Setting up the Integrated Configuration File for CMM Generation 10-323
Controlling mmx_view and cell_view physical models separately 10-324
CMM static model support 10-325
Generating a Custom Macro Model 10-325
CMM Generation Using the ‘export model’ Command in Totem 10-325
Loading/Importing CMM Cells for Analysis 10-326
Loading a CMM Binary Model 10-326
Loading a CMM RAW Model 10-326
Generate & use CMM mmx-view, cell-view and post-sim views in a single directory 10-327
Importing in Mixed Mode 10-328
CMM Compatibility between Releases 10-328
Finding the CMM DB version 10-328
CMM database compatibility across RedHawk and Totem releases 10-329

Chapter 11 - Signal EM Analysis

Introduction 11-330
Methodology for Transistor-level Signal EM Analysis 11-331
Overview 11-331
Current Estimation 11-332
Importing Simulation Currents 11-332
Input Data Requirements 11-332
Setting Up for Transistor-level Signal EM Analysis 11-334
“Static” Estimated Waveform GSR Setup 11-334
“Dynamic” True SPICE Simulation-based Waveform Setup 11-338
Running Transistor-level Signal EM Analysis 11-344
Estimated Waveform Method 11-344

True SPICE Simulation-based Method 11-344
Tips for Using the Totem GUI 11-345
Analyzing EM Results 11-347
Text Reports 11-347
Multi-cycle and Multi-state results 11-347
Debugging tips 11-348
DCOP support for signal EM flow 11-348
DMP (Distributed Machine Processing) Support for Signal EM 11-348
Methodology for Cell-level Signal EM Analysis 11-349
Overview 11-349
Input Data Requirements 11-350
Setup for Cell-level Signal EM Analysis 11-350
Waveform Specifications 11-351
EM Limits 11-351
Custom Current File 11-352
Running Signal EM Analysis 11-352
Tips for Using the Totem GUI 11-353
Analyzing Results 11-353
Debugging Tips 11-354
Integrated Power and SignalEM flow 11-354

Chapter 12 - Pathfinder™ ESD Analysis

Introduction - The ESD Problem 12-356
ESD Analysis 12-357
Overview 12-357
Clamp Cell Definition 12-358
Clamp Files - Cell 12-358
Clamp Files - Type 12-363
Clamp DB Creation 12-363
Clamp-file-driven GDS-based Pathfinder-Static flow 12-364
ESD Rules Files 12-364
Defining Groups of Nets 12-365
Reducing Design Size for Improved ESD Analysis 12-365
Topology and Connectivity Checking of Bumps and Clamps 12-365
Layout Resistance Checking of Bumps and Clamps 12-367
Overview 12-367
Bump-to-Bump (BUMP2BUMP, or B2B) 12-367
Bump-to-Clamp (BUMP2CLAMP, or B2C) 12-368
Bump-to-Instance (B2I) 12-368
Clamp-to-Clamp (CLAMP2CLAMP, or C2C) 12-368
Clamp-to-Inst (CLAMP2INST or C2I) 12-368
Clamp-to-Macro (CLAMP2MACRO or C2M) 12-368
Including Package Resistance 12-369
Data Flow 12-369
Description of Inputs 12-370
Resistance Checking for B2B, B2C, and C2C Rules 12-372
Rules File Syntax for Types B2B, B2C, and C2C 12-375

Sample Rules File 12-381
Resistance Checking for CLAMP2INST (C2I) and CLAMP2MACRO (C2M) Rules .. 12-381
Rules File Syntax for Types C2I and C2M 12-381
Examples of Rule Checking 12-384
Connectivity Checking 12-386
Combined Rules and Clamp Cell Pin Location File 12-388
Invocation 12-388
Resistance Checking for BUMP2INSTANCE Rules 12-389
Rule File Syntax for Types B2I 12-389
Resistance and Voltage Checking from Bumps to Cross-domain Driver-Receivers . 12-391
Introduction 12-391
Cross-domain Resistance/Voltage Check Rule Keywords 12-393
ESD Resistance Checking Reports 12-395
Report esdcheck command 12-395
Resistance Checking Output Reports 12-396
B2I results reports 12-397
B2C Results Reports 12-398
Clamp Info Reports 12-398
Pass/Fail Reports 12-399
ESD Info Reports 12-402
ESD Summary Reports 12-403
Special Resistance Calculations for ESD Analysis 12-404
Current Density Checking 12-406
Mode 1 Rules Files - all clamp paths 12-406
Mode 2 Rules Files - Specified clamp paths 12-411
JEDEC HBM Checks 12-412
Guard-ring Latch-up Checks 12-413
Arc-based Current Density Checking 12-413
Rules File 12-413
CD and arc-based resistance checks include clamp resistance 12-416
Viewing Current Density Checking Results 12-416
ESD CD Report Command 12-416
Results in compressed mode 12-418
Reporting voltage at probed points in B2B CD check 12-419
ESD-CD report esd_summary.rpt 12-419
ESD Current Density Reports for Pads 12-419
ESD EM Report for ESD-CD 12-420
Displaying Resistance and Current Density Results in the GUI 12-420
GUI Results for Resistance Checking 12-420
GUI for EM Current Density Rule Check 12-425
Clampviewer utility for viewing I-V curves for clamps 12-428
General Rule File Inclusions and Exclusions 12-428
Clamp Element Exclusions 12-429
Bump Inclusions/Exclusions Using the Pad File 12-429
3DIC flow in Pathfinder 12-429

Chapter 13 - Substrate Noise Analysis

Introduction 13-431
Quick experiments with substrate analysis 13-431
(1) Experimenting with GDS and noise injection 13-431
(2) Experimenting with digital vectorless analysis 13-432
(3) Experimenting with an SoC 13-432
What is CSE (Canyon Substrate Extension)? 13-432
What CSE cannot model 13-433
Introduction to substrate-noise analysis 13-434
What is substrate noise? 13-434
Chips with radios 13-435
CMOS imagers 13-436
High-voltage and high-current chips 13-437
Low-cost packages 13-438
Substrate noise in triple-well CMOS 13-438
Setting up a design for substrate analysis with CSE 13-440
Settings needed for CSE 13-441
Standard-cell GSR keywords and configuration files 13-442
Creating a SUBSTRATE_LIBRARY_FILE 13-442
Creating a SUBSTRATE_AUTO_DNW_CONFIG 13-445
Modeling substrate-noise sources 13-446
Modeling substrate structures 13-447
Modeling victims 13-447
Modeling the package 13-448
CPMs that include substrate 13-448
Noise Injection Analysis 13-448
Inject_current Tcl command 13-449
Time to steady state 13-449
Injection at PLOCs 13-450
Injecting user-supplied waveforms 13-450
Inject_voltage Tcl command 13-450
S21 Tcl command 13-451
Frequently-used analyses for understanding substrate noise 13-451
Guard-ring weakness check 13-451
Point-to-point resistance calculation 13-453
Manual noise injection 13-453
Coupling analysis using S21 13-455
Peak-to-peak voltage maps 13-455
Current maps 13-457
Voltage and current probes 13-457
Frequency-domain results 13-458
Substrate Back-annotation for Spice Simulation 13-460
Debugging substrate noise 13-460
Overall noise levels unexpectedly high/low 13-460
Isolated substrate hot spot 13-460
Guard ring ineffective 13-461
High-frequency noise in victim region 13-461
Run spice simulation by back annotating noise voltage 13-461

Early substrate analysis to avoid late-stage problems 13-461
Reference A: Recipe for adding substrate analysis to your setup 13-462
Steps for Totem setup with substrate analysis 13-462
Task 1: Check Technology Files and Layermap 13-462
Task 2: Run without substrate analysis 13-463
Task 3: Run with substrate analysis through extraction 13-464
Task 4: Run dynamic analysis, including substrate analysis 13-465
Steps for Redhawk setup with substrate analysis 13-465
Task 1: Check Technology Files and Layermap 13-465
Task 2: Successful non-substrate run 13-466
Task 3: Model standard cells for substrate analysis 13-467
Task 4: Create models for custom blocks 13-467
Reference B: Describing substrates for CSE 13-468
Overview of substrate model 13-468
Examples of substrate cross-sections 13-468
Substrate Model 13-471
Substrate features directly modeled by CSE 13-471
Overlaying features to create a cross-section 13-474
Describing implicit (undrawn) shallow wells 13-474
Electrical model of substrate 13-475
Substrate features not directly modeled by CSE 13-477
Substrate technology file (.subtech) 13-480
Overview of contents 13-480
Substrate technology file keywords 13-481
Examples of substrate technology files 13-484
CMOS triple-well .subtech 13-485
SOI BCD process 13-487
Bulk BCD process 13-488
Visualizing the substrate cross-section 13-490
Configuring Totem-CSE for substrate analysis 13-491
Determining the substrate cross-section 13-491
“Drawing rules” for substrate shapes 13-492
Tuning the substrate cross-section 13-493
Default simplifications 13-493
Totem-CSE geometry source 13-494
GDS layer map file for CSE 13-495
Creating the substrate technology file 13-496
Determining depths of substrate features 13-496
Specifying feature depth 13-496
Determining resistivity of conductors 13-497
Specifying resistivity 13-497
Determining capacitance values 13-498
Calculating capacitance from parasitic substrate diodes 13-499
Choosing the bias voltage 13-499
Alternative capacitance calculation 13-499
Specifying capacitance 13-500
Calculating capacitance from parasitic substrate diodes 13-500

Specifying surface fill 13-501
Substrate GSRs interacting with .subtech 13-501
Reference C: Characterizing substrate-pin currents with APL 13-503
Additions to APL configuration for substrate 13-503
Specifying nominal voltages for substrate pins 13-504
Controlling bulk-pin resistance 13-505
Reference D: LDMOS noise sources 13-505
White-box LDMOS model 13-507
Reference E: Rarely-used and obsolete CSE GSRs 13-507
Rarely-used controls 13-507
Obsolete settings 13-510

Chapter 14 - Totem™ Standard Cell EM Checks

Introduction 14-511
Vectored ttemcheck flow(Beta Version) 14-511
Totem Vectorless Standard Cell EM Checks Flow 14-524
Key inputs and Usage 14-524
Sample Configuration File <ttemcheck.config>: 14-525
BSUB config file 14-527
Cell specific Config file 14-527
Output Reports 14-529
Support for multiple subckt in DSPF in ttemcheck flow 14-530

Chapter 15 - Totem™ CCI Flow

Introduction 15-532
Benefits of Totem CCI Flow 15-532
Flow Overview 15-533
Totem CCI Flow - Input Requirements 15-533
Requirements for CCI data 15-534
Calibre LVS 15-534
CCI Database 15-534
Requirements for RC Mapping File 15-535
Requirements for DSPF 15-536
Netlist requirements 15-536
Running the CCI Flow 15-536
Query Calibre LVS results(svdb) 15-537
Run CCI2EX 15-538
Run cci2ex standalone 15-538
Run cci2ex in integrated aplmmx 15-539
CCI Flow related keywords 15-541
Mandatory CCI2EX keywords: 15-541
Optional CCI2EX Keywords: 15-542
APLMMX keywords: 15-544
GSR keyword: 15-544
Special device handling 15-544
MOM cell handling: 15-544

P/NLDMOS cell handling 15-545
Pseudo Flattened NetList Support 15-545
Totem CCI Flow Debugging Tips 15-545

Appendix A - Installation Procedure

Introduction A-548
Downloading Totem Software A-548
Program Installation A-548
Totem Operating System/Platform Support A-549
Setting Up the SCBU Product License A-549
Setting Up the Totem Environment A-550
License File and Library Directory Setup A-550
Binary Setup A-551
Platform-Specific Binaries A-551
Platform-Independent Binaries A-551
Invocation A-551

Appendix B1 - Power Integrity GUI and Menus

Introduction B1-553
Totem Graphic User Interface Description B1-553
General GUI Features B1-555
GUI Look and Feel Preference Setting B1-555
Layers Window B1-555
Configuration Window B1-557
GUI View Control and Query Toolbars B1-560
Status Toolbar B1-562
Advanced Filter Function for Table Dialogs B1-562
Multiple selection and multiple highlighting B1-562
GDS pre-selected cross-probing B1-563
Customizable forms for all GUI table dialogs B1-563
SPT debugging from any pin-based colormap B1-564
Net selection B1-564
Displaying maps of switch current and voltage B1-565
TCL Command Line B1-565
Text Message and Log Window B1-566
Exporting Marker Coordinates from “List of ...” Dialogs B1-566
Other GUI Support Features B1-566
Middle Mouse Button/wheel Functions B1-566
Bindkey Support (Virtuosotm-like behavior) B1-567
Special GUI support B1-567
User-preferred GUI settings B1-569
Power Integrity GUI Menus B1-569
Tools -> Power Integrity B1-569
Tools -> Signal EM B1-569
Tools -> ESD B1-569
Files -> Import GDS B1-570
Files -> Import Netlist B1-570

Files -> Import Design Data B1-571
Files -> Import Database B1-571
Files -> Import ECO... ..... B1-571
Files -> Export Flatten GDS B1-571
Files -> Export Database B1-572
Files -> Export ECO B1-572
Files -> Import Third Party Technology Data B1-572
Files -> Import GUI Config B1-572
Files -> Export GUI Config B1-572
Files -> Playback B1-572
Files -> Auto Play B1-573
Files -> Snap Shot B1-573
Files -> Exit B1-573
Edit -> Undo B1-573
Edit -> Redo B1-573
Edit -> Add Pad B1-573
Edit -> Delete Pad B1-573
Edit -> Add Strap B1-573
Edit -> Edit Strap B1-573
Edit -> Add Via B1-574
Edit -> Delete Via B1-574
Edit -> Add Decap Cell B1-574
Edit -> Add/Delete Probe B1-575
Edit -> Add Rectangle B1-575
Edit -> Delete Objects B1-575
Edit -> Create Ruler B1-575
Edit -> Clear All Rulers B1-576
View -> GDSII View B1-576
View -> OA Database View B1-582
View -> Layout View B1-582
View -> Net Display Map B1-582
View -> Nets... ..... B1-582
View -> Waveform View B1-582
View -> Connectivity -> B1-587
View -> Technology Layers (Metal and Via) B1-593
View -> Technology Layers (Substrate) B1-593
View -> Hierarchy Level... ..... B1-594
View -> Netlist Hierarchy B1-594
View -> Map Configurations -> B1-595
View -> EM Mode B1-598
View -> Power Maps -> B1-598
View -> Resistance Maps -> B1-599
View -> Voltage Drop Maps B1-600
View -> Electromigration Map B1-602
View -> Current Maps B1-602
View -> Voltage Maps B1-602
View -> Transistor Pin Maps -> B1-602

View-> Dynamic Instance DvD -> B1-604
View -> Decap Maps -> B1-605
View -> Switch Instance Maps B1-606
View -> Substrate Noise Map B1-606
View -> View Control B1-607
View -> Query B1-608
View -> Switch Operating Mode B1-608
Job Monitor B1-609
Static-> Power B1-609
Static -> Network Extraction B1-611
Static -> Package Constraints B1-611
Static -> Static IR-drop Analysis B1-611
Static -> EM Check B1-611
Dynamic -> Power B1-611
Dynamic -> Network Extraction B1-611
Dynamic -> Package Constraints B1-612
Dynamic -> Vectorless Dynamic Voltage Drop Analysis B1-612
Dynamic -> VCD-based Dynamic Voltage Drop Analysis B1-612
Dynamic -> EM Check B1-612
Results -> Log Message Viewer B1-612
Results -> Design Summary Report B1-612
Results -> List of ECOs B1-613
Results -> List of Pads B1-613
Results -> List of Transistor Information B1-613
Results -> List of Transistor Resistance B1-613
Results -> List of Resistance Constraint Checks B1-614
Results -> List of Guard Ring Actual Resistance B1-614
Results -> List of Worst EM B1-614
Results -> List of Worst IR for Wires and Vias B1-614
Results -> List of Worst IR Instances (Static) B1-615
Results -> List of Worst Instance DVD B1-615
Results -> List of Worst Transistor Pin Voltages... ..... B1-615
Results -> List of Worst Transistor Pin Current B1-617
Results -> List of Highest Power Instances B1-617
Results -> Supply Current Waveforms B1-617
Results -> Analysis Histograms... ..... B1-618
Results -> Movie -> B1-618
Connection -> Connect to layout editor B1-618
Connection -> Connect to schematic B1-622
Explorer -> Generate B1-622
Explorer -> View B1-624
Windows -> Display Multiple Frames B1-624
Windows -> Preferences B1-625
Windows -> Bring main window to top B1-627
Windows -> Detach/Attach Tcl console B1-627
Windows -> Detach/Attach LSW B1-627
Windows -> Tcl Console B1-627

Windows -> Default View B1-627
Windows -> Text Message Window B1-627
Help -> About... ..... B1-627
Help -> Manual B1-627
Bindkey Functions B1-627
Functional Summary B1-627
Reconfiguring Bindkeys B1-628

Appendix B2 - Signal EM GUI and Menus

Introduction B2-630
Totem Signal EM GUI Description B2-630
Introduction B2-631
General GUI Features B2-632
GUI Look and Feel Preference Setting B2-632
Layers Window B2-632
View Configuration Window B2-634
GUI View Control Toolbar B2-637
Status Toolbar B2-638
Multiple selection and multiple highlighting B2-639
Advanced Filter Function for Table Dialogs B2-640
GDS pre-selected cross-probing B2-640
Customizable forms for all GUI table dialogs B2-640
SPT debugging from any pin-based colormap B2-641
Net selection B2-641
TCL Command Line B2-642
Text Message and Log Window B2-642
Exporting Marker Coordinates from “List of ...” Dialogs B2-642
Other GUI Support Features B2-642
Middle Mouse Button/wheel Functions B2-642
Bindkey Support (Virtuosotm-style behavior) B2-643
User-preferred GUI settings B2-643
Signal EM GUI Menus B2-643
Tools -> Power Integrity B2-643
Tools -> Signal EM B2-644
Files -> Import Netlist B2-646
Files -> Import Design Data B2-647
Files -> Import Database B2-647
Files -> Import ECO B2-647
Files -> Export Flatten GDS B2-647
Files -> Export Database B2-648
Files -> Export ECO B2-648
Files -> Import Third Party Technology Data B2-648
Files -> Import GUI Config B2-648
Files -> Export GUI Config B2-648
Files -> Playback B2-648
Files -> Auto Play B2-649
Files -> Snap Shot B2-649

Files -> Exit **B2-649**
Edit -> Add Strap **B2-649**
Edit -> Edit Strap **B2-649**
Edit -> Add Via **B2-649**
Edit -> Delete Via **B2-650**
Edit -> Create Ruler -> **B2-651**
Edit -> Clear All Rulers **B2-651**
View -> GDSII View * **B2-651**
View -> OA Database View **B2-656**
View -> Layout View **B2-657**
View -> Net Display Map **B2-657**
View -> Nets **B2-657**
View -> Waveform View **B2-657**
View -> Connectivity -> **B2-662**
View -> Hierarchy Level **B2-666**
View -> Netlist Hierarchy **B2-666**
View -> Map Configurations -> **B2-667**
View -> EM Mode **B2-669**
View -> Electromigration Map **B2-669**
View -> Current Maps **B2-669**
View -> Transistor Pin Maps -> **B2-669**
View -> Temperature Map **B2-669**
View -> View Control **B2-669**
View -> Query **B2-670**
View -> Switch Operating Mode **B2-671**
View -> Job Monitor **B2-671**
Analysis -> Static Signal EM Analysis **B2-671**
Analysis -> Dynamic Signal EM Analysis **B2-671**
Results -> Log Message Viewer... ..... **B2-671**
Results -> Design Summary Report **B2-672**
Results -> List of Worst EM **B2-672**
Results -> List of Signal Nets Transistor Pin Currents **B2-672**
Results -> Analysis Histograms **B2-672**
Connection -> Connect to layout editor... ..... **B2-673**
Connection -> Connect to schematic **B2-676**
Explorer -> Generate B2-677
Explorer -> View B2-678
Windows -> Display Multiple Frames **B2-678**
Windows -> Preferences **B2-680**
Windows -> Bring main window to top F1 ..... **B2-681**
Windows -> Detach/Attach Tcl console F2 ..... **B2-681**
Windows -> Detach/Attach LSW F4 ..... **B2-681**
Windows -> Tcl Console **B2-681**
Windows -> Default View **B2-682**
Windows -> Text Message Window **B2-682**
Help -> About... ..... **B2-682**
Help -> Manual **B2-682**

Bindkey Functions B2-682
Functional Summary B2-682
Reconfiguring Bindkeys B2-683

Appendix B3 - ESD GUI and Menus

Introduction B3-684
Totem ESD GUI Description B3-684
Introduction B3-685
General GUI Features B3-686
GUI Look and Feel Preference Setting B3-686
Layers Window B3-686
View Configuration Window B3-688
GUI View Control Toolbar B3-691
Status Toolbar B3-693
Advanced Filter Function for Table Dialogs B3-694
TCL Command Line B3-694
Text Message and Log Window B3-694
Exporting Marker Coordinates from “List of ...” Dialogs B3-694
Other GUI Support Features B3-695
Middle Mouse Button/wheel Functions B3-695
Bindkey Support (Virtuosotm-style behavior) B3-695
User-preferred GUI settings B3-696
ESD GUI Menus B3-696
Tools -> Power Integrity B3-696
Tools -> Signal EM B3-696
Files -> Import Netlist... ..... B3-699
Files -> Import Design Data... ..... B3-700
Files -> Import Database... ..... B3-700
Files -> Import ESD DB B3-700
Files -> Import ECO B3-701
Files -> Export Flatten GDS B3-701
Files -> Export Database B3-701
Files -> Export ESD DB B3-701
Files -> Export ECO B3-701
Files -> Import Third Party Technology Data B3-701
Files -> Import GUI Config B3-701
Files -> Export GUI Config B3-702
Files -> Playback B3-702
Files -> Auto Play B3-702
Files -> Snap Shot B3-702
Files -> Exit B3-702
Edit -> Add Pad B3-702
Edit -> Delete Pad B3-702
Edit -> Add Strap B3-702
Edit -> Edit Strap B3-703
Edit -> Add Via B3-703
Edit -> Delete Via B3-703

Edit -> Clamp Pin Setup B3-704
Edit -> ESD Clamp ECO B3-705
Edit -> Create Ruler -> B3-705
Edit -> Clear All Rulers B3-705
View -> GDSII View B3-705
View -> OA Database View B3-711
View -> Layout View B3-711
View -> Net Display Map B3-711
View -> Nets B3-712
View -> Waveform View B3-712
View -> Connectivity -> B3-717
View -> Hierarchy Level B3-722
View -> Netlist Hierarchy B3-722
View -> Map Configurations -> B3-723
View -> ESD Current Density Test List B3-725
View -> ESD Clamp Current/Voltage List B3-725
View -> Driver-Receiver Net Pair Voltage Check B3-725
View -> Electromigration Map B3-728
View -> Current Maps B3-728
View -> Voltage Maps B3-728
View -> ESD Resistance Check Maps B3-728
View -> Transistor Pin Maps -> B3-729
View -> View Control B3-730
View -> Query B3-731
View -> Switch Operating Mode B3-731
View -> Job Monitor B3-732
Static -> Network Extraction B3-732
Static -> P2P Current Density Check B3-732
Static -> Resistance and Current Density Check B3-732
Results -> Log Message Viewer B3-732
Results -> Design Summary Report B3-733
Results -> List of Effective Grid Resistance B3-733
Results -> List of Effective Instance Resistance B3-733
Results -> List of Pads B3-733
Results -> ESD Connectivity Lists B3-733
Results -> List of Wire /Via Actual Resistance... ..... B3-734
Results -> List of Worst EM... ..... B3-734
Results -> List of ESD Static Rule Checks B3-734
Results -> List of ESD P2P Resistance B3-735
Results -> Analysis Histograms B3-735
Connection -> Connect to layout editor B3-736
Connection -> Connect to schematic B3-739
Explorer -> Generate B3-740
Explorer -> View B3-741
Windows -> Display Multiple Frames B3-742
Windows -> Preferences B3-743
Windows -> Toolbars B3-744

Windows -> Bring main window to top F1 ..... B3-744
Windows -> Detach/Attach Tcl console F2 ..... B3-744
Windows -> Detach/Attach LSW F4 ..... B3-744
Windows -> Tcl Console B3-745
Windows -> Default View B3-745
Windows -> Text Message Window B3-745
Help -> About... ..... B3-745
Help -> Manual B3-745
Bindkey Functions B3-745
Functional Summary B3-745
Reconfiguring Bindkeys B3-746

Appendix C - File Definitions

Introduction C-747
Input Files C-747
Output Files C-747
Keyword Syntax Conventions C-747
Apache Technology File (*.tech) C-748
Encrypting and Decrypting a Tech File C-748
Partial File Encryption C-749
Technology File Keywords C-749
Global Switching Configuration (GSC) File C-774
Global System Requirements File (*.gsr) C-776
GSR File Keywords C-777
Input Data Keywords C-777
Parameter Keywords C-811
Custom Cell Modeling Keywords C-822
Electromigration Keywords C-829
Extraction and Netlisting Keywords C-843
Characterization Keywords C-858
Clock, Toggle Rate and Power Scaling Keywords C-860
Timing Keywords C-870
Wire Load Specification Keywords C-871
Dynamic Simulation Keywords C-874
FAO General Keywords C-883
Grid Fixing and Optimization Keywords C-885
Decap Optimization Keywords C-889
Low-power Design Keywords C-893
ESD keywords C-898
Name Mapping Keywords C-902
Warning and Error Message Keywords C-904
Ignore Function Keywords C-905
Place and Route Block Design Keywords C-910
GSR Macro Keywords C-914
Substrate Analysis Keywords C-915
Pad, Power/Ground and I/O Definition Files C-915
Unified Pad Input File Format C-915

Individual Pad File Specification C-919
Pad Cellname File (*.*pcell*) C-919
Pad Instance Name File (*.*pad*) C-920
Pad Location File (*.*ploc*) C-920
Pad PSS File C-921
Library Technology Files C-922
Design Netlist Files C-922
Synopsys Library Files C-922
Custom LIB File Syntax C-922
P/G Arc Definitions in Custom LIB Files C-925
Result Files C-926

Appendix D - TCL Command Reference

Introduction D-927
Invoking Totem D-927
TCL / Script Commands D-928
TCL Syntax Conventions D-928
TCL Command Summary D-928
Running Totem in the TCL Script Mode D-994
Starting the GUI at a designated step in batch mode D-995
TCL Script Execution Examples D-996
Sample TCL Scripts D-996
Static IR Drop Analysis Example D-996
Dynamic Voltage Drop Analysis Example D-997
Automated Colormap Generation D-998

Appendix E - Utility Programs

Introduction E-999
vcdtrans E-999
vcdscan E-1000
fsdbtrans E-1001
fsdbscan E-1001
rhtech E-1002
Native GDS2DB Handling E-1004
Creating the GDS2DB Configuration File E-1005
Special Applications E-1006
Cell Hierarchy Modeling E-1006
Optimized geometry with Apache Physical Models E-1007
Mini LVS Features E-1008
Layer Map Definition E-1008
Boundary Layer Definition E-1011
Extracting multiple via layers between two metal layers E-1012
Modeling through-via support E-1012
Clamp Cell identification based on marker layer E-1013
GDS2DB support for MOM device handling E-1013
Bump Via Support E-1014
pt2timing E-1015

ACE Decap and ESR Characterization E-1016
Identifying tied-off devices as decaps E-1017
ACE Configuration File E-1017
Running ACE Characterization E-1023
Output Result Files E-1024
sim2iprof E-1024
Running sim2iprof E-1024
sim2iprof Configuration File E-1025
Configuration File Example E-1036
Output E-1037
aplreader E-1038
Running aplreader E-1038
aplreader Output E-1038
Current Outputs E-1038
Equivalent Device Capacitance and Resistance Outputs E-1040
Piecewise Linear Capacitance and Resistance Outputs E-1041
aplcdev2pwc E-1042
aplcdev2pwc Configuration File E-1042
APLMMX Template Utility E-1043
clampviewer E-1043

Appendix F - Third-Party Software Licenses

ALPHABETIC LIST OF GSR KEYWORDS

ACCURATE_METAL_DENSITY C-818
ADD_LEF_CELL_FOR_POWER C-752
ADD_PLOC_FROM_DEF C-786
ADD_PLOC_FROM_TOP_DEF C-786
ALLOW_SEPARATED_METAL_PSEUDO_VIAS C-819
ANALOG_ANALYSIS C-848
ANALYZE_NETS_FILE C-804
APACHE_DB_OVERWRITE C-786
APACHE_FILES C-786
APL_FILES C-833
AUTO_PAD_CONNECTION_LAYERS C-752
BLOCK_INSTANCE_POWER_FILE C-752
BLOCK_PAR C-849
BLOCK_POWER_ASSIGNMENT C-753
BLOCK_POWER_ASSIGNMENT_FILE C-760
BLOCK_POWER_FOR_SCALING C-835
BLOCK_POWER_FOR_SCALING_FILES C-837
BLOCK_POWER_MASTER_CELL C-761
BLOCK_POWERUP_FILES C-868
BLOCK_SAIF_FILES C-762
BLOCK_STA_FILES C-762
BLOCK_TOGGLE_FILES C-838
BLOCK_TOGGLE_RATE C-838
BLOCK_TOGGLE_RATE_FILES C-839
BLOCK_VCD_FILES C-883
BOUND_SLEW_TO_MAX_TRANSITION C-845
BPA_BY_CURRENT C-763
BPA_BY_LAYER C-763
BPA_CONN_DISTANCE C-764
BPA_CONN_MARGIN C-764
BPA_CURRENT_DENSITY C-764
BRIDGE_WIRE_CONNECTION C-819
BUS_DELIMITER C-876
BUS_DELIMITER_STA C-876
CACHE_DIRECTORY C-787
CACHE_MODE C-787
CAP_LIMIT C-864
CELL_CURRENT_DIST_FILE C-819
CELL_GEOMETRIES_AS_MFILL C-764
CELL_PIN_FILE C-765
CELL_RC_FILES C-846
CELL_TOGGLE_RATE C-839
CELL_TOGGLE_RATE_FILE C-839
CELL_WELL_CAP_FILE C-885
CEXTRACTION_SPEF_LAYER_MAP C-819

CEXTRACTION_USE_SPEF C-819
CHARGE_SWITCH C-868
CHECK_SWITCH_POWERON C-868
CLEAN_VIAS_AFTER_WIRES C-820
CLOCK_ROOTS C-845
CMM_CELLS C-797
CMM_CONSTRAINT_FILE C-798
CMM_CREATE_PINS C-797
CMM_EXCLUDE_FILES C-799
CMM_EXPAND_PINS_AT_TOP C-800
CMM_IGNORE_BLOCK_GSC C-800
CMM_INCLUDE_APL C-800
CMM_INSTANCES C-800
CMM_LAYER_MAP_FILES C-800
CMM_MODEL_CREATION C-802
CMM_PIN_BOUNDARY_LIMIT C-802
CMM_POWER_OVERRIDE_IPF C-802
CMM_PROCESS C-802
CMM_RIVETED_CONN C-820
CMM_VIA_MODEL_LAYER C-803
COMPLEX_PIN_MAX_RECT_NUM C-787
CONFIGURABLE_REPORT_FILE C-804
CONNECT_SWITCH_PINS C-820
CONSISTENT_SCENARIO C-849
COUPLEC C-820
DECAP_CELL C-864
DECAP_CELL_FILES C-864
DECAP_DENSITY C-865
DECAP_TILE_MAX C-865
DECOUPLE_LDO_GROUND C-869
DEF_FILES C-765
DEF_IGNORE_LAYERS C-766
DEF_IGNORE_PIN_LAYERS C-766
DEF_IGNORE_SPECIFIC_LAYERS C-766
DEF_IMPORT_REGION C-767
DEF_PG_NETS_FILE C-787
DEF_READ_ALL_IO_NETS C-767
DEF_SCALING_FACTOR C-787
DEFINE C-888
DELTA_T_RMS_EM C-804
DETAILED_CONN_ISSUE_RPT C-805
DIELECTRIC C-724
DO_PININST_INTERNAL_CONN C-820
DRIVER_RECEIVER_IMBALANCE_CHECK C-805
DSPF_MAP C-767
DUMP_NODE_MIN_MAX_VOLTAGE C-849
DVD_GLITCH_FILTER C-849

DYNAMIC_ADAPTIVE_RON C-869
DYNAMIC_APL_RESAMPLE C-834
DYNAMIC_BYPASS_SHORT C-850
DYNAMIC_CELL_CROSS_CHECK C-851
DYNAMIC_DETACHED_POSTPROCESS C-851
DYNAMIC_EM_FRAME C-805
DYNAMIC_FF_ADJUST_NTRIG C-788
DYNAMIC_FREQUENCY_AWARE C-851
DYNAMIC_GENERATE_BRANCH_WAVEFORM C-788
DYNAMIC_GROUP_WIRECAP C-851
DYNAMIC_LGCS_MIN_PINCAP C-852
DYNAMIC_LGCS_REPORT C-851
DYNAMIC_LOCAL_GROUND_CURRENT_SCALING C-852
DYNAMIC_MBFF_MODE C-852
DYNAMIC_MCYC_TW C-852
DYNAMIC_MSTATE_FILTER C-788
DYNAMIC_PEAK_CURRENT_AWARE C-789
DYNAMIC_PGARC_REPORT_BASE C-852
DYNAMIC_PRECHECK C-789
DYNAMIC_PRESIM_TIME C-853
DYNAMIC_REPORT_DECAP C-853
DYNAMIC_REPORT_DVD C-853
DYNAMIC_SAVE_WAVEFORM C-789
DYNAMIC_SELECTIVE_SAVE C-789
DYNAMIC_SELECTIVE_SAVE C-853
DYNAMIC_SIMULATION_TIME C-854
DYNAMIC_SOLVER_MODE C-854
DYNAMIC_SORT_BY_PERCENTAGE C-854
DYNAMIC_TIME_STEP C-805
DYNAMIC_TIME_STEP C-854
DYNAMIC_VOLTAGE_CHECK C-805
EFFECTIVE_VDD_WINDOW C-854
EM_CCF_ONLY C-806
EM_CHECK_2D C-806
EM_CUSTOM_CURRENT_FILE C-806
EM_DUMP_PERCENTAGE C-807
EM_DUTY_RATIO_ROOT C-807
EM_LENGTH_USE_MAX_LENGTH C-807
EM_MISSION_PROFILE C-808
EM_MODE C-809
EM_NET_INFO C-809
EM_REPORT_<mode>_PERCENTAGE C-811
EM_REPORT_LINE_NUMBER C-811
EM_REPORT_MODE_ONLY C-810
EM_REPORT_PERCENTAGE C-810
EM_REPORT_PERCENTAGE_BY_LAYER C-811
EM_RULE_SET C-725

EM_RULE_SET_DC C-812
EM_RULE_SET_PEAK C-812
EM_RULE_SET_RMS C-812
EM_SLEW_NO_STA C-812
EM_TECH_DC C-813
EM_TECH_FILE C-812
EM_TECH_PEAK C-813
EM_TECH_RMS C-813
EM_TOPOLOGY_USE_ELECTRON_FLOW C-813
ENABLE_AUTO_EM C-813
ENABLE_AUTO_SCANLINE C-821
ENABLE_BLECH C-814
ENABLE_POLYNOMIAL_EM C-814
ERROR_COUNTS C-878
ERROR_LOG_COUNTS C-878
ESD_BSIM3_DIODES C-872
ESD_CLAMP_FILES C-872
ESD_CLAMP_PIN_FILE C-873
ESD_CLAMP_PIN_NODE_DISTANCE C-873
ESD_DEF_IGNORE_LAYER C-874
ESD_EXTRACT_CLAMP_NET C-874
ESD_GSR_OPTIMIZE C-874
ESD_LOCAL_NETS C-875
ESD_RULE_FILES C-875
ESD_SHORT_CLAMP_PIN C-875
ESD_SIGNAL_NET_FILE C-876
ESD_SIGNAL_NETS C-876
ESD_SUBSTRATE C-876
EVA_PG_AWARE C-789
EXPAND_CELL_PIN_FILE C-821
EXTRACT_INTERNAL_NET C-869
EXTRACT_PIN_VOL_INSTS C-822
EXTRACTION_INC C-821
EZ_MERGE_NON_RECT_WIRE C-822
EZ_MERGE_NON_RECT_WIRE_MAX_LENGTH C-822
FAO_ACCURATE_VOLTAGE C-857
FAO_ADD_STACK_VIA C-858
FAO_BYPASS_CUTSIZE_CHECK C-858
FAO_DECAP_FILL_ALG C-865
FAO_DECAP_FILL_NEW_FLOW C-865
FAO_DECAP_OVERLAP C-866
FAO_DRC_DROP_RATIO C-860
FAO_DRC_OBS C-866
FAO_DRC_PL_OBS C-866
FAO_DVD_TYPE C-860
FAO_DYNAMIC_MODE C-860
FAO_ECO_NAME C-866

FAO_HOLD_LIC C-858
FAO_LAYERS C-861
FAO_MAX_SHIFT C-866
FAO_MISVIA_DISTANCE C-861
FAO_MISVIA_ONE_FILE C-858
FAO_MISVIA_RPT_SHORT C-861
FAO_MISVIA_RPT_STACK C-861
FAO_NETS C-861
FAO_OBJ C-858
FAO_PLACE_GRID C-866
FAO_PRECISE_VOLTAGE C-862
FAO_RANGE C-862
FAO_REGION C-859
FAO_ROW_SITE C-867
FAO_ROW_VDD_SITE C-862
FAO_SUB_GRID_NETS C-862
FAO_SUB_GRID_SPEC C-863
FAO_TURBO_MODE C-859
FAO_WIDTH_CNSTR C-863
FAO_WIRE_WIDTH_LOW_LIMIT C-867
FAST_DEF_READ C-768
FAST_IMPORT_APL_MODE C-834
FIX_WINDOW C-859
FREQUENCY C-839
GDS_CELLS C-769
GDSII_FILE C-768
GDSII_SCALING_FACTOR C-790
GENERATE_CPM C-790
GND_NETS C-790
GSC_FILES C-769
GSC_OVERRIDE_IPF C-840
HALF_NODE_SCALE_FACTOR C-726
HIER_DIVIDER C-877
HIER_DIVIDER_STA C-877
HONOR_LEF_PIN_TYPE C-770
IGNORE_APL_CHECK C-834
IGNORE_APL_CHECK_SWITCH C-834
IGNORE_APL_PROCESS_CORNER C-835
IGNORE_BPA_ERROR C-879
IGNORE_CELLS C-879
IGNORE_CELLS_FILES C-879
IGNORE_DEF_ERROR C-879
IGNORE_DUMMY_PGNET C-823
IGNORE_ESCAPE_CHAR C-880
IGNORE_FILE_PREPARSE C-880
IGNORE_FILLER_DECAP_CELL_REPORT C-880
IGNORE_FLOATING_PAD C-880

IGNORE_GDSMEM_ERROR C-880
IGNORE_HALF_NODE_SCALE_FOR_EM C-814
IGNORE_INSTANCES C-880
IGNORE_INSTANCES_FILES C-881
IGNORE_INSTANCES_ON_TOP_DEF_REGIONS C-885
IGNORE_LEF_DEF_MISMATCH C-881
IGNORE_LEF_MACRO C-881
IGNORE_LIB_CHECK C-881
IGNORE_NETS_FILES C-882
IGNORE_OPC_METAL C-823
IGNORE_PLOC_INTERNALNETS C-770
IGNORE_PRECHECK_ERROR C-770
IGNORE_ROUTE C-882
IGNORE_SHORT C-882
IGNORE_SIMTIME_CHECK C-882
IGNORE_TECH_ERROR C-883
IGNORE_THICKNESS_VARIATION C-823
IGNORE_UNDEFINED_LAYER C-883
IGNORE_UPF_PGARC C-883
IGNORE_VP_CONTROL_ERROR C-883
IMPORT_NETS C-770
IMPORT_NETS_FILE C-770
INACTIVE_NETS C-840
INCLUDE C-888
INPUT_TRANSITION C-845
INST_CURRENT_DIST_MODE C-823
INSTANCE_POWER_FILES C-771
INSTANCE_TOGGLE_RATE C-840
INSTANCE_TOGGLE_RATE_FILES C-841
INTERCONNECT_GATE_CAP_RATIO C-846
INTERNAL_CONNECT_PIN_CELLS_FILES C-823
IP_MODEL_CELL_MAP C-803
IP_MODEL_INST_MAP C-803
IP_MODELS C-803
KEEP_POWER_DATA C-771
LEAKAGE_LIMIT C-867
LEF_FILES C-772
LEF_IGNORE_PIN_LAYERS C-772
LEF_SCALING_FACTOR C-791
LEXTRACTION_FREQ C-824
LEXTRACTION_MODE C-824
LIB_FILES C-772
LIB_IGNORE_IO_VOLTAGE C-773
LIB_PIN_CAP C-773
LIB2AVM C-835
LICENSE_WAIT C-791
LONG_WIRE_RES_CALC C-824

LOWEST_METAL C-824
MACRO_IRDROP C-855
MACRO_POWER_FILES C-773
MAX_IPROF_POINT_NUMBER C-855
MERGE_ABUTTED_ASYM_CUTS C-824
MERGE_ABUTTED_CUTS C-814
MERGE_WIRE C-825
MESH_VIAS_FILE C-825
METAL C-726
METAL_DENSITY_BOUNDS C-825
MINWIDTH_FROM_LEF C-825
MISSING_VIA_CHECK_IGNORE_CELLS C-883
MMX_ADAPTIVE_SAMPLING C-855
MMX_DVD_GLITCH_FILTER_FILE C-856
MMX_DVD_GLITCH_FILTER_FILE C-791
MMX_RES_MAP_LIMIT C-792
MMX_XTOR_HIER_DIVIDER C-877
MPR_MODE C-825
MPR_PARTITION_REDUCTION C-826
MPR_POWER_LIMIT_FOR_RED C-826
MULTI_CYCLE_SCENARIO C-856
MULTI_THREADS C-792
NAME_CASE_SENSITIVE C-877
NATIVE_GDS_SUPPORT C-774
NET_LOAD_FILE C-826
NET_NAME_CASE_SENSITIVE C-774
NET_NAME_CASE_SENSITIVE C-827
NEW_MERGE_WIRE C-814
NOISE_LIMIT C-859
NOISE_REDUCTION C-860
NUM_HOTINST C-867
NUM_HOTSPOT C-863
OASIS_INPUT_FILE C-775
PACKAGE_SPICE_SUBCKT_INFO C-775
PAD_FILES C-775
PARA_CALC_POWER C-841
PERFORMANCE_IMPACT_REPORT C-793
PGNET_HONOR_DEF_TYPE C-776
PGPLOC_DEBUG C-793
PIECEWISE_CAP_FILES C-869
PIECEWISE_SWITCH_INPUT C-856
PIN_DELIMITER C-877
PIN_DELIMITER_STA C-877
PIN_SLICE_CELL_LIST_FILE C-827
PIN_SLICE_LIMIT C-827
PLOC_SHORT_INTERNAL_NET C-827
POWER_ALLOW_MULTIPLE_STATE C-886

POWER_ALLOW_PER_PIN_IPF C-886
POWER_ANALYSIS_MODE C-886
POWER_DISABLE_SWITCH C-841
POWER_DRIVER_TOGGLE_RATE C-841
POWER_IGNORE_ASYNCH_PIN C-777
POWER_LEAKAGE_SCALING_FACTOR C-777
POWER_LEAKAGE_SCALING_FILE C-777
POWER_MISSING_IPF_POWER C-841
POWER_MODE C-776
POWER_STATE_DEPENDENT_LEAKAGE C-841
POWER_TRANSIENT_ANALYSIS C-842
POWER_VCD_IGNORE_ERROR C-842
POWER_VCD_NUM_PROCESS C-842
POWER_VCD_REUSE_EVENT C-842
POWERUP_OUTPUT_HIGH_PROB C-870
POWERUP_RANDOM_TOGGLE C-828
POWERUP_SAVE C-870
PPI_CELL_EDGE_MAX_NM_THRESHOLD C-828
PPI_CELL_EDGE_THRESHOLD_PERCENT C-828
PPI_STD_IGNORE_TOUCH_VIA_MET C-828
PRIMARY_OUTPUT_LOAD_CAPS C-847
PRINT_EM_VIA_BOX C-793
PRINT_EM_VIA_INFO C-793
PRINT_ONE_PLOC_PER_PADINST C-777
PROBE_NODE_FILE C-829
PS_IN_FLOW_APL C-843
PUSH_PG_PININST C-830
PUSH_PININST_CELLS_FILES C-829
RDL_CELL C-777
READ_LEF_OBS C-778
REMOVE_PARENTLEF_GEOS C-779
REPORT_ALL_UNCONNECT_PORTS C-830
REPORT_DISCONN_MIN_LENGTH C-793
REPORT_FLATTEN_LOG C-793
REPORT_MAXCAP_VIOLATION C-779
REPORT_PEAK_MEMORY C-794
REPORT_REDUCTION C-794
RTL_FAST_MODE C-856
SAIF_FILE C-779
SCALE_CLOCK_POWER C-843
SCAN_CONSTRAINT_FILES C-886
SCANLINE_MERGE_LAYERS C-830
SEM_ANALYZE_NET_ONLY C-815
SEM_ENABLE_SHORTS_REPORT C-815
SEM_NET_CHECK_SHORT C-815
SEM_NET_TOGGLE_RATE C-815
SEM_NET_TOGGLE_RATE_FILE C-816

SEM_RECOVERY_FACTOR C-816
SEM_SLEW_OPTIMIZATION C-816
SEM_VECTORLESS_TIME_STEP C-816
SIMULATION_CACHE_DIRECTORY C-857
SINGLE_CUT_SLICE_LAYERS C-831
SIZE_BASED_PININST_CURRENT_DISTRIBUTION C-831
SKIP_CONNECTIVITY_CHECK C-831
SKIP_LAYERS_FILE C-831
SLEW_MIN_TRANSITION | SLEW_MAX_TRANSITION C-846
SPLIT_SPARSE_CONTACT_VIA_ARRAY C-831
SPLIT_SPARSE_VIA_ARRAY C-832
SPLIT_VDD_EXTRACT_LP C-794
SPLIT_VDD_EXTRACT_LP_FSIZE C-794
SPLIT_VIA_ARRAY C-832
STA_CRITICAL_PATH_FILE C-780
STA_FILES C-780
STA_VCD_FREQ_RATIO C-843
STANDARD_CELL_HEIGHT C-781
STATE_PROPAGATION C-843
STATIC_CONNECT_INST_FLOAT_GND C-832
STATIC_IR_HIDE_DISCON_INST C-832
STD_CELL_SINGLE_NOMINAL_VOLTAGE_ONLY C-795
STEINER_TREE_CAP C-847
SUBSTRATE C-743
SUPPLY_NETS C-795
SWITCH_MODEL_FILES C-870
SWITCH_MODEL_XTR_FILES C-870
SWITCH_RES_FILES C-871
TECH_CONNECTIVITY_FILE C-782
TECH_FILES C-782
TEMPERATURE C-781
TEMPERATURE_EM C-817
TEMPERATURE_REPORT_LINE_NUMBER C-796
TEMPERATURES_EM C-817
THERMAL_COUPLING_PITCH C-817
THERMAL_MODEL C-887
THERMAL_PROFILE C-782
TOGGLE_RATE C-844
TOGGLE_RATE_RATIO_COMB_FF C-782
TOTEM_IR_REPORT_LINE_NUMBER C-796
UNITS C-743
USE_DEF_VIARULE C-783
USE_DRAWN_WIDTH_FOR_EM C-817
USE_DRAWN_WIDTH_FOR_EM_LOOKUP C-818
USE_FAST_DECAP_ALG C-848
USE_INTERP_ETCH_FOR_R C-832
USE_LIB_MAX_CAP C-848

USE_MF_SWITCH_MODEL C-871
USE_MVM_PIN_MODEL C-832
USE_ZERO_WIRE_R_COMP C-833
USER_STA_FILE C-783
VCD_CONSISTENT_SCENARIO_FILTER C-786
VCD_FF_EDGE_TRIG C-857
VCD_FILE C-784
VCD_PREPARE_SCENARIO C-857
VCD_SCENARIO_COMPRESS C-857
VCD_X_LOGIC_STATE C-887
VDD_NETS C-796
VECTORLESS_BLOCK C-887
VIA <via_name> C-744
VIA_COMPRESS C-818
VIA_IR_REPORT C-797
VIAMODEL C-749
VP_CONTROL C-872
WARNING_COUNTS C-878
WARNING_LOG_COUNTS C-878
WIRE_R_COMPENSATION C-833

Chapter 1
Introduction

Silicon Integrity Analysis

ANSYS's Totem™ silicon integrity solution is a transistor-level product for analyzing and verifying the design integrity of power/ground structure, substrate, power and signal electro-migration, static and dynamic power performance, and electrostatic discharge, as shown in the Figure 1-1 diagram, from early in the design flow through sign-off. Static IR drop, which determines the IR-drop based on the average value of the power distribution, is merely the DC component of the solution. The more significant components are the AC effects involving temporal relationships between switching events of transistors and the impact that capacitance and inductance have on power integrity.

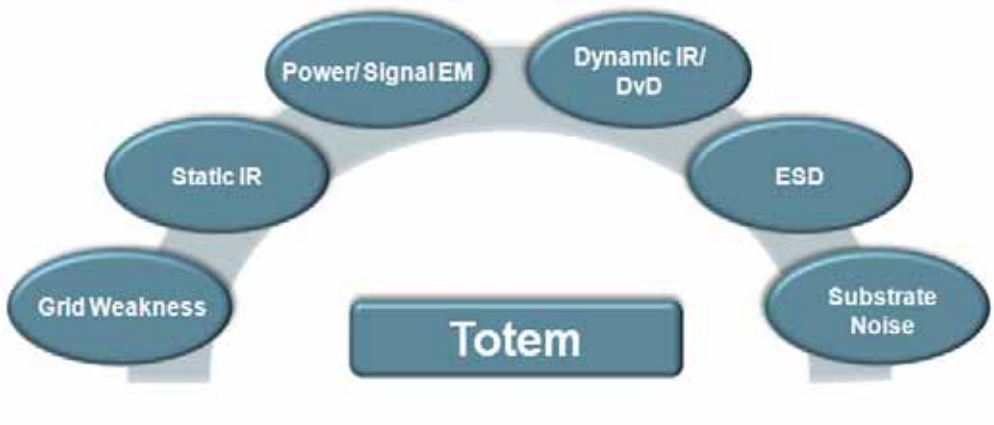


Figure 1-1 Totem analysis functions

The Totem product suite consists of two core applications, Totem-Power Integrity and Totem-Signal EM. Totem-Power Integrity supports early stage P/G weakness detection, and static IR, Power EM (electro-migration) and dynamic voltage drop analyses. Dynamic Voltage Drop (DVD) analysis includes on-chip, as well as package parasitics, for accurately modeling the design and predicting worst-case voltage drop due to issues such as simultaneous switching, insufficient strapping, and inadequate decoupling capacitance.

The comprehensive physical power integrity solution offered by Totem encompasses several breakthrough technologies, including analog/digital and mixed-signal full-custom, as well as semi-custom designs that have a combination of place and route digital blocks along with full-custom IP blocks.

One of the underlying technologies is Totem's Vectorless Dynamic statistical analysis engine, which computes realistic worst-case switching scenarios without requiring functional vectors for standard cells inside digital P & R blocks. For full custom digital and analog designs, Totem uses the current profiles generated from the functional simulation(s) in APL. During dynamic simulation, it back annotates the current drawn by the transistor(s) to the physical location in layout. It has an internal extraction engine that can extract RC values, as well as inductance, for the P/G network, so there is no need to rely on external extraction engines for power grid parasitics. Totem can concurrently analyze multiple P/G domains, which is very useful in analyzing the impact of noise injected by switching activity in one domain on other domains. This is a critical issue in analog designs, which are much more sensitive to injected digital noise. Severe peak voltage drop areas are identified as hotspot regions, which can be fixed using new decap placement, grid resizing, supplementary power routing, and/or swapping key cell instances at power hotspots. Upon identifying these hotspots in the power networks, Totem helps determine how to modify the grid or the decap placement, or swap a cell instance at needed locations, to reduce the voltage drop.

Totem is fully interoperable with industry standard file formats for technology, design, and library descriptions. To support the requirements of complex custom and semi-custom designs, such as mixed signal designs, Totem accepts LEF/DEF and other standard inputs for place and route blocks, as well as GDSII for memories, RDL, I/O and other custom IP blocks. In addition, Totem supports SPICE, Spectre, and DSPF netlist formats to characterize and generate Apache Power Libraries data to ensure SPICE level accuracy.

While accuracy is one of the primary goals of Totem's physical power flow, it is equally important to provide designers with an easy-to-use product. Totem has embedded RLC power-ground-substrate extraction, power calculation, and transient simulation to enable electro-migration, static IR and dynamic voltage drop analysis, and can back-annotate the noise waveform to the SPICE netlist in a single integrated run. Totem also can be used from early design stages with a minimal set of inputs. Since it is a layout-driven tool it provides visual feedback to the designers about the weaknesses in their designs with IR maps at the wire, instance and transistor level. “What-if” analysis of wires/vias, and power pad and decap optimization can be performed on-the-fly to asses the impact of the design changes on the voltage drop. Totem also has a cross-probing capability that can interface with industry standard layout editors such as Virtuoso and Laker so that designers can easily verify the feasibility of their optimization changes in their layout environment. Totem can also extract and concurrently analyze multiple P/G domain designs that are more common in analog and custom digital designs.

Using Totem in the Design Flow

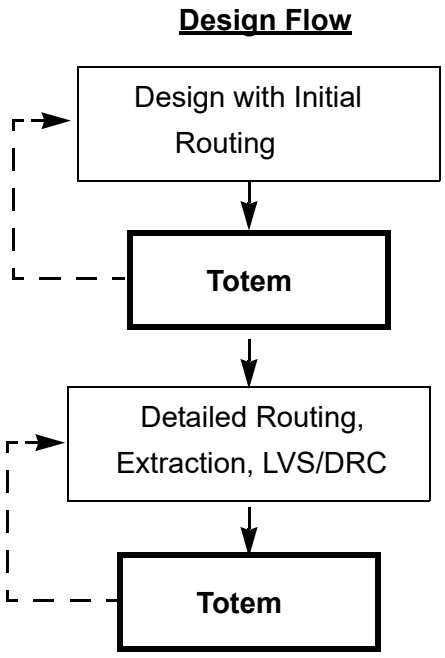
Totem is fully compatible with industry standard formats and easily fits into existing custom design flows. Totem’s physical power methodology, illustrated in Figure 1-2, is easily integrated into the two primary stages of chip design: Early Design and Design Verification. The use of Totem in these design phases is described in the following sections.

Totem supports two methodologies for dynamic analysis, depending on where you are in the design flow. The first method is based on user-defined power specification, which provides early feedback on static and dynamic hotspots. The second method is Apache Power Library (APL), based dynamic analysis, which provides transistor-level accuracy during verification, based on transistor-level current waveforms in the characterized APL.

Early Design Analysis

Totem enables early design analysis of static IR drop, and dynamic hotspot estimation using a user-defined power specification for analysis. Typically, graphic presentation of IR drop and both power and signal EM maps can reveal weaknesses of the power/ground mesh structures or the shortage of power/ground pads. Power-grid distribution issues can be easily remedied by wire-sizing or adding more straps or vias. At this point, switching to a flip-chip package may alleviate the static IR power distribution problems. In addition, by black-box consideration of memories and other content which is not yet available, dynamic hotspots due to simultaneous switching can be flagged in individual blocks or on the entire design.

For designs with embedded place and route blocks, after detailed cell placement, the design is represented by hybrid of input formats, such as LEF/DEF for logic core and GDSII for memories and flip-chip power bumps. If the SPEF or DSPF of the signal nets is unavailable, Totem uses Steiner tree wire loading to compute power distribution. The use of slew information from the static timing analyzer (STA) for place and route blocks further increases the accuracy of power analysis. Potential EM problems can be identified. Once the dynamic hotspot regions are identified, a rapid “what-if” analysis can be performed to assess the impact of package model inductance and intentional decap. Early analysis of the decaps ensure that proper area and location are allocated during placement, rather than after routing. Decap must be placed as close as possible to the victim to ensure proper protection from current spikes.



Early Design Analysis

- Perform user-defined power spec analysis of static and dynamic hotspots and repair
- Evaluate static IR distribution and EM problems
- Estimate dynamic hotspots
- Evaluate and fix power grid design and current demand imbalances
- Explore and modify decoupling capacitance

Design Verification

- Perform precise APL-based analysis of dynamic hotspots and repair
- Optimize decap usage and protection
- Analyze dynamic voltage and its impact on timing
- Evaluate and fix remaining power-related design issues
- Verify design with SPICE sign-off accuracy

Figure 1-2 Using Totem throughout the design process

Design Verification

Totem uses detailed accurate RC loading information from DSPF for signal EM sign-off for full custom digital blocks, and uses SPEF/STA information for place and route blocks. For power integrity and reliability sign-off Totem uses the entire power grid network and Apache Power Library with transistor-level current draw and decoupling capacitor information to achieve SPICE level accuracy, with significantly shorter turn-around times. It also provides a single interface for performing power grid and decap optimization and verification. Based on this dynamic analysis, transistor-level voltage drop information can be back-annotated to critical path simulations to asses the impact of voltage drop on timing.

Summary

Due to higher operating frequencies, higher power densities and lower operating voltage, power noise is a key concern in 65nm processes and below. Dynamic power and peak voltage drop is difficult to analyze and correct, being a transient phenomena, and its impact on chip timing and yield is a growing concern.

Major custom design houses and IP supplier companies benefit by using ANSYS's Totem tools to design and verify the dynamic power integrity of designs. At the 65nm process node and below, dynamic power integrity and signal EM integrity are important design sign-off requirements.

A comprehensive mixed transistor-level and cell-level methodology for power integrity analysis must address the following issues:

- power noise, reliability analysis and verification
- accurate modeling of standard cells, as well custom macros in the design
- the impact of on-chip and package parasitics on P/G noise
- ease of performing design analysis and optimization
- concurrent analysis of multiple P/G domains, to accurately asses the impact of noise coupling from one P/G domain to another

Totem's physical power solution delivers all of these capabilities in a dramatically faster, and easy-to-use design environment. Design teams can confidently deploy Totem's physical power methodology throughout their design flows, taping out on schedule, and with power integrity insured to silicon.

Chapter 2
General Setup and Flow

Introduction

Totem™ design integrity analysis performs power/ground grid checking, static IR drop, electro migration (EM) and dynamic voltage drop analysis on designs with transistor-level accuracy. The different types of design styles handled are as follows:

- Embedded memory macros, such as compiled memories, register files, TLBs, and cache memories
- Full custom hierarchical designs, such as flash memory, CAM, DRAM, SRAM, and FPGA chips
- Analog designs
- Advanced power gated and LDO designs
- Mixed signal designs

The Totem setup and analysis procedure is similar to that for cell-based designs using RedHawk, with some modifications in data input and processing methodology to account for the fact that the analysis is performed at the transistor level of the design rather than at the cell level. This chapter describes the basic setup steps that are independent of the type of design For modifications to the basic setup flow that are required for specific types of designs, see Chapter 5, "Design-Specific Setup".

Support of Multiple Design Styles

For each of the typical chip design styles, Totem can provide several types of analyses, as follows:

Custom Block/Macro Only

- GDS-only
  - P/G grid check
  - Static IR and EM analysis
  - Device current estimation based on the size and number of contacts
- SPICE-based
  - P/G grid check
  - Static IR and EM
  - Device size estimation based on the SPICE netlist
  - SPICE-based custom macro DVD

System on a Chip

- Top-level LEF-DEF
- Custom macros with GDS and SPICE netlist
  - P/G grid check
  - Static IR and EM analysis
  - DvD analysis with mixture of custom macros and cell-based blocks

Full Custom Chip

- Top-level GDS only
- Custom macros with GDS and SPICE netlist
  - P/G grid check
  - Static IR and EM analysis
  - DvD analysis with mixture of custom macros and cell-based block

Totem Flow Overview

Introduction

The Totem flow is summarized in Figure 2-1. Some of the key advantages of this flow are:

- easy single-pass setup
- multiple analysis modes
- integrated package handling
- on-die P/G RLC extraction
- model creation for SoC

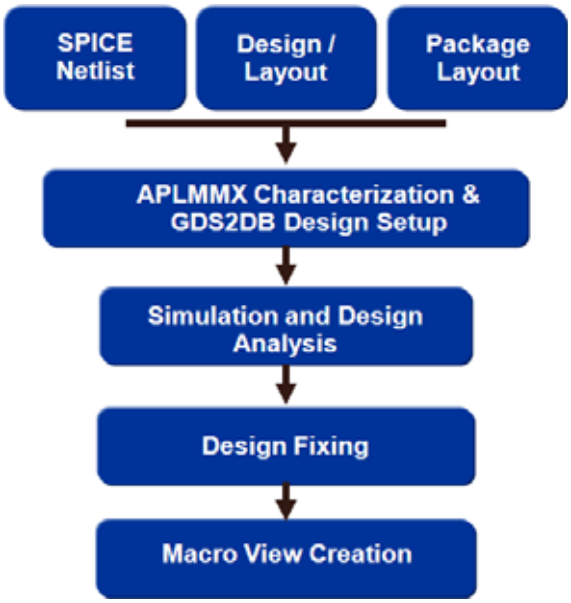


Figure 2-1 Totem flow

Details of setting up Totem analysis are described in the following sections.

Static and Dynamic Analysis Design Input Data Requirements

Totem uses industry-standard design input data for analyzing power noise and reliability on all types of analog and mixed signal designs. The design data needed are described in the following table, and in more detail in the section "APLMMX Data Preparation", page 2-12.:

Design Data	Relevant Information
Technology File	Includes definitions of all metal and device layers. Required.
GDSII File	GDS layout file. Required.
GDS Layer Mapping File	Drawing and Text Layer information, including all metal/device layers. Required.
Device Model File	Spice file for the specific technology. Required
Schematic Netlist	Flat/Hierarchical Spice netlist (*.*cdl/*.sp/*.cir*). Required.
DSPF Netlist	Signal-only DSPF netlist, either Signal RC Extracted or C-only with Placement Info File. Required
LEF File	Master LEF File, if available. Optional.
P/G domain to be analyzed	Definitions of top-level P/G domains to be analyzed. And also internal domains if the design has power switches. Required.
Pad Location (Voltage Source)	Can be any of the following: Specific Text labels in GDS Pin layers in GDS Explicit locations (*.ploc) Required
Switch Cell names	Switch cell master names. Required only for power-gated designs

For more details on preparing the various types of input design data, also see section "Totem Flow Description", page 2-9.

Totem Data Preparation

Once the required design input data is available, the first step in preparing to run Totem analysis is to generate probes, perform “pre-characterization”, and create electrical and physical device models for he circuit to be utilized by Totem. Pre-characterization is a process in which the pre-layout netlist of the design is simulated in a Spice-compatible simulator and the device currents are captured. Once the device currents are captured, a current model is written out. Then the layout and current models are read in along with the optional package netlist and a full analysis in Totem is performed. The following outlines the steps involved in creating data required for dynamic analysis:

- probe generation with APLMMX
- running Spice simulation
- generating electrical models with APLMMX
- preparing the physical models with the GSR file.

These steps are described in more detail in the following sections.

Probe Generation

This step generates the necessary inputs for the Totem flow, which involves identifying the transistors that are connected to the power and ground network from the netlist and creating a set of probes for these transistors, as shown in Figure 2-2. What is a “probe”? The Totem utility 'aplmmx' is used to generate the probes. The configuration file for *aplmmx* has the required pointers to the netlist files and also information regarding the power and ground nets. The *aplmmx* output file has an extension *.*aplmmx,* and has a list of all the probes.

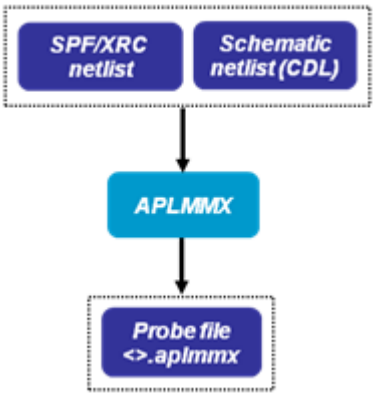


Figure 2-2 Probe generation flow

Running Spice Simulation

Next you must pre-characterize the currents consumed by all the devices in the design. This pre-characterization step involves running a Spice simulator with a set of vectors to capture the currents consumed by all devices, as shown in Figure 2-3. The Spice simulation is run on a pre-layout extracted netlist for power and ground. That is, the netlist that is used for simulation should not have parasitics for the power and ground network.

Typically this pre-characterization simulation takes a relatively short time in any fast Spice simulator. The probe file that was generated in the previous step should be included during Spice simulation to ensure that the device currents are being probed. The simulation output file can be any Spice-compatible format, such as *fsdb*, *raw*, or *tr0*.

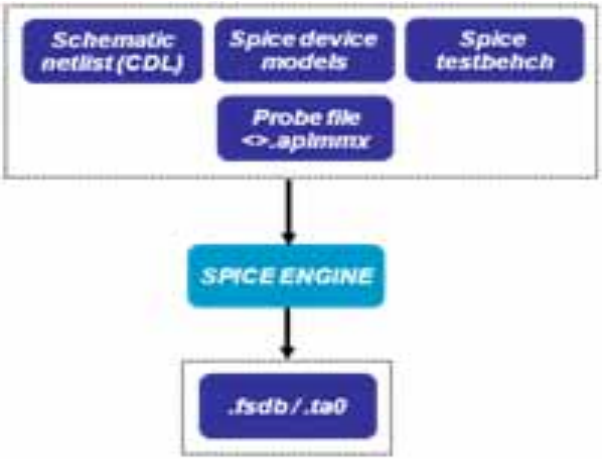


Figure 2-3 Spice simulation flow

Generating Electrical models in APLMMX

After Spice pre-characterization is completed, the simulation outputs for the device currents are converted to an electrical model for use in Totem analysis, as shown in Figure 2-4. The electrical model is generated using the 'aplmmx' utility with the proper configuration file. The outputs of this stage are the current files (*<cell>.spcurrent)*, the transistor model mapping files (*<cell>.gdsmmx),* and the capacitance models (*<cell>.cdev*). See Chapter 3, "Characterization and Modeling", for descriptions of all APLMMX configuration file keywords.

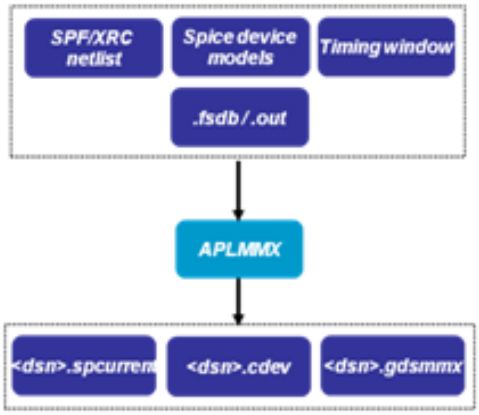


Figure 2-4 APLMMX characterization flow

Note that if you *only* want to run static IR analysis, a GDS-only flow is available and APLMMX is not required.

Preparing the physical models

The final step in preparing to run Totem dynamic analysis is to prepare the Global System Requirements (GSR) file, with references to all the relevant design and technology data. The GSR file is a global configuration file that contains pointers to the technology file, current models, GDS configuration file, pad locations, and any other required files. When Totem is run in the GUI or in batch mode, the GSR file is read in for setting up the database and doing simulations. Descriptions of GSR keywords are provided in section "Global System Requirements File (*.gsr)", page C-772.

When the required design data and Totem-specific data are ready, as described in this chapter, see Chapter 4, "Static and Dynamic Analysis".

Totem Flow Description

GDS-based Setup

Layout only (Grid-Check, Static)

Totem can be run in a 'gridcheck' and 'static' mode when only the GDS is available. During early stage analysis, the netlist, DSPF or any electrical information is not available. Totem can read in a GDS only and assign currents to devices based on active/transistor geometries.

The current assignment is based on the contact density. Devices with a larger number of contacts are assigned larger currents Tap cell contacts are intentionally ignored during

this analysis. When the contact and base layers are not specified in the GDS2DB layer map file, the bottom-most via is used for current assignment. The current depends on the density of the vias present. This current assignment, although not electrically accurate, provides significant value during the early stages of a design.

In some cases, when current information for specific regions is available, Totem can assign currents using the BLOCK_POWER_ASSIGNMENT GSR keyword to 'regions'. This approach can yield a more accurate current distribution at an early stage.

Netlist-based Setup

The following section summarizes netlist-based setup in the APLMMX characterization flow and the tools used.

Type	LVS Based Flow	Extraction Based Flow
Netlist	Schematic Netlist	Schematic Netlist(optional)*
Layout back-annotation	Layout Netlist from LVS Instance cross-reference file (IXF) Net cross-reference file (NXF)	DSPF with RC extraction Layout Netlist (no back-annotation) DSPF with C-extraction + Placement Info File
Relevant tools	Calibre or Hercules LVS Assura	Calibre-XRC/Star-RC/ Fire & Ice

Layout + DSPF/Layout netlist (Static)

A pure static analysis can be performed with only layout, or with layout and the LVS netlist. When providing only layout to Totem, it performs the current assignment based on the size of the contact via arrays connecting the lowest metal layer to the device. There is no back annotation to the schematic names.

Typically LVS tools dump out a layout netlist that may or may not correspond to the schematic netlist names or hierarchy. If you want to run pure static analysis flow, this information is good enough to get started with the analysis. You must ensure that the layout netlist has “$X and $Y” co-ordinates for the transistors and other devices. With this flow, the current assignment is based on the size of the devices in the netlist. The GUI has back-annotation to the netlist provided to this flow.

Layout + Schematic netlist + DSPF/Layout netlist (Static/Dynamic)

Extraction-based flows rely just on the DSPF netlist. The DSPF should be a signal RC extracted DSPF. In absence of RC extraction the co-ordinates of the devices are not available. You can also provide a Signal C-only extracted DSPF along with a Placement Info File (available only in StarRC-XT flows).

In the typical Totem flow, the schematic netlist (*.cdl, .sch, .sp*) are used for simulation. The currents are captured from this simulation and then mapped to the transistor locations in the DSPF netlist using the APLMMX utility. For this method to be accurate, the mapping/hierarchy between the DSPF netlist and the Schematic netlist should be consistent. Refer to section "Device mapping between DSPF and the schematic netlist", page 2-15, for details on mapping between schematic and DSPF netlists.

With Layout netlist for Decap co-ordinates

This is the most commonly used functional simulation setup for large analog and custom digital designs (especially memories). Typically, fast spice simulators take advantage of the hierarchical nature of the design and therefor those who want to simulate large

hierarchical designs with repetitive instantiations use them. Note that an identical simulation setup for Totem current profile generation should be used, but in order to get the X,Y locations of the decaps, the Layout netlist from LVS (w/o any back annotation) as DECAP_NETLIST is used. APLMMX provides this DECAP_NETLIST to ACE to get the capacitance values, and distributes it at appropriate X,Y locations, as shown below:

```
SPICE_NETLIST <schematic netlist>
DSPF_NETLIST <DSPF file>
DECAP_NETLIST <Layout netlist>
```

Sometimes the device names (models) in the LVS layout netlist are different (changed for LVS purposes) from the names in the schematic or DSPF file, so you must ensure that the layout netlist provided by the LVS flow can be simulated in Spice, so that ACE can successfully perform AC analysis.

In some cases you can extract a DSPF w/ MOS X,Y locations in the '*|Instance' section, instead of the '*|I(NETS)' section. In this case this netlist can also be used as a layout netlist with back-annotation, and there is no need to provide an additional LVS layout netlist for APLMMX to get decap X,Y co-ordinates.

Note that when the DSPF has C-only extraction, the NETS section in DSPF have no device coordinates, so you must enable dump out of PLACEMENT_INFO_FILE (available in the StarRC flow) to get the device X,Y coordinates. The keywords are given below.

```
#APLMMX config file specification of DSPF and placement info
SPICE_NETLIST <Schematic netlist>
DSPF_NETLIST <DSPF netlist> <Placement_info_file>
```

LVS-based Flows

When using LVS results as input, you must get the cross-referencing between the schematic and layout generated by LVS tools. Different tools have different LVS results formats. Totem has native support for Calibre LVS results. Hercules (Synopsys) and Calibre (Mentor) are predominantly used in the industry. A few customers, especially analog design teams, use Assura (Cadence). You must be clear about what inputs are needed to get from your LVS tool for Totem, and then Ansys AEs can help develop a script to convert the results of the LVS tool into APLMMX format.

LVS netlists are read into the APLMMX configuration file using the following keywords.

```
SPICE_NETLIST <Schematic netlist>
LAYOUT_NETLIST <LVS Layout netlist> <IXF file>
```

Calibre-based flow

Typically LVS flows generate only a summary pass /fail report, and may not generate a detailed back-annotation report. You need to enable certain Totem switches to dump out detailed back-annotation, or cross-references between schematic and layout. The command to generate a back-annotated cross-reference (XREF) file from Calibre-LVS is shown following:

```
# Calibre command to generate the back annotated netlist
% caliber -lvs -ixf -spice <spice_netlist> <SCRF_rule_file>
```

Typically Calibre dumps out an IXF file along with the layout netlist. If the net names in the layout and schematic are different, Totem needs an NXF file, which Totem and APLMMX support.

Hercules-based flow

Note: Please refer to the latest Hercules User Manual for the most accurate information. The information provided here is only for guidance. The following Figure 2-5 shows the Hercules data flow.

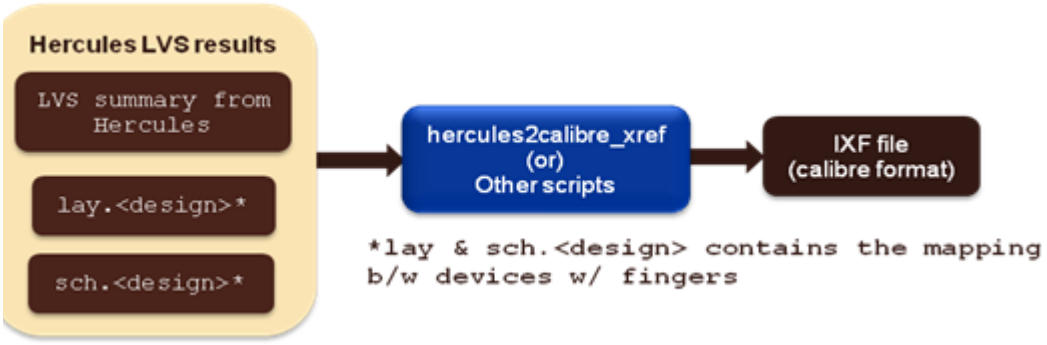


Figure 2-5 Hercules data flow

Hercules LVS results are not natively supported by Totem. Hercules can dump out a layout netlist in *<>.net* format by default, which can be converted into *<>.sp* (HSpice) format using the 'nettran' utility, with the following commands:

```
# Hercules command
% Hercules <lvs.ruledeck>
# Nettran command to convert to spice format
% nettran -hercules <dsn>.net -outType spice -outName <dsn>.spi
```

Hercules dumps out only the LVS summary report that has the top 1000 cross-references between the schematic and layout. In order to get the needed data for Totem-APLMMX, you must run a Hercules compare run using the following steps.

1. Modify the Hercules equiv file (*<design>.equiv*) to add "EQUIV <design_name> = <design_name>"
2. Edit the *<>.ev* file to enable CMP-60,CMP-61,CMP-62,CMP-225 with the message_enable option, for Hercules to dump out the cross-reference files.
3. Run the Hercules Compare run as follows

```
% hercules -C  <design>.equiv  <>.ev
```

The Hercules-Compare run then dumps out the LVS summary report, along with two additional files, *lay.<design>* and *sch.<design>,* which have the back-annotation information, along with the locations of the multi-finger devices. This file is then fed into script "*hercules2calibre_xref.pl*", or any custom utility to convert it to Calibre IXF format for APLMMX.

Other LVS flows (such as Assura) can also be supported if the cross-reference files are provided to APLMMX in IXF format.

APLMMX Data Preparation

Requirements for Spice Netlists

A legal HSpice, Spectre or ELDO netlist is supported in APLMMX. The netlist can be hierarchical or flat; however, it is best to use a hierarchical netlist if the Spice simulation setup uses a hierarchical netlist. The netlist can be a schematic netlist or layout netlist generated from LVS flows. If SPICE_NETLIST is the only keyword in the APLMMX configuration file, then the tool expects the netlist to have the device x,y co-ordinates for layout back-annotation.

Probe generation from the netlist

- Generate the probes using “aplmmx -s “ option.
- Ensure that the top level instance name is the same in APLMMX and in the Spice simulation setup. You can modify the top level instance call name in the Spice test-bench using the following keyword in the *aplmmx.config* file:

  ```
  TOP_XCELL < >
  ```

- Sometimes the Spice simulation setup does not need to have a top level subckt instantiation, in which case you must be sure to specify the keyword TOP_XCELL and provide an empty string (“ “). By default APLMMX uses ‘XCELL’ as the top X instance name. For example:

  ```
  TOP_XCELL ""
  ```

- Ensure that you are setting up the appropriate Spice simulator name in the APLMMX configuration file using the keyword SPICE_SIMULATOR. Different Spice simulators have different syntax conventions for probes. For example, HSIM and HSpice simulators have one convention for current direction, whereas other simulators have a different convention. Also, some simulators require the syntax to be “probe i1(xtor)”, whereas others require it to be “probe i(xtor:D)”. An example of specifying a correct Spice simulator as HSIM is shown below

  ```
  SPICE_SIMULATOR HSIM
  ```

- If you are using SBA (Structural Back Annotation) in HSIM, be sure that you are setting the keyword ‘HSIMSBA’ in the APLMMX configuration file, in order for APLMMX to handle the multi-finger transistor currents correctly.

  ```
  HSIMSBA 1
  ```

- Usually APLMMX uses “par” statements (parameterized variables) to encode the transistor probe names to *<domain>.apl** names. If the simulator is having difficulty in understanding it, use the ‘PROBE_NO_PAR’ keyword in the configuration file to generate simple probes. For example:

  ```
  PROBE_NO_PAR 1
  ```

- Sometimes it is necessary to probe the devices in a netlist at the model subckt for transistors, such as when the model contains extra parasitic information in addition to the .model statements. In order to probe at the model subckt level, use the keyword:

  ```
  PROBE_STOP_AT_MODEL_SUBCKT 1
  ```

- APLMMX also supports instance-level probes. However, keep in mind that the accuracy of the analysis in this case may be questionable because of the ambiguity in snapping the current sinks to correct locations in the layout.
- The netlist that is used to generate the probes should also be used for pre-characterization simulations, to ensure that the device hierarchy of the probes match the hierarchy of the actual simulation netlist. For example, you cannot generate the probes for a hierarchical CDL netlist and use it for simulating a DSPF netlist in Spice.

Requirements for DSPF file (Star/Calibre/Assura)

- Any DSPF provided to APLMMX should be a signal RC extracted DSPF.
- The DSPF should have the coordinate information in the *|I section of the file.
- The DSPF should have schematic back-annotated names for proper handling by APLMMX.
- If the DSPF is a C-only DSPF, then you need a placement info file for APLMMX to map the device locations.
- If you have power switches in the design, do not extract the RC of the power gated domain also, and that DSPF has both NET and INSTANCE sections in it.

- Refer to the sample rule deck below to be sure that all relevant options are set to get the correct DSPF netlist.

Rule deck requirements for StarRC-XT

- XREF: YES <YES is required, turns on back-annotation, requires Hercules LVS run>
- NET_TYPE: SCHEMATIC <SCHEMATIC required, back-annotation>
- POWER_NETS: <net name or string> defined power nets in schematic, wild cards accepted, such as *VDD* *VSS*>
- POWER_EXTRACT: NO <NO required, NO does not extract P/G nets>
- EXTRACTION: RC <RC required, must not use C or CONLY because DSPF format is not supported with C or CONLY>
- NETLIST_FORMAT: SPF <Required, DPSF format>
- NETLIST_FILE: <output file name>
- NETLIST_GROUND_NODE_NAME: GND <ground node name for grounding capacitors in DSPF>
- COUPLE_TO_GROUND: YES <grounds all coupling capacitor, recommended for faster runtime. perhaps some analog blocks need coupling caps>
- PLACEMENT_INFO_FILE <YES|NO> Required when a c-only DSPF is extracted

Refer to the Assura and QRC user guides to set up DSPF generation.

Refer to the Calibre user guides to set up DSPF generation.

Coordinate Requirements in DSPF

Ensure that the DSPF has a “*|I” section in it. Sometimes DSPF can also have co-ordinates in the 'INSTANCE' section, the section with transistor instantiation and its x,y co-ordinates, in which case this DSPF can also be used as a layout netlist to get transistor x,y locations. You can also use a C-only DSPF file and use a PLACEMENT_INFO_FILE generated from StarRC in addition to DSPF to back-annotate the transistor x,y co-ordinates, using the syntax in Star-RC file:

```
DSPF_NETLIST <dspf_file> <placement_info_file>
```

Note: This option is only available in Star-RC.

Verify if the co-ordinates in DSPF are to the original scale or shrink. Typically when doing extraction for a half-node, layouts are not scaled, but the extraction tools are smart enough to shrink the geometries for extraction. Extraction tools have the option to dump out original polygon co-ordinates, or to dump out a shrink version. If scaled/shrunk coordinates are present in DSPF, then use the ‘NETLIST_XY_SCALE’ keyword in the APLMMX configuration file and set it to 1/<scaling_factor> to make the netlist x,y locations match the layout. Later the LEF/DEF_SCALING_FACTOR keyword (in the GSR) can be used to get scaled x,y co-ordinates in Totem.

Sample DSPF format

DSPF netlist is always a flat netlist. A sample DSPF format is provided below, with details on different subsections needed by APLMMX. For power net analysis, Totem only needs the transistor x,y locations and does not need any RC parasitic information.

```
.SUBCKT TOP vdd1 vss vdd2
...
*|GROUND_NET 0
...
*|NET AAA 0pF
```

```
*|I ( xi80/mxpa:S xi80/mxpa S B 0 360.0 350.0)
*|I ( xi80/mxpa:D xi80/mxpa D B 0 360.0 351.0)
*|I ( xi80/mxna:S xi80/mxna S B 0 340.0 350.0)
*|I ( xi80/mxna:D xi80/mxna D B 0 340.0 351.0)
...
*|Instance
Mxi80/mxpa xi80/mxpa:D VDD xi80/mxpa:S VDD pch w=0.1u
l=0.01u
...
.ENDS
```

Device mapping between DSPF and the schematic netlist

Device mapping mismatching between the schematic and DSPF/layout netlist is one of the main reasons for APLMMX failures. Typically the DSPF netlist and the schematic/simulation netlist can have mismatches in terms of hierarchy, device naming conventions, finger delimiters, or bus delimiters. APLMMX has several mapping keywords that can be used to fix these mapping issues natively, which avoids having to modify the netlist.

Finger delimiter setting

Typically “@” or “__” are used as finger delimiters, but LVS/extraction tools provide flexibility to customize the finger delimiters, so they can use any character as delimiters. If that is the case, you can define the finger delimiters in the APLMMX configuration file, using the keyword:

```
FINGER_DELIMITER <finger_delimiter>
```

Hierarchy, finger and bus delimiter mapping

The FLAT netlist is usually the layout/DSPF netlist. The hierarchical netlist is usually the netlist that is used for simulation. The default hierarchy mapping between layout -> hierarchical mapping is “/” -> '.' In order to change the mapping, you can use the keyword

```
FLAT_*HIER_MAP <flat_delimiter> <hierarchical_delimiter>
```

Prefixes to device types can also be controlled during the mapping process. The following keywords can be used:

```
FLAT_XHIER_MAP
FLAT_MHIER_MAP
FLAT_DHIER_MAP
FLAT_RHIER_MAP
FLAT_QHIER_MAP
```

Multiple device name mapping rules for each X, M, D, R or Q element are supported. For example:

```
FLAT_MHIER_MAP {
  "/MM" ".XM"
  "/MM" ".M"
}
```

Bus bits can also be mapped between a flat netlist and a hierarchical netlist using the following keyword:

```
FLAT_BUS_DELIMITER_MAP {“<DSPF bus delimitr>” “<schematic bus delimitr>”
    ...
}
```

The FLAT_BUS_DELIMITER_MAP keyword can also be used for finger delimiter mapping between flat and hierarchical netlists.

APLMMX can also resolve mapping issues between the Spectre extracted netlist and Calibre DSPF netlist by supporting non-default hierarchical delimiters and complex finger delimiters inside the SPICE_NETLIST.

Some Spectre netlists use “|” as a hierarchical divider and use finger delimiters that have complex syntax, such as “_<id>__rcx”. In such cases, the following keywords can be used to resolve mapping issues:

```
HIER_DIVIDER  <divider_character>
    FLAT_BUS_DELIMITER_MAP {
    "<flatten_netlist_finger_delimiter>"  "___rcx"
    ...
}
```

For example, if in the Spectre netlist the device name is:

```
I3|TP3|I1_1__rcx
```

and in DSPF netlist, the device name is:

```
XDXI3/XTP3/DI1@1
```

then in the APLMMX configuration file you can use the following settings to define the mapping rules:

```
HIER_DIVIDER |
# define the finger/bus delimiter mapping
FLAT_BUS_DELIMITER_MAP {
"@" "___rcx"
}
FLAT_DHIER_MAP {
"DI" "I"
}
FLAT_XHIER_MAP {
"X" ""
}
```

APLMMX also supports same symbol for bus-delimiter, finger-delimiter, and hierarchy divider.For example:

```
DSPF: XXI25<0>/MM33@2 -> SPICE: I25_0__MM33\@2
```

Here, DSPF is using “/” for hierarchy delimiter and “<>” for bus-delimiter whereas CDL is using “_” for both. User can use below mapping for it:

```
HIER_DIVIDER _
FLAT_BUS_DELIMITER_MAP  "<>" "__"
```

Simulation Requirements and Settings

Current polarity requirements

Totem internally defines current flow from the battery through the power net to devices and from devices through ground net back to the battery. Totem must modify simulator current polarity to fit its polarity convention, so defining the “SPICE_SIMULATOR” keyword in the APLMMX configuration file is very important.

The current flowing through different terminals of a MOS device is usually represented by *i<terminal_id>*. Typically, the mapping of the terminal IDs is as follows:

- i1 - current flowing into drain of device
- i2 - current flowing into gate of device
- i3 -current flowing into source of device

- i4 - current flowing into bulk of device

There are three categories in terms of simulator current polarity conventions, as summarized in the table following.

Current direction	**i1, i2, i3, i4 flowing into device**	**i1, i2, i4 flowing into device; i3 flowing out**	**I1, i2 flowing into device; i3, i4 flowing out**
Simulator	XA, Ultrasim, Spectre, aps, Eldo, Adit, NSpice, BDA, SmartSpice	HSpice, HSim, NanoSim	FineSim

Settings for different Spice simulators are described in the following sections.

HSpice Simulator

HSPICE is the industry's standard for accurate circuit simulation and offers foundry-certified MOS device model support. Some checks can be performed on the test bench before simulating:

- Probe file from Totem included.
- HSPICE uses '*.include*' to include the probe file as an input.
- Transient simulation settings (*.tran* statement should have correct start and end time).

The Totem-specific APLMMX configuration file setting for HSpice is:

```
SPICE_SIMULATOR HSPICE
```

HSIM Simulator

HSIM is a FAST Spice simulator that has two primary flows:

- standard SPICE-like analysis with a DSPF/CDL.
- Structural Back-Annotation flow (SBA), where the tool back-annotates the DSPF to CDL.

An example Test Bench file is shown following:

```
.TITLE FF_Fres_1p08_125C

.param mismatchflag_mos_18=1
.param mismatchflag_mos=1

.GLOBAL vdd!
.GLOBAL gnd!

.tran 10e-12 10e-9 START=0

.INCLUDE models.sp
.INCLUDE netlist.sp

.INCLUDE "../../dynamic_run/topcell.aplmmx"

.end
```

HSIM-specific settings are as follows:

```
**HSIM performance specific:
```

```
.param hsimspice=3  *default=0
.param hsimanalog=3 *default=1
.param hsimspeed=3 *default=3
.param HsimTstep2TauMax=1
.param hsimdetailbsim4 = 1
```

SBA-specific settings are as follows:

```
**HSIM back annotation
.param HSIMSPF= <DSPF NETLIST>.dspf
.param HsimSbaAutoSubModels=1
.param HsimSba=1
```

Some checks can be done on the test bench:

- Probe file included in the test bench.
- Accuracy mode (HSIMSPICE) should not be set too low, as this impacts the run-time.
- Check if it is a regular flow or an SBA flow.
- In an SBA flow, the first finger transistor has the sum of current consumed by all the fingers. The remaining fingers have zero current. If this is the case, use the keyword 'HSIMSBA 1' in the APLMMX configuration file.
- Check if the correct DSPF is included in the HSIM run. You should not include the netlist with P/G.

Totem APLMMX-specific settings when using HSIM/HSIMSBA are as follows:

```
# Turn on HSIM simulator for correct current directions
SPICE_SIMULATOR HSIM

# Turn this on when using the SBA flow in HSIM
HSIMSBA 1
```

XA Simulator

XA is fast Spice simulator similar to HSIM. The input testbench and Spice netlists are typically HSpice-compatible. Certain extra keywords are needed to perform back annotation.

An example of an XA DSPF back annotation file is as follows:

```
.option XA_CMD='load_ba_file -file filename.dspf'
.option XA_BA_DPF_SCALE_RESET=off
.option XA_CMD='enable_ba_error_net 1'
```

Totem APLMMX-specific settings when using XA are described in section "Support for XA as a Third Party Simulator", page 3-87.

UltraSim Simulator

UltrasSim can be run on a Spice-type netlist, or on a Spectre-type netlist. Refer to the latest UltraSim user manual for information regarding simulation settings.

Spectre Simulator

Refer to the latest Spectre simulator user manual for information regarding simulation settings.

Eldo Simulator

Refer to the latest ELDO simulator user manual for information regarding simulation settings.

Modules like Totem, APLMMX, ACE, rawutil and APL support 15.4 version of Eldo libraries. But these binaries are limited to support waveforms from 12.2 and higher version of Eldo only. APLMMX also reads the simulation output file dumped from latest ELDO 15.4 simulator. To enable this, User needs to set environment variable 'MGC_AMS_HOME' to the latest ELDO 15.4

Example:

```
setenv MGC_AMS_HOME /appls/mentor/15.4
```

In APLMMX config:

```
SPICE_SIMULATOR /appls/mentor/15.4/bin/eldo
```

NanoSim Simulator

NanoSim removes current probes for resistors from its simulation database. If you include resistor probes in the probe list, NanoSim does not take the syntax. APLMMX can automatically remove resistor probes from probe list when you use the keyword 'SPICE_SIMULATOR nanosim' in the APLMMX config file.

Finesim Simulator

Refer to the latest Finesim simulator user manual for latest information regarding simulation settings.

GDS Processing Setup

Introduction

The GDS2DB function in Totem provides efficient GDSII data processing inside to create physical models by reading in GDSII data directly and processing it in a native manner in the Totem database. GDSII stream format is binary, for representing planar geometric shapes, text labels and other information in hierarchical form. Totem accepts all GDSII data directly into the database through GDS2DB. The following are the key steps performed in modeling GDSII natively:

1. Prepare and read in GDSII input data and populate a hierarchical DB inside Totem, which includes the GDS2DB configuration file that describes GDSII data input, the top cell, net types, and layer mapping
2. Perform Net Tracing. Based on the P/G nets defined in the VDD_NETS or GND_NETS section in the GDS2DB configuration file, geometries for the defined nets are extracted and traced.
3. Perform Mini-LVS steps.
    a. Layer extraction and device recognition, such as MOS, BJTs, Diodes, and Poly resistors.
    b. Logical layer operations on device layer geometries.
    c. Detection of well taps (tap cells).
    d. Pin snapping. Identification and connection of the devices to the corresponding X,Y locations in layout.
4. Automatically create current sinks to assign current for static and dynamic analysis.
5. Optional Substrate analysis. Extract substrate features (EXTRACT_SUBSTRATE GSR keyword). GDS2DB can extract the well geometries and process them for Totem-CSE, the substrate extraction and noise coupling analysis tool. See Chapter 13, "Substrate Noise Analysis".

6. Optional transistor-level power gate analysis. For designs with power switches, the power-switch hierarchy must be described in GDS. For a detailed description of this setup step, see the section "LDO Low Power Designs", page 5-202. Relevant GDS2DB configuration file keywords are described in section "GDS2DB Configuration File Keywords", page 3-99.

These setup steps are described in the following sections.

GDS2DB Configuration File

Overview

You must create the GDSII configuration file, *<gds2db_filename>.config*. There are four required sections in the GDSII configuration file:

- a GDSII file that specifies a set of GDSII files for conversion
- top cell definition, which is the name of the top cell structure(s) in GDSII
- nets definition that identifies the power, ground and signal nets that are labeled in text in the GDSII
- layer map definition, which specifies the map file that contains information for mapping the GDSII layer numbers to the corresponding layer names defined in the technology file

In addition, there are several optional sections in the GDSII configuration file:

- extraction layer geometries
- auto pad location generation
- other keywords

The following is an example of a basic GDS2DB configuration file. TOP_CELL RAM256

```
GDS_FILE user_data/RAM256.gds
GDS_MAP_FILE user_data/RAM256.gds.map
XTOR_MAP_FILE ../aplmmx/RAM256.gdsmmx
VDD_NETS {
  VDD {
  VDD:
  VDD
 }
}
GND_NETS {
  VSS {
  VSS:
  VSS
 }
}
```

Descriptions of all configuration file keywords are in section "GDS2DB Configuration File Keywords", page 3-99.

Nets definition

Net name requirements for geometry extraction

The Power and Signal nets in GDS are traced inside Totem based on the text labels or a coordinate system. Without labels in the design, Totem cannot trace the nets. Therefore, it is a good practice to have labels in the top level of the GDS for the power and ground nets. By default, Totem uses the labels in all hierarchies in the GDS to trace the P/G nets, but this may sometimes cause shorts in the design because of improper label placement in lower level hierarchies. To avoid this issue, you can use the 'USE_TOP_LEVEL_TEXT_ONLY 1' keyword in the GDS2DB configuration file.

The text labels and layer numbers should be provided in the layer-mapping file accurately.

Location-based tracing is also available, where you provides the start point and name of net to trace in the GDS and Totem uses that start point as origin

LEF file requirements for geometry extraction

Totem uses a LEF file for a macro for two different functions:

1. To make sure the origin and prBoundary of the LEF macro is carried over to the analysis.
2. To use the LEF power/ground pins for tracing coordinates.

The macro origin and boundary is important when Totem is generating a model for SoC level consumption. By default, the origin and boundary of the macro depend on the metal geometries that were traced during GDS2DB operation. This can lead to improper placement of the macro at the top level during SoC validation in RedHawk. Always use the LEF_FILE keyword in the g*ds2db.config* file to avoid this placement issue.

Sometimes labels may not be available in the GDS top level hierarchy to trace a power or ground net. In this situation, the LEF file with the pin definitions for power and ground can be used as an input to guide Totem to trace the nets. The keyword 'USE_LEF_PINS_FOR_TRACING 1' can be used in the configuration file to do this.

GDS2DB Layer Map file

The Layer Map file contains information for mapping the GDSII layer numbers to corresponding layers defined in the Totem database. All the base layers (ndiff, pdiff, poly) in the GDS should be retained so that Totem can identify the current sinks accurately, and also extract the resistance of contacts and diffusion routes, if needed.

Some of the layers in a layermap must appear in a particular order, as follows:

- Entries for metal 1 and above, and the associated vias, must appear in ascending order, such as M1, V12, M2, V23. If some metal layers require BOOLEAN derivations, the BOOLEANs should appear in stack order, such as BOOLEAN M1, V12, M2, ...
- Entries for temp layers used in BOOLEANs must appear before they are used.
- Other entries may appear in any order, including metal 0 interconnect for finFETs or other processes, base layers, and substrate layers.

The format of the GDS2 layer map file allows multiple layer numbers to be specified, either as a layer number or as a text layer number. The layer map file syntax is as follows:

```
<Layer_name> <Layer_type> <Layer_number> <Text_layer_number>
[ <Via_name> v <Layer_number> - ( <upper_margin> <lower_margin> ) ]
```

where

<Layer_name>: specifies the name of the layer

<Layer_type>:

m = metal,

v = via

mim | mcap| c = metal cap

cont | contact = contact layer connecting to meta1 to diffusion or poly

pd | pdiff = p-diffusion

nd | ndiff = n-diffusion

nw | nwell = n-well layer

pw | pwell = p-well layer

poly = poly silicon layer

polyres = poly-resistor marker layer to recognize poly resistor geometries

d | diode = diode marker layer to identify a diode device

bjt = BJT marker layer to identify a BJT device from layout

sil = Silicide layer to identify active silicide region from layout

temp = A temporary layer used for doing further boolean operation to generate final layer geometries. See the Logical layer Operations section for more details.

<Layer_number> and <Text_layer_number> : can be specified by several layer-datatype pairs and layer numbers, separated by ';' (semi-colon). Layer and datatype are separated by ':' (colon). Text_layer_number can be ignored by inserting a dash '-'.

<Via_name> : specifies the via name

<upper_margin> <lower_margin> : specifies the margin around the via on the upper layer and lower layer, respectively (default: 20 nm)

The following is an example of a layer map file:

```
#Layer_name Layer_type Layer_number Text_layer_number
METAL1         m      1:5;10:0       11:1;13
VIA1           v      2;12           - (10 10)
METAL2         m      13:0;15:50     -
VIA2           v      3:12;3:43      -
```

In the above example, all GDS layers with a layer number 1, of Datatype 5 having a text layer 11 are extracted as METAL1 in the Totem database. Typically, you can get the above layer mapping information from the GDS stream-in configuration file in the library. Note that Boolean expressions can be used to define logical layers such as 'diffusion', 'active', 'pplus', and 'nplus'. GDS layer logical operations support four modes: A&B, A-B, !A&B, and A|B. This can be helpful in correctly identifying special cases, such as tap cells:

```
BOOLEAN pdiff pd active&pplus
BOOLEAN ndiff nd active-pplus
```

BOOLEAN operations are supported for ESD marker layers too. Example

```
esd_tmp1 temp 98:0 -
esd_tmp2 temp 99:0 -
BOOLEAN ESD_MARKER ESD (esd_tmp1&esd_tmp2)
```

See the description of Logical Layers operations in the Mini LVS Features section below for more details.

The keyword setting ‘CONTACT_RESISTANCE 1’ should be specified in the GDS2DB configuration file to extract the pdiff/ndiff/poly layers from GDS. GDS2DB identifies well taps based on overlap between nwell/ndiff and contact, or pwell/pdiff and contact, so be sure that these layers are specified in the layer map file for GDS2DB to identify the tap contacts.

When a layer is used in a Boolean operation, be sure that it is defined as a “temp” layer earlier. If multiple layer numbers/Booleans are used for PDIFF, “OR” functions can be used to group them as one layer. For example, if the GDS has layer 25:0 used as pdiff in region A, but 25:0 &6:0 active as PDIFF in region B, then the layer map file should have the following definition:

```
active temp 6:0  -
pd_temp temp 25:0 -
contact cont 30:0 -
M1      m    31:0 -
...
BOOLEAN pdiff pd  (pd_temp |( pd_temp &active))
```

Any routing connections through the silicide layer (typically called OD or active) in the layout is handled by using the keyword “SILICIDE” in the layer map file, and also by specifying “TAP_CONTACT_EXCEPTION 1"

```
Layer1 temp -
Layer2 temp -
SILICIDE sil (layer1|layer2) -
```

The keyword “text” recognizes text- based device markers that support BJT/Diode/ resistor(poly|diff|cm) devices, as shown in the following example:

```
GDS_LayerMap :
marker temp 10:0 -
#<Name>  text   <text layers>  <Text value for device>
dio_txt text 10:1;10:2 pw_diode_t;nw_diode_t
bjt_txt  text 10:1 bjt_t
# To recognize device with marker layer and text overlapped

BOOLEAN dio diode marker >> dio_txt
BOOLEAN bjt_layer bjt marker >> bjt_txt *
```

Mini-LVS Functions

Layer extraction and device recognition

When you specify various device layers in the layer map file, GDS2DB extracts all of them to the DB. For example:

```
nwell nw 14 -
pwell pw 15 -
pdiff pd 43 -
ndiff nd 10 -
poly poly 30 -
cont contact 50 -

#ID layers for special devices
DIODE diode     119:0 -
BJT   bjt       250:0 -
PR    polyres   115:0 -
```

Note that special devices such as diodes, BJTs and poly-resistors should have a marker layer that is specified in GDS. For poly-resistors to be handled correctly during analysis, the name of the poly resistor, 'PR' in this case, needs to be defined in the Apache tech file as a metal. For more details on extraction of special analog devices, see section "Analog Designs", page 5-177.

Layermaps can take Boolean expressions for marker layers, and the same layer marker definition can be used for both the ESD and device identification (diode) purposes in GDS2DB. For example, in the GDS2DB layermap you can specify:

```
HBM_MARKER temp :
CDM_MARKER temp :
BOOLEAN diodes diode [HBM_MARKER|CDM_MARKER]
BOOLEAN ESD_HBM ESD HBM_MARKER
BOOLEAN ESD_CDM ESD CDM_MARKER
```

In the above example, HBM_MARKER and CDM_MARKER layers mark the HBM and CDM diodes. Totem marks these as both diode and marker layers. So, for example, clamps can be set for various devices and can be defined as Boolean expressions in the layer map.

To extract contact layer geometries into the Totem DB, use the keyword setting “CONTACT_RESISTANCE 1” in the GDS2DB configuration file. The contact layer geometries are then modeled as via arrays. In the Totem Tech file, specify 'p-diffusion' and 'n-diffusion' as lower metal layers of the model, and 'contact' as a via layer.

You can trace through a poly resistors using TRACE_THROUGH_POLY_RESISTOR keyword. When this keyword is specified, GDS2DB traces through poly resistors identified by Totem and treats them as metal routes. Default is set to 0 (off). The polyRes layer must be defined in the layer map.

Logical Layer Operations

You can use Boolean expressions to define logical layer operations in the GDS2DB layermap file. GDS logical layer definitions support four Boolean expressions, as follows:

(A&B) - intersection of the geometries of A and B

(!A&B) - inclusion of only the geometries of B that do not intersect with any of the geometries in A.

(A|B) - either geometries of A or geometries of B

(A-B) - inclusion of only the geometries of A that do not intersect with any of the geometries in B.

The syntax for defining logical layers is:

```
BOOLEAN <logic_layername> <layer_type> <Boolean_definition>
```

Note that the BOOLEAN keyword works on a temp type layer only. In the following layermap example, the first few layer definitions are in standard form, and the last three use new logical operator definitions:

```
active temp 6:0 -
pplus temp 25:0 -
npplus temp 26:0 -
nwell nw 14 -
pplus temp 43 -
active temp 10 -
poly poly 30 -
cont contact 50 -
M1 m 61:0 61:22
LDMTAG ldmos 94:0  -
...
BOOLEAN pdiff pd (active&pplus)
BOOLEAN ndiff nd (active-pplus)
BOOLEAN TRENCH substrate (def|fg)
```

Support of LDMOS marker layer

As shown in the layermap example above, you can define an LDMOS marker layer in the GDS layer map file to help GDS2DB identify LDMOS devices for pin snapping. Transistor

pins with locations from the XTOR_MAP_FILE that overlap the LDMOS marker layer are then considered LDMOS devices. If an LDMOS transistor location from XTOR_MAP_FILE is outside the marker, you can relax the LDMOS pin snapping by using the following GDS2DB configuration file keyword:

```
LDMOS_MARKER_LAYER_BBOX_TOLERANCE <threshold_um>
```

Tap Cell Detection

GDS2DB also identifies well taps when all device layers are specified in the GDS2DB layer map file.

Pin Snapping

After running APLMMX current characterization, the *<>.gdsmmx* file is generated to pass electrical data to GDS2DB, using the GDS2DB keyword XTOR_MAP_FILE. For each device terminal pin in the *<>.gdsmmx* file, GDS2DB identifies the device types defined in the XTOR_MAP_FILE and extracts the layout and performs pin snapping. Two summary files are generated describing how terminal pin snapping is performed on the contacts (considered to be the current carriers for the devices), as follows:

```
adsRpt/GDS/xtor.pin_snapping.report.txt (Report on full pin snapping)
adsRpt/GDS/xtor.pin_snapping.error.txt (Errors during pin snapping)
```

General Notes on GDS Handling

- Totem supports flat or hierarchical GDS. However, hierarchical GDS is preferred and is typically available. The GDS should be a full-detail GDS, and using abstract frame-view GDS is not recommended. GDS is translated into internal DB format with the GDS2DB utility.
- The GDS can be in microns or lambda (a scaled layout). If lambda scale is used, then you must use LEF/DEF_SCALING_FACTOR or GDS_SCALING_FACTOR in order to shrink the GDS to micron scale.
- The GDS should have no power/ground shorts. The GDS translation engine cannot detect any shorts between power and ground or between power and signal.
- The GDS database should be consistent with the Netlist and DSPF used in APLMMX to get correct device back-annotation locations.
- For configuration file keywords specific to mini-LVS, see section "Mini LVS and Device Handling", page 3-117.

Special GDS2DB Steps

PLOC Automation Procedure

In the GDS2DB configuration file you can use the keyword GENERATE_PLOC USE_TEXT_LABEL to generate pad locations in the PINS section of the DEF file. Then in the Totem-MMX GSR file, you can specify ‘ADD_PLOC_FROM_TOP_DEF 1’ to obtain the pad location from the PINS section of the DEF file.

For Embedded Switches within the LVS flow, use GDS2DB configuration keywords “GENERATE_PLOC USE_NIF_FILE” and “IGNORE_PLOC_INTERNALNETS 0", and GSR keyword “ADD_PLOC_FROM_TOP_DEF 1" to create plocs on the switch output.

OASIS Processing

Introduction

Totem can also work from OASIS database instead of GDS to create physical models and process it in native manner in the Totem DB

Input Data Preparation

Figure 2-6 shows the steps in preparing the OASIS data for the Totem database

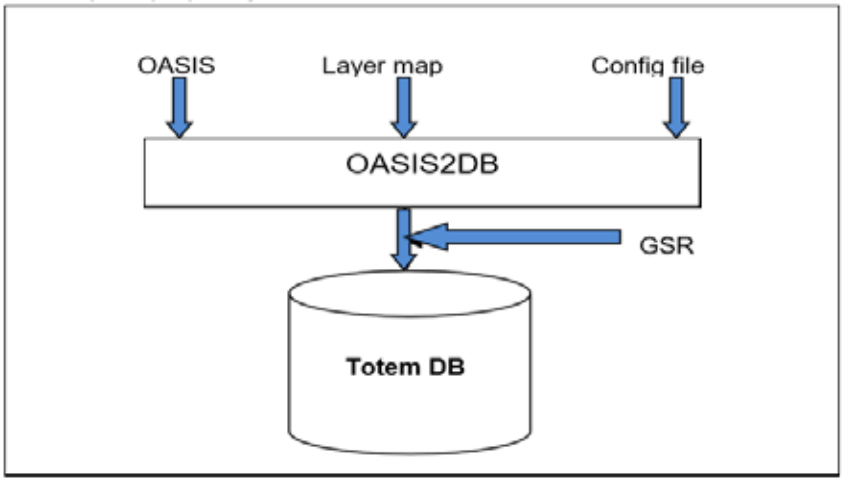


Figure 2-6 Data flow in Oasis Processing

To run Totem OASIS2DB, the following input data needs to be prepared:

GSR keywords

GSR file is same as that provided for GDS2DB except that GDSII_FILE keyword is replaced by OASIS_INPUT_FILE. For details, please refer the section GSR Keywords of GDSII

OASIS_INPUT_FILE

This keyword specifies the OASIS cell name, the configuration file and other settings.

Syntax:

```
OASIS_INPUT_FILE {
<cell_name> <oasis2db_config_file> <mode> top?
}
```

where,

cell_name: Specifies the OASIS cell name.

mode: Specifies the detailed transistor view. It can be full | extended | reduced

top: is an optional sub-keyword indicating it is the top of the hierarchy

Configuration Keywords

This configuration file is similar to that provided for GDS2DB except for following difference:

In place of GDS_FILE keyword use OASIS_FILE keyword to specify OASIS file

Syntax:

```
OASIS_FILE       <oasis_file_path>
```

In place of GDS_MAP_FILE keyword use OASIS_MAP_FILE to specify layer map file.

Syntax:

```
OASIS_MAP_FILE <layer_map_file>
```

The following is an example of a typical OASIS2DB configuration file.

```
TOP_CELL TOP
OASIS_FILE ./data/Top.OAS
OASIS_MAP_FILE ./data/layer.map
VDD_NETS
{   VDD {
    VDD:
    VDD  } }
GND_NETS
{   VSS {
    VSS:
    VSS  } }
```

For detail on all other configuration keywords refer the section GDS2DB Configuration File keywords

Chapter 3
Characterization and Modeling

Introduction

The Totem™ characterization engine extracts the effective intrinsic or intentional capacitance, equivalent resistance and time-variant and voltage-dependent current models for each transistor-level device. In this detailed characterization modeling approach, the device models are both spatially (relative to location) and temporally (relative to time of switching) accurate.

The advantages of MMX modeling are:

- Full-chip dynamic analysis on designs with a mixture of custom macros and cell-based digital blocks
- Large capacity and shorter runtime (compared to a fast SPICE approach)
- Layout-based GUI for ease of debugging and diagnosis

Modeling Method

Totem modeling involves the creation of both electrical and physical models. To create the electrical model, the APLMMX utility characterizes the custom block by partitioning the design and selecting and grouping active and inactive devices in each partition. It invokes a vectorless simulation method to extract equivalent circuit capacitance and resistance of each device and a set of user-provided test bench simulations to produce the accurate voltage-dependent and time-variant switching current model, I(v,t), for each transistor-level device. The current profiles created by APLMMX do not have fixed time steps, but are in PWL (piecewise linear) format, which has the following advantages over the normal current profile:

- if the current profile is relatively smooth and well-behaved throughout, then the PWL waveform contains fewer points and is more compact, requiring significantly less memory
- if the current profile has fast changes and spikes, then the PWL waveform captures the details of waveform more accurately, even with rapid glitches

GDS2DB creates geometric models of the custom macro or memory block from GDSII. Figure 3-1 presents an overview of the modeling strategy.

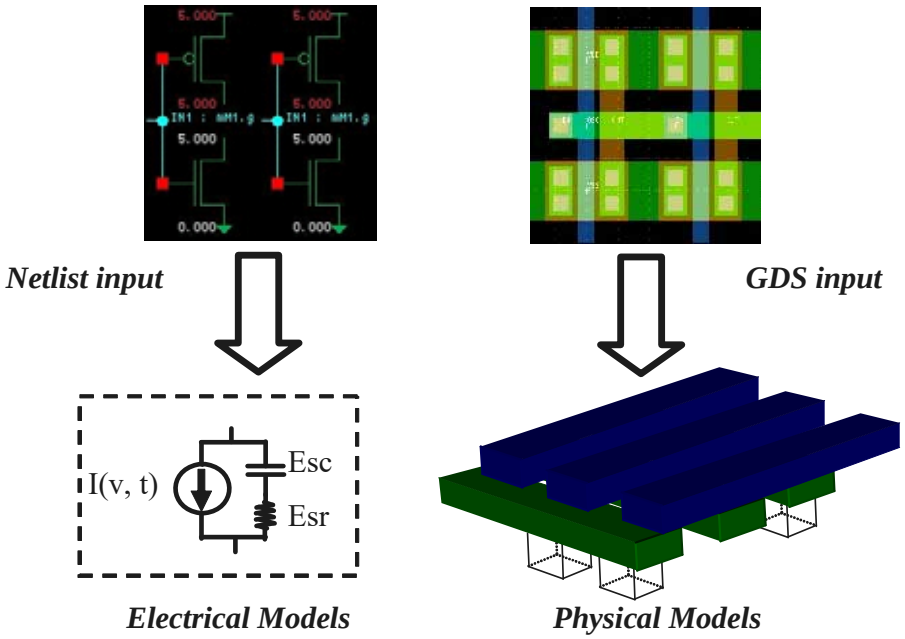


Figure 3-1 MMX Modeling

These models contain the physical power/ground structures, the effective power circuit resistance (Esr) and capacitance (Esc) electrical models, along with the hookup points represented by macro power pins. Totem then imports these physical geometric models and electrical models to perform analysis.

Characterization: Creating Electrical APLMMX Models

Running APLMMX Characterization

APLMMX is a pre-characterization step in Totem that creates the electrical models for the analysis. The electrical models can be created for either a static or a dynamic analysis. After creating an APLMMX configuration file, *aplmmx.config,* containing the necessary keywords, you can run APLMMX in the Totem environment:

Static Analysis Characterization Setup

1. To perform a characterization for static analysis of the design, the you need the layout and the netlist, with proper x, y coordinates for transistors. Refer to section "Netlist-based Setup", page 2-10, for details on the netlist requirements in Totem. The configuration file is updated with the necessary APLMMX keywords, as described in the next section.
2. Use the following command to run static characterization:

   ```
   % aplmmx -gds aplmmx.config
   ```

   which generates the following set of output files needed for physical model creation:

   ```
   <cell>.gdsmmx
   ```

The cell files contains coordinates and relative drive strengths of each device in the netlist. The relative drive strength of each device is determined based on the width and length of each device in the netlist.

Note: for more accurate estimates of device strength, use the IDSAT characterization method (see section "IDSAT-based power assignment", page 4-161).

Dynamic Analysis Characterization Setup - Outside Totem

Note: You cannot use physical models created in static characterization setup in dynamic analysis, since the electrical models are different.

1. To perform characterization for dynamic analysis of the design, you need additional data regarding the testbench and simulation currents for each transistor. Since dynamic analysis for Totem is vector-dependent, you need access to a Spice simulator to simulate the functional vectors of the design. Once the currents for all the devices in the netlist are captured, APLMMX generates the current model for Totem. The steps involved in performing dynamic analysis are described below:
2. Create current probes from netlist using the command:

   % aplmmx -s aplmmx.config

   The output of this command creates a file *<cell>.aplmmx* which contains a list of probes for all devices connected to a power or ground network. The format of this probe list depends upon the Spice simulator being used.
3. Run the Spice simulator using the probes.

   In this step you include the probes generated by APLMMX in the Spice test bench, and then simulate the test bench in Spice. The command to simulate Spice depends on your simulator of choice. The simulation also needs to be performed to capture multiple sets of vectors for better coverage of the switching activity. The simulation output typically is a *.raw/.fsdb/.out* file that has transient current information for the switching devices.
4. Create the electrical model in APLMMX.

   After Spice simulation has been performed, the output currents in the form of a waveform file, such as *.raw/.fsdb/.out*, are imported by APLMMX to create the current model for Totem. The command used to perform this step is:

   ```
   % aplmmx -n aplmmx.config
   ```

   The following output files are generated by this stage:

   ```
   <cell>.spcurrent
   <cell>.cdev
   <cell>.gdsmmx
   ```

   The *<cell>.spcurrent* files contain the transient current model for all transistors in the design. The *<cell>.cdev* files contain the capacitance information for all transistors in the design. The *<cell>.gdsmmx* files contains device location and type information for the physical model creation stage.

   To parallelize spcurrent file dump out function in multi-state flows, users can add the option `'-new_waveform_flow'` along with `'aplmmx -n'` command.

APLMMX also creates a basic configuration file template needed for creating the physical GDS models with GDS2DB.

To check and debug the APLMMX results, see section "Checking and Debugging APLMMX Characterization Results", page 3-90.

APLMMX New Device Name Mapping Engine

From 14.2.1, APLMMX device new name mapping engine has simpler usage model, faster runtime and better default mapping rate and a GUI interface for mapping rule suggestion. The new name mapping engine is default ON from 18.1.1.

Advantages:

- Faster
  - Hierarchical mapping
  - Device name mapping stage speedup > 10X
  - Overall aplmmx runtime speedup 30% for large design
- Simpler
  - Only three config file keywords setting is required
    a. HIER_DIVIDER
    b. FINGER_DELIMITER
    c. FLAT_HIER_TOKEN_RENAME_MAP
- Better mapping rate
- GUI interface to help setup mapping rule

  Usage:
  `aplmmx [ -n | -gds ] aplmmx.config -mlog`
  -mlog : to dump netlist device names database into adsRpt/NameMatch directory for later GUI interface setup

Pattern search and replace can be performed from begin, end, and anywhere in each hierarchy. Regular expression search and replace is supported. Default mapping setup at `<$APACHEROOT>/bin/aplmmx_name_map_config`.

```
HIER_DIVIDER |
FINGER_DELIMITER _

#operand scope : 1. All hierarchy, 2. Top only, 3. Bottom only, 4. Flatten device
#operator mode : 1. Begin with, 2. End with, 3. Anywhere, 4. Regular expression.

FLAT_HIER_TOKEN_RENAME_MAP {
#           match          replace     scope      operator
            ""             ""          All        Anywhere
            -1             ""          All        Begin
            ""             "m"         All        Begin
            ""             "x"         Top        Begin
            "x"            "m"         Top        Begin
            -2             ""          All        Begin
            "m"            "x"         Top        Begin
            ""             "xm"        Top        Begin
            "_unmatched"               ""         Bottom     End
#           "[Xx]*_([0-9]+)_([0-9]+)"  "%1\.%2_%3"  Bottom   Regular
}
```

Table 3-1 The three keywords required in the new device name mapping engine

Improved finger handling

Totem has been enhanced to better handle fingers in spice netlist. Some of the features has been turned on by default and for rest, user need to set keyword "AUTO_SPICE_FINGER_SUPPORT 1" in aplmmx configuration file to enable them.

Following features are made default on:

- Automatic detection of serial resistor chain
- Automatic checking of device name with "==" or "_" patterns as mixed finger delimiter.
- Automatic recognition of hierarchical divider pattern.
- Support wildcard (*) patterns in configuration keyword FINGER_DELIMITER.

If multiple finger delimiter combinations are used in the same hierarchy name, user need to explicitly set the finger pattern with wildcard in configuration keyword FINGER_DELIMITER for APLMMX to recognize them. For example, device X1/M1__2@3, user needs to define "FINGER_DELIMITER __*@*" in aplmmx configuration file to recognize this device as finger for X1/M1.

Other more complicated finger pattern can also be defined now with wildcard pattern expression. For example, "FINGER_DELIMITER _*__rcx _*__dmy0 _m*_0".

TSR Flow Setup Inside Totem

A TSR file is a combination of all needed information previously specified separately in the GSR file, APLMMX configuration file, GDS_CELLS, and the GDS configuration file. You can create the integrated keyword TSR file and then use the Tcl command ‘import tsr <integrated_gsr_file>’ inside Totem to control the APLMMX and GDS characterization stage, referred to as the “TSR flow”. A sample integrated GSR file template is shown below, with a mixture of GSR and APLMMX keywords.

Sample Template TSR file for Totem Power IR/EM Analysis

```
############################################################

    TOP_CELL ABC_core
    TOP_XCELL_NAME Xdm_cp_core
    MACRO_TYPE analog
    DSPF_NETLIST ./design/extract/abc
    DEVICE_MODEL_LIBRARY /proj/exam/hspice/a.lib
    DEVICE_MODEL_LIBRARY /proj/modz/hspice/b.lib
    ...

    VOLTAGE_SOURCES {
        <pin_name1> <volt_src_name1>
        ...
    }
    SIM_OUTPUT_FILE /xyz_core.fsdb
    MULTI_STATE_FILE ./mnop.vec
    #  GDS2DB Options
    GDS_FILE ./design/gds/mnop.gds
    GDS_MAP_FILE ./new_gds.map
    GDS_OPTION {
            GENERATE_LEF_DEF ./defout
        SNAP_THRESHOLD_DISTANCE 0.09
    }
```

```
    ACE_OPTION {
        ACE_HSPICE /tools/abc/hspice/
        ...
    }
    ##  GSR Keywords
    TECH_FILE ./tech/projV/apache.tech

    ## EM Options
    USE_DRAWN_WIDTH_FOR_EM 1
    USE_DRAWN_WIDTH_FOR_EM_LOOKUP 1
    TEMPERATURE 100
    TEMPERATURE_EM 100

    ## Ploc file options
    PAD_FILES {
        ./design/pad4/URPG.ploc
        ...
    }

    ## Power nets for voltage drop analysis
    VDD_NETS {
        V1  0.9500
        V2  1.100
        ...
    }
    GND_NETS {
        Vss1  0.0
        ...
    }
GSC_FILE ./gsc/abc_core.gsc
    DYNAMIC_SIMULATION_TIME 6.25n
    DYNAMIC_TIME_STEP 10p

    # Custom lib with P/G arcs for voltage derating
    LIB_FILES {
        ./libA/custom.lib customA
    }
#########################################################
```

Using a TSR file Totem generates the needed aplmmx and GDS configuration files to perform APLMMX characterization, electrical modeling, and GDS translation, as shown in the Figure 3-2 diagram.

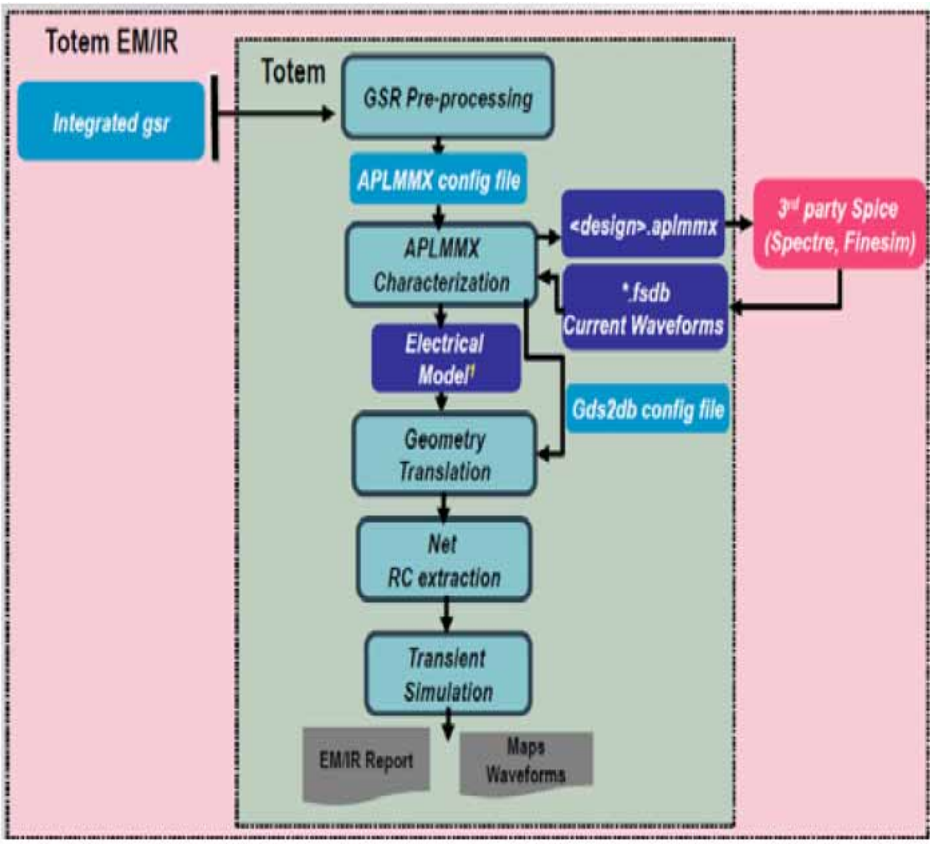


Figure 3-2 Integrated GSR flow (TSR)

Once the TSR file is created you can easily include new APLMMX or GSR keywords, or modify keyword settings, by either executing a new ‘import tsr <modified_file>’ Tcl command in Totem, or add new keywords to the existing TSR file by using ‘-add <GSR/APLMMX keyword>’ option. The Tcl syntax is:

```
import tsr <tsr_file> ?-add <APLMMX/GSR keyword>?
?-aplmmx_dir <dir>? ?-gds_cells_dir <dir>?
?-gds_config <gds_cofig_file>?
?-dump <file>?
```

where

-add : adds GSR/APLMMX keywords that are not otherwise specified in the TSR file.

-dump <file> : dumps the latest values from all TSR file keywords in the latest Totem DB.

-aplmmx_dir <dir>, -gds_cells_dir <dir>, -gds_config <gds_cofig_file> : includes relevant data from the specified APLMMX or GDS cells directories, or relevant data from the specified GDS configuration file. For example, if <tsr_file> only contains TECH_FILE and nothing else, you can use -aplmmx_dir <dir> to include electrical models in the GSR APL_FILES, -gds_cells_dir <dir> allows you to include physical models in the GSR GDS_CELLS, -gds_config <gds_cofig> allows you to include physical model configurations in the GSR GDSII_FILES keyword.

Note that both ‘import tsr’ and ‘import gsr’ commands CANNOT be used at the same time; if they both exist, an error is displayed and Totem stops.

You can also control ‘aplmmx –s’ and ‘aplmmx –n’ runs in 3rd party simulator flows using this TSR tcl command interface. The usage is described in the following paragraphs: Depending on the type of methodology desired, the following methods can be used:

Method 1: To generate probes for the 3rd party Spice simulator flow, the commands are:

```
import tsr <integrated_gsr_file>
characterize aplmmx -s
```

Method 2: To generate an electrical model after importing simulation output waveforms for the 3rd party Spice simulator flow, the commands are:

```
import tsr <integrated_gsr_file>
characterize aplmmx –n
setup design
...
```

where the integrated GSR file has SIM_OUTPUT_FILE specified. This allows use of one configuration setup for the entire Totem flow, but with the flexibility to run a 3rd party simulator flow.

Note that you can execute multiple “import tsr” or “characterize aplmmx” commands before “setup design”, and that the latter command overwrites the previous command results.

APLMMX Configuration File Keywords

Required Keywords

Before running APLMMX, you must create an APLMMX configuration file. Several APLMMX configuration file keywords must be defined, including specifying several other required files, as described in this section. Unix environment variables can also be specified for paths under these keywords.

Example:

```
DSPF_MAP $PROJ_ROOT/dir1/dir2/totem_${PROCESS_PDK}
${PROCESS_STACK}.dspf.map
```

DECAP_NETLIST

Specifies the netlist with detailed intrinsic and intentional decap transistors if intentional decap transistors are not defined in SPICE_NETLIST, DSPF_NETLIST, or LAYOUT_NETLIST.

Syntax:

```
DECAP_NETLIST <netlist_for_decap_extraction>
```

DEVICE_MODEL_LIBRARY

Defines the device model library files, which are used for internal decap characterization and netlist tracing. APLMMX supports global environment variables inside the device model library. Global variables must start with a “$” character. For example, use a line inside a model file such as “*.lib ./user_data/$process28nm/diode.lib TT*”. In this case “$process28nm” represents the value set in the global environment variable 'process28nm'.

Syntax:

```
DEVICE_MODEL_LIBRARY <library_path> <process_corner>
```

DSPF_NETLIST

Specifies the DSPF netlist with device x,y coordinates defined in the "*|I" line in the Net Section and the device line in the Instance section. If the DSPF is extracted from STAR-RCXT™, APLMMX can use the placement info file to get device x,y coordinates, and you can extract signal capacitance only to reduce the DSPF file size. See Case 2 in section "Netlist-based Setup", page 2-10, for more information.

Syntax:

```
DSPF_NETLIST <dspf_netlist> ?<placement_info_file>?
```

GND_PIN_NAME

Specifies the ground domain names. Also supported for signal EM analysis.

Syntax:

```
GND_PIN_NAME <gnd1> <gnd2> ...
```

LAYOUT_NETLIST

Specifies the post-layout netlist with x,y coordinates defined on the device line. If the device name is different in the SPICE_NETLIST and LAYOUT_NETLIST, and the layout netlist is extracted from Calibre, APLMMX can use the *ixf* file for device name mapping. See Case 4 in section "Netlist-based Setup", page 2-10, for more information. If there is SS (Smashed Source) notation in the ixf file, APLMMX looks for its corresponding layout and CDL device name in the ixf file and splits the SS device into fingers. Note that if the domain name has been changed using the Calibre LVS flow, you must also specify the third argument to input the Calibre NXF file.

Syntax:

```
LAYOUT_NETLIST <layout_netlist> ?<ixf_file>? ?<nxf_file>?
```

MACRO_TYPE

Specifies the types of devices to be included in modeling for each application type. Since current conservation for signal nets is very important for EM accuracy, and signal nets only connect to a few devices, ignoring some device current may cause inaccurate EM results. So in the signal EM flow, by default Totem includes all device types in modeling and analysis, and the MACRO_TYPE settings are ignored.

In the power analysis flow, MACRO_TYPE specifies which types of devices are included by default in modeling. The main motivation is to reduce the number of probes for large memory array applications, but still provide full device support for analog/RF applications.

Syntax:

```
MACRO_TYPE [ memory| analog | PMIC | io | rf ]
```

The devices included for each MACRO_TYPE setting are as follows for power analysis:

memory : MOS, Resistor

For memory designs with over 100K devices, the keyword setting 'PROBE_THRESHOLD 1pA' is turned on by default to ignore devices with absolute peak currents less than 1pA.

analog: MOS, Resistor, Diode, BJT

PMIC: MOS, Resistor, Diode, BJT

The pmic option (power management IC) enables the PMIC flow in APLMMX to handle non-power switch devices connecting to multiple power domains. For MOS devices, if both drain and source are connected to power/ground domains, in the PMIC flow MOS devices are not ignored. But in non-PMIC flow, this type of MOS device is treated as a power switch and ignored.

io: MOS, Resistor

rf: MOS, Resistor, Diode, BJT, Inductor

For MOS devices, if both drain and source are connected to power/ground domains, in the RF flow (same as PMIC flow), MOS devices are not ignored.

Notes on use:

1. If you want to change the default device type settings for different MACRO_TYPE applications, you can use “INCLUDE_PROBE_TYPE” and “IGNORE_PROBE_TYPE” configuration file keywords to modify the default settings.
2. If the same device type is defined in both INCLUDE_PROBE_TYPE and IGNORE_PROBE_TYPE keywords, the IGNORE_PROBE_TYPE setting takes precedence.
3. In spite of the application type setting in power analysis, if you want to include the gates of MOS devices or Bases of BJT devices in the modeling, add the “INCLUDE_PROBE_TYPE GB” keyword setting.
4. If you want to include bulk (i4) of MOS devices, add “PROBE_BULK_NODE 1" configuration file keyword setting.
5. Linear capacitor devices by default are not included in power analysis because they are considered parasitic capacitors and this avoids double counting, since Totem P/G extraction extracts capacitance. However, if you know that the linear capacitors are designed capacitors, and want to include them in modeling, use the “INCLUDE_PROBE_TYPE C” keyword setting.
6. To support netlist parameters “nf” and “nr” in the RF device finger flow for signal EM, using the 'MACRO_TYPE RF' setting, these special RF devices must be covered by a marker layer in GDS as specified as follows in the layer map file, such as

   ```
   finger_box mosfinger <marker_leyer_id:data_type>
   ```

   The GDSMMX file contains the extra finger information for these RF devices. Without this, all current for the RF device would be carried in a single finger and not be distributed among all fingers.

SPICE_NETLIST

Specifies the SPICE netlist for simulation.

Syntax:

```
SPICE_NETLIST <cdl_netlist>
```

SPICE_SIMULATOR

Specifies the type of third-party SPICE simulator to be used, or the full binary path, and additional simulator arguments. Note that the full binary path option should contain the simulator name. The simulator used affects the current model lookup table file *<cell>.aplmmx,* as well as determines the current polarity for modeling. Default simulator is NSpice. Correctly handles Eldo simulator current polarity when you run Eldo in Spice-compatible mode (using the “-compat” option, or the “.option compat” setting in the test-bench).

Also allows passing simulator command options to aplsw using the <simulator_options> subkeyword in the embedded switch characterization flow. The content of

<simulator_options> is copied exactly into the aplsw configuration file with the following syntax: SIMULATOR_COMMAND_OPTION <simulator_options>. This line is added to the command line when aplsw is launched inside APLMMX.

Syntax:

```
SPICE_SIMULATOR [[hsim | nanosim | ultrasim | eldo | Adit |
    smartspice | hspice ? -hpp ?| hpp | nspice | xa | xps |
    spectre | aps | titan_apache | finesim | bda | <path to
afs_simulator> ] |
    <full_path_Spice_simulator> ] ? -compat ?
? <simulator_options>?
```

Note: APLMMX assumes BDA simulator setup in Spectre netlist environments, so if it is in an HSpice environment, you must set the “USIM_SAVE 0” keyword in the APLMMX configuration file.

TOP_CELL

Specifies the top-level hierarchy (subckt) in MMX analysis.

Syntax:

```
TOP_CELL <design_name>
```

TOP_XCELL_NAME

Specifies the top-level hierarchy (subckt) instance name in MMX analysis, which is used to match the top subckt instantiation in SPICE simulation test bench. So the ‘.probe’ statement in the <cell>.aplmmx file inherits the specified TOP_XCELL_NAME as prefix.

Syntax:

```
TOP_XCELL_NAME <design_instance_name>
```

Example

```
TOP_XCELL_NAME xtop
```

Then in <cell>.aplmmx:

```
.probe vdd.apl23 =par(‘i3(xtop.X1.MI)’)
```

VDD

Specifies the voltage level of each power domain, in the same order as VDD_PIN_NAME.

Syntax:

```
VDD <vdd1_volt> <vdd2_volt> ...
```

VDD_PIN_NAME

Specifies the power domain names. Also supported for signal EM analysis.

Syntax:

```
VDD_PIN_NAME <vdd1> <vdd2> ...
```

Optional APLMMX Configuration File Keywords

The following keywords can be used in the APLMMX configuration file as needed.

ACE_OPTION

Specifies ACE options using APLMMX configuration file.

Syntax:

```
ACE_OPTION {
    <ACE_options>
    ...
    }
```

Example

```
ACE_OPTION {
    ace_hspice /usr/spice/hspice
    }
```

ACE_OUTPUT_FILE

Defines the *cdev* output file path generated by the ACE utility.

Syntax:

```
ACE_OUTPUT_FILE <ace_output_path>
```

ANALYZE_NETS

Specifies a subset of nets to be analysed for quick performance(default all nets are analysed).

Syntax:

```
ANALYZE_NETS {
    <net_name_1>
    <net_name_2>
    ...
}
```

APLSW_OPTION

To enable user to specify different simulators for switch characterization (aplsw) vs current/cdev characterization on designs with embedded switches. If user specifies simulator related option inside the APLSW_OPTION {}, aplmmx will then use the specified simulator in aplsw characterization instead of using the simulator defined in SPICE_SIMULATOR keyword.

Syntax:

```
APLSW_OPTION {
APL_<simulator> <simulator_binary_path_for_aplsw>
<...>
}
```

Aplmmx will copy and paste the content in the {} of APLSW_OPTION into the internally created aplsw config file for embedded switch flow support. Example for APL_<simulator> inside the APLSW_OPTION {} can be APL_HSPICE, APL_SPECTRE, APL_FINESIM, APL_ELDO etc. When the option “SIMULATOR_COMMAND_OPTION -no-spice” is set inside “APLSW_OPTION {}” , APLSW will not use '-spice' option when callsing finesim in Embedded switch flow.

APL_VOLTAGES

Defines voltage derating factor for characterization. Voltage factors must be specified either in ascending or descending order.

Syntax:

```
APL_VOLTAGES <#_of_sweeps> <1st_sweep_volt_derating>
    <2nd_sweep_volt_derating> ...
```

AUTO_SPICE_FINGER_SUPPORT

When set to 1, automatically handle finger index differences between dspf and spice netlist while mapping. Default 1.

Syntax:

```
AUTO_SPICE_FINGER_SUPPORT [1 |0]
```

CELL_BASED_FLOW

When set, invokes the Totem CPM cell-based flow, and automatically generates physical models in the file *.*gds.config*.

Syntax:

```
CELL_BASED_FLOW [ 0 | 1 ]
```

CELL_NAME_MAP

Can be used where the top cell name in GDS is different from that in CDL. Then while executing the GDS2DB flow, you can rename the top cell name in the generated LEF/DEF views to match the top cell name in CDL. Any desired name can be given, and the name is retained in the LEF/DEF.

Syntax:

```
CELL_NAME_MAP  {
    <gds_name> <desired name>
}
```

CELL_SHORT_PORT_LIST

Allows tracing through custom electro-magnetic devices, such as spiral inductors, in the Spice netlist and treating them as black boxes in the static analysis flow. This allows you to define a list of custom EM cells as black box cells and list a group of ports to be shorted during netlist tracing. This keyword specifies the path to the input file to describe the custom electro-magnetic cell list and port shorting definitions.

Syntax:

```
CELL_SHORT_PORT_LIST_FILE <file_path>Inside the specified
file, you should define the following:
 <subckt_name_or_model_name1>  {
   <group_id1> {
   <port1>
   <port2>
   ...
   <portN>
   }
   <group_id2> {
   <port1>
   <port2>
   ...
   <portN>
 }
  ...
 }
 <subckt_name_or_model_name2> {
   <group_id1> {
```

```
  <port1>
  <port2>
  ...
  <portN>
 }
  <group_id2> {
  <port1>
  <port2>
  ...
  <portN>
  }
 ...
}
```

Notes in using the port file syntax:

- the subckt_name or model_name should come from the netlist, and APLMMX then supports the model name from the '$.MODEL=<model_name>' definition in the X-instance line (assuming that $MODEL is unique for each instantiation).
- group_id is only for tracking purposes and has no physical meaning
- if there are ports in multiple groups of the same cell in the file, you should merge them into the same group. APLMMX errors out if it finds a port is defined in multiple groups of the same cell.

If there are two P/G domains defined in the APLMMX configuration file shorting through a custom EM cell, Totem stops shorting them through the custom EM cell.

CKT_PATTERN_FILE

To turn off selected devices in a Signal EM Vectorless flow.

Syntax:

```
CKT_PATTERN_FILE <off_devices_file>
```

Syntax of <off_devices_file>:

```
$$OFF_CHANNEL &ATTRIBUTE
TopInstance/SubInstance/Device1
TopInstance/SubInstance/Device2
```

COMPLEX_PORT_FILE

Specifies a file defining I/O nets with complex layout port structures, such as when an I/O net contains multiple layout ports. When this keyword is specified (or the keyword HANDLE_COMPLEX_LAYOUT_PORT), Totem creates one dummy node where the schematic current source is hooked up, and all layout ports of this schematic port are then shorted to this node.

Syntax:

```
COMPLEX_PORT_FILE <file>
```

CURR_DIFF_TOLERANCE'

When set, checks the difference between total charge in the power domain and the total charge in the ground domain when the PGARC configuration file keyword is specified, such as:

```
PGARC {
  vddi vss
```

```
    vdd2 vss
  }
```

In this case a summary is recorded in the log file, as the following sample:

```
Info: Summary of Power to Ground consistency check on current/
charge conservation (Tolerance=1%):
Power to Ground Total Charge Summary (Coulomb):
PGARC state_name power ground diff_percent(%)  consistency
-------------------------------------------------------------
(vdd1 vdd2 <-> vss) ST1 7.77689e-11 7.93604e-11 2.04642 Failed
(vdd1 vdd2 <-> vss) ST2 2.03461e-11 2.09693e-11 3.06304 Failed
```

BUS_DELIMITER

Specifies the bus delimiter used in the netlist to assist in bus notation mapping defined in PROBE_BUS_DELIMITER.

Syntax:

```
BUS_DELIMITER “<start_bus_notation>” “<end_bus_notation>”
```

Example

```
BUS_DELIMITER “[“ “]”
```

CHARACTERIZE_IDSAT

When set, performs Idsat characterization internally instead of providing a look-up table. You can specify a global toggle rate in the APLMMX configuration file, and a template for BLOCK_POWER_ASSIGNMENT is generated in the *adsRpt/<design>.bpa_file* to use in static analysis. In this static flow, you do not need to provide BLOCK_POWER_FOR_SCALING power specifications. Instead, Idsat and toggle rate information is used to determine the overall block power using BLOCK_POWER_ASSIGNMENT constraints. You can further fine tune the region powers by setting MMX_REGION or MMX_PIN options in BLOCK_POWER_ASSIGNMENT for transistor-pin static analysis. A static DC spcurrent profile with static current values is dumped for the transistor pins by default. *Default: 0.*

Syntax:

```
CHARACTERIZE_IDSAT  [0|1]
```

CHARACTERIZE_SWITCH

When set to 1 (or ‘embedded’), APLMMX automatically characterizes pmos/nmos power-switch devices and dumps all information like width, length, Idsat, Ron, voltage etc. in output *.gdsmmx file. The *.gdsmmx file generated can be passed to gds2db without need of SWITCH_MODEL_FILE in GSR and [DEFINE|EXTRACT]_SWITCH_CELL in gds2db config. This also removes the need to specify switch cell properties explicitly using SWITCH_MODEL_TABLE config file keyword. This flow supports only simple pmos/nmos switch devices that are not connected in serial/parallel and not defined as subckt. *Default: 0 (off)*

Syntax of *.gdsmmx file:

```
<switch_pin_name>  <xtor_name>  <x>  <y>  <pratio>
TYPE{<switch_TYPE>} TID{<TID>}  MDL{<MDL_name>}  W{<W>}
L{<L>}  RON{<RON>}  IDS{<IDS>}  ILK{<ILK>} NIF_ID{<NIF_ID>}
VOLT{<voltage>}
```

Syntax:

```
CHARACTERIZE_SWITCH [ 0 | 1 ]
```

OR

```
CHARACTERIZE_SWITCH [ off | embedded ]
```

CHECK_MODEL_CORNER

When set, honors process corner settings when reading the DEVICE_MODEL_LIBRARY.

Syntax:

```
CHECK_MODEL_CORNER [0|1]
```

CURR_CONSISTENCY_REPORT

When turned on, reports on the transistor current in the simulation output waveform file and in the APLMMX converted current profile. A report is generated in the file *adsRpt/<design>.current_diff_rpt*. An example file is shown below:

```
<xtor_name> <pin/probe_name> <spcurr_peak> <sim_peak> <diff%>
<spcurr_charge> <sim_charge> <diff%>
sweep: 0  state: WRITE
================================================================
X0.M3   vdd    311.947    323.131    3.46097    4.00195e-07    3.79932e-07    5.3332
----------------------------------------------------------------------------------------------------------------
X0.M3   vdd.1  311.947    323.131    3.46097    4.00195e-07    3.79932e-07    5.3332
----------------------------------------------------------------------------------------------------------------
X0.M4   vdd.2  89.2139    91.0456    2.01191    8.2996e-08     6.6360e-08     25.068
```

Default: 0.

Syntax:

```
CURR_CONSISTENCY_REPORT [0 | 1]
```

CURR_DIFF_TOLERANCE

When set, checks the difference between total charge in a domain and the sum of transistor charges in that domain. The specified tolerance difference must be a fraction between 0 and 1.0. A tolerance limit of 0.1 means that the charge difference must be within 10%, or else the process stops and an ERROR MMX-317 message is displayed. If the PROBE_THRESHOLD keyword is defined, the relative peak current difference after current redistribution is also checked. This feature supports multi-state cases, and reports current/charge consistency checking data for different states in *aplmmx.log*.

For designs with internal power or ground pins, the charge checking depends on the configuration file keyword VP_PAIRING (see VP_PAIRING...). In such cases, the relative difference between the total charge at parent level pin and the sum of the charges at all internal power pins (equal to the sum of the transistor charges within the internal block) under the parent level, plus the total charge of the transistors that are directly connected to the parent level power pin, are taken into account. An error message is issued depending on the following conditions:

- With VP_PAIRING and with CURR_DIFF_TOLERANCE set, the total difference of charge is reported in the log file, and an error displayed, if required.
- With VP_PAIRING set and without CURR_DIFF_TOLERANCE, the total difference of charge is reported in the log file.
- With neither VP_PAIRING nor CURR_DIFF_TOLERANCE set, no check is performed.
- Without VP_PAIRING and with CURR_DIFF_TOLERANCE set, Totem errors out, as there is not sufficient data available to run the check.

Syntax:

```
CURR_DIFF_TOLERANCE <tolerance_diff>
```

CURR_DIFF_TOLERANCE_MIN_CUR

When set, allows you to set an absolute value threshold for current consistency checks, so that small currents below the specified threshold are filtered out, and only current values above the threshold are compared for current consistency.

This keyword works along with CURR_DIFF_TOLERANCE <tolerance_diff>, which is used to set a relative difference threshold. If the absolute current is greater than the threshold value, and the relative difference is greater than the relative difference threshold <tolerance_diff>, a tolerance check is triggered and a warning/error message is issued if the difference is over the user-defined threshold value.

Syntax:

```
CURR_DIFF_TOLERANCE_MIN_CUR <threshold_value>
```

CURRENT_MODEL_MODE

When set to 1, the subckt current effect at transistor level is not considered in the analysis. By default (0), the effect is considered when any of the following is true:

1. there are no internal nets defined, such as with INTERNAL_POWER_PIN_NAME or INTERNAL_GND_PINNAME APLMMX configuration file keywords.
2. there are no multiple nets grouped into one net.
3. the difference between the net current and the sum of device terminal currents is within 30%.

A current consistency report is generated regardless of the CURRENT_MODEL_MODE setting.

Syntax:

```
CURRENT_MODEL_MODE [ 0 | 1 ]
```

CUSTOM_STATE_SIM_FUNC

This keyword picks up valid cycles based on specified Boolean expressions and clock activity. APLMMX chooses cycles based on the first Vdd pin defined in VDD_PIN_NAME in the configuration file. If no Boolean equation is defined, APLMMX chooses the cycle with maximum current (charge) among all valid clock cycles. This keyword supports both power analysis and signal EM flows. There are also two optional configuration file keywords that allow you to refine your settings in finding the peak power cycle, PROBE_MAP and CUSTOM_STATE_TARGET_PIN.

Syntax:

```
CUSTOM_STATE_SIM_FUNC {
    <state> <Boolean_eq> <clk_name>
    ...
}
```

Example:

```
CUSTOM_STATE_SIM_FUNC {
    s1 “!wea_n & !mea_n & !meb_n” “clka”
    ...
    }
```

Long Boolean functions, can be defined using multiple lines with a character limit of 64000. Each line should be separated by a “\” character.

Example2:

```
CUSTOM_STATE_SIM_FUNC {
```

```
    write "!CE_N & \
    !GWE_N" "CLK"
}
```

This is equivalent to the Boolean function "!CE_N & !GWE_N"

CUSTOM_STATE_SIM_TIME

Defines the capture window of each state. See the following section for more information on multi-state operations.

Syntax:

```
CUSTOM_STATE_SIM_TIME <options>
```

CUSTOM_STATE_TARGET_PIN

If the keyword CUSTOM_STATE_SIM_FUNC is specified, APLMMX chooses cycles based on the first Vdd pin defined in VDD_PIN_NAME in the configuration file. Then you can use CUSTOM_STATE_TARGET_PIN to redefine which P/G pin to be used to determine the peak power cycle.

Syntax:

```
CUSTOM_STATE_TARGET_PIN <vdd_pin_name/gnd_pin_name>
```

Example:

If there are multiple P/G pins named vdd1, vdd2, gnd1, gnd2, in the design and you want to use 'gnd1' to calculate current/charge for cycle selection, instead of 'vdd1' you can use the following specification in the configuration file:

```
CUSTOM_STATE_TARGET_PIN gnd1
```

Note that the CUSTOM_STATE_SIM_FUNC and CUSTOM_STATE_SIM_TIME keywords can be used at the same time, as long as state names are not in conflict.

CYCLE_SKIP

Specifies skipping extraction of undesired cycles, such as an initial spike cycle. For example, the first clock cycle may include a large amount of noise that is not to be considered. Cycle count starts from 0. *Optional.*

Syntax:

```
CYCLE_SKIP <cycle0> < cycle1> ...
```

DECAP_SUBCKT

Defines sub-circuit-based intentional decap instances in the SPICE netlist. APLMMX finds the first device that has non-bulk pins connected to the same P/G net the instance pin is connected to, and takes this first device pin (x,y) location as the decap_subckt instance P/G pin (x,y) location. Supports negative values in the input netlist.User need not specify intentional decaps constructed using regular mos-device (single xtor decap) as they are auto-recognized and modelled by aplmmx.

Syntax:

```
DECAP_SUCBKT {
    <decap_subckt_name1> <port_type_list1> param <param_list1>
    <decap_subckt_name2> <port_type_list2> param <param_list2>
    ...
    <decap_subckt_nameN> <port_type_listN> param <param_list3>
}
```

where

<decap_subckt_nameN>: describes the subcircuit name of the intentional decap instance

<port_type_listN>: defines the type of subckt port connection, either power, ground, or signal net, designated as 'power', 'ground', or 'na' in the port type list ('na' designates a signal net connection)

Note that there must be one item in the <port_type_list> for each pin name.

param <param_list>: defines the parameters that determine the size of the intentional decap instance. For example, in a SPICE netlist as follows:

```
XDECAP1   PLUS   MINUS   NMOSCAP   lr=6.3u   wr=3.92u   m=4 ...
XDECAP2   PLUS   MINUS   NMOSCAP   lr=3.3u   wr=1.92u   m=8 ...
...
```

You can construct the following section in the APLMMX configuration file to define the decap subckt:

```
DECAP_SUBCKT {
    NMOSCAP power ground param lr wr m
    Decap*200Z power ground
    DecapLibx5?X power ground
    }
```

The wild card characters '*' and '?' can be used with sub-circuit names.

'*' represent a string and '?' represents a single character.

Note: Port-order/param-list is optional for MOM and device-decaps (like nmoscap, etc). User can define only subckt/model name under DECAP_SUBCKT without port connection and size parameter.

For example,

```
DECAP_SUBCKT {
    crtmom2t
    crtmom3t
    nmoscap
}
```

1. Aplmmx will pick all x-instance of specified model/subckt having at least one pin connected to power and ground nets each.
2. If there are x-instances with only one pin connecting to power or ground (all other pins connecting to signal), APLMMX will turn on "INCLUDE_PROBE_TYPE C" by default to model such device pins.
3. For decap x-instance qualified in #1, APLMMX will pass the PG connection and parameter list to ACE (called internally) for cdev characterization.
4. Aplmmx will dump file 'adsRpt/mmx.miss_spice_decap_subckt_definition' or 'adsRpt/mmx.miss_layout_decap_subckt_definition' if it detects any non-mos/bjt/diode/resistor/inductor "leaf" level instance in spice or layout netlist connected to both power and ground nets but not list under DECAP_SUBCKT setting.

DEFAULT_NET_CURRENT

To assign basic current in vectorless Signal EM to perform EM check on nets that belong to circuits wherin it can't identify the current channel between power and ground. DEFAULT_NET_CURRENT applies only to nets with non-zero capacitance and at least two connected devices that can carry DC current, e.g., MOS S/D or resistors. Nets belong to un-recognized channel and not satisfying these criteria will typically be listed in adsRpt/em_dropped_nets.rpt report and dropped from analysis. Default: none.

Syntax:

```
DEFAULT_NET_CURRENT <option>, where <option> is one of:
```

none | drop: do not analyze; drop net and report (default)

low_drive: use low-drive waveforms

high_drive: use high-drive waveforms

DEFINE_SWITCH_CELLS

When used with 'IDENTIFY_INTERNAL_PG_NETS 1', APLMMX finds switch pins and internal P/G nets automatically and identifies device terminals connected to the internal P/G nets. APLMMX also adds DEFINE_SWITCH_CELLS and EXTRACT_SWITCH_CELLS keywords in the GDS configuration file template. In this case you do not need to define INTERNAL_POWER_PIN_NAME or INTERNAL_GND_PIN_NAME keywords.

If 'IDENTIFY_INTERNAL_PG_NETS 1' is not set, APLMMX only uses the DEFINE_SWITCH_CELLS information to identify switch pins and dump out to the *.gdsmmx* file. In this case you need to define the INTERNAL_POWER_PIN_NAME and/or INTERNAL_GND_PIN_NAME keywords, as well as EXTRACT_SWITCH_CELLS in GDS_OPTION in the GDS configuration template.

Syntax:

```
DEFINE_SWITCH_CELLS {
<switch_cell> [HEADER|FOOTER] <external_pin>
<internal_pin> <control_pin1> <control_pin2> ...
}
```

DEFINE_SWITCH_CELLS

When used with 'IDENTIFY_INTERNAL_PG_NETS 1', APLMMX finds switch pins and internal P/G nets automatically and identifies device terminals connected to the internal P/G nets. APLMMX also adds DEFINE_SWITCH_CELLS and EXTRACT_SWITCH_CELLS keywords in the GDS configuration file template. In this case you do not need to define INTERNAL_POWER_PIN_NAME or INTERNAL_GND_PIN_NAME keywords.

If 'IDENTIFY_INTERNAL_PG_NETS 1' is not set, APLMMX only uses the DEFINE_SWITCH_CELLS information to identify switch pins and dump out to the *.gdsmmx* file. In this case you need to define the INTERNAL_POWER_PIN_NAME and/or INTERNAL_GND_PIN_NAME keywords, as well as EXTRACT_SWITCH_CELLS in GDS_OPTION in the GDS configuration template.

Syntax:

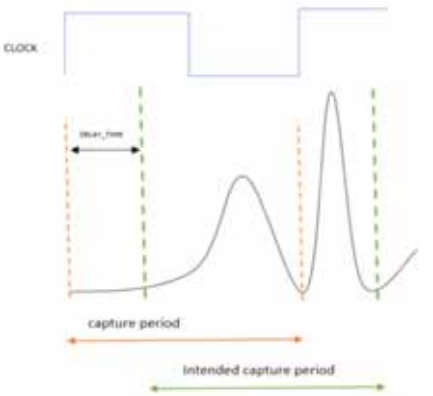
```
DEFINE_SWITCH_CELLS {
<switch_cell> [HEADER|FOOTER] <external_pin>
<internal_pin> <control_pin1> <control_pin2> ...
}
```

DELAY_TIME

Specifies a delay after the triggering edge of clock to start capture period. This considers the insertion delays of the clock, when the states are defined using CUSTOM_STATE_SIM_FUNC. *Optional. Default:0*

Syntax:

```
DELAY_TIME <delay_time_in_sec>
```



DEVICE_MODEL_TABLE

In the vectorless Signal EM flow, this keyword specifies a device model look-up table, so that device parameters such as Ron, Cgate, Cdrain, Csource, ESC, and ESR can be calculated without any external dependency on Spice (ACE). Each device W/L combination can have multiple lines based on different parameters (VGS/VDS/NFIN). The embedded switch flow supports DEVICE_MODEL_TABLE to specify all properties such as RON and IDSAT, instead of separately specifying them using SWITCH_MODEL_TABLE and other inputs. In vectorless signal EM flow, APLMMX can also apply Ron of MOS from ACE Idsat characterization results to improve the current estimation accuracy.

Wildcard support is provided for the device model name field. Interpolation/extrapolation is also supported for device parameters. When you do not have specific information, you can use “-” or “ace” entries to allow ACE to determine the Idsat or device/pin cap for such devices.

Syntax:

```
DEVICE_MODEL_TABLE <table_filename>
```

The device model look-up table content and format is as follows:

```
<device_model1> {
WL <W> <L> {
    VTH{<vth>} CGATE{<cap_in_F>} CSOURCE{<cap_in_F>}
IDSAT{I_in_A} RON{<cap_in_F>} ESR{<R_in_ohms>
ESC{<cap_in_F>} CDRAIN{<cap_in_F>} VGS{<volts>}
  }
}
<device_model2> {
  WL <W> <L> {
    ...
  }
}
```

DEVICE_MODEL_CONFIG

This keyword defines which model files are encrypted so that APLMMX and ACE netlist parsers can bypass the files, assists APLMMX in recognizing the device model type, and in getting the necessary subckt port number and name information.

Syntax:

```
DEVICE_MODEL_CONFIG <device_model_config_filename>
```

The format of the contents of the device model configuration file is as follows:

```
<device_type> <model_name> <marker_name> <pin1>(<pin1_layer_name>
<pin2>(<pin2_layer_name> ... <pin_n>(<pin_n_layer_name>) [SIZE_PARAM
<param1> <param2> ... <paramn>]
```

You can specify the following device types in the <device_type> column:

[NMOS | PMOS | NPN | PNP | DIODE | RESISTOR | CAPACITOR | INDUCTOR | MOM | GENERIC]

In this format, the pin name assists APLMMX in creating probes of the x-instance port current. The pin layer name column is required, but if you do not know the pin snapping layer name, specifying “NA” is acceptable. The [size_param ...] column is optional, except when the default w and l parameters are not available, or when certain netlist parameters are required for ACE characterization.

Embedded switch flow supports LDMOS transistors as switches. To enable this, user needs to set LDMOS device as "NLDMOS" or "PLDMOS" in the DEVICE_MODEL_CONFIG file immediately followed by pin-pair grouping (in addition to defining them under SWITCH_MODEL_TABLE or using “CHARACTERIZE_SWITCH 1”).

Example to provide LDMOS transistors in embedded switch flow:

```
[NLDMOS | PLDMOS] <model/subckt_name> <marker>
<pin1>(<pin_snapping_layer>) <pin2>(<pin_snapping_layer>) ...
PIN_PAIR {
# input_pin
    PIN_GROUP1 {
     <pin_name1>
    }
# output_pin
    PIN_GROUP2 {
     <pin_name2>
    }
    ...
    IGNORE_PINS {
     <pin_name3>
    }
}
```

Along with PIN_GROUP definition APLMMX will also perform topology tracing to identify the direction of LDO in/out pins for swapping.

To ignore selected pins for models and subcircuits, user can define them using IGNORE_PINS under PIN_PAIRS option through DEVICE_MODEL_CONFIG file.

You can use a “GENERIC” device type to handle non-standard CMOS process devices, such as BJT, diode, resistor, capacitor, inductor, LDMOS, or other RF devices. APLMMX then probes the current for this type of device without dropping it.

Example file:

```
PMOS tpmc tpmc_mask n1(pdiff) n2(poly) n3(pdiff) n4(nw)
    [SIZE_PARAM w l nf dfm_flag]
NMOS tnmc tnmc_mask n1(ndiff) n2(poly) n3(ndiff) n4(pw)
    [SIZE_PARAM w l nf dfm_flag]
GENERIC <model/subckt_name> NA/marker_layer_name <pin1>(NA)
<pin2>(NA) … pinN(NA)
```

Also, for GENERIC devices APLMMX supports definition of the w and l parameters as equations based on other parameters with following conditions.:

- Each parameter/equation is separated by space
- No space within the equation
- Operand can be floating number literal, integer number literal, or variable with name starting with letter or '_'
- Operator can be +-*%() with */% higher precedence than +-

Example1:

```
GENERIC dmos1 NA D(NA) G(NA) S(NA) EPI(NA) GSUB(NA)[
SIZE_PARAM w=dw*dl l=nl]
```

To identify GENERIC devices, a marker layer must be defined in GDS2DB layer map using the keyword 'gmarker'

Example2:

Definition in device model configuration file:

```
GENERIC < model_name> GENMRK port1(NA) port2(NA) port3(NA)
… portN(NA)
```

Definition in gds layer map:

```
GENMRK   gmarker   117:0 –
```

APLMMX supports Inductors with S-parameter models in the schematic netlist. Schematic netlist has nport definition with reference node per port whereas DSPF Netlist contains a different model name without any reference ports. Aplmmx will identify the instances of S-parameter models and will insert an extra "0" reference node for each pin, there by ensuring proper mapping of instance ports between schematic and DSPF netlists. In <device_model_config_file>, user needs to define the S-parameter model name in DSPF that needs to be replaced by nport with "<port_number>"

Syntax of <config_file> to provide s-parameter

```
GENERIC <model> <marker> <pin>(<PSL>) ... nport=<port_number>
```

Example:

Sample DSPF syntax for a 4 port Inductor:

```
Xind Xind:t1 Xind:t2 Xind:t3 sparam4_model devname="…
```

In aplmmx config file:

```
DEVICE_MODEL_CONFIG device.cfg
```

Contents of device.cfg :

```
GENERIC sparam4_model NA t1(M1) t2(M2) t3(M3) t4(M3)
nport=4
```

Aplmmx will process this information and when it finds instance in netlist, for these models, it will insert the extra "0" reference node for each pin to form nport pin pairs.

DEVICE_NAME_TOKEN_RENAME

Helps to avoid mismatching instance names in DSPF file "*|I" section when there are inconsistent delimiters in net names. Schematic netlist names used for probes and DSPF netlist names used for location mapping can be different, but mapping can be made consistent by renaming tokens using this keyword. Supported in signal EM flow also.

Syntax:

```
DEVICE_NAME_TOKEN_RENAME
"<original_name_token>" "<new_name_token>"
```

Example

```
DEVICE_NAME_TOKEN_RENAME
 ".X" "_X"
```

DSPF_TRACEPOINT_PER_LAYER

limit the number of tracing points for each layer in gds config. User can set ‘default <limit_number>’ to non-defined layers.

Example

```
DSPF_TRACEPOINT_PER_LAYER {
M1  3
M2  5
default 4
}
```

ENABLE_LDO_PINS

Creates LDO pins for regulated power nets when the pin layer is on substrate. To enable this feature set 'ENABLE_LDO_PINS 1' in the aplmmx config file (default is off). To get the substrate pins in the CCI2EX database user must enable DUMP_SUBSTRATE_PINS in the cci2ex configuration or cci2ex_option section in the aplmmx config file.

In the gsr use ADD_PLOC_FROM_TOP_DEF 1 to enable LDO pin creation into the def file.

In case the regulated power net is connected to an output port and the output port shouldn't get a ploc, use IGNORE_REGULATED_NET_PORTS 1 within the gds configuration file or in the GDS_OPTION section of the aplmmx config file. In case the output port of the regulated net should be probed and the current waveform applied to the port use 'PROBE_POWER_OUTPUT_PORT 1' in the aplmmx config file.

Example:

```
ENABLE_LDO_PINS 1
PROBE_POWER_OUTPUT_PORT 1
REGULATED_POWER_NETS {
    ...
}
CCI2EX_OPTION {
    DUMP_SUBSTRATE_PINS 1
    ...
}
GDS_OPTION {
    IGNORE_REGULATED_NET_PORTS 1
    ...
}
GSR_OPTION {
    ADD_PLOC_FROM_TOP_DEF 1
}
```

EMPRO_MODE

Specifies the type of signal EM analysis to be performed. For vectorless analysis, Totem performs “CCR tracing”, which means finding charging paths from VDD, and also finding discharging paths to ground, based on MOS channel connectivity. The default is static.

Syntax:

```
EMPRO_MODE [ vectorless | vectorbased | static ]
```

ENABLE_DERATING

When set to 1, APLMMX will generate virtual voltage derating table automatically for more accurate Dynamic IR results.

Syntax:

```
ENABLE_DERATING [1 | 0]
```

Usage in `aplmmx.config`:

```
APL_VOLTAGES <#_of_sweeps> <1st_derating_ratio>
<2nd_derating_ratio>
...
ENABLE_DERATING 1
```

Note that one of the APL_VOLTAGE should have <derating ratio> as ‘1’, which will be considered as the reference voltage in spice simulation.

Example:

```
APL_VOLTAGES 5 1 0.95 0.9 0.85 0.8
ENABLE_DERATING 1
```

Based on APL_VOLTAGES setting example above, If VDD=0.8 volts, then there will be five voltages, 0.8, 0.76, 0.72, 0.68, and 0.64 which will be used for APLMMX sweep.

ENABLE_SUBCKT_REPLACE

When set to 1, probe generation considers the last provided sub-circuit definition. Otherwise, the first sub-circuit definition is used for probe generation. This can be useful when user has a main netlist (schematic) but selectively adds include statements redefining some blocks with a back-annotated netlist. Default 0.

Syntax:

```
ENABLE_SUBCKT_REPLACE [ 0 | 1]
```

ENCRYPTED_FILES

In advanced process nodes, the foundry may encrypt model files. In order to bypass encrypted model file parsing, and still be able to get the necessary model information, use this keyword to define which model files are encrypted so that APLMMX and ACE netlist parsers can bypass these files.

Syntax:

```
ENCRYPTED_FILES {
    <encrypted_file_name>
    ...
}
```

EXPORT_MODEL

Creates a macro model for I/O ports during electrical model creation for Totem signal EM analysis. The port model can be re-used for bigger designs in hierarchical analysis, and takes into account the receiver load capacitance from the macro model for vectorless static signal EM and IR analyses, and rolls up the signal net current to power for IR analysis.

After you finish electrical model creation for a small block, such as “A”, you can import a model for block “A” using the IMPORT_MODEL configuration file keyword. The port model contains information to model the current and load capacitance for the I/O ports of

block “A”. So if you then have a bigger design, such as “B”, and this design contains block “A”, you can use IMPORT_MODEL to import the model so APLMMX does not need to go inside block A, but still creates an electrical model for later Totem signal EM analysis.

Modeling requirements:

- For block level, whether vector-based or vectorless is used or not, APLMMX generates a vectorless model for the I/O ports.
- Top/upper level analysis must be vectorless to use IMPORT_MODEL.
- For top/upper level DSPF netlist creation, you must “black box” those subckt/blocks using IMPORT_MODEL.

The <custom_net_file> option helps APLMMX define input or output ports, or a custom DC current source. If <custom_net_file> is not specified, APLMMX uses the DC current polarity to decide whether the net is an input or output port. The syntax of the specified <custom_net_file> is:

```
#DC_I
<net1>

#DC_O
<net2>
```

where

#DC_O means that current is flowing out of the block (flowing into net at top-level)

#DC_I means that the current is flowing into the block (flowing out of net at top-level). TCL wild cards are supported for specifying <net>.

Syntax:

```
EXPORT_MODEL <model_dir> ?<custom_net_file>?
```

EXTRACT_BIAS_NETWORK

Allows Vectorless Signal EM to extend handling of bias networks to treat all nets that are channel-connected as DC nets, with no switching transitions. The default is propagate:

Syntax:

```
EXTRACT_BIAS_NETWORK [ propagate | isolate]
```

where

propagate: labels all nets in the same CCR driven by a bias net through a transmission gate as a bias net

isolate: identifies bias nets (constant DC values) only

EXTRACTED_VIEW_NETLIST

When set to 1, ignores parasitic R and C not to be modeled as devices when a Virtuoso^{tm} extracted_view netlist is provided as input to the SPICE_NETLIST option in the APLMMX configuration file.

The schematic/Spectre netlist may have bus bits smashed under a single bus bit (either MSB or LSB). When this keyword is set to “QRC” APLMMX automatically identifies and maps these devices between the schematic Spectre netlist to flatten the DSPF netlist. The default is 0 (off).

Syntax:

```
EXTRACTED_VIEW_NETLIST [ 0 | 1 | QRC]
```

EXTRACT_LEAKAGE

Extracts leakage current from current model and stores in the *cdev* model. The <state_name> option allows you to specify which state to use in extracting leakage values from switching current, such as one of the states among multiple states defined in CUSTOM_STATE_SIM_TIME, CUSTOM_STATE_SIM_FUNC, or MULTI_STATE_FILE APLMMX configuration file keywords.

Syntax:

```
EXTRACT_LEAKAGE [ ace | ace_decap_only | apl
?<state_name>?]
```

where

ace: uses leakage from ACE characterization

ace_decap_only: uses leakage from ACE characterization, but only applies leakage to decap

lib: uses leakage from *.lib* file

apl : extracts leakage from <state_name>. Default is the first state in aplmmx DB. *Note that if multiple custom states are specified in the APLMMX configuration file, you must specify the state name, or an Error message is displayed.*

EXTRACT_PURE_MOS_PG_PATH

When set, identifies pure nmos/pmos resistor P/G paths in Vectorless Signal EM, to improve CCR tracing. The output is saved in *adsRpt/n(p)mos_pg_path.rpt*. Default 0 - off.

Syntax:

```
EXTRACT_PURE_MOS_PG_PATH [ 0 | 1 ]
```

FILE_SIZE_REDUCTION

Allows generation of smaller optimized current profiles, if specified. The “Max” setting provides significant file size reduction and an accuracy loss less than 10%. If file size reduction is set to ‘off’, best accuracy is achieved. This keyword supports both power analysis and signal EM flows.

The second argument controls gzipped output format, where ‘0’ is for ungzipped and ‘1’ is for gzipped format (default), with moderate file size reduction. Generally, you do not need to set this keyword, but turning off the second argument allows generation of a current profile that can be imported into 13.1 and prior release versions of Totem or RedHawk, since gzip format was not supported in previous releases.

Syntax:

```
FILE_SIZE_REDUCTION [ off | default | max ] ?[0|1]?
```

FIND_INTERNAL_NET

When set, automatically identifies stacked switches and populate the associated nets to the VIRTUAL_NET_PAIRS section of GDSII_FILE (for gds2db)

Syntax:

```
FIND_INTERNAL_NET 1 <max_stages> ?maxstage{stagenum}?
?mixpnmos? ?useleafnettype?
```

Where:

<max_stages>: Define the maximal number of internal nets tool can find out along the channel connected path from user specified external nets and internal net. Destination internal net is not counted for the max stage setting. Default is 3

User can use either max_stages or maxstage{<#_of_stage>} to define this option.

mixpnmos: Tool traces through both pmos and nmos for internal nets, default is to trace through only pmos or only nmos, separately.

useleafnettype: Tool will use leaf net type for all intermediate nets, currently only regulated net type is propagated when set, default just marks net as regular internal net.

For regulated net with both pmos and nmos as passgate on the same path, usage can be the following

```
FIND_INTERNAL_NET 1 3 mixpnmos useleafnettype
FIND_INTERNAL_NET 1 maxstage{3} mixpnmos useleafnettype
```

For regular internal net tracing, possible usages:

```
FIND_INTERNAL_NET 1
FIND_INTERNAL_NET 1 maxstage{5} mixpnmos => has both pmos
and nmos as switch
```

For example:

Sample aplmmx.config file:

```
INTERNAL_POWER_PIN_NAME {
    vcc_int 0.9
}
 FIND_INTERNAL_NET 1 <max_stages>
```

Then, GDSII_FILE will subsequently get populated with the nets associated with stacked switches found by aplmmx.

```
VIRTUAL_NET_PAIRS {
  vcc_ext vint1
  vint1 vint2
  vint2 vcc_int
}
```

Note: Please set <max_stages> based on number of intermediate internal nets between user defined external and internal nets. Setting a large number may increase run time and cause tool to find out false internal nets.

For example, "ext" is an external net and both "int" and "int_int" are internal nets defined in the aplmmx configuration file

ext -> int1 -> int2 -> int

There are two intermediate internal net and three stacked switches in between ext and int nets. So in the example above, user should set "FIND_INTERNAL_NETS 1 2".

If user sets "FIND_INTERNAL_NETS 1 4", aplmmx may find other path up to 4 intermediate nets between "ext" and "int" net and end up with false internal net and switches identification.

For example, aplmmx may find the following path ext -> sig1 -> sig2 -> sig3 -> int and ext->int1->int2->int->sig4->int_int but the mos devices between these nets are not switches.

If user sets "FIND_INTERNAL_NETS 1 1", aplmmx will not be able to find path with "int1" and "int2" because the actual path from "ext" to "int" requires more than 1 intermediate internal nets.

FINGER_DELIMITER

Specifies the delimiter character for finger device current and cdev splitting, by which a single device in the schematic netlist can be split into multiple finger devices in the layout

netlist. Since finger delimiters used at the X-instance level can be different than those used at the device level, different finger delimiters at different hierarchy levels are supported, such as “xtop/xblk@2/M _2”. Note that Totem automatically detects whether one of the delimiter characters “@” or “_ _” (double underscore) is used in the netlist. If both are used, or if some other character is used, then you *must* define the characters using this keyword.

Syntax:

```
FINGER_DELIMITER <delimiterA> <delimiterB> ...
```

Example

```
FINGER_DELIMITER _ @
```

APLMMX also supports complex serial, parallel, or mixed topology resistor fingers with bus bit structure.

Example:

DSPF / CCI database vs Spice netlist

```
"XXcore/XRes<1>@2" "Xcore.XRes<1>_1__dmy0"
"XXcore/XRes<1>@4" "Xcore.XRes<2>_1__dmy0"
```

This can now be addressed using following settings:

```
FINGER_DELIMITER _*__dmy0
AUTO_SPICE_FINGER_SUPPORT 1 ### default ON
```

FINGER_DELI_ON_LAST_HIER

Distinguishes FINGER_DELIMITER characters from a M_FACTOR delimiters. If FINGER_DELI_ON_LAST_HIER is not set (default), both leaf instances and devices treat finger delimiters in the usual way. If FINGER_DELI_ON_LAST_HIER is set, the finger delimiter only applies to the device's last hierarchy, and the M_FACTOR defined is used on instance hierarchy.

Syntax:

```
FINGER_DELI_ON_LAST_HIER [ 0 | 1 ]
```

FLAT_BUS_DELIMITER_MAP

Specifies flattened netlist and hierarchical netlist bus delimiter mapping. Multiple device name mapping rules for each X, M, D, R or Q element are supported.

Syntax:

```
FLAT_BUS_DELIMITER_MAP “<flat_bus>”  “<hier_bus>”
```

Example

```
in DSPF: x[0]/M1
in CDL: x<0>.M1
FLAT_BUS_DELIMITER_MAP “[]”  “<>”
```

FLAT_CHIER_MAP

Specifies flattened netlist and hierarchical netlist capacitor hierarchy mapping. Multiple device name mapping rules for each X, M, D, R or Q element are supported. Using a single item without brackets is also allowed, as well as different user-defined name mapping rules to map to the same pattern.

Syntax:

```
FLAT_CHIER_MAP “<flat_token>” “<hier_token>”
```

FLAT_DHIER_MAP

Specifies a flattened netlist and one or more hierarchical netlist diode hierarchy mappings. A single item without brackets is also allowed. Multiple device name mapping rules for each X, M, D, R or Q element are supported. Using a single item without brackets is also allowed, as well as different user-defined name mapping rules to map to the same pattern.

Syntax:

```
FLAT_DHIER_MAP {
    “<flat_token>”  “<hier_token>”
    ...
    }
```

Example

```
“/XD”  “.D”
“/DD”  “.D”
in DSPF: XTOP/DD1
in CDL: XTOP.D1
FLAT_DHIER_MAP “/D”  “.”
```

FLAT_HIER_TOKEN_RENAME_MAP

When specified, allows you to set generic device name token mapping rules to back-annotate device names between the hierarchical Spice netlist and the flattened layout/ DSPF netlist. INCLUDE_DEFAULT_RULES sub keyword within FLAT_HIER_TOKEN_RENAME_MAP appends default rules (default rules defined under $APACHEROOT/bin/aplmmx_name_map_config) in addition to user-defined rules in aplmmx config file.

Syntax:

```
FLAT_HIER_TOKEN_RENAME_MAP {
    <token_name_flat_netlist> <token_name_hier_netlist>
}
```

Example: Device name in Spectre netlist file

```
u_ptatcal\<3\>.MP1
u_bias_ana.Iptatcal.Mdum_REF\.1_0
```

Example: Device name in DSPF netlist file,

```
XXu_ptatcal<3>/XMP1
XXu_bias_ana/XIptatcal/XMdum_REF_1_0
```

```
HIER_DIVIDER |
FINGER_DELIMITER _

#operand scope : 1. All hierarchy, 2. Top only, 3. Bottom only, 4. Flatten device
#operator mode : 1. Begin with, 2. End with, 3. Anywhere, 4. Regular expression.

FLAT_HIER_TOKEN_RENAME_MAP {
#          match       replace     scope       operator
           ""          ""          All         Anywhere
           -1          ""          All         Begin
           ""          "m"         All         Begin
           ""          "x"         Top         Begin
           "x"         "m"         Top         Begin
           -2          ""          All         Begin
           "m"         "x"         Top         Begin
           ""          "xm"        Top         Begin
           "_unmatched"            ""          Bottom      End
#          "[Xx]*_([0-9]+)_([0-9]+)"   "%1\.%2_%3"    Bottom    Regular
}
```

Example: If the user defined rules must have higher priorities than the default rule table, then user must declare the following,

```
      FLAT_HIER_TOKEN_RENAME_MAP  {
          "MQ"      ""      TOP     Begin
          INCLUDE_DEFAULT_RULES
}
```

Then the rules will be expanded automatically as below:

```
FLAT_HIER_TOKEN_RENAME_MAP  {
    #match  replace  scope   operator
    "MQ"      ""      TOP     Begin
    ""      ""      All     Anywhere
    -1      ""      All     Begin
         .........
    ""      "c"     Top     Begin
}
```

FLAT_LHIER_MAP

Specifies flattened netlist and hierarchical netlist inductor hierarchy mapping. Multiple device name mapping rules for each X, M, D, R or Q element are supported.

Syntax:

```
FLAT_LHIER_MAP “<flat_token>” “<hier_token>”
```

FLAT_MHIER_MAP

Specifies a flattened netlist and one or more hierarchical netlist MOSFET hierarchy mappings. Multiple device name mapping rules for each X, M, D, R or Q element are supported. Using a single item without brackets is also allowed, as well as different user-defined name mapping rules to map to the same pattern.

Syntax:

```
FLAT_MHIER_MAP {
    “<flat_token>”  “<hier_token>”
    ...
    }
```

Example

```
“/XMP”  “.MM”
“/XMN”  “.MM”
in DSPF: XTOP/MM1
in CDL: XTOP.M1
FLAT_MHIER_MAP “/M”  “.”
```

FLAT_QHIER_MAP

Defines a flattened netlist and one or more hierarchical netlist BJT hierarchy mappings. Multiple device name mapping rules for each X, M, D, R or Q element are supported. Using a single item without brackets is also allowed, as well as different user-defined name mapping rules to map to the same pattern.

Syntax:

```
FLAT_QHIER_MAP {
    “<flat_token>”  “<hier_token>”
    ...
    }
```

Example

```
in DSPF: XTOP/QQ1
in CDL: XTOP.Q1
FLAT_QHIER_MAP “/Q”  “.”
```

FLAT_RHIER_MAP

Specifies a flattened netlist and one or more hierarchical netlist resistor hierarchy mappings. Multiple device name mapping rules for each X, M, D, R or Q element are supported. Using a single item without brackets is also allowed, as well as different user-defined name mapping rules to map to the same pattern.

Syntax:

```
FLAT_RHIER_MAP {
    “<flat_token>”  “<hier_token>”
    ...
    }
```

Example

```
in DSPF: XTOP/RR1
in CDL: XTOP.R1
FLAT_RHIER_MAP “/R”  “.”
```

FLAT_XHIER_MAP

Specifies a flattened netlist and one or more hierarchical netlist x-instantiation hierarchy mappings. Multiple device name mapping rules for each X, M, D, R or Q element are supported. Using a single item without brackets is also allowed, as well as different user-defined name mapping rules to map to the same pattern.

Syntax:

```
FLAT_XHIER_MAP {
    “<flat_token>”  “<hier_token>”
    ...
    }
```

Example

```
in DSPF: XTOP/I12/M1
in CDL: XTOP.XI12.M1
FLAT_XHIER_MAP “/I”  “.XI”
```

FREQ_CHECK

Provides a quick check on the simulation timing window(s). A warning is displayed if any simulation time for a current profile is inconsistent with the FREQ_CHECK value (frequency times timing window should be approximately = 1).

Syntax:

```
FREQ_CHECK <freq>
```

GDS_OPTION

Specifies GDS2DB options using the APLMMX configuration file.

Syntax:

```
GDS_OPTION {
    <GDS_option1> <value>
    ...
    }
```

Example:

```
GDS_OPTION {
    CONTACT_RESISTANCE 1
   }
```

GDS_SETUP_DIR

Specifies the GDS2DB output data directory, which will be passed down to the automatically generated GDS2DB template configuration file.

Syntax:

```
GDS_SETUP_DIR <GDS2DB_output_dir>
```

GND_NETS

Specifies the voltages for ground domains and also define the equivalent ground domain names in the SPICE netlist, which are merged in APLMMX for Totem analysis. Do not define duplicate entries in GND_PIN_NAME, INTERNAL_GND_PIN_NAME and VDD_NETS, which will cause program error. Instead use the GSR keyword GND_NETS. Also supported for signal EM analysis.

Syntax:

```
GND_NETS
```

```
<Vss_domain_net_name> <value_Volts> {
    <equiv_Vss_net_name1>
    ...
    }
 ...
}
```

GENERATE_PROBE_FOR_PWR_SEM

When set, generates probes for both Power and Signal nets for EM analysis in one APLMMX run, with results recorded in the *<output_directory>/<cell>.aplmmx* file, which simplifies the flow and reduces the number of Spice simulation runs. Default is off.

Syntax:

```
GENERATE_PROBE_FOR_PWR_SEM  [0 | 1]
```

GSR_OPTION

When specified, generates a GSR template file in the power analysis flow. You can define baseline GSR keyword settings. APLMMX generates a template GSR file *<design>_template.gsr* under the run directory, which can be edited later for Totem analysis setup.

Syntax:

```
GSR_OPTION {
    <all_baseline_keywords_setup>
}
```

Example:

```
GSR_OPTION {
    TECH_FILE demo.tech
    GSC_FILE demo.gsc
    DYNAMIC_SIMULTION_TIME 10ns
    DYNAMIC_TIME_STEP 10ps
    TEMPERATURE 125
    USE_DRAWN_WIDTH_FOR_EM_LOOKUP 1
    ENABLE_BLECH 1
}
```

HANDLE_COMPLEX_LAYOUT_PORT

When set, provides special handling for I/O nets with multiple layout ports (default 0). Totem creates one dummy node where the schematic current source is hooked up, and all layout ports of this schematic port are shorted to this node. APLMMX and Totem create the following report files listing the I/O nets created by this feature:

apache.sigem.complex_layout_port : APLMMX

apache.sigem.complex_layout_port.engine :Totem

Syntax:

```
HANDLE_COMPLEX_LAYOUT_PORT [0 | 1]
```

HIER_DIVIDER

Helps to resolve mapping issues between the Spectre extracted netlist and Calibre DSPF netlist by supporting non-default hierarchical dividers and complex finger delimiters inside the SPICE_NETLIST.

Syntax:

```
HIER_DIVIDER  <divider_character>
    FLAT_BUS_DELIMITER_MAP {
    "<flat_netlist_finger_delimitr>"  "<hier_netlist_finger_delimitr>"
    ...
}
```

HSIMSBA

Provides special HSIM-SBA support when finger device current is not included in the master device current probe. With HSIMSBA set to 1, APLMMX scales the finger device current based on its size ratio to the master device, instead of splitting current into multiple fingers. Default is 0.

Syntax:

```
HSIMSBA  [ 0 | 1 ]
```

HSIMSBAPARAM

For devices being process in HSIM using the structural back-annotation flow, the device-probing strategy should be changed using this keyword in order for characterization to be done correctly.

Syntax:

```
HSIMSBAPARAM <subckt_name>
```

HSIMWRAPPERSUB

For devices that are defined as subcircuits in the device model file, but are being instantiated as MOS, Diodes, BJTs, or resistor switch M-line, D-line, Q-line and R-line syntax in the SPICE netlist instead of X-lines, this keyword can be used to specify these device subcircuit names. This allows netlist transfer through the subckt to reach the device level for device probing in characterization.

Syntax:

```
HSIMWRAPPERSUB <subckt_name>
```

IDENTIFY_INTERNAL_PG_NETS

When set, along with DEFINE_SWITCH_CELLS, identifies internal P/G nets for switches automatically.

```
IDENTIFY_INTERNAL_PG_NETS [0 | 1]
```

IDSAT_FILE

Specifies a file that can be used to input saturation current data for MMX static analysis, before running 'aplmmx -gds <config_file> '. If W / L effects are not linear, APLMMX uses interpolation or extrapolation to process the IDSAT_FILE to get a more accurate idsat current values. The input file contains the Idsat data in a format as follows:

```
# model  Type    W(um)  L(um)  Vds(v) Vgs(v) Isat(uA/um)
=================================================
    -    pmos    0.195   0.060  -      -       600
    -    nmos    0.195   0.060  -      -       1020
nch_lvt   nmos    0.150   0.060  2.5    2.5     1020
nch_lvt   nmos    0.300   0.080  -      -       2020
nch_lvt   nmos    0.500   0.100  -      -       3020
pch_lvt   pmos    0.150   0.060  -      -       638
```

```
pch_lvt   pmos    0.300   0.080  -      -       1038
pch_lvt   pmos    0.500   0.100  -      -       1538
*         d        *      *      -      -       18
*         r        *      *      -      -       18
```

For models whose W, L, Vds, and Vgs values cannot be matched with any entry in the table, interpolation/extrapolation is used to get correct idsat values. Characters "-" and "*" can be used as wildcards to denote all models of a particular type.

The model 'Type' can be pmos, nmos, q, d, r, or l. Sample current distribution details are recorded in the log file, such as:

```
Info: in Domain 'vcc1p0_pe', static current was distributed on
19351 devices based on the specified Idsat.
```

Syntax:

```
IDSAT_FILE <input_file>
```

IDSAT_PRATIO

When set, distributes the current of multiple channels based on their effective resistance, so the equivalent resistance of each MOSFET is derived from the Idsat current through characterization. The MOSFET device stacking, with serial and /or parallel connections, is also considered. If the keyword is off (default), APLMMX uses the width/length ratio of the last MOSFET device in the pull-up or pull-down stacking channel to distribute current.

Syntax:

```
IDSAT_PRATIO [0 |1]
```

IGNORE_ACE_SIGNAL_CAP

When set to 1 (default), ignores ACE characterized signal cap loading as part of intrinsic device capacitance when the design has intentional decaps. Setting this keyword to "0" includes this signal cap loading from ACE.

Syntax:

```
IGNORE_ACE_SIGNAL_CAP [ 0 | 1 ]
```

IGNORE_ESCAPE_CHAR

This keyword is obsolete in the new name mapping engine and user need to provide WAVEFORM_TOKEN_RENAME "\" "" in aplmmx config file if the probes names in the FSDB have an escape character.

IGNORE_INSTRINSIC_CAP

When set, ignores intrinsic device cap in the output cdev file. The default is 0 (off).

Syntax:

```
IGNORE_INSTRINSIC_CAP [0 | 1]
```

IGNORE_LAYOUT_FINGER_SPLIT

When set to 1, finger devices are ignored from layout netlist to perform a quick Totem Static/Dynamic analysis. This feature only does early analysis for quick design iteration and is not recommended for sign off. In the aplmmx output *.spcurrent, *.cdev and *.gdsmmx files, all the fingers will be ignored.

Syntax:

```
IGNORE_LAYOUT_FINGER_SPLIT [0 | 1]
```

(default is 0 (off))

IGNORE_NETS

Specifies the list of nets to be ignored for analysis.

Syntax:

```
IGNORE_NETS
{
    <net_name1>
    <net_name_2>
    ...
}
```

IGNORE_NEW_WAVEFORM_FLOW_CURRENT_CONSISTENCY_CHECK

When set, disables the current consistency check in new waveform flow. This will improve the runtime for spcurrent file generation in `aplmmx -n` run for large memory design. Note that this keyword works only with "-new_waveform_flow" option. Default is 0.

Syntax:

```
IGNORE_NEW_WAVEFORM_FLOW_CURRENT_CONSISTENCY_CHECK [1 | 0]
```

IGNORE_OVERLAP_POINT

When set to 1, this keyword ignores simulation output waveform file processing errors when there are multiple data points at the same time. Default is 0.

Syntax:

```
IGNORE_OVERLAP_POINT [ 0 | 1 ]
```

IGNORE_PARENTHESIS_CHAR

Used to ignore the “(“ “)” characters inside the .subckt port line. If this option is not used, APLMMX treats “(“ “)” characters as part of the port name or port list. Default On.

Syntax:

```
IGNORE_PARENTHESIS_CHAR [0|1]
```

IGNORE_PARENT_PROBE_CHECK

For cases in which the parent waveform is a DC current and should be used for current distribution, parent probe checking can be bypassed by setting this keyword (default off).

Syntax:

```
IGNORE_PARENT_PROBE_CHECK [0|1]
```

IGNORE_PROBE_INST

Specifies device instances to be ignored during characterization and modeling. Instance names using wildcard characters are allowed, where “*” is used as a wildcard for multiple characters, and “?” is used as a wildcard for a single character.

Syntax:

```
IGNORE_PROBE_INST <inst_dev_name1> <inst_dev_name2> ...
```

Regular expressions are also supported.

Example:

```
IGNORE_PROBE_INST regexp::^D.*(\||\.|\/|\:)ld_D.*
IGNORE_PROBE_INST regexp::^Dld_D.*
```

In order not to confuse the regular expression from string name, user need to define special syntax regexp:: in front of the pattern to trigger the regular expression.

IGNORE_PROBE_INST_FILE

Specifies a file listing the device instances to be ignored during characterization and modeling. Each ignored instance name must be listed on a separate line in the file. Instance names using wildcard characters are allowed, where "*" is used as a wildcard for multiple characters, and "?" is used as a wildcard for a single character. A report file *mismatch.ignore.list* is saved in the *adsRpt* sub-directory when there are transistors or instances in the IGNORE_PROBE_INST_FILE that are not present in the design.

Syntax:

```
IGNORE_PROBE_INST_FILE <filename>
```

IGNORE_PROBE_SUBCKT

Ignores all devices under the specified sub-hierarchy. It can also define subckts or model names to be bypassed in the missing model type definition warning report, *adsRpt/mmx.miss_model_definition*. For example, some models or subckts are not real device types, so IGNORE_PROBE_SUBCKT can be used to bypass the warning checks on these missing model definitions.

Syntax:

```
IGNORE_PROBE_SUBCKT <model_name/subckt_name>
```

IGNORE_PROBE_SUBCKT_FILE

Ignores all devices listed in the specified sub-hierarchy file. Each sub-hierarchy must be specified in a separate line in the file.

Syntax:

```
IGNORE_PROBE_SUBCKT_FILE <filename>
```

IGNORE_PROBE_TYPE

Specifies the category of devices that are to be ignored. M represents MOS type, D represents diode, R represents resistor, Q represents BJT, GB represents gate or base terminal (i2), PI represents primary input pin, PO represents primary output pin, and BI represents primary inout pin. Only one argument is allowed, but multiple device symbols can be specified without spaces between them. No default. If you want to probe for only power or signal EM analysis, the pwr/sig options can be selected (by default both are probed). Note that by default power analysis ignores some device types based on MACRO_TYPE definitions and signal EM analysis probes all device types.

Syntax:

```
IGNORE_PROBE_TYPE [ M D R Q GB PI PO BI ] ?[pwr | sig]?
```

Example

```
IGNORE_PROBE_TYPE DRPO pwr
```

IGNORE_R_MODEL_IN_PARASITIC_RESISTOR

Defines the list of resistor models to be ignored during parasitic resistor detection. The trace through function in extracted view netlist flow will avoid tracing through 'designed resistor' to avoid pin snapping error in GDS2DB.

Syntax:

```
IGNORE_R_MODEL_IN_PARASITIC_RESISTOR {
     <model_name1>
     <model_name2>
     ...
     }
```

IGNORE_SPARE_CELL

When set, allows spare MOS transistors (all 4 terminals connected to same P/G domain) to be ignored. By default (0), assigns a 0 power ratio to spare transistors. .

Syntax:

```
IGNORE_SPARE_CELL
```

IGNORE_SPARE_XTOR

Transistors with source, drain and gate shorted together in the SPICE netlist can be ignored when creating the DSPF netlist. When set, this keyword reduces the number of redundant pins and probes. You can retain spare and dummy transistors in modeling and analysis by turning the keyword Off. Default: 1 (ignore spare transistors).

Syntax:

```
IGNORE_SPARE_XTOR [ 0 | 1 ]
```

IMPORT_MODEL

Imports macro models created using the EXPORT_MODEL keyword. TCL wild cards are supported for <file> option. APLMMX uses the DC current for nets defined in <file>; if the net is not found in <file>, APLMMX uses the default method.

If there are inconsistent current source specifications in some nets it is noted in a detailed report, *adsRpt/mmx.net_macromodel.rpt*. Any mismatches in drivers/receivers are reported and the receiver's total DC current is scaled to match the total driver current.

Syntax:

```
IMPORT_MODEL {
  <model_dir1> ? <file> ?
  <model_dir2> ? <file> ?
  ...
}
```

INCLUDE

Defines ‘Include’ file for device model library, or any other parameters that need to be included in SPICE simulation.

Syntax:

```
INCLUDE <device_library_include>
```

INCLUDE_PROBE_TYPE

Specifies the device type to be included in probing in addition to the default included device types.

The GB option is used to model the gate terminal of MOS devices and the base terminal of BJT devices. These two terminals usually have very minimum current compared to the drain/source of MOS and collector/emitter of BJTs, so they do not need to be modeled by default. Setting this keyword to GB can force APLMMX to model the 2nd terminal of MOS and BJT devices. No default. If you want to probe for only power or signal EM analysis, the pwr/sig option can be selected (by default both are probed). Note that by default power analysis ignores some device types based on MACRO_TYPE definitions and signal EM analysis probes all device types.

Syntax:

```
INCLUDE_PROBE_TYPE [ M | D | R | Q | GB ] ?[pwr | sig]?
```

INTEGRATE_CELL_FLATTEN

When set, ACE generates an integrated flatten netlist for bottom-up PG pin connectivity tracing.

Syntax:

```
ACE_OPTION {
INTEGRATE_CELL_FLATTEN [ 0 | 1]
}
```

INTERNAL_POWER_PIN_NAME

INTERNAL_GND_PIN_NAME

Defines internal ground domain names for designs with power switches and internal ground domains that do not connect to a voltage source. Supports internal power/ground domain netlist extraction for cases in which the internal domains cannot be defined as global in the SPICE netlist because the internal domain name is the same as external domain name. You must define these keywords to accept full hierarchy names, to avoid using the SPICE_GLOBAL option, since in some designs the internal node name is the same as the external pin name. You can also group multiple internal domains into one domain using the syntax below. Also supported for signal EM analysis.

Syntax:

```
INTERNAL_POWER_PIN_NAME {
    <final_power_domain_name1>   <voltage1>
    ...
    <final_power_domain_nameN>   <voltageN> ?{
        <full_hierachy_internal_power_domain_name1>
        ...
        <full_hierachy_internal_power_domain_nameN>
}? }
 INTERNAL_GND_PIN_NAME {
    <final_ground_domain_name1>   <voltage1>
    ...
    <final_ground_domain_nameN>   <voltageN> ?{
        <full_hierachy_internal_ground_domain_name1>
        ...
        <full_hierachy_internal_ground_domain_nameN>
}? }
```

IOPORT_FILE

When set, the vectorless Signal EM flow supports feed-through nets connected by metal resistors, where the feed-through may have two “*|P”(port) and no dummy devices for the net.

Syntax:

```
IOPORT_FILE <port_file>
```

IPROF_SAMPLING_METHOD

Switch the sampling method from preserving total charge to preserving peak current when generating current profile.

Syntax:

```
IPROF_SAMPLING_METHOD [charge | peak]    (Default: charge)
```

IPROF_SAMPLING_MODE

Defines the number of sampling points for current profiles. If the capture window defined in CUSTOM_STATE_SIM_TIME or MULTI_STATE_FILE, or specified simulation time is too long or too short, you must adjust the IPROF_SAMPLING_MODE to make sure the sampling time step is within the range of 2 ps to 200 ps. Default is 100 sampling points.

Syntax:

```
IPROF_SAMPLING_MODE [reduced| extended| accurate|
       <#_of_points> ]
```

Example

```
IPROF_SAMPLING_MODE 200
```

IPROF_REDUCTION_MODE

Controls the *spcurrent* file size and waveform resolution. When set to “off”, uses the original waveforms from Spice, with no waveform shape loss. The “default” setting provides moderate file size reduction, with moderate waveform shape loss. The “max” setting provides more file size reduction, with more waveform shape loss. *Default: off*

Syntax:

```
IPROF_REDUCTION_MODE [off | default| max]
```

IXF_LAYOUT_TOKEN_RENAME_MAP

IXF_CDL_TOKEN_RENAME_MAP

Allows renaming the name token in the Calibre IXF file before mapping to the Spice netlist and layout netlist.

Syntax:

```
IXF_LAYOUT_TOKEN_RENAME_MAP <name_in_ixf_layout_token> <new_name>
IXF_CDL_TOKEN_RENAME_MAP <name_in_ixf_cdl_token>  <new_name>
```

Example

For a device name “XI1.XI2” in the layout netlist, and “I1/I2” in ixf file, use the following settings to map the name.

```
IXF_LAYOUT_TOKEN_RENAME_MAP “/I”  “.XI”
```

IXF_LAYOUT_XX

These keywords support Calibre *ixf* file mapping between the layout netlist and Spectre netlist.

Syntax:

```
IXF_LAYOUT_BUS_DELIMITER_MAP
     <ixf_bus_delimiter>  <layout_netlist_bus_delimiter>
IXF_LAYOUT_XHIER_MAP  <ixf_xinst_hier>  <layout_xinst_hier>
IXF_LAYOUT_MHIER_MAP <ixf_mos_hier> <layout_mos_hier>
IXF_LAYOUT_QHIER_MAP <ixf_bjt_hier> <layout_bjt_hier>
IXF_LAYOUT_RHIER_MAP <ixf_resistor_hier> <layout_resistor_hier>
IXF_LAYOUT_DHIER_MAP <ixf_diode_hier> <layout_diode_hier>
IXF_CDL_BUS_DELIMITER_MAP
      <ixf_bus_delimiter> <cdl_netlist_bus_delimiter>
IXF_CDL_XHIER_MAP <ixf_xinst_hier> <cdl_xinst_hier>
IXF_CDL_MHIER_MAP <ixf_mos_hier> <cdl_mos_hier>
IXF_CDL_QHIER_MAP <ixf_bjt_hier> <cdl_bjt_hier>
```

```
IXF_CDL_RHIER_MAP <ixf_resistor_hier> <cdl_resistor_hier>
IXF_CDL_DHIER_MAP <ixf_diode_hier> <cdl_diode_hier>
```

IXF_SS_FINGER_SUPPORT

By default, smashed schematic (SS) devices handling in Calibre LVS ixf file flow is disabled. When set to 1, SS finger handling support for Calibre IXF has improved run time and EM accuracy, which is useful when APLMXX netlist back annotation time is too long. ,

By default (0), Totem merges SS device current with its corresponding master schematic device. The total merged current is then split into fingers based on the layout netlist.

Syntax:

```
IXF_SS_FINGER_SUPPORT [ 0 | 1 ]
```

LDO_INST

LDO_INST_FILE

If you do not want to include the LDO transistors in the analysis and want to treat the input and output of LDOs as two separate nets, when this keyword is used APLMMX probes only the input terminal (connected to external supply) and not the output terminal of LDO transistors.

Syntax:

```
LDO_INST {
<ldo_xtor_inst_name>
}
```

or

```
LDO_INST_FILE <ldo_inst_list_file>
```

Example:

```
LDO_INST {
XXI70/XI51/XI52/XIregu/MM4*
}
```

or

```
LDO_INST_FILE  ldo_xtor_list.txt
```

Then the *ldo_xtor_list.txt* file would have the following:

```
XXI70/XI51/XI52/XIregu/MM4*
```

By specifying the LDO instance full hierarchy names, APLMMX can identify all LDO transistors, and only probe and model the inputs of LDO transistors, with no probing of output transistors.

Note that to use this method you need to define the LDO output nets using the INTERNAL_POWER_PIN_NAME configuration keyword.

LEAD_DSPF_INST_NAME_TOKEN_MAP

Specifies user-defined name mapping rules to map the following cases:

- device names between the Instance section and NET section of DSPF files, in the DSPF-only flow
- device names between the instance name in the Spice netlist and the instance name in the NET section of the DSPF/layout netlist ((Spice netlist + DSPF netlist flow) *or* (Spice netlist + layout netlist flow))
- device names between the instance name in the Spice netlist and the instance name in Instance section of DSPF/layout netlist ((Spice netlist + DSPF netlist flow) *or*

(Spice netlist + layout netlist flow)).

Syntax:

```
LEAD_DSPF_INST_NAME_TOKEN_MAP {
  <lead_char_in_Instance_section_in_DSPF>
    <lead_char_in_Net_section_in_DSPF>
}
```

or

```
LEAD_DSPF_INST_NAME_TOKEN_MAP {
  <lead_char_in_Spice/hierarchical_netlist>
     <lead_char_in_Instance_section_in_DSPF/layout>
  <lead_char_in_Spice/hierarchical_netlist>
     <lead_char_in_Net_section_in_DSPF/layout>
}
```

Example

```
LEAD_DSPF_INST_NAME_TOKEN_MAP {
“XXX” “XX”
“XD” “D”
}
```

LEAD_FINGER_MISMATCH_PARENT

Typically APLMMMX ignores all finger devices without a leading master instance definition, such as a list: MM36@2, MM36@3, MM36@4. However, if the LEAD_FINGER_MISMATCH_PARENT keyword is set to 1, APLMMX also supports finger definitions that do not have a master instance definition. If there are inconsistencies between the first finger device name in the layout netlist and master device name in the schematic netlist, you should set this keyword to 1 for correct handling (default off).

Syntax:

```
LEAD_FINGER_MISMATCH_PARENT [ 0 | 1]
```

LEAKAGE_I

Defines cell-level leakage current using the APLMMX configuration file. Leakage current is then redistributed to each individual transistor.

Syntax:

```
LEAKAGE_I {
    <Vdd_name> <Gnd_name> <leakage_Amps>
}
```

LVS_FLOW

When specified, directly takes device location and signal net tracing information from the Calibre LVS or Assura LVS database, which bypasses the requirement to generate a large DSPF with parasitic RC and *|I, *|P, *|S sub-node information for GDS tracing. So you need to generate C-only DSPF for Totem signal EM analysis, and this allows you to import the net info file **.nif* generated by the Totem LVS integration script.

By default (0), Totem connects MOS S/D pins to areas under the contacts connecting to S/D diffusion. These area connections prevent false EM errors due to unequal current-sharing among contacts in a split contact array.

If the second argument is set to 1, Totem sets pin snapping layers for all devices in *<cell>.gdsmmx*. Totem uses information derived from LVS output data to determine x/y/layer for device pins, and attaches each pin to an individual node in its electrical model of

the interconnect/substrate. Unlike “raw GDS” inputs, using LVS output guarantees that each pin connects to the correct net and to the layer specified by the LVS configuration.

The LVS flow supports equivalent net specifications in VDD_NETS and also METAL_RESISTOR settings. You also can define hierarchical node names in net name settings.

Syntax:

```
LVS_FLOW <nif_filepath-LVS_integ_script> [ 0 | 1 ]
```

NEW_WAVEFORM_MODULE

By default, when processing simulation waveform files such as fsdb and tr0, significant improvements in run-time and memory demand are achieved, especially when processing extensive waveform data. Note that when the PROBE_THRESHOLD keyword is set, this module only considers the peak current within specified windows defined in CUSTOM_STATE_SIM_TIME. When this keyword turned Off, this filtering is based on the entire waveform duration across multiple non-adjacent windows, which can result in a different number of pins in the APLMMX output *<design_name>.gdsmmx* file, but this does not affect overall QoR in final IR/DvD/EM results.

Syntax:

```
NEW_WAVEFORM_MODULE [ 0 | 1 ]
```

M_FACTOR

When set, specifies a string or character to represent the instance multiplication delimiter.

Syntax:

```
M_FACTOR <delim_char>
```

MACRO_TYPE RF'

When set, turns on RF device handling for grouped LVS finger X-instance devices (using the Calibre NR factor in the netlist). APLMMX scales the power ratio based on the NR factor, and provides the NR factor and the total width of grouped LVS devices for the GDS utility to create transistor pin geometry for all fingers in the LVS group. This allows Totem to distribute the pin current to all finger contacts in the LVS group.

Syntax:

```
MACRO_TYPE RF [0|1]
```

MEMORY_BIT_CELL

Specifies the memory bit cells in the design. This enables APLMMX to identify and perform more accurate cdev modeling of such cells.

Syntax:

```
MEMORY_BIT_CELL {
        <memory_bit_cell_model_name1>
       <memory_bit_cell_model_name2>
   ...
   }
```

MEMORY_BIT_CELL_INST

Specifies the memory bit cell instances in the design. This enables APLMMX to identify and perform more accurate cdev modeling of such instances.

Syntax:

```
MEMORY_BIT_CELL_INST {
```

```
        <memory_bit_cell_inst1>
        <memory_bit_cell_inst2>...
    }
```

Wildcard characters(*) are supported for instance name specification.

METAL_RESISTOR

Specifies one or more metal resistor device model names, so that APLMMX can trace serially-connected single or multi-level metal/poly resistors in the SPICE netlist, in order to accurately model devices connecting to metal resistors. Metal resistor short tracing is automatically checked. This keyword can be used to group different signal nets bridged by metal/poly resistors into one net for EM analysis to improve accuracy and avoid GDS trace shorting issues. Note that APLMMX automatically applies EM_NET_INFO GSR settings to nets connected using this keyword.

Syntax:

```
METAL_RESISTOR <dev_model_name1> ...
```

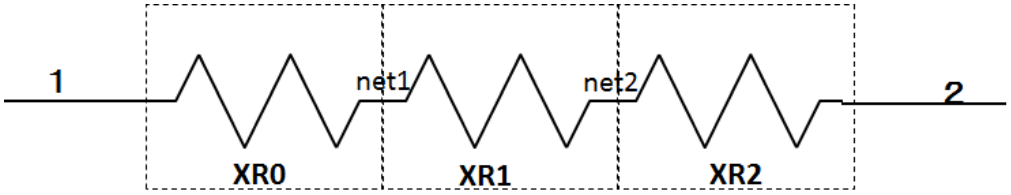
METAL_RESISTOR_FILE

Specifies a separate file listing METAL_RESISTOR device names on separate lines. To help identify special types of inductive resistors, APLMMX treats devices with an L-prefix in the SPICE netlist as a resistor type of device when defined in the metal resistor file. Supports the list of ports that need to be shorted during netlist tracing. You can provide a file that contains the device names and a list of ports to be shorted. APLMMX handles metal resistors connected by abutment even if the internal net connecting them is not present in the *NET section of the DSPF if their port numbers are defined in METAL_RESISTOR_FILE.

The format of the metal resistor file is as follows:

```
<subckt_name_or_model_name1> {
  <group_id1> {
    <pin_order_index1>
    <pin_order_index2>
    ...
  }
  ...
}
<subckt_name_or_model_name2> {
  <group_id1> {
    <pin_order_index1>
    <pin_order_index2>
    ...
  }
  ...
}
```

Example:



In the picture above, in DSPF Instance section, net1 connects XR0 and XR1, and net2 connects XR1 and XR2. Net1 and net2 may or may not be be present in *NET section.

Sample metal resistor definition in netlist:

```
subckt rm1w n1 n2 n3 n4 l=length w=width  scale='scale_res'
multi=1
```

To short n1 and n3 ports and to short n2 and n4 ports, use the GSR:

```
METAL_RESISTOR_FILE met_res_file
```

where `met_res_file` contains:

```
rm1w_test {
     ### port numbers to be shorted
     group1 {
        1
        3
     }
     ### port numbers to be shorted
     group2 {
        2
        4
     }
}
```

Syntax:

```
METAL_RESISTOR_FILE <metal_resistor_file>
```

MOBILITY_FILE

In the MMX flow, specifies a file that defines the mobility factor of devices. For vectorless signal EM, Totem by default assumes that the mobility of NMOS devices is twice that of PMOS devices. Hence in mux/pass-gate circuits, more current is drawn by the NMOS devices. The general format of the mobility file is as follows:

```
<model_name1>  <mobility_factor1>
<model_name2>  <mobility_factor2>
...
```

In this example the HVT devices have the default 2:1 mobility ratio, but the LVT, PMOS and NMOS devices are assigned similar mobility.

Syntax:

```
MOBILITY_FILE <file>
```

Example

```
# < model>
pch_lvt 1
nch_lvt 1
pch_hvt 1
nch_hvt 2
```

MULTI_THREADS

When set, provides multi-threading support in the netlist back annotation stage, which improves runtime, with a slight increase in memory usage. The maximum number of CPUs to use can be selected. Multiple-threading is supported only if the input file contains one of the following:

- an RC-extracted DSPF netlist

- a DSPF netlist and a place info file

Syntax:

```
MULTI_THREADS [On=1|Off=0] ? <num_CPUs> ?
```

MULTI_STATE_FILE

Defines capture window of each state. See the following section for more information on multi-state operations. The format of the file specified by MULTI_STATE_FILE is exactly the same as the content described inside the curly brackets of the keyword CUSTOM_STATE_SIM_TIME.

Syntax:

```
MULTI_STATE_FILE <multi-state_control_file>
```

NANOSIM_ELDO

The Nanosim Eldo interface uses different syntax for current probing. When using the NanoSim simulator and the Eldo SPICE netlist format, setting this keyword to 1 in the configuration file turns on the Nanosim-Eldo interface that eliminates manual setup work that otherwise would require users to modify the *<design>.aplmmx* file with *.defwave* and *.probe* statements.

Syntax:

```
NANOSIM_ELDO [0|1]
```

NETLIST_XY_SCALE

When set, provides a scaling factor to correct the values of x/y coordinates to match GDS. Totem attempts to convert annotations in DSPF, layout netlists, or other sources to microns. However, sometimes these annotations have unexpected values -- f or example, due to half-node scaling by extraction, or the choice of database units in LVS tools. Totem multiplies the coordinates it calculates by the value of NETLIST_XY_SCALE to produce final coordinate values.

Improperly scaled pin coordinates are a common cause of pin-snapping errors reported during GDS processing . For example, pin snapping errors can give coordinates for device pins that are 10X too large, based on what are in GDS. This can occur using a layout netlist (*.sp*) created by a Calibre-LVS run using a database unit of 0.1nm. Default: 1.0.

Syntax:

```
NETLIST_XY_SCALE <scale_factor>
```

NEW_WAVEFORM_FLOW

When set to 1, along with keyword ‘NEW_WAVEFORM_MODULE 1’ in config file, APLMMX will handle large wdb waveform file (created by Eldo simulator) with better performance and memory footprint using the new waveform processing engine.This flow can also invoked using “-new_waveform_flow” option in TCL command to run APLMMX. The flow is turned on automatically for designs with large simulation output file (>1GB). This improves accuracy for device-pins with low peak currents, and better pin-cap distribution (total cap remains same).

Syntax:

```
NEW_WAVEFORM_FLOW [0 | 1]
```

***_MODEL_NAME**

Provides user definitions for devices of model types NMOS, PMOS, NPN, PNP, BJT, DIODE, RESISTOR, CAPACITOR, and INDUCTOR. In advanced process model files, devices are generally defined as subckts inside the model file, and there are multiple types of devices inside the subckt, so without specific definitions using these keywords it is difficult to accurately determine the device type among different device naming conventions. IDsat characterization flow for special MOS devices that have more or less than 4 terminals is supported. You must use this keyword to specify the special MOS devices if the model type defined in device model library file is not standard (type=bsim, fet, psp, …).

Syntax:

```
<model_type>_MODEL_NAME <model_name>
```

NON_METAL_RESISTOR_INST_FILE

To specify instances of a given resistor model that are to be treated as 'open' during aplmmx net tracing. Input file should have one entry per instance as follows:

<Resistor_instance_name> <resistor_model_name>

All the other instances of the specified model type will be treated as metal resistors. It is not required to define these resistor models in METAL_RESISTOR section.

Syntax:

```
NON_METAL_RESISTOR_INST_FILE  <file name>
```

OUTPUT_DIRECTORY

Defines the desired output file directory for APLMMX. When specified, all outputs from APLMMX, both physical and electrical, are placed in the specified OUTPUT_DIRECTORY. (Prior to v 13.2 physical model files were output in the run directory, and the electrical model files were output in OUTPUT_DIRECTORY.) If there is a GDS_SETUP_DIR keyword, then the physical-model related files are created in GDS_SETUP_DIR specified, while the electrical model files are created in the specified OUTPUT_DIRECTORY.

Syntax:

```
OUTPUT_DIRECTORY <applmmx_output_directory>
```

OUTPUT_GSC_HIGHEST_POWER

When set, writes only the state with highest power/charge in the output GSC file, instead of all states (default: 0). Note that the total charge of all power domains is considered for identifying the highest power state. The <power_domain> option uses the charge of the specified domain for identifying the highest power state. If the charge/power ratio for a P/G net is negative, APLMMX issues a warning message. This features is not applicable to the signal-EM flow.

Syntax:

```
OUTPUT_GSC_HIGHEST_POWER [0 | 1 | <power_domain>]
```

PARASITIC_CAPACITOR

PARASITIC_RESISTOR

Supports extracted view netlists processing, to automatically recognize parasitic capacitors and resistors. If non-standard parasitic resistor/capacitor devices are detected in the extracted view netlist, an error message reminds you to define these resistors and

capacitors using these keywords. Note that if the parasitic devices are not properly specified, APLMMX may stop and probe the devices in analysis.

Syntax:

```
PARASITIC_RESISTOR {
    <R_model_name1>
    <R_model_name2>
    ...
}
PARASITIC_CAPACITOR {
    <C_model_name1>
    <C_model_name2>
    ...
}
```

PARASITIC_RESISTOR_RLIMIT

Defines a maximum threshold for parasitic resistance value. APLMMX will then trace through the resistors whose value is lesser than the defined threshold.

Syntax:

```
PARASITIC_RESISTOR_RLIMIT <resistor_limit_in_ohms>
```

PIN_SWAPPING_DETECTION

Allows support for net tracing through metal resistors. If this keyword is set to 1, Totem detects nodes and P/G ports that are shorted by metal resistors, and also removes false violations in the *mmx.xtor_miss_in_CDL_from_DSPF_netsection* file. APLMMX drops devices if all terminals are connected to P/G directly, or indirectly through the metal resistors, since those devices should be considered as part of power network.

In the LVS flow the LVS tool is allowed to swap MOS drain/source or resistor pins between the input CDL/Schematic netlist and output LVS database if the pins are on the same layer. To support this flow, APLMMX detects the pin swapping situation and automatically adjust current polarity if this keyword is set. Default: 1.

Syntax:

```
PIN_SWAPPING_DETECTION [ 0  | 1 ]
```

PLOC_EXTERNAL_NET

When set to 1, generates ploc for external PG nets (in non-CCI flows). Default: 0

Syntax:

```
PLOC_EXTERNAL_NET [1 | 0]
```

PRIMARY_OUTPUT_LOAD_MODEL

Specifies parameters to support Pi modeling for primary output loads in vectorless signal EM analysis.

Syntax:

```
PRIMARY_OUTPUT_LOAD_MODEL {
    <net> <total_C1> <total_R> <total_C2> <num_segments>
}
```

Example:

```
PRIMARY_OUTPUT_LOAD_MODEL {
    NET1 1 200 2 4
}
```

The above keyword specifies a pi model with the following elements (C in pF, R in Ohms):

C1a to gnd .25, Ra 50, C2a to gnd .5, C1b to gnd .25, Rb 50, C2b to gnd .5, C1c to gnd .25, Rc 50, C2c to gnd .5, C1d to gnd .25, Rd 50, C2d to gnd .5.

PROBE_BULK_NODE

When turned on, performs probing and modeling of the 4th terminal of MOS transistors. For a typical design, bulk node current is minimum and does not affect DvD/IR/EM analysis results, so APLMMX by default does not model it (default 0). But for some analog circuits, the bulk node current may be significant and this keyword allows modeling the bulk node in the analysis.

Syntax:

```
PROBE_BULK_NODE  [ 0  | 1 ]
```

PROBE_BUS_DELIMITER

Specifies the bus delimiter to be used in the 'probe' or 'save' statement in the file *<design>.aplmmx,* to replace the original bus delimiter in the netlist. This keyword must be used with the keyword BUS_DELIMITER.

Note: This keyword only changes the bus delimiter in the *<design>.aplmmx* file, and not in schematic to layout name mapping.

Syntax:

```
PROBE_BUS_DELIMITER "<start_bus_notation>" "<end_bus_notation>"
```

Example

```
PROBE_BUS_DELIMITER "\[" "\]"
BUS_DELIMITER "[" "]"
```

replaces the device name bus 'xyz[10]' with 'xyz\[10\]'.

PROBE_BUS_DELIMITER_MAP

This single keyword specifies bus delimiter conversion from netlist to probe files. Otherwise two keywords are needed, PROBE_BUS_DELIMITER and BUS_DELIMITER.

Syntax:

```
PROBE_BUS_DELIMITER_MAP
    <"front_delimiter-netlist"> <"front_delimiter-probe">
    ?<"back_delimiter-netlist"> <"back_delimiter-probe">?
```

Example:

```
PROBE_BUS_DELIMITER_MAP "<" "\<" ">" "\>"
```

PROBE_DEVICE_DELIMITER

Allows changing the device name hierarchy delimiter in the *<design>.aplmmx* file, and avoiding device name conflicts in certain SPICE simulation DSPF back-annotation flows.

Syntax:

```
PROBE_DEVICE_DELIMITER <character>
```

PROBE_FILE_OPTION

Defines custom settings in the APLMMX output prob file `*.aplmmx` in power analysis flow or `*.probe` file in signal EM flow.

Syntax:

```
PROBE_FILE_OPTION {
```

```
    <custom_setting>
    ...
    }
```

Example:

```
    PROBE_FILE_OPTION {
    .option xa_cmd = set_waveform_option -format fsdb_5.5
     }
```

PROBE_FILE_THREADS

When set, generates multiple aplmmx probe files, along with the original probe file with a subset of probes. This can help launch multiple simulation runs in parallel with smaller probe sets, to reduce simulation time. These multiple simulation output files can be specified and imported during the 'aplmmx -n' stage to create a single electrical model. The multiple aplmmx probe files for N threads are *<design>.aplmmx0, <design>.aplmmx1*, and. *<design>.aplmmxN-1.*

Syntax:

```
PROBE_FILE_THREADS  <no_of_threads>
```

PROBE_LEAF_CELL

Creates electrical models only at the specified leaf cell hierarchy. APLMMX can specify the subcircuit names to create probes, so that tracing stops beyond that subcircuit hierarchy. In the Signal EM flow this allows handling of black-box instances with multiple layout ports, to probe instance level currents and analyze only interface nets between different hierarchies. Groups of cells can be specified with wild-card characters as needed for efficiency. Outputs find probe points at the ports of subckt leaf instances in the *<design>.aplmmx* file. For example, in a Spectre environment the probe points appear as follows:

```
<inst1>:<port_id>
<inst2>:<port_id>
...
```

In an HSPICE-compatible environment, they appear as follows:

```
<inst1>.<port_name>
<inst2>.<port_name>
...
```

Syntax:

```
PROBE_LEAF_CELL <cell_name1> <cell_name2> ...
```

PROBE_MAP

Specifies probe variable alias name in a simulation file. If probe variable names are different in the simulation output file compared to the original probe statement you can use the keyword 'PROBE_MAP' to redefine the probe name mapping to solve this problem. For example, if the power pin name is VDD and the probe variable name in the simulation output file is iVVDD (not the same as the probe statement variable iVDD), you can specify the two names to help APLMMX find the right probe variable for VDD current.

Another use of PROBE_MAP is to allow you to redefine mapping for signals specified with keywords CUSTOM_STATE_SIM_TIME and CUSTOM_STATE_SIM_FUNC--for example, in the APLMMX configuration file.

Syntax:

```
PROBE_MAP {
    <original_probe_name> <alias_probe_name>
    ...
```

```
}
```

Example:

```
PROBE_MAP {
    ivdd i(vdd)
    ivss i(vss)
}
```

PROBE_NO_PAR

Specifies that the device name is to be used for probing. The keyword can be used to turn off parameter probing to speed up 3rd-party simulator current characterization flow, such as the performance of HSIM and other simulators. Default is 1.

Syntax:

```
PROBE_NO_PAR [ 0 | 1 ]
```

PROBE_PASS_GATE

When set, enables proper APLMMX probe generation for pass gate devices . APLMMX then always probes pass gate drain and source terminal current when it identifies VDD_NETS or GND_NETS connecting power switches or pass gates. Default: 0.

Syntax:

```
PROBE_PASS_GATE [ 0 | 1 ]
```

PROBE_SHARING

When set to 0 in APLMMX Signal EM flow, there is no probe sharing between drain and source terminal , if device is defined as model char for the mos topology where drain and source terminal connecting to the same net and the net is merged from several nets connected by metal resistor.. Aplmmx will generate probe for both drain and source terminals. For xmos (mos defined as subckt) with the above topology, aplmmx always generates probe for both drain and source terminals, no change. Default 1.

Syntax:

```
PROBE_SHARING [0|1]
```

PROBE_STOP_AT_MODEL_SUBCKT

APLMMX stops netlist tracing at the SPICE netlist level, without tracing and probing inside the device model file when 'PROBE_STOP_AT_MODEL_SUBCKT' is set to 1. Default 1.

Syntax:

```
PROBE_STOP_AT_MODEL_SUBCKT [ 0 | 1 ]
```

PROBE_THRESHOLD

Specifies that only transistors with peak current larger than the specified threshold are modeled. If there are performance and capacity concerns, when 'MACRO_TYPE MEMORY' is defined and the design has more than 100K transistor devices to be modeled, the default threshold value of 1 pA is generally satisfactory. If you have different design conditions, turn PROBE_THRESHOLD OFF, or set a more appropriate threshold value. This keyword can be used to skip signal nets with driver/receiver current smaller than the user-defined criteria. To analyze nets with small current has no impact on EM checks, so to reduce memory usage and improve performance, you can skip these nets in dynamic signal EM analysis. Note that when VECTORLESS_MODE is on, Totem compares either the average or peak current against the threshold, depending on the setting of PROBE_THRESHOLD_MODE.

Syntax:

```
PROBE_THRESHOLD [ <peak_current_threshold_Amps> | OFF ]
```

PROBE_THRESHOLD_MODE

Selects whether to compare average or peak current to the value of PROBE_THRESHOLD. Applies only when keyword VECTORLESS_MODE is set. Default is PEAK.

Syntax:

```
PROBE_THRESHOLD_MODE [AVG | PEAK]
```

PROBE_VAR_DELIMITER

Allows changing the probe variable delimiter to fit the SPICE simulation environment. The default delimiter is ".".

Syntax:

```
PROBE_VAR_DELIMITER   <delimiter>
```

Example:

If you specify 'PROBE_VAR_DELIMITER _' , the probe statement in the *<design>.aplmmx* file changes from

```
<pin_name>.apl0, <pin_name>.apl1 ...
```

to

```
<pin_name>_apl0, <pin_name>_apl1 ...
```

PROCESS

Specifies the process corner to be used in the header of APLMMX-generated current profile and cdev file.

Syntax:

```
PROCESS <corner_name>
```

SAVE_ALL_NETLIST_PAR

When set, writes all parameters of subckt/MOS from the netlist to the IDSAT file for running ACE (default 0).

Syntax:

```
SAVE_ALL_NETLIST_PAR [0|1]
```

SCALE

Specifies device width and length scale factor in the SPICE netlist.

Syntax:

```
SCALE <value>
```

Example

```
SCALE 1e-6
```

This scales the MOS device width and length by a factor of 1e-6.

SCALE_FINGER_CURRENT_BY_WL_RATIO

Provides special simulator support when finger device current is not included in the master device current probe. When set to 1, APLMMX scales the finger device current based on its width/length size ratio to the master device. Default is 0.

Syntax:

```
SCALE_FINGER_CURRENT_BY_WL_RATIO [0 | 1]
```

SEM_WST_EXCLUDE_FILE

To specify a list of nets that should be excluded from multi-cycle SEM_WST_ANALYSIS in a Signal EM Vectorless flow. Nets listed in the exclude list will be analyzed only in single cycle instead of multi-cycle worst case analysis flow. This way user can customize the analysis on complex structures.

Syntax:

```
SEM_WST_EXCLUDE_FILE <exclude_net_file>
```

Syntax of <exclude_net_file>:

```
Net1
Net2
```

SEM_IGNORE_PIN_CHECK

Totem automatically generates a warning report for nets with missing pin locations and/or a missing probes simulation output file. You can ignore these problematic nets in the output electrical model, so the nets are not analyzed in signal EM analysis, by setting this keyword to 1. *Optional. Default: 0.*

Syntax:

```
SEM_IGNORE_PIN_CHECK [0 | 1]
```

SEM_PRESIM

When turned on, supports signal nets with significant DC/leakage current, to avoid false EM violations due to incorrect DC condition setting. This keyword setting includes pre-simulation current in the current profile, so the signal EM engine can predict accurate DC conditions for transient simulation. If it is not on, and APLMMX detects that there is significant DC current from Spice simulation, a warning message is issued to inform you to turn on the option to get more accurate results. Default is Off.

Syntax:

```
SEM_PRESIM [0|1]
```

SEM_USE_HIER_NAME

When turned on, allows you to use schematic netlist device names in the Totem DB for signal EM analysis. Default: 0.

Syntax:

```
SEM_USE_HIER_NAME [0|1]
```

SEM_VECTORLESS_TIME_STEP

Handles very low frequency nets in the vectorless signal EM flow by default. Also, skips or adjusts the time step to reduce run-time according to the option specified, while analyzing low frequency nets. The syntax is:

```
SEM_VECTORLESS_TIME_STEP <time_step> ? [1|2|3] ?
```

where

1 - skips time points to the next transition

2 - adjusts time step to larger of <GSR_time_step> and <DYNAMIC_SIMULATION_TIME/100>

3 - both of above operations

SEM_WST_ANALYSIS

When turned on, identifies devices connected to the same input signal and turns them on together (in the same cycle), and separately turns on different parallel driver channels in different cycles for multi-cycle signal EM analysis, to capture the worst-case conditions. In this way each driver channel has a chance to drive the entire load of the net to potentially create the worst-case single-cycle AVG/RMS/PEAK current for each wire segment, and allow capture of worst-case AVG/RMS/PEAK EM violations. An output file, *adsRpt/apache.sigem.net_multicyc,* is generated, which lists the signal nets with multiple drivers. Default is 0 (off).

Syntax:

```
SEM_WST_ANALYSIS [0 |1]
```

SEM_WST_ANALYSIS_ON_MULTIPLE_LAYOUT_PORT

When set, Totem writes the total load cap of each net to the *dspf.rh* file, to support worst-case analysis on nets connected to multiple primary output ports. At the analysis stage, each output port of a net is hooked up with the total load cap of the net and the current flow is derived by solving the network. The worst-case current for each resistor is then selected for analysis. Default is 0 (off)

Syntax:

```
SEM_WST_ANALYSIS_ON_MULTIPLE_LAYOUT_PORT [0 |1]
```

SEM_WST_PATH_TRACING

When set, a report is generated in *AdsRpt/apache.sigem.worst_path.rpt*, a Signal EM report file that has Current/EM values for all states run in Signal EM Vectorless Analysis: Default 0 (Off).

Syntax:

```
SEM_WST_PATH_TRACING  [ 0 | 1 ]
```

SET_UNIT_CURRENT

For static analysis without characterization data, specifies the current assigned per-device, and the duration of the unit current. This feature is mainly used to flush dynamic EMIR flows and facilitate modification of device currents interactively.

Syntax:

```
SET_UNIT_CURRENT <unit_domain_current> <duration>
```

Example

```
SET_UNIT_CURRENT 500ma 1ns
```

SHORT_PG_NETS_THROUGH_METAL_RESISTOR

When set to 1, short the metal resistors bridging PG net having same nominal voltage settings in the configuration file. When APLMMX identifies that there is a metal resistor bridging two PG nets, it will first check if these two PG nets have the same nominal voltage setting in config file. If the voltage is different, it will not align and group these two nets.

Otherwise, APLMMX will automatically group these two PG nets and put them into alias list. The output *.gdsmmx file, *.spcurrent, *.cdev, *.gds.config, *_template.gsr will use the updated alias setting. Default: 0

Syntax:

```
SHORT_PG_NETS_THROUGH_METAL_RESISTOR [0 | 1]
```

SIM_OUTPUT_FILE

Defines the simulation output file path. APLMMX supports **.fsdb, *.hout, *.tr0, *.ta0, *.raw*, *.wdb (from Eldo), and **.tran* (from UltraSim or Spectre, with psf ascii format only) formats. However, for UltraSim and Spectre simulators, APLMMX prefers the **.fsdb* simulation output file.

Syntax:

```
SIM_OUTPUT_FILE <simulation_output_path>
```

SIM_OUTPUT_FILES

Characterization simulation output files are sometimes too large because of long simulation times. You can divide the SPICE simulation into different periods, as defined by their state, and then dump out output files for each state using the SIM_OUTPUT_FILES keyword. Note that this keyword should be used with CUSTOM_STATE_SIM_TIME or CUSTOM_STATE_SIM_FUNC keywords. All states defined using CUSTOM_STATE_SIM_TIME and CUSTOM_STATE_SIM_FUNC must be defined in SIM_OUTPUT_FILES if they are used in the APLMMX configuration file. Domain currents can be given in separate waveform files for same state.

Syntax:

```
SIM_OUTPUT_FILES {
    <state1> <sim_output_wavefile1>
    <state2> <sim_output_wavefile2>
    <state2> <sim_output_wavefile2A>
    ...
    <stateN> <sim_output_wavefileN>
}
```

Example:

```
SIM_OUTPUT_FILES {
    WRITE write.fsdb
    READ    read.fsdb
    PRESET preset.fsdb
}
```

SIM_OUTPUT_PREFIX

Defines the prefix path for the simulation output file (for multiple simulation output file case and when voltage derating APL_VOTLAGES is set).

Syntax:

```
SIM_OUTPUT_PREFIX <simulation_output_path_prefix>
```

For example, if simulation output files are */user/simulation/hsim.so.fsdb* and */usr/simulation/hsim.s1.fsdb*, you can use ‘SIM_OUTPUT_PREFIX /user/simulation/hsim’ in the APLMMX configuration file.

SLEW_ESTIMATION

When set to 1, internally characterizes Ron/Idsat/pin-cap of devices to auto-estimate net slew & current waveform for early analysis.

Syntax:

```
SLEW_ESTIMATION [ 0 | 1]
```

SPECTRE_NETLIST_MODE

Specifies all netlist and device model files should be in Spectre format. Default is 0.

Syntax:

```
SPECTRE_NETLIST_MODE [ 0 | 1 ]
```

SPICE_GLOBAL

Allows you to set a specified P/G pin as a global pin in SPICE netlist tracing. Also, APLMMX now honors the '.global' statement in the SPICE netlist such that global nets are connected directly to lower level pins with the same name without performing hierarchy tracing. Either of these keywords can be used when the internal power or ground domain is only defined in the intermediate sub-hierarchy and not in the top-level hierarchy

Syntax:

```
SPICE_GLOBAL <pin_name>
```

SPICE_NO_PIN_CHECK

When set to 1 does not check for devices connecting only to a power domain or only to a ground domain in the SPICE netlist, which would otherwise cause an Error condition and abort. Default is 0.

Syntax:

```
SPICE_NO_PIN_CHECK [ 0 | 1 ]
```

SPICE2LEF_PIN_MAPPING

Allows using LEF pin names instead of SPICE pin names to map pin names between simulation result files and LEF data.

Syntax:

```
SPICE2LEF_PIN_MAPPING [ 0 | 1 ]
```

SUPRESS_NETLIST_PARSE_INFO

When set, suppresses detailed Info messages while APLMMX parses the netlist. The default value is 0 (Info messages not suppressed).

Syntax:

```
SUPRESS_NETLIST_PARSE_INFO [ 0 | 1 ]
```

SWITCH_MODEL_TABLE

Specifies a switch model table file to be imported, which dumps out width, length, Idsat current, leakage current and Ron information in the output **.gdsmmx* file for power switch pins. This embedded power switch flow handles situations in which the power switch hierarchy is not preserved in GDS layout. To support multiple-vdd switch models, you can specify switch model parameters RON, Idsat, etc., for different voltage values for each transistor instance. APLMMX picks up the appropriate model based on its VDD domain voltage.

The format of the switch model table file is:

```
#<DeviceModel> <Width(um)> <Length(um)> <Ron(Ohms)> <IdSat(Amps)>
    <Ileak(Amps)> <Voltage(Volts>
```

Example:

```
# DeviceModel Width(um) Length(um) Ron(Ohms) IdSat(Amps) Ileak(Amps) Voltage
     *       2.295    0.041209     190.80   0.00162355   2.83115e-08  0.850
    pch      2.295    0.041209     190.80   0.00162355   2.83115e-08  0.990
    nch      0.810    0.03864      310.188  0.00106106   1.82968e-08  0.950
```

Wildcards '*' and '-' are accepted for the DeviceModel column. Also, APLMMX performs interpolation for width and length when MOS device width and length values in the netlist do not have exact matches with the input table. With this input table, APLMMX dumps extra entries in the output *<design>.gdsmmx* file for power switch devices.

The format of power switch pins data in the output *<design>.gdsmmx* file is:

```
<pin_name_sw> <Xtor_name> <x_loc> <y_loc> <p/n ratio> TYPE{}
    TID{} MDL{} W{} L{} RON{} IDS{} ILK{}
```

Note that On-state resistance and saturation current are scaled based on the W/L ratio of the device when multiple device models with the same W or L parameters are specified in SWITCH_MODEL_TABLE.

Syntax:

```
SWITCH_MODEL_TABLE <file_name>
```

SWITCH_SUBCKT

Specifies switch subcircuit definitions to be passed to ACE for internal power/ground domain handling.

Syntax:

```
SWITCH_SUBCKT <subckt_name>
```

TCL_REGULAR_SQUARE_BRACKET

When set, typical wild card characters “[“ and “]” are treated as regular characters in GSR keyword ‘EM_NET_INFO’ and APLMMX configuration file keyword ‘PRIMARY_OUTPUT_LOAD_MODEL’, the same as is in keywords ‘IGNORE_NETS’, ‘IGNORE_NETS_FILE’, ‘ANALYZE_NETS’, and ‘ANALYZE_NETS_FILE’. Default is off (characters “[“ and “]” in the TCL wrapper system are treated as wildcards).

Syntax:

```
TCL_REGULAR_SQUARE_BRACKET [ 0 | 1 ]
```

. Example

When `TCL_REGULAR_SQUARE_BRACKET` is set, the IGNORE_PROBE_INST_FILE expression “abc/def[*]” ignores instances such as *abc/def[1], abc/def[2],* and *abc/def[3]* .

TEMP

Defines the temperature used in simulation.

Syntax:

```
TEMP <simulation_temperature>
```

TEST_BENCH_FILE

Defines testing vectors/ stimulus for simulation (only needed if simulation is run inside APLMMX).

Syntax:

```
TEST_BENCH_FILE <test_vector_file>
```

TIEOFF_XTOR_SUPPORT

When set, identifies the types of devices listed below as tied off when performing static and dynamic analysis. When this keyword is turned on, the tie-off transistors have a 0 pratio in the output **.gdsmmx* file, and when BPFS or BPA are used to assign power,

these tied-off transistors get zero current. This keyword also handles tied-off intentional decap devices. Default = 0 (off).

NMOS

- source/gate ties to ground, drain to power (signal)
- drain/gate ties to ground, source to power (signal)

PMOS

- source/gate ties to power, drain to ground (signal)
- drain/gate ties to power, source to ground (signal)

Syntax:

```
TIEOFF_XTOR_SUPPORT [0 | 1]
```

TOGGLE_RATE

Defines the toggle rates to be used for vectorless flow. If only one toggle rate is specified, it is applied to the bidirectional nets. By default, the clock toggle rate is 2, and the unidirectional net toggle rate is calculated as 50% of the bidirectional toggle rate.

Syntax:

```
TOGGLE_RATE <bidirect_net_TR> <Clk_TR> ?<unidirect_net_TR>?
```

TSTOP

Defines the simulation stop time for multiple voltage derating simulation. It also can be used in the DEVICE_CURRENT_FILE flow to set up the static spcurrent duration as 1/ frequency.

Syntax:

```
TSTOP <stop_time><unit>
```

USE_XY_FROM_INST_SECTION

When set, picks transistor location from the Instance section of the DSPF netlist. When this keyword is set to “1”, locations from the Instance section get precedence over those in '*|I' line of NET section of the DSPF netlist. Default is 0 (off).

Syntax:

```
USE_XY_FROM_INST_SECTION [ 0 | 1 ]
```

USIM_LOOKUP_SPICE

When set to 1, adds the line “`simulator lang=spectre lookup=spice`” at the beginning of the <*design*>*.aplmmx* file to assist UltraSim map the device name in simulation.

Syntax:

```
USIM_LOOKUP_SPICE [ 0 |1 ]
```

USIM_SAVE

Specifies the use of Ultrasim or Spectre ‘save’ statements in <*cell*>*.aplmmx*. This keyword is On by default when UltraSim, APS or Spectre simulators are used for current characterization, and Off by default otherwise. For example, in <*cell*>*.aplmmx*,

```
save xtop.XI10.M15:1
```

You can also use ‘USIM_SAVE 0’ to turn off ‘save’ probe syntax and use ‘.probe’ when running UltraSim in the regular HSpice simulation environment. Default is 0 (off).

Syntax:

```
USIM_SAVE [ 0 | 1 ]
```

VDD_NETS

Specifies the voltages for power domains and also define the equivalent power domain names in the SPICE netlist which are merged in APLMMX for Totem analysis. Do not define duplicate entries in VDD_PIN_NAME, INTERNAL_POWER_PIN_NAME and VDD_NETS, which will cause program error. Instead use the GSR keyword VDD_NETS. Also supported for signal EM analysis.

Syntax:

```
VDD_NETS
{
    <Vdd_domain_net_name> <value_Volts> {
    <equiv_Vdd_net_name1>
    ...
    }
    ...
}
```

VECTORBASED_NET_FILE

In mixed mode Signal EM analysis, by default Totem compares every driver current with the PROBE_THRESHOLD value, and even if one of the driver currents is less than the value of PROBE_THRESHOLD Totem analyzes the net in vectorless mode. To provide more analysis control, use this keyword to specify a file containing a list of nets to be analyzed as part of vector analysis, but not vectorless.

Syntax:

```
VECTORBASED_NET_FILE <filename>
```

VECTORLESS_MODE

By default, when Totem calculates power EM (PEM), it uses currents from external simulation (vector-driven static or dynamic PEM), or from static block power assignments. When this keyword is set, Totem combines current values calculated by vectorless signal EM (SEM) with externally supplied currents--that is, “rolls up” SEM values to use for static PEM.

When performing a roll-up, Totem calculates AVG, RMS, and PEAK current metrics for each signal net or port connected to the rail of interest, based on the multi-cycle, worst-case-scenario, vectorless SEM analysis. It also calculates vector-driven AVG/RMS/PEAK metrics based on the external waveforms provided, and ignores waveforms with metrics below the value of keyword PROBE_THRESHOLD. Totem checks the AVG or PEAK metric depending on the setting of keyword PROBE_THRESHOLD_MODE.

Totem then determines which values to roll up, based on the VECTORLESS_MODE setting, as follows:

- VECTORLESS_MODE 1: Totem calculates SEM metrics based on external waveforms for pins/ports on signal nets only, and then rolls up the larger AVG, RMS, and PEAK values from the vector-driven and vectorless current calculations.

  With this setting, Totem calculates probe mapping percentage based only on signal-net waveforms. Unless >10% of these waveforms exist and have names that map to simulation-netlist names, Totem issues an error and stops because the mapping rate

is too low.

- VECTORLESS_MODE 2: Totem calculates SEM metrics based on external waveforms for pins/ports on power nets only, and then rolls up the larger AVG, RMS, and PEAK values from vector-driven and vectorless current calculations.

  With this setting, Totem calculates probe mapping percentages based only on power-net waveforms. Unless >10% of these waveforms exist and have names that map to simulation-netlist names, Totem issues an error and stops because the mapping rate is too low.

With either setting, using the larger of vector-driven and vectorless metrics assures full coverage of all load currents on each power rail. Totem substitutes vectorless values for pins that were inactive during external simulation, and external waveforms supply currents for non-digital signal nets not modeled by vectorless SEM.

Use these roll-up calculations for static PEM, when you want to check AVG EM limits on the power rails. RMS and PEAK EM checks can give pessimistic results, since the roll-up calculation works only with RMS and PEAK metrics, not with the actual load waveforms. Generally, you should use vector-driven PEM for accurate RMS and PEAK checks. Default is Off (0).

Syntax:

```
VECTORLESS_MODE [ 0 | 1 | 2]
```

VOLTAGE_SOURCES

Specifies the voltage source names of each power and ground domain (only required in the ‘APLMMX -s’ flow if the voltage source is not defined in test bench file). The <pin_name> should match the power/ground pin name in the netlist and the <volt_src_name*> should match the actual voltage source names in the test bench during SPICE simulation.

Syntax:

```
VOLTAGE_SOURCES {
    <pin_name1> <volt_src_name1>
    <pin_name2> <volt_src_name2>
    ...
}
```

VP_PAIRING

VP_PAIRING defines internal and external power and ground pin pairs.

Syntax:

```
VP_PAIRING (
    <intern_power/ground_pin1> <extern_power/ground_pin1>
    ...
)
```

WAVEFORM_TOKEN_RENAME

When set, allows you to rename tokens in the probed signal names in the waveform file. Note that the mapping rule 'WAVEFORM_TOKEN_RENAME “^” “/”' is set as default to avoid probe mapping issues after upgrading to the FSDB 5.0 reader library, which automatically converts the “/” hierarchy divider to “^”. Configuration file keyword IGNORE_ESCAPE_CHAR is obsolete in the new name mapping engine and user need to provide WAVEFORM_TOKEN_RENAME "\" "" in aplmmx config file if the probes names in the FSDB have an escape character.

Syntax:

```
WAVEFORM_TOKEN_RENAME “<original_token>” “<new_token>”
```

Example

```
WAVEFORM_TOKEN_RENAME "^" "/"
```

This setting maps the probed signal name “x1^x2^m1” in the simulation output waveform file to “x1/x2/m1” during APLMMX waveform processing.

XTOR_VIRTUAL_DERATING_FILE

APLMMX perform spatial voltage derating in addition to temporal derating when the aplmmx current profile is captured at different voltage samples, which produces more accurate analysis compared to single voltage level spcurrent data, but requires only one voltage simulation. MMX pin current derating is performed based on the voltage drop at the particular pin at every time point This keyword specifies the table lookup derating file.

Syntax:

```
XTOR_VIRTUAL_DERATING_FILE <derating table filename>
```

XMOS_INST_PARAM

When set to 1, APLMMX passes all x-instance parameters in netlist to embedded switch flow during switch characterization for increased accuracy. *Default: 1*

Syntax:

```
XMOS_INST_PARAM [0 | 1]
```

Sample APLMMX Configuration File

The following is a typical APLMMX configuration file :

```
TOP_CELL RAM256
MACRO_TYPE memory
VDD_PIN_NAME VDD
GND_PIN_NAME VSS
VDD 1.1
SPICE_NETLIST user_data/RAM256.cdl
DSPF_NETLIST user_data/RAM256.dspf
DEVICE_MODEL_LIBRARY user_data/RAM256.lib TT
TEST_BENCH_FILE user_data/RAM256.tbh
```

Modeling MOS intentional device capacitance

APLMMX recognizes intentional decap devices made by MOS junction capacitance, using characterized junction cap parameters (cj_area, cj_sw, cj_w) from ACE and the PS, PD, AS, AD, W parameters from the Spice netlist to calculate total junction capacitance of the following types:

- PMOS - power (drain), ground (gate), power (source)
- NMOS - ground (drain), power (gate), ground (source)
- NMOS - power (drain), power (gate), power (source), ground (bulk)
- PMOS - ground (drain), power (gate), ground (source), power (bulk)

In the last two cases drain and source are tied to the same potential, so there can be no switching current, and hence these devices are automatically recognized by APLMMX as intentional decap.

Using ‘m-factor' to split subcircuit X-instances

APLMMX supports the use of 'm-factor' division of subcircuit x-instances in the SPICE netlist, which splits the schematic x-instance with m-factors into multiple x-instances with a finger delimiter defined in the post-layout netlist or in the DSPF netlist. This increases the accuracy of modeling by avoiding false violations due to the concentration of all device current and device capacitance in only one master x-instance when instances are not split. Note that x-instances using 'm-factor' need to have finger delimiters at the x-instance level in order to trigger the x-instance finger splitting. For example, in the schematic netlist for an 'm-factor' of 4:

```
XI_mod   in   net0   inv   m=4
```

and in the layout netlist, the four “fingers” are described:

```
XI_mod   in   net0   inv   $x=100   $y=150   $t=…
XI_mod@2   in   net0   inv   $x=120   $y=180   $t=…
XI_mod@3   in   net0   inv   $x=150   $y=240   $t=…
XI_mod@4   in   net0   inv   $x=150   $y=240   $t=…
```

The default finger delimiter “@” is used to denote the 'm-factor' x-instances in the layout netlist. You can specify a different finger delimiter using the APLMMX configuration file keyword 'FINGER_DELIMITER <symbol>'.

Support for XA as a Third Party Simulator

Totem supports the Synopsys XA simulator as a third party simulator for APLMMX, which has a different usage flow, as described below. In the APLMMX configuration file you must have the keyword:

```
SPICE_SIMULATOR <XA_binary_path> <XA_simulation_options>
```

The '-i' XA option should be put at the end of option set, since APLMMX appends a SPICE netlist at the end of internal SPICE command submission. APLMMX can support XA Eldo, HSPICE, and Spectre interfaces, as described below.

To run the XA HSPICE interface:

```
SPICE_SIMULATOR  <binary_path_of_XA>  -hspice <options>
```

or

```
SPICE_SIMULATOR  <binary_path_of_XA>  -n <options>
```

To run the XA Eldo interface:

```
SPICE_SIMULATOR  <binary_path_of_XA>  -neldo <options>
```

or

```
SPICE_SIMULATOR  <binary_path_of_XA>  -eldo <options>
```

To run the XA Spectre interface:

```
SPICE_SIMULATOR  <binary_path_of_XA>  -nspectre <options>
```

or

```
SPICE_SIMULATOR  <binary_path_of_XA>  -spectre <options>
```

While using XA Spectre interface, APLMMX by default will write probes in Spectre syntax. For generating probes in spice syntax, set configuration keyword ‘USIM_SAVE 0’ in APLMMX config file.

The default output file format of XA is the same as an FSDB file. If you use a different simulation output format, you must define the simulation output filename prefix using the APLMMX keyword ‘SIM_OUTPUT_PREFIX <prefix>. For example, if there are three voltage sweeps in the simulation output, XA generates simulation output files such as *xxx.fsdb0, xxx.fsdb1, xxx.fsdb2,* or *xxx.tr0, xxx.tr1, xxx.tr2.*

So you then define the following option in APLMMX configuration file:

```
SIM_OUTPUT_PREFIX  xxx.fsdb
```

or

```
SIM_OUTPUT_PREFIX  xxx.tr
```

Special APLMMX Applications

Multi-state Analysis Support

Two special APLMMX configuration file keywords, CUSTOM_STATE_SIM_TIME and MULTI_STATE_FILE, are used to define the capture window for different states. Note that multi-state mode supports only memory/IP cells, and not combination and sequential cells.

NOTE: When creating multiple-state models for mixed signal designs with digital Selements, you must capture current profiles for reserved states ‘standby_trig’ and ‘standby_ntrig’. During dynamic analysis (vectorless/ VCD), if a memory *does not switch* in one of the other custom states (read/write/compare, and so on), they are assumed to be in standby mode and a current profile corresponding to a standby state is used. If standby states are not captured, Totem treats the instances not switching in other custom states as decaps and they do not appear in the output *adsRpt/*dvd** reports and in instance DvD colormaps.

CUSTOM_STATE_SIM_TIME

Supports custom multiple-state Boolean expressions in LIB files. APLMMX also allows specifying custom states based on summation and scaling of current profiles. Each line is a state definition (evaluated in order). User can specify different capture window for each net for a given state. However, the duration of different nets for a given state should remain consistent. Different states can hold different duration but duration of nets within each state should be the same. The syntax for the CUSTOM_STATE_SIM_TIME is described below. You can define one, or more than one, state inside the curly brackets:

```
CUSTOM_STATE_SIM_TIME {
    <state_name1> <Boolean_express> <active_input> VDD_PIN <vdd_pin_name1>
<start time> <end time> VDD_PIN <vdd_pin_name2> <start time> <end time>
GND_PIN <gnd_pin_name1> <start time> <end time> GND_PIN <gnd_pin_name2>
<start time> <end time>
    <state_name1> <Boolean_express> <active_input> <start_time> <end_time>
    <state_name2> <Boolean_express> <active_input> <state_operation>
    ...
    }
```

where

- <state_name>: defines the state symbol for each custom state for Totem-MMX dynamic analysis
- <Boolean_expression>: defines the Boolean expression to be used in Totem-MMX VCD mode analysis to determine when the specific state will be On (active)
- <active_input> : defines switching input pin(s)
- <start_time> : defines start time of capture window for the state
- <end_time> : defines end time of capture window for the state

<state_operation>: allows linear combinations of multi-state currents with Boolean definitions

Vectorless Mode Example:

```
  a “” “” 0 2e-9
  b “” “” 0 2e-9
  c “” “” 1e-9 3e-9
  d “” “(!ali)”  “ c+a+b “
  e “(a|b)!c” “” “a+b+2*c”
}
```

VCD Mode Example:

```
CUSTOM_STATE_SIM_TIME {
WRITE “!CEN&!WEN&CLK” “CLK” VDD_PIN VDDPE 1.5ns 1.837ns
VDD_PIN VDDCE 2.3ns 2.637ns VSS_PIN VSS 10.0ns 10.337ns
READ “!CEN&WEN&CLK” “CLK” 5ns 10ns
STANDBY_TRIG “CLK” “CLK” 15ns 20ns
STANDBY_NTRIG “!CLK” “!CLK” 25ns 30ns
}
```

You can also use the MULTI_STATE_FILE keyword to define capturing window for states in an additional control file.

The GSC_FILE keyword must be specified in multi-state vectorless dynamic analysis, to determine which state is used in dynamic simulation. Please refer to Case 2 in the Netlists section for GSC_FILE setup.

Linear Combinations of Multi-state Currents

APLMMX allows linear combinations of multi-state currents with Boolean definitions, by summing multi-state currents from each contributing pin, defined using keywords CUSTOM_STATE_SIM_TIME or CUSTOM_STATE_SIM_FUNC. For example:

```
CUSTOM_STATE_SIM_TIME {
 a “” “” 1e-9 2e-9
 b “” “” 3e-9 4e-9
 c “a+2.5*b”
}
```

In this example, after running APLMMX, the final current profile has three states in the *.*spcurrent* file, including the combination pin current of state 'c', which is the pin current of state 'a' plus 2.5 times the pin current of state 'b'.

Multiple Characterization Simulation Output Files

Characterization simulation output files are sometimes very large because of long simulation times. You can divide the SPICE simulation into different periods, as defined by their state, and then create separate output files for each state. The APLMMX configuration file keyword syntax for specifying these partial output files is:

```
SIM_OUTPUT_FILES {
    <state1> <sim_output_file1>
    <state2> <sim_output_file2>
    ...
    <stateN>  <sim_output_fileN>
    }
```

For example:

```
SIM_OUTPUT_FILES {
```

```
WRITE  write.fsdb
READ    read.fsdb
PRESET preset.fsdb
}
```

Note that this keyword must be used with keywords CUSTOM_STATE_SIM_TIME or CUSTOM_STATE_SIM_FUNC. All states defined in CUSTOM_STATE_SIM_TIME or CUSTOM_STATE_SIM_FUNC must be included in the SIM_OUTPUT_FILES specification to ensure a complete set of output files.

Peak Power Cycle Identification

In addition to using the GSR keyword 'CUSTOM_STATE_SIM_TIME <start_time> <end_time>', and calculating the area under the curve for that duration, APLMMX can also automatically pick the peak power cycle for dynamic analysis from the ivdd profile (total sum of currents at the block level). The target net to calculate peak power is the first Vdd net defined in VDD_PIN_NAME in APLMMX configuration file. Two additional Boolean methods to pick the cycle that has peak charge are described following:

1. Define the capture window duration, then the start time is based on the Boolean expression and clock pin activity using the CUSTOM_STATE_SIM_TIME keyword:

   ```
   CUSTOM_STATE_SIM_TIME {
       <state> <Boolean_eq> <clk> <duration>
       ...
       }
   ```

   For example:

   ```
   CUSTOM_STATE_SIM_TIME {
       write  WEN&SET  clk  2e-9
       ...
       }
   ```

2. CUSTOM_STATE_SIM_FUNC picks up the valid cycle based on Boolean expression and clock activity. The syntax is:

   ```
   CUSTOM_STATE_SIM_FUNC {
       <state> <Boolean_eq> <clk>
       ...
       }
   ```

   For example:

   ```
   CUSTOM_STATE_SIM_FUNC {
       s1 "!wea_n & !mea_n & !meb_n" " clka "
       ...
       }
   ```

If no Boolean equation is defined, APLMMX chooses the cycle with maximum current (charge) among all valid clock cycles.

There are also two optional configuration file keywords that allow you to refine your settings in finding the peak power cycle, PROBE_MAP and CUSTOM_STATE_TARGET_PIN, as described in the following paragraphs.

PROBE_MAP

If you find that probe variable names are different in the simulation output file compared to the original probe statement, you can use the keyword 'PROBE_MAP' to redefine the

probe mapping with an “alias” name to solve this problem. Also, to specify current waveform polarity inversion, add “1” to the end of the line.

The syntax is:

```
PROBE_MAP {
  <PG_pin_name-signal_name> <alias_probe_name_in_sim_output> ? 1 ?
  }
```

For example, if the power pin name is VDD and the probe variable name in the simulation output file is iVVDD (not the same as the probe statement variable iVDD), you can use the following setting to help APLMMX find the right probe variable for VDD current.

```
PROBE_MAP {
vdd   ivvdd   1
vss  ivvss
}
```

In the above example, APLMMX finds the ivvdd probe variable in the simulation output file for pin 'vdd' and inverts the waveform during waveform processing.

Another way to use PROBE_MAP is to help you redefine mapping for signals specified with keywords CUSTOM_STATE_SIM_TIME and CUSTOM_STATE_SIM_FUNC. For example, in a APLMMX configuration file:

```
CUSTOM_STATE_SIM_TIME {
write WEN&SET clk 2e-9
}
CUSTOM_STATE_SIM_FUNC {
read WEN&SET clk
}
PROBE_MAP {
clk clock
}
```

In this example simulation output file, the signal 'clk' has a different probe variable name, 'clock'. So you can use PROBE_MAP to insure that APLMMX performs the correct mapping.

CUSTOM_STATE_TARGET_PIN

If you specify the keyword CUSTOM_STATE_SIM_FUNC, APLMMX chooses cycles based on the first Vdd pin defined in VDD_PIN_NAME in the configuration file. Then you can use CUSTOM_STATE_TARGET_PIN to redefine which P/G pin is used to determine the peak power cycle, as follows:

```
CUSTOM_STATE_TARGET_PIN <vdd_pin_name/gnd_pin_name>
```

For example, suppose there are multiple P/G pins named vdd1, vdd2, gnd1, and gnd2, in the design. If you want to use 'gnd1' to calculate current/charge for cycle selection instead of 'vdd1' you can use the following specification in the configuration file:

```
CUSTOM_STATE_TARGET_PIN gnd1
```

Note that the CUSTOM_STATE_SIM_FUNC and CUSTOM_STATE_SIM_TIME keywords can be used at the same time as long as the state names are not in conflict. This is shown in the following example:

```
CUSTOM_STATE_SIM_TIME {
aa "" "" 0e-9 1e-9
bb b b 0.4e-9 0.9e-9
}
CUSTOM_STATE_SIM_FUNC {
f d[0]&(!cen) clk2
```

```
g "!wen & !cen & ( d[0]* !a[0] )" !clk
h "cen+ wen" clk
i " ! cen & !wen" !clk
}
PROBE_MAP {
clk2 clkii
vdd_low VDDL
}
CUSTOM_STATE_TARGET_PIN vdd_low
```

Output

The output file *adsRpt/<design>.state_cyscle_scan.rpt* contains the calculated charge in each cycle, as in the following example:

```
# Cycle Scan Report to find out peak total charge cycle
# Simulation time: from 4210 ps to 60000 ps
# Target net: VDD
Sweep Number: 0
-----------------------------------
State: s1
Boolean expression:!wea_n&!mea_n&!meb_n <=> wea_n ! mea_n ! & meb_n ! &
Clock pin: clka
from 4210 ps to 12610 ps : total charge = 9.758370 pCoul
from 21010 ps to 29410 ps : total charge = 9.785883 pCoul
from 37810 ps to 46210 ps : total charge = 9.739245 pCoul
from 54610 ps to 60000 ps : total charge = 8.842555 pCoul
```

If you want to choose a different peak cycle, you can do it from the report of ranked cycles, and then use CUSTOM_STATE_SIM_TIME, with <start_time> and <end_time> specifications, to redefine the capture window.

Static IR analysis using DC-OP information

On designs in which the activity is predominantly static (DC), you can provide a DC OP file as an input to Totem to perform static IR and EM analyses. To model devices with static currents, you should specify a DEVICE_CURRENT_FILE in the APLMMX configuration file.

The format of the information in the file is either from the HSim dc op file or from a Spectre **info.oppoint* file. The following is a Hsim dc op file format example:

```
*i(<device_name>)=<device_dc_current_A>
...
```

For the Spectre DC-operating point file in DC-op flow, use the following steps:

1. Create a probe file using the aplmmx –s stage
2. Include the *< >.aplmmx* probe list in the Spectre simulation testbench, add the options “dcOpInfo info what=oppoint where=rawfile” in the testbench to dump out a *<file_name>.info.oppoint* file, and use the Spectre command-line option “-format psfascii”.
3. Specify the dcop file in the aplmmx configuration file using DEVICE_CURRENT_FILE, and run the ‘aplmmx –gds’ step to generate a static DC current profile.
4. Set ANALOG_ANALYSIS 1, POWER_MODE APL and specify the static DC current profile from step 3 in the GSR, then perform static analysis in Totem.

The following is a Spectre dcOpInfo file format example:

```
**************************************
Operating-Point information `dcOpInfo':
**************************************
Operating point computed in DC analysis `dcOp' at T = 70 C.
...
Instance: Itop.M0 of pch_mac
Model: Itop.M0.pch_mac.24
Primitive: bsim4
d : V(Itop.net016) = 1.8 V
g : V(Itop.vdda) = 101.322 nV
s : V(vcp) = 1.8 V
b : V(vcp) = 1.8 V
region = triode
reversed = no
ids = -104.471 pA
vgs = -1.8 V
vds = -458.837 nV
vbs = 0 V
vgd = -1.8 V
vdb = -458.837 nV
...
```

In the APLMMX configuration file, you should specify the dcop file as follows.

```
DEVICE_CURRENT_FILE <dcop_file>
```

Use the “aplmmx -gds” option to generate the *<design>_static.spcurrent* file.

The current of each transistor device from the dc op file is dumped into a static dc current profile, '*<design>_static.spcurrent*. You then can specify this static current profile using the GSR keyword GSR APL_FILES <files> and turn on POWER_MODE APL keyword, as well as set ‘ANALOG_ANALYSIS 1’ in the Totem GSR file to perform static analysis.

Example in GSR:

```
...
APL_FILES {
<design> <design>_static.spcurrent current
}
POWER_MODE APL
ANALOG_ANALYSIS 1
...
```

Note: Do not set the FREQ keyword in the GSR when turning on ANALOG_ANALYSIS (FREQ takes priority).

Voltage Derating

To provide voltage derating in APLMMX, use one of the following procedures. If you use NSPICE, you can use the APL_VOLTAGE keyword in the MMX configuration file. For example,

```
APL_VOLTAGE 2 1.0 0.9
```

If you use a third party simulator like HSIM, do the following:

1. Use the ‘SIM_OUTPUT_PREFIX <filepath_prefix>’ keyword in the APLMMX configuration file, and use the file suffix *.fsdb.*
2. Also specify ‘SPICE_SIMULATOR [hsim | nanosim]’, since APLMMX assumes that *fsdb* is the default output format for these two simulators.

3. Rename the *fsdb* file:

```
<sim_output_fix_pattern>.s0.fsdb
<sim_output_fix_pattern>.s1.fsdb
...
```

4. The order of the *fsdb* files should follow the APL_VOLTAGE derating order. For example, for 'APL_VOLTAGE 2 1.0 0.9', the result of derating 1.0 should be placed at *<sim_output_fix_pattern>.s0.fsdb*, and the result of derating 0.9 should be placed at *<sim_output_fix_pattern>.s1.fsdb.*

Table Lookup-based Voltage-derated spcurrent

For large designs, running current characterization on multiple voltage levels can be very time-consuming. To overcome this issue, APLMMX can create voltage-derated spcurrent values based on a specified current derating ratio, which produces more accurate analysis compared to single voltage level spcurrent data. The APLMMX configuration file keyword to specify the table lookup derating file is:

```
XTOR_VIRTUAL_DERATING_FILE <filename>
```

For example, an APLMMX configuration file could be as follows:

```
VDD_PIN_NAME VDD1 VDD2
GND_PIN_NAME VSS
VDD 1.6  1.2
APL_VOLTAGES 3 1 0.9 0.8
XTOR_VIRTUAL_DERATING_FILE deratingfileABC
```

The format and syntax of the derating file is:

```
* {
    <current derating ratio for voltage level 1>
    <current derating ratio for voltage level 2>
    ...
 }
 <device_model_name1> {
    <current derating ratio for voltage level 1>
    <current derating ratio for voltage level 2>
    ...
}
<device_model_name2> {
   <current derating ratio for voltage level 1>
   <current derating ratio for voltage level 2>
   ...
}
...
```

where "*" indicates wildcard support for all device model types not specifically defined.

An example of derating file contents is:

```
* {
    1
    0.9
    0.75
    }
 N18 {
    1
    0.88
```

```
0.72
}
```

In the example above, the transistor pin current for device model name 'N18' at voltage level 1 is scaled by factor 1, at voltage level 2 it is scaled by factor 0.88, and at voltage level 3 it is scaled by factor 0.72. Devices with all other device model names use the derating factors 1, 0.9, and 0.75 for voltage levels 1, 2, and 3.

Characterization of Leakage Current

There are two methods of extracting leakage current for transistor pins :

1. Leakage current is determined as an average of starting current and ending current values, using 'EXTRACT_LEAKAGE APL' keyword in the configuration file.
2. The leakage for each power-ground arc is directly specified using the 'LEAKAGE_I { <vdd_name> <gnd_name> <leakage> }' keyword in the configuration file.

   For example:

   ```
   LEAKAGE_I {
   vdd vss 1e-6
   }
   ```

In both cases, the leakage current is subtracted in the output *spcurrent* file and put in the *.cdev* file as leakage current for each transistor pin.

Characterization of Designs with Power Switches

APLMMX and ACE decap characterization setups are integrated for MMX-based designs. When there are switch subcircuits or switch transistors with internal and external nets in the netlist APLMMX generates the ACE configuration file automatically inside the flow. The integrated APLMMX configuration file setup syntax for switch IP support is as follows:

```
VDD_PIN_NAME <extern_power_pin_name1> <extern_power_pin_name2> ...
GND_PIN_NAME <extern_ground_pin_name1> <extern_ground_pin_name2> ...
VDD <extern_power_pin1_voltage> <extern_power_pin2_voltage> ...
INTERNAL_POWER_PIN_NAME {
<intern_power_pin_name1> <voltage_level1>
<intern_power_pin_name2> <voltage_level2>
...
}
INTERNAL_GND_PIN_NAME {
<intern_ground_pin_name1>
<intern_ground_pin_name2>
...
}
VP_PAIRING (<intern_power/ground_pin1> <extern_power/ground_pin1>)
VP_PAIRING (<intern_power/ground_pin2> <extern_power/ground_pin2>)
VP_PAIRING (...)
? SWITCH_SUBCKT <switch_subckt>?
...
```

See previous sections of this chapter describing the use of these keywords. \

Automatic modelling of Output port current for regulated nets

For voltage-regulators/LDO or any other design with pass-gates between primary and regulated supplies, Totem will model the internal terminal of the pass gate (on regulated

net) as ploc and also probes output (load) current at the port of regulated net. A similar method would be followed for multistage pass gates also(as shown in Figure-3 below).

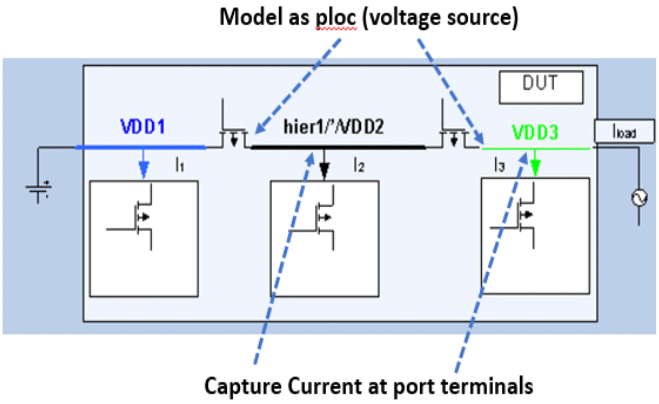


Figure 3-1 modeling multi-stage pass-gates.

The port definition can be either from RC-DSPF or CCI flow nif file. Either of following two options are required

1. Using regulated-nets flow

```
REGULATED_POWER_NETS {
VDD2 1.1
VDD3 1.1
}
```

optionally DEVICE_MODEL_CONFIG if there are multistage pass gates.

```
DEVICE_MODEL_CONFIG model.conf
```

Example DEVICE_MODEL_CONFIG:

```
PMOS pchana 1(NA) 2(NA) 3(NA) 4(NA)
PIN_PAIR {
  pchana {
    PIN_GROUP1 {
       1
    }
    PIN_GROUP2 {
       3
    }
  }
}
```

2. Using switch flow:

```
INTERNAL_POWER_PIN_NAME {
VDD2 1.1
VDD3 1.1
}
LDO_INST {
Inst1
Inst2
}
```

optional DEVICE_MODEL_CONFIG if there are multistage pass gates.

Output:

- `adsRpt/<cell>.aplmmx.ploc` for usage #1 above only: This ploc contains location for regulated nets VDD2 and VDD3. It also contains location for other external nets.
- `gdsmmx:` Aplmmx will creates pins and current source for LDO output port.

Checking and Debugging APLMMX Characterization Results

Resistor Lists

Resistor Models

APLMMX automatically creates an output file *adsRpt/mmx.identified_resistor_models* listing all resistor models identified in the netlist, to confirm if they match design specifications.

Resistors Bridging Power Domains

During netlist tracing a file *adsRpt/mmx.metal_resistor_bridge_domains* is generated listing all resistors bridging two different power/ground nets. When two nets are connected through a designed resistor, Totem analyzes the nets separately, but current waveforms on either side of the resistor would be the same. On one net, the resistor terminal is treated as receiver with bi-directional current. For the other net, the resistor terminal is treated as a current source with bi-directional current. This feature is turned on by default.

Other Checks

Checking if FSDB has the probe currents

Most Spice simulators can generate an FSDB (Fast Signal Database) file that contains the output current probes. The following checks should be made.

- Verify that all parent current probes are present and probed correctly in the simulation. You may use any of the FSDB viewers (nWave, Meta-Waves, C-scope). For example, Figure 3-2 following shows a three-parent current probe in a design (Vddce, Vddpe, Vsse).

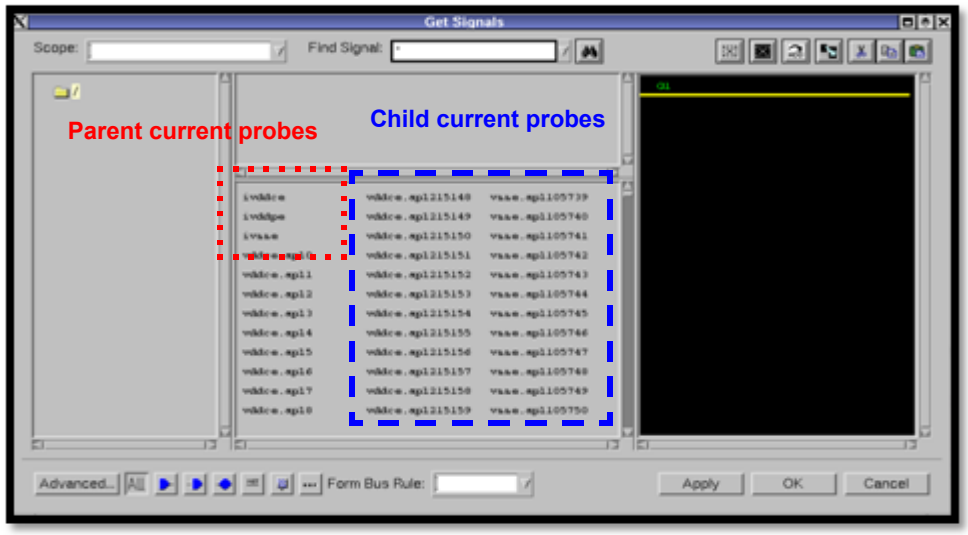


Figure 3-2 Parent and child current probe lists

- Verify the probe currents in the viewer, as shown in Figure 3-3.

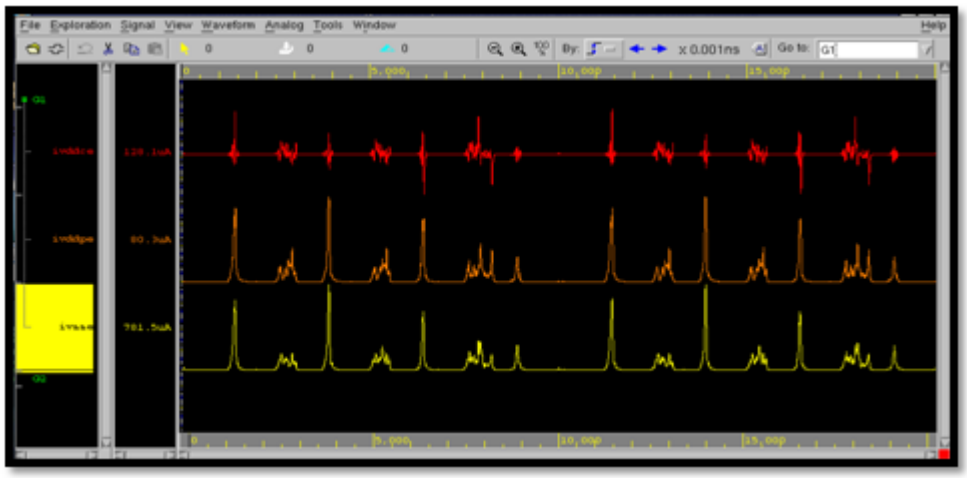
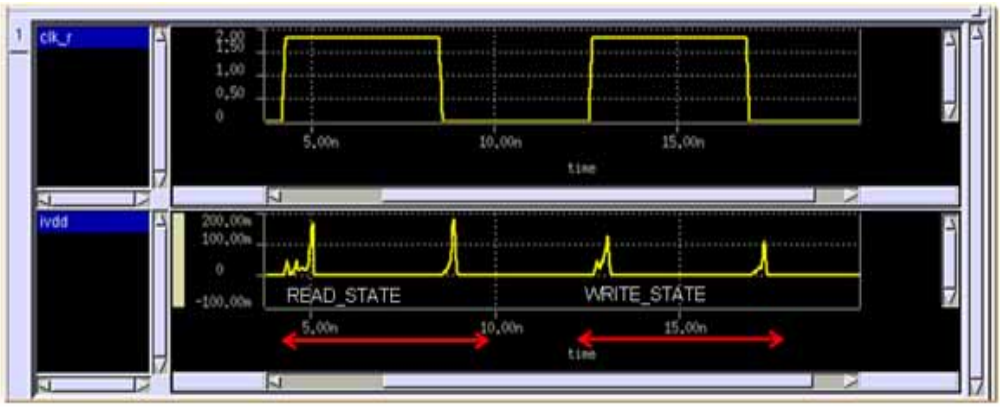


Figure 3-3 Probe current display

- Verify that all child current probes are present and probed correctly in the simulation. (These are the probes created by 'aplmmx -s ').

Picking time frames

To perform Totem analysis on an IP you must make sure that the electrical model (**.spcurrent*) is generated with the correct functional states, which are periods of current demand that can be captured for analysis in Totem. An example of a state capture during the APLMMX stage is shown in Figure 3-4 below:




```
# <state> <trigger_function> <trigger_input> <start time> <end time>
READ_STATE "" "" 4e-9 10e-9
WRITE_STATE "" "" 12e-9 18e-9
```

Figure 3-4 Functional state capture

The MULTI_STATE_FILE or the CUSTOM_STATE_SIM_TIME keywords can be used to specify the time frames to capture the state. An example of the syntax in the APLMMX configuration file is shown below:

```
CUSTOM_STATE_SIM_TIME {
  READ_STATE "" "" 4e-9 10e-9
  WRITE_STATE "" "" 12e-9 18e-9
}
```

The requirements for proper Totem analysis using custom states are as follows:

- The state names should be unique.
- The stop and start difference for each state should be the same.
- The FREQUENCY keyword that is used during analysis in the GSR file should be lower than or equal to (1/stop_time - start_time) setting, so that the instance switching profile can be simulated completely.

The states that you pick during the electrical model generation can be used during Totem analysis using the GSC_FILE keyword in the GSR file.

GDSII Processing: Creating Physical Models

After performing the GDS setup as described in Chapter 2, create a GDS2DB configuration file using the keywords described in the following section. Issues with GDS processing can be resolved using the techniques in the following section "Checking and Debugging GDS2DB Modeling Results", page 3-148.

GDS2DB Configuration File Keywords

There are four required sections in the GDS configuration file:

- GDSII file: specifies a set of GDSII files for converting.
- Top cells definition: the name of the top cell structure(s) in GDSII.
- Nets definition: identifies the power, ground and signal nets that are labeled in text in the GDSII.

- Layer map definition: specifies the map file that contains information for mapping the GDSII layer numbers to the corresponding layers defined in LEF/DEF.

To create the GDS2DB database, you must include two GSR keywords, NATIVE_GDS_SUPPORT and GDSII_FILE:

NATIVE_GDS_SUPPORT

Specifies the type of GDS2DB support to be provided, where

2 (default): provides optimized GDS2DB modeling

-1 : reverts to original GDS2DB modeling behavior

In version 14.1 and later, settings of 0 or 1 are automatically reset to "NATIVE_GDS_SUPPORT 2" and Totem issues a warning message. Default: 2.

Syntax:

```
NATIVE_GDS_SUPPORT [ -1 | 2 ]
```

GDSII_FILE

Specifies the GDS2DB configuration file path and type of view for the specified cell.

Syntax:

```
GDSII_FILE {
    <cell.name> <gds2db_config_file> full ?top?
}
```

where

full: indicates a detailed transistor view

top: is an optional sub-keyword indicating it is the top of the hierarchy

Example:

```
GDSII_FILE {
    ram26-b gds2db.config full top
}
```

After creating the layermap file and the GDS2DB configuration file containing the necessary keywords as described in the following section, the GDS2DB database is created during the Totem run.

There are a number of optional sections in the GDS2DB configuration file, as follows:

- Input LEF
- Geometry extraction
- Mini LVS and device handling
- Selective cell hierarchy handling
- Auto pad location generation
- Automated switch cell handling
- ESD analysis
- Other keywords

GDS configuration file keywords are described in the following sections.

GDSII Files

GDS_FILE

Specifies the GDSII file path, which will be passed down to the automatically generated GDS2DB template configuration file to represent design layout.

Syntax:

```
GDS_FILE <hier_gds2_file_path>
```

GDS_MAP_FILE

Specifies the GDSII mapping file, which is passed down to the automatically generated GDS2DB template configuration file, and maps layer numbers to layer names. Note that to properly support BJT and diode pin snapping to diffusion layer geometry, at least one of the well layers (nwell/pwell) must be present in the file.

Syntax:

```
GDS_MAP_FILE <GDSII_layer_map_file>
```

Top Cell(s) Definitions

DEF_FILE_DEFINITIONS

Defines default resolution units (default 2000) and die size and location. DIE_AREA specifies the opposite die corners, in microns. OFFSET specifies the coordinate offsets, in microns.

Syntax:

```
DEF_FILE_DEFINITIONS {
    UNITS <DEF resolution unit>
    DIEAREA <llx> <lly> <urx> <ury>
    OFFSET <x_off> <y_off>
}
```

FLATTEN_TOP

When set, flattens the GDS top level hierarchy of the design. Default: 0.

Syntax:

```
FLATTEN_TOP [ 0 | 1 ]
```

GENERATE_TOP_LEF

When set, creates top cells LEF in ‘reduced’ and ‘extended’ modes, as defined in the GSR GDSII_FILE <file>. This is useful for stitching a cell properly at the next (parent) level in the hierarchical flow (box_cell flow). Note the following behavior:

- no parent LEFs are created when there is a parent LEF already specified in the GDS configuration file.
- if a parent LEF already exists in the output directory, the file is not overwritten, but instead a new file **_1.lef* is generated.
- If you do not provide GENERATE_PLOC and GENERATE_PLOC_FILTER keyword settings, GDS2DB automatically finds the top-most metal layer per net and dumps those geometries into the LEF file.

Default: 0.

Syntax:

```
GENERATE_TOP_LEF [0 |1]
```

TOP_CELL

TOP_CELLS

TOP_CELL and TOP_CELLS keywords specify the name of the top cell or the list of top cells to be extracted as defined in the GDSII file. Totem supports multiple top cells in GDS2DB to reduce the time needed to read in multiple top cells in large GDSII files.

```
TOP_CELL {
<TopCell_name>
}
```

OR:

```
TOP_CELLS {
<TopCell_name>
...
<TopCell_nameN>
}
```

where

<TopCell_name>: the top cell name to be extracted.

<TopCell_nameN>: additional cell names to be extracted

TOP_CELL_MAP

This keyword is used along with TOP_CELL (which finds the block/cell name in the GDSII data) to specify the names of cells inside the Totem DB, particularly when the macro cell name in LEF (input parent) and cellname in GDSII data do not match. For example, if “cellA1” is the name in GDSII data (TOP_CELL), but you want to call the top cell “cellA2”, this keyword allows it, as follows:

TOP_CELL cellA1

TOP_CELL_MAP cellA2

which generates the design name “cellA2” in the Apache DB. Syntax:

```
TOP_CELL_MAP <desired_top_cell_name>
```

or

```
TOP_CELL_MAP {
        <desired_top_cell_name1>
        ...
        <desired_top_cell_nameN>
    }
```

TOP_CELLS_MAP

If there is a master cell name mismatch between LEF and GDS, use the TOP_CELLS_MAP keyword to rename the master cell in the Totem DB. GDS2DB then extracts geometries from TOP_CELLS and renames them to the mapped name as specified by TOP_CELLS_MAP.

```
TOP_CELLS_MAP {
   <cell_name1> <cell_name1_map>
   <cell_name2> <cell_name2_map>
    ...
}
```

A sample GDS configuration file would look as follows:

```
TOP_CELLS {
   VDD_ST_TF3V3_LIN_50u
}
```

```
LEF_FILE cell.lef
VDD_NETS {
  vdde3v3
}
GND_NETS {
    gnde
}
GDS_FILE design.gds
GDS_MAP_FILE gds.map
TOP_CELLS_MAP {
    VDD_3V3_LIN_50u VDD_3V3_LIN
}
```

Nets Definition

CHECK_TRACING

Enables checking of any two nets that are shorted. The default is 0 (no check for shorts).

```
CHECK_TRACING [0 |1 ]
```

EXTRACT_ALL_LEF_PIN_SIGNALS

When set, Totem GDS2DB preserves the signal names in the output DEF file, instead of renaming the signals “adsN1*”, “adsN2*” ... , which is the default behavior.

```
EXTRACT_ALL_LEF_PIN_SIGNALS [0 | 1]
```

EXTRACT_ISOLATED_SUBDOMAINS

When set to 1, for different geometry islands with the same text labels (nets), sub-domains on that domains are assigned unique names, prefixed with the original net name and new generated suffixes. In the example case below, block pins x1, x2 and x3 from XTOR_MAP_FILE originally are on net “vdd”. After sub-domain creation and name assignment, pin snapping is also modified so that x1 and x3 are snapped to “vdd”, whereas x2 is snapped to “vdd_gdsNet1”. See Figure 3-6.

When set to 2, different isolated net islands with same text labels (nets), gets a unique DEF net name. A typical scenario where this is applicable is for handling stacked diode or clamps in case of ESD analysis, where the intermediate nets between the stacked devices can have same net-name but it needs to be uniquified for ensuring correct modeling and analysis by downstream engines like ESD analysis, Chip ESD Model generation.See Figure 3-7 for a sample scenario.

```
EXTRACT_ISOLATED_SUBDOMAINS [0 | 1 | 2]
```

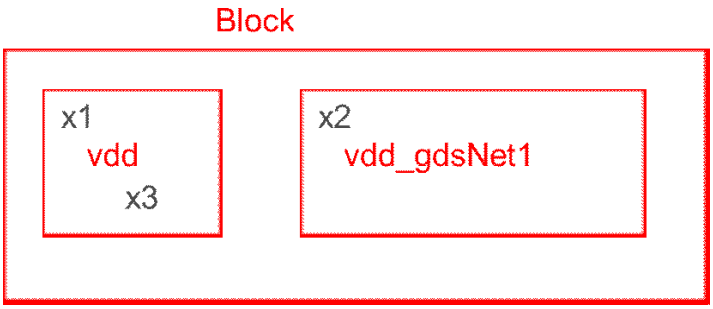


Figure 3-5 Geo island naming

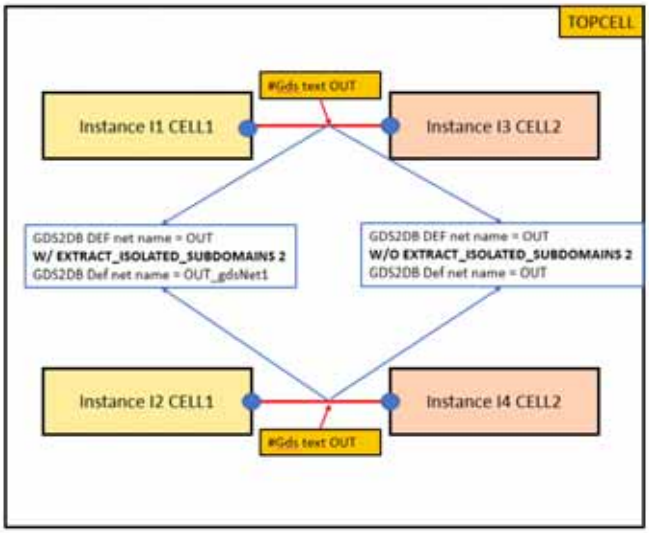


Figure 3-6 Isolated net island naming

IGNORE_CORNER_TOUCH

When set, decomposes non-rectangular shapes of layers that participate in Boolean operations into rectangles. The default value is 0, which retains shapes that touch each other only at corner points. When set to 1, shapes that touch each other only at corner points are discarded.

```
IGNORE_CORNER_TOUCH [ 1 | 0 ]
```

IGNORE_LAYER_TYPE

Specifies the layer types in GDS layer map to be ignored during GDS2B modeling. Layer types include specific devices, such as metal-resistors, device-markers (ESD, Diode, BJT), mfill layers, etc.

Syntax:

```
IGNORE_LAYER_TYPE {
<Layer_type1>
<Layer_type2>
…
}
```

Example:

```
IGNORE_LAYER_TYPE {
    metalres
    esd
   }
```

IGNORE_LEF_PG_PIN_GEOMETRY

When set, GDS2DB does not merge LEF P/G pins and GDSII in the generated DEF and preserves them independently. Based on the GDS2DB command, results are as follows:

- when GDSII_FILE <mode> is set to “reduced”

  The LEF P/G pins go into generated LEF, irrespective of any keyword.

- when GDSII_FILE <mode> is set to “extended”

  The LEF P/G pins do not go into generated LEF by default. If 'IGNORE_LEF_PG_PIN_GEOMETRY 1' is set, the LEF P/G pins go into LEF.

For the DEF PINS section, when IGNORE_LEF_PG_PIN_GEOMETRY is set to 1, no input LEF P/G pin geometries go into the DEF PINS section; when the keyword is set to 0, they do.

IGNORE_PSL_MAP

Specifies device types for which pin snapping layer creation should be turned off in gdsmmx. This will prevent incorrect pin snapping and current crowding in Totem.

Syntax:

```
IGNORE_PSL_MAP <deviceType/modelName>
```

Device types supported are "M" "B" "D" "R" "L" and "C".

Example:

```
IGNORE_PSL_MAP R pmos
```

Here, there will be no PSL behind Resistor type and pmos model in `<design>.gdsmmx` file.

IGNORE_RES_SHORTS_BW_NET_GROUPS

When specified, GDS2DB will trace through metal resistors (instead of creating OPEN) for the defined nets.

Syntax:

```
IGNORE_RES_SHORTS_BW_NET_GROUPS {
<net1>
<net2>
….
}
```

Also note that the metal resistors should be defined in the layer map.

NET_NAME_CASE_SENSITIVE

Setting NET_NAME_CASE_SENSITIVE to ‘no’/’0’/’false’ turns the search of text layer case insensitive. By default, text label name searching is case sensitive.

NET_NAME_CASE_SENSITIVE { 1 | no | ’0 | false]

PRIORITY_NETS

When specified, GDS2DB preserves the defined net names in generated DEF file for nets having multiple nets names.

Syntax:

```
PRIORITY_NETS {

<net_name1>
```

```
<net_name2>
….
}
```

Shorts between nets listed in this keyword are reported in adsRpt/GDS/short_file.txt

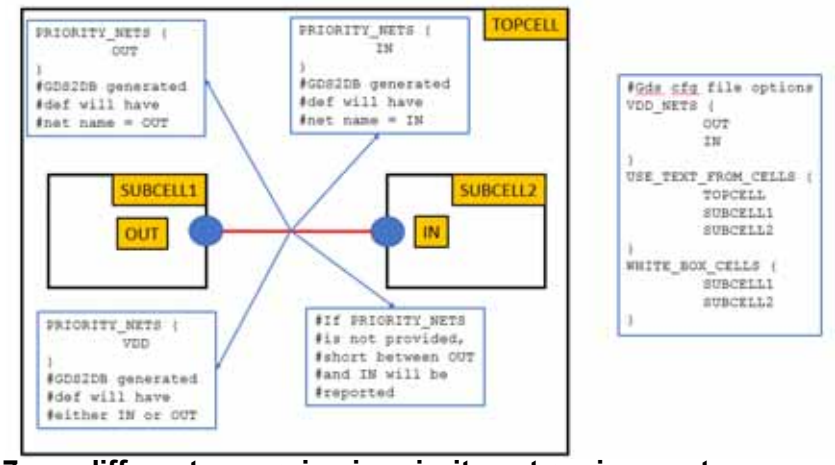


Figure 3-7 different scenarios in priority net assignment.

SIGNAL_NETS

Specifies signal net mapping and tracing can be specified with different types of syntax. Wildcards are supported for net names

Syntax:

```
SIGNAL_NETS {
    <DEF_signal_net_name-1>
{
    <signal net-a>
    <signal net-b>
    ...
}
...
{
<DEF_signal_net_name-N>
{
    <signal net-Na>
...
*<cell_name> @ <layer_name>
<NetName> <cell_name> @ <layer_name>
...
}
}
```

where

<DEF_signal_net_name-N> : net name in DEF for group of signal nets

<signal net-a> : signal net name in GDS or Spice

*<cell_name> @ <layer_name> : specifies all isolated geometries/domains traced using geometries from the specified GDS layer inside cell_name, create nets with internal names such as cell_name_Net1, cell_name_Net2.

<NetName> <cell_name> @ <layer_name>: specifies all isolated geometries/ domains traced using geometries from the specified GDS layer inside cell_name, create nets with names prefixed with NetName such as NetName1, NetName2.

To trace and extract all signal nets from their corresponding I/O pins in the selected region, use the keyword:

```
EXTRACT_ALL_LEF_PIN_SIGNALS 1
```

START_POINT_TOLERANCE

When location-based tracing is used, allows control over the GDS2DEF algorithm that uses DSPF x,y locations to trace and extract signal nets from the GDSII file in the signal EM flow. The default behavior of the algorithm attempts to automatically correct any imprecision between input data files, and optimizes the extraction of nets for most designs. But occasionally some x,y locations cannot be mapped precisely, which leads to shorts or missed traces, and errors later in the analysis. Under these conditions, this keyword gives you control of the mapping error correction, using the specified fixed radius to snap DSPF x,y locations to the next polygon found. Specifies the maximum distance in microns from the starting point specified in the GDS configuration file that can to be used to start tracing. The unit for the radius is the design unit from the tech file. *Optional. Default: None*

Syntax:
START_POINT_TOLERANCE <search_radius>

Example:
START_POINT_TOLERANCE 0.05

USE_TEXT_FROM_CELLS

Describes cells from which the text labels are taken for net tracing. USE_TOP_LEVEL_TEXT_ONLY must be set to 0. The syntax is:

```
USE_TEXT_FROM_CELLS {
<cell_1>
<cell_2>
...
}
```

Example:

```
USE_TEXT_FROM_CELLS {
    cellABC
    cellXY
    Top_ab
}
```

USE_TEXT_FROM_HIERARCHY_BLOCK

Allows specifying GDS hierarchy for the text search feature (instead of just top level or all levels). Net tracing and PLOC creation is performed to the specified hierarchy level. USE_TOP_LEVEL_TEXT_ONLY must be set to 0. The syntax is:

```
USE_TEXT_FROM_HIERARCHY_BLOCK {
    <Hierarchy_name1>
    <Hierarchy_name2>
    ...
}
```

Examples:

```
USE_TEXT_FROM_HIERARCHY_BLOCK {
    TOP/A
    TOP/A/B
    TOP/A/B/C
    TOP
}
```

In this example cell B is inside cell A and A is inside the TOP cell.

USE_TEXT_FROM_HIERARCHY_LEVEL

Describes hierarchy levels from which the text labels should be taken for net tracing. USE_TOP_LEVEL_TEXT_ONLY must be set to 0. The syntax is:

```
USE_TEXT_FROM_HIERARCHY_LEVEL <hierarchy_level_description>
```

Examples:

```
USE_TEXT_FROM_HIERARCHY_LEVEL 0 2 5 ... # 0 is top-level
USE_TEXT_FROM_HIERARCHY_LEVEL 2-5
USE_TEXT_FROM_HIERARCHY_LEVEL 2-MAX
USE_TEXT_FROM_HIERARCHY_LEVEL 2 - MAX
USE_TEXT_FROM_HIERARCHY_LEVEL 0-5
```

USE_TOP_LEVEL_TEXT_ONLY

When set to 1, only the text in the top cell are extracted and used for power/ground net tracing purposes. The syntax is:

```
USE_TOP_LEVEL_TEXT_ONLY [0|1]
```

Names must be defined in VDD_NETS and GND_NETS. The default is 0.

VDD_NETS/GND_NETS

Specifies lists of Vdd /Vss nets for power/ground net geometry extraction. You can use wild cards, such as “VDD*” and “VSS*”, in specifying net name groups. Also, VDD and GND nets can be retrieved by specifying bit select and part select patterns, such as in the following pattern examples: VDD(1), VDD(1:2), VDD(2), VDD(3), VDD(4), VDD[1:2], VSS*[2], VSS:G, VSS[1], VSS[2], VSS[3], VSS[4], VSS[3:4].

Syntax:

```
GND_NETS
{
<ground_net_name_in_output_DEF> {
    <ground_net_name> @ <gds_layer_number>?:<data_type>?
    <gds_x_position>  <gds_y_position>
...
    <ground_pin_name_in_DEF>
    <ground_net_name_in_GDS>
...
}
*<cell_name> @ <layer_name>
<NetName> <cell_name> @ <layer_name>
}

VDD_NETS
    {
```

```
<power_net_name_in_output_DEF> {
    <power_net_name> @ <gds_layer_number>?:<data_type>?
  <gds_x_position>  <gds_y_position>
(...)
<power_pin_name_in_DEF>
<power_net_name_in_GDS>
(...)
}
*<cell_name> @ <layer_name>
<NetName> <cell_name> @ <layer_name>
}
```

where

<power/ground_net_name_in_output_DEF> : specifies net names in DEF for groups of power nets. Net naming can use wild cards, such as “VDD*” for “VDD3” and “VDD:”. There is also support for bit select definitions, such as “VDD(0)” or “VDD[0]”, and part select, such as “VDD[1:3]”, in Net Names.

@ <gds_layer_num> <gds_x_position> <gds_y_position> : specifies the GDS layer number, and optionally data type, and x,y position. The coordinate format is just to provide a tracing starting point; the name label is not used for text label searching.

<power/ground_pin_name_in_DEF> : specifies power pin name in DEF

<power/ground_net_name_in_GDS> : specifies power net name in GDS or Spice

*<cell_name> @ <layer_name> : specifies all isolated geometries/domains traced using geometries from the specified GDS layer inside cell_name, create nets with internal names such as cell_name_Net1, cell_name_Net2

<NetName> <cell_name> @ <layer_name>: specifies all isolated geometries/ domains traced using geometries from the specified GDS layer inside cell_name, create nets with names prefixed with NetName such as NetName1, NetName2

The GDS2DB conversion utility allows multiple power net tracing. It searches the GDSII file for nets that are labeled with the same names as in the power net section, trace the geometries that belong to the net and generate the *.def* description with those geometries.

The power net and ground net can be specified by simply using a net name also. For example, if a power net only has one entry and the power net name to be used in output DEF is the same as the net name in the entry, then a simplified format can be used, as shown below:

```
VDD_NETS {
    <power net1>
    <power net2>
...
}
```

An example power net section is shown below:

```
VDD_NETS {
    VDD {
    VDD
    VDD1_core
    VDD2_core
    VDD3_core
```

```
}
}
```

The power net in the output *.def* has the given name in the power net section. In this example, VDD, VDD1_core, VDD2_core, and VDD3_core is included under VDD in the generated *.def* file. Note that the power net names are case sensitive. Wildcards are supported for net names under VDD_NETS and GND_NETS keywords.

Layer Map Definition

DATATYPE

Sub-layers can be defined optionally using the following keyword:

```
DATATYPE {
    <valid sub-layer numbers>
}
```

GDS_MAP_FILE

Specifies the name of the GDS layer map file. Specify DESC if layers are listed in descending order. Required.

```
GDS_MAP_FILE <layer_map_file> ? DESC ?
```

MOM_SNAP_THRESHOLD_DISTANCE

If MOM cap marker layers are provided in the GDS layer map file for GDS2DB, all geometries overlapping MOM markers of cells containing the MOM marker layer are ignored, to avoid double counting the parasitic coupling effect inside MOM devices. If the transistor location in the APLMMX file does not overlap a MOM marker layer, a threshold value can be specified to help identify which MOM cell it belongs to, by using the following GDS2DB configuration file keyword syntax (optional, no default):

```
MOM_SNAP_THRESHOLD_DISTANCE <threshold_um>
```

NEW_MOM_HANDLING

When set, improves MOM device handling. Default 0 (off). The syntax is:

```
NEW_MOM_HANDLING [0 | 1]
```

Following are the key steps in using the keyword:

1. You can specify layer-specific markers instead of a global marker layer for MOM cells, such as:

   ```
   m1MOMmark     MOM     155:1   M1
   m2MOMmark     MOM     155:2   M2
      …
   ```

   MOM-finger marker layer removal is on a layer basis.

2. MOM area pins are created on all left-over wires. This provides better current distribution to avoid false EM violations, as well as improving pin snapping/creation
3. With no PSL definition, pins are created on all available layers inside MOM-cells based on each individual MOM instance pin connection situation.
4. MOM pins are snapped to nearest metal geometry if the pin locations from the DSPF netlist are not falling on the terminals of the MOM cell and lie inside the MOM marker layer.

   Note. If pins are desired only on specific layers, use DEVICE_MODEL_CONFIG in the aplmmx configuration file to define the pin and corresponding layer for accurately modeling the MOM devices.

The following are optional sections of the GDS2DB configuration file.

Input LEF

GENERATE_LEF_DEF

After reading in the GDSII data natively inside Totem/GDS2DB, you can also generate the LEF/DEF files and PRATIO values using this keyword to specify the directory. Default : none.

Syntax:

```
GENERATE_LEF_DEF <lef_def_out_dir>
```

GENERATE_LEF_DEF_ONLY

When set, Totem/GDS2DB only generates LEF/DEF in the output directory specified with GENERATE_LEF_DEF keyword, and then quits.

```
GENERATE_LEF_DEF_ONLY [ 0 | 1 ]
```

GENERATE_SEPARATE_PG_PINS_LEF

Allows static GDS2DB to keep the original LEF by setting this keyword to 1, which creates another LEF macro with power/ground nets and their geometries only, and creates an instance of this macro in the generated top cell DEF file. The default is off (0).

```
GENERATE_SEPARATE_PG_PINS_LEF [0|1]
```

IGNORE_LEF_FOREIGN_OFFSET

Many designs that consider the LEF ‘FOREIGN offset keyword in the original LEF file generate GDS2DB output DEF with misplaced geometric elements, which results in shorts in the Totem run. The keyword ‘IGNORE_LEF_FOREIGN_OFFSET’ can be used to ignore the offset values. The default is 0 or off, which means the FOREIGN offset value is used when GDS2DB writes out DEF geometry. If ‘IGNORE_LEF_FOREIGN_OFFSET’ is set to 1, GDS2DB sets both x and y offsets to 0.

```
IGNORE_LEF_FOREIGN_OFFSET [0 | 1 ]
```

Currently, there is no guideline to help you decide whether or not set this keyword. It depends on how the LEF file is prepared.

Note: Even when this keyword is set to 1, GDS2DB still uses the LEF foreign name for GDS structure name mapping.

IGNORE_LEF_ORIGIN_FOR_START_PT

When set to '1', the ORIGIN from the start points imported from LEF file for net tracing are ignored. This becomes active when USE_LEF_PINS_FOR_TRACING is set. Default is 0.

```
IGNORE_LEF_ORIGIN_FOR_START_PT [0 | 1]
```

LEF_FILE

Specifies import of existing top-level or sub-block LEF file for hierarchical block/cell extraction (optional). You must specify the original LEF file in the GDS2DB configuration file if the physical model is to be used in the top-level VCD mode dynamic analysis.

If the *.lef* files for GDSII are specified, the GDS2DB utility parses these files and search for the top cell macro information. If the FOREIGN keyword is specified for the macro, the GDS2DB utility converts the structure whose name is the same as macro foreign name, instead of the specified top cell name.

For the most accurate approach to obtaining the die area, the GDS2DB utility takes DIEAREA specified by the SIZE of the MACRO keywords in the *.lef* file. If the *.lef* is not available, then the GDS2DB utility estimates the area based on the bounding box of all geometries in the GDSII file.

Syntax:

```
LEF_FILE {
    <LEF file name>
    ...
    }
```

LEF_FOREIGN_OFFSET_HANDLE

When set to 1, shifts the input LEF pin-geometries of the top-cell such that it matches the GDS of the top-cell. Default is 1.

```
LEF_FOREIGN_OFFSET_HANDLE [0 | 1] …
```

LEF_PIN_POWER

Enables current to be distributed to vias connected to the LEF pins. When LEF_PIN_POWER is On, GDS2DB traces only vias under power pin geometry in the input LEF file and outputs the corresponding pins under those vias in the output LEF file. Default: 0 (off)

```
LEF_PIN_POWER  [0|1]
```

RETAIN_PRBOUNDARY_FOR_TOP_BBOX

When set, uses prboundary for the top BBOX, rather than using LEF or other GDS geometries (default: 0, off). The syntax is:

```
RETAIN_PRBOUNDARY_FOR_TOP_BBOX [0 | 1]
```

USE_LEF_PINS_FOR_TRACING

Where LEF pins defined as VDD_NETS and GND_NETS are used for tracing, uses geometries defined in the PINS section of the LEF file of the block being extracted to trace its power and ground nets during GDS translation. No text or label based extraction is performed. However, user-specified coordinates are honored. Default: 0 (off, no LEF pin based pin tracing).

```
USE_LEF_PINS_FOR_TRACING [0 | 1]
```

USE_LEF_FOR_TOP_BBOX

By default (1), forces extraction to pick up the cell bounding box from the LEF file. The syntax is:

```
USE_LEF_FOR_TOP_BBOX [0|1]
```

Geometry Extraction

ADJUST_POLYGON

If set, removes any residual polygons with a dimension less than 0.01 microns that can occur from conversion of non-rectangular areas. It merges adjacent thin metal strips to create wide metal, which avoids multiple thin metal strips in the database and thus potentially inaccurate EM results. This function is supported only in the NGS2 flow. I

Syntax:

```
ADJUST_POLYGON [0 |1]
```

APPLY_ESL_FOR_SIGNAL_NET

When set to 0, GDS2DB exclude the signal nets from global EXTRACTION_STARTING_LAYER definition. *Default: 1*

Syntax:

```
APPLY_ESL_FOR_SIGNAL_NET [0 | 1]
```

DOUBLE_SIDED_MCAP

When set, traces MIM caps whose terminals are connected to adjacent metal layers in GDS. An example structure for a MiMCap with a double side connection is shown in Figure 3-8

The GDS layer map file should be as shown below:

```
    #<MIM cap layer> c <layer number> <text layer number> <via name
for the via connecting mim cap layer specified in first column>
    MiM_T c 50:1 – vm1
    MiM_B c 50:0 – vm2
```

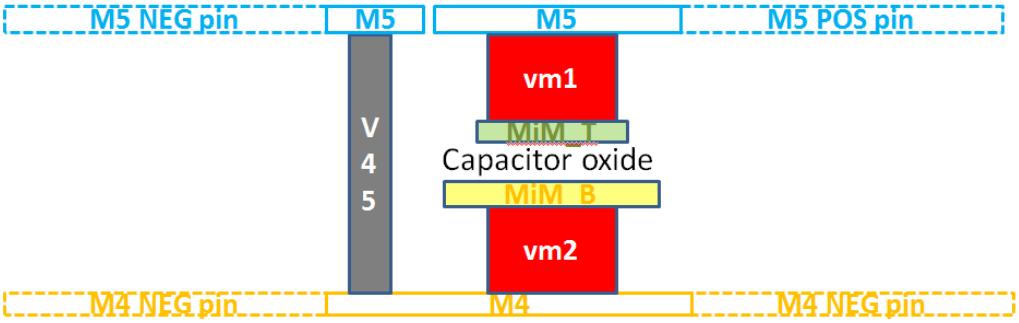


Figure 3-8 MiM Cap with a double side connection

Syntax:

```
DOUBLE_SIDED_MCAP [0 |1]
```

CORE_EXTRACTION_STARTING_LAYER

Specifies the metal layer name for starting power/ground net extraction for the memory_bit_cell core region. By default the metal layer used is two layers higher than the metal layer specified in EXTRACTION_STARTING_LAYER. Optional. Default: Extraction_Starting_Layer + 2.

Syntax:

```
CORE_EXTRACTION_STARTING_LAYER <layer_name>
```

ENABLE_POLY_VIA_MERGE

When set, large circular vias are merged into a single via, rather than split into segments when set to 0. Default: Off (0).

Syntax:

```
ENABLE_POLY_VIA_MERGE [ 0 | 1 ]
```

EXTRACTION_STARTING_LAYER

Sets the lowest starting layer for extraction and tracing. Then GDS2DB does not include excluded layers from the parent LEF, which are ignored.To define the starting metal layer name for power/ground net extraction in the GDSII file, use the following specification:

```
EXTRACTION_STARTING_LAYER <lowest_metal_layer> ? Traceall ?
```

The default is to trace and extract all the layers specified in the layermap file. If the option ‘Traceall’ is also specified, GDS2DB traces all layers, but only extracts starting from the lowest layer specified. Both ‘GDS2DB’ and ‘GDS2DB -m’ utilities honor this specification. Also, sets the switch pin creation layer internally and snaps all type{sw} transistors in switch instances instead of in the top cell.

This keyword also can selectively extract lower layers only for ESD cells that are defined using GBC or marker layers, and restrict the rest of the design to a higher ESL. This improves the overall runtime for ESD analysis, without much impact on the accuracy of ESD results. You can go down to base-layers inside ESD devices and start at a much higher metal layer (say M4) outside the ESD devices for improved GDS translation and ESD simulation performance. A example configuration file syntax for this is:

```
# Relevant settings in gds2db configuration file
...
# ESL for entire design
EXTRACTION_STARTING_LAYER <higher_metal_layer>
CONTACT_RESISTANCE 1
...
...
ESD_CELLS {
    ...
#ESL for ESD Cells
    EXTRACTION_STARTING_LAYER ALL
    ...
}
BOX_CELLS_PROPERTIES {
    ...
    PIN_CREATION_LAYERS POD NOD
    ...
}
```

EXTRACT_METAL_FILLER

Metal fills are segments of metal added due to DRC rules to increase the metal density. These metals are floating and not connected to any power domain. Although as conductors they are meaningless, they are very important in calculating the metal density. Metal density in turn is used in calculating metal strip resistance. If this keyword is turned on, GDS2DB extracts all floating net geometries in the GDS database in each layer to get the correct metal density calculation. The floating nets preserved in the database are only used for cross-cap extraction and reduction. They are not analyzed for IR drop or EM. Default: Off (0).

```
EXTRACT_METAL_FILLER [0 |1 ]
```

An example of the output in the DEF file is as follows:

```
- MFILL
+ ROUTED M1 10400 ( 501120 -10 )  ( * 88250 )
  NEW M1 460 ( 672290 800050 )  ( 675670 * )
  NEW M1 1000 ( 687220 829780 )  ( 697940 * )
  NEW M1 240 ( 701920 786750 )  ( 705010 * )
```

HANDLE_PATH_45DEGREE

When set, correctly handles 45-degree geometries using trapezoids, rather than creating a staircase-like rectangular structure (applicable only for 'NATIVE_GDS_SUPPORT 2' GSR setting). Default: 0.

Syntax:

```
HANDLE_PATH_45DEGREE [ 0 | 1 ]
```

IMPORT_REGIONS

IMPORT_REGIONS is used to specify regions in the top cell for extraction, using corner coordinates.

Syntax:

```
IMPORT_REGIONS {
    <low_left_x> <low_left_y> <up_right_x> <up_right_y>
    ...
}
```

MEMORY_BIT_CELL

Specifies the name of the memory bit cell in the core array. Optional.

Syntax:

```
MEMORY_BIT_CELL [auto_detect | OFF |
    <memory_bit_cell_name> ]
```

MEMORY_CELL

Specifies the name of the memory cell sub-circuit. Optional.

Syntax:

```
MEMORY_CELL [ auto_detect | off | <memory_cell_name> ]
```

MEMORY_CORE_REGIONS

Specifies a file that defines the boundaries for memory core regions, such as when you want to remove the assignment of current from a user-specified area. For example, this option can be used to eliminate the pratio current assignment in areas of a macro specified in the core regions filename. The rectangular regions are specified using lower-left upper-right coordinates, one coordinate set per line, as follows:

```
3.085 219.34 40.1 219.67
3.085 185.56 40.1 186.05
58.8 219.34 96.36 219.67
58.8 185.56 96.36 186.05
```

The effect of this option is to zero out the pin pratio data within the rectangles specified, and re-assign them to pratio values outside of the areas. The number of created pins may be modified from previous GDS2DEF outputs, so the GDS2DEF total pin numbers may change when compared to previous version runs.

Syntax:

```
MEMORY_CORE_REGIONS <filename>
```

MERGE_WIRE

Merges geometries (On by default).

Syntax:

```
MERGE_WIRE [ 0 | 1 ]
```

MERGE_WIRE_NETWISE

After net tracing, when set, merges all wire geometries by net. This keyword is 1 (On) by default for gds2db -m design flows, and 0 by default for other flows.

Syntax:

```
MERGE_WIRE_NETWISE [ 0 | 1 ]
```

NEW_IMPORT_REGION

When set, allows importing only a specified region of a top cell. *Optional. Default: 0.*

Syntax

```
NEW_IMPORT_REGION [0 | 1 ]
```

NO_METAL_IN_VIA_MODEL

When turned on, generated via models have a minimum size (1x1 nm) on top and bottom metal layers. By default (0), via models generated are the same size as cut rectangle geometries of top and bottom metal layers. The syntax is:

Syntax:

```
NO_METAL_IN_VIA_MODEL [ 0 | 1 ]
```

OUTPUT_VIA_ARRAY

Setting this keyword allows grouping adjacent vias so that you can select an entire group of arrays with a single click.

```
OUTPUT_VIA_ARRAY [0 | 1 ]
```

SLOT_VIA_LAYERS

To slice long slot vias into single via based on slotting distance mentioned. Slot vias described as rectangular and non-rectangular or paths in GDS are supported. In the GDS layer map files, <Layer_type> for these vias should be mentioned separately as “slot_via”. Slot vias can be specified separately from main metal/via stack in layermap. GDS2DB can handle designs with regular and slot vias in the same height and with same or different Tech layer name (only the GDS layer/datatype will be different).

Syntax

```
SLOT_VIA_LAYERS {
 #<via_layer> <slotting_distance_in_um>
 CAslot 1
 via1 1
}

#Sample gds layer map file
-------------------------------------
# slot via layers
#<via_layer> slot_via <temp_layer (or) layer:datatype> -
CAslot slot_via CAtemp1 -
V1 slot_via 15:0 –
```

VIA_NAME_PREFIX

Normal via naming creates very long names for via models, resulting in very large output DEF files. When 'VIA_NAME_PREFIX' is set to 0, the typical “GDSVia_<topcell>” via name prefix is removed in order to shorten it, and reduce file size. The syntax is:

```
VIA_NAME_PREFIX [ 0 | 1 ]
```

For example, with VIA_NAME_PREFIX set to 1 (default), a via model name could be *GDSVia_logic_die_iosso_0_M7_RV_AP_0.* With VIA_NAME_PREFIX set to 0, the via model name would be *0_M7_RV_AP_0.*

Mini LVS and Device Handling

10NM_SUPPORT

Provides support for 10nm technology designs (default 0). Automatically set to 1 when a TSMC 10nm design is detected.

Syntax:

```
10NM_SUPPORT [0|1]
```

ANNOTATED_AREA_PIN_CREATION

When set, creates area pins in the overlapping area of the pin snapping layer (PSL), and its upper layer if the PSL is a device layer. This avoids false EM violations on contacts due to point pins on device layers. Default is 0 - off.

```
ANNOTATED_AREA_PIN_CREATION [0|1]
```

ANNOTATED_GDS_FLOW

ANNOTATED_GDS_MAP_FILE

ANNOTATED_GDS_KEY

These three keywords support the annotated GDS2DB flow. Annotated GDS has a key and a net ID associated with every geometry object in the GDS, so GDS2DB can bypass GDS tracing to speed up runtime. ANNOTATED_GDS_FLOW turns on the annotated GDS capability. Use the keyword ANNOTATED_GDS_MAP_FILE to specify the net mapping file that maps net ID in GDS with net names. The ANNOTATED_GDS_KEY keyword specifies the key value to find the net ID in the GDS map file. The keyword syntax is :

```
ANNOTATED_GDS_FLOW [0 |1 |2 ]
ANNOTATED_GDS_MAP_FILE <GDS map file>
ANNOTATED_GDS_KEY <key value>
```

To support a hierarchical annotated GDS flow, set ANNOTATED_GDS_FLOW 2, and provide the following inputs, as generated by the Caltrans utility:

- Hierarchical annotated GDS file
- Hierarchical Annotated GDS Map File (*.map) defined in the ANNOTATED_GDS_MAP_FILE keyword .
- Schematic Spice Netlist File (*.*schm.sp*) defined in LAYOUT_NETLIST keyword

APACHE_PHYSICAL_MODEL

When set, reduces the node count for extraction and provides significant speed-up in network processing, with a minimal reduction in accuracy. To reduce unneeded geometries, below the specified APM layer only those geometries that are necessary to

keep connectivity between the pins on the APM layer are retained. All geometries (wires/vias) are retained on or above the specified APM metal layer.

For current sources/sinks:

- pins are created on the specified APM metal layer.
- for the transistors in transistor map file, the snapping locations (at device/metal1 layer) are found and stored temporarily. Then, based on the connectivity from APM layer geometries to the original snapping locations, pins are created at APM layer, and pratios from transistors are assigned to the pins. The keyword 'DUMP_APM_STATS 1' can be used, which generates a file '*adsRpt/GDS/apm_pin_map_report.txt* that provides information on which transistor is mapped to which pin at the APM layer.
- the bounding-box for a group of vias above the APM layer are formed, based on the intersection of wires at APM layer and the metal-layer above APM layer. Pins are created using this bounding-box, instead of creating a single pin below each via.
- in memory core regions, pins are created with a 0 pratio value at the core-extraction-starting-layer. Core-extraction-starting layer should be the same or above APM layer.

For ESD analysis, APM supports expanded ESD clamp cell modeling by extracting detailed views inside clamps and a higher level of abstraction outside the ESD cells. If APACHE_PHYSICAL_MODEL is set and clamp-cells are auto-detected (using the ESD marker layer in the layermap), APM ESD flow is automatically invoked.

- for P/G nets, only geometries (wires/vias) on or above the specified metal layer are retained outside clamp instances, and all geometries are retained inside clamp instances. Also, no current sources/sinks are created outside clamp-instances, and current sources/sinks are created only inside clamp-instances.
- for signal nets, there is no change in modeling geometries and current sources/sinks.

This provides faster resistance and current density checks, without compromising accuracy.

If there are switch-cells in the design, during reduction of the wire-via network below APM layer, Totem retains the wires/vias that are necessary to maintain connectivity to switch-instance pins, to ensure that switches are not disconnected.

The CMM cell_view generated using APACHE_PHYSICAL_MODEL supports multiple states by capturing state-dependent pratios. The pratios are available in the file RawModelSP_CMM_state.pratio in the following format:

```
#index state1 state2
40 10609904 10595700
41 10837962 10824250
42 0 0
```

Default: none. *For GDS2DB only.*

Syntax:

```
APACHE_PHYSICAL_MODEL <APM_metal_layer>
```

AUTOFIT_DIFFUSION_WITH_POLY

When set, shrinks diffusion to poly height for advanced technology nodes where diffusion can be extending beyond poly-cut height.

Syntax:

```
AUTOFIT_DIFFUSION_WITH_POLY [ 0 | 1]
```

BJT_MARKER_LAYER_BBOX_TOLERANCE

When set, increases the size of the BJT marker layer geometry by the specified tolerance when it is smaller than the BJT device.

Syntax:

```
BJT_MARKER_LAYER_BBOX_TOLERANCE <tolerance_um>
```

COMPLETE_GATE_PIN_SNAPPING

When set, GDS2DB looks for directly overlapping polyGates of the MOS device and snaps them to the center of the poly geometry. If set to 0, transistor pins are snapped to the nearest poly geometry. Default: 1.

Syntax:

```
COMPLETE_GATE_PIN_SNAPPING [ 0 | 1 ]
```

CONTACT_RESISTANCE

After defining the diffusion, poly, and contact layers in the GDS map file, turn on this keyword to extract contact resistance for MMX analysis. Default: 0.

Syntax:

```
CONTACT_RESISTANCE [ 0 | 1 ]
```

where

0: pins are created on M1, at the same location as the real contacts of the same net. (tap contacts are NOT used)

1: pins are created on the overlapping diffusion layer, at the same location as the real contacts of the same net (tap contacts are NOT used). If “CONTACT_RESISTANCE 1" is specified, and ‘PIN_CREATION_LAYERS’ is not specified, then the creation of pins is on diffusion, at the same location as that of the contacts (tap contacts are not considered).

DEVICE_SNAP_THRESHOLD_DISTANCE

Defines separate snap threshold distance for different device types in the design. *Default: None.*

Syntax:

```
DEVICE_SNAP_THRESHOLD_DISTANCE {
    <device_type1> <distance in um>
    <device_type2> <distance in um>
    …
    …
    ALL <distance in um>
}
```

Example:

```
DEVICE_SNAP_THRESHOLD_DISTANCE {
MULTI_FINGER_MOS 1.5
LDMOS 2.5
RESISTOR 0
INDUCTOR 3
METAL_ON_METAL 1
BJT 0
DIODE 2
MOS 1
```

```
    ALL 0
    }
```

Where,

ALL is equivalent to config keyword "SNAP_THRESHOLD_DISTANCE", which sets snap threshold distance for all devices that are not explicitly specified in the block.

DISABLE_MINILVS

If you cannot provide all device layers, such as p-well, n-well, p-diffusion, n-diffusion, poly and contact layer, in the GDS layer map file, this keyword must be set to 1 to disable the LVS features, such as tap contact recognition, device contact array extraction and modeling. Otherwise, GDS2DB aborts with an error. Layers below M1 are ignored when this keyword is set, thus improving run-time and memory footprint. However, statistics information pertaining to the layers below M1 then are *not* present in the file *<run-dir>/adsRpt/GDS/gdsii.info.*

Note that this keyword is automatically set to 1 when CONTACT RESISTANCE is disabled in the Virtuoso cross-probing “Layout based Static Analysis” setup. Default: Off (0).

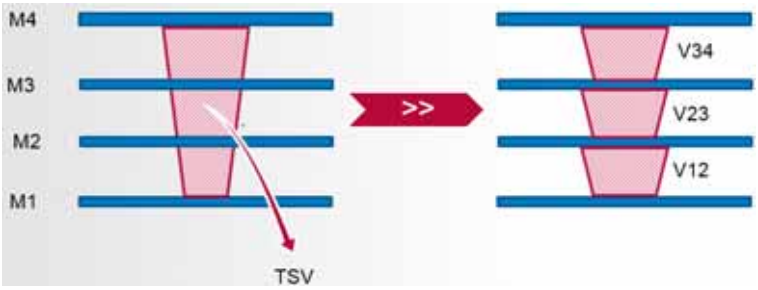
Syntax:

```
DISABLE_MINILVS [ 0 | 1 ]
```

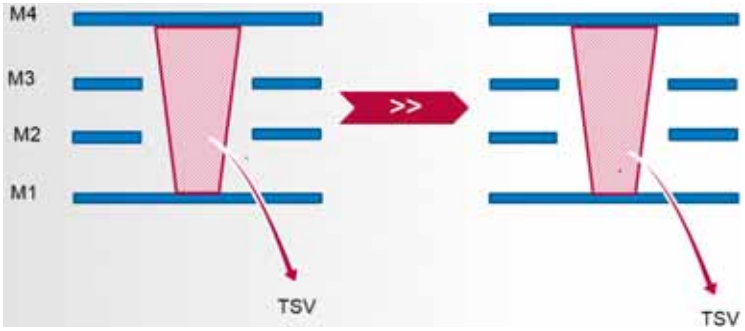
ENABLE_NATIVE_THROUGH_VIAS

When enabled, allows handling a mix of through-vias and stack-vias in the same design, depending on how the through-vias are used, as shown in the case diagrams following.

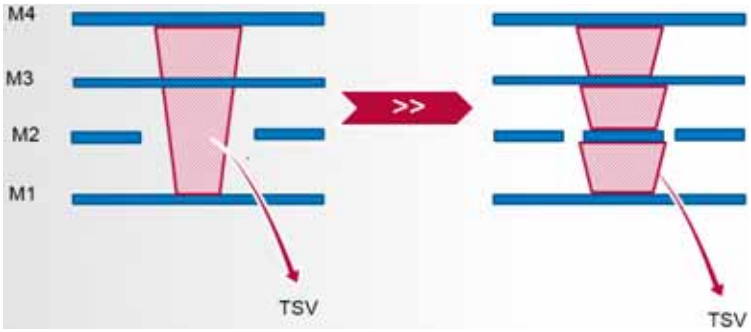
Case-1: Through-via (TSV) is between M1 and M4. M2 and M3 metals are overlapping the TSV layer. The keyword setting provides cuts through-vias as stack-vias.



Case-2: When M2 and M3 metals are not overlapping the TSV layer. This keyword setting retains the TSV as it is.



Case-3: When only M3 metal is overlapping the TSV layer, this keyword setting cuts through-vias as stack-vias and places dummy M2 metal between via2 and via3.



Syntax:

```
ENABLE_NATIVE_THROUGH_VIAS [ 0 | 1 ]( default : 0 )
```

ENHANCED_PIN_GROUPING

When set, ENHANCED_PIN_GROUPING improves TYPE{M} pin snapping and pin grouping based on the overlapping metal1 and diffusion geometries. For a non-co-vertical flow, the XTOR pin snapping for a MOS device is based on the contacts overlapping diffusion of the device. The pin is created after grouping these contacts. See Figure 3-9.

ENHANCED_PIN_GROUPING is automatically turned off for co-vertical base layer-based flow (20 nm and lower process technology nodes). Hence run time is improved because performing pin snapping based on contact is no longer required for co-vertical layer-based flow (the process requires pseudo via modeling).

Default behavior (0) is to group the contacts overlapping a single metal and single diffusion geometry, as follows:

- Pin is snapped over the contacts overlapping:
  - Single metal
  - Single diffusions

When set to 1, pin grouping considers multiple contiguous metal1 and diffusions, as follows:

- Pin is snapped over the contacts overlapping:
  - Multiple contiguous metal
  - Multiple contiguous diffusions

When set to 2, pin grouping considers multiple contiguous diffusions, but multiple non-contiguous metal1, as follows:

- Pin is snapped over the contacts overlapping:
  - Multiple broken metal
  - Multiple contiguous diffusions

This keyword is required in the **gds.config* file to handle such scenarios.

Syntax:

```
ENHANCED_PIN_GROUPING [ 0 | 1 | 2 ]
```

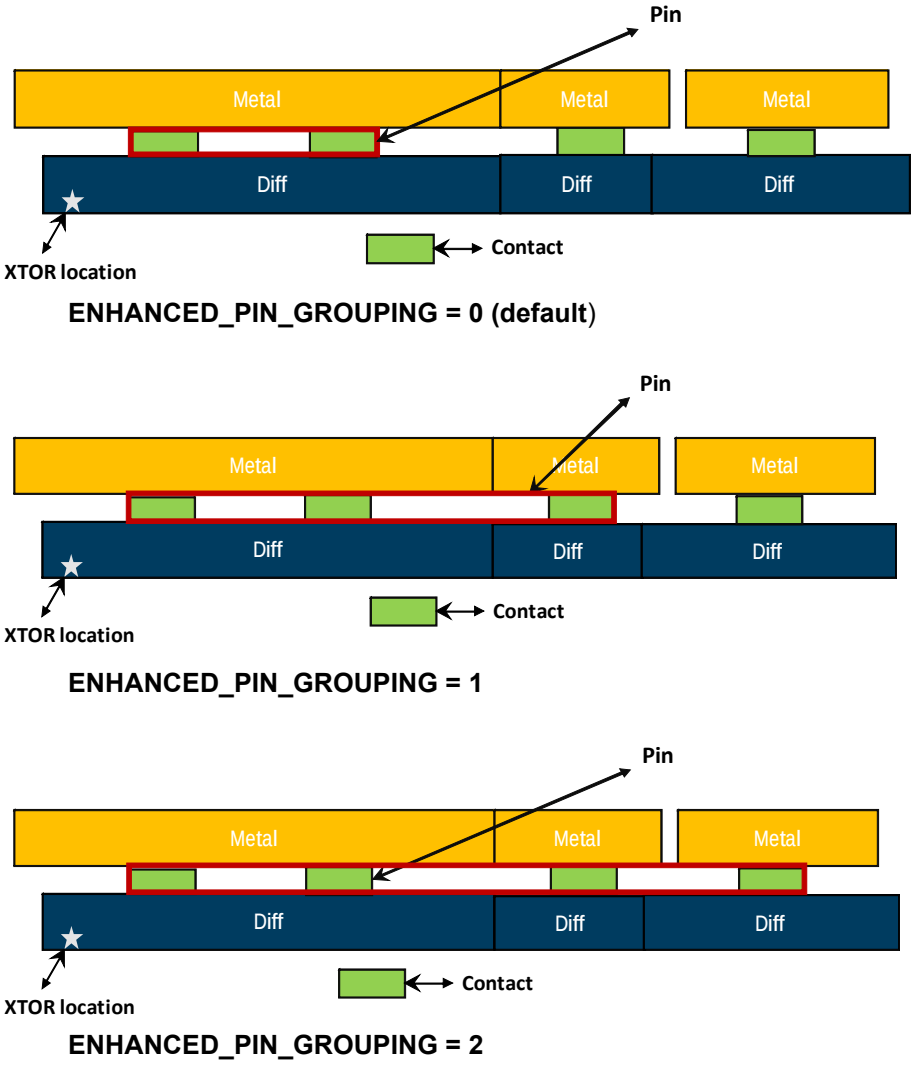


Figure 3-9 ENHANCED_PIN_GROUPING behavior

EXTENDED_RESISTOR_HANDLING

Required for proper handling of poly/diffusion/CM resistors.

Syntax:

```
EXTENDED_RESISTOR_HANDLING [ 0 | 1 | 2 ]
```

where

0: Off

1: Old legacy behavior (not recommended) - cm/poly/diff and cm/poly/diff resistor geometries are connected by inserting dummy via geometry between the layers. This may result in false EM violations

2: Default and Recommended -

- cm/poly/diff and cm/poly/diff resistor geometries are directly connected through abutment (no resistor-via geometries created)
- Creates a small disconnect between original layer and resistor border at

one end and snaps pin geometries (complete width) on either side

- Results in better current distribution and avoids false EM
- Reports resistor shorts in separate file (`adsRpt/GDS/resistor_short_file.txt`) that can be ignored

See Figure 3-10 for the default behavior

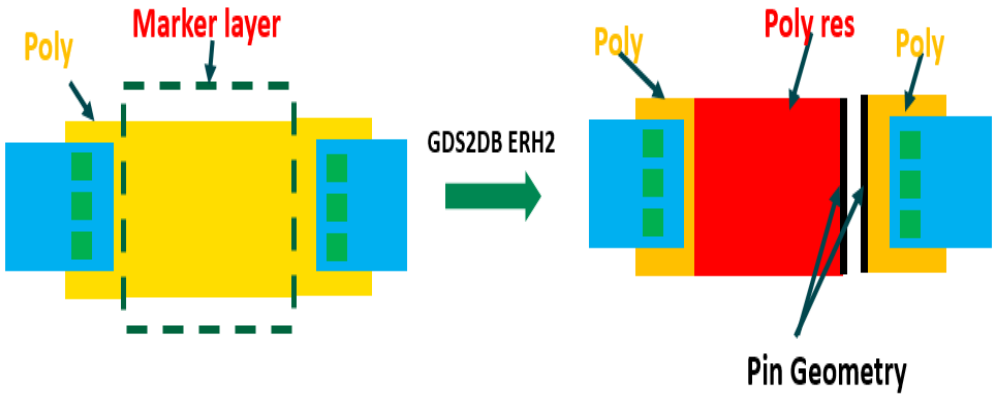


Figure 3-10 EXTENDED_RESISTOR_HANDLING 2 behavior

EXTENDED_SERIES_RESISTOR_HANDLING

To handle rhim resistors with shared terminals as in the below figure. Default: 1 (On)

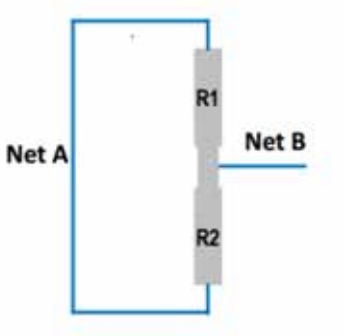


Figure 3-11 RHIM resistor with shared terminals

Syntax:

```
EXTENDED_SERIES_RESISTOR_HANDLING [0|1]
```

EXTRACT_FLOATING_PODE

When set, retains the floating poly-geometries in the Totem DB.

Syntax:

```
EXTRACT_FLOATING_PODE [1 | 0]
```

EXTRACT_SUBSTRATE

When set, extracts all geometries for special substrate layers without tracing. These geometries are assigned to a preserved net named *adsSubstrateNet*. Totem analysis is

then based on the substrate technology file layer stacking and n- or p-type doping information to make the proper connections and modeling in the substrate network.

For example, if the design has a deep trench n-buried layer beneath the RF region, you can define the following in the GDS layer map file in addition to the standard supported layers:

```
DTRENCH sub 10:1 -
Nburied sub 11:1 -
```

Totem issues a warning if a metal or via has no geometries in GDS, and issue errors if any diffusion/poly/well layers do not have geometries in GDS. When EXTRACT_SUBSTRATE is 1, Totem exits if all well layers do not have geometries.

Syntax:

```
EXTRACT_SUBSTRATE [ 0 | 1 ]
```

GDS_INSTANCE_NAME_KEY

When set, reads user-specified instance annotation in GDS. If the GDS has instance name annotation in custom properties, you can provide key-value pairs to the GDS translation engine to retain the instance names in the generated LEF/DEF files using this keyword. If no key number is provided, the instance names are unknown. The syntax is:

```
GDS_INSTANCE_NAME_KEY <key_number>
```

LATCHUP_MODELING

Enables resistance-checks for tap contacts (substrate/well contacts) for latch-up purpose.

```
LATCHUP_MODELING {
    EXTRACT_WELL_TAPS 1
    RETAIN_XTOR_PINS 0
}
CONTACT_RESISTANCE 1
# Optional input
## XTOR_MAP_FILE <>
```

Where:

EXTRACT_WELL_TAPS : When set to 1, extracts tap-pins and name them as TAP_<domain>.gds* (in non xtor_map_file flow) or as TAP_xtorName (in xtor_map_file flow) under 3rd column in the pratio file.

RETAIN_XTOR_PINS : When set to 0, GDS2DB creates only tapPins and ignores active device pins. Otherwise all the pins are retained. Default :1 for both non-xtor and xtor based flow.

MERGE_DEVICE_WIRE

When set, the mergeable geometries in diffusion/poly/co-vertical (or any layers defined in the layermap as Boolean) are merged, for proper shrinking of diffusion geometries. When turned off, no geometry merging on these layers is performed. The default is on (1).

```
MERGE_DEVICE_WIRE [0|1]
```

MOS_PIN_SNAPPING

By default (1), Totem follows the complete MOS device (TYPE{M}) pin snapping algorithm. However, for certain types of MOS device layouts in which there is no direct source or drain contact, but which are weakly connected to diffusion with a distant

contact, ‘MOS_PIN_SNAPPING 1’ can cause high runtimes. For such situations, a ‘MOS_PIN_SNAPPING 2’ setting should be used for runtime improvement. Syntax:

```
MOS_PIN_SNAPPING [ 1 | 2 ]
```

MMX_HIERARCHY_FILE

Specifies the netlist-driven hierarchy file generated by APLMMX to generate a hierarchical LEF/DEF file.

Syntax:

```
MMX_HIERARCHY_FILE <filename>
```

NEAREST_NET_ERROR_PIN_SNAPPING

When set, prevents shorting between different domains when the XTOR_MAP_FILE file generated from APLMMX has multiple pins with the same location, but which belong to different net domains.

Syntax:

```
NEAREST_NET_ERROR_PIN_SNAPPING [ 0 | 1 ]
```

NEW_POLY_RES_HANDLING

When set, reduces runtime with more efficient polyRes handling (default 0).

Syntax:

```
NEW_POLY_RES_HANDLING [ 0 | 1]
```

OPTIMIZED_BOOLEAN_OPERATIONS

When set, improves run time during logical layer handling by creating nested Boolean layer operations for complex Boolean expressions; the layer map file is pre-parsed to check all complex Boolean layer operations for redundant expressions. Temporary internal layers are created for all such expressions so that logical layer processing on all such layers is performed only once (default 1). A temporary layerMap is generated in *adsRpt/GDS/temp_layermap_file.txt* for debugging and verification purpose.

Syntax:

```
OPTIMIZED_BOOLEAN_OPERATIONS [0|1]
```

QOR_VALIDATION_MODE

When set, enables QoR validation, which compares the number of devices and topology between the input XTOR_MAP_FILE and GDS2DB pin recognition. Default is 0 - off.

```
QOR_VALIDATION_MODE [0|1]
```

The following files are generated in the *adsRpt/Validation/* file:

1. *layout_summary.rpt*. This report file has the following data tables in it.
   - The first table checks whether each metal/via layer in the tech file is present in the LayerMap file or not.
   - The second table checks whether each non-temp layer present in the LayerMap file is present in the tech file or not.
   - The third table lists the number of geometries in each GDS layer flattened to the top cell.
   - The forth table lists connectivity information for different layers specified in the LayerMap file
   - The 5th table is created only when XTOR_MAP_FILE keyword is provided in

GDS2DB configuration file. This table lists different device types that need special markers, checks the presence of marker layers in the layer-map file and the number of marker geometries found in the GDS.

2. *QOR_validation_summary.rpt*. This file is generated only when XTOR_MAP_FILE is provided in GDS2DB configuration file. It has two tables.
   - The first table lists the net-wise percentage of devices recognized by GDS2DB against the APLMMX netlist output (XTOR_MAP_FILE).
   - The second table lists device-type-based percentage of devices recognized by GDS2SB against the APLMMX netlist output (XTOR_MAP_FILE).
3. *gds_device_summary.rpt*: Lists detailed information on devices recognized per net per type. For every failed net (less than 100% mapping), it shows the number of devices present in the netlist and the number of devices recognized properly in GDS2DB for every device type.

RELAX_PINSNAPPING

Generally, to snap a transistor to contact, GDS2DB follows an algorithm based on the device type (such as TYPE M = MOS, TYPE D = Diode, TYPE Q = BJT) using the specific layout topology or marker layer for that device. By default, if the device cannot be connected to any net using the device-specific algorithm, a dummy pin is created on the lowest metal layer at the device location.

If RELAX_PINSNAPPING is set to 1, a device that cannot be snapped using the device-specific algorithm is snapped to the nearest geometry of the same net on the lowest metal layer. The file *adsRpt/GDS/xtor_snapping_error.txt* listing unsnapped transistors is still generated for those devices, even though they are connected to the lowest metal layer of the same net. Pins with PSL (pin snapping layer) specification are also supported. If there's no geometry available at the DSPF location on specified PSL, it will search for nearest geometry on same net and create a point pin. All devices that are connected to a specific net using the above option are also present in the *adsRpt/GDS/xtor_snapping_report.txt* file with the corresponding Pin-Snapping-Layer (PSL).

Syntax:

```
RELAX_PINSNAPPING [0|1]
```

REPORT_TRACED_VAIMODELS_ONLY

When set, does not create via models when they are not instantiated in the DEF generated by GDS2DB. If this keyword is set to 0 and GDS2DB is creating via models with none of them instantiated in DEF, then an error message is displayed:

Syntax:

```
REPORT_TRACED_VIAMODELS_ONLY [0|1]
```

RESISTOR_PIN_GROUPING

When set, improves grouping of contacts for resistor pins. The default is 1.

Syntax:

```
RESISTOR_PIN_GROUPING [0|1]
```

RESISTOR_SNAP_THRESHOLD_DISTANCE

Diffusion resistors used to connect nets and pins of TYPE{R} that do not overlap any resistor marker layer geometries can be incorrectly snapped to the nearest metal of the same net. When this keyword is used, Totem finds a resistor marker layer within the specified radius around the transistor and snaps it to a terminal having contacts of the

same net, and pins within the specified distance are snapped to the correct device edge. Results of snapping are recorded in the file *xtor.pin_snapping_report.txt*.

Syntax:

```
RESISTOR_SNAP_THRESHOLD_DISTANCE <radius_um>
```

SNAP_THRESHOLD_DISTANCE

Specifies the transistor snapping threshold distance. If the distance between the transistor location from XTOR_MAP_FILE and the closest contact or metal wire of the same net/pin type is larger than the specified threshold distance, GDS2DB does not snap the transistor to the closest metal wire; the transistor remains unconnected and a warning is issued.

Note that option SNAP_THRESHOLD_DISTANCE of configuration file keyword SWITCH_CELLS_PROPERTIES, if specified, takes precedence over this keyword for MOS devices. A list of unconnected transistor pins can be viewed using the menu command **View -> Connectivity -> Highlight Disconnected Transistor Pin**. Details about contact snapping can be reviewed in the file *adsRpt/GDS/xtor.contact_snapping_report*. Details about disconnected transistors can be reviewed in the file *adsRpt/GDS/xtor.disconnected_to_metal*.

Syntax:

```
SNAP_THRESHOLD_DISTANCE  <distance_microns>
```

SUPPORT_LDMOS

To reduce tap-cell detection runtime in GDS2DB processing when there are no LDMOS cells in the design, set ‘SUPPORT_LDMOS 0’. The default value is 1.

Syntax:

```
SUPPORT_LDMOS [ 1 | 0]
```

TAP_CONTACT_EXCEPTION

In general a MOS device pin is snapped to a real contact that is not a well tap for that device. Specifying this keyword also considers tap contacts for pin snapping. Default: Off.

Syntax:

```
TAP_CONTACT_EXCEPTION [ 0 | 1 ]
```

TRACE_THROUGH_CM

When set, supports net tracing through co-vertical metal base-layers (between M1 and poly/diffusion), as shown in Figure 3-12. This is supported in NATIVE_GDS_SUPPORT settings 1 and 2, When this keyword is set, GDS2DB traces through CM layer geometries, and traces up to other metal layer geometries. The default is 1 (On) when CM layers are present.

Syntax:

```
TRACE_THROUGH_CM [0 | 1]
```

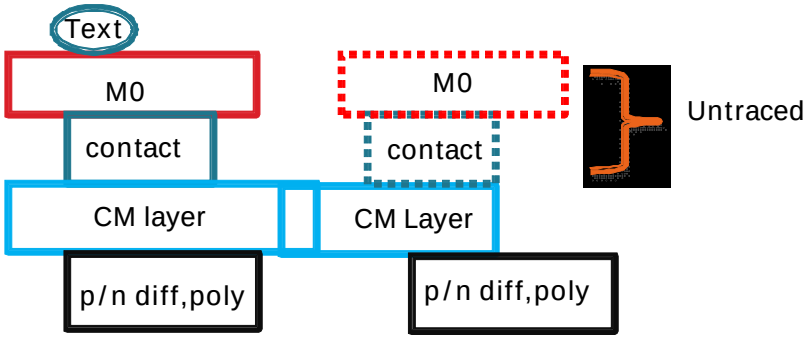


Figure 3-12 Tracing with TRACE_THROUGH_CM off

TRACE_THROUGH_CMRES

When set, traces through the co-vertical metal resistor layer (rh_tn) present in different GDS hierarchies without pre-flattening the hierarchy in GDS layout. (Another way to do the same thing is to put this keyword in the GDS_OPTION section of the APLMMX configuration file.) Default: 0.

Syntax:

```
TRACE_THROUGH_CMRES [ 0 | 1 ]
```

A sample GDS_MAP_FILE setting to trace the rh_tn layer (must be defined as a CM layer) is as follows:

```
BOOLEAN rh_tn cm RH_TN-RPDMY
BOOLEAN rh_tn_body cmres RH_TN&RPDMY rh_tn
```

where 'rh_tn_body' is the cmres for 'rh_tn'.

TRACE_THROUGH_DEVICE_LAYER

In some layouts there are 2 or more disjoint geometry domains, which are only connected though device layers poly, pdiff, or ndiff. If one of the domains has associated text labels, but others do not, the domain without text labels is not traced by default, since net-tracing does not go upward from the device layer by default. If the domain without text labels has associated transistor locations in XTOR_MAP_FILE there will be pin-snapping errors also. In such a case, setting TRACE_THROUGH_DEVICE_LAYER to 1 allows geometry without text to be traced through device layers. This situation often occurs in process nodes 20nm and smaller. The syntax is:

```
TRACE_THROUGH_DEVICE_LAYER [ 0 | 1 ]
```

TRACE_THROUGH_DIFFUSION_RESISTOR

TRACE_THROUGH_POLY_RESISTOR

TRACE_THROUGH_CM_RESISTOR

These keywords trace through diffusion, poly, or Co-vertical Metal layer resistors, to allow proper pin snapping and analysis (in conjunction with "EXTENDED_RESISTOR_HANDLING 2") . User must define diffusion/cm/poly resistor marker layers in GDS layer map appropriately. Default ON (Turned on by "EXTENDED_RESISTOR_HANDLING 2"). When set to 0, ignores resistor geometry completely.

Syntax:

```
TRACE_THROUGH_DIFFUSION_RESISTOR [ 0 | 1 ]
```

```
TRACE_THROUGH_POLY_RESISTOR [ 0 | 1 ]
TRACE_THROUGH_CM_RESISTOR [ 0 | 1 ]
```

TRACE_THROUGH_POLY

When set to 1, enables tracing net through poly geometries. Default ON for advanced technology nodes (<= 10). Does not trace through poly-resistor geometries (poly geometries with poly-resistor marker layer over them).

Syntax:

```
TRACE_THROUGH_POLY  [ 0 | 1]
```

XTOR_MAP_FILE

Specifies the transistor mapping file that includes transistor name, location, power strength ratio (optional if it is a GDS- only flow).

Syntax:

```
XTOR_MAP_FILE <transistor_mapping_filename>
```

XTORMAP_SANITY_CHECK

If set to 0, skips time-consuming sanity checks inside GDS2DB. By default, Totem checks if the net names in the XTOR_MAP_FILE are defined in the GDS2DB configuration file, which can be time- consuming if the net names are very long, especially in Signal EM. Default: On.

Syntax:

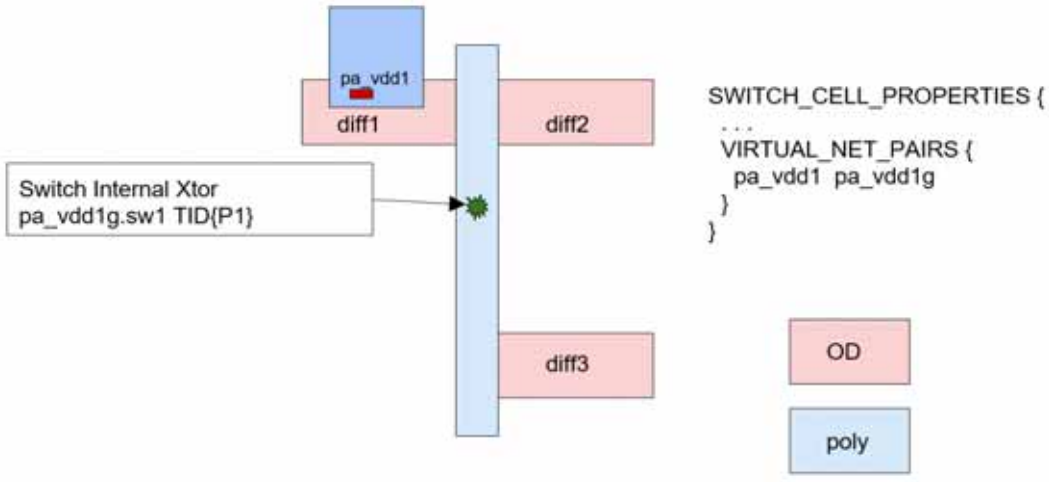
```
XTORMAP_SANITY_CHECK [0|1]
```

XTOR_BASED_NET_PAIR_CHECK

When set to 1, GDS2DB will support shared poly in AUTO_TRACE_INTERNAL_NETS flow for embedded switch designs. *Default : 0*

Syntax:

```
XTOR_BASED_NET_PAIR_CHECK [ 0 | 1 ]
```



For ex, in picture above, shared poly is touching two untraced diffusion geometries (on the right), though “diff1” geometry (belonging to external net pa_vdd1) which is sharing

the effective channel with “diff2” geometry. With this flow, “diff2” geometry is identified as start geometry to trace Internal Net pa_vdd1g (for pa_vdd1g.sw1 xtor).

Selective Cell Hierarchy Handling

BOX_CELL_LEF_PROPERTIES

This keyword provides more control in defining the behavior of box cells. The options are

TRACE_INTERNAL_NETS : when set to 1, traces all internal nets. Otherwise, for BOX_CELL geometries connecting to the top level nets, PINs based on these geometries are created. Can improve run-time when set to 0. Default value: 1.

BOX_CELL_EXTRACTION_STARTING_LAYER <layer_name>: specifies the metal layer below which pins are not created for the BOX_CELL.

OUTPUT_TOP_LAYER_PINS_ONLY : when set to 1, extracts only the top layer geometries for each internal net for the BOX_CELL. Otherwise, pins are created based on all the geometries for each internal net. Can improve run-time when set to 0. Default value: 1.

The full syntax is:

```
BOX_CELL_LEF_PROPERTIES {
    TRACE_INTERNAL_NETS [0 | 1]
    BOX_CELL_EXTRACTION_STARTING_LAYER <layer_name>
    OUTPUT_TOP_LAYER_PINS_ONLY [ 0 |1 ]
}
```

BOX_CELLS_PROPERTIES

BOX_CELLS_PROPERTIES is used in the GDS2DB flow to streamline all box cell-related options and functionality. The syntax is:

```
BOX_CELLS_PROPERTIES {
    PIN_CREATION_LAYERS [ ALL | <layer1> <layer2> ... ]
    UNIQUIFY [0 | 1 ]
    PIN_CREATION_LEVEL [min|max]
    TRACE_FROM_PIN [0|1]
    XTOR_BASED_ESL [0|1]
    PIN_NAME_TEXT_LABELS <text_label1 <text_label2> ...
    MEMORY_SAVING_MODE [0|1]
    USE_LEF_PINNAME [0|1]
    EXTRACTION_STARTING_LAYER {
        <cell_name> <layer_name>
        }
}
```

The PIN_CREATION_LAYERS option specifies the layers within the box cell on which the box cell pins are created, which can be specified as follows:

ALL - uses the metal geometries from all layers within the box cell to create the switch cell pins. This is the default behavior when BOX_CELLS_PROPERTIES is not specified.

<layer1> <layer2> - uses only geometries from the specified metal layers for pin creation. Combinations of device layers and metal layers may also be specified, including diffusion or co-vertical base layers, such as

```
PIN_CREATION_LAYERS ndiff pdiff
PIN_CREATION_LAYERS ndiff met3
```

The UNIQUIFY option invokes the uniquification feature for box cells when ‘UNIQUIFY 1’ is specified. Note that MERGE_INTERNAL_NETS is automatically enabled when ‘UNIQUIFY 1’ is set.

The MERGE_INTERNAL_NETS option specifies the internal nets of the box cell that have the same connectivity to top level nets across all instances of a particular cell, and are merged together if ‘MERGE_INTERNAL_NETS 1’ is specified.

The PIN_CREATION_LEVEL option, when set to “min” considers the text directly inside block, and when set to “max” considers text from all hierarchy inside block .

The TRACE_FROM_PIN option, when set to “0” creates pins with geometries that are connected to the text-label at the corresponding layer (should be specified in PIN_CREATION_LAYERS), and when set to “1” creates pins with geometries that are connected to text label at any layer. For example, if there is text on M2 geometry inside the block and the M2 geometry is connected to M3 geometry through v23 geometry (inside the block), then PIN_CREATION_LAYER is specified as M3. If 'TRACE_FROM_PIN 1' is set, Totem uses M3 geometry for pin creation, even though the text label is on M2.

The XTOR_BASED_ESL option auto-determines the cell-specific EXTRACTION_STARTING_LAYER based on the lowest Pin Snapping Layer (PSL) found in the GDSMMX file for each cell-type

Using the option PIN_NAME_TEXT_LABELS, GDS2DB can create pins on box cells based on text labels specified (regular expressions are accepted). A pin is created on the metal geometries touching the same layer as the text label. So if a text labels is on M1, then only M1 geometries touching the text label layer are part of the pin geometry. This feature allows creation of standard hierarchical data structures (similar to LEF/DEF). One application in which it can be used is the GDS-based ESD analysis flow, which allows standardizing pin names and locations on clamp cells (extracted as box cells), easing subsequent clamp info file creation. This option also honors text labels at lower hierarchies within box cells, as well as text labels at box cell hierarchy level. GDS2DB also supports aliased names for box cell pins. Note that only directly-connected metal segments are used to create pins, and not those connected through upper layers.

Example for PIN_NAME_TEXT_LABELS:

```
PIN_NAME_TEXT_LABELS {
    <text_label1>
    <text_label2>
    VDD {
        A*  ### alias A* and B* text labels to VDD
        B*
    }
}
```

The option MEMORY_SAVING_MODE reduces memory usage for box cell modeling.

USE_LEF_PINNAME annotates box cell pin names from input LEF when it is provided.

The option EXTRACTION_STARTING_LAYER allows to specify an extraction starting layer for each box cell separately, instead of a global setting. GDS2DB then preserve only the layers and pin geometries from the specified layer to the top layer.

CREATE_LEF_MACRO_FOR_BOX_CELLS

When specified along with GRAY_BOX_CELLS, generates a LEF file called *<original_top_level_cell>_adsgds1.lef* containing power and ground pins for the

specified output cells. Default is 0 (no LEF file) for GRAY_BOX_CELLS and 1 for WHITE_BOX_CELLS.

Syntax:

```
CREATE_LEF_MACRO_FOR_BOX_CELLS [ 1 | gray | white | 0]
```

where

1: creates LEF for both gray and white box cells

gray: creates LEF for gray box cells only

white: creates LEF for white box cells only

0: no LEF file

BLACK_BOX_CELLS

When this GDS2DB keyword is used, all cells listed in BLACK_BOX_CELLS are ignored when reading in GDS2 files, and are *not* included in the DEF file. Wildcards are supported in specifying cell names. A report *boxcell_wildcard_match.txt* is generated in the *adsRpt/GDS* directory, which contains a list of the cells found in GDS. The syntax is:

```
BLACK_BOX_CELLS {
    <sub-block_name1>
    ...
}
```

GRAY_BOX_CELLS

The GRAY_BOX_CELLS keyword defines separate instances of cells in the output DEF file. For a hierarchical design flow, if this keyword is set to 0 for a listed cell, only cell-instances are created, with no feed-through geometries converted. If set to 1, boundary boxes and feed-through geometries crossing top-level and sub-block boundaries are preserved for use in the top level view. See Figure 3-13 and Figure 3-14 showing the behavior of the two settings. This can be useful in analog blocks with a digital sub-block flow, by instantiating and placing sub-blocks within the block. It can also be used to resolve capacity issues, such as memory and run-time, and avoid extraction of the sub-block geometries, while properly placing bounding boxes of sub-blocks in the master cell. You use the CREATE_LEF_MACRO_FOR_BOX_CELLS keyword, or must provide sub-block LEF files, in which case the LEF files are created automatically. If the keyword CREATE_LEF_MACRO_FOR_BOX_CELLS is defined, a cell LEF macro is generated for gray box cells in the file *<top>_adsgds1.lef.* Default: 0. Wildcards are supported in specifying cell names. A report *boxcell_wildcard_match.txt* is generated in the *adsRpt/GDS* directory, which contains a list of the cells found in GDS. The syntax is:

```
GRAY_BOX_CELLS {
    <cellname1> [ 0|1 ]
    ...
}
```

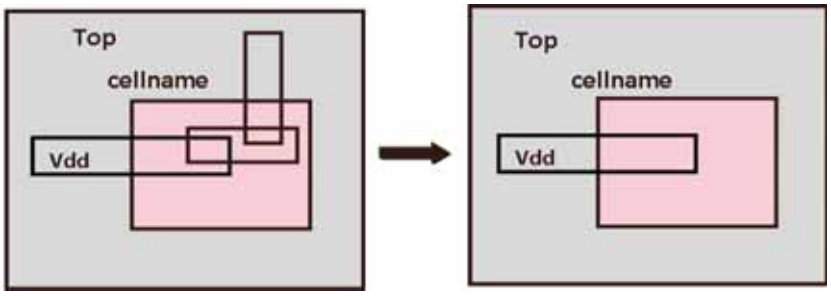


Figure 3-13 Gray_Box_Cells 0

When set to 0 Gray_Box_Cells discards the geometries of the ‘cellname’ block before tracing the net. So any geometries at a higher level of hierarchy that can be traced only through the ‘cellname’ block are *not* retained.

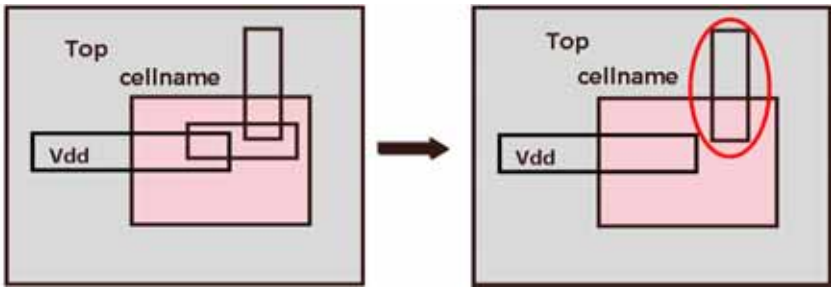


Figure 3-14 Gray_Box_Cells 1

When set to 1 Gray_Box_Cells discards the geometries of the ‘cellname’ block after tracing the net. So any geometries at a higher level of hierarchy that can be traced only through the ‘cellname’ block *are* retained.

Note that base layer checking is relaxed when “GRAY_BOX_CELLS” is present in the GDS configuration file or in hierarchical flows. This enables creating pins on the top of hierarchical blocks, while allowing tracing and pin creation on lower level metals, to capture devices that are not contained within a hierarchical block

WHITE_BOX_CELLS

This keyword creates separate instances of the cells in the output DEF file, and retains all geometries inside cell-instances. Current sinks are created inside the box and hierarchy is retained. A LEF file is also generated for the cell. Wildcards are supported in specifying cell names. A report *boxcell_wildcard_match.txt* is generated in the *adsRpt/GDS* directory, which contains a list of the cells found in GDS. Syntax:

```
WHITE_BOX_CELLS {
    <sub-block_name1>
    ...
}
```

MMX_HIERARCHY_FILE

Specifies the netlist driven hierarchy file generated by APLMMX to generate hierarchical LEF/DEF file. Refer to section "Netlist-Driven Hierarchical Dynamic Analysis", page 4-165, for more details.

Syntax:

```
MMX_HIERARCHY_FILE <filename>
```

UNIQUIFY_BOX_CELLS

Creates unique LEF cells when instances of the cell have different connections to Vdd and Vss, based on the external net connectivity of the cell. The keyword is used for the uniquification of box cells specified using either the GRAY_BOX_CELLS or WHITE_BOX_CELLS keywords. Using the recommended and default '2' value assigns a LEF pin name based on the parent cell (top) net connection. If specifically desired, the previous functionality can be invoked using the keyword setting '1'. With this setting, if the same box cell pin connects to both power and ground (parent) nets in different instances, the created LEF pins are assigned names that do not relate to their parent cell connections.

```
UNIQUIFY_BOX_CELLS [ 2| 1]
```

USE_LEF_FOR_BOX_CELL_BBOX

When set, GDS2DB uses input LEF bbox from the box cell macro ORIGIN/SIZE.

Syntax:

```
USE_LEF_FOR_BOX_CELL_BBOX [ 0 | 1 ]
```

Auto Pad Location Generation

GDS2DEF_PLOC_CREATION

If set to 1, PLOC creation matches the old, legacy gds2def flow. The following two aspects of PLOC creation are changed during GDS processing to make it similar to old/ legacy GDS2DEF flow:

1. LEF-port PLOCs. If LEF_FILE is present in config file, PLOCs will be created for the input LEF ports in addition to the PLOCs created for text labels.
2. Larger PLOC area. Each non-LEF-pin PLOC will be represented by a 2um square centered on the PLOC coordinate (clipped to fit overlapping wire geometry), instead of tiny 0.01 um square earlier. This can avoid potential EMIR inaccuracy caused due to tiny plocs.

Syntax:

```
GDS2DEF_PLOC_CREATION [ 1 | 0 ]
```

GENERATE_PLOC

When specified, GDS2RH creates pads in the PINS section of the DEF file it creates. The locations where these pads are created depends on the option specified. During analysis you must set the GSR keyword 'ADD_PLOC_FROM_TOP_DEF 1' to allow GDS2RH to create pads as ploc points.

```
GENERATE_PLOC [USE_TEXT_LABEL | USE_USER_START_PT |
    USE_INPUT_LEF_PIN | USE_PIN_LAYERS | USE_TEXT_PIN_NAME
    USE_NIF_FILE <nif_file_path> | USE_PIN_MARKER
```

where

USE_TEXT_LABEL: generates plocs at the location of text labels. Refer to the following GDS configuration file keywords to determine the selection of the text labels:

- USE_TEXT_FROM_HIERARCHY_LEVEL
- USE_TEXT_FROM_CELLS
- USE_TEXT_FROM_HIERARCHY_BLOCK

USE_USER_START_PT: if you have specified custom start points for net tracing, then those locations are used for ploc creation when this keyword is specified. This keyword also may be used either independently or in combination with any of the above specified options. Can be used with USE_TEXT_LABEL option.

USE_LEF_PINS: generates plocs that have the same size and position as all pin geometries that are present in the input LEF.

USE_INPUT_LEF_PIN: generates plocs that have the same size and position as all pin geometries that are present in the input LEF. To have finer control over which layer and area the plocs are created, the keyword GENERATE_PLOC_FILTER may be used.

USE_PIN_LAYERS: generates plocs that have the same size and location as all metal geometries. To have finer control over which layer and area the plocs are created, the keyword GENERATE_PLOC_FILTER may be used.

USE_TEXT_PIN_NAME: generates plocs for power, ground and signal domains.plocs generated for signal domain can be named as per text label . For example, e.g., If config signal net is as below, then PLOC created will be SIG and SIG:

```
SIGNAL_NETS {
    SIG {
        SIG
        SIG: }
    }
```

USE_NIF_FILE <file>: creates PLOCs for all ports in power and ground domains occurring in the NIF file, and generates the PINS section in the generated DEF (GDS2DB) based on the “port” section of the network information file (NIF), which is only valid in the Calibre LVS flow. The generated DEF uses the “port” section from the NIF file for the PINS section in DEF. The file name in the configuration file keyword 'NIF_FILE <filename>' must be specified as well.

USE_PIN_MARKER: generates PLOC on metal layers or PINS in the generated DEF file using a special pin-marker layer in the GDS layer map file.

Note: The GENERATE_PLOC_FILTER keyword following can be used for controlling the plocs generated by ALL of the above keyword options, and have the same effect in each case.

GENERATE_PLOC_FILTER

GENERATE_PLOC_FILTER maybe be used with any of the GENERATE_PLOC options (USE_LEF_PINS, USE_TEXT_LABEL, USE_PIN_LAYERS) to fine tune pad location creation, and have the same effect in each case.

```
GENERATE_PLOC_FILTER {
    [<metal_layer_name> | ALL] [ <x1 y1 x2 y2> | ALL]
    [ <net_name> | <net_type> | ALL ]
  ...
```

```
}
```

where (x1, y1) and (x2, y2) represent the bottom left and top right x,y co-ordinates, respectively, of the region bounding box.

For example,

```
GENERATE_PLOC_FILTER {
M1 10 20 15 40
M3 120 200 240 300
}
```

Note that the combination of region-based and also individual net-based filtering is presently not supported.

The layer and region combinations for the filter may be specified in several ways:

1. Combination of layer and region specified example:

   ```
   GENERATE_PLOC_FILTER {
   M1 10 20 15 40
   M3 120 200 240 300
   }
   ```

   Here only those geometries that are inside the specific bounding boxes for M1 and M3 are created as DEF pins.

2. Layer only example:

   ```
   GENERATE_PLOC_FILTER {
   M1 ALL
   M3 120 200 240 300
   }
   ```

   Here, ALL geometries in M1 go to the DEF pin section, while only those geometries that are inside the specific bounding box for M3 are created as DEF pins.

3. Region only example,

   ```
   GENERATE_PLOC_FILTER {
   ALL 120 200 240 300
   ALL 400 60 700 250
   }
   ```

   In this case for ALL metal layers, only those geometries in the specified regions are in the DEF pin section.

4. Example of all regions and nets:

   ```
   GENERATE_PLOC USE_PIN_LAYERS
   GENERATE_PLOC_FILTER {
     TOP all all
   }
   ```

   All top layer geometries are pushed out as pins in the DEF pins section.

NIF_FILE

Specifies the file name when the generated DEF uses the “port” section from the NIF file for the PINS section in DEF. The syntax is:

```
NIF_FILE <filename>
```

Automated Switch Cell Handling

Note: For this flow the DEFINE_SWITCH_CELLS and EXTRACT_SWITCH_CELLS configuration file keywords are not needed.

EXTEND_SWITCH_BBOX <extension_um>

Specifies the added amount to be added to the switch bounding box on each side, in microns. The default is 0.

OD_SUPPORT

When set, supports distributed switch modeling (switch Ron distribution between multiple fingers) for the 20 nm technology node, with 'TECHNOLOGY 20' set in the GSR.

```
OD_SUPPORT [ 0 | 1 ]
```

MAX_SWITCH_WIDTH <width>

MAX_SWITCH_HEIGHT <height>

These keywords specify the granularity of the switch cells, as shown in Figure 3-15. GDS2DB uses DSPF switch device coordinates to group switch pins together. The default value of MAX_SWITCH_WIDTH is "2*XTOR_FINGER_WIDTH". The default value of MAX_SWITCH_HEIGHT is "20*XTOR_FINGER_PITCH". Internal defaults are set for these keywords based on power switch pin information obtained from the netlist defined in the APLMMX output **.gdsmmx* file.

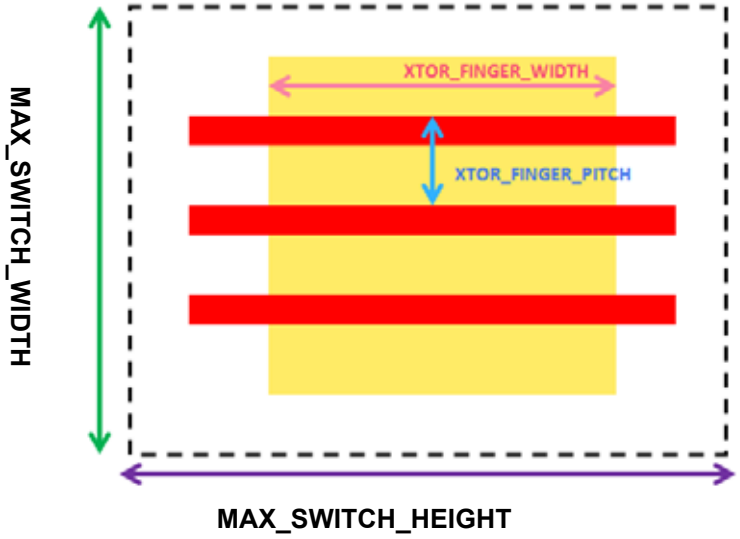


Figure 3-15 Switch cell specifications

XTOR_FINGER_PITCH <pitch_u>

Specifies the poly pitch between fingers of the switch, as shown in Figure 3-15. The default is "8*L" based on the maximum L of all switches, based on power switch pin information obtained from the netlist defined in the APLMMX output **.gdsmmx* file.

XTOR_FINGER_WIDTH <width_u>

Specifies the maximum switch device layout width, as shown in Figure 3-15. The default is “1.5*W”, based on maximum W of all switches, based on power switch pin information obtained from the netlist defined in the APLMMX output **.gdsmmx* file.

SWITCH_CELLS_PROPERTIES

This keyword controls switch cells pin creation, and automatically creates the power switch hierarchy and a switch model file to handle cases in which the switch hierarchy is flattened in GDS layout. Note that option SNAP_THRESHOLD_DISTANCE, if specified, takes precedence over specifying keyword SNAP_THRESHOLD_DISTANCE for MOS devices.

After you run APLMMX with keyword SWITCH_MODEL_TABLE to dump out the power switch transistor information in the **.gdsmmx* file, then you use this keyword for the embedded power switch hierarchy and power switch model file creation:

```
SWITCH_CELLS_PROPERTIES {
    AUTO_TRACE_INTERNAL_NETS 1
    CREATE_HIERARCHY {
        USE_SWITCH_XTOR
        USE_SWITCH_XTOR_FINGER
        XTOR_FINGER_PITCH
        XTOR_FINGER_WIDTH
        EXTEND_SWITCH_BBOX
        MAX_SWITCH_WIDTH
        MAX_SWITCH_HEIGHT
    }
    SNAP_THRESHOLD_DISTANCE <dist>
    VIRTUAL_NET_PAIRS {
        <ext_net_name> <int_net_name>
    }
    SMASH_GEOMETRIES [1|0]
    USE_SWITCH_INT_PIN_FOR_TRACING [1|0]
    PIN_CREATION_LAYERS [ ALL| <MET2>| <MET1>| <cont_lay>]
    MEMORY_SAVING_MODE [0 | 1]
    }
    VDD_NETS{
        <external_net_name> {
        [ <text_label> |<trace_point>]
        }
        <external_net_name1>
    }
    GND_NETS{
        <internal_net_name> {
        [ <text_label> |<trace_point>]
    }
        <internal_net_name1>
}
    ANNOTATED_GDS_MAP_FILE <GDS map file>
```

where

AUTO_TRACE_INTERNAL_NETS - when set, automatically traces the internal nets in the embedded switch flow if all external and internal nets are defined using VDD_NETS and GND_NETS keywords. You do not need to define the texts/start points; GDS2DB internally uses the *.*sw* locations from the **gdsmmx* file to trace the nets. All GDSII texts/start points are completely ignored for the internal nets. Default 1 (On).

USE_SWITCH_XTOR - keyword to enable embedded power switch handling

USE_SWITCH_XTOR_FINGER – creates switch cell per MOS devices. This keyword can be used instead of USE_SWITCH_XTOR and it will not merge multiple MOS Xtors.

XTOR_FINGER_PITCH - poly-gate perpendicular direction grouping threshold, [8*L], based on max L of all switches

XTOR_FINGER_WIDTH - poly-gate parallel direction grouping threshold, [1.5*W], based on max W of all switches

EXTEND_SWITCH_BBOX - to extend identified switch finger bounding box, default =0

MAX_SWITCH_WIDTH - to restrict the maximum width of the switch, [2*XTOR_FINGER_WIDTH]

MAX_SWITCH_HEIGHT - to restrict the maximum height of the switch, [20*XTOR_FINGER_PITCH]

SNAP_THRESHOLD_DISTANCE - when specified, takes precedence over specifying keyword SNAP_THRESHOLD_DISTANCE for MOS devices.

VIRTUAL_NET_PAIRS - defines the internal and external net pairs of the switch cells. In the embedded switch flow, automatically identifies external and internal net pairs and lists them under VIRTUAL_NET_PAIRS.

SMASH_GEOMETRIES: when set to 1 (default), all switch cell internal geometries are pushed to the top level, which makes it similar to a white box cell approach. When set to 0, none of the switch cell internal geometries are pushed to the top level, which makes it like a gray box cell.

USE_SWITCH_INT_PIN_FOR_TRACING: when set, forces GDS2DB to use text labels for internal pins inside the switch cell hierarchy to trace the internal P/G nets. This keyword can only be used when the switch cell hierarchy inside the GDS file has text labels corresponding to <internal pin>, as defined in the keyword DEFINE_SWITCH_CELLS. This eliminates the need for you to specify coordinates for tracing internal P/G nets when text labels for them are not brought out at top level hierarchy.

PIN_CREATION_LAYERS: specifies layers on which pins are created. When a contact layer (name from layer map file) is specified, the pins are created on MET1 based on the contact via top MET1 geometries.

MEMORY_SAVING_MODE reduces memory usage for box cell modeling.

VDD_NETS, GND_NETS : defines nets for auto internal net tracing flow

ANNOTATED_GDS_MAP_FILE - GDS map file with attributes for internal net geometries

The following is a sample switch cell GDS2DB configuration file:

```
SWITCH_CELLS_PROPERTIES {
    PIN_CREATION_LAYERS ALL
    CREATE_HIERARCHY {
        USE_SWITCH_XTOR
        XTOR_FINGER_PITCH 0.1  # Switch grouping keyword
        XTOR_FINGER_WIDTH 0.6  # Switch grouping keyword
```

```
            EXTEND_SWITCH_BBOX 10  # Switch grouping keyword
            MAX_SWITCH_WIDTH 4     # Switch grouping keyword
            MAX_SWITCH_HEIGHT 4    # Switch grouping keyword
        }
        VIRTUAL_NET_PAIRS {
            <external net_name> <internal net_name>
            <external net_name1> <internal net_name1>
            ...
      }
    }
```

SWITCH_MODEL_TABLE

Specifies a switch model table file that has a list of device models with the following information:

- On-state resistance of the power switch
- On-state saturation current of the switch IDS
- Off-state leakage current of the power gate

A sample switch_model.tbl file is shown below:

```
# DeviceModel Width(um) Length(um) Ron(Ohms) IdSat(Amps) Ileak(Amps)
    *       0.8         0.04        60       1.2e-3      0.1e-8
    pch     0.8         0.04        60       1.2e-3      0.1e-8
    nch     0.8         0.04        34       2.2e-3      0.2e-8
...
```

Note that only one width and length combination per device model must be specified in the lookup table. The syntax is:

```
SWITCH_MODEL_TABLE <switch_table.mdl>
```

ESD Analysis

ESD_CELLS

When this keyword is specified, GDS2DB can create hierarchy for clamp cells from a flattened GDS for GDS-based ESD static analysis. A unique virtual cell is created from the specified ESD components with all the geometries in all the layers from all the hierarchies in that region. The geometries that extend outside of the specified ESD regions are cut and made part of the ESD cell. Syntax:

```
ESD_CELLS {
    CREATE_HIERARCHY {
    [USE_CELL_NAMES |USE_ID_LAYER |USE_REGION ] }
    USE_CLAMP_FILE_PIN_LAYERS [0|1]
    RESTRICT_ID_LAYER_MERGE_ACROSS_HIERARCHY [0|1]
    TRACE_THROUGH_STACKED_DEVICE [0|1]
    TRACE_THROUGH_SERIAL_DIODE [0|1|2]
    CLAMP_NETS_ONLY [0|1]
    POLARITY_BASED_CLAMP_INFO [0|1
    DUMMY_DIODE_NAMES {
        <dummy_diode_name1>
        <dummy_diode_name2>
        ...
    }
```

```
        CELL_NAMES {
            <cell_1>
            ...
        }
        REGION {
            <llx> <lly> <urx> <ury> <region_name>
            ...
        }
        TOLERANCE <ID_layer_tolerance>
        TOLERANCE_PER_ID {
         <id_layer1> <tolerance1>
         <id_layer2> <tolerance2>
        ...
       }
    }
        CREATE_PIN_PAIR {
            #<clamp type>  <marker>  PIN_PAIR {<> <>} [(RON {} |
    ivname { })]
    }
        CLAMP_CONN_FILE <create_pin_pair file>
    }
```

where

USE_CELL_NAMES/ CELL_NAMES: uses the specified CELL_NAMES to find the region of the overlapping instances of the cell. All instances of this cell are identified and the overlapping instances, or those meeting the specified criteria, are considered together to form a virtual ESD cell. The created virtual ESD cell covers all geometries inside all these instances and is instantiated under the TOP cell. The instances of the cell specified in CELL_NAMES section are not retained in the database or in the generated LEF/DEF files. Usually ESD cells are multi-finger devices formed by small parameterized cells called pcells. These pcell names should be specified in the CELL_NAMES section. Wildcards are supported for the cell-names under ESD_CELLS in GDS2DB config.

USE_ID_LAYER: uses the GDS ID layer specified in the layer map file to find the region enclosing the geometries from these layers.
With overlapping layers, all the ID layer geometries are found from all the hierarchies, and the ID layers are then flattened to the top. The overlapping ID layers are grouped together to form a virtual ESD cell.
With non-overlapping layers, the cell is created by considering the bounding box covering the region of ID layer geometries within the maximum distance specified.

For example, in the GDS layer map file:

```
ESD_diode ESD 122:0 -
```

This keyword can create Clamp Cells hierarchy independent of the GDS hierarchy if the RESTRICT_ID_LAYER_MERGE_* option is set to 0.

RESTRICT_ID_LAYER_MERGE_ACROSS_HIERARCHY : the default value of 1 means that the Clamp Cell Hierarchy is created only within the same GDS cell or hierarchy.

TRACE_THROUGH_STACKED_DEVICE: traces through internal nets between stacked ESD devices to perform ESD checks. Default 0.

USE_REGION/ REGION: uses the defined REGION to create a virtual ESD cell that covers all the geometries in the region, and has all the geometries from all layers in the region. Geometries extending outside of the region are cut and made part of the ESD cell. This cell is instantiated under the TOP cell.

USE_CLAMP_FILE_PIN_LAYERS : creates pins based on user-specified layers in the clamp file.

TRACE_THROUGH_SERIAL_DIODE : traces internal nets connecting any serial diodes created by the CREATE_HIERARCHY option, which is not traced if this keyword is not used. Top-level nets are created for these internal trace-through nets. This keyword is used with the CREATE_HIERARCHY option. A file, *clamp.ppi*, is generated for this feature that has the pin and layer information. Example portion of PPI file:

```
serial_diode_D1_1 PIN { pad } LAYER { PDIFF NDIFF } BBOX { 0.0000 0.0000
    0.8150 1.1330 }
serial_diode_D2_1 PIN { pad vdd } LAYER { PDIFF NDIFF } BBOX { 0.0250 0.0000
    0.8400 1.1460 }
serial_diode_D2_1 PIN { Net1 } LAYER { PDIFF NDIFF } BBOX { 0.0000 2.9170
    0.8660 4.0630 }
```

CLAMP_NETS_ONLY: When set to 1, this will retain only the nets that are connected to ESD devices, from auto-tracing of clamp connected nets; rest of the signal nets are ignored.

POLARITY_BASED_CLAMP_INFO - when set, identifies the actual cathode/ anode of diode devices and creates pin pairs with the correct polarity.

DUMMY_DIODE_NAMES : specifies the diode cell names that can be shorted to its own terminal. This kind of shorted diode areas could be present inside the custom hierarchy and are not considered for pin creation when this keyword is used.

REGION : defines coordinates for the region used to create the virtual ESD cell. The option <region_name> specifies the user defined region name for custom ESD cell naming. ESD cells get named as `<design name>_<region name>_<seq number>_0.`

TOLERANCE <ID_layer_tolerance> : globally specifies the maximum extension of the distance between regions, based on the GDS ID layer, or between instances defined in CELL_NAMES, for purposes of merging them (microns).

TOLERANCE_PER_ID: optionally specifies the maximum distance between regions based on the GDS ID layer, or between instances defined in CELL_NAMES, to merging them (microns).

Note that even if two regions are overlapping, or within the tolerance limit, if they belong to different hierarchies, they are retained as different cells.

CREATE_PIN_PAIR: categorizes clamps under different types (pdiode, ndiode, pwrclamp, etc) based on user defined connectivity. The PIN_PAIR option can have S(SIGNAL),P(POWER),G(GROUND values. Also, with PIN_PAIR defined under CREATE_PIN_PAIR, tool automatically derives clamp-file including ESD_PIN_PAIR settings.Example:

```
CREATE_PIN_PAIR {
    Pdiode  HIA_DIODE_DUMMY   PIN_PAIR { S P }
    Ndiode  HIA_DIODE_DUMMY   PIN_PAIR { G S }
    PwrdiodeHIA_DIODE_DUMMY   PIN_PAIR { P G }
    Ggnmos  HIA_GGNMOS_DUMMY  PIN_PAIR { P G }
    Pwrclamppwrclamp          PIN_PAIR { G S }
    ..
}
```

CLAMP_CONN_FILE <create_pin_pair file> : specifies CREATE_PIN_PAIR info within a file. Default connectivity is defined under `$APACHER0OT/scripts/clamp_conn_default.txt`

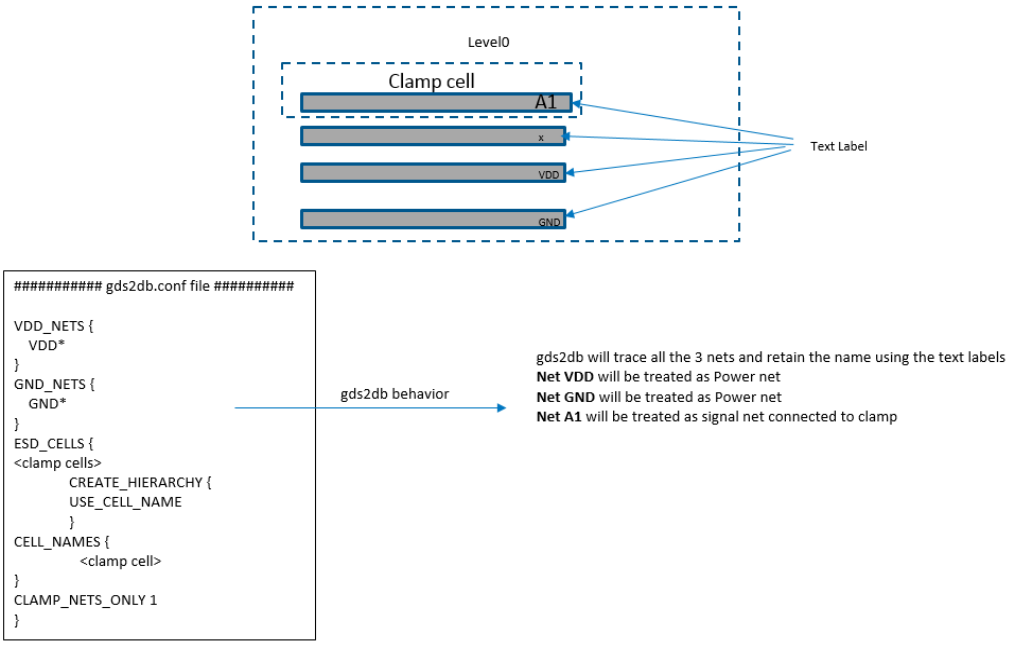
EXTRACT_ALL_NETS

When set to 1, APLMMX will auto-trace all clamp connected nets and name them using highest level hierarchy.

- Pads will be created based on GENERATE_PLOC option for all nets (power/ground/signal)
- Power/Ground nets will be passed on to VDD|GND_NETS section in GSR respectively, whereas Signal nets will be passed on to ESD_SIGNAL_NETS section in GSR.
- “CLAMP_NETS_ONLY 1” under "ESD_CELLS" can be used to retain only the nets that are connected to ESD devices, rest of the signal nets are ignored.
- Nets that do not have any text-labels in GDS and are connected to clamps will be named as “clamp_net_<id>”

Syntax:

```
EXTRACT_ALL_NETS [ 0 | 1 ]
```



With the option CLAMP_NETS_ONLY enabled, only Net “A1” will be retained, Net”X” will be dropped as it does not connect to any clamp_cell.

EXPAND_ESD_LAYERS

When the ESD marker layer is not placed completely over the ESD cell, it causes incomplete ESD Cell modeling. This keyword expands the ESD marker layer in both horizontal and vertical directions (in um) to cover the entire ESD cell, to ensure proper ESD cell modeling. Default: none.

```
EXPAND_ESD_LAYERS {
   <id_layer_1> <extension_Horiz1> <extension_Vert1>
   <id_layer_2> <extension_Horiz2> <extension_Vert2>
   ...
```

```
}
```

Other Keywords

14NM_SUPPORT

When set, provides proper support for 14nm technology designs.

```
14NM_SUPPORT [0 | 1]
```

BUSBITCHARS

Defines the characters used for bus bits in the generated DEF/LEF file. If not defined, GDS2DB uses either BUSBITCHARS defined in the original LEF file, or the default value “[ ]”.

```
BUSBITCHARS <char>
```

COMPRESS_DEF

This GDS2DB keyword performs a gzip on the output DEF file. When this keyword is turned on, the output DEF file is gzipped and given the filename *<top_cell>.def.gz*. The default value is '0 (not gzipped).

```
COMPRESS_DEF [0 | 1]
```

CONTACT_SANITY_CHECKS

When set to 1, GDS2DB performs sanity checks for partially connected vias, etc. Default:1.

```
CONTACT_SANITY_CHECKS [ 0 | 1 ]
```

COVERTICAL_BASE_LAYER_SUPPORT

Enables support of co-vertical base layers which are present between contacts and poly or diffusion layers. Without co-vertical layers, the current will go to diffusion only at contact points which might have some losses and hence these layers are used to spread the current on the complete diffusion layer.GDS2DB has the capability to handle all cases of co-vertical layers connecting with other co-vertical layer, diffusion, or poly. The type should be defined as cmcont in the GDS layer map. Usually these are derived using boolean operation of other layers. The typical syntax to define these in the layer map is:

```
BOOLEAN <co-vertical layer name> cmcont <Boolean expression>

COVERTICAL_BASE_LAYER_SUPPORT [0 | 1 ]
```

COVERTICAL_DIFFUSION_PINS

When set to 1, this will create pins on the diffusion or poly with the pin size being the overlap of the covertical layer and the diffusion or poly. GDS2DB has the capability to handle all cases of co-vertical layers connecting with other co-vertical layer, diffusion, or poly.

```
COVERTICAL_DIFFUSION_PINS [0 | 1 ]
```

DEFAULT_NET_CURRENT_PIN_CREATION

When using the command ‘gd2db’ and the “Extended” option of the GSR keyword GDSII_FILE, by default current pins are created on the

EXTRACTION_STARTING_LAYER and/or CORE_EXTRACTION_STARTING_LAYER based on the vias above that layer. However, when a net does not have any geometry on the lowest via layer (that is, if the EXTRACTION_STARTING_LAYER is M1), then pins are created on VIA12, or the net does not have any vias traced, the default current pins creation methodology for GDS2DB “extended” does not create any pins for such nets. However, if you set 'DEFAULT_NET_CURRENT_PIN_CREATION 1', pins are created on the lowest metal layer. Default: 0.

```
DEFAULT_NET_CURRENT_PIN_CREATION [0 |1 ]
```

ERH2_SUPPORT_FOR_METALRES

When set to 1, model the metal resistors as device resistors and perform better pin snapping like other device resistors. GDS handling will trace through the metal resistor and create a small 1db unit gap between the two nets connected through metal resistors and create pins on either side of the gap. If not specified, then GDS2DB will rely on PSL layer to create pins for the metal resistors and this can lead to inaccurate results if specific layer names are not specified in PSL

```
ERH2_SUPPORT_FOR_METALRES [0 |1 ]
```

FULL_GDSII_INFO

By default, GDS2DB generates the imported GDS data summery to the *adsRpt/GDS/gdsii.info* file. It has information about the first 500 GDS structures (blocks) that have highest number of geometries. In order to see all the GDS structures (blocks), use this keyword. Default 0.

```
FULL_GDSII_INFO [0|1]
```

HANDLE_BOX_IN_BOX

When set to 0 and if some ESD/White-box/Gray-box cell instances are placed inside some other ESD/White-box/Gray-box cell, the geometries of the such inside-cell gets flattened to the outer one and instances of the inside-cell are removed from outer cell. Such cells may be found missing in the GUI or lef-def if all instances meet the same above criteria.When set to 1 and if some ESD/White-box/Gray-box cell instances are placed inside some other ESD/White-box/Gray-box cell, the instances of the inside-cells are brought directly inside top and removed from outer-cell. So, inside-cells' instances are retained and would be found in the GUI and lef-def. Default 0.

```
HANDLE_BOX_IN_BOX [0|1]
```

INTERNAL_DBUNIT

Specifies the internal GDS2DB dimensional unit resolution (multiplier for actual user unit, such as microns). If ‘UNITS’ keyword in DEF_FILE_DEFINITIONS is changed to 1000, this keyword value must match. For designs with metal layers with 4 decimal point coordinate values, setting to 1000 is recommended to resolve the high resolution coordinates.GDS2RH/GDS2DB will set DBUNIT to 20000 automatically for 10nm or lower nodes to increase precision required at these nodes. Default is 2000.

```
INTERNAL_DBUNIT <value>
```

MAGIC_COMPATIBLE

When `MAGIC_COMPATIBLE` is set to 1, GDS2DB treats reference array (AREF) in the GDS file in a manner compatible with the Magic tool. The default is 0.

```
MAGIC_COMPATIBLE [0|1]
```

MERGE_DUP_STRUCTURE

When turned on, merges multiple GDS files with the same name. By default, the most recently processed structure replaces the previously-processed structure, when there are multiple GDS files with the same name. For merging, set the value to “yes”, ”1”, or ”true”.

```
MERGE_DUP_STRUCTURE [ 0 | yes | 1 | true ]
```

MEMORY_SAVING_MODE

The MEMORY_SAVING_MODE keyword enables reduced peak memory use when more than 16 GB may be needed, and reduces memory swapping. The default is 1 (On).

```
MEMORY_SAVING_MODE [ 0 |1 ]
```

OPTIMIZE_BOOLEAN_OPERATION

When set, GSD2DB optimizes Boolean operations by automatically creating nested Boolean layers to handle complex expressions, which improves GDS2DB runtime. This is supported only in “NATIVE_GDS_SUPPORT 2" mode. Default is 0.

```
OPTIMIZE_BOOLEAN_OPERATION [0 | 1]
```

OUTPUT_DIRECTORY

To write out the DEF/LEF files to a different directory than the current run directory, use the following specification:

```
OUTPUT_DIRECTORY <output_directory>
```

or use the GDS2DB command options ‘-out_dir <dir_name>’ or ‘-log <dir_name>’.

OUTPUT_APACHECELL

When `OUTPUT_APACHECELL` is set to 1, “_APACHECELL” is appended to original top cell name. When set to 0, cells are generated in COMPONENTS session with the original top cell name. The default is 1.

```
OUTPUT_APACHECELL [0|1]
```

OUTPUT_DETAILED_DEF_PIN

The DEF output produced by GDS2DB does not have physical geometries for the power and ground pins. By turning on OUTPUT_DETAILED_DEF_PIN, the physical geometries of power and ground pins in the LEF file are included in the DEF file. The size and location of these pins are obtained from the user-provided LEF file of the macro cell. Default: 1.

```
OUTPUT_DETAILED_DEF_PIN [0 | 1 ]
```

PREFLATTEN_CELLS

Specifies that GDS structures (blocks) are flattened before hierarchical net tracing.

```
PREFLATTEN_CELLS {
  Cell1
  Cell2
}
```

This keyword can also be used to flatten the top cell as well, as follows:

```
TOP_CELL top_cell
PREFLATTEN_CELLS {
    top_cell
```

```
}
```

Note: pre-flattening the top cell has a run time impact.

SLOT_VIA_PROPERTIES

Merge the slotted via and report the total current flowing through the via (and compare against the rule in tech file). The keywords needed to trigger this EM check are below

```
SLOT_VIA_PROPERTIES {
  SLOT_VIA_SPACING 0
  CLUSTER_SLOTTED_VIAS 1
}
```

UNIFORM_OUTPUT

Creates consistent geometry and pin order in the database, eliminating potential small QoR randomness changes. This keyword can be turned off to improve run time by 10% to 20%. Default On (1).

```
UNIFORM_OUTPUT [ 0 | 1 ]
```

Checking and Debugging GDS2DB Modeling Results

Debugging GDS-related issues in Totem

Not all GDS -related issues require you to be expert with layouts. Some issues can be resolved without opening the GDS to view. However, it is always beneficial to understand the GDS layout and hierarchies in order to perform the analysis in a more informed way. You can use the OA viewer built into Totem to view the GDS hierarchy, or use other free tools such Klayout to view GDS files.

GDS debugging - Shorts

GDS2DB traces the power and ground net geometries from the GDS file and converts it into a Totem-compatible format. This translation usually involves tracing the geometries from a particular label name or a location.

It is common to see shorts reported during the GDS2DB translation. This is because GDS2DB relies heavily on user input such as labels, coordinates, and layer mapping, and there can be inaccuracy in these inputs. To debug the issues, use one or more of the following approaches.

Method 1: Find misplaced labels

Always ensure that you have the keyword ‘USE_TOP_LEVEL_TEXT_ONLY 1’ in the GDS configuration file to avoid any shorts due to label mismatching at lower hierarchies. In this method, GDS2DB picks the x,y location on <layer>:<datatype> and uses it as starting point for tracing, which requires accurate coordinates, but needs no knowledge of what text labels are present. If there are no labels at the top level, but the labels in the topmost metal layer are reliable, then you can change the layer map file to remove the labels in the lower hierarchies.

Method 2: Find layer number and data type mismatches

In this method, add one data type at a time in your layer map file for each layer. GDS2DB searches for text vdd on that layer and datatype. So this method restricts the starting point to available vdd texts. However, there could be several vdd texts, so pick the one which is at an x,y location on <layer>:<datatype>. This is more reliable, since it requires that text labels with the given name and location are present.

In general, if text labels are available and trustworthy, it's a good idea to use method 2. However, there is a possibility of errors, since the x, y location must match the vdd text location. And if you are not sure about the available text labels, method 1 should be used.

You must make sure to have at least one data type in your layer map file. For example, you might have

```
metal1 m 31 131
```

in this case, GDS2DEF traces all the data types (drawing, pin, label, boundary, ...) belonging to layer 31. In some cases, this might cause shorts. Make sure your LVS stream layer mapping file and the GDS2DEF layer map file is exactly same. Example:

```
metal1 m 31:0 131:0
```

Always specify the data types used for different metal/via layers in layer map file. Custom designs providing layer numbers without a data type are very difficult, since the same layer may have data types for metal fuse or metal resistor layers, and considering those layers would definitely cause false violations.

Method 3: Coordinate-based tracing

If using methods 1 and 2 does not resolve all the problems, try pure location-based tracing in the GDS2DB configuration file, as shown below. This ensures that only one tracing point is used for the actual net tracing.

```
VDD_NETS {
VDD {
- @ <layer:datatype> <x> <y>
}
}
```

Do not use this method if you are not sure if that location has a label. In the case below, GDS2DEF tries to find labels in the given x,y location. If it cannot find any label, the process fails. So, to be on the safe side, just use “- @ <layer:datatype> <x> <y>”, when you try to do pure location-based tracing, as follows.

```
VDD_NETS {
VDD {
   vdd @ <layer:datatype> <x> <y>
}
}
```

Approach 4: Layer by layer debug

When you cannot resolve issues with other methods, then try to have only a few layers in the layer map file and start tracing the GDS. In this way you can actually find the layer in which you are seeing some shorts. For example, try three layers at a time from topmost layer to run GDS2DEF.

```
M5 m 35 135
v5 v 36 136
M6 m 37 137
```

If this runs through correctly, then add two more layers below M5, and then run through the flow.

```
M4 m 33 133
v5 v 34 134
M5 m 35 135
v5 v 36 136
M6 m 37 137
```

This approach helps you find the layer that is shorting.\

Approach 5: Find Pin mismatches

There can be cases in which shorts occur because of false pin locations in the *<>.gdsmmx* file. GDS2DB creates a small pin geometry based on the location specified in the *<>.gdsmmx* file. If there are mismatches between the netlist and layout, these pin locations may be incorrect, which may cause false shorts. A difference in scaling factors (see APLMMX keyword NETLIST_XY_SCALE in section "NETLIST_XY_SCALE", page 3-72) between the schematic netlist and layout may also cause shorts in the GDS translation.

If you see shorts in GDS2DB, try to run GDSMMX first to make sure that there are no shorts. If GDSMMX flags the shorts in the design, then it is likely a real short, or an issue with the layer map file. If GDSMMX cannot handle the design, try flattening the GDS in Totem2 GUI File ' Export Flattened GDS, and then use it in GDS2DB to verify the shorts.

Approach 6: Find ESD/Fuse cells that cause intentional shorts

In some designs you may have real intentional shorts, such as metal fuse cells and ESD clamp cells. In this case, the LVS may still pass, because you may have specific rules for the LVS engine to ignore such shorts. But GDSMMX/GDS2DEFmay detect these valid objects as shorts. In this case, add these cells in IGNORE_CELLS list in your GDS2DEF configuration file and then rerun GDSMMX. This should resolve the short issue.

General GDS debugging tools

OA viewer

Not all GDS-related issues require users to be expert with layouts. Sometimes you can debug the issue without opening the GDS view. However, it is always beneficial to understand the GDS layout and hierarchies to do the analysis in a more informed way. You can use the OA viewer built into Totem to view the GDS hierarchy.

Other free tools such as “Klayout” also can be used to view GDS files.

Viewing layouts in commercial viewers

Commercial layout viewers such as Virtuoso Layout, Laker or Calibre-RVE can be used to stream in a GDS and view the layout. Refer to the latest user guides for these tools to understand how to import and view GDS files.

Viewing layouts in Totem

If you do not have a design loaded in Totem, you can use **File ->Import GDS.** If you already have a design loaded, then you can load the GDS and layer map using the cross-probing option in the configuration panel. In cross-probing mode, all the layers specified in the layer map are modified into layer names in the LSW window. The rest of the layers are specified as layer numbers.

Chapter 4
Static and Dynamic Analysis

Introduction

After performing basic setup and characterization steps as described in Chapters 2 and 3, different design types require application-specific modifications to the setup procedure. See Chapter 5 for the steps needed for your design type before starting the analysis phase.

Totem can perform static analysis and dynamic analysis. There are three basic types of static power assignment available, as shown in Figure 4-1.

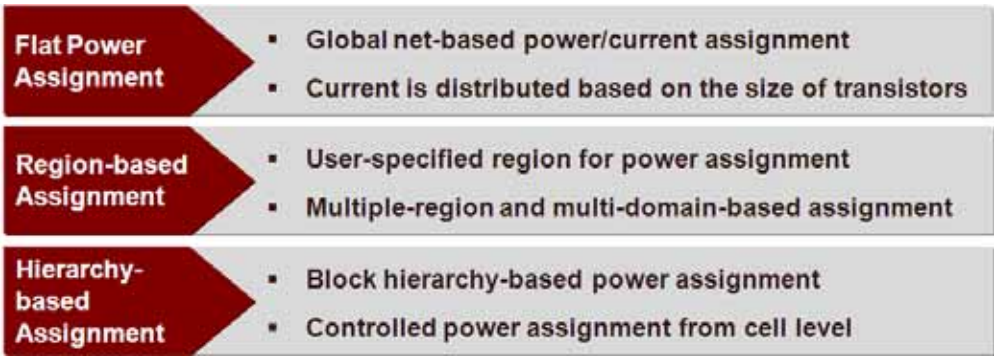


Figure 4-1 Totem static power assignment methods

For more information on setup for each of these power assignment methods, see section "Setup Details for Static-based Analysis", page 4-159.

Power Calculation

Power calculation uses information from a number of design files to evaluate the average cycle power consumption of all cell instances for both flat and hierarchical designs. The calculated power is used for both static and dynamic analysis. It then displays the power distribution results in a graphical “map” of the design. Power calculation is typically performed *once for each design*, and the results are summarized in several power reports. The power reported by power calculation is “ideal” power, based on the supply voltage specified in the VDD_NETS keyword in the GSR, or otherwise the nominal voltage specified in the LIB file. The computed total power also includes power due to instances not connected to power nets or instance power pins connected to non-Vdd nets, hence the actual power may be different due to instances not connected in the design.

The following GSR files are used to specify the key design parameters and set up power calculation:

- STA_FILE
- CLOCK_ROOTS
- FREQUENCY

See section "Global System Requirements File (*.gsr)", page C-772 for descriptions of the use of these keywords.

Power Calculation Procedure and Results Evaluation

With all design data imported and the appropriate GSR keyword values set, you can perform power calculation from the TCL command line or using the GUI. The TCL command is:

```
perform pwrcalc
```

If you have already run power calculation on the design and have made no changes, you can import the calculated power directory data from your previous calculation by using the menu command **Static > Power > Import,** and select the previous power directory, such as *adsPower*.

Or perform the import using the TCL command:

```
import power <Power_Dir_Name>
```

Or perform power calculation using the GUI and then evaluate the results, following the GUI-based steps below.

1. Calculate the power using the **Static > Power > Calculate** or **Dynamic > Power > Calculate** command.
2. Review the power summary in the file *power_summary.rpt* in the *adsRpt* directory, as shown in the Figure 4-2 example. The Power Summary Report recommends the simulation time needed to capture a majority of power events, and lists the calculated total_pwr, leakage_pwr, internal_pwr, switching_pwr and percentage of total power consumption by frequency domain, by power domain, and by cell type. Note that imported power values are scaled during power calculation to meet instance-specific power consumption numbers.

```
------------------------------ Power Summary Report ----------------------------------------------
Recommended dynamic simulation time, 5000psec, to include 100%
of total power for DYNAMIC_SIMULATION_TIME in GSR.

INFO: Importing user specified power file: demo.inst.power
INFO: Instances not specified in this file will have zero power.
INFO: 100% of instances specified in instance_power_file have corresponding
instances in the design.
0 out of 4142 instances do not have corresponding instance in the design.
INFO: For complete list of PWR-121 and PWR-122 WARNINGS, please see adsRpt/
apache.inst_pwr_file.mismatch.

The power is based on the INSTANCE_POWER FILE: demo.inst.power
Totem honors instance-by-instance power as specified in the above file.
Individual components of power, like switching power, internal power,
however, are calculated by Totem and may have been scaled to
meet the total instance-specific power number.
```

```
Power of different frequency (MHz) domain in Watts:

Frequency    total_pwr leakage_pwr   internal_pwr switching_pwr  %_total_pwr
4.000e+02    1.729e-02  1.704e-04     1.107e-02     6.052e-03     8.333e+01%
2.000e+02    3.4595e-03  2.1869e-05   3.1000e-03    3.3758e-04    1.666e+01%
0.000e+00    0.000e+00   0.000e+00    0.000e+00     0.000e+00     0.000e+00%

Power of different Vdd domain in Watts:
Vdd_domain   total_pwr leakage_pwr  internal_pwr  switching_pwr %_total_pwr
VDD (1.1V)   2.075e-02  1.922e-04   1.417e-02     6.389e-03      1.000e+02%

Power of different cell types in Watts:
cell_type    total_pwr  leakage_pwr  internal_pwr  switching_pwr %_total_pwr
combinational 7.111e-03   1.221e-04   1.754e-03      5.235e-03   3.426e+01
latch_and_FF  4.292e-03   3.890e-05   3.323e-03      9.301e-04   2.067e+01
memory         0.000e+00   0.000e+00   0.000e+00      0.000e+00   0.000e+00
I/O            0.000e+00   0.000e+00   0.000e+00      0.000e+00   0.000e+00
clocked_inst  9.352e-03   3.119e-05   9.096e-03      2.246e-04   4.505e+01
decap          0.000e+00   0.000e+00   0.000e+00      0.000e+00   0.000e+00
where clocked_inst are instances that cannot be classified as latch_and_FF,
memory, or I/O, but have clock pin(s).

Total chip power, 0.020757 Watt including core power and other domain power.
Total clock network only power, 0.003225 Watt. Total clock power, including
clock network and FF/latch clock pin power, 0.005992 Watt.
```

Figure 4-2 Example Power Summary Report

Some of the information that can be checked in the power summary report is:

- Does the total power consumption make sense?
- Are clock network and the clock power values reasonable?
- Is the power reported by clock frequency what would be expected?

Having several elements that consume a lot of power close to several elements that consumes little power may not create a voltage drop problem. But having an area where many elements consume a lot of power can create a voltage drop problem if the area is not well supplied. The power consumption maps give important feedback regarding the power distribution, i.e., the quality of the placement/floorplanning vs. power consumption.

Some aspects of the power calculation results to investigate:

- What is the total power consumption in the design?
- Is the overall power demand well distributed?
- Will the power pads handle this load safely?
- How is the power balanced between the different frequency domains?
- How much power is used in the clock network?

You can click on any instance to see a text report of its power consumption, as well as its associated capacitance and applied voltage.

Power Grid Resistance Extraction

After power calculation, the next step is to perform network resistance extraction, using either TCL or GUI commands. At technology nodes now being used for design, very precise estimates of wire resistance, and therefore wire thickness, is necessary. This effect of metal density on resistance calculation is described in the following section.

Metal Density Calculation

To obtain very accurate resistance values for wire segments, both width and thickness values must be accurately determined. To achieve an accurate estimate of wire thickness, an accurate assessment of the metal density for all metal geometries in the region of the wire is needed, because of the effects of CMP on wire thickness. The procedure for estimating metal density around wire segment is as follows.

1. Read geometries from DEF/LEF, or from GDS2RH/DB. Include all geometries from LEF/DEF (NETS, SPECIALNETS, PINS, MFILL), as well as all geometries from GDS2DB (including MFILL).
2. Merge and flatten all metal geometries per layer, which eliminates redundant geometries from different hierarchies or from GDS.
3. Calculate the metal density over the whole design in small gridded regions that contain wires.
4. Perform extraction. Calculate the density of segments based on average density of regions around each segment.
5. By default each uniform wire geometry uses the density from an average of all regions that the wire passes through.
6. When you require the most accurate resistance calculation, such as for long wires that go through multiple regions of varying metal density, set the GSR keyword LONG_WIRE_RES_CALC 1. In this case, the metal density of every region that contains the long wire geometry is used in the calculation.
7. Then calculate the final wire thickness based on the calculated metal density-- for example, using the POLYNOMIAL_BASED_THICKNESS_VARIATION Tech file keyword setting.

TCL Command R Extraction

After power calculation, the next step is to perform network extraction, using either TCL or GUI commands.To run static resistance extraction for IR drop analysis using the TCL command, execute:

```
perform extraction [-power | -ground]
```

To run RLC extraction for dynamic voltage drop analysis using the TCL command, execute:

```
perform extraction [-power | -ground] -l -c.
```

The ‘`perform extraction`’ command builds connectivity and performs extraction for the selected elements of the selected power/ground nets.

GUI Extraction

To use the GUI commands to perform extraction on both VDD and VSS, use the following steps:

1. For static IR analysis, use the extraction command **Static -> Network Extraction**. From the extraction form displayed, select resistance (R), and both VDD and VSS.
2. For faster static analysis of either the power or the ground network, either VDD or VSS can be extracted separately. However, both VDD and VSS must be selected for dynamic analysis.

The key information you can get from the extraction step is:

- Are all power and ground nets connected to a source? If not, the a warning message is displayed: “Net VDD in cell not driven by any pad”.
- Are there any missing connections? (reported in the file *adsRpt/apache.*.unconnect*).

Examining Power/Ground Grid Weakness

Excessive voltage drop can occur due to weak power/ground grid structures. Totem can identify problems in P/G structures early in the design cycle after placement and CTS is completed, even without STA and SPEF files. P/G weakness analysis can report two different measures of grid weakness:

- perform gridcheck - an estimate of the upper bound on grid resistance for all instances in the design, normalized to the highest instance grid resistance
- perform res_calc - the calculated effective grid resistance for a specified number of instances in the design

For gridcheck, an instance that has a weakly connected Power or Ground network shows up as high impedance in the generated report. Using the GUI you can also review power grid resistance maps showing the distribution of power and ground resistance across the design. Also, pins not connected to VDD or VSS are listed as “floating” in the PG Weakness report, and are listed together at the end of the report.

Note that you do not need to run power calculation or static analysis prior to doing this assessment. Once the design is imported and P/G extraction is performed, the design environment can generate the resistance report using the TCL interface. This can be done early in the design cycle when detailed routing or timing information is not available.

The procedure for generating and evaluating a P/G grid weakness report is as follows:

1. **Normalized resistance estimation.** On the TCL command line execute:

   ```
   perform gridcheck -o <output_file_name> ? -limit <line_limit>?
   ```

   The format of the resistance report displays R_{VDD} and R_{VSS} for each instance or arc as percentages of the total P/G resistance distribution, as shown in the example report below (Figure 4-3). For multiple P/G arc designs, a*dsRpt/apache.gridcheck* is constructed as follows:

   a. The arc that has the highest relative R value is displayed first and continues up to the specified line limit.

   b. The rest of the arcs are displayed up to the specified line limit, or until the relative R becomes smaller than the last displayed relative R in the previous arc that reached the line limit.

```
------------------------------------------------------------------------
    # Power/Grnd arcs listed separately, there are 2 P/G arcs w/ non-zero R.
    #
    # Max. Resistances (%) of Power/Ground Arcs:
    # Arc1 ( VDD_INT VSS) 100.0000
    # Arc2 ( VDD VSS) 93.7410
     { Power/Ground Arc1:
    # Total VDD_INT(%) VSS(%) Location (x y) ArcID Instance_name
```

```
100.000  49.5167 50.4833 3316.37 3950.55 Arc1 inst_129973/inst_366358
99.6966  50.8138 49.1862 3318.21 3965.32 Arc1 inst_129973/inst_366355
99.5191  50.4213 49.5787 3316.83 3961.62 Arc1 inst_129973/inst_366346
...
}
{ Power/Ground Arc2:
# Total  VDD(%)  VSS(%)  Location (x y)  ArcID   Instance_name
93.7410 50.0202 49.9798 2099.90 3888.64 Arc2 inst_129228/inst_465808
92.9774 50.7174 49.2826 2131.64 3903.41 Arc2 inst_129228/inst_412486
92.9138 50.8630 49.1370 2131.18 3903.41 Arc2 inst_129228/inst_412485
92.8791 50.0223 49.9777 2131.64 3896.03 Arc2 inst_129228/inst_466552
...
}

# Floating Instances: Location (x y)     Name
floating        0       977.5   io/pad/VSSC/extra/left/4/adsU1
floating        10000   2956.24 io/pad/VDDC/extra/right/23/adsU1
floating        5435.09 191     io/pad/extra/VDDC/11/adsU1
floating        3935.09 191     io/pad/mpi/VDDC/2/inst/adsU1
floating        7300.05 0       io/pad/extra/VSSC/15/adsU1
...
-----------------------------------------------------------------------
```

Figure 4-3 Sample P/G Weakness Report

The instance P/G resistance, R_{inst}, is normalized such that the instance or arc with the highest total effective resistance is assigned a value of 100 and the instance or arc with the lowest total effective resistance has a normalized value of 0, using the following equation:

$$(R_{inst} - R_{min})/(R_{max}, - R_{min})*100 \quad \text{(EQ 1)}$$

where, R_{inst} is the total effective P/G resistance for an instance ($R_{VDD} + R_{VSS}$), R_{min} is the minimum value of all R_{inst} and R_{max} is the maximum value of all R_{inst}.

The first column of the report, Total, lists the normalized resistance for every cell as a percentage of the maximum R_{inst}. The second column lists the percentage of R_{VDD} grid resistance to the total effective resistance for that instance. For multiple P/G arcs, the % resistance numbers are normalized to the highest resistance of all the domains.

The third column provides a similar number for R_{VSS}. The report values are sorted based on the highest total relative resistance percentage. The sum of ‘VDD(%)’ and ‘VSS(%)’ entries is always 100.

A very large imbalance between R_{VDD} and R_{VSS} (in percentage terms) indicates a significant weakness in the P/G grid at that instance, particularly for instances that have large values of total R.

The list of P/G weakness instances also can be viewed without the TCL 'perform gridcheck' command by using the GUI pulldown menu **Results -> List of Weak PG Instances**, which brings up an ordered list of high total resistance instances.

Clicking on any entry and on 'Go To Location' zooms to and highlights the weak P/G instance. The list can be sorted based on Total resistance, VDD(%) or VSS(%).

2. View Resistance Maps

   The resistance maps can be displayed using the GUI menu option **View -> Resistance maps**

The Resistance maps display either Total Resistance, R_{VDD} , or R_{VSS}, which are the relative effective resistances for all instances in the design. To see the meaning of each color, click on the 'Set Color Range' Configuration button at the right side of the GUI. A 'Resistance Color Map' dialog is displayed, indicating the resistance “gradient” (normalized resistance) for each color. The display and the resistance ranges can be changed using the dialog.

3. The **View -> Nets** menu option can be used to select the resistance maps to be displayed by net.

4. To view a grid problem area, you can highlight the weak instances in GUI using the resistance report described above. Use the following TCL command to highlight weak P/G instance:

   ```
   select addfile high_VDD_res_instance
   ```

   to highlights instances with weak VDD structures, or

   ```
   select addfile high_VSS_res_instance
   ```

   to highlight instances with weak VSS structures.

5. **Effective Grid Resistance Calculation.** Reviewing the highlighted high impedance instances, you can now start debugging the causes of weak P/G structures in more detail. An important tool in finding out more about weak grid areas is the ‘perform res_calc’ TCL command (see section "perform", page D-952, for command syntax description).

   The ‘perform res_calc’ command calculates the effective P/G grid resistance from all pads to selected instances or locations, and provides absolute resistance values. The default invocation, without any options, creates a resistance report of the worst instances, as indicated by an initial quick estimation. Using the options ‘-instance’, ‘-inst_file’, or ‘-box’ allow the effective resistance of particular instances or identified weak areas of the grid to be investigated in more detail. The -cell_file <CellFile> option specifies a text file that contains the cell names, one per line, for res_calc to compute the equivalent grid resistances. A sample ‘perform res_calc’ output file is shown below.

   ```
   # Ohm   Location(x y)   Layer   Net     Instance
   4.54617 1005.4  8755.2  MET3    VSS     inst_1234
   4.39744 1055.6  8855.3  MET5    VDD     inst_4134
   3.32929 1855.7  8455.6  MET3    VDD     inst_2324
   ...
   ```

6. **Special Node Resistance Reports.** Two other options allow you to select the type of node resistance reports, for standard cells or macro blocks, as follows:

   ```
   perform gridcheck ?-stdcell [ ave| min| max| all| none ]?
             ?-macro [ ave| min| max| all| none ]?
   ```

   The option ‘ave’ is the average resistance for all nodes in each instance selected, and is the default. For the 'min' and 'max' options, the node with the minimum or the maximum resistance value for the instance is reported, instead of the average of all nodes in the instance. The 'none' option is used to eliminate reporting on all standard cell or macro instances. For 'all', the largest 5000 (default) node resistances in the design are reported.

   The report format is like 'perform res_calc', where VDD/VSS values are listed point-by- point, without pairing up. An example report follows:

   Example all-point gridcheck resistance listing for instances:

   ```
   Resistance(%) Location( x y) Layer Net   Instance
   87.4927 4459.77 443.855 METAL3  VDD     inst_129747/adsU1
   87.3918 2839.76 443.855 METAL3  VDD     inst_129747/adsU1
   ```

```
42.4231 3741.59 609.345 METAL3  VDD     inst_129747/adsU1
33.1267 1874.49 119.18  METAL3  VDD     inst_129747/adsU1
```

Defining Pad and Package Parameters

Before performing static IR drop and EM analysis, the pad, package wirebond or flip-chip bumps and associated electrical package parameters must be defined.

Command Line Procedure for Package Modeling

The pad, wirebond or flip-chip bump, and package parasitics are typically specified in the Totem *.tech* file. The units used for the package TCL commands are: R in Ohms, C in picoFarads, and L in picoHenrys.

The following three TCL commands define simplified lump models for all pad RC, wire-bond/bump RLC, and package RLC circuits, respectively.

```
setup pad [-power | -ground] [-r <R_Ohms> | -c <C_pF>]

setup wirebond [-power | -ground ]
    [-r <R_Ohms> | -l <L_pH> | -c <C_pF>]

setup package -r <R_Ohms> -l <L_pH> -c <C_pF>]
```

where

-power -ground : selects which P/G net to define

-r <R_Ohms> : specifies equivalent resistance value in Ohms

-c <C_pF> : specifies equivalent capacitance value in picoFarads

-l <L_pH> : specifies equivalent inductance value in picoHenrys

To specify individual bump/pad RLC values, see the PLOC file specifications in section "Pad Location File (*.ploc)", page C-915. If you need a more accurate Spice package subcircuit for your design, see Chapter 7, "Package and Board Modeling", for more information about package subcircuit modeling.

To set up your package parameters such that you can easily look at the effects of different package designs on power integrity, after extraction you can use the TCL command ‘setup pss’, which defines PLOC and package subcircuit files to be used in simulation, using the follow syntax:

```
setup pss -pad_file <filename> -subckt <pss_ckt_name>
```

where

-pad_file <filename> : specifies a PLOC pad definition file that must have a *.ploc* extension and be in PSS PLOC format.

-subckt <pss_ckt_name> : specifies the Spice package subcircuit file

After running Totem with one set of PLOC and package subcircuit files, you can then modify one or both of the files, rerun the ‘setup pss’ command, and then rerun Totem to see the effects of the new package data.

GUI Procedure for Package Modeling

1. Set the package and pad constraints by selecting the **Static -> Package Constraint** command. The data form is displayed, as shown in Figure 4-4.
2. Enter appropriate RLC pad and package values for your design.

The package model RLC values are needed to include the impact of package parameters on the circuit analysis. The package can have a significant impact on voltage drop. If you

do not have accurate RLC values from testing or Spice models, you can provide reasonable default values to see the effects on the voltage drop results.

Note: For static analysis only, R values are sufficient. However, for dynamic analysis good L and C values are necessary for accurate results.

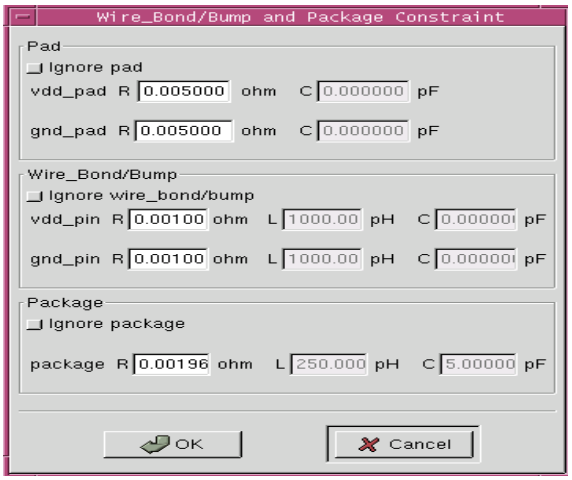


Figure 4-4 Pad, Wire-bond/Bump, and Package constraints form

Package Compiler Utility

The Totem Package Compiler is a versatile utility that checks the die-package interface, as follows:

- checks package Spice syntax
- checks package RLCK passivity
- calculates effective package Inductance for each voltage domain
- matches die and package pins and creates an annotated PLOC

Totem Package Compiler makes use of the Chip Package Protocol (CPP) header information to determine the following:

- identifies package and die pins
- identifies which nodes belong to the die side and which belong to the PCB side
- uses CPP header data to differentiate between power nets and ground nets

For details on using Package Compiler, see section "Chip-Die Mapping Using Package Compiler", page 7-239.

Setting up and Running Static Analysis

Before running static analysis you must create an appropriate GSR control file and a TCL command file. These operations are described in the following sections.

GSR File

See section "GSR File Keywords", page C-773, for details on using the GSR keywords and their options. Following is an example of a typical GSR file for running static IR/EM analysis:

Sample GSR File

```
TECH_FILE user_data/RAM256.tech
PAD_FILES {
    user_data/RAM256.ploc
}
GDSII_FILE {
   RAM256 gds2db.config full top
}
VDD_NETS {
  VDD 1.8
}
GND_NETS {
   VSS 0
}
BLOCK_POWER_FOR_SCALING {
  FULLCHIP RAM256 1.2 VDD
  FULLCHIP RAM256 0.9 VSS
}
FREQ 1e08
```

Other Files

The format for information in a custom LIB file is as follows:

```
cell <macro_name> {
    type memory
}
```

Pads are defined in a Pad file, which has the following data format:

Format 1 (use when there is no package netlist):

```
<die_pad_name> <X-coord> <Y-coord> <layer> <POWER | GROUND>
```

Format 2 (use when there is a package netlist):

```
<die_pad_name> <X-coord_um> <Y-coord_um> <layer>
    <power/ground_domain_name> <Spice_pkg_port_name>
```

TCL Command File for Static Analysis

A sample TCL command file for Static analysis is shown below:

```
import gsr user_data/RAM256.gsr
setup design
perform pwrcalc
perform extraction -power -ground
perform gridcheck
perform analysis -static
```

Running Static Analysis

.To run Totem-MMX static analysis using a TCL command file, execute the command:

```
totem2 -f static.tcl
```

For standard MMX analysis, skip to section "Setting Up and Running Totem Dynamic Analysis", page 4-163.

Setup Details for Static-based Analysis

Flat Static Analysis using Block Power for Scaling

Refer to section "BLOCK_POWER_FOR_SCALING", page C-856, for details on assigning power using the GSR keyword BLOCK_POWER_FOR_SCALING.

Region-based Static Analysis

For designs that are in early stages and do not have complete dynamic simulation environment information, the BLOCK_POWER_ASSIGNMENT GSR keyword can be used to define power parameters for regions, hierarchical blocks, specific pins, and also the top level.

Block power estimation with BLOCK_POWER_ASSIGNMENT

Using the BLOCK_POWER_ASSIGNMENT GSR keyword, power values can be assigned and others estimated by power calculation based on the information provided. You must provide correct and complete power calculation data so that power for the block can be calculated accurately. Use one line in the BPA specification for every domain (net) in the block. The area over which this assignment is performed comes from the specified REGION bounding box, or from the specified BPA instance placement and bbox.

You can also exclude regions from power assignment in early analysis for designs at a stage where place and route is complete for some hierarchical blocks, and you want to use power calculation data for the completed blocks and use power assignment for rest of the design.

For power/current assigned to hierarchical elements in a design, power/current assigned to child elements are included in the parent‘s assignment. So a parent block’s power assignment is the sum of the power assigned specifically to its children and also the power assigned to areas that are outside of its children. Multiple layer specifications can be provided for the same instance and net, in which case the power or current is added and distributed equally between nodes on the P/G grid on the specified layers. If ALL layers are specified, all nodes on the grid are considered.

BLOCK_POWER_ASSIGNMENT supports MMX pin-based region power assignment for MMX instances that have many P/G pins inside to represent transistors. So power is only distributed to P/G pins (regular region-based BPA assigns power to nodes on the given layer within the region). You can use this feature to change the power distribution inside a MMX instance for static analysis only. The basic BPA syntax for MMX designs is:

```
BLOCK_POWER_ASSIGNMENT {
    <DEF_inst_name> MMX_PIN ALL <domain_name> <power/current>
    <DEF_inst_name/region_name> MMX_REGION ALL <domain_name>
        <power/current> <llx> <lly> <urx> <ury>
    <DEF_inst_name/region_name/...> MMX_REGION ALL <domain_name>
        [<power_W>|<gnd_I> | -1 ] <llx> <lly> <urx> <ury>
}
```

You can assign power:

- to all transistor pins inside MMX instances for the entire domain by using the MMX_PIN option in BPA
- to regions inside MMX instances by using the MMX_REGION option and using “/” to indicate hierarchy in the first column.
- hierarchically by defining regions inside other regions with the MMX_REGION option. Use “/” to indicate hierarchy.

- the power calculation engine can estimate power and current for early stage blocks by setting their BLOCK_POWER_ASSIGNMENT values to -1, which means that power calculation determines the appropriate power/current based on other information available, such as toggle rate, BPFS and characterization.

An example region-based design scheme and the associated power/current assignments are shown in Figure 4-5. The BPA power assignments for VDD and current assignments for VSS are as follows:

```
BLOCK_POWER_ASSIGNMENT {
    ABC/ram64x8 MMX_PIN all VDD 0.18
    ABC/ram64x8/R1 MMX_REGION  all VDD 0.04 1789 980 1822 1031
    ABC/ram64x8/R1 MMX_REGION  all VSS 0.02 1789 980 1822 1031
    ABC/ram64x8/R2 MMX_REGION  all VSS 0.05 1000  300  1100 320
    ABC/ram64x8/R2/R3 MMX_REGION  all VSS 0.02 1800 1000 1810 1020
```

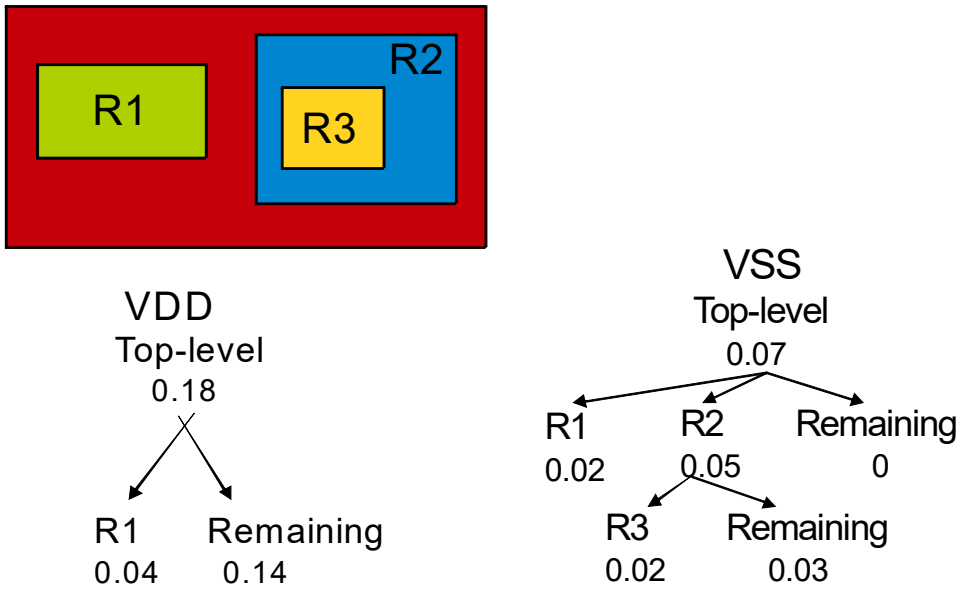


Figure 4-5 Region-based BPA power/current in static analysis

Usage notes:

1. Unlike metal/via region-based BPA, pin-based region BPA does not create an instance hierarchy for the given region. It only redistributes and adjusts the transistor pin power/current inside the MMX instance, while keeping the total power/current of the MMX instance unchanged.
2. Pin-based region BPA and metal/via based region BPA can be used in the same design at the same time.
    a. You can apply pin-based BPA to MMX instances while assigning metal/via-based BPA to place outside MMX instance.
    a. You can assign pin-based region BPA and metal/via based region BPA to the same MMX instance. But the metal/via-based region BPA cannot cover any MMX transistor pins, because it would conflict with the assumptions. Use MMX_REGION to handle such cases. In the above example, pin-based region BPA is used while assigning power/current to “MHE/MCFIFO/ram64/adsU1/RR” with metal/via existing, but the metal/via region BPA does not include any MMX transistor pins.

Usage restrictions. There are several restrictions on the MMX pin-based region BPA methodology that are automatically checked to avoid errors:

1. an MMX instance cannot be inside an MMX_REGION.
2. an MMX_REGION must be defined inside an MMX instance.
3. an MMX_REGION can be under another MMX_REGION only if it belongs to the same instance.
4. regions cannot overlap

For full BPA syntax, usage details, and other examples, see section "BLOCK_POWER_ASSIGNMENT", page C-774, and also the *RedHawk User Manual.*

IDSAT-based power assignment

If you do not have power parameters for all power domains for BLOCK_POWER_ASSIGNMENT in Totem static analysis, you can also use the APLMMX IDSAT characterization method to estimate domain power. The steps are:

1. Set 'CHARACTERIZE_IDSAT 1' in the APLMMX configuration file to characterize each transistor Idsat current. For this method you must provide a device model library in order to perform characterization.
2. Set the 'TOGGLE_RATE <toggle_rate>' APLMMX keyword to scale the idsat current of each transistor. APLMMX sums the scaled idsat current for all transistors to get the total power for the blocks in each domain, based on the circuit topology and connectivity. The pratio (power ratio) of the transistors is also passed to Totem later for static analysis. The current of each transistor is distributed based on the power specification in BLOCK_POWER_ASSIGNMENT and the pratio of each transistor in the database.
3. A template for the BLOCK_POWER_ASSIGNMENT keyword is generated in the file *adsRpt/<design>.bpa_file* to use in static analysis. In this static flow, you do not need to provide BLOCK_POWER_FOR_SCALING power specifications.

Netlist-Driven Hierarchical Static Analysis

If your input netlist hierarchy matches the GDS physical hierarchy, you can use the following MMX netlist-driven hierarchical analysis flow, which allows you to assign power for all selected subckt hierarchies. If you have a power specification for each sub-block, you can use the GSR keyword BLOCK_POWER_FOR_SCALING to specify the power for each subblock. The subblock power (DC current) is then distributed to each transistor pin based on its PRATIO (power ratio) derived from the device electrical properties, such as width and length. The diagram of Figure 4-6 below shows the general scheme. Key elements of the flow are:

- Supports individual assignment of macro power to multiple VDD domains and macro current to multiple VSS domains.
- Automatically-distributed current/power to custom macros whose power are not assigned, based on the device size.
- Mandatory top-level custom macro power/current assignment.
- User-assigned custom macro power/current is not altered by MMX.
- In the figure example, Pwr(BLK B) = Pwr(TOP) – Pwr(BLKA)
- The calculated Pwr(BLK B) is distributed to BLK B.1 and BLK B.2 based on the relative transistor size.
- In the same way, user-assigned Pwr(BLK A) is divided between BLK A.1 and BLK A.2

according to relative transistor size.

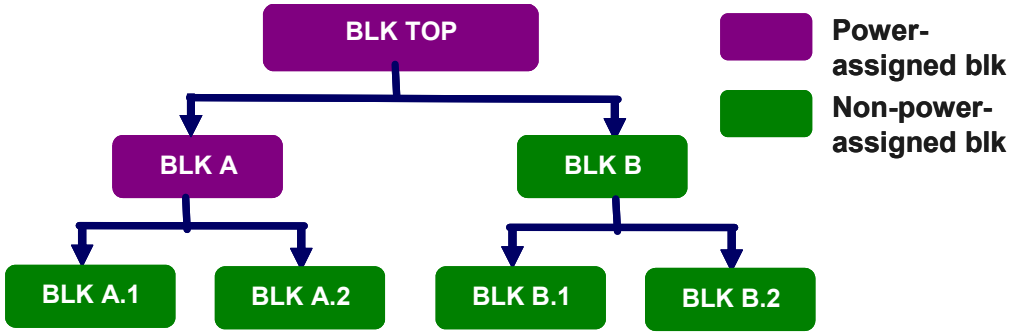


Figure 4-6 Power/current assignment for hierarchical static flow

Setting Up and Running Netlist-Driven Hierarchical Static Analysis

In APLMMX you must specify all subckt hierarchies that you want to assign power in PROBE_SUBCKT_FILE keyword in the APLMMX configuration file. Specify each subckt(s) on a different line in the PROBE_SUBCKT_FILE. Example of a PROBE_SUBCKT_FILE:

```
cell_top
cell_mid
cell_bot
...
```

Then run APLMMX, using one of the following procedures:

```
characterize aplmmx  aplmmx.config -hier_config  or
characterize aplmmx  aplmmx.config -gds
```

In the GSR file, use the keyword:

```
GDSII_FILE {
    <cell> <cell>.gds.config FULL HIER_GDS
}
NATIVE_GDS_SUPPORT 1
...
VDD_NETS {
VDD  1.2
VDDA  3
}
GND_NETS {
VSS 0
VSSA 0
}
BLOCK_POWER_FOR_SCALING {
CELLTYPE  cell_top  0.12 VDD
CELLTYPE  cell_top  0.03 VDDA
CELLTYPE  cell_top  0.11 VSS
CELLTYPE  cell_top  0.01 VSSA
CELLTYPE  cell_mid  0.06 VDD
CELLTYPE  cell_mid  0.01 VDDA
CELLTYPE  cell_mid  0.05 VSS
CELLTYPE  cell_mid  0.033 VSSA
```

```
CELLTYPE  cell_bot  0.024 VDD
CELLTYPE  cell_bot  0.015 VDDA
CELLTYPE  cell_bot  0.02 VSS
CELLTYPE  cell_bot  0.005 VSSA
...
}
...
```

P/G grid integrity checking

After P/G grid extraction and before static analysis, you can perform a basic evaluation of power and ground grid resistance using the command 'perform gridcheck', which provides a normalized and ranked listing of potential node weaknesses. For more accurate grid resistance data, use the ‘perform res_calc’ command, which calculates and reports the actual resistance at the transistor pins. For the usage and syntax for these commands, see section "perform", page D-952

Setting Up and Running Totem Dynamic Analysis

A few changes should be made to the GSR and TCL command files to prepare to run dynamic analysis, as described below.

GSR File for Dynamic Analysis

In addition to the GSR file keywords previously described for Totem static analysis, the following GSR file keywords are needed for dynamic analysis.

- APL_FILES (only required to import APL data if running APLMMX outside of Totem)
- USER_STA_FILE
- GSC_FILE
- DYNAMIC_SIMULATION_TIME
- DYNAMIC_TIME_STEP

Refer to Appendix C for details on the options and use of these keywords. A typical GSR file for Totem dynamic voltage drop analysis is as follows:

```
TECH_FILE user_data/RAM256.tech
LIB_FILES {
  user_data/custom.lib custom
}
PAD_FILES {
  user_data/RAM256.ploc
}
APL_FILES {
  user_data/APLMMX/RAM256.spcurrent current
  user_data/APLMMX/RAM256.cdev cdev
}
GDS_CELLS {
  RAM256 user_data/GDS2DB mmx
}
USER_STA_FILE user_data/RAM256.sta
GSC_FILE user_data/RAM256.gsc
VDD_NETS {
```

```
  VDD 1.8
}
GND_NETS {
  VSS 0
}
BLOCK_POWER_FOR_SCALING {
  FULLCHIP RAM256 1.2 VDD
  FULLCHIP RAM256 0.9 VSS
}
FREQ 500e6
DYNAMIC_SIMULATION_TIME 2e-09
DYNAMIC_TIME_STEP 5e-12
```

Other Files

The data format in an STA user file is as follows:

```
<inst_name> TW <min_timing_window> <max_timing_window>
```

Example:

```
adsU1 TW 0 0
```

The data format in a GSC file is as follows:

```
<inst_name>  <switching_state>
```

where <switching state> could be HIGH/LOW if multi-state is not used in APLMMX, state symbol defined in CUSTOM_STATE_SIM_TIME or MULTI_STATE_FILE. See section "Special APLMMX Applications", page 3-89, for details on multi-state descriptions.

Example:

```
adsU1 PRESET
```

TCL Command File for Dynamic Analysis

A sample TCL command file for DvD analysis is as follows:

```
import gsr user_data/RAM256.gsr
setup design
perform pwrcalc
perform extraction -power -ground
setup pad  -power -r 0.0  -c 0.0  -ground -r 0.0  -c 0.0
setup wirebond -power -r 0.002 -c 30.0 -l 250.0
    -ground -r 0.002 -c 30.0 -l 250.0
setup package  -r 0.0016  -c 5.0  -l 25.0
perform gridcheck
perform analysis -vcd
```

DvD Backannotation to Timing Flow

To generate the SPICE deck for DvD backannotation to the timing flow, use the TCL command ‘generate simulationdeck’ with the following syntax:

```
generate simulationdeck -mmx -aplmmx_cfg <filename>
-sim_time <time> ? -inst <inst_name>? ? -xtor_list <filename>?
? -dir <output_dir>? ? -aplmmx_run_dir <run_dir_name>?
```

where

-mmx: specifies generating MMX SPICE simulation deck

-aplmmx_cfg <filename>: specifies APLMMX configuration file

-sim_time <time>: specifies total simulation time for DvD back-annotated SPICE simulation (ps)

-inst <inst_name>: specifies the MMX macro instance name (only required in custom design flow)

-xtor_list <filename>: specifies the file that lists the transistors for which you want to back-annotate DvD noise waveforms.

-dir <output_dir>: specifies the output directory to dump out SPICE deck for back-annotation simulation. Default is 'aplmmx_ba'.

-aplmmx_run_dir <run_dir_name>: specifies the APLMMX run directory

Running Dynamic Analysis

To run Totem-MMX using the GSR and a TCL command files, execute:

```
totem -f dynamic.tcl
```

The procedures so far have dealt with transistor-level custom macro MMX analysis. The following sections describes netlist-driven hierarchical analysis and custom design MMX analysis.

Netlist-Driven Hierarchical Dynamic Analysis

If your input netlist hierarchy matches the GDS physical hierarchy, you can use an MMX netlist-driven hierarchical dynamic flow to generate the electrical models hierarchically.

Setting Up and Running Netlist-Driven Hierarchical Dynamic Analysis

In APLMMX, you can determine which x-instance(s) characterization results are used to generate the hierarchical electrical model. In the PROBE_SUBCKT_FILE, you can specify the instance name after the subckt name, and APLMMX then honors the given instance to create the model. Otherwise, APLMMX chooses the first instantiated instance in the hierarchical netlist for modeling. An example of PROBE_SUBCKT_FILE:

```
cell_top xtop
cell_mid xtop.xcoreregion
cell_bot xtop.xcoreregion.bitcell
...
```

You then can use 'aplmmx -s' and 'aplmmx -n' to generate the *spcurrent* and *cdev* files for all selected hierarchies.

In the Totem GSR, specify all subblock *spcurrent* and *cdev* files using the APL_FILES keyword, and the GDS2DB-generated header file using the GDS_CELLS keyword option '*HIER_GDS <cell>.gdstop'*.

Then you can use regular dynamic analysis TCL or GUI commands to perform dynamic analysis, and check results.

Note: In APLMMX, if you have different test bench files for different subblocks, you can put all subblock test bench files under the same directory specified in 'TEST_BENCH_DIR <dir>'. All test bench files should have the same file suffix specified in 'TEST_BENCH_SUFFIX <suffix>' in APLMMX configuration file.

You can use the '-hier_config' option when executing APLMMX to dump out an APLMMX configuration file for each sub-block to generate the necessary *spcurrent* and *cdev* files by using the commands 'aplmmx -s' and 'aplmmx -n' on each individual subblock.

The rest of GDS2DB and Totem steps are the same as normal netlist-driven hierarchical dynamic flow.

Mixed Mode VCD and Vectorless Analysis

Totem supports input data including both VCD and Vectorless settings, which provides more freedom to fine-tune the settings at block level. Some of the mixed mode features are:

- A block level VCD with a top-level vectorless approach is supported, where you can provide one or more block VCDs.
- The GSR keyword 'MIXED_MODE 1' and the TCL command option 'perform analysis -dynamic' is used in Mixed Mode simulation, instead of options '-vectorless' and '-vcd', which are invalid in Mixed Mode.
- The GSR keyword BLOCK_VCD_FILE defines one or more VCD blocks for analysis. Each block is specified using the VCD_FILE option with a block name (hierarchy path).
- Keywords DYNAMIC_SIMULATION_TIME and DYNAMIC_PRESIM_TIME define the VCD simulation time span. VCD_FILE 'Start_Time' is honored, but 'End_Time' is discarded. DYNAMIC_PRESIM_TIME defines the presim time span at 'Start_Time' backward. Auto Presim, that is the default DYNAMIC_PRESIM_TIME, or set to -1, is honored.
- Mixed mode allows users to specify a mix of True time and Non-True time VCDs.
- Mixed mode supports S-parameter package models.
- Mixed mode allows for scaling VCD and STA frequencies with STA_VCD_FREQ_RATIO. Block level STA_VCD_FREQ_RATIO is supported only in Mixed Mode.

The following GSR keywords are supported for non-VCD blocks:

- GSC_FILE
- BLOCK_POWER_FOR_SCALING(_FILE)
- STATE_PROPAGATION
- STA_FILE CONST
- TOGGLE_RATE_RATIO_COMB_FF
- INSTANCE_TOGGLE_RATE(_FILE)
- BLOCK_TOGGLE_RATE(_FILE)
- TOGGLE_RATE

If you provide a block VCD file for a block that has a sub-block not covered in the VCD, then the instances inside sub-block are always Off, unless otherwise specified in the GSC file. If simulation time is a multiple of FREQ, then the multi-cycle vectorless approach is used for the instances that are not covered by VCD.

Setup for Mixed-Mode Custom Design Flow

In Totem-MMX analysis, you can mix P&R blocks with standard cell-based APL models together with custom macros having transistor-level MMX models in the same simulation environment to simulate a full custom design.

For an analysis that includes P&R blocks, you can characterize the standard cells inside the P&R blocks using regular cell-based APL methodology. For more details on APL characterization, please refer to the *RedHawk User Manual*.

In addition to the basic GSR keywords described for custom macro analysis previously, the keywords required for custom mixed-mode analysis are as follows:

- DEF_FILES <def_files> - top-level and P&R blocks DEF files
- LEF_FILES <lef_files> - top-level and P&R blocks LEF files
- APL_FILES <apl_files> - electrical models from APLMMX and regular APL
- STA_FILE <sta_file> - STA timing window file of the custom design to describe the instance switching timing windows
- VCD_FILE <vcd_file> - VCD file of the custom design to describe instance switching activity

The following is an example of a typical GSR file for Totem-MMX analysis for custom design analysis:

```
VDD_NETS {
  VDD 1.8
}
GND_NETS {
  VSS 0
}
TECH_FILE user_data/TOP.tech
LIB_FILES {
  user_data/LIB/stdcell.lib
  user_data/custom.lib custom
}
DEF_FILES {
  user_data/pnr.def block
  user_data/TOP.def top
}
LEF_FILES {
  user_data/stdcell.lef
  user_data/pnr.lef
  user_data/TOP.lef
}
GDS_CELLS {
  RAM256  user_data/GDS2DB mmx
}
APL_FILES {
  user_data/RAM256.spcurrent current
  user_data/RAM256.cdev cdev
  user_data/stdcell.spcurrent current
  user_data/stdcell.cdev cdev
}
PAD_FILES {
  user_data/TOP.ploc
}
STA_FILE
{
  FREQ_OF_MISSING_INSTANCES 125e6
  TOP user_data/TOP.chip.timing
}
```

```
VCD_FILE {
    TOP user_data/TOP.vcd
    FILE_TYPE VCD
    front_path "top/mem/"
    substitute_path ""
    frame_size 8000
    TRUE_TIME 1
}
GSC_FILE                user_data/RAM256.gsc
INPUT_TRANSITION        0.1n
TOGGLE_RATE             0.2
FREQUENCY               500e6
DYNAMIC_SIMULATION_TIME 2e-09
DYNAMIC_TIME_STEP       5e-12
```

Power/Ground EM Analysis

Static EM Analysis Overview

Totem™ performs power EM analysis as a part of static or dynamic voltage drop analysis. The primary difference is that static EM analysis is based on true average current values, which is covered in this section, whereas dynamic power EM analysis is based on either peak, true average or RMS current values, although average may also be reported, as described in the following sections.

There is no separate command required for performing EM analysis. It is performed by default if you have EM limits specified in the technology file. I n electro-migration analysis, Totem checks the actual current density for any METAL wire, or current per cut or area for a VIA segment, against the limit specified in the technology file. The setup for power EM analysis only involves defining the desired EM limits in the tech file, which can be done in several different ways, as described in the following section.

Dynamic EM Analysis Overview

Dynamic power EM analysis is very similar to static EM analysis. In static analysis, the EM analysis is based on the true average current density for wires or current per cut or area for vias. In dynamic analysis, Totem performs EM wire or via analysis based on either peak current density (the default value of the GSR keyword EM_MODE), or true average or RMS current density, if EM_MODE is set to AVG or RMS. A set of EM limits different than in the tech file can be specified using the GSR keyword TECH_FILE <file>. Or EM limit values overriding those in the tech file or TECH_FILE can be separately passed to Totem using the GSR keywords EM_TECH_PEAK <file>, EM_TECH_RMS <file>, or EM_TECH_AVG <file>. In these tech files, you need to define only the EM-related parameters for different metals and vias. The format for specifying the limits in these files is the same as in the regular tech file EM keywords. Totem takes the other limits from the default tech file passed through the TECH_FILE keyword in the GSR. If you do not provide separate EM limits in one or more of these files, Totem uses the EM limits specified in the regular tech file for dynamic analysis.

Dynamic EM analysis generates the report with the same filename (*<design>.em.worst*) as in static analysis, located in the *adsRpt/Dynamic* directory. The report format is also the same.

Setting Up EM Limits

The simplest EM limit is specified per layer, which defines the allowed current density value for a specific METAL or VIA layer. For a metal layer, the current density limit is defined as the current flowing per unit width. It can be specified in the tech file, as in the following example:

```
metal METAL1
{
   Thickness   1.45
   Resistance 0.2343
   EM          2.7
   above       PASS4
}
```

In the above example the current density limit for layer METAL1 is defined as 2.7. The unit for current density comes from the units for length and current in the 'Units' section of the technology file, as in the following example:

```
units {
  capacitance 1p
  resistance  1
  length      1u
  current     1m
  voltage     1
  time        1n
  frequency   1me
}
```

For VIA layers, the EM limit is defined by layer on a per-cut basis: the allowed current per cut of the via or via array, as in the following example:

```
via VIA78
{
  Area         { 9 }
  Resistance  0.041
  T           25
  Tnom        110
  Coeff_RT1   0.00337
  Coeff_RT2   -7.91e-7
  EM          7
  UpperLayer  METAL8
  LowerLayer  METAL7
}
```

There are more advanced methods for specifying the EM limits in the tech file, using the following options of the ‘metal’ tech file keyword:

```
EM_Adjust
Width_Based_EM
Blech_JLC
EM_Temp_Rating
Tnom_EM
```

For a description of how to use these options to define metal and via EM limits, see the section "METAL", page C-747, of section "Apache Technology File (*.tech)", page C-744.

If you are creating the tech file using the Apache utility 'rhtech', you can use the option '-e <EM_FILE>' to populate the tech file with two of the EM-related keywords. Only the EM

and EM_ADJUST options can be used inside this rule file; you should include the other EM-related keywords manually in the tech file. The following is the syntax for using the 'rhtech' utility:

```
rhtech -i <input_file> [-o <output_file>]
[-m <layer_mapping_file>]
[-e <EM_file>] [-t <temp_file>]
```

Following is an example of an EM rule file for passing EM tech file keywords:

```
## <layer_in_itf/nxtgrd> <EM_limit-mA/um for metal; mA for vias> <EM adjust>
  metal5 0.9308 0.02
  metal4 0.9308 0.02
  metal3 0.9308 0.02
  metal2 0.9308 0.02
  metal1 0.716 0.02
  via4 0.06766
  via3 0.06766
  via2 0.06766
  via1 0.0676
```

Temperature Setting

Operating temperature for Static and Dynamic EM calculation can be set independently of the temperature value used for analysis, if desired, using the required tech file keyword TNOM_EM and the required GSR keywords TEMPERATURES_EM and TEMPERATURE_EM. The associated power analysis temperature keywords are TNOM in the tech file and TEMPERATURES and TEMPERATURE in the GSR. These temperature keywords affecting EM calculation are described below.

In the Tech file, specify the nominal and final temperatures with the syntax:

```
metal  <metal_layer_name> {
    EM <max_wire_current_density>
     TNOM_EM <EM_nom_temp>
     ? T_EM <EM_final_temp> ?
     TNOM <Power_nom_temp>
     ? T <Power_oper_temp>?
     ...
```

In the GSR, use the following keywords for EM calculation:

```
TEMPERATURES_EM {
        <layer_1> <temp_1 ºC>
        ...
        <layer_n> <temp_n ºC>
}                                   , or
TEMPERATURE_EM <temp ºC>
```

The associated extraction GSR keywords for temperature are:

```
TEMPERATURES {
        <layer_1> <temp_1 ºC>
        ...
        <layer_n> <temp_n ºC>
}                                   , or
TEMPERATURE  <temp ºC>
```

Then for EM calculation, the priorities for temperature-setting keywords are:

1. TEMPERATURES_EM - layer-based, in GSR
2. TEMPERATURE_EM - global, in GSR
3. T_EM - layer-based, in the tech file
4. TEMPERATURES - layer-based, in GSR, for extraction
5. TEMPERATURE - global, in GSR, for extraction
6. T - in the tech file, for extraction
7. TNOM_EM - global, in tech file

Totem records the temperatures used in extraction and in EM checking in the *totem.log* file.

Running Power EM Analysis

Totem by default does EM analysis as a part of static and dynamic simulation. However, you can set the GSR keyword 'ENABLE_EM_PROCESS' to disable the EM check (default On) during regular post simulation processing. Then the TCL command `'perform emcheck'` can be used to perform power EM analysis for specific modes and nets, using the command:

```
perform emcheck ?-mode [AVG |RMS |PEAK |all ]?
?-net <netname>?
```

The default mode is 'all'.

Analyzing Power EM Analysis Results

Once the simulation is performed you can click on the **EM** button in the **View Results** panel to see the EM violations map, such as shown in Figure 4-7, which displays the EM violations in different ranges in different colors. Totem highlights all metal or via segments with current density or current per cut exceeding the specified EM limit (default) in red, and lesser fractions of the limit in other colors.

You can change the EM ranges and their color display using the ElectroMigration Color Map dialog, available by clicking on the 'Set Color Range' button in 'Configuration' panel in the GUI.

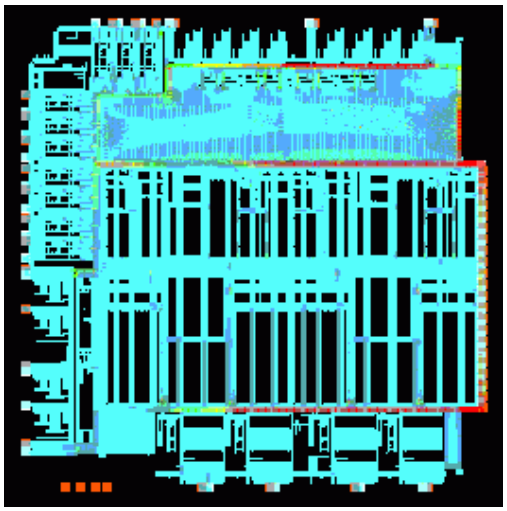


Figure 4-7 EM violation display

EM violations are also listed in the report *adsRpt/Static/<design_name>.em.worst*. This file lists the details of all METAL or VIA segments with current density or current exceeding the default EM limit of 100% (in decreasing order). You can change this default limit setting using the GSR keyword, EM_REPORT_PERCENTAGE. By default, Totem dumps only top 1000 violations. You can increase this number using the GSR keyword EM_REPORT_LINE_NUMBER. If you want to dump the EM violations in a specific range, you can use the GSR keyword EM_DUMP_PERCENTAGE. For example, if you specify “EM_DUMP_PERCENTAGE 50 60" in the GSR, it dumps all violations having an EM ratio between 50% and 60% in the file *adsRpt/Static/<design_name>.em.* This file is not in sorted order and it is not generated unless you specify the above keyword in GSR.

In the EM report file, Totem reports all METAL EM violations using the following format.

```
#Layer   #End-to-end_coordinates                    #EM_Ratio #Net #Width
METAL4   (4905.670,3398.849 4905.670,3400.562)    469.016%  VDD  25.000
```

This report is also available using the GUI menu command **Results -> List of Worst EM**. In this GUI display list you can zoom in to any violation using the **Go To Location** button. Then click on the geometry to see more details on the violation in the log window. This information can be used for calculating the EM ratio of this segment. Actual current density is calculated by dividing the current by effective wire width. If you specify the EM_ADJUST parameter in the tech file, Totem subtracts this value from the actual width to get the effective width.

For dynamic power EM, Totem reports a list of worst EM violations for each mode separately in the file *adsRpt/Dynamic/<design>.em.worst.<mode>*. This is useful with several EM file limits, such as:

TECH_FILE <file>.tech

EM_TECH_RMS <file>.em_rms.tech

EM_TECH_PEAK <file>.em_peak.tech

In this case Totem automatically generates a detailed report for all the modes for which EM analysis has been performed. An example report generated after performing EM analysis for PEAK mode is shown below:

```
EM MODE is PEAK
# This file reports the EM violations, i.e. EM_Ratio =
Actual_Current_Density/Current_Density_Limit > 100% (default or from
“em_report_percentage”) for wire pieces and vias in decreasing order. Unit
used for coordinates and dimensions is um.

# For wires: #layer #end-to-end_coordinates #EM_Ratio #net #width
# Blech_length
# For vias: #via_name #x-y_coordinates #EM_Ratio #net #blech_length

METAL3  (886.190,1018.712 887.192,1018.712)     1081.01%  VDD  2.003  141.951
METAL3  (885.190,1020.695 886.190,1020.695)     1081.01%  VDD  2.000  141.951
METAL3  (2944.210,2479.555 2944.210,2480.055)   1024.24%  VDD  2.000  10.500
METAL3  (2942.980,2479.305 2942.980,2479.555)   1024.24%  VDD  0.500  10.500
METAL3  (1108.770,2479.555 1108.770,2480.055)   1023.81%  VDD  2.000  10.500
METAL3  (1107.540,2479.305 1107.540,2479.555)   1023.81%  VDD  0.500  10.500
...
```

When input Tech files have BLECH_JLC (Blech product) specified, Totem dumps out the Blech length used in the *<design>.em.worst* file. A sample portion of an *adsRpt/Static/<design>.em.worst* file follows:

```
# For wires: #layer #end-to-end_coordinates #EM_Ratio #net #width
#blech_length
# For vias: #via_name #x-y_coordinates #EM_Ratio #net #blech_length
```

```
METAL3  (3212.145,3293.045 3212.145,3293.295)   1525.37%  VSS  0.500  126.660
METAL3  (3210.914,3293.295 3210.914,3293.795)   1525.37%  VSS  2.001  126.660
METAL4  (4146.085,1824.885 4146.585,1824.885)   1132.84%  VSS  2.000  119.865
METAL4  (2150.795,1822.905 2151.295,1822.905)   968.839%  VSS  2.000  119.865
METAL4  (2151.295,1824.885 2151.795,1824.885)   968.839%  VSS  2.000  119.865
METAL3  (1240.175,3295.595 1240.175,3296.095)    502.985%  VDD  2.000  10.500
METAL3  (2998.330,2477.005 2998.330,2477.255)    378.323%  VSS  0.500  10.500
METAL3  (2999.560,2477.255 2999.560,2477.755)    378.323%  VSS  2.000  10.500
...
```

In the example in Figure 4-8, EM_ADJUST for METAL4 is defined as 0.016. So Totem calculates the actual current density as follows:

```
Actual I density = (Current/Eff_Width) = 219.664 /(25-0.016) = 8.7922 mA/u
```

And the EM limit for METAL4 in the tech file is defined as 1.874 mA/u.

Totem calculates the EM_Ratio using the following equation.

EM_Ratio = 100 * (Actual density of current) / (EM limit in tech file).

Therefore

EM_Ratio = 100 * 8.7922 / 1.874 = 469.2

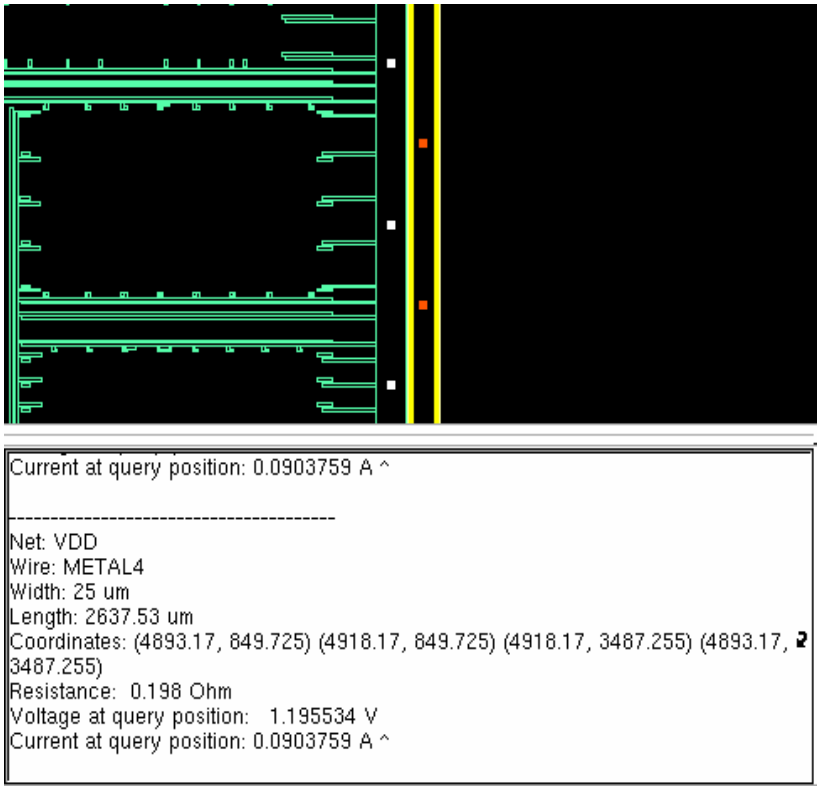


Figure 4-8 Investigating a potential metal EM violation

For VIAs, Totem reports the EM violations in the following format:

```
#Via_name          #x-y_coordinates      #EM_Ratio #net
via via3Array_87  (3154.820,893.390)    172.29%   GND
```

The EM_ratio of a VIA layer is calculated as follows (percentage):

```
EM_Ratio = (Current through the VIA)*100 /(EM limit)
```

When you click on a via in the GUI, the Totem log window displays additional information that helps you in analyzing the violation, as shown following:

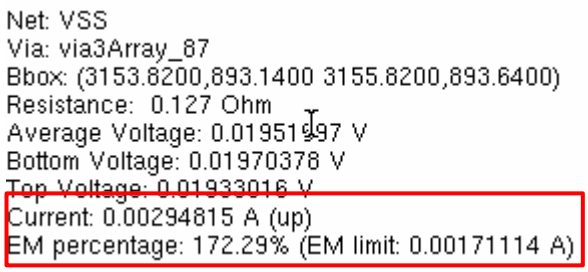


Figure 4-9 Investigating a potential via EM violation

In the example above, Totem reports both EM percentage and the EM limit value used in the calculation. The EM limit displayed is the limit calculated for the specific VIA array. As described earlier, inside the tech file Via EM limits are defined on a per-cut basis. Totem calculates the EM limit for every via array in the design, considering the number of via cuts in the via array and total area of the cuts included. For example, if the via array has 10 cuts, the EM limit reported is (Tech file EM limit * 10).

Current direction and EM violations

To accurately analyze EM violations in a design, you need to understand the actual current directions. After EM analysis, when you click on any METAL or VIA geometry in Totem, the assumed current direction is indicated by a symbol in the log display window, along with the current value, as shown in Figure 4-10:

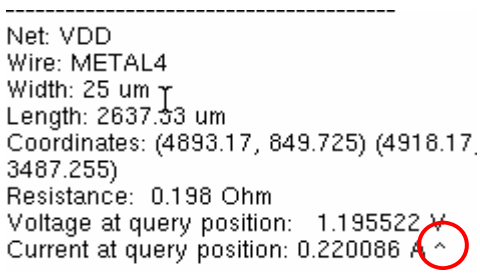


Figure 4-10 Log display showing current direction

The symbol conventions followed for displaying the current direction are:

> : For metals, current flow from left to right.

< : For metals, current flow from right to left.

^ : For metals, current flow from bottom to top.

V : For metals, current flow from top to bottom

Up : For vias, current flow from lower metal layer to upper metal layer

Down : For vias, current flow from upper metal layer to lower metal layer.

Using the correct current direction, you can identify the actual path for the current flow. You can also look at the Current map (**CUR** button in **View Results** GUI panel) to see any discrepancy in the current flow.

Fixing Power EM Violations

EM violations are mostly caused by weak power grid connections feeding current to high power-consuming regions or blocks in the design. If this is the case, increasing the metal width to reduce the current density is a typical solution. Similarly, for a via EM violation, you can increase the number of vias to fix potential EM issues. You also can provide additional straps for the current supply, thereby reducing the current-per-strap value. Layer switching is another option; typically, upper metal layers in the technology have higher current driving capability (due to greater thickness). So you can use these layers for designing the major power grids (grids with higher current flow) in the design.

You can use the what-if and Fix and Optimize capabilities (see Chapter 9, "Fixing and Optimizing Grid and Power Performance") in Totem to modify the grid for fixing EM violations. Using this method you can edit any existing strap or add any new straps or vias. Totem has incremental extraction capability, which helps you in analyzing the design quickly after making any modification.

The following example helps in understanding how you can resolve EM issues quickly using the “What-if” capability in Totem. An example EM violation is caused by an insufficient number of vias between the power straps. More vias are added in this area using the 'Add Via' option in Totem to fix the problem, as shown in Figure 4-11 below:

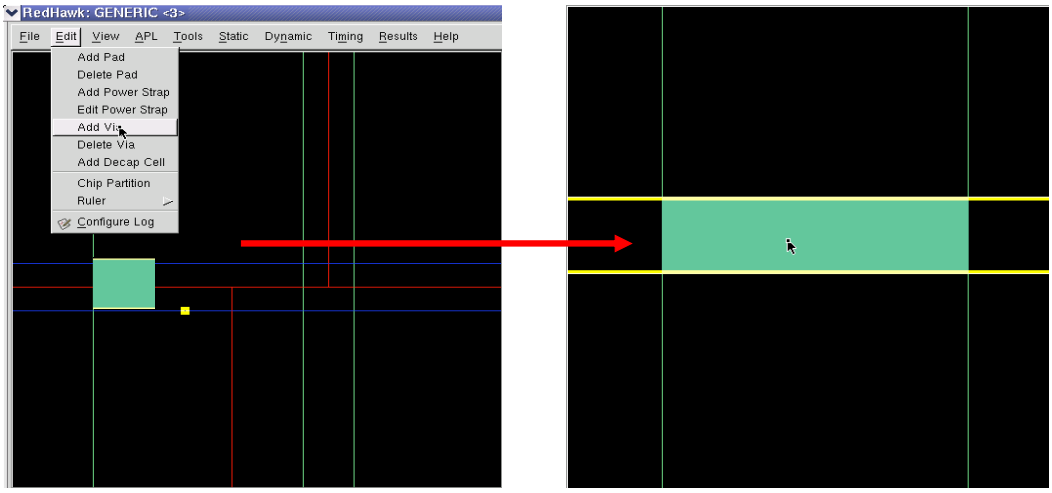


Figure 4-11 Using FAO to repair EM violations

Chapter 5
Design-Specific Setup

Introduction

Different design styles require somewhat different procedures to set up and run Totem. The primary different types of designs are:

- Embedded memory macros, such as compiled memories, register files, TLBs, and cache memories
- Analog designs, such as PLLs, ADCs, DACs
- Advanced low-power designs, power gating and LDO
- Full custom hierarchical designs, such as flash memory, CAM, DRAM, SRAM, and FPGA chips
- Mixed signal designs

In addition to the setup steps described in Chapter 2, modifications for these specific types of designs are described in the following sections.

Embedded Memory Macros

Simulating RFs, SRAMs and CAMs in Totem does not require any special handling of the data. To speed up the performance you can chose to ignore the core bit-cell for these types of memories. This can be done because the power consumed by these cells is minimal and the geometry is repetitive in nature. For CAM cells it is important to model the cells due to high current flow through the actual CAM logic in the cells. A sample architecture of an RF/SRAM is shown in Figure 5-1.

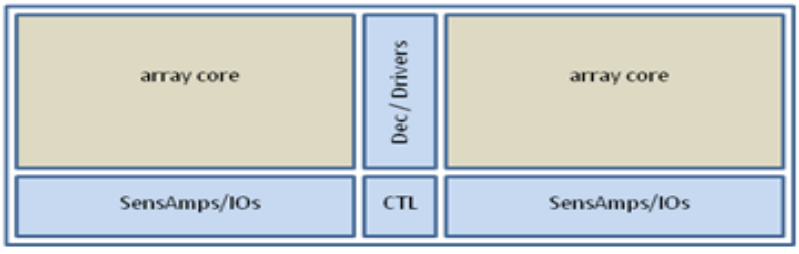


Figure 5-1 Embedded RF/SRAM architecture

Typically, large cache memories are made up of small building blocks of memories. These blocks of memories can be simulated for capturing the currents accurately. Once these currents have been captured, the currents can be rolled up to the top level for analysis.

Analog Designs

The setup required for analog designs is very similar to the generic Totem setup for any custom digital design. However, there are special types of devices that are specific to analog designs that need to be handled correctly during netlist and GDS processing. The following sections described how to handle the special devices for these types of designs.

Diode Devices

Marker layer

For Totem to correctly identify the diodes in a layout, a marker layer should be present for identification. Without this ID layer, the diodes cannot be identified. The GDS2DB layer map should have an entry to identify the diodes using the 'DIODE' keyword, as shown below:

```
DIODE diode 119:0 -
```

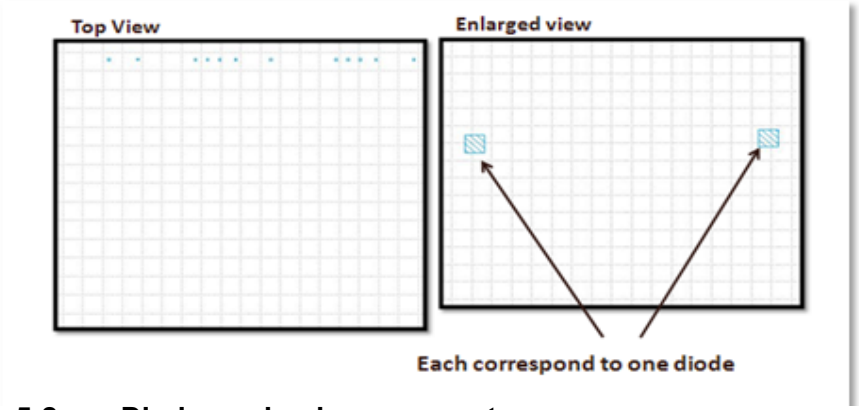


Figure 5-2 Diode marker layer geometry

PIN snapping

Diode marker geometries have either an NDIFF or a PDIFF within.

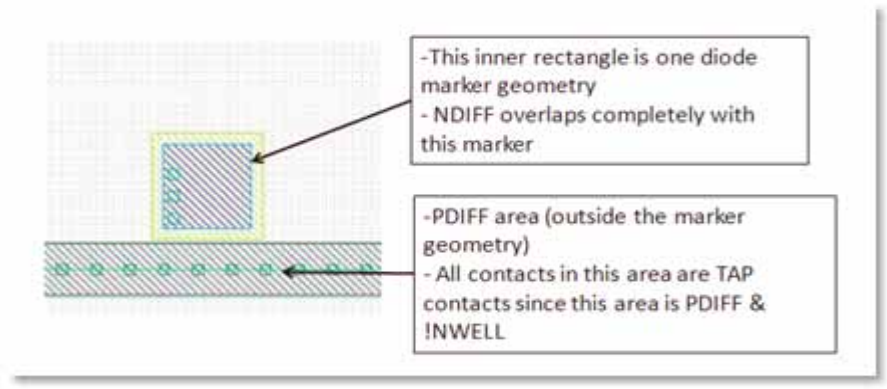


Figure 5-3 Diode illustration

Created pins after both pins with TID = PN1 and TID = PN2 are processed.

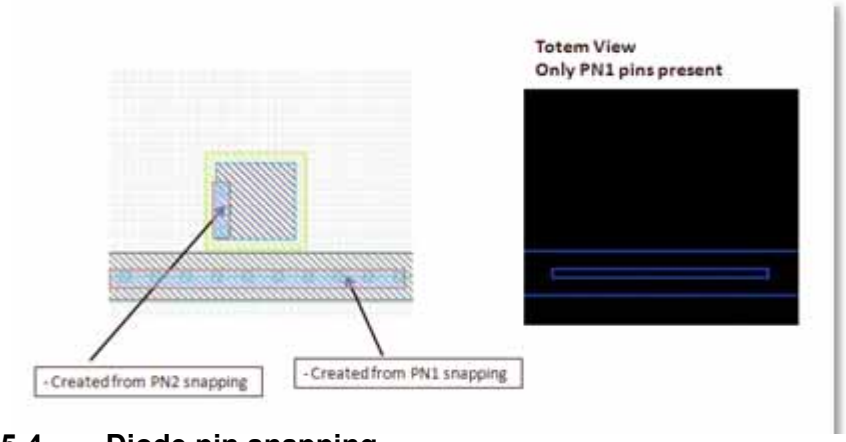


Figure 5-4 Diode pin snapping

BJT Devices

Marker layer

BJTs are similar to diodes, and also have a marker layer defined in the GDS, which is required to identify the devices. The GDS2DB layer map should contain the following entry, along with the correct layer numbers to identify the BJTs. The keyword used is 'BJT':

```
BJT bjt 110:0 -
```

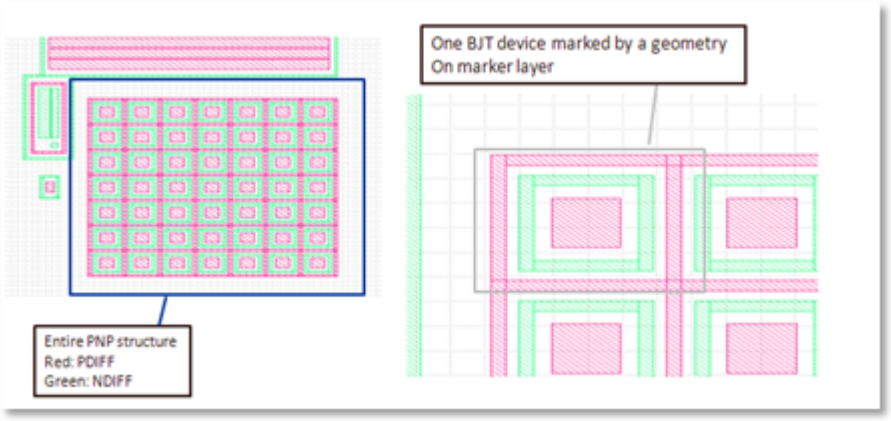


Figure 5-5 BJT layout

Totem also supports lateral BJT devices, which are lateral NPN transistors that have a special structure (like interleaved fingers of the emitter-base) compared to regular BJTs.

A sample GDS layer map definition follows:

```
BOOLEAN base_npn          poly          (BP&SC)-EP
BOOLEAN emitter_npn_temp temp        (EP&active&SC)
#Poly layer corresponding to emitter and collector
BOOLEAN emitter_npn poly emitter_npn_temp | (CS&active)
```

Poly Resistor Devices

Marker Layer

A marker layer is also required to identify the POLY resistor devices in GDS2DB. Without the marker layer gds2db cannot identify these types of devices, even if the layout suggests their presence. The marker layer should be specified in the GDS2DB layer map file, as shown in Figure 5-6.



Figure 5-6 Sample map file showing poly resistor identification

Technology file changes for poly resistors

Once GDS2DB extracts the poly resistors in the design, Totem needs an updated technology file to specify the resistance of the poly resistors identified. An example of a tech file including the poly resistor definition is shown below:

```
metal POLYRES {
  Tnom 25.0
  Tnom_em 125.0
  Coeff_RT1 2.972e-04
  MINWIDTH 0.027000
  Coeff_RT2 -1.868e-07
  em 1.0
  THICKNESS 0.048000
  MINSPACE 0.072000
  Resistance 100.200000
  above GOX
  ...
}
```

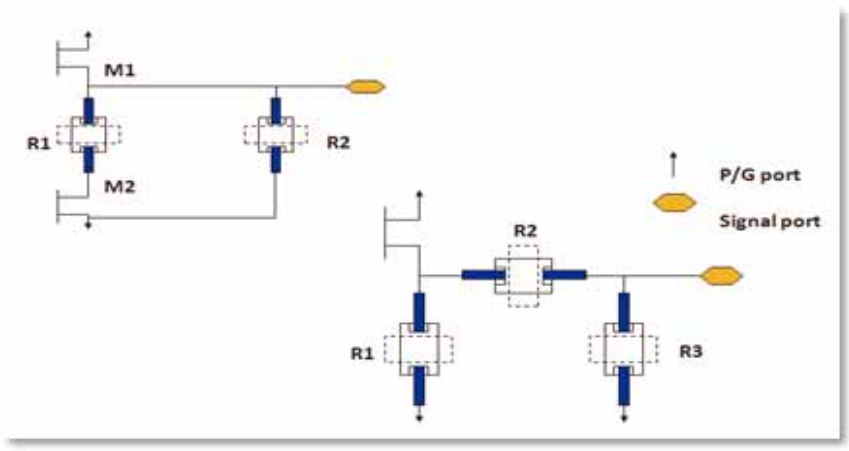


Figure 5-7 Example of poly resistor devices in a circuit

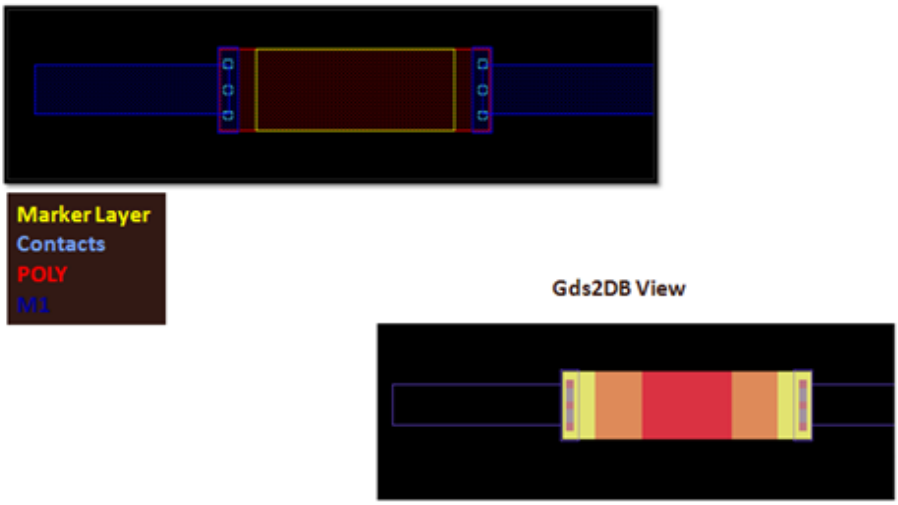


Figure 5-8 Layout view and pin snapping

Power-gated Designs

Overview

VLSI designs, especially those manufactured in a 90nm scale process or smaller, and those targeted for wireless, mobile computing, and other low-power applications, are employing power saving modes for extended battery performance, greater control over the speed-versus-power trade-off and better thermal performance. Sub-micron designs suffer from increased device leakage currents, less control of threshold voltage levels from negative bias temperature instability (NBTI), increased current drive strengths, and higher device densities. Designers are becoming increasingly aware of the need to make their designs “power-aware” at several levels – system, architectural, design, and

technological. Though the most significant impact on power saving may be achieved through changes at the system or architectural levels, most of the visible efforts are focused on design and technological level changes to mitigate the harmful effects of higher power consumption in today's designs.

Some commonly employed power saving design techniques, include:

- Power Gating, in which Vdd /Vss supplies for selected areas of the circuit, or individual cells are controlled by one or more switches or sleep transistors. Techniques such as this are also sometimes referred to as "MTCMOS".
- Low DropOut (LDO) linear voltage regulation, in which a feedback loop maintains the output voltage (see section "LDO Low Power Designs", page 5-202).

Totem™ can provide accurate dynamic power analysis for designs using the power saving techniques described above. Designers can perform full custom chip and block-level *transient* dynamic voltage analysis and average static voltage drop analysis by accurately modeling one or more of the above power saving modes in their designs. They can use the textual result reports and the graphical displays, including time point based movie mode display of dynamic voltage drop results, to analyze and improve their designs.

IP macros may have a gated power domain controlled by a power switch. The power switch usually is the interface between an external power domain and an internal power domain. Totem has the ability to handle power gated domains natively, provided certain requirements are met in the netlist and the gds hierarchy. An illustration of a power gated domain is shown in Figure 5-9.

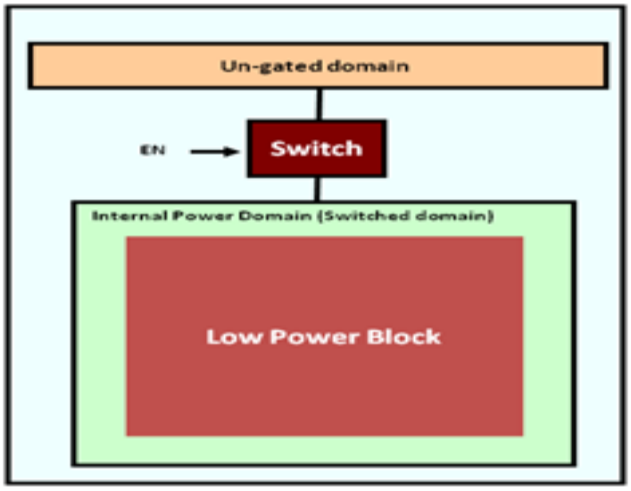


Figure 5-9 Power-gated domain

In order for Totem to handle power gates in the design, a clear hierarchy of the power gates must be present in the design data. The power gate must have the same hierarchy definition in the netlist as in the GDS file. The requirements for the netlist, GDS, and other data inputs are provided in the following sections.

The basic power gating setup and analysis procedure is shown in Figure 5-10:

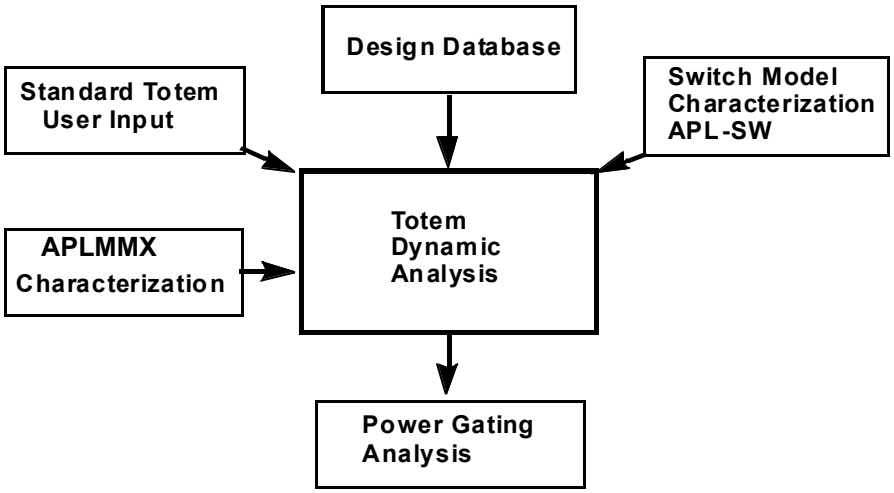


Figure 5-10 Flow for Power Gating and Switch Analysis

Low power designs with header and footer switches generally do not have domain text labels. The Figure 5-11 diagram outlines this type of data flow.

Note: only Power ON state is supported in Totem low power analysis.

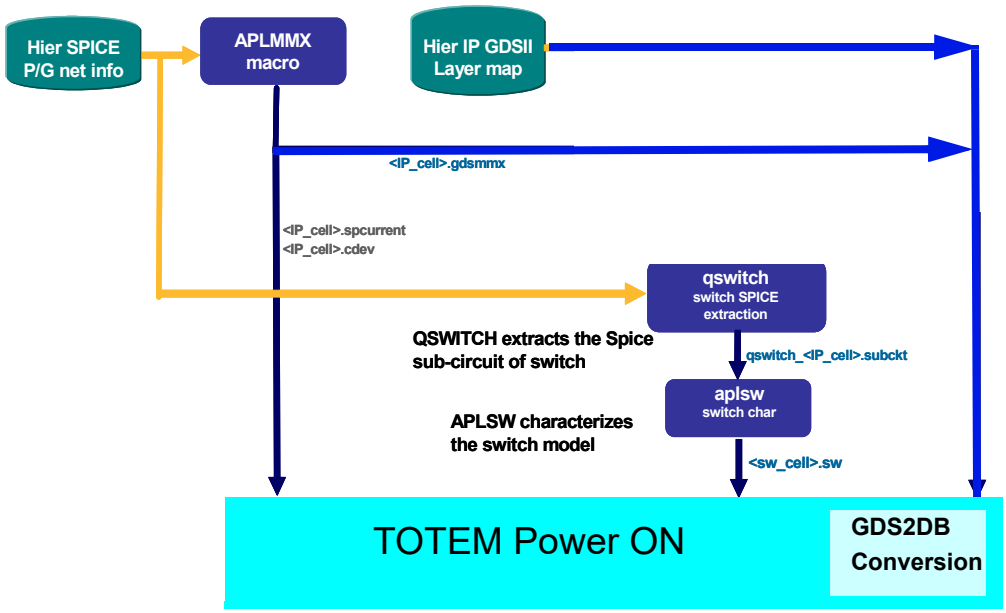


Figure 5-11 Flow for power gating designs

Initially three input data elements should be validated:

1. Check that the Spice netlist for the design includes complete switch definitions. See the following section for details.
2. Check the GDS files for the required switch-related content. See the following section for details.

3. Check the switch cell name and switch pin name consistency in the APL switch models.

The basic steps for performing power gating analysis are:

4. Create individual power gating switch models for ON and OFF conditions using the APLSW configuration file (*.*conf*) and SPICE netlists/models. Perform characterization of switch models using the 'aplsw' utility.
5. Create and check required APLMMX/ACE configuration file keywords for switches, then run APLMMX/ACE electrical model characterization.
6. Create and check required GDS configuration file keywords for switches, then run GDSMMX/GDS2DB to create the physical models.
7. Define power gating On-Off switching with the GSC file.
8. Specify and import switch definitions using required GSR keywords.
9. Run Totem.

Details on input data preparation and power gating analysis are provided in the following sections.

On-state Analysis

A diagram of a typical design for On-state analysis is shown in Figure 5-12.

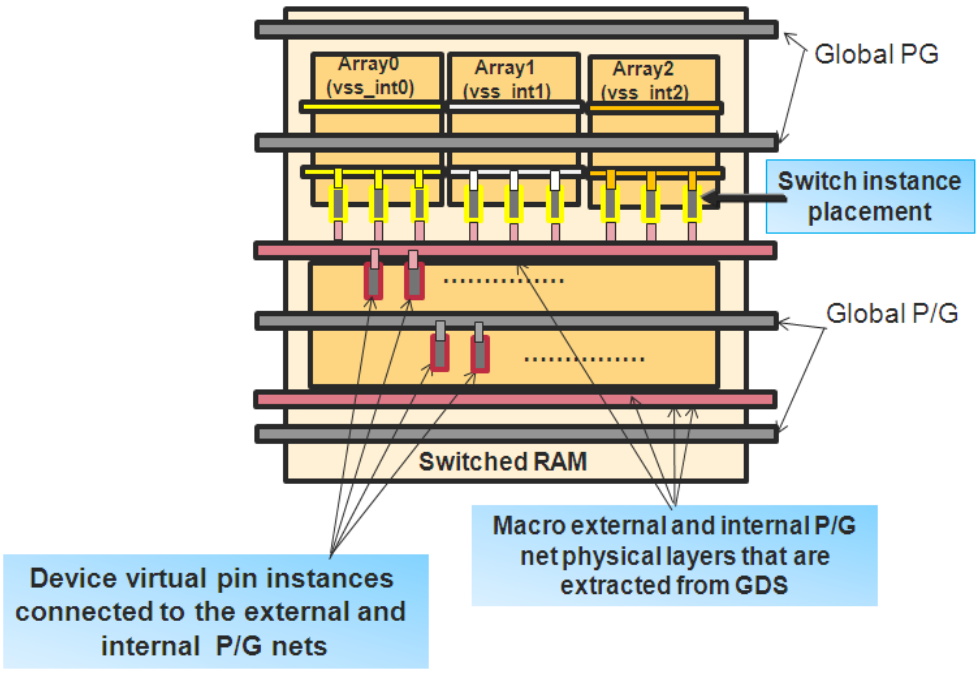


Figure 5-12 On-state analysis scheme

The following are key elements of switched RAM On-state analysis:

- Supports standalone macro On-state analysis.
- Switched cell APL model must be specified.
- Only the external nets must be specified with VDD_NETS or GND_NETS keywords.

- By setting EXTRACT_INTERNAL_NET 1, flow automatically traces the internal nets.
- Optional power ON control using VP_CONTROL keyword to assign internal delay to the block switching time.
- Results checking:
  - Make sure the power is successfully calculated.
  - Check the connectivity of the P/G pin instances.
  - Perform ‘plot current –net –name <internal net>’ command to make sure appropriate current is assigned.

Power Gating Flow

General data requirements

In order to perform an analysis of power gated designs, the following data rules must be met:

- Top-level and cell-level switch control definitions must be available for characterization.
- The switch cell hierarchy should be present in the GDS file. Have hierarchical GDS with the switches defined as placed master cells.
- The switch cell name should match the name in the switch name in the netlist. The SPICE for the memory with switch subckt names should match GDS cell names.
- External and internal (virtual) P/G name mapping must be available between LEF/GDS view and SPICE view.
- GDS text labels should be available for the internal power nets and be present at the top level, or GDS tracing points for the internal nets must be manually defined.

Some important aspects of the power gated design flow are:

- GDSMMX/GDS2DB can extract the LEF for the switch automatically.
- APLSW characterizes the switch model.

The following sections describe Totem power gated design analysis.

Spice netlist data requirements

The switch cell must have hierarchy defined in the Spice netlist, as shown below:

```
...
.SUBCKT switch_header VDD VDD_sw in1
MM0_pwrsw<0> VDD_sw in1 VDD VDD TP W=1.5u L=0.05u m=1
MM0_pwrsw<1> VDD_sw in1 VDD VDD TP W=1.5u L=0.05u m=1
MM0_pwrsw<2> VDD_sw in1 VDD VDD TP W=1.5u L=0.05u m=1
.ENDS
...
```

In this example the switch cell is connected between the external (VDD) and internal (VDD_sw) power domains. The APLMMX utility can automatically recognize the switch transistors between the external and internal domains provided the net names are specified correctly in the aplmmx config file. For APLMMX to correctly recognize the coordinates of the switch transistors, the DSPF file that provides the x,y coordinates or the cross reference file should have these devices.

GDS switch data requirements

Figure 5-13 shows the data flow for GDS switch models.

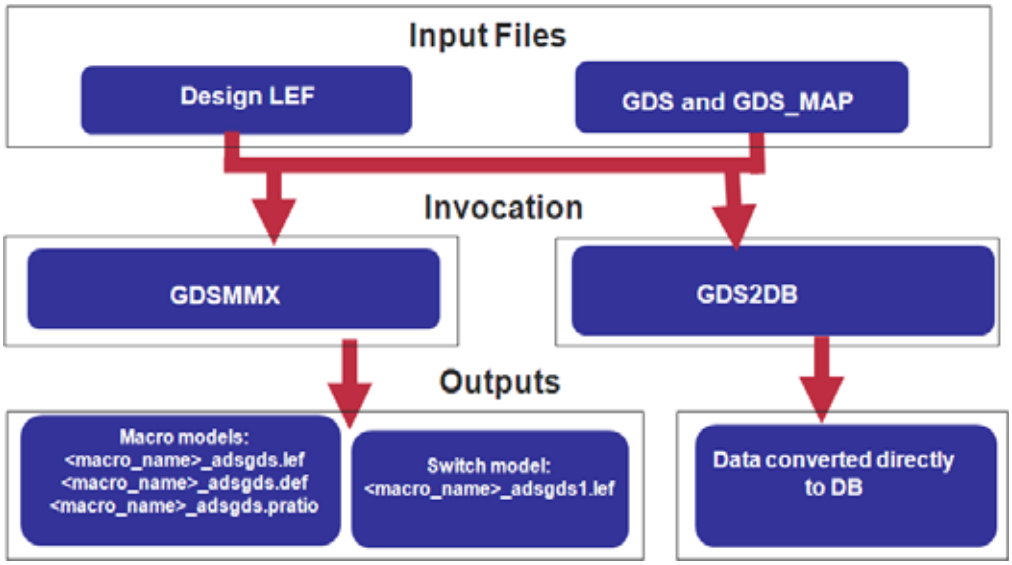


Figure 5-13 GDS switch model data flow

Switch model creation and characterization

Ensure that the switch models generated from the Spice netlist and the physical models from GDS. To ensure switch pin name matching, use the SPICE2LEF_PIN_MAPPING keyword for switch model characterization.

Some designs include circuitry that provides the ability to turn power to a cell, block or subcircuit ON or OFF as needed. This is usually accomplished through the addition of a switch circuit in series with the power or ground net serving the subcircuit. An example of such a network is shown in Figure 5-14, with the external VDD supply connected to the power supply internal to the subcircuit through a switch circuit. This type of switch is commonly referred to as a *header* switch.

Note: Power-up and power-down analyses are currently not available in Totem.

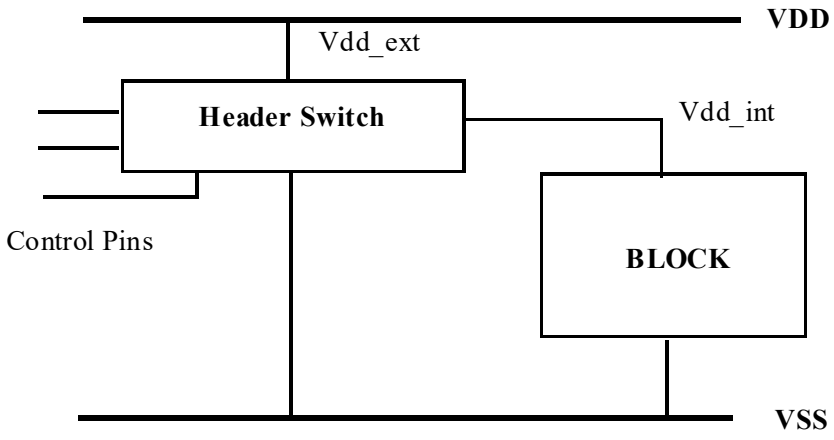


Figure 5-14 Example using VDD power switching with a header switch.

When a switch controls the conduction path between the external and internal ground network, it is commonly referred to as a *footer* switch, shown in Figure 5-16. A particular subcircuit or block typically would have only one or the other type of power gating, but it could have both if there was a design reason.

The UNIX shell command syntax is:

```
aplsw [-c] [-d] [-o <output_file>] <sw_config_list>
```

where

-c : check generated result

-d : debug mode

-o <output_file> : output file name

<sw_config_list> : switch configuration list file. Create a file defining all the switches you want to characterize, with the following contents:

```
<cell1> <cell1.config>
<cell2> <cell2.config>
```

ON State

During the ON-state, the header and footer switches form a low resistance element between the “external” and “internal” power networks. Regular dynamic and static analysis of a design can be performed in this mode. Totem reports the voltage across the switches and the current through the switches from the dynamic and/or static analyses. Figure 5-15 shows the equivalent circuit model for a header switch in the ON state, which is characterized using the *aplsw* utility, which automatically generates a switch model file containing switch performance data for ON and OFF states. The key components of the On model are a small saturation R value, some capacitance, and a small leakage current that is relatively independent of the voltage applied or the internal resistance.

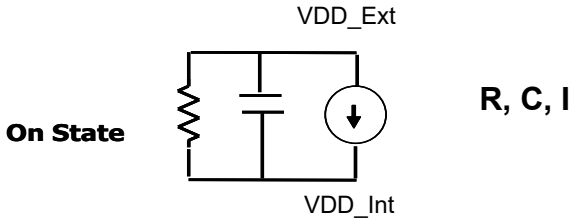


Figure 5-15 ‘ON’ state electrical model for a header switch

Adaptive ON Resistance

For improved accuracy, Totem supports adaptive modification of the switch On resistance (Ron) based on the supply voltage variation seen at the control pin of the PFET (header) or the NFET (footer). For example, for a footer-based device, if the supply voltage variation at the gate during transient simulation is significant compared to a particular threshold, then Totem chooses the appropriate switch Ron for that particular combination of terminal voltages at the simulation time point. The gate voltage is obtained by monitoring the appropriate voltages (VDD for footers and VSS for footers) during an ON-state dynamic analysis in the neighborhood of the switch cells. Then Totem uses this gate voltage relationship to determine the appropriate switch Ron from the switch model file (see section "APLSW Switch Model Generation", page 5-192). This feature is activated using the GSR keyword:

```
DYNAMIC_ADAPTIVE_RON [0|1] <variation_threshold_%>
```

For example, 'DYNAMIC_ADAPTIVE_RON 1 8' specifies that the threshold for Ron change is an 8% variation in power or ground voltage. The default value is 0 (Off).

The power switch should ideally be characterized using the APLSW keyword 'MD_PWL 1' to use the multi-dimensional switch model file, which provides the most accurate electrical representation of the switch. If the default switch model is used, Totem uses internal heuristics to scale the switch Ron.

Control of Power Gating Switches

Control pins are used to manage *when* power is applied to the block or subcircuit and when it is removed, as shown in Figure 5-14 and Figure 5-16. Pins required to connect and control a *header* switch have the following types: control (one or several), vss supply, vdd_external/supply, vdd_internal, vdd_high/bias. A corresponding set of pin types are needed for a *footer* switch: control (one or several), vdd supply, vss_external/supply, vss_internal.

Enhanced mode switch characterization takes enable pin transitions into account.

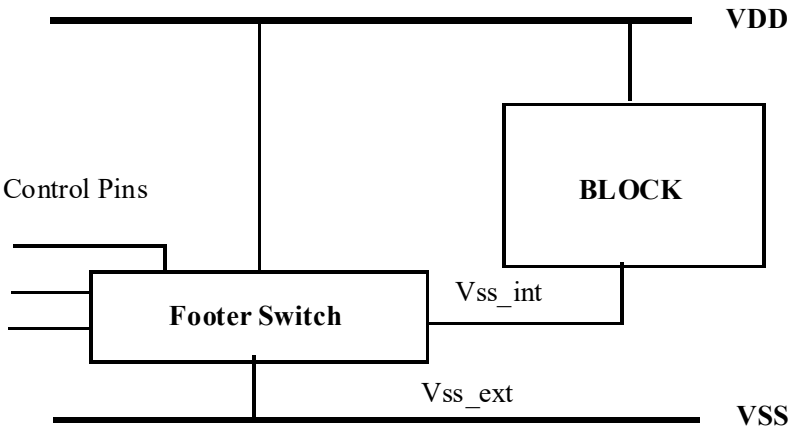


Figure 5-16 General power gating circuit using a footer switch

The next section describes the configuration and characterization process that defines the actual pin names, how they are controlled, and the voltage levels associated with each condition.

Checking Switches with Two Enable Pins

When one power switch is controlled by two enable pins, you need to make sure that the proper threshold voltage is reached when the second pin is enabled. Using the GSR keyword CHECK_SWITCH_POWERON ensures the proper working of the switch. The syntax for the keyword is:

```
CHECK_SWITCH_POWERON {
  [ THRESHOLD <threshold_all_V> |
  <cellname> <cell_threshold_V> ]
...
  }
```

where

<threshold_all_V> : specifies the threshold voltage (Volts) for all switch cells

<cellname> <cell_threshold_V> : specifies the threshold voltage (Volts) for switch cells of type <cellname>

A report, *switch_poweron.rpt*, lists switches in the design by threshold voltage and identifies any violations, as shown in sample report in Figure 5-17. In this example, for a

timing window of 2.2 ns, the footer switch ‘sw_inst1’ only discharges to 0.272049 V, while it should reach 0.1 V for proper operation, so a threshold violation is reported.

Threshold = 0.1 , TW for sw_inst1/ECLK is 22.6e-10
=> **Violation**

```
#Report switch power-on voltage
#instance_name  threshold      power_on_voltage       violation
sw_inst23         0.1      n/a      n/a
sw_inst22         0.1      n/a      n/a
sw_inst21         0.1      n/a      n/a
sw_inst20         0.1      n/a      n/a
sw_inst19         0.1      n/a      n/a
sw_inst18         0.1      n/a      n/a
sw_inst17         0.1      n/a      n/a
sw_inst16         0.1      n/a      n/a
sw_inst15         0.1      n/a      n/a
sw_inst14         0.1      n/a      n/a
sw_inst13         0.1      n/a      n/a
sw_inst12         0.1      n/a      n/a
sw_inst11         0.1      n/a      n/a
sw_inst10         0.1      n/a      n/a
sw_inst9          0.1      n/a      n/a
sw_inst8          0.1      n/a      n/a
sw_inst7          0.1      n/a      n/a
sw_inst6          0.1      n/a      n/a
sw_inst5          0.1      n/a      n/a
sw_inst4          0.1      n/a      n/a
sw_inst3          0.1      n/a      n/a
sw_inst2          0.1      n/a      n/a
sw_inst1          0.1      0.272049         YES
```

Figure 5-17 Sample switch_poweron.rpt for a footer switch

APLSW Configuration Files

A configuration file must be created for each switch type to specify the different pins in the switch and their voltage settings. The following are examples of the keywords required in the switch configuration file, along with a definition and the proper syntax.

TEMPERATURE 25

Defines characterization temperature. Syntax:

TEMPERATURE <temperature_value>

SPICE_NETLIST my_switches.sp

Defines switches’ SPICE netlist file. Syntax:

SPICE_NETLIST <switch_name.sp>

SWITCH_TYPE HEADER

Specifies either header or footer switch type. Syntax:

SWITCH_TYPE [HEADER | FOOTER]

DEVICE_MODEL_LIBRARY 0.13um.lib TT

Defines SPICE device model library. Syntax:

DEVICE_MODEL_LIBRARY <model_file> <process_corner>

INCLUDE spice.models

Defines SPICE model card file. Syntax:

INCLUDE <model_card_name>

EXT_PIN vdd_ext

Defines switch input (supply connection) pin name. Syntax:

EXT_PIN <pin_connection_to_external_supply>

INT_PIN vdd_int

Defines switch output (circuit connection) pin name. Syntax:

INT_PIN <pin_connection_to_cell>

GND_PIN_NAME vss

Defines GND pin name -- for HEADER switches ONLY. Syntax:

GND_PIN_NAME <gnd_pin_name>

VDD_PIN_NAME vdd

Defines VDD pin name -- for FOOTER switches ONLY. Syntax:

VDD_PIN_NAME <vdd_pin_name>

VDDVALUE 1.1

Defines VDD supply value. Syntax:

VDDVALUE <vdd_supply_value>

ON_STATE {

```
    ENABLE1 0
    ENABLE2 1.1
}
```

Defines the voltage on control pins needed to turn the switch fully ON. Syntax:

ON_STATE {<pin> <V_ON_value>}

OFF_STATE {

```
    ENABLE1 1.1
    ENABLE2 0
}
```

Defines the voltage on control pins needed to keep the switch OFF.

Syntax: OFF_STATE {<pin> <V_OFF_value>}

CONTROL_PIN ENABLE1 R F

CONTROL_PIN ENABLE2 R F

Specifies the control pin names and the transitions (rise or fall) that turn the switch On. This uses the proper transitions in the STA file during Totem analysis. For no rise or fall spec, use “-”. (don’t care).

Syntax: CONTROL_PIN <subckt_pin_name> <for ramp-up: R | F | - >
<for ramp-down: F | R | - >

DC_BIAS vdd_high 1.1

Defines input control pins’ required DC value. Syntax:

DC_BIAS <subckt_pin_name> <value>

OPENPIN open1

Defines pins that should be kept open. Syntax:

OPENPIN <subckt_pin_name>

SPICE2LEF_PIN_MAPPING {

Defines SPICE to LEF pin mapping. Can be applied to match the switch LEF pin names in the physical model: *<macro_name>_adsgds1.lef*. Syntax:

SPICE2LEF_PIN_MAPPING {
<subckt_pin_name> <LEF pin name>
...

```
}
```

SWITCH_TIMING_CHAR 1

Turns on switch timing characterization. Syntax:

```
SWITCH_TIMING_CHAR [0 | 1]
```

EXTERNAL_CONTROL_PIN poweron_ads

Defines external control pin; required keyword for timing characterization. Syntax:

```
EXTERNAL_CONTROL_PIN <external control pin name>
```

INTERNAL_CONTROL_PIN poweron_ads_int

Defines internal control pin; required keyword for timing characterization. Syntax:

```
INTERNAL_CONTROL_PIN <internal control pin name>
```

CONTROL_SLEW 2 100p 200p

Defines transition times in ps (default, 100 ps). Syntax:

```
CONTROL_SLEW <number of values> <transition time 1>
    [ <transition time 2> ..]
```

INT_TIMING_ON2OFF POWERON_ADS F

```
4.18423e-11 4.64905e-11 6.99287e-11 5.5511e-11
```

Specifies internal pin timing for On-to-Off transitions. Syntax:

```
INT_TIMING_ON2OFF <external_ctrl_pin> [R | F]
< In>Out delay in S for transition 1 >
<internal slew in S for transition time 1>
[< In>Out delay in S for transition 2 >
<internal slew in S for transition time 2>...]
```

where [R | F] specify the <external_ctrl_pin> waveform is rising or falling to make the switch OFF2ON or ON2OFF.

INT_TIMING_OFF2ON POWERON_ADS R

```
4.16594e-11 4.44381e-11 7.21476e-11 5.44803e-11
```

Specifies internal pin timing for Off-to-On transitions. Syntax:

```
INT_TIMING_OFF2ON <external_ctrl_pin> [R | F]
< in>out delay in S for transition time 1>
<internal slew in S for transition time 1>
[< in>out delay in S for transition time 2>
<internal slew in S for transition time 2> ...]
```

A sample aplsw configuration template is shown below:

Sample APLSW config file to characterize a power switch

```
SPICE_NETLIST <spice_simulation_netlist.cir>

# Specify the switch type
SWITCH_TYPE <HEADER|FOOTER>
DEVICE_MODEL_LIBRARY <spice_model_library> <corner>
INCLUDE <includes_for_parameters.inc>
EXT_PIN <external_vdd_pin>
INT_PIN <internal_vdd_pin>
GND_PIN_NAME <gnd_pin>
VDD_PIN_NAME <external_vdd_pin>
VDDVALUE <volage_value>
```

```
# Specifies that the <ctl_pin> has to rise to be turned on
CONTROL_PIN <ctl_pin> R -

# Specify the voltage level for control pin during on state
ON_STATE {
   <ctl_pin>  1.1
}

OFF_STATE {
   <ctl_pin>  0
}

POWER_UP {
   <ctl_pin>  1.1
}

# Specifies the DC bias conditions for other pins in the
switch netlist
DC_BIAS <bias_pinA> 1.1
DC_BIAS <bias_pinB  0
```

Advanced Switch Characterization

Note: If all switch enable waveforms are relatively fast and linear, with transition times on the order of a few picoseconds, the advanced switch characterization described in this section is *not* required.

For switches that have multiple control pins, significant transition times (for example, a single driver might be controlling a group of switches), or non-linear behavior, using the MD_PWL keyword in the switch configuration file is recommended ('MD_PWL 1'). The behavior of non-linear or slow transition control pin waveforms can be specified in the GSR using the keyword 'PIECEWISE_SWITCH_INPUT <filename>', which specifies a switch input file that describes the input waveforms for the required switch pins in piecewise linear detail. The format of the switch input file is as follows:

```
<switch_inst_name> <ctrl_pin_name> ( <t1> <volt1> <t2> <volt2> ... )
...
```

where times are in seconds and voltages are in volts.

APLSW Switch Model Generation

Once a configuration file is created for each switch, list the location of all configuration files in another master configuration file, one configuration file per line, along with the name of the switch cell (should be the same as that specified in the configuration file and as specified in the SPICE netlist). For example, a design with two switches, switch_ab and switch_cd needs two configuration files, such as *switch_ab.conf* and *switch_cd.conf.* The location of the individual switch configuration files must be listed in a master configuration list file, with the following internal format:

```
switch_ab <path to config file ab>/switch_ab.conf
switch_cd <path to config file cd>/switch_cd.conf
```

The APL utility, `aplsw`, is then executed on the master switch configuration list file to generate the switch models used in low power analysis, using the syntax:

```
aplsw [-c] [-d] [-o <output_file>] <sw_config_list.conf>
```

where

-c : check generated result

-d : debug mode

-o <output_file> : output file name

<sw_config_list.conf> : switch configuration list file

Non-ideal piecewise linear control pin inputs can be accommodated in the current modeling.

After *aplsw* completes execution, a switch model file, called *aplsw.out* by default, is automatically generated, containing data for each switch model in the APL run directory. The *aplsw.out* file contains a header to identify the nature of the contents, which includes the pin connections and the electrical parameters for each of the four switch states, as well as the voltage and leakage current relationships for a specified number of data points for each condition. Leakage current outputs are provided by 2D or 3D table lookup for up to two control pin inputs, as well as Vdd, so if piecewise linear control pin voltages are available, this additional characterization accuracy can be used.

Note: You should not attempt to edit the switch model files.

APLMMX electrical model processing

In APLMMX configuration file define the internal power domain names and voltage levels using the keyword INTERNAL_POWER_PIN_NAME and internal ground domain names using the keyword INTERNAL_GND_PIN_NAME, as described on page 3-65. Do not specify internal power/ground domain pin names using configuration file keywords VDD_PIN_NAME and GND_PIN_NAME.

Sample APLMMX configuration file for power gated designs

A sample of the APLMMX config file with the keywords specific to power gated designs is shown below. These keywords are to be used in as well as the basic APLMMX keywords for any design.

```
# Specify the external power pins here.
VDD_PIN_NAME VDD
GND_PIN_NAME VSS
VDD    1.0

# Specify the internal power pins here.
# The hierarchical name of the internal nets
# are specified in the format shown below.
INTERNAL_POWER_PIN_NAME {
   VDD_sw 1.0 {
       Xtop.XblkA.vvdd_sw
       Xtop.XblkB.vvdd_sw
  }
}

INTERNAL_GND_PIN_NAME  {
    VSS_sw 0 {
        Xtop.XsubA.vss_sw
    }
}
# Specify the pairing of internal/external pins here.
VP_PAIRING (VDD_sw VDD)
VP_PAIRING (VSS_sw VSS)
...
```

ACE Characterization

The ACE utility provides equivalent power circuit resistance, device capacitance, and leakage current data. For details on running ACE, see the *RedHawk User Manual,* Chapter 9.

GDS2DB physical model processing

Creating the Switch Macro Model

The following keywords added to the basic GDS2DB configuration file provide the information needed to build the switch model:

- USE_LEF_PINS_FOR_TRACING [0 | 1] - when set to 1, uses geometries defined in the PINS section of the LEF file of the block being extracted to start tracing its power and ground nets during GDS translation. No text or label-based extraction is performed. However, user-specified coordinates are honored. Remove this if the Text tracing method is used
- EXTRACT_SWITCH_CELLS {
  <switch name> <HEADER | FOOTER>
  <external net name> <internal net name>
  }
- DEFINE_SWITCH_CELLS {
  <switch_name> [HEADER| FOOTER] <input/external LEF pin name>
  <output/internal LEF pin name> <control_pin1> <control_pin2> ...
  ...
  }
- SWITCH_CELLS_PROPERTIES {
  SMASH_GEOMETRIES <0/1>
  PIN_CREATION_LAYERS [ALL | <layer1> <layer2> ... | <contactLayer>]
  }

  where

  SMASH_GEOMETRIES - specifies whether the geometries within the switch cell are brought to the top cell or not. “1” means that the geometries within the switch cell are extracted and brought to the top level. Default is “1”

  PIN_CREATION_LAYERS - specifies the layers within the switch cell on which the switch cell pins are created.

  ALL - uses the metal geometries from all layers within the switch to create the switch cell pins.

  <layer1> <layer2> ... - uses only geometries from the specified metal layers for pin creation.

  <contact_layer_name> - If the contact layer name is specified as one of the PIN_CREATION_LAYERS, then the result depends upon ‘CONTACT_RESISTANCE’ keyword setting.
- CONTACT_RESISTANCE [0 | 1] - turn on this keyword to extract resistance below metal geometry (default 0)

  where

  0 : pins are created on M1, at the same location as the real contacts of the same net. (tap contacts are NOT used)

1: pins are created on the overlapping diffusion layer, at the same location as the real contacts of the same net (tap contacts are NOT used). If “CONTACT_RESISTANCE 1" is specified, and ‘PIN_CREATION_LAYERS’ is not specified, then the creation of pins is on diffusion, at the same location as that of the contacts (tap contacts are not considered).

- COMPLETE_PIN_SNAPPING [0 | 1] - when set, provides the most accurate transistor pin snapping based on device layers in GDS layout (default On).
- XTOR_MAP_FILE <filename> - specifies the transistor mapping file that includes transistor name, location, and pratio (power strength ratio) (optional if it is a GDS- only flow).
- SWITCH_MODEL_MODE ONSTATE - sets the mode of the switch to On state (the only power gated setting now available).

Sample GDS configuration file

```
MACRO_TYPE memory_mmx
XTOR_MAP_FILE ./MNOP123.gdsmmx >> generated by APLMMX
TOP_CELL Top_EFG123
GDS_FILE ./user_data/gds/XYZ_345.gds
GDS_MAP_FILE ./user_data/gds/gdsABC.map

LEF_FILE {
./user_data/lef/XYZ_345.lef
}

VDD_NETS {
  VDD {
    VDD:
    VDD
  }
  VVDD {
     - @ 31:0 26.5333 71.6488
     - @ 31:0 26.9444 29.7189
  }
  VDD_sw {
     - @ 31:0 53.4588 74.9033
  }
}

GND_NETS {
  VSS {
    VSS:
    VSS
  }
  VVSS {
     - @ 31:0 55.5344 25.4889
  }
}

USE_TOP_LEVEL_TEXT_ONLY 1
```

```
EXTRACT_SWITCH_CELLS  {
gwd_ftr_TUV_33 FOOTER VSS VVSS
vvdd_hdr_TUV_44 HEADER VDD_sw VVDD
}

DEFINE_SWITCH_CELLS   {
gwd_ftr_TUV_33 FOOTER VSS VSS2 sleepx
vvdd_hdr_TUV_44 HEADER VDDM VVDD abcx_rb45
}

SWITCH_CELLS_PROPERTIES {
  PIN_CREATION_LAYERS <tech_layer_name>
}

SWITCH_MODEL_MODE ONSTATE
## only ONSTATE is available Totem; no RAMPUP support
```

GDS configuring file for a single switch case



A sample file for the single switch case above is shown below:

```
TOP_CELL < design name >
GDS_FILE < gds pointer >
GDS_MAP_FILE < layer map pointer>
VDD_NETS {
  Ext_VDD1
  Int_VDD1
}
GND_NETS {
      VSS
}
EXTRACT_SWITCH_CELLS {
    sw1  HEADER  EXT_VDD1  INT_VDD1
}
DEFINE_SWITCH_CELLS {
         sw1 HEADER VDD_EXT VDD_INT EN1
}
# For Net tracing using lef pins
LEF_FILE {
<design LEF file pointer>
}
USE_LEF_PINS_FOR_TRACING 1
SWITCH_MODEL_MODE ONSTATE
```

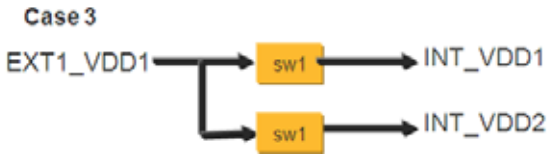
GDS configuring file for the two independent switch case



A sample file for the case above, involving multiple external/internal P/G net pairs connected by unique switch cells, is shown below:

```
TOP_CELL                < design name >
GDS_FILE                 < gds pointer >
GDS_MAP_FILE        < layer map pointer>
VDD_NETS {
  Ext_VDD1
  Ext_VDD2
  INT_VDD1
  INT_VDD2
}
GND_NETS {
      VSS
}
EXTRACT_SWITCH_CELLS {
    sw1 HEADER EXT_VDD1 INT_VDD1
    sw2 HEADER EXT_VDD2 INT_VDD2
}
DEFINE_SWITCH_CELLS {
        sw1 HEADER VDD_EXT VDD_INT EN
        sw2 HEADER VDD_EXT VDD_INT EN
}
# For Net tracing using LEF pins
LEF_FILE {
    <design LEF file pointer>
}
USE_LEF_PINS_FOR_TRACING 1
SWITCH_MODEL_MODE ONSTATE
```

GDS configuring file for parallel switch case



A sample file for the case above, involving a single external P/G net connecting to multiple internal P/G nets by the same switch cell, is shown below:

```
TOP_CELL                < design name >
GDS_FILE                 < gds pointer >
GDS_MAP_FILE        < layer map pointer>
VDD_NETS {
  Ext_VDD1
```

```
  Int_VDD1
  Int_VDD2
}
GND_NETS {
      VSS
}
EXTRACT_SWITCH_CELLS {
    sw1 HEADER EXT_VDD1 INT_VDD1 EN1
    sw1 HEADER EXT_VDD1 INT_VDD2 EN1
}
DEFINE_SWITCH_CELLS {
         sw1 HEADER VDD_EXT VDD_INT EN
}
# For Net tracing using LEF pins
LEF_FILE {
    <design LEF file pointer>
}
USE_LEF_PINS_FOR_TRACING 1
SWITCH_MODEL_MODE ONSTATE
```

GDS configuring file for two switches in parallel case



A sample file for case above, involving a single external P/G net connecting to multiple internal P/G nets by different switch cells, is shown below:

```
TOP_CELL < design name >
GDS_FILE < gds pointer >
GDS_MAP_FILE < layer map pointer>
VDD_NETS {
  Ext_VDD1
  Int_VDD1
  Int_VDD2
}
GND_NETS {
      VSS
}
EXTRACT_SWITCH_CELLS {
    sw1 HEADER EXT_VDD1 INT_VDD1 EN1
    sw2 HEADER EXT_VDD1 INT_VDD2 EN2
}
DEFINE_SWITCH_CELLS {
        sw1 HEADER VDD_EXT VDD_INT EN
        sw2 HEADER VDD_EXT VDD_INT
# For Net tracing using LEF pins
LEF_FILE {
    <design LEF file pointer>
}
```

```
USE_LEF_PINS_FOR_TRACING 1
SWITCH_MODEL_MODE ONSTATE
```

LEF Extraction

If the LEF for the embedded switches is not available, then you can use gds2def to create a LEF model for use in the flow. The following example of a basic configuration file, and the embedded comments, explain the procedure.

```
TOP_CELL <switch cell> # ---> switch cell name in the GDS file
GDS_FILE <IP cell GDSII file>
GDS_MAP_FILE <layer map file>
VDD_NETS {
# < list texted nets connected to the switch; examples below >
    <VDD_EXT>
    <VDD_INT>
}
GND_NETS {
    <VSS>
}
OUTPUT_DIRECTORY .
GENERATE_LEF_PINS 1 # '_ADS' is added to each pin name
OUTPUT_APACHECELL 0 # '0' means to extract the LEF file
```

Extract the LEF file with the command

```
gds2def <config_file>
```

After running GDS2DEF, the switch LEF file, *<cellname>_adsgds.lef,* is generated. Repeat this process for each unique switch master cell in the switched IP GDS file.

Define power gating On-Off switching with the GSC file

The state of instances for power gating analysis must be defined in the GSC file. You have three options to specify the state in low power analysis:

1. Specify the state of BOTH switch instance and power-gated block/instance:
   ```
   <switch_instance> OFF
   <gated_block> ON
   ```
2. Specify domain_name to include everything in that domain:
   ```
   *<domain_name> ON
   ```
3. Combine 1 and 2 for more detailed case:
   ```
   <switch_instance> <domain_name> OFF
   <gated_block> <domain_name> ON
   ```

Wildcards (*) can be used for defining sets of blocks and instances.

For information on setting states for switches using the GSC file, see section "Global Switching Configuration (GSC) File", page C-770).

Defining and Importing switches using the GSR file

In addition to the regular GSR keywords for analysis, use the following GSR entry to import the switch models generated by the *aplsw* utility into Totem:

```
SWITCH_MODEL_FILE {
<APL_switch_model_file>
   }
```

You must ensure that all internal and external power and ground networks that are connected through the switches and that need to be simulated by Totem are listed in the

GSR file, along with their associated voltage values. Following is an example of the referenced GSR keywords.

```
VDD_NETS {
    VDD_INT 1.08
    VDD_EXT 1.08
}

GND_NETS {
    VSS_INT 0
    VSS_EXT 0
}
```

Some additional GSR keywords that can be used for power gating are:

- FREQ <frequency> - defines the operating frequency.
- DYNAMIC_SIMULATION_TIME <sec> - defines dynamic simulation time.
- EXTRACT_INTERNAL_NET 1 - sets internal net extraction.
- GSC_FILE <GSC file> - defines GSC filename
- POWER_MODE APL - defines APL power mode.
- APL_FILES {
  <current model> current
  <cap model> cap
  }
- LIB_FILES
  {
  <lib file pointer >
  }
- LEF_FILES {
  <User IP lef file>
  #switch Lef file generated in gds2def flow
  <gds2def path>/<macro_name_adsgds1.lef>
  }
- DEF_FILES {
  <def pointer>
  }
- GDS_CELLS {
  <cellname> <gds2def path for the on-state physical model>
  }
- PAD_FILES {
  <pad pointer>
  }
- STA_FILE {
  <design name> <timing file pointer>
  }

Adding and deleting power switches

Power gating switches can be added and deleted using the TCL commands ‘eco add switch’ and ‘eco delete switch’. See section "eco", page D-936, for details on command usage.

Running Totem Low Power Analysis

Once the switch models are read in and the database is set up, connections are made between the external power and/or ground networks through the header and/or footer switches, respectively, using the electrical parameters defined in the switch models.

Note that a new switch model file with different switch models can be substituted during a Totem run using the TCL command

```
gsr set SWITCH_MODEL_FILE <switch_model_filename>
```

which overrides the existing switch model file. Then design extraction and analysis can be easily rerun with different switch models specified in the new file.

Now you are ready to invoke Totem analysis for the power gated design.

Mixed Mode Low Power Analysis

To execute mixed mode low power analysis, set the GSR keyword 'MIXED_MODE 1'. Then use the commands 'setup analysis_mode lowpower' and 'perform analysis -dynamic' to run low power analysis in mixed mode. A sample TCL file is shown below:

```
setup design GENERIC.gsr
perform pwrcalc
setup analysis_mode lowpower
perform extraction -power -ground -c
setup package
setup wirebond
setup pad
perform analysis -dynamic
```

Power Gating Results

Reports on switches in the design

Totem reports the switch names traced in a separate file, *adsRpt/apache.memsw.info*. The *totem.log* file points to the file *adsRpt/apache.memsw.info* containing the names of the macros containing switches, the switch master names, the switch count, and the switch internal domains.

switch_static. rpt File

Following Totem static analysis, the *adsRpt/Static/switch_static.rpt* file is generated. This lists the voltage at the internal node of all the switches, the voltage drop across them, and the current through them, as seen in the static analysis. The report has the following format:

```
#6.1.RD (Oct 31 10:09:57 2006)
#Report static results of switch voltage and current (mA)
#instance_name   type  internal_node_volt  volt_across_switch  avg_current
    SW/FSW10        footer  1.946112e-05  1.515508e-05  2.117815e-04
    SW/FSW20        footer  1.857796e-05  1.482933e-05  2.072293e-04
    SW/FSW30        footer  1.940953e-05  1.542746e-05  2.155878e-04
    SW/FSW40        footer  1.888943e-05  1.490737e-05  2.083198e-04
    SW/FSW50        footer  1.862162e-05  1.502620e-05  2.099804e-04
```

switch_dynamic.rpt File

Following Totem dynamic analysis, the *adsRpt/Dynamic/switch_dynamic.rpt* file is generated, which lists the peak current through active header and footer switches for ramp-up analysis (in Amps), and includes the status of each header or footer switch at the end of simulation (OFF or ON), allowing you to easily see which switches are turned on and their real efficiency. The report has the following format:

```
#Report dynamic results of switch voltage and current
# Instance_name    Type    Max_Isw(Amp)   End_simul_state
    SW/FSW1           footer  OFF
    SW/FSW2           footer  3.1390e-10 ON
    SW/FSW9           footer  1.0030e-12 ON
    SW/FSW3           footer  OFF
    SW/FSW2           footer  3.137452e-10 OFF
    SW/FSW7           footer  OFF
```

LDO Low Power Designs

Overview

To support advanced techniques related to supply voltage control, such as multi-supply multi- voltage (MSMV), and dynamic voltage and frequency scaling (DVFS), on-chip low drop-out regulators (LDOs) are widely used to control each power domain. So LDO-aware voltage drop analysis has become significant for verifying stability of the voltage supplied from the LDOs. This section describes the generation of LDO models and how to use them in Totem to perform static and dynamic voltage drop analysis that accurately capture the response of on-chip LDOs. The LDO-based flow in Totem is shown in Figure 5-18.

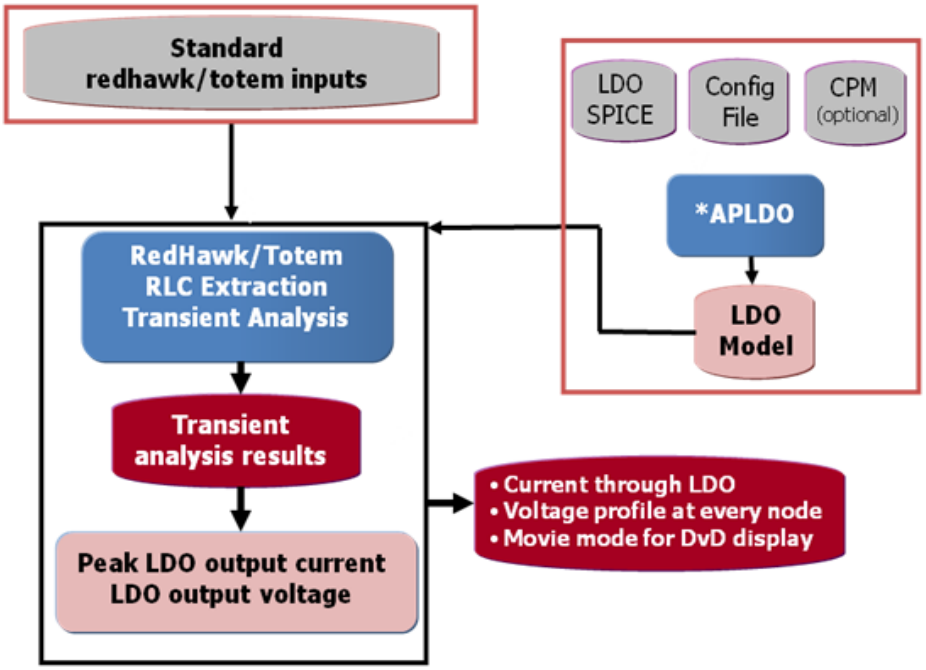


Figure 5-18 LDO analysis flow

LDO design modeling

Figure 5-19 shows a schematic of a typical LDO used in designs. To generate the LDO model, get DC biasing information for all the inputs to the LDO netlist.

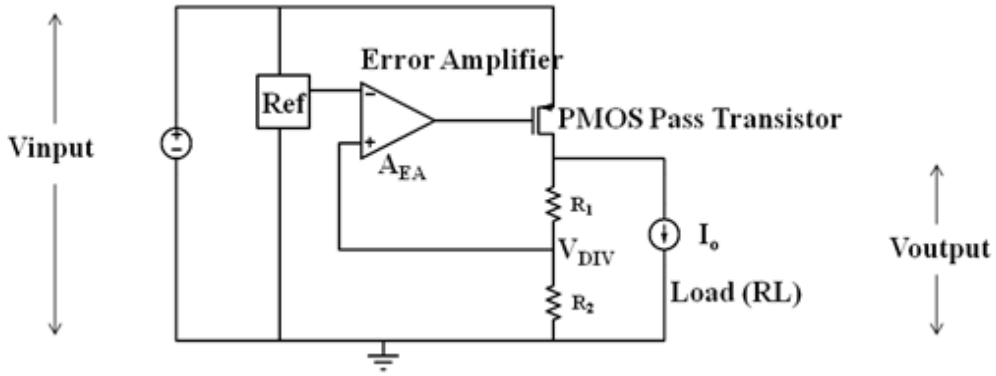


Figure 5-19 Typical LDO schematic

There are two types of LDO models generated by Totem, static and transient. Static models are current-controlled voltage sources, while transient models also capture the transient behavior of LDOs, which account for the instantaneous drop in the voltage caused by high current demand.

The LDO-based data flow is shown in Figure 5-20. The details of the procedure are given in the following paragraphs.

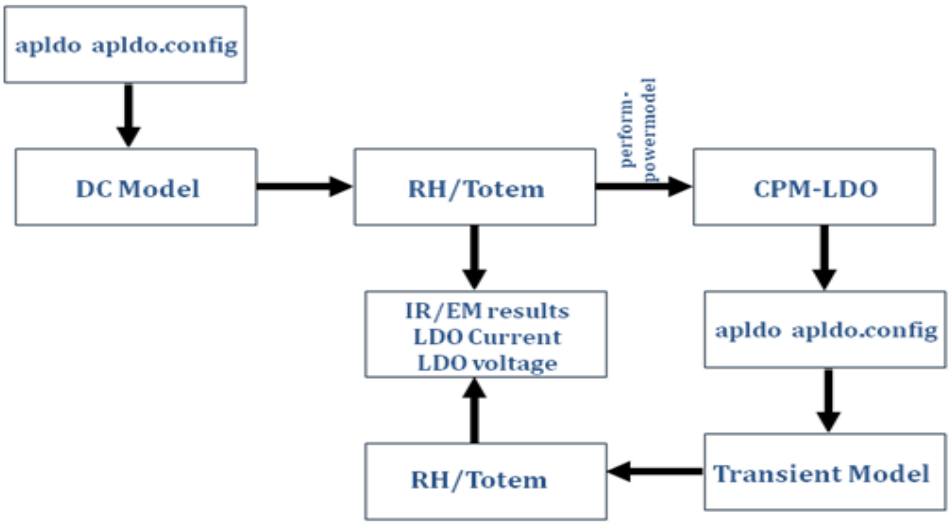


Figure 5-20 LDO data flow

There are four basic steps in LDO-based dynamic flow. In the LDO DC-based flow, only the first two steps are required.

1. Generate an LDO DC model using the APLDO utility (see section "LDO Low Power Designs", page 5-202).

2. a. Generate a CPM model that includes the LDO input/output ports and possibly some internally-probed nodes.

   To use LDO models, the following keyword must be in the GSR file:

   ```
   LDO_MODEL_FILE {
       <ldo_model_file_from_APLDO>
    }
   ```

   The 'DECOUPLE_LDO_GROUND 1' GSR keyword setting is recommended for the LDO-based flows. When running the LDO flow in Totem and 'DECOUPLE_LDO_GROUND 0' is used, the LDO output voltage subtracts the Vss voltage at each time point. If 'DECOUPLE_LDO_GROUND 1' is specified, the original LDO output voltage is reported, which is usually desired. The default is 0.

   Also, for LDO analysis the following keyword should be set in the *gds.config* file if an LDO is present in GDS:

   ```
   WHITE_BOX_CELLS {
      <ldo_cell>
   }
   ```

   b. Use the generated DC LDO model in the Totem to create a CPM-LDO of the die, using the following command:

   ```
   perform powermodel vcd -pincurrent -global_gnd -o CPM_LDO
       -probe  -reportcap
   ```

   where

   -probe : reads the <probe_node_file> defined in the PROBE_NODE_FILE GSR keyword, which is required if you want to check the DvD of the nodes in CPM+LDO_xtor+pkg runs outside of Totem. The format of <probe_node_file> is the following:

   ```
   <x in um> <y in um> <layer> <node_name>
   ```

3. For dynamic models, generate an AC LDO model, either for a load regulation dynamic model considering load variation, or for a line regulation dynamic model considering input variation as well, with CPM+LDO_xtor [+pkg] with APLDO (see the APLDO section for details).

4. Run Totem again with the AC LDO model for either VCD-based or vectorless-based analysis, to get accurate LDO output voltage/current responses depending on the load.

Controlling LDO switching state using GSC file

RedHawk can turn on/off LDO instances in dynamic analysis based on the settings in GSC file.

1. User can decide on-state or off-state for each LDO instance based on GSC setting. The state for a LDO instance cannot be changed during dynamic analysis. Wildcard(*) characters are accepted in this case.

   Syntax:

   ```
   <LDO instance name> [DISABLE|ENABLE]
   ```

   Example:

   ```
   LDO_* DISABLE
   LDO_1 ENABLE
   ```

It means that all LDO instance with name matching LDO_* will be off-state, except for LDO_1, which will be on-state.

2. User can change any state for a LDO instance during dynamic analysis. This usage does not support wildcard(*) characters. The two new GSC file keywords `'LDO_INST_CONFIG'` and `'END_LDO_INST_CONFIG'` should be specified as in below format to enable this.

   Syntax:

   User should add comments using "#" before these 2 keywords.

   ```
   # LDO_INST_CONFIG
   <LDO instance name> [disable|enable] <time:pico sec.>
   [disable|enable] <time:pico sec.> [disable|enable]
   <time:pico sec.> ……...
   # END_LDO_INST_CONFIG
   ```

   Example:

   ```
   # LDO_INST_CONFIG
   LDO_3 enable 10000 disable 15000 enable 20000 disable 25000
   # END_LDO_INST_CONFIG
   ```

   Example for Case1 + Case 2:

   This usage doesn't support wildcard(*). Assuming that there are 4 LDO instances :

   ```
   LDO_1 DISABLE
   LDO_2 ENABLE
   LDO_4 DISABLE
   # LDO_INST_CONFIG
   LDO_3 enable 10000 disable 15000 enable 20000 disable 25000
   # END_LDO_INST_CONFIG
   ```

To check the state of an LDO instance, user can run "`plot current -ldo -name <LDO instance name>`" . The file `adsRpt/Dynamic/ldo.current` will be impacted by this feature.

Outputs generated in LDO-based analysis

The following outputs are generated in LDO-based analysis in the directory *adsRpt/Dynamic*.

- ldo.current (output current waveforms)
- ldo.voltage (output voltage waveforms)
- ldo_dynamic.rpt
  - Min/Max load current value
  - Min/Max output voltage value
  - Min/Max input voltage value
  - Min/Max load current di/dt value

LDO Modeling with APLDO

APLDO is the LDO modeling utility that generates behavioral LDO models used by Totem. The following are the inputs and outputs for running APLDO:

- Input Data
  - Spice netlist and model parameters
  - proper configuration (keywords to enable modeling features and parameters)
  - CPM file (optional- for optimal dynamic LDO models)

    - package (optional- when package decap data available for LDO output net)
- Output Data
    - encrypted LDO model file (NSpice/HSpice accept it as a subckt)
    - DC model - I-V table
    - transient model - RLC & control sources

Following are example configuration templates for the generation of DC, load regulation dynamic, and line regulation dynamic LDO models used in the APLDO utility:

LDO DC model example configuration file

```
# name of the LDO cell (required)
LDO_CELL LDO_cell

# name of output file (optional with default <ldo_cell>_apldo.mdl>)
#OUTPUT_FILE

# Spice netlist containing subcircuit definitions. Can be more than one
SPICE_NETLIST ../totem/netlist_subckt_vcc

# temperature condition (optional with default value 25 degree C)
TEMPERATURE 25

# DEVICE_MODEL_LIBRARY defines Spice device model (optional)
DEVICE_MODEL_LIBRARY ./user_data/SPICE/device.lib TT

# include files (optional)
INCLUDE ./totem/tt

# special Spice options (optional)
OPTION GSHUNT=1e-12

# path to the Spice executable (optional -- NSpice is used by default).
#SPICE_SIMULATOR hspice /appls/synopsys/C-2009.09-SP1/hspice/linux/hspice

# minimum idle current (required)
#  start current value in the model
MIN_IDLE_CURRENT 1.0mA

# minimum load current (A) (required)
# the minimum load current in the application
# generally, may set it the same as MIN_IDLE_CURRENT
MIN_LOAD_CURRENT 1.0mA

# maximum load current (A) (required)
MAX_LOAD_CURRENT 320mA

# dc current sweep step
DC_CURRENT_SWEEP_STEP 2mA

# output power pin name and ideal voltage (required)
```

```
POWER_OUTPUT_PIN vccr 2.2

# minimum output voltage (optional)
# MIN_OUTPUT_VOLTAGE 1.08

# minimum input voltage (optional)
# MIN_INPUT_VOLTAGE 2.40

# analog power input pin name (required)
POWER_INPUT_PIN vcc_in 3.0

# general ground pin name (required)
GROUND_PIN gnd 0

# dc bias
# <pin_name> <voltage>
DC_BIAS vcc_in         3.00
DC_BIAS gnd            0.0
DC_BIAS vbgr           1.25
```

APLDO example configuration file for load regulation dynamic model

```
# name of the LDO cell (required)
LDO_CELL LDO_cell

# name of output file (optional with default <ldo_cell>_apldo.mdl>)
#OUTPUT_FILE

# Spice netlist containing subcircuit definitions.  Can be more than one.
SPICE_NETLIST ../totem/netlist_subckt_vcc

# temperature condition (optional with default value 25 degrees C)
TEMPERATURE 25

# DEVICE_MODEL_LIBRARY defines Spice device model (optional)
DEVICE_MODEL_LIBRARY ./user_data/SPICE/device.lib TT

# include files (optional)
INCLUDE ./totem/tt

# special Spice options (optional)
OPTION GSHUNT=1e-12

# path to the Spice executable (optional -- NSpice is used by default).
#SPICE_SIMULATOR hspice /appls/synopsys/C-2009.09-SP1/hspice/linux/hspice

# minimum idle current (required)
# the start current value in the model
MIN_IDLE_CURRENT 1.0mA
```

```
# minimum load current (A) (required)
# the minimum load current in the application
# generally, may set it the same as MIN_IDLE_CURRENT
MIN_LOAD_CURRENT 1.0mA

# maximum load current (A) (required)
MAX_LOAD_CURRENT 320mA

# dc current sweep step
DC_CURRENT_SWEEP_STEP 2mA

# output power pin name and ideal voltage (required)
POWER_OUTPUT_PIN vccr 2.2

# minimum output voltage (optional)
# MIN_OUTPUT_VOLTAGE 1.08

# minimum input voltage (optional)
# MIN_INPUT_VOLTAGE 2.40

# analog power input pin name (required)
POWER_INPUT_PIN vcc_in 3.0

# general ground pin name (required)
GROUND_PIN gnd 0

# dc bias
# <pin_name> <voltage>
DC_BIAS vcc_in         3.00
DC_BIAS gnd            0.0
DC_BIAS vbgr           1.25

# Defines the top level subckt name of CPM + LDO test bench (required)
CPM_LDO_MODEL adsPowerModel

# CPM_LDO_FILE <file name>
# To specify filename of the CPM + LDO test bench (required)
CPM_LDO_FILE ./power_model_CPM-LDO

# CPM_LDO_PORT <port name> <value>
# Defines the bias conditions for CPM_LDO_MODEL.  Users must check the
# PLOC and set the corresponding pad value to the ports of CPM_LDO_MODEL.
# Wildcards supported to specify the port name of CPM_LDO_MODEL (required)
CPM_LDO_PORT *:VCC 3.0
CPM_LDO_PORT *:VSS 0

# CPM_OPEN_PORT <port name>
# Defines the open ports of CPM_LDO_MODEL. The defined ports remain floating
# while characterizing the LDO model. Wildcards are supported to specify the
# port name (optional)
CPM_OPEN_PORT VBB:*
```

```
# CPM_LDO_TRANSIENT_TIME <time>
# Defines the transient simulation time in APLDO characterization.
# Users may refer to the Totem simulation time for CPM model generation
# and specify the same in CPM_LDO_TRANSIENT_TIME for LDO models (optional).
CPM_LDO_TRANSIENT_TIME 120ns

# the subckt name of PKG model (optional)
# but is required when CPM_LDO_PORT is not specified
# PACKAGE_MODEL <subckt name>
PACKAGE_MODEL REDHAWK_PKG

# the PKG model filename (optional)
# it is required when PACKAGE_MODEL is given
# PACKAGE_FILE <pkg_file_name>
PACKAGE_FILE redhawk_pkg_0.spi
```

Generating the DC LDO model

Use the following command to generate the DC LDO model:

```
% apldo apldo.config
```

For APLDO debugging help, set “DEBUG 1" in configuration file, which keeps the .apache/APLDO directory for the following files:

- ldo_dc_vdd.cir (for dc sweep simulation)
- ldo_ac_vdd.cir (for ac sweep simulation)

Testing LDO models

Generated LDO models are encrypted, so you can perform sanity checking on the model by running Spice simulations on the *< >_LOAD_REGULATION_TEST_BENCH.sp* file in the apldo working directory.

Also, the command ‘aplreader -ldo <LDO model>’ can be used to plot LDO output I-V curves.

Other practical LDO applications

The LDO methodology described not only analyzes voltage drop, but also can assist in adjusting the on-chip LDO size (that is, the drive strength) and the locations for power reduction and chip shrink. For example, by replacing a normal-sized on-chip LDO with one that is half-sized, designers can compare the two voltage waveforms at the output pin of those LDOs, as shown in Figure 5-21. If the half-sized one meets design constraints in voltage drop, designers can use it instead of the normal-sized one. This means that as another application, this method provides a what-if analysis for adjusting on-chip LDO size. Moreover, designers can adopt the method without any modifications to low power techniques such as MSMV and DVFS. And designers have other advantages in chip modeling, such as Chip Power Model (CPM), for chip-package co-design, using this method.

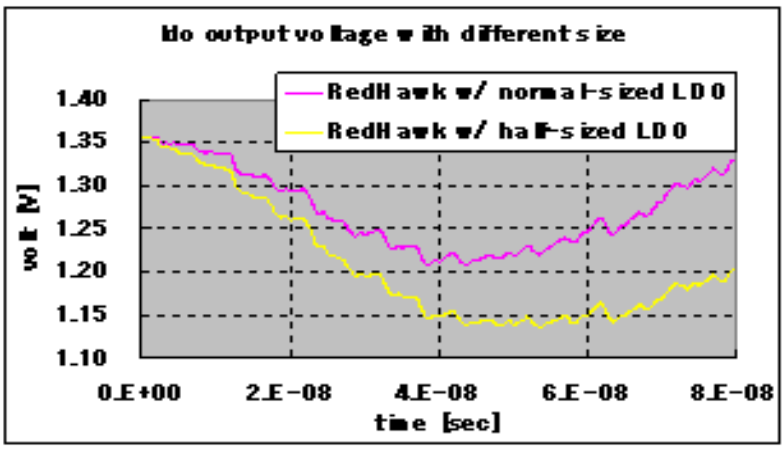


Figure 5-21 LDO output voltage for normal and half drive strength

Stand-along power-gate analysis

For a stand-alone power-gate IP (GDS containing only a large/custom power-gate) , Totem allows creating custom output port pins for accurate analysis. It could also be used for voltage regulator / LDO block design. User need to set the configuration keyword PROBE_POWER_OUTPUT_PORT to 1 in APLMMX configuration file.

There are two options to provide the switch pin details in `aplmmx.config` file:

Option 1:

Using the DSPF_MAP file. In this method, APLMMX will create TYPE{T} pins with PSL {} corresponding to the layer in DSPF.

Defintion in `aplmmx.config`:

```
PROBE_POWER_OUTPUT_PORT 1
DSPF_MAP <>.map
```

Sample of generated gdsmmx file:

```
##<>.gdsmmx
#<pin> <device> <x> <y> <pratio> <mdl>
vddA.gds1 vddA 20 30 100 TYPE{T} PSL{M5}
vddB.gds1 vddC 20 10 100 TYPE{T} PSL{M1}
vddC.gds1 vddD 10 20 100 TYPE{T} PSL{M1}
```

Option 2:

- This method uses the keyword, POWER_OUTPUT_PORT_MODEL which allows to customize the power output port layer, bounding box and pitch of node creation as below syntax:

In `aplmmx.config`:

```
PROBE_POWER_OUTPUT_PORT 1
POWER_OUTPUT_PORT_MODEL
```

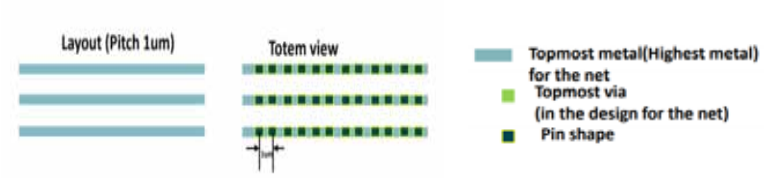
```
{
    <P/G net1> <metal_layer> <pitch> <bbox>
<P/G net2> <metal_layer> <pitch> <bbox>
    ……
}
```

The `<layer>, <pitch> and <bbox>` definitions are optional. Also, the default value of layer is highest metal layer for that net.

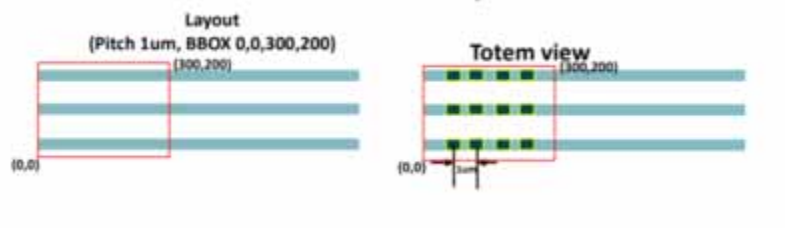
Example aplmmx config:

```
POWER_OUTPUT_PORT_MODEL
{
    vddA M5 10 0,0,300,200
    vddB M1 - -
    vddC - - 0,0,300,200
}
```

Here, for `vddB`, APLMMX will create multiple `TYPE{T}` pins on M1 at 1u pitch as shown below since the pitch and bbox is defined with `'- -'`



For, vddC, APLMMX will create multiple pins on the highest metal layer within the bbox (0,0,300,200) with a pitch of 1um as shown below since the metal layer is defined with '-'.



Sample of generated gdsmmx file:

```
##<>.gdsmmx
#<pin> <device> <x> <y> <pratio> <mdl>
vddA.gds2 XM1 10 20 101 TYPE{M} TID{P1}
...
vddB.gds1 vddC - - 100 TYPE{T} PSL{M1}
vddB.gds2 XM2 10 20 101 TYPE{M} TID{P1}
...
vddC.gds1 vddD - - 100 TYPE{T} PSL{hm} BBOX{0,0,300,200}
vddC.gds2 XM3 10 20 101 TYPE{M} TID{P1}
```

Full Custom Hierarchical Designs

DRAM/Flash memories

Typically, DRAMs and FLASH types of memories are made up of highly repetitive cell architectures. To handle large designs in Totem it is necessary to understand the requirements for analysis before tying to simulate the design. Totem has a unique capability to perform hierarchical analysis by combining simulation currents from different hierarchies to the top level. This gives the ability to switch the different modules or hierarchies at different times and use different switching states. Guidelines on analyzing FLASH and DRAM designs are provided below.

FLASH design operating modes

Analysis of power noise integrity for FLASH designs requires a knowledge of the different operating states of a FLASH memory. The following are different properties associated with the different modes:

Read mode

- Read mode has lower average power consumption, but has much shorter access times. The peaks and di/dt values are very important during read operation. Read mode is also the most frequently used operation in the lifetime of a FLASH memory, so it is important to analyze this mode accurately for Power Noise and Reliability issues.
- Page buffers (including Sense Amp) are major power consuming components during this operation. The page buffer is highly repetitive in structure and in timing during the read operation. It is important to model this switching activity accurately.

Erase mode:

- Erase mode occurs over a larger time period and has lower peak currents. However, the average current is much higher over the duration. The Charge Pump domains mainly work for erase and program models.

Program mode:

- Page buffer domain works for Read and Program mode.

A typical FLASH design architecture is shown in Figure 5-22

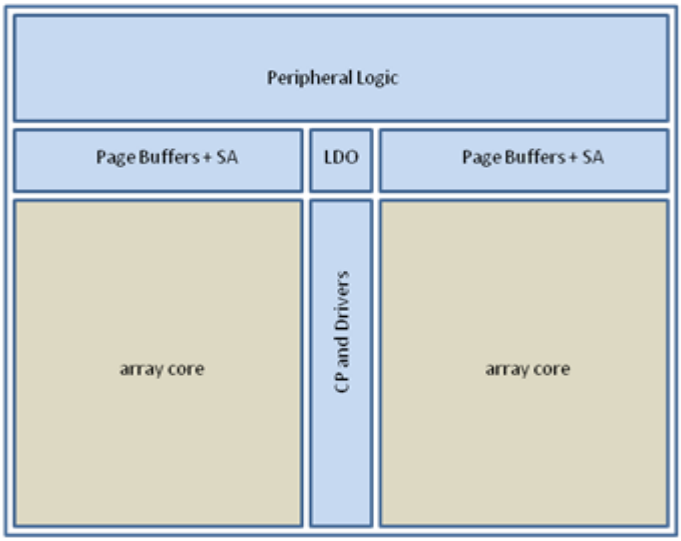


Figure 5-22 Typical flash architecture

Handling large design sizes

FLASH designs are typically very large to simulate as a single monolithic piece. All Spice level simulations are performed at different hierarchies in the design and then verified. However, for power noise analysis it is important to be able to simulate the noise at the full chip level. Totem can combine different GDS hierarchies and different hierarchy currents at the top level. The physical model and the electrical model is created for smaller pieces of the design and then combined automatically during Totem top level analysis. A schematic explanation of the flow is shown in Figure 5-23.

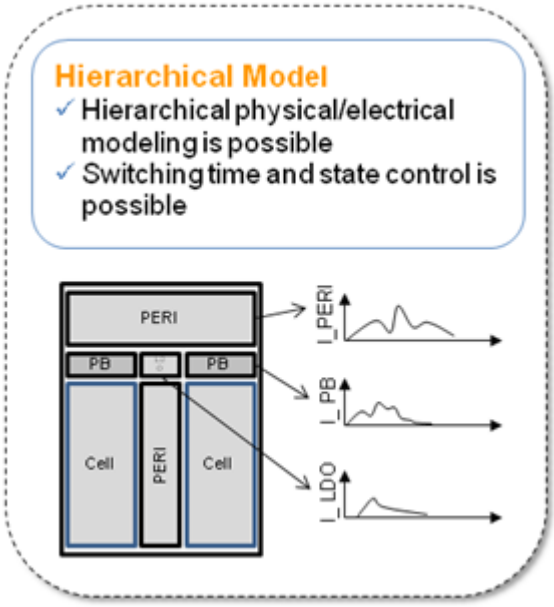
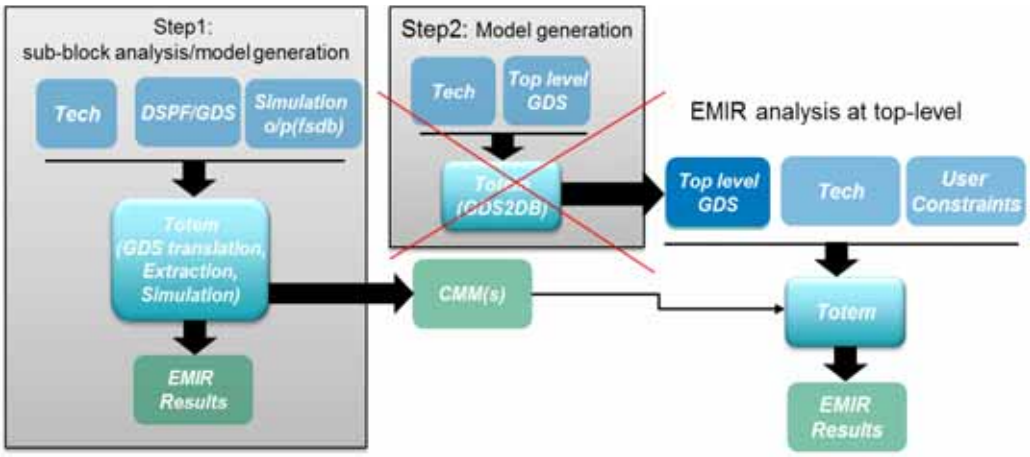


Figure 5-23 Totem hierarchical modeling flow

The Hierarchical flow enables running top-level GDS2DB and analysis in a single run. User need to specify top-cell under GDSII_FILES and provide hierarchy (box-cell) views under GSR (CMM / DEF)

Earlier users had to run it in two steps:

1. Gds2db on top-level (by specifying box-cells as required) to generate LEF/DEF
2. Totem analysis using LEF/DEF of top-level (from step 1) + CMM / def of box-cells.



Usage:

```
CMM_CELLS {
block1 block1_cmm
block2 block2_cmm
}
GDSII_FILE {
Top top.gds.config full
}
```

GDS config file gds.config:

```
GRAY_BOX_CELLS {
block1 def_view
block2 def_view
}
GDS_FILE top.gds
GDS_MAP_FILE <layer_map>
..
```

Totem will automatically determine the logical connections based on the physical data in CMM model.

Mixed Signal Designs

Setup for Mixed-Mode Custom Design Flow

In Totem-MMX analysis, you can mix P&R blocks with standard cell-based APL models together with custom macros having transistor-level MMX models in the same simulation environment to simulate a full custom design.

For an analysis that includes P&R blocks, you can characterize the standard cells inside the P&R blocks using regular cell-based APL methodology.

In addition to the basic GSR keywords described for custom macro analysis previously, the keywords required for custom mixed-mode analysis are as follows:

- DEF_FILES <def_files> - top-level and P&R blocks DEF files
- LEF_FILES <lef_files> - top-level and P&R blocks LEF files
- APL_FILES <apl_files> - electrical models from APLMMX and regular APL
- STA_FILE <sta_file> - STA timing window file of the custom design to describe the instance switching timing windows
- VCD_FILE <vcd_file> - VCD file of the custom design to describe instance switching activity

The following is an example of a typical GSR file for Totem-MMX analysis for custom design analysis:

```
VDD_NETS {
  VDD 1.8
}
GND_NETS {
  VSS 0
}
TECH_FILE user_data/TOP.tech
LIB_FILES {
  user_data/LIB/stdcell.lib
  user_data/custom.lib custom
}
DEF_FILES {
  user_data/pnr.def block
  user_data/TOP.def top
}
LEF_FILES {
  user_data/stdcell.lef
  user_data/pnr.lef
  user_data/TOP.lef
}
GDS_CELLS {
  RAM256  user_data/GDSMMX mmx
}
APL_FILES {
  user_data/RAM256.spcurrent current
  user_data/RAM256.cdev cdev
  user_data/stdcell.spcurrent current
  user_data/stdcell.cdev cdev
}
PAD_FILES {
```

```
  user_data/TOP.ploc
}
STA_FILE
{
  FREQ_OF_MISSING_INSTANCES 125e6
  TOP user_data/TOP.chip.timing
}
VCD_FILE
{
TOP user_data/TOP.vcd
FILE_TYPE VCD
front_path "top/mem/"
substitute_path ""
frame_size 8000
TRUE_TIME 1
}
GSC_FILE                user_data/RAM256.gsc
INPUT_TRANSITION        0.1n
TOGGLE_RATE             0.2
FREQUENCY               500e6
DYNAMIC_SIMULATION_TIME 2e-09
DYNAMIC_TIME_STEP       5e-12
```

For more details about mixed signal design setup, see the *RedHawk User Manual*.

3DIC setup for EMIR

Totem support Power EMIR flows on 3DIC WoW and interposer structure for concurrent and model-based analysis. The flow requires user to prepare the multi-die configuration file to setup the connections between different DIE using “`setup3dic`” utility. (Please contact Ansys AE for latest documentation & support on this utility).The physical database of individual DIE should be prepared upfront using a chip level Totem run. The overall setup is similar to RedHawk 3DIC analysis. The individual DIE and Interposer can be full-custom or digital block with corresponding setup GSR.

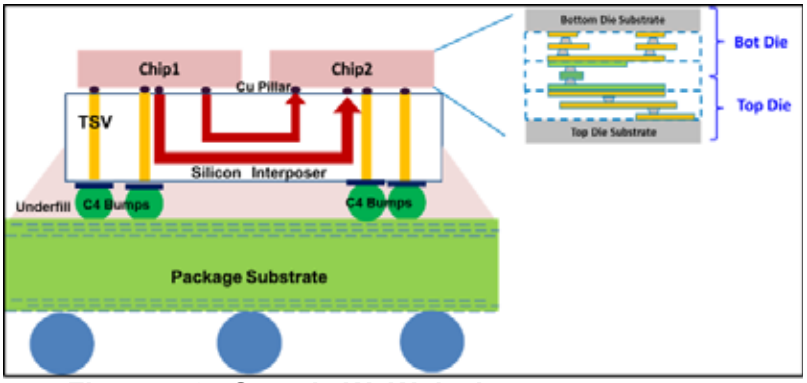


Figure 5-24 Sample WoW design

Sample Tcl command file:

```
#Import 3DI config generated using “setup3dic” utility
setup 3dic WoW.config
#multi-DIE GSR
import gsr system.gsr
#Import the Interposer GSR
import gsr FCCC_Interposer.gsr -dir Interposer
#Import GSR for individual DIE
import gsr FCCC_TOP.gsr -dir DIE_C1_TOP
import gsr FCCC_BOT.gsr -dir DIE_C1_BOT
import gsr FCCC_TOP.gsr -dir DIE_C2_TOP
import gsr FCCC_BOT.gsr -dir DIE_C2_BOT
setup design
# Rest of the flow
perform pwrcalc
perform extraction -power -ground -l -c
setup pad
setup wirebond
setup package
perform analysis -dynamic
perform emcheck
export db TT3DIC.db
```

User can filter individual DIEs or interposer layers using Layer Selection Window(LSW) in Totem GUI. Totem Current and IR maps can be visualized concurrently on multiple DIEs like RedHawk.

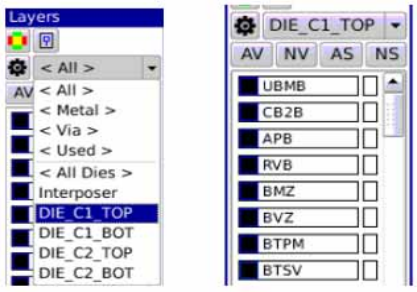


Figure 5-25 Totem DIE based layer selection in LSW

Totem also supports multi-die analysis with CPM model for other DIE like RedHawk.

Note: Totem can support digital design only up to 1 Million instances. For multi-die designs with large digital blocks it is recommended to use RedHawk-3DIC flow. Please consult your local ANSYS support team for further guidance on bringing up multi-die analysis in Totem.

I/O Devices

The power delivery network for high-speed I/Os such as SERDES and DDRs need to be analyzed with caution. Typically, the I/O power domain is isolated from the main core domains to prevent cross-coupling of I/O noise with core voltages. The I/O power domains handle a large amount of current due to large off-chip capacitive loading. When analyzing voltage domains that interact with off-chip components it is important to consult with your PI/SI/SSO application engineers.

Chapter 6
Results Evaluation

Introduction

After grid check, static IR drop, electromigration and dynamic voltage drop analysis on memories and custom macros, a number of types of results are available for detailed examination, such as

- P/G grid weakness checks
- static IR drop
- electromigration analysis
- dynamic voltage drop (DvD) analysis
- mixed-mode custom design analysis
- “what-if” analysis

Evaluating these types of results are described in the following sections.

P/G Grid Integrity Checking

After the TCL command 'perform gridcheck' or Static IR drop analysis has been run, view the results using the GUI command **View -> Connectivity -> List of PG Weakness** for either Instances or Transistor Pins. A ‘Report of gridcheck weakness for transistor pins’ dialog is displayed, as shown in Figure 6-1, which ranks effective power grid resistance for all transistor pins in order of severity. Look for anomalies and differences on the list. From the **Connectivity** menu you can also display a list of Missing Vias or Shorts.

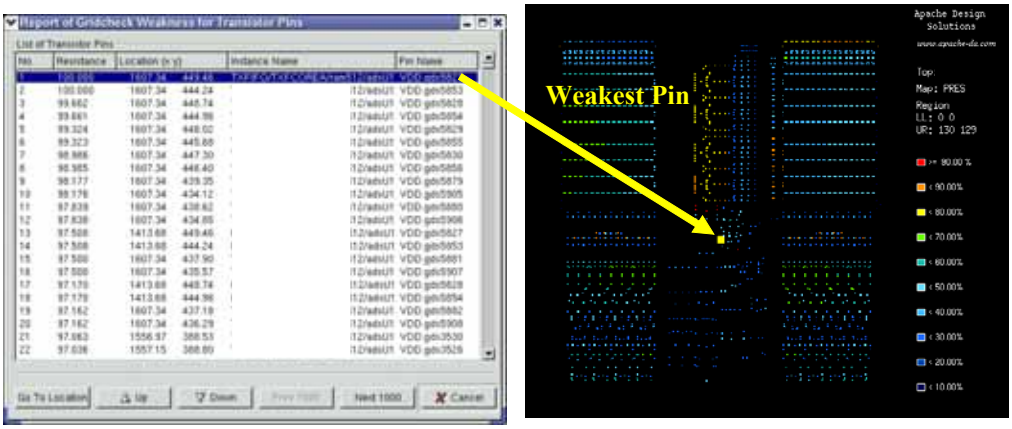


Figure 6-1 Report and color map of P/G weakness for transistor pins

Select a pin on the list and click on the 'Go to Location' button to zoom to the highlighted pin in the layout. Click on the menu command **View -> Transistor Pin Maps -> Resistance Map**. Look at the pattern of color-coded pins on pin resistance map; by default red represents the most serious grid weaknesses, such as undersized grids and missing vias.

The output of the ‘perform res_calc’ command is displayed as a list of the highest actual grid resistances (worst 500 by default). Review the list for unusually high pin resistances.

Make changes to weak grid areas and vias with the Fixing and Optimization (FAO) grid modification tools (see the FAO chapter in the Totem Users' Manual). Rerun static IR analysis and recheck the transistor pin resistance report to see if the unbalanced resistance problems have been fixed. The grid check report is in the *apache.macro.gridcheck* file in the *adsRpt* directory.

To get transistor pin information, use the following procedure:

1. Click on ‘View Layers’ Configuration button to bring up the Layers dialog box.
2. Check the ‘Pin Inst’ box on the transistor layer (usually it is the bottom layer, such as Metal1).
3. Select ‘Instance Outline’ and then the ‘Apply’ button.

   The transistor pin geometry is displayed in the layout window.
4. Click on the desired geometry in the layout window and the transistor information is displayed in the message window.

Static IR Drop Analysis

After static IR analysis has been run, use the GUI commands **Results -> List of worst IR for Wire and Via**, and **Results -> List of Worst Transistor Pin Voltages**, which rank the locations of worst average voltage drops for wires and vias, and for transistor average pin voltages, respectively, in order of severity. Look for anomalies and differences in entries on the two lists. Select an item from a list and zoom to the highlighted display of the problem area on the layout, as shown in Figure 6-2.

Use the GUI command **View -> Transistor Pin Maps** to display full chip color-coded maps of Voltage Drop, Resistance, and Current. Look for locations of high voltage drop, high pin resistance and high current to help determine the cause of the voltage drop hot spots.

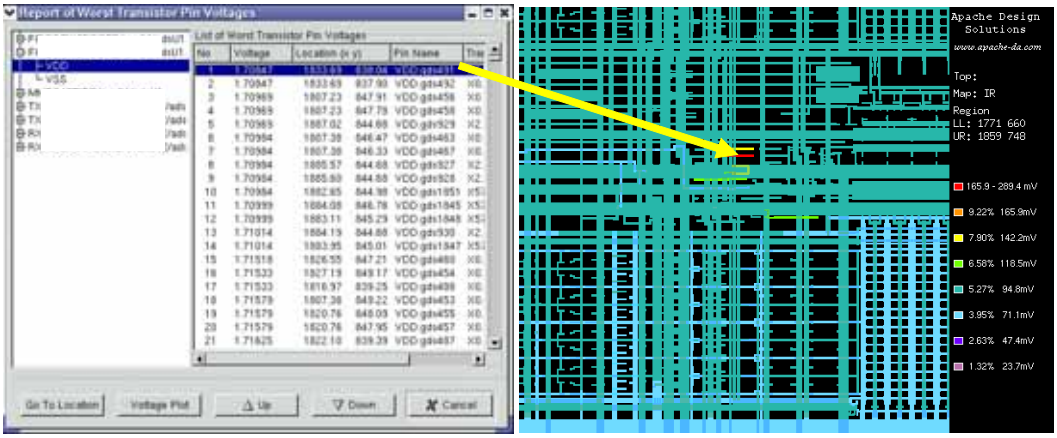


Figure 6-2 Report and colormap of worst transistor pin voltages

Look at the pattern of color coding of the IR drop problems, which could be caused by undersized grids and/or missing vias. Use the 'Set color range' button in the 'Config' panel of the GUI to bring up a color range dialog to understand the meaning of the display colors.

Click on the 'View Layers' button on the 'Config' panel in the GUI. Compare voltage drop and current maps layer-by-layer, starting at the top layer, to discover the potential cause of the hot spots, such as missing vias and undersized grid structures.

Make appropriate changes to grids and vias with the Fixing and Optimization (FAO) tools. Perform incremental extraction, then rerun static IR drop analysis, and recheck reports of static IR drop.

The transistor pin-based static IR report is in the *<design>.ir.mmx* file in the *adsRpt/Static* directory.

EM Analysis

List of Worst EM display

After running static IR and EM analysis, use the GUI command **Results -> List of Worst EM**, which ranks locations of worst average current EM locations in order of severity, and identifies the end points of the high EM segments, the net name, and the percent of EM limit represented. Select an item from the list and zoom to the highlighted location, as shown in Figure 6-3. The left hand side of the dialog shows the net browser (separated by Power and Ground domains). The right hand side is the detailed report of Power or Ground domains, or a specific net. Note that the List of Worst EM by default brings up the Power domain selection. If you want to see Ground domain or all EM violations for the entire design, either switch to Ground domain by clicking the Ground domain entry in the left hand side browser, or use the multi-select feature to hold the “Ctrl” button, and then select both Power and Ground domains to display all design EM violations.

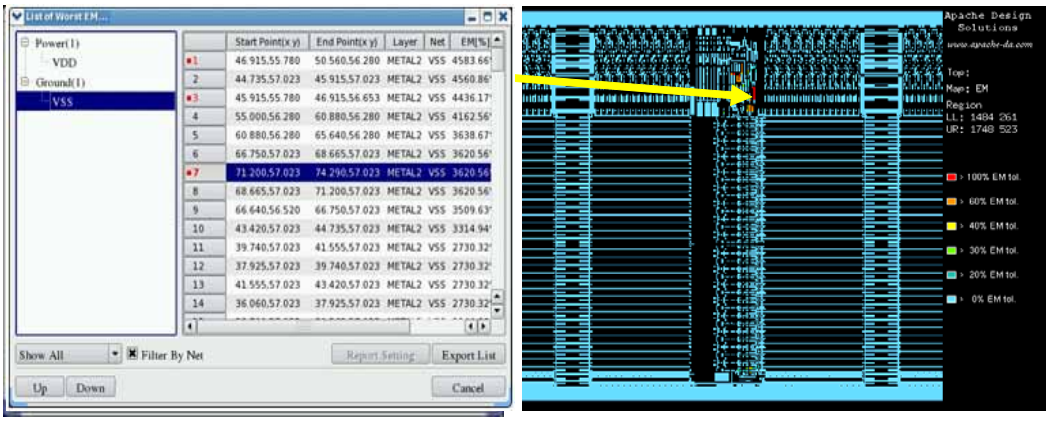


Figure 6-3 Report and color map of worst EM locations

You can mark certain EM violations by clicking on them the list. After marking the EM violations, use “Show Marked”, “Show Unmarked” and “Show All” options in the dialog box to change which violation items are displayed. This feature is supported in the save/reload flow, so the markings are stored in the database and restored after reloading the database.

Note that columns in “List of xx” results displays can be moved by clicking on the header and dragging it to the desired new location.

Other results display

You can display color maps using the 'EM' button and 'CUR' button on the GUI, or look at the EM and current values layer-by-layer, starting at the top layer. Layers to be displayed can be selected using the 'View Layers' button, which brings up the 'Layers' dialog. Look at the pattern of color-coded EM and current values. You can use the 'Set color range' button in ‘Config’ panel of the GUI to bring up a color range dialog to see the meaning of the display colors. Review combinations of high EM and high current values for each layer to evaluate hot spots and critical EM wire segments and vias.

Also you can look at the EM results in the text report *<design>.em.worst* in the *adsRpt/Static* directory, which lists in tabular form: the via or metal layer name, the coordinates of the violating segment or via, the EM percentage (EM value * 100, divided by the specified EM limit), the net name, and the wire width (if it is a wire EM violation). See section "Apache Technology File (*.tech)", page C-744, for the ‘metal’ and ‘via’ keywords used to specify EM limits for the design.

Also you can make appropriate changes to grids and vias with the FAO modification tools, or perform incremental extraction, then rerun static IR drop and EM analysis, and recheck reports and color maps of EM.

Dynamic Voltage Drop Analysis

MMX Lists and Colormaps of Critical Conditions

After dynamic analysis has been run, use the GUI command **Results -> List of worst Transistor Pin Voltages**, which ranks the locations of worst dynamic voltage drop for transistor pins in order of severity, as well as the location, the pin name and transistor name, as shown in Figure 6-4. Look for anomalies and differences on the list. Select a location from a list and zoom to a highlighted display of its location. You can also click on the column header to sort the ranked list based on either minimum voltage, maximum voltage, or average voltage over the whole simulation time.

Use the GUI command **View -> Transistor Pin Maps** for displaying color maps of either Voltage Drop or Current. Look at the DvD and current values layer-by-layer, starting at the top layer. Layers to be displayed can be selected using the 'View Layers' button, which brings up the 'Layers' dialog. Look at pattern of color coding of dynamic voltage drop - by default red represents most serious DvD problems. Use the 'Set color range' button in Config panel of the GUI to bring up a color range dialog to see the meaning of the display colors.

To see a list of disconnected transistor pins, check the following:

- file *adsRpt/MMX/disconnected_xtor_pins.rpt*
- the TCL command 'dump mmx_pin_info' (without -avg_volt_tw) shows all pins, but if the option '-report_disconnected' is used, a flag indicates which pins are connected and which pins are not.
- the menu command **Results-> List of worst transistor pin voltages** shows the disconnected pins with voltages listed as “NA”.

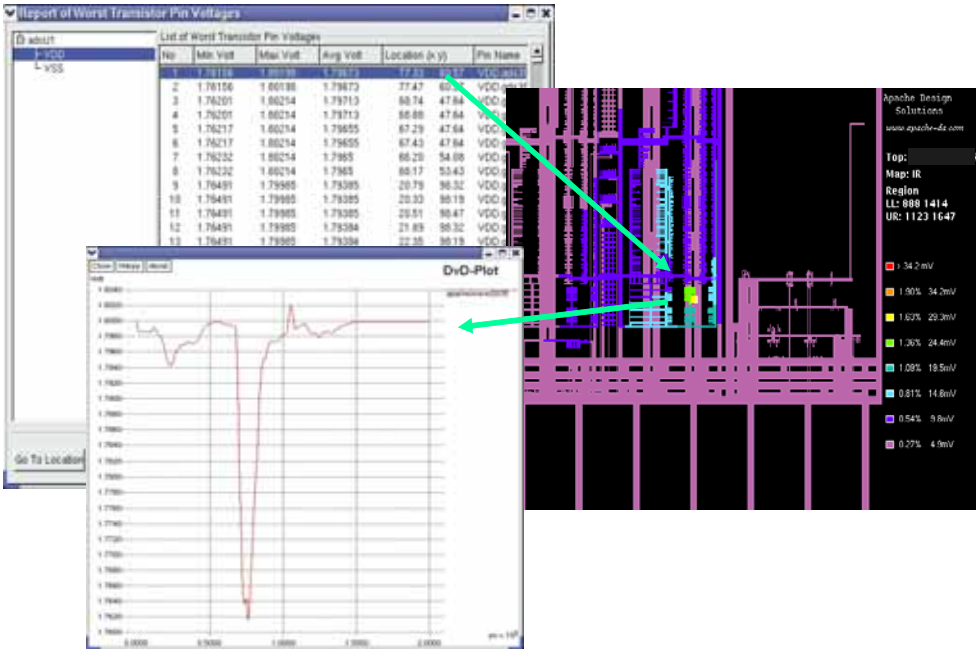


Figure 6-4 List, color map and plot of worst transistor pin DvD

View plots of worst-case current and voltage waveforms using the TCL command 'plot current <options>' and 'plot voltage <options>' for worst case DvD conditions (the GSR keyword DYNAMIC_SAVE_WAVEFOM, must be left at its default value of 1), as shown in Figure 6-4. Use the TCL command 'plot analysis <options>' to display histograms of specific types of DvD results. See section "plot", page D-972 for syntax and details on using the plot commands.

To see step-by-step changes in transistor-level voltage during logic sequencing over the timing window, use the TCL command 'movie make -transistor', and then 'movie show -transistor'.

Make appropriate changes to decap or grids with FAO modification tools. Perform incremental extraction, rerun DvD analysis, and then recheck reports of DvD issues. A transistor pin-based DvD report is saved in the *<design>.dvd.mmx* file in the *adsRpt/Dynamic* directory.

To get transistor pin dynamic voltage waveforms, use the following GUI procedure:

1. Click on the menu command **Results -> List of Worst Transistor Pin Voltages.**
2. Select Instance and a power or ground domain in the pop-up dialog.
3. Enter either a transistor name using Transistor Name or a pin name using Pin Name, then click the Find button.
4. Switch to search ‘By Transistor Name’ or ‘By Pin Name’ by clicking on the appropriate button.
5. When the selected object has been highlighted, click on ‘Voltage Plot’ to plot the waveform.

To get dynamic voltage waveforms using a TCL command, use:

```
plot voltage -name <inst_name> -pinname <power_pin_name>
```

For example, a TCL command to plot waveforms could be:

```
plot voltage -name adsU1 -pinname VDD.gols525
```

Histograms

You can plot a transistor pin voltage drop histogram of MMX analysis results to identify voltage drop extreme values and visualize the voltage drop distribution among all transistor pins. Use the option '-mmx_pin' with the 'plot analysis' command to generate the transistor pin histogram. The syntax is:

```
plot analysis -type [DvD | StaticIR | StaticEM | DynamicEM]
?-layer [<metal> | ALL]?
?-net [<netname> | ALL]? ?-instance [avgTW | minTW | maxTW | minCyc |
    instDrop | vddDrop | vssBounce]?
?-lower <lower>? ?-upper<upper>? ?-o <filename>?
?[-binsize <size> | -binnumber <number>]?
?-nograph? ?-mmx_pin?
```

If '-mmx_pin' is specified, valid values for '-type' are 'DvD' and 'StaticIR' only. Note that options '-mmx_pin', '-instance', and '-layer' are mutually exclusive, and only one of them can be specified. For example:

```
plot analysis -type DvD -mmx_pin
plot analysis -type DvD -mmx_pin -net vss -lowerLim 10 -upperLim 300
    -binnumber 2 -o xtor_pin_hist1
```

A screen shot of a sample transistor pin DvD histogram is shown in Figure 6-5 below:

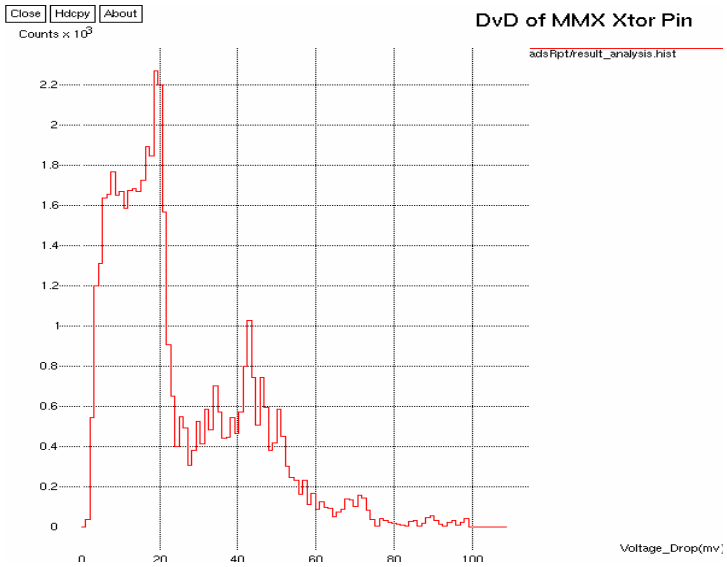


Figure 6-5 Transistor pin DvD histogram

Decap Results

To display a list of intrinsic and intentional decap by power/ground domain, use the following TCL command:

```
print decap -net <netname>
```

For example,

```
print decap -net VDD
```

A sample decap list printout:

```
DECAP STATISTICS:
Rectangular region: Full chip
```

```
Total decap on net VDD: 4.569 pF
Total transistor intentional decap on net VDD: 0.202 pF
Total PG cap on net VDD: 0.165 pF
```

To generate an intentional transistor decap report, which contains information such as maximum transient decap current and minimum effective voltage over simulation time of all intentional decap transistors. The TCL syntax is:

```
print mmx_decap_report ?-o <outfile>?
```

The default output file is *adsRpt/Dynamic/decaps_mmx.rpt*. Example:

```
# MMX intentional decap report
#time unit: ps
#current unit: amp
#voltage unit: volt
#Note: 'max_current@time' indicates maximum current and time point of the
intentional decap transistor
#Note: 'min_dvd_sim@time' indicates minimum effective voltage over
simulation time and time point of the intentional decap transistor

#decap_xtor_name max_current@time min_dvd_sim@time VDD_Pin GND_Pin InstName
X0.X69.M53 2.913237e-06@1905 1.6127@1840 VDD.gds2435 VSS.gds2257 adsU1
X0.X69.M53 3.121334e-06@1905 1.5933@1795 VDD.gds2435 VSS.gds2257 adsU1
X0.X15.M53 1.040567e-06@1935 1.6615@1855 VDD.gds2197 VSS.gds2047 adsU1
X0.X25.M53 3.276817e-06@1905 1.6018@1900 VDD.gds11011 VSS.gds10609 adsU1
X0.X25.M53 3.225260e-06@1905 1.5739@1900 VDD.gds11011 VSS.gds10609 adsU1
X0.X25.M53 3.172728e-06@1905 1.5812@1900 VDD.gds11011 VSS.gds10609 adsU1
X0.X25.M53 3.173094e-06@1905 1.6229@1900 VDD.gds11011 VSS.gds10609 adsU1
```

MMX Design Summary Report

After running dynamic analysis (or static, if desired), you can generate a Design Report for transistor-level results in MMX designs using the menu command **Results -> Design Summary Report**. This report allows you to quickly review voltages and currents that exceed selected limits set in a constraint file. The following MMX-specific constraint file keywords are used to specify unacceptable transistor-level performance to be flagged:

- DYNAMIC_XTOR_DVD - dynamic voltage drops exceeding specified limit
- DYNAMIC_XTOR_MAX_VOLTAGE - dynamic voltages exceeding specified limit
- DYNAMIC_XTOR_MIN_VOLTAGE - dynamic voltages below specified limit
- STATIC_XTOR_IR - average static voltages exceeding specified limit

Normal instance-based constraint file keywords are also supported in MMX Design Reports:

STATIC_IR

STATIC_PAD_CURRENT

DYNAMIC_EFFVDD_TW

DYNAMIC_MINVDD_TW

DYNAMIC_MINVDD_CYC

DYNAMIC_PEAK_PAD_CURRENT

OFF_STATE_LEAKAGE_CURRENT

The general syntax for specifying keywords in a constraint file is:

```
<constraint_file keyword> {
    [ <net_name> | all ] <constraint value>
     ...
   }
```

For example:

```
DYNAMIC_XTOR_DVD {
    all 0.05
    }
DYNAMIC_XTOR_MAX_VOLTAGE {
   VDD 1.25
   VSS 0.05
   }
DYNAMIC_XTOR_MIN_VOLTAGE {
   VDD 1.15
   VSS -0.05
   }
 STATIC_XTOR_IR {
   all 0.003
   }
```

After running dynamic (or static) analysis and selecting the menu command **Results -> Design Summary Report -> Generate**, and then **-> Show**, a Design Summary report dialog is displayed, such as in Figure 6-6:

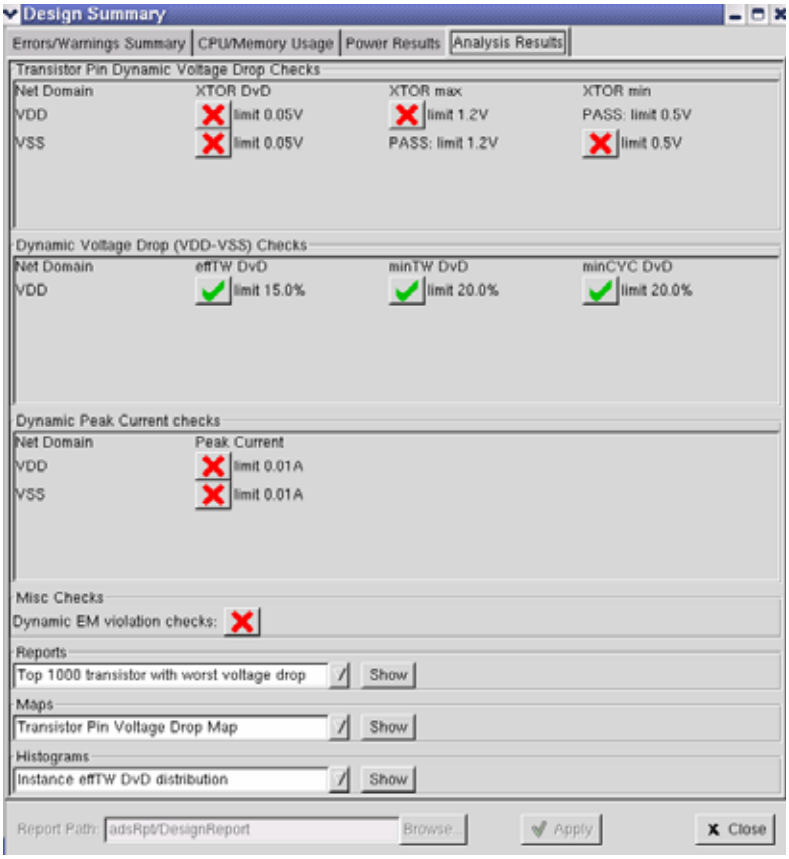


Figure 6-6 Design Summary Report for MMX

The top panel of the dialog relates to transistor level results. The ‘Transistor Pin Dynamic Voltage Drop Checks’ panel lists violations of transistor pin DvD values (‘XTOR DvD’) violations of transistor pin maximum voltage values (‘XTOR max’), violations of transistor pin minimum voltage values (‘XTOR min’), and after static analysis, indicates violations of transistor pin static voltage limits set by ‘XTOR IR’. Red X's indicate constraints violated and green checks indicate constraints met.

The next two panels report on general Totem design constraints set.

Additional MMX reports and maps can be displayed using the selection boxes at the bottom of the Design Summary dialog, as follows:

- ‘Top 1000 transistor with worst IR drop report’ for static analysis,
- ‘Top 1000 transistor with worst voltage drop report’,
- ‘Top 1000 transistor with worst ground bounce report’ for dynamic analysis.
- ‘Transistor Pin Voltage Drop Map’
- ‘Transistor Pin PG Weakness Map’
- ‘Transistor Pin Current Map’, and
- ‘Transistor Pin Decap Map’.

Examples of several of these reports and maps follow.

An example “Top 1000 Transistors with Worst Voltage Drop” list is shown in Figure 6-7:

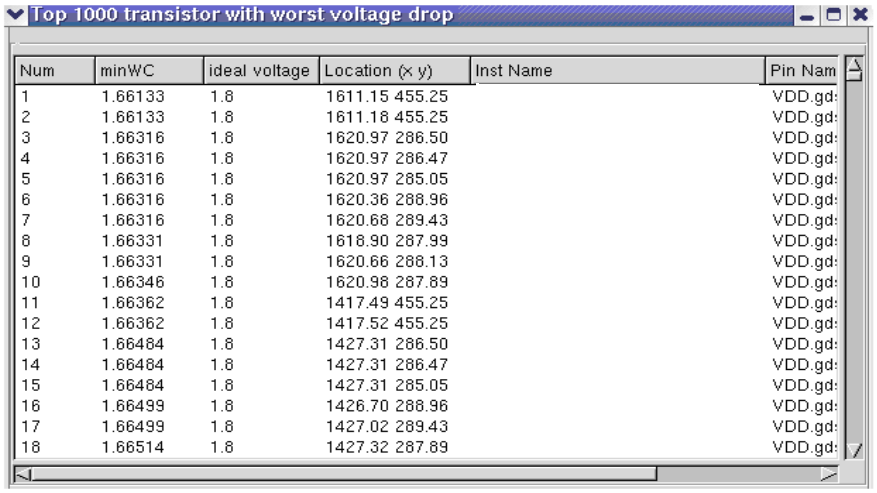


Figure 6-7 Top 1000 Transistors with Worst Voltage Drop list

An example “Transistor Pin Voltage Drop colormap” is shown in Figure 6-8:

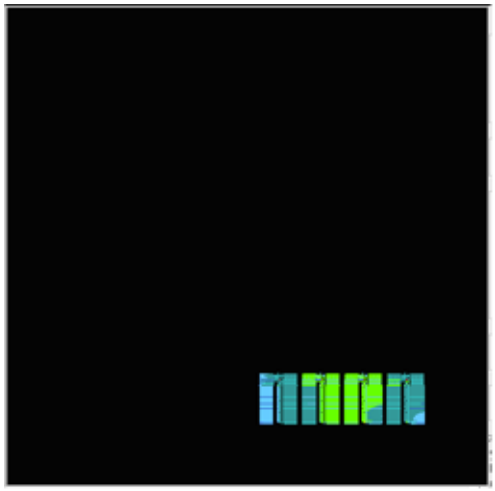


Figure 6-8 Transistor Pin Voltage Drop colormap

Using the menu command **Results -> Supply Current Waveforms**, both current demand and supply current can be plotted. An example set of current plots is shown in Figure 6-9:

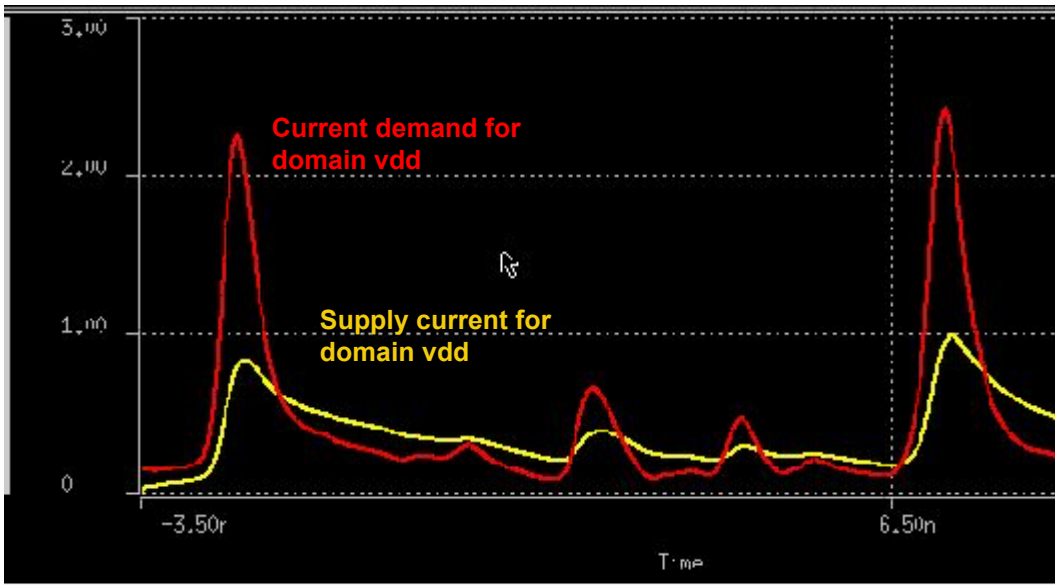


Figure 6-9 Current demand and supply current plots

Mixed-Mode Custom Design Analysis

In mixed-mode custom design analysis you can see both cell-based results for P & R blocks and transistor-level results for custom macros in the same analysis, as shown in Figure 6-10.

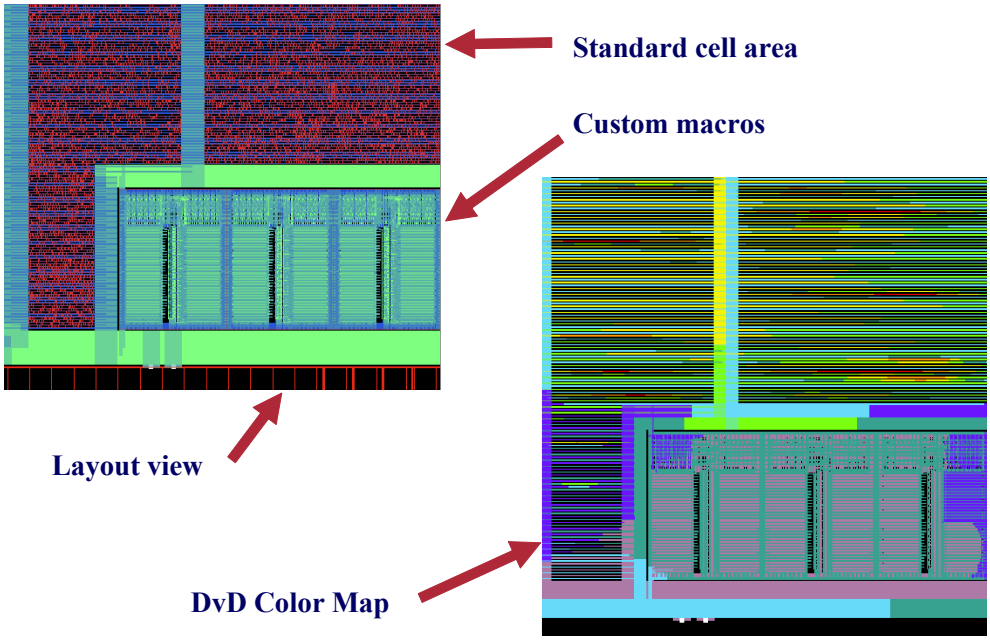


Figure 6-10 Simultaneous analysis of custom macros and standard cell elements

"What-if" Analysis

When the worst resistance, current, and voltage hot spots have been identified, use Totem's easy-to-use Fixing and Optimization (FAO) tools to make trial design modifications to see the effects of the fix, and then rerun the analysis. Design changes such as changing wire width, adding P/G grids, or changing spacing, and modifying or adding vias, in order to solve voltage drop, high current and EM problems, are all easily performed and tested using FAO tools (note that full FAO capabilities to modify decap for DvD repair are not yet available for custom design style). Design changes that result in acceptable fixes can be documented using the ECO features of Totem. For more information on FAO commands and procedures, see Chapter 9, "Fixing and Optimizing Grid and Power Performance".

CMM Reports

CMM Statistics

You can review statistics for nodes, resistors, wires, and for generated CMM cells. When simulation is performed, the *adsRpt/apache.CMM.rpt* file is generated, with a detailed elements counts for all CMM cells in the design. A sample output in *adsRpt/apache.CMM.rpt* file is shown below:

```
#################################################################
#CREATE REDUCED MODEL FOR CMM CELLS {
#CELL_NAME                          ORIG_NODE     REDUCED_NODE REDUCTION
ram1                                   25942           14331  44.76 %
ram2                                   10460            6256  40.19 %
```

```
}
Total CMM original nodes: 526308  Total CMM reduced nodes: 289310
Total CMM original resistors: 675893  Total CMM reduced resistors: 438895
526308 nodes are in CMM blocks out of a total of 538028

#CONNECTION COUNT FOR CMM MODEL CELL INSTANCES {
#CELL_NAME      NET(COUNT)...   INSTANCE_NAME
ram1    VDD(11) VSS(8)  FIFO/RXCORE/ram2
ram1    VDD(9)  VSS(4)  FIFO/TXCORE/ram2
ram2    VSS(4)  MHE/MCFIFO/ram64
}
526308 nodes are in CMM blocks out of a total of 538028

#CALCULATING NODE VOLTAGES FOR CMM INSTANCES {
FIFO/RXCORE/ram2
FIFO/TXCORE/ram2
}

#POST PROCESS RESULT FOR CMM MODEL CELL INSTANCES {
ram1    VDD     VSS     FIFO/RXCORE/ram2
ram2    VSS     MEM/FIFO/ram64
}
```

Wire Voltage Drops Report

Data regarding wire voltage drops are also reported for all CMM instances, either compact or original, in *adsRpt/apache.CMM.rpt*.

Below is a sample section of the report:

```
The worst dynamic voltage drop for the sub block <FIFO/RXCORE/ram2>:
NET<VDD> :
        voltage = 1.770709 at node (1759.360000,656.695000)
        voltage = 1.770862 at node (1772.950000,656.695000)
        voltage = 1.770862 at node (1777.215000,656.695000)
        voltage = 1.770862 at node (1759.360000,656.695000)
NET<VSS> :
        voltage = 0.023494 at node (1763.960000,668.675000)
        voltage = 0.023494 at node (1763.960000,667.335000)
        voltage = 0.023494 at node (1763.960000,661.295000)
        voltage = 0.023341 at node (1763.960000,661.295000)
```

Constraint Violation Summary

After top-level analysis, if there are any constraints defined in sub-block CMMs, a violation summary table is recorded in the *totem.log* file after post-processing of simulation results. An example is shown below:

```
Summary of CMM Constraint Violation:
-----------------------------------------------------
Constraint Type No. of violations
-----------------------------------------------------
Top Connection Min Voltage 50
Top Connection Max Voltage 50
```

```
Pin Min Voltage 500
Pin Max Voltage 500
There are total 1100 constraint violations in the CMM
instances, please look at adsRpt/Dynamic/
CMM_constraint_violation.rpt for detail!
```

Dynamic Analysis Constraint Summary

For dynamic analysis, the final CMM constraint violation report is recorded in the file *adsRpt/Dynamic/CMM_constraint_violation.rpt.* The file format is:

```
# Top connection max voltage constraint violation:
# <inst> <cell> <x,y,layer> <net_name> <constraint> <voltage_value>
#-----------------------------------------------------
# Top connection min voltage constraint violation:
# <inst> <cell> <x,y,layer> <net_name> <constraint> <voltage_value>
#-----------------------------------------------------
# pin max voltage constraint violation:
# <inst:pin> <cell> <x,y> <constraint> <voltage_value>
#-----------------------------------------------------
# pin min voltage constraint violation:
# <inst:pin> <cell> <x,y> <constraint> <voltage_value>
#-----------------------------------------------------
```

For example:

```
# Top connection min voltage constraint violation:
# <inst> <cell> <x,y,layer> <net_name> <constraint> <voltage_value>
#-------------------------------------------------------------
ABC_CORE/ram64/adsU1 ram32x8x2 (1759.3600,783.0350, METAL2) VDD 1.7820
    1.7309
# Top connection max voltage constraint violation:
# <inst> <cell> <x,y,layer> <net_name> <constraint> <voltage_value>
#-------------------------------------------------------------
ABC_CORE/ram64/adsU1 ram32x8x2 (1763.9600,778.4350, METAL2) VSS 0.0010
    0.0528
# Pin min voltage constraint violation:
# <inst:pin> <cell> <x,y> <constraint> <voltage_value>
#-------------------------------------------------------------
ABC_CORE/ram64/adsU1:VDD ram32x8x2 (1792.3700,727.8150) 1.7460 1.7294
# Pin max voltage constraint violation:
# <inst:pin> <cell> <x,y> <constraint> <voltage_value>
#-------------------------------------------------------------
ABC_CORE/ram64/adsU1:VSS ram32x8x2 (1790.1550,727.6450) 0.0100 0.0534
```

Static Analysis Constraint Summary

For Static, the final report is recorded in the file *adsRpt/Static/CMM_constraint_violation.rpt.* The file format is:

```
# Top connection IR constraint violation:
# <inst> <cell> <x,y,layer> <net_name> <constraint> <voltage_value>
#-------------------------------------------------------------

# pin IR constraint violation:
# <inst:pin> <cell> <x,y> <constraint> <voltage_value>
```

```
#-----------------------------------------------------------------
```

For example:

```
# Top connection IR constraint violation:
# <inst> <cell> <x,y,layer> <net_name> <constraint> <voltage_value>
#-----------------------------------------------------------------
ABC_CORE/ram64/adsU1 ram32x8x2 (1759.3600,783.0350, METAL2) VDD 1.7960
    1.7909
# Pin IR constraint violation:
# <inst:pin> <cell> <x,y> <constraint> <voltage_value>
#-----------------------------------------------------------------
ABC_CORE/ram64/adsU1:VDD ram32x8x2 (1792.3700,727.8150) 1.7910
    1.7909
```

Chapter 7
Package and Board Modeling

Introduction

High quality models of off-chip RLC circuit elements such as the package and board can be very significant in achieving an accurate simulation of circuit power. This chapter describes the procedures for creating the required package and board-related circuit models and mapping package port names to die pad names.

Three types of package models are available, a simple model in which all power pads have the same RLC values and all ground pads have the same RLC values. More complicated package modeling, in which different values can be assigned to each pad, are provided by a SPICE subcircuit model. These three types of models are described in the following sections.

Integrated CPA Flow

Totem has been enhanced to support integrated Chip plus package flow within single GUI. The keyword CPA_FILES enables this feature. User needs to specify the package layout and the CPA model output directory inside the CPA_FILES.

CPA Tcl Commands are not supported in Totem flow yet (GUI needs to be used to customize setup).

Usage in GSR:

```
CPA_FILES {
    PACKAGE  <package_layout>
    MODEL <output_model_path>
}
```

PACKAGE: specifies the package layout file to be displayed and imported into Totem-CPA. The supported formats are xfl, mcm, sip and brd.

MODEL: defines the path to CPA project directory. A new directory will be created if it doesn't exist. If directory already exists and contains a previously saved project, it will be re-loaded.

If CPA_FILES is included in GSR, GUI will create CPA menu automatically, and include the 'Package' view into main page window ('Layout' View is still the default view).

The CPA Integrated GUI in Totem is shown in Figure 7-1.

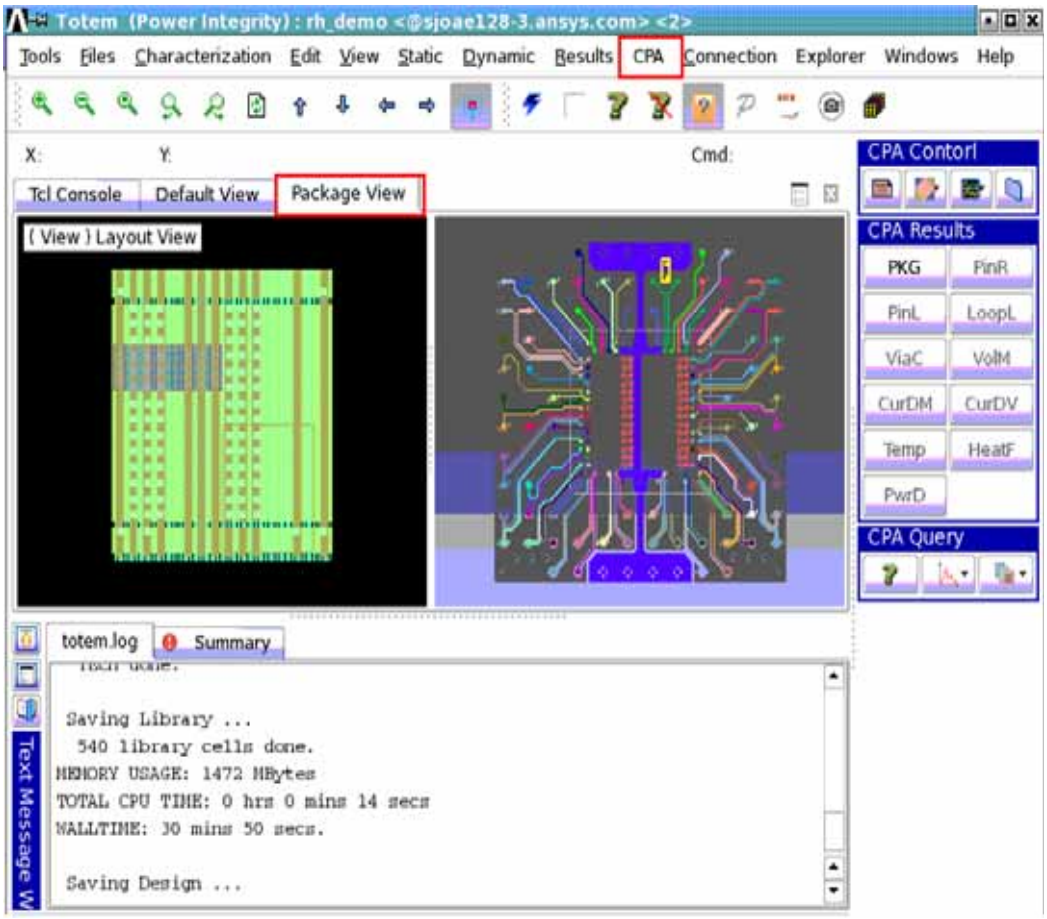


Figure 7-1 CPA Integrated GUI in Totem

Simple Package RLC Model

A simple package circuit model representation in Totem is shown in Figure 7-2.

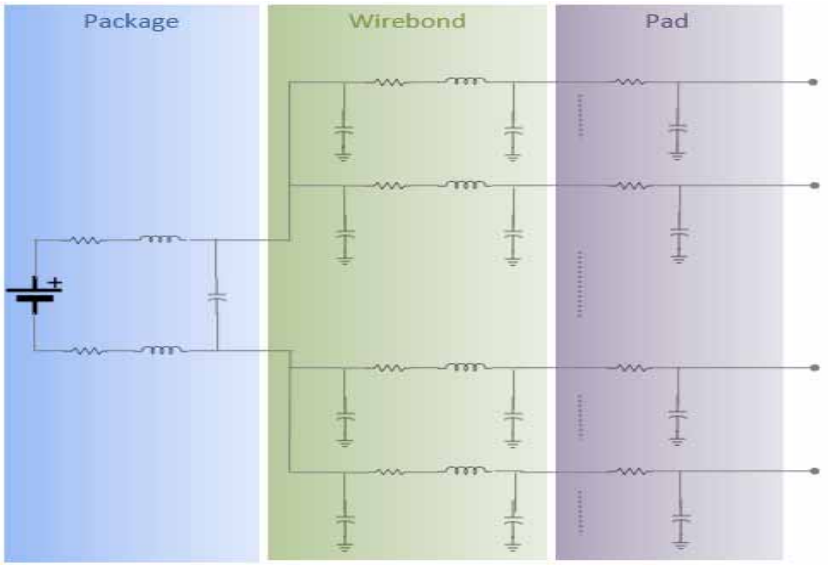


Figure 7-2 simple package RLC model

Package parasitics represent RLC values associated with the package substrate itself. The wirebond represents the bondwire (lead frame to bondpad connection) parasitics. For Flip Chip designs, where there is no lead frame, this can be included as a part of package parasitics. Padwire parasitics represent the parasitics of the pad (or bump) wire routing (bondpad/bump to I/O pad cell connection). If this routing is already included in the DEF, Totem extracts the information automatically; you do not need to specify these values.

The simple model annotates single RLC values for all pads in the design using TCL commands. This can be done separately for power pads and ground pads. All power pads and all ground pads have the same parasitic values with this command.

The commands used for annotating simple RLC package values are:

```
setup package -r <in Ohm> -c <in pF> -l <in pH>
setup wirebond -power/-ground -r <in Ohm> -c <in pF> -l <in pH>
setup pad -power/-ground -r <in Ohm> -c <in pF>
```

Note that asymmetric power and ground package models are supported, and wirebond and pad constraints can be annotated separately for power and ground. Also, an inductance value is not annotated with pad parasitics, as this routing is significantly small, and generally is considered to be zero in simulation.

An example of the simple model specification follows:

```
setup pad -power -r 0.010000 -c 0.5
setup pad -ground -r 0.010000 -c 0.5
setup wirebond -power -r 0.245000 -l 1420 -c 5
setup wirebond -ground -r 0.245000 -l 1420 -c 5
setup package -r 0.001000 -l 10 -c 10
```

Totem also supports more detailed distributed RLCK package and board models for simulation; these models are described in the following sections.

Distributed RLCK Package and Board Subcircuit Model

For RLCK package subcircuit models to accurately analyze the dynamic voltage drop associated with these variations, the package parasitics must be in the form of a

subcircuit description following conventional SPICE syntax. Ideal voltages are applied at the package balls through the subcircuit, which is connected to the die during analysis. The list of ports in the subcircuit definition may include signal, power and ground ports. Note that asymmetric power and ground models are supported. The following elements in the subcircuit netlist are supported.

Symbol	Element
R	Resistance
L	Self inductance
Kxx	Mutual inductance
C	Capacitance
H	Current-controlled voltage source
I	Current source
V	Voltage source
E	Linear voltage-controlled voltage source
F	Linear current-controlled current source
G	Linear voltage-controlled current source
H	Linear current-controlled voltage source

All elements must be fixed, with no dependencies on any other parameter.

In addition, the following functions in the subcircuit netlist are supported.

Function	Purpose
.inc or *.include*	Inclusion of other SPICE-compatible files
subckt	Definition of additional subcircuits.

The entire off-chip network must be captured in a SPICE subcircuit netlist named ‘REDHAWK_PKG’. In addition, the power and ground ideal sources must be defined in the top-level netlist. All definitions and instantiations in the netlist must follow conventional SPICE syntax.

Note: Voltage values provided in the package netlist are used in Totem analysis and the voltages specified in the GSR file are overridden.

Figure 7-3 is an illustration of the way package and board parameters are modeled in Totem. Each port of the REDHAWK_PKG subcircuit can be connected to one or more pads on the die (note that no node can have “0” as a name).

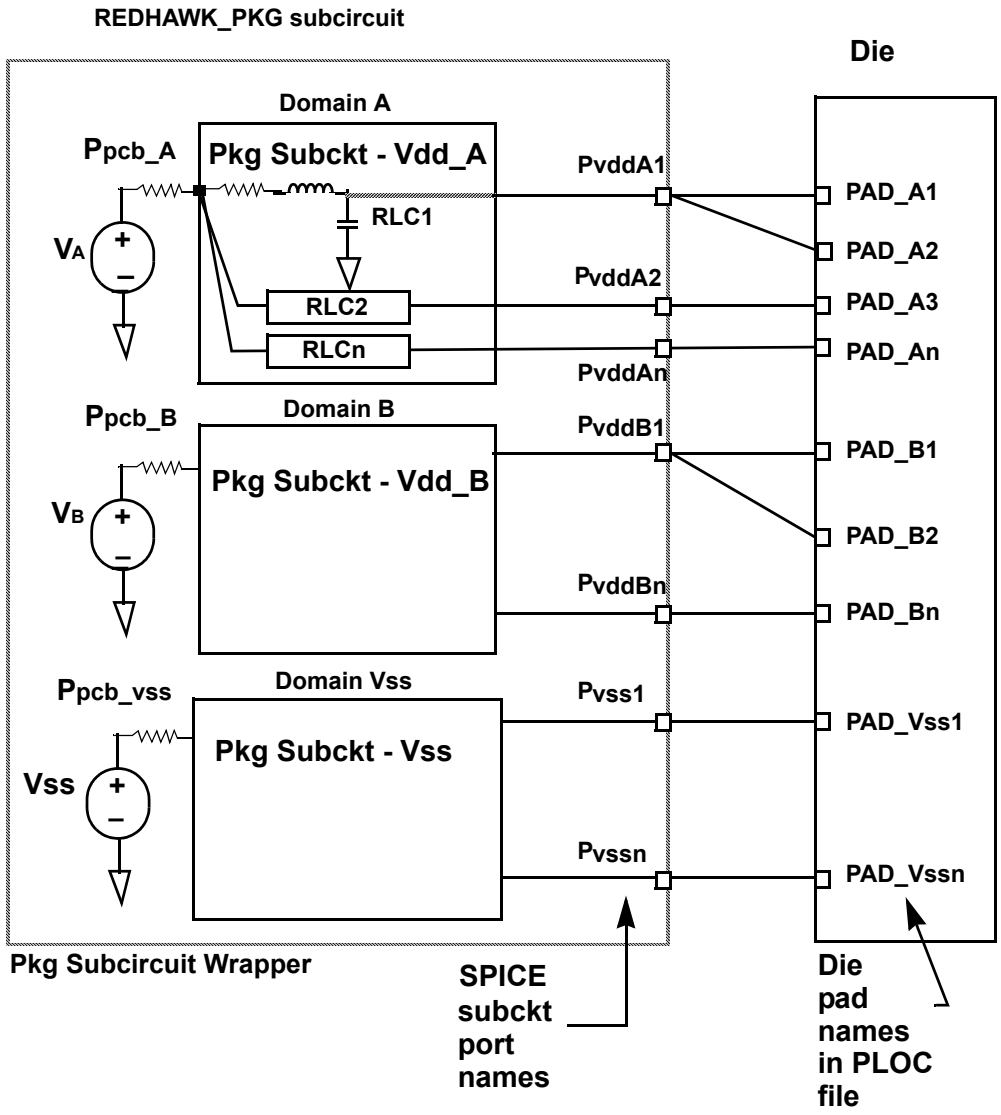


Figure 7-3 Modeling package parameters in Totem

Support for Package K-parameters

Totem supports package subcircuit models to more accurately analyze the dynamic voltage drop with off-chip package and board effects. The mutual inductor coefficient K is supported, in addition to basic SPICE circuit components (RLC). The mutual inductor coefficient represents the coupling effects among inductors. If the mutual inductance between inductors L1 and L2 is M12, the coefficient K is defined as ‘M12 / sqrt(L1*L2)’. In the package SPICE netlist it appears as:

```
K_L1_L2   L1   L2 0.5
```

The mutual inductance coefficient must have an absolute value between 0 and 1.0. You can specify the K parameter for each pair of inductors for an inductor group (i = 1...N). So the self inductances L11 to LNN, along with mutual inductances Mij (i = 1...N, j = 1...N)

constitutes an inductance matrix. In package modeling, such an inductance matrix is usually extracted by field solvers.

Following is an example of a package SPICE netlist, *top.sp*, in a single file, for the model illustration above, including mutual inductance between inductors, K12:

```
* Top level subcircuit
* The file name must be specified with a GSR keyword
* All ports are mapped (i.e, connected) to Totem pads defined
* in the *.ploc file.
* The subcircuit must be named REDHAWK_PKG

.subckt REDHAWK_PKG PvddA1 PvddA2 PvddB1 PvddB2 Pvss1 Pvss2

* PCB voltage supplies
Va Ppcb_A 0 1.1v
Vb Ppcb_B 0 1.0v
Vvss Ppcb_vss 0 0v
* Instantiate domain subcircuits. Connect them to the
* corresponding PCB voltage supplies and output ports of
* REDHAWK_PKG
XVDDA_RLC Ppcb_A PvddA1 PvddA1 VDDA_RLC
XVDDB_RLC ...
XVSS_RLC ...
.ends

* Package RLC for each voltage domain can be captured in a
* separate subcircuit
.subckt VDDA_RLC Ppcb Pdie1 Pdie2
R1 Ppcb N1 1e-9
L1 N1 Pdie1 1e-12
K12 L1 L2 0.3
C1 Pdie1 0 1e-10
R2 Ppcb N2 0.5e-09
L2 N2 Pdie2
.ends
.subckt VDDB_RLC Ppcb Pdie1 Pdie2
...
.ends
.subckt VSS_RLC Ppcb Pdie1 Pdie2
...
.ends
```

In order for Totem to include the subcircuit model in the simulation, the top-level SPICE netlist must be specified in the Totem GSR file, as follows:

PACKAGE_SPICE_SUBCKT top.sp

Linear Current- and Voltage-Controlled Source Models

The following SPICE linear current- and voltage-controlled sources are available for package modeling in Totem:

- Type “E” - Linear Voltage-Controlled Voltage Source
- Type “F” - Linear Current-Controlled Current Source

- Type “G” - Linear Voltage-Controlled Current Source
- Type “H” - Linear Current-Controlled Voltage Source

The syntax for describing these linear source models is given below. Note that SPICE source-type keywords, such as ‘VCVS’ and ‘CCCS’, are optional in the Totem syntax.

Linear Voltage-Controlled Voltage Source (E)

```
E* N+ N- ?VCVS? NC+ NC- VGain
```

where

E*	name of the source, starting with E
N+	name of positive node
N-	name of negative node
NC+	name of positive controlling node
NC-	name of negative controlling node
VGain	voltage gain

Linear Current-Controlled Current Source (F)

```
F* N+ N- ?CCCS? Vname IGain
```

where

F*	name of the source, starting with F
N+	name of positive node
N-	name of negative node. Current flows from the positive node through the source to the negative node
Vname	name of the voltage source through which the controlling current flows.
IGain	current gain

Linear Voltage-Controlled Current Source (G)

```
G* N+ N- ?VCCS? NC+ NC- TransC
```

where

G*	name of the source, starting with G
N+	name of positive node
N-	name of negative node
NC+	name of positive controlling node
NC-	name of negative controlling node
TransC	transconductance (voltage to current conversion factor)

Linear Current-Controlled Voltage Source (H)

```
H* N+ N- ?CCVS? Vname TransR
```

where

H*	name of the source, starting with H
N+	name of positive node

N-	name of negative node
Vname	name of the voltage source through which the controlling current flows
TransR	trans-resistance in Ohms (current to voltage conversion factor)

Note: The dummy sources for types “F” and “H” must have a voltage of 0.

Mapping Package Port Names to Die Pad Names in the PLOC File

Mapping of ports on the package to one or more pads on the die is achieved using definitions in the PLOC file. Some package vendors have protocols that can be used in mapping ports and pads. If a protocol is not available, the port/pad locations must be mapped manually.

Following is the syntax for a standard **.ploc* file with no package model:

```
<die_pad_name> <X-coord> <Y-coord> <layer> <POWER | GROUND>
```

If you want to include a package subcircuit model, then the syntax of the *.ploc* file should be modified as follows:

```
<die_pad_name> <X-coord_um> <Y-coord_um> <layer>
    <power/ground_domain_name> <Spice_pkg_port_name>
```

where the power/ground domain name is the name specified in the VDD_NETS GSR keyword, and none of the ports can be named “0’. Signal ports in the package subcircuit can be floating. Multiple pads can be connected to a single port of the package. For example, in the package model provided multiple Vdd_A domain pads can be connected to port PvddA1.

The following is a sample *.ploc* file corresponding to the example design and package:

```
Pad_A1 3125 4340 metal6 VDD_A PvddA1
Pad_A2 3225 4340 metal7 VDD_A PvddA1
Pad_A3 3325 4340 metal6 VDD_A PvddA2
Pad_A4 3425 4340 metal6 VDD_A PvddA2
Pad_B1  5125 4340 metal7 VDD_B PvddB1
Pad_B2  5225 4340 metal7 VDD_B PvddB1
Pad_B3  5335 4340 metal6 VDD_B PvddB2
Pad_B4  5335 4340 metal6 VDD_B PvddB2
Pad_VSS1  6125 4340 metal7 VSS Pvss1
Pad_VSS2  6225 4340 metal6 VSS Pvss1
Pad_VSS3  6325 4340 metal7 VSS Pvss2
Pad_VSS4  6425 4340 metal7 VSS Pvss2
...
```

The **.ploc* file is imported into Totem using standard procedure.

Chip-Die Mapping Using Package Compiler

Accurate analysis of chip designs require a model of the associated package, in terms of an accurate RLCK SPICE subcircuit. The Totem Package Compiler can provide this as an extension of the Chip Package Protocol (CPP) header. The Package Compiler has several functions:

- wrap the package SPICE model into Totem-compatible format.
- match die and package pins
- compute effective inductance
- perform RLCK passivity checks

- perform package SPICE syntax checks

If Package Compiler has already been run, if invoked again it identifies the already-wrapped Totem package model and the already-annotated PLOC file, and reruns the same set of checks on the output files for completeness and correctness.

Overview

An overview of the Package Compiler process and data flow is shown in Figure 7-4.

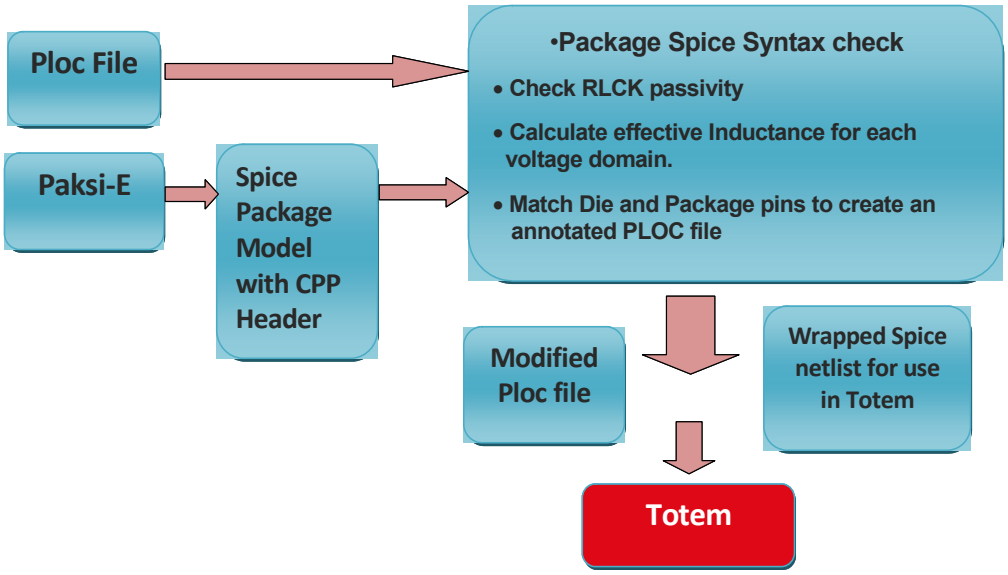


Figure 7-4 Overview of Package Compiler flow

The key features of Package Compiler is “wrapping” the package SPICE model subcircuit into one that is Totem-compatible, as well as calculating effective inductance for each voltage domain and matching die and package pins.

Inputs

The require inputs are:

- PLOC file - pin location file to be annotated (`*.ploc`). The format of the PLOC file is as follows:

  ```
  <pin/pad_name> <x_loc> <y_loc> <layername> <P/G_domain_name>
  ```

  Note that the last column should **not** be “Power” /”Ground”, but the domain name.
- package SPICE file - target package SPICE model with Chip Package Protocol (CPP) header to be mapped, wrapped, syntax checked, passivity checked, and effective inductance calculated.
- GSR file : standard Totem GSR file that contains information about the voltages to apply to each die power and ground net, specified with keywords

  ```
  VDD_NETS {
      <die_VDD_name> <voltage>
      ...
      }
  GND_NETS {
  ```

```
<die_VSS_name> <voltage>
...
}
```

Command Syntax

The syntax for using package compiler from a UNIX command line is:

```
compile_pkg
    [ -m <pkg_model_file>] [ -p <ploc_file> ] [-n <p_n_t_file>]
    [ -t <transform_file> ] [ -vv <port_value_file> ]
    [ -vg <gsr file> ] [ -g <generic_pkg_file> ]
    [ -c ] [ -nma <#_of_match_iter> ] [ -no_leff_calc ]
    [ -op <output_ploc_file> ] [ -om <wrapped_model> ]
    [ -ow <working directory> ] [ -icf <input_config_file> ]
    [ -start_tol <min_dist> -end_tol <max_dist> ]
    [-snap ] [ -ee]
```

where

-m <pkg_model_file> : specifies the package model SPICE file.

-p <ploc_file> : specifies the die pin locations and corresponding die nets.

-n <p_n_t_file>: specifies the file defining the package net types

-t <transform_file> : specifies the transformation operations to use for die/package pin matching.

-vv <port_value_file> : specifies the voltages to assign to package power/ground PCB pins.

-vg <gsr file> : reads the die net voltages from the GSR file.

-g <generic_pkg_file>: If no -m file supplied, this file can be used to indicate package pin locations and corresponding package nets.

-c : Checks the data between the already annotated ploc file and -m pkg file.

-nma <#_of_match_iter> : specifies the maximum number of matching attempts.

-no_leff_calc : the effective inductance is not calculated, which reduces runtime.

-op <output_ploc_file> : specifies the name of the output PLOC file.

-om <wrapped_model> : specifies the name of the wrapped SPICE model

-ow <working directory> : specifies the working directory for the intermediate files

-icf <input_config_file> : specifies the file that contains input arguments to this utility

-start_tol <min_dist> -end_tol <max_dist> : specifies the minimum and maximum separation distance in mapping die and package pins (meters)

-snap : snaps each die pin to the package pin closest to it, even if outside the tolerance radius.

-ee : forces die ground pins to attach only to package ground pins and die power pins to attach only to package power pins

Note that if you run Package Compiler *without* the '–ee' and '–snap' options, a file with a list of unmatched ploc pins, called *adsPackage/unmatchedPlocPins.txt*, is generated, with entries such as the following:

```
+ Best Match: 1/3126 pins lying outside of tolerance 1.8um,
with greatest offset of 264.552452um
+ The list of unmatched ploc pins: [ adsPackage/
unmatchedPlocPins.txt ]
!! NOTE: Since -snap option was NOT specified, all pins
```

```
lying outside of the tolerance will be commented out in the
annotated ploc file.
```

Using the '–ee' and '–snap' options, Package Compiler snaps any PLOC points lying outside of the tolerance threshold to the closest related package point and no *unmatchedPlocPins.txt* file is generated.

Outputs

The primary output is the conversion of the original package SPICE netlist, <*pkg_spice*>, of the form:

```
subckt pkg die1 die2 pcb1 pcb2
...
.ends
```

into the "wrapped" Totem-compatible package SPICE netlist, *redhawk_pkg*, that has the format:

```
include <pkg_spice>
.subckt redhawk_pkg die1 die2
V1 pcb1 0 1.2
V2 pcb2 0 0
Xpkg die1 die2 pkg
.ends
```

Note that the new SPICE file includes the original package SPICE file. The subckt netlist excludes the nodes on the side of the PCB, which are assigned appropriate voltages described by the voltage file.

Additional outputs are described following:

- <ploc_file>.1 - fully annotated ploc file after successful die and package pin pair mapping.
- standard checks of circuit syntax, such as:
  - annotated SPICE nodes and nets exist in the package SPICE model
  - annotated SPICE nodes belong to the correct package net
  - die side SPICE nodes are not used as voltage sources in the SPICE file
  - SPICE nodes do not have multiple types
  - SPICE nodes do not belong to multiple nets in CPP header
  - PLOC file is fully annotated
  - PCB SPICE nodes have valid voltages
- passivity check - checks RLCK passivity for the package SPICE model
- package net types file (*.pnt) - automatically created upon successful pin-pair mapping, and required for effective inductance calculation. In the event that the file is unavailable (if Package Compiler has not been run yet), it can be provided by using the '–n' option. The file format is:

```
.POWER
  <pkg_net1>
  <pkg_net2>
...
.GROUND
  <pkg_neta>
```

```
        <pkg_netb>
      ...
```

- effective inductance calculation for each Power and Ground net pair.

  For example, for the example PNT file above, it calculates the inductances:

  ```
  <pkg_net1> to <pkg_neta>
  <pkg_net1> to <pkg_netb>
  <pkg_net2> to <pkg_neta>
  <pkg_net2> to <pkg_netb>
  ```

  A testbench is created for each calculation, named:

  <pkg_model>.<power_name>_<gnd_name>.leff.sp

  By default, the utility uses the NSPICE binary to extract the effective impedance ($APACHEROOT/bin/nspice)

  You can use the ‘sv’ waveform viewer to view the waveform file generated [*.aa0].

- *<ploc_file>.map.1* file - contains the list of die to package pin mappings.
- *<ploc_file>.T* file - contains the transformation (rotation, mirror, translation) used to match the die to the package
- **.leff.sp* files - testbench SPICE files used to extract the effective inductances between power and ground package net pairs.

**.leff.aa0* files - contain the effective inductance graphs that can be viewed with the ‘sv’ viewer.

Known Restrictions

The following restrictions must be observed in package models:

- Only the previously-described SPICE keywords are supported.
- Mapping the nodes of the package to their locations on-chip is currently supported only through the *.ploc* file.
- The user-created top-level SPICE netlist must use the ‘REDHAWK_PKG’ keyword as the subcircuit model.

Chapter 8
Chip Power Models (CPM)

Introduction

Chip package and printed circuit board (PCB) designers need an accurate and relatively simple IC power model to design and optimize effective chip packages and boards -- including key parameters such as impedance and resonant frequencies of the global power delivery network (PDN). An equivalent circuit for the chip PDN must provide not only an accurate multiple-terminal impedance model, but also accurate current waveforms to represent a realistic worst-case switching scenario for the chip.

Ansys Chip Power Model (CPM) enables die-package-board co-design and co-verification for dynamic power integrity. Built on the Totem full custom chip dynamic power integrity platform, CPM generates a compact and accurate model of the full custom chip power delivery network at various key stages of chip and package design.

CPM supports the VectorLess™ clock gating switching mode, in addition to the default vectorless and VCD modes. In clock gating Vectorless mode, Totem generates a switching scenario that mimics the transition from one clock gating mode to another during multi-cycle transient analysis. A CPM created in this mode captures the current transitions when this clock gating scenario change occurs. This change in current demand is particularly useful to understand the associated Ldi/dt effect in package and board level analyses. CPM has a Resonance Frequency Aware Excitation Mode, in which the Totem VectorLess engine generates a multiple-cycle switching scenario for the current signature that introduces most of the energy around the chip-package-system resonance frequency. You only need to specify the system resonance frequency when creating the model, and the CPM technology automatically creates the frequency-based form of on-die excitation, while maintaining the logic and timing properties of the circuit.

CPM bridges the design of the power delivery network between the IC and the associated package and PCB. System designers can use CPM to guide and verify the off-chip PDN design by evaluating the impact of on-die parasitics over the global PDN impedance, diagnose potential chip-package LC resonance, validate the package/board dynamic voltage noise margin, as well as to optimize the off-chip decoupling capacitor placements.

Both current signatures and parasitic network are distributed across a multiple-terminal equivalent circuit to reflect their temporal and spatial dependencies. The simplest element of the CPM model can be considered to be a serially connected R_{DIE} and a C_{DIE}, in parallel with a current switch, as shown in Figure 8-1.

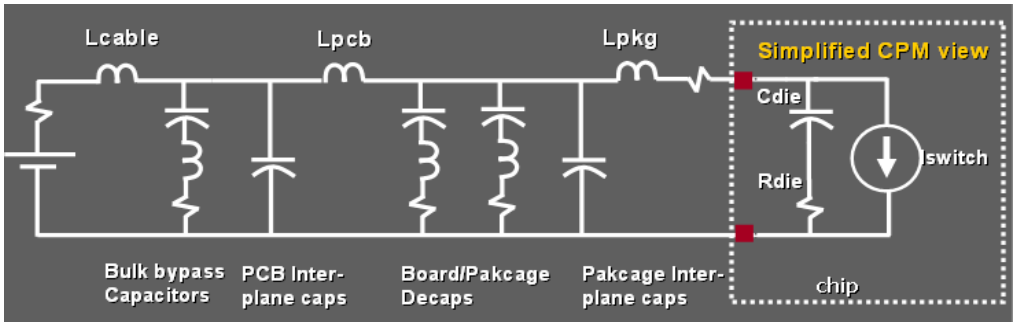


Figure 8-1 Power Delivery Network and simple Chip Power Model

SPICE-compatible CPM includes parasitics of non-linear switching and non-switching devices, parasitics of power/ground wires, decoupling capacitors, and effective RC's of the loading capacitors from signal interconnects. In addition, CPM contains full-chip switching current signatures based on transistor-level SPICE simulations.

Design Flow

CPM technology can be useful at all stages of the package-IC system co-design process, as shown in the Figure 8-2 flow diagram.

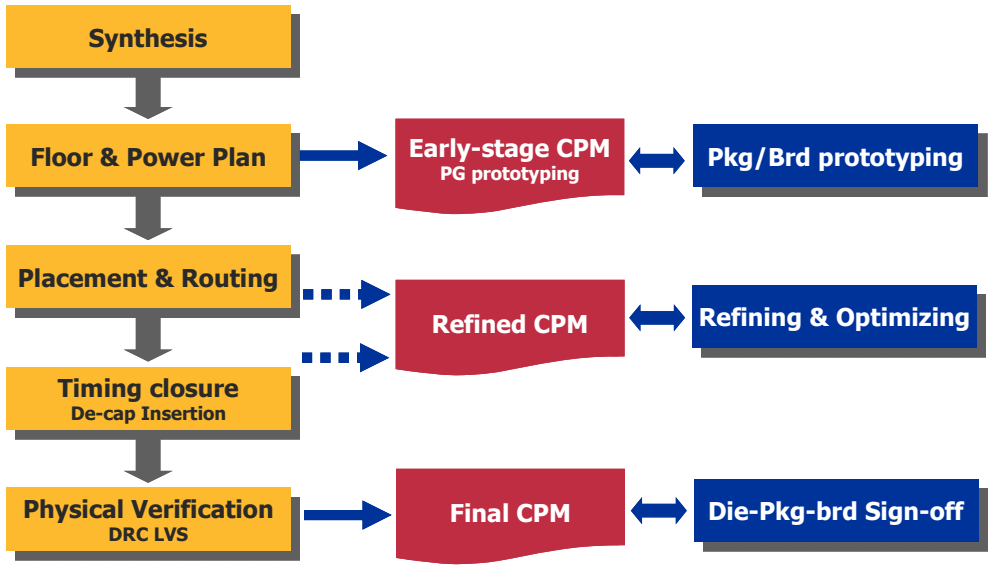


Figure 8-2 CPM-based IC system co-design process (for digital P & R blocks)

Totem Modeling of Chip Power Delivery Network

A chip power delivery network can be modeled in several ways by CPM, depending upon the number of power/ground pads involved and also the desired trade-off between speed and accuracy of analysis.

Modeling Choices Based on Number of Pads

If a large number of die pads are involved, such as several thousand in a typical flip chip package design, the die pad area is partitioned into smaller rectangular areas and CPM generates a set of terminals corresponding to each Vdd and Vss net within each partition. For wirebond designs with a large number of pads, they also can be grouped into individual partitions of pads for better analysis based on the SPICE node name specification in the PLOC file. For designs with a smaller manageable number of power/ground pads, all pads are represented individually by CPM terminals.

All significant sources of capacitance in a chip, including parasitics and decoupling, are included in the Totem chip model, as shown in the circuit of Figure 8-3:

- Intentional de-caps, from Totem characterization
- Intrinsic device de-caps, from Totem characterization
- Signal loading capacitance, from SPEF (StarRC)
- Coupling capacitance between power and ground wires, from Totem extraction

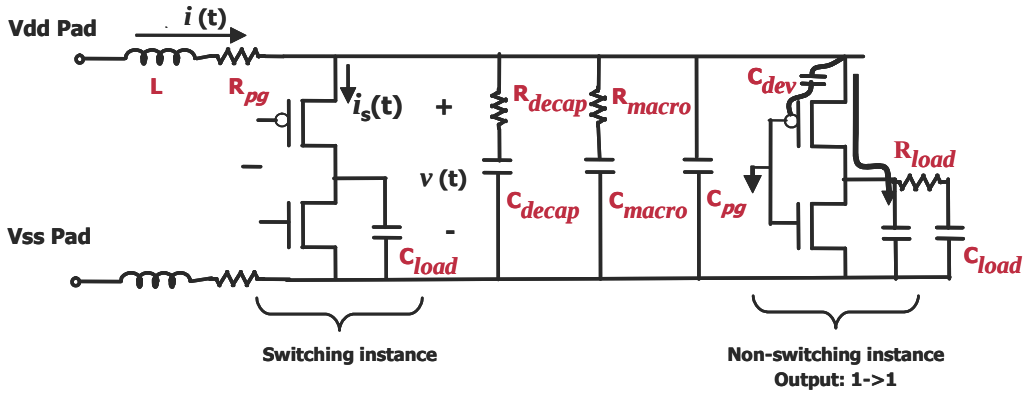


Figure 8-3 Totem chip power modeling circuit

CPM for Flip Chip Designs

For flip chip designs, the chip bump area can be partitioned into N x M tiles. Assuming there are K power/ground nets (that is, one or more Vdd nets and one or more Vss nets), then the maximum number of external terminals of the resulting CPM will be K x N x M.

Figure 8-4 shows a flip chip example with one VDD net and one VSS net. The number of partitions is 4 (N=2, M =2).

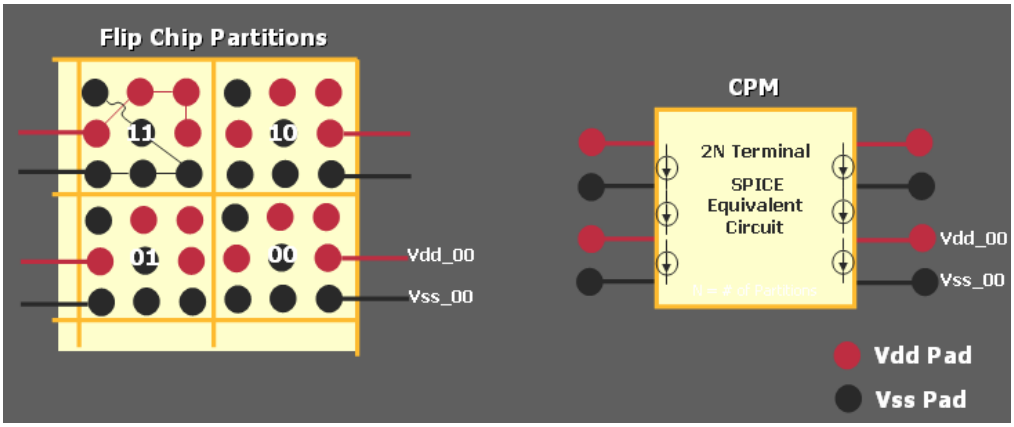


Figure 8-4 Modeling a chip power circuit for a flip-chip package

CPM for Wirebond Designs

For wire-bond designs, an N-terminal CPM is created, for which typically each wire-bond pad corresponds to a terminal of the model, as shown in Figure 8-5.

For PLOC files that have grouped ports (package nodes) and named them accordingly, CPM by default uses the PLOC port groups and generates a set of external ports that follow the grouping pattern in the PLOC file.

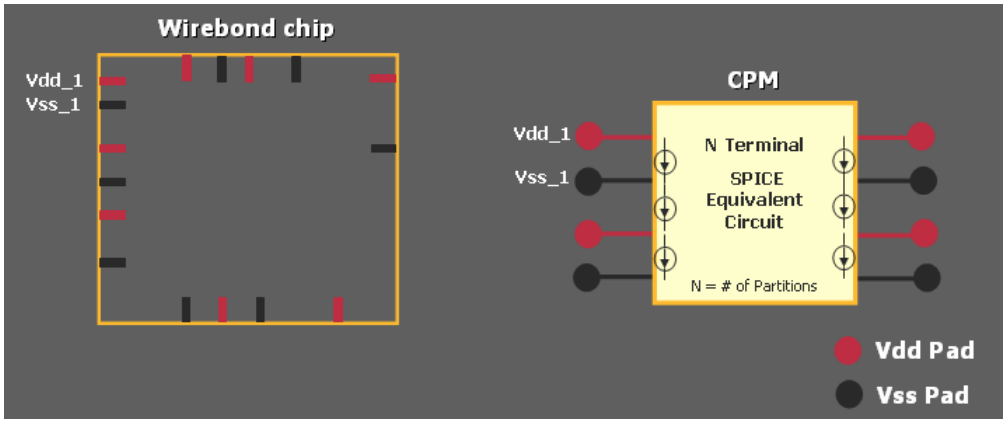


Figure 8-5 Modeling a chip power circuit for a wirebond package

Modeling Choices Based on Analysis Speed and Accuracy

High speed modeling

The “model order reduction” (MOR) methodology is based on a DC/zero frequency-centered solution, that becomes more inaccurate as the switching frequencies of the design become higher, but saves runtime for conditions that do not require high frequencies and high accuracy.

In general an assumption of power-ground symmetry is made, which means that package power and ground impedances are considered similar. However, setting the GSR keyword DYNAMIC_SOLVER_MODE to 1 allows a P/G solution without this assumption, increasing the accuracy, although this makes it a more time-consuming and memory-intensive methodology.

High Accuracy Modeling

The default “AC” analysis-based version of CPM significantly improves the accuracy of the model, and has far better correlation with Totem simulations. CPM model construction consists of two parts: calculation of current signatures and the reduction of the passive part of the circuit.

The AC analysis-based CPM generation uses grid RC reduction applied to the original RC network that contains not just the resistance and capacitance representing the parasitics of the power/ground wires, but also the series RC branches that model the parasitics of non-switching instances. Switching instances are modelled as time and voltage-dependent current sources.

The analysis takes the large network of resistors and capacitors (typically with millions of nodes) and generates a much smaller network (typically with tens or hundreds of nodes), whose frequency domain response at the specified ports for the specified frequency range matches that of the original network, with a tolerance better than 0.2 %. The resulting network, which consists of R, C, and possibly L and G (constant Voltage Controlled Current Source) elements, is exported as a SPICE netlist. The netlist is guaranteed to be passive by construction, and thus suitable to be used in transient simulations with SPICE or another transient simulator. This netlist can be used with any version of SPICE (such as NSPICE from Apache and HSPICE from Synopsys) as it only uses classic circuit elements.

CPM Simulation Procedure

Setup and Preparation

The required input data for CPM is the same as you need for a Totem DvD analysis. Refer to APLMMX setup description in section "APLMMX Data Preparation", page 2-12. Refer to section "Characterization: Creating Electrical APLMMX Models", page 3-27 for the proper characterization procedure.

In order to include signal loading capacitance in CPM generation, to better predict global LC resonance, add “IGNORE_ACE_SIGNAL_CAP 0" in the APLMMX configuration file. You can fine tune the signal loading capacitance during ACE characterization by setting K_GROUND_CAP/K_FLOATING_CAP keyword ACE_OPTION { } in the APLMMX configuration file. Refer to ACE setup in section "ACE Decap and ESR Characterization", page E-1010, to get more detailed information.

Additional input parameters to be decided and specified for a flip chip CPM model are the number of partitions desired in the X and Y directions.

Running CPM

The Chip Power Model is created from the Totem flow, by reading the input data, performing power calculation and on-die P/G extraction, and then creating the electrical model of the chip power delivery network and current profiles based on the defined partitioned network.

The Totem command line setup and invocation command steps are as follows:

1. Import and set up design.

   The GSR file contains the paths to all design and characterization output files from APLMMX and simulation conditions. See section "Global System Requirements File (*.gsr)", page C-772, for a general description of the GSR keywords. The file is executed using the command:

   ```
   setup design <design>.gsr
   ```

   To provide ESD-aware CPM models, use the commands

   ```
   setup analysis_mode esd
   setup design <design>.gsr
   ```

2. Use special GSR keywords for CPM models, including multi-VDD, multi-ground cases using coupled solver mode.

   ```
   GENERATE_CPM 1
   DYNAMIC_SOLVER_MODE 1
   ```

3. Hierarchy identification. If the hierarchical divider in the transistor name is not “/” and “.”, use the following GSR keyword to specify the hierarchical divider, to help Totem identify the transistor hierarchy:

   ```
   MMX_XTOR_HIER_DIVIDER <hierarchy_string>
   ```

   For example, if the transistor name in the Totem database is:

   ```
   IswArray\|ILSB0\|M15_2__rcx
   ```

   you can define the use following setting to identify the transistor hierarchy.

   ```
   MMX_XTOR_HIER_DIVIDER \|
   ```

4. Multiple power/ground domains. You must remember to define pg arcs in the custom lib if there are multiple power/ground domains in the design. For example, use the following custom lib file entry defined in GSR LIB_FILES keyword:

   ```
   cell ram256x8 {
     type memory
     pgarc {
        VDDA_1ST VSSA_1ST
        VDDA_2ND VSSA_2ND
        VDDA_SH  VSSA_SH
     }
   }
   ```

5. Current sources can have port voltage derating applied.

   Voltage derating for all power ports is written to a file named *curr.derat* in the run directory. Derating is performed based on the V-I table created by the Totem. To specify the V-I curve, you must edit a file *vc_coef.txt* with content/format as follows:

   ```
   # <normalized voltage> <normalized current>
   0.1 0.1
   0.5 0.5
   1.0 1.0
   1.5 1.5
   ```

   This represents a linear V-I curve. When voltage is standard, current has no derating: (1.0 1.0).

   To consider voltage derating, you can remove the current source part in the *PowerModel.sp* file and include the contents of the *curr.derat* file, using the new sources with voltage derating. Derating is only performed for power nets, and not for ground nets.

6. Specify a temporary file location on a local disk. Particularly when multiple processors are used and for large designs (more than 10 M nodes), writing temporary files to a disk on the network slows down execution because of I/O operations. Put temporary files on a local disk by setting the CACHE_DIR GSR keyword. After chip power modeling finishes, the temporary files (linear.*) in CACHE_DIR are deleted.
7. Determine appropriate toggle rates and calculate average power.

```
perform pwrcalc
```

8. Extract RLC parasitics of on-die P/G networks.

```
perform extraction -power -ground -c -l
```

9. The command syntax for generating a CPM is as follows:

```
perform pwrmodel [ -wirebond | <flip chip partitions>| -cdie |
    -static |-vcd | -dvd | -mixed | -esd ] ?-esd_clamp <file>?
? -parasitic ? ?<no model option-default>? ? -pincurrent ?
? -rleak ? ? -rleak_par? ?-solver mor? ?-plocname? ? -ind?
? -no_afs? ? -passive ? ? [ -noglobal_gnd | -global_gnd ] ?
? -repeat_current [ <start_time> | presim | best ]? -probe
? -internal_node -cell_file <cell_list_filename> ? -reportcap ?
? -o <output_filename> ? -reuse ? -io ? -esd_combine_shorted_clamp ?
```

where

-wirebond : specifies a wirebond package

<flip chip partitions> : -nx <num_x_ partitions> -ny <num_y_partitions>, specifies the number of partitions in the x and y directions. For -nx 1, -ny 1, a Cdie/Rdie report is generated in the *adsRpt/CPM/apache.Cdie* file.

-cdie : sets nx=1 and ny=1, and connects all power nets together and all ground nets together to obtain a single-port solution to obtain the equivalent Cdie and Rdie values for the chip. No current waveform is generated. A Cdie/Rdie report is generated in the *adsRpt/CPM/apache.Cdie* file. See the -reportcap option for the

Note that the -cdie option just provides faster run time by not calculating the current waveforms. For all other purposes it is equivalent to using '-nx1 -ny1'.

-static : creates a static analysis chip power model, using DC conditions, a resistance-based circuit, and average current.

-vcd : creates a VCD file based dynamic voltage drop model

-dvd: creates vectorless-based model (default)

-mixed: creates a model for mixed vectorless and VCD voltage drop analysis

-esd : creates an ESD-aware CPM model for system-level ESD analysis. Clamp connections are modeled as ports in the CPM model, and sub-circuits for clamp devices are included in the CPM model.

-esd_clamp <file> : specifies the ESD clamp file for ESD-aware CPMs. Note that this is not needed if a clamp file is specified using the ESD_CLAMP_FILE GSR keyword.

-parasitic: generates only the passive part of the CPM, without performing transient simulation, to generate the current signatures of the CPM ports, an extension of -cdie option for multi-partition CPMs. This can save time in transient simulation. However, the CPM model generated with -parasitic can only be used for DC and AC analysis (not usable for transient analysis).

-pincurrent: specifies that CPM generate the model without current conservation (balanced current between Vdd and Vss) to achieve better correlation with Totem dynamic simulation results. In general, CPM enforces current conservation. However, in cases when Totem does not produce balanced VDD and VSS currents, this option can be used.

-rleak: causes the leak resistance to be added between ports on the VDD (power) net and the reference port on the VSS net.

-rleak_par: inserts leakage resistance between the VDD and VSS ports of each defined partition. Note that this option only works with partitioned CPM models. See the following section for more details on usage.

-solver mor: turns off the default ‘solver ac’ function, which is an accurate frequency-based linear solver AC solution with passivity enforcement.

-plocname: specifies pad/group names for CPM port names. If the '-wirebond' option is used, the subcircuit terminal names are taken from the pad names (if no grouping is specified), or the group names from the 6th column of the *.ploc* file. These group names are also known as SPICE node names, as this mechanism is used to connect a package using the keyword 'PACKAGE_SPICE_SUBCKT'. If this option is used with '-nx # -ny #' options, the node names are generated in the following form: PAR_0_0_VDD1, PAR_0_0_VSS2, ... For wirebond CPMs without any grouping, the subckt terminal name is the ploc name.

-ind: accounts for on-chip inductance, if the option '-l' of the Totem 'perform extraction' command has been used

Note: If you specify the '-l' option of the 'perform extraction' command, but not '-ind', inductance is ignored. Not specifying the ‘-l’ option and using ‘-ind’ is an error.

-no_afs : turns off the default AFS function. Adaptive Frequency Sweep for AC mode execution intelligently selects seven to nine frequencies (enough to reach convergence) to perform AC analysis, as opposed to 26 frequency samples that are performed by default in ‘solver ac’ mode. This option is recommended for design sizes exceeding 50 ports. Port count can be determined by computing N * M * P, where N = number of X partitions in the CPM (-nx option), M = number of Y partitions in the CPM (-ny option), and P = number of power and ground domains.

-passive : only effective with MOR function, which uses MOR to generate a passivity-enforced SPICE model, which can reduce accuracy. This mode can be useful if the SPICE simulation of the package/PCB CPM has convergence issues. The SPICE netlist is significantly smaller than using the default mode. But CPM is a compact model, so complexity is not a concern for cases with, for example, 100 bond pads, or up to 10x10 partitions for a flip-chip design.

-noglobal_gnd | -global_gnd: specifies the type of parasitic modeling in CPM, either (a) without Spice Node 0, using option '-noglobal_gnd' (the default), where there is a direct connection between the power and ground ports without going through Spice node 0, or (b) using Spice Node 0, using option '-global_gnd’, in which the RLC parasitics from power and ground are connected to Spice global ground (node 0).

-repeat_current: specifies that the CPM current signature is repeated starting from the specified time point. Either a <start_time> in ns, the ‘presim’ time, or ‘best’ (chosen to cause the best continuity at the repeat point), can be chosen as the starting point of the repeating waveform. A warning is issued for incorrect values (such as a negative value or value greater than the maximum time of the PWL source). For non-zero values the closest time in the PWL definition is used. For example, if a ‘-repeat_current 1n’ option is specified, and PWL time values ..., 900, 930, 960, 990, 1020, ... ps are

defined, ‘R=990 ps’ is used. This is required for SPICE to accept the netlist. For the ‘best’ option to work well, the presim time and the transient simulation time need to be set to “n*T”, where n is a positive integer, and T is the period of the clock frequency. With this option, the repeat time may be different for each individual PWL current source. Usage examples:

```
perform powermodel -nx 2 -ny 2 -repeat_current 0
```

which repeats starting from the beginning, t=0. Or,

```
perform powermodel -nx 2 -ny 2 -repeat_current 2n
```

which repeats starting from the time value in the PWL source definition closest to the 2ns time specified.

-probe: invokes the iCPM utility that enables the visibility of sensitive P/G connections in the design, and allows you to probe device locations inside the chip. You must set the PROBE_NODE_FILE GSR keyword, as described in section "iCPM- Internal Node Probing", page 8-255.

-internal_node: specifies additional ports located at P/ G pins on the same net that are to be shorted together to form one internal port. These nodes are named with the format '<instance name>_<netname>' in the CPM, such as “inst_1_VDD”.

Note that the options '-internal_node' and '-cell_file' are required to execute this feature, and the '-pincurrent' option should be used to keep the CPM currents at the correct value without further modifying the port currents, so that the sum of all port currents is zero. This feature also allows you to include/exclude the instance current profile and device capacitance from CPM creation, using the EXCLUDE option in the GSC file. To exclude instance current profiles and device capacitance, use the GSC syntax: '<instance name> EXCLUDE'.

-cell_file <cell_list_filename>: specifies a file containing a list of the instances whose internal P/G pins are to be exposed.

-reportcap: creates a log file report of all capacitance components included in the CPM generation. Example output:

```
Capacitance components -
     Intentional Decap - 0.000000e+00 pF
     Intrinsic   Decap - 2.404236e+02 pF
     Load        Decap - 1.945542e+02 pF
     Power grid  Decap - 3.260729e+00 pF
     Well        Decap - 0.000000e+00 pF
Total - 4.382385e+02 pF
```

-o <output_filename> : specifies output filename (default - *PowerModel.sp*, with the passive part in the file *PowerModel.sp.inc*)

-reuse : allows reuse of the generated current waveforms after one CPM run, if chip power models with different partitioning schemes are desired

-io : specifies that the I/O cells are to be included in the chip power model

-esd_combine_shorted_clamp : prevents the inter cell short being created in output CECM netlist, due to the overlapping/abutted pins between instances in ESD_CLAMP_PIN_FILE based flow.

10. So to generate a CPM for a flip chip design:

a. to use a vectorless methodology for an accurate solution:

```
perform pwrmodel
    -nx <num_x_partitions>
    -ny <num_y_partitions> -o <output_filename>
```

b. to use a VCD scenario, use the same command as in step (a) and use the '-vcd' option.

c. to generate a CPM quickly for a wirebond design:

```
perform pwrmodel -wirebond -solver mor
-o <output_filename>
```

d. to generate the passive part of a CPM (in a file *cpm.sp.inc*) with 2 x 3 partitioning:

```
perform powermodel -nx 2 -ny 3 -parasitic -o cpm.sp
```

Note that AC mode analysis in Chip Power Modeling can provide a fast solution using the linear solver with multi-threading, which supports multiple-CPU computation. You can set the maximum number of processors to be used, which by default is the lesser of the number of processors available or 8. To use more than 8 processors, or fewer, you can set the Linux/UNIX environment variable 'MAX_CPU <number>'. Then the number of processors used is the lesser of the processors available and the MAX_CPU value. For example, using 'setenv MAX_CPU 4' limits the maximum number of processors to 4.

Setting up for power integrity analysis

The following GSR keywords settings are recommended for creating CPMs for power integrity analysis:

DYNAMIC_TIME_STEP <ps> : time step should be less than 20ps

DYNAMIC_PRESIM_TIME < time> <TSM=1> : do not use TSM > 1.0. For example:

```
DYNAMIC_PRESIM_TIME 10n 1.0
```

For multiple-Vdd designs, the following GSR keywords must be used. Note that run time and memory use will increase significantly for these cases.

COUPLEC 1

DYNAMIC_SOLVER_MODE 1: use if COUPLEC keyword used, or for asymmetric package models

The recommended command options to generate CPM for power integrity analysis are:

```
perform pwrmodel [-wirebond | -nx -ny ] –plocname
    –repeat_current <start_time>
```

The effects of using the -pincurrent option are described in Table 8-1 below.

Use of option -pincurrent	Dynamic Solver	I(t)	Passive portion
Not used	0	Current from N-1 ports flow to one common VSS port.	Coupled between VDD and VSS. Not affected by these options.
Not used	1	Current from N-1 ports flow to one common VSS port.	Coupled between VDD and VSS. Not affected by these options.
Used	0	Current for all N ports flow to SPICE node 0.	Coupled between VDD and VSS. Not affected by these options.
Used	1	Current for all N ports flow to SPICE node 0.	Coupled between VDD and VSS. Not affected by these options.

Table 8-1 Effects of -pincurrent option use

The effects of using the -noglobal_gnd and -global_gnd options are described in Table 8-2 below.

Use of options -noglobal_gnd and -global_gnd	Dynamic Solver	I(t)	Passive portion
-global_gnd	0	Not affected by these options. Controlled by –pincurrent.	Capacitance modeled to SPICE node 0.
-global_gnd	1	Not affected by these options. Controlled by –pincurrent.	Coupled between Vdd and Vss.
-noglobal_gnd (default)	0	Not affected by these options. Controlled by –pincurrent.	Coupled between Vdd and Vss.
-noglobal_gnd (default)	1	Not affected by these options. Controlled by –pincurrent.	Coupled between Vdd and Vss.

Table 8-2 Effects of -noglobal_gnd and -global_gnd option use

Setting up for EMI modeling

The following GSR keywords settings are recommended for creating CPMs for EMI analysis:

COUPLEC 1

DYNAMIC_SOLVER_MODE 1

DYNAMIC_TIME_STEP <ps> : time step should be less than 20ps

DYNAMIC_PRESIM_TIME < time> <TSM=1> : do not use TSM > 1.0

The recommended command options to generate CPM for EMI analysis are:

```
perform pwrmodel –pincurrent –emi [-wirebond | -nx -ny ]
    –plocname –repeat_current <start_time>
```

Modeling leakage resistance using arbitrary partitioning

You can use the '-rleak_par' option to capture leakage resistance in CPM between POWER/GROUND ports defined in arbitrary partitions. Arbitrary partitions are created by adding a 6th column to the PLOC file and then using the '-wirebond' option of the 'perform powermodel' command. Pads with the same “package node” (the entry in the 6th column of the PLOC file) are grouped together, forming a so-called “CPM port”. Unlike geometric grouping using '-nx # -ny #" partitioning, an additional file *partition.txt* is needed to define which VCC and VSS ports belong to the same partition. The format of the *partition.txt* file is '<package node> < group name>'.

The following is an example partition definition in a *partition.txt* file:

```
bump_vcc1 part1
bump_vcc2 part2
ref1 part1
ref2 part2
```

In this example, the partition named 'part1' has'bump_vcc1' as its VCC port, and 'ref1' as its 'VSS port'. The partition named 'part2' has 'bump_vcc2' as its VCC port, and 'ref2' as its VSS ports. So if you specify '-rleak_par', two leakage resistances are added, one between 'bump_vcc1' and 'ref1', and the other between 'bump_vcc2' and 'ref2'. The file *partition.txt* should be placed in the run directory.

The CPM command is:

```
perform pwrmodel  -rleak_par -wirebond
```

Example PLOC file

```
#source name #loc_x #loc_y #layer name #POWER/GROUND #Group name
    DVDD41 35 3426 METAL6 POWER GROUP41_POWER
    DVSS41 35 3548 METAL6 GROUND GROUP41_GROUND
    DVDD42 35 3730 METAL6 POWER GROUP42_POWER
    DVSS42 35 3790 METAL6 GROUND GROUP42_GROUND
    DVDD43 35 3976 METAL6 POWER GROUP43_POWER
    DVSS43 35 4028 METAL6 GROUND GROUP43_GROUND
    DVDD44 35 4212 METAL6 POWER GROUP44_POWER
    DVSS44 35 1270 METAL6 GROUND GROUP44_GROUND
```

The leakage resistance in the output CPM model appears as follows:

```
R0_A320 GROUP44_GROUND GROUP44_POWER 99660.431993
R0_D821 GROUP43_POWER GROUP43_GROUND 64882.488351
R0_N863 GROUP42_POWER GROUP42_GROUND 87442.938928
```

iCPM- Internal Node Probing

iCPM enables access to sensitive P/G connections in the design, and allows you to probe device locations inside the chip. This capability can evaluate the drop through the on-die PDN in a fast system-level simulation. You can not only see the impact of changes at the boundary between the die and the package (at the pads or C4 bumps), but also deep inside the chip at critical locations, such as near the PLL or the FGU blocks.

CPMs created iCPM technology have terminals at user-defined device locations, in addition to the traditional C4 bump and/or pad locations. The iCPM model is created in an incremental step after simulation:

```
perform powermodel -nx 1 -ny 1 -probe -o cpm_probe.sp
```

In the GSR file, add the keyword 'PROBE_NODE_FILE <probe.txt>'. The file format is:

```
<X_location> <Y_location>  <Metal_layer_name>  <Port_name>
```

For example:

```
68.81  47.76   METAL1  port_vdd
67.25  40.10   METAL1  port_vss
```

Or you can use the options '-internal_node' and '-cell_file <instance_list_file> to specify particular instances that should have probe access.

A warning is reported when probing is attempted on an unextracted net (not in VDD_NETS/GND_NETS).

Resonance frequency-aware mode

In the Resonance Frequency Aware mode, the Totem VectorLess™ engine generates a multiple-cycle switching scenario for the current signature that introduces most of the energy around the chip-package-system resonance frequency. You only need to specify the system resonance frequency when creating the model, and CPM analysis automatically creates the frequency-based form of on-die excitation, while maintaining the logic and timing properties of the circuit, as shown in the Figure 8-6.

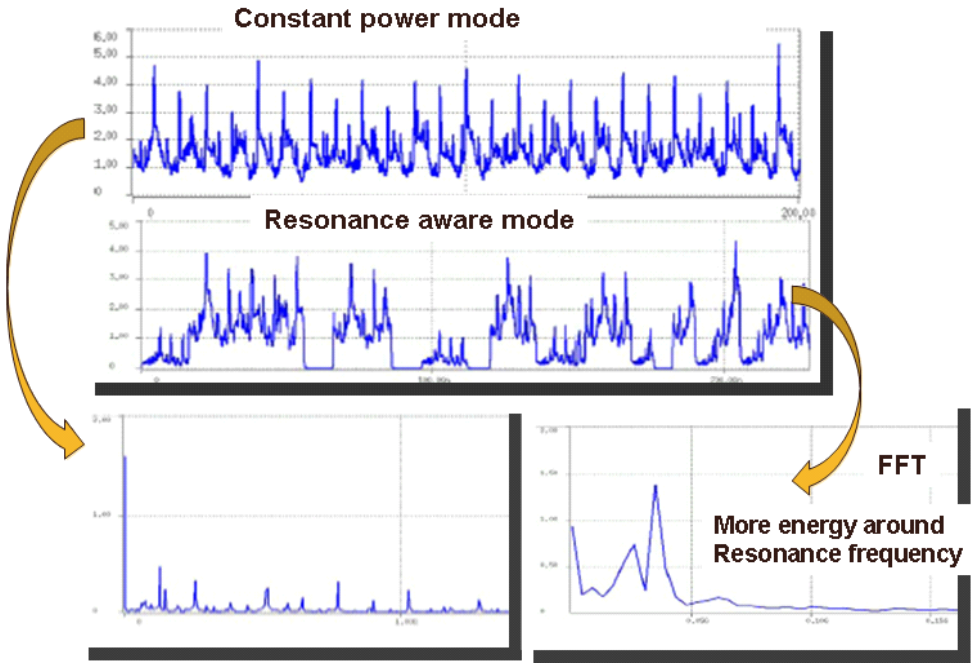


Figure 8-6 Frequency-aware CPM mode

To use this option, first generate a regular CPM model and use its passive part to perform AC analysis for the circuit that includes the CPM and package (and PCB if desired). This AC analysis yields the resonance frequency. Then a second CPM generation run is performed with the GSR setting: DYNAMIC_FREQUENCY_AWARE <frequency in Hz>

For example, for a resonance frequency of f= 40 MHz:

```
DYNAMIC_FREQUENCY_AWARE 40e6
```

Power Transient Mode (Mixed Signal /Digital P&R Designs)

Power Transient Analysis, also known as Variable Power, provides greater flexibility to simulate frame-by-frame power/current transient behavior, such as for Gated Clock operations, and more accurately models power transients on the chip.

While the resonance-aware mode generates current transients that match a particular resonance frequency, variable power mode has more flexibility to include a range of frequencies. By capturing the power step-up, the impact of various chip transitions--such as from reset to running mode, from traffic to no-traffic mode, or from memory-on to memory-off mode--can be modeled, as shown in Figure 8-7. These power transients affect the entire VRM, board, package, and die system to varying degrees, based on the duration and amplitude of the power step. The variable power CPM also can be used to create a low-frequency spectrum, as opposed to a traditional high-frequency spectrum, to allow testing of the package and PCB in the low-to-middle frequency range.

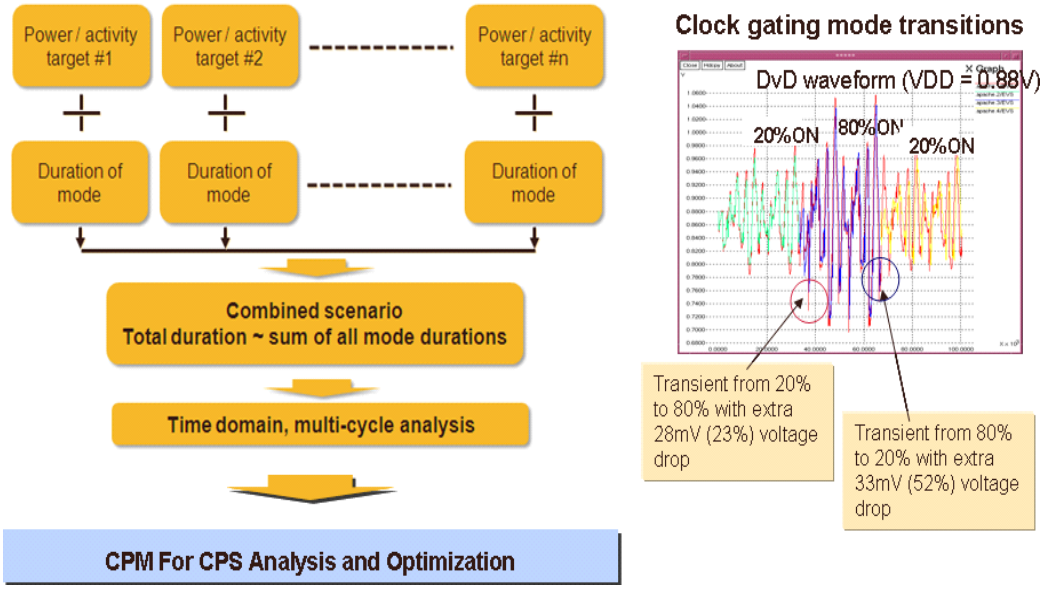


Figure 8-7 CPM Power Transient mode

CPM gets the Power Transient Analysis from the Totem dynamic run. The following GSR keyword are used to specify its operation:

```
POWER_TRANSIENT_ANALYSIS {
    <duration_frame_1_sec> <config_file_1>
    <duration_frame_2_sec> <config_file_2>
    <duration_frame_3_sec> <config_file_3>
    ...
}
```

Power transient configuration file

The CPM configuration file supports the following GSR keywords:

- GSC_FILE
- INSTANCE_POWER_FILE
- GSC_OVERRIDE_IPF
- BLOCK_POWER_FOR_SCALING (and BLOCK_POWER_FOR_SCALING_FILE)
- STATE_PROPAGATION {
  GATED_ON_PERCENTAGE <On_fraction>
  PROPAGATION_MODE PROBABILITY
  ? CONSTRAINT_FILE <constraint_filename> ?
  ? GATED_CLOCK_COVERAGE [0 | 1] ?
  ? GATED_CONTROL_FILE <control_filename> ?
  }
- TOGGLE_RATE_RATIO_COMB_FF
- INSTANCE_TOGGLE_RATE (and INSTANCE_TOGGLE_RATE_FILE)
- BLOCK_TOGGLE_RATE (and BLOCK_TOGGLE_RATE_FILE)
- TOGGLE_RATE

Settings in the configuration file versus the GSR file

The configuration keyword, if defined, overrides the same keyword in the GSR. The GSR keyword is used if the configuration file keyword is not defined, except for

- GSC_FILE
- STATE_PROPAGATION

which are ignored in the GSR if the Config file keyword is not defined.

Note that at present Power Transient Analysis works only in the Vectorless mode, and it does not support keywords other than the list in above. It does not support:

(a) VCD_FILE or BLOCK_VCD_FILE

(b) STA_FILE or BLOCK_STA_FILE

(c) BLOCK_POWER_ASSIGNMENT

(d) PAR or BLOCK_PAR

Also note that simulation uses charge from the first frame for its simulation reference, so the dynamic simulation result may vary depending on the first frame config setting.

User-configurable mode (Mixed Signal /Digital P&R Designs)

The user-configurable mode enables package and board designers to divide and customize a CPM for multiple individual system simulations, each targeting a specific operating mode or area of the chip, without having to regenerate the model for each single operating mode. The user-configurable CPM allows package and board engineers to mix and match contributions from different user-specified regions, or functional blocks of the die, creating unique scenarios that reflect different operating states of the chip. For example, an analysis of EMI from different blocks on the chip could be performed separately with one run.

First, you must partition the chip into groups of functions by hierarchical blocks and instances. Each group is represented by its unique current signature that can be enabled or disabled independently of the current signatures of other groups. The current signature sources corresponding to different groups are connected in parallel to each other, with sources belonging to each group marked by SPICE comments to activate one or more groups in a particular SPICE simulation, as shown in the Figure 8-8. Note that the sum of currents of all groups is equal to the current of the original CPM model. The passive part is also identical to that of the original CPM model.

The partitioning of the IC into multiple groups is achieved by specifying a list of blocks and instances that belong to each group in a “group file.” The resulting CPM model can be constructed for a vectorless or a VCD case.

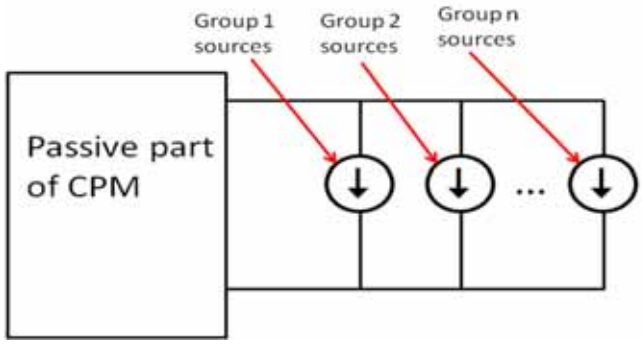


Figure 8-8 Creating a user-configurable CPM

To use the user-configurable feature, the following options of the 'perform powermodel' command are used:

-cpm_emi: enables the user-configurable feature (note that it does nothing specifically involving emi)

-group_file <file name>: specifies the name of the file used to define the partitioning of the chip into groups of hierarchical blocks and instances. A full or relative path to the file is allowed, but tilde '~' and environment variables are not allowed. The purpose of the group file is to assign switching instances (in the apache.scenario file) to groups. Note that instances can be grouped in hierarchical blocks, and the separator for blocks is slash ('/'). Users can assign a whole hierarchical block, a sub-block, or an individual instance to a group. Any instances that do not belong to any user-defined group are assigned to group Ng+1 (Ng = number of groups), with the name “others”.

The format of the group file is as follows:

```
GROUP  <group name1>
INSTANCE <inst_name_1A> ...  ;
GROUP <group name2>
INSTANCE <inst_name_2A> ...  ;
...
```

The keywords are case-sensitive, as are instance names. Wildcard names are supported, and lines starting with '#' are considered comments.

Example group file:

```
GROUP GS1
INSTANCE Bl_1
Bl_2/Inst1
Inst2
Bl_3/MEM*;
...
```

In this example, 'Bl_1' could be a hierarchical block name, 'Bl_2/Inst1' could be an instance name (or level 2 hierarchical block name), and 'Inst 2' may be an individual instance name. The syntax does not specify this. So group 1 contains all instances whose full name starts with “Bl_1”, followed either by '/' or white space, and also “Bl_2/Inst1” followed either by '/' or white space, and “Inst2” followed either by '/' or white space. If P = number of CPM ports, the resulting CPM model has (P-1)*Ng current signature PWL sources without the '-pincurrent' option, or P*Ng current sources with the '-pincurrent' option. The PWL sources belonging to the same group have names starting with the group name, such as “Icsg1_1”, “Icsg1_2”, for a group named “csg1”, and so on. This allows third party tools to easily enable or disable sources belonging to a particular group.

User-configurable CPMs can easily be applied to power delivery analysis because the current contribution from different portions of the chip has impact on voltage drops. For example, the worst voltage drop can be observed and optimized by turning off the current contribution of certain blocks in the chip. The source of noise can be identified using this approach, where different die activity scenarios are explored, to understand the block-

level contribution to the overall system noise. Results for a sample design are shown in Figure 8-9 and Figure 8-10.

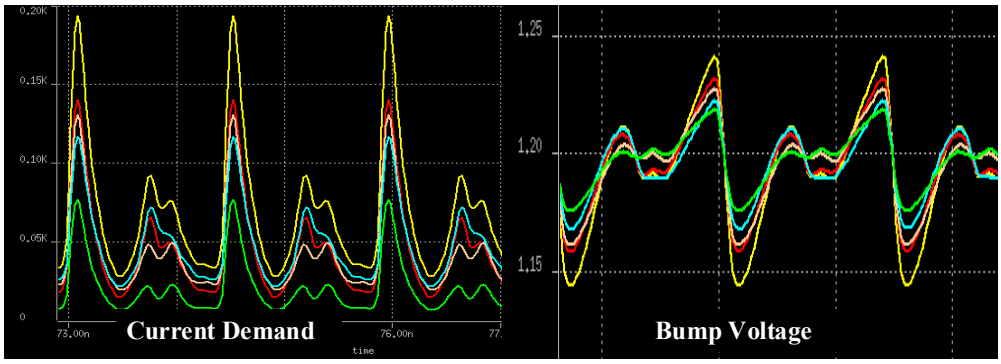


Figure 8-9 Analysis of multiple group current/voltages

Block 1	Block 2	Rest of Chip	Peak Current	Peak-to-Peak DvD
ON	ON	ON	192A	95mV
OFF	ON	ON	139A	72mV
ON	OFF	ON	130A	66mV
ON	ON	OFF	116A	53mV
OFF	OFF	ON	76A	42mV

Figure 8-10 Example CPM analysis results

CPM LDO analysis support

CPM models can be generated for chips with LDO features using the 'perform powermodel' command with no additional options. The number of terminals created corresponds to the behavioral LDO models that APLDO generates. The presence of LDO features is detected automatically. The CPM option '-plocname' is always enabled if LDOs are present. CPM generates additional internal ports at the pins (terminals) of the LDO models. The main CPM subcircuit (.SUBCKT adsPowerModel) has terminals only for the CPM ports at the pads, the same as it would without the LDO models. The CPM contains comments indicating the name of the LDO instance and the name of the SPICE subcircuit instantiated for the LDO instance. For example:

```
* ****** CPM-LDO ***
* LDO cell | SPICE subciruit name
* --------------------------------
* VDD_REG_FC_ISO_580173    VDD_REG_FC_ISO
* --------------------------------
* VDDCO_REG_FC_ISO_580174    VDDCO_REG_FC_ISO
* --------------------------------
```

The subcircuit *adsPowerModel* instantiates the LDO subcircuits with one per LDO instance. For example:

```
**** Instantiation of the LDO subcircuits : ***
X_VDD_REG_FC_ISO_580173 VDD_REG_FC_ISO_580173__vddhv
    VDD_REG_FC_ISO_580173__vddcore0
```

```
+ VDD_REG_FC_ISO_580173__vddcore1 VDD_REG_FC_ISO_580173__vss
    VDD_REG_FC_ISO
X_VDDCO_REG_FC_ISO_580174 VDDCO_REG_FC_ISO_580174__vddhvcore
    VDDCO_REG_FC_ISO_580174__vddcore0
+ VDDCO_REG_FC_ISO_580174__vddcore1 VDDCO_REG_FC_ISO_580174__vss
    VDDCO_REG_FC_ISO
*****************************************
```

To perform system-level simulation with CPM, you must create a SPICE netlist that includes the models for package/PCB and supply sources. If LDOs are present, you must also provide the LDO subcircuits, which can be either transistor-level SPICE models or behavioral models. Transistor-level LDO models provide the greatest accuracy, while the behavioral models may have better simulation speed. If transistor-level LDO models are used, they typically have additional terminals, such as for the bias pins. They must be “wrapped”, along with the bias voltage sources and other required circuitry, to generate the subcircuit that is invoked by the CPM.

For best accuracy, the following two options of the 'perform powermodel' command are recommended: -pincurrent and -global_gnd.

Notes on using the LDO methodology:

1. No LDO subcircuits are instantiated in the binary CPM (Sentinel-PI format).
2. The CPM header includes coordinates and layer information for the LDO subcircuit terminals.
3. LDO can be used with the CPM '-probe' option, but not with the '-internal_node' option of the 'perform powermodel' command. Example:
   ```
   perform pwrmodel -wirebond -global_gnd -pincurrent
       -o ABCD_ldo_de BZ.sp -probe
   ```
4. The -probe option requires that the PROBE_NODE_FILE GSR keyword be used, as shown below:
   ```
   PROBE_NODE_FILE <file_path>
   ```
   Format of the probe node file:
   ```
   <x coordinate> < y coordinate> <layer> <probe name>
   ...
   ```
5. It is possible to have a probe node exactly at the location of the LDO terminal. In this case, the probe node becomes one CPM port and it becomes exposed. Normally, the terminals of the LDO subcircuits are not exposed.

CPM Outputs

The outputs from CPM processing are the following:

- CPM model files
- *get_cdie.sp* NSPICE netlist file
- **.cdie* file, effective die series resistance and capacitance values for ‘1 X 1’ partition models

These outputs are described in the following paragraphs.

CPM Model Files

The CPM model has several elements: the passive part of the CPM model (**.sp.inc*), and the current signatures by partition (**.sp*), as shown in Figure 8-11. The output of Totem

CPM simulation is a hierarchical SPICE netlist that includes the switching current signature, as well as the reduced-order model of the die power delivery network. The complexity of the SPICE netlist is determined by the number of partitions over which the CPM model is created.

Three types of models are generated simultaneously during the run:

- standard SPICE (default)
- CPM optimized for HSPICE (generated with the '-hspice' option of 'perform powermodel' command). For designs with a large number of ports (100+), the use of HSPICE-specific elements (Foster VCCS) is important to provide a way to eliminate internal nodes, improve simulation speed and reduce memory consumption.
- Sentinel-PI CPM, which is the same as the HSPICE format, but in binary form.

get_cdie.sp File

The NSPICE netlist is exported to the file *get_cdie.sp*, which can be used to calculate Rdie/Cdie at the specified frequency (50 MHz by default, unless you modify it in the *get_cdie.sp* file), and to plot Cdie(f) and Rdie(f) for frequencies ranging from 1 MHz to 1 GHz. This file eliminates user errors, which are likely for a CPM model with a large number of ports, and also saves time that would be required to create the netlist by hand.

For designs with multiple power/ground nets Totem generates a separate *get_cdie*.sp* file for each pair of nets. If there are only two nets, the file name is still g*et_cdie.sp*. Otherwise file names of the form *get_cdie_<pwr_net>_<gnd_net>.sp* are written to the working directory.

The CPM model corresponding to the specified option (default, -hspice, -sentinelpi) is placed in the run directory. .

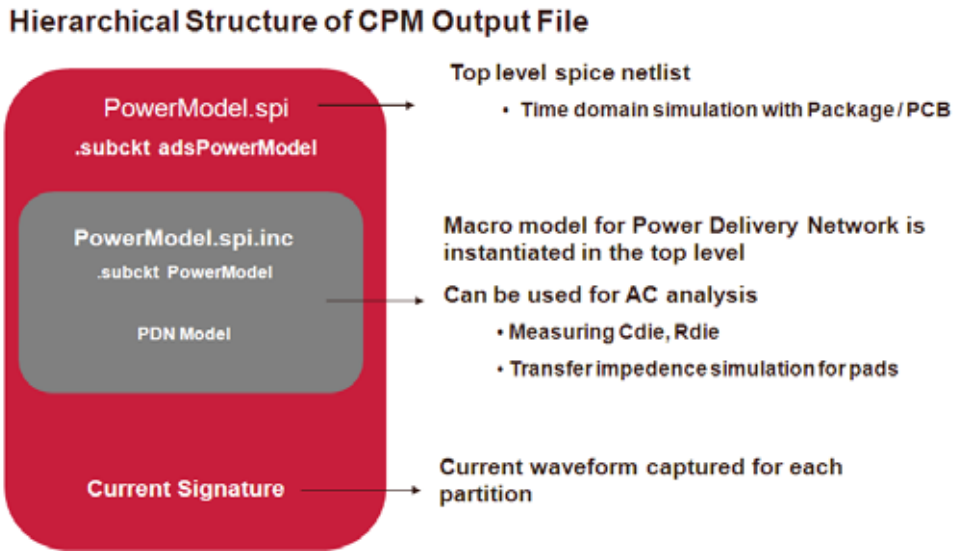


Figure 8-11 CPM outputs

*.cdie File

If you run CPM simulation with 1 x 1 partition option using '-nx 1 -ny 1' (even for a wirebond design), or the -cdie option, a **.Cdie* file is generated in the *<workdir>/adsRpt/CPM/apache.Cdie* directory. The **.Cdie* file contains the effective capacitance, as well as the effective series resistance, of the die PDN at various frequency points. This is useful as a quick ballpark assessment of die parasitics.

The following is an example of a *.*Cdie* file:

```
Cdie and Rdie for net VDD__VSS
...
1.000000e+06Hz   Rdie = 9.180318e-01   Cdie = 6.778965e-10
1.000000e+07Hz   Rdie = 9.079731e-01   Cdie = 6.778965e-10
1.000000e+08Hz   Rdie = 9.078725e-01   Cdie = 6.778965e-10
1.000000e+09Hz   Rdie = 9.078715e-01   Cdie = 6.778965e-10
...
```

For AC analysis, you should connect to the parasitics part only (*.inc* file), and do not connect the CPM port to global ground (SPICE node 0). For current signature, by default, CPM uses the Norton current model. It picks one of the ground ports as the ground for every pwl. The total current should be zero. If you generate a CPM with the option' -pincurrent' it doesn't use the Norton current model. Every pwl is referenced to node 0. Note that some tools do not support non-Norton current models.

The CPM is described in hierarchical subcircuit-based SPICE netlists. The main CPM SPICE deck includes piecewise linear switching current signatures, which are connected to the parasitic model of the die power delivery network, which is in turn described in a second SPICE netlist file (*.inc* file). A two-port equivalent model is shown in Figure 8-12.

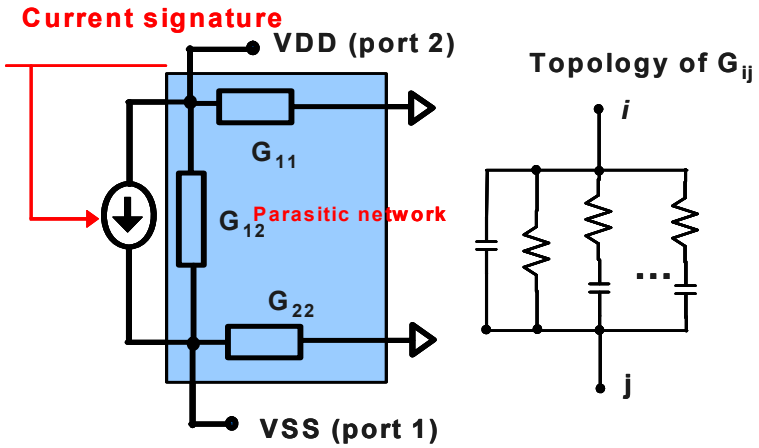


Figure 8-12 Two-port CPM equivalent circuit

The following is a simple CPM generated for a die (one Vdd and one Vss net) using a 1 x 1 partition. The Chip Package Protocol section lists the name of the die pad, x and y location of the pad, the corresponding SPICE node name, as well as the partition and net information of this pad.

```
*****************************************
* Apache Totem Chip Power Model [ Ver 1.00 ]
* @Apache Design Solutions 2002 - 2006
*****************************************

.INCLUDE tutorial_cpm.sp.inc

* Begin Chip Package Protocol --->
```

```
* VSS29  :   (308.000000 7196.000000)    :   p2   = PAR_0_0_VSS
* VSS28  :   (308.000000 7047.000000)    :   p2   = PAR_0_0_VSS
* VSS27  :   (308.000000 6900.000000)    :   p2   = PAR_0_0_VSS
* VSS26  :   (308.000000 3021.000000)    :   p2   = PAR_0_0_VSS
* ...
* VDD29  :   (308.000000 7645.000000)    :   p1   = PAR_0_0_VDD
* VDD28  :   (308.000000 7489.000000)    :   p1   = PAR_0_0_VDD
* VDD27  :   (308.000000 7343.000000)    :   p1   = PAR_0_0_VDD
* ...
* End Chip Package Protocol <---
.subckt adsPowerModel
+ p1 p2

Xpdn
+ p1 p2
+ PowerModel

Icursig1 p1 p2 pwl(
+ 0.000000ps -0.000000
+ 30.000000ps 0.014396
+ ...
+)
.ends
```

Using the Chip Power Model

You can attach the SPICE netlist created from CPM to the package/PCB models and simulate those using NSPICE or any other SPICE compatible simulator.

Some of the key applications of Chip Power Model technology are:

- Determining global Power Delivery Network impedance, identifying IC-package resonance using AC analysis (use the parasitic part only).
- Performing dynamic voltage noise budgeting at board and package level
- Package and board decap optimization

Validating the Model

One type of accuracy validation of the CPM model involves the following steps:

1. Run Totem without any package models to generate the CPM model. Accuracy improves if a sufficiently small transient time step is used.
2. Attach a package model to the CPM model and run using SPICE.
3. Measure the current and the voltage at the connection points (die pads) of the CPM model to the package model.
4. Attach the same package model to Totem and run DvD analysis (with the same pre-simulation time as used for CPM generation). Remember to shift the waveform by the pre-simulation time, which is prior to t=0 in Totem and after it in SPICE.
5. Measure the current profile for all the bumps and the VDD/GND waveform at any bump.
6. Compare the two waveforms.

Figure 8-13 below shows waveforms from the measurement of current from one test-case using the above procedure. The red current profile is from measuring the bump current in a Totem run, while the yellow current profile is from measuring the bump current from the SPICE run with the CPM model.

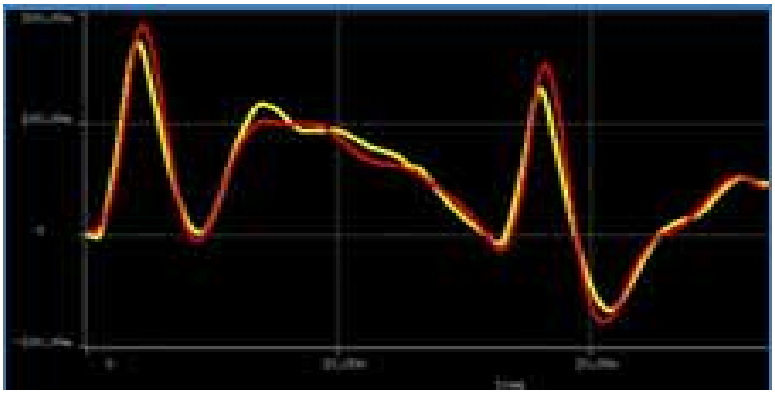


Figure 8-13 Comparison of CPM model with full Totem chip simulation

Chapter 9
Fixing and Optimizing Grid and Power Performance

Introduction

After Totem static (IR) or dynamic voltage drop (DvD) analyses have identified areas in the design that have unacceptable voltage drops, Totem Fixing and Optimization (FAO) functions allow you to modify the power grid and/or allocate new decoupling capacitance to meet specific static and dynamic voltage drop targets, and also power-related circuit timing problems. Fixing voltage drop problems also can reduce noise from ground bounce. Operations on instances with multiple Vdd power sources is handled transparently.

Once a trial power grid and initial placement information is available, Totem FAO can be used to globally modify power grid parameters to meet specific voltage drop targets. Totem enables you to easily add or delete power/ground pads, straps, and vias at any stage of a run to explore their impact on voltage drop and electro-migration (EM). Totem is capable of incrementally taking in new changes during IR drop and EM analysis. After detailed placement, and prior to detailed routing, Totem can be used to identify high dynamic voltage drop areas, or areas with significant timing impacts from DvD, and fix them through a combination of power grid wire changes and targeted decap placement.

The first part of this chapter describes the manual grid and decap modification commands available in Totem, as follows:

- Changing existing layer or via resistivity
- Adding a power/ground pad
- Deleting a power/ground pad
- Adding a via
- Deleting a via
- Adding a power strap
- Editing/deleting a power strap
- Adding a decap cell
- Adding metal layers and via or via arrays
- Writing an ECO file
- Reading an ECO file

As the design moves through final placement and routing, you can use Totem Fixing and Optimization functions to automatically modify the grid parameters and decap placement to meet specified goals for both voltage drop and also power-related circuit timing. When

all of your voltage drop constraints are met, Totem can export an ECO file with the needed design changes and then perform a final SPICE-accurate power analysis sign-off of the design.

You can perform Undo or Redo operations after making FAO changes, allowing you to try out FAO changes and then reverse them if the results are not satisfactory. Any number of Undo or Redo commands can be performed consecutively until the commands in the stack are exhausted.

These various fixing and optimization functions are described in the following sections.

Manual Power Grid Modification

Changing a Metal Layer or Via Resistivity

An existing metal layer or via resistivity can be changed in the Totem technology (*.tech)* file and then Totem can be rerun to see the effects of the changes. Please refer to section "Apache Technology File (*.tech)", page C-744, for the format of the Apache *.tech* file.

Adding a Power/Ground Pad

New power or ground pads can be created using the **Edit > Add Pad** command, as shown in Figure 9-1. A pop-up menu allows the selection of the metal layer to which the pad will be connected. Pick the location for the new pad on the metal layer by clicking at the desired location. The new pad is added to the design in the selected location.

New power pads are displayed as orange rectangular boxes and new ground pads are displayed as white boxes. Use **File > Export ECO** to export the added pad locations and use **File > Import ECO** to import the ECO file to back annotate for future runs. See the section "Saving Design Changes with the ECO Command", page 9-294, for more information on the **ECO** command.

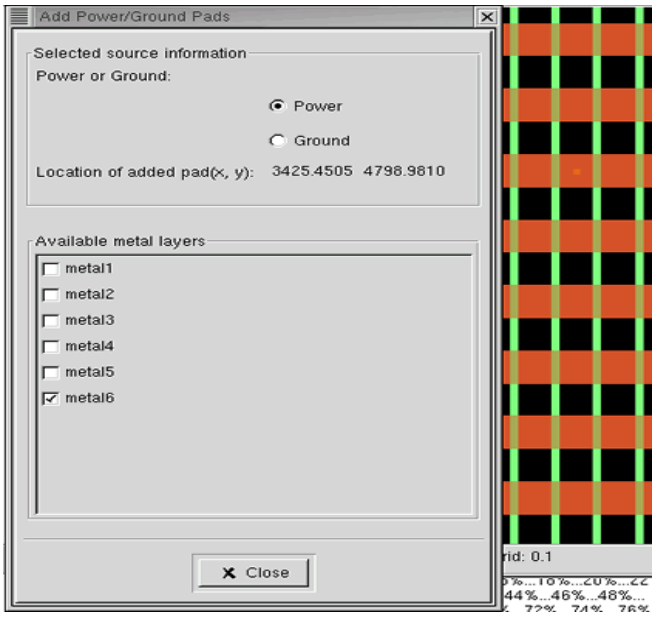


Figure 9-1 Edit > Add Pad Menu

Adding a Set of Power/Ground Pads over a Specified Area

The command 'fao add pad' adds a set of power /ground pads over an entire mesh, or in a defined window. You must specify a window, a metal layer, a net, and also a pitch in both x and y directions by which to distribute pads evenly over the specified region. **Undo**/**Redo** commands can be used. The command line syntax is:

```
fao add pad -window {llx lly urx ury} -metal <layer>
    -net <net_name> -pitchX <microns> -pitchY <microns>
    -r <resistance> -l <inductance> -c <capacitance>
```

where

-window { }: defines lower left and upper right x,y coordinates of the rectangular area in which pads are added (by default, the entire mesh).

-metal: defines the routing layer (required). Only one metal layer is allowed, since every metal has its own placement.

-net: defines the net name (required)

-pitchX -pitchY: defines the incremental distance between pads, starting at lower left corner of the rectangle specified by '-window { }' (required) .

For example, for “-window {a b c d}”, starting at {a b}, Totem adds pads at {a b}, {a+pitchX b}, {a+2*pitchX b} ..., {a b+pitchY}, {a+pitchX b+pitchY}, {a+2*pitchX b+pitchY} ... , over the entire window.

-r -l -c: specifies pad RLC parameters (units the same as in the PLOC file)

Deleting a Power/Ground Pad

An existing power/ground pad can be selected and deleted using the command **Edit > Delete Pad**. Use the pop-up menu to choose a pad for deletion.

Adding a Via

New vias can be added using the command **Edit->Add Via**. Click on the design in the desired location. Use the pop-up form to define the top and bottom metal layers for the inserted via stack. When adding stacked vias, make sure that the appropriate number of vias and via layers are selected. The new via or stacked vias are added to the design and displayed as a blue box, after saving the changes.

Deleting a Via

An existing via can be selected and deleted using the command **Edit > Delete Via**. You may have to make the vias visible by turning them on with the **View Layers** Configuration button, or the command **View -> Map Configuration -> Layers Color Map**, and then select **Fill** or **Outline** display for the set of vias you want to look at. First select the via for deletion by clicking on it with the left mouse button. The selected via is highlighted in yellow. Use the pop-up menu to choose an existing via for deletion.

Adding One or Multiple Power Straps

1. To add power straps graphically (recommended), select **Edit -> Add Power Strap**. A ‘Power Strap’ dialog box is displayed.
2. Deselect **Add strap by text input**.
3. Display the layers that are important in relation to the new strap.

4. Use the right mouse button to draw a desired strap on the layout. If you want to add several identical straps, draw the first strap representing one edge of the set you want to add, and select **Add multiple straps**.
5. Calculate and enter in the dialog box the location of the outside edge of the last strap to be added and also the desired pitch. Totem will add the maximum number of legal straps meeting the constraints specified.
6. Select the layer on which to add the new straps (metal1 is the default).
7. Click on **Commit adding** when the specification and drawn straps are correct (there is no undo function). If there is a physical error or short, an error message will be displayed. A successful strap addition is illustrated in Figure 9-3.
8. If you make a mistake or want to change the added straps, use the **Edit->Power Strap** function to select and then delete or change a strap.

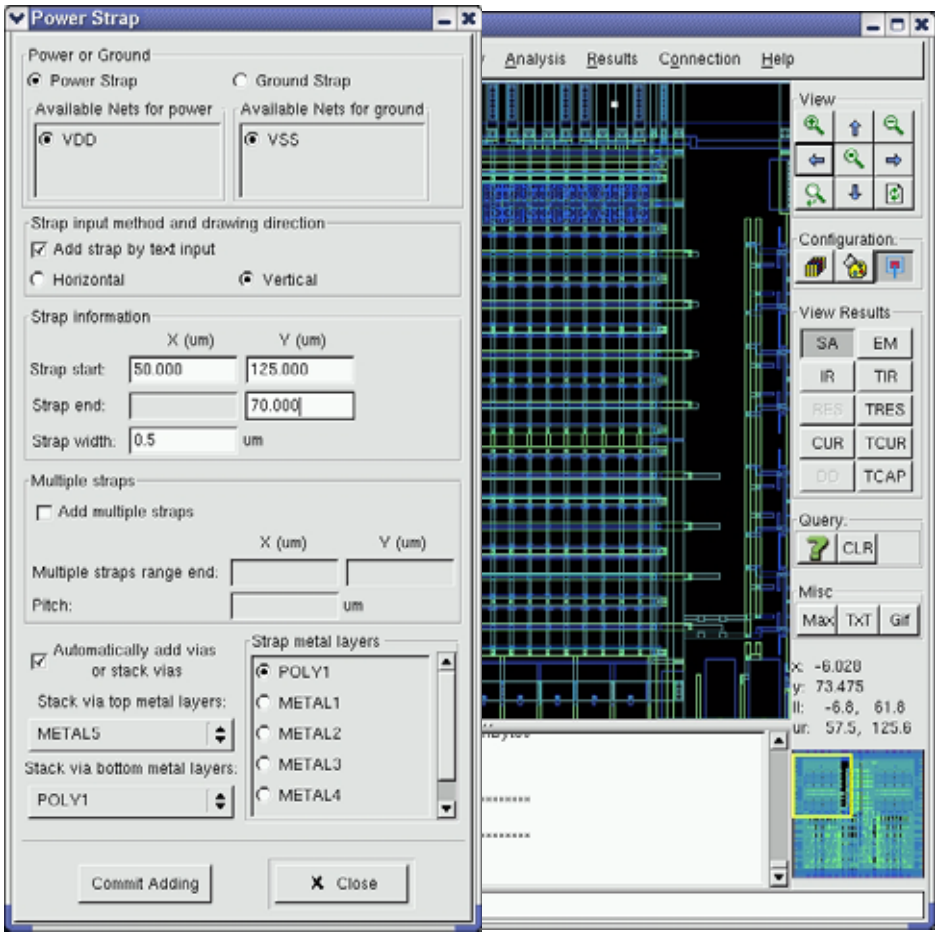


Figure 9-2 Form for adding a strap by text input

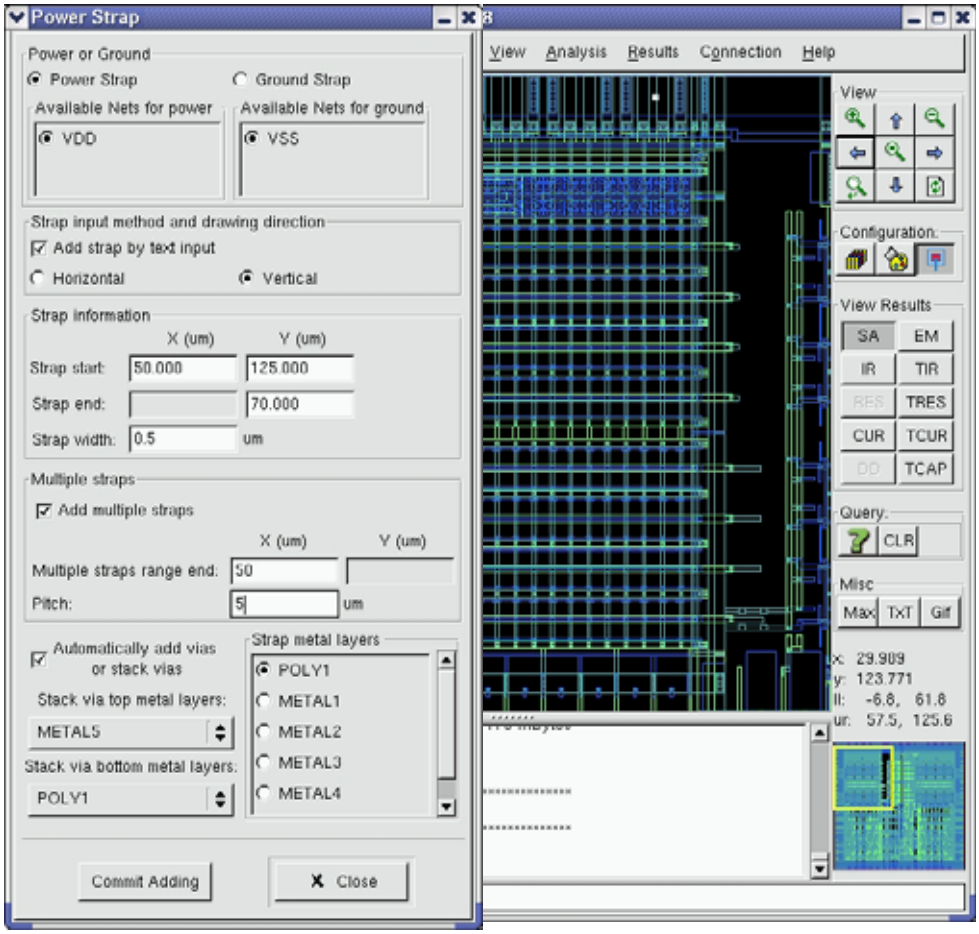


Figure 9-3 Adding multiple straps with Add multiple straps

Editing/Deleting a Power Strap

An existing power strap can be selected and edited by using the command **Edit > Edit Power Strap**. First, select the power strap by clicking on it with the left mouse button. The selected strap will be highlighted in yellow. Use the pop-up menu to change the length and width, as well as the x and y start locations of the selected power strap, as illustrated in Figure 9-4. The strap can also be deleted from the design database.

NOTE:	New vias are NOT automatically added as a result of editing an existing power strap.

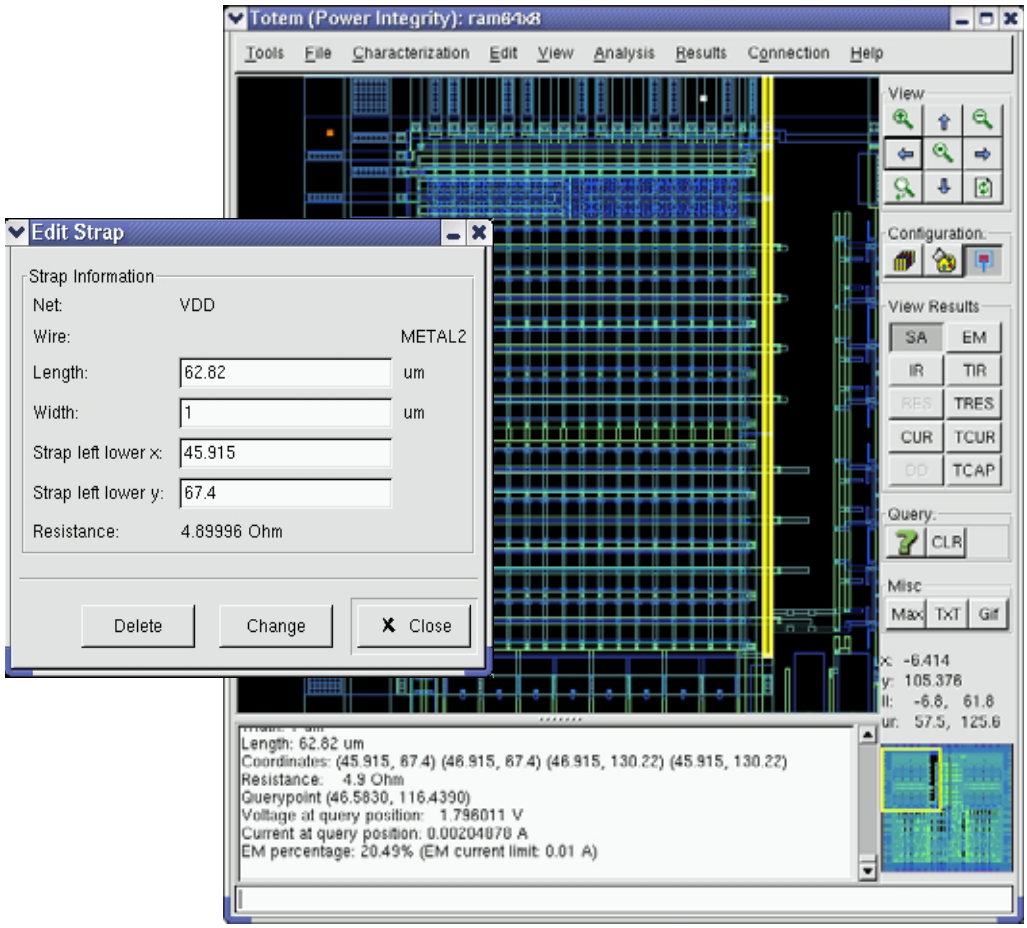


Figure 9-4 **Edit->Edit Power Strap**

Adding a Decap Cell

One or more intentional decap cells can be added by using the command **Edit > Add Decap Cell**. Click on the **Select Vdd** button on the Add Decap Cell form and then click on the VDD wire location on the lowest metal layer where the decap should be inserted. Next, click on the **Select Gnd** button on the form and click on a VSS location very close to the previously selected VDD location. A list of available decap models will be shown on the form, which were previously defined by the 'DECAP_CELL' keyword in the GSR file. If no appropriate models have been defined, you must provide them in the GSR file.

NOTE:	Corresponding DECAP cell definitions must exist in the LEF file and in the GSR file.

After intentional decaps are characterized through the Totem APL flow, obtain the ESC (effective power circuit capacitance) and ESR (effective power circuit resistance) values from the *<cell>.cdev* file. Click on **Commit Adding** to see the decap appear on the layout. The addition of decap is now complete, as illustrated in Figure 9-5 and Figure 9-6. Use **File -> Export ECO** to write out the added decaps and export the ECO files for inclusion in the design on future runs.

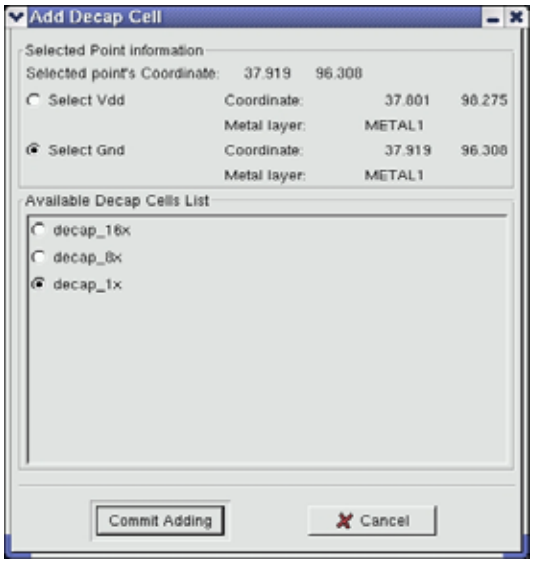


Figure 9-5 **Edit -> Add Decap Cell**

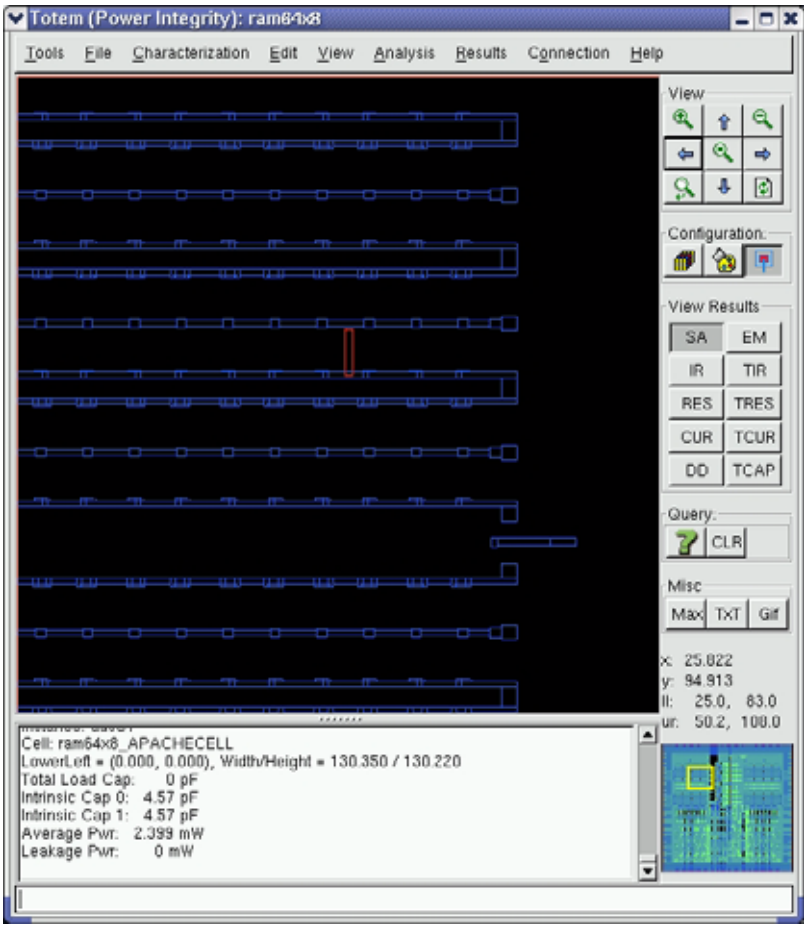


Figure 9-6 **Decap added on layout**

Adding Metal Layers and Vias or Via Arrays

The following steps allow you to add straps on an additional layer of metal, in addition to the existing metal layers. For example, if the existing layers of metal are metal1 to metal6, the additional layer of metal should be specified as metal7, on top of metal6.

1. Add the extra layer definition to LAYER section of the *.lef* file, which contains the technology definition for layers. In the example below, there are six metal layers. Edit the *.lef* file to add metal7 and via67 as shown below. Make sure that the sequence and numbering are correct (i.e., specify VIA67 and METAL7 after METAL6).

```
LAYER VIA67
  TYPE CUT ;
END VIA67
LAYER METAL7
  TYPE ROUTING ;
END METAL7
VIA via6 DEFAULT
# (Worst case resistance model for via5 = 2.54 ohm/ct) = 2.5400e+00
RESISTANCE 2.5400e+00 ;
LAYER METAL6 ;
  RECT -0.240 -0.190 0.240 0.190 ;
LAYER VIA67 ;
  RECT -0.180 -0.180 0.180 0.180 ;
LAYER METAL7 ;
  RECT -0.270 -0.270 0.270 0.270 ;
END via6
```

Add the metal layer and via information the **.tech* file, as shown below.

```
metal METAL7 {
  Thickness 0.4
  Resistance 0.065
  TC 0
  EM 1.22
  Above   DIEL6
}
via VIA67
{
 Width { 0.19 }
 Resistance 1.2
 EM 0.126
 UpperLayer metal7
 LowerLayer metal6
}
```

2. Create the required via arrays in the VIAS section of the top-level *.def* file. Since there is no existing via array definition, you must create the required via arrays in the VIAS section. The via arrays for VIA67 can be created by copying the VIA56 definition in the existing *.def* file and changing the parameters to match VIA67, as shown below

```
- $TCARY$via6_0_2_1_1420_1420_F + RECT METAL6 ( -1200 -380 )
( 1200 380 ) + RECT METAL7 ( -1260 -540 ) ( 1260 540 ) + RECT VIA67
( -1080 -360 ) ( -360 360 ) RECT VIA67 ( 360 -360 ) ( 1080 360 ) ;
```

3. After the metal layer and via arrays are created, follow the same steps to add pads and straps, as previously discussed.

Undo and Redo

You can perform Undo or Redo operations after making FAO changes, allowing you to try out FAO changes and then reverse them if the results are not satisfactory. Any number of **Edit -> Undo** or **Edit -> Redo** menu commands can be performed consecutively until the commands in the stack are exhausted. On the TCL command line, ‘fao undo’ and ‘fao redo’ commands also can be executed with the same results.

Automated Grid Optimization and Fixing Procedures

Fixing and Optimization Flow for Static IR Drop Improvement

The primary steps in the FAO static voltage drop optimization flow are outlined in Figure 9-7 below.

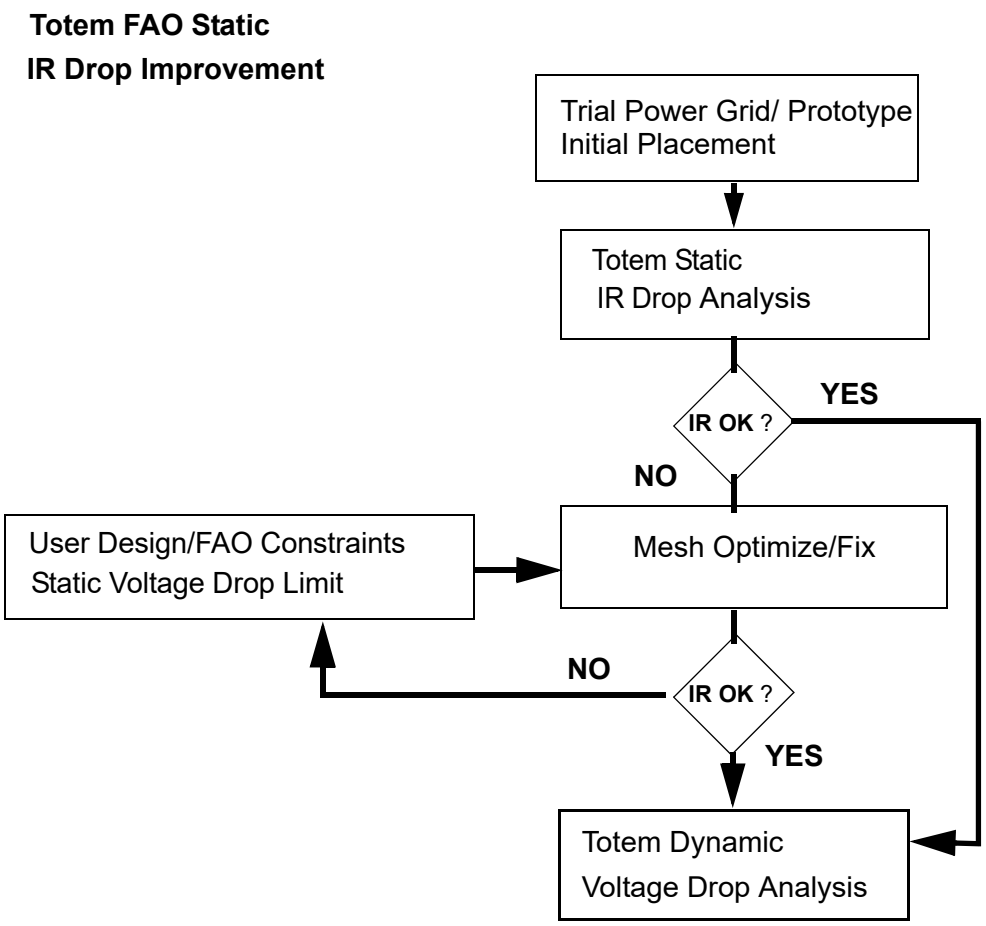


Figure 9-7 Totem flow for static IR drop analysis

FAO Procedure for Multiple Vdd Designs

For multiple Vdd domain designs, only one power domain is analyzed and modified by FAO at a time. Specify each power domain in order using the GSR keyword FAO_NETS . For example, if there are two power domains, VDD and VDD_B, first invoke:

```
gsr set fao_nets VDD
```

Now you can run FAO analysis, and perform the needed VDD grid and decap modifications. Following fixing and optimization on the VDD net, use the command:

```
gsr set fao_nets VDD_B
```

Then run FAO analysis and power problem mitigation on the VDD_B net.

If there is more than 1 power net defined in fao_nets, only the first one listed is considered, and Totem returns the following warning:

```
WARNING: ================================================
WARNING: *** FAO_NETS has > 1 VDD net: Only consider <VDD>
WARNING: *** Set fao_nets for EACH power net and apply fao separately.
WARNING: ================================================
```

Grid Optimization

Mesh Commands

The key commands for using FAO to adjust grid widths are invoked in the GUI or TCL format, and have the following functionality:

- `mesh optimize` - investigates a grid width solution within a defined area (fao_region). All wire segments on the specified metal layers will be adjusted to the same width to solve the voltage drop problem, either for the full chip or within the width of the specified region (but the full height of the chip). With the -taper option, grid wire segments are only resized within fao_region, and not outside of it (not the full height of the chip).
- `mesh fix` - investigates a grid width solution within a rectangular region that includes all “hot spots” (high voltage drop nodes) and their associated fix_window areas. For all hot spots within fao_region, wire segments in vertical and horizontal bands (full width and height of chip) defined by the dimensions of the fix_window, will be adjusted to an appropriate width to solve the voltage drop problem. With the -taper option, grid wire segments are only resized within the specified region (fao_region), and not outside of it.
- `mesh snscalc` - for a selected region and set of layers and nets, mesh snscalc calculates and reports a number representing the relative average sensitivity for modifying the width of wires associated each layer and net selected--the higher the sensitivity number reported (in mv of voltage drop reduction per micron of additional wire width), the more effective a wire width change would be in reducing the worst-case voltage drop within the selected region. Using this command is recommended before performing any grid resizing.
- `mesh sub_grid` - adds a new subgrid mesh, with an area defined by the GSR keyword 'fao_region', including power nets defined by 'fao_sub_grid_nets' and with physical characteristics defined by 'fao_sub_grid_spec'. The new subgrid is targeted to achieve a voltage drop specified by the GSR keyword 'noise_reduction' . The min/max width of the new grids to be sized is specified by 'fao_range'. The minimum width is also used to create the initial subgrid before sizing analysis.

- `mesh set_width` - allows the mesh wire width to be modified for the specified layer(s). By default the wire width is modified equally on both sides of the mesh center line, but the width change can be made on only one side of the wire by specifying the 'expand_dir' (or it could be the contracting direction if you are making the width smaller).

Several additional commands are available for modifying the power grid, but are not needed frequently, and are summarized below.

- `mesh add` - allows you to add a specified mesh section to the grid, including specific layers and regions of the design. Useful for easily modifying the grid and evaluating the effect on voltage drop
- `mesh delete` - allows you to delete a specified mesh section of the grid, including specific layers and regions of the design. Useful for easily modifying the grid and evaluating the effect on voltage drop
- `mesh vias` - allows you to add an optimal set of vias in a region of the design using specified via models
- `mesh generate` - generates a prototype grid based on the user constraints that are provided in the design constraints file (*.dcf*)
- `ring add/delete` - adds or deletes a specified power/ground ring

Following is a brief summary of the GSR keywords available to control grid modification. The general TCL syntax for setting GSR keyword values is: `gsr set <variable name> <value>`. All FAO options and GSR keyword values will be collected in Totem and then the GSR file can be used to set initial values and then save-and-reload will set the GSR keyword values properly. More detailed descriptions of these GSR keywords and their options and syntax are provided in section "Grid Fixing and Optimization Keywords", page C-881

GSR Keywords for Region-based Grid Width Sizing (mesh optimize)

1. To select IR drop or DvD analysis: fao_dynamic_mode (default - off)

2. To set a target voltage drop reduction: noise_reduction (input required)

3. To specify a net or group of nets for optimization: fao_nets (input required)

4, 5. To specify layer and grid width constraints: fao_layers, fao_range (input required for both)

6. To select accurate or turbo mode: fao_turbo_mode (default is on)

7. To specify an optimization area: fao_region (default - whole chip)

8.To select the number of high voltage drop areas to optimize: num_hotspot (default - 1)

9. To specify layer/width relationships: fao_width_cnstr (default: no specification)

10. To specify the percentage voltage drop reduction to be achieved on specific layers: mesh_search (default - even distribution on selected layers)

GSR Keywords for Hot-spot Based Grid Width Fixing (mesh fix)

GSR keywords 1 through 10 described above for mesh optimize also can be used for `mesh fix`. In this command 'fao_region' is used to define the overall evaluation area, and a "fix window" around each hotspot within the region is investigated for a grid width modification, as follows:

11. To select the horizontal and vertical dimension of the area, centered on each hotspot, in which to modify the grid widths: fix_window (default - 100u x 100u)

Several example static IR drop analyses are provided in the following section.

Examples of Grid Optimization

Example A - Full Chip Optimization - Relaxing Static IR Drop

Conditions: Early in design development. Your design allows you to relax (increase) the existing worst case voltage drop and perhaps save grid resources. You want to use full chip uniform grid optimization methodology, so **mesh optimize** is used.

Goal: Perform full power grid optimization, increasing the static IR drop target 15%.

1. Decide on what GSR keyword settings you want to set for performing a ‘mesh optimize’ operation. Assume in this case the initial worst case static IR drop is 18mV on VDD, and we can accept a slightly higher voltage drop without problems. Therefore we will do a full grid optimization to relax this drop by 15%, as shown in the following TCL input lines:

   ```
   gsr set fao_turbo_mode 0
   gsr set fao_nets VDD
   gsr set fao_layers { {metal6 hor <= 29.4} }
   gsr set fao_range { {metal6 8 29.4}}
   gsr set noise_reduction -15
   ```

   No region has been defined using ‘fao_region’, so the entire power grid will be optimized (default). For this run FAO will do an 'accurate' mode search of low static voltage drop areas and optimize only the VDD net. It will examine all METAL6 wires running horizontally of width less than or equal to 29.4um (the existing grid width) as its target wires. FAO will try to relax the worst case static voltage drop by 15%. Note that this will create a reduction in metal usage, so you must specify a width range less than the existing width, such as the 8 to 29.4u range in this example.

2. Run mesh optimize as follows, specifying an output eco file:

   ```
   mesh optimize -eco <filename>.eco
   ```

3. Based on the requested grid optimization parameters, Totem will report the wires it optimized in the region defined (the whole grid in this case), as follows

   ```
   ECO File: fao_mesh_opt_23:15:47_022242005.eco
   Region: <0 0 6817.32 8316.08>
   FAO Mode: accurate
   Simulation Mode: static
   Net: VDD
   Noise Reduction: -15%
   ++++++++++++++++++++++++++++++++++++++
   Initial Worst Static Noise: 17.578mv
   Voltage Noise Constraints: 20.2147mv
   *=========================================*
   *          Searching Target Grids         *
   *=========================================*
   Net <VDD>:
      Layer <metal6>: identified 44 wires
   *=========================================*
   *           Sizing Target Grids           *
   *=========================================*
   ```

   Following the run, FAO reports 44 wire change recommendations, as shown in the log file below. In this case, there will be less metal usage in the metal6 VDD net as

a result of the relaxation in the static voltage drop limit. Note that new via models also are specified to match the wire changes

```
Optimization succeed: no error
Optimized width of layer metal6: 23.8um
*=========================================================*
 Metal Usage Change Report
    Net <VDD>:
        Layer <metal6>:5.05281e+06um^2 => 4.09119e+06um^2 (-19.03%)
    Total: 5.05281e+06um^2 => 4.09119e+06^2 (-19.03%)
*=========================================================*
Replacing optimized grids...
Replace <metal6> mesh grids :
    net <VDD> : 44 wires replaced.
DEF file for new via models: SH_VIA_VDD.def
```

To do a final check on the fix, rerun Totem with the same setup commands and options as described in the initial preparation steps. Package RLC units are Ohms, pH and pF, respectively:

```
perform extraction
setup pad
setup wirebond
setup package
perform analysis -static
```

The final static voltage drop on the VDD network is 21mV (a 15% relaxation), with a 19% saving in metal resources over the entire design.

Example B - Full Chip Grid Fixing - Reducing Static IR Drop

Conditions: Early in design development. Your design requires a lower voltage drop. You want to use full chip uniform grid optimize methodology, so **mesh optimize** is used.

Goal: Perform full power grid optimization for static voltage, reducing the static IR drop target from the existing 18 mv, as shown in Figure 9-8.

1. Decide on what GSR keyword settings are necessary. This time we will tighten the voltage drop, evaluating the grid width on metal6 for whole design.

   Keep in mind if a tighter voltage drop limit is specified (using ‘noise_reduction’), then the fao_range should be increased to allow FAO to increase grid widths . For example, if you want to reduce the voltage limit by 10%, then the following GSR keyword settings should be used:

   ```
   gsr set fao_turbo_mode 0
   gsr set fao_nets VDD
   gsr set fao_layers {{metal6 hor <= 29.4}}
   gsr set fao_range {{metal6 10 35}}
   gsr set noise_reduction 10
   ```

   The allowable range of horizontal metal6 widths for this analysis is now 10-35 microns, instead of 8-29.4u in the previous example, and the ‘noise_reduction’ value is now a positive number, 10 percent.

2. Run mesh optimize as follows:

   ```
   mesh optimize -eco <filename>.eco
   ```

3. Look at your results as described previously.

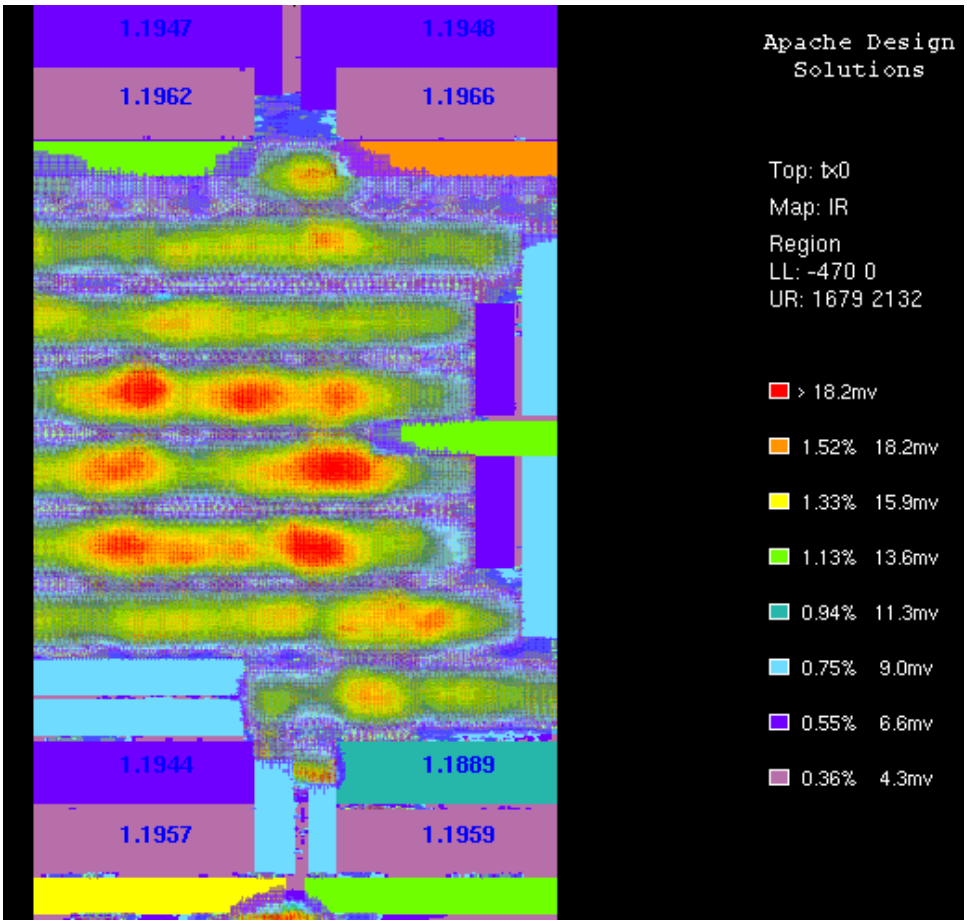


Figure 9-8 Static IR drop map

Example C - Partial Chip Optimization, Relaxing Static IR Drop

Conditions: Later in design development. Your design can accept a higher voltage drop than 18mv, but you do not want to make changes over the entire grid. You want to specify a “fix window” of a particular size, so mesh fix is used.

Goal: To perform a limited power grid fix to relax the static IR drop 15% by modifying wire widths for only a selected area of the power grid.

1. Decide on what GSR keyword settings are necessary. This time we will allow the voltage drop to increase, evaluating the grid width on metal6 for part of the design, as shown in the GSR keyword values set below:

```
gsr set fao_region {2600 2677 6487 5304}
gsr set fao_turbo_mode 0
gsr set fao_nets VDD
gsr set fao_layers { {metal hor <= 29.4} }
gsr set fao_range { {metal6 8 29.4} }
gsr set noise_reduction -15
gsr set fix_window {600 600}
gsr set num_hotspot 1
```

   By using the above GSR keyword values, we have defined a bound area with corner locations (2600,2677) and (6487,5304) for grid fixing. Totem FAO will

search for “hot-spots” within this area (in this run only one is being evaluated), perform an ‘accurate’ mode voltage drop analysis, and fix only the VDD net within the specified area. The recommended changes to the grid will be performed in the selected “fix window” with the hotspot at the center. Note that the grid will not be modified just within the fix window, but across the entire width and height of the chip with the specified x,y dimensions (as modified by the metal layer constraints). FAO will search all METAL6 wires running horizontally of width less than or equal to 29.4um in the “fix window”. It will fix the specified portion of the grid to relax the voltage drop by 15%. Since the specified operation will entail a reduction in metal usage, you must specify an analysis range for METAL6 width less than the existing width (in this case between 8 and 29.4 microns).

2. Run mesh fix as follows:

```
mesh fix -eco <filename>.eco
```

3. Based on the specified grid fixing parameters, Totem reports the wires to be fixed within the defined region, finding 1 hotspot, as shown in the following output to the screen and to the log file:

```
ECO File: fao.static.eco
Region: <2600 2677 6487 5304>
FAO Mode: accurate
Simulation Mode: static
Net: VDD
Noise Reduction: -15%
Mesh Style: uniform
Hot-spots Number: 1
Fix-Window Size: 600x600
++++++++++++++++++++++++++++++++++++
Initial Worst Static Noise: 17.578mv
Voltage Noise Constraints: 20.2147mv
Identified 1 hots-spots:
    (3123.26, 3058.42)
Fixing Window: <2823.26 2758.42 3423.26 3358.42>
*=========================================*
*          Searching Target Grids         *
*=========================================*
Net <VDD>:
    Layer <metal6>: identified 11 wires
*=========================================*
*           Sizing Target Grids           *
*=========================================*
Optimization succeed: no error
Optimized width of layer metal6: 17.9um
*=========================================================*
 Metal Usage Change Report
    Net <VDD>:
        Layer <metal6>: 5.05281e+06um^2 => 4.56558e+06um^2 (-9.64%)
    Total: 5.05281e+06um^2 => 4.56558e+06^2 (-9.64%)
*=========================================================*
Replacing optimized grids...
Replace <metal6> mesh grids :
    net <VDD> : 11 wires replaced.
DEF file for new via models: SH_VIA_VDD.def
```

The recommended fix involves 11 metal6 wire changes. Since the voltage drop was allowed to increase, there will be less metal usage in the wires identified for modification.

4. To check on the new static IR drop, execute the standard Totem IR analysis again:

```
perform extraction
setup pad
setup wirebond
setup package
perform analysis -static
```

The final static IR drop on the VDD network is 21mV (a 15% relaxation) with a 10% saving in metal resources.

Example D - Partial Chip Fixing, Reducing Static IR Drop

Conditions: Later in design development. Your design requires a lower voltage drop than 18mv, but you do not want to make changes over the entire grid. You want to specify a fix window, so **mesh fix** is used.

Goal: To perform a limited power grid fix to reduce the worst static IR drop 15% by modifying wire widths for only a selected area of the power grid.

1. Decide on what GSR keyword settings are necessary. This time we will tighten the voltage drop, evaluating the grid wire widths for a portion of the design. Since a tighter limit is specified for noise_reduction, the fao_range must be increased to allow wider metal strips. For example, if you want to tighten the voltage drop by 10%, the following GSR keyword settings should be used:

```
gsr set fao_region {2600 2677 6487 5304}
gsr set fao_turbo_mode 0
gsr set fao_nets VDD
gsr set fao_layers { {metal hor <= 29.4} }
gsr set fao_range { {metal6 10 40} }
gsr set noise_reduction 10
gsr set fix_window {1200 1200}
gsr set num_hotspot 1
```

As before, a limited part of the grid is specified for evaluation, and a fix window of 1200 by 1200 microns is specified around each hotspot. The range of grid width is widened to allow wider wires this time to achieve a noise reduction.

2. Run mesh fix as follows:

```
mesh fix -eco <filename>.eco
```

3. The screen and log file output summarizing the results from this run is shown below, and are shown graphically in Figure 9-9.

```
ECO File: drop_improv_15.eco
Region: {2600 2677 6487 5304}
FAO Mode: accurate
Simulation Mode: static
Nets: VDD
Noise Reduction: 10%
Mesh Style: uniform
Hot-spots Number: 1
Fix-Window Size: 1200x1200
.....................................
Initial Worst Static Noise(mv): 17.578
Voltage Noise Constraint(mv): 15.8202
```

```
Identified 1 hot-spots:
 (3123.26, 3058.42)
Fixing window: <2523.26 2458.42 3723.26 3658.42>
*=======================================*
*         Searching Target Wires        *
*=======================================*
Net <VDD>:
   Layer <metal6>: identified 17 wires
*=======================================*
*          Sizing Target Wires          *
*=======================================*
Optimization succeed:  no error.
Optimized width of layer metal6: 37.47um
*=========================================================*
 Metal Usage Change Report
    Net <VDD>:
        Layer <metal6>:5.05281e+06um^2 => 5.58121e+06um^2 (+10.46%)
 Total: 5.052814e+06um^2 => 5.581216e+06um^2 (+10.46%)
*=========================================================*
Replacing optimized grids...
Replace <metal6> mesh grids :
 net <VDD> : 17 wires replaced.
DEF file for new via models: SH_VIA_VDD.def
*********************************************************
****   FAO Verify Power Grids
*********************************************************

Resulting Voltage Noise(mv): 15.8578
```

Note that the target reduction in voltage drop was achieved by increasing the width of 17 metal6 wires by 10.46%.

4. As before, verify the new static IR drop by executing the standard Totem IR analysis:

```
perform extraction
setup pad
setup wirebond
setup package
perform analysis -static
```

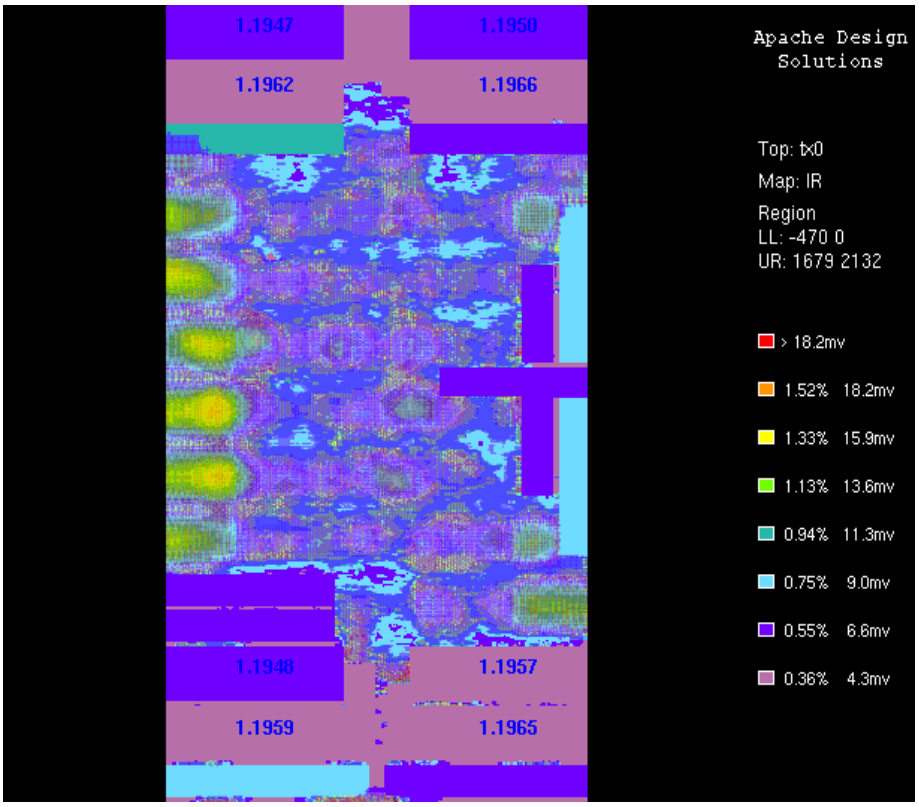


Figure 9-9 Static IR drop map after FAO operations

Example E - Mesh Optimize, -taper option, to reduce regional static IR drop

Conditions: Your design requires a lower voltage drop on two layers in a particular region, so **mesh optimize -taper** is used.

Goal: To perform a 5% static voltage drop reduction with changes on METAL3 and METAL4 layers in a defined FAO_REGION. We choose to limit the width of new wires to between 2 and 5 microns. The '-taper' option is used to ensure that the grid modifications are performed only within the 'fao_region' and not across the entire width and height of the chip.

1. Decide on what GSR keyword settings are necessary. With the conditions described above, the following scripted GSR keyword settings should be used:

```
gsr set fao_nets VDD
gsr set fao_drc 0
gsr set fao_region {x1 y1 x2 y2}
gsr set fao_layers { {METAL4} {METAL3} }
gsr set fao_range { { METAL4 2 5 } { METAL3 2 5 } }
gsr set noise_reduction 5
mesh optimize -taper
export eco mesh_optimize.eco
```

2. By executing the above script, FAO searches for all METAL4 and METAL3 wires in the defined fao_region that can be modified to achieve the voltage drop of 5%. The '-taper' option ensures that the grid fix is performed only within the 'fao_region' and not across the entire width and height of the chip.

3. The ECO changes from this procedure would be the deletion of a number of small wire segments within the defined fao_region on METAL3 and/or METAL4, and larger wires added in their place.

Example F - Mesh Fix, -taper option, to reduce hot spot static IR drop

Conditions: Your design requires a lower voltage drop for hot spots on two layers in a particular region, so **mesh fix -taper** is used.

Goal: To achieve a 2% static voltage drop reduction for hot spots in a defined FAO_REGION with changes on METAL3 and METAL4 layers. We choose to limit the width of new wires to between 2 and 5 microns. The '-taper' option is used here to ensure that the grid modifications are performed only within the 'fix_window' areas around each hot spot and not across the entire width and height of the chip.

1. Decide on what GSR keyword settings are necessary. With the conditions described above, the following scripted GSR keyword settings should be used:

```
gsr set fao_dynamic_mode 1
gsr set fao_nets VDD
gsr set FAO_DRC 0
gsr set fao_region { x1 y1 x2 y2 }
gsr set fao_layers { {METAL4} {METAL3} }
gsr set fao_range { { METAL4 2 10} { METAL3 2 10 } }
gsr set noise_reduction 2
gsr set fix_window { 600 600 }
gsr set num_hotspot 5
mesh fix -taper
export eco mesh_fix.eco
```

2. By using the above GSR keyword values, we have defined an FAO_REGION area with corner locations (x1,y1) and (x2,y2) for identifying hot spots and grid fixing. FAO searches for 5 hot-spots within this area and finds grid changes that can reduce their voltage drop by at least 2%.
3. The ECO changes from this procedure would be the deletion of a number of small wire segments within the specified 600x600 fix_window area around 5 identified hot spots, and larger wires added in their place.

Fixing and Optimization (FAO) for DvD and Timing in Digital P & R Blocks

Overview

An overview of the flow for using Totem to fix and optimize dynamic voltage drop and timing problems for digital place and route blocks is shown in Figure 9-10.

Note: This FAO feature is not yet available for custom design.

Three criteria for running FAO are available:

- instances having the worst-case dynamic voltage drop (DvD) (hot instances)
- all instances having the highest delta slack and delta delay caused by DvD
- instances in critical paths having the highest delta slack and delta delay caused by DvD

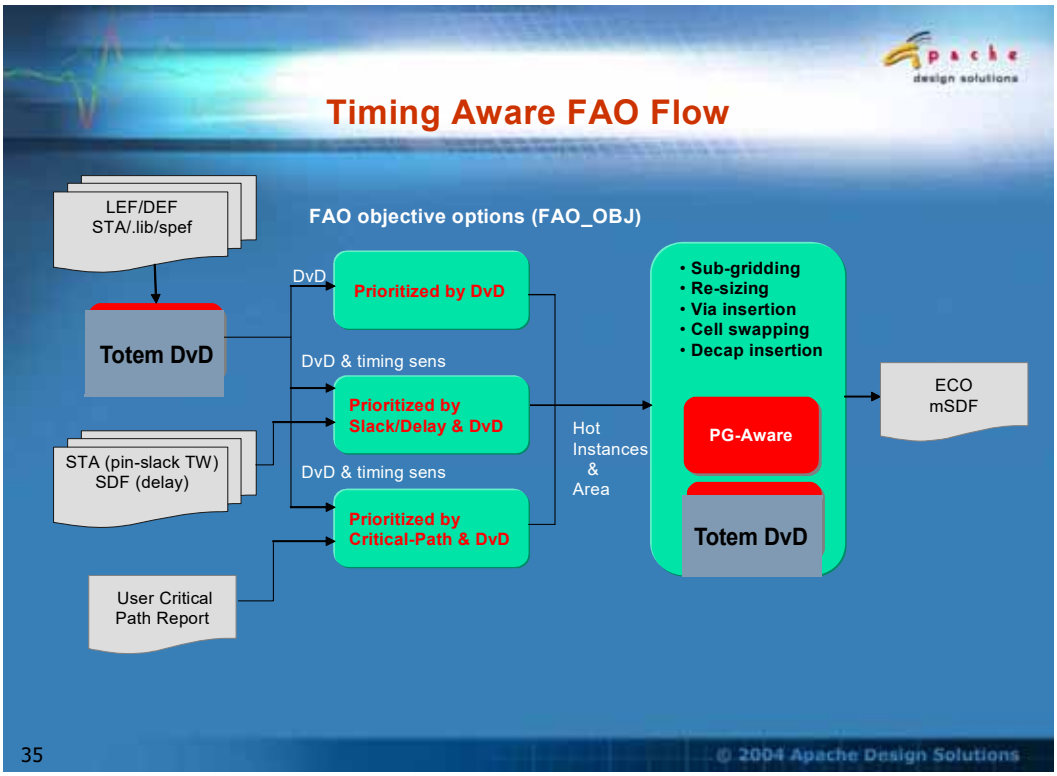


Figure 9-10 FAO Flow

Based on the specified value of the GSR keyword/FAO GSR keyword FAO_OBJ, the program creates an ordered list of instances that, for pure DvD runs have the worst case DvD values, or for the timing-based runs are most likely to be seriously impacted by DvD. The three types of FAO analysis are described below as invoked using the FAO_OBJ keyword:

FAO_OBJ

FAO keyword and GSR keyword that selects the type of instance prioritization and results desired. *Optional; default: DVD*

Syntax

```
FAO_OBJ [ DVD | TIMING ?<slack threshold>? |
    PATH ?<slack threshold>? ]
```

where

DVD : instances with highest voltage drop are selected for FAO

TIMING <slack_threshold>: all instances in the design with high delta delay and delta slack values caused by DvD are selected for FAO. The default value of the slack threshold is 0.
Note: To use FAO_OBJ TIMING you must execute '`import sdf xxx.sdf`' before cell swapping, or the candidates for swapping will be zero. Also, fao_region is still honored to filter out candidates from critical path or timing/slack info.

PATH <slack_worse than> : instances in critical timing paths with high delta delay and delta slack values caused by DvD are selected for FAO. The default value of the slack threshold is 0.

The target instances to be investigated and fixed can be further filtered using parameters such as the value of DvD, the total number of instances, or a special region of interest, as described in the following sections.

Preparation for Decap FAO

Prior to running decap FAO, verify that the available decap cells are properly prepared as follows:

1. Define the list of decaps in the GSR file, using the option DECAP_CELL:

   ```
   DECAP_CELL
   {
   <decap_cellname> ? <width_um> <height_um> <C_pF> <R_ohm>
   <metal_layer_name> ?
   ...
   }
   ```

   Usually only the list of decap cell names is required, since the other information is in the LEF files.

2. Run APL characterization for decaps in Totem, which produces the *<cell>.cdev* file containing the R, C and/or leakage power definitions.

3. Insure that the proper physical information is defined in the LEF file and included in LEFS with power/ground pins defined for the layer.

   ```
   MACRO cell_name
       CLASS <class_type> ;
       ORIGIN <value> ;
       SIZE <value> BY <value> ;

       PIN GND
       USE GROUND ;
       PORT
           LAYER <layer_name> ;
           RECT <x1> <y1> <x2> <y2> ;
       END
       END GND

       PIN VDD
       USE POWER ;
       PORT
           LAYER <layer_name> ;
           RECT <x1> <y1> <x2> <y2> ;
       END
       END VDD
   END cell_name
   ```

Decap Modification Operations

Totem FAO also allows you to investigate the use of decoupling capacitance to fix dynamic voltage drop and timing problems.

Decoupling Capacitance Modification Commands

The key commands for fixing decoupling capacitance can be invoked in TCL command line or from the GUI. The TCL commands have the following functionality:

- decap advise - finds the total amount of decap that can be placed very quickly, without providing specific placements. The '-place' option uses an algorithm to determine the largest amount of decap that fits in the available row width; saves the result in a user-specified decap placement file (*dpf*). For MMX designs with pure GDS custom blocks, use the '-mmx' option.
- decap fill - executes '`decap place`' using one of several methods:
  - targeted - within a defined rectangle around each specified hot instance (DvD)
  - hot instance area - within a rectangular area encompassing a specified set of hot instances
  - region - within a specified area
  - prefill - adjacent to hot instances after making space in the row
- decap remove - removes all ineffective inserted decaps within a specified area, as defined by a specified peak current (default: 30 uA or less).

Additional information on the functions, syntax and usage of these commands is provided in the FAO command reference at the end of the chapter.

Following is a brief summary of GSR keywords available to control the operation of FAO to investigate modifying decoupling capacitance. The general TCL syntax for setting GSR keyword values is: '`gsr set <variable name> <value>`'.

A detailed description of these GSR keywords and their options and syntax is provided in section "Global System Requirements File (*.gsr)", page C-772.

Decap Modification Constraints and Interfaces with Other Programs

- Commands 'decap advise -place', 'decap fill' and 'cell swap' honor the DEF file option '+FIXED' so that no modifications are made to fixed instances.
- Commands 'decap advise -place', 'decap fill' and 'cell swap' honor DEF placement blockage if the GSR keyword 'FAO_DRC_PL_OBS' is set to 1 (On).
- In case decaps include metal1 structures, decap placement detects DRC check against metal1 signal routes with GSR keyword 'FAO_DRC_OBS' set to 1 (On). If set to 0 (Off), there is no DRC metal1 signal route check.
- Decaps listed in GSR are not moved during cell swap if designated with option '-fixed_inst/cell' in cell swap command.
- Cells with tie high/low connections are not moved unless the '-include_thtl' option is used (but can move prior to routing).

GSR Keywords Controlling Decap Modification

The following GSR keywords can be used to control the operation of FAO commands `decap advise`, `decap advise -place`, and `decap fill`.

1. To select the target voltage drop reduction: noise_reduction (input required)
2. To specify the percent improvement in voltage drop: noise_limit (no default)
3. To select a fixing area: fao_region (default - whole chip)
4. To select the area around a hot instance to be fixed: fix_window (default - 40 X 40 u)
5. To select the number of the highest voltage drop areas to fix: num_hotinst (default - 10)

6. To limit the total decoupling capacitance added: cap_limit (default - infinity)
7. To limit the power leakage from all added decaps: leakage_limit (default - infinity)

The following GSR keywords can only be used for '`decap advise -place`':

8. To allow decap placement in illegal locations where there is no available space: fao_decap_overlap (default - no overlap)
9. To specify placement of a multiple of each legal decap for placement: decap_density (default - 1)

Other GSR keywords are available that are specific to individual commands. See the command reference at the end of the chapter.

Additional Tools for Fixing High Voltage Drop Areas

Supplemental Power Routing with 'route fix'

In some situations a high voltage drop node can be repaired by routing a supplementary power wire from it to a nearby low voltage drop node on the same net, providing a parallel path to equalize the voltage drop between the two nodes. This can be performed using the 'route fix' command, which uses a shape-based point-to-point router. It is invoked by the following syntax:

```
route fix [-from { x1 y1 x2 y2 <layer>} | -from_pt { xf yf <layer>} ]
[-to {x1 y1 x2 y2 <layer>} | -to_pt { xt yt <layer>} ] -width <float>
```

This command finds the shortest DRC-clean legal route for new power/ground wires and vias from a specified target "from" point (or area) and layer to another specified target "to" point (or area) and layer. The "export eco" command defines the added wires/vias for the design database. If the existing via models are not correct for the new routing, any new vias required will be specified and documented in the file 'adsRpt/faoVIAS.def'. The width of the new routing is specified by the value of option '-width'.

This command is used to improve power supply wiring from a stronger point to a weaker point (in terms of voltage drop), as identified from PG-Aware reports.

Cell Moving or Cell Swapping of "Hot Instances"

Some high voltage drop areas can be mitigated by moving a "hot instance" cell to a nearby node on the same grid with better voltage drop conditions, redistributing and equalizing current demand on the chip.

After running Totem dynamic analysis, the FAO process for moving or swapping instances to improve DvD or timing is as follows:

1. Select the analysis criteria by setting the value of FAO_OBJ keyword:
   e. DVD - evaluate all instances that have significant DvD impacts
   f. TIMING - evaluate all instances that have significant slack and delay impacts from DvD
   g. PATH - evaluate critical paths instances that have significant slack and delay impacts from DvD
2. Select fix_window size - the area around each hot instance to investigate swapping cells (default: 40 X 40 microns)
3. Select DvD criteria desired setting 'cell swap' command options:
   -eff_vdd_tw : use average effective voltage (Vdd-Vss) over the timing window to define swapping targets (default DvD criteria)
   -max_vdd_tw : use maximum effective voltage (Vdd-Vss) over the timing window to define swapping targets

-min_vdd_tw : use minimum effective voltage (Vdd-Vss) over the timing window to define swapping targets

-min_vdd_all : use minimum effective voltage (Vdd-Vss) over clock cycle to define swapping targets

4. Eliminate any cells from swapping by declaring them “fixed”, or define an exclusive list of cells or instances to swap, using ‘cell swap’ options.
5. Invoke ‘cell swap’ using the command syntax at the end of this procedure.
6. FAO generates a list of candidate hot instances to attempt to swap based on the selected criteria and instance constraints.
7. For each hot instance in the candidate list, FAO first attempts to move it to all open locations in the fix_window rectangle surrounding the instance. If the criteria are improved by moving any candidate cells, it makes the change.
8. Next FAO attempts to swap the candidate with all other legal instances in the fix_window. If the selected performance criteria is improved by any attempted swaps, it makes the change.
9. FAO iterates through all swap candidates on the specified instance list.
10. An ECO report of instances swapped is automatically prepared.
11. Run the -report and/or -plot options to view the swapping results.

The detailed syntax and usage for the ‘cell swap’ command is as follows:

```
cell swap ?-eco <file_name>?
    ?-decap_cells {cell1 cell2 ...}?
    ?-fixed_cells {cell1 cell2 ...}?
    ?-fixed_insts {inst1 inst2 ...}?
    ?-removable_cells {cell1 cell2 ...}?
    ?-glob|-regexp|-exact?
    ?-include_clock ?
    ?-inst_list {inst1 inst2 ...}?
    ?-inst_file <file_name>?
    ?-search_only?
    ?-eff_vdd_tw? ?-max_vdd_tw?
    ?-min_vdd_tw? ?-min_vdd_all?
    ?-report? ?-plot?
```

where

-eco: specifies the filename of the ECO file describing the changes to the design

-decap_cells: : specifies the names or families of master decap cells that can be used in cell swapping with hot instances. Default: any cells with no logic connection can be used for swapping. Three options define how the cells are specified:

-glob: allows global wildcard symbols (such as *) to be used to define families of decap cells allowed for swapping

-regexp: allows regular expressions to be used to define categories of decap cells allowed for swapping

-exact: supplies the specific names of decap cells allowed for swapping

-fixed_cells {cell1 cell2 ...} : specifies the names of fixed master cells *not allowed* for swapping

-fixed_insts {inst1 inst2 ...} : specifies the names of fixed instances *not allowed* for swapping

-removable_cells {cell1 cell2 ...} : specifies the names or families of decap cells that can be removed from the design, if necessary, during cell swapping. Default: no decap cells can be removed. The -glob, -regexp, and -exact sub-options are also available for this option, with the same usage as for the -decap cell option.

-include_clock : allows clock elements to be included in cell swapping

-inst_list {inst1 inst2 ...} : specifies exclusive list of instance names for cell swapping

-inst_file <file_name> : specifies file containing exclusive list of instance names for cell swapping

-search_only : only generates file with list of candidate instances for swapping based on selection criteria; no swapping is performed (default file name: *cell_swap_<time>.list.* If you want to change the name, use the '-inst_file <filename>' option to specify.)

-eff_vdd_tw : use average effective voltage (Vdd-Vss) over the timing window to define swapping targets (default DvD criteria)

-max_vdd_tw : use maximum effective voltage (Vdd-Vss) over the timing window to define swapping targets

-min_vdd_tw : use minimum effective voltage (Vdd-Vss) over the timing window to define swapping targets

-min_vdd_all : use minimum effective voltage (Vdd-Vss) over clock cycle to define swapping targets

-report : generates text table of last cell swapping result *cell_swap.rpt*, showing instances swapped, and the change in voltage drop (see Figure 9-11)

-plot : generates histogram of last cell swapping result in file *cell_swap.hist*, showing the number of instances vs. the change in criteria Vdd value selected.

```
*******************************************************************************
****   FAO Cell Swapping Report
*******************************************************************************
Cell Swapping Result Summary
  Total swapping instance number  : 2182
  Actually Moved instance number  : 436
  Actually Deleted instance number: 0
  maximum voltage change (mv)     : 171
  minimum voltage change (mv)     : -18
**** Please see adsRpt/cell_swap.rpt for detail DvD change.
**** To view histogram of DvD change, please execute xgraph
adsRpt/cell_swap.hist.

MEMORY USAGE: 2265 MBytes
TOTAL CPU TIME: 1 hrs 17 mins 52 secs
WALLTIME: 1 hrs 27 mins 56 secs.
*******************************************************************************
```

Figure 9-11 Sample Cell Swapping Report

There are several GSR keywords that specify how the cell swap command operates:

fao_region: specifies the target analysis area containing high power instances to be fixed

fao_nets: when there are multiple VDD nets in the design, specifies the name of the VDD net with which the hot instances are associated

noise_limit: for the list of worst case hot instances, the noise_limit specifies the voltage drop threshold below which the hot instance will not be considered for swapping. For example, if noise_limit is set at 5%, hot instances with less than a 5% voltage drop would not be considered for swapping.

num_hotinst: specifies the number of worst case hot instances to be swapped in the region

fix_window: specifies the size of the area centered on each identified hotinst within which the cell would be moved or swapped (default: 40 x 40 u)

fao_verbose: (0/1) controls the volume of output messages. (default: 0, small amount)

Examples of FAO for DvD

Several examples of dynamic voltage drop improvement using FAO are provided in the following section.

Example G - Full Chip DvD Reduction - Non-overlap Decap and Grid Fix

Conditions: Early in design development. For this example, the initial worst case dynamic voltage drop (DvD) has been evaluated at 306mv by Totem. Your design requires a voltage drop at least 10% lower.

Goal: Perform 'decap advise -place' over the full design, attempting to reduce DvD by 10% (to 276mV) by adding decoupling capacitance. If the target is not met with decap additions, follow up with a grid fixing run.

1. Decide on the GSR keyword settings to use. For this run, use all default settings for decap insertion, except specify a noise reduction of 10%, using the command:
   ```
   gsr set noise_reduction 10
   ```
2. Run decap placement as follows:
   ```
   decap advise -place
   ```
3. During the evaluation, FAO reports the following:

```
|Target Worst Average DvD percent over TW: 24.1895%(0.27576)(v.s. Vdd=1.14)
```

   This report shows the that the target DvD reduction of 276mv would be 24.2% of Vdd. The measured DvD is the worst case average over the timing window (TW). (The timing window is the range of time in which a particular gate could be switching, according to a Static Timing Analysis (STA) tool).

4. After completing the evaluation, FAO reports the following decap changes:
   ```
   Placed decap cells for 29 hot instances.
   Added decap 5.092 pF
   ```
   FAO then automatically performs another dynamic analysis to determine how close to the target the result is with the initial decap changes, and adds more capacitance for the hot instances if needed. It then reports the total decap added in the second run, so the total decap added in the two placements is now 209.8pf.
   ```
   Placed decap cells for 29 hot instances.
   Added decap 204.718 pF
   ```

5. Now rerun Totem extraction and dynamic analysis as follows, to check the improvement achieved:

```
perform extraction -power -ground -c
setup pad -power -r 0.005 -ground -r 0.005
setup wirebond -power -r 0.001 -l 1000 -ground -r 0.001 -l 1000
setup package -r 0.001 -l 250 -c 5
perform analysis -vectorless
```

   In this case the dynamic voltage drop has improved to 284mV, with a target value of 276mV.

6. Obtain a report of the decaps added and the improvement achieved using the 'decap report' command. Then export the required changes into an ECO file, as follows:

```
decap report
export eco decap.1.eco
```

   The decap report for the operation follows:

```
************************************************************
****   FAO Decap Report on top Hot Instances
************************************************************
Inserted Decap Inst Num : 3579
Inserted Decap Total    : 209.81 pF
V0/V1 : Average (VDD - GND) over TW before/after decap placement.
DvD0/DvD1 : Dynamic Voltage Drop (VDD - V0/V1) before/after decap
placement. Improvement % : (DvD0 - DvD1)/DvD0 %.
inst    V0    V1    DvD0    DvD1    Improvement%
-------------------------------------------------
INFO(FAO-75): inst194510 : 0.8343 0.8568 0.3057 0.2832 7.36017%
INFO(FAO-75): inst117100 : 0.8346 0.8574 0.3054 0.2826 7.46563%
INFO(FAO-75): inst76925 : 0.8395 0.8607 0.3005 0.2793 7.05491%
-------------------------------------
New Top Hot Instances:
inst    &   avgVoltage_TW
-------------------------------------
inst194510 0.8568
inst117100 0.8574
inst76925 0.8607
-------------------------------------
Overall DvD improvement = 7.3601 % v.s. initial DvD
MEMORY USAGE: 622 MBytes

************************************************************
```

7. In the ECO file, filter the list of instance names of the added decaps (those with a decap* prefix), then use the Totem command:

```
select add "decap*" -glob -linewidth 3
```

   to highlight them for viewing on screen.

8. Using the 'decap report' (above) showing a DvD improvement of 7.36%, assign the remaining improvement needed (2.64%) to the next step, wire fixing. Set the new target voltage drop as follows:

```
gsr set noise_reduction 2.64
```

9. Now on the target layers run ‘`mesh optimize`’ (if you want to look at the entire grid), or ‘`mesh fix`’, with a specified ‘fix window’ if you want a localized fix, and see if that achieves the desired voltage drop goal.
10. As in the previous examples, rerun Totem dynamic simulation to check on your results.

Example H - Full Chip DvD Reduction - Decap Overlap

Conditions: Early in design development. For this example, the initial worst case dynamic voltage drop (DvD) is again found to be 306mv by Totem. Your design requires a 10% lower voltage drop. This time we are going to search for a solution in which decaps can be placed anywhere, whether there is legal space or not.

Goal: Perform decap placement over the full design, attempting to reduce DvD by 10% to 276mV by adding decoupling capacitance.

1. Decide on the GSR keyword settings to use. Using the same Totem design inputs, this time use the 'overlap true' GSR keyword setting for ‘`decap advise -place`’. This allows the needed decaps to be inserted both legally and also where there is no placement space presently available, to attempt to reduce the local DvD. Set GSR keyword values as follows to request a 10 % reduction in DvD and allow decaps to be placed in illegal areas:

```
gsr set fao_decap_overlap 1
gsr set noise_reduction 10
```

2. As in previous example, verify the amount of decap added and the final DvD savings achieved. A sample output report is shown below:

```
*********************************************************
****   FAO Decap Report on top Hot Instances
*********************************************************
Inserted Decap Inst Num : 305
Inserted Decap Total    : 229.938 pf
V0/V1 : Average (VDD - GND) over TW before/after decap fix.
DvD0/DvD1:Dynamic Voltage Drop (VDD - V0/V1) before/after decap fix.
Improvement % : (DvD0 - DvD1)/DvD0 %.
    inst               V0     V1 DvD0 DvD1 Improvement%
-------------------------------------------------------
INFO(FAO-75): inst117100 : 0.8346 0.8573 0.3054 0.2827 7.43288%
INFO(FAO-75): inst194510 : 0.8344 0.8609 0.3056 0.2791 8.67146%
INFO(FAO-75): inst76925 : 0.8396 0.8623 0.3004 0.2777 7.55658%
-------------------------------------------------------
New Top Hot Instances:
inst    &   avgVoltage_TW
------------------------------
inst117100 0.8573
inst194510 0.8609
inst76925 0.8623
------------------------------

Overall DvD improvement = 7.73499 % v.s. initial DvD
```

Reviewing and Implementing ECOs in a layout editor

Once a voltage drop target is met, the ECO file created by the mesh and decap placement operations can be read into Totem to review and implement the ECOs. You can set up a cross-probing connection to a 3rd-party layout editor and use the list of ECOs available in the GUI to go through the ECOs one-by-one, as follows:

1. Set up cross-probing to the layout editor.
2. Select **Results -> List of ECOs** to bring up a dialog box listing all ECO items, as shown in Figure 9-12.
3. Click on **List of ECOs** dialog. The geometry bounding box is highlighted in the 3rd party layout editor.
4. Make the listed modifications in the layout editor.

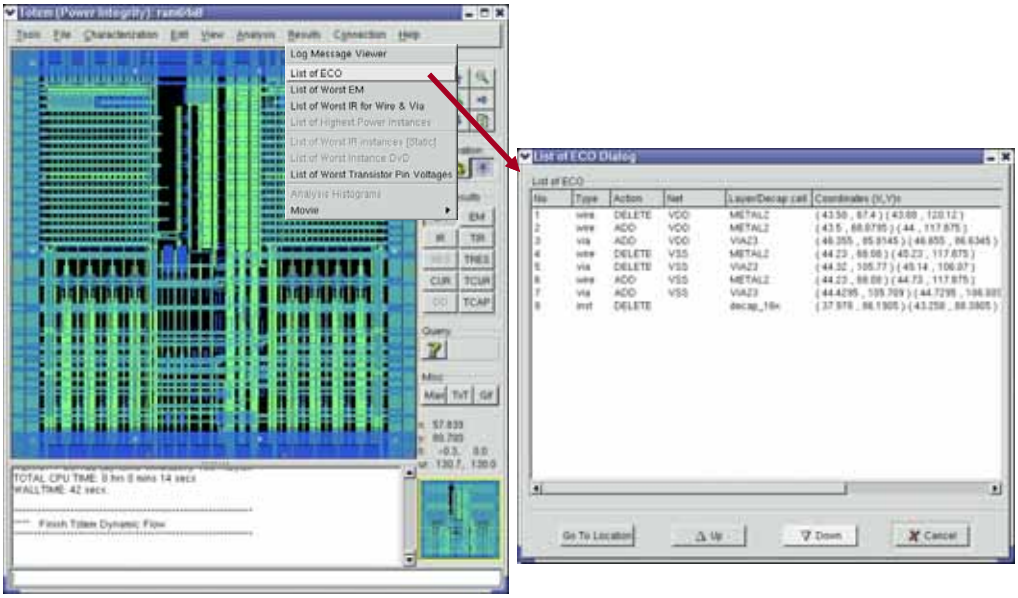


Figure 9-12 Reviewing the List of ECOs

Saving Design Changes with the ECO Command

Writing an ECO File

The command **File > Export ECO** exports all the recent design changes to an ECO file in the Totem working directory. When selected, the pop-up form will query you for a file name, otherwise use the default name *apache.eco.* The format of the ECO file is described below. For the TCL invocation of the ‘export eco’ command, see section "TCL Command Summary", page D-923.

Reading an ECO File

The ECO file can be imported by using the **File > Import ECO** command after the design is read in with the **File > Import Design Data** command. Database setup must be completed also. When this is performed, all what-if changes as specified in the ECO file or *apache.eco* will be included in the design during IR drop and EM analysis.

ECO File Format Definition

NOTE: The ECO file syntax is described below to help you understand the file contents, which are generated automatically by Totem. Do not try to create or edit an ECO file manually.

The following is an example of a tabular **.eco* file created by the **File -> Export ECO** command:

Keyword	Obj	Obj name (uniq ID)	P/G name	Layer/ via	Rotation	Coordinates	Direction
add	pad	pvdd1	Vdd	metal1	None	x_new y_new (center)	None
delete	pad	pvdd2	Gnd	metal1	None	x_old y_old (center)	None
add	via	via67_new	Vdd1	via7_1	<code>	x_new y_new (center)	None
delete	via	via67_old	Vdd1	via7_2	None	x_old y_old (center)	None
add	wire	wire1	Vdd2	metal7	None	x1 y1 (lower left) x2 y2 (upper right)	[horizontal | vertical]
add	wire45	wireCD	Vdd	metal7	None	x3 y3 x4 y4 x5 y5 x6 y6 (trapezoid corners)	None
delete	wire	wire2	Vdd	metal7	None	x1 y1 (lower left) x2 y2 (upper right)	None

The ECO file format description, by column, is as follows:

Column 1: ‘Keyword’ represents the type of ECO change:

add - a new object added to database

delete - an existing object deleted from database

Column 2: ‘Obj ’ represents the type of object that was changed:

pad - a pad object

via - a via object

wire - a horizontal or vertical wire object

wire45 - a wire object, not on grid

Column 3: ‘Obj name (uniq ID)’ represents the unique name of the object.

Column 4: ‘P/G Name’, for pads is ‘VDD’ or ‘GND’, and for wires and vias is the net name

Column 5: ‘layer/via’ is the name of metal layer or via defined in LEF:

metal1 - metal layers defined in LEF

via7_1 - via definition defined in LEF

Column 6 : represents the coordinates of the object being changed by ECO, except for ‘add via’:

For pads, indicates the x,y coordinates of the center of the power/ground pad being added or deleted

For ‘add via’, indicates the rotation code associated with the via added, as follows:

-1 No rotation

0-6 Totem rotation code

For ‘delete via’, indicates the center of the via to be deleted.

For wires, indicates the lower left and upper right x,y coordinates of the bounding box for the wire being added or deleted. Note that you can delete a non-grid wire segment with the command ‘delete wire’.

For wire45 objects, x3 y3, x4 y4, x5 y5, and x6 y6 represent the four corners of the trapezoidal wire segment to be added. Any corner can be the first corner specified, but the corners must be listed in counterclockwise order.

Column 7: For ‘add via’, indicates the x,y coordinates of the center of the via to be added. This via and its size/layer must have already been defined in LEF.

Column 8: For ‘add wire’, indicates the direction--vertical or horizontal-- of the wire to be added.

Note: ECO added wires are not merged by Totem, so if an added wire overlaps another wire on the same net and on the same layer, the capacitance extracted for the overlap portion of the two wires will be double counted (too large), and the resistance for the overlap portion will be that much smaller due to the double counting. For this reason, the amount of overlap should be minimized.

Fixing and Optimization Command Reference

Descriptions of Mesh Optimization Commands

The following section describes the commands available for performing grid optimization and fixing in Totem FAO (alphabetic order). The GSR keywords associated with each command are described in the next section. See the last section for syntax conventions.

Mesh and ring commands - FAO functions to optimize and fix grids.

```
mesh [ add| delete| fix| generate| optimize| snscalc |
    sub_grid | set_width | vias ] ? args ?

ring add ? args ?

ring delete <ring_name>
```

Table 9-1 Grid Optimization and Fixing Commands

Command	Description
mesh add	Allows you to add a specified mesh to the power grid.
	Associated GSR keywords: none
	Options
	[-vertical | -horizontal] Select either horizontal or vertical direction for adding mesh.
	-window { <x1> <y1> <x2> <y2> } Add mesh in rectangular area specified by corner coordinates ‘x1’,’y1’, and ‘x2’,’y2’
	-layer <layer_name> Add mesh layer named ‘layer_name’.
	-offset <offset_value> Offset new layer by ‘offset_value’ microns from the left edge of the selected window for vertical addtion, or from the bottom edge of the selected window for horizontal addition.
	-space <space_amount> Add ‘space_amount’ in microns between the power nets specified in the -net option.
	-pitch <pitch> Add mesh with pitch of ‘pitch’ microns.
	-width <mesh_width> Add mesh with width of ‘mesh_width’ microns.
	[-novia | -viatop <metal_layer_name_above> -viabottom <metal_layer_name_below>] Add mesh either without a via or with vias going from ‘metal_layer_name_below’ up to ‘metal_layer_name_above’
	-net { <net1> <net2> ... <netn>} Add mesh associated with net names ‘net1’ to ‘netn’
	--exclude_regions {{x1 y1 x2 y2} {<exclude_region2_coords>} ... } Excludes specified regions for mesh additions/deletions. Note that to specify only one region, double braces should still be used -- for example, {{ <llx> <lly> <urx> <ury> }}
mesh snscalc	Calculates a number representing the relative voltage drop sensitivity (in mv/um) for changes to wire width for the selected layers and nets within the selected region.
	Associated GSR keywords: fao_region, fao_nets, fao_layers
	Options
	? -taper ? Resizes grid wires on target layers only within the specified region (fao_region), not the full height or width of the chip.

Table 9-1 Grid Optimization and Fixing Commands

mesh delete	Allows you to delete specified parts of the power grid.
	Associated GSR keywords: None
	Options
	[-vertical | -horizontal] Delete mesh in either horizontal or vertical direction.
	-window { <x1> <y1> <x2> <y2> } Delete mesh in rectangular area specified by corner coordinates ‘x1’,’y1’, and ‘x2’,’y2’
	-layer <layer_name> Delete mesh layer named ‘layer_name’.
	-net { <net1> <net2> ... <netn>} Delete mesh associated with net names ‘net1’ to ‘netn’.
	-width { ?<Oper>? <width-1> ?<Oper> <width-2>? ? <width>? } Defines grids to be deleted by function of width. <Oper> is a relational operator of the group: =, >, <, >=, or <= . “Width” is defined orthogonal to the wire direction
	-length { ?<Oper>? <length-1> ?<Oper> <length-2>? ?<length> ?} Defines grids to be deleted by function of length. <Oper> is a relational operator of the group: =, >, <, >=, or <= . “Length” is defined in the wire direction.
	-out_of_ratio { n m } Defines partial grid wire deletion, or how many grid wires, “n wires out of every m wires”, are to be deleted.
	-all Specifies that all available mesh object parameters are deleted except as specifically listed in the ‘mesh delete’ command.
	-exclude_regions {x1 y1 x2 y2} {excluded_region2_coords} ... Specifies one or more regions to be excluded from the ‘mesh delete’ command specified, where x1, y1, x2, y2 are lower left and upper right coordinates for the region to be excluded.

Command	Description
mesh fix	Allows you to perform hot-spot based grid width fixing.
	Associated GSR keywords: fao_region, mesh_search, fao_layers, fao_turbo_mode, fao_dynamic_mode, fao_nets, fao_range, fao_width_cnstr, noise_reduction, fix_window, num_hotspot
	Options
	? -taper Resizes grid wires on target layers only within the specified region (FAO_REGION), or FIX_WINDOW if defined, not the full height or width of the chip.
	? -eco <eco_filename>? Prepare ECO report file ‘eco_filename’ in Totem ECO text format.

mesh optimize	Allows you to investigate and fix voltage drop problems using uniform grid modifications on selected parts of the grid system.
	Associated GSR keywords: fao_region, mesh_search, fao_layers, fao_turbo_mode, sim_mode, fao_nets, fao_range, fao_width_cnstr, noise_reduction, num_hotspot
	Options
	? -taper Resizes grid wires on target layers only within the specified region (FAO_REGION), not the full height or width of the chip.
	?-eco <eco_filename> Prepare ECO report file ‘eco_filename’ in Totem ECO text format.
mesh sub_grid	Adds a set of subgrids defined in a region defined by 'fao_region', nets defined 'fao_sub_grid_nets', and with layers, pitch and spacing defined by 'fao_sub_grid_spec'. The program sizes the new subgrid to meet a preset voltage drop target 'noise_reduction' . The min/max width of the new grids considered is specified by 'fao_range'. The minimum width is also used to create the initial subgrid before adjusting the size.
	Associated GSR keywords: fao_region, fao_sub_grid_nets, fao_sub_grid_spec, fao_range, noise_reduction, fao_turbo_mode, fao_dynamic_mode
	Options
	?-eco <eco_filename> Prepare ECO report file ‘eco_filename’ in Totem ECO text format.
mesh set_width	Modifies the mesh wire width on specified layers.
	Associated GSR keywords: fao_region, fao_sub_grid_nets, fao_layers
	Options
	{ { <layer1> <new_mesh_width1> ? <width_change_dir1> ?} ... { <layerN> <new_mesh_widthN> ? <width_change_dirN> ?} } where <layerN> Specifies name of layer to have mesh width changed.
	<new_mesh_widthN> Specifies new mesh wire width for <layerN> in microns.
	? <width_change_dirN> ? Specifies direction of width change, if not equal on both sides of centerline (default). Direction values are [left | right | top | bot].

mesh vias	Adds specified optimal-sized stacked vias to improve voltage drop-- that is, adds the via with the largest possible dimension in the rectangular region overlapping the upper and lower metal layers.
	Associated GSR keywords: none
	Options
	-toplayer <top_layer_name> Specifies top layer connection of vias to be added.
	-bottomlayer <bottom_layer_name> Specifies bottom layer connection of vias to be added.
	[-window { <x1> <y1> <x2> <y2>} | -inst {< inst1> <inst2> ... }] Specifies corner coordinates ‘x1’,’y1’ and ‘x2’,’y2’ for window in which to add stacked vias, *or* specifes windows determined by the locations of cell instances ‘inst1’ to ’instn’.
	?-net { <n1> <<n2> ... }? Specifies nets ‘n1’ to ‘nn’ for which to add stacked vias.
	?-topwidth {<tw1> <tw2> ... }? ?-bottomwidth { <bw1> <bw2> ... }? Defines the top widths ‘tw1’ to ‘twn’ of the existing top layer and bottom widths ‘bw1 to ‘bw2’ for which stacked vias will be added in the overlapped area.
	?-viamodel { <m1> <m2> ... }? ?-replace? Specifies viamodels ‘m1 to ‘mn’ to be used.
	?-between_layer {<layer1> <layer2>} Specifies a via to be connected between defined layers. The -toplayer and -bottomlayer options having a different direction are used to define the location of the new via using their intersections with <layer1>/<layer2>. No via connections are made outside of the <layer1> to <layer2> range in this usage.
	?-eeq_net {<net1> <net2>} Specifies that a via be dropped between the wires of the specified top and bottom layers, even though they have different names.
	? -pt_file <filename> Specifies a file listing potential via x,y coordinates. The file format is '<x_coord> <y_coord>', with one entry per line, or when using a script such as *report_missing_vias.tcl,* entries may use either the form '<x_coord> <y_coord>', or the command form 'marker add -position <x_coord> <y_coord>'.
	?-delete -window { x1 y1 x2 y2} -viamodel {Via1 Via2 ...} -cut_layer <layer> Specifies vias to be deleted in a specified rectangle, viamodel, layer, or viamodel. If you do not define -window, the FAO_REGION GSR defines the window. If FAO_REGION is not defined, the whole chip is used. If -viamodel is not defined, all viamodels in the box are deleted. If neither -window nor -viamodel are defined, then all types of vias in FAO_REGION, or on whole chip are deleted. Viamodels are defined in Tech LEF or in top DEF.
	? -viarule { <viarule1> <viarule2> ... | ALL } For <viarule1> <viarule2>, Totem selects the best from the named viarules, and for ALL, Totem selects the best viarule from all the viarules in the LEF/Tech files.

mesh vias (cont.)	Manipulates and reports on specified stacked vias to improve voltage drop.
	Associated GSR keywords: FAO_LAYERS, FAO_LAYERS, FAO_NETS.
	Options
	?-report_missing ? -exclude_stack_via ? ? '-check_valid_vias? : Reports if metal overlap can accept a valid via model. Note this option does not work with '-min_width' and 'FAO_LAYERS' GSR. All other switches and keywords are supported. ? -exclude_cell {<cells>}/-exclude_inst {<instances>}: excludes area covered by all instances of a given cell, or an area covered by a specified list of instances from the missing via report. ? [-inst {<inst1> ...} | -cell { <cell1> ...} | -window { x1 y1 x2 y2 } ? ? -sort_by_hot_inst ? ? -toplayer <layer1> -bottomlayer <layer2> ? ? -gds ? ? -threshold [-1 | <voltage_V>] ? ? -min_width <value>? ? -fao_layers { {layer1 [h|v]} {layer2 [h|v] } ...}? ?-marker ? ? -pitch <via_center-center distance> ? -from_edge [0|1] ? ? -ignore_inter_met : Specifies missing vias to report, by default has stacked vias. Supports FAO_REGION and 45-degree wires. If the '-exclude_stack_via' option is used, stack vias are excluded. Using -inst, -cell or '-window' options reports only missing vias of specified inst/cell type, or overlaps of defined window boundaries. Default is full chip. Using the '-sort_by_hot_inst' option sorts missing vias by voltage drop, where default sorting is by average delta V between defined layers. The -gds option flags missing via locations both inside the GDS block region and in routes over GDS blocks. By default missing via check ignores all items inside or overlapping the area of a GDS2DEF/GDSMMX block. To flag all missing vias within and overlapping the gds*-based cell region, the -gds option is required. '-threshold < >' sets the voltage threshold between overlapped geometries that trigger a missing via report. If the option is set to '-1' the voltage check is not used as a filtering criteria-- for example, if dynamic analysis has not been performed and there are no voltage drop values. The default threshold is 1mV. -min_width ignores segments with less than specified width. -fao_layers <layers...> specifies certain layers and their horiz/vertical orientation for which missing vias are reported. -marker specifies the missing via is highlighted in the GUI -pitch <via center-center dist> specifies the pitch for missing vias on parallel wires crossing or touching GDS blocks. A pitch may start from either left-edge or right-edge for a horizontal overlap (that is, width > height), or a pitch may start from either bottom-edge or top-edge for a vertical overlap. Use option '-from_edge [0|1]' to specify this, using '-from_edge 0' for parallel nets from left/bottom edge and '-from_edge 1' for parallel nets from right/top edge. -ignore_inter_met ignores wires on all layers between the specified -toplayer < tl> and -bottomlayer <bl>.

Table 9-1 Grid Optimization and Fixing Commands

Command	Description
ring add	Adds specified power/ground rings to design.
	Associated GSR keywords: None
	Options
	?-name <ring_name>? ?-window { x1 y1 x2 y2 } ? { -[top| bottom| left| right] -layer <layer_name> -offset <offset> -width <width> -space <space> -net <net1 net2 ...netN> } Specifies location and dimensions of two or more power/ground rings. [top| bottom| left| right] are sides of the rings to be added -window { x1 y1 x2 y2 } are corners of a specified window in the design (or chip boundary by default) <layer_name> - name of layer where ring segment is added <offset> - distance (u) from defined window (negative value is toward inside of window) <width> is the width of ring (u), measured toward outside of the chip <space> is the space (u) between rings. Multiple 'width' and 'space' pairs are added (equal to the number of nets specified) <netN> is the net name associated with the rings in the order defined.
ring delete	Deletes all elements of the specified power/ground ring from the design.
	Associated GSR keywords:None
	Options
	<ring_name> Name of previously-added ring to be deleted.

Descriptions of Decap Modification Commands

The following section describes the commands available for performing decoupling capacitance fixing in Totem FAO. The GSR keywords associated with each command are described in the next section.

Note: The decap and cell swap FAO commands are only available for digital place and route blocks.

Decap commands - the following commands provide FAO functions to perform modifications on decoupling capacitance. They are described in the table below.

```
decap [ advise | place | fill | remove | report ] ? arg ?

cell swap ? arg?
```

Table 9-2 Decap Modification Commands

decap report	After running a decap fao command creates an output report in the file *adsRpt/faoOverlapDecap* listing the total number of decaps, the cap value, and the associated DvD improvement.
	Associated GSR keywords: none
	Options
	? -overlap ? Includes report of number of overlapping decaps in the decap report in the format: `<decap_master> loc_x loc_y <num_decaps>`
	? [-eff_vdd_tw | -min_vdd_tw | -max_vdd_tw | -min_vdd_all] ? Sets up 'decap report' for the specified DvD criteria, which are defined as: eff_vdd_tw : average effective voltage (Vdd-Vss) over the timing window to define fixing targets (default DvD criteria) max_vdd_tw : maximum effective voltage (Vdd-Vss) over the timing window to define fixing targets min_vdd_tw : minimum effective voltage (Vdd-Vss) over the timing window to define fixing targets min_vdd_all : minimum effective voltage (Vdd-Vss) over clock cycle to define fixing targets. (Default: -eff_vdd_tw)
decap drc	Creates a DRC report on FAO decap changes, and, with the '-fix' option, fixes DRC violations for user-placed decap cells.
	Associated GSR keywords: none
	Options
	? -fix ? Uses flipping and removal techniques to fix DRC violations on pin geometries in user-placed decap cells that violate signal/shield routes.
	? -o <filename> ? Specifies the output file describing DRC locations and fixes in a decap DRC report.

Table 9-2 Decap Modification Commands

decap advise	Provides an assessment of the amount decap that could be placed in specified areas of the design, and creates a **.dpf* file of results.
	Associated GSR keywords: fao_region, noise_reduction, noise_limit, num_hotinst, cap_limit, fao_decap_overlap, cap_limit, leakage_limit, decap_density,
	Options
	? -place ? Implements maximum decap placement based on 'advise' analysis. Creates **.dpf* file describing placements.
	? -dpf <dpf_filename> ? Specifies the name of the input decap placement file. (Default: *adsRpt/fao.dpf)*
	? -distribute ? Ignores 'noise_limit' and 'noise_reduction' constraints and fixes decap cells up to 'cap_limit' around 'num_hotinst' of hot instances in proportion to their criteria DvD values. (Default: off)
	? [-eff_vdd_tw | -min_vdd_tw | -max_vdd_tw | -min_vdd_all] ? Specifies which type of DvD criteria is used to select candidate hot instances to investigate for fixing. Instances and associated criteria values are defined in the *adsRpt/Dynamic/<design>.dvd* lookup table. The criteria are: eff_vdd_tw : use average effective voltage (Vdd-Vss) over the timing window to define fixing targets (default DvD criteria) max_vdd_tw : use maximum effective voltage (Vdd-Vss) over the timing window to define fixing targets min_vdd_tw : use minimum effective voltage (Vdd-Vss) over the timing window to define fixing targets min_vdd_all : use minimum effective voltage (Vdd-Vss) over clock cycle to define fixing targets. (Default: -eff_vdd_tw)
	? [-inst_file <filename> | -inst {inst1 inst2 ... }] ? -inst_file <file_name> : defines a file containing exclusive list of instances as target areas for filling. -inst { inst1 inst2 ... } : specifies exclusive list of instances as target areas for filling
	? repeat [-iter <n>] ? Specifies automatic repeat up to n times of 'decap advise -place', then extraction, and DvD analysis. At the end of each iteration, the DvD value is checked against the noise_limit. If the noise_limit is met, less than 1% DvD improvment is achieved, or n is reached, iteration stops. Default n =1.
	?-mmx? Provides an assessment of the amount decap that could be placed in a custom blocks or design modeled with MMX methodology, and records the results in the log file.

Table 9-2 Decap Modification Commands

decap place	Provides specific decap placements at hot instance locations based on 'decap advise' results showing how much is needed.
	Associated GSR keywords: fao_decap_overlap, cap_limit, leakage_limit, decap_density, noise_limit
	Options
	? -dpf <dpf_filename> ? Specifies the input decap placement file. (Default: *adsRpt/fao.dpf)*
	? -eco <eco_filename> ? Prepares ECO report file in Totem ECO text format.
decap qor	Measures QoR (results quality) of decaps inserted by FAO.
	Associated GSR keywords: decap_fill.
	Options
	? -i <input_decap_inst_file> Specifies input decap instance list file.
	? -o <output_dir> Specifies the output directory name.

Table 9-2 Decap Modification Commands

decap fill	Provides decap filling in available spaces for partial or full chip. If neither '-fao_region' nor '-hot_area' are defined, spaces in the fix_window area around each hot instance are filled.
	Associated GSR keywords: fao_region, noise_reduction, noise_limit, num_hotinst, cap_limit, fao_decap_overlap, cap_limit, leakage_limit, decap_density, fao_max_shift
	Options
	? repeat [-iter <n>]? Specifies automatic repeat of 'decap fill', followed by extraction and DvD analysis. At the end of each iteration, the DvD value is checked against the noise_limit. After the first normal 'decap fill' run, subsequent iterations place *s* decaps (as specified with the 'density_step' option) at each of the top 5% hot instance locations in the 'fix_window', followed by extraction and DvD analysis. When the noise_limit is met, less than 1% DvD improvment is achieved, *or n* is reached, iteration stops. Default n =3.
	? -density_step <s> ? Specifies the number of decaps to add at each of the top 5% hot instance locations in the 'fix_window' for each 'repeat' iteration after the first 'decap fill'. Default s=1.
	? -eco_name_pattern <string> ? Allows a specified alpha-numeric string to be added to the names of a group of decaps being placed during a 'decap fill' operation. Default string = SH. Names can be of the form: '<decap_cell>_<eco_string><number>' The value of this option overrides any value set using the ECO_NAME_PATTERN GSR keyword.
	? -no_adjust_fix_window ? Turns off the default proportional adjustment of the fix window size based on amount of voltage drop so that all fix_windows are the specified size (default 300x300 microns). If this option is *not* invoked the highest DvD hot instance has the specified fix_window size, and the size of the fix_window for other hot instances is reduced in proportion to the DvD down to 50% of the maximum fix_window dimension for the lowest hot instance specified.
	? -clone_cells { <list of block cellnames> } ? For a block instantiated multiple times at top level, replicates the fill of decap cells exactly the same in all instances of the block, including different orientations. An ECO report is generated listing the decaps added and the numbered clones of the decap instantiation in each block.
	?-prefill ? -no_ff_move ? ? -ignore_fixed ? First performs the decap prefill operation described in the '-prefill_only' option, and then performs the selected 'decap fill' command. If '-no_ff_move' used, no FFs defined in LIB file are moved. Typically used after CTS. If '-ignore_fixed' is used, "fixed" elements such as instances can be moved in executing the decap fill operation.

Table 9-2 Decap Modification Commands

	Options
decap fill (cont.)	? -fao_region ? If selected, uses definition in GSR keyword for x,y coordinates of rectangle within which all spaces will be filled with decap. (default: whole design)
	?-hot_area? If selected, uses definition in GSR keyword for rectangular area to be filled that includes all hot instances meeting constraints on total capacitance, leakage, and noise_limit; (default: if -fao_region and -hot_area are not defined, the fix_window area around each hot instance is filled).
	? [-eff_vdd_tw | -min_vdd_tw | -max_vdd_tw | -min_vdd_all] ? Specifies which type of DvD criteria is used to select candidate hot instance areas to investigate for filling. Instances and associated criteria values are defined in the *adsRpt/Dynamic/<design>.dvd* lookup table. The criteria are: eff_vdd_tw : use average VDD over the timing window to define filling targets (default) max_vdd_tw : use maximum VDD over the timing window to define filling targets min_vdd_tw : use minimum VDD over the timing window to define filling targets min_vdd_all : use minimum VDD over clock cycle to define filling targets
	? [-inst_file <filename> | -inst { inst1 inst2 ... }] ? -inst_file <file_name> : defines a file containing exclusive list of instances as target areas for filling. -inst { inst1 inst2 ... } : specifies exclusive list of instances as target areas for filling (Default: null)
	? -prefill_only ? Moves cells adjacent to each defined hot instance as far as possible away from them in the row, up to the value of GSR keyword FAO_MAX_SHIFT (5 um default). Decap cells are then placed in the larger of space available on either side of hot instance. Cells that have +FIXED flag in DEF, or are listed in the '-fixed_insts' or '-fixed_cells' options, are not shifted.
	? -replace_filler {<filler1> ... } ? or ? replace_filler_file <fillename> ? Specifies existing filler cells to be replaced with decap cells (GSR keyword DECAP_CELL). Either list the filler cells, or specify a file containing a list of filler cells to be replaced. -inst_type specifies a 'FIXED' or 'PLACED' attribute for -replace_filler', or 'ALL' includes all types of filler cells (default). -replace_filler_hot_area automatically generates a "hot box" to replace ineffective fillers (the most significant area to replace fillers with decaps, based on the selected DvD criteria option.
	? [-fixed_insts | -fixed_cells] { ... } ? Fixes the location of listed instances or cells, and prevents any movement by the '-prefill_only' or '-prefill' commands.

Table 9-2 Decap Modification Commands

decap remove	Removes all “ineffective” decap cells, as defined by the value of the ‘-imax_less_than’ criteria (default: 30uA).
	Associated GSR keywords: fao_nets, fao_region, fix_window, noise_limit, noise_reduction, num_hotinst, cap_limit, fao_decap_overlap, cap_limit, leakage_limit, decap_density, fao_verbose
	Options
	? -imax_less_than <peak_current> ? Specifies the smallest peak decap current to keep a decap in place and not remove it (default: 30 uA). Decap list: *adsRpt/Dynamic/decaps.rpt*.
	? -dvmax_less_than ? Specifies a small delta voltage associated with the peak decap current as a threshold below which no decap removal is performed (default: 0)
	? -fao_region ? If selected, specifies that decaps would be removed only within the rectangular region defined by the GSR keyword, according to other option values set or their default values.
	? -hot_area ? If selected, specifies a rectangular area that includes all hot instances (as defined by noise_limit and num_hotinst) that will have *no* decaps removed.
	? -user_decap ? If selected, also removes user-specified decaps, in addition to FAO-added decaps.
	? [-eff_vdd_tw | -min_vdd_tw | -max_vdd_tw | -min_vdd_all] ? Specifies which type of DvD criteria is used to select candidate hot instances to investigate for decap removal. Instances and associated criteria values are defined in the *adsRpt/Dynamic/<design>.dvd* lookup table. The criteria are: eff_vdd_tw : use average effective voltage (Vdd-Vss) over the timing window to define removal targets (default) max_vdd_tw : use maximum effective voltage (Vdd-Vss) over the timing window to define removal targets min_vdd_tw : use minimum effective voltage (Vdd-Vss) over the timing window to define removal targets min_vdd_all : use minimum effective voltage (Vdd-Vss) over clock cycle to define removal targets
	? -ignore_cap_limit ? After removing ineffective decaps based on the imax criteria, this option stops decap removal, even if cap_limit is exceeded (default: off - decap removal continues on decaps associated with lowest-ranked “best” DvD instances in the target list until both capacitance and leakage limits are achieved).
	? -no_decap_report Cancels decap removal reporting to speed up repetitive removal process.

Table 9-2 Decap Modification Commands

decap remove (cont.)	? -ignore_leakage_limit ? After removing ineffective decaps based on imax criteria, this option stops decap removal, even if leakage_limit is exceeded (default: off - decap removal continues on decaps associated with lowest-ranked “best” DvD instances in the target list until both capacitance and leakage limits are achieved).
	? -clone_cells ? Removes all identical decap cells from all cloned instances if none of them cause a degradation of DvD performance. If any decap removals would increase DvD, none of cloned decap cells are removed.
	? -all ? Regardless of other options set or their default values, removes all decaps on the chip, or, if the -fao_region option is set, within the specified region.

Table 9-2 Decap Modification Commands

cell swap	Moves defined hot instance cells to open spaces, or swaps them with either decap cells or cells with lower power demand in a specified area.
	Associated GSR keywords: fao_region, fao_nets, fix_window, noise_limit, num_hotinst, fao_verbose
	Options
	? -eco <ECO_filename> ? Prepares ECO report file '*eco_filename*' in Totem ECO text format.
	? -decap_cells: ? Specifies the names or families of master decap cells that can be used in cell swapping with hot instances. Default: any cells with no logic connection can be used for swapping. Three options define how the cells are specified: -glob: allows global wildcard symbols (such as *) to be used to define families of decap cells allowed for swapping -regexp: allows regular expressions to be used to define categories of decap cells allowed for swapping -exact: supplies the specific names of decap cells allowed for swapping
	? -fixed_cells { cell1 cell2 ...} ? Specifies the names of fixed master cells *not allowed* for swapping
	? -fixed_insts { inst1 inst2 ...} ? Specifies the names of fixed instances *not allowed* for swapping
	? -removable_cells { cell1 cell2 ...} ? Specifies the names or families of decap cells that can be removed from the design, if necessary, during cell swapping. The '-glob', '-regexp', and '-exact' sub-options are also available for this option, with the same usage as for the '-decap cells' option (default - no decap cells are removed).
	? -include_clock ? Allows clock elements to be included in cell swapping.
	? [-inst_file <filename> | -inst { inst1 inst2 ... }] ? -inst_file <file_name> : defines a file containing an exclusive list of instances as targets for swapping. -inst { inst1 inst2 ... } : specifies an exclusive list of instances as targets for swapping
	? search_only Only generates a file with the list of candidate instances for swapping based on selection criteria; no swapping is performed (default file name: *cell_swap_<time>.list*. If you want to change the filename, use the '-inst_file <filename>' option to specify).

cell swap (cont.)	[-eff_vdd_tw | -min_vdd_tw | -max_vdd_tw | -min_vdd_all] Specifies which type of DvD criteria is used to select candidate hot instances to investigate for decap swapping. Instances and associated criteria values are defined in the *adsRpt/Dynamic/<design>.dvd* lookup table. The criteria are: eff_vdd_tw : use average effective voltage (Vdd-Vss) over the timing window to define swapping targets (default) max_vdd_tw : use maximum effective voltage (Vdd-Vss) over the timing window to define swapping targets min_vdd_tw : use minimum effective voltage (Vdd-Vss) over the timing window to define swapping targets min_vdd_all : use minimum effective voltage (Vdd-Vss) over the clock cycle to define swapping targets
	? -report ? Generates text table of the last cell swapping results in file *cell_swap.rpt*, showing instances swapped and the change in voltage drop.
	? -plot ? Generates a histogram of the last cell swapping results in file *cell_swap.hist*, showing the number of instances vs. the change in the criteria DvD value selected.
	?-swappable_cells {cell1 cell2 ...} ? Lists master cells available for swapping.
	? -ff_only ? Specifies that only instances directly connected to a flip-flop input should be moved. Used in conjunction with the '-swappable_cell' option, only listed cells that connect to a flipflop input are swapped to improve their voltage drop.
	? -pre_proc ? Specifies that hot instances in the row with the highest average voltage drop have their cells swapped first. Then hot instances in rows with the next highest average voltage drop are done in order.
	? -to_scan_in_only <cell1 cell2 ...> ? Specifies that only instances that are attached to scan-in pins of specified scan cells are included in cell swapping.
	? -include_thtl ? Specifies that instances with tie-high and tie-low pins can be included in cell swapping. By default these cells are excluded.
	? -drc_m2 ? Specifies that any instances that have metal2 features cannot be swapped to a target area with metal2 access.

Supplementary Voltage Drop Fixing Commands

Table 9-3 Route Fix Commands

Command	Description
route fix	Finds the shortest DRC-clean legal route for a new power/ground wire and vias from the specified target “from” point (or area) and layer to another specified target “to” point (or area) and layer.
	Associated GSR keywords: None
	Options
	[-from { x1 y1 x2 y2 <layer>} | -from_pt { xf yf <layer>}] Specifies the origin area or x,y point and layer for new pwr/grnd routing.
	[-to {x1 y1 x2 y2 <layer>} | -to_pt { xt yt <layer>}] Specifies the terminating area or x,y point and layer for new pwr/grnd routing.
	-width <routing-width> Specifies the width of the new pwr/grnd routing in um.

GSR Keywords Supporting FAO Functionality

Special GSR keywords modify the function of the ‘mesh optimize’, ‘mesh fix’, ‘route fix’, ‘decap advise’/-place’, and ‘cell swap’ commands, and are described in detail in the section "FAO General Keywords", page C-879, section "Grid Fixing and Optimization Keywords", page C-881, and section "Decap Optimization Keywords", page C-885.

Command and GSR Keyword Syntax Conventions

```
< x >          x is a variable name or value to be specified
[ a | b | c ]  one of a, b, or c must be selected
? x ?          x is an optional parameter
{ a b c }      a, b and c parameters all must be specified
{ x }...       an element such as {x} can be added
+ - * /        standard arithmetic operators for add, subtract,
               multiply, divide
```

Chapter 10
Custom Macro Models

Introduction

Totem can create a Custom Macro Model, a compact optimized power model that contains both electrical and physical data for the custom macro. You can create both detailed transistor-level models (“mmx_view”) and cell-level models (“cell_view”).

Totem ‘cell_view’ CMMs preserve transistor switching/toggling information, but with a very compact view, to allow you to load and run them in SOC full-chip dynamic simulation. Totem ‘mmx_view’ CMMs are detailed transistor-level models, which therefore can have significant performance and memory use impacts when used in large SOC designs.

Note: model generation and analysis must be done on the same type of binary OS platform.

CMM Interfaces

Since CMM is an extracted model, it is critical to define the interface correctly between the model and top level at which the model is used. CMM connects to the top level based on the pins or ports defined during model generation. It is important to define interface pins correctly in order to ensure accuracy of the results.

Tech files

The tech files of CMM blocks and top level runs are merged automatically. The tech files do not need to be exactly identical; there can be differences in following parameters:

- number of layers
- names of the layers

However, the layer stacking order and matching layers' properties must be the same.

Defining CMM Pins

There are four methods of creating CMM pins, which are described in this section.

1. Pins defined in LEF for the top level cell of the CMM, which are always inherited as pins of the model.
2. All the pins in defined in DEF file become CMM pins if the GSR keyword 'ADD_PLOC_FROM_TOP_DEF 1' is specified for model creation. If you generate a CMM for a GDS block (such as a memory), then you can generate the DEF pins using GENERATE_PLOC* options discussed in the GDS2DEF/GDSMMX section.

3. You can also define the pins by importing a PLOC file, using the GSR keyword:

```
PAD_FILES {
    CMM.ploc
}
```

   In this case, the PLOC points are converted to large pins on the geometries on which they lie. Enlarging the CMM pins is so that all points in the enlarged geometry can connect to top level design. You must define these pins for CMM in the ploc file following the DEF convention, which means that additional pins of the same net should be suffixed with *.extra<number>*, using contiguous numbering.

4. Finally, you can let the software create the pins automatically, using the GSR keyword 'CMM_CREATE_PINS 1', in which case no other pins should be specified. That is, the keywords PAD_FILES and 'ADD_PLOC_FROM_TOP_DEF 1' should *not* be used, to avoid redundant pins being created.

   With the CMM_CREATE_PINS, Totem creates surface pins that cover the entire layer geometry for the top two routing layers by default, and in lower metal layers it creates boundary pins on the edges of wire shapes that touch the bounding box of cells. These small pins are created to connect to other geometries at the top level by abutment.

   To modify the default behavior you can use the GSR keyword CMM_EXPAND_PIN_LAYERS as in the following example:

```
CMM_EXPAND_PIN_LAYERS {
    M10
    M9
    M8
    M7
}
```

   The example creates large pins on metal layers M7, M8, M9 and M10. If you want to avoid large pins altogether, specify a single non-existent layer name in this section, such as 'NONE'. Using automatic pin creation is recommended if you are not sure how the CMM model gets connected to the top level, such as in IP CMMs.

Embedding Signatures in a CMM

A CMM is PVT-dependent, since it is an extracted model, and resistance values depend on the corner and temperature values in the technology file that were used to generate the CMM. Library teams can embed signatures in the CMM so that the designer using the CMM can query the CMM to identify the CMM-PVT, as well as any other specific settings used for CMM generation. To embed a user-specified signature string in a CMM, use the GSR keyword:

```
CMM_PROCESS <string>
```

Keywords for including data in CMM models - analog

Some files can be specified during model generation can be included in the CMM and IP models, and then imported at top level run. In addition, some of the information already present in the generated CMM can be excluded. GSR keywords to perform data inclusion and exclusion are described below.

BLOCK_POWER_FOR_SCALING & BPFS_FILE

The Block Power For Scaling (BPFS) section or any BPFS_FILE are saved to the CMM database as BPFS_FILE. These are then appended to top level GSR internally, so that they are processed at top level.

You can save Block Power Assignment (BPA) information for instances inside CMMs and modify them in the top level run. If you create a BPA region REGION1 inside a CMM with a specified power in mW, and use the 'save design' command, the BPA is saved in the CMM DB. Then at the top level you can modify this power value using BLOCK_POWER_FOR_SCALING. The region name given in BPFS must be the full hierarchical instance name. For example:

```
BLOCK_POWER_FOR_SCALING {
    FULLCHIP INST1/REGION1 0.006
    FULLCHIP INST2/REGION2 0.002
 }
```

If you do not modify the value, the original power assignment inside the BPA region is honored.

APL_FILES

All cdev, current and AVM files in APL_FILES are automatically saved in the CMM model, and imported back at top level. CMM also includes the AVM automatically created using LIB2AVM flows. The APL files are automatically embedded in CMM models.

BLOCK_POWER_ASSIGNMENT

BPA constructs specified in the GSR are saved automatically in the CMM. CMM supports both via-based as well as transistor-based assignments that are available in BPA.

Keywords for including data in CMM models - digital place and route

The following keywords are relevant specifically for mixed signal and digital place and route blocks.

STA_FILE, BLOCK_STA_FILE, and USER_STA_FILE

While saving the CMM model for a hierarchical block (such as place and route), either STA_FILE or BLOCK_STA_FILE keywords can be used. These files are saved in the CMM model. At the top level, all the timing files from CMM models are included in the BLOCK_STA_FILE GSR section for the top level run. If the top level timing file includes data for instances inside CMM blocks as well, then the top STA should be specified as STA_FILE, so that the block level STA files coming from CMM models are ignored.

Similarly, user STA information is saved in the CMM model and is automatically processed at the top level for equivalent effect.

CELL_RC_FILE (SPEF files)

The SPEF files from CMM blocks are automatically included in top level run by including them in the CELL_RC_FILE section of the top level run.

VCD_FILE

Any VCD file specified in the model generation is saved in the CMM model. However, CMM generation does not support block level VCD files (BLOCK_VCD_FILE keyword). The parameters of the VCD_FILE section are also saved. At the top level, VCD files and their parameters (such as start time and end time) from CMM blocks are automatically included in the BLOCK_VCD_FILE section in the GSR.

MCF file

MCF files from the CMM are included in the MCF_FILE section in the top level run. To include the MCF in CMM, you need to perform power calculation during the CMM generation run in order to save the MCF file.

INSTANCE_POWER_FILE

Information from the Instance Power File included during CMM generation is included in the top level run. The instance names in the file are prefixed with the CMM instance name, so power is assigned to instances contained in each CMM instance correctly.

ESD-aware CMM Creation

To provide ESD-aware and compatible CMMs (raw and binary), you must specify the following for both model creation and top runs:

```
setup analysis_mode esd
```

The contents of the files specified in the following GSR keywords are saved into the CMM models and loaded during top runs:

```
ESD_CLAMP_FILES
PUSH_PININST_CELLS_FILE
ESD_CLAMP_PIN_FILE
```

Keywords for excluding data from CMMs

By default, all available files in CMM models are imported at top level while importing the CMM. To exclude some of these files by model and file type, the GSR keyword 'CMM_EXCLUDE_FILES' can be used, as follows:

```
CMM_EXCLUDE_FILES {
    <modelname1> <file_type1> <file_type2> ...
    <modelname2> <file_type1> <file_type2> ...
...
}
```

where

<modelname1>: CMM model name

<file_type1> : type of file to be excluded

Example:

```
CMM_EXCLUDE_FILES {
    M1 ipf vcd spf sta ...
    M2 bpfs
}
```

So in the example, file types ipf, vcd, spf and sta should not be imported for CMM model M1, and bpfs information should not be imported for model M2. Note that BLOCK_POWER_ASSIGNMENT (BPA) *cannot be excluded* from the CMM.

Extra caution is needed when importing any instance power file (IPF), since any instance not covered by IPF is assigned a power of 0, which may not be desirable. So if any IPF file is imported, a warning is displayed that an IPF was imported for some CMM models but not for top level or other CMM.

Choosing appropriate hierarchy levels for CMMs

To maximize benefits from the CMM flow, you must choose the hierarchy at appropriate levels. Since there is some overhead in the interfaces for CMMs, if the CMM blocks are too small compared to the design size, then the interfaces may dominate, and the expected performance benefits may not be achieved. It is beneficial to use CMMs for blocks that are instantiated a number of times in the design, as RedHawk/Totem leverage the already-extracted views for CMM during extraction.

If you are creating a model of a well-defined hierarchical block, such as a memory cell or an IP block, to interface to other groups, and where performance is not the sole reason to create CMMs, then the above guideline can be relaxed.

CMM database compatibility across RedHawk and Totem releases

Totem software can read CMM databases generated with the current release of RedHawk or Totem, and also databases generated by older releases back to version 9.1. In other words, you can read CMM databases generated with RedHawk/Totem 9.1 or newer with a RedHawk or Totem release that is the same or newer than the version that was used to generate the CMM model. You cannot read a CMM database created in a newer release than you are running.

At the beginning of every major release of RedHawk and Totem, the CMM DB version is incremented to the next higher integral number.

Preparing APLMMX and GDSMMX Model Input

Initially you must decide if you want to do optimization of the existing electrical and physical input models. Examples of two levels of optimization of APLMMX and GDSMMX input models follow.

No APLMMX or GDSMMX model optimization

When you specify APL_FILES and GDS_CELLS in the GSR, Totem uses the existing input electrical and physical models, without rerunning APLMMX and GDSMMX for optimization. To use the original electrical and physical models, the required GSR keywords are:

- TECH_FILE
- FREQ
- VDD_NETS
- GND_NETS
- PAD_FILE (optional if GENERATE_PLOC is defined when generating physical model)
- APL_FILES
- GDS_CELLS

Optional keyword:

- CMM_CONSTRAINT_FILE (user-defined block-level constraint file to pass to top-level checking)

GSR Example 1 - no APLMMX or GDSMMX optimization

```
====================================
        CHARACTERIZE_BLOCK 1
        TECH_FILE ./user_data/totem/gli8188.tech
        LIB_FILES {
        user_data/totem/TYPICAL
        }
        VDD_NETS {
        VDD 1.8
        }
        GND_NETS {
```

```
VSS 0
}
FREQ 0.5e9
APL_FILE {
ram64x8/ram64x8.spcurrent current
ram64x8/ram64x8.cdev cdev
}
GDS_CELLS {
ram64x8 ram64x8/gdsmmx mmx
}
```

APLMMX and GDSMMX model optimization

GSR Example 2

Totem regenerates an electrical and physical model based on the input file defined in APLMMX{ } section. The GDSMMX configuration is created based on the default keyword settings. If you need to add optional keywords in GDSMMX setup, put them inside the GDS_OPTION {...} keyword inside the APLMMX {...} section, which are passed to the internally-created GDSMMX configuration file.

```
====================================
        MMX {
        APLMMX {
        CMD_OPTIONS -n
        TOP_CELL ram64x8
        VDD_PIN_NAME VDD
        GND_PIN_NAME VSS
        VDD 1.8
        SPICE_NETLIST ./user_data/aplmmx/ram64x8.spi
        TEST_BENCH_FILE ./user_data/aplmmx/ram64x8.tbh
        DEVICE_MODEL_LIBRARY ./user_data/aplmmx/smic18.lib TT
        GDS_FILE ./user_data/gdsmmx/ram64x8.gds Required
        GDS_MAP_FILE ./user_data/gdsmmx/gds.map Required
        SPICE_XY_SCALE 1
        PROBE_THRESHOLD off
        GDS_OPTION
        {
        LEF_FILE {
        user_data/totem/GLI8188_0601C.lef
        }
        APACHE_PHYSICAL_MODEL 0
        DISABLE_MINILVS 1
        GENERATE_PLOC USE_TEXT_LABEL
        }
        CUSTOM_STATE_SIM_TIME {
        a "" "" 0 2e-9
        b "" "" 1e-9 3e-9
        }
        VOLTAGE_SOURCES {
        VDD vVDD
```

```
VSS vVSS
}
SIM_OUTPUT_FILE ./user_data/aplmmx/ram64x8.ta0
DB_PROBE_FILE SKIP
}

CHARACTERIZE_BLOCK 1
TECH_FILE ./user_data/totem/gli8188.tech
LIB_FILES {
user_data/totem/TYPICAL
}
VDD_NETS {
VDD 1.8
}
GND_NETS {
VSS 0
}
FREQ 0.5e9
CMM_CONSTRAINT_FILE tmcmm.constraint
}
```

GSR Example 3

Totem regenerates electrical and physical models based on the input file defined in APLMMX { } and GDSMMX { } sections.

Note: To avoid confusion in the integrated GSR flow, the keywords 'GDS_VDD_TEXT_NETS' and 'GDS_GND_TEXT_NETS' must be used inside the 'GDSMMX { }' section instead of keywords 'VDD_NETS' and 'GND_NETS'.

```
=========================================================
        MMX {
        APLMMX {
        CMD_OPTIONS -n
        TOP_CELL ram64x8
        VDD_PIN_NAME VDD
        GND_PIN_NAME VSS
        VDD 1.8
        SPICE_NETLIST ./user_data/aplmmx/ram64x8.spi
        TEST_BENCH_FILE ./user_data/aplmmx/ram64x8.tbh
        DEVICE_MODEL_LIBRARY ./user_data/aplmmx/smic18.lib TT
        SPICE_XY_SCALE 1
        PROBE_THRESHOLD off
        CUSTOM_STATE_SIM_TIME {
        a "" "" 0 2e-9
        b "" "" 1e-9 3e-9
        }
        VOLTAGE_SOURCES {
        VDD vVDD
        VSS vVSS
        }
        SIM_OUTPUT_FILE ./user_data/aplmmx/ram64x8.ta0
        DB_PROBE_FILE SKIP
```

```
}
GDSMMX {
GDS_FILE ./user_data/gdsmmx/ram64x8.gds
MACRO_TYPE memory_mmx
XTOR_MAP_FILE /data/chunta/QA/xtro_map.file
GDS_MAP_FILE ./user_data/gdsmmx/gds.map
LEF_FILE {
user_data/totem/GLI8188_0601C.lef
}
APACHE_PHYSICAL_MODEL 0
DISABLE_MINILVS 1
GENERATE_PLOC USE_TEXT_LABEL
GDS_VDD_TEXT_NETS {
    VDD {
    VDD:
    VDD
}
}
GDS_GND_TEXT_NETS {
    VSS {
    VSS:
    VSS
}
}
}
CHARACTERIZE_BLOCK 1
TECH_FILE ./user_data/totem/gli8188.tech
LIB_FILES {
user_data/totem/TYPICAL
}
VDD_NETS {
VDD 1.8
}
GND_NETS {
VSS 0
}
FREQ 0.5e9
CMM_CONSTRAINT_FILE tmcmm.constraint
}
```

Setting up the Integrated Configuration File for CMM Generation

The Totem integrated configuration file includes GSR, APLMMX and GDSMMX keywords. To generate an optimized CMM, the required GDS-related keywords are:

- LEF_FILE (specifies LEF file for the macro for top-level connection)
- GDS_FILE
- GDS_MAP_FILE
- GENERATE_PLOC USE_TEXT_LABEL (if no PAD_FILE is defined)

Other general required GSR keywords are:

- TECH_FILE
- FREQ
- VDD_NETS
- GND_NETS
- PAD_FILE (optional if GENERATE_PLOC is defined)

An optional APLMMX-related GSR keyword is:

- PROBE_THRESHOLD (if you want to reset or turn off the probe reduction)

Optional GDS-related keywords are:

- APACHE_PHYSICAL_MODEL
- CORE_EXTRACTION_STARTING_LAYER
- MEMORY_BIT_CELL

A CMM-related GSR keyword is:

- CMM_CONSTRAINT_FILE (user-defined block-level constraint file to pass to top-level checking). See section "CMM_CONSTRAINT_FILE", page C-819, for contents and syntax of the CMM constraint file.

Controlling mmx_view and cell_view physical models separately

Using the integrated configuration file or the ‘export model’ TCL command, you can define the following keywords to control separately the 'mmx_view' and 'cell_view' physical models:

```
GDS_OPTION {
    EXTRACTION_STARTING_LAYER <layer_name> ?[cell_view | mmx_view]?
    CORE_EXTRACTION_STARTING_LAYER <layer_name> ?[cell_view |mmx_view]?
    APACHE_PHYSICAL_MODEL <layer_name> ?[cell_view | mmx_view]?
}
```

For example:

```
GDS_OPTION {
    EXTRACTION_STARTING_LAYER Metal1 cell_view
    CORE_EXTRACTION_STARTING_LAYER Metal3 cell_view
    EXTRACTION_STARTING_LAYER Metal1 mmx_view
    CORE_EXTRACTION_STARTING_LAYER Metal1 mmx_view
}
```

A comparison of the differences in MMX_VIEW and CELL_VIEW models are summarized in Table 10-1 below.

Totem Macro Model	GDSMMX	GDS2DB
Summary	No difference in terms of geometries between MMX_VIEW and CELL_VIEW models	Significant geometry reduction between MMX_VIEW and CELL_VIEW models

Totem Macro Model	GDSMMX	GDS2DB
MMX_VIEW	• Automatically identifies memory core region. • Inside core, MET3 and higher geometries are considered. • Outside core, MET1and higher geometries (all) - selectively remove M1 geometries. • Currently this has limitations, as all the transistor pins are snapped to the outside boundaries of the memory core.	• No automatic identification of memory cores. • M1+ geometries (all) have full content, no reduction.
CELL_VIEW	• Same as MMX	• Temporarily snap transistor pins to contacts first. • Trace pins through the actual geometry path up to M3/VIA34 layers. • Create current pins on M3. • No geometries are extracted below M3 (and hence no connectivity is preserved).

Table 10-1 Comparison of MMX_VIEW and CELL_VIEW models

CMM static model support

Totem CMM model generation can generate static CMM cell_view and mmx_view if you do not have APLMMX characterization results (**.spcurrent* and **.cdev*) for dynamic analysis. You can use GSR keywords BLOCK_POWER_FOR_SCALING or BLOCK_POWER_ASSIGNMENT to define static power for the block and create a CMM model by using the same export model TCL command. The block power values are stored automatically in the CMM model so you can later load them into top-level analysis. The region constraints defined in BLOCK_POWER_ASSIGNMENT are automatically transformed into top-level coordinates based on the top-level instance's offset and orientation definitions.

Generating a Custom Macro Model

After design data setup is completed, you can use the following approach to generate a CMM in Totem:

CMM Generation Using the ‘export model’ Command in Totem

The TCL command is:

```
export model -mmx ?-config <macro_model_cfg_file>?
?-mmx_view? ?-cell_view? -top_only ? -dir <outdir> ?
```

where

-config <macro_model_cfg_file>: specifies the configuration file for creating the macro model. Default configuration is from “import gsr <tsr_style_config>

-mmx_view: generates ‘mmx_view’ model

-cell_view: generates ‘cell_view’ model

-top_only :enables post-simulation CMM model generation for digital blocks.

-dir <dir_name>/: specifies the output directory; default output directory is 'macro'
<dir_name>/<cell>_model/mmx_view/<cell>_cmm
<dir_name>/<cell>_model/cell_view/<cell>_cmm

The default output directory is named 'macro' if you do not specify a directory. The configuration file is very similar to the APLMMX configuration file, except:

- GDS_FILE and GDS_MAP_FILE keywords are required
- SIM_OUTPUT_FILE is required

Totem uses the characterization configuration file as *macro_model config*, so you must make sure that the above keywords are already defined in the configuration file. Note that SIM_OUTPUT_FILE is not required if NSPICE is run inside APLMMX.

Loading/Importing CMM Cells for Analysis

Loading a CMM Binary Model

To load (import) one or more Totem CMMs for top-level SOC analysis, define the cells using the keyword ‘CMM_CELLS’ in the GSR file. The ‘optimize’ option reduces the RLC network within CMM instances, resulting in fewer nodes and resistors, and thus reducing simulation memory use and runtime. If desired, you can use the full model by specifying the ‘original’ option, with the following syntax:

```
CMM_CELLS {
    <cell_name1> <macro_model_path1> [original | optimize]
    <cell_name2> <macro_model_path2> [original | optimize]
    ...
}
```

Note: model generation and analysis must be done on the same type of binary OS platform. Also, note that the default mode is ‘optimize’ for 'cell_view' models and ‘original’ for 'mmx_view' models. A compressed *.tarz file may be specified as a model folder.

For example:

```
CMM_CELLS {
  ram256x8x2 user_data/ram256x8x2_model/cell_view/
    ram256x8x2_cmm original
  ram64x8 user_data/ram64x8_model/mmx_view/ram64x8_cmm
    original
}
```

Loading a CMM RAW Model

Totem CMM model generation automatically generates a RAW model in the output CMM directory. The RAW model is not the original input design data, such as a Spice netlist and GDS; it contains the processed electrical and physical data, including a GSR and TCL template, which allows you to regenerate a CMM. This can solve issues with different release versions, or allows regenerating a CMM with a different technology file to resolve a CMM version incompatibility, or a technology layer stacking consistency limitation.

If you want to directly load a CMM RAW model, you can use the following setting in the GSR to directly import the RAW model for analysis:

```
CMM_CELLS {
  <cell_name> <CMM_directory_path> raw
}
```

The CMM Raw model can save GSC_FILES information during model creation. The GSC information is saved in the CMM Raw model and later can be loaded back into the top level. The GSC data stored in the model is used only if top level GSC data is not available. When the top level GSC is provided, it takes precedence, and overwrites the GSC constraints defined and stored in the CMM raw model.

Generate & use CMM mmx-view, cell-view and post-sim views in a single directory

User can generate different CMM views (mmx-view, cell-view and post-sim view) in a single directory in Totem. This “bundling” of views simplifies selecting appropriate views for different instances of a CMM cell. For enabling this, use `‘-ipdir’` option in `‘export model’` TCL command to specify the directory and create the views.

Example TCL:

```
# Generate mmx view
export model –mmx -mmx_view –ipdir ./parent_dir/mmx_model
-config cell_mmx.gsr

# Generate cell view
export model -mmx -cell_view -ipdir ./parent_dir/
cell_view_model -config ./cell_mmx.gsr

# Generate post-sim CMM
setup design cell_mmx.gsr
...
export model -mmx -cell_view -ipdir ./parent_dir/
postsim_view_model -config ./cell_mmx.gsr
```

This will generate mmx-view, cell-view and post-sim view in `parent_dir` directory. The `‘.modeltag’` file in each of view sub-directory denotes the type of view stored there.

If there are multiple states in input APL, while generating a post-sim view, export model generates the current profile corresponding to each state. Use the ‘-no_state_pratio’ option to instead capture current only for state used in dynamic analysis.

Example GSRs for instantiating models:

```
IP_MODELS {
    #<cell_name> <parent_dir>
    cell_mmx      parent_dir
}

## Use post-sim view by default
IP_MODEL_CELL_MAP {
    #<cellname> <model_directory>
    cell_mmx    cell_view_model
}

## Instance specific override
IP_MODEL_INST_MAP {
```

```
    # <instance_name> <Cellname>  <model_dir_in_IP_model>
    instance1 cell_mmx    mmx_model
    instance2 cell_mmx    postsim_view_model
}
```

Importing in Mixed Mode

Custom Macro Models also can be imported in mixed mode. That is, for the same master cell, cell-view and mmx-view models can be applied for different instances in the same session.

1. To generate mixed-mode models inside Totem:

   ```
   export model -dir <output_dir> -mix
   ```

   When the '-mix' option is used during model generation, the '_mmx' suffix is automatically appended to the original master cell names in the transistor-level mmx_view model.

2. To use the mixed mode model in the top-level assembly stage, define the mmx_view with the extra suffix '_mmx' in the cell name and the cell_view with regular cell name in the GSR CMM_CELLS keyword.

   a. To specify the instances to apply mmx_view models, use the GSR keyword CMM_INST_CELL_MAP with the following syntax:

   ```
   CMM_INST_CELL_MAP {
    <inst_name> <cell_name>
   }
   ```

   For example,

   ```
   CMM_CELLS {
        memoryA   CMM_model/cell_view/memoryA_cmm/
        memoryA_mmx  CMM_model/mmx_view/memoryA_mmx_cmm/
   }
   CMM_INST_CELL_MAP {
        blockC/inst1 memoryA_mmx
        blockD/inst2 memoryA_mmx
   }
   ```

   The original cell name in top DEF is memoryA, and instances ‘blockC/inst1’ and ‘blockD/inst2’ use the mmx_view transistor level model and other instances with the same master cell use cell_view.

CMM Compatibility between Releases

This section describes what CMM database versions are supported by a given Totem or RedHawk release.

Finding the CMM DB version

For each major release of RedHawk and Totem, the CMM DB version is incremented to the next higher integral number. To determine the CMM DB version, in any RedHawk version, use the command 'perform cmmcheck -path <CMM_DB_path>', and the CMM DB version is reported.

Also, the *totem.log* and *redhawk.log* files are included in the CMM model. In case of an IP model, only the first seven lines are saved. From the log file you can determine the RedHawk/Totem version and build date used to generate the CMM model.

CMM database compatibility across RedHawk and Totem releases

Totem and RedHawk software can read CMM databases generated with the current release of RedHawk or Totem, and can read databases generated by older releases back to version 9.1. In other words, you can read CMM databases generated with RedHawk 9.1 or newer with a RedHawk or Totem release that is same or newer than the RedHawk version that was used to generate the CMM model.

You cannot read a CMM database created in a newer release than you are running.

If your model is generated with Totem and you are using RedHawk for reading in the model at the top level, you must use a RedHawk build that is compatible with the model. To identify suitable RedHawk builds for a top level run, look up the Totem build date from the first section of the file *<CMM_DB_path>/totem.log*, and then use a RedHawk build that is of the same or later major release, with a build date that is the same or later than the Totem build date used to generate the model.

At the beginning of every major release of RedHawk and Totem, the CMM DB version is incremented to the next higher integral number.

Chapter 11
Signal EM Analysis

Introduction

Electromigration (EM) is the movement of material that results from the transfer of momentum between electrons and metal atoms under an applied electric field. This momentum transfer causes the metal atoms to be displaced from their original positions. This effect increases with increasing current density in a wire, and at higher temperatures the momentum transfer becomes more severe. Thus in sub-100nm designs, with higher device currents, narrower wires, and increasing on-die temperatures, the reliability of interconnects and their possible degradation from EM is a serious concern.

The transfer of metal ions over time from EM can lead to either narrowing or hillocks (bumps) in the wires. Narrowing of the wire can result in degradation of performance, or in extreme cases can result in the complete opening of the conduction path. Widening and bumps in the wire can result in shorts to neighboring wires, especially if they are routed at the minimum pitch in the newer technologies.

Foundries typically specify the maximum amount of current that can flow through a wire under varying conditions. These EM limits depend on several design parameters, such as wire topologies, widths, and metal densities. EM degradation and EM limits depend significantly on the temperature at which interconnects operate, as well as on the material properties of the wires and vias, on the direction of current flow in the wire, and on the distance of the wire segment from the driver(s).

One common EM check employed is to measure the average or DC current density flowing through the wire and compare it against foundry-specified limits. The impact of average or DC current in the wires in a design is typically quantified using Black's equation, which is used to measure and compare the Mean-Time-To-Failure for interconnects with different parameters, such as the average current density, temperature, and activation energy. Another common check employed is to measure the peak and RMS current flowing through interconnects and check them against foundry-specified targets. These checks are to ensure that metal failures do not occur because of Joule or self-heating in the wires.

Totem™ provides a single platform approach in which to analyze EM of both power grid and signal interconnects in a design. Power EM analysis is performed as an integral part of static and/or dynamic analysis (see section "Power/Ground EM Analysis", page 4-168). Signal EM analysis, which is performed in a separate run, checks for average (uni-directional or bi-directional), RMS, and peak current densities in all signal wires and vias in a design. Totem signal EM analysis is performed on flat transistor-level designs, but if you have digital place and route blocks, you also can perform block-level inter-cellular signal EM analysis.

Methodology for Transistor-level Signal EM Analysis

Overview

After extraction, signal EM analysis is performed on a flat transistor-level design by calculating Iavg, Irms and Ipeak for every wire. The calculated current densities are compared against their respective EM limits, and violations are reported in detail. Current values and their directions are reported for each signal wire, in addition to any EM violations. The current direction can be either bi- or uni-directional. In general, pull-up currents are responsible for charging the nets, while pull-down currents are responsible for discharging. If the magnitudes of both currents are comparable (up to 20% difference allowed), the current is reported as bi-directional.

Totem supports multi-cycle multi-states analysis set up in the GSC file. The default average/RMS/Peak current and EM values are calculated based on the worst cycle among all states (the average and RMS current and EM are calculated based on each state not based on the whole simulation time).

The Totem dynamic signal EM flow requires similar inputs to the P/G EM/IR flow, as follows:

1. GDSII of the layout
2. GDSII layer map
3. Netlist for the design
4. Signal only RC DSPF of the design with device x,y coordinates
5. Simulation testbench
6. Spice models
7. Extraction technology file

Once the required input data are available, the first step in running Signal EM is to perform the setup and the “pre-characterization” of the circuit. Pre-characterization is a process in which the netlist of the design is simulated in any Spice-compatible simulator, capturing device currents, and a current model is written out. Then the layout and current models are simulated to perform a full Signal EM analysis. The steps in generating required Totem input data are described following:

Step 1: Probe generation (aplmmx -sig -s)

This step identifies the instances or devices whose ports are connected to non-power nets. The output file contains a list of these corresponding probes, and can be found in the EMP directory as *<design_name>.probe*.

Step 2: Running Spice simulation (spice)

This step pre-characterizes the currents consumed by all devices in the design, which involves running a Spice simulator with a set of vectors to capture the currents drawn by all devices. For accurate signal EM analysis, the simulation netlist should contain the RC parasitics for your signal nets. The probe file that was generated in the previous step should be included during Spice simulation to ensure that device currents are being probed. Just as in dynamic analysis, the simulation output file can be any Spice-compatible format such as *fsdb*, *raw*, or *tr0*.

Step 3: Generating an electrical model (aplmmx -sig -n)

Once the Spice pre-characterization is completed, the simulation outputs for the device currents are converted to an electrical model for use in Totem. This electrical model is

generated using the 'aplmmx -n -sig' command with the proper configuration file. The output of this step is a current file (**.spcurrent*). A template for the 'aplmmx' config file is provided in the following section as a reference.

To determine the currents at the driving points, Totem can either perform current estimation or import currents based on actual SPICE simulation. If estimation is preferred, for performance reasons, then the currents are based on design parameters and user input.

Current Estimation

For Iavg, the currents are calculated based on the total capacitance, the source voltage, the frequency, and the given toggle rate for each signal net. Totem rectifies the current before reporting, since the estimated current flowing in most nets is generally bi-directional, and without rectification the effects in each direction would nearly offset each other.

Estimation of the RMS current requires understanding of the current profiles, since it is defined as the integral from time 0 to Tend. Tend can be Tclk in digital designs. In general, the current profile is observed during the charging and discharging of the signal net at the physical location of each driver. Totem approximates the switching current using a polynomial-based profile, whose shape depends on the average current of the net, as well as the transition times for the rising and falling edges. That current profile is used to estimate Ipeak as well. These relationships are as follows:

Irms = SQRT(SUM(I_t^2)) / Tend)

Ipeak = MAX(ABS(I_t))

All computed currents are highest at the driver points and gradually decrease towards the end of the net at each receiver location. There can be multiple driving points; the following relationships are used to evaluate the average current and the actual net load:

Iavg = SUM(I_avg_at_driving_location) = f*tr*V*Cload

Importing Simulation Currents

For best accuracy, SPICE simulation-based analysis is the preferred option. The estimation of driver current is replaced by true simulation results, which eliminates spurious EM violations. Also, the EM analysis is no longer limited to switching current, so current flowing between drivers is also correctly analyzed. Totem integrates the SPICE simulation and waveform current importing through a comprehensive user interface. Using SPICE netlists, device libraries and testbench files Totem supports different SPICE simulation environments and SPICE products. After importing current profiles, Totem performs a true transient simulation to get current waveforms at each wire segment, and calculates Iavg, Irms, Ipeak, and the corresponding current densities, to check for EM violations in each EM mode.

Totem reads the GDSII layout file, as well as a post-extracted DSPF netlist, to obtain the physical and electrical model of the cell being analyzed. Based on these models, all currents are computed. To find EM violations, current densities based on wire width are determined and compared to EM limits specified in the technology files, which can depend on physical parameters such as wire width or via size, and die temperature.

Input Data Requirements

Transistor-level signal EM analysis requires the following data:

- GDSII file. Similar to the Totem P/G EM/IR flow, the Signal EM flow requires the nets in the design to be 'short free'. Because Totem traces the nets based on an input set of coordinates, any shorting of nets in the design leads to the analysis dropping the

shorted nets. When Totem traces the nets it creates device pins for all types of devices. For example, MOS pins are snapped to the closest source/drain/gate contacts shared by M1 and diffusion/poly.

If poly resistors are defined in aplmmx, a polyres marking layer should be defined in the GDS layer map file to help the tool create pins and break connectivity in the right place. See section "Analog Designs", page 5-177, for specification of ID layers for various types of analog devices such as BJT, Diodes, and Poly-Resistors, since the requirements for the Signal EM flow are similar.

DSPF netlist. Totem Signal EM relies on the coordinates from a netlist or DSPF file to trace the signal nets in the GDS. Totem supports Mentor Calibre and Synopsys Star RC extracted netlists. The extracted layout netlist/DSPF must contain sub-node coordinates and layer information for each net extracted. Device locations should be listed in the *|I section of the DSPF. Only co-ordinates under *|S section are used for net-tracing purpose to avoid any false shorts observed due to approximate start points.

Suitable Calibre XRC settings are:

RLOCATION RLAYER RWIDTH

PEX NETLIST DISTRIBUTED

Sample Calibre XRC DSPF file:

```
...
...
*|NET ABC 3.41881e-15
*|S (ABC:2 187.82 70.715)
*|I (ABC/Mmsbn:s ABC/Mmsbn s B 0.0 187.927 70.705)
rABC /210 ABC:2 XI95/NB:4 12.1333 $a=2.43e-14
$layer=metal1  $X=1.00 $Y=70.715 $X2=1.82 $Y2=70.715
...
```

Suitable Star RCXT settings are:

NETLIST_NODE_SECTION: YES

NETLIST_COMMENTED_PARAMS: YES

NETLIST_INSTANCE_SECTION: YES

NETLIST_PASSIVE_PARAMS: YES

NETLIST_CONNECT_SECTION: YES

NETLIST_TAIL_COMMENTS: YES

EXTRA_GEOMETRY_INFO: NODE

Sample Star RCXT DSPF file:

```
...
*|NET mout 0.00127675PF
*|I (XG015_cplx_1x/MN1:S XG015_cplx_1x/MN1 S B 0 1.852 0.467) //
    $llx=1.852 $lly=0.467 $urx=1.852 $ury=0.467 $lvl=31
*|S (mout:4 1.768 1.78) // $llx=1.718 $lly=1.75 $urx=1.818 $ury=1.81
    $lvl=1
...
```

- a metal layer mapping configuration file, to identify metal layers both in DSPF and GDSII, using the DSPF_MAP GSR keyword. This mapping is required by Totem to identify what layer is present in the DSPF for the sub-node section.

  The layer specification in the DSPF file typically can be found in the header of the LAYER_MAP section.

Derived base layers (between M1 & diffusion) can be defined from the GDS layer map file for GDS tracing, for signal nets, which only have base layers (below M1). You must add the GDS configuration file keyword “TRACE_THROUGH_CM 1" in the APLMMX configuration file keyword 'GDS_OPTION'.

Example in GDS layer map file:

```
rx_fin   temp 5:0   -
diffcon_t  temp 7:0  -
polycon_t  temp 8:0   -
...
BOOLEAN  polycon  cm  polycon_t&rx_fin
BOOLEAN  diffcon  cm  diffcon_t&rx_fin
```

Example in the DSPF_MAP file:

```
diffcon 181 trace
metal1 6 trace
```

- technology file, to specify EM limits, as well as physical parameters. See section "Setting Up EM Limits", page 4-169.
- optional signal net configuration file, to set transition times and toggle rates for each net (if input originates from a digital design)
- optional SPICE environment settings and files to support SPICE simulation

Setting Up for Transistor-level Signal EM Analysis

Totem needs first to be set in the right mode for signal EM analysis. Then, in addition to pointing to the input data, basic information needs to be supplied to extract signal nets from the GDSII file, by specifying the layer parameters used in the corresponding DSPF file to one metal layer of choice-- for example, the layer “M1” in the given technology file referring to metal1. After running, all EM modes can be displayed one after another by changing the setting in “ElectroMigration Color Map” dialog box, which is accessed using the “Set Color Range” button in the GUI. Also, by clicking on each wire segment, current information (its value, direction and type - bidir or unidir) can be obtained.

There are two types of signalEM analysis, with different input requirements and different accuracy of results:

- “static” analysis, using estimated current waveforms, a one step procedure that requires a combined file with both GSR and APLMMX keywords.
- “dynamic” analysis, using true SPICE waveforms. This is a two-step process of characterization, which also generates a basic GSR file needed for the analysis step, and then transient simulation.

These two analysis methods and their setup procedures are described in the following sections.

“Static” Estimated Waveform GSR Setup

To obtain correct current profiles Totem needs to know about transition times for rising and falling edges. These can be supplied in the form of global settings (most pessimistic) or more fine-tuned for individual signal nets through a special file-based setup. Signal nets that do not have individual settings automatically receive pessimistic settings through the following global GSR keywords:

- EM_SLEW_SIG_PERCENTAGE <value> (default 0.25 --25%)
- EM_SLEW_SIG_TIME <value> (default 0.8 ns)

All other nets receive their transition times from the file specified by the keyword

- EM_NET_INFO <filename>

which specifies a file that lists for each signal net its combined transition times (rise and fall). Note that wildcards are accepted in net name specifications. Other GSR keywords for estimated current waveform flow are described below. See section "GSR File Keywords", page C-773, for more details on each keyword. Note that there is no separate configuration file required for the static estimated current flow.

Required keywords

- FREQ <frequency> - used to specify the design global frequency. To specify a net-specific frequency, please use EM_NET_INFO file.
- INPUT_TRANSITION <value> - specifies global input slew rate for signal EM analysis. To specify net specific slew rate, please use EM_NET_INFO file.
- TECH_FILE <file>
- VDD_NETS {
  <vdd_name> <voltage>
  }
- GND_NETS {
  <gnd_name>
  }
- DSPF_NETLIST <file> - specifies a compliant DSPF netlist for EM analysis (same definition as used in APLMMX).
- DSPF_MAP <map_file> - specifies the file that defines mapping between the layers in the DSPF file and the layers in the Apache tech file. This mapping is required by Totem to identify what layers are present in the DSPF for the sub-node section. The format of this map file is as follows:

```
?#DSPF_PAR_METAL [ LVL | LAYER | <other_layer_parameter> ]?
?#USE_DSPF_LAYER_NAME ?
<tech_layer_name> <dspf_layer_param1> <dspf_layer_param2>
    ... ? trace?
```

where

trace: can be optionally specified if the layers is used for GDS2DB geometry tracing. If none of the layers are specified, APLMMX uses the lowest metal layer from TECH_FILE for GDS2DB geometry tracing.

DSPF_PAR_METAL < > : optional and need not be specified if the value is “lvl” or “layer”, since these two-layer parameters are automatically detected.

Note that DSPF_PAR_METAL is not required if the TRACE_FROM_ILINE keyword is set in the configuration file. APLMMX automatically uses the DSPF *|I line of the MOS transistor as the GDS2DB trace point, so there is no need to provide DSPF metal layer trace setup information.

Note that multiple DSPF layer IDs can be mapped to the same tech file layer.

USE_DSPF_LAYER_NAME : defines a DSPF layer name instead of layer ID in the file, because layer ID in DSPF file may vary when the process node migrates, but the layer name usually is not changed. To specify layer name instead of layer ID, a comment must be put into the file to introduce it, as follows:

```
#USE_DSPF_LAYER_NAME
<Tech_file_layer_name> <DSPF_layer_name> ? trace?
```

Example DSPF layer name specification:

```
#USE_DSPF_LAYER_NAME
```

```
M1 metal1  trace
M2 metal2  trace
...
```

where ‘metal1’ and ‘metal2’ are layer names defined in the LAYER_MAP section in the DSPF file.

Example for a DSPF map file based on Star-RC-based DSPF:

```
*|S (mout:4 1.768 1.78) // $llx=1.718 $lly=1.75 $urx=1.818 $ury=1.81 $lvl=1
```

Example for a DSPF map file based on Calibre XRC- based DSPF:

```
rXI95/NB/163 XI95/NB:101 XI95/NB:102 0.107376 $w=5.4e-07  $l=2.8e-07
+ $layer=metal1 $X=188.66 $Y=70.292 $X2=188.94 $Y2=70.292
```

- GDS_FILE <file> - specifies the GDSII file pointer. The same definition as the one used in APLMMX and GDSMMX.
- GDS_MAP_FILE <file> - specifies the GDSII layer mapping file between Totem Tech file and the GDSII file. Same definition as the one used in APLMMX and GDSMMX.
- TOP_CELL <cell_name> - specifies the top subckt hierarchy name. Same definition as the one used in APLMMX and GDSMMX.
- NMOS_MODEL_NAME <model_1> ... - specifies NMOS model names. Same definition as the one used in APLMMX.
- PMOS_MODEL_NAME <model_1> ... - specifies PMOS model names. Same definition as the one used in APLMMX.
- EM_NET_INFO <file> - specifies per signal net slew rate, toggle rate, frequency information for signal EM. Recommended for more accurate signal EM analysis.

Optional Keywords

- ANALYZE_NETS <net1> <net2> ... <netN> - specifies nets to be analyzed in signal EM analysis.
- ANALYZE_NETS_FILE <file> - only the nets listed in the specified file are analyzed in signal EM analysis.
- DEFAULT_PULLUP <resistance> - specifies the default equivalent pull-up resistance for nets without input drivers (primary input), to avoid applying unreasonable slew rates in analysis.
- DEFAULT_PULLDOWN <resistance> - specifies the default equivalent pull-down resistance for nets without input drivers (primary input), to avoid applying unreasonable slew rates in analysis.
- EM_SCALE_AVG <value> - specifies the scaling factor for the average simulation current density.
- EM_SCALE_RMS <value> -specifies the scaling factor for the RMS simulation current density.
- EM_SCALE_PEAK <value> - specifies the scaling factor for the Peak simulation current density.
- EM_SCALE_UNI <value> -specifies the scaling factor for the uni-directional simulation current density.
- EM_SCALE_UNI_AVG <value> -specifies the scaling factor for the uni-directional Average simulation current density.
- EM_SCALE_UNI_RMS <value> - specifies the scaling factor for the uni-directional RMS simulation current density.
- EM_SCALE_UNI_PEAK <value> - specifies the scaling factor for the uni-directional

Peak simulation current density.

- EM_TECH_RMS/AVG/PEAK <filename> -specifies the Average, RMS and Peak EM limits.
- ENABLE_BLECH [0 | 1] - turns on Blech effect polynomial-based EM rule checking. For more details on polynomial EM rules, see section "Technology File Keywords", page C-745.
- FILE_SIZE_REDUCTION - when set to “max”, generates much smaller optimized current profiles and significant file size reduction, with an accuracy loss less than 10%. If file size reduction is set to ‘off’, best accuracy is achieved.
- IGNORE_NETS <expr1> <expr2> **...** <exprN> - specifies a list of nets to be ignored during signal EM analysis.
- IGNORE_NETS_FILE <file> - specifies a file containing a list of nets to be ignored during signal EM analysis.
- LIMIT_CAP_RATIO <ratio> - allows ignoring nets with low load capacitance from EM analysis. The cap ratio is the ratio of the highest capacitance net to any other net. For example, assume that the capacitance of the largest net is 120 fF. If you specify a LIMIT_CAP_RATIO of 10, all nets with net Cload smaller than 12 fF are ignored in signal EM analysis.
- MERGE_WIRE [0 | 1] - tries to merge all wires that overlap, unless there are 45-degree wires in the design, when wire merging is disabled. For more details, see section "MERGE_WIRE", page C-846.
- NETLIST_XY_SCALE <scale> -specifies the netlist-to-GDS x,y scaling factor. Same definition as the one used in APLMMX.
- NET_NAME_CASE_SENSITIVE [0 | 1] - turns on net name case sensitivity during netlist parsing. Default is off.
- PRIMARY_OUTPUT_LOAD_CAPS - specifies the capacitance of the output driver nets, usually with a fanout of 1.
- PROBE_NO_PAR - specifies that the device name is to be used for probing. Supports signal EM analysis for the following Spice simulators: Nanosim_eldo, BDA/Ultra_sim/Spectre, Adit Spectre, Eldo/HSpice, Hsim/Nanosim/Finesim, and Smart_Spice/NSpice.

Also, several ‘metal’ keyword options in the in the technology file can be used, such as ‘Width_Based_EM’ and ‘length-based polynomial EM’. See the section "Electromigration Keywords", page C-825, for more information about relevant GSR options.

An example <*tsr*>*.gsr* file for estimated current flow is as follows:

```
FREQ 1e9
EM_NET_INFO des.info
#
DSPF_NETLIST dspf.spef
DSPF_MAP dspf.map
#
GDS_FILE design.gds
GDS_MAP_FILE design.map
#
VDD_NETS {
  vdd 1
}
GND_NETS {
```

```
  gnd
}
#
DEFAULT_PULLUP 100
DEFAULT_PULLDOWN 200
```

“Dynamic” True SPICE Simulation-based Waveform Setup

Totem sets up a SPICE simulation environment, which integrates everything needed for running SPICE and imports the SPICE-based device current results. In addition to the keywords described in estimated current waveform flow, the following are the most commonly used required and optional configuration and GSR keywords for characterization and dynamic simulation-based analysis. See section "GSR File Keywords", page C-773, for more details on each keyword.

Required APLMMX configuration file keywords

- DEVICE_MODEL_LIBRARY <file> <corner> - specifies the device model library file and the design corner to be used in SPICE simulation. Same definition as the one used in APLMMX.
- EMPRO_MODE dynamic - turns on transient SPICE simulation-based analysis flow.
- INCLUDE <file> - specifies the ‘include’ file for SPICE simulation. Same definition as the one used in APLMMX.
- SIM_OUTPUT_FILE <file> - specifies the simulation current waveform output file path. Same definition as the one used in APLMMX.
- SPICE_NETLIST <file> - specifies SPICE netlist for SPICE simulation if DSPF netlist is not directly used as netlist in SPICE simulation. The probe file is created based on the SPICE_NETLIST, if it is specified. Partial encrypted spice library is also supported
- SPICE_SIMULATOR <full_path_to_simulator> - specifies SPICE simulator binary path. Same definition as the one used in APLMMX.
- TEST_BENCH_FILE <test_bench_file> - specifies the test bench file to set up for SPICE simulation. Same definition as the one used in APLMMX. Note that setup for signal EM analysis needs the TEST_BENCH_FILE separately specified; it cannot be integrated into the DSPF file or other “include” file.

Optional APLMMX keywords

- APPLY_ALL_TRACEPOINT [<num_points> | all] - specifies the number of starting trace points to be dumped in the *.gds.config* file per net for physical net tracing. The default value and the minimum number of tracing points per net is 100. If set to “ALL”, APLMMX dumps all tracing points.
- CUSTOM_STATE_SIM_TIME - specifies the capture window; same function as in APLMMX
- FLAT_BUS_DELIMITER_MAP, FLAT_XHIER_MAP, FLAT_MHIER_MAP, FLAT_DHIER_MAP, FLAT_RHIER_MAP, FLAT_QHIER_MAP specify hierarchical SPICE netlist to flatten DSPF netlist name mapping rule. Refer to the APLMMX chapter for detailed setup procedures.
- IGNORE_OPERAND_STATE [0 | 1] - ignores operand states in output spcurrent file when linear Boolean operations are applied to custom states defined in keyword CUSTOM_STATE_SIM_TIME to reduce disk space and memory use when operand states are not used in analysis.
- IGNORE_PROBE_TYPE [`M D R Q GB PI PO BI`] - the specified device type or

types are ignored in modeling and analysis. M represents MOS type, D represents diode, R represents resistor, Q represents BJT, GB represents gate or base terminal (i2), PI represents primary input pin, PO represents primary output pin, and BI represents primary inout pin. Only one argument is allowed, but multiple device symbols can be specified without spaces between them.

- MULTI_STATE_FILE <file> - specifies the capture window to calculate the Average, RMS and Peak currents from simulation output file. Same definition as the one used in APLMMX.
- PROBE_THRESHOLD <thresh>
- PROCESS <process_corner>
- TEMP <temp>
- TOP_XCELL_NAME <top_xinst_name> - specifies the top subckt X-instantiation name. Same definition as the one used in APLMMX.
- GSR_IN_FILE <file> - specifies regular baseline GSR keywords, such as FREQ, INPUT_TRANSITION, TECH_FILE, DYNAMIC_SIMULATION_TIME, DYNAMIC_TIME_STEP, GSC_FILE in the file

Sample aplmmx.config file:

```
# turn on Transient SignalEM mode
EMPRO_MODE dynamic

# Probe level setting: (Default is 1)
# 1: Add probe 1st, 2nd, 3rd terminal of mos, bjt, any terminal
    of diode and resistor
# 2: Besides 1, Add probes for primary input if spice simulator
    can handle them

# Top cell name
TOP_CELL <top level cellname>
TOP_XCELL_NAME <name of instantiation of top cell in spice test bench>
VDD_PIN_NAME <vdd_pin1> <vdd_pin2> ...
GND_PIN_NAME <gnd_pin1> <gnd_pin2> ...
VDD <vdd_value1> <vdd_value2> ...

# If simulator does not support parameterized probes,
# disable parameterize probe generation using the keyword
PROBE_NO_PAR 1
TEST_BENCH_FILE <testbench>
SPICE_NETLIST <spice netlist>
DSPF_NETLIST <layout extracted dspf>

# Define Apache tech layer to dspf layer id/name map
DSPF_MAP <dspf map file>
TEMP <temperature>
PROCESS <process corner>

SIM_OUTPUT_FILE  <simulation_output_file>

# GSR file that contains baseline GSR keywords like FREQ,
    INPUT_TRANSITION, TECH_FILE, SIM_TIME
GSR_IN_FILE <sigEM.gsr file pointer>
```

```
# Define state name and capture window to use from simulation
CUSTOM_STATE_SIM_TIME {
TEST1 “” “” 0e-9 10e-9
}
DEVICE_MODEL_LIBRARY <pointer to spice model file> <corner>
SPICE_SIMULATOR <path to spice simulator>
GDS_FILE <path to gds>
GDS_MAP_FILE <path to gds layer map file>
GDS_OPTION {
# Include Contact Resistance in Extraction
CONTACT_RESISTANCE 1
# Optional: specify a snapping threshold distance for your device pins
SNAP_THRESHOLD_DISTANCE <distance_microns>
# Only use top level text for power and ground net tracing
USE_TOP_LEVEL_TEXT_ONLY 1
}
# Trace signal nets from multiple trace points specified in DSPF file
APPLY_ALL_TRACEPOINT 1

# Specifies to stop netlist tracing at the Spice subckt level,
# and not probe inside device model subckt.
PROBE_STOP_AT_MODEL_SUBCKT 1

# Specify names of blackboxed inductor cells, and metal layer to snap
    pins to.
# If a specific metal layer is not known, then specify
# LM for lowest metal, or HM for highest metal
INDUCTOR_MODEL_NAME {
<names of spiral inductor cells> <metal layer to snap pin to |LM|HM>
}

# For capacitor models (xinstance) in netlist, specify the model names
CAPACITOR_MODEL_NAME <mos_cap_model_name>
```

a. Sample 1 GSR_IN file (baseline GSR file):

```
-----------------------------------------------
  VDD_NETS
  {
    VDD 1.05
  }
  GND_NETS
  {
    VSS 0
  }
  DEF_TRUE_PATH_EXTENSION 0
  PGNET_HONOR_DEF_TYPE 1
  TEMPERATURE 125

  EM_TECH_AVG     ./user_data/tech/em_avg.limits
```

```
EM_TECH_PEAK    ./user_data/tech/em_peak.limits
EM_TECH_RMS     ./user_data/tech/em_rms.limits

ENABLE_BLECH 1
USE_DRAWN_WIDTH_FOR_EM_LOOKUP 1
EM_REPORT_PERCENTAGE 75
EM_REPORT_LINE_NUMBER 30000
EM_REPORT_PERCENTAGE 0
IGNORE_TECH_ERROR 1
TECH_FILE       ./user_data/tech/ totem_avg.tech
FREQ 1e+09

GSC_FILE        ./user_data/top.gsc
DYNAMIC_SIMULATION_TIME 1ns
DYNAMIC_TIME_STEP 10ps
-----------------------------------------------
```

b. Sample 2 GSR (cookbook)

```
# Main tech file specification
TECH_FILE       <main tech file>

# EM tech files for RMS/AVG/PEAK (Optional)
EM_TECH_RMS     <rms tech file>
EM_TECH_AVG     <avg tech file>
EM_TECH_PEAK    <peak tech file>

TEMPERATURE 125
FREQ 1e+08

VDD_NETS {
  VDD <vdd_value>
}
GND_NETS {
  VSS 0
}

LIB_FILES {
}

# Apl file specification from aplmmx run
APL_FILES {
    <pointer>/<cell_name>.spcurrent sigemcurrent
}

# EM model file from aplmmx run
MACRO_EM {
  DSPF <pointer>/<cel_namel>_dspf.rh
}

# GDS2DB config creates from aplmmx run
GDSII_FILE {
```

```
    <cell_name> <pointer>/<cell_name>.gds.conf full
}
NATIVE_GDS_SUPPORT 1

# Dynamic simulation time and resolution
DYNAMIC_SIMULATION_TIME 10ns
DYNAMIC_TIME_STEP 50ps

# Use drawn width for em calculation and lookup from tech
files
USE_DRAWN_WIDTH_FOR_EM_LOOKUP 1
USE_DRAWN_WIDTH_FOR_EM 1

# Use the Delta T of 5C from tech file. Rule should be
specified correctly in tech file
DELTA_T_RMS_EM 5

EM_DUMP_PERCENTAGE 0
EM_REPORT_LINE_NUMBER -1
```

c. Sample APLMMX configuration file

A sample APLMMX configuration file is shown below:

```
EMPRO_MODE dynamic

############################################################
# Top cell name
############################################################

TOP_CELL  demo

############################################################
# This should have the testbench instance name
############################################################

TOP_XCELL_NAME  Xtop

GDS_FILE        ./user_data/demo.gds
GDS_MAP_FILE    ./user_data/gds_layer.map
GDS_OPTION {
CONTACT_RESISTANCE 1
START_POINT_TOLERANCE 0.005
}

SPICE_NETLIST   ./user_data/demo.sp
DSPF_NETLIST    ./user_data/demo.dspf
FINGER_DELIMITER @
PROBE_STOP_AT_MODEL_SUBCKT 1

############################################################
# Include device model libraries here
```

```
############################################################

DEVICE_MODEL_LIBRARY ./user_data/log40.l corner
DEVICE_MODEL_LIBRARY ./user_data/log40.l fastmid
DEVICE_MODEL_LIBRARY ./user_data/hd_cell_242_ll.l  ff

DSPF_MAP ./user_data/signal_em_dspf.map

INCLUDE ./user_data/options.inc

SPICE_SIMULATOR hsim
SIM_OUTPUT_FILE ./user_data/top.fsdb
MULTI_STATE_FILE ./user_data/top_timeframe.inc
IGNORE_OVERLAP_POINT 1

GSR_IN_FILE ./user_data/base.gsr
```

Required GSR keywords

The initial APLMMX characterization step creates a basic GSR file and is saved in <*out_dir*>/<*design*>*.gsr.* You must then add the following keywords:

- VDD_PIN_NAME <vdd1> <vdd1> ... <vddN> - specifies power domain names. Same as defined in aplmmx.
- GND_PIN_NAME <gnd1> <gnd2> ... <gndN> - specifies ground domain names. Same as defined in aplmmx.
- VDD <vdd1_volt_level> <vdd2_volt_level> ... <vddN_volt_level> - specifies power domain nominal voltage level. Same as defined in aplmmx.

Optional GSR keywords

Additional GSR keywords can be added as necessary. See previous sections or Appendix C for details.

- EM_TECH_RMS/AVG/PEAK <filename> -
- DEFAULT_PULLUP <resistance>
- DEFAULT_PULLDOWN <resistance>
- PRIMARY_OUTPUT_LOAD_CAPS
- EM_SCALE_AVG <value>
- EM_SCALE_RMS <value>
- EM_SCALE_PEAK <value>
- EM_SCALE_UNI <value>
- EM_SCALE_UNI_AVG <value>
- EM_SCALE_UNI_RMS <value>
- EM_SCALE_UNI_PEAK <value>
- ENABLE_BLECH [0 | 1]

Sample dynamic GSR file

A sample <*tsr*>*.gsr* file for probe-based dynamic flow is as follows:

```
EMPRO_MODE dynamic
DYNAMIC_SIMULATION_TIME 10ns
DYNAMIC_TIME_STEP 10ps
GSC_FILE top.gsc
GDS_OPTION {
    CONTACT_RESISTANCE 1
}
MULTI_STATE_FILE top_state.txt
TOP_CELL top
#
VDD_PIN_NAME VDD
GND_PIN_NAME VSS
VDD 1.1
#
DSPF_NETLIST dspf.sp
DSPF_MAP dspf.map
#
GDS_FILE des.gds
GDS_MAP_FILE gds.map
#
TEST_BENCH_FILE dut.sp
#
DEVICE_MODEL_LIBRARY mos.lib TYP
DEVICE_MODEL_LIBRARY res.lib tt
#
DEFAULT_PULLUP 100
DEFAULT_PULLDOWN 100
#
TECH_FILE des.tech
#
FREQ 70e6
TEMPERATURE 25
PRIMARY_OUTPUT_LOAD_CAPS {
    DEFAULT_LOAD 10f VDD
    DEFAULT_LOAD 10f VSS
}
```

Running Transistor-level Signal EM Analysis

Estimated Waveform Method

To run signal EM, a TCL run file, such as the following sample, can be used:

```
setup analysis_mode -signalEM
import gsr <tsr>.gsr
setup design
perform analysis -signalEM
```

True SPICE Simulation-based Method

There are two ways of running EM analysis using SPICE simulation waveforms:

a. run SPICE simulation directly inside Totem (default)

b. run SPICE simulation using a third-party simulator flow

These methods are described below.

Run SPICE Simulation Directly Inside Totem

To run SPICE simulation inside Totem signal EM analysis, after performing the setup steps described in the previous section, execute the following command with a TCL command file :

```
totem2 -f cmd_signalEM.tcl
```

Example command file:

```
setup analysis_mode signalEM
import gsr Design_ABC.gsr
setup design
#cell.gsc specifies which state to use in analysis.
import gsc <cell>.gsc
perform analysis -signalEM -transient
```

The last line of the command file also automatically invokes the following functions:

```
perform extraction -signal -r -c
perform emcheck
```

If the GSR keyword “ENABLE_EM_PROCESS” is set to 1, the additional two commands are *not* automatically performed. Its default value is 0, which runs the added EM steps.

Run SPICE Simulation Using a Third-party Simulator Flow

This procedure is similar to the APLMMX third-party simulator characterization flow, as described in the following steps:

1. Create an APLMMX configuration file with specific keywords for signalEM analysis, as described in the previous section. The example APLMMX configuration file in the following section indicates those keywords that are specific to the signalEM flow.
2. Create a probe file for the third-party simulator

   ```
   aplmmx -sig -s ?-o <out_dir>? <aplmmx_config_file>
   ```

   The output probe file is placed in *<output_directory>/EMP/<design>.probe.*
3. Run third-party simulator simulation. You must include the probe file generated from step 2 in your SPICE deck and run simulation.
4. Read back the simulation output waveform file and create models for signal EM analysis using the command:

   ```
   aplmmx -sig -n ?-o <out_dir>? <aplmmx_config_file>
   ```

   Specify the SIM_OUTPUT_FILE in the APLMMX configuration file.
5. Run signal EM analysis with the command:

   ```
   totem2 -f cmd_signalEM.tcl
   ```

   The contents of the *cmd_signalEM.tcl* file are the same as in the example above in the section "Run SPICE Simulation Directly Inside Totem", page 11-342. The basic GSR file created in this step is saved in *<out_dir>/<design>.gsr.*

Tips for Using the Totem GUI

In the GUI are several useful ways to analyze EM -related results, as described below.

1. Current and EM information can be obtained by clicking on any metal wire, which displays wire data in the log window:

```
Bidirectional current:
RMS 5.989e-05 A >
EM percentage 13.6452% (EM current limit: 4.389e-04 A)
Peak 1.157e-03 A >
EM percentage 80.796% (EM current limit: 1.432e-03 A)
Avg  4.574e-06 A >
EM percentage 1.04211% (EM current limit: 4.389e-04 A)
```

or, in case one current exceeds its opponent by more than 20% : (uni-directional, in the example below, the pull-up current)

```
Pull-up current:
RMS 6.104e-05 A <
EM percentage 22.0185% (EM current limit: 2.772e-04 A)
Peak 1.179e-03 A <
EM percentage 36.7285% (EM current limit: 3.211e-03 A)
Avg  4.661e-06 A <
EM percentage 1.6816% (EM current limit: 2.772e-04 A)
```

Current and EM information can be also obtained by clicking on a via:

```
Pull-up current:   (scale factors none)
RMS 4.910e-04 A o
Peak 6.666e-04 A o
Avg  3.704e-04 A o
EM percentage 74.0711%  (EM current limit: 5.000e-04 A)
```

To give a practical example, for a typical inverter, as shown in Figure 11-1, with its geometry resembling a “T” on its side, the shorter n1-n2 and n2-n3 segments are connected to either a PMOS or a NMOS transistor. Each of these shorter segments has uni-directional current reported. The load segment n2-n6 has bi-directional current.

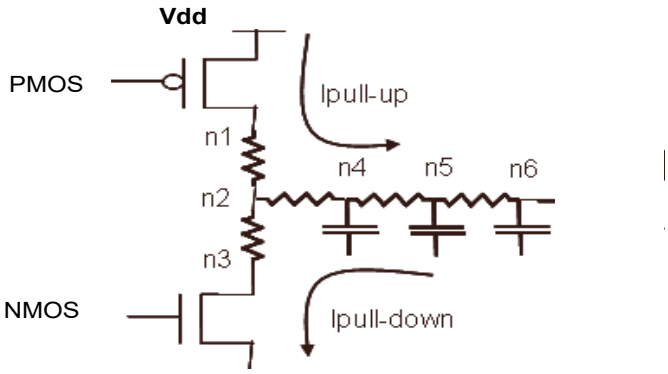


Figure 11-1 Current in a typical inverter

2. You can filter the displayed nets by going into **View->Nets.** Wild cards are accepted. This is helpful when the design is big and may contain multiple hierarchical levels, which are now flattened. By name matching you can focus on the original instances.
3. In probe-based simulation flow, Totem2 can display a transistor pin current map, TCUR, based on various EM modes defined by users. These maps display distribution and magnitude of the current sources. You can also select the pin and use the popup menu with the middle mouse button to plot current waveforms for the selected transistor. In addition, using the **Results -> List of Signal Nets**

Transistor Pin Current, you can display a table of signal nets analyzed, and transistor pins with their AVG, RMS, and PEAK current values. You can also plot the current waveform of transistor pins in the GUI, as shown in Figure 11-2.

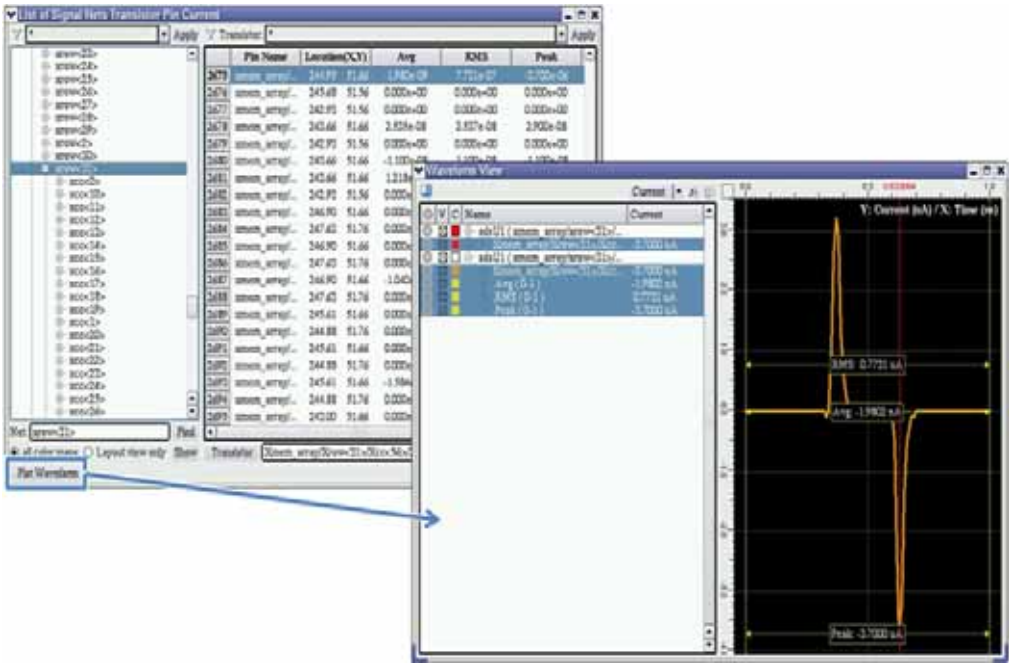


Figure 11-2 Displaying transistor pin waveforms

Analyzing EM Results

Text Reports

After Totem has completed its signal EM analysis, the following text reports are available:

- *adsRpt/apache.sigem.info* file - indicates if a signal net is driven, and if so, by how many PU PD devices (typically transistors).
- *adsRpt/<top_cell>.disconnected_nets* - file generated when Totem gives an ANL-411 message, which means that due to missing connectivity (most susceptible primary inputs) an EM analysis was not successful for these nets.
- *adsRpt/SignalEM/<top_cell>.rms_em.worst*
- *adsRpt/SignalEM/<top_cell>.peak_em.worst*
- *adsRpt/SignalEM/<top_cell>.avg_em.worst* - contains a list of potential violations. It only becomes active if the violation exceeds 100% (a true violation). To dump out info about current density and the impact on their respective limits, lower the EM_REPORT_PERCENTAGE to the level of interest (such as 10%)

Multi-cycle and Multi-state results

Totem provides analysis for multi-cycle and multi-state conditions set up in the GSC_FILE. The default average/RMS/Peak current and EM values are calculated based on the worst cycle among all states (the average and RMS current and EM are calculated based on each state not based on the whole simulation time).

The Tcl command 'report em_state' allows you to load and import the results from the worst state of specified state(s) back into the text/GUI reports.

Debugging tips

If the results do not look reasonable, first check the file *adsRpt/totem.err* for the following errors:

- ANL-411 - indicates some of the signal nets could not be properly analyzed. This could be due to undriven nets (check the *adsRpt/apache.sigem.info* file, as well as *adsRpt/<top_cell>.disconnected nets*), or the existence of isolated regions due to missing connectivity.
- INS-620/621 - indicates complete failure of the analysis.

These additional files may also contain clues:

- *empro.log* - may contain warnings / errors coming from a pre-analysis run of the supplied DSPF netlist.
- *gds2def.log* - may contain warnings/ errors coming from the GDSII file extraction.
- if you run the SPICE probe-based flow, check the file **adsRpt/Log/<top_cell>_spice.log,* which may describe other problems, such as license queuing required for SPICE simulation only.

The last step is to verify the format requirements of the supplied DSPF netlist used with Totem.

DCOP support for signal EM flow

Totem signal EM flow supports enabling DCOP (DC operating point). The usage is same as for static IR analysis using a DCOP file.

Usage:

```
DEVICE_CURRENT_FILE <DC_Op_file> ?[ 0| 1]?(Default : 1)
```

With `DEVICE_CURRENT_FILE <file> 1`, aplmmx checks the circuit connections to make sure the current polarity is compatible with the simulator setting. With `DEVICE_CURRENT_FILE <file> 0`, it uses the current polarity as it is from the DC Op file.

For DCOP-driven analysis, signal EM checks driver/receiver current balance. You specify a threshold for filtering nets with high imbalance. Signal EM calculates imbalance as $|I_{driver} - I_{receiver}|$. When imbalance is less than the threshold specified, it scales the receiver current to force balance. If imbalance exceeds the threshold, it drops the net.

The dropped nets due to current imbalance filtering are listed in the file `adsRpt/em_dropped_nets.rpt`.

Usage:

```
DRIVER_RECEIVER_IMBALANCE_CHECK 1 ?[threshold]?
```

Imbalance checking is ON with a threshold of 10uA by default. Note that this setting is just a sanity check and may not affect vectorless or vector-based Signal EM analyses.

If `DEVICE_CURRENT_FILE,` is missing some DC currents, signal EM will report them in `the file adsRpt/mmx.xtor_miss_dcop.`

DMP (Distributed Machine Processing) Support for Signal EM

Vector-based Signal EM simulation supports DMP using LSF, SSH and SGE gridtypes. To invoke DMP for SEM, specify a DMP configuration file in Tcl one of these two ways:

1. Declare using “`setup analysis_mode`” as the first Tcl command:
   `setup analysis_mode signalEM -dmp <dmp config file>`

```
...
perform analysis -signalEM
```

2. Declare using “`setup dmp`” before perform signal EM analysis:

```
setup dmp -dmp <dmp config file>
...
perform analysis -signalEM
```

Specify DMP settings in the DMP config file, using the same format and content as for DMP simulation, but with the addition of OPTION_CHECKOUT:

```
NUMBER_OF_JOBS <number>
GRID_TYPE LSF/SSH/SGE
OPTION_CHECKOUT <TM|RH_DMP> # optional, tool checks out
                            #multiple totem by default

# List of LSF hosts (Optional)
CANDIDATE_HOST_LIST {
 Host1
 Host2
 ...
}
```

The GSR MULTI_THREAD *n* can be used to limit SEM to *n* threads per host. By default, SEM will use all cores on each host.

- For SSH mode, if CANDIDATE_HOST_LIST is not defined, tool will run multiple jobs on master machine. Also, host-names can be repeated in the list and jobs will be assigned in the order in which host-names are specified in list.
- GSR keyword “MULTI_THREAD <> <>” can still be used to control threads per machine. If not specified, tool will use all processors per machine
- Even for single machine, 4-threads run, DMP flow in 18.1 is around 20%~30% faster than Totem 17.1.
- Host-names can be repeated in the list and job assignment will occur in the order in which host-names are enlisted.

As of now, only following optional GSR keywords are supported for Vectorbased signal EM DMP flow,

```
EM_USE_DUTY_RATIO 2
EM_SCALE_(AVG|RMS|PEAK)/ EM_SCALE_UNI_(AVG|RMS|PEAK)
EM_BI_UNI_RATIO
SEM_SCALE_VECTORLESS_CURRENT_ONLY
SEM_RECOVERY_FACTOR
PROBD_NODE_FILE   ### OR TCL “probe add”
```

Special peak current calculation or flows such as longpeak, multiple_layoutport shorting, are not supported as of now.

Methodology for Cell-level Signal EM Analysis

Overview

Very similar to transistor-level signal EM analysis, Totem cell-level signal EM analysis estimates the I_{avg}, I_{rms}, and I_{peak} for every wire in the design, especially those belonging to the clock tree network, and compares them against their respective EM limits. The

signal EM flow in Totem estimates the different current values based on cell design parameters. Totem analyzes signal EM for all wires in a design -- both inside cells (intra-cell) and between cells (inter-cell).

The average current flowing in every wire (I_{avg}) is estimated from the capacitive load seen at the output of each cell, its operating frequency, its supply voltage, and toggle rate (which is defined as 2.0, or 200% for clock pins).

$$I_{avg} = f \text{ (capacitive load, frequency, toggle rate, supply voltage)}$$

Since the current flowing in a signal net is usually bi-directional, the true average current is usually very small. Therefore in Totem a rectified average current is calculated. Estimation of RMS current requires an understanding of the current profile that is seen during the charging or discharging process in any output net. The RMS current is defined as follows, where the current is integrated from 0 to T_{clk}, the clock period:

$$Irms = \sqrt{\frac{\int i^2(t)dt}{Tclk}}$$

Totem by default approximates the switching current profile at the output of a cell using a polynomial-based profile whose shape and size depend on the average current of the net and the transition times (for rising or falling edges). Once the current profile is constructed, Irms can be calculated using the above equation. You also have the choice of creating a different polynomial- based waveform for estimation of Irms. Using that profile, Totem determines the peak current (I_{peak}).

The currents (I_{avg}, I_{rms}, and I_{peak}) are estimated for every signal wire segment in the design. The current values are typically highest closest to the driver cell and decrease gradually as distance from the drivers increases. Totem reads in the signal net routing and geometry information, along with signal net parasitic data, to determine the current values in each net, from its driver(s) to its receiver(s).

Once the current values are determined for each of these modes, the current density is estimated at each wire segment and via and compared to the relevant EM limits. The EM limits can be specified in the technology file as dependent on physical parameters such as the width of the wire, size of the via, and temperature of the die.

Input Data Requirements

An inter-cell EM analysis (wires outside of cells) requires the same data preparation as a regular Totem static analysis. Some of the key data requirements for an inter-cell EM analysis include:

- Routed design netlist in the DEF format
- Signal parasitic information in the DSPF format
- Timing information (frequency, slew, clock nets)
- VCD or instance-specific toggle information [optional]
- Technology file with specified EM limits

Setup for Cell-level Signal EM Analysis

The command files and setup needed for signal EM analysis are similar to those needed for static analysis. Totem estimates three different measures of current for every net in a single run. You can display different EM modes in the GUI by changing setting in ‘ElectroMigration Color Map’ dialog box, which is accessed using the ‘Set Color Range’ button in the GUI.

Waveform Specifications

The current profile used to estimate the RMS and peak current can be either a triangular or a polynomial waveform. You can select the mode using the GSR keyword (polynomial is the default):

```
EM_WAVEFORM_TYPE [polynomial | triangle ]
```

Besides waveform types, you can set the slew type with a GSR keyword to estimate the RMS and peak current, as follows (default is Min):

```
EM_USE_SLEW [ Max | Min | Average]
```

When set to 'Max', Totem uses the maximum available slew value (transition time) for the net. If set to 'Min', the minimum available slew value is used. If 'Average' is specified, an average of the minimum and maximum slew values are used (in ns units). Using the 'Min' option is recommended.

The current profiles that are constructed depend on the output slew and load at every net, among other parameters. Totem gets slew information for signal and clock nets from the STA data provided and RC loading data. For cells missing the STA and RC data, you can control the profile construction using the following keywords (which also have default values to cover cells with missing slew data).

```
EM_SLEW_SIG_PERCENTAGE    p1 (default 0.25)
EM_SLEW_SIG_TIME          t1 (default 0.8 ns)
EM_SLEW_CLK_PERCENTAGE    p2 (default 0.1)
EM_SLEW_CLK_TIME          t2 (default 0.3 ns)
```

For signal nets, slew is calculated as

```
min(p1*clock_period, t1)
```

For clock nets, slew is calculated as

```
min(p2*clock_period, t2)
```

For signal nets missing load information in the SPEF/DSPF data, Totem estimates the missing slew information based on routing data. The GSR keyword TOGGLE_RATE should be set in order to assign a default toggle rate to signal and clock nets.

```
TOGGLE_RATE <signal toggle rate> <clock toggle rate>
```

EM Limits

For signal EM analysis on high technology designs, the peak current EM spec includes a DUTY_RATIO value for each net. To include the DUTY_RATIO data in Totem signal EM analysis, set the following GSR keyword to 1 (supported for signal EM analysis only):

```
EM_USE_DUTY_RATIO 1 (default 0)
```

In addition to the parameters needed for regular analysis set in the Totem technology file, the following additional EM limits can be set in the GSR:

```
EM_TECH_RMS <tech_file_nameA>
EM_TECH_AVG  <tech_file_nameB>
EM_TECH_PEAK <tech_file_nameC>
```

The format of these tech files is the same as a standard Totem tech file, except that only EM-related data are included. Three formats are acceptable, as follows:

```
metal <metal_layer> {
    EM <limit_value>
}
```

or

```
metal <matal_layer> {
WIDTH_BASED_EM {
    width { <silicon_width1> ... <silicon_widthN>}
```

```
        em { <limit_per_u_sil_width1> ... <limit_per_u_sil_widthN+1>}
    }
  or
    via <via_name> {
    EM <limit_value>
    }
```

The list of EM limit values for WIDTH_BASED_EM correspond to the list of silicon widths, with an additional EM value for any wire widths bigger than the largest width in the list.

These special tech files should ***only contain EM limits*****.** All other tech file information, such as resistivity, thickness, and EM_adjust, should be specified in the regular Totem tech file, since they are the same across all EM modes (signal and power EM). If the additional tech files for EM limits are not provided, the EM limits in the regular tech file are used.

If the special tech files for an EM mode contains no via-related information, via EM will not be checked for that EM mode. For example, the signal EM specification has no Peak or RMS EM limit for vias.

Custom Current File

You can create a custom current file to specify current at signal net driving points using the GSR keyword 'EM_CUSTOM_CURRENT_FILE <filename>'. The format of the custom current file is as follows:

```
<net_name> INST <inst_name> <pin_name> <avg> <rms> <peak>
...
<net_name> CELL <cell_name> <pin_name> <avg> <rms> <peak>
...
```

where

net_name: specifies the net name

INST <inst_name>: specifies the instance name

pin_name: specifies the pin name

<avg> <rms> <peak>: specifies the current value for each type of current, in Amps. Current types that are not to be set should be given values of '-1'.

Example custom current file:

```
net1 INST ANA1 VREGO -1 -1  1e-3
net2 INST ANA1 SIGPIN -1 -1 2e-3
```

The example sets PEAK current to '1e-3' for net1 and '2e-3' for net2 at driving points VREGO and SIGPIN, respectively. Other current values are unspecified.

Using the custom current file you can also limit EM analysis to those nets that are specified. In above example, only nets 'net1' and 'net2' would be reported if you set the GSR keyword EM_CCF_ONLY to 1. To specify the custom current file.

Running Signal EM Analysis

The signal EM flow is divided into different command steps, as described below, so that you have access to the results of each step individually:

```
# setting the analysis mode to signal EM
setup analysis_mode signalEM

# importing the GSR
import gsr <gsr_file_name>
```

```
# setting up the design
setup design

# power calc step to find the toggle rate of instances
perform pwrcalc

# extracting signal nets using the SPEF file
perform extraction -signal

# performing signal EM analysis
perform analysis -signalEM

# creating EM analysis reports
perform emcheck
```

Note: The 'setup analysis_mode' step must be first for signal EM, as opposed to the power EM analysis procedure, when it is run after 'setup design'.

Tips for Using the Totem GUI

In the GUI, there are several useful ways to analyze EM-related results:

- Current information can be obtained by clicking on any metal wire:

```
Bidirectional current:
RMS 9.061e-02 A
Peak 1.812e-01 A
Avg 4.530e-02 A
```

- To check current EM_MODE, use the command:

```
gsr get EM_MODE
```

- To change the EM_MODE, use the command:

```
gsr set EM_MODE [ rms | avg | peak ]
```

  or use the appropriate GUI button in the EM Color Map dialog.

- To view nets, select from the menu **View->Nets**
- To highlight instances related to a net, use the command :

```
select add [get instbynet <netname>]
```

- Also, the command

```
select add [get instbynet -driver <netname>] -color red
```

  can be used to highlight a driver in a different color, such as red.

Analyzing Results

After completing signal EM analysis, the following text reports and maps are available:

- *adsRpt/apache.sigem.info* file

  Check signal nets that have no loading capacitance, timing, or driver instance information.

- *adsRpt/Totem.err* file

Reports all signal EM flow setup and analysis errors.

- *adsRpt/SignalEM/<design_name>.rms_em.worst* file

  Reports the highest 1000 J_{rms} (RMS current density) violations.

  To report more than 1000 violations, use the GSR keyword:

```
EM_REPORT_LINE_NUMBER [ Max_line]
```

Also, EM_REPORT_PERCENTAGE (default 100) can be used to specify the threshold for reporting violations, the same as Power EM. Sometimes you must use a lower EM_REPORT_PERCENTAGE to get any entries in the **em.worst* output files.

- *adsRpt/SignalEM/<design_name>.avg_em.worst*

  Reports highest 1000 J_{avg} (average current density) violations.

- *adsRpt/SignalEM/<design_name>.peak_em.worst*

  Reports highest 1000 J_{peak} (peak current density) violations.

Debugging Tips

- When encountering unexpected EM results, first check the *adsRpt/apache.err* file for error messages.
- Check the *adsRpt/SignalEM/<design>.em.warn* file.
- Use the 'IGNORE_NETS { <netname> ... }' GSR keyword to eliminate from consideration nets that are not of interest and reduce output data.
- Use the GSR keyword:

```
ANALYZE_NETS {
    <net_name_1>
    <net_name_2>
    ...
}
```

  to only process specific nets for quick probing and fast turnaround.

- To perform incremental debugging in the GUI or a TCL shell, use the following:

```
setup design
...
gsr set ANALYZE_NETS {net1}
perform analysis -signalEM
gsr set ANALYZE_NETS {net2}
perform analysis -signalEM
...
```

Integrated Power and SignalEM flow

Totem now supports Power-integrity and Signal-EM analysis in an integrated fashion across all steps, that is from aplmmx to final analysis step.

Initially for running the aplmmx in the integrated mode, execute the commands as follows:

1. To generate probes (for both power and signal nets):

   `aplmmx -integrated -s <aplmmx.config>`

   The integrated probe file will be saved as `'.apache/APL/<cellName>.probe'.`

2. To create electrical model for powerEM and signalEM run:

   `aplmmx -integrated -n <aplmmx.config>`

   Following files are generated after electrical modeling in integrated mode:

   - spcurrent and sigemcurrent for power and signal flows, cdev for power EM
   - dspf.rh for signal nets
   - Consolidated `<cell_name>.gdsmmx` file with power and signal nets.

To perform the Integrated Power and Signal EM simulation, generated GSR and GDS config file can be used in a run command file as below:

```
setup analysis_mode power_signal
setup design <gsr_file>
perform pwrcalc
perform extraction -signal -power -ground -r -c
perform analysis -dynamic
perform analysis -signalEM
perform emcheck
```

After simulation the result files get dumped separately in directories `'adsRpt/Dynamic'` and 'adsRpt/SignalEM' for Dynamic and SignalEM run respectively.

GUI Features:

- EM map simultaneously shows Power and Signal EM violations in the same GUI.
- Option to get results file for EM and IR drop.

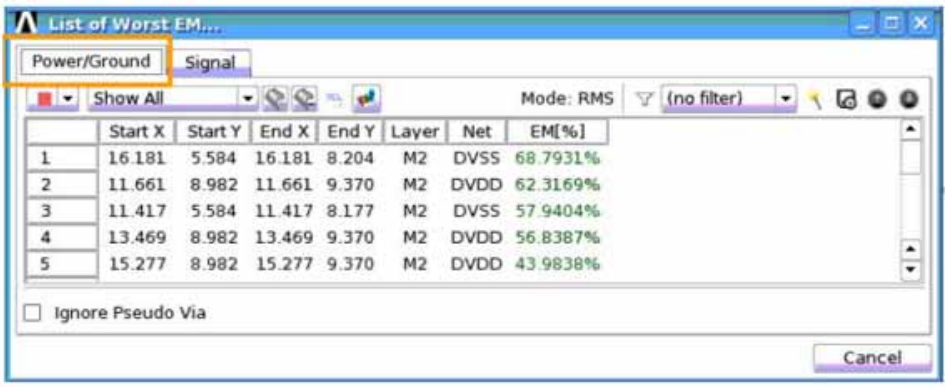


Figure 11-1 List of Worst EM

Chapter 12
Pathfinder™ ESD Analysis

Introduction - The ESD Problem

Electrostatic Discharge (ESD) is a common reliability problem in chip design that involves very high discharge currents that can fatally damage sensitive circuit elements, or adversely affect their operation. As many as 35% of total IC field failures are reported as ESD-induced. ESD represents a serious problem for the electronics industry, which must insure proper operation of chips during all phases of manufacture, test, packaging, and installation into products. Serious semiconductor problems caused by ESD are: oxide punch-through, junction burnout, and metalization burnout. ESD phenomena are typically classified into three discharge/test models:

- Human Body Model (HBM), an ESD discharging event occurring when a charged human body contacts an electronic device directly
- Machine Model (MM), an ESD pulsing event when charged machinery discharges when touching an IC during testing
- Charged-device Model (CDM), a self-induced discharging ESD event when un-grounded electronic parts are charged up during assembly and then discharge through a ground pin.

These three ESD modes and their discharge waveforms are shown in Figure 12-1.

The traditional approach to ESD design and verification consists of several steps:

1. design at circuit level and verify on test chips
2. integrate ESD test cells into I/O cells of the SoC
3. review results by ESD experts before tape-out.

These empirical approaches are error-prone and involve high costs when the chip fails due to ESD problems.

A further complication is that ESD is a time-domain transient behavior, and has a very short duration. Some ESD failures can only be captured in time-domain circuit-level transient simulation. Unfortunately, there is no circuit-level simulator that can do dynamic ESD simulation at this time. Pure SPICE-level simulation fails in properly handling negative resistance from the snap-back behavior of clamp devices under ESD stress. The tools that designers can use now are SPICE analysis plus a device simulator. Capacity and long run-times are significant limitations of such solutions, so they can only be applied to a very small portion of a design.

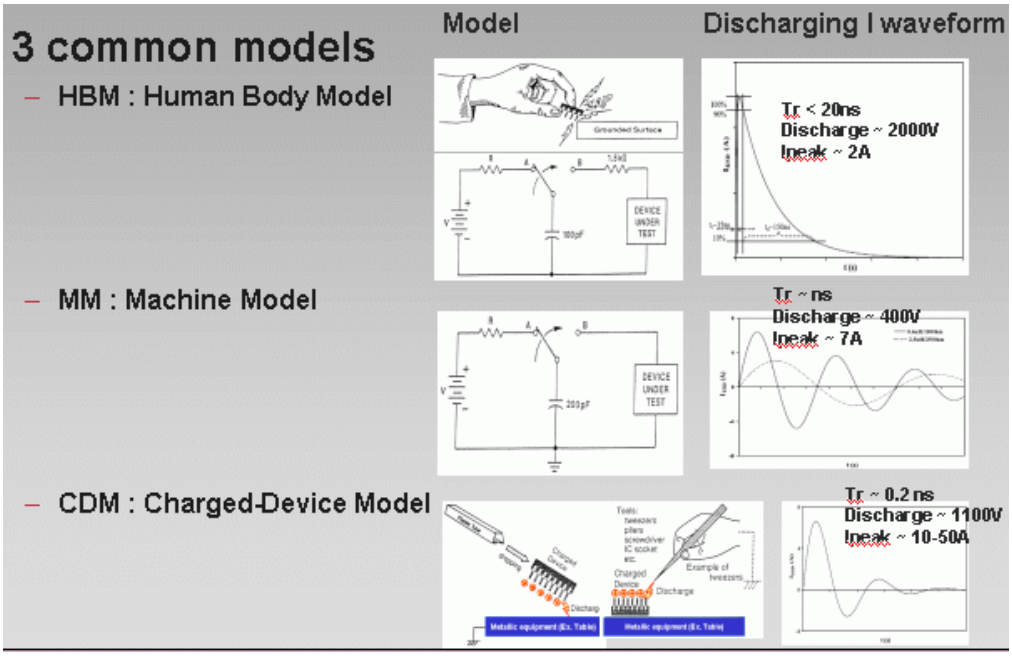


Figure 12-1 Three common ESD event models and discharge waveforms

Chip design has benefited from EDA solutions in many areas, but an EDA solution for ESD analysis has been a missing piece of the complete EDA design and analysis flow. Furthermore, ESD problems are now occurring more often due to advanced processes, larger design sizes, and mixed-signal and low power design methodologies.

Dealing with the difficulty of ESD transient simulation and long run times involved for covering all the discharging paths, a set of static techniques have been developed, primarily checking the resistance of the potential discharge paths, assuming ESD protection clamps are turned on. The RC time constant that controls the turn-on order of the clamps is constrained by the threshold resistance value, so the assumed actual resistance for the discharge path from a bump to another bump, or from a core instance to a clamp, can be calculated reasonably. If the resistance of a discharge path is much larger than the threshold resistance value, it often indicates a potential problem that will show up during the ESD testing, since there is no obvious discharge path for a bump in HBM/MM condition, or for a large custom macro in CDM condition.

ESD Analysis

Overview

PathFinder™-S provides a comprehensive set of technologies for performing ESD verification and analysis for both analog and SoC designs. The PathFinder tool uses static techniques (rule-based checks), while Pathfinder-D uses dynamic transient simulation techniques for ESD analysis. Differences in PathFinder and PathFinder-D analyses are shown in Figure 12-2.

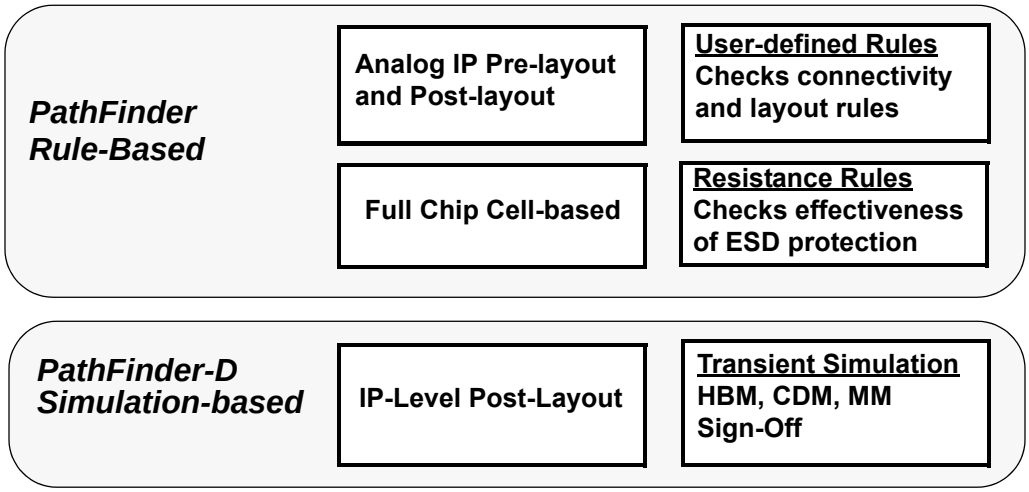


Figure 12-2 ESD analysis and verification coverage of PathFinder

PathFinder can be used for design planning (such as clamp cell placement, number of clamp cells, and design of clamp cells) or for sign-off (such as meeting discharge guidelines, and passing device-level threshold criteria). It can be used both on pre-layout circuits and also on post-layout designs. The checks provided by PathFinder are applicable for all three primary modes of electrostatic discharge events:

- Human Body Model (HBM)
- Machine Model (MM)
- Charged Device Model (CDM)

PathFinder can be used to evaluate potential ESD problems in both a layout topology environment and also in a netlist environment, which are both considered in this document.

The capabilities provided in PathFinder can be classified into two broad categories:

- Verification of the placement of clamp circuits with respect to each other, the pads, and other cells in the design.
- Verification of the design of individual clamp circuits (at the schematic or layout level).

Totem PathFinder also provides current density analysis for user-specified B2B, P2P, and P2PM ESD paths.

One set of checks evaluates the connectivity and topology of all elements of the circuit. In addition, resistance-based checks calculate accurate resistance values between the pads and the clamps, the clamps to the clamps, or from the clamps to other cells in the design. This capability is described in this chapter.

Clamp Cell Definition

Clamp Files - Cell

For all types of ESD analysis, you must specify I-V curves for the relevant diodes/clamps. A clamp cell can be defined either in its own clamp cell file or in a combined clamp cell/rule file. In a manually-generated clamp file, you can also specify the clamp I-V curve as in the following example:

```
BEGIN_CLAMP_CELL
    NAME  <cell name>
    TYPE <user_type_name>
    ? PIN [<pin_name> | NA] [<x_loc> <y_loc>]
        [ BOTTOM | TOP |<layername>] <locID> ?
    ?PIN_GROUP [POWER | GROUND | SIGNAL]
    ? XTOR <xtorName>:<net> <x_loc> <y_loc> <layer> <locID> ?
        ...
    ? RON <clamp_On_Resistance_Ohms> ?
    ? ESD_PIN_PAIR <loc_ID1> <loc_ID2> [
        [ <Ron> | [ <Ron+>|OFF] [<Ron->|OFF] ]?
        <I-V_clamp_name> ]? IS=<iscale>
    ? IMAX <I1> [<I2>] ?
    ? VMAX <V1> [<V2>] ?
    ? RAIL_VOLTAGE <voltage> ?
    ? VDIFF <Pin1> <Pin2> <vmin> <vmax>
END_CLAMP_CELL

BEGIN_CLAMP_IV
  NAME <name of I-V clamp>
  Ron <Ron+> [<Ron->]
  VT1 <VT1+> [<VT1->]
  VH <VH+> [<VH->]
  ROFF <Roff+> [<Roff->]
  ION2   0.9
  RON2   0.1
  IT2    3.0
  RT2    1.0
  VT2    0.5
END_CLAMP_IV
```

where

NAME: specifies the name string to identify this I-V data.

TYPE : user-specified name for the clamp cell type

PIN : specifies clamp cell pins by either <pin name>, by `<x_loc> <y_loc>,` by BOTTOM /TOP layer or <layername>, or by <locID>.
Note that only "-" should be used as a placeholder for clamp pin names, and *not* "NA"; "NA" is treated the same as any other pin name in the cell. If "-" is specified, PF-S gets the pin location from wires/vias of the closest top level domain net for the BOTTOM/TOP layer or <layername> specified. If BOTTOM or TOP is chosen, the specified <x/y> location is snapped to the closest bottom or top layer of the <pin_name> net defined in LEF. If only a legal pin name is specified and no other information, by default a node with near minimum resistance to the pads is chosen. If no clamp cell pins are specified, one pin per domain is identified automatically that has near the minimum resistance to a pad. If only a subset of all clamp cell pins is specified, only the pins specified are used for ESD check. If a legal pin name is given, then this is also the default <locID> used in the ESD_PIN_PAIR option (defined below).

PIN_GROUP : Enables shorting/grouping of the specified pin nodes. PF-S will find the instances of a cell, and examine the net connections from those instances. The pins connected to VDD_NETS in GSR are added to the

'POWER' pin group and similarly pins connected to GND_NETS and ESD_SIGNAL_NETS are grouped as ‘GROUND’ and ‘SIGNAL’ pins respectively.

XTOR: defines ESD clamp device connections on transistors for MMX blocks. Options are defined the same as for PIN, except that ‘:<net>’ is the name of the net that the transistor pin is connected to.

Ron : resistance value when the clamp device is On; default: 0.0001 Ohms, RON+ is the resistance for current flowing from the positive to negative terminal; RON- is the resistance for current flowing in the opposite direction.

OFF: specifies the zapping direction is “off”; that is, Totem treats the clamp device as an open circuit with that zapping polarity. Note that it is considered a syntax error if both directions of a pin pair is specified “off”.

ESD_PIN_PAIR : defines a legal ESD discharge path for B2B rules, and resistance values for both current discharge directions. <Ron+> defines the On resistance for discharges from `<loc_ID1> to <loc_ID2>`, and <Ron-> defines the On resistance for discharges from `<loc_ID2> to <loc_ID1>`). <Ron> defines symmetric resistance between the node pair; if defined, it overrides the RON definition above, or otherwise a default value of 0.0001Ohms is used. IS=<iscale> defines a scaling factor to scale the current on clamp I-V if it is assigned to the pin pair

IMAX: for a clamp cell rule, specifies the current thresholds for the positive and negative breakdown checks for the associated ESD_PIN_PAIR. When only one value is specified, the threshold is the same in both directions.
For a clamp I-V device rule, specifies the current thresholds for that device. For an ESD_PIN_PAIR, the limits specified in the clamp I-V rule have the highest precedence, followed by those in the clamp cell rule, then in the ESD rule.

VMAX : for a clamp cell rule, specifies the voltage thresholds for the positive and negative breakdown checks for the associated ESD_PIN_PAIR. When only one value is specified, the threshold is the same in both directions. For a clamp I-V device rule, specifies the voltage thresholds for that device. For an ESD_PIN_PAIR, the limits specified in the clamp I-V rule have the highest precedence, followed by those in the clamp cell rule, then in the ESD rule.

RAIL_VOLTAGE : when specified, reports clamp instances connected to the incorrect power domains

VDIFF: perform differential nominal voltage check during clamp check. VDIFF defines the threshold differential voltage range between the clamp. If only one value is defined instead of a range, it will be considered as the absolute threshold.

For the CLAMP_IV file, the keywords are described below and identified in Figure 12-3

Roff: resistance value when the clamp device is Off; default: 1e6 Ohms.

Roff+ is the off resistance for current flowing from the positive to negative terminal;

Roff- is the off resistance for current flowing in the opposite direction.

VT1: threshold voltage that turns On the clamping device; default: 0. That is, the clamp is modeled as a linear resistor with RON.

VH: holding voltage when the clamping device is On; default for VH: VH = VT1; the supported range for VH is 0 <= VH <= VT1.

Ion2: current threshold for On resistance to change from Ron to Ron2.

Ron2 : resistance in the upper conducting region

It2: current threshold for I-V curve to enter breakdown region.

Rt2: resistance in the breakdown region.

Vt2: threshold voltage at the beginning of the breakdown region.

Most clamp I-V parameters have two versions, '+' for the positive-to-negative current direction, and the '-' for the reverse direction.

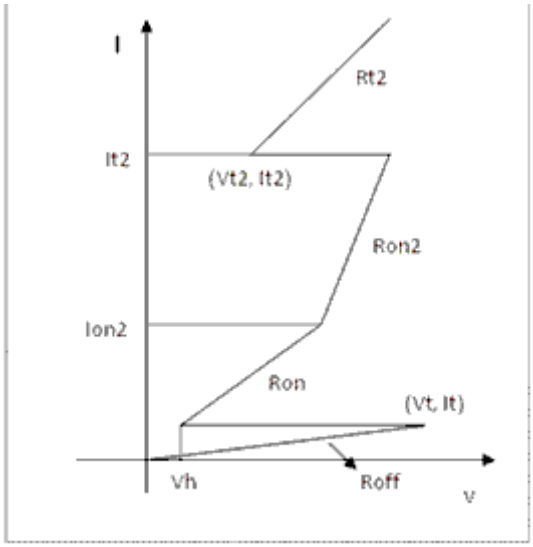


Figure 12-3 Diode/clamp device I-V characteristics

Sample clamp cell file:

```
BEGIN_CLAMP_CELL
    NAME IO_cell
    PIN PAD 7.29 362.14 METAL1 pad_1
    PIN VSS 7.39 362.04 METAL2 vss_1
    PIN VDD 7.29 363.66 METAL1 vdd_1
    PIN VDD 7.29 369.4 METAL2 vdd_2
    PIN VSS 7.0 334.5 METAL3 vss_2
    RON 0.01
    ESD_PIN_PAIR pad_1 vss_1 0.001 0.05
    ESD_PIN_PAIR vdd_1 vss_1 0.02 OFF
    ESD_PIN_PAIR vdd_2 vss_2 OFF 0.0001
END_CLAMP_CELL
```

Sample transistor clamp cell settings are shown following:

```
BEGIN_CLAMP_CELL
NAME IO
# INST : mos1__2 PIN : VDD
XTOR mos1__2:VDD 2.24 328.14 BOTTOM
# INST : mos1__2 PIN : VSS
XTOR mos1__2:VSS 2.34 328.14 BOTTOM
# INST : mos4__1 PIN : VDDO
XTOR mos4__1:VDDO 1.81 309.23 BOTTOM
# INST : mos4__1 PIN : VSS
```

```
XTOR mos4__1:VSS 1.91 309.23 BOTTOM
# INST : mos4__2 PIN : VDDO
XTOR mos4__2:VDDO 1.81 317.31 BOTTOM
# INST : mos4__2 PIN : VSS
XTOR mos4__2:VSS 1.91 317.31 BOTTOM
ESD_PIN_PAIR mos1__2:VDD mos1__2:VSS diode
ESD_PIN_PAIR mos4__1:VDDO mos4__1:VSS sback
ESD_PIN_PAIR mos4__2:VDDO mos4__2:VSS b2b_diode
END_CLAMP_CELL

BEGIN_CLAMP_IV
  NAME diode
  RON 0.1  100K
  VT1 0.4
  ROFF 100K
END_CLAMP_IV

BEGIN_CLAMP_IV
  NAME b2b_diode
  RON 0.2
  VT1 0.2
END_CLAMP_IV

BEGIN_CLAMP_IV
  NAME sback
  RON 0.3 0.2
  VT1 1.2 1.5
  VH  0.1 0.15
END_CLAMP_IV
```

In this example, VT1, VH, and RON are specified according to the parameters defined in Figure 12-3. By default, VH = VT1, <RON-> = <RON+>, <VT1-> = <VT1+>, and <VH-> = <VH+>. If none of <RON->, <ROFF->, <VT1->, <VH-> are specified, they assume the default values defined above.

PathFinder supports Transmission Line Pulse (TLP) data in ESD clamp I-V definitions for both resistance and CD checks, with the TLP test results filename provided by the rule keyword TLP_FILE. The clamp file syntax is:

```
BEGIN_CLAMP_IV
   NAME <ivname>
   TLP_FILE <tlp_filename>
   TLP_I_RESOLUTION <value>
END_CLAMP_IV
```

TLP_I_RESOLUTION: Based on the TLP data PF-S engine automatically decides a value for 'TLP_I_RESOLUTION' by default and user does not have to set it. The typical range is from 0 to 0.001.

For cases where one wants to fine tune the model, use 'clampviewer' utility (on unix terminal) to interactively fine tune through a new option '-di <value>' (di i.e. ‘delta I' is for 'TLP_I_RESOLUTION' in clamp IV file.

A sample TLP test results file (first column voltage, second column current) is shown below:

```
0,0
1.075228,-5.57006E-06
2.181616,3.30643E-05
3.273627,7.19229E-06
4.283446,7.90741E-05
5.371665,7.80592E-05
```

The ',' value separator can also be a space or tab. And '#' at the beginning of a line for comments is allowed.

For more information on clamp device connection and modeling, see GSR keyword section "ESD_CLAMP_PIN_FILE", page C-894, and TCL command section "pfs", page D-969.

You can perform B2B checks before doing DC analysis. Loop resistance used for DC analysis does not need to be the same as that used for the B2B check. The clamps in “pass loops” in the B2B check are used for DC analysis. When there are no “pass loops”, all clamps are off, and DC analysis cannot be performed. If the B2B check has not been performed, Totem performs iterative convergence to decide which clamps are on and off in the final solution.

Clamp Files - Type

You can also specifying clamps by cell type and pin type to help simplify the setup, similar to clampgen. The syntax is as follows:

```
BEGIN_CLAMP_TYPE
  CLAMP_TYPE <type>
  CELL_NAME <cell1> <cell2> ...
  PIN_TYPE <pin_type> <pin1> <pin2> ...
  PIN_PAIR <type1> <type2> [R |Ron+ Ron-| IV]
  PIN_GROUP [POWER | GROUND | SIGNAL]
  EXCLUDE_GUARD_RING [0|1]     ## Default : 0
END_CLAMP_TYPE
```

This specification can be included a clamp file defined using the GSR keyword “ESD_CLAMP_FILE <clamp_file>”, the same as a regular clamp file, and can be used for ESD checking without requiring the clamp file to be specified again, using :

```
perform esdcheck -setupClamp
```

or

```
perform esdcheck -rule <rule_file>
```

Option ‘EXCLUDE_GUARD_RING 1’ enables automatic filtering out nets/pins belonging to guard-ring structures. CLAMP_TYPE processing will not create ESD_PIN_PAIR when an instance does not have PIN_PAIR specified. Default: 0

Clamp DB Creation

To make clamp definition and reuse more efficient, PathFinder creates a binary DB for clamp devices (such as cells, instances, nodes, esd pin pairs, I-V data) to be used by the general ESD checking command 'perform esdcheck' (see details in section "perform esdcheck", page 12-369) for both clamp rules and also for bump rules. With general clamps definition in the DB, specific clamp parameters can be added or removed from particular rules files. The basic controls on checking relative to clamps are described in this section, using the clamp-related syntax:

```
perform esdcheck
    -clamp <clamp_file> ? -setupClamp ?
```

where

-clamp <clamp_filename> : specifies the clamp filename. Clamp statements can be combined in the rule file or separately defined in a clamp file. See the clamp file section for a description of the clamp file syntax and an example clamp file.

-setupClamp : imports and saves the clamp info into the DB. You can check Errors/Warnings in the clamp file before running ESD checks. Saves significant run-time without having to import a clamp file using "-clamp <file>" each time ESD checks are run. You can save or load the clamp data by including the following keywords in the rule file:

```
SAVE_CLAMP_DB 1
LOAD_CLAMP_DB 1
```

Both keywords are default off. Note that

- Only one of SAVE or LOAD can take effect. When both are specified, LOAD is used.
- When you specify an ESD rule without LOAD_CLAMP_DB 1, and if you do not specify clamp info either in rule file or with -clamp <file>, Totem loads the existing clamp DB.
- If neither option -setupClamp or the rule 'SAVE_CLAMP_DB 1' are specified, if the clamp DB does not exist, the clamp info is saved into the DB.

Clamp-file-driven GDS-based Pathfinder-Static flow

The clamp-file-driven PF-S flow allows clamp cell names to be picked from the ESD_CLAMP_FILE specified in the GSR, and they do not have to be specified in the GDS2DB configuration file.

Input requirements:

- GDS master cell names must match clamp names specified in the clamp file.
- For each pin specified in ESD_PIN_PAIR (in the clamp file), there must be matching text-labels inside the clamp cell in GDS.

This flow also supports the following advanced features:

- Series/parallel connected clamps, auto tracing of internal net connections between the series-connected clamps.
- Irregular placement and interconnections of clamps (such as series /parallel connections or star connections) are also supported.
- The ESD paths containing disconnected, partially connected, and shorted clamp instances are ignored in the PF-S checks.

Syntax in the GSR:

```
ESD_CLAMP_FILE <clamp file name>
```

Syntax in the GDS2DB configuration file:

```
ESD_CELLS {
    USE_CLAMP_FILE 1
}
```

ESD Rules Files

The parameters for ESD checks must be specified using a rules file that defines key elements of each desired check, such as TYPE (BUMP2BUMP, BUMP2CLAMP, CLAMP2CLAMP, PIN2CLAMP, PIN2PIN), a user-assigned rule NAME, limits on ARC_R, LOOP_R, and PARALLEL_R, the number of stages, and also the clamp types, bumps and nets that are to be specifically included or excluded in that check.

Defining Groups of Nets

ESD rules files can specify user-defined net groups to simplify including or excluding certain groups of nets in ESD testing. The rule syntax for defining a net group is as follows:

```
BEGIN_NET_GROUP
    NAME <userDefinedGroupName>
    NET <netName1>
    NET <netName2>
    ...
END_NET_GROUP
```

The creation of various types of specific rules files are described in the following sections.

Reducing Design Size for Improved ESD Analysis

To speed up performance on large designs, PathFinder can generate a special GSR file that selectively removes ESD-irrelevant cells/layers to reduce design size, while keeping acceptable accuracy. The methodology is as follows:

1. In the original GSR file, add the following GSR keywords:

   ```
   ESD_CLAMP_FILE <clampFile>
   ESD_RULE_FILE <ruleFile>
   ```

   where <clampFile> and <ruleFile> are the clamp and rule files to be used for the “perform esdcheck” command.
2. Add the “ESD_GSR 1" rule keyword in the appropriate rule file.
3. Run Totem through the setup design stage. A new *esdGsr.gsr* file with modified GSR keywords is generated:
   a. Unnecessary standard-cell LEF cells are removed from analysis with the IGNORE_CELLS keyword, when no C2I/C2M/B2I rules are involved.
   b. SPLIT_VIA_ARRAY or SPLIT_SPARSE_VIA_ARRAY keywords are removed, if no current density rules are involved.
   c. Lower metal layers from the block/top level DEF files are ignored using the DEF_IGNORE_SPECIFIC_LAYERS keyword.
4. Replace the original GSR file with the *esdGsr.gsr* file and rerun Totem.

The performance and capacity improvement of running with the new *esdGsr.gsr* could be up to 50%, depending on the design.

Topology and Connectivity Checking of Bumps and Clamps

PathFinder evaluates and reports on clamp instances that are not connected to P/G and signal nets, and bumps isolated from clamps. You can obtain a text report using the command 'perform clampcheck' as follows:

```
perform clampcheck <net/bump_conn_options>
? -rule <rule file> ?-o <file>? ?-instConn?
? -isolatedBump? ?-cell <cell_name>? ?-celltype <type>?
? -inst <inst_name>? ? -volt <voltage>? ?-net <net_name>?
? -netConn <net1> <net2> ...? ? -allNetConn ?
? -bumpConn <bump_name> ? ? -allBumpConn ? ? -loop?
? -loopLength <stage_num>? ? -b2bLoopLength <stage_num>?
? -roff <R_thresh> ? ? -rptDisConn ?
```

```
? -rule <rule_file> ? ? -esdStage {min,max} | <stage_num>?
? -detail? ?-append? -diffVolt ?
```

where

<net/bump_conn_options> : indicates how net/bump connectivity is checked. The options are : -allNetConn, -allBumpConn, -netConn <Nets>, and -bumpConn <Nets>.

-rule: specifies the name of the rule file for checking

-o <file>: specifies the output filename

-instConn: reports clamp instances that have missing connections to the P/G grid.

-isolatedBump: reports bumps that do not connect to any clamp

-cell <cell_name>: reports information on specified clamp cell(s)

-celltype <type>: reports information on clamp cells of a particular clamp type

-inst <inst_name>: reports information on specified clamp instance(s).

-volt <voltage>: reports a list of clamps connected to a particular node voltage.

-net <net_name>: reports information on all clamp nodes connected to specified net(s), both directions.

-netConn: checks connectivity of each net pair (both directions), so for "-netConn {netX netY}", the connectivity of both netX->netY and netY->netX is checked. If "-netConn netX" is specified, the connectivity from netX to all other nets and from all other nets to netX is checked. For each net pair reported, a symbol "->" or "<->" is added between the two nets to indicate the connectivity direction between them.

-allNetConn: reports clamp connectivity to all nets and domains, in both directions

-bumpConn: reports clamp connectivity to all bump connectivity from the specified bump

-allBumpConn: reports clamp connectivity of all bumps

-loop: lists the clamp instances of all B2B loops for queried net pairs.

-loopLength: -b2bLoopLength : set the EXACT stage number(s) to be reported for queried net pairs. Note that "-esdStage" sets the RANGE of stage numbers, while "-loopLength" sets the EXACT stage numbers. -loopLength can specify more than 2 numbers.

-roff: sets the resistance threshold value at which clamp devices are considered "OFF". The default threshold value is 1e6 Ohms.

-rptDisConn: reports net-pairs with no clamps between

-esdStage: specifies either a specific stage or a range of clamp stages between which the net/domain pair checks have to be performed.

Example:

```
-perfom clampcheck -esdstage {1,3}
```

Here, net/domain pair check happens from stage 1 to stage 3.

-detail: reports detailed information on net/domain pair.

-append: appends results to existing results in the output file

An example invocation would be:

```
report clampcheck -rptDisConn -allNetConn
```

which reports all net-pairs between which no clamps are found.

A sample 'Unconnected Clamp Instances' report is shown following for the command:

```
perform clampcheck -o instConn.txt -instConn
```

```
######### List of Unconnected Clamp Instances #########
```

```
#
# <CELL NAME>:
#     <PIN_NAME> <X> <Y> <LAYER> <LOCID> <INSTANCE_NAME>
inst_ndiode_0:
VDDO 1.07374e+06 1.07374e+06 metal1 VDDO inst_ndiode_0/adsU4
```

A sample ‘Bumps Isolated from Clamps’ report is shown following for the command:

```
perform clampcheck -o isolated_bumps.txt -isolatedBump
```

```
######## List of Bumps Isolated from Clamps ########
#
# <BUMP_NAME> <BUMP_X> <BUMP_Y> <NET_NAME> <BUMP_LAYER>
1_pad  4569.13 10992.5 net1 metal8
```

-diffVolt : Pathfinder will check the differential voltage across the clamp and compares it against the provided threshold to report pass/fail. When option -o is not specified, the result will be printed to the Totem text window and the log file, it will not use default file 'adsRpt/ESD/diff_check.rpt'. Tcl Command:

```
perform clampcheck -diffVolt -rule <> -clamp <> -o
<filename>
```

Layout Resistance Checking of Bumps and Clamps

Overview

PathFinder can perform the following six types of ESD-related resistance checks:

- Bump-to-Bump (BUMP2BUMP, or B2B), including multi-stage checks
- Bump-to-Clamp (BUMP2CLAMP, or B2C)
- Bump-to-Instance (BUMP2INSTANCE, or B2I)
- Clamp-to-Clamp (CLAMP2CLAMP, or C2C)
- Clamp-to-Inst (CLAMP2INST, or C2I)
- Clamp-to-Macro (CLAMP2MACRO, or C2M)

Bump-to-Bump (BUMP2BUMP, or B2B)

For bump-to-bump resistance checking, you must specify the list of bumps/pads (or bump locations) that form pairs. Each bump pair must be connected by one or more clamp cells.

The first step in this calculation is the estimation of the loop resistance, which is the resistance for each bump->clamp->bump path. Loop resistance is defined as the actual Vdd/Vss resistance from a bump to another bump. For bump pairs that have multiple clamp cells connecting them, an equivalent number of paths for loop resistances is calculated.

Once the loop resistances are calculated, they are compared against the loop resistance threshold that has been defined. The loops (bump->clamp->bump paths) having an effective resistance greater than the specified threshold are considered as failing the loop resistance check. The clamps that are in these failing loops are deemed “invalid” and are discarded from the subsequent bump-to-bump parallel resistance calculation, in which the effective resistance from one bump to another is calculated, considering all the valid

clamp cells connecting these two bumps. The invalid bumps (from the loop resistance checks) are considered to ***not*** provide a discharge path between a pair of bumps.

This rule also calculates and performs ESD resistance checks on multi-stage ESD BUMP-to-BUMP pairs. PF-S calculates the resistance from each bump to ESD clamp instances, forms multi-stage bump to bump paths, and performs ESD resistance checks on these multi-stage bump2bump paths.

Bump-to-Clamp (BUMP2CLAMP, or B2C)

Next, the effective resistance from a pad to a clamp cell is calculated and reported. You can provide the pad location, or the tool can determine it automatically. Similarly, for the clamp cell, the location to which the resistance is calculated can be specified (recommended), or it can be determined by the tool. If the tool determines it automatically, it attempts to identify the location with the highest effective resistance. This pad-to-clamp resistance is compared against the threshold specified using the command line option '-arc_R'.

Totem automatically finds clamp cell devices based on an ID layer specified in the layer map file, specifies them as white_box_cells, and then generates a LEF macro for them.

For B2C resistance checking, to apply different resistance thresholds from the pad to the anode of the HBM diode, and from the pad to the cathode of the HBM diode, use the following keywords to select the clamp pins to be tested in separate B2C runs:

- To apply the rule only to the anode of the HBM diode:

  ```
  CLAMP_POS_PIN 1
  ```

- To apply the rule only to the cathode of the HBM diode:

  ```
  CLAMP_NEG_PIN 1
  ```

Bump-to-Instance (B2I)

B2I rules support Bump-to-Instance resistance checks. Using this feature you can check if the resistance to an instance from the bump is greater than the resistance to the clamp.

Clamp-to-Clamp (CLAMP2CLAMP, or C2C)

Finally the effective resistance between any two clamp cells connected to each other is reported, and compared against the threshold specified in the '-maxR' command line option.

Clamp-to-Inst (CLAMP2INST or C2I)

The effective resistance from a core instance to a clamp cell is calculated and reported to identify potential CDM failures. You must provide a list of the instances. Then locations with minimum effective resistances are chosen for standard cells and locations with maximum effective resistances are selected for macro cells (such as memories, IPs, etc.) automatically. The rationale for these choices is that macro cells are much larger in size compared to standard cells and therefore the worst location with maximum effective resistance should be considered to catch potential C2I failures. The same location selection method can be applied to clamp cells. Both loop_R and arc_R are reported with pass/fail criteria with respect to arc_R and loop_R constraints specified in the rule.

Clamp-to-Macro (CLAMP2MACRO or C2M)

The effective resistance from a macro instance to a clamp cell is calculated and reported to identify potential CDM failures. You must provide a list of the instances. Then all node locations on the bottom/top/<layer> of the LEF pin geometries with minimum effective resistances with respect to one of the clamp cells are chosen (such as memories, IPs, etc.) automatically. The difference from C2I selection is that many more node locations are considered for checking potential CDM failures (therefore achieving better location

coverage) . The same location selection method can be applied to the clamp cells. Only arc_R is reported with pass/fail criteria with respect to arc_R constraints specified in the rule.

Including Package Resistance

PathFnder ESD analysis can include the effects of package subcircuits. The package can have a significant impact on some ESD results, since it modifies the resistive paths between many nodes in the chip. For ESD resistivity analysis, inductances in the package subcircuit are short-circuited, and mutual inductances and capacitances are open-circuited. Clamps are identified, a radius formed, valid bumps are identified, and then resistances are calculated and compared. The same ESD checks are performed as in chip analysis, except that the package resistance from the bumps to their package BGA sources (for both the VDD and VSS) is included. So the check starts from the BGA side of the package, through the package, through the VDD bumps, through the P/G network to the clamp, and then returns to VSS bumps and through the package to the BGA pins.

Several types of rules are added to the 'perform esdcheck' command for computing resistances beginning or ending at the package pins. The setup of these rules is similar to the corresponding BUMP-related checks. The package-related rules are:

- PIN2CLAMP (or P2C) - external package pins to clamp nodes, similar to B2C
- PIN2PIN (or P2P) - external package pins to external package pins, similar to B2B, including multi-stage cases.

Package subcircuits can be included in Totem analysis by the same mechanism as is used for static and dynamic simulation. The most common method is to specify the package subcircuits using the GSR keyword PACKAGE_SPICE_SUBCKT. Once a package subcircuit is included, ESD checks can be performed with or without the package subcircuits. The package subcircuit is required and is automatically included for the P2C, P2P and P2PM checks, in which all resistive paths start or end at the package pins. For other resistive ESD rule types, the default behavior is to exclude the effects of the package.

Data Flow

The Layout Topology Checker of PathFinder can be run inside both RedHawk and Totem tools. When running inside RedHawk, it helps verify the placement of clamp cells in SoCs and other digital circuits. It typically works with a LEF/DEF-based input, although GDS is also supported. When running inside Totem, it helps perform similar checks, particularly for analog, custom, and mixed signal circuits. In this case, it works with GDS-based inputs, with additional data coming from the SPICE netlist. The data flow is shown in Figure 12-4.

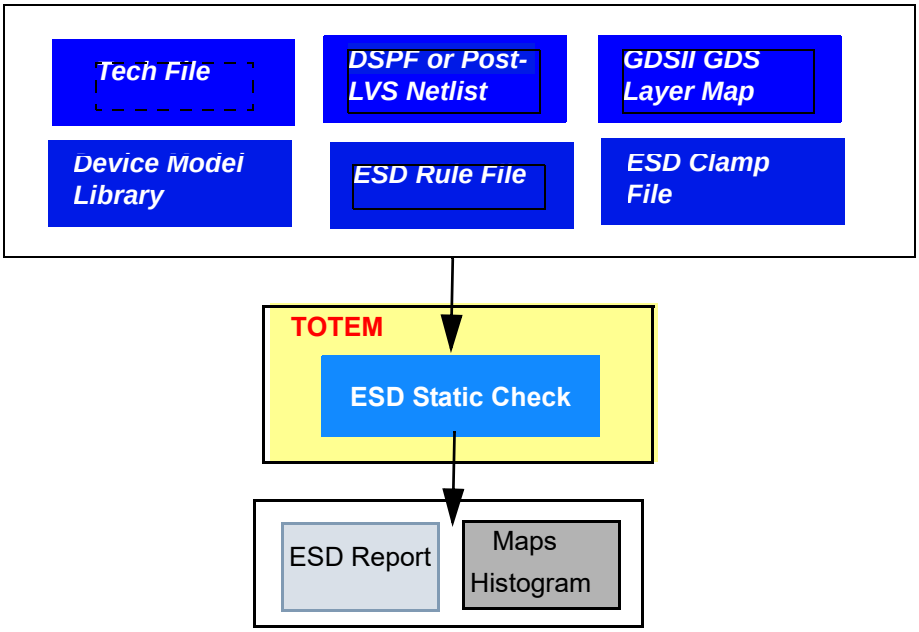


Figure 12-4 PathFinder Data Flow

Description of Inputs

The inputs for the Layout Topology Checker are as follows:

- Technology LEF file - contains the layer and via descriptions for a particular library or process node in the LEF format available from the library team.
- Apache Technology file - specifies key technology parameters, such as resistance and dielectric constant, for Totem to use for extraction.
- Pad location (ploc) file - specifies the locations of voltage sources in the regular PLOC file format, as used in Totem. An example of a PLOC file with single VDD and VSS source points and is as follows:

```
# pad location points
<unique name>  <x-loc>    <y-loc>    <metal>    <power/ground>
vdd1    6808    11091   metal7      POWER
vss1    7038    11612   metal7      GROUND
```

- GDSII files and GDS layer map - can be used for GDSMMX, or through GDS2DB using GDS_FILE keyword in the GSR.
- ESD static check includes geometry-based resistance checks for HBM/MM/CDM modes, and current density checking for HBM/M
- In the command file, include ‘setup analysis_mode ESD’ for ESD-specific control and GUI display.
- Rule definitions - see following sections
- Clamp cell definitions - a clamp cell can be defined either in its own clamp cell file or in a combined clamp cell/rule file, as described in section "Clamp Cell Definition", page 12-355.
- Job control - PathFinder supports multi-threading and multi-processing capabilities. For multi-threading, use the '-thread' option; and for multi-processing, use the

'-jobCount' option. To perform esdcheck with the -jobCount option, it is advisable to set up the clamp DB using 'perform esdcheck -clamp <clampfile> -setupClamp', and then execute 'perform esdcheck -rule <ruleFile> -jobCount <N>'. You can do multi-processing by starting a session from scratch, or by importing a previously-exported DB. In that case, make sure the DB is exported from a session with 'setup analysis_mode esd' as the first TCL command executed.

If the machine has multiple CPU's, PathFinder splits the job and executes them. One job is performed by the parent PathFinder session, and the rest of the jobs are executed in different sessions in adsESD1, adsESD2, … as the working directory. The job count and save directory options of 'perform esdcheck' are defined as follows:

-jobCount <n>: the number of jobs to execute at a time.

-saveDBDir : directory to save the DB and then import it when executing esdcheck in the same run

For detailed 'perform esdcheck' syntax, see section "perform esdcheck", page 12-369.

You can also control splitting of rules into separate jobs by creating a job file using the '-jobFile <job_defin_file>' option. Using PF-S job assignment file:

```
perform esdcheck -rule <ruleFile> -jobFile <jobFile> ....
?-jobCount <cuont>?
```

When -jobCount is not used <jobFile> must have more than 1 job in it. When both -jobFile and -jobCount are specified, -jobCount determines the number of jobs to be executed. A sample job file below:

```
BEGIN_ESD_JOB
 NAME <jobName>
 WORK_DIR <path>
 RULE <type> <name> <percent>
 RULE_COUNT <type> <name> <arcCount>
 EXCLUDE_RULE <type> <name>
END_ESD_JOB
```

where NAME, CLAMP and DIR specifications are optional.

<jobName>: User specified name for the job, unique among all jobs. Default is Job1, Job2, ... following the order as it appears in the job assignment file.

<path>: to specify directory path to execute the job. Default is adsESD1, adsESD2,..., in working directory, following the order in the job assignment file. Each job needs to have it's own working directory, not to be shared with other jobs.

<type>: the type of the ESD rule as in ESD rule file (B2B, B2C, ... CD, etc.), to be assigned to this job.

<name>: The name of the ESD rule of the <type> to be executed in this job.Wildcard (*) is supported in/as <name>..

<percent>: 0.0 to 100.0, in RULE, specifies the percentage of from/to zaps fora TYPE CD or R rule to be executed in this job.

<arcCount>: Specifies the number of from/to zaps for a TYPE CD or R rule to be executed in this job.

EXCLUDE_RULE: Not to assign the specified rule of <type>, <name> in this job.When 0 is used for <percent> or <arcCount> for RULE or RULE_COUNT, resp., the effect is same as 'EXCLUDE_RULE'.

When there are redundant assignment (same rule is assigned multiple times in a job);

a). <arcCount> overrides <percent>.

b). larger value will be used.

It is not required that <percent> or <arcCount> to cover exactly all arcs for a rule. When over-assigned, it gives PF-S engine more flexibility trying to reach shortest possible completion time. When under-assigned, PF-S will fit the unassigned arcs into the jobs.

For ESD rules in ESD rule file not covered in the job assignment file, PF-S will assign the rules into the jobs such that it tries to reach the shortest execution time for all jobs.

Resistance Checking for B2B, B2C, and C2C Rules

The resistance checking command is 'perform esdcheck'. There are a number of basic options available, which are described in the following syntax descriptions. Additional controls on the checking are provided by rules file keywords in the following section.

```
perform esdcheck
  -rule {<rules_file1> <rules_file2> ...}
  -clamp {<clamp__file1> <clamp_file2> ...}
  ? -thread <num_threads> ?
  ? -optimize <mode>? ? -detail ? ? -append ?
  ? -outDir <results_dir> ? ? -ignoreError ?
  ? -saveDBDir <path> ? ? -jobCount <num_jobs> ?
  ? -jobFile <job_defin_file> ? -addNode ? -rerun execFail
  ? –rerun emFail ? -incremental ? -reportByPGArc
```

where

-rule <rules_file_names> : specifies the names of the rules files containing the rules to be checked. At least one rule must be specified. If the ESD check for a rule has an error, the default behavior is to stop execution. The '-ignoreError' option allows checking to continue past any errors, if possible, and attempt to finish all rules in the rule file. See the following section for a definition of the rules file and a sample rules file.

-clamp <clamp_file_names> : specifies the names of the clamp files to be used. Clamp statements can be combined in the rule file or separately defined in a clamp file. See the Clamp file section for a description of the clamp file syntax and an example clamp file.

-thread <num_threads>: specifies the number of threads/processes to be used, to speed up the calculation. All available CPUs may be used to support the multiple threads unless ‘-mcore’ is specified. Default=2.

-optimize <mode>: as the mode specified increases from 0 to 3, more optimization is done, for greater performance gains.

Mode values 0 and 1 are only recommended to work around issues that have caused resistance computation to fail:

0: all optimization off

1: performs only basic netlist optimizations (up to 2X performance gain)

Mode values 2 and 3 are recommended for normal use.

2: performs aggressive netlist optimization for greater performance gain (up to 20X possible). This is the default.

3. In addition to optimization of mode 2, performs additional optimization for greater coverage (can double the performance as with 2). This setting would be effective only for C2I, C2M, and B2I resistance checks.

-detail : provides detailed reports on B2B and multi-stage checks, including data on x,y locations, netnames, metal layers and the cell names of the clamp instances, so you do not have to refer to any other files to get details.

-append : appends results to the output files, instead of overwriting the previous output file

-outDir <results_dir>: specifies the directory for the results files. Default: *adsRpt*.

-ignoreError : where the <rules_file> contains multiple rules to be checked, if there are data issues or error conditions such that the ESD check for a rule has an error, the default behavior is to stop execution. '-ignoreError' allows checking to continue past any errors, if possible, and attempt to finish checking all rules in the rule file.

-saveDBDir: saves the DB into the specified directory. You can then execute esdcheck in the same run used for exporting the DB.

-jobCount: specifies the number of jobs to execute at a time when the rule file has multiple rules in it, or when an ESD current density check rule has many 'from/to' point pairs to be analyzed. This option can be used to partition the rule check into multiple sub-tasks, which are executed in parallel when the host machine has multiple CPUs. One job is performed by the parent PF session and the rest of the jobs are executed in different sessions in *adsESD1, adsESD2*, … as the working directory.

The recommended usage is to start the session by importing a previously-exported DB, then to run ESD checking with the -jobCount option specified.

-jobFile : specifies the job definition rules file

-addNode : creates a node for accurate esdcheck. Options "-from {<x1 y1> <layerName1> ?<netName>? }" and "-to {<x2 y2> <layerName2> ?<netName>?}" must also be specified. A node is created at that location, provided there is no node within a 1 uM radius, and there are wires at the specified location.

-rerun execFail : If the job fails due to LSF submission or other tool/rule errors, the command will help rerun the failed jobs

–rerun emFail : rerun the Zaps that are failing CD checks

-incremental : support incremental instance and node selection for C2I resistance check in DMP flow. Pathfinder will sample a new batch of instances for performing esdcheck

-reportByPGArc : to only report pin pairs in pgarcs for the cell, if exist.

To export ESD checking results to a specified directory, the command is:

```
export esdcheck <esdcheck_db_directory>
```

To Import ESD checking results from specified directory, or optionally import results for only a specific rule type, use the command:

```
import esdcheck <esdcheck_db_directory> ?-<ruleType>?
```

To specify the type of reports to be created on the ESD checking, the command is:

```
report esdcheck -type <ruleType> ? -outDir <output_dir>?
? -name <rule_name>? ?-[glob |-regexp ] ?
? -arcR <arc_threshold>? ? -loopR <loop_threshold>?
? -parallelR <parallel_threshold>? ? -append?
? -failedOnly? ? -detail?
```

where

-type: reports results for the specified type of ESD checks. Default is all types.

-outDir : specifies the report files directory. Default is *adsRpt/ESD*

-name: specifies the name of the rule tested. If multiple checks have been specified within a single rule-- for example, two bump pairs are specified in the same rule-- PathFinder internally splits them into subrules, each with a single bump pair. Then it appends '(_\d+)+' to the base rule name. So when using 'report esdcheck', you can specify this derived name to look at the particular results desired.

-glob|-regexp: specifies the type of wildcard expressions used. PathFinder reports the results for all subrules that match the specification; 'glob' supports regular wildcard expressions and 'regexp' supports normal wild cards and also extended character strings. For example:

```
report esdcheck -detail -perNet -type B2C -name abc* -glob
report esdcheck -type B2C -name {M_test_[3-5]} -regexp
```

-arcR: specifies the threshold for arc-R checking; required for B2C, C2C, C2I, C2M

-loopR: specifies the threshold for loop-R checking; required for C2I

-parallelR: specifies the threshold for parallel-R checking; required for B2B

-append: appends results to existing results in the output directory.

-failedOnly: only reports failed results.

-detail : provides more detailed report results in B2B

The default B2B report format is as follows:

```
# BUMP PAIR:(<BUMP1> <X> <Y> <LAYER> <NET>) <-> (<BUMP2>
    <X> <Y> <LAYER> <NET>)
# PASS/FAIL: <LOOP_R> <ARC1_R> <ARC2_R> <CLAMP> <LOCID1>
    <LOCID2>
# PARALLEL R: <VALUE>
```

For example:

```
PASS: 7.29865 2.63136 4.5673 inst_92318 VSS VDD
```

With the -detail option, the report format is:

```
# BUMP PAIR: (<BUMP1> <X> <Y> <LAYER> <NET>) <->
    (<BUMP2> <X> <Y> <LAYER> <NET>)
# PASS/FAIL: <LOOP_R> <ARC1_R> <ARC2_R> <CLAMP> (<X> <Y>
    <LAYER> <NET>) (<X> <Y> <LAYER> <NET>)
# PARALLEL R: <VALUE>
```

For example:

```
PASS: 7.29865 2.63136 4.5673 inst_92318 (1268.27 3145.09 METAL3 VSS)
    (1268.27 3142.63 METAL3 VDD)
```

With the -detail option, the report format is:

```
# BUMP PAIR: (<BUMP1> <X> <Y> <LAYER> <NET>) <->
    (<BUMP2> <X> <Y> <LAYER> <NET>)
# PASS/FAIL: <LOOP_R> <ARC1_R> <ARC2_R> <CLAMP> (<X> <Y>
    <LAYER> <NET>) (<X> <Y> <LAYER> <NET>)
# PARALLEL R: <VALUE>
```

For example:

```
PASS: 7.29865 2.63136 4.5673 inst_92318 (1268.27 3145.09 METAL3 VSS)
    (1268.27 3142.63 METAL3 VDD)
```

With the -detail option:

```
# BUMP PAIR: (<BUMP1> <X> <Y> <LAYER> <NET>) <->
    (<BUMP2> <X> <Y> <LAYER> <NET>)
# PASS/FAIL: <LOOP_R> <ARC1_R> <ARC2_R> <CLAMP> (<X> <Y>
    <LAYER> <NET>) (<X> <Y> <LAYER> <NET>) <INT_NET>
```

```
    <CLAMP> (<X> <Y> <LAYER> <NET>) (<X> <Y> <LAYER> <NET>)
# PARALLEL R: <VALUE>
```

For example:

```
FAIL: 4.34594 3.0401 1.30564 adsU1 (573.825 32.585 M6Z
RAIL_gnde_001) (535.85 105.21 M7Z gndo) gndo adsU2 (535.85
105.21 M7Z gndo) (573.825 53.585 M6Z RAIL_vdde_001)
```

Rules File Syntax for Types B2B, B2C, and C2C

A description of the syntax supported in the ESD rules file is shown below, including package integration.

```
BEGIN_ESD_RULE
  TYPE [ BUMP2BUMP | BUMP2CLAMP | CLAMP2CLAMP |
      PIN2CLAMP | PIN2PIN | RESISTANCE ]
  NAME  <rule name>
  ARC_R <value in ohm>
  LOOP_R   <value in ohm>
  PARALLEL_R <value in ohm>
  RADIUS <value>
  INCREMENTAL_MODE 1
  ESD_STAGE [1 or 2]
  PACKAGE [0 |1]
  PACKAGE PIN [ 0 | 1]
  PACKAGE_PIN_FILE <pkg_fileName>
  EXCLUDE_BUMP {<bump1> ... <bumpN>}
  EXCLUDE_BUMP_PAIR <bump1> <bump1A> <bump2> <bump2A> ...
  EXCLUDE_BUMP_PAIR_FILE <file_name>
  BUMP_LIST {<bump1> ... <bumpN>}
  NET_PAIR <net1> <net2>
  NET_PAIR_FILE <filename>
  INTERNAL_NET <net1> <net2> ...
  TERMINAL_NET <net1> <net2> ...
  TERMINAL_NET_GROUP { POWER | GROUND | SIGNAL }
  INTERNAL_NET_GROUP { POWER | GROUND | SIGNAL }
  B2B_NET1 <net1> <net2> ...
  B2B_NET2 <net1> <net2> ...
  REPORT_TO_CLAMP [ OFF | MIN |MAX | ALL ]
  REPORT_FROM_CLAMP [ OFF | MIN |MAX | ALL ]
  ZAP_VOLTAGE <voltage_Volts>
  ZAP_R <resistance_Ohms>
  ZAP_CURRENT <current_in_Ampere>
  CLAMP_TYPE <type_name>
  CLAMP_ROFF <R_limit>
  SHORT_CLAMP_NODE [ 0 | 1 ]
  FROM_CLAMP_TYPE <clamp_type>
  TO_CLAMP_TYPE <clamp_type>
  CLAMP_POS_PIN [0 |1]
  CLAMP_NEG_PIN [0 |1]
  USE_CLAMP_FILE <file>
  B2B_LOOP_LENGTH <clamp_count1> <clamp_count2> ...
  RADIUS <value_u>
  REPORT_B2C [0 |1]
```

```
  REPORT_C2B [0 |1]
  REPORT_C2C [0 |1]
  REPORT_BY_PGARC [0 |1]
  EXCEL_REPORT [ 0|1]
  B2B_PAIR_COUNT <num>
  B2B_LOOP_COUNT <num>
  B2B_CHECK_IV [0|1]
  MAX_ESD_STAGE <num>
  CLAMP_SELECT [min |max]
  EM_MODE [ ave | peak ]
  DETAILED_MODE [0 | 1}
  SHORT_BUMP_IN_NET <net name1> <net name2> ...
  NO_SHORT_BUMP_IN_NET <net name1> <net name2>...
  SHORT_BUMP_IN_NET_GROUP [POWER| GROUND| SIGNAL ] ? ALL?
  SHORT_MODE [0 |1]
  B2B_MODE [0 |1]
  ALL_PAD_NODE [0 |1]
  SAME_CLAMP_INST [0|1]
  DIFF_CLAMP_INST [0|1]
  SAVE_MAP [ EM | CURRENT | VOLTAGE]
END_ESD_RULE
```

where

TYPE: specifiies the checking rule type. TYPE R(ESISTANCE) allows resistance checks from multiple point to multi-point ZAPS.

NAME : user-specified rule name; reserved TYPE names are not allowed as rule names. PIN* rule names involve package pin checks.

ARC_R : specifies resistance check threshold (Ohms) for BUMP2CLAMP or CLAMP2CLAMP rule. specifies the resistance threshold value for the BUMP2CLAMP, CLAMP2CLAMP, CLAMP2INST and CLAMP2MACRO checks. If specified, the value overrides the arc_R value defined in the ESD rules file. Optional for CLAMP2INST and CLAMP2MACRO checks, if specified in ESD rules file; required if '-rule CLAMP2INST' or '-rule CLAMP2MACRO' are specified.

LOOP_R : loop resistance check threshold for BUMP2BUMP. specifies the resistance constraints for checking the loop resistance of a single ESD path (bump->clamp (ON resistance)->bump or core->clamp (ON resistance)->same_core). Overrides the loop_R value defined in the ESD rules file. Optional for BUMP2BUMP, CLAMP2INST, or CLAMP2MACROcheck, if specified in ESD rules file; required if '-rule BUMP2BUMP, CLAMP2INST, or CLAMP2MACRO' is specified.

PARALLEL_R : parallel resistance check threshold for BUMP2BUMP. Specifies the pass/fail checking criteria for the total effective resistance of a bump pair. Optional for B2B checks. If not specified, the following message is issued. “INFO(ESD-111): PARALLEL_R not specified, only LOOP_R is used for pass/fail checking on B2B resistances.In this case, we also skip parallel R computation and only report the min B2B loop R.”

Note: Bump to bump checking can be performed without specifying PARALLEL_R in the rule file. In such cases, the LOOP_R value is used as the pass/fail criteria for the B2B check. If any single Loop R passes, the B2B pair is considered to pass. If all the Loop R tests fail, then B2B pair fails. In the ESD reports, the PARALLEL_R is displayed as the minimum of all Loop R values for the B2B pair.

RADIUS <value> : specifies the radius in um around each bump within which to include bumps or clamps in the resistance checking topology.

ESD_STAGE <min> <max> : used in B2B multistage rules. PathFinder checks all B2B net pairs that are connected by at least <min> clamps and no more than <max> clamps, and computes the bump-to-bump equivalent resistances for every bump pair in the net (subject to the RADIUS constraint). The value of <max> is limited to 5. If only one value is specified for ESD_STAGE, all bump pairs that have at least one bump-bump path (loop) with the specified number of clamps are analyzed. You can also control the number of best bump-clamp and bump-bump loops (loops with the lowest loop resistances) in the output report by using the keyword B2B_LOOP_COUNT. The maximum number of stages can be set using the option MAX_ESD_STAGE <num>, which can override the limit of 5 in ESD_STAGE.

PACKAGE 1 - includes the package subcircuits in ESD checking for the rule in which the keyword is included.

PACKAGE_PIN : specifies pin-to-pin checks with net-group options, such as FROM_NET/TO_NET/TERMINAL_NET between package pins. This option is different from the option 'PACKAGE 1', in which PathFinder uses bumps as reference, but ‘PACKAGE_PIN 1’considers package pin connections as reference.

PACKAGE_PIN_FILE <pkg_fileName> : for P2P, P2C and P2PM rules, specifies a file identifying nodes in the package subcircuit that should be treated as external pins, so that ESD checks compute resistances beginning and ending at those nodes. The format of this external file is as follows:

```
<Spice_nodename> <x_location> <y_location>
  ...
```

The <x,y locations> are only used for package reporting purposes. They are specified in user units, and must be floating point numbers. The node names in this file are case-insensitive, since SPICE is case-insensitive. If no file is specified, the voltage sources in the package subcircuit are used as the external pins, and resistance computations originate and terminate at those nodes. The package pins specified are used only when checking the rule in which the keyword is included, and is meaningful only if package subcircuits are included for this rule.

EXCLUDE_BUMP {<bump1> ... <bumpN>} : list of bumps to be excluded in ESD check

EXCLUDE_BUMP_PAIR : specifies sets of bump pairs that should be excluded

EXCLUDE_BUMP_PAIR _FILE: specifies a filename that provides a list of bump pairs to be excluded.

BUMP_LIST {<pad1/bump1> … <padN/bumpN>} : lists names of pads/bumps on which the ESD check is to be performed. Optional.

NET_PAIR <net1> <net2>: designates net pairs to be analyzed for ESD interaction. B2B and CD checks are performed for all the bump pairs between the specified nets.

NET_PAIR_FILE <filename>: specifies a file path containing a list of net pairs to be analyzed.

INTERNAL_NET <net1> : specifies bumps in domains to be filtered out and *not included* in B2B checking, for either 1 or 2-stage systems.

TERMINAL_NET: specifies the **only** net(s) between bump pairs to be analyzed in B2B checking. All other nets are considered INTERNAL_NET and are not used in B2B checking. In the following sample rules file:

```
BEGIN_ESD_RULE
    NAME  esd_b2b_rule
    ...
    TERMINAL_NET VSS A[3] VDDS
END ESD_RULE
```

all bump pairs connected between nets VSS, A[3] and VDDS are investigated and reported.

TERMINAL_NET_GROUP : specifies the **only** net types included in the analysis -- POWER, GROUND or SIGNAL.

INTERNAL_NET_GROUP : specifies the only net types **not** included in the analysis -- POWER, GROUND or SIGNAL.

B2B_NET1 : specifies the net(s) in the B2B_NET1 group that are to be paired with the net(s) in the B2B_NET2 group for B2B checks. Previously called ‘FROM_NET’.

B2B_NET2 : specifies the net(s) in the B2B_NET2 group that are to be paired with the net(s) in the B2B_NET1 group for B2B checks. Previously called ‘TO_NET’.

REPORT_BY_PGARC : to only report the pin pairs in pgarcs for the cell,if exist.

REPORT_TO_CLAMP : specifies TO_NET end on clamps and not bumps. The options are:

OFF: Disabled (default)

MIN: Report at the clamp with minimal resistance

ALL: Report resistances to all clamps

MAX: Report the resistance to the clamp on the ‘to’ side which has the highest resistance

ZAP_VOLTAGE: specifies the voltage of the zapping source for more accurate results, where non-linear clamp I-V behavior is involved (including cases with different Ron and Roff for ESD_PIN_PAIR) .

ZAP_R: specifies the series resistance to the zapping voltage source.

ZAP_CURRENT: directly specifies the zapping current. When this is used, ZAP_VOLTAGE and ZAP_R keywords are ignored.

CLAMP_TYPE : user-specified name for the clamp cell type to be considered, which allows limiting checking only to specified clamp type(s)

CLAMP_ROFF: sets the clamp resistance limit for reporting. B2B checking treats clamps with R >= CLAMP_ROFF value as 'off'; and not involved in the loop/parallel R computation, and these clamps are not reported.

SHORT_CLAMP_NODE : shorts clamp nodes “on-the-fly” in ESD-R and ESD-CD checks when resistances are computed.

FROM/TO_CLAMP_TYPE : specifies checking FROM/TO <clamp_type> as defined in the clamp cell file with the 'TYPE” keyword. The clamps specified in FROM_CLAMP_TYPE are used as reference clamps and all reporting is done with respect to the “FROM*” list items. Outputs are available in the *adsRpt/ESD/esd_*.rpt* file.

CLAMP_POS_PIN [0|1] : when set to 1 (default is 0), ESD B2C resistance check will pick up only the nodes connected to the positive/negative pins of a clamp diode device; for example, as in:

```
ESD_PIN_PAIR Vsig Vdd 0.1 off
```

where the positive pin is Vsig, and the negative pin is Vdd.

CLAMP_NEG_PIN [0|1] : same behavior as CLAMP_POS_PIN for negative pins

USE_CLAMP_FILE <file> : in the specified file, the following syntax can be defined.

```
USE_CLAMP_CELL
    <cellName> [<locID1> [<locID2>]]
      ...
END_CLAMP_CELL
USE_CLAMP_INST
    <instName> [<locID1> [<locID2>]]
      ...
END_CLAMP_INST
```

Note that wildcards are honored in the clamp cell and instance names, and the locIDs are optional. The above keywords can be applied to B2B/B2C/CD rules for clamp selections.

B2B_LOOP_LENGTH : <clamp_countN> values specify the number of clamps in each B2B path (positive integers less than or equal to MAX_ESD_STAGE). Values of <clamp_countN> *not included* mean that B2B paths with that number of clamps are excluded from parallel R computation.

RADIUS : specifies the maximum distance between B2B, B2C or C2C pairs that is checked (in microns) for the rule. Note that in B2B checking, if no bump pairs fall within the radius specified, PathFinder automatically extends the RADIUS value to include about 10% of the number of bump pairs specified in the option B2B_PAIR_COUNT. A message about what the RADIUS value is and how many pairs are included is displayed.

INCREMENTAL_MODE : when the RADIUS option is used in B2B resistance checks, provides incremental resistance check coverage when users have already run with a given radius and want to rerun with an increased radius.

REPORT_B2C [0 |1] : default 0; reports B2C results as well with B2B

REPORT_C2B : default 0; reports C2B results as well with B2B checks (clamp-centric reporting). the report format in *esd_fail.rpt/esd_pass.rpt* is :

```
# CLAMP: <X> <Y> <LAYER> <NET> <LocID> <CLAMP_INST>
# <R> <X> <Y> <LAYER> <NET> <BUMP_NAME>

CLAMP:  2606.25 3170    METAL3  VDD     ADS_VDD
cinst_123(diode)
5.65142 1616    4968    METAL6  VDD     DVDD31
4.79307 4316    39      METAL6  VDD     DVDD30
...
```

-o : specifies the output filename

REPORT_C2C [0 |1] : default 0; reports C2C results as well with B2B

EXCEL_REPORT: when set, creates a consolidated ESD report in Excel format for resistance and CD checks *(adsRpt/ESD/esd_excel.rpt)*, so that it can be easily sorted and processed using Excel, and also easily parsed by other flows downstream.

B2B_PAIR_COUNT <num>: defines the maximum number of B2B pairs to be computed. The default pairs limit is 100,000, which is a “soft limit” in that the actual B2B count is based on the RADIUS value, but it should be close to the specified number in practice. Whether a RADIUS is specified or not, when the B2B count exceeds the B2B_PAIR_COUNT value, PathFinder automatically adjusts the RADIUS value so that the B2B count is close to the B2B_PAIR_COUNT. When a small RADIUS is used, and no B2B pair exists

within the specified RADIUS, the RADIUS is automatically increased to compute about 10% of B2B_PAIR_COUNT (or 10% of the pairs in the design, whichever is smaller).

B2B_LOOP_COUNT: controls the number of lowest-R bump-clamp and bump-bump loops (loops with loop resistances less than LOOP_R) in the output report. The default for B2B_LOOP_COUNT is 5.

B2B_CHECK_IV : To include clamp voltage and current and I-V data in B2B resistance check report. This keyword supports new B2B report format (that has additional VT1, RON information of current format) with or without ZAP_CURRENT for B2B checking.

MAX_ESD_STAGE : specifies the maximum number of stages that can be checked (default 5).

CLAMP_SELECT : specifies the method of selecting nodes for clamp cells in each net using the minimum or maximum resistance from grid checking. Used only if x,y location is not specified for all the pins in a domain. Default is minimum.

EM_MODE : specifies ‘avg’ or ‘peak’ values to be used for current density checking (default peak). If EM MODE is specified in both the GSR and in the ESD rules file, the rules file setting has priority.

DETAILED_MODE : B2C reports a pair as failing only if all the arc resistances to it are failing. In other words, if at least one arc to a clamp passes (arc resistance less than or equal to ARC_R), then all arc resistances for that clamp are reported as passing. If this keyword is set to 1, then if any of the arcs fail, a clamp failure is reported in *esd_fail.rpt*. Default off.

SHORT_BUMP_IN_NET: specifies a one or more nets for which all bumps are shorted together to calculate minimum resistance, and the minimum resistance path is then traced from the clamp to the nearest bump on the shorted net. Multiple entries of this keyword can be use to specify multiple nets, but only nets specified in a single rule entry are shorted together.

NO_SHORT_BUMP_IN_NET : allows excluding specified nets otherwise shorted as a group using SHORT_BUMP_IN_NET_GROUP.

SHORT_BUMP_IN_NET_GROUP : shorts bumps together in each domain in the specified group. If ‘All’ is specified, shorts bumps in all domains in the specified group together.

SHORT_MODE : when set, produces a one-line text report for each B2B pair, with the format:

```
(<BUMP1> <X> <Y> <LAYER> <NET>) <-> (<BUMP2> <X> <Y>
   <LAYER> <NET>) <para_R>
```

B2B_MODE : specifies N- stage B2B checking. Default 0, Uses original methodology and reporting for 1- and 2-stage B2B checks.

ALL_PAD_NODE : controls node selection when plocs are created from pins in pcells. If set, all ploc nodes are picked up for resistance and CD checks. By default, only one node is picked up for each pcell pin.

SAME_CLAMP_INST & DIFF_CLAMP_INST : to compute and report resistances between same/different clamp instances when more than one point are declared on a pin of clamp cell, for C2C resistance computation and ESD-CD analysis. Both keywords are default 0 (off), i.e, will report all clamp-to-clamp resistances.

SAVE_MAP: Saves the analysis map specified with the keyword, in both GUI mode and batch mode.

Note: PathFinder recognizes wildcard characters in the rule file for keywords related to domain filtering, to allow you to more easily filter nets in ESD analysis. The ESD keywords that support wildcards are:

DOMAIN
B2B_NET1 (previously 'FROM_NET')
INTERNAL_NET
SHORT_BUMP_IN_NET
TERMINAL_NET
B2B_NET2 (previously 'TO_NET')

Sample Rules File

An example of the syntax used in the ESD rules file for the types above is shown below:

```
BEGIN_ESD_RULE
  TYPE [ BUMP2BUMP | BUMP2CLAMP | CLAMP2CLAMP ]
  # Note that a TYPE statement is required
  # Syntax for CLAMP2INST and CLAMP2MACRO rule types are described below
  NAME  rules1  # the reserved TYPE names are not allowed for the rule name
  RADIUS   1000 # described following
  ARC_R 1.5   # resistance check threshold for BUMP2CLAMP or CLAMP2CLAMP
  LOOP_R   10.85  # loop resistance check threshold for BUMP2BUMP
  PARALLEL_R 2.45 # parallel resistance check threshold for BUMP2BUMP
  ESD_STAGE <min> <max>
  B2B_LOOP_COUNT <count>
  MAX_ESD_STAGE <stage>
  PACKAGE 1
END_ESD_RULE
```

For example,

```
BEGIN_ESD_RULE
    TYPE BUMP2BUMP
    NAME  rules1
    RADIUS   1000
    LOOP_R   10.85
    PARALLEL_R  1.45
    ESD_STAGE 1 6
    B2B_LOOP_COUNT 4
    MAX_ESD_STAGE 6
    PACKAGE 1
END_ESD_RULE
```

For rules1, BUMP2BUMP is checked, since LOOP_R and PARALLEL_R are defined. The *esd.clamp* file contains the clamp cell definition, as described in the following section.

Resistance Checking for CLAMP2INST (C2I) and CLAMP2MACRO (C2M) Rules

The same basic resistance checking command, 'perform esdcheck', is used for the bump rules as for clamp rules (see section "perform esdcheck", page 12-369 for detailed syntax).

Use the 'CHECK_CONNECTION 1' rule setting to invoke this check.

Rules File Syntax for Types C2I and C2M

A description of the syntax in the ESD rules file for CLAMP2INST and CLAMP2MACRO checking is shown below:

```
BEGIN_ESD_RULE
 TYPE [CLAMP2INST(or C2I) | CLAMP2MACRO (or C2M)]
 NAME <rule name>
 LOOP_R <R_Ohms>
 LAYER [TOP |BOTTOM |<layer>]
 DOMAIN <net1> [<net2> ... ]
 DOMAIN_GROUP { POWER | GROUND | SIGNAL | <userDefinedGroupName> ]
 CLAMP_CELL <cell1> [<cell2> ... ]
 CLAMP_INST_FILE <filename>
 CLAMP_INST_NAME <inst1> [<inst2> ... ]
 CLAMP_AREA <llx> <lly> <urx> <ury>
 CELL <cellname1> [<cellname2> ... ]
 SHORT_CLAMP_NODES [ 0 | 1]
 CELL_FILE <filename>
 INSTANCE <inst1> [<inst2> ... ]
 INST_FILE <filename>
 ARC_R <maxR>
 THREAD_BY_NET [ 0 | 1 ]
 AREA <llx lly urx ury>
 SAMPLE_INST <%_instances_sampled>
 INST_COUNT <num_instances_sampled>
 SAMPLE_POINT <%_points_per_inst>
 POINT_COUNT <num_points_per_inst>
 SAMPLE_MODE [ UNIFORM ?<region_num>? | RES_BY_AREA ?<region_num>? ]
 B2B_LOOP_LENGTH <clamp_count1> <clamp_count2> ...
 PACKAGE 1
 PACKAGE_PIN_FILE <pkg_fileName>
 EXCLUDE_CELL <cellName>
 EXCLUDE_CELL_FILE <fileName>
 EXCLUDE_INSTANCE <instName>
 EXCLUDE_INSTANCE_FILE <fileName>
 IGNORE_MID_BLOCK [ 0 | 1]
 CLAMP_SELECT [min| max]
 STDCELL_SELECT [min| max]
 MACRO_SELECT [min| max]
END_ESD_RULE
```

where

TYPE : specifiies the checking rule type, either CLAMP2INST(C2I) or CLAMP2MACRO (C2M). Required

NAME : user-specified rule name; reserved TYPE names are not allowed as rule names. Required input that specifies the resistance check threshold (Ohms) for BUMP2CLAMP or CLAMP2CLAMP rule, and CLAMP2INST and CLAMP2MACRO checks. If specified, the value overrides the arc_R value defined in the ESD rules file.

ARC_R : specifies resistance check threshold (Ohms). If not specified, defaults to 0.5X LOOP_R. Either ARC_R or LOOP_R must be given.

THREAD_BY_NET : by default (0), allows using more than one thread per net when arc-based C2I analysis is performed (if set to 1, allows only one thread per net)

LOOP_R: loop resistance checking threshold (Ohms) of a single ESD path. Overrides the loop_R value defined in the ESD rules file. If not specified, defaults to 2X ARC_R. Either ARC_R or LOOP_R must be given.

LAYER [TOP| BOTTOM | <layer>] : checks nodes of pins on the selected layer

DOMAIN <net1> [<net2> …] : specifies only nets to be considered

DOMAIN_GROUP - allows specifying a group of domain nets of a certain type to be analyzed in arc-based C2I (and other checks that support the DOMAIN option). Also allows user-defined and named net groups, using the BEGIN_NET_GROUP rules definition.

CLAMP_CELL <cell1> [<cell2> ...] : specifies clamp cells. Multiple entries can be used. ESD clamp cells can be automatically identified and extracted from GDS by specifying marker layers in the layer map used in GDS2DEF. The presence of a marker layer inside the cells causes GDS2DEF to classify them as ESD devices and output them in LEF/DEF format.

CLAMP_INST_FILE <filename> : specifies an instance file filename. Multiple entries can be used.

CLAMP_INST_NAME <inst1> [<inst2> …]: specifies clamp instance names to be considered. Multiple entries can be used.

CLAMP_AREA <llx lly urx ury> : specifies clamp area in a bbox to be considered.

SHORT_CLAMP_NODES : supports ESD-R and ESD-CD checks by shorting clamp nodes “on-the-fly” when the resistances are computed.

CELL : specifies one or more cell names to be checked. Wildcard “*” can be used to represent one or more characters to specify groups of cells.

CELL_FILE: specifies a file that lists cells to be checked. Wildcard “*” can be used to represent one or more characters to specify groups of cells.

INSTANCE <inst1> [<inst2> …] : specifies core instance names to be considered. Multiple entries can be used. Wildcard “*” can be used to represent one or more characters to specify groups of instances.

INST_FILE <filename>: specifies a file that contains core instance names. Multiple entries can be used. Wildcard “*” can be used to represent one or more characters to specify groups of instances.

ARC_R <maxR> : specifies the threshold for arc resistance checking; required for B2C, C2C, C2I, and C2M rules (resistance above threshold considered a failure)

AREA <llx lly urx ury> : specifies a core area in a bbox to be considered

SAMPLE_INST : specifies the percentage of instances in the design to sample in analysis in C2I analysis.

INST_COUNT : specifies the number of instances to sample in the design

SAMPLE_POINT: specifies the percentage of points per instance to be sampled for macro instances. At least one point per net per instance is sampled.

POINT_COUNT: specifies the number of points per instance to be sampled for macro instances. At least one point per net per instance is sampled.

SAMPLE_MODE : by default C2I samples at least one node from each net of every macro/instance. The distribution of instance sampling can be modified using the options:

UNIFORM: samples an equal number of instances from each design region

RES_BY_AREA: samples more instances from design regions with higher gridcheck resistance

<region_num>: specifies the number of regions to be created in the design for either UNIFORM or RES_BY_AREA modes. For example, if the <region_num> is set to 5, the design is uniformly partitioned into 5 X 5 = 25 regions. The default <region_num> is 10 (100 regions), and the maximum region number setting is 31 (961 regions).

B2B_LOOP_LENGTH : <clamp_countN> values specify the number of clamps in each B2B path (positive integers less than or equal to MAX_ESD_STAGE). Values of <clamp_countN> *not included* mean that B2B paths with that number of clamps are excluded from parallel R computation.

PACKAGE 1 : includes package subcircuit in analysis

PACKAGE_PIN_FILE <pkg_fileName>: specifies a file identifying nodes in the package subcircuit that should be treated as external pins, so that ESD checks compute resistances beginning and ending at those nodes.

EXCLUDE_CELL <cellName> : excludes instances with specified cell names. Wildcard "*" can be used to represent one or more characters to specify groups of cells.

EXCLUDE_CELL_FILE: specifies a file that lists cells to be excluded from checks. Wildcard "*" can be used to represent one or more characters to specify groups of cells.

EXCLUDE_INSTANCE <instName> : specifies instances to be excluded. Wildcard "*" can be used to represent one or more characters to specify groups of instances.

EXCLUDE_INSTANCE_FILE <fileName>: specifies a file name that contains a list of instance names to be excluded from CDM checks. Wildcard "*" can be used to represent one or more characters to specify groups of instances.

CLAMP_SELECT : specifies the pin location for each power/ground domain for all clamp cells. Provides the minimum or maximum resistance to all pads if the <x/y> location is not specified for all the pins in a domain. Default is minimum.

IGNORE_MID_BLOCK : filters mid-level block instances from the instance list for C2M/C2I analysis With this option specified, PathFinder checks B2I/C2I/C2M only for leaf level instances. If the instance list or native sampled instance list contains block instances, they are filtered out. Also, it reports the selected instances in the *esd_info.rpt* file in the *adsRpt/ESD* directory by default, or in the specified output directory. Default: off (0).

Note that for MMX blocks, where the original cell is a DEF cell, it has an 'adsU1' instance in it, and selecting the adsU1 instance in this case is the only way for PF-S to perform the analysis, regardless of the IGNORE_MID_BLOCK flag setting.

STDCELL_SELECT : specifies the pin location for each power/ground domain for all sampled standard cells. Provides the minimum or maximum resistance to all the pads. Default is minimum.

MACRO_SELECT : specifies the pin location for each power/ground domain for all sampled macro cells. Provides the minimum or maximum resistance to all the pads. Default is maximum.

Examples of Rule Checking

Assume two instantiated clamp instances C1 and C2 in a design specified in the clamp file using '-clamp' option, and three core instances, I1, I2, and I3, specified in the file i*nst.list*. Several examples of rule files for this case are given following.

Example 1 for C2I Rule Check

To run a C2I rule check, run ESD check on the three core instances I1, I2 and I3 to all clamp instances, including C1 and C2. Use the following settings in the rule file:

```
BEGIN_ESD_RULE
    NAME core_usage_1
    TYPE C2I
    INST_FILE inst.list
    ARC_R 2.3
    LOOP_R 4
END_ESD_RULE
```

Example 2 for C2I Rule Check

Run an ESD check on the three core instances I1, I2 and I3 to the clamp instance 'C1' while a clamp file is defined with ‘-clamp’ option on the ‘perform esdcheck …’ command line. Use the following rule settings:

```
BEGIN_ESD_RULE
    NAME core_usage_2
    TYPE C2I
    CLAMP_INST_NAME C1
    INST_FILE inst.list
    ARC_R 2.3
    LOOP_R 4
END_ESD_RULE
```

Example 3 for C2I Rule Check

Run an ESD check on the full chip, with automatic instance sampling to all the instantiated clamp instances from the clamp file defined with the ‘-clamp’ option in ‘perform esdcheck ... ‘ command line. You can optionally control the instance sampling percentage using the SAMPLE_INST keyword. Use the following rule settings:

```
BEGIN_ESD_RULE
    NAME core_usage_2
    TYPE C2I
    ARC_R 2.3
    LOOP_R 4
    SAMPLE_INST <value in %>
END_ESD_RULE
```

Example 1 for C2M Rule Check

Run an ESD check on the three macro instances I1, I2 and I3 to all clamp instances instantiated from a clamp file defined with the ‘-clamp’ option in ‘perform esdcheck …’ command line. Use the following rule settings:

```
BEGIN_ESD_RULE
    NAME C2M_usage_1
    TYPE C2M
    INST_FILE inst.list
    LAYER BOTTOM # choose nodes from the bottom layer
    SAMPLE_POINT 20 #
    ARC_R 2.3
END_ESD_RULE
```

Note in this case 20% of nodes are selected; if this option is not specified, automatic sampling is performed that considers both resistance weakness and a reasonable run time.

Example 2 for C2M Rule Check

Run an ESD check on three MMX instances I1, I2 and I3 (can be a Custom Macro Model from Totem) to all clamp instances instantiated from a clamp file defined with the '-clamp' and options in 'perform esdcheck …' command line. To check transistor pins in MMX instances, use the 'TRANSISTOR_PIN 1' rule setting, as shown in the rule file following (transistor names are included in *esd_pass.rpt* and *esd_fail.rpt* reports).

```
BEGIN_ESD_RULE
    NAME C2M_usage_1
    TYPE C2M
    INST_FILE inst.list
    TRANSISTOR_PIN 1
    LAYER BOTTOM # choose the nodes from the bottom layer
    SAMPLE_POINT 20 # with 20% of node selections; if
notspecified, automatic sampling that consider both
resistance weak-ness in a reasonable run time
    ARC_R 2.3
END_ESD_RULE
```

Connectivity Checking

For C2I and C2M modes, the rule 'CHECK_CONNECTION 1' is provided to check the connectivity of core instances to the clamp instances. The rule and clamp files are the same as for a C2I or C2M run. Example rule file:

```
BEGIN_ESD_RULE
    NAME C2I_rule
    TYPE CLAMP2INST
    CHECK_CONNECTION 1
END_ESD_RULE
```

CLAMP2INST Connectivity

Most of the other C2I or C2M option/rule entries are inactive with the '-checkConn' option, except for the ones that are related to clamps, such as clamp cells/insts/pins. For a given set of clamps, these are checked for all instances in the design.

1. If an instance is connected to clamp(s) for both power and ground, it lists those that are completely disconnected or partially disconnected. For example, in the checkConn report, the PWR and GND status are reported separately, so if either P or G is not connected to clamp, it is reported. An example is:

   ```
   # POWER   GROUND  <instName>
   PWR_OK  NO_GND   I1/I2
   NO_PWR  GND_OK   I3/I4
   NO_PWR  NO_GND   I5/I6
   ```

2. For the connected instances, the ones that are marked as "macro" as determined by Totem, are listed first. A "macro" is usually a memory, IP, or very large cell, which is automatically selected for C2M analysis if there is no user-specified instance file.

3. All switch instances in the design are listed so you know that it is justified for switch instances to have a one-ended connection in the loop-R report. When the rule type is CLAMP2INST, PF-S checks and reports the selected node per domain in each instance that is not connected to any clamp instances. The connectivity info is recorded in the *adsRpt/ESD/esd_info.rpt* file.

A sample Instance and Connectivity Report for Rule <C2I> (CLAMP2INST) follows:

```
BEGIN_ESD_RULE
    NAME C2I
    TYPE CLAMP2INST
    CHECK_CONNECTION 1
END_ESD_RULE


# Total number of unprotected instances: 8
# POWER GROUND  <INSTANCE>
NO_PWR  NO_GND  inst_129199
PWR_OK  NO_GND  inst_129203
NO_PWR  NO_GND  inst_129208
NO_PWR  GND_OK  inst_129216
...

# Total number of macro instances: 44
# MACRO INSTANCE NAMES:
inst_129400/adsU1
inst_129422/inst_7693
...

# Total number of switch instances: 884
# SWITCH INSTANCE NAMES:
inst_129973/switch_inst_R1_C1
inst_129973/switch_inst_R2_C1
inst_129973/switch_inst_R3_C1
...
```

CLAMP2MACRO Connectivity

When the rule type is CLAMP2MACRO, PF-S checks and reports points (nodes) in macros (as automatically determined in the Totem database, or as specified with INSTANCE_FILE or '-instfile <file>'), that are not connected to any clamp instances. The report format is:

```
# Macro Instance and Connectivity Report for Rule <C2IM>
(CLAMP2MACRO)

BEGIN_ESD_RULE
  NAME C2IM
  TYPE CLAMP2MACRO
  CHECK_CONNECTION 1
END_ESD_RULE

# INST <NAME> <X1 Y1 X2 Y2> <NUM_UNPROTECTED_POINTS>
# <XY LOCATION> <LAYER> <DOMAIN> <PIN>
```

```
INST I1/u428/I356 1931.8 0 3900 3915 103
2572   3516.6 m5 vdd vdd
2658.4 3574.2 m5 vss vss
2658.4 2969.4 m5 vdd1 vdd1
2572   3574.2 m5 vvdd vdd
...

Summary: Found 3 instances with 204 unprotected points.
```

When the ‘-transistor’ option is used, only nodes on transistor pins for MMX blocks are checked and the transistor names are listed in the report:

```
# INST <NAME> <X1 Y1 X2 Y2> <NUM_UNPROTECTED_POINTS>
# <XY LOCATION> <LAYER> <DOMAIN> <PIN> <TRANSISTOR>
INST FIFO0/TX0FCORE/ram64/adsU1 1757.36 785.035 1903.34 915.375 105
1818.41 871.955 METAL1 VSS VSS.gds299  X0.X67.X0.X2.M5
1844.05 858.335 METAL1 VDD VDD.gds1651 X50.X0.M30
1859.39 858.335 METAL1 VDD VDD.gds1729 X53.X0.M30
```

Combined Rules and Clamp Cell Pin Location File

The ESD rules file, which defines the rule name, and the clamp cell pin location file, which specifies the pin location of each clamp cell, can also be merged into a single file, as specified in the '-rule' option. The combined file can be a simple concatenation of the two files as described above. The option '-clamp' is not needed for a combined file.

Invocation

Execute PathFinder from the Totem2 GUI using pull-down menus, by selecting **Tools -> ESD** to display the ‘ESD Analysis’ menu. Then import design data, perform **Network Extraction**. Then run ESD static resistance checks from the menu **Static -> ESD Resistance Check**, which displays the ‘ESD Resistance Check’ dialog box, as shown in Figure 12-5.

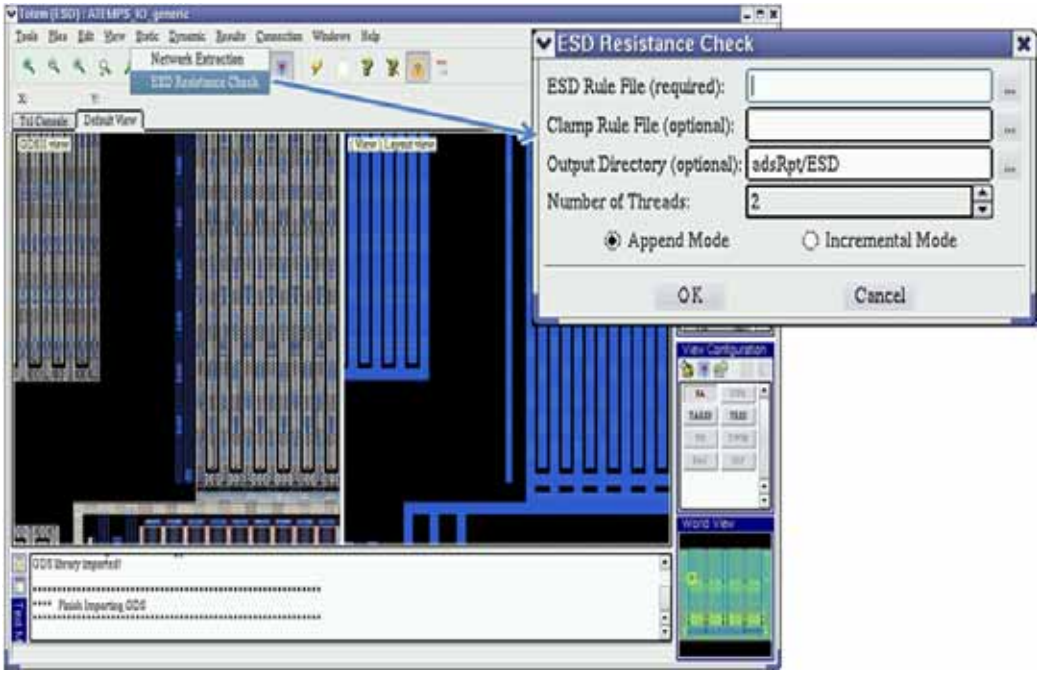


Figure 12-5 Running ESD Resistance Checks

Fill in the following information in the Resistance Check dialog box:

- ESD Rule File (required): a file that describes the type of the checking, the thresholds, and other settings related to ESD checking rule selected.
- Clamp Rule File (optional): a file that describes information related to clamps of interest
- Output directory (optional) : specifies the directory for the output files. Default is adsRpt/ESD.
- Number of threads: selects the number of parallel jobs to be run.

Select the **Append Mode** button, which appends results to the output files instead of overwriting the previous output, or the **Incremental Mode** button, which performs incremental instance and macro point selection to continue from previous 'perform esdcheck' runs for C2I and C2M rules.

In addition to using the GUI invocation, the ‘perform esdcheck’ TCL command can also be used to launch static ESD resistance checking.

Resistance Checking for BUMP2INSTANCE Rules

PathFinder also supports Bump-to-Instance resistance checks. For every bump, it compares the resistances to instances and the resistance to its protecting clamp. The protecting clamp of a bump is the clamp with the smallest resistance to the bump. If a bump has any resistance to an instance that is smaller than the resistance to its protecting clamp, the bump fails the B2I check. The same ESD resistance checking command, ‘perform esdcheck’, is used for performing B2I checks. The most common options used for performing B2I checking are similar to those for performing C2I checking.

Rule File Syntax for Types B2I

A description of the syntax in the ESD rules file for BUMP2INSTANCE checking is shown below:

```
BEGIN_ESD_RULE
    TYPE [ BUMP2INST | B2I ]
    NAME <rule_name>
    B2C_NAME <b2c_rule_name>
    B2I_ARC_COUNT <b2i_arc_num>
    CELL <cellname1> [<cellname2> ... ]
    CELL_FILE <filename>
    INSTANCE <inst1> [<inst2> ... ]
    INST_FILE <filename>B2I_ARC_COUNT <b2i_arc_num>
    EXCLUDE_CELL <cellName>
    EXCLUDE_CELL_FILE <fileName>
    EXCLUDE_INSTANCE <instName>
    EXCLUDE_INSTANCE_FILE <fileName>
    EXCLUDE_BUMP <bump1> ... <bumpN>
    BUMP_FILE <filename>
    EXCLUDE_BUMP_FILE <filename>
    DOMAIN <net1> <net2> ...
    B2I_REPORT_PARTIAL_PASS [ 0 | 1 ]
    B2I_MACRO_MODE  [ 0 | 1 ]
    SAMPLE_INST <%_instances_sampled>
    INST_COUNT <num_instances_sampled>
    SAMPLE_MODE [ UNIFORM ?<region_num>? | RES_BY_AREA ?<region_num>? ]
    RES_RATIO <ratio>
END_ESD_RULE
```

where

B2C_NAME: specifies the previously-performed B2C rule name to obtain the resistance to the protecting clamp of each bump. If this keyword is not specified, B2C resistance for the bumps is computed.

B2I_ARC_COUNT: specifies the number of B2C arcs to be reported in *esd_pass.rpt* and *esd_fail.rpt*. The default number of B2C arcs reported is 5.

CELL_FILE: specifies a file that lists cells to be checked. Wildcard "*" can be used to represent one or more characters to specify groups of cells.

INSTANCE <inst1> [<inst2> …] : specifies core instance names to be considered. Multiple entries can be used. Wildcard "*" can be used to represent one or more characters to specify groups of instances.

INST_FILE <filename>: specifies a file that contains core instance names. Multiple entries can be used. Wildcard "*" can be used to represent one or more characters to specify groups of instances.

EXCLUDE_CELL <cellName> : excludes instances with specified cell names. Wildcard "*" can be used to represent one or more characters to specify groups of cells.

EXCLUDE_CELL_FILE: specifies a file that lists cells to be excluded from checks. Wildcard "*" can be used to represent one or more characters to specify groups of cells.

EXCLUDE_INSTANCE <instName> : specifies instances to be excluded. Wildcard "*" can be used to represent one or more characters to specify groups of instances.

EXCLUDE_INSTANCE_FILE <fileName>: specifies a file name that contains a list of instance names to be excluded from CDM checks. Wildcard "*" can be used to represent one or more characters to specify groups of instances.

EXCLUDE_BUMP : list of bumps to be excluded in checking

BUMP_FILE : name of file that lists bumps to be included in checking

EXCLUDE_BUMP_FILE : name of file that lists bumps to be excluded in checking

DOMAIN : list of nets to be checked. Others are excluded.

B2I_REPORT_PARTIAL_PASS : when set to 1 (default is 0), "partial pass" bumps are reported in *esd_pass.rpt*. A partial pass bump is a bump with both failing and passing B2I arcs. By default only bumps without failing B2I arcs are reported in *esd_pass.rpt.*

B2I_MACRO_MODE : when set to 1 (default is 0), multiple points per net per instance for larger instances (macro instances) are sampled. By default only one point per net per instance is sampled.

SAMPLE_INST : specifies the percentage of instances in the design to sample in analysis in B2I analysis.

INST_COUNT : specifies the number of instances to sample in the design

SAMPLE_MODE : by default B2I samples at least one node from each net of every macro/instance. The distribution of instance sampling can be modified using the options:

UNIFORM: samples an equal number of instances from each design region

RES_BY_AREA: samples more instances from design regions with higher gridcheck resistance

<region_num>: specifies the number of regions to be created in the design for either UNIFORM or RES_BY_AREA modes. For example, if the <region_num> is set to 5, the design is uniformly partitioned into 5 X 5 = 25 regions. The default <region_num> is 10 (100 regions), and the maximum region number setting is 31 (961 regions).

RES_RATIO : when specified, a B2I path is reported as failed if the Resistance (B2C path)/ Resistance(B2I path) > RES_RATIO

B2I rules also support all instance sampling rule keywords used for C2I/C2M, such as INSTANCE and AREA.

The associated 'report esdcheck' command syntax is :

```
report esdcheck -type b2i -name esd_b2i_rule -b2cName esd_b2c_rule
```

The default output directory is *adsRpt/ESD/,* which can be changed using the '-outDir' option. By default in the *esd_pass.rpt* and *esd_fail.rpt* report files five instance names and five B2I arcs are specified. You can control this feature using the rule keyword 'B2I_ARC_COUNT'.

Resistance and Voltage Checking from Bumps to Cross-domain Driver-Receivers

Introduction

Cross-domain driver-receiver design is generally susceptible to CDM discharging events. A typical design example that includes a VDD1 pin discharging in a CDM event is shown in Figure 12-6.

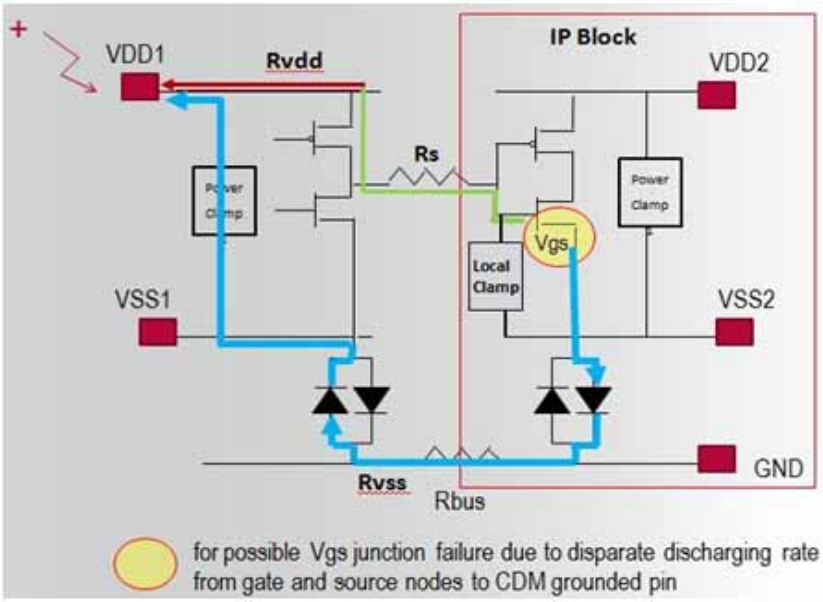


Figure 12-6 Cross-domain driver-receiver discharging paths

Usually, the Vgs junction on the receiver side sees the most stress, potentially leading to oxide breakdown (particularly for thin-oxide) due to different discharging rates from gate and source nodes to the CDM grounded pin. The common solution to this potential weakness includes insertion of resistance from the driver output of driver to the fan-in of the receiver, resulting in a voltage divider effect on the stress seen across the Vgs junction. In addition, a local clamp such as a GGNMOS can be added between the gate, the local ground for both parasitic BJT snapback discharging paths, and a parasitic diode to ground. A potential significant problem in today's multiple-IP multiple-domain SoC designs is that the driver is outside of the driven macro where the receiver resides. So the responsibility for protecting the cross-domain driver-receiver system therefore falls on the SoC designer to provide the needed ESD protection. The same macro also can be reused in different SoC designs, and therefore the Vgs stress observed is different in this case. One of the key cross-domain driver-receiver CDM failure mechanisms can be caused by a very unbalanced impedance path from the gate and source node of the Vgs junction to the CDM grounded pin. This is the reason for the following checks on cell-based designs as the first step in assessing this potential weakness.

The first check for cross-domain driver-receiver systems is focused on the cell-based cross-domain driver-receivers, as illustrated in Figure 12-7. The check compares the resistance value of path A1 versus A2, and path B1 versus B2, in absolute value, and also in percentage of R difference. After you provide a resistance difference threshold, PF-S highlights those driver-receiver pairs that failed the check. When there are not sufficient core clamps, you may see a large resistance difference for the cross-domain driver-receiver pair, and this may contribute to failure in a CDM discharging event.

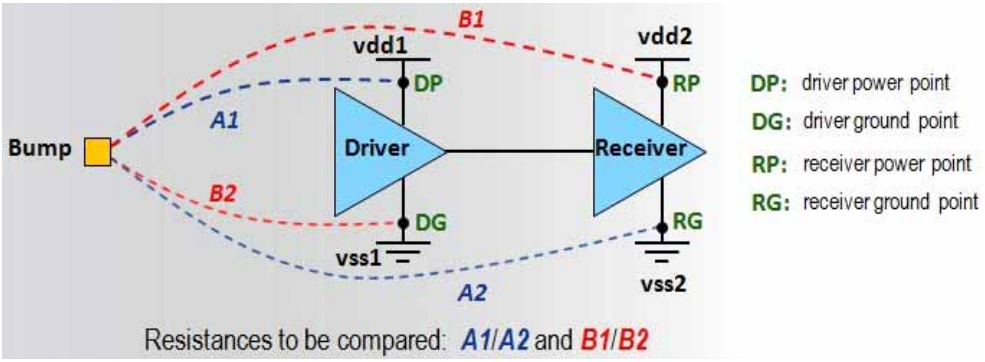


Figure 12-7 Cell-based cross-domain driver-receiver checking

Cross-domain Resistance/Voltage Check Rule Keywords

Rules for checking driver-receiver circuits that cross voltage domains are called DRIVER2RECEIVER (D2R) rules, and are used to check the difference or the ratio of the resistance from P/G pins of cross-domain driver-receiver instance pairs to the resistance of all bumps in designs that are connected directly or through clamps. D2R ESD rule keywords are:

D2R_P2P_CHECK [1|0] : activates driver-receiver point-to-point P/G resistance checks. Default: 0.

D2R_PG_MODE [G2G| P2P |P2G | G2P | DIFF]

G2G : computes ground-to-ground point pair checks

P2P: computes power-to-power point pair checks.

P2G - only resistance pairs of drivers' POWER nets and receivers' GROUND nets are checked

G2P only resistance pairs of receivers' POWER nets and drivers' GROUND nets are checked

DIFF : chooses point pairs on different nets, if there are any (default)

D2R_PAIR_MODE [ALL | VDD_DIFF | GND_DIFF]: specifies the types of driver-receiver pairs to be checked (if not specified, by default all driver-receiver pairs with different P/G domains are checked). The values are:

ALL - all driver-receiver pairs are checked (default)

VDD_DIFF - driver-receiver pairs with different POWER domains are checked.

GND_DIFF - driver-receiver pairs with different GROUND domains are checked.

D2R_PAIR_COUNT <number> : specifies the maximum number of estimated worst-case D2R point pairs to be computed. Default: 1000.

D2R_MIN_BOUND [1 |0] : activates the capability to reduce the total pair count for arc resistance calculation (if the ARC_R rule keyword is also specified). Default: 0.

D2R_DIFF <res_diff>: specifies the resistance difference between driver and receiver P/G nets as a pass/fail threshold.

D2R_RATIO <res_ratio>: specifies the resistance ratio between driver and receiver P/G nets as a pass/fail threshold.

D2R_NODE_FILE <node_file>: specifies a file listing driver-receiver node pairs to be checked. The node file syntax is as follows:

<Inst1> <Pin1/Tran1> ?<x> <y> ?<layer>? ? <inst2> <Pin2/Tran2> ?<x> <y> ?<layer>? ?

Note that one of either D2R_DIFF or D2R_RATIO is required for valid D2R checking, and that D2R_PAIR_MODE/D2R_PG_MODE and D2R_NODE_FILE keywords are mutually exclusive

ZAP_D2R_NET <net_name> : specifies the signal net to have P2P effective resistance checking, of those defined in GSR keyword ESD_SIGNAL_NETS

CDM_ZAP_ALL_NET 1 : distributes current on M1 nodes on all nets

CDM zap pin can be specified using any of the following command:

ZAP_CDM_BUMP <bumpname> : discharge to a specific bump (or)

ZAP_CDM_PIN <pin_name> : discharge to a specific pkg pin (or)

ZAP_CDM_NET <nets> : discharge to bumps in net one at a time (or)

ZAP_CDM_NET_GROUP <gname> : discharge to bumps in a net group one at a time

IP_D2R_NET_FILE: Defines the file which contains all primary IO signals in the IP.

REPORT_INSTANCE <N> : turns on the instance voltage check and reporting in ESD-CD check. The supported values for <N>:

0: No instance voltage check and report. Default value

1 (or D2R): Check and report driver-receiver voltages

2 (or DIFFV): Check and report differential instance voltage in esd_inst.rpt as before

3: To turn on both D2R and DIFFV

Sample rule file would be as follows:

```
BEGIN_ESD_RULE
NAME d2r
TYPE D2R
D2R_DIFF 20
END_ESD_RULE
```

Sample Rules for Driver-Receiver Resistance check

```
BEGIN_ESD_RULE
  NAME d2r_gg
  TYPE R
  ARC_R 30   # Pass/Fail Threshold
  D2R_P2P_CHECK 1  # to perform P2P res_calc
  D2R_PG_MODE G2G  # to restrict check on Ground nets
  D2R_PAIR_MODE ALL  # For all D2R, not just for cross-
domains
  D2R_MIN_BOUND 1  # To rank vulnerable D2R pairs
  D2R_PAIR_COUNT 10000  # to restrict count to
50k(default:1k)
  SHOTGUN_MODE 1  # Enable fast computation when possible
END_ESD_RULE
```

Sample Rules for Driver-Receiver differential voltage check

```
BEGIN_ESD_RULE
  NAME cdm_zap
```

```
  TYPE cdm
  ZAP_CURRENT 5
  ZAP_CDM_NET gnd
  LAYER M1 #CDM Zap current distributed on specified net/
layer
  DIFF_VOLT 5 #Differential voltage b/w driver-receiver
Ground pins
  REPORT_FAIL_ONLY 1
  CDM_ALL_NET 1 #Include all nets connected to VSS by
clamps
  D2R_PG_MODE G2G  # Restrict reporting to ground-2-ground
voltages
  D2R_PAIR_MODE ALL # For all D2R, not just for cross-
domain
  COMPUTE_EM 0 # Disable EM processing(for speed-up)
END_ESD_RULE
```

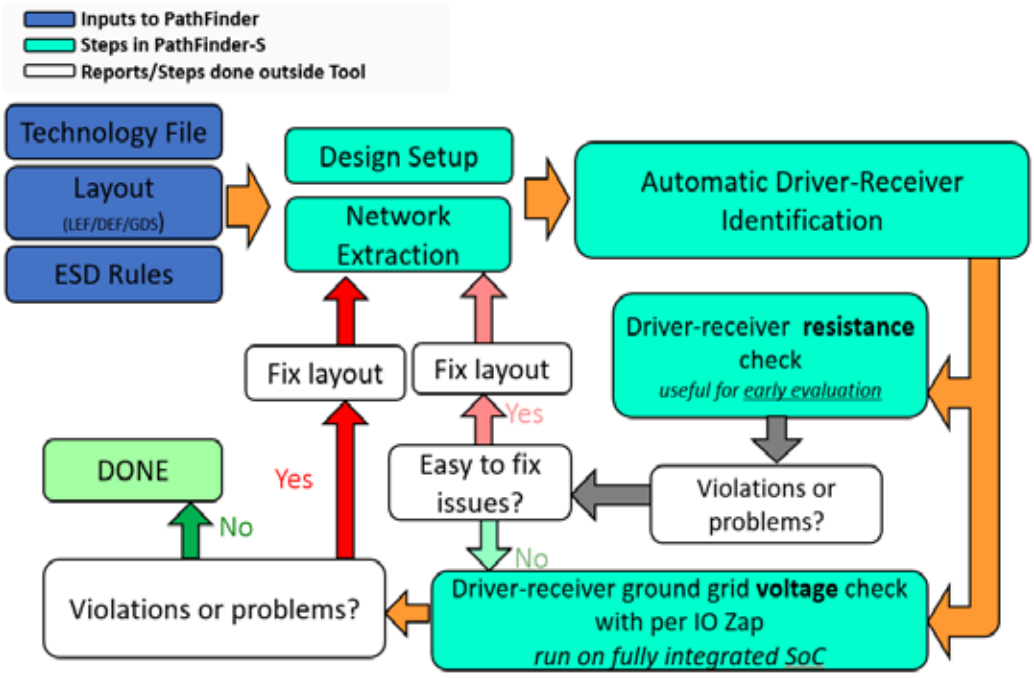


Figure 12-8 Pathfinder-S Driver-Receiver Check Flow

ESD Resistance Checking Reports

Report esdcheck command

After an ESD resistance check is performed with the 'perform esdcheck' command, Totem saves the results into the DB. The data is identified by the rule name. You can get text reports for any ESD resistance check with the TCL command ‘report esdcheck’ to specify the type of reports to be created :

```
report esdcheck -type <ruleType> ? -outDir <output_dir>?
? -name <rule_name>? ?-[glob |-regexp ] ?
? -arcR <arc_Res_thresh>? ? -loopR <loop_Res_thresh>?
? -parallelR <parallel_threshold>? ? -append?
```

```
? -failedOnly? ? -detail?
```

where

-type: reports results for the specified type of ESD checks. Default is all types.

-outDir : specifies the report files directory. Default is *adsRpt/ESD*

-name: specifies the name of the rule tested. If multiple checks have been specified within a single rule-- for example, two bump pairs are specified in the same rule-- PathFinder internally splits them into subrules, each with a single bump pair. Then it appends '(_\d+)+' to the base rule name. So when using 'report esdcheck', you can specify this derived name to look at the particular results desired.

-glob|-regexp: specifies the type of wildcard expressions used. PathFinder reports the results for all subrules that match the specification; 'glob' supports regular wildcard expressions and 'regexp' supports normal wild cards and also extended character strings. For example:

```
report esdcheck -detail -perNet -type B2C -name abc* -glob
report esdcheck -type B2C -name {M_test_[3-5]} -regexp
```

-arcR: specifies the threshold for arc resistance checking; required for B2C, C2C, C2I, and C2M rules (resistance above threshold considered a failure)

-loopR: specifies the threshold for loop-R checking; required for C2I

-parallelR: specifies the threshold for parallel-R checking; required for B2B

-append: appends results to existing results in the output directory.

-failedOnly: only reports failed results.

-detail : provides more detailed report results for B2B rules

You can perform basic filtering of the resistance checking results based on nets, clamp instances and clamp types. The types of filtering rules supported are:

CLAMP_CELL <cellName>: reports only B2C (C2C) arcRs that connect to clamp instances belonging to cell <cellName>

CLAMP_INST <instName>: reports only B2C (C2C) arcRs that connect to clamp instance <instName>

CLAMP_TYPE <clampType>: reports only B2C (C2C) arcRs that connect to clamp instances with type <typeName>

TERMINAL_NET <netName>: reports only B2C (C2C) arcRs that connect to net <netName>

Resistance Checking Output Reports

PathFinder resistance checking generates five types of resistance check report files for each rule, describing pad information and the resistance of each ESD path, in the directory *adsRpt/ESD*, as follows:

- *ClampInfo.rpt* - information on ESD clamp cells and instances
- *esd_pass.rpt* - information on passed ESD arcs/loops for each rule, listing one arc/loop for each passed instance, and resistance range.
- *esd_fail.rpt* - information on failed ESD arcs/loops for each rule, listing one arc/loop for each failed instance, and resistance range.
- *esd_info.rpt* - more information on passing and failing instances
- *esd_summary.rpt* - contains a summary of the passing and failing results for each rule check, including resistance range. Reports the exact number of resistances computed

for B2C and C2C checks.

Following are descriptions of different types of ESD reports for different rules.

B2I results reports

The results of a B2I check are generated after the ‘perform esdcheck’ command is executed with a B2I-type rule. The default output directory is *adsRpt/ESD/*. The results also can be generated by using the ‘report esdcheck’ command. An example ‘report esdcheck’ command follows:

```
report esdcheck -type b2i -name esd_b2i_rule -b2cName esd_b2c_rule
    -outDir b2i_report
```

Using the ‘-b2cName’ option is the same as using the “B2C_NAME” rule keyword. A sample *esd_fail.rpt* is shown below:

```
# ESD Check Results for Rule <esd_b2i_rule> (BUMP2INST)

BEGIN_ESD_RULE
  NAME esd_b2i_rule
  TYPE BUMP2INST
END_ESD_RULE

# Bump: <B2C_R> <X> <Y> <LAYER> <NET> <Bump_NAME>
        <MIN_B2I_RES> <MAX_B2I_RES> <PASS/FAIL_RATE>
# <B2I_R> <XY location> <layer> <Net> <Inst_name> <Pin> <Transistor>
Bump: 1.3208 4880 938.85 METAL4  VSS DVSS1 0.7537 4.0961 0.29%(7/2391)
     0.7537 4690.91 925.52 METAL4 VSS inst_129425/inst_7769
     1.1636 4460.62 893.39 METAL1 VSS inst_149915
     1.1969 4562.74 893.39 METAL1 VSS inst_148929
     1.20924589.42 893.39 METAL1 VSS inst_149950
     1.2929 4459.7 893.39 METAL1 VSS inst_148901

# Bump: <B2C_R> <X> <Y> <LAYER> <NET> <Bump_NAME> <MIN_B2I_RES>
        <MAX_B2I_RES> <PASS/FAIL_RATE>
# <B2I_R> <XY LOCATION> <LAYER> <NET> <INST_NAME>
Bump: 1.3923 4880 1058.85 METAL4 VSS DVSS2 0.9712 4.3061 0.08%(2/2391)
     0.9712 4690.91 925.52 METAL4 VSS inst_129425/inst_7769
     1.3458 4460.62 893.39 METAL1 VSS inst_149915
```

Passing and failing bumps and arcs are summarized in the report *esd_summary.rpt*, as shown in the sample following:

```
# ESD Check Summary for Rule < esd_b2i_rule > (BUMP2INST)

BEGIN_ESD_RULE
 NAME esd_b2i_rule
 TYPE BUMP2INST
END_ESD_RULE

Bump summary:
          8 bumps are checked.
          8 failed bumps. (100.00%)
          0 passed bumps. (0.00%)
```

```
Instance summary:
     348382 instances.
       2391 instances computed completely. (0.69%)
          0 instances computed partially. (0.00%)
     345991 instances to be computed. (99.31%)
Arc-R summary:
      19128 arc-Rs are checked.
         74 failed arc-Rs. (0.39%)
      19054 passed arc-Rs. (99.61%)
```

B2C Results Reports

B2C resistance checking dumps clamp-centric reports when the RADIUS option is set in the rule file and the keyword 'REPORT_C2B 1' is in the rule file, as follows:

```
BEGIN_ESD_RULE
NAME rulec2b
TYPE B2C
ARC_R 1
REPORT_C2B 1
RADIUS 1000
END_ESD_RULE
```

Clamp-centric reports have the following format:

```
# CLAMP: <X> <Y> <LAYER> <NET>  <LocID>  <CLAMP_INST>
# <R> <X> <Y> <LAYER>  <NET>  <BUMP_NAME>
CLAMP:  2606.25 3170   METAL3  VDD    ADS_VDD  cinst
 5.65142 1616  4968    METAL6  VDD  DVDD31
 4.79307 4316  39      METAL6  VDD   DVDD30
...
```

Bump-centric/Clamp-centric reports also can be generated later using the “report esdcheck” command, regardless of the setting in the RULE file. The syntax is:

```
report esdcheck -type b2c -name <xyz> -reportC2B -outdir
...
```

Clamp Info Reports

Detailed information on clamps is saved in the report file *adsRpt/ESD/ClampInfo.rpt* for B2B, B2C, C2C, C2I, and C2M runs. Since only clamp cell names are specified, PF-S saves all ESD instances found in *adsRpt/ESD/ClampInst.rpt*. ESD cells that are undefined in the clamp info file are reported in `adsRpt/ESD/esd_clamp_unused.rpt`

Sample ClampInfo Report (ClampInfo.rpt) for all ESD rule runs

```
For example,
RULE rules2 (CLAMP2INST):
BEGIN_ESD_RULE
  NAME rules2
  TYPE CLAMP2INST
  ARC_R 6
  LOOP_R 10
END_ESD_RULE
COVERAGE INFO:
Coverage area: 0 0 8640 9570
Number of instances: 782404
Number of selected instances: 4129
```

```
Number of valid instances: 4129
CLAMP CELL INFO:
BEGIN_CLAMP_CELL
  NAME TEST_VDDO_APACHECELL
  PIN VDDC 20 314 metal3
  PIN VSSC 20 10 metal3
  RON 0.1
END_CLAMP_CELL
CLAMP INSTANCE INFO:
CLAMP CELL: TEST_VDDO_APACHECELL {
#<INSTANCE> <X1 Y1 X2 Y2> <ORIENT>
#<XY LOCATION> <LAYER> <NET> <PIN> [<locID>]
chip_pads/test_VDDO_6_186/adsU1     5900.000000
0.000000        5930.000000     382.000000      N
5915.000000     313.000000      metal3  VDD     VDDC
5920.280000     10.000000       metal3  VSS     VSSC
}
```

Pass/Fail Reports

For a **BUMP2BUMP** rule, an example report file for *esd_fail.rpt* is shown below (the format for *esd_pass.rpt* and *esd_fail.rpt* files is the same). Only one arc/loop is listed for each passed instance.

```
****************************************************************

# ESD Check Results for Rule <B2B_RULE> (BUMP2BUMP)

BEGIN_ESD_RULE
  NAME B2B_RULE
  TYPE BUMP2BUMP
  LOOP_R 3.4
  PARALLEL_R 2.5
END_ESD_RULE

# BUMP PAIR:(<BUMP1> <X> <Y> <LAYER> <NET>) <-> (<BUMP2> <X> <Y> <LAYER>
<NET>)
# PASS/FAIL: <LOOP_R> <ARC1_R> <ARC2_R> <CLAMP> <LOCID1> <LOCID2>
# PARALLEL R: <VALUE>

BUMP PAIR: (VDD_1 700 1600 M4 VDD) --> (VSS_2 1300 400 M4 VSS)
PASS:   3.18274 2.20349 0.969255        PVSS_D1_I17     VDD     VSS
PASS:   3.39066 1.49413 1.88652 PVSS_D1_I16     VDD     VSS
FAIL:   3.43285 2.12204 1.21081 PVDD_D1_I15     VDD     VSS
PASS:   3.30182 1.2371  1.96472 PVDD_D1_I14     VDD     VSS
PARALLEL R: 2.7047
```

The report for B2B shows resistance from one power pad to another power pad. The first line shows the I-th pad pair, pad names and pad locations. Then each ESD loop (single ESD path) is reported, including the checking results (pass or fail), resistance, location of the ESD clamp cell on this ESD path, and its instance name. After all ESD loops are reported, the total parallel effective resistance is calculated from the multiple ESD paths, and the result is compared against the parallel R limit.

The format in the example above is for a normal run. Using the '-detail' option, the output report format syntax is as follows:

```
BUMP PAIR: (<BUMP1> <X> <Y> <LAYER> <NET>) <-> (<BUMP2>
    <X> <Y> <LAYER> <NET>)
PASS/FAIL: <LOOP_R> <ARC1_R> <ARC2_R> <CLAMP> (<cell>)
    (<X> <Y> <LAYER> <NET>) (<X> <Y> <LAYER> <NET>)
PARALLEL R: <VALUE>
```

A sample *esd_fail.rpt* for **BUMP2BUMP for multiple stages** is shown following :

```
************************************************************

# ESD Check Results for Rule <B2B_RULE> (BUMP2BUMP)

BEGIN_ESD_RULE
  NAME B2B_RULE
  TYPE BUMP2BUMP
  LOOP_R 3.5
  PARALLEL_R 2.5
  ESD_STAGE 2
END_ESD_RULE

# BUMP PAIR: (<BUMP1> <X> <Y> <LAYER> <NET>) <-> (<BUMP2> <X> <Y> <LAYER>
<NET>)
# PASS/FAIL: <LOOP_R> <ARC1_R> <ARC2_R> <CLAMP> <LOCID1> <LOCID2> <INT_NET>
<CLAMP> <LOCID3> <LOCID4>
# PARALLEL R: <VALUE>

BUMP PAIR: (VDD_1 700 1600 M4 VDD) --> (VSSIO_9 1600 1600 M4 VSSIO)
FAIL:   5.00087 2.20349 1.75207 PVSS_D1_I17     VDD     VSS     VSS
PVSS_D1_I16     VSS     ADS_VSS1
PASS:   3.35621 1.49413 1.75207 PVSS_D1_I16     VDD     VSS     VSS
PVSS_D1_I16     VSS     ADS_VSS1
FAIL:   5.08782 2.12204 1.75207 PVDD_D1_I15     VDD     VSS     VSS
PVSS_D1_I16     VSS     ADS_VSS1
PASS:   3.4518  1.2371  1.75207 PVDD_D1_I14     VDD     VSS     VSS
PVSS_D1_I16     VSS     ADS_VSS1
PASS:   3.36413 1.2371  2.02703 PVDD_D1_I14     VDD     ADS_VSS1
PARALLEL R: 3.13897
```

In the report the resistance from one power pad to another power pad is reported in this report file. The first line shows the I-th pad pair, pad names and pad locations. Then each ESD loop (single ESD path) is reported, including the checking results (pass or fail), resistance, location of the ESD clamp cell on this ESD path, and its instance name. After all ESD loops are reported, the total parallel effective resistance is calculated from the multiple ESD paths, and the result is compared against the parallel R limit.

The format in the example above is for a normal run. With the '-detail' option the report syntax is:

```
BUMP PAIR: (<BUMP1> <X> <Y> <LAYER> <NET>) <->
    (<BUMP2> <X> <Y> <LAYER> <NET>)
PASS/FAIL: <LOOP_R> <ARC1_R> <ARC2_R> <CLAMP> (<cell>)
    (<X> <Y> <LAYER> <NET>) (<X> <Y> <LAYER> <NET>)
```

```
            <INT_NET> <CLAMP> (<cell>) (<X> <Y> <LAYER> <NET>)
            (<X> <Y> <LAYER> <NET>)
        PARALLEL R: <VALUE>
```

A sample *esd_fail.rpt* for **BUMP2CLAMP i**s shown following

```
**************************************************************

# ESD Check Results for Rule <B2C_RULE> (BUMP2CLAMP)

BEGIN_ESD_RULE
  NAME B2C_RULE
  TYPE BUMP2CLAMP
  ARC_R 2
END_ESD_RULE

# BUMP: <X> <Y> <LAYER> <NET> <BUMP_NAME>
# <R> <X> <Y> <LAYER> <NET> <LocID> <CLAMP_INST>

BUMP:   700     1600    M4      VDD     VDD_1
2.20349 1325    180     M3      VDD     VDD     PVSS_D1_I17
2.12204 1025    180     M3      VDD     VDD     PVDD_D1_I15

BUMP:   1000    400     M4      VDD     VDD_0
2.20083 1025    1820    M3      VDD     VDD     PVSS_D1_I16
2.11938 725     1820    M3      VDD     VDD     PVDD_D1_I14

BUMP:   1600    400     M4      VSSC    VSSC_15
2.07156 1025    20      M3      VSSC    ADS_VSS1        PVDD_D1_I15

BUMP:   1600    1600    M4      VSSIO   VSSIO_9
2.02703 725     1980    M3      VSSIO   ADS_VSS1        PVDD_D1_I14

Summary for rule <B2C_RULE> (BUMP2CLAMP):
        Failed bump-to-clamp resistances: 6 (30.00% of 20)
```

For each power pad, its location, layer name and net name are reported. Then the resistance from this pad to each ESD cell which is connected to this net are reported, including the resistance, port location on the ESD clamp cell, and cell name.

A sample *esd_fail.rpt* for **CLAMP2CLAMP** rule is shown following:

```
**************************************************************

# ESD Check Results for Rule <C2C_RULE> (CLAMP2CLAMP)

BEGIN_ESD_RULE
  NAME C2C_RULE
  TYPE CLAMP2CLAMP
  ARC_R 1
END_ESD_RULE

# CLAMP POINT: <X> <Y> <LAYER> <NET> <LocID> <CLAMP_INST>
```

```
# <R> <X> <Y> <LAYER> <NET> <LocID> <CLAMP_INST>

CLAMP POINT:    725     1870    M3      VSS     VSS     PVDD_D1_I14
1.05727 1025    130     M3      VSS     VSS     PVDD_D1_I15
1.01382 1325    130     M3      VSS     VSS     PVSS_D1_I17

CLAMP POINT:    1025    1820    M3      VDD     VDD     PVSS_D1_I16
1.02501 1325    180     M3      VDD     VDD     PVSS_D1_I17

CLAMP POINT:    1025    130     M3      VSS     VSS     PVDD_D1_I15
1.01371 1025    1870    M3      VSS     VSS     PVSS_D1_I16

Summary for rule <C2C_RULE> (CLAMP2CLAMP):
        Failed clamp-to-clamp resistances: 4 (28.57% of 14)
```

For each ESD clamp cell, its port location, layer name, port name, and cell name are reported. Then the resistance from this ESD clamp cell to other ESD clamp cells is reported.

The format for output reports in *esd_fail.rpt* for **C2I** rules is shown following:

```
************************************************************
# <loop_r> <arc_r> <x,y location> <layer> <domain> <pin> <arc_r>
<x,y location> <layer> <domain> <pin> <inst> <cell> <clamp> <dist>
<locid1> <locid2>
```

The format for output reports in *esd_fail.rpt* for **C2M** rules is shown following:

```
************************************************************
# inst <name> <cell> <x1 y1 x2 y2> <num_arcs> <num_passed_arcs>
<num_failed_arcs>
# <arc_r> <x,y location> <layer> <domain> <pin> <clamp>
<x,y location> <layer>
```

ESD reports the cell name in the output file for the instances analyzed in C2I or C2M analysis when either 'DETAILED_MODE 1' is specified in the rule file, or the '-detail' option is specified with the 'perform esdcheck' command.

ESD Info Reports

The ESD Info file reports contains additional information on clamps and circuit ESD protection, as shown in the following examples:

Bumps Not Connected to Clamps Info report

```
# ESD Check for Bumps Not Connected to Clamps for Rule <B2B_RULE> (BUMP2BUMP)

BEGIN_ESD_RULE
  NAME B2B_RULE
  TYPE BUMP2BUMP
  LOOP_R 3.5
  PARALLEL_R 2.5
  ESD_STAGE 2
END_ESD_RULE

# <BUMP> <X> <Y> <LAYER> <NET>
```

```
DVSS_11 400     1300    M4      DVSS
AVSS_7  1600    1000    M4      AVSS
AVSS_6  700     400     M4      AVSS
TX_22   1750    1150    M4      TX
S<4>_21 1750    850     M4      S<4>
S<3>_20 1750    550     M4      S<3>
```

Bumps and Domain Info Report

```
# ESD Check for Bump and Domain Info for Rule <B2B_RULE> (BUMP2BUMP)

BEGIN_ESD_RULE
  NAME B2B_RULE
  TYPE BUMP2BUMP
  LOOP_R 3.5
  PARALLEL_R 2.5
  ESD_STAGE 2
END_ESD_RULE

# DOMAIN PAIR <ID>: (<NET1> <NET2>) {
#   INTERNAL DOMAINS: <NUM>
#      <NET_NAMES>
#   BUMP PAIRS CHECKED: <NUM>
#   BUMP LIST FOR DOMAIN <NET1> (<COUNT>) {
#     <BUMP_LIST1> (<NAME> <XY_LOCATION> <LAYER> <NET>)
#   }
#   BUMP LIST FOR DOMAIN <NET2> (<COUNT>) {
#     <BUMP_LIST2> (<NAME> <XY_LOCATION> <LAYER> <NET>)
#   }
# }

DOMAIN PAIR 1: (VSSIO VSSC) {
  INTERNAL DOMAINS: 2
    VSS
    VDD
  BUMP PAIRS CHECKED: 2
  BUMP LIST FOR DOMAIN VSSIO (1) {
    VSSIO_9     1600    1600    M4      VSSIO
  }
  BUMP LIST FOR DOMAIN VSSC (1) {
    VSSC_15     1600    400     M4      VSSC
  }
}
```

ESD Summary Reports

A sample *esd_summary.rpt* for **BUMP2BUMP** rule is shown following

```
**************************************************************

# ESD Check Summary for Rule <B2B_RULE> (BUMP2BUMP)
BEGIN_ESD_RULE
```

```
  NAME B2B_RULE
  TYPE BUMP2BUMP
  LOOP_R 3.4
  PARALLEL_R 2.5
END_ESD_RULE

DOMAIN PAIR 1: (VSSIO VSS) {
  Clamp connections between (VSSIO VSS): 1
  Bump-to-bump pairs checked: 4
  Bump-to-bump loop resistances:
        Pass:      2 ( 50.0% of      4)
        Fail:      2 ( 50.0% of      4)
  Bump-to-bump parallel resistances:
        Pass:      0 (  0.0% of      4)
        Fail:      4 (100.0% of      4)
}
```

Special Resistance Calculations for ESD Analysis

The ‘perform res_calc’ command can be used for a number of special ESD analyses, and has some options that require a PF license to execute. These options are described in this section.

```
perform res_calc
? [-instance <name_list>]? ?[-instFile <filename>]?
? [-cell <name_list>]? ?-cellFile <cell_filename>?
? -from {<x1 y1> <layerName1> ?<netName>? }?
? -to {<x2 y2> <layerName2> ?<netName>?} ?
? -fromFile <file1> -toFile <file2> [-pairwise|-straightPair]?
? -fullchip? ?-constrFile <res_Constr_file> ?
```

where

-instance : displays a report of the worst resistance location for the specified instance(s). The report contains worst case resistance for one location per domain per instance.

-instFile : same as the -instance option, except that you can specify a file containing a list of instances. The report contains worst case resistance for one location per domain per instance.

-cell : creates a resistance report of the worst resistance locations of all instance(s) for the specified cell masters. The report has one location per domain for each instance. <name_list> can be represented in two formats: {<name1> <name2> ... }, or <name1>,<name2>, ... (no spaces in this form)

-cellFile : specifies a text file that contains the cell names, one per line, for res_calc to compute the equivalent grid resistances.

-from/-to : performs resistance calculation from one node at or near <x1 y1> on layer <layerName1> to another node at or near <x2 y2> on layer <layerName2>. Specifying the associated net names is optional.

-fromFile/-toFile: performs resistance calculation between points listed in two files in one of two ways: “-pairwise” calculates p2p resistances for all pairs of points in the two files, while the “-straightPair” option only calculates and reports p2p resistances for “straight pairs”-- that is, between the 1st point in the -fromFile and the 1st point in the -toFile, and between the 2nd points in

each file, and so on. One requirement of the -straightPair' option is that the -fromFile and -toFile must have the same number of points. Also, the content of the specified files has the following format:

```
<x> <y> <layer> <net>
...
```

where <layer> and <net> are optional, but highly recommended.

-fullchip: specifies that effective resistances for all nodes in the design, not just the pin nodes in the instances, and including those for wires/vias, are to be computed.

-constrFile <res_Constr_file> : specifies a file defining bump-to-instance pin resistance checks to be performed based on user-specified constraints.

The constraint file format is:

```
IP_RESISTANCE
   CHECKBUMP [ RES_TOP|RES_BOT|DIS_TOP|DIS_BOT | SHORT |
   ALL | RES_MAX | RES_MIN | DIS_MAX | DIS_MIN ]
       <num_bumps>
   CELL <cell_name>
   PIN <Pin_name> <Res_threshold> ?<x> <y> ?<layer>? ?
   ...
   END CELL
  INSTANCE <inst_name>
      PIN <Pin_name> <Res_threshold> ?<x> <y> ?<layer>? ?
   END INSTANCE
END IP_RESISTANCE
```

where

CHECKBUMP specifies the type of check and the number of bumps to be checked, which applies to all pins in the constraint file, and:

RES_TOP - checks nodes with the highest resistance, up to <num_bumps>

RES_BOT - checks nodes with the lowest resistance, up to <num_bumps>

DIS_TOP - checks nodes at longest distance, up to <num_bumps>

DIS_BOT - checks nodes at shortest distance, up to <num_bumps>

SHORT: checks and reports the effective resistance from IP pins to all associated bumps shorted for calculation purposes.

ALL: checks and reports point-to-point effective resistance from each pin to all associated bumps.

RES_MAX : checks and reports the maximum point-to-point effective resistance among all pin-bump pairs.

RES_MIN : checks and reports the minimum point-to-point effective resistance among all pin-bump pairs.

DIS_MAX : checks and reports the maximum geometric distance among all pin-bump pairs.

DIS_MIN : checks and reports the minimum geometric distance among all pin-bump pairs.

The default output file is *adsRpt/<design_name>.res_calc*.

A sample output file format follows:

```
# Grid Resistance Calculation Report
# CELL CONSTRAINT: <cell> <Pin> <Res_Constr> ?<x> <y> ?<layer>? ?
# INSTANCE CONSTRAINT: <cell> <Pin> <Res_Constr> ?<x> <y> ?<layer>? ?
```

```
# PIN: <pin>: (<x> <y> <layer> <net>) <instance> <cell>
# CELL CONSTRAINT: cell_55  VSS 3 100 200 METAL4
# Check against the resistive furthest 5 bumps
{
# PIN: VSS (2346.05 1935.28 METAL4 VSS) inst_129422/inst_7768 cell_55
# <Res> PAD:(<x> <y> <layer> <net>)
# FAIL:
    3.19957 DVSS45  (4256 39  METAL6  VSS)
# PASS:
    2.9993  DVSS26  (1978 39  METAL6  VSS)
    2.95331 DVSS32  (2696 39  METAL6  VSS)
    2.829 DVSS39  (3596 39  METAL6  VSS)
    2.71838 DVSS52  (35 910 METAL6  VSS)
```

Current Density Checking

Totem **PathFinder** provides current density analysis for user-specified B2B and P2P ESD paths, and reports the IR voltage and EM values in report files, as well as displaying results in GUI maps, to help you perform DC analysis of ESD paths and analyze possible IR voltage and EM violations. Current density analysis is implemented in two modes:

Mode 1 - performs IR voltage analysis considering all clamp paths with iterative convergence to reach a solution that includes a set of On clamps and Off clamps for the specified bump pair.

Mode 2 - specifies the clamp paths on which the analysis should be performed.

The ESD command is ‘perform esdcheck’. For details, see section "perform esdcheck", page 12-369.

Mode 1 Rules Files - all clamp paths

The format of the rule file for Mode 1 current density analysis is as follows:

```
BEGIN_ESD_RULE
NAME <name>
TYPE CURRENT_DENSITY # or "DC", or "CD" for short
LOOP_R <resistance_Ohms>
ESD_STAGE 2
B2B_RULE_NAME <rule_name>
BUMP_PAIR/PIN_PAIR <bump/pin Name1> <bump/pin Name2>
BUMP_PAIR_FILE <filename>
PIN_PAIR_VTH <V_diff>
SHOTGUN_MODE [0 | 1 ]
NET_PAIR_DOMAIN [ SAME | DIFF ]
FROM_NET <net_name1> <net_name2> ...
TO_NET <net_name1> <net_name2> ...
TERMINAL_NET <net_name1> <net_name2> ...
FROM_NET_GROUP ?Power? ?Ground? ?Signal?
TO_NET_GROUP ?Power? ?Ground? ?Signal?
TERMINAL_NET_GROUP ?Power? ?Ground?
CDM_ALL_NET [ 0 | 1 ]
SHORT_BUMP_IN_NET <net_name>
```

```
NO_SHORT_BUMP_IN_NET <net name1> <net name2> ...
JEDEC_MODE [BIDIR |POS |NEG ]? OPEN ?
B2B_LOOP_LENGTH <clamp_count1> <clamp_count2> ...
ZAP_B2B_NET <net1> <net2> ...
ZAP_B2B_NET_GROUP {POWER | GROUND | SIGNAL ]
ZAP_VOLTAGE <voltage_Volts>
ZAP_VOLTAGE_RANGE level_1 level_2 ....
ZAP_R <resistance_Ohms>
ZAP_CURRENT <current_in_Ampere>
ZAP_CURRENT_RANGE level_1 level_2 ....
ZAP_FROM <name> <x> <y> ?<layer>? ?<net>?
ZAP_TO <name> <x> <y>  ?<layer>? ?<net>?
EXTEND_CLAMP_CONN [ 0 | 1 ]
EM_SCALE <scale_factor>
EM_RULE_SET <rule_set_name>
NET_PAIR_VTH <Vth value> ?<net1>? ...
PEAK_VOLT <peak_voltage_thresh>
DIFF_VOLT <diff_voltage_thresh>
CLAMP_IMAX <I1> [<I2>]
CLAMP_VMAX <V1> [<V2>]
SHORT_CLAMP_NODE [ 0 | 1 ]
CACHE_EM <value>
EM_THRESHOLD  <value>
REFF_THRESHOLD <value>
I_THRESHOLD  <value>
V_THRESHOLD  <value>
SAVE_CLAMP_FAIL [0|1]
PACKAGE_PIN [ 0 | 1]
SAVE_EM [ 0 | 1]
IGNORE_EM_IN_CLAMP [ 0 | 1 ]
END_ESD_RULE
BEGIN_VGS_THRESHOLD
<model_name1> <threshold>
...
END_VGS_THRESHOLD
BEGIN_VDS_THRESHOLD
<model_name1> <threshold>
...
END_VDS_THRESHOLD
DISCHARGE_FROM_GUARDRING
DISCHARGE_TO_GUARDRING
```

where

NAME : rule name

TYPE CURRENT_DENSITY: performs current density analysis

LOOP_R: loop resistance threshold for a single ESD path

ESD_STAGE <min> <max> : for multistage rules, PathFinder checks all net pairs that are connected by at least <min> clamps and no more than <max> clamps.

Note that if you specify only the ESD_STAGE rule, PathFinder does not compute B2B resistance, to reduce runtime. If either LOOP_R or PARALLEL_R keywords are specified, then B2B resistance is computed.

B2B_RULE_NAME: uses previously-executed B2B Resistance check results in ESD current density analysis to improve runtime when non-linear clamp devices are used.

BUMP_PAIR: specifies the bump pair to be tested. The first bump in the pair is connected to the positive terminal of the zapping source. Multiple BUMP_PAIR line entries can be used in the same rule file.

PIN_PAIR: specifies the pin pairs to be tested.

PIN_PAIR_VTH : reports pin pair differential voltages in ESD-CD checks. If the CD check involves more than one net, the pin pair differential voltages that are above the positive voltage threshold <V_diff> are reported in the file *esd_pinPair.rpt.*

BUMP_PAIR_FILE: specifies a file containing a list of bump pairs to be analyzed.

SHOTGUN_MODE: when turned on, analyzes multiple nets together to reduce the run time for arc-based analyses, such as when two terminals of the zapping source are connected in the same net and CD checks are needed from all I/O signal bumps to the protecting clamp devices.

NET_PAIR_DOMAIN : checks and reports violating pairs connected to the same power/ground domain (SAME) or different power/ground domains (DIFF).

FROM_NET/TO_NET: Unidirectional current density checks are made FROM all bumps on the specified net(s) TO all bumps on the specified net(s). Note that B2B_NET1/B2B_NET2 perform bi-directional checks.

When clamp device has asymmetric nonlinear I-V characteristics, B2B_NET1/B2B_NET2, will perform and report resistances for both directions, i.e. from B2B_NET1 to B2B_NET2 and also from B2B_NET2 to B2B_NET1, respectively if conduction paths exist. On the other hand, FROM_NET/TO_NET will only check and report from FROM_NET to TO_NET.

TERMINAL_NET: current density checks are made from/to all bumps on the specified net(s)

FROM_NET_GROUP: current density checks are made from all bumps belonging to one or more of the specified net groups Power, Ground, and Signal.

TO_NET_GROUP: current density checks are made to all bumps belonging to one or more of the specified net groups Power, Ground, and Signal.

TERMINAL_NET_GROUP: current density checks are made from/to all bumps belonging to one or more of the specified net groups Power, Ground, and Signal.

CDM_ALL_NET : applies static charge to all domains connected to the discharge point by clamp devices. By default the static charge is applied only to the domain where the discharge point is.

SHORT_BUMP_IN_NET: specifies the net in which all bumps are to be shorted, in order to consider the effects of the package, since the bumps are connected through a very low resistance path through package.

NO_SHORT_BUMP_IN_NET : allows excluding specified nets otherwise shorted as a group using SHORT_BUMP_IN_NET_GROUP.

JEDEC_MODE : (default Off) specifies zapping from bumps in each individual net in a defined net group to the rest of the nets in the group, where the zap direction is chosen. Unless OPEN is selected, the bumps in the “to” group are shorted by default.

B2B_LOOP_LENGTH : <clamp_countN> values specify the number of clamps in each B2B path (positive integers less than or equal to MAX_ESD_STAGE). Values of <clamp_countN> *not included* mean that B2B paths with that number of clamps are excluded from parallel R computation.

ZAP_B2B_NET : provides zapping between bump pairs belonging one or more specified nets.

ZAP_B2B_NET_GROUP : provides zapping to all bump pairs of one type of net--POWER, GROUND, or SIGNAL. Depending on the number of bumps, CD checks are performed on all combinations.

ZAP_VOLTAGE: specifies the voltage of the zapping source.

ZAP_VOLTAGE_RANGE: specifies multiple zap voltage levels for CD check. After checks for all levels are finished , an "ESD Zap Sweep Summary" will be generated for each rule that has used the zap range in the esd_summary.rpt .

ZAP_R: specifies the series resistance to the zapping voltage source.

ZAP_CURRENT: directly specifies the zapping current. When this is used, ZAP_VOLTAGE and ZAP_R keywords are ignored.

ZAP_CURRENT_RANGE: specifies multiple zap current levels for CD check. After checks for all levels are finished , an "ESD Zap Sweep Summary" will be generated for each rule that has used the zap range in the esd_summary.rpt .

ZAP_FROM/ ZAP_TO : specifies the user-assigned name for the “from” and “to” points, x, y, and the optional layer and nets involved.

EXTEND_CLAMP_CONN: uses the discharge paths through clamp devices even for cases when from/to zap points are part of same net. For example, when checking resistance between two ground bumps, PF also takes into account the parallel discharge path through the power-rails that are connected to ground through clamp devices. When set, this keyword also supports clamp devices that have both terminals connected to the same net for ESD-CD and P2P R checks. Also note that when 'EXTEND_CLAMP_CONN 1' is set, B2B-related ESD rule keywords such as ESD_STAGE, LOOP_R, and PARALLEL_R have no effect. 'EXTEND_CLAMP_CONN 1' is set along with ZAP_B2B_NET/ZAP_B2B_NET_GROUP specified in ESD rule, Totem performs bi-directional ESD-CD/R check

EM_SCALE: scales the ESD currents to match different Zap scenarios. You can provide one EM current limit in the technology file and then scale the current limit to that desired for ESD.

EM_RULE_SET : specifies a rule set file that enables importing different EM current density limit checks for different rules in same Totem session. Note that EM_RULE_SET values should be present as a part of the EM_TECH_FILE keyword in the GSR file.

NET_PAIR_VTH: reports driver-receiver voltage threshold pairs for specified nets that violate the voltage difference VTH. Optionally, one or more nets with the same VTH value can be specified.

PEAK_VOLT : specifies the peak voltage to be used in current density checks

DIFF_VOLT : specifies the differential voltage to be used in current density checks

CLAMP_IMAX: specifies the current thresholds for the positive and negative breakdown checks for power clamps and diodes. When only one value is specified, it is applied to all ESD_PIN_PAIRs, and the threshold is the same

in both directions. You can also specify the I-V limits at clamp cell level, or for individual clamp devices, using the clamp I-V specification. The default value is 0.0, which means no check is performed.

CLAMP_VMAX: specifies the voltage thresholds for the positive and negative breakdown checks for power clamps and diodes. When only one value is specified, it is applied to all ESD_PIN_PAIRs, and the threshold is the same in both directions. You can also specify the I-V limits at clamp cell level, or for individual clamp devices, using the clamp I-V specification. The default value is 0.0, which means no check is performed.

SHORT_CLAMP_NODE : shorts clamp nodes “on-the-fly” in ESD-R and ESD-CD checks when resistances are computed.

CACHE_EM: for ESD checks for multiple from/to points, setting this to 2 significantly reduces the EM checking runtime and CD analysis time. Setting to 1 gives an intermediate speed-up. Note that when using this keyword EM results for a few corner cases can be slightly less accurate (<5%). Setting to 3 stores EM limits in the PF-S DB, resulting in improved run and reload time for ESD CD checks with multiple 'from/to' zap points. Default is 0 (off).

EM_THRESHOLD : saves the ESD-CD database if the EM % is >= this threshold value.

REFF_THRESHOLD : saves the ESD-CD database if the effective R between zap from/to points is >= this threshold value.

I_THRESHOLD : saves the ESD-CD database if the zapping current between from/to points is >= this threshold value.

V_THRESHOLD : saves the ESD-CD database if the zapping voltage between from/to points is >= this threshold value.

SAVE_CLAMP_FAIL : when set to 1, the DB is saved when any clamps fail the breakdown threshold

Note: when none of the above thresholds are specified, all the ESD-CD test results are saved to the database.

PACKAGE_PIN : specifies pin-to-pin CD checks with net-group options, such as FROM_NET/TO_NET/TERMINAL_NET between package pins. This option is different from the option 'PACKAGE 1', in which PathFinder uses bumps as reference, but ‘PACKAGE_PIN 1’considers package pin connections as reference.

SAVE_EM : when set, saves EM results. The saved EM results can be directly loaded back into Totem without re-running EM checking, using the TCL command ‘import esdcd <rule_name>’.

IGNORE_EM_IN_CLAMP: when set to 1, disable EM violation in Clamp instance in DC check. Default value is 0

BEGIN_VDS_THRESHOLD/END_VDS_THRESHOLD:

BEGIN_VGS_THRESHOLD/END_VGS_THRESHOLD: Defines the device-model specific VDD/VGS threshold voltage values for Driver-Receiver checks. It can be defined anywhere in the rule file or between BEGIN/END_ESD_RULE statements if different thresholds are desired for a particular rule.For generating <design name>.esdinfo file with all the xtors VDS/VGS and corresponding device model names, user needs to specify the signal pad nets using ESD_SIGNAL_NETS, and set IO_PIN_CHECK / D2R_CHECK to 1 in aplmmx config file. Also, the <design_name>.esdinfo file should be defined inside the ESD_RULE using keyword IP_SIG_NET_FILE.

Example rules file:

```
BEGIN_ESD_RULE
    NAME mydc
    TYPE DC
    LOOP_R 6.0
    BUMP_PAIR VSS1 VDD5
    ZAP_VOLTAGE 100
    ZAP_R 1000
 END_ESD_RULE
```

DISCHARGE_FROM_GUARDRING/DISCHARGE_TO_GUARDRING: When set to 1, defines guardring nodes as the source/sink for discharging during current density check for latch-up on guard ring taps.

Sample ESD rule file for Latch-up checks:

```
BEGIN_ESD_RULE
 NAME latchup_from
 TYPE CD
 ZAP_CDM_NET_GROUP POWER GROUND
# or use ZAP_CDM_NET, ZAP_CDM_BUMP, etc.
 DISCHARGE_FROM_GUARDRING 1
# DISCHARGE_TO_GUARDRING 1 for negative discharge
ZAP_CURRENT 1.3
# SHORT_BUMP_IN_NET_GROUP POWER
REPORT_INSTANCE 3
END_ESD_RULE
```

User can specify the zap current and net-groups of interest. All guard ring taps in that specific domain will act as source or sink. Any current crowding or voltage build-up on issue due to poor hook-up on guard ring tap connections will be highlighted from this analysis. A detailed report of individual tap-point voltages are dumped when "REPORT_INSTANCE 3" is specified in adsRpt/ESD/esd_inst.rpt.

Mode 2 Rules Files - Specified clamp paths

The format of the rule file for Mode 2 current density analysis is as follows:

```
BEGIN_ESD_RULE
  NAME <name>
  TYPE CURRENT_DENSITY
  BUMP_PAIR <bumpName1> <bumpName2> # or use PIN_PAIR
  ZAP_VOLTAGE <voltage_in_Volts>    # or use ZAP_CURRENT
  ZAP_R <resistance_in_ohms>
  CLAMP_INST_PIN <instance> <locID1> <locID2>
  USE_CLAMP_FILE <file>
END_ESD_RULE
```

where

USE_CLAMP: specifies an ESD_PIN_PAIR in a clamp instance for DC analysis. Multiple en-tries of CLAMP_INST_PIN are allowed.

USE_CLAMP_FILE <file> : in the specified file, the following syntax can be defined.

```
USE_CLAMP_CELL
```

```
    <cellName> [<locID1> [<locID2>]]
      ...
  END_CLAMP_CELL
  USE_CLAMP_INST
    <instName> [<locID1> [<locID2>]]
      ...
  END_CLAMP_INST
```

Note that wildcards are honored in the clamp cell and instance names, and the locIDs are optional. The above keywords can be applied to B2B/B2C/CD rules for clamp selections. All clamps specified in CLAMP_INST_PIN statements and in USE_CLAMP_FILE are combined for DC analysis.

Example rules file for Mode 2:

```
BEGIN_ESD_RULE
  NAME dc
  TYPE DC
  BUMP_PAIR VDD8 VSS4
  ZAP_VOLTAGE 100
  ZAP_R 1500
END_ESD_RULE
```

Outputs are reported in the following files:

Out_dir/esd_summary.rpt : reports IR/EM values across the analyzed clamp instances.This also reports the total number of Wire/Via EM violations.

Out_dir/esd_em.rpt : reports EM values of the B2B paths

In addition to these output reports, results of the analysis are also viewable in several maps, using the menu options identified in ESD results map menu: See next section.

JEDEC HBM Checks

The ESD rule file also allows user-defined net groups to be analyzed as a group for ESD resistance and current density checks, using syntax as follows:

```
BEGIN_NET_GROUP
  NAME <net_group_name>
  NET <net1> <net2> ...
END_NET_GROUP

BEGIN_ESD_RULE
  NAME CD_test
  TYPE CD
  ZAP_CURRENT 1.0
  FROM_NET_GROUP <net_group1>
  TO_NET_GROUP  <net_group2>
END_ESD_RULE
```

The following special ESD rule keywords can be used for bump shorting in resistance and CD checking :

- SHORT_BUMP_IN_NET_GROUP POWER ALL : shorts bumps in all PWR nets together
- NO_SHORT_BUMP_IN_NET VDD : excludes VDD, so bumps are not shorted
- SHORT_BUMP_ONE_NET VDD1 : shorts bumps for VDD1separate from other power bumps

The ESD rule keyword 'JEDEC_MODE' zaps one net against the rest of the nets in the group, with bumps shorted together. The syntax is:

```
BEGIN_ESD_RULE
  NAME jedec1
  TYPE CD
  ZAP_CURRENT 1.5
  TERMINAL_NET_GROUP SIGNAL
  JEDEC_MODE <zap_dir> ? <bump_option>?
END_ESD_RULE
```

where

<zap_dir>: specifies the zap direction. POS means the 'to be zapped net' is connected to the positive terminal of the zapping source, and the rest of the nets (bumps) are shorted and grounded. NEG means the reverse polarity of POS. BIDIR specifies both POS and NEG polarity checks.

<bump_option>: by default all bumps in the net group are shorted. Use 'OPEN' to not short the bumps for the net to be tested (optional).

Guard-ring Latch-up Checks

The following ESD Rule keywords allow simulating current-zap from guard rings to check / prevent latch-up.

- DISCHARGE_FROM_GUARDRING : When set to 1, defines guard-ring nodes as the source for discharging during current density check for latch-up on guard ring taps.
- DISCHARGE_TO_GUARDRING: When set to 1, defines guard-ring nodes as the sink for discharging during current density check for latch-up on guard ring taps.

Sample ESD rule file for Latch-up checks:

```
BEGIN_ESD_RULE
NAME latchup_from
TYPE CD
ZAP_CDM_NET_GROUP POWER GROUND
# or use ZAP_CDM_NET, ZAP_CDM_BUMP, etc.
DISCHARGE_FROM_GUARDRING 1
# DISCHARGE_TO_GUARDRING 1 for negative discharge
ZAP_CURRENT 1.3
# SHORT_BUMP_IN_NET_GROUP POWER
REPORT_INSTANCE 3
END_ESD_RULE
```

User can specify the zap current and net-groups of interest. All guard ring taps in that specific domain will act as source or sink. Any current crowding or voltage build-up on issue due to poor hook-up on guard ring tap connections will be highlighted from this analysis. A detailed report of individual tap-point voltages are dumped when "REPORT_INSTANCE 3" is specified in adsRpt/ESD/esd_inst.rpt.

Arc-based Current Density Checking

PathFinder also supports arc-based current density checks for bump-to-clamp, clamp-to-pin and clamp-to-clamp rules, as well as for a pair of bumps, pins or nets through clamps.

Rules File

The following rules file keywords are available for arc-based ESD-current density checks:

ZAP_BUMP_CLAMP <bumpName> <instName> <locID> : checks from bump to clamp instance

ZAP_CLAMP_BUMP <instName> <locID> <bumpName>: checks from clamp to bump

ZAP_CLAMP_CLAMP <inst1> <locID1> <inst2> <locID2> : checks from clamp to clamp

ZAP_PIN_CLAMP <pinName> <instName> <locID> : checks from pin to clamp

CLAMP_PIN <instName> <locID> <pinName> : checks from clamp to pin.

To enable current density checks within a specific net, the following keywords are available:

ZAP_B2C_NET <netName> : bump to clamp for the net(s) specified

ZAP_C2B_NET <netName> : clamp to bump for the net(s) specified

ZAP_C2C_NET <netName> : between clamps belonging to the net(s) specified

ZAP_P2C_NET <netName> : pin to clamp for the net(s) specified

ZAP_C2P_NET <netName>: clamp to pin for the net(s) specified

Also, the same set of CD checks can be run for all nets of a specified type, as follows:

ZAP_B2C_NET_GROUP {POWER | GROUND | SIGNAL}

ZAP_C2B_NET_GROUP {POWER | GROUND | SIGNAL}

ZAP_C2C_NET_GROUP {POWER | GROUND | SIGNAL}

ZAP_P2C_NET_GROUP {POWER | GROUND | SIGNAL}

ZAP_C2P_NET_GROUP {POWER | GROUND | SIGNAL}

These ZAP_*_NET_GROUP rules provide zapping to all bump/clamp/pin pairs of one type of net -- POWER, GROUND, or SIGNAL.

For C2C and B2C current density checks, you can define a RADIUS within which a clamp instance check is selected. And for C2C current density checks, you can specify the type of clamps between which the check is to be performed. The keywords for these functions are:

FROM_CLAMP_TYPE <type>

TO_CLAMP_TYPE <type>

SAME_CLAMP_TYPE [0 | 1] : if On, checks the same clamp type

DIFF_CLAMP_TYPE [0 | 1] : if On, checks different clamp types

For arc-based current density checking you can also select the closest set of clamps, based on the ESD rule keyword 'FROM_TO_SELECT', which allows selecting points to be connected to the zapping source in the analysis. The syntax is:

FROM_TO_SELECT <method> <count> ?<1_to_many [0|1]>?

where <method> is one of the options:

FROM_MIN_DIST

FROM_MIN_RES

TO_MIN_DIST

TO_MIN_RES

and

FROM*/TO* : indicates at which end of the zapping source the selection takes place. For example, for a net specified by 'B2C_NET', the bumps in the net are connected to the positive terminal of the zapping source, and the points in the clamps are grounded.

*MIN_RES : selects from/to points that are connected with smallest effective resistances.

*MIN_DIST: selects from/to points that are close in physical distance in the layout,

<count>: optional, specify how many points to selected by <method>. It takes a positive integer value, default 1.

<1_to_many [0|1]>: specifies if to tie (short) all selected points together to do 1-to-many zapping. Default is 0 (off). In effect only when <count> setting is greater than 1.

Example: “FROM_TO_SELECT FROM_MIN_DIST 10" means to select 10 bump points to connect to the positive terminal of the zapping source, and to be zapped with each clamp point, respectively.

An example of this rule file syntax is:

```
BEGIN_ESD_RULE NAME rule_name
  TYPE CURRENT_DENSITY
  ZAP_CURRENT 1
  B2C_NET <> (or) C2C_NET
  FROM_CLAMP_TYPE clampA
  TO_CLAMP_TYPE clampB
  FROM_TO_SEL TO_MIN_DIST 5 1
END_ESD_RULE
```

Also, to allow multiple parallel clamps to be included (shorted together) for the referenced side as “zap froms”, the rule keyword 'TIE_REF_CLAMP' can be used.The syntax is the same as 'TIE_CLAMP' and it is used for C2C zap for the clamps on 'from side':

Syntax:

TIE_REF_CLAMP <method> ?<num_nodes>?

This enables handling of multi-finger diodes in clamp-to-clamp ESD-CD checks. For example, if the design has n-diodes connected between signal and power nets (in parallel), and there are n power clamps in the design, and you can perform an ‘m X n’ parallel zap using the following rules:

```
FROM_CLAMP_TYPE DIODE
TO_CLAMP_TYPE PWRCLAMP
FROM_TO_SEL TO_MIN_DIST <n> 1
TIE_REF_CLAMP <method> [<m>]
```

Choices for clamping <method> are:

PARALLEL: clamps connected in parallel

INST: clamps in the same instance (like many fingers in a cell)

MIN_DIST: get the <num_nodes> closest clamp nodes.

ALL : to short all clamps

TYPE : to short clamps of same type

For PARALLEL and INST, when <num_nodes> is not specified, all such clamps are included. For MIN_DIST, the default for <num_nodes> is 1; that is, the same as when 'TIE_REF_CLAMP' is not specified. So for 'MIN_DIST', you should specify <num_nodes> >1.

Note that TIE_REF_CLAMP has higher priority over rules FROM_TO_SELECT and RADIUS.

CD and arc-based resistance checks include clamp resistance

CD and arc-based resistance checks can include clamp resistance for better current distribution. This includes R-type point-to-point resistance checks as well to perform a subset of the potential path resistances for B2C and C2C arc-based analysis.

ESD rule syntax example:

```
BEGIN_ESD_RULE
  NAME BUMP_TO_D2_D4
  TYPE CD     # 'TYPE R' also supported
  ZAP_CURRENT 1.3

  ZAP_B2C_NET_GROUP SIGNAL
# ZAP_C2C_NET, ZAP_BUM_CLAMP, also supported

# To include all clamps connected to the zap from/to
points:
  USE_CLAMP_IV 1
# To specify particular clamp devices to be used:
  USE_CLAMP_IV
    CELL  <cellName(s)>
    INST  <instName(s)>
    TYPE  <clampType(s)>
  END_CLAMP_IV
END_ESD_RULE
```

To display Diode/Clamp instance current in the `esd_summary.rpt` file during CD checks, keyword `'USE_CLAMP_IV 1'` is set in the CD check rule file and the `'-jobCount'` option is specified in the `'perform esdcheck'` command. The syntax in the tcl command file is:

```
perform esdcheck -jobCount 4 -rule <cd_check_rule> ...
```

In the rule file <cd_check_rule> the syntax is:

```
BEGIN_ESD_RULE
  NAME <cd_name>
  TYPE CURRENT_DENSITY
  ...
  ...
  USE_CLAMP_IV 1
END_ESD_RULE
```

Viewing Current Density Checking Results

ESD CD Report Command

After an ESD CD check is performed with the 'perform esdcheck' command, Totem saves the results into the ESD DB. The data is identified by the rule name. If multiple ESD-Current Density checks are performed with the same rule name, a suffix “_1”, “_2” … is added to the specified rule name to provide separate current density check files. You can get text reports for any ESD current density check with the TCL command:

```
report esdcheck -type cd -name <rule_name>
?-[glob|-regexp] ?
?-inst <inst_names>? ?-detailed? ?-outDir <dir_path>?
?-sort <sort_criterion>? ?-limit <inst/node_num>?
```

where

-name: specifies the name of the rule tested. If multiple checks have been specified within a single rule-- for example, two bump pairs are specified in the same rule-- PathFinder internally splits them into subrules, each with a single bump pair. Then it appends '(_\d+)+' to the base rule name. So when using 'report esdcheck', you can specify this derived name to look at the particular results desired.

-glob|-regexp: specifies the type of wildcard expressions used. PathFinder reports the results for all subrules that match the specification; 'glob' supports regular wildcard expressions and 'regexp' supports normal wild cards and also extended character strings. For example:

```
report esdcheck -detail -perNet -type CD-name abc* -glob
report esdcheck -type CD -name {M_test_[3-5]} -regexp
```

-inst <inst_names>: specifies the instance(s) to be reported

-detailed: displays the detailed node information of specified instance(s).

-sort <sort_criterion>: sorts reported instances by different criteria. Currently two criteria are supported: 'maxV' (peak voltage) and 'diffV' (differential voltage). The default sorting criterion is 'maxV'. See Figure 12-9 for how the results of different voltage criteria are displayed in the GUI.

-limit <inst/node_num>: sets the limits for the number of items to be reported. When both “-inst” and “-detailed” options are given, the specified number limits the number of nodes of each instance to be reported. Otherwise the specified number limits the number of instances to be reported.

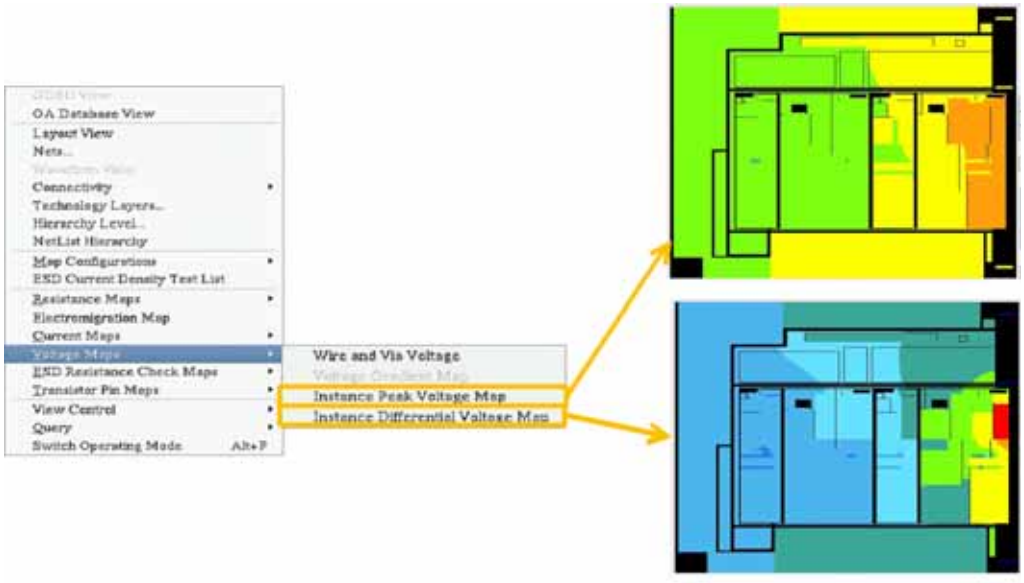


Figure 12-9 Displaying different instance voltage maps

You can load results from a current density check named, for example, ‘RuleABC’ with the following command :

```
import esdcd RuleABC
```

A sample output report in the *adsRpt/ESD/esd_inst.rpt* file is shown following (voltage drop):

```
# ESD-CD Analysis Instance Voltage for Rule <esd_cd> (CURRENT_DENSITY)
BEGIN_ESD_RULE
 NAME esd_cd
 TYPE CURRENT_DENSITY
 BUMP_PAIR DVSS13 DVDD14
 ZAP_VOLTAGE 1000 +
 ZAP_R 1000
END_ESD_RULE
# <MaxVoltage> <DiffVoltage> <X> <Y> <Inst Name> <Cell Name>
0.7956  0.7202  4820.535   2377.665 inst_129425/inst_508698 CellABC
0.7956  0.7208  4820.535   2381.345 inst_129425/inst_508699 CellFGH
```

In detailed reporting mode, 'report esdcheck -detail <options>', for the header lines representing the instances of the detailed node voltages, the format is:

```
# <Inst Name> <Cell Name> (<MaxVoltage> <DiffVoltage> <X_loc> <Y_loc>)
        <node_data> ...
```

A sample output file is shown following for a rule “esd_cd_1” (current density):

```
Rule name: esd_cd_1
EM mode: PEAK
Worst EM: 7748.7%
Zap From:
Bump(DVSS11) (4880 2138.85 METAL4 VSS)
Zap To:
Bump(DVDD11) (4905 2078.85 METAL4 VDD)
Zapping source: 1000 V @ 1000 Ohms
Equivalent resistance: 1.4031 Ohms (1.40114 V, 0.998599 A)
Currents on clamp devices: min 157.482 mA, max 421.800 mA
# Clamp device I(mA)/V(volt)/R(Ohm) list:
# <I> <V> <Ron> (<V> <X> <Y> <LAYER> <NET>)
# (<V> <X> <Y> <LAYER> <NET>) <INST>   <locID1> <locID2>
421.800 0.0000  0.0001  (0.7372 4521.65 2201.16 METAL4 VSS)
(0.7371 4543.78 2197.32 METAL4 VDD)   inst_129425/inst_7773  VSS   VDD
 252.563 0.0000  0.0001  (0.7593 3196.97 2201.16 METAL4 VSS)
(0.7593 3199.43 2197.32 METAL4 VDD)
```

Regular output results for current density checks are reported in the following files:

Out_dir/esd_summary.rpt : reports IR values across the analyzed clamp instances. The worst metal/via EM violation summary and total number of Wire/Via EM violations info are also reported in this file.

Out_dir/esd_em.rpt : reports EM values of the B2B paths

Results in compressed mode

Current density checking results are saved in compressed mode by default, which provides a 50% to 90% data size reduction. You can choose *not* to save the results in compressed form by using the '-noCompress' option in the 'perform esdcheck' command, or by specifying the 'COMPRESS_DB 0' keyword in the ESD rule file.

Reporting voltage at probed points in B2B CD check

PathFinder dumps voltage info at probe points defined by the GSR `PROBE_NODE_FILE` for B2B current density checks. The file `adsRpt/ESD/esd_probe.rpt` contains node voltages for each B2B current density check.

ESD-CD report esd_summary.rpt

The following is an sample esd_summary.rpt for **ESD-CD**:

```
# DC Analysis Summary for Rule <DC_RULE> (CURRENT_DENSITY)

BEGIN_ESD_RULE
  NAME DC_RULE
  TYPE CURRENT_DENSITY
  BUMP_PAIR VDD_1 VSS_2
  ZAP_VOLTAGE 1000
  ZAP_R 1000
END_ESD_RULE

# BUMP PAIR: (<BUMP1> <X> <Y> <LAYER> <NET>) --> (<BUMP2> <X> <Y> <LAYER>
<NET>)
BUMP PAIR: (VDD_1 700 1600 M4 VDD) --> (VSS_2 1300 400 M4 VSS)
Zapping source: 1000 V @ 1000 Ohms
Equivalent bump-to-bump resistance: 3.47856 Ohms (3.4665 V, 0.996533 A)
Clamp device current: 0.000 mA to 550.579 mA

# Clamp device I(mA)/V(volt)/R(Ohm) list:
# <I> <V> <Ron> (<V> <X> <Y> <LAYER> <NET>) (<V> <X> <Y> <LAYER> <NET>)
<INST> <locID1> <locID2>
550.579 0.6955 1.2632 (1.6574 1325 180 M3 VDD) (0.9619 1325 130 M3 VSS)
PVSS_D1_I17 VDD VSS
445.952 0.6945 1.5573 (2.0644 1025 1820 M3 VDD) (1.3700 1025 1870 M3 VSS)
PVSS_D1_I16 VDD VSS
0.000   0.9109 off (2.2396 725 1820 M3 VDD) (1.3288 725 1870 M3 VSS)
PVDD_D1_I14 VDD VSS0.000   0.7686 off (1.7610 1025 180 M3 VDD) (0.9925 1025
130 M3 VSS) PVDD_D1_I15 VDD VSS
0.000   0.4350 off (1.8050 1025 1980 M3 VSSIO) (1.3700 1025 1870 M3 VSS)
PVSS_D1_I16 ADS_VSS1 VSS
```

ESD Current Density Reports for Pads

PathFinder creates a Pad current and Pad voltage reports and maps for ESD current density BUMP2BUMP checks, as shown in the following sample output report, *adsRpt/ESD/esd_pad.rpt*:

```
    # ESD-CD Analysis Pad Current for Rule <esd_cd_1> (CURRENT_DENSITY)
        BEGIN_ESD_RULE
          NAME esd_cd_1
          TYPE CURRENT_DENSITY
          BUMP_PAIR DVDD11 DVSS11
          ZAP_VOLTAGE 1000
          ZAP_R 1000
          B2B_MIN_BOUND 1
        END_ESD_RULE
```

```
# <Current(mA)> <X/Y Location> <Pad Name>
    0.0000 (  4880.000,  2138.850) DVSS11
    0.0000 (  4880.000,  2378.850) DVSS13
```

You can view a color map of pad current using the command **View -> Current Maps -> Pad Current Map**.

ESD EM Report for ESD-CD

Totem supports the 'get em' command for ESD current density checking. The following is a sample output *esd_em.rpt* for **ESD-CD**, which reports EM violations, that is, EM_Ratio = Actual_Current_Density/Current_Density_Limit > 100% (default or from "em_report_percentage") for wire pieces and vias in decreasing order. The unit used for coordinates and dimensions is um.

```
# ESD EM Check Results for Rule <DC_RULE> (CURRENT_DENSITY)

BEGIN_ESD_RULE
  NAME DC_RULE
  TYPE CURRENT_DENSITY
  BUMP_PAIR VDD_1 VSS_2
  ZAP_VOLTAGE 1000
  ZAP_R 1000
END_ESD_RULE

# For wires: #layer #end-to-end_coordinates #EM_Ratio #net #width
# For vias: #via_name #x-y_coordinates #EM_Ratio #net

M2  (1334.000,106.000 1334.000,112.000) 4044.41%  VSS  5.000
M2  (728.000,1838.000 728.000,1894.000) 3932.45%  VDD  5.000
M2  (704.000,1838.000 704.000,1894.000) 3733.18%  VDD  5.000
M2  (1310.000,106.000 1310.000,112.000) 3621.23%  VSS  5.000
via VIA2_800_800_9 (704.000,1838.000)   2828.35% VDD
via VIA2_800_800_9 (1310.000,106.000)   2743.53% VSS
M4  (1320.858,224.142 1320.858,226.232) 1916.46%  VSS  20.000
M4  (1312.501,232.500 1304.142,238.768) 1916.46%  VSS  20.000
M4  (1320.858,226.232 1312.501,232.500) 1916.46%  VSS  20.000
```

Displaying Resistance and Current Density Results in the GUI

GUI Results for Resistance Checking

Totem provides a rich set of GUI features to graphically display ESD resistance and current density rule checking results, in addition to the text results in the directory *adsRpt/ESD* (the default ESD report directory, or in a user-specified directory).

After running PF-S analysis in Totem, you can find the results in the Totem2 GUI table browser using the menu command **Results->List of ESD Static Rule Check**, which shows all ESD rules that have been analyzed, as shown in Figure 12-10. All user-defined rules of each type (such as B2B, B2C, C2C, C2I, and C2MACRO) are displayed on the ESD rule check table browser. The 2nd column in the check-box can reset flight line color.

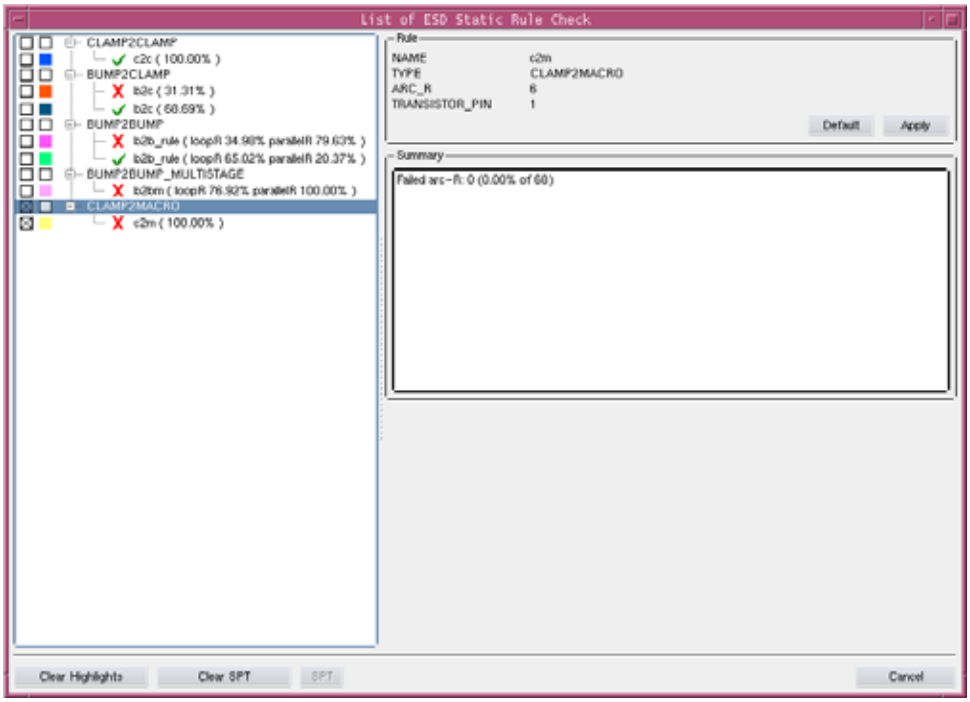


Figure 12-10 List of ESD Static Rule Check browser

You can turn on flight line cross-probing with the following steps:

1. Select the first column in the check-box of the **List of ESD Static Rule Check** browser.
2. Click the **Alt** button to switch the pre-select mode from wire/via pre-select mode to flight line pre-select mode (see Figure 12-11). Selecting the first column check-box for a BUMP2CLAMP rule type, the flight lines can be shown with **Alt** button, switch from pre-select mode to “flight line” mode. When moving the mouse to the flight line, a more detailed summary of the flight line is shown in the GUI.
3. Move mouse cursor on top of a flight line to see detailed summary of each flight line, as shown in Figure 12-11.

4. If you want to see more details, you can click the right mouse button to cross-probe the flight line and get a detailed report on the right hand side of browser.

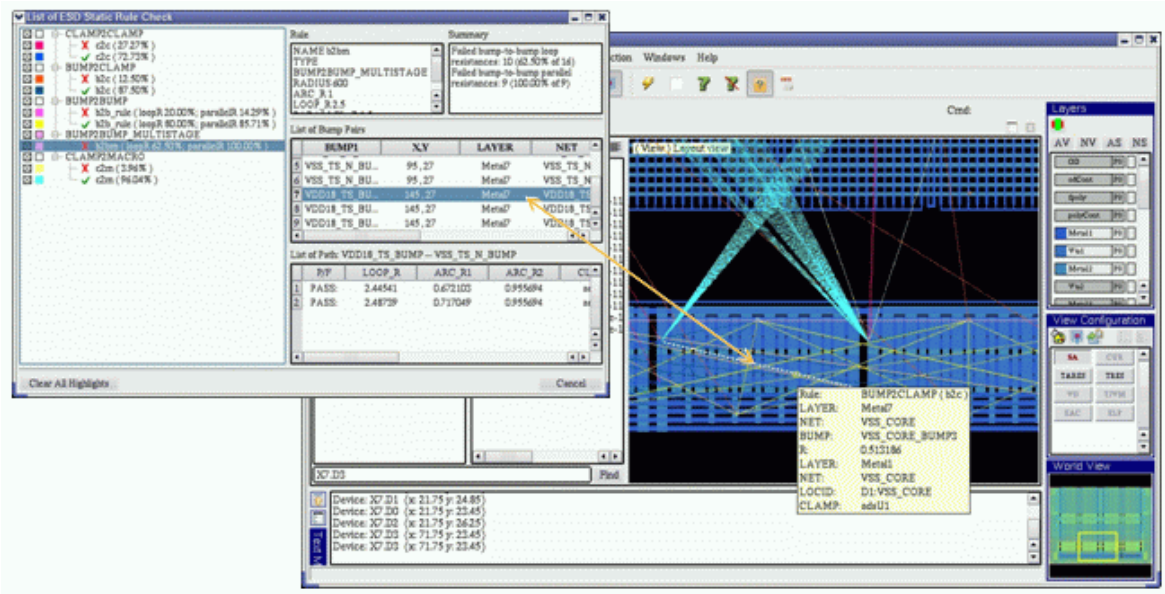


Figure 12-11 Flight line displays

If a B2B type of rule is highlighted on the left of the **List of ESD Static Rule Check** browser, all processed bump pairs and paths of loop_R for a selected bump pair are shown on the right (see Figure 12-12). Similarly, the relevant path information is shown for other rule types when selected.

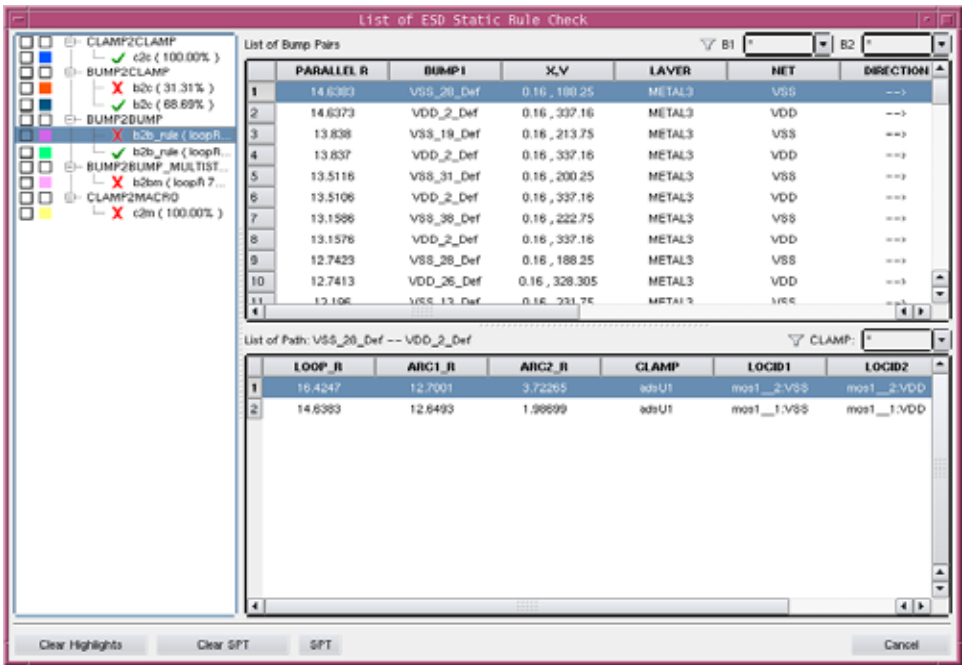


Figure 12-12 Display of bump pairs for loop_R

With a selected loop_R path from a bump pair in a BUMP2BUMP type of rule checking, you can click on **SPT** (Shortest Path Tracing) to display the minimum resistance path for a given bump-to-bump discharge path at the bottom of the GUI table. Figure 12-13 shows the individual branch of wire and via constituting a SPT path. The highlighted METAL1 path (on the table and layout view) shows a branch of a wire with a 33.757 Ohm resistance, which is quite large and turns out to be due to missing vias on the wire.

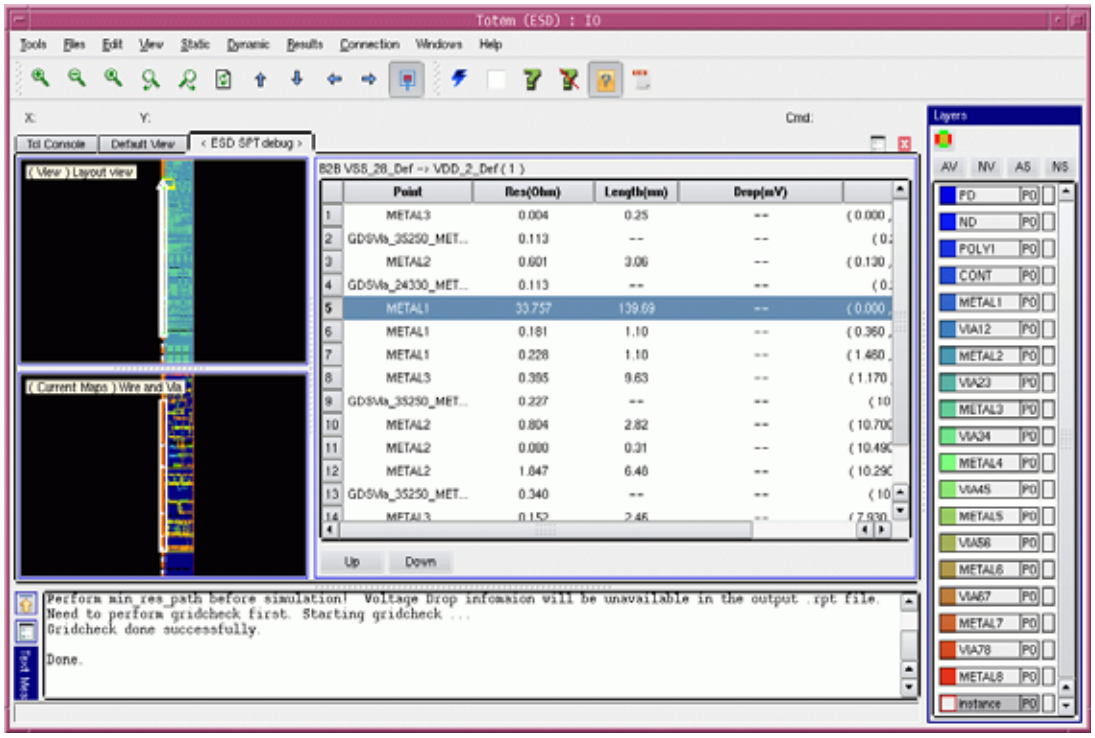


Figure 12-13 SPT minimum resistance path tracing

A list of relevant ESD clamps can also be displayed by selecting **Results->List of ESD Clamps** from the menu. The list of clamps is shown in Figure 12-14, and the corresponding locations are displayed in the layout window (see Figure 12-15).

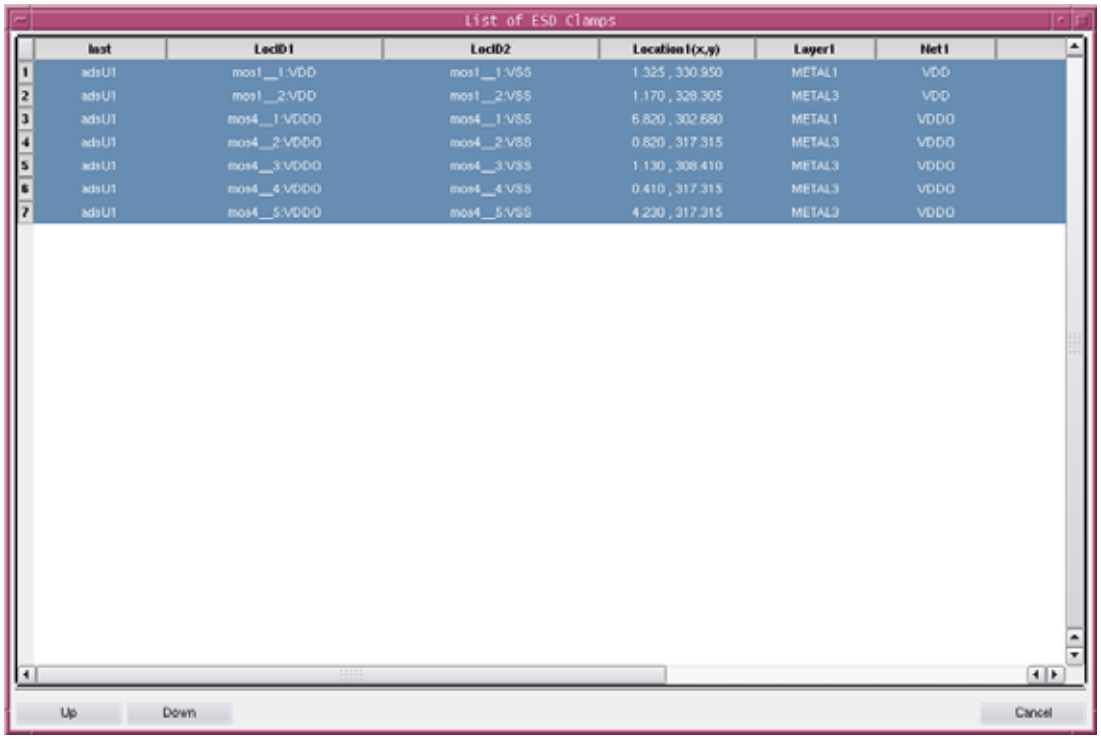


Figure 12-14 List of ESD clamps

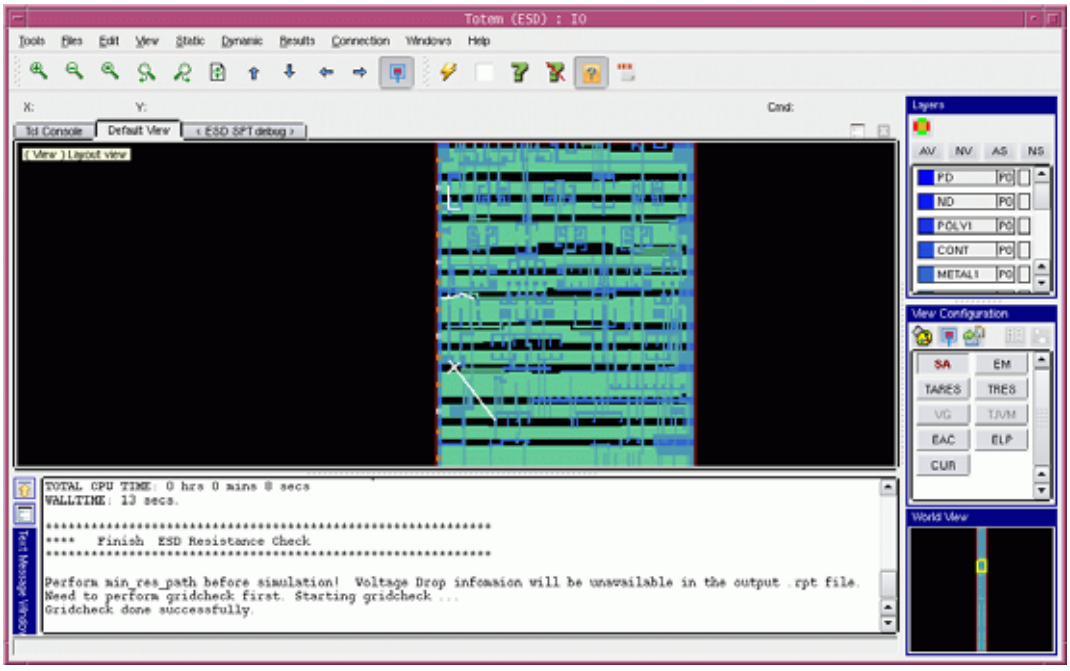


Figure 12-15 Locations of the listed ESD clamps

A histogram of individual rule types also can be generated, showing the distribution of resistance for different paths, using the GUI menu, as shown in Figure 12-16. For example, a histogram of parallel_R for all the B2B paths can be tabulated, as shown in the example in Figure 12-17.

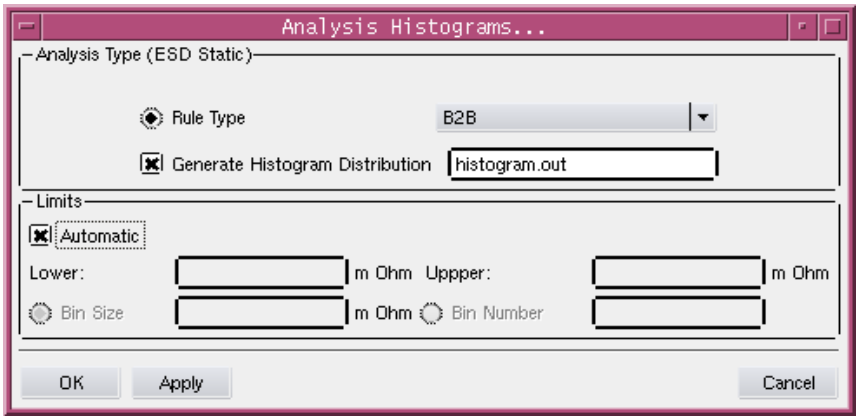


Figure 12-16 Histogram dialog

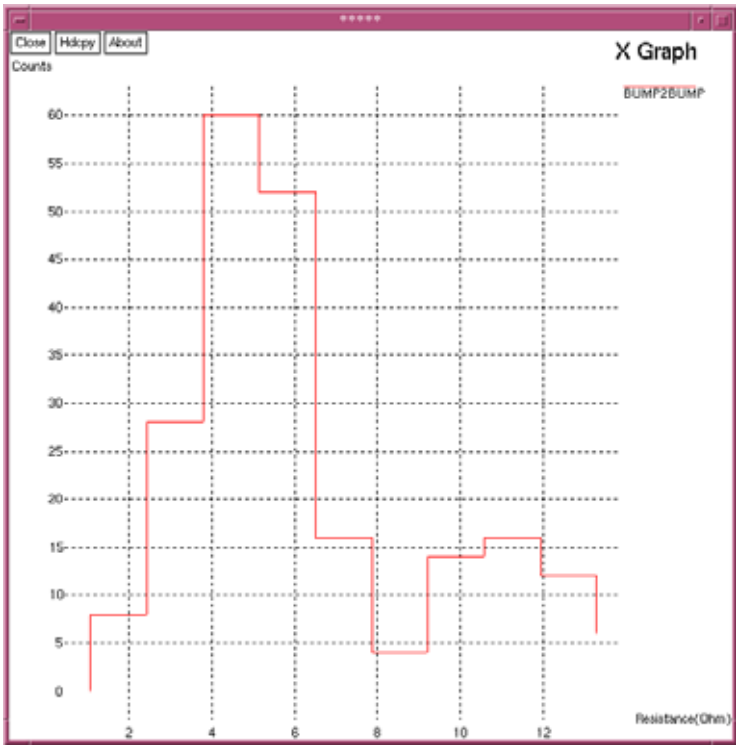
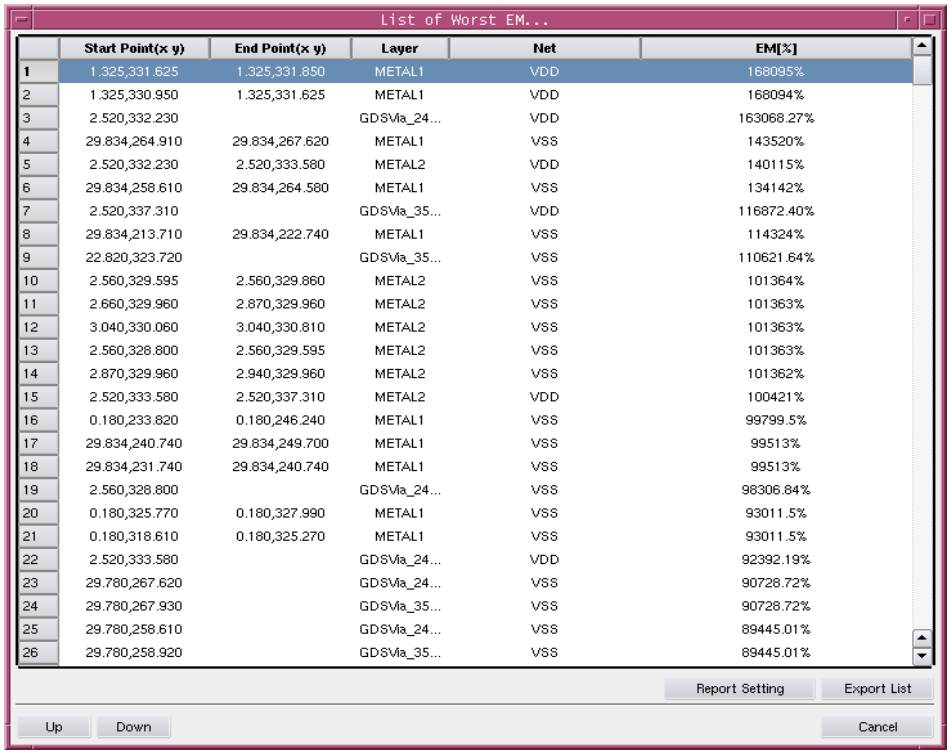


Figure 12-17 Sample B2B parallel-R histogram display

GUI for EM Current Density Rule Check

Given a bump pair and rule type CURRENT_DENSITY, the zapping current goes into the starting bump/package_pin and discharges through on-clamps, wires/bias, and arrives at

the destination bump/package_pin. Figure 12-18 shows the worst ranked violations of the peak EM limit in the GUI. If you click on any of the entries in the table, the corresponding location is shown in the layout view (Figure 12-19).



	Start Point(x y)	End Point(x y)	Layer	Net	EM[%]
1	1.325,331.625	1.325,331.850	METAL1	VDD	168095%
2	1.325,330.950	1.325,331.625	METAL1	VDD	168094%
3	2.520,332.230		GDSVia_24...	VDD	163068.27%
4	29.834,264.910	29.834,267.620	METAL1	VSS	143520%
5	2.520,332.230	2.520,333.580	METAL2	VDD	140115%
6	29.834,258.610	29.834,264.580	METAL1	VSS	134142%
7	2.520,337.310		GDSVia_35...	VDD	116872.40%
8	29.834,213.710	29.834,222.740	METAL1	VSS	114324%
9	22.820,323.720		GDSVia_35...	VSS	110621.64%
10	2.560,329.595	2.560,329.860	METAL2	VSS	101364%
11	2.660,329.960	2.870,329.960	METAL2	VSS	101363%
12	3.040,330.060	3.040,330.810	METAL2	VSS	101363%
13	2.560,328.800	2.560,329.595	METAL2	VSS	101363%
14	2.870,329.960	2.940,329.960	METAL2	VSS	101362%
15	2.520,333.580	2.520,337.310	METAL2	VDD	100421%
16	0.180,233.820	0.180,246.240	METAL1	VSS	99799.5%
17	29.834,240.740	29.834,249.700	METAL1	VSS	99513%
18	29.834,231.740	29.834,240.740	METAL1	VSS	99513%
19	2.560,328.800		GDSVia_24...	VSS	98306.84%
20	0.180,325.770	0.180,327.990	METAL1	VSS	93011.5%
21	0.180,318.610	0.180,325.270	METAL1	VSS	93011.5%
22	2.520,333.580		GDSVia_24...	VDD	92392.19%
23	29.780,267.620		GDSVia_24...	VSS	90728.72%
24	29.780,267.930		GDSVia_35...	VSS	90728.72%
25	29.780,258.610		GDSVia_24...	VSS	89445.01%
26	29.780,258.920		GDSVia_35...	VSS	89445.01%

Figure 12-18 Violation list for EM current density path

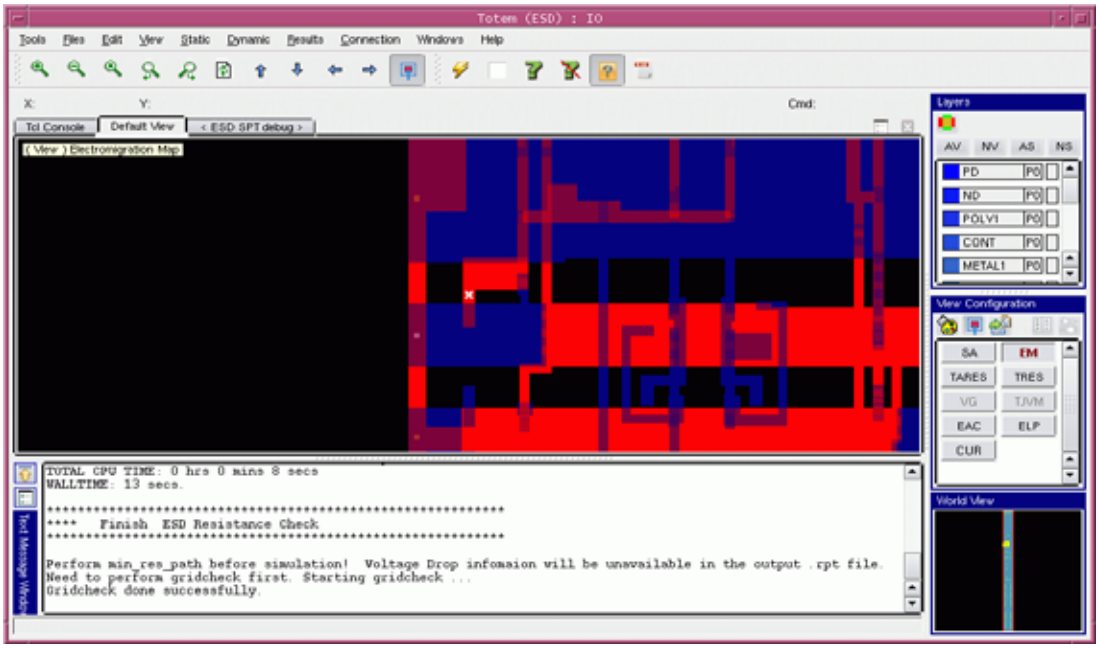


Figure 12-19 Layout display of current density violations

Figure 12-20 shows how to set up and load a Current Density Map from a Current Density Test List dialog in the Totem2 GUI:

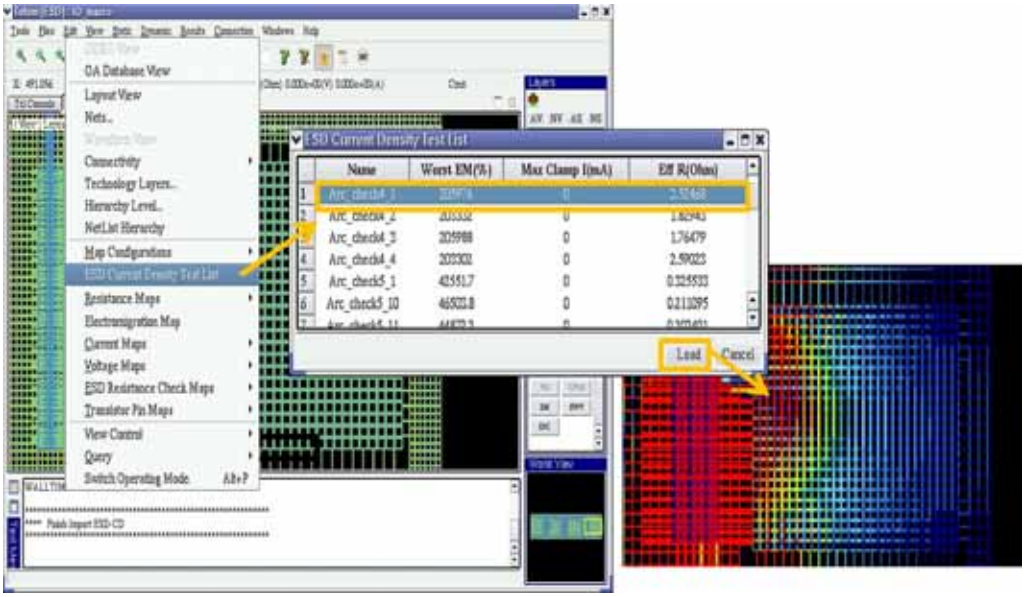


Figure 12-20 Displaying current density color maps

Current density and voltage drop maps associated with the EM violation list are also available, as shown in examples in Figure 12-21 and Figure 12-22.

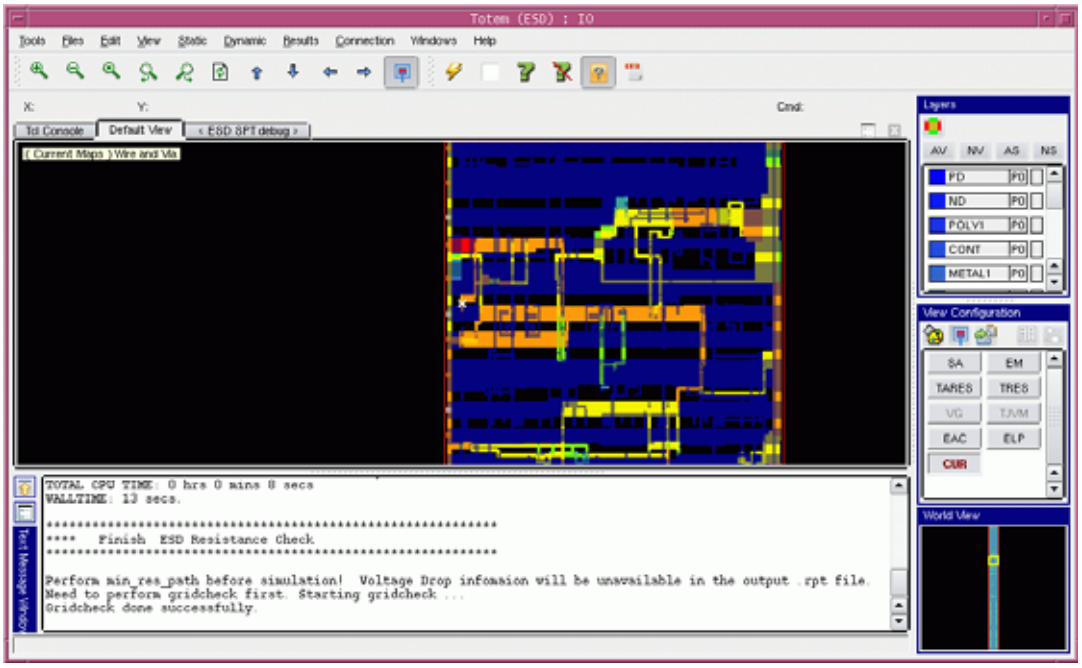


Figure 12-21 ESD rule check current density color map display

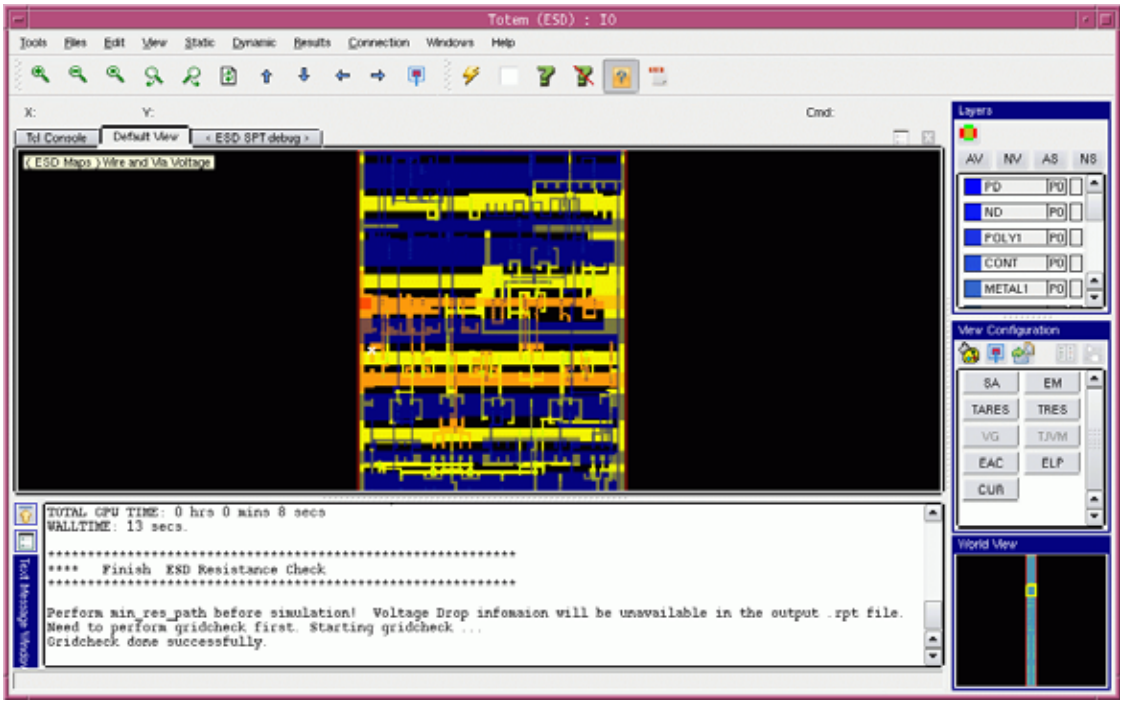


Figure 12-22 ESD rule check voltage drop map color map display

Clampviewer utility for viewing I-V curves for clamps

Totem can display clamp I-V curves with the new utility ‘clampviewer’.

Usage:

```
% clampviewer <IVFileName> [<IVName(s)>] [options]
```

List of options:

<name1>,<name2>,…name(s) of I-V curve defined in file to plot
Default: All I-V curves in the file will be plotted

-i <min>,<max> : current range to plot, Default: [-10,10]

-v <min>,<max> : voltage range to plot Default: [-10,10]

-marker : draw marker

-oneview : plot all I-V curves in one plot window ,Default: plot all IV curves in separate windows

-tlp : Plot TLP data, when I-V is derived from TLP_FILE.

-h : Print help message

-di <value> : di means ‘delta I’, for specifying TLP_I_RESOLUTION value in clamp IV file

-is<scale> : works together with the -o option to save the scaled clamp I-V

General Rule File Inclusions and Exclusions

Both resistance checking and current density checking rules files can have specifications for clamps, bumps, instances, or pins that are to be included or excluded for a particular ESD check. Some examples are given in this section.

Clamp Element Exclusions

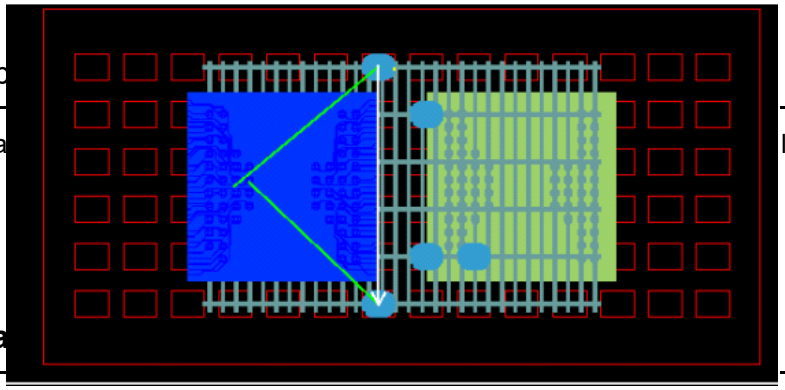
You can exclude specific clamp cells, instances, or pins from an ESD check by including the following keywords in the rule file:

EXCLUDE_CLAMP <cellname>
EXCLUDE_CLAMP_INST <instance_name>
EXCLUDE_CLAMP_CELL_PIN <cellname> <loc_ID>
EXCLUDE_CLAMP_INST_PIN <cellname> <loc_ID>

Bump

You ca

le file keywords:



3DIC flow in Pa

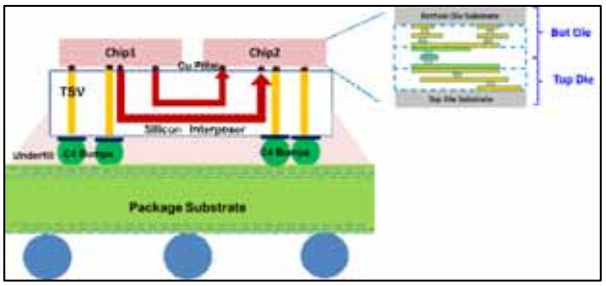


Figure 12-1 Sample Wow Design

To enable ESD analysis (R and CD checks) for new 3DIC flow, a multi-die configuration file must be created to setup the connections between different DIE using the "`setup3dic`" utility. (Please contact Ansys AE for latest documentation & support on this utility).The physical database of individual DIE and the clamp definitions for the clamps on individual DIE in a multi-DIE framework should be prepared upfront. The overall setup is similar to RedHawk 3DIC analysis.

The sample Tcl command setup for ESD check on 3DIC is as follows:

```
#Setup ESD mode
Setup analysis_mode esd
#Import 3DI config generated using “setup3dic” utility
Setup 3dic WoW.config
#Import IDE level GSR with definition of clamps present in
each DIE.
Import gsr system.gsr
Import gsr FCCC_Interposer.gsr -dir Interposer
```

```
Import gsr FCCC_TOP.gsr -dir DIE_C1_TOP
Import gsr FCCC_BOT.gsr -dir DIE_C1_BOT
Import gsr FCCC_TOP.gsr -dir DIE_C2_TOP
Import gsr FCCC_BOT.gsr -dir DIE_C2_BOT
setup design
perform extraction -power -ground
perform -esdcheck -setupClamp
perform -esdcheck -rule esd.rule -ignoreError
```

Once the connection between the DIE and interposer is established in the 3DIC interface, the ESD checks can be performed between the Interposer bump and clamps present inside the individual dies. The figure below shows the ESD discharge path from Interposer C4 VDD bump to VSS bump through the clamp present inside the DIE

Figure 12-2 Multi-Die GUI View

Chapter 13
Substrate Noise Analysis

Introduction

This chapter describes Canyon Substrate Extension (CSE), an option that adds substrate modeling to the Totem and RedHawk power-noise analyses. It explains how to configure CSE, how to add substrate modeling to an existing setup, and how to view and interpret results from analysis. It assumes you are already familiar with Totem/Redhawk dynamic analysis.

Quick experiments with substrate analysis

A complete, accurate substrate-analysis setup requires care – follow the procedures in section "Reference A: Recipe for adding substrate analysis to your setup", page 13-458 before production use. However, to see how CSE works with Totem or Redhawk, you can quickly add substrate modeling to an existing, reasonably clean dynamic-analysis setup.

(1) Experimenting with GDS and noise injection

Suppose that you have GDS for a block, including its isolation structures, if any. Totem runs through extraction, tracing VDD/GND nets, and the result looks OK in SA view in Totem's GUI. To experiment with CSE:

1. Get the interconnect-technology file (.tech), substrate-technology file (.subtech), and substrate-capable layermap for a process similar to the one used by the design. For example, your AE or the ANSYS foundry team can provide a generic triple-well configuration. If the block uses an SOI or high-voltage process, you or your AE will have to create a generic configuration, perhaps based one of the examples shown in section section "Reference B: Describing substrates for CSE", page 13-464.
2. Add these GSRs to enable CSE:

   ```
   SUBSTRATE_ANAYSIS 1
   SUBSTRATE_TECH_FILE <process.subtech>.
   ```

3. Run setup design and extraction again, and check in the logs and the GUI that nothing looks seriously wrong.
4. Use inject_voltage or inject_current to create VDD and/or GND rail noise. Look at peak-to-peak noise maps on substrate layers and waveforms to see noise propagation.

(2) Experimenting with digital vectorless analysis

Suppose you have a small design with a digital block, and Totem already runs vectorless simulation on it. To experiment with it, follow the steps in (1), with two exceptions. In step 2, if the digital block uses multi-height cells, you may have to create a substrate library file to avoid errors when CSE builds the wells for the digital block; see section "Standard-cell GSR keywords and configuration files", page 13-438 for details. In step 4, run perform analysis -vectorless, since you already have it working.

(3) Experimenting with an SoC

If you want to experiment with a full SoC, first experiment with one of its GDS blocks using Totem, as in (1). Next, run Totem-CSE to convert the block's GDS data, including substrate shapes, for Redhawk.

Now suppose that Redhawk already runs vectorless simulation of the SoC that includes this block. Running with the converted GDS data, run Redhawk-CSE, following the steps in (2).

What is CSE (Canyon Substrate Extension)?

CSE, or Canyon Substrate Extension, adds substrate-noise modeling to the analyses performed by the Totem and Redhawk power-integrity tools. To run CSE as part of an existing chip/package analysis, add substrate technology information and a few control settings. CSE extracts an R/C electrical model for the substrate, couples it to chip interconnect, connects noise sources, and includes the substrate in the overall power-delivery network for dynamic simulations, CPM generation, and other analyses.

Among CSE's key capabilities are its ability to model almost any substrate, its full-chip capacity, its ability to analyze substrate in full-chip context, and the qualitative and detailed analyses it provides.

CSE supports *general substrate cross-sections*, sufficient to describe the silicon substrates for all customer processes to date. It supports multiple shallow wells, multiple (possibly nested) deep wells, and conducting and insulating sinkers. The substrate bulk may mix n-type, p-type, and insulating layers, enabling CSE to model the complex n-epi and SOI substrates used for high-voltage designs and the specialized, thinned substrates used for backside-illuminated CMOS imagers. See section "Reference B: Describing substrates for CSE", page 13-464 for details.

Using CSE with Totem or Redhawk gives you the speed and capacity needed to analyze *full-chip* substrate noise, including all power domains simultaneously. For SoCs and analog/mixed-signal (A&MS) designs, CSE efficiently models substrate noise generated by standard-cell logic and memories.

Accurate substrate analysis requires that you analyze your design *in context* of a chip's complete power distribution network, using all available global and block-level layouts and package models. Package/board impedance and coupling affect the levels of on-chip rail noise, but they play an equally important role in the effectiveness of guard rings and other on-chip substrate isolation structures. Unlike standalone substrate analysis, Totem-CSE and Redhawk-CSE provide this context.

CSE substrate-noise color maps give a *qualitative picture of substrate noise:* aggressor "hot spots," whether isolation is functioning, and how noise propagates to victim areas. You can select which substrate layers to display, adjust levels represented by colors, and display frequency-specific maps. Figure 13-1 shows a noise map for surface wells in an A&MS block.

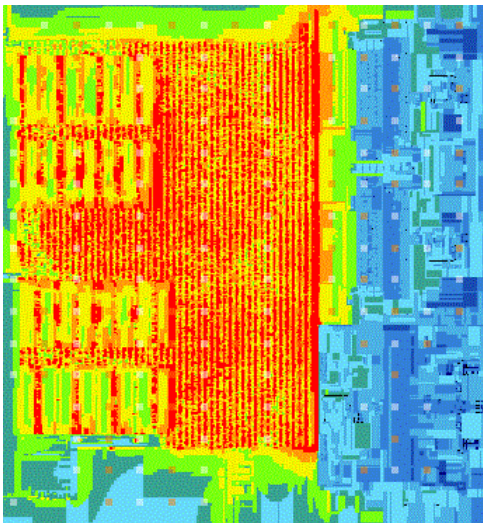


Figure 13-1 Color map for A&MS block

For *detailed noise measurement,* you can capture and view voltage and current waveforms anywhere in the design: at any point on any substrate layer, on chip interconnect, or at pads. For example, you can capture and display noise voltage for each substrate layer at a point in a victim, to understand how noise varies with depth in the substrate.

What CSE cannot model

To achieve full-chip capacity, CSE represents substrate as a linear R/C model. It models each P/N junction as a linear capacitor with a fixed value, i.e., it assumes small voltage variations around a fixed back-bias. This approximation works well for most wells, which tie to power rails.

However, in LDMOS devices, the drift-region/bulk diode sees large voltage variations. Fortunately, LDMOS SPICE models include these diodes and can take the large-signal variation into account. Alternatively, you can get good results by modeling the diode as a linear C that delivers the correct total charge for the voltage swing across it.

Because CSE uses a passive substrate model, it can't analyze situations where parasitic substrate devices become active: forward-biased substrate diodes, active lateral parasitic BJTs, or active SCRs causing latch-up. If you extract the devices some other way, say, with LVS, CSE may be able to provide an approximate R/C model of the interconnects among their pins. However, there is no clear way to separate parasitics within the devices from substrate parasitics or to define pins for the parasitic devices.

Nevertheless, CSE may provide some useful information. For example, to avoid latchup, you need to assure that the feedback loop forming the SCR is well grounded, since that will kill the gain needed to sustain latchup. With CSE, you can check resistance to VDD/GND under devices forming potential SCRs. Low enough R ensures that wells are sufficiently tapped to prevent latchup, even in areas that cannot support the tap density required by geometry-based rules.

CSE *can* analyze test structures designed for correlation to measurements, but doing so requires extreme care. For example, the small Cs between interconnect and substrate

have no effect on real designs, where device Cs and decaps completely dominate; in a small test structure, they affect results at high frequency. Probe impedances in the test fixture strongly affect isolation structures, so precisely modeling the measurement setup becomes critical. Contact R&D for guidance.

Introduction to substrate-noise analysis

This section introduces substrate noise and the problems it causes in the main classes of designs analyzed by CSE customers. It also introduces some the terminology related to substrate noise and gives a thorough introduction to substrate noise in CMOS triple-well processes.

What is substrate noise?

"Substrate noise" refers to noise conducted across a chip through its substrate. For example, as shown in Figure 13-2 chips often include both digital and analog blocks. The logic, memory, and I/O blocks generate voltage transients on their power rails. Sensitive analog circuits on the same chip use separate, isolated power supplies. However, rail noise can still propagate through the substrate, compromising the performance of the analog circuits.

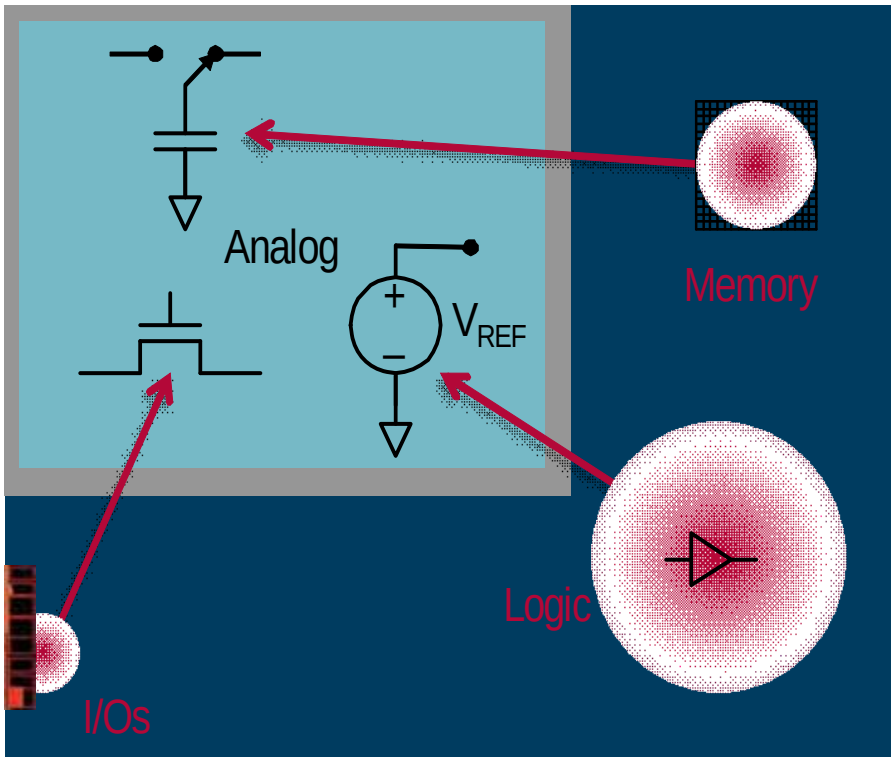


Figure 13-2 Digital/Analog Example

Figure 13-3 shows the role of substrate noise in a complete noise model. Noise "aggressors" generate noise that couples to the substrate through wells connected to the aggressor's power rails and parasitic device diodes. The substrate conducts the noise through its wells and bulk layers. Isolation structures, such as guard rings and deep wells,

break surface conduction paths, absorbing or blocking some of the noise. At the sensitive "victim" circuits, the remaining substrate noise couples to the victim devices or rails.

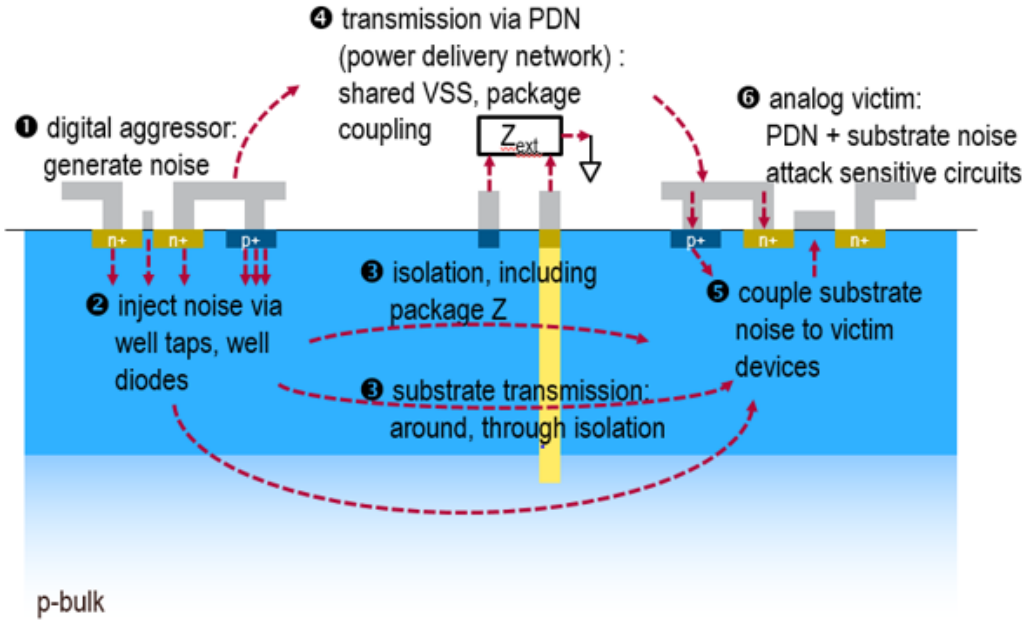


Figure 13-3 Combined Noise Model

The chip's package can have big impact on substrate noise. Increasing package impedance increases voltage swings at aggressors, increasing noise injected into the substrate. Package impedance also reduces noise absorption by isolation structures, increasing noise conduction. It can also reduce noise absorption by wells connected to rails in the victim area. Occasionally, parasitics internal to the package can couple aggressor rails or signals to victim signals, which then conduct the noise back to sensitive victim circuits.

Therefore, you should analyze substrate-conducted noise in context, and as part of the total noise affecting victim circuits. Substrate conduction will increase the total noise reaching the victim, but the noise may travel over complex paths through on-chip and package wiring as well. Package parasitics may significantly increase the noise that travels past isolation structures.

Due to increasing integration, more chips now mix noisy aggressors with sensitive victims. Substrate-conducted noise may cause these designs to fail, resulting in re-spins and costly project delays.

Chips with radios

Wi-fi, Bluetooth, AM/FM receivers, and similar chips look like SoCs with embedded RF IP blocks, as shown in Figure 13-4. The RF IPs may react to µV-level noise, but they share the die with logic, memories, and I/Os generating 100mV-level noise. These designs typically use a bulk-CMOS logic process, which provides limited isolation features: guard rings and deep n-wells.

Failures occur when digital noise reaches VCOs, mixers, and other sensitive circuits inside an RF IP. Noise levels as low as -30dBV to -40dBV may compromise overall performance by causing noise spurs, decreased sensitivity, or an increased noise floor.

Analyzing these designs requires modeling full-chip noise, including substrate noise, starting during floorplanning and extending through signoff. The analysis must also model noise signals over a much wider dynamic range than required for digital noise analysis.

CMOS imagers

Almost all analog designs contain significant digital support circuits. CMOS image sensors are typical of this class of circuit: a large pixel array surrounded by digital controls, data pipe, and buffer memories, and ADCs, sample/holds, and other analog interface circuits. The pixel array is heavily isolated using special substrate features. Also, for back-side illuminated sensors, wafer thinning eliminates most of the bulk layer, eliminating noise conduction through deep bulk layers.

However, substrate-noise conduction still causes failures in these sensors. As shown in Figure 13-5, the ring of substrate surrounding the pixel array conducts noise from the digital section to the interface circuits. You must carefully isolate the pixel-array interface circuits to prevent noise artifacts such as smearing or banding in the captured image.

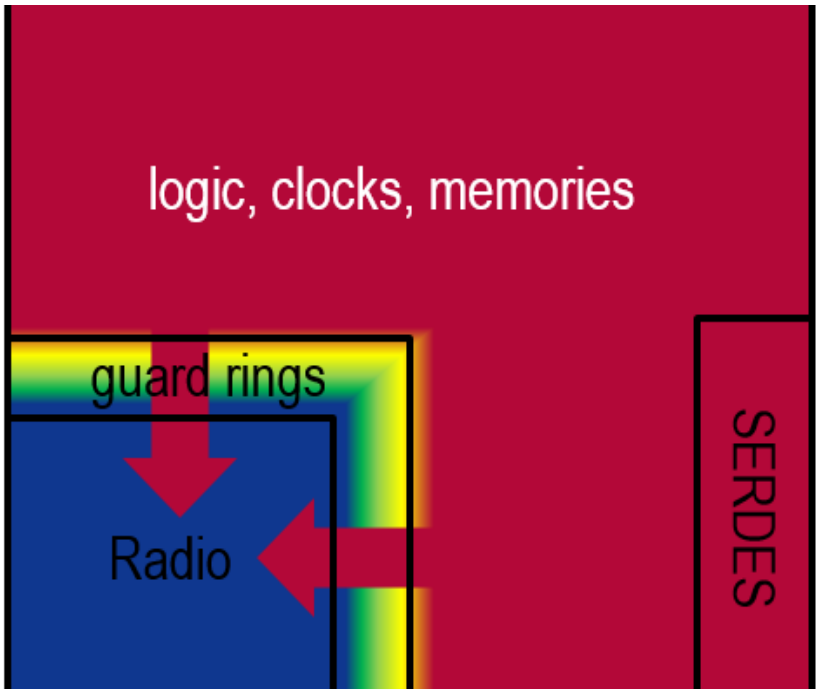


Figure 13-4 Substrate Noise in an SoC with an analog block

Analyzing image sensors requires modeling the special substrates and isolation structures used for these sensors. Also, because the digital noise may depend strongly on the image content, you must be able to simulate the data pipe and memories processing specific data.

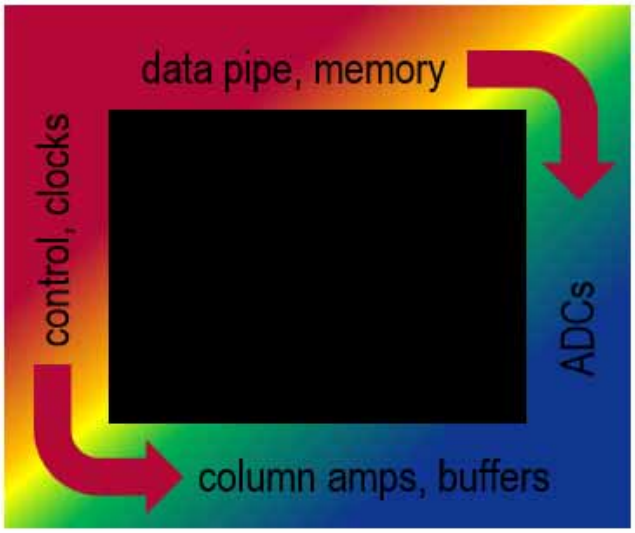


Figure 13-5 Substrate noise in a CMOS imager

High-voltage and high-current chips

Automotive-control, lighting-control, and other high-voltage analog/mixed-signal (A&MS) designs often use bipolar/CMOS/DMOS (BCD) processes. These designs mix high-voltage/high-current DMOS drivers with lower voltage logic, memory, or analog functions. They require processes with special substrate features, both to enable high-voltage operation and to isolate victims from LDMOS or DMOS aggressors.

Because these designs may mix DMOS transistors switching 40-50V with logic operating at 1V, failures may occur despite the strong isolation structures supported by the process. As shown in Figure 13-6 aggressors and victims switch roles: DMOS devices generate noise that can overwhelm their own digital control logic.

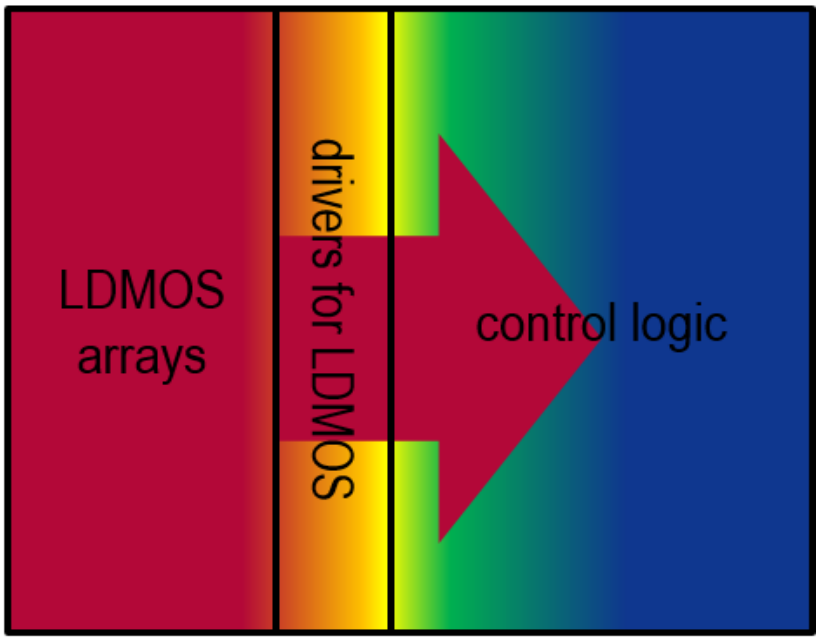


Figure 13-6 Substrate noise in a high-voltage A&MS design

Analyzing high-voltage designs requires modeling the special substrates that they use. These substrates may have multiple shallow wells, deep wells, and sinkers of various kinds. The bulk may comprise combinations of n-type, p-type, and insulating (SOI) layers. Also, you must be able to model injection from LDMOS and other devices not found in CMOS logic processes.

Low-cost packages

To compete, commodity designs of all kinds require low-cost packages. Bond wires and the packages themselves all contribute series inductance between on-die nets and AC grounds.

This inductance poses a threat, especially at video frequencies and above. Inductance in the VDD and VSS paths for on-chip logic increases power-rail noise, which couples as through wells under logic to the substrate. At the same time, inductance in the AC-grounding path for guard rings lessens their ability to absorb noise.

The trick is to choose the lowest-cost package that avoids substrate-noise-induced failures later. Early analysis using a floorplan and estimated bond wire and package parasitics enables you to quantify the likely requirements for your package. Later, you should analyze the chip and substrate together with the package to verify that noise levels are low enough.

Substrate noise in triple-well CMOS

Most designs use a triple-well CMOS process. Figure 13-7 shows a simplified substrate cross-section for the typical p-bulk process. “Triple-well” refers to the three main well types: shallow n-well, shallow p-well; and deep n-well. Same-type wells and bulk connect resistively; opposite-type wells connect capacitively, through back-biased diodes. P-well inside of deep n-well is DC-isolated from p-bulk by junction diodes.



Figure 13-7 Simplified 3well cross-section

Triple-well processes also support another choice of shallow well, known as non-well or native well. Non-wells have no well doping, so have electrical properties similar to the underlying p-type, lightly doped bulk.

P-well and n-well both have moderate resistivity. AC noise travels fairly easily through these surface wells, including through the junctions between them.

Isolation structures protect sensitive victim circuits by absorbing or blocking substrate noise. Typical layouts include isolation rings, known as guard rings, around victim circuits. Guard rings break the noise path through surface wells, absorbing noise before it reaches the victim.

Over long distances, conduction through bulk also becomes important. Although its resistivity is high (typically 10 Ohm-cm), bulk is hundreds of times thicker than wells, so viewed as a single layer, bulk has low sheet rho. However, noise traveling laterally in bulk must also travel a long vertical distance to see this lower resistance, which helps limit the amount of bulk conduction.

However, at frequencies above a few GHz, bulk impedance begins to drop. The intrinsic capacitance of silicon “shorts” the bulk’s resistance, reducing its overall impedance. Typically, intrinsic capacitance starts to significantly increase bulk conduction at 5 – 10GHz and above, reducing the effectiveness of isolation structures by conducting noise underneath them.

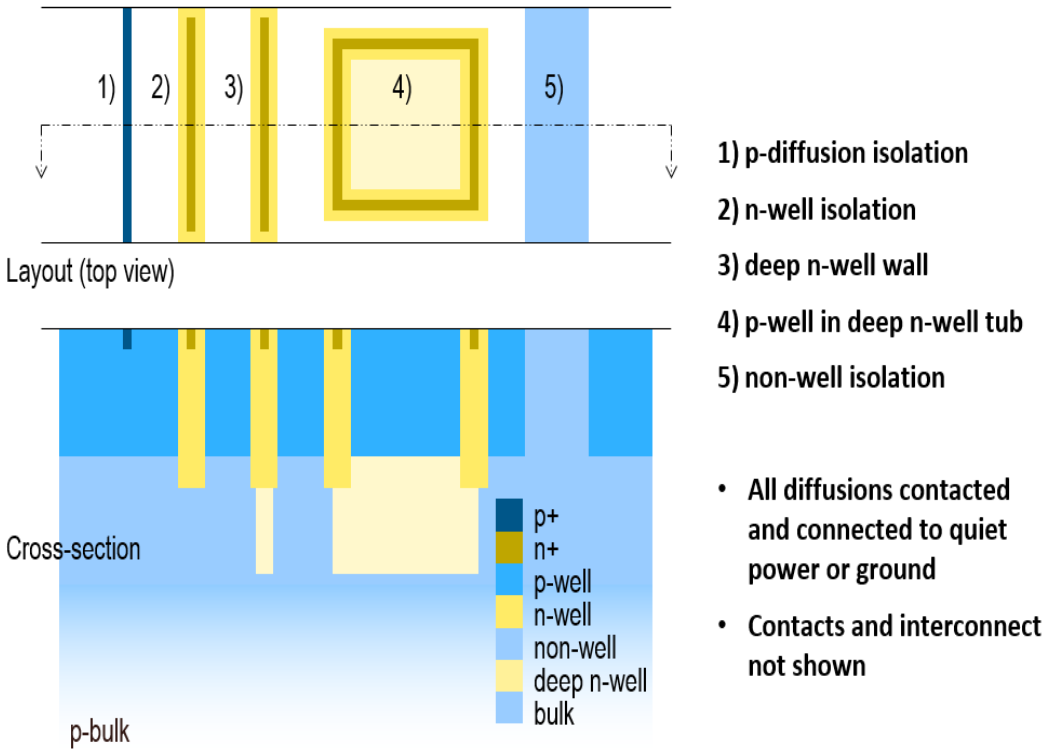


Figure 13-8 Examples of triple-well isolation

Figure 13-8 shows isolation options for a triple-well process. Structures 1 – 3 show options for building absorbing guard rings. To block noise, their well taps require a low-impedance connection to a quiet AC ground, i.e., a quiet ground or a quiet VDD. N-well guard rings absorb best when back-biased as weakly the design permits, since the n-well/p-well diode capacitance is then as large as possible. Of options 1 – 3, p-diffusion guard rings use least area, have the simplest grounding requirement, and can often provide sufficient isolation.

Deep n-well is primarily used for DC isolation of nFETs from bulk, for example, for a circuit requiring below-ground voltages or for embedded flash memories. A pocket such as structure 4 can shield a victim circuit inside the deep n-well from noise outside it. Equally, it can absorb noise from an aggressor inside the pocket to protect victims outside it.

However, this shielding relies on a low-impedance connection to a quiet AC ground. If you connect the deep n-well to a typical, noisy VDD, you will see the opposite effect: the well strongly radiates the VDD noise into the substrate.

Structure 5 shows a section of a non-well guard ring. It works by introducing high series resistance to break surface conduction paths through normal shallow wells. It has the advantage of not requiring a ground connection. Instead, it relies on other structures to absorb noise, e.g., well taps in a victim block. However, it may require significant area due to its design rules.

Setting up a design for substrate analysis with CSE

Use RedHawk for substrate analysis of SoCs, which usually contain a variety of digital, analog and IO blocks, and for designs with a large number of digital instances. Use Totem for analyzing substrates of smaller designs like custom IP blocks and smaller SoCs, including converting GDS geometry for use in RedHawk.

For substrate-noise analysis of custom IP blocks or certain kinds of analog/mixed-signal full-chip designs, such as automotive/high voltage designs with LDMOS aggressors, use Totem. CSE flow for custom designs shows key inputs for substrate analysis in Totem. Usually, you will receive the layermap, the interconnect-tech, and the substrate-tech files from the ANSYS foundry team. If you need to prepare the substrate configuration yourself, or just want to understand CSE modeling capabilities better, please see section "Reference B: Describing substrates for CSE", page 13-464. The GDS must include shapes describing the substrate features on layers consistent with the layermap.

For substrate analysis of SoCs with RF/analog victims, CMOS image sensors, and other SoCs with considerable LEF/DEF digital content, use either RedHawk-CSE or Totem-CSE. CSE flow for cell-based designs illustrates this flow. For GDS blocks of an SoC, use Totem with CSE enabled to create geometry with substrate data for RedHawk; when you export the model, Totem will store it in DEF-like format. Redhawk or Totem can then load these GDS cells. For standard cell regions defined by LEF/DEF data, by default CSE will automatically generate suitable well geometry. These options will be discussed in later sections. Use the same technology files for RedHawk as for Totem.

Settings needed for CSE

Every Totem-CSE or Redhawk-CSE run requires two basic keywords.

SUBSTRATE_ANALYSIS

Required to enable substrate modeling and analysis; also required when using Totem to export GDS blocks for Redhawk-CSE runs. If CSE is disabled, Totem and Redhawk ignore all CSE keywords. *Default: 0.*

Syntax:

```
SUBSTRATE_ANALYSIS [ 0 | 1]
```

SUBSTRATE_TECH_FILE <file>

Required to specify the substrate technology file; ignored in runs where CSE is not enabled.

Syntax:

```
SUBSTRATE_ANALYSIS [ 0 | 1]
```

Additionally, TSMC substrate configurations require `SUBSTRATE_WELL_PORTION`. We will supply the proper settings to use when you receive with the substrate tech file. Otherwise, do not use it:

SUBSTRATE_WELL_PORTION

Forces CSE to create sublayers for shallow wells (see section "Reference B: Describing substrates for CSE", page 13-464 to understand substrate terminology). Needed to maintain accurate resistance vertically and horizontally for substrate tech files that describe shallow wells or other features using sublayers with widely varying resistivity. CSE will create additional sublayers (resistor meshes) in the substrate model, increasing memory use and runtime.

Syntax:

```
SUBSTRATE_WELL_PORTION "<f1> <f2> ..."
```

Each value gives the thickness of a sublayer as a fraction of the total thickness of a shallow well. The sum of the fractions must equal 1. The first fraction refers to the top-most sublayer.

For example:

```
SUBSTRATE_WELL_PORTION "0.3 0.7"
```

specifies a top sublayer with 30% of the overall thickness and a bottom sublayer with 70%.

Standard-cell GSR keywords and configuration files

For a design that includes standard-cell regions (as LEF/DEF), CSE can automatically create wells and well taps under the standard-cell rows. The resulting model gives good accuracy for noise generated by or impinging on the standard cells, and it creates a smaller, faster-simulating model than if it models the detailed well layout directly. It eliminates the need for GDS files describing the standard-cell well structure, except in rare cases with relying on specialized well structures for isolation under or around the standard cell regions.

SUBSTRATE_LIBRARY_FILE <file>

Required for designs using cell libraries with tap cell or multi-height cells and sometimes required to properly connect substrate body pins. See below to understand when to use this file and how to format it.

SUBSTRATE_AUTO_DNW [0 | 1]

SUBSTRATE_AUTO_DNW_CONFIG <file>

SUBSTRATE_AUTO_DNW enables automatic generation of deep N-wells under P&R standard-cell areas. Requires a DNW configuration file. See below to understand when to use this file and how to format it.

Creating a SUBSTRATE_LIBRARY_FILE

The substrate library file comprises these sections: tap-cell declarations; n-well /p-well configuration; and substrate body-pin declarations. Each section is optional, but you will usually need to provide the first two, since most cell libraries contain tap cells and multi-height cells.

Tap-cell declarations

In this section, declare tap cells for the standard-cell libraries used in the design use tap cells. CSE will then connect standard-cell wells to rails only at these tap cells. Otherwise, CSE assumes that every standard cell has integral taps. Use this format to declare the contact resistance and the pin name for each tap:

```
TAPCELLS {
    <tapcell1> <nwell contact R> <pwell contact R> <nwell
pin> <pwell pin>
    ...
}
```

For example:

```
TAPCELLS {
    HS90BB_LH_FILLERNPW2    0.5 0.5 vdd gnd
    HS90BB_LH_FILLERSNPW6   0.5 0.5 CNW CPW
    HS90_LHDN_LSSIPX15      0.5 0.5 vddin  gnd
    HS90_LHDN_LSSIPX30      0.5 0.5 vddin  gnd
    HS90_LHDN_LSSOPX38      0.5 0.5 vddout gnd
```

```
    HS90_LHDN_LSSOPX19      0.5 0.5 vddout gnd
    mco_c090_pwrsw5         0.5 0.5 vdd    gnd
    mco_c090_pwrsw8         0.5 0.5 vdd    gnd
}
```

Well configuration

By default, CSE generates wells under each standard-cell row. It covers the bottom half of the cell with p-well and the top half with n-well. If the substrate tech file defines multiple n-wells or p-wells, CSE uses the shallowest well of each type, which are usually the wells for logic circuits. It extends the wells along the width of the row to fill gaps, in case the row does not have filler cells.

Use this section to override these defaults. For example, if a cell contains double-height cells, i.e., cells that span two standard-cell rows, each such cell usually contains p-well near the bottom ground rail, n-well surrounding power rail in the middle of the cell, and p-well near the top ground rail. In this case, the substrate library file must define the well structure for each cell height.

Use this format:

```
pwell_config [<p-well name>]{
    [extension <distance along row>]
    <cell_height1> <from0> <to0> [<from2> <to2> ...]
    ...
}
nwell_config [<n-well name>]{
    [extension <distance along row>]
    <cell_height1> <from1> <to1> [<from3> <to3> ...]
    ...
}
```

The optional well name overrides the default choice of well based on well depth.

The optional `extension` specification limits how far CSE extends wells from the sides of cells to cross gaps in a row. If the gap between two cells is smaller than twice the specified distance, CSE fills it; otherwise, it does not. Without this specification, CSE fills gaps of any size.

The cell-height entries override the default 50%/50% well structure for cells. Each line defines one of more stripes of wells inside cells of the specified height. Stripes of alternating well types must abut and must exactly cover the cell height.

For example, the cells in Figure 13-9 contain p-well (blue) and n-well (yellow) as shown, requiring this well specification:

```
# heights in microns
pwell_config {
    #ht  fr  to  fr  to
    1.2  0.6 1.2
    2.4  0.0 0.3 2.1 2.4
}
nwell_config {
    #ht  fr  to  fr  to
    1.2  0.0 0.6
    2.4  0.3 2.1
}
```

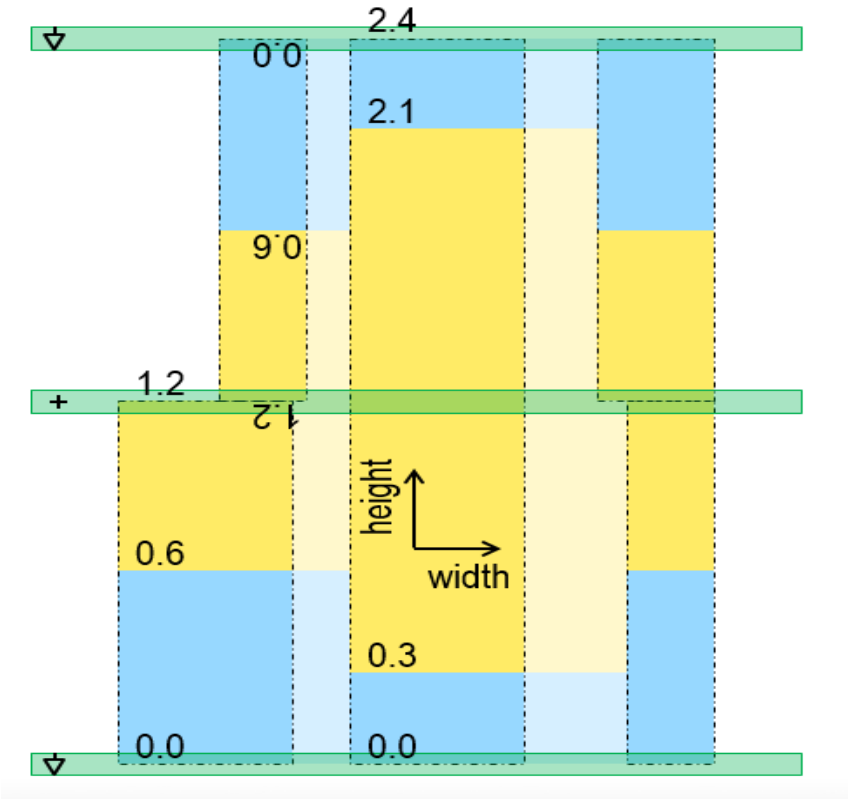


Figure 13-9 Standard-cell row with multi-height cells

CSE will fill gaps between pairs of cells with the same p-well (light blue) or n-well (light yellow) as in the left-hand cell, as shown in the figure.

Note that CSE applies well configurations globally. For example, it uses the same well configuration for all cells of a given height, even if they belong to different libraries, say, a logic library and a pad library.

Substrate body pins

For most applications, you can ignore body pins in cells. Standard cells create noise on their rails, which couple noise into the underlying wells, and they couple it to the substrate. Substrate pins carry small currents compared to drain/source pins of devices in cells.

However, CSE can automatically connect substrate body pins if it has sufficient information. The substrate body pin must appear in the LEF file defining the cell, assigned to Nwell or Pwell layers. The pin also must appear in the *.lib* file.

If substrate-pin data is missing, or to override this automatic processing, you can manually specify the substrate body pins one of two ways. If all cells use the same body pin names, declare the names globally:

```
BODY_PWELL_PIN <P_pin_name>
BODY_NWELL_PIN <N_pin_name>
```

If cells use different pin names, enumerate them:

```
SUBSTRATE_BODY_PIN {
```

```
    <substrate_layer_name> <cellname> <body_pin_name>
    ...
}
```

For example:

```
SUBSTRATE_BODY_PIN {
    PW HS90BB_LH_AND2ABX10 gnds
    NW HS90BB_LH_AND2ABX10 vdds
    PW HS90BB_LH_AND2ABX19 gnds
    NW HS90BB_LH_AND2ABX19 vdds
}
```

Creating a SUBSTRATE_AUTO_DNW_CONFIG

Some designs use deep-n-well tubs and n-well guard rings to isolate P&R standard-cell areas to reduce the noise they couple to the substrate or to provide additional decap. CSE can create these tubs and rings automatically, based on the description in this configuration file.

The SUBSTRATE_AUTO_DNW_CONFIG file controls where and how CSE creates tubs and rings, using these specifications (dimensions are in microns):

DNW_BOUNDARY_WIDTH <distance>

The distance deep n-well extends beyond the standard cell boundary. *Default: 0.5*

DNW_NWRING_GAP_FROM_EXTERNAL_INSTS_GEOS <margin>

If a deep-n-well or n-well ring collides with other geometries and/or external instances, CSE automatically adjusts it to avoid overlaps by this margin. *Default: 0.5*

DNW_NWRING_REGIONS

Specifies the lower left and upper right corners of one or more rectangular n-well rings. Otherwise, one ring surrounds all standard cell regions. The syntax is:

```
DNW_NWRING_REGIONS {
    <ll_x> <ll_y> <ur_x> <ur_y>
  ...
  }
```

IGNORE_EXTERNAL_INSTS [0 | 1]

By default, CSE will not create DWN wells inside of non-standard-cell instances. Setting this control causes CSE to ignore neighboring instances and always create the specified wells.

NW_RING_EXTN_FROM_DNW_BOUNDARY <distance>

Specifies how far the n-well ring extends beyond the DNW boundary. *Default: 0.5*

NW_RING_DIST_FROM_STDCELL_BOUNDARY <spacing>

Specifies the spacing between the n-well ring and the outside of the standard cell boundary. *Default: 0.0.*

Modeling substrate-noise sources

The problem of modeling substrate noise divides into three main pieces, as suggested by Figure 13-3 modeling noise sources (aggressors); modeling the substrate itself, including all the substrate features in the design (noise-conduction “channel”); and modeling noise victims. Additionally, Totem can export the substrate model as part of a CPM, for simulating externally with SPICE.

Standard-cell aggressors

Standard-cell logic often generates most of the substrate noise in an SoC. To model logic blocks as aggressors, follow these steps:

1. Set up logic blocks for DVD analysis exactly as you would in Redhawk or Totem. You do not need to characterize bulk-pin current in APL, since it makes an insignificant contribution to generated noise.
2. Use SUBSTRATE_LIBRARY_FILE to enable CSE to build a suitable well structure.
3. Run simulation `(perform analysis)` as for Redhawk or Totem

Switching activity creates rails noise, which then couples through well taps to the substrate.

Memory aggressors

Memories can also create significant noise in an SoC. To model them as aggressors, follow these steps:

1. Characterize memories for DVD analysis exactly as you would in Redhawk or Totem.
2. Use ERV models to represent their layouts. To create the substrate model of an ERV block, CSE retains the well shapes from its GDS layout, and creates taps from the rails in the ERV. This approach efficiently couples memory noise to substrate without requiring details of the memory’s complex interconnects and taps.
3. Run simulation `(perform analysis)` as for Redhawk or Totem.

Similarly to logic, memory switching activity creates rails noise, which then couples through well taps to the substrate.

Manual noise injection for logic and memories

Suppose you have the layout for logic and memories in a design, but you do not have DVD analysis set up yet. To approximate the noise they will generate, use manual noise injection to apply the maximal rail noise you expect to the VDD/GND pads supplying their domains.

For an initial analysis, inject clock-frequency sinusoidal voltage with a magnitude of roughly the maximal allowed ripple (e.g., 10% of VDD), applying it to all VDD pads. Optionally, apply this noise differentially all VDDs and all GNDs, to model the partial cancelling of substrate noise caused by simultaneous VDD drop and GND bounce. See section "Manual noise injection", page 13-449 for inject_voltage usage.

Of course, this approximation won’t exactly match the spatial distribution of rail noise. It also won’t match the spectrum of actual rail noise – instead, you can perform multiple simulations at clock harmonics or other frequencies of interest instead. Nevertheless, it gives an easy way to take a quick look at substrate noise, for example, as you begin setting up a big design, or early in design, where details of logic and memory blocks may be missing.

LDMOS and other noisy devices

You can model noisy devices one of two ways: using manual noise injection; or using aplmmx to characterize device currents. Manual injection is much easier to set up and run; characterized currents are more precise spatially and spectrally.

For manual noise injection, connect the noise source to the noisy nets that will couple to the substrate. For standard FETs, you can usually inject on VDD/GND, since they will couple to the substrate through wells, as in digital areas. However, for LDMOS, you inject noise onto the nets connecting to the n-wells under them: for nLDMOS, the signal net connecting to the drain pin; and for pLDMOS, the VDD connecting to the body/source pins. (Declare the nLDMOS drain nets as VDDs so that Totem includes them in analysis.)

For more information about modeling LDMOS, see section "Reference D: LDMOS noise sources", page 13-501.

Modeling substrate structures

CSE builds an RC model of the substrate based on drawn substrate features in GDS and depth and electrical descriptions of the features in the substrate technology file. The GDS layermap maps GDS layers to their substrate-technology layers.

During extraction, Totem-CSE traces from all VDD/GND diffusion to same-type substrate features, and from them, through the entire resistor network in the substrate model. Substrate conductors that without resistive connections to a PLOC are said to be “floating”.

In bulk CMOS designs, CSE should find very few floating features. Almost every well has one or more taps to a rail. P-wells over p-bulk all connect to bulk, forming one big net.

Except in SOI processes, CSE doesn’t include floating features in simulation. Floating features display as black areas in voltage maps, allowing you to see more easily if VDD/GND connect as expected. For more information on including floating features in simulation, see section "SUBSTRATE_CONNECT_FLOATING_GEOMETRY [0 | 1]", page 13-504

Check that your setup includes all isolation structures – substrate analysis without them gives wrong results! Similarly, check that noise-absorbing (active) isolation structures connect to AC grounds as intended, since a floating guard ring may spread noise instead of absorbing it.

Modeling victims

CSE has no information about which parts of a design are victims and does no special modeling for them. You are responsible for analyzing them, based on each victim’s noise sensitivity.

To model victims, include their layouts with their substrate shapes. Connect any active isolation they contain to PLOCs.

Assure that victim rails connect to PLOCs. Wells and other substrate-coupled features within a block change noise levels inside it. Tapped wells near the boundaries of a victim will absorb noise that passes through or under intentional isolation around the victim. They may also spread the absorbed noise through interconnect to other wells deeper in the block.

At frequencies above a few GHz, intrinsic capacitance of silicon significantly reduces impedance of the 10 Ohm-cm bulk common in MOS processes. High-frequency noise may then travel underneath guard rings, rendering them less effective. Bulk-conducted noise hits the victim’s wells, which then transfer the noise to victim rails. Model victim wells and their external connections carefully in such cases.

Modeling the package

At a minimum, include estimated L/R package impedances for substrate analysis. A guard ring may work unrealistically well without series impedance to AC ground, but fail when grounded through realistic package impedance. Similarly, wells inside a victim may absorb noise unrealistically well without series impedance for the victim rails.

Also, if isolation structures do not have dedicated AC grounds inside the package (or board), consider using a package model that represents this coupling. A guard ring may work as expected with series, decoupled impedance between its pads and its voltage source, but fail if the package couples it to other, noisy pads before it connects to the external voltage source.

CPMs that include substrate

If you set up a design with CSE, then create a CPM, the CPM will include the substrate model – no special settings are required. You can also add probes as ports to the CPM (an iCPM), for example, to monitor substrate noise at selected x/y/layer locations.

Using CPMs with substrate can make some analyses easier. For example, suppose you want to choose among several packages, checking the noise in well in a victim block. If a separate package team performs this task, a CPM lets them run SPICE, instead of having to learn Totem. Create an iCPM that includes probes for the wells. Then create a SPICE netlist to instantiate the iCPM and connect it to each package model, simulate in SPICE, and check the noise on the probe ports.

Of course, you can perform the same checks with Totem-CSE. Unlike a CPM, Totem-CSE gives you full visibility inside the design. Therefore, even if you use SPICE simulation to select a likely package, it makes sense to confirm the choice using Totem-CSE.

Noise Injection Analysis

To understand how noise *propagates* through a design, instead of simulating the actual noise-generating activity in the design, you can often model noise using a simple “signal generator.” Using a signal generator requires far less input data and setup for the noise source than vector-driven or vectorless noise generation requires. Of course, it does not create the exact noise distribution within the aggressor, but it gives you a way to easily estimate propagated noise, even during early design.

For example, to check how much substrate-conducted noise travels from a digital aggressor block to a victim block, impose sinusoidal ripple at frequencies of interest on the rails in the aggressor. This noise will couple through well taps in the aggressor to the substrate and propagate through the design to the victim. Using color maps and probes, you can gain a detailed understanding of how well substrate isolation structures are working.

The Tcl commands *perform analysis -inject_current* (or its original name, *-inject_noise*) and *-inject_voltage* can apply noise signals to multiple points in a chip or block. They automatically disable other noise sources and set simulation controls, allowing you to use them with few or no changes to existing GSRs.

Each *inject* command supports one sine, one step, or multiple user-supplied waveforms. Use a sine for most applications, e.g., determining substrate-conducted noise at frequencies of interest. Use a step to determine how long it takes the design to settle RC charging and RLC resonances to steady state.

If necessary, provide your own waveforms to determine the response to specific stimuli, e.g., supply current drawn during a typical clock cycle. You can provide multiple stimulus waveforms for a single analysis command to see the superposed response.

Each *inject* command allows you to attach copies of the source(s) to multiple locations. For each set of locations, you may provide an optional, signed magnitude for the sources. For example, you can inject balanced (differential) current into power and ground rails by specifying magnitudes of 1 and -1.

Additionally, for RF designers, *perform analysis -S21* simplifies finding values of the S-parameter S21 between two PLOCs at specified frequencies. It allows them to easily calculate S21 between two ports in the presence of grounded guard rings or other isolation structures.

Inject_current Tcl command

The Tcl command for `-inject_current` using built-in waveforms has the general form:

```
perform analysis -inject_current <waveform> <?magnitude?
locations>... ?-time2ss <sec>?
```

For example:

```
perform analysis -inject_current -sine 100e6 { probeVDD1 -mag 1e-3
probeVDD2 -mag 2e-3 } -time2ss 50e-9
```

attaches a 1mA, 100MHz sinusoidal current source to the probe *probeVDD1*, another 1mA source to *probeVDD2,* pre-simulates 50ns to settle the design to steady state, then simulates one cycle (10ns) to produce the result of interest.

<*waveform*> specifies a built-in waveform:

- -sine <frequency, Hz>: 1A, 0-phase sine of the specified frequency
- -step: 1A step at t=0+

<*mag/locations*> specifies one or more locations at which to apply sources with the preceding waveform, optionally scaled by a signed magnitude:

- -mag <factor>: signed scale factor, e.g., -1 to invert the waveform
- locations: a Tcl list of one or more locations *{ loc1 loc2... }*, where each *loc* has one of these forms: a PLOC name; the name of a probe added with *add probe;* an absolute location as *x y layer* or as *layer x y*

For example, this location specification:

```
{ 1 100 PWell PDiff 100 1 plocGNDll gndprobe3 }
```

attaches sources to: (1, 100) on the substrate layer PWell; (100, 1) on the interconnect layer PDiff; the PLOC *plocGNDll*; and the ground-net probe *gndprobe3*.

Time to steady state

In general, the chip/package response to a periodic stimulus is the sum of a zero-state response and a steady-state response. The zero-state response goes to zero once Cs and Ls reach states consistent with the stimulus, leaving only the steady-state response.

For noise analysis, you almost always want only the steady-state response. Unfortunately, there is no practical way to simulate only the steady-state response using Totem simulation methods, so in general, you must “pre-simulate” enough cycles of the stimulus to allow the system to reach steady state.

Use the option *-time2ss* <*sec*> (time to steady state) to specify sufficient settling time for the chip/package you are analyzing. The *inject* commands then pre-simulate for an

integral number of cycles of the injected waveform equaling or exceeding the time you specify. (See also the GSR DYNAMIC_PRESIM_TIME, which solves the same problem for other simulation commands.)

To determine settling time, simulate using a *-step* waveform with the injection locations you are planning to use. Normally, specify in *-time2ss* the time at which the step response reaches steady state.

Occasionally, settling time >> period of the stimulus. For example, on-chip C may charge slowly through high series impedance. Since the zero-state response will change very little during one cycle of the stimulus, the combined response nearly equals the desired steady-state response. Do not specify a settling time, which will cause a long simulation time with no benefit in accuracy.

Injection at PLOCs

When attaching a source to a PLOC, *inject_current* isolates the PLOC by connecting a 10M Ohm resistor between it and its external supply or package pin. Otherwise, current would flow directly into the external supply or into the package, instead of into the design.

If you want to inject noise at a PLOC for a rail, you usually must inject noise at all PLOCs for that rail. Otherwise, non-injected PLOCs remain externally connected, shunting the injected noise externally.

Injecting user-supplied waveforms

The Tcl command for *-inject_current* using user-supplied waveforms has the general form:

```
perform analysis -inject_current  -wave <waveA>  <?magnitude? locations>... [ -wave <waveB>  <?magnitude? locations>... ]… ?-time2ss <presimTime>?
```

That is, unlike with built-in waveforms, you may specify more than one waveform with its own mag/locations parameters. For example:

```
perform analysis -inject_current -wave waveA.txt  { plocVDDA } -wave waveB.txt { plocVDDB }
```

attaches a source with waveform *waveA.txt* to *plocVDDA* and a source with waveform *waveB.txt* to *plocVDDB*.

The files waveA.txt, etc., each contain a piecewise-linear waveform, with one time/value pair per line, in ascending time order. Start the waveform with 0 at t=0, to match simulation assumptions. Give time in psec, and value in A, e.g.:

```
0       0
100     1e-6
200     2.3e-6
...
```

For built-in waveforms, noise injection controls all simulation settings, and ignores GSR settings for DYNAMIC_SIMULATION_TIME, DYNAMIC_TIME_STEP. However, for user-supplied waveforms, you should set these GSRs yourself. Otherwise, Totem will use the same values as for other analyses, based on the FREQUENCY GSR and other defaults.

Inject_voltage Tcl command

The Tcl command for *-inject_voltage* has the same form as for *-inject_current.* As you would expect, the only difference is that it attaches voltage sources, instead of current

sources, to the specified locations. Also as for *-inject_current,* when you attach to PLOCs for a rail, you attach sources to all PLOCs.

S21 Tcl command

The Tcl command for *-S21* using user-supplied waveforms has the general form:

```
perform analysis -S21 <frequency, Hz> <PLOC1> <PLOC2> ?-time2ss
<presimTime>?
```

It applies 50-Ohm terminations to both PLOCs and a 40mA sinusoidal stimulus PLOC1, then runs time-domain simulation. The response at port 2 is a sinusoid with magnitude and phase equaling S21 at the specified frequency. Because it leaves all other PLOCs connected externally, it accounts the effects of isolation structures, such as guard ring, that absorb noise between the two ports.

Frequently-used analyses for understanding substrate noise

This section describes analyses commonly used when understanding substrate noise. Resistance-based checks require only design setup and extraction. Simulation-based checks require performing a vectorless, vector-driven, or noise-injection dynamic simulation.

For simulation-based checks, Totem and Redhawk support two main ways to model noise sources: manually injecting sinusoidal or other noise waveforms; and simulating activity in aggressor blocks. For example, you can manually inject a noise tone, i.e., sinusoidal noise at a specific frequency, at one or more locations in a design.

At chip level, you can use manual noise injection early in design, when you are still prototyping floor plans and choosing isolation structures. Analyzing substrate noise early helps avoid expensive surprises later in design. Even after you have a full layout, you can manually inject noise onto power rails to compare noise at different frequencies.

Alternatively, you can simulate the activity of aggressors. In the common case of digital aggressors, you can perform vectorless or VCD-driven simulation to generate noise on their power rails, which then couples to the substrate through well taps. You can also perform waveform-driven simulation at device level in Totem to model noise generated by analog or mixed-signal blocks.

After simulation, you can visualize and measure noise using one of the methods described below. Most methods apply to both interconnect and substrate noise.

Guard-ring weakness check

Guard rings require low-impedance connections to quiet VDD/GND pins to absorb noise and shield areas of substrate properly. Weakly grounded guard rings will not block substrate noise, and they may even provide conduction paths that increase noise.

This check identifies guard rings weakly connected to their pads. Separately check the impedance in the package model between the pads and external supply.

Use the menu command **View -> Substrate Noise Map -> Guard Ring Resistance Weakness Map** to bring up a color map displaying guard rings with high resistance. Figure 13-1shows an example of the results.

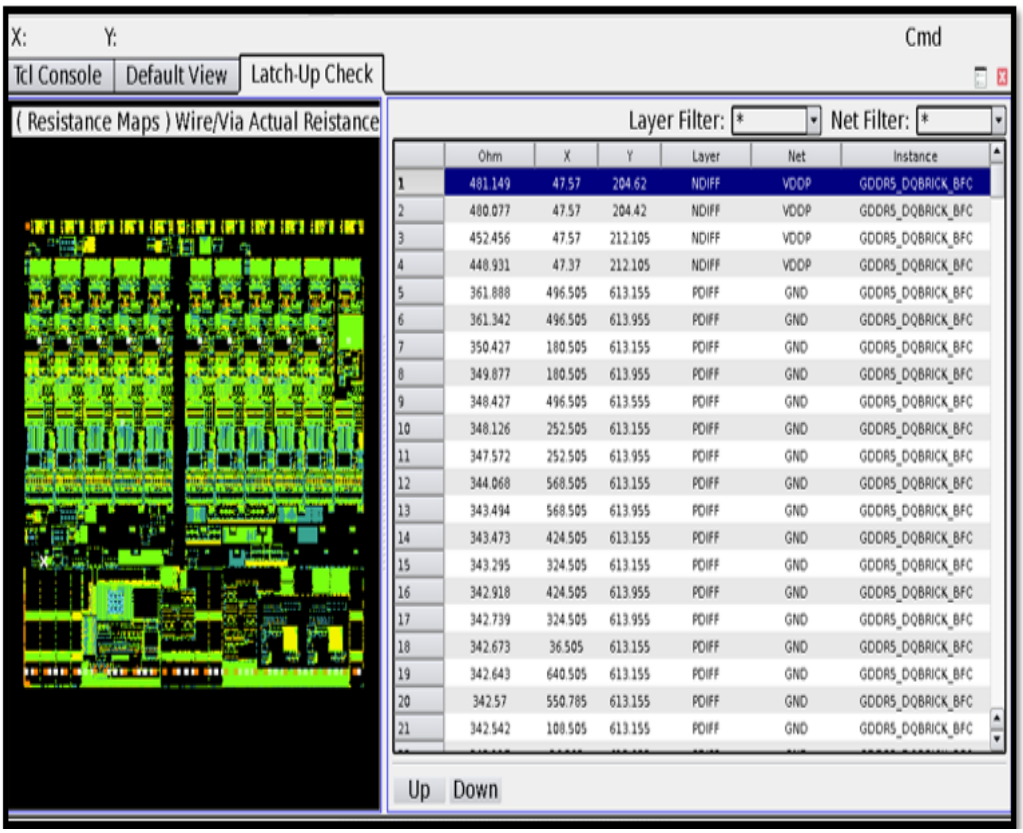


Figure 13-1 GR check

Use the menu command **View -> Connectivity -> List of Guard Rings Weakness** to execute the guard ring weakness check. Checking guard ring structures shows a least-resistance path from a guard ring, traversing the entire design from left (guard ring in white box) to right (bump).

You can also invoke guard-ring checking using the TCL command:

```
perform gridcheck -guardring
    ? -box <lowleft_x> <lowleft_y> <upright_x> <upright_y>?
```

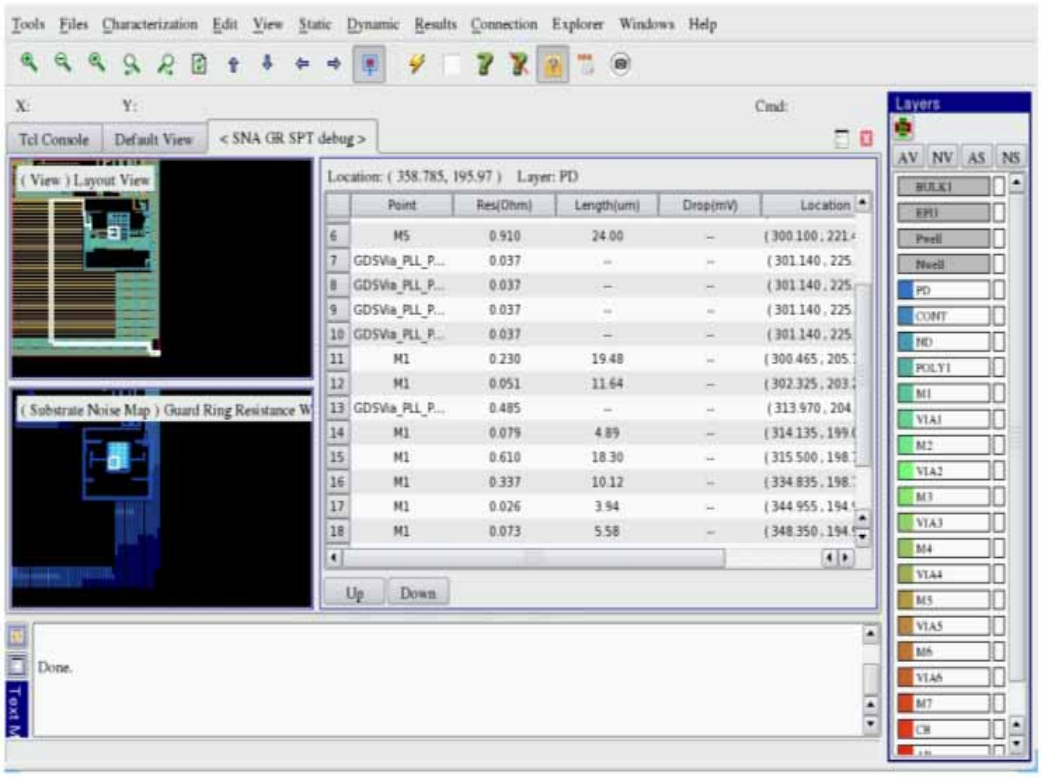


Figure 13-2 Checking guard ring structures

Point-to-point resistance calculation

Point-to-point resistance calculation (“p2p resistance”) calculates the resistance, and optionally, the least-resistive (“shortest”) path, between two points you specify. Middle-clicking in an SA window of the Totem GUI invokes p2p calculations interactively, as does the Tcl command `perform res_calc`. P2p has several applications for substrate analysis, for example:

- Check resistance between a pad and a guard ring
- Check resistance between a well taps, for example, between a noise injection point and a guard ring or a tap inside the guard ring
- Check whether two nodes are DC connected – p2p resistance calculation will fail if no DC path exists

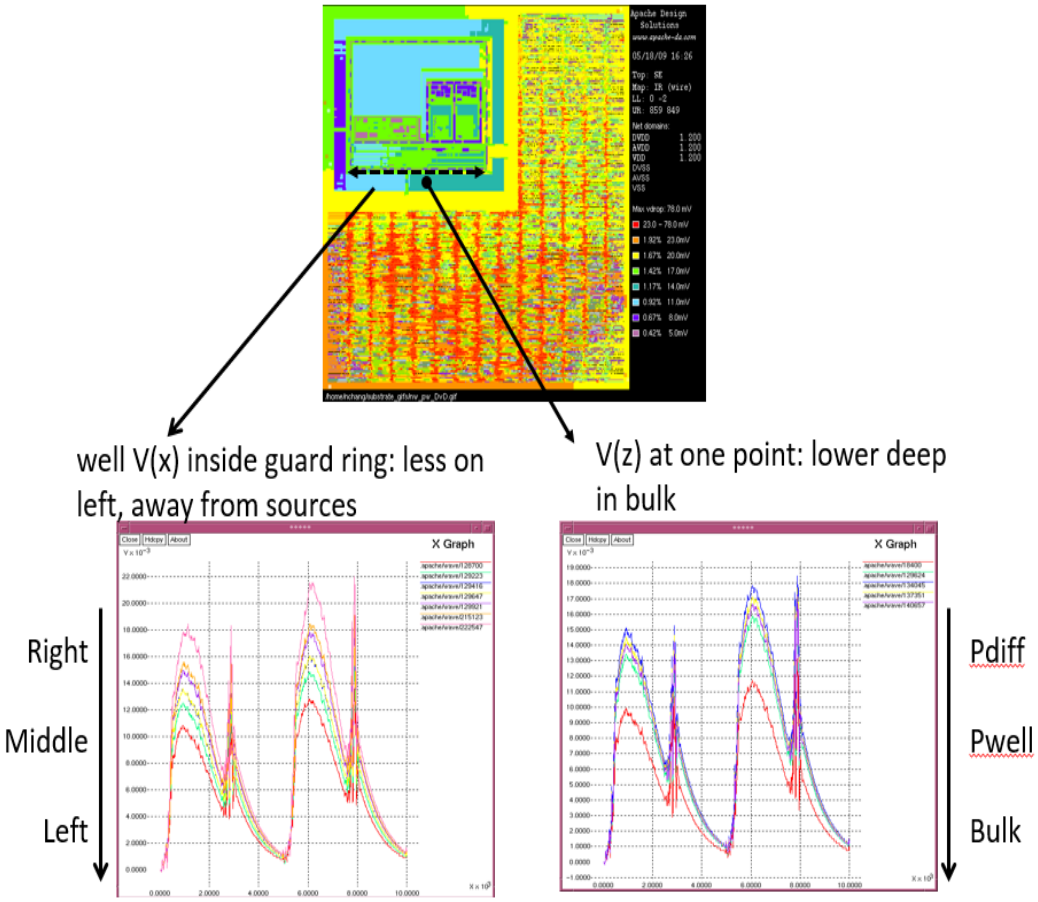
When checking the resistance of substrate structures, specify points on metal or possibly diffusion. Use start/end points on substrate layers for only continuity checking. Starting or ending on a well or other substrate layer probes a single node in the 3D substrate model, not an area. The resistance from such a virtual “point contact” with the substrate is unpredictable, but often large.

Manual noise injection

The options `–inject_current` and `-inject _voltage` to the Tcl command `perform analysis` apply a noise waveform at one or more locations in a design. You can attach these noise sources to PLOCs or to any location on any layer, including

substrate layers, to act as aggressors, and see the effects on victim areas. Some common uses include:

- *Determine settling (presim) time to reach steady-state.* To determine settling time for initial transients, use `-inject_voltage -step` to disturb the circuit, then observe the resulting voltage response. Specify the observed settling time in subsequent noise-injection analyses using `-time2ss <seconds>` to see steady-state responses to sines or other waveforms. In steady state, responses to sinusoidal noise will be symmetric, with equal positive and negative peaks. Prior to steady state, you may see a sine combined with a decaying exponential, or a sinusoidal-looking waveform with the wrong apparent frequency due to a decaying phase shift.
- *Analyze noise propagation versus frequency.* Using the option `-sine <frequency>`, inject noise tones at frequencies of interest, e.g., at the clock frequency (and harmonics) of digital aggressors. For example, `perform analysis inject _voltage -sine 100e6 -time2ss 50e-9 <list of PLOCs>` will inject a 100MHz tone on the specified PLOCs, giving at least 50ns of pre-simulation time to settle the transient response. Simulation runs for one full cycle. Peak-to-peak voltage or current maps will then show how noise spreads at that frequency, and capturing and plotting waveforms with probes will give magnitude and phase of noise at specific locations.
- *Check a substrate-analysis setup.* Before modeling aggressor activity, you can inject



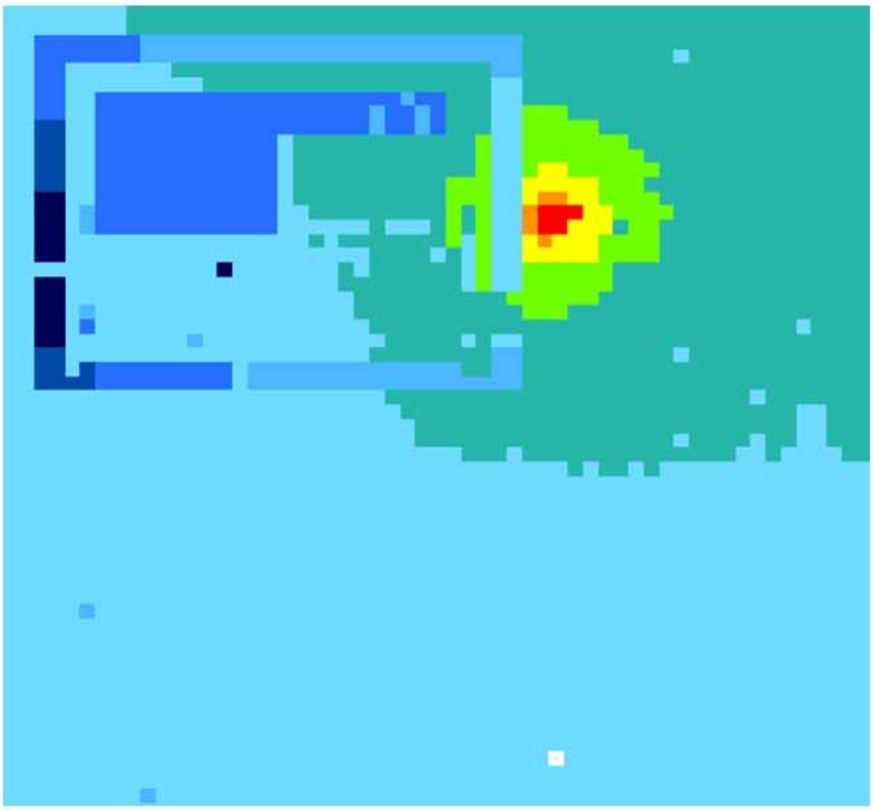


Figure 13-3 Using injected noise

Figure 13-3 shows noise injected near one side of a custom block surrounded by a guard ring. The guard ring works poorly because the process uses a low-resistivity substrate, allowing noise to easily leak under the guard ring. Also, the gap in its lower-right corner causes even more noise leakage, as the color map shows.

Coupling analysis using S21

RF designers usually characterize networks in terms of s-parameters. Totem's `-S21` analysis simplifies calculating S21 through the substrate, say, for the substrate-noise path from an aggressor port into the substrate, through a guard ring, and out to a victim port.

For example, the Tcl command `perform analysis -S21 -sine 1e9 PLOC1 PLOC2` disconnects both PLOCs externally, terminates each with 50 Ohms, then injects a 40mA sinusoidal current at PLOC1. The voltage waveform at PLOC2 then reflects the magnitude and phase of S21 at 1GHz. By running at several frequencies and measuring the magnitude of the response waveform, you can graph S21 between the two PLOCs, including the effect of guard rings or other noise absorbers between them.

As when you use `-inject_voltage` and `-inject_current`, specify enough settling time to reach steady state.

Peak-to-peak voltage maps

Available only in Totem, peak-to-peak voltage maps display full-chip, peak-to-peak ripple. They give full spatial resolution on interconnect and substrate layers. You can use these

maps to understand qualitatively chip-level substrate-noise propagation, noise “hot spots,” and detailed noise transmission at the boundaries of victim blocks. Use them with noise injection to study frequency-specific propagation or with noise simulation to understand overall noise levels.

Use the menu pick **View→Voltage Maps→Wire and Via Peak-to-peak noise map**. Make visible all the layers at the same depth, for example, all shallow wells, then display the map, possibly adjusting the color range to better resolve noise levels in interesting area. To compare maps for different conditions, use matching color ranges for all the maps you want to compare. For example,Figure 13-4 shows noise magnitudes using the same color scale at different depths in a design with a high-resistivity substrate.

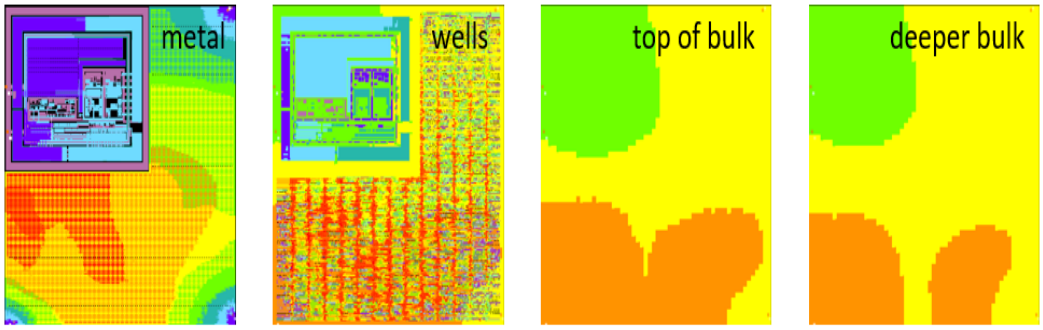


Figure 13-4 Peak-to-peak noise maps

Both RedHawk and Totem can display a voltage-drop map, showing the magnitude of VDD drop or GND bounce. For substrate noise, voltage drop *per se* is rarely interesting – you want to see noise magnitude, not drop across an instance. Also, a voltage-drop map omits voltage “rises” outside the rails -- VDD bounce or GND drop – so may fail to show some noise altogether. However, as shown in Figure 13-5 the maps usually show similar patterns, although color scales differ.

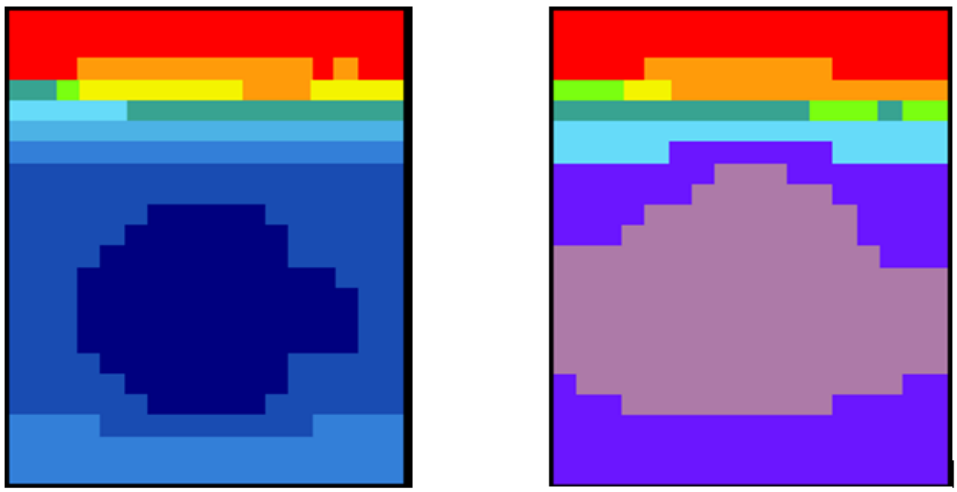


Figure 13-5 Peak-to-peak-voltage and voltage-drop maps of the top bulk layer around a rectangular guard ring. The maps show qualitatively similar noise distributions.

Current maps

Current maps let you visualize noise currents. In substrate layers, they help show the RC paths that noise follows. Use them similarly to peak-to-peak voltage maps. For example, view all shallow wells to see current-flow patterns near the surface of the substrate.

Voltage and current probes

Voltage probes capture the full-precision simulated waveform at specified locations, including in substrate layers. Use them to quantify the noise at specific locations, notably, on substrate layers in victim areas.

You can display probed waveforms in Totem's waveform viewer, in time domain or frequency domain. Since they are saved as text files, you can also post-process captured waveforms with scripts or other tools.

To capture voltage waveforms at specific locations, use `probe add` to define probes before extraction. You can probe any location on any layer, including substrate layers. After simulation, use `plot voltage -sub` to display the waveforms in the Totem GUI or write them to files.

Figure 13-6 Voltage waveforms

Figure 13-6 shows probe waveforms at varying x locations and various depths in a design. Noise drops at x locations farther from aggressors and at greater depths.

Use `plot current` on interconnect wires, where the wire's shape constrains current to flow in one dimension. Since extraction models an interconnect wire as a series of parasitic resistors, current at a point in the middle of a wire segment is well-defined.

Avoid using current waveforms from probes in substrate layers. Current in substrate structures travels in three dimensions, so current may travel horizontally, vertically, or some combination into and out of a point. Use CUR maps instead.

Frequency-domain results

As discussed above, to study noise conduction at frequencies or interest, you can inject a sinusoid, simulate, and use peak-to-peak voltage maps and probes to view the results. However, Totem also supports frequency-specific color maps to view the effect of sources with complex spectra, e.g., an active block of digital logic.

To generate and display frequency-specific color maps after dynamic analysis, use the pairs of commands `generate sfreq/show sfreq` and `generate swvf/show swvf`. These maps show full-chip results, but with limited spatial resolution.

1. To generate and display substrate-noise color maps based on magnitude response at specified frequencies, use `sfreq` commands:

   ```
   # calculate map
   generate sfreq ?-freq <Hz>? ?-freq_resolution <Hz>? ?-
   gnuplot?
   # display in GUI
   show sfreq
   # and/or write to a file (default file: image.gif)
   dump gif -map sfreq ?-o <output_file>?
   ```

   where:

   -freq <Hz>: specifies frequency of interest. *Default:* value of the GSR FREQ

   -freq_resolution <Hz>: specifies frequency resolution used for Fourier transformation

   -gnuplot: generates data for the 'gnuplot' utility in the directory *./adsRpt/Substrate/freq*

   The `generate sfreq` command transforms dynamic-analysis voltage waveforms to frequency domain, then creates color-map data for the specified frequency. Simulation time step determines the highest (Nyquist) frequency; simulation time determines the minimal frequency resolution. The first `generate` command after simulation performs the transform; subsequent `generate` commands reuse these results.

2. Use the SWVF function ("Substrate Wire and Via Frequency Domain Noise Map") to generate a frequency-specific noise map on wires and vias. By default, it displays p-well and n-well layers.

   From the GUI interface use the command **View -> Substrate Noise Maps -> Substrate Wire/Via Frequency Domain Noise Map** to generate and display the map. Alternatively, use `swvf` commands:

   ```
   # calculate map
   generate  swvf  ?-layer {<layer_name1> ...}? ?-freq <Hz>?
       ?-freq_resolution <Hz>?
   ```

```
# display in GUI
show swvf
```

where:

-freq <Hz> and -freq_resolution <Hz>: same as for `generate sfreq`

-layer <list>: interconnect or substrate layers to include in the map

Figure 13-1 shows sfreq maps for a design with noise generated by logic around a block protected by a guard ring. The map for each frequency shows the combined effect of the spectrum and location of the noise sources and the isolation structures.

To generate a spectrum of the response at a location, use the `plot voltage` as follows:

```
plot voltage -sub -freq ?-o <output_file>?
    ?-freq_resolution <Hz>?
    {<Xloc> <Yloc> ?<layerName>?}
```

where:

-sub: enables subtrate-layer plotting

-freq: plots the spectrum of a voltage waveform

-o <output_file>: specifies output file name

-freq_resolution <Hz>: specifies frequency resolution used for Fourier transformation

<layername>: specifies the layername (default BULK1)

Example:

```
plot voltage -sub {5 5} -o x5 y5_time
plot voltage -freq -sub {5 5} -o x5 y5_freq
```

To use an FFT for these operations, Totem linearly interpolates time-domain waveforms as necessary to create a power-of-two number of time/frequency points. This interpolation distorts the resulting spectrum, although its envelope is roughly correct. To avoid this limitation, add a probe, plot the voltage to a file, and use a DFT externally to show the spectrum.

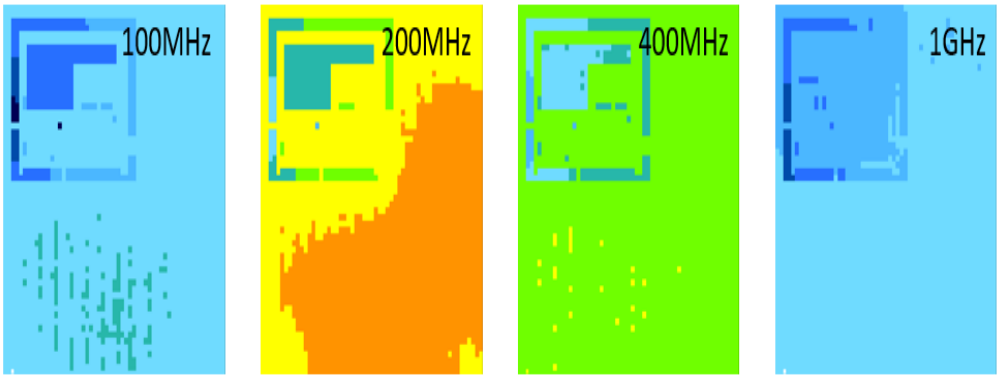


Figure 13-1 Frequency-specific maps

Substrate Back-annotation for Spice Simulation

Totem can back-annotate noise to a SPICE netlist, enabling you to externally simulate a victim circuit to assess its response to substrate noise. First, simulate noise in Totem, then use the `generate simulationdeck` command:

```
generate simulationdeck -sub ?-mmx? -aplmmx_cfg <file>
    -inst <name>
    ?-xtor_list { <xstor1> ... }?
    ?-xyscale <netlist_scale>?
```

where:

-mmx: specifies back-annotation of P/G noise to SPICE deck for a custom block

-aplmmx_cfg <file>: aplmmx configuration file for the block, used to find the SPICE netlist and device model file

-inst <name>: name of instance in netlist to back-annotate

-xtor_list: annotate only instances of the listed transistors

-xyscale: scale for $x/$y in SPICE netlist or DSPF to convert to um. For example, if the netlist is in nm units, specify '-xyscale 1000'.

Totem generates the back-annotated SPICE deck in the file *sub_noise_backannotation*

Debugging substrate noise

This section gives some ideas for debugging suspected substrate-noise problems revealed in a design.

Overall noise levels unexpectedly high/low

Check noise levels on the rails in the aggressor regions – are they in the range you expect? Substrate noise results from rail noise coupled into the substrate, so if rail noise levels are wrong, substrate noise levels will also be wrong.

Enabling substrate modeling should not cause a jump in rail noise. Therefore, start diagnosis by running a comparable non-substrate simulation:

```
# Disable CSE
SUBSTRATE_ANALYSIS 0
# CSE requires coupled simulation, so forces this setting
internally:
DYNAMIC_COUPLED_SIMULATION 1
```

If noise levels are still too high, debug the cause of excessive DVD. Once you fix it, you can resume substrate analysis.

If rail noise is reasonable without CSE, but too high with it, consult your AE. Theoretically, enabling substrate analysis could perturb chip RC to cause chip/package resonance that does not appear in non-CSE simulation, but we have not observed it in practice.

Isolated substrate hot spot

Suppose you see a hot spot surrounded by quieter (bluer-colored) regions. Noise monotonically decreases in areas of substrate not connected to noise sources –the substrate cannot focus noise to create a hot spot. Therefore, noise must reach the hot spot through the power distribution network.

To debug hot spots, look for noisy nets connected to the spot, then track backwards to understand why that net is noisy. For example, well taps in the hot spot may connect to a noisy VDD or GND net. Occasionally, you may find noisy pads on an otherwise quiet net due to package or board coupling.

Guard ring ineffective

Suppose you observe noise penetrating into p-wells that are inside a guard ring. Looking at noise maps covering the guard ring, does noise enter in a localized area? If so, do you see a gap in the guard ring, or a gap in the Metal1 ring or contacts connecting to the ring? Is there a hot spot just outside the guard ring?

Alternatively, suppose you see uniform, but unexpectedly high, noise inside the guard ring. This pattern suggests that the guard ring is not working properly, perhaps even floating. Is it connected to the correct net? Is the diffusion ring well tied to Metal1? How high is the impedance to AC ground? Run guard ring weakness checks to confirm reasonable resistance to pads. Also, check series R/L in the package.

High-frequency noise in victim region

At frequencies above 5 – 10GHz, intrinsic capacitance of silicon starts to significantly reduce the impedance of bulk layers. As a result, noise travels more easily under guard rings. In the limit, the guard ring itself becomes quieter than the areas inside and outside of it, because it is absorbing noise from both sides.

To see if intrinsic C is causing unexpected conduction, set `SUBTRATE_INTRINSIC_CAP 0` in your GSR file and rerun the analysis. If noise drops, bulk-layer conduction is causing the problem.

In a bulk CMOS process, to reduce noise conducted this way usually requires more noise absorbers in victim areas, for example, more, smaller guard rings. You may also be able to use deep N-wells to isolate pFETs, provided you can connect the N-wells to a quiet VDD. Also, grounding the back side of the chip may provide a sink for the noise.

Run spice simulation by back annotating noise voltage

After completing noise analysis in RedHawk/Totem, you can capture voltage waveforms at substrate nodes that correspond to bulk terminals of critical transistors. Also, the Tcl command `generate simulationdeck` described earlier writes a netlist that back-annotates substrate noise onto bulk terminals.

Early substrate analysis to avoid late-stage problems

Discovering excessive substrate noise in the later stages of chip assembly, and especially near tapeout, will likely mean delays and major rework. On the other hand, excessive isolation makes the die unnecessarily large or reduces space available for on-die logic, memory, and other functions. Designing and testing isolation early helps to avoid these problems.

Guard rings can reduce noise, but they require extra area and additional connections to rails. To understand these tradeoffs, you can perform what-if experiments during early layout. If you need additional guard rings, you plan them and their rail connections then.

Package impedance can significantly boost substrate noise. To check the effect of a package, add a simple R/L package model for it. Simulate to check whether guard rings and other active isolation structures perform adequately for various choices of pinout and package.

Accordingly, a prototype layout should include approximate power grids, tapped wells, and guard rings or other isolation structures, properly grounded as necessary; you can conveniently create the prototype using the Totem/Redhawk power-grid prototyping tool. You can also include whatever detailed layouts are available, e.g., layouts of IP blocks. Include estimated series R and L for the package for a more realistic picture, especially for pads used to connect isolation structures to their AC grounds. Then to analyze isolation, inject noise on power rails of aggressor blocks, and observe the induced noise in victim blocks.

Consider designing the substrate for an SoC with an embedded radio. During floorplanning, you need to place the radio appropriately and include sufficient isolation so that it will operate without substrate-noise-induced performance degradation or failure.

Using power-grid prototyping, you can create an approximate power grid, including wells and taps. You can also prototype the isolation around the radio, possibly including a layout of the radio. To simulate, inject worst-case voltage noise on the digital rails, then use color maps and probes to evaluate the results.

As your design progresses, you can replace prototype power rails and blocks with more detailed, complete versions, then re-check substrate noise, avoiding late-stage surprises. See *Proc. of DesignCon 2015*, "Substrate noise full-chip level analysis flow from early design stages till tapeout", Guterman and Toublanc for an example of this approach.

Reference A: Recipe for adding substrate analysis to your setup

This section gives step-by-step procedures for adding substrate to noise-analysis setups for a design. Enabling CSE in a setup that already has errors can cause new, mysterious errors in substrate analysis. Therefore, substrate setup proceeds more smoothly and quickly if you find and fix data and setup errors as early as possible. Following these procedures will help you avoid many common, time-consuming problems.

Steps for Totem setup with substrate analysis

This section gives a step-by-step process on how to set up Totem for substrate noise analysis and covers key issues that you need to address while creating the setup.

Task 1: Check Technology Files and Layermap

Obtain the interconnect (.tech) and substrate (.subtech) technology files for the process node you are using. The .tech file should contain all interconnect layers, including contacts, diffusion, and other layers below Metal1. The following examples show diffusion layer sections that should be present in the technology file (.tech):

```
#Diffusion sections in .tech file (same with/without CSE)
metal ND {
    Thickness <v>
    Resistance <v>
    Height <v>
    Type       N
}
metal PD {
    Thickness <v>
    Resistance <v>
    Height <v>
    Type       P
}
```

and the substrate technology file (.subtech):

```
#Diffusion sections in .subtech file, used by CSE
Pdiff PD
Ndiff ND {
   capacitance_fringe DPW <v>
   capacitance DPW <v>
   capacitance_fringe PW <v>
   capacitance PW <v>
}
```

The layer map file must contain layer numbers for all drawn substrate layers in GDS that define the features present in the substrate. Diffusions should have type `pd` or `nd`; shallow wells should have type `pw` or `nw`; and all other substrate layers should have type `substrate`. Here is a snippet of the layer map file for a hypothetical process with multiple shallow wells, one deep well, and two kinds of sinkers/trenches:

```
#LayerTypeGDStext
PW  pw  3:0 -
NW  nw  4:0 -
PW2 pw  5:0 -
NW2 nw  6:0 -
FG  substrate7:0-
NSinksubstrate8:0-
Trenchsubstrate9:0-
PD  pd  10:0-
ND  nd  11:0-
```

Task 2: Run without substrate analysis

In this step, eliminate errors and serious warnings in your setup before enabling CSE. Enabling CSE when you have basic errors in non-substrate setup can cause mysterious CSE errors or give incorrect results. Ensure that you get the expected results from non-CSE analysis.

Also, setting up the non-CSE run correctly will allow you to directly compare it to the CSE run. You can then see how noise changes when modeling the design with substrate noise included.

Here is a checklist for this run:

1. Add these keywords to your GSR file to prepare for running CSE later. Disable substrate analysis for now:

   ```
   SUBSTRATE_ANALYSIS 0
   SUBSTRATE_TECH_FILE <.subtech>
   DYNAMIC_COUPLED_SOLVER 1
   ```

You must set `DYNAMIC_COUPLED_SOLVER 1` for comparable simulations between non-CSE and CSE runs. When you enable CSE, CSE will always enable this option because correct substrate-noise analysis requires it.

2. Use an appropriate package model. For substrate analysis, package impedance is crucial in checking effectiveness of on-die isolation structures.
3. Allow sufficient presim time to settle initial transients, notably, to damp any initial ringing due to package inductance.
4. Run dynamic analysis, and check that the expected noise appears on power rails. If logic generates the noise of interest, run standard dynamic analysis. If custom circuits generate the noise, use `–inject_current` or `–inject_voltage`

initially, since they require no external simulation or FSDBs. Later, run vector-driven analysis as usual.

After the run completes, check errors and warnings reported in totem.log and fix the setup as needed:

a. Debug and fix shorts and floating/missing PG nets. If you leave these problems unattended, CSE will likely report them for substrate layers, and the debug at that stage is unnecessarily complicated.

b. Check and fix any issues related to placement of blocks, such as unexpected overlap of blocks. If block placement is incorrect, CSE is likely to report shorts due to overlapping wells connecting to different rails.

c. Check if package connections are correct.

d. Check if dynamic results are reasonable.

e. If you are using manual noise injection to test the setup, check that it creates noise as expected.

5. If you use different layer maps for non-CSE and CSE analysis, compare the two maps to ensure they only differ in substrate layers.

Task 3: Run with substrate analysis through extraction

By now, the setup should be stable, and you should be able to invoke CSE to create the substrate model. Next, we want to ensure that when CSE models the substrate, it captures all substrate layers correctly and does not show any shorts or opens.

Enable CSE using `SUBSTRATE_ANALYSIS 1`, start Totem, and stop after `perform extraction`. Then:

a. Check if SA view displays all substrate geometries for the process. Surface wells should cover the design area. Guard rings should be present wherever drawn in GDS.

b. Spot-check the VDD nets assigned to some substrate shapes, to help determine if connectivity has been traced correctly. In p-bulk processes, many GND nets typically connect to each other through bulk, so CSE does not report the connections as shorts. However, CSE chooses the GND net that it assigns to each grounded substrate shape unpredictably from the set of connected GNDs. Also, by default, CSE assigns an arbitrary VDD (or GND) net to any floating n-type (or p-type) substrate features.

c. View unconnected substrate “wires” layer by layer. In typical bulk CMOS processes, nearly all substrate shapes should be connected. However, you may see some floating shapes, for example, a PW inside a DNW, where the PW connects to a signal line instead of a rail.

   Also, some design styles allow wells to connect to each by wiring between taps, without the taps themselves connecting to a rail. Totem-CSE cannot trace such daisy-chain connections, so some of the wells involved may float. The only way to model such connections is to put PLOCs on the metal wires between the taps. Unless you need detailed substrate noise in the affected area, leave the wells floating.

d. Look for errors/warnings related to substrate shorts and well overlaps. Use the GUI to visualize overlaps (**View > Connectivity > Overlapping substrate wires**). For example, NW and PW shapes from different blocks may touch or overlap if the blocks are not placed correctly – did you miss some placement errors in Task 2? Incorrect placement is the most common cause of PW/NW overlaps.

e. If placement appears to be correct, but CSE reports NW/NW overlaps, check the substrate shapes at some of the reported locations. You may see substrate shapes touching or protruding beyond the block boundary. If the connecting NWs connect to VDDs in different domains inside and outside the block, CSE will report them as shorting. If the VDDs are in fact in the same domain, fix VDD_NETS in the GSRs to reflect that equivalence. If not, there is probably either a placement error you need to fix or a layout error (for example, if the design is not LVS-clean).

Task 4: Run dynamic analysis, including substrate analysis

Finally, run dynamic analysis. When it completes, compare results with results obtained earlier without CSE enabled. Here are a few guidelines on what to expect from this comparison:

a. Voltage drop maps should only a few black substrate shapes. Black areas have been excluded from simulation. See the discussion of SUBSTRATE_CONNECT_FLOATING_GEOMETRY for more information.

b. Compare demand currents (*.ipwr file) without and with CSE. Demand currents should be identical. To assure a match with vectorless simulation, you must use the same switching scenario; see below for more information.

c. Compare supply currents (*.ivdd file)

d. from this run with the run without CSE. There could be minor differences in these currents, for example, because the substrate capacitance estimated without CSE may differ from the capacitance associated with the detailed CSE model. Large differences indicate some discrepancy, for example, high-C deep wells or other structures ignored during non-CSE analysis.

Absent unresolved errors/warnings, assume that your setup is clean. More info on verifying substrate analysis results is provided in the final section below, “Analyzing substrate noise”.

Steps for Redhawk setup with substrate analysis

To set up substrate-noise analysis in RedHawk, follow the same general process as for Totem. However, you must first use Totem to model IP blocks or other GDS layouts that RedHawk will need. (You cannot use *gds2rh* for preparing such blocks, because it cannot process substrate data.)

We recommend this pattern:

1. Get and check substrate-aware technology file and layer map files
2. Set up the chip in RedHawk, including all GDS blocks, and run successfully without CSE
3. Setup substrate modeling for standard cells
4. Create substrate-enabled block models using Totem-CSE for memories and custom blocks
5. Run RedHawk with substrate-enabled data and options, but with CSE disabled
6. Run RedHawk with CSE enabled

Please read the preceding section on Totem setup before continuing with Redhawk setup, since the same principles apply in both cases.

Task 1: Check Technology Files and Layermap

RedHawk and Totem use the same .tech and .subtech files; refer to Totem setup above. RedHawk does not need the layermap. However, you will need it to run Totem to prepare GDS blocks for RedHawk.

Task 2: Successful non-substrate run

Eliminate errors and serious warnings in setup that can cause CSE errors or give incorrect results. Check that you get expected results from this analysis.

Here is a checklist to prepare for this run:

1. Set DYNAMIC_COUPLED_SOLVER 1 (same requirement as for Totem-CSE setup)
2. Use an appropriate package model. For substrate analysis, package impedance is crucial in checking effectiveness of on-die isolation structures.
3. Provide sufficient presim time to allow settling of initial transients.
4. Ensure that all block level power and ground net names are added to VDD_NETS and GND_NETS section of GSR file. Without these, top level power/ground net connectivity to block level nets may not be established correctly, causing false shorts between wells or other substrate shapes.

   Example:

```
VDD_NETS {
        vdd     1.1 {
                adsU1/vdd
                adsU2/vdd
                adsU3/vdd
                adsU4/vdd
        }
}
GND_NETS {
        gnd     0 {
                adsU1/gnd
                adsU2/gnd
                adsU3/gnd
                adsU4/gnd
        }
}
```

5. Set up digital noise generation, or use `–inject_voltage` or `–inject_current` to manually inject noise

After the run completes, check errors and warnings reported in the RedHawk log files and fix the setup as needed. Issues that you should address include:

1. Debug and fix shorts and floating/missing rails, if any, on PG nets, to avoid mysterious, hard-to-debug CSE shorting/floating errors later
2. Check and fix any issues related to placement of blocks, especially unexpected overlap of blocks. If block placement in RedHawk is incorrect, later CSE will report shorts due to incorrect connection or overlap of wells.
3. Check if package connections are correct.
4. Check if dynamic noise results are reasonable.
5. If design contains logic aggressors and you are using vectorless analysis mode, export the switching scenario. Import this switching scenario for substrate analysis. Then demand currents for non-CSE and CSE runs will match, allowing you to compare results.

Task 3: Model standard cells for substrate analysis

Set up standard-cell wells as explained in section "Standard-cell GSR keywords and configuration files", page 13-438

RedHawk requires that you run APL to characterize cell currents. Normally this characterization ignores bulk-pin currents. You may use an existing characterization: omitting bulk-pin currents will not significantly affect results.

While you can re-characterize standard cells with APL to include bulk-pin currents, these currents are small and are captured by the wells under the standard cells. The great majority of standard-cell noise comes from source/drain “demand” currents causing voltage-drop noise, which couples to wells through rail taps.

Task 4: Create models for custom blocks

Model memories for substrate analysis

For substrate analysis, create ERV models for memory IPs and use these in RedHawk. When Totem-CSE generates an ERV model, it retains tapped wells and well shapes. It also provides connectivity from rails to wells inside the IP, allowing the IP to inject noise from its rails.

For each master cell of a placed memory instance, create an ERV model using Totem-CSE. Include the resulting ERV models in RedHawk analyses using the `ERV_CELLS`.

Model other custom blocks for substrate analysis

For GDS blocks in the design, export DEF descriptions containing substrate shapes from Totem-CSE. Include these files in RedHawk analyses using the `GDS_CELLS`.

Ensure that isolation structures associated with GDS blocks are available in RedHawk for analysis. Many analog blocks include their own guard rings or other isolation structures. Totem-CSE will include these in the exported model.

Sometimes isolation structures appear in separate GDS layouts. For example, a cover macro for the top-level LEF/DEF for a chip might include them. Model the cover macro using Totem-CSE, then include it in your RedHawk analyses.

Full-chip GDS overlay for substrate analysis

An alternative way to supply substrate geometry is to get it from a full-chip GDS. This approach has the virtue of simplicity – you can avoid individually modeling custom blocks – but it creates risks of mismatched data, untapped wells, and much larger runtime and memory.

Therefore, we recommend using full-chip GDS overlay method only when necessary. For example, a design may use special well layouts under standard cells that significantly reduce substrate noise from logic. If CSE cannot automatically derive a sufficiently good approximate layout for them, use the exact well structure from GDS.

For more details, see SUBSTRATE_GDS_OVERLAY**.**

Final tasks: Run Redhawk without and with CSE

At this stage, you have a clean RedHawk setup without substrate analysis and have created models for blocks present in design. Update the setup with the block models and run RedHawk without enabling substrate analysis. Check that the run completes cleanly with expected results.

Next, enable substrate analysis (`SUBSTRATE_ANALYSIS 1`) and run, stopping after performing extraction. Check that when CSE models the substrate, it captures all substrate layers correctly and does not show any shorts or opens. Use the same approach as for Totem-CSE:

1. Check that the SA view displays all substrate geometry. Surface wells should cover the design area; guard rings should be present where expected, and connected to rails.
2. Spot-check VDD nets assigned to substrate shapes to see if the connectivity has been traced correctly. See Totem setup regarding checking GND nets and floating substrate features. Check if rails connect correctly.
3. Look for errors/warnings related to shorts, especially well overlaps.
4. Look for placement errors if any; check if there are unintended substrate shapes protruding from blocks.

After clearing these checks, run dynamic analysis. On completion of analysis, compare results with results obtained earlier without CSE setup:

a. Voltage drop maps should not have black areas on substrate shapes
b. Compare demand currents (*.ipwr file) between the runs. If they are not identical, check that you are using the same switching scenario for both runs.
c. Compare supply currents (*.ivdd file)
d. from this run with the run without CSE. There could be minor differences in these currents, as explained previously.

Absent unresolved errors/warnings, assume that your setup is clean and that the CSE results are meaningful.

Reference B: Describing substrates for CSE

Canyon Substrate Extension (CSE) creates an electrical model of the silicon substrate for substrate noise analysis. This reference section describes CSE's substrate modeling and the data it needs to extract the electrical model.

Overview of substrate model

CSE can model substrates of both logic and specialized semiconductor fabrication processes. CSE's substrate model has proven general enough to analyze nearly every type of IC substrate including:

a. Triple-well substrates used in CMOS logic and mixed signal designs
b. Substrates used for CMOS imagers
c. SOI and FDSOI substrates for logic and mixed-signal designs
d. n-EPI/p-bulk and SOI high-voltage substrates

The generic term “substrate feature” refers to regions in substrate that have unique electrical characteristics and specific physical dimensions. CSE directly models most substrate features: bulk layers, including buried oxide and EPI; diffusion wires and taps; shallow surface wells; deep wells; and diffused sinkers and trenches. It also supports Through-Silicon Vias (TSVs), back-side metal, and other complex features, using combinations of directly-supported features.

Examples of substrate cross-sections

The figures in this reference usually show cross-sections of substrate, i.e., 2D vertical slices through substrate. A 2D cross-section can show the location and depth of a feature and how it interacts with other features. These cross-sections also show material type

and relative doping. Blue indicates p-type material; yellow, n-type; and black, insulating material. Darker shades indicate heavier doping compared to lighter shades.

Bulk CMOS substrate

Most logic and mixed signal designs use a bulk CMOS process. It is fabricated on a uniform p- bulk wafer. It has a deep n-well (DNW) and 3 shallow wells: n-well (NW), p-well (PW), and non-well (nonW). nMOS transistors lie inside surrounding PWs; pMOS transistors, in NWs. In the cross-section shown in Figure 13-1, the PW lying over DNW is DC-isolated from p-bulk, allowing nMOS transistors whose bulk terminals do not connect to wafer p-bulk.

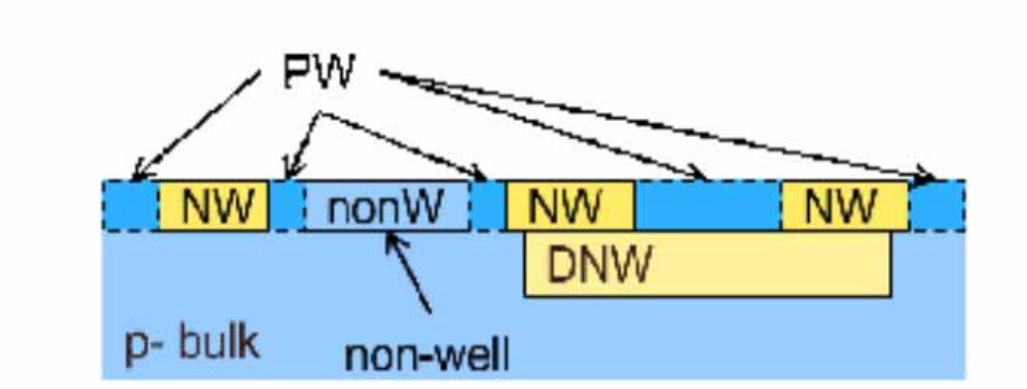


Figure 13-1 Cross-section of a bulk CMOS substrate

The non-well feature is an un-doped surface region. Inductor devices lie over non-well regions to reduce eddy currents that lower their Q. Some processes support nMOS devices with high-resistance bodies in non-well. Non-well rings can also act as isolation structures for substrate noise: Because their doping is similar to bulk, they have high resistance, so they help to break low-impedance NW/PW paths conducting substrate noise.

BCD SOI substrate

Automotive/high-voltage designs requiring highest isolation use a BCD (bipolar/CMOS/DMOS) process on an SOI (silicon-on-insulator) substrate. The blank wafer is multi-layered, with two n-type EPI layers on top of buried oxide (BOX), on top of a p-bulk wafer handle. The deeper EPI layer has heavier doping than surface EPI (a “retrograde” doping profile).

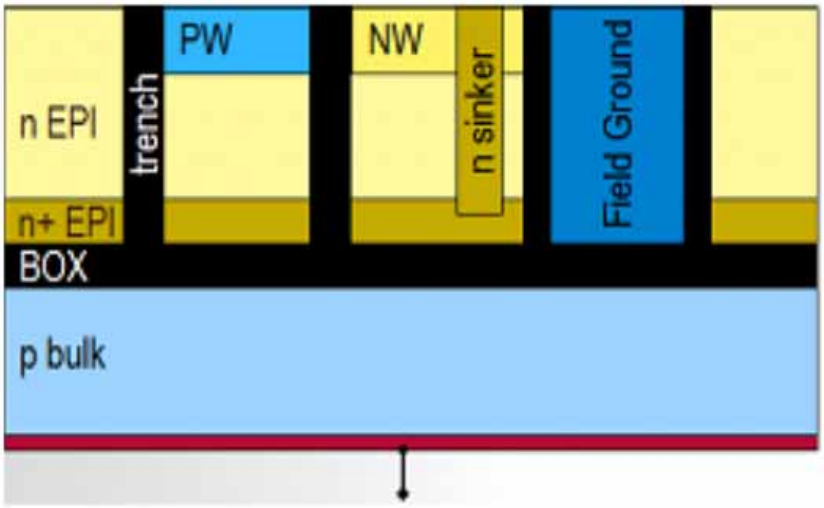


Figure 13-2 Cross-section of BCD SOI substrate

The trench features shown in Figure 13-2 are etched holes or slots filled with oxide. The combination of the buried oxide layer and oxide-filled trenches between the top surface of the substrate and buried oxide can split the EPI to form DC-isolated pockets. Typically a pocket contains one type of MOS device, e.g., nMOS devices formed in a PW shallow well, or other kinds of devices, such as LDMOS transistors. The process shown also supports field grounds, heavily-doped pockets that act as AC shields between pockets containing devices.

This process also provides n-type sinkers, highly-doped implants that create low-resistance connections from the top surface of substrate to the buried, more heavily doped n+ EPI layer. Optional back-side metal shown by a thin red strip above may be used to ground the bulk.

BCD EPI process substrate

Automotive/high-voltage designs that do not require the extreme isolation provided by SOI may use a BCD process built on an n-EPI/p-bulk substrate, as shown in Figure 13-3. The blank wafer is multi-layered, with two n-type EPI layers on top of a p-bulk wafer handle. As with a BCD SOI process, the deeper EPI layer has heavier doping than surface EPI, and n-type sinkers allow low-resistance connections from the top surface of substrate to the buried, more heavily doped n+ EPI layer.

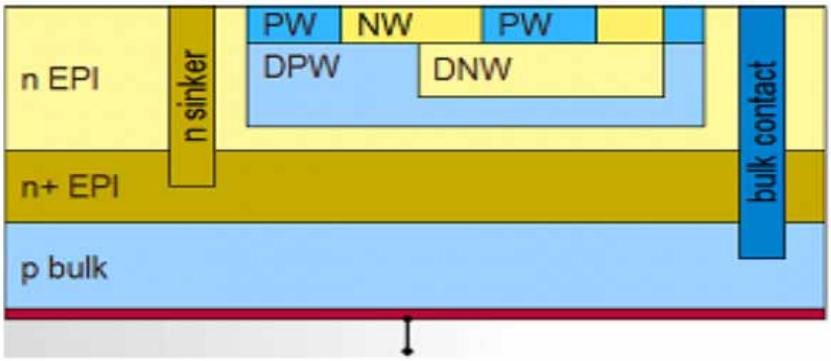


Figure 13-3 Cross-section of BCD EPI substrate

Bulk contacts provide connections from the substrate surface to bulk, and also can form walls to strongly isolate EPI regions. Physically, bulk contacts typically have oxide sidewalls and a polysilicon core.

The process shown supports triple-well CMOS blocks inside of deep p-well (DPW) regions. The DPW plays the role of p-bulk in a bulk CMOS process. It encloses all the wells typical of bulk CMOS, including deep n-well (DNW).

Shallow wells also exist in n-EPI regions, for high-voltage devices. An optional back-side metal shown by a thin red strip above can be used to ground the bulk.

Substrate Model

CSE uses physical and electrical characteristics of substrate features to create a substrate model. These characteristics include the depth and electrical type (p, n, or insulating), resistivity and junction capacitance for conducting features, and dielectric constant for insulating features.

Substrate features directly modeled by CSE

Bulk

Bulk represents the blank/empty portion of a wafer where all other features are formed. Therefore, the layout does not contain drawing shapes for bulk layers. All bulk layers exist everywhere in the design, except where overlaid by other substrate features.

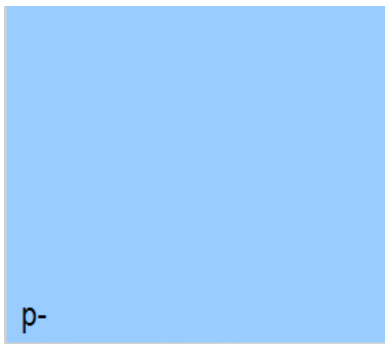


Figure 13-4 Cross-section showing single layer bulk

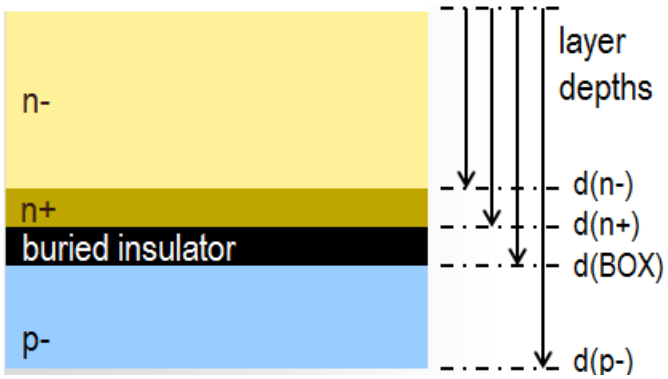


Figure 13-5 Cross-section showing multi-layer bulk

Figure 13-4 illustrates bulk with uniform p-type material. CMOS logic processes use this type of bulk. Figure 13-5 illustrates the multi-layer bulk from the SOI high-voltage described previously. The bottom of each sub-layer lies at depth d() from the surface of wafer. Therefore, the buried insulator (BOX) layer starts at depth d(n+), the depth of the bottom of the n+ EPI above it, and ends at depth d(BOX). Its thickness is the difference of these two depths, d(BOX) - d(n+).

Shallow wells

Shallow wells (or surface wells) lie at the surface of the substrate. They may contain diffusions, but they do not contain or overlap other shallow wells. Their depths are less than those of deep wells; their doping concentrations are usually higher than those of deep wells and bulk.

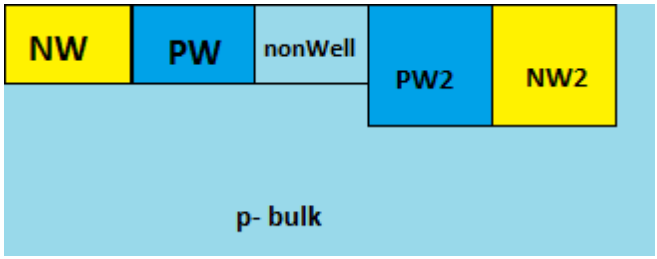


Figure 13-6 Cross-section showing shallow wells in bulk

Figure 13-6 shows a cross-section for a process with two different n-wells and two different p-wells. In this case, NW2 and PW2 are deeper than NW and PW. The figure also shows a non-well region, modeled in CSE by its own p-type shallow well.

Deep wells

Deep wells extend deeper than all shallow wells. They may contain or overlap shallow wells and other deep wells. Deep wells generally have lighter doping than shallow wells, except for non-well shallow wells.

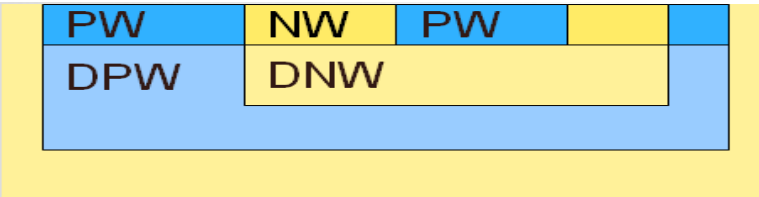


Figure 13-7 Cross-section showing deep well features

Figure 13-7 shows a portion of substrate's cross-section containing deep wells and shallow wells in n-type surrounding material, e.g., n EPI. The deep p-well (DPW) is deepest. DPW contains a deep n-well (DNW) region, isolated from n-type surrounding material by the DPW. Shallow n-wells (NW) and p-wells (PW) lie on top of DPW or DNW.

Sinkers

CSE models implanted sinkers, deep trenches, and substrate contacts as conductive or insulating sinkers, depending on the material forming them. Sinkers may overlay shallow or deep wells, but sinkers may not overlap other sinkers.

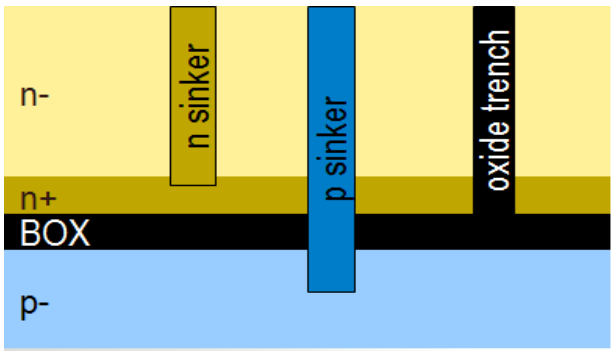


Figure 13-8 Cross-section showing sinkers and trenches

If a trench is filled with insulating material (typically silicon dioxide), it DC-isolates conductors that it splits. CSE models insulating trenches as insulating sinkers and models the capacitance coupling between conducting features surrounding the insulator.

Implanted sinkers and substrate contacts provide low-resistance connections between the substrate surface and features deeper in the substrate. CSE models them as conductive sinkers. It models their resistance using the resistivity of the conducting material composing sinker. It also models capacitance to surrounding structures with the opposite doping type.

Figure 13-8 shows three kinds of sinkers:

- an implanted n-type sinker connecting the top of the substrate to surrounding n- EPI and buried n+ EPI
- a substrate contact modeled as a p-type sinker connecting the top of the substrate to p- bulk below BOX, the buried oxide layer; and
- an oxide-filled deep trench modeled as an insulating sinker between top surface and BOX.

Diffusion wires/taps

Diffusion wires and taps lie in shallow wells. Their doping concentrations are higher than the wells that contain them. Diffusions may lie over any substrate feature, including deep wells and sinkers, but they may not overlap opposite-type diffusions.



Figure 13-9 Cross-section showing diffusion features in shallow wells

In Figure 13-9, PD represents p-type diffusion; ND represents n-type diffusion. The left PD taps the PW under it; the right ND taps the NW under it. The middle ND and PD form diffusion wires (active area) in their respective wells. CSE models the resistance of well taps, i.e., between a diffusion and a well of the same type. It models capacitance for the active area, i.e., for the diode that forms between a diffusion and an opposite-type well.

Overlaying features to create a cross-section

CSE overlays the features described in the previous section to create a 3D model of the substrate for a layout. To construct this model, CSE overlays features in an idealized fabrication order: bulk is fabricated first, then deep wells, shallow wells, and finally, sinkers.

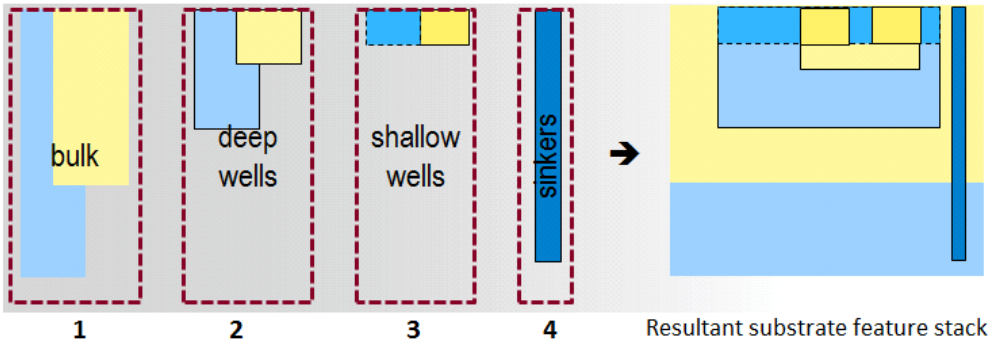


Figure 13-10 CSE sequence of feature overlay

Figure 13-10 illustrates the sequence of overlays used to form the overall model:

1. Stack shallower bulk layers on top of deep ones to form multilayer bulk
2. Stack deep wells on the multilayer bulk, deepest first
3. Stack shallow wells on the bulk + deep-well model
4. Overlay sinkers from top surface of substrate to their designated depths in substrate.

This sequence gives the cross-section shown in the figure.

When stacking features, CSE checks for prohibited overlap of features, such as:

- Shallow wells may not overlap other shallow wells
- Sinkers may not overlap other sinkers

Describing implicit (undrawn) shallow wells

Some shallow wells do not appear as drawn shapes in layouts. For example, in CMOS triple-well processes, the layout typically includes shapes for n-wells and non-wells, but

no shapes for p-wells. By convention, p-well covers the surface of the substrate except where n-well or non-well shapes exist. Figure 11 illustrates this convention for drawing shallow wells.

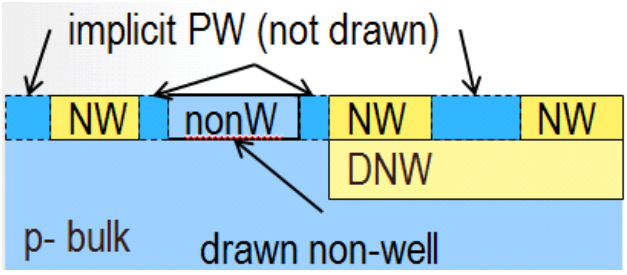


Figure 13-11 Cross-section of CMOS triple well process with implicit PWs

The rules for determining implicit surface wells may depend on context. Areas without deep wells may have one type of implicit shallow well (or no shallow well at all). Each type of deep-well area may have its own implicit shallow well. Figure 13-11 shows an example in which areas over DPW or DNW have implicit PW shallow wells, but areas without DPW or DNW have no implicit surface well, i.e., lightly doped n EPI starts at the surface of the substrate.

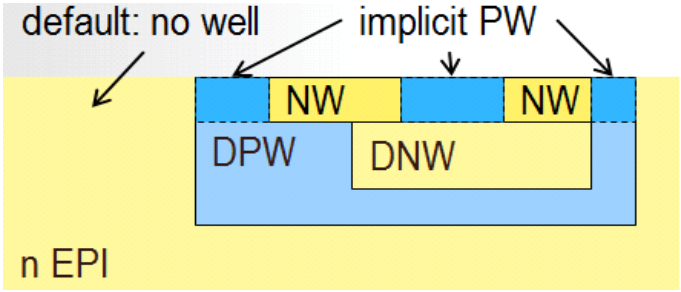


Figure 13-12 Cross-section of n-EPI process with varying implicit shallow wells

CSE derives implicit shallow wells based on information in the substrate-technology file for the process. For example, for the bulk CMOS cross-section shown in Figure 13-12, CSE will derive PW = !(NW | nonW), that is, it will create PW wherever there are no NW nor nonW shapes.

Electrical model of substrate

CSE models substrate as a 3D linear resistance-capacitance mesh. It models resistance based on resistivity and physical dimensions of each conductor and between conductors of the same doping type.

CSE models capacitance for these two scenarios:

- Where conductors with opposite doping types touch: CSE includes the p-n junction (diode) capacitance as a linear capacitor. The substrate-technology file contains a single value for each such capacitance, based on the reverse bias normally applied across the junction.

  Junctions may exist along bottom-walls (area) and side-walls (fringe) of interacting features. For example, a PW inside a DNW will typically have bottom-wall capacitance

to DNW and side-wall capacitance to NW.

- Between conductors on opposite sides of an insulating bulk layer or an oxide sinker: CSE calculates this linear capacitance based on the insulator's dielectric and thickness in the direction of interest. For example, it models the vertical capacitance between EPI and bulk across the buried oxide in an SOI process; and it models the lateral capacitance between conductors on either side of an oxide-filled trench.

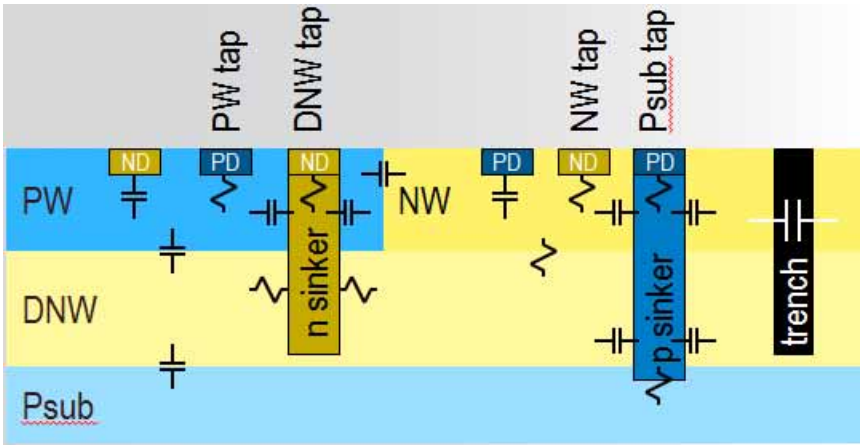


Figure 13-13 Cross-section showing Rs and Cs for a variety of situations

Figure 13-13 summarizes the various Rs and Cs required to model a particular substrate cross-section. For example, for the n-type sinker shown, CSE creates resistors internal to the sinker and connecting it to the ND and DNW touching it, and capacitors for the side-wall diodes between the sinker and PW. For the trench, it creates capacitors connecting the PW and DNW regions sandwiching it.

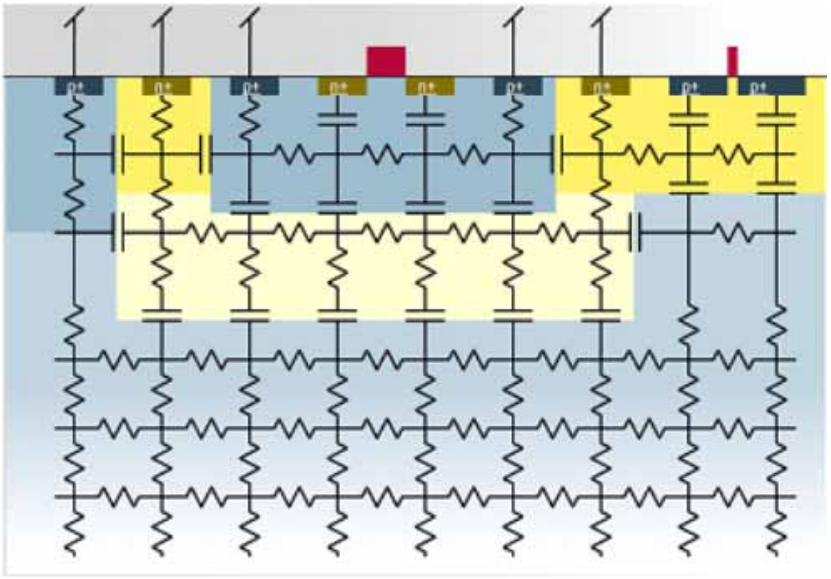


Figure 13-14 2D cross-section with associated RC mesh

Figure 13-14 illustrates the R/C mesh created for a 2D cross-section of a MOS triple-well process substrate. The well taps shown connect to power and ground nets. Of course, the complete mesh created for a layout by CSE is 3D mesh.

Substrate features not directly modeled by CSE

Not every substrate feature maps directly into a feature that CSE natively supports. However, CSE can usually model other features using its native features. Non-native features that can be modeled using this approach include back-side connections, insulated sinkers such as substrate contacts, and through-silicon vias (TSVs).

Backside connections

Ohmic backside connections are typically made using metal layers plated onto the back of the wafer. This metal can then be connected though the die landing on the package. Backside grounds can absorb substrate noise that could otherwise couple over long distances within bulk. Also, specialized processes support vertical MOS transistors, whose drain terminals connect through bulk to the back-side of the die.

Backside metal can be patterned or un-patterned. CSE can model both of these cases.

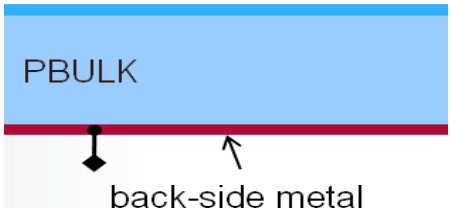


Figure 13-15 Cross-section of unpatterned backside metal below p bulk

Figure 13-15 shows unpatterned backside metal on the bottom of p-type bulk. CSE models this metal as a “fake” p-type bulk layer with the metal's resistivity and thickness. Use PLOCs on this fake bulk layer to connect to it.

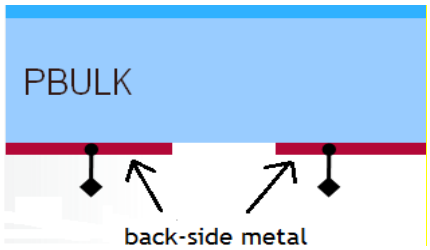


Figure 13-16 : Cross-section patterned backside metal below p bulk

Figure 13-16 shows patterned back-side metal on the bottom of p-type bulk. CSE cannot model the patterning directly. However, you can use a separate array of PLOCS connected to the p-bulk layer to model each connected back-side area.

Bulk contacts

Some fabrication processes create bulk contacts or other insulated sinkers from conducting material clad with an insulating material. This conducting material provides a resistive vertical connection, while the insulator prevents a DC connection to features that the contact passes through.

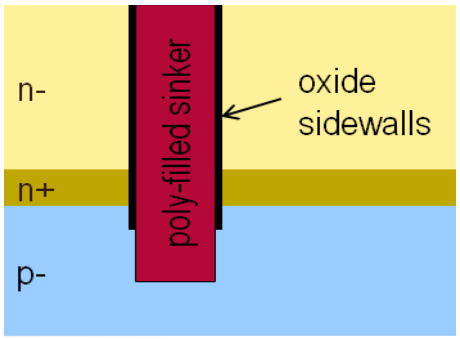


Figure 13-17 Cross-section showing poly-filled oxide clad sinker

Figure 13-17 shows the physical structure of a poly-filled substrate contact. Its oxide cladding insulates it from the n-type layers it passes though, so it has only capacitive coupling to them. At the bottom of the sinker, its poly core touches p-bulk, connecting with it.

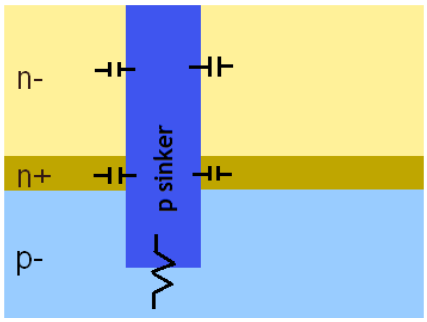


Figure 13-18 Cross-section with CSE model of poly-filled oxide clad sinker

For this specific case, CSE can model the substrate contact as a p-type sinker, as shown in Figure 13-18. Use the resistivity of the poly core for the sinker, and use the actual capacitance between the core and surrounding features as the sidewall capacitance between the sinker and the n-type layers around it.

Obviously, this simple trick only works for a substrate contact that passes through only n-type material before connecting to a p-type buried layer, or vice versa. In the general case, CSE requires a conducting sinker of the proper type surrounded by a wall of insulating sinker. Fortunately, substrate contacts usually pass though material of only one doping type.

Back-gate pins and deep-trench decaps in SOI logic processes

SOI logic processes have no wells. Instead, transistors form in a surface layer of epitaxial silicon on top of buried oxide. Shallow-trench isolation (STI) forms between diffusion wires, completely DC-isolating transistors from each other, as Figure 13-19 shows. However, diffusion wires can capacitively couple to bulk beneath the buried oxide, and CSE models this coupling without any special setup.

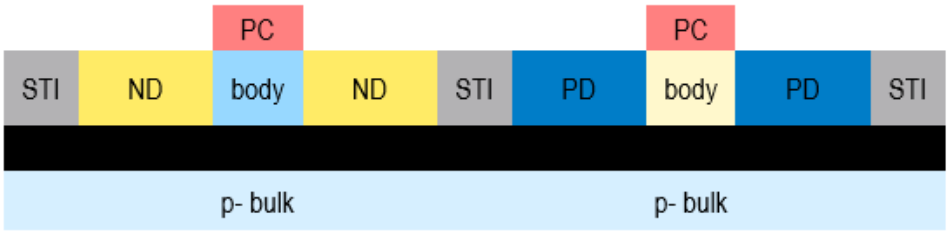


Figure 13-19 Cross-sections of nFET and pFET in SOI logic process

Some SOI processes support connections to bulk terminals of selected FETs, to provide better characteristics needed for analog circuits. CSE does not model these back-gate contacts directly. However, by defining virtual shallow wells, you can model them.

Some SOI processes support capacitors formed in deep trenches that drop through the buried oxide into the bulk (see Figure 13-20). If deep-trench caps are used as decaps, they directly couple rail noise into the bulk. You can model these capacitors using deep wells and sinkers.

Contact R&D if you need to configure CSE for an SOI process with capabilities like these.

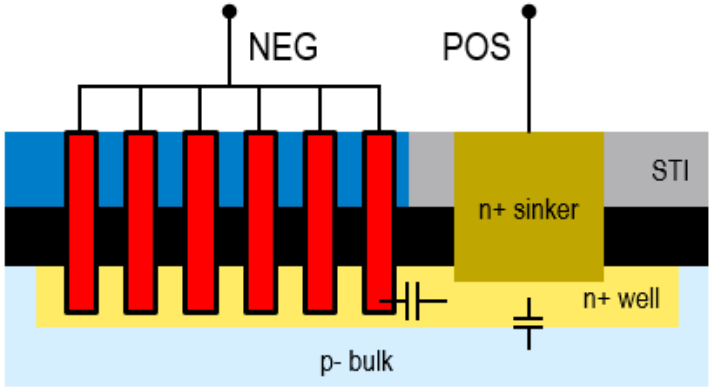


Figure 13-20 Cross-section of deep-trench capacitor array

Through-Silicon Vias (TSVs)

TSVs connect lowest metal on the front side of a die with bumps or metal routes on the backside of the die. For example, CMOS backside-imagers (BSI sensors) may use TSVs to route power to lens/shutter modules attached to the back of the imager.

Figure 13-21 below shows a TSV for a bulk CMOS process. It consists of an oxide-clad conductive metal core, reminiscent of a substrate contact. However, the TSV extends through to the bottom of the bulk. To avoid damaging devices, each TSV must be placed in an empty area of the die, so it passes through only p-type material.

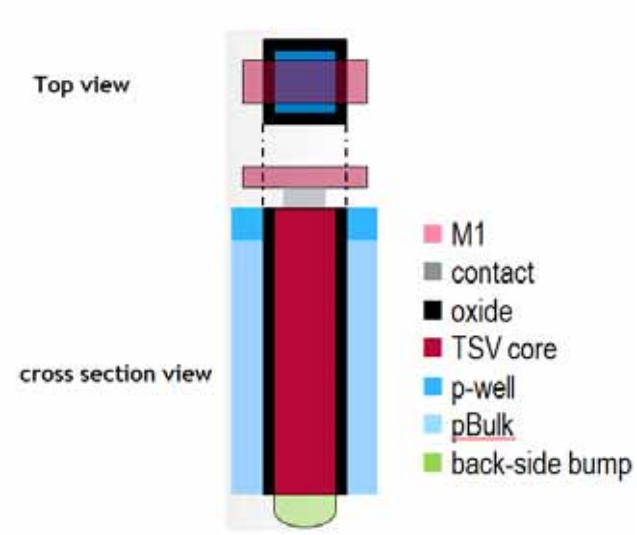


Figure 13-21 Top view and cross-section of a TSV

CSE can model this type of TSV as an n-type sinker. For details, see the preceding discussion of body contacts.

Note: Totem and Redhawk cannot use this CSE TSV model in combination with backside metal routing for 3D ICs. You must model substrate noise coupled from TSVs without the actual backside metal routing.

Substrate technology file (.subtech)

CSE requires a technology file, an ASCII file that provides physical and electrical characteristics of substrate features based on foundry data. This section provides: an overview of the contents of substrate technology files; examples of substrate tech files for a few processes; a description of all keywords in substrate tech files; and instructions on viewing the generic substrate cross-section using Totem.

Overview of contents

Foundry data captured in the substrate technology file can be broadly categorized into physical and electrical data, as described in the following paragraphs.

Physical data

The *.subtech* file specifies the depth of each substrate feature. Measure depth from the top surface of the substrate to the bottom of a feature; refer to Figure 13-5 for an illustration. Foundries can estimate feature depth from fabrication targets and other information, e.g., implant depths, doping profiles, and wafer parameters.

CSE gives accurate results even with an idealized depth profile. Actual well doping varies with depth. Also, wells over the same-type material, e.g., PW over bulk, have no well-defined bottom. However, a simplified model assuming a single, idealized well depth suffices for CSE analyses, provided that the combination of depth and resistivity implies the correct sheet rho value for the feature.

Electrical data

Electrical data includes the inputs needed to estimate resistance and capacitance associated with a feature.

CSE calculates feature resistance based on its resistivity. The substrate tech file includes resistivity for all conductors. The SPICE library for a process often contains sheet rho for wells, which you can use to determine resistivity; see the “Creating a substrate technology file” section below.

As described above, CSE models the capacitance of p/n junctions. Also, it models capacitive coupling across insulating substrate features. The substrate technology file provides data for extracting these capacitance values:

Capacitance type	Substrate tech file data for feature
Fringe/side-wall capacitance	capacitance per unit length between interacting features
Area/bottom-wall capacitance	capacitance per unit area between interacting features
Across insulator	Dielectric constant of insulator material

If only one of side-wall or bottom-wall capacitance is provided, CSE determines the other capacitance using $C_{missing}$ per unit area = $C_{specified}$ per unit area. More info on sources for this data is available in the section “Creating substrate technology file”. The SPICE library for a process usually contains parasitic substrate diodes, which you can use to determine capacitance; see “Creating a substrate technology file” below.

Substrate technology file keywords

Subtech format supports three classes of keywords: feature types, physical properties, and rules. This section summarizes each class.

Feature types

Types describe the type of material and the depth “group” of features. Table 1 shows all available types, by depth group.

Types	Description
Pdiff, Ndiff	p- or n-type diffusions; exactly one of each required
Pwell, Nwell	p- or n-type shallow wells; at least one required; may not overlap
DeepPwell, DeepNwell	p- or n-type deep wells; optional; multiple allowed; may nest or overlap if depths differ
Psinker, Nsinker, INsinker	p-type, n-type, or insulating sinker; multiple allowed; may not overlap
Psub, Nsub, INsub	p-type, n-type, or insulating substrate (bulk) layer; at least one required; never drawn; depths must differ

All types support the same syntax:

```
<type> <layername> {
…<properties>…
}
```

The <layername> should match the corresponding name in the Totem layermap for all drawn substrate features. Psub/Nsub/INsub layers must not appear in the layermap, since bulk layers are never drawn. However, they appear in the Totem GUI, and you may use them elsewhere, for example, to specify what layers to display in a substrate-noise color map of a substrate layer for a pad location (PLOC) or probe.

The Pdiff layer should not have properties; it inherits them from the interconnect technology file (.tech). Similarly, Ndiff inherits depth and resistance, so it should only have capacitance properties to Pwell features.

Physical Properties

Physical properties define the depth and electrical characteristics of features. Table 2 shows all available properties.

Table 13-1 Substrate feature properties

Property	Unit	Description
depth	µm	depth of bottom of feature, measured from substrate surface
resistivity	Ohm-cm	resistivity of a p- or n-type feature
resize	µm	amount ≥0 to increase x/y dimensions of feature
capacitance	pF/ µm^2	areal capacitance of a p/n interface; specify for n-type features
capacitance_fringe	pF/ µm	fringe capacitance of a p/n interface; specify for n-type features
dielc	none	relative dielectric constant of an insulating feature

All properties support the same basic syntax:

```
<property> <number>
```

Additionally, depth and resistivity support sublayers with different resistivity within a feature:

```
depth <d1> <d2>…
resistivity <r1> <r2>…
```

Sublayers must have increasing depths and corresponding resistitivities. For correct modeling, you must also set SUBSTRATE_WELL_PORTION; as described in the next section. Generally, defining each feature as a single, “average” layer gives sufficient accuracy.

Surface-fill rules

CSE supports one rule declaration, as described in the table below:

Rule	Description
SURFACE_FILL_LAYER <shallow well>	fill substrate surface with <shallow well> wherever no drawn shallow well exists

If SURFACE_FILL_LAYER appears outside of a deep well declaration, it applies globally to all substrate surfaces not covered by any drawn shallow well. If it appears inside deep-well or bulk declarations, but not globally, it applies only when those features are present.

As discussed previously, in typical triple-well bulk CMOS, you draw only NW and nonW shapes; PW covers everywhere else. To describe this implicit PW fill, include a global SURFACE_FILL_LAYER PW declaration in *.subtech* such as:

```
# Triple-well CMOS substrate
#  PW default everywhere
SURFACE_FILL_LAYER PW

# Diffusion names/capacitance
Pdiff PD
Ndiff ND {
…
}
…
```

Note that a layout may include explicitly drawn shapes for the shallow well used as surface fill. For example, if a triple-well process has both of the rules just described, you must draw NW shapes over non-DNW and PW shapes over DNW.

Alternatively, instead of a single, global surface-fill rule, you can declare separate surface-fill rules for each deep well and for the shallowest bulk layer. Such a rule applies wherever its feature is present (which means everywhere for a rule associated with the shallowest bulk layer). CSE gives priority to the rule for the shallowest feature present, that is, the feature that would exist at the substrate surface if no shallow wells or sinkers were

present. Looked at another way, CSE overlays surface-fill rules in the same order it overlays substrate features.

Alternatively, instead of a single, global surface-fill rule, you can declare separate surface-fill rules that depend on what deep wells are present in an area. You can optionally define a surface-fill rule for each deep well and for the shallowest bulk layer. CSE gives priority to the rule for the shallowest deep well present in each area--that is, the deep well or substrate layer that would exist at the substrate surface if no shallow wells or sinkers were present. Figure 13-22 illustrates this priority order. Note that DNW, DPW, and nEPI may have different implicit surface wells.

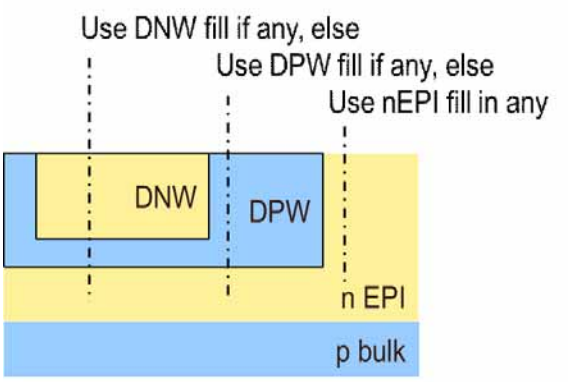


Figure 13-22 Prioritized surface-fill rules in a complex process.

Consider the n-EPI/p-bulk process illustrated in Figure 13-23, in which DNW always nests inside of DPW, DPW has implicit PW fill, and there is no implicit shallow well outside of DPW.

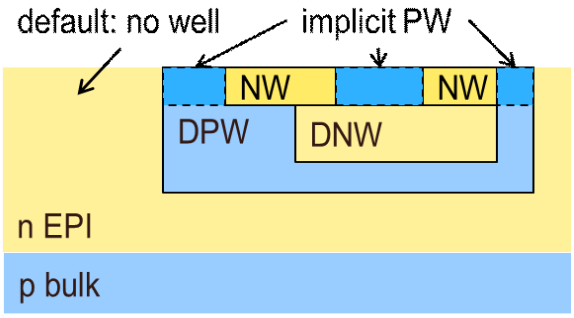


Figure 13-23 Example process with implicit PW only in DNW areas

To specify these fill rules, declare a fill only for DPW. as follows:

```
DeepPwell DPW {
SURFACE_FILL_LAYER PW
...
}
```

Do not declare a global PW fill, since nEPI areas have no implicit fill. DNW does not need its own fill rule, since it and DPW have the same implicit fill.

Finally, to illustrate the general case, consider a hypothetical n-EPI/p-bulk process shown in Figure 13-24 with these complex fill rules: DNW has implicit PW fill, DPW has no implicit fill, and n-EPI areas outside of DNW/DPW have implicit HVPW fill.

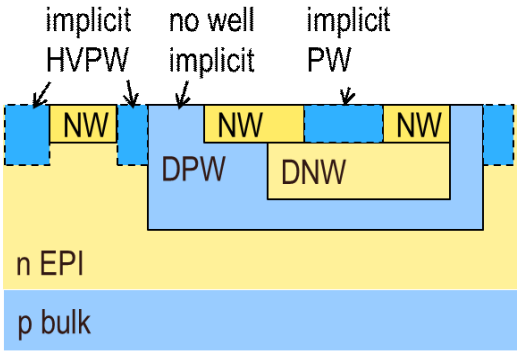


Figure 13-24 Example process with complex rules for implicit wells

To specify these fill rules, declare fill rules for nEPI, DPW and DNW as follows:

```
Pwell PW {
...
}
Pwell HVPW {
...
}
# native "fill" in DPW, same rho as DPW
Pwell nonPW_in_DPW {
...
}
...
DeepNwell DNW {
SURFACE_FILL_LAYER PW
...
}
DeepPwell DPW {
SURFACE_FILL_LAYER nonPW_in_DPW
...
}
Nbulk nEPI {
SURFACE_FILL_LAYER HVPW
...
}
...
```

You cannot specify “no fill” for a deep well that lies in the context of a feature with fill, e.g., DNW over nEPI in this example. Instead, use a non-well with the same properties as the unfilled deep well to fill the deep well, the way that nonPW_in_DPW fills “empty” DPW areas in this case.

Examples of substrate technology files

To illustrate *.subtech* format, this section shows *.subtech* content for three substrate variants. See “Substrate technology file keywords” for a detailed description of *.subtech* keywords and values.

CMOS triple-well .subtech

Figure 13-25 shows a typical triple-well cross-section. It includes five substrate features: PW, NW, and nonW surface wells; a DNW deep well; and p- bulk. The .subtech file must include a depth and a resistivity for each feature.

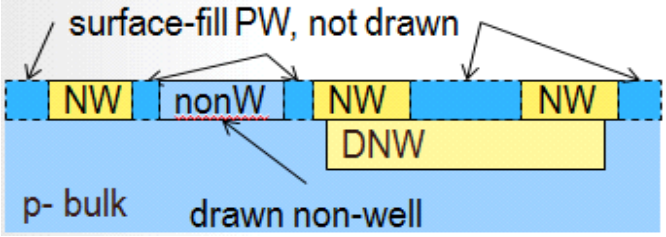


Figure 13-25 Cross-section of a CMOS triple-well substrate

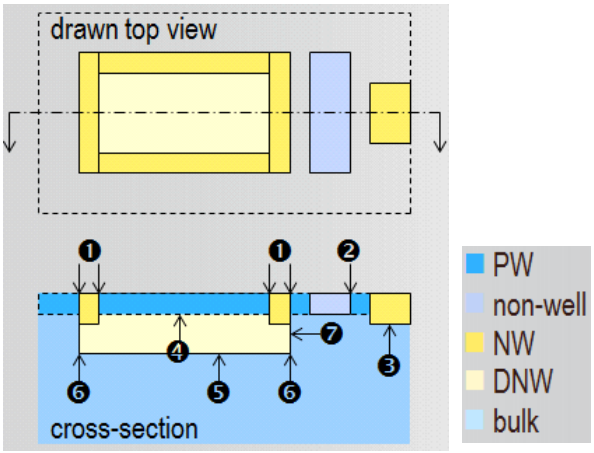


Figure 13-26 Top and cross-section views--all possible p/n junctions

Figure 13-26 shows all p/n junctions that can exist in a layout for this process. The *.subtech* file must include capacitance values for each of these seven junctions, as part of the description of the n-type feature creating the junction.

Note that Figure 13-26 does not show PW/DWN, NONWELL/NW, or NONWELL/DNW junctions. Typical triple-well design rules prohibit layouts that would create these junctions. If CSE finds such a junction, but *.subtech* does not provide information to model it, CSE reports an error.

The substrate-technology file for this substrate typically has this general form:

```
################################
## Triple-well CMOS .subtech ##
################################
SURFACE_FILL_LAYER PW
Pdiff PD
```

```
Ndiff ND {
capacitance_fringe PW <v>
capacitance PW <v>
}
Nwell NW {
depth <d>
resistivity <v>
capacitance PD <v>
capacitance_fringe PW <v>
capacitance PBULK <v>
}
Pwell PW {
depth <d>
resistivity <v>
}
Pwell NONWELL {
depth <d>
resistivity <v>
}
DeepNwell DNW {
depth <d>
resistivity <v>
capacitance PW <v>
capacitance PBULK <v>
}
Psub PBULK {
depth <d>
resistivity <v>
}
```

Per-feature information. Each bracketed block of data describes one substrate feature--for example, an Nwell feature named NW. In a block, <d> is the depth of the feature in um. Each n-type feature includes a capacitance value for each p/n junction that it can form, e.g., the NW block lists fringe capacitance for an adjacent PW and areal capacitance for PD touching above it, and PBULK touching below it. Each <v> is a fringe (sidewall) capacitance in pF/um or a bottom wall (areal) capacitance in pF/um2.

Implicit surface wells. SURFACE_FILL_LAYER PW means that PW exists everywhere except where the layout includes explicit drawn surface wells, either NW or NONWELL in this case, as shown in Figure 13-25.

NONWELL surface well. NONWELL, or native, areas occur where no wells were implanted during fabrication. As a result, the surface properties in NONWELL areas generally match the properties of the undoped, bulk wafer: p-type, lightly doped. Therefore, to model NONWELL, declare it as a Pwell, use the same resistivity as PBULK, and use the same depth at PW.

Diffusions. You must declare each type of diffusion in the interconnect technology file (*.tech*). They should also appear in *.subtech*. The Pdiff declaration has no associated information, because the *.tech* contains sheet rho and thickness for it, and NW includes the capacitance between PD and NW. The Ndiff declaration has only capacitance to PW, because *.tech* contains its sheet rho and thickness.

Note that while *.subtech* supports capacitance_fringe specifications for Pdiff and Ndiff, CSE ignores them. It uses only the capacitance specifications when modeling diffusion/well junctions. In practice, this approximation has negligible effect on block- and chip-level results, since rail noise couples strongly to wells through well taps.

SOI BCD process

Now consider the more complex BCD SOI process shown in Figure 13-27. It includes five surface features, plus a complex substrate comprising four layers, including buried oxide. The actual process may include more shallow and deep wells, not shown in the figure.

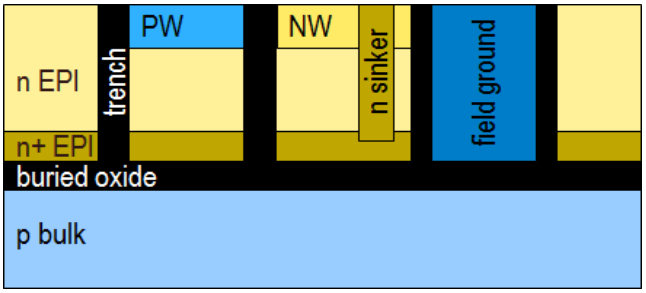


Figure 13-27 Cross-section of SOI BCD process substrate

The corresponding substrate-technology file has this general form:

```
######################
## BCD SOI .subtech ##
######################
Pdiff PD
Ndiff ND {
capacitance_fringe PW <v>
capacitance PW <v>
}
Nwell NW {
depth <v>
resistivity <v>
capacitance PD <v>
}
Pwell PW {
depth <v>
resistivity <v>
}
DeepPwell FG {
depth <v>
resistivity <v>
}
INsinker TRENCH {
depth <v>
dielc <v>
}
Nsinker NSINK {
depth <v>
resistivity <v>
```

```
}
Nsub NEPI {
depth <v>
resistivity <v>
capacitance PW <v>
}
Nsub NplusEPI {
depth <v>
resistivity <v>
capacitance PW <v>
}
INsub BOX {
depth <v>
dielc <v>
}
Psub PBULK {
depth <v>
resistivity <v>
}
```

This type of description has several notable differences from the CMOS triple-well subtech previously described.

Multi-layer bulk. As shown in Figure 13-27, the blank substrate for this process has four layers. The topmost EPI layers are n-type (Nsub), the buried oxide is an insulator (INsub), and the wafer handle is p-type (Psub). To create the desired cross-section, the four substrate layers have progressively increasing depths, as previously shown in Figure 13-5. Because BOX is an insulator, it has a relative dielectric value, of approximately 4.1 for SiO2, but no resistivity.

Field ground. FG fills a pocket completely, deeper than a shallow well. Therefore, FG is not a shallow well. In this case it has type DeepPwell, but you could also declare it as a Psinker with the same parameters.

Sinkers. TRENCH is an insulator, so it has a dielectric value, but no resistivity. On the other hand, NSINK is an n-type implant, so it has resistivity.

Few capacitance specifications. In this process, each pocket created by TRENCH walls and BOX contains only one kind of well, so layouts contain only a few types of p/n junctions.

No surface fill. Per the cross-section, without drawn wells, NEPI appears as the substrate surface. Therefore, the *.subtech* does not specify a global SURFACE_FILL_LAYER well.

Bulk BCD process

Finally, consider the even more complex bulk BCD process shown in Figure 13-28. It includes six surface features, plus a three-layer substrate. It supports embedded triple-well CMOS, where deep p-well areas provide a p-bulk-like islands for logic. The actual process supports multiple types of NWs and PWs not shown in the figure

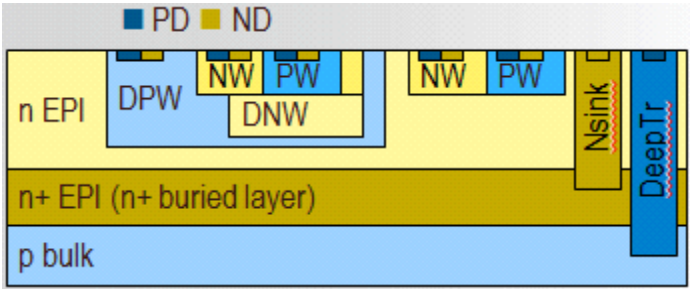


Figure 13-28 Cross-section of Bulk BCD substrate

The corresponding substrate-technology file has this general form:

```
########################
## Bulk BCD .subtech ##
########################
Pdiff PD
Ndiff ND {
capacitance_fringe DPW <v>
capacitance DPW <v>
capacitance_fringe PW <v>
capacitance PW <v>
}
Psinker DeepTr {
depth <v>
resistivity <v>
}
Nsinker Nsink {
depth <v>
resistivity <v>
}
Nwell NW {
depth <v>
resistivity <v>
capacitance_fringe PD <v>
capacitance PD <v>
capacitance_fringe PW <v>
capacitance DPW <v>
capacitance_fringe DPW <v>
}
Pwell PW {
depth <v>
resistivity <v>
}
DeepNwell DNW {
depth <v>
resistivity <v>
capacitance DPW <v>
capacitance_fringe DPW <v>
}
DeepPwell DPW {
depth <v>
resistivity <v>
}
Nsub NEPI {
depth <v>
resistivity <v>
```

```
capacitance_fringe PW <v>
capacitance PW <v>
capacitance_fringe DPW <v>
capacitance DPW <v>
capacitance_fringe DeepTR <v>
}
Nsub NBL {
depth <v>
resistivity <v>
capacitance PBULK <v>
capacitance_fringe DeepTr <v>
}
Psub PBULK {
depth <v>
resistivity <v>
}
```

Compared to the SOI BCD process previously described, there are a few differences to note.

Nested deep wells. The cross section shows that DNW nests inside DPW. Therefore, in *.subtech*, DPW has a larger depth than DNW.

DeepTr sinker. Physically, this sinker consists of an oxide-lined trench and a conductive core exposed only to PBULK. Process design rules require an empty area around each DeepTr sinker. Therefore, in the *.subtech*, it can appear as a Psinker, coupled through fringe capacitance to NEPI and NBL, and resistively connected to PBULK.

No surface fill. Per the cross-section, without drawn wells, NEPI appears as the substrate surface. Similarly, without drawn shallow wells, DPW appears as the substrate surface in DPW areas. Therefore, the *.subtech* does not specify either a global SURFACE_FILL_LAYER well or a SURFACE_FILL_LAYER well local to DPW.

Visualizing the substrate cross-section

Totem can display a cross-section of the features in the active substrate technology file. This view helps check relative depth of features. To create the cross-section view from the Totem GUI, select **View à Technology Layers à Technology Layers (Substrate)…**

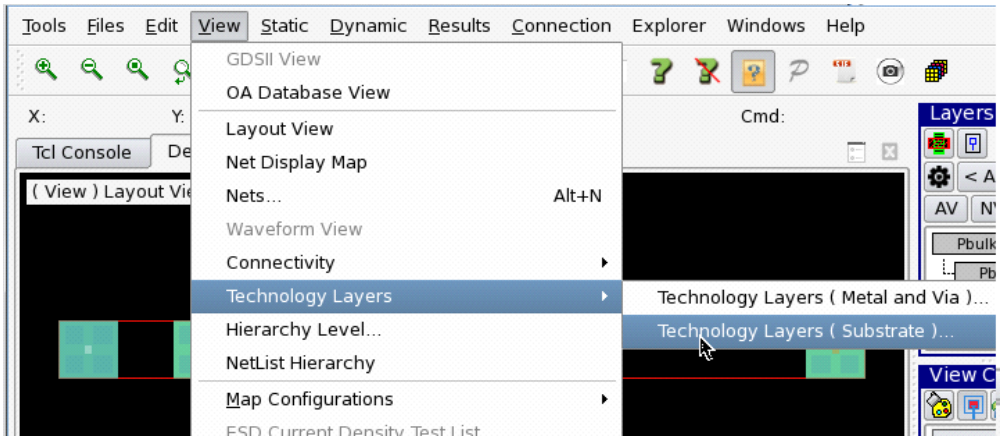


Figure 13-29 Invoking the subtech viewer

As shown in Figure 13-30, Totem display a substrate cross-section, showing all substrate features and their depths. The viewer organizes the display by depth for compactness. It has no knowledge of design rules, so adjacent layers at a specific depth shown in this view may or may not touch each other in a layout. For example, in Figure 13-30, the shallow non-well NTN lies between PW and NW, where NTN actually cannot touch NW due to design rules.

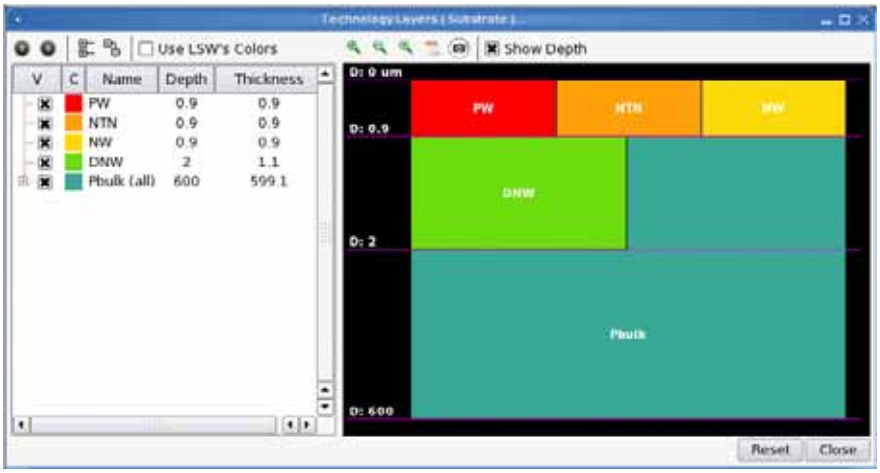


Figure 13-30 Subtech view of a bulk CMOS process

Configuring Totem-CSE for substrate analysis

To perform substrate analysis, Totem-CSE requires the mapping from drawn shapes in the layout to substrate features and the physical/electrical mapping for creating the electrical model. The layermap or LVS configuration maps shapes to features, and the substrate technology file provides the electrical characteristics for the features.

To create a complete CSE configuration, start by understanding the substrate cross-section and the rules for drawing substrate features in layout. Then create the substrate technology file and set up the geometry flow.

Determining the substrate cross-section

Usually, there is no problem listing all the physical substrate features; however, you need to decide how to represent them in CSE. Use an idealized, simplified cross-section for simplicity and efficiency. If you discover later that your analyses require more detail, you can always add it later.

For example, consider a triple-well CMOS process. To choose the cross-section for CSE:

- Define one type of PW and one type of NW. The actual process has multiple wells of each type--for example, logic wells and deeper I/O wells. However, using core/logic wells to represent all wells provides good accuracy.
- Define non-well “PW” separately, since it has much higher resistivity than doped PW.
- Assume that PW inside DNW (the profile PW/DNW) has the same depth (and resistivity) as PW over p-bulk. PW inside DNW has somewhat different characteristics than PW over p-bulk, but the differences do not significantly impact substrate-noise conduction through surface wells in a block or full chip. Similarly, use the same NW for NW/DNW as for NW/p-bulk.
- DNW under PW has a higher resistance than DNW under NW. However, the surface

wells dominate noise conduction in DNW areas, so model a single type of DNW.

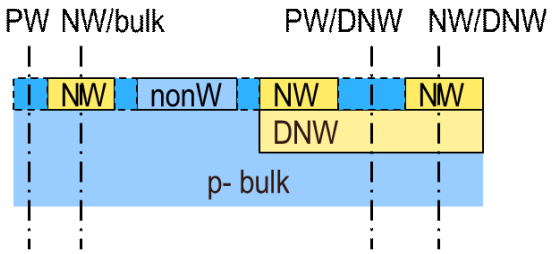


Figure 13-31 Triple-well cross-section, showing implicit (not drawn) PW surface fill

"Drawing rules" for substrate shapes

To configure layout processing, you need to know what shapes on what drawn layer(s) determine each possible substrate cross-section of interest. Some substrate features may appear as drawn shapes in a design layout; others must be derived based on drawn shapes. CSE can derive some features. For example, for a triple-well MOS substrate such as in Figure 13-31, NW and nonW have drawn shapes on specified GDS layers; PW is implicitly present everywhere without NW or nonW, and CSE can derive it (see SURFACE_FILL_LAYER below).

However, in complex cases, CSE requires derived features in its input geometry. For example, some BCD processes implicitly include guard rings generated by LVS. CSE cannot calculate these shapes or their connectivity, so it must use LVS output geometry to model them.

Answer the following questions to determine the key drawing rules for substrate shapes.

1. What layer combinations imply different substrate cross-sections?

   For example, even if PW implicitly fills the substrate surface where other wells are absent, layouts might contain explicit PW shapes. If so, what do they mean? For example, perhaps you can draw a PW shape inside an NW shape to punch a PW hole in it, i.e.: NW feature = drawn NW - drawn PW.

2. Are there implicit shallow wells in some substrate areas?

   For example, in a bulk CMOS process, PW typically fills the surface outside of all drawn NW and non-well shapes. Implicitly, PW feature = ! (drawn NW | drawn nonW).

3. Do different areas have different implicit fills?

   For example, in a bulk CMOS process, perhaps NW implicitly fills DNW areas, but PW implicitly fills non-DNW (bulk) areas.

4. Are there other implicit substrate features?

   For example, some BCD processes implicitly include a complete diffusion/contact/metal guard ring connected to a power rail at the edge of certain deep wells. LVS synthesizes these rings, since it requires them for LVS checks. However, CSE clearly cannot derive them accurately-- it must use the geometry derived by the LVS deck.

   To help decide how to include implicit features:
   - Identify the step in the design flow that derives their shapes.
   - Determine the structure of these shapes in layout.

- Check the connection of shapes to layout nets.

5. Can shallow wells overlap?

   If yes, determine the substrate cross-section for each allowed combination.

6. Can sinkers overlap?

   If yes, determine the substrate cross-section for each allowed combination.

7. In LVS, which derived layers determine substrate features?

   For example, the LVS rules may separate drawn NW into many different derived layers. You may need to combine a set of them to create NW features for CSE.

8. Which shapes can touch each other?

   For example, perhaps DNW must always have NW around its outer edges. Then PW will never abut DNW laterally. If some shallow wells can only occur inside deep wells, they can never touch bulk.

   For processes with conducting sinkers, check if design rules require blank substrate, particular wells, etc., to surround them, which is typically the case. If so, you can often use efficient tricks to model them in CSE.

Tuning the substrate cross-section

CSE models the substrate as an RC mesh, as illustrated in Figure 13-14. It adds a layer of R/C mesh based on unique bottom depths for features. The cross-section in Figure 13-31 requires at least 3 layers, representing the shallow wells, DNW and adjacent Pbulk, and Pbulk below DNW. So by default, CSE discretizes this cross-section into three depths: 0 - 0.9, 0.9 - 2, and 2 - 600.

Increasing the discretization can improve accuracy for noise paths that travel vertically. Long-distance paths are less sensitive to the details of discretization, so full-chip noise rarely requires increasing the number of mesh layers.

On the other hand, increased discretization increases memory use, and it can significantly increase runtimes. Therefore, CSE tries to use as few layers as possible without significant effects on accuracy.

Default simplifications

CSE snaps shallow wells and sinkers to reduce the number of Z layers in the substrate profile. Unlike interconnect, substrate connects densely in 3D. If you create too many Z layers, especially for full chips, the combined interconnect/substrate model may become too big to solve in reasonable time/memory. Therefore, CSE tries to use as few Z layers as necessary to achieve accuracy.

By default, CSE makes these simplifications to the substrate cross-section:

- Use one layer to represent all shallow wells by “snapping” the depth of all shallow wells to depth of the shallowest shallow well

  In a triple-well process, NW is usually shallower than PW. CSE snaps the depth of PW to match the depth of NW, adjusting PW resistivity to keep its sheet rho constant.

  Most substrate noise travels through the shallow wells, because they provide the lowest impedance path due to their high doping. P-well guard rings are effective because they ground this path. Because noise travels laterally mostly in the shallow wells, CSE can accurately model noise while representing the shallow wells with only a single layer.

- Snap each sinker to the bottom of the deepest layer it travels through

  Suppose a triple-well process supports deep-trench isolation, and that CSE models a

deep trench as an insulating sinker. Depending on the relative depths d() of shallow wells, the deep-trench sinker, and DNW, CSE snaps the sinker one of two ways. If d(shallow wells) <= d(sinker) < d(DNW), CSE snaps d(sinker) to d(shallow wells). If d(DNW) <= d(sinker), CSE snaps d(sinker) to d(DNW).

For many types of typical sinkers, snapping is no problem. Harmless examples include: INsinker snapping to BOX in an SOI process; Nsinker snapping to n- EPI in an n-EPI/b-bulk BCD process; DTI sinkers in a CMOS imager process, where the sinker reaches the bottom of the thinned wafer.

If you want to force CSE to model the actual depth of a sinker, use SUBSTRATE_Z_DISCRETIZATION (see below). For example, in a process with 1um-deep shallow wells, a 2um-deep sinker, and a 3um-deep DNW, by default CSE snaps the sinkers to 1um depth. To use the actual 2um depth, add the GSR: SUBSTRATE_Z_DISCRETIZATION “2”.

Totem-CSE geometry source

Within Totem, CSE can use either layout GDS or Calibre-LVS CCI as the source of geometry data. Figure 13-32 illustrates these flows.

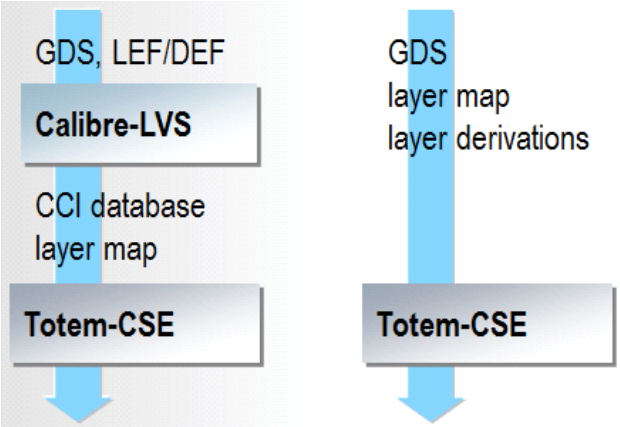


Figure 13-32 CSE flows using LVS or GDS geometry data

The choice primarily depends on how a layout represents substrate features. If drawn shapes in GDS map nearly directly to substrate features, then GDS data can be used as input. Layouts in triple-well bulk CMOS usually work well with layout GDS. On the other hand, for BCD or other processes with a large number of substrate features, or complex rules for deriving them, use LVS geometry for ease of setup and maintenance. Otherwise, you have to maintain complex derivation rules in parallel in the LVS deck and the Totem layermap. If actual layouts contain implied features that do not appear in GDS, use LVS geometry, which must derive them for complete LVS checking. For more guidance on choosing and configuring a CSE flow, contact Applications Engineering.

The table following compares how CSE derives data using GDS and LVS geometry.

	Calibre LVS data	Raw GDS data
Derived substrate layers	from LVS geometry operations	from Boolean operations specified in layermap
Implicit substrate features	from LVS geometry operations	not supported
Layer mapping	Calibre derived layers mapped to CSE substrate layers	GDS layers and results of Boolean operations mapped to substrate layers

GDS layer map file for CSE

When using GDS geometry for substrate analysis, you must extend the standard interconnect layermap to also supply shapes for the substrate layers.

In the layer map file, use the types “pw” and “nw” for all shallow well shapes. (Non-CSE analyses need these shapes to differentiate active diffusions from tap connects.) Use “substrate” type for all other substrate features. With this assignment of layer types, the same extended layermap file works for both CSE and non-CSE analyses.

Figure 13-33 shows an SOI BCD substrate and extended layer mapping for it. Note that the map gives pw/nw types for shallow wells, but uses the generic type substrate for other features.

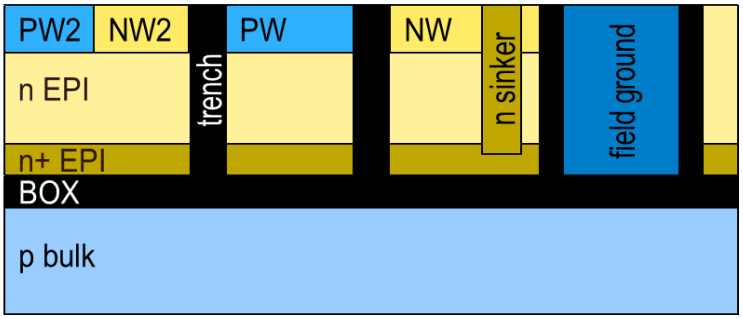


Figure 13-33 BCD SOI process and layermap (below) with two types of wells and all features explicitly drawn

```
#layer          type     GDS text
PW               pw      3:0     -
NW               nw      4:0     -
PW2              pw      5:0     -
NW2              nw      6:0     -
FG          substrate    7:0     -
Nsink       substrate    8:0     -
TRENCH      substrate    9:0     -
#EPI, BOX not drawn

PD               pd      10      -
ND               nd      11      -
CONT        contact      20      -

M1          m       21      -
...
```

Creating the substrate technology file

Before trying to create a substrate setup for your process of interest, contact your applications engineering team to see if one already exists. Substrate technology files are usually available for TSMC and Global Foundry processes, and they may be available for other major foundries.

Otherwise, use this section as a guide for creating a substrate technology file. It provides suggestions to determine substrate cross section with its features and options to estimate electrical parameters of features. Also, the Totem product team has additional material to help create and test a *subtech*.

The full PDK for a process contains the two main sources that usually give most of the information you need: the design manuals for the process, and the SPICE models for devices. For some processes, you may need to look at the LVS rule deck to understand how substrate features derive from drawn shapes in layouts. Finally, only fab engineers may know some of the substrate parameters.

The following subsections describe the recommended steps for creating a substrate technology file, which are basically: describe the cross-section; set the resistance parameters; set the capacitance parameters; and specify surface fill. In general, specifying surface fill requires an understanding of how to draw each substrate feature in layout, covered in a separate section below; this section discusses surface fill only.

Determining depths of substrate features

First, determine the depths of all substrate features. Again, simplify where possible - CSE does not need exact depths to deliver accurate results. Fabs can often provide target cross-section and depth information for a process, especially for bulk layers. Doping profiles for non-device areas can also provide depth information, since the depth of each p/n junction is well-defined.

For example, considering the triple-well CMOS cross-section in Figure 13-31:

- Choose one depth for all shallow wells. For example, NW has a well-defined depth in a p-bulk process, due to the NW/p-bulk junction at the bottom (the profile labeled NW/bulk in Fig. 26). Use NW depth for PW, which does not have a junction to define its depth when it lies over p-bulk (the profile labeled PW in Figure 13-31).
- Determine DNW depth. DNW ends at a well-defined depth, due the DNW/bulk junction below it. Note that DNW must extend deeper than all shallow wells; physically, the DNW implant is normally much deeper than shallow wells.
- Determine bulk depth, i.e., bulk thickness. Use the nominal thickness. If some designs use thinned substrates, you can either modify the depth given in *.subtech* or use GSR keyword SUBSTRATE_MODIFY_BULK_LAYER_DEPTH.

Specifying feature depth

Specify the depth of each feature as the distance from the top surface of the substrate to its bottom edge, using a depth property. For a CMOS process, either STI or diffusion covers the top of the substrate. Figure 13-34 illustrates the depth of NW, and the corresponding declaration in *.subtech* follows it.

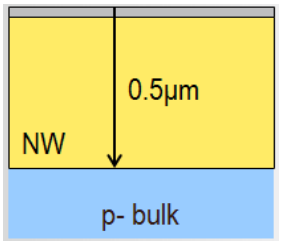


Figure 13-34 Specifying NW depth. The gray layer on top of NW represents STI

```
#nwell in substrate tech file
Nwell NW {
   depth 0.5
   …
}
```

Determining resistivity of conductors

SPICE device models for a process often include direct or indirect specifications of the resistivity of features. For example, device models often need well resistances, so the library may define sheet resistance for shallow wells; resistivity equals sheet rho times thickness. The resistance of characterized well resistors also implies well resistivity.

Fabs often know the resistivity of some substrate features, in particular, bulk resistivity. Most CMOS processes use the same 10 Ohm-cm high-resistivity p- bulk.

However, only doping profiles may provide information about some layers-- for example, the resistivity of DNW under PW. Preferably, use profiles away from devices, where implants may affect the profiles, e.g., the retrograde profiles around MOS transistors. Your fab may have process-specific formulas for determining resistivity from profiles.

However, general formulas exist. See, for example: Dieter K. Schroder, “Semiconductor Material and Device Characterization”, Third Edition, John Wiley and Sons, Hoboken, New Jersey, 2006, which provides formulas based on measurements for phosphorous (n-type) and boron (p-type) dopants.

Specifying resistivity

Specify resistivity for each conducting feature using a resistivity property.Figure 13-35 gives an example specification for NW.

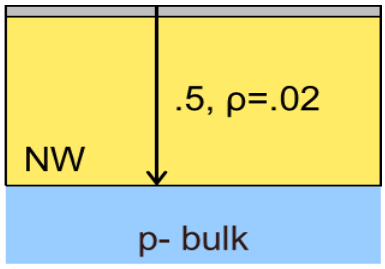


Figure 13-35 Specifying NW resistivity

The layermap for the figure above is as follows:

```
Nwell NW {
    # bottom depth, um
    depth .5
    # rho, Ohm-cm
    resistivity .02
...
}
```

Note that the surface of some substrate features are always covered by other features. In such cases, choose the resistivity for the deeper, non-covered material to reflect the correct sheet rho.

For example, the surface of DNW in a bulk CMOS process is always covered by PW or NW -- there are no non-well areas. Choose the resistivity of DNW appropriate for the non-covered part of DNW underneath the surface wells--for example, based on an appropriate doping profile. That is, for the example shown in Figure 13-36, the resistivity of DNW should reflect the appropriate sheet rho for a 1?m-thick layer of DNW.

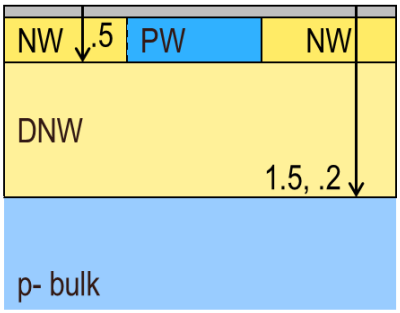


Figure 13-36 DNW depth and resistivity.

In the figure above, the 1um-thick layer of “pure” DNW under the surface wells has a resistivity of 0.2 Ohm-cm. The layermap would look as shown below:

```
DeepNwell DNW {
# bottom depth, um
depth 1.5
# rho, Ohm-cm
resistivity .2
...
}
```

Determining capacitance values

Specify the capacitance value(s) for each p/n junction that exists in a layout, taking into account design-rule restrictions on which opposite-doping substrate features may touch. Making a table of all n-type features vs. all p-type features can capture the requirements clearly. For example, data for a triple-well CMOS is shown in the table below:

	ND	NW	DNW	Notes
PD	n/a[1]	A, F	n/a[2]	[1]shorted by salicide; [2]DNW covered by PW/NW
PW	A, F	F	A	
non-well	n/a[3]	n/a[3]	n/a[3]	[3]only touches PW, Pbulk
pBulk	n/a[4]	A	A, F	[4]in native nMOS FETs only

In the table above, “A” refers to the areal capacitance for a top/bottom junction between opposite-doping-type features and “F” refers to the fringe capacitance along a sidewall junction. Also, specify the relative dielectric constant for each insulating feature (INbulk or INsinker). Do not include STI, which CSE does not model as a bulk layer.

Calculating capacitance from parasitic substrate diodes

Foundries characterize the parasitic substrate diodes for each process. Consequently, the SPICE models for these diodes supply the junction capacitance values for them. These diodes usually have obvious types, e.g., dpwnw for a PW/NW parasitic diode.

Other device models may also include substrate diodes. For example, MOS models contain parasitic diffusion/well diodes.

To calculate a substrate capacitance from a diode model, use the appropriate set of bottom-wall (cj0/pb/mj) or sidewall (cjsw0/php/mjsw) SPICE parameters:

Parameter	Symbol	Units	Description
cj0/cjsw0	C_{j0}	F/m^2, F/m	Zero bias junction capacitance
pb/php	ϕ_0	V	Contact potential
mj/mjsw	m	–	Junction grading coefficient 1/3 to 1/2

Use the standard diode-capacitance formula:

$$C_j(v) = C_{j0}/[1-(v/\phi_0)]^m$$

α β χ δ ε ϕ γ η ι φ κ λ μ ν ο π θ ρ σ τ υ ϖ ω ξ ψ ζ >>?? << ALPHA- SYMBOLS

where v is the bias voltage for this type of diode in the design of interest. Finally, scale all sidewall (fringe) capacitance values to substrate technology file units: $C_{subtech}$ pF/µm = 10^6 $C_{formula}$ F/m. Areal Bottom wall (areal) capacitance needs no scaling, because 1 pF/µm2 = 1 F/m2.

Choosing the bias voltage

Because CSE uses a fixed, linear capacitance for each type of junction, you must choose an appropriate bias voltage. For actual designs, most junctions have a (nearly) constant back-bias applied. For example, if V_{pbulk} = 0 and V_{nw} = VDD due to well taps, use v = VDD for the bias for NW diodes to Pbulk. For test structures, use the bias applied when probing the structures, typically v = 0. If you need to model multiple bias conditions with CSE, you must create one *.subtech* per set of bias points.

Alternative capacitance calculation

Alternatively, you can estimate substrate junction capacitance from doping profiles and standard semiconductor-physics formulas. Obviously, this approach has poorer accuracy than using calculations based on characterized models.

Estimate dopant concentrations from doping profiles. For a sidewall capacitance, use concentrations at several different depths, then choose a result that you feel represents the junction.

Then calculate C_j using the textbook equation:

$$C_j(N_a,N_d,A,V_d) = A*SqRt\{[q\varepsilon N_a*N_d/(N_a+N_d)]/[\phi_0 - V_d]\}$$

where:

- built-in potential: $\phi_0 = V_t * \log(N_a*N_d/Ni2)$
- intrinsic Si carrier concentration at 300K: Ni = 1.5e10/cm^3
- threshold: V_t = kT/q = 26e-3 V
- permittivity of Si: ε = 1.04e-12 F/cm
- electron charge q = 1.6e-19 Coulombs

Specifying capacitance

Specify capacitance for each type of junction in the specification of its n-type feature. Figure 32 shows an example for NW in a CMOS process.

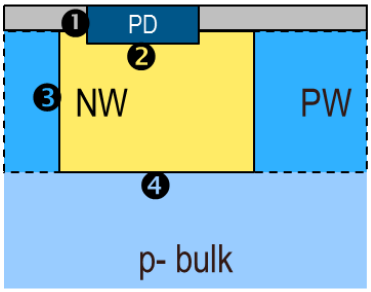


Figure 13-37 Cross-section of NW in a CMOS process

The figure above shows related capacitances for NW in a CMOS process. with corresponding specifications as follows:

SUBSTRATE_MODIFY_BULK_LAYER_DEPTH

```
Nwell NW {
```

Calculating capacitance from parasitic substrate diodes

```
# (1) nwell/pdiff sidewall, pF/um
  capacitance_fringe PD 3e-03
# (2) nwell/pdiff, pF/um^2
  capacitance PD 3e-04
# (3) nwell/pwell sidewall, pF/um
  capacitance_fringe PW 9e-04
# (4) nwell/psub, pF/um^2
  capacitance Pbulk 3e-05
}
```

Specifying surface fill

Finally, *.subtech* usually requires SURFACE_FILL specifications for areas without drawn shallow wells. To determine the correct specification, determine which shallow wells are drawn and which are not for each deep well and for bulk areas.

As discussed previously, triple-well CMOS processes require drawn shapes for NW and non-well, but not for PW. PW covers all substrate surface not covered by an explicitly drawn well. To specify this PW-default well-drawing rule in *.subtech*, include a global surface-fill declaration: SURFACE_FILL PW

However, in addition to this global default, suppose that NW instead of PW covers the surface of DNW areas when other shallow wells are present. In that case, specify the following rule using a surface-fill declaration local to DNW:

```
DeepNwell DNW {
  SURFACE_FILL NW
...
}
```

This local default overrides the global default whenever DNW exists.

Substrate GSRs interacting with .subtech

Three substrate GSRs affect how CSE interprets *.subtech* by modifying the default vertical discretization:

- SUBSTRATE_MODIFY_BULK_LAYER_DEPTH: changes the thickness of bulk to account for wafer thinning, without editing the *.subtech*
- SUBSTRATE_WELL_PORTION: splits shallow wells vertically; which is required for substrate technology files that use multiple sublayers to describe shallow wells. Otherwise, do not use this keyword.
- SUBSTRATE_Z_DISCRETIZATION_POINTS: splits the substrate model into sub-layers at specified depths. May be needed to model vertical currents in more detail, e.g., to see how deeply substrate noise penetrates bulk. Normally, not used.

SUBSTRATE_MODIFY_BULK_LAYER_DEPTH

When set, changes the depth of the deepest (bottom-most) bulk layer-- the overall thickness of the substrate. Use it to conveniently model thinned wafers, without editing the depths specified in *.subtech* or maintaining multiple *.subtech* files. In particular, use it if you need to modify bulk depth for an encrypted *.subtech*. You must specify the deepest bulk layer for this setting. The new depth must satisfy two bounds:

- new depth > depth of the next-deepest feature--for example DNW in a triple-well CMOS process
- new depth </= default depth in *.subtech* (that is, the wafer may only get thinner, not thicker)

Syntax:

```
SUBSTRATE_MODIFY_BULK_LAYER_DEPTH <deepest bulk layer>
<depth um>
```

For example, to set the thickness of a PBULK bottom layer to 400 um, the specification would be:

```
SUBSTRATE_MODIFY_BULK_LAYER_DEPTH PBULK 400
```

SUBSTRATE_WELL_PORTION

When set, CSE slices all shallow wells into sub-layers, each of which covers the specified fraction of the depth of the original layer. The fractions must total 1.0. CSE models each sub-layer as a separate resistor mesh, increasing the vertical discretization of the shallow wells.

You must use this setting with substrate technology files that specify sub-layers for shallow wells with significantly different resistitivities. Otherwise, CSE combines the sub-layers into a single layer with an “average” resistivity that it uses for both vertical and lateral conduction, giving too high a sheet rho for lateral conduction. The syntax for specifying sub-layers in .subtech is:

```
depth <d1> <d2> ...
resistivity <r1> <r2> ...
```

where each pair (di, ri) specifies a sublayer with the corresponding depth and resistivity.

Use SUBSTRATE_WELL_PORTION to approximately split the substrate model into these sub-layers-- the fractions need not correspond exactly to the split implied by the depths.

Syntax:

```
SUBSTRATE_WELL_PORTION "<fraction 1> <fraction 2> …"
```

SUBSTRATE_Z_DISCRETIZATION_POINTS

When set, s CSE splits the substrate model at the specific depths di um. In addition to the default discretization based on depths of the features in .subtech, CSE also creates a resistor mesh for each conductor at each additional depth, increasing the vertical resolution of the substrate model. Clearly, this additional discretization increases model size, and therefore, run time and memory, so use it sparingly, especially for full-chip analysis.

Figure 13-38 shows the effect of adding discretization points for a triple-well CMOS process. By default, CSE splits the bulk into 3 vertical layers, based on the depth of shallow wells, DNW, and bulk. Adding discretization points at depths of 1um, 10um and 20um results in a total of 6 vertical layers, approximately doubling the complexity of the substrate model, but providing additional information about noise vs. depth and improving the accuracy of “corner-turning” currents (changes in current direction from vertical to horizontal).

Syntax:

```
SUBSTRATE_Z_DISCRETIZATION_POINTS "<d1> <d2> <d3> ..."
```

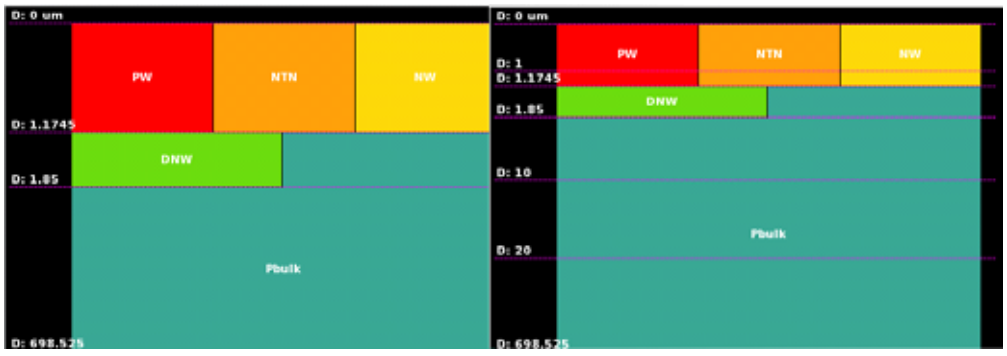


Figure 13-38 Substrate cross-section without and with setting SUBSTRATE_Z_DISCRETIZATION_POINTS “1 10 20”

Reference C: Characterizing substrate-pin currents with APL

Generally, you do not need to model currents on substrate pins of FETs inside of standard cells. Noise on power rails couples directly to standard-cell wells; substrate-pin currents make little or no contribution. Accordingly, you may use existing, non-substrate characterization results when running CSE.

Nonetheless, APL can characterize substrate-pin currents for logic an decap cells in most cases:

- Single-rail cells with or without separate substrate power/ground nets
- Multi-rail cells with separate substrate power/ground nets

Additions to APL configuration for substrate

To enable substrate characterization, set `SUBSTRATE_CURRENT 1`, and declare substrate Power/Ground pins must be specified using `VDDB_NAME` and `VSSB_NAME`.

SUBSTRATE_CURRENT [0|1]

Enables substrate-pin current characterization. Default: 0

VDDB_NAME {

<substrate_power_pin_1> [<paired_vdd_domain_pin_1>]

[<substrate_power_pin_2> [<paired_vdd_domain_pin_2>]]

...

}

Specifies which VDD domain each substrate (bulk) power pin belongs to; required if the design has multiple VDD domains

VSSB_NAME {

<substrate_ground_pin_1> [<paired_vss_domain_pin_1>]

[<substrate_ground_pin_2> [<paired_vss_domain_pin_2>]]

...

}

Specifies which VSS/GND domain each substrate (bulk) power pin belongs to; required if the design has multiple GND domains

Note that <paired_vdd_domain_pin_x> and <paired_vss_domain_pin_x> refer to VDD/VSS pin names in the SPICE netlist for cells, *not* LEF pin names.

Example 1

A single Vdd/Vss cell without separate substrate power/ground nets, i.e., where the substrate pins are connected with VDD/VSS. The cell has the following VDD/VSS and substrate pins:

vdd domain: vdd

vss domain: vss

substrate power pin: vdd

substrate ground pin: vss

To specify them in the APL configuration file, use:

```
SUBSTRATE_CURRENT 1
VDDB_NAME {
    vdd vdd
}
VSSB_NAME {
    vss vss
}
```

Example 2

A multi-rail cell with separate substrate power/ground nets. The cell has the following VDD/VSS and substrate pins.

```
vdd domains: vdd1 vdd2
vss domains: vss1 vss2
substrate power pins: vbbnw1 vbbnw2
substrate ground pin: vbbpw1 vbbpw2
```

If `vbbnw1` belongs to the `vdd1` domain; `vbbnw2,` to `vdd2`; `vbbpw1`, to `vss1`, and `vbbpw2` to `vss2`, specify them in APL configuration file this way:

```
SUBSTRATE_CURRENT 1
VDDB_NAME {
vddnw1 vdd1
vddnw2 vdd2
}
VSSB_NAME {
vbbpw1 vss1
vbbpw2 vss2
}
```

Specifying nominal voltages for substrate pins

By default, APL uses the nominal voltage of the paired domain for each substrate pin. With `SUB_VOLTAGE`, you can specify a different voltage (in Volts)”

SUB_VOLTAGE <substrate_pin_name> <voltage>

Example

A cell with separate substrate nets has the following VDD/VSS and substrate pins.

vdd domain: vdd

vss domain: vss

substrate power pin: vddnw

substrate ground pin: vsspw

Use the basic specification, then set explicit voltages to use during characterization:

```
SUBSTRATE_CURRENT 1
VDDB_NAME {
```

```
vddnw vdd
}
VSSB_NAME {
vsspw vss
}
sub_voltage vddnw 1.0
sub_voltage vddpw 0.2
```

Controlling bulk-pin resistance

During characterization, APL uses substrate resistivity to set series R between bulk pins and their nets (essentially, well resistance to their power rails), using the formula:

series R = (substrate resistivity)/(10 Ohm-cm) * 2k Ohm

For a typical bulk-CMOS process with a 10 Ohm-cm substrate, APL therefore uses a 2k series resistor. SUBSTRATE_RESISTIVITY tells APL to use a different resistivity value for this calculation.

SUBSTRATE_RESISTIVITY <resistivity_Ohm-cm>

Example:

```
SUBSTRATE_RESISTIVITY 20
```

in which case APL uses (20/10)*2k = 4k resistors for substrate characterization.

Reference D: LDMOS noise sources

Typical LDMOS devices have the cross-sections shown in Figure 13-39 Both devices lie in deep n-wells. However, they differ in how they couple noise to the substrate.

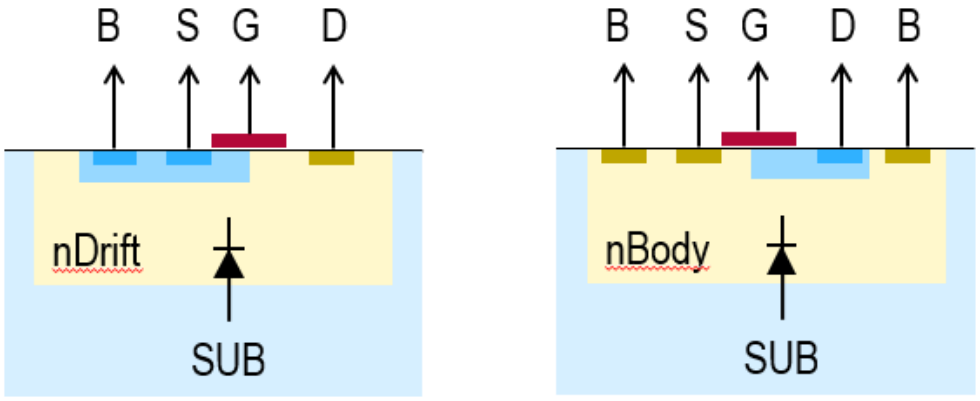


Figure 13-39 nLDMOS and pLDMOS cross-sections

For the nLDMOS device, the deep n-well forms part of the drift region for the device, and it connects to the D pin, which typically drives an external load. Thus, nLDMOS devices primarily inject the noise from the D pin. Also, the voltage on the drift well may swing over

a wide range, say, 0 – 40V, significantly changing the back-bias on the well/SUB diode, and therefore, its capacitance.

For the pLDMOS device, the deep n-well forms part of the body well for pFET, and it connects to the B and S pins. These pins typically connect to VDD. Thus, pLDMOS devices primarily inject noise for a high-voltage VDD rail. The body/substrate diode sees a relatively constant back-bias, giving a contact C value.

Each LDMOS device can play one or more roles with respect to substrate noise: injection, absorption, and conduction. As Figure 13-40 shows, devices with noisy DNWs inject noise: an nLDMOS with a noisy net on D pin, or a pLDMOS with noisy VDD on B pin. Note that device itself need not switch to cause the noise.

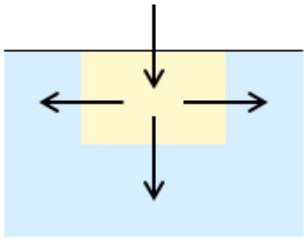


Figure 13-40 LDMOS roles in substrate noise (a) noise injection

On the other hand, the same devices connected to quiet nets can absorb noise (Figure 13-41). The nets must have a low-Z connection between its deep n-well and quiet AC ground, which is often, but not always, true.

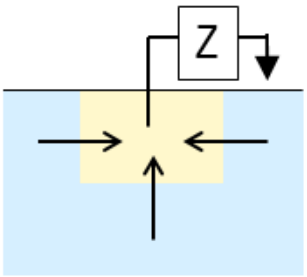


Figure 13-41 LDMOS roles in substrate noise (b) noise absorption

Rarely, an LDMOS can conduct noise through its n-well (Figure 13-42). Since conduction requires a high-Z connection between the deep n-well and AC ground, you can disregard this situation except in special cases (or if your setup is wrong!).

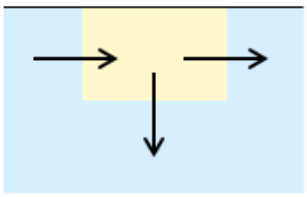


Figure 13-42 LDMOS roles in substrate noise (c) noise conduction

LDMOS devices act purely as aggressors in most designs containing them, so focusing only on noise injection describes their role accurately. For simplicity, model them as white boxes. Alternatively, you can consider other models, for example, that use characterized SUB pin currents to account the non-linear C of the n-well/substrate diodes.

White-box LDMOS model

The white box model uses all substrate features in the device. It couples noise on the n-well to the substrate through the diode between them.

Physically, model the complete cross-section: the enclosing deep n-well, the enclosed shallow p-well (to avoid shorting D/S), and the diffusions. In the substrate tech file and layer map, represent the nDrift and nBody wells as unique deep n-wells. Set C between nDrift and bulk to a value that will couple the correct amount of total charge for the D-net voltage swing, and C between nBody and bulk based on the body pin's VDD.

Note that the D pin of an nLDMOS device usually drives an external load – it's a signal net. To include nDrift, you must declare the D-pin net as a VDD. Connect its PLOC to an impedance approximating the load it drives, so that the net responds appropriately to current injected on the net, if any.

For simulating with manual noise injection, use `-inject_voltage` on the PLOC for the D-pin net for nLDMOS and/or on the VDD PLOC for the S/B-pin net for pLDMOS.

For simulating using aplmmx-characterized pin currents, ignore the SUB pin when configuring nLDMOS devices (see **DEVICE_MODEL_CONFIG**), to avoid double-counting noise injection from nDrift wells. Connect the other pins physically.

Reference E: Rarely-used and obsolete CSE GSRs

This section documents GSR controls used only for special flows or experimentation. It also lists obsolete keywords that you should not use; eliminate them if they exist in your CSE setups.

Rarely-used controls

BACK_BIAS_ANALYSIS [0 | 1]

Experimental setting for analyzing well currents due to standard-cell bulk connections. *Optional. Default: 0.*

Enables limited substrate modeling. Requires APL cell characterization including bulk pins. Creates tap and bulk-pin connections in standard areas; sets

DYNAMIC_COUPLED_SOLVER (required when simulating substrate models). Does not enable frequency-domain analysis, guard-ring weakness checks, or GUI substrate features.

Usage:

```
BACK_BIAS_ANALYSIS 1
SUBSTRATE_ANALYSIS 0
```

SUBSTRATE_ADD_INTRINSIC_CAP [0 | 1]

Disables intrinsic-capacitance modeling in special situations. *Optional. Default: 1.*

By default, CSE includes intrinsic capacitance of bulk silicon when modeling substrate features with relaxation time constant greater than 10ps. For layers with sufficiently high resistivity, intrinsic capacitance may significantly reduce impedance at high frequencies, usually 5GHz or above. For typical CMOS processes, only 10 Ohm-cm bulk layers require modeling intrinsic C.

Use this GSR to disable intrinsic-C modeling, for example: to force the substrate-capacitance reported in Totem logs to reflect only junction capacitance; or to reduce substrate model size for low-frequency applications.

SUBSTRATE_CONNECT_FLOATING_GEOMETRY [0 | 1]

Include all floating substrate features in simulation. *Optional. Default: 0 for typical processes, e.g., for CMOS triple-well; 1 for SOI or other processes with insulating substrate features*

"Floating geometry" refers to substrate features that CSE cannot trace from any PLOC.

Setting this GSR forces CSE to include floating substrate features in the model it passes to simulation. It includes their internal resistor networks and the capacitors connecting them to the rest of the substrate model.

Do not enable this setting simply because Totem reports floating substrate wires: You may mask design or setup errors that will cause major errors later. Typical CMOS designs contain few wells that intentionally float.

Instead, first understand what is causing geometry to float:

a. Incorrect design features with unconnected geometries – fix the design

b. Unconnected geometry due to missing trace points or labels – fix the setup

c. Intentional floating geometry in the design – enable this GSR after checking (a) and (b)

If the substrate tech file contains insulating layers, CSE enables this setting automatically. For example, a design using a BCD SOI process may include many floating pockets. In a design using an SOI logic process, the entire bulk will float unless the block you are analyzing includes and connects bulk tap. In both cases, you typically want to include these floating features in simulation.

SUBSTRATE_CONTACT <file>

Use for experimentally adding connectivity between interconnect wires and substrate features, for example, for what-if analyses or quick ECOs. *Default: none. Locations are in microns.*

Syntax:

```
R <layer_name1> <x1> <y1> <layer_name2> <x2> <y2> <R_Ohm>
```

```
C <layer_name1> <x1> <y1> <layer_name2> <x2> <y2> <C_pF>
```

Connects non-physical resistors or capacitors between pairs of x/y/layer locations. You may connect capacitors between locations regardless of their domains. Do not short domains with added resistors, however.

For example, to connect a 1M Ohm resistor between NW near (80,30) and M9 near (130,25):

```
R NW 80 30 M9 130 25 1e6
```

To connect a 231pF capacitor between an n-well and a p-diffusion wire:

```
C NW 46.97 43.86 PD 50.8 48.2 231
```

SUBSTRATE_CONTACT_RESISTANCE <R Ohms>

Specifies globally the resistance used to connect standard-cell tap pins to their wells when cell layouts do not come from GDS. Preferably, use `SUBSTRATE_LIBRARY_FILE` instead. *Default: 1 Ohm.*

SUBSTRATE_GDS_OVERLAY [0 | 1]

Specifies exact full-chip substrate geometry in standard-cell areas for designs where CSE cannot automatically generate sufficiently accurate wells, for example, when a design uses specialized isolation for standard cells. *Default: none.*

The GDS-overlay flow requires DEF/LEF including standard-cell areas and matching full-chip GDS data. Prepare the DEF by editing it to add an instance named `<topcell>_APACHECELL` in the COMPONENTS section, where `<topcell>` is the top cell in the GDS.

Use the two-step flow shown in **[Figure 2: SoC substrate analysis in RedHawk].** First, use Totem-CSE to prepare the GDS data. Then include the data in the Redhawk-CSE run by specifying it with `GDS_CELLS` and setting `SUBSTRATE_GDS_OVERLAY 1`. CSE will disable automatic well generation in standard-cell areas and merge the GDS geometry and with the LEF/DEF layout.

Note that Totem-CSE models well taps in detail. If the overlaid GDS includes diffusions creating well taps, by default Totem includes all of them. To reduce memory and runtime, discard diffusion in logic and memories, instead of relying on automatically generated taps. CSE will generate taps based on tap cells in standard-cell areas and on well shapes in ERV memory models.

SUBSTRATE_MODIFY_BULK_LAYER_DEPTH <thickness, um>

Use to model designs with thinned substrates. Overrides the bulk thickness given by the substrate technology file, including when you use an encrypted .subtech. *Optional. Default: none.*

Specifies the actual wafer thickness to model the substrate, which changes the depth of the bottom of the lowest substrate bulk layer. You may use any depth greater than the greatest depth of all non-bulk layers.

For example, to model a thinned substrate, you may use the standard substrate technology file for the process, using this setting to model the thinning. Physically, some of the deepest bulk layers must remain after thinning. Therefore, you may not thin a CMOS triple-well wafer so much that the bottom of deep n-wells is exposed.

SUBSTRATE_GRID <grid size, um>

Use to experiment with the effect of substrate x/y discretization, for example, when resistance between metal wires connected through substrate features is too high. *Optional. Default: 10um.*

CSE creates a 3D mesh (or grid) of resistors to model the substrate. It adapts the lateral mesh based on substrate features and diffusion in the design, to keep the model size manageable for blocks or a full chip while producing accurate results in most situations. In “blank” areas, including entire deep bulk layers in most cases, it uses a default grid size.

Use this setting to force CSE to use a smaller (or larger) default mesh size. Remember that the number of resistors in the resulting model roughly equals the square of the mesh size, so expect runtime and memory to increase dramatically for small mesh sizes.

See also SUBSTRATE_Z_DISCRETIZATION_POINTS, which forces additional vertical discretization.

Obsolete settings

Use **SUBSTRATE_LIBRARY_FILE** instead of these GSRs:

SUBSTRATE_STDCELL_DIFF_PORTION

TAP_CELLS

Do not use these settings. Improvements to CSE have superseded them:

SUBSTRATE_BULK_PORTION

SUBSTRATE_CAP_CURRENT_MAP

SUBSTRATE_CELL_NWELL_PORTION

SUBSTRATE_EPI_PORTION

SUBSTRATE_MERGE_WIRE

SUBSTRATE_SMART_GRID_BIN

SUBSTRATE_SMART_GRID

SUBSTRATE_WIRE_SLICE_DEPTH

SUBSTRATE_WIRE_SLICE_DEPTH

WIRE_SLICE_MIN_DIM

WIRE_SLICE_WIDTH

Chapter 14
Totem™ Standard Cell EM Checks

Introduction

As designs move to smaller technology nodes, EM verification within the standard cells becomes more critical for ensuring the robustness of the standard cells over several SOC designs. Unlike timing characterization, EM validation is dependent on supply voltage, slew-load and also frequency at which the cell is switching. Also, the standard cells can have several operating models and worst case scenarios, coming up with sample space for EM characterization is non-trivial and may not be able to capture all corner cases. Hence an automated approach for validating EM on standard cells is proposed here.

The flow has 2 approaches:

- Totem Vectored EM check that provides spice-level accuracy for the standard cell reliability checks such as power EM, signal EM, and optionally thermal / self-heating feedback.
- Totem Vectorless EM flow that is fast, heuristics based, depends on user inputs like activity, slew etc. This flow could be performed to get early results

This chapter describes the different steps involved in setting up the Totem Standard Cell Check (SCC) flow. It will provide a flow description, key inputs required and templates for setup and analysis.

Vectored ttemcheck flow(Beta Version)

Totem can now support vectored ttemcheck flow(Beta version). Overall flow is similar to the vector-less run, main differences being:

- User needs to run FXM flow that takes DSPF, lib etc as inputs and generates test-bench for various combinations of functional-arcs/loads/slews etc. Contact ANSYS AE for details.
- Point the test-bench directory in ttemcheck config file using “`TEST_BENCH_DIR <path>`”.
- Run ttemcheck with additional switch “-vec”.

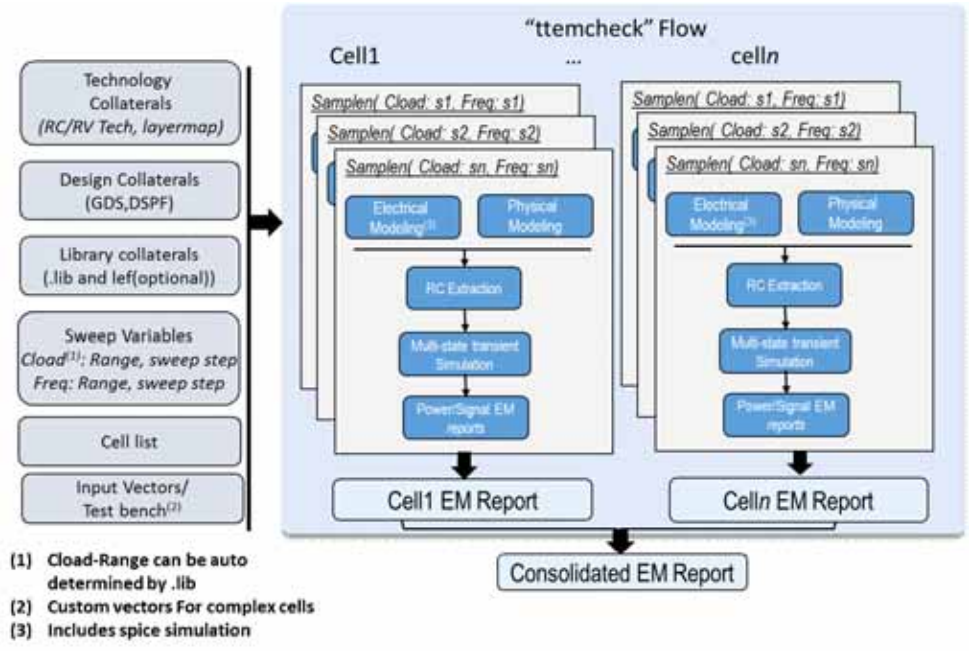


Figure 14-43 Vectored ttemcheck flow

The main features include:

- A comprehensive slew/load/frequency sweep for all timing arcs
- Multi-threaded with distributed computing support
- Support for thermal aware EM sign-off
- Enables EM limited Max-cap tables for SOC level PnR environment

For more details, please contact Ansys AE.

The flow can be broadly classified into two steps.

(1) **Vector generation** for all slew/load and timing arcs for the standard cells in the library
(2) **EMIR and SelfHeat analysis** for the standard cells, which also includes running spice simulations to capture the device currents and temperature using spice-testbench generated in step (1)

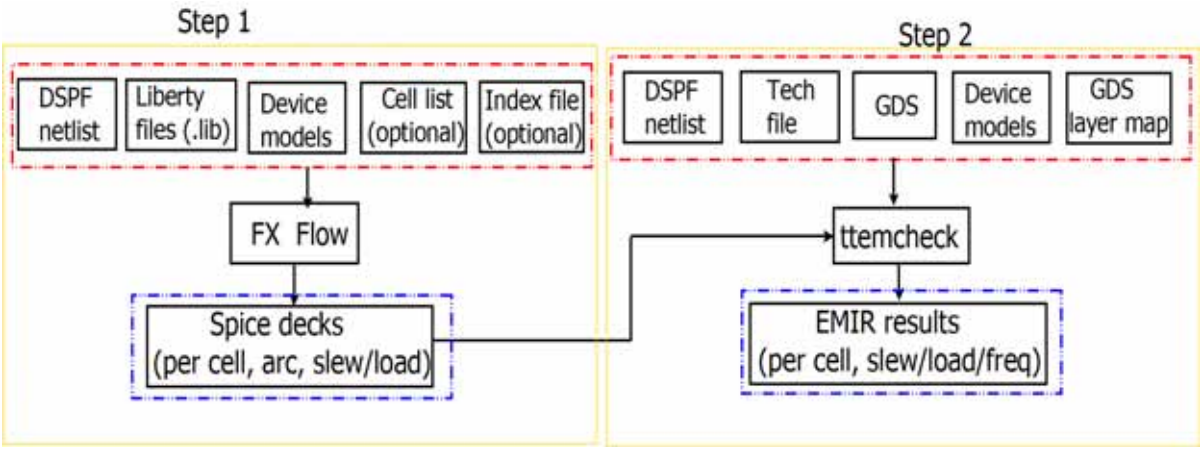


Figure 14-44 Overview of Totem SCC tool

The following inputs are required for running the flow.

Inputs	Description
Tech files	Totem tech file with RC parasitic, stack-up and EM limits
GDS/GDS layer map	Metal/via layer name and number along with Booleans for base layers
Device models	Spice models along with appropriate model-cards for functional simulation and SelfHeat analysis
DSPF map	**Optional** contains metal layer LVS layer name/index to tech layer name mapping
Liberty Files (.lib)	Used to generate the slew/load/timing-arc combinations
DSPF netlist	Used for spice simulation and Totem analysis
Cell list	File containing the name of the cells to be analysed
Index file	Contains the description of how load/slew indexing is applied to cells

Step1: Vector generation

Figure 14-44 shows the overview of Totem SCC tool. The totem_fx will take the devices models, DSPF netlist and the liberty files to generate the spice decks. The timing information and arc will be created based on the information in the liberty files. However, if the user wants to analyze only few cells, then cell list file is required. Cell list file consists of cell names to be analyzed. If there is no cell list file, then all the cells in the liberty file will be analyzed.

Commands to execute fx step:

```
source run.steps
```

Configuration Files:

In order to successfully run the totem_fx step, we need the following scripts and files. Some of these steps are subsequently explained. Please contact Ansys AE for more details

1. Input_files.tcl
2. Configuration.tcl
3. File containing device model files (ffgnp.inc)

4. Makefile
5. Cell list (optional)
6. FX_expert_guidance.py
7. run.steps

Files	Comments
Input files.tcl	Contains the pointer to the library file, DSPF, spice model file (Template: Fx_release_area/starter_scripts/FXM_char/input_files.tcl)
Configuration.tcl	Contains machine details, worker launch command, queues, cell list file, index file, etc. (Template:Fx_release_area/starter_scripts/FXM_char/Configuration.tcl)
Makefile	Setup the location of the FX release as well as the LSF/GRID launch command and queue (Template:Fx_release_area/starter_scripts_FXM_char/)
FX_expert_guidance.py	For reporting the error messages (available in Installation directory)
Run.steps	Contains all the commands to generate the spice decks (Template:Fx_release_area/starter_scripts/FXM_char/run.steps)
Cell_list (optional)	Contains the name of the standard cells need to be analyzed

Table 14-2 FX input files and Significance

<input_files.tcl> ----------

```
set library_file  <>
# pointer to the liberty file

set spice_netlist_dir   <>
# pointer to the DSPF netlist

set corner_spice_model_file  <models.inc>
# pointer to the device model file

set configuration_file   configuration.tcl
set initialization_mode      1/0
# set initialization mode 1 --→ only one cell will be
characterized. If set initialization mode 0 ---→ all the cells in
the cell will be characterized.
```

Note: The file which contains device model files should also contain the self-heat device model file if the thermal analysis is enabled. If the self-heat model files are not included in the thermal analysis file, then XML files cannot be generated and there will be a failure in spice simulations. An example of the file which contains the device model files is shown below:

<models.inc>----------

```
.lib  '../../../  models.l' <model_card>_
..
```

==========

<configuration.tcl> ----------

```
set queue "<>"
# queue to use. Ex: set queue “sjo_all_hosts”

set worker_launch_command "qsub -sync y -V -b y -cwd -j y -q
$queue -N fxm.%id -o $working_dir/worker.%id.out %worker %addr --
logfile $working_dir/worker.%id.log --queue_license"

# GRID AUG Example:
# set worker_launch_command "qsub -sync y -V -b y -cwd -j y -q
$queue -N fxm.%id -o $working_dir/worker.%id.out %worker %addr --
logfile $working_dir/worker.%id.log --queue_license"

# LSF Example:
# set worker_launch_command "bsub -K -q normal -W 167:59 -R
\"select\[type=='LINUX64' && mem>=3500\] rusage\[mem=3500\]\" -v
32000000 -o $working_dir/worker.%id.out %worker %addr --logfile
$working_dir/worker.%id.log --queue_license"


#=============================================================
#### FXM CHAR SETTINGS
####=============================================================
#option
set spice "fxgen"
## this is to generate accurate spice deck for cells having
resistors and diode
set_option build_fxm_spice_for_diode_polyr hspice
## USER MAY NEED TO DEFINE, if not in search path as hspice
set_option cell_arc_sim_hspice_command "/appls/synopsys/I-
2013.12/hspice/amd64/hspice -i %s -o %s.out"
set_option fxm_hspice_command  "/appls/synopsys/I-2013.12/hspice/
amd64/hspice %spi -o %spi.out"

#=============================================================
#### USER DEFAULTS
####    CAN BE OVERRIDED BY PASSING ARGS VIA MAKE
####    make ARGS=" .....  "
####    -char_type      : Valid inputs are:   corner, statistical
or both
####    -num_worker     : Integer number of workers desired.
####    -celllist_file  : File containing list of cells
####=============================================================
```

```
#
set char_type         corner
set num_workers       20
### number of workers
set celllist_file     <cell.list>
### pointer containing the file which contains the cell to be
characterized. If this command is commented, then all the cells in
the library file will be analyzed.
```

An example of the cell list file is shown below. Cell list should provide the names of the standard cells which need to be analyzed.

<cell.list> ----------

```
## Names of the standard cells to be analyzed
inv2x1
clkbuf2x1
mbff8x2
```

Following steps must be followed in running the FX step:

Step 1:

Copy Makefile from FX installation area and modify appropriately.

Make file Template

```
RELEASE = <FX release area>
export QUEUE = " sjo_all_hosts “
# If the user wants to run in local machine, this command can be commented out
export SUBMIT = qsub -N PathFX -V -sync y -b y -j y -q $(QUEUE) -pe pe_smp 2 –cwd
# If the user wants to run in local machine, this command can be commented out
export META_QUEUE = 1
export NUM_CPUS = 4
BUILD_FXM      := $(RELEASE)/bin/totem_fx
ANSYS_PY       := $(RELEASE)/bin/clkda_python
EXPERT         := $(RELEASE)/FX_expert_guidance.py
SCRIPTS        := ./scripts
```

Additional Makefiles are supplied for:

Makefile.threaded_grid (For threaded operation)

Makefile.distributed (For distributing the jobs in bigger machines)

Makefile.distributed_grid (For distributing and threading the jobs in bigger machines)

*_grid versions apply the SUBMIT command as provided by user.

Update all of the following environment variables in MakeFile:

RELEASE -- Path to install.

QUEUE -- Grid to queue to use.

SUBMIT -- Grid launch command.

NUM_CPUS -- Number of cpus running threaded. build_clkdb always runs threaded.

Step 2:

Please update all parameters in the input_files.tcl

Step 3:

Please update below TCL variables in configuration.tcl
worker_launch_command
queue

Step 4:

If necessary, update the celllist. Default is to char/sim all cells in library.
When running in intialization_mode, only the FIRST cell is used for quick validation of flow.

Step 5:

run.steps has below steps.

```
make build_fxm                  # generating fxm models
make build_clkdb                # generate clkdb
make TTMCheck_spice_gen         # generate spice decks
```

Source run.steps or ./run.steps to characterize the cells and create the vectors for the spice simulation.

Output of FX step:

Once the run is complete, user will see a **"database"** directory. Database directory consists of **TTMCheck_spice_gen**, **build_clkdb**, **build_fxm** and **data** sub directories. **TTMCheck_spice_gen** consist the vectors in the **validate_dir** sub directory.

In other words, user need to navigate to the following directories to see the generated spice vectors:

Database--→TTMCheck_spice_gen--->validate_dir--→ <spice vectors>

User can look at the stage_summary.log to see whether fx steps is successful or not.

Step2: EMIR and SelfHeat analysis

In step 2, probes are generated, and the spice simulations will be performed on the vectors (spice decks) generated in the step 1. The generated current waveforms for each vector (spice decks) is analyzed using ttemcheck flow. Output of the step 2 generates the EMIR results for each chosen corner with respect to slew/load and frequency. In step 2, we will also get a comprehensive report of number of EMIR violation in each cell. Further, the tool will dump out a library file which contains the allowable maximum capacitance for the given slew and frequency for each cell. If the thermal options are enabled, then the Thermal Resistance File (TRF) is generated for each cell with the worst Rth (thermal resistance) value being reported.

Command to execute ttemcheck step (without thermal options):

```
ttemcheck -l cell.list -c ttemcheck.cfg -vec
```

where cell.list contains the cell list of all the standard cells to be analyzed and ttemcheck.cfg is the configuration file used in step 2.

In summary, FX step will take the inputs to generate the spice decks and the ttemcheck will perform dynamic analysis on the spice decks generated. Since we are running the spice simulations for analysis, this tool has a spice level accuracy in identifying the EMIR bottle necks for each standard cell.

In order to successfully run the ttemcheck step, we need to have the following files.

1) Config file (ttemcheck.cfg).
2) Cell list
3) Spice decks (generated from the fx step)

<ttemcheck.cfg > ----------

```
DSPF_NETLIST  <>
## Pointer to the DSPF file
DEVICE_MODEL_LIBRARY <>
## Pointer to the device model library file
DSPF_MAP <>
## Pointer to the DSPF map file
GDS_FILE <>
## Pointer to the GDS file
GDS_MAP_FILE <>:
## Pointer to the GDS map file
LIB_FILE <>
## Pointer to the liberty file
……..
TECH_FILE <>
## Pointer to the tech file
……
SAVE_TOTEM_DB 1
## Saves the totem db after the ttemcheck step
……..
FREQ_SAMPLE  <freq1> <freq2>
## Provide the frequency samples at which simulation needs to be
performed
SPICE_SIMULATOR <spice>
## Pointer to the spice simulator
TEST_BENCH_DIR ../ totem_fx/spice_decks
## Pointer to the spice decks generated from the totem_fx step
```

Once the all the required options are enabled and all the required files are pointed in the configuration file (ttemcheck.cfg), then ttemcheck step will be executed. _

Output directory structure will be as follows for the ttemcheck step:

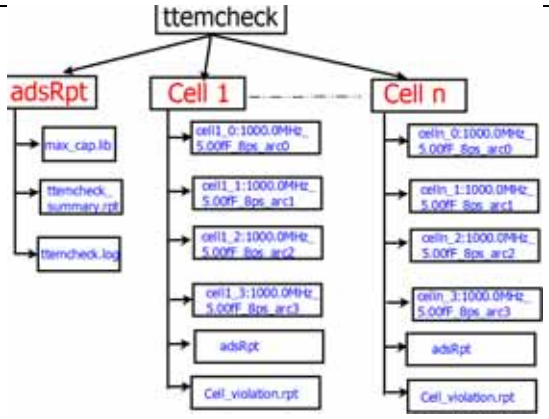


Figure 14-1 Output directory structure of the ttemcheck step

The output directory of the ttemcheck contains the adsRpt directory and the directory with the cell names is shown in Figure 14-1

- adsRpt directory contains the max_cap.lib, ttemcheck_summary.rpt, and ttemcheck.log subdirectory.
  - ttemcheck.log file contains the information about the run and can be used to debug, if there is an unsuccessful run.
  - Max_cap.lib provides the maximum allowable load capacitance across the standard cell without EM violation for the given frequency and input transition time (slew).
    - A sample max_cap.lib file is shown below:

```
library (em_limit_table) {
time_unit : 1ns ;
capacitive_load_unit(1, pF);
em_lut_template (output_by_freq_and_trans) {
  variable_1 : frequency ;
  variable_2 : input_transition_time ;
  index_1 ("1.0, 2.0");
  index_2 ("1.0, 2.0");
}
cell (inv2x1) {
  pin (o) {
    electromigration () {
      related_pin : "i0";
      when : "i0";
      max_cap(output_by_freq_and_trans) {
        current_type : "average";
        index_1 ("100000000.0000, 300000000.0000");
        index_2 ("0.0080, 0.2480");
        values ( \
          "2.22507e-3, 2.22507e-3", \
          "2.22507e-3, 2.22507e-3" \
        );
      }
      max_cap(output_by_freq_and_trans) {
        current_type : "rms";
        index_1 ("100000000.0000, 300000000.0000");
        index_2 ("0.0080, 0.2480");
        values ( \
          "2.22507e-3, 2.22507e-3", \
          "2.22507e-3, 2.22507e-3" \
        );
      }
      max_cap(output_by_freq_and_trans) {
        current_type : "peak";
        index_1 ("100000000.0000, 300000000.0000");
        index_2 ("0.0080, 0.2480");
        values ( \
          "2.22507e-3, 2.22507e-3", \
          "2.22507e-3, 2.22507e-3" \
        );
      }
    }
```

- From the above file, we can see that for frequency of 1GHz and the input transition of 0.08ns, the maximum allowable capacitance without average EM violation is 2.22e-3 pF.

- ttemcheck_summary.rpt will provide a high level summary report for all the standard cells analyzed. A sample ttemcheck_summary report is shown below.

```
# <cell_name> <cload_range(fF)> <?input_slew_range(ps)?> <freq_range(MHz)>/<toggle_rate(Sig/Clk)> <powerEM_status> <signalEM_status>
inv2x1  1.25-1.25  8-8  1800.00-1800.00  PASS  PASS
clkbuf2x1  12.50-12.50  8-8  1800.00-1800.00  FAIL  FAIL
mbff8x2  0.62-0.62  8-8  1800.00-1800.00  PASS  PASS
```

- In the ttemcheck_summary report, PASS means, there is no average, rms or peak power or signal EM analysis. FAIL indicates that there is atleast one average or rms or peak power or signal EM analysis. In the above case, the cell “clkbuf2x1” has atleast one average or rms or peak power and signal EM analysis

Along with the adsRpt directory, directories of each standard cell to be analyzed is created.

- A directory for every standard cell analyzed and sub-directory for every sample for the cell will be generated as follows:

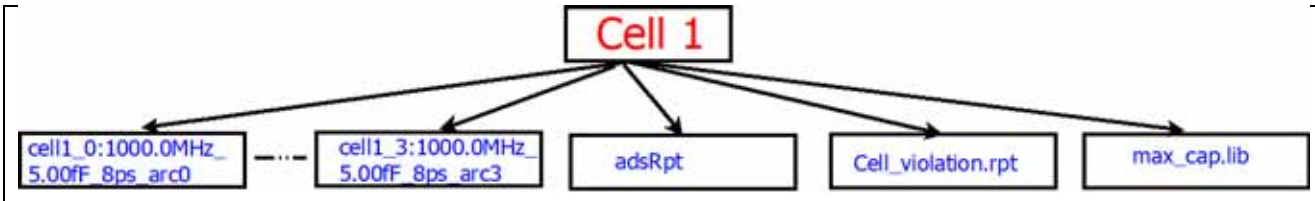


Figure 14-2 Output directory of each standard cell

As seen from Figure 14-2, each standard cell will consists of the sub-directories with each corners analyzed and the adsRpt, cell_violation.rpt and max_cap.lib.

- adsRpt contains the information about the aplmmx run. If there is any issue in the probe generation, the user can debug the aplmmx by looking log file available in the adsRpt directory.
- Cell_violation.rpt provides a detailed summary report for each load, slew, frequency and arc combinations along with the database for each combination of that particular standard cell. An example cellviolation.rpt is shown below:

From the below table, user can get an idea of for which load, slew, frequency and arc combinations, the standard cell is having the EM violations. Further, the user can also see the number of average, rms and peak EM violations happening. P in the above report represents the Power EM while S represents the Signal EM.

Any cell that has a violation for a particular EM mode and a particular Freq,Cload sample can be obtained from the file Cell_Violation.rpt. This file also has the corresponding path for the totem vector based EM which can be loaded in Totem GUI for further debug.

```
#<cell_name> <EM_mode> <(CLoad(fF), Freq(MHz), InputTransition(ps), arc)> <avg_vio> <rms_vio> <peak_vio> <dir>
mbff8x2 P (40.00, 1800.00, 248, arc0) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_0:1800.00MHz_40.00fF_248ps_arc0/EM
mbff8x2 S (40.00, 1800.00, 248, arc0) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_0:1800.00MHz_40.00fF_248ps_arc0/EM
mbff8x2 P (40.00, 1800.00, 248, arc1) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_1:1800.00MHz_40.00fF_248ps_arc1/EM
mbff8x2 S (40.00, 1800.00, 248, arc1) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_1:1800.00MHz_40.00fF_248ps_arc1/EM
mbff8x2 P (40.00, 1800.00, 248, arc2) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_2:1800.00MHz_40.00fF_248ps_arc2/EM
mbff8x2 S (40.00, 1800.00, 248, arc2) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_2:1800.00MHz_40.00fF_248ps_arc2/EM
mbff8x2 P (40.00, 1800.00, 248, arc3) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_3:1800.00MHz_40.00fF_248ps_arc3/EM
mbff8x2 S (40.00, 1800.00, 248, arc3) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_3:1800.00MHz_40.00fF_248ps_arc3/EM
mbff8x2 P (40.00, 1800.00, 248, arc4) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_4:1800.00MHz_40.00fF_248ps_arc4/EM
mbff8x2 S (40.00, 1800.00, 248, arc4) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_4:1800.00MHz_40.00fF_248ps_arc4/EM
mbff8x2 P (40.00, 900.00, 248, arc0) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_5:900.00MHz_40.00fF_248ps_arc0/EM
mbff8x2 S (40.00, 900.00, 248, arc0) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_5:900.00MHz_40.00fF_248ps_arc0/EM
mbff8x2 P (40.00, 900.00, 248, arc1) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_6:900.00MHz_40.00fF_248ps_arc1/EM
mbff8x2 S (40.00, 900.00, 248, arc1) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_6:900.00MHz_40.00fF_248ps_arc1/EM
mbff8x2 P (40.00, 900.00, 248, arc2) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_7:900.00MHz_40.00fF_248ps_arc2/EM
mbff8x2 S (40.00, 900.00, 248, arc2) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_7:900.00MHz_40.00fF_248ps_arc2/EM
mbff8x2 P (40.00, 900.00, 248, arc3) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_8:900.00MHz_40.00fF_248ps_arc3/EM
mbff8x2 S (40.00, 900.00, 248, arc3) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_8:900.00MHz_40.00fF_248ps_arc3/EM
mbff8x2 P (40.00, 900.00, 248, arc4) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_9:900.00MHz_40.00fF_248ps_arc4/EM
mbff8x2 S (40.00, 900.00, 248, arc4) 0 0 0 /scratch0/UserRuns/dselvaku/SCC/ttmstdchk/ttemcheck/mbff8x2/mbff8x2_9:900.00MHz_40.00fF_248ps_arc4/EM
***MARK_END***
```

- As explained above, max_cap.lib will provide the maximum allowable load capacitance across the standard cell without EM violation for the given frequency and input transition time (slew) for the particular standard cell.
- Further, each corner of the standard cell will have the following output directory structure.

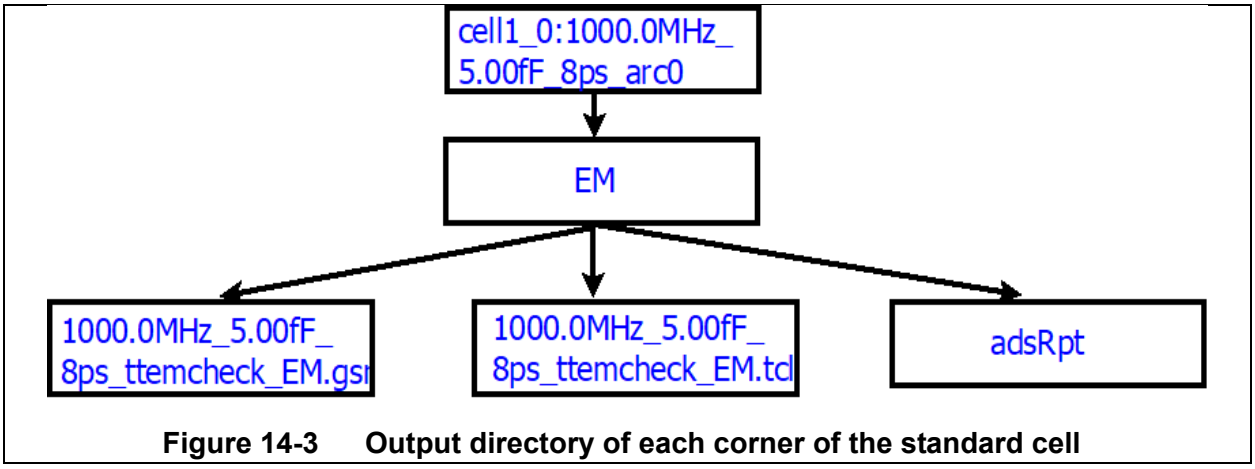


Figure 14-3 Output directory of each corner of the standard cell

From the Figure 14-3, we can see that each corner of the standard cell will have a gsr file, tcl script and adsRpt directories. Now user, can run individual totem run they are interested to load the GUI and look in to the results. If "**SAVE_TOTEM_DB 1"** in the ttemcheck.cfg file, then the Totem db will be saved along with the gsr, tcl scripts.

Usage model 1:

Freq scaling flow:

To enable the frequency scaling flow, following keyword must be added in the ttemcheck.cfg.

SCALE_FREQ 1

In this flow, aplmmx -n will run to decide maximum frequency (against voltage switching check). Totem will try to find out the maximum frequency for each sample. Once the max frequency is determined, tool will scale the current based on the frequency ratio (lower_freq/max_freq) for the remaining lower frequencies and will perform the emcheck iteratively.

When thermal options are enabled in this flow, “perform selfheat_analysis” will be performed for each frequency.

Usage model 2:

Max cap flow:

User can enable the max_cap flow by adding the following keyword in the ttemcheck.cfg

REPORT_MAX_CAP 1

In this flow, max-cap is selected which passes for all em check (average/rms/peak).

LogN on number of CLOAD using binary search algorithm.

Note: Usage model 1 and 2 cannot be used at the same time

Usage model 3:

Thermal Flow:

Apart from the basic settings to run the Totem SCC flow as discussed above, user can also enable thermal settings to generate the Thermal Resistance File (TRF).

```
## The following commands need to be set in ttemcheck.cfg file if
the user wants to generate the thermal resistance file (enabling
the thermal options). If the user didn’t enable the thermal
options, then the following commands are not necessary.
      THERMAL_ANALYSIS 1
      THERMAL_PARAMETERS {
      THERMAL_EM_MODE RMS
      SH_FILE <>
 ## Pointer to the self-heat file provided by the foundry
      THERMAL_FLOW 1
      GENERATE_RTH 1
      }
```

If the user wants to enable the thermal flow, the above commands need to be added in the ttemcheck.cfg.

Command to execute ttemcheck step with thermal options:

```
ttemcheck -l cell.list -c ttemcheck.cfg -vec -thermal -samplebased
```

User needs to make sure that the device model files should consist have self heat model. Once the thermal options are enabled, then the worst_Rth.rpt (worst thermal resistance report) will also be generated in the adsRpt directory.

Note: Usage model 2 (max_cap flow) and usage model 3 (freq_scaling flow) cannot be used at the same time

LSF environment setting:

User can enable the qsub or bsub options in the ttemcheck flow to submit the jobs to the bigger machines.

If the user needs to enable the qsub options, the following command need to be added in the ttemcheck command.

Syntax:

```
ttemcheck -l cell.list -c ttemcheck.cfg -vec -qsub -j
number_of_totem_license_available> ?-sim_lic
<number_of_spice_license_available> ?
```

For example:

```
ttemcheck -l cell.list -c ttemcheck.cfg -vec -qsub -j 10 ? -simlic
3 ?
```

In the above example, number of totem licenses available is 10 and the number of spice simulation available is 3.

Debug files:

After the ttemcheck stage is done, user can look at various files in case some some error is encountered. Some of the files which can be used for debug are:

adsRpt/ttemcheck.log – This is the log files which includes all the steps which are executed and the warning and errors encountered during a particular stage.

adsRpt/MMX/Ttemcheck/ttemcheck.error.<date and time> - This file contains all the errors that are encountered during the run at the particular date and time.

adsRpt/MMX/Ttemcheck/ttemcheck.warning.<date and time> - This file contains all the warnings that are encountered during the run at the particular date and time.

<cellname>/adsRpt/aplmmx.log- This file contains the probe generation failure occurred in the run.

<cellname>/spice/<cload_slew>/<arc>/*.lis (hspice)- This file contains information about the failure happened during the spice simulation.

<sample_directory>/adsRpt/totem.log – This file contains information about the Totem failure.

Totem Vectorless Standard Cell EM Checks Flow

The “ttemcheck” wrapper flow built on top of Totem engine enables automation of EM validation on entire standard cell library. It gives a variety of reports from high level audit reports to cell-specific EM reports with the flexibility to load the results in Totem-GUI on demand basis. Totem provides visualization to debug the results on net by net basis and review the device currents to root-cause the violations. This flow provides the following advantages:

- Spice independent EM analysis on entire standard cell library
- Accurate results based on device,net-attributes, taking into account worst-case conditions
- Integrated debug cockpit to visualize and root-cause EM & IR violations

Key inputs and Usage

The key inputs to the vectorless power flow are as follows:

Input	Description
Technology file	Regular Totem tech file
GDS / GDS layermap	Layout information along with ID mapping
DSPF netlist	Used to identify the PMOS/NMOS stack, and device x,y co-ordinates
spice/spectre netlist	simulation netlist(optional)
Device model library	Used to characterize the gate cap for receivers
IDSat table	Provides the max current of devices for pull-up/dn channels
Liberty Files	.lib files for the standard cell library
Device Model table	Optional, but available for table-lookup based IDSAT/ Cap estimation

Table 14-1 The key inputs

The flow basically takes the above mentioned inputs creates a sample space for every standard cell in the library, it creates the setup, runs through extraction and performs the vectorless signal EM analysis and power EM analysis to identify any EM violations. It is also used to identify the Fmax or the maximum frequency at which the cell can operate without any EM issues. It gives out a consolidated audit report for library teams to verify the overall coverage on the standard cell library and also gives detailed EM violations on every cell and sample. It has an option to save the database on demand basis for end users to load, visualize and root-cause the EM bottlenecks in the standard cells and fix the same using the powerful Totem GUI. The syntax is:

```
ttemcheck -l cell.list -c ttemcheck.config –m [sig|pwr]
    -j <no. of threads> [-lmwait <min> ] -bsub
```

-l <cellList> : List of cells to be analyzed. Default, all cells present in lib files will be analysed.

-m [sig|pwr] : It controls either to do signal EM or power EM analysis. By default, both will be done.

-j <no. of threads> : allows user to set the thread numbers to launch Totem. Default, half of the Physical core. Maximum allowed thread number is equals to physical cores.

-bsub : option to use bsub command for cell based analysis internally

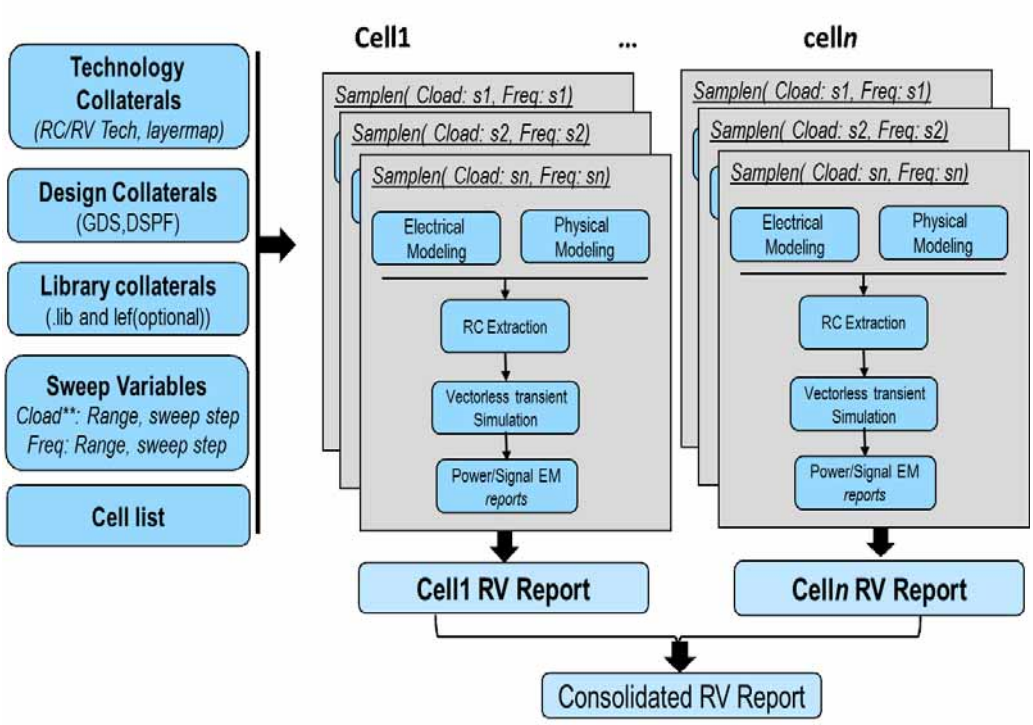


Figure 14-1 Vectorless ttemcheck flow

Totem can now verify EM on std-cells. Details are given below:

- Cload range is obtained from liberty file or user specified range
- Frequency samples are provided using the keyword FREQ_SAMPLE.
- Total no. of samples for a cell will be #(freq_samples) x #(Cload_samples).
- Current is modelled according to the vectorless power EM/ Signal EM flows already in place.

Sample Configuration File <ttemcheck.config>:

```
# if jobs are being fired in bjobs
BSUB_CONFIG_FILE <file_name>
#########################
# Global Settings#
#########################
VDD_PIN_NAME VDD
GND_PIN_NAME VSS
VDD <voltage>
TEMPERATURE <>
TECH_FILE <>

# Wild-card support is available for spice/dspf netlist.
# the file names should be <cell_name>\.<extension>

DSPF_NETLIST {
    # ../design_data/spice/*.cir
    <netlist>
```

```
}

# Wild-card support is available for GDS files.
# the file names should be <cell_name>\.<extension>

GDS_FILE {
    # ../design_data/spice/*.gds.gz
    <gds_files>
}

# Library Files
    LIB_FILE {
    <liberty_files>
}

# User inputs for cells not having lib file (optional)
STD_CELL_CONFIG_INFO <cell_specific_config>

# Optional inputs
EM_REPORT_PERCENTAGE 85
EM_REPORT_LINE_NUMBER -1
DELTA_T_RMS_EM 5

GDS_OPTION {
    LEF_FILE <lef>
    USE_TOP_LEVEL_TEXT_ONLY 1
}

# Frequency Samples(explicit values)
FREQ_SAMPLE <freq1> <freq2> <freq3>

## OR
## Frequency range
## N → No. of Samples
## VAL_i → Fraction of frequency specified in FREQ keyword
FREQ_SAMPLE_RANGE N <VAL_1> <VAL_2> …..<VAL_N>
FREQ <>

# Cload Samples(explicit Cload values)
CLOAD_SAMPLE <load1> <load2> <load3>
## Cload Samples
## N → No. of Samples
## VAL_i → Ratio of Max Cload from lib to MAX_CLOAD
setting in config
CLOAD_SAMPLE_RANGE N <VAL_1> <VAL_2> …..<VAL_N>
# Cload samples(optional), default from lib file

##Note: MAX_CLOAD usage
## 1. Priority: MAX_CLOAD(cell)> max_capacitance(lib)>
MAX_CLOAD(global)
## 2. If MAX_CLOAD(cell) is larger than
```

```
max_capacitance(lib), use     max_capacitance(lib) as
upper-bound
## 3. If MAX_CLOAD(cell) is smaller than the minimum
capacitance in lib file, tool will ignore lib value
MAX_CLOAD max_capacitance

# Toggle rate(optional,default 1 2)
TOGGLE_RATE <signal> <clock>
## Toggle-rate sweep(instead of Freq Sweep)
TOGGLE_RATE_SAMPLE {
    sigTR1 clkTR1
    sigTR2 clkTR2
    .
    .
    sigTRN clkTRN
}

## Additional aplmmx options
SEM_WST_ANALYSIS 1

# Spice libraries
DEVICE_MODEL_FILE <>
PMOS_MODEL_NAME <MODELNAMES>
NMOS_MODEL_NAME <MODELNAMES>

# DEVICE MODEL TABLE
DEVICE_MODEL_TABLE <>

# To save totem database for every sample
SAVE_TOTEM_DB [1|0]
```

BSUB config file

```
#BSUB option1 …
#BSUB option2 …
```

Cell specific Config file

```
# Support wild-card in cell-names

CELL <cell_name> {
    FREQ_RANGE <freq1> <freq2>...<freqn>
    TOGGLE_RATE <signal_TR> <clock_TR>
    # to support cell-specific capacitive loads
    CLOAD_SAMPLE <cload1><cload2>...

    # to override lib file info or to provide missing info
use the following options
    VDD_PIN_NAME <>
    GND_PIN_NAME <>
    VDD <>
```

```
    EM_NET_INFO_CELL {
        ## Net specific attributes (like EM_NET_INFO)
        # slew_range 0.8
        # <net_name> <trans_time_sec> <toggle_rate>
        #<freq_in_Hz> <uni-dir_scale> # <bi-dir_scale>
        #<extra-cap>
        CLK* 50e-12 2 2e09 1.3 - - -
    }
    #Custom PLOC for power EM flow (default: use LEF pin)
    PAD_FILES <user_ploc>
}
```

Device Model Table File:

Syntax is :

```
DEVICE_MODEL_TABLE <device model look-up table>
```

The look-up table file contains the following

```
<device_model1> {
    # Each and every W/L combination can have multiple
#lines based on different Parameters (VGS/VDS/NFIN)
    WL <W> <L> {
        VTH{<vth>} CGATE{<cap_in_F>} CSOURCE{<cap_in_F>}
        IDSAT{I_in_A}     RON{<cap_in_F>} ESR{<R_in_ohms>}
        ESC{<cap_in_F>} CDRAIN{<cap_in_F>} VGS{<volts>}
    }
}
<device_model2> {
    WL <W> <L> {
    ..
    }
}
```

Wildcard support is provided for device model name field. Interpolation/Extrapolation is also supported for device parameters. This can be used to make this flow completely spice-independent to reduce the overall TAT.

Output Directory Structure will be as follows:

A sub-directory for every cell and sub-directory for every sample for the cell will be generated as follows:

```
<cell_name>/

  |-<cell_name>_sample_id1:freq1_cload1

    |-powerEM/

    |-signalEM/

  |-<cell_name>_sample_id2:freq1_cload2

    |-powerEM/
```

```
|-signalEM/
```

Other Config file keywords:

CLOCK_PIN_FROM_LIB

When CLOCK_PIN_FROM_LIB 1 is set, ttemcheck will auto- identify clock vs non-clock nets and apply 2/1 toggle rate respectively. If no clock pin is found for the cell, every pin (including internal nets) will use toggle rate 2. If at-least one clock pin is found, clock pins will be assigned toggle rate 2 and other pins (including internal nets) will be assigned toggle rate 1. Default :0

Syntax:

```
CLOCK_PIN_FROM_LIB [0|1]
```

Output Reports

ttemcheck will dump out a high level audit report (adsRpt/ttemcheck_summary.rpt)

#<cell_name>	<cload_range(fF)>	<freq_range(MHz)>/<toggle_rate(Sig/Clk)>	<powerEM_status(failed/total_checks)>	<signalEM_status(failed/total_checks)>
CRIV_X12B_F12TL_C35	0.10-518.79	180.00-6000.00	ERROR(1/6)	ERROR(1/6)
CRIV_X6B_F12TL_C35	0.10-518.79	180.00-6000.00	PASS	FAIL
CRIV_X4B_F12TL_C35	0.10-518.79	180.00-6000.00	PASS	PASS

Also, a detailed cell-level report (<cell_name>/cellViolation.rpt) is generated.

```
#<cell_name> <EM_mode> <(CLoad(fF), Freq(MHz))> <avg_vio> <rms_vio> <peak_vio> <dir>
CRIV_X12B_F12TL_C35 P (0.10, 180.00)       0 0 72     ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_0:180.00MHz_0.10fF/powerEM
CRIV_X12B_F12TL_C35 S (0.10, 180.00)       0 0 0      ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_0:180.00MHz_0.10fF/signalEM
CRIV_X12B_F12TL_C35 P (104.73, 180.00)     0 12 93    ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_1:180.00MHz_104.73fF/powerEM
CRIV_X12B_F12TL_C35 S (104.73, 180.00)     0 9 52     ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_1:180.00MHz_104.73fF/signalEM
CRIV_X12B_F12TL_C35 P (518.79, 180.00)     2 39 93    ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_2:180.00MHz_518.79fF/powerEM
CRIV_X12B_F12TL_C35 S (518.79, 180.00)     3 43 65    ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_2:180.00MHz_518.79fF/signalEM
CRIV_X12B_F12TL_C35 P (0.10, 6000.00)      0 0 71     ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_3:6000.00MHz_0.10fF/powerEM
CRIV_X12B_F12TL_C35 S (0.10, 6000.00)      0 0 0      ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_3:6000.00MHz_0.10fF/signalEM
CRIV_X12B_F12TL_C35 P (104.73, 6000.00)    63 70 92   ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_4:6000.00MHz_104.73fF/powerEM
CRIV_X12B_F12TL_C35 S (104.73, 6000.00)    68 74 73   ./CRIV_X12B_F12TL_C35/CRIV_X12B_F12TL_C35_4:6000.00MHz_104.73fF/signalEM
```

Any cell that has a violation for a particular EM mode and a particular Freq,Cload sample can be obtained from the file adsRpt/cellViolation.rpt. This file also has the corresponding path for the totem vectorless EM which can be loaded in Totem GUI for further debug.

Also, .lib em table “em_max_toggle_rate” and “max_cap” are generated after ttemcheck flow. `em_max_toggle_rate` is a table with indices `‘input_transition’` and `‘output_load` and `max_cap` table has `‘frequency’` and `‘input_transition’` as the indices.





Support for multiple subckt in DSPF in ttemcheck flow

"ttemcheck" flow supports multiple subckt definition in single DSPF file. Tool will automatically use the respective subckt for cells defined in the cell list to be analysed.

To enable this feature, user need to use below ttemcheck.config file setting:

```
DSPF_NETLIST <dspf with multiple subckts>
STD_CELL_PRE_PROCESSING {
    DSPF_ONLY 1
    DSPF_OUTPUT_DIRECTORY <dir_name>   ## optional, default output
                                       directory is ./DSPF
}
```

If cell_list is specified, tool will internally split the input dspf file into multiple dspf (with single subckt definition) for specified cells, keep them under directory <dir_name>, and use it.

If cell_list is not specified, input dspf file will be split for all the cells and put under directory <dir_name>. Default <dir_name> is ./DSPF.

APLMMX Config file:

```
SPICE_SIMULATOR /appls/mentor/15.4/bin/eldo
```

Summary:

Standard cells form the basic building blocks for the complex SoC designs. EM verification within the standard cells becomes more critical for ensuring the robustness over several SOC designs. So, analyzing and understanding the EM violations at the standard cell level is very important. In this document, we are presenting Totem SCC to analyze and the understand the EM bottle necks at the standard cell level. Further, SCC utilizes the spice simulations to calculate the EM current carrying density of the metal wires and EM violations, there by providing the spice level accuracy. Further, SCC also have thermal enable options to generate the Thermal Resistance File which can be used for various purposes. In summary Totem SCC has the following advantages:

- Spice level accuracy for determining standard cell reliability with thermal feedbacks.
- Accurate results based on device,net-attributes, taking into account worst-case conditions.
- Integrated flow to have both power EM and signal EM analysis in one run.

Chapter 15
Totem™ CCI Flow

Introduction

The CCI Flow in Totem allows to use Calibre generated LVS data (CCI) to run Totem Static/Dynamic EMIR analysis and SignalEM as well as substrate analysis. CCI stands for Calibre Connectivity Interface, Totem-CCI flow uses Calibre generated CCI database instead of raw GDS and DSPF that used in regular Totem flow. Following analysis are supported in Totem CCI Flow:

- Static EMIR Analysis
- Dynamic EMIR Analysis
- SignalEM Analysis
- Substrate Analysis

Benefits of Totem CCI Flow

- Robust support for custom/analog PDKs
  - Supports all device types and device layouts
- No DSPF required for powerEM, coupled-C DSPF is enough for Signal EM, no need detailed DSPF for Signal EM.
- No need for gds map file with Boolean operation
- Robust support for custom geometry
  - Exact net identification and labeling
  - Exact device identification and labeling
  - Proven pin placement
- Improved name mapping
  - Exact mapping between source and layout netlists
- Support of regulated power nets
  - Automated handling of regulated xtor output as internal power domain.
- Support of selective metal resistor tracing
  - Trace or not trace through specific metal resistor instances.

Flow Overview

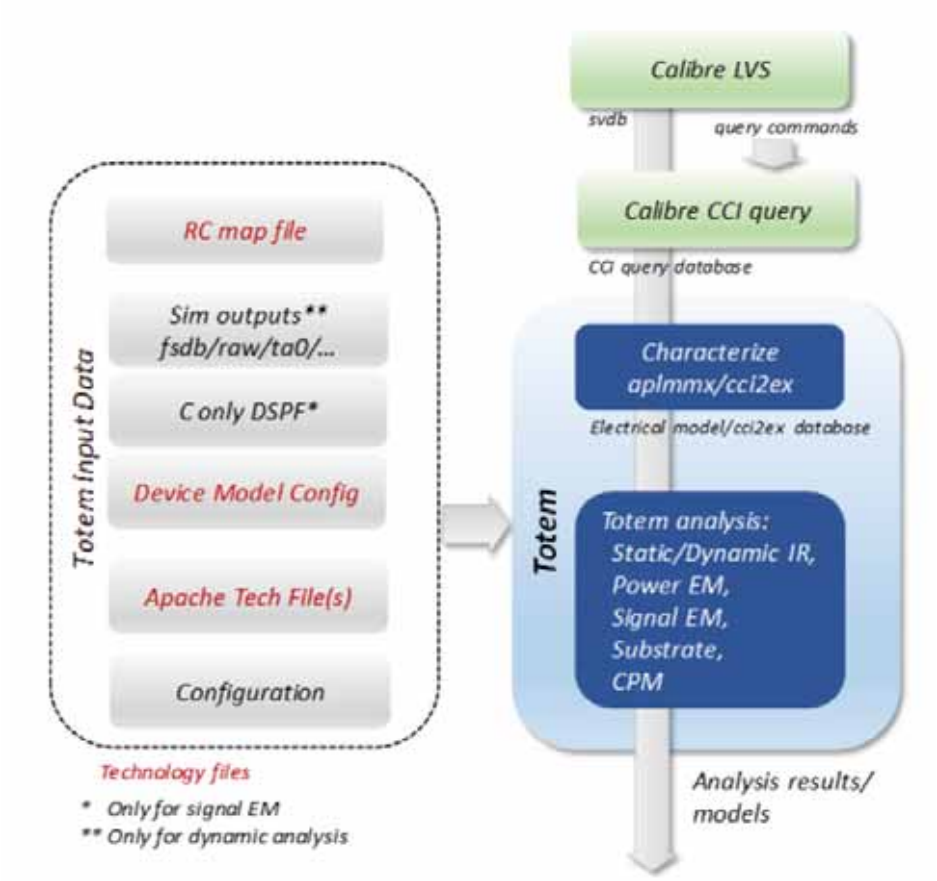


Figure 15-1 Flow Diagram

Totem CCI Flow - Input Requirements

Totem uses industry standard OADB input data for analyzing power noise and reliability on analog and mixed signal designs. The inputs for Totem in OADB Flow are as follows:

Inputs	Information	Flow	Comment
Apache Tech file	Layer stack(metal,via,base) with parasitic info, can be derived from itf/nxtgrd	All	Must
Substrate Tech file	physical and electrical characteristics of substrate features	Substrate analysis	Must
CCI database	LVS database from Calibre	All	Must

RC Mapping file	Mapping file w/ LVS layer-name to Apache Tech layer name mapping	All	Must
C-only DSPF	With signal net capacitance	Signal EM	Highly recommended
Simulation netlist	Spice/Spectre netlist used for simulation	Dynamic	Must
Device models	Spice models for ACE characterization	Dynamic	Recommended
Simulation current profiles	Device current profiles for Dynamic EMIR and Signal EM analysis	Dynamic / Signal EM	Must

Requirements for CCI data

Calibre LVS

When you run Calibre LVS, make sure that “CCI” option is set in calibre rule deck as shown below, this option is required to generate Calibre Connectivity Interface (CCI) data

```
MASK SVDB DIRECTORY “svdb” QUERY CCI
```

CCI Database

Calibre query extracted database contains following files and these files are used for cci2ex and Totem.

CCI DB Files	Information Extracted
CCI_DB.agf CCI_DB.GDS_MAP	Annotated GDS file w/ GDS Map file
CCI_DB.devicetab	Device models, marker layers, terminal names and layers
CCI_DB.extents CCI_DB.lph	BBOX information and Hierarchical Cell BBOX
CCI_DB.ixf CCI_DB.nxf CCI_DB.lnn	Device, Instance and Net Cross-references
CCI_DB.nl	Layout Device Names and its co-ordinates
CCI_DB.ports CCI_DB.ports_cells	Top Level and Cell Level Port x, y locations
CCI_DB.lvs_deck	Key LVS settings (like connectivity, etc.)

Requirements for RC Mapping File

RC Mapping file provides mapping between Calibre layers and Totem tech layer names, the format of the map file is similar with the map file used by StarRCXT.

Following are the rules for creating an RC Map file.

- RC Mapping file should contain all FEOL/BEOL/MEOL layers
- 2nd column MUST be consistent with the Totem Tech file layer-names
- All layers present in CCI GDS map must be mapped in the RC mapping file. A sanity check verifies that all layers are mapped and consistent with the
- Totem technology file. This check can be downgraded to “Warning” by setting RELAX_SANITY_CHECKS 1
- Layers that are mapped to “SUBSTRATE” for RC name are getting ignored unless DUMP_SUBSTRATE_PINS 1 is set. In that case pins that are on a substrate layer will be assigned to “lm” (lower-metal)
- Many LVS layers can be mapped to one Tech RC name but vice-versa is not possible
- Comments are represented by proceeding “*” sign
- Embedded line-comments can be supported if necessary
- A StarRCXT Map file can be used

Sample RC Map file:

```
*************************
**<LVS name> <RC name>
*************************
conducting_layers
* <lvs_layer_names> <apache_tech_layer_name>
met1       M1   ** comments
met2       M2
..
tpdiff     POD
tndiff     NOD
..
dnw        SUBSTRATE
..
via_layers
n_polyCont polyCont
p_polyCont polyCont
marker_layers
..
```

When using Calibre-LVS CCI data for substrate analysis, the substrate layers need to be mapped to appropriate Totem tech layers also.

In rc map file, use substrate_layers section for mapping substrate layers. All layers that are mapped within the substrate_layers section will be ignored running Totem in non-CSE mode therefore the same rc map file can be used for both CSE and non-CSE analyses.

Sample RC Map file with substrate layer mapping:

```
*************************
**<LVS name> <RC name>
*************************
substrate_layers
```

```
* <lvs_layer_names> <substrate_tech_layer_name>
Sub         BULK    ** comments
Bulk        BULK
..
nepi        EPI
NWell       NW
PWell       PW
..
dnw         DNW
..
conducting_layers
..
```

Requirements for DSPF

CCI Flow uses device pin coordinates from CCI database, therefore DSPF is not needed for power analysis and no need for detailed DSPF with sub-node info for signalEM, C-only DSPF is enough for signalEM to get total signal net capacitance.

Netlist requirements

If you use same netlist for both simulations and Calibre LVS, aplmmx will not see any device mapping issues. sometimes customers use Spectre netlist for simulations and Spice netlist for Calibre LVS, in such cases you may see device name mapping and should use mapping keywords in aplmmx config to map device names with CCI database as we do in regular flow.

Running the CCI Flow

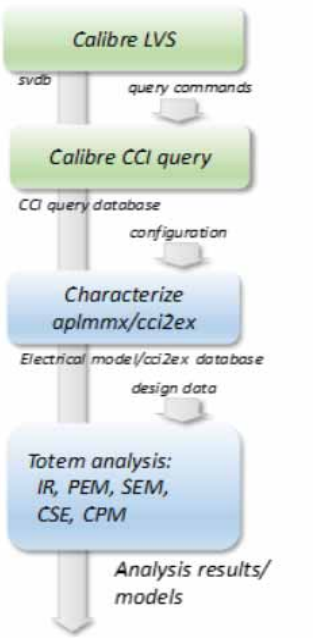


Figure 15-2 Steps in Totem CCI Flow

Query Calibre LVS results(svdb)

If you have calibre LVS results in svdb directory, run Calibre query commands to extract CCI data

Calibre Query Command:

```
calibre -query ./svdb  < query_cmd > cci.log
```

Sample query_cmd file:

```
## Calibre query command
# Define property numbers in annotated GDSII
GDS NETPROP NUMBER 5
GDS PLACEPROP NUMBER 6
GDS DEVPROP NUMBER 7
# Output seed polygon with net ID
GDS SEED PROPERTY DEVICE ORIGINAL
# Output annotated GDSII mapping file
RESPONSE FILE CCI_DB/CCI_DB.GDS_MAP
GDS MAP
RESPONSE DIRECT
# Output annotated GDSII file
GDS WRITE CCI_DB/CCI_DB.agf
# Output device table file containing descriptions of device layers
RESPONSE FILE CCI_DB/CCI_DB.devtab
DEVICE TABLE
RESPONSE DIRECT
# Output layout net name / net ID mapping
LAYOUT NETLIST TRIVIAL PINS YES
LAYOUT NETLIST EMPTY CELLS YES
LAYOUT NETLIST NAMES NONE
LAYOUT NAMETABLE WRITE CCI_DB/CCI_DB.lnn
# Output ideal layout netlist
LAYOUT NETLIST PRIMITIVE DEVICE SUBCKTS YES
LAYOUT NETLIST PIN LOCATIONS YES
LAYOUT NETLIST HIERARCHY AGF
LAYOUT NETLIST DEVICE LOCATION CENTER
LAYOUT NETLIST WRITE CCI_DB/CCI_DB.nl
# Output net / device instance cross referencing tables
NET XREF WRITE CCI_DB/CCI_DB.nxf
INSTANCE XREF WRITE CCI_DB/CCI_DB.ixf
# Output ports file
PORT TABLE CELLS WRITE CCI_DB/CCI_DB.ports_cells
# Output cell extents file
CELL EXTENTS WRITE CCI_DB/CCI_DB.extents
# LVS Settings Report
LVS SETTINGS REPORT WRITE CCI_DB/CCI_DB.lvs_deck
```

Query outputs the the CCI database.

Run CCI2EX

CCI2EX translate CCI data and creates Totem inputs, includes totem layermap, *.nif, layout netlist with source names etc.

We need following inputs to run cci2ex.

- cci2ex config
- CCI DB
- RC Map file
- Apache Tech file

Mandatory keywords in cci2ex config.

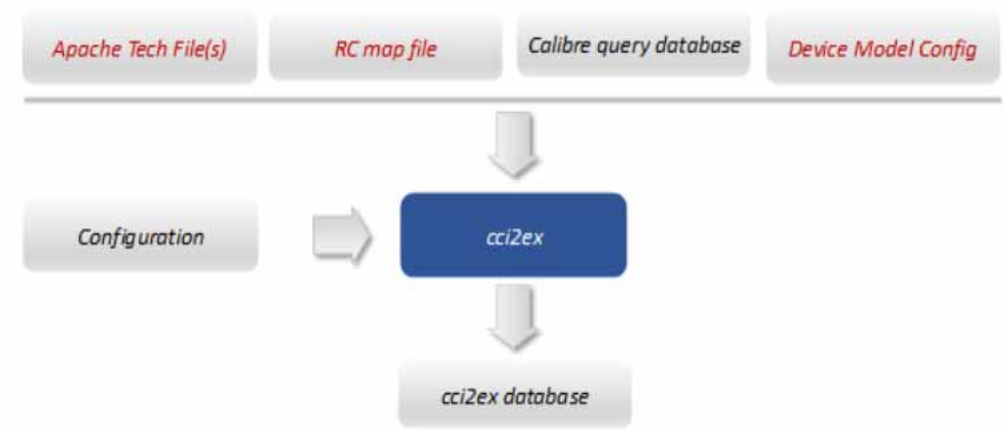
```
TOP_CELL   <string>
RC_MAP_FILE <file_path>
TECH_FILE <file_path>
CCI_DB directory_path>
```

cci2ex can be executed in two ways, either as standalone utility or integrated into aplmmx.

Run cci2ex standalone

Sample cci2ex.cfg

```
## cci2ex.cfg ##
## Design-specific configuration
TOP_CELL <top_cell>
TECH_FILE <apache_tech_file>
CCI_DB <cci_db_path>
## Configuration for the process/PDK
RC_MAP_FILE <rc_map_file>
DEVICE_MODEL_CONFIG <device_configuration_file>
# Other process configuration
#PSEUDO_VIA_LAYERS { … }
#UNMATCHED_DEVICES { … }
#UNMATCHED_NETS { … }
# Put all cci2ex output in one place
OUTPUT_DIRECTORY <cci2ex_output>
```

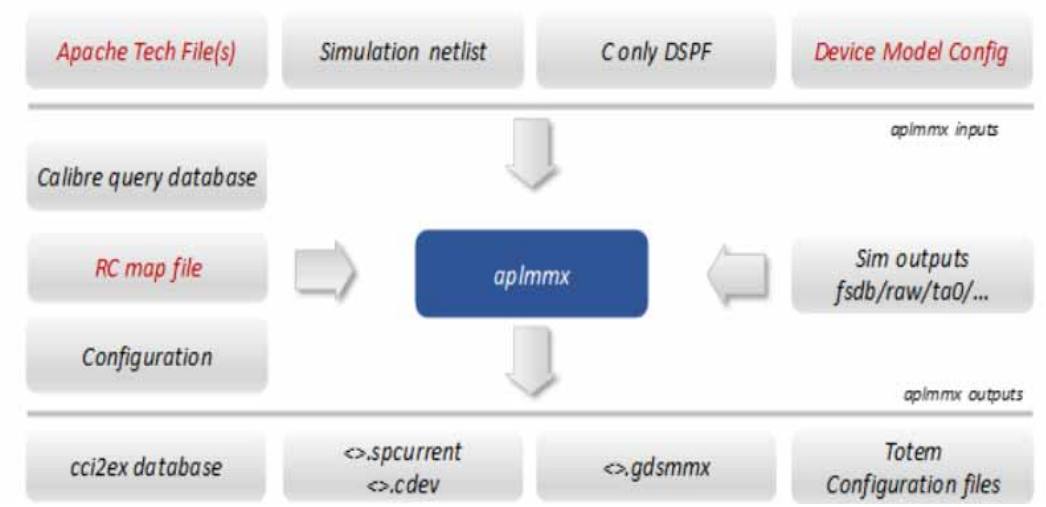


Command to run cci2ex: **`cci2ex cci2ex.cfg`**

If we run cci2ex standalone, we must supply cci2ex output directory in aplmmx config using CCI2EX_DB keyword and run aplmmx like in the GDS based flow to generate the electrical model for Totem analysis runs.

Run cci2ex in integrated aplmmx

In integrated aplmmx flow, aplmmx internally invoke cci2ex binary to translate CCI DB and then run aplmmx to generate Totem inputs.



Sample aplmmx.cfg file:

```
## aplmmx.cfg ##
TOP_CELL <top_cell>
# spice netlist;for static analysis the CCI2EX db netlist
# can be used while for dynamic analysis the
# simulation netlist has to be used.
#SPICE_NETLIST <cci2ex_output>/<top_cell>.schm.flatsp
SPICE_NETLIST <simulation_netlist>
DEVICE_MODEL_CONFIG <device_configuration_file>..
CCI_DB <cci_db_path>à ..
CCI2EX_OPTION {
  ## Configuration for the process/PDK
  RC_MAP_FILE <rc_map_file>
# Other process configuration
  #PSEUDO_VIA_LAYERS { … }
  #UNMATCHED_DEVICES { … }
  #UNMATCHED_NETS { … }
  # Put all cci2ex output in one place
  OUTPUT_DIRECTORY <cci2ex_output>
}
GSR_OPTION {
  TECH_FILE <apache_tech_file>
  ..
  ## Use Calibre ports for ploc
  PAD_FILES {
```

```
    adsRpt/<cell_name>.aplmmx.ploc
  }
}
GDS_FILE <cci_db_path>/<agf file>
## Misc options needed in CCI flow
USE_GATE_XY_FOR_DS 0
PRECISION_XY_LOCATION 1
NET_NAME_CASE_SENSISTIVE 1
```

Run Aplmmx

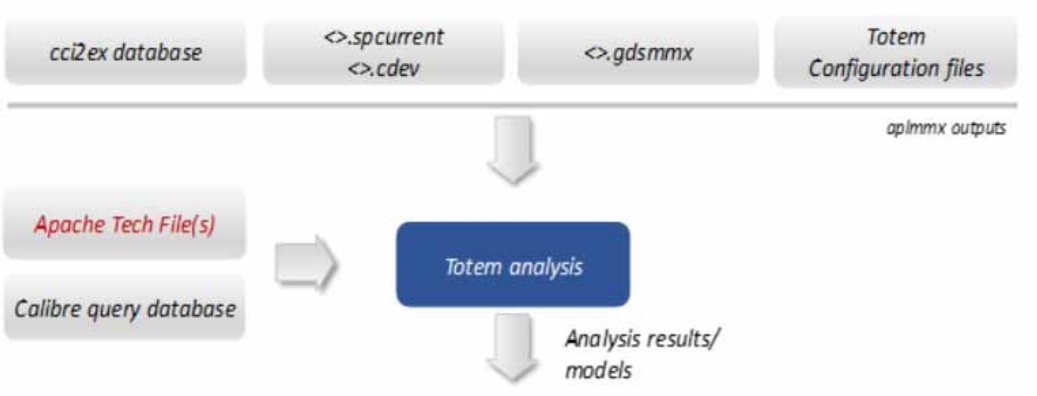
The command options are same as regular Totem Flow

```
## Static analsyis
aplmmx –gds aplmmx.config
## Dynamic analysis
## probe generation
aplmmx –s aplmmx.config
## Electrical Modeling
aplmmx –n [-no_cci2ex] aplmmx.config
## Signal EM analysis
## probe generation
aplmmx –s -sig aplmmx.config
## Electrical Modeling
aplmmx –n -sig [-no_cci2ex] aplmmx.config
```

In case the cci2ex database has been generated already in a previous run and should not be generated again user can add the '-no_cci2ex' option.

Totem Analysis

In CCI flow, all geometries in Annotated GDS File(*agf) are labelled, no need tracing and no start points are required for GDS2DB. Since the Calibre CCI DB contains accurate device locations, the pin-snapping will be accurate. Declaring the keyword CCI2EX_DB in *gds.config triggers the CCI flow, aplmmx automatically generates *gds.config and template GSR files with required keywords to enable CCI Flow.



Sample GDS config:

```
##GDS config(auto-generated by aplmmx)
TOP_CELL <cell_name>
CCI2EX_DB <path_to_cci2ex_output_db>
CCI_DB <path_to_cci_db>
XTOR_MAP_FILE <aplmmx genetated *gdsmmx>
GDS_FILE <agf file>
GDS_MAP_FILE <cci2ex generated layermap>
<PG net_names>
<other_keywords/options>
```

Sample GSR:

```
TECH_FILE <apache_tech_file>
## Tech connectivity is derived from LVS rule-settings file
TECH_CONNECTIVITY_FILE <tech_connectivity>
..
GDSII_FILE {
  <cell_name> <gds_config> full top
}
## Ploc file automatically derived from Calibre ports
PAD_FILES {
  adsRpt/<cell_name>.aplmmx.ploc
}
..
#<PG net definitions and other typical options>
```

Sample run TCL file:

```
setup analysis_model [static|dynamic|signalEM]
setup design <>.gsr
perform pwrcalc ## only for power EM run
perform extraction [-power|-ground|-signal]
perform analysis [-static|-dynamic|-signalEM]
perform emcheck
```

Totem run command:

```
% totem –f run.tcl
```

CCI Flow related keywords

Mandatory CCI2EX keywords:

```
TOP_CELL <cell_name>
RC_MAP_FILE <file_path>
TECH_FILE <file_path>
CCI_DB <directory_path>
```

Optional CCI2EX Keywords:

CCI2EX_DEVICE_MODEL_CONFIG / DEVICE_MODEL_CONFIG

Specifies the path of the device model configuration file that explicitly defines the types of device models.

Syntax:

```
CCI2EX_DEVICE_MODEL_CONFIG / DEVICE_MODEL_CONFIG
<Path_to_file>
```

Devices supported under keyword DEVICE_MODEL_CONFIG :

MOM
DIODE
RESISTOR
NMOS, PMOS
GENERIC
INDUCTOR
CAPACITOR
NPN, PNP
LNDMOS, LPDMOS

CCI_DEVICE_KEY

Defines the DEVICE PROP value from Caliber Query Command. Default :7

Syntax:

```
CCI_DEVICE_KEY <number>
```

CCI_INSTANCE_KEY

Defines the INSTANCE PROP value from Caliber Query Command. Default: 6

Syntax:

```
CCI_INSTANCE_KEY <number>
```

CCI_NET_KEY

Defines the NET PROP value from Caliber Query Command. Default : 5

Syntax:

```
CCI_NET_KEY <number>
```

DMC_PRIORITY

When set to 0, device table file in CCI_DB will get priority than device Types in Device Model Config . Default: 1.

Syntax:

```
DMC_PRIORITY [0|1]
```

DUMP_SUBSTRATE_PINS

When set to 1, honors substrate pins in analysis.Default : 0

Syntax:

```
DUMP_SUBSTRATE_PINS [0|1]
```

FINGER_DELIMITER

Defines the finger Delimiter used for dumping in NIF,ALS,Flat NetList ,default is ‘@’

Syntax:

```
FINGER_DELIMITER <delimiter_string>
```

OUTPUT_DIRECTORY

Defines the output directory path.

Syntax:

```
OUTPUT_DIRECTORY <Path_to_file>
```

PIN_DELIMITER

Defines the pin Delimiter used for dumping NIF/ALS/Flat NetList, default is ':'

Syntax:

```
PIN_DELIMITER <delimiter_string>
```

QUERY_COMMAND_FILE

Specifies the path to Calibre Query Command File.

Syntax:

```
QUERY_COMMAND_FILE <Path_to_file>
```

RELAX_SANITY_CHECKS

When set to 0, if any sanity checks failed, the tool will error out, else it will not, Default:1

Syntax:

```
RELAX_SANITY_CHECKS [0|1]
```

RC_MAP_HEADERS {}

There will be default block headers(conducting_layers, via_layers, color_layers, remove_layers, ignore_cap_layers, custom_bbox_layers, marker_layers, viewonly_layers) in RC_MAP_FILE for the layers in cci2ex flow. Any new block header used in RC_MAP_FILE can be provided under RC_MAP_HEADERS.

Syntax:

```
RC_MAP_HEADERS {
<layer_headers>
}
```

UNMATCHED_DEVICES

UNMATCHED_NETS

Defines a prefix and suffix string for the PODE device and their Net names. The CCI database will contain string "_unmatched" appended to the layout device name and net name, if a layout device cannot be found in the cross-referencing file (IXF) and a layout net is not found in (NXF&&LNN) file.

The DSPF contains the conventions "Xld_" prefix for PODE devices and "ln_" for their nets as they don't have any corresponding schematic entry in source netlist.

Hence, to rename the layout-only devices (eg: PODE) and their Nets these keywords are used:

Syntax:

```
UNMATCHED_DEVICES {
  PREFIX   <string>
  SUFFIX   <string>
}
```

```
UNMATCHED_NETS {
  PREFIX <string>
  SUFFIX <string>
}
```

PSEUDO_VIA_LAYERS

Defines the pseudo via layers in the design.

Syntax:

```
PSEUDO_VIA_LAYERS {
  <layer_name_1>
  ..
  <layer_name_n>
}
```

APLMMX keywords:

CCI_DB

Specifies the Calibre query output directory

Syntax:

```
CCI_DB <path>
```

CCI2EX_DB

Specifie the cci2ex output directory in standalone flow.

Syntax:

```
CCI2EX_DB <path>
```

CCI2EX_OPTION {}

Used for integrated flow, this keyword defines all CCI2EX options expect CCI_DB. Keywords that can be derived from the APLMMX or TSR configuration do not need to be listed in the CCI2EX_OPTIONS section.

Syntax:

```
CCI2EX_OPTION {
<options>
....
}
```

GSR keyword:

TECH_CONNECTIVITY_FILE

APLMMX automatically pass the CCI2EX_DB/<cell_name>.connectivity file into TECH_CONNECTIVITY_FILE in template GSR.Generated adsRpt/<cell_name>.aplmmx.ploc can be use as PAD_FILES.

Special device handling

MOM cell handling:

MOM CELLS can be mentioned in Device Model Config as below:

```
MOM_CELL    <list of mom_cell wildcards separated by
commas>
```

```
E.g.: MOM_CELL *CRTMOM*
```

PS: If ambiguous wildcards like *MOM* are used and if some block in <>.nl/agf which is not a MOM_CELL has the string, you may end up finding multiple cells as MOM. Hence, we will have to make sure that unambiguous strings are used for MOM_CELL.

- *All MOM Devices are tagged as 'USER' type in CCI_DB.*
- *Hence CCI2EX needs a input File (similar to device model config file in aplmmx) to represent certain device Models as MOM which are mentioned as 'USER' type in CCI_DB*
- *Usage Model of device model config file is as follows:*

  ```
  <device_type>    <model_name>
  <cell_type>         <cell_name>
  ```

- *Cell_name should support WildCards as well*

  Example

  ```
  MOM    crtmom_2t              # Per line one device Model
  MOM    fmom_2t
  MOM_CELL   *CRTMOM* , *FMOM*   # You can list all Mom Cell
  Wildcards in one line
  ```

P/NLDMOS cell handling

The CCI flow supports LDMOS transistors. To define the device type, user needs to set LDMOS device as "NLDMOS" or "PLDMOS" in the device model config file. In case the LDMOS device is extracted without width and/or length parameter W and L can be extracted from other parameters using simple equations in the device model config setup.

Example:

```
NLDMOS  ldnmos1 NA D(NA) G(NA) S(NA) B(NA) SUB(NA)[
SIZE_PARAM w=(nx+cw) l=ny*le-6 ]
```

In case no parameter is available a fixed value can be assigned to w and l also. The parameters nf, nfend and nr are used for RF finger array and cannot be used for the W/L Calculation.

In case the pins of the LDMOS devices are mapped onto a substrate layer totem will map these pins to "lm" (lowest metal). To enable the generation of substrate pins into the CCI database user must enable DUMP_SUBSTRATE_PINS in the cci2ex configuration.

Pseudo Flattened NetList Support

Sometimes AGF and Netlist can have a mismatch, due to Query Command 'LAYOUT NETLIST SEPARATED PROPERTIES YES' in Calibre query command file

Netlist can be pseudo Flattened, but AGF is not, when CCI2EX need to handle this scenario by identifying the correct devices in Netlist map to the physical Geometries.

The keyword _PSEUDO_FLATTENED_NETLIST for this purpose

Totem CCI Flow Debugging Tips

Running cci2ex standalone the cci2ex log/err/warn files can be found be found in the working directory.

Running cci2ex as part of aplmmx the cci2ex logfile can be found under 'adsRpt/Log/<cell_name> _cci2ex.log'. In case aplmmx fails because a file generated by cci2ex is missing check the cci2ex logfile to ensure that cci2ex run was successful.Additional reports can be found under 'adsRpt/CCI_Reports/'.

Report/Debug Files	Info

cci2ex.log/err/warn	Log/error and warning message info
adsRpt/CCI_Reports/cci2ex.stats	Cell name and instance, net, device counts per cell
adsRpt/CCI_Reports/flatDevices.rpt & adsRpt/CCI_Reports/ hierDevices.rpt	Flat/Hierarchy based info (device names, type, models, marker(ID), pin and pin-marker and ID)
adsRpt/CCI_Reports/cci_sanity.log/ err/warn	CCI flow related sanity checking for error, warning and info messages.

The sanity check verifies the consistency b/w Tech, RCMAP, GDSMAP and AGF layers(route/marker).

In case the sanity check fails, the cci2ex will error out unless RELAX_SANITY_CHECKS is enabled in the cci2ex configuration.

Sanity check error codes can be read as follows:

E.g. **LAYER[ROUTE]_GDSMAP[1]_RCMAP[0]_TECH[0]**

This code indicates that

- **LAYER [ROUTE]** <- 'ROUTE' layer type is routing layer
- **GDSMAP [1]** <- '1' This layer appears in the calibre query database in the gds map file
- **RCMAP [0]** <- '0' This layer is not mapped to an Totem tech layer in the RC map file
- **TECH [0]** <- '0' This layer is not a Totem tech layer

Note aplmmx will proceed but complain that input data from the cci2ex run are missing. In such case check the cci2ex logfile and verify if the cci2ex run was successful.

Sample sanity check report:

```
------------------------------------------------------------
LVS_LayerName  AGF(Layer:Datatype)   Tech_LayerName
Device_Model   ERROR_CODE
------------------------------------------------------------
********* RESISTOR MARKER LAYERS *************
# MAPPED IN GDSMAP/RCMAP/TECH
# MAPPING ERRORS
RPPOLY1NULLRpolyrppoly
LAYER[DEV]_GDSMAP[0]_RCMAP[1]_TECH[1]
********* DEVICE MARKER LAYERS *************
# MAPPED IN GDSMAP/RCMAP/TECH
# MAPPING ERRORS
ngate_macz   57:0 NULL nch_mac
LAYER[DEV]_GDSMAP[1]_RCMAP[0]_TECH[0]
..
********* ROUTE LAYERS *************
# MAPPED IN GDSMAP/RCMAP/TECH
AP     28:0 AP NULL NULL
M1           16:0 M1 NULL NULL
```

```
tpdiff_RC    26:0 POD NULL NULL
# MAPPING ERRORS
all_sd       62:0 NULL NULL
LAYER[ROUTE]_GDSMAP[1]_RCMAP[0]_TECH[0]
all_sd_cop   63:0            NULL      NULL
LAYER[ROUTE]_GDSMAP[1]_RCMAP[0]_TECH[0]
```

Appendix A
Installation Procedure

Introduction

This chapter describes how to download and install the Totem program, in the following steps:

1. Download software from ANSYS Customer Portal.
2. Perform the installation
3. Set up the Totem software license
4. Set up the Totem environment

Downloading Totem Software

Please log in to Ansys Customer Support Center website (https://support.ansys.com/AnsysCustomerPortal/en_us/Downloads/Semiconductor+Products) for download instructions.

Program Installation

Download the software in the <*tarball_directory*>, such as */disk1/ecad/*, from where you want to install Totem. Follow the instructions below, using the *csh/tcsh* shell (or other shells).

```
% cd <tarball_directory>
% gtar xzvf Totem_<platform>_<version>.tar.gz>
% setenv APACHEROOT <Totem Installation Directory>
```

This will allow execution of the installed and associated Totem binaries.

Note: For installations that contain both RedHawk and Totem versions, you can specify Totem versions using $TOTEMROOT and the RedHawk versions using $APACHEROOT in the same script/environment. Since the “aplmmx” binary is shared between RedHawk and Totem, to avoid confusion you should invoke the aplmmx binary with the absolute path “$TOTEMROOT/bin/aplmmx” in order to invoke aplmmx from Totem. It also ensures that other binaries that aplmmx calls internally, such as ACE and NSPICE, are invoked from binaries under the Totem installation.

After installing the Totem software, the following directory structure will be created in the *<installation_directory>*, along with the *README* file and software Release Notes.

Directory	Partial Contents
license/	ANSYS license file
bin/	Contains all the *Totem* related executables, such as "flow setup" utilities *rh_setup.pl* and *gds_setup.pl.*
lib/	Machine dependent libraries
etc/	Machine independent files and data
lmbin/	FlexLM manager and utilities
platform/	LSF support for APL
scripts/	Useful Perl scripts for assisting in Totem execution.
scripts/atcl	Helpful TCL utilities for TCL query capabilities in Totem
doc/	Software Release Notes and latest *Totem Users' Manual*

Totem Operating System/Platform Support

Totem supports RHEL (RedHat)/CentOS versions 5 and 6 and SLES (SUSE) version 11.

Setting Up the SCBU Product License

Totem uses the industry-standard FlexLM licensing scheme. The license file (apacheda.lic) can be obtained from your ANSYS technical support contact. Write the license file into the *license/* directory. It is recommended that your system administrator installs the license and executables and makes a backup copy of the release directory and license.

To start the license daemon, issue the following commands on the license server:

```
% cd /disk1/ecad/apache (as the <Totem Installation Dir>)
% lmbin/<platform>/lmgrd -c license/apacheda.lic
    -l /var/tmp/apacheda.log
```

where

`lmgrd` : the FlexLM license daemon program,

apacheda.lic : the license file, and

apacheda.log : the license log file.

Note that the log file is put under the */var/tmp* directory. The system will automatically remove the log file when it reboots. If you want to maintain the license log file even after the system is shutdown and reboots, you can specify another directory other than */var/tmp* (for example, use */tmp*). However, you must ensure that you have write permission to that directory when you start the license daemon process.

NOTE: Do not start lmgrd as root.

You can now check the status of the license daemon by using the following commands,

```
% lmbin/<platform>/lmstat -v apacheda
```

```
% ps -al | grep apacheda
```

If you need to stop the license daemon for any reason, use following command:

```
% lmbin/<platform>/lmdown -c license/apacheda.lic
```

where *apacheda.lic* is the license file.

NOTE: Do not use the Unix command `% kill -9`, as this may leave the vendor daemon in the background and cause problems.

Setting Up the Totem Environment

License File and Library Directory Setup

The license file setup is required for every user who runs Totem. The lines below may be added to the *.cshrc* file. Set up the license file in the *csh/tcsh* environment:

```
% setenv APACHEDA_LICENSE_FILE <absolute_path_to_license-file>
```

or,

```
% setenv APACHEDA_LICENSE_FILE port_number@lic_server_ip_addr
% set path=($path <abs_path_to_release>/Totem)
```

For example,

```
% setenv APACHEDA_LICENSE_FILE /disk1/ecad/apache/license/apacheda.lic
```

or through network for the license file,

```
% setenv APACHEDA_LICENSE_FILE 1881@129.186.1.10
% set path=($path $APACHEROOT/bin)
% rehash
```

If the library directories are not in the same location as Totem, set the following environment variables in the `.cshrc` shell environment:

```
setenv LD_LIBRARY_PATH <Totem_installation_dir/lib>
```

or if *$LD_LIBRARY_PATH* already exists, set up the following instead:

```
setenv LD_LIBRARY_PATH "$LD_LIBRARY_PATH:<Totem_installation_dir/lib>"
setenv SHLIB_PATH <Totem_installation_dir/lib>
```

or if *$SHLIB_PATH* already exists, set up the following instead:

```
setenv $SHLIB_PATH "$SHLIB_PATH:<Totem_installation_dir/lib>"
```

If `'source ./setup.ksh'` was not executed in the `ksh` shell environment, then set the following environment variables in the `.kshrc` shell environment.

```
set LD_LIBRARY_PATH=<Totem_installation_dir/lib>
```

or if *$LD_LIBRARY_PATH* already exists, execute the following instead:

```
set LD_LIBRARY_PATH="$LD_LIBRARY_PATH:<Totem_installation_dir/lib>"
export LD_LIBRARY_PATH
set SHLIB_PATH=<Totem_installation_dir/lib>
```

or if $SHLIB_PATH already exists, execute the following instead:

```
set SHLIB_PATH="$SHLIB_PATH:<Totem_installation_dir/lib>"
```

Binary Setup

There are two Totem binary options:

- platform-specific
- platform-independent

These two types of invocations are discussed in the following sections.

Platform-Specific Binaries

Available platform-specific binaries for Totem are “Linux32” and “Linux64”, which can be run on RedHat Enterprise3 and later versions.

This binary is for customers with only one or two types of machines that want platform-specific binaries for reasons of disk space or FTP download time.

Before using Totem, source *.cshrc* or refresh *.kshrc*. Alternatively, if ‘`source ./setup.csh`’ was not executed in the `csh` or `tcsh` shell environment, then set the following environment variables in the `.cshrc` shell environment:

```
setenv APACHEROOT <Totem_installation_dir>
set PATH=($APACHEROOT/bin $path) [if desired]
```

If ‘`source ./setup.ksh`’ was not executed in the `ksh` shell environment, then set the following environment variables in the `.kshrc` shell environment.

```
set APACHEROOT=<Totem_installation_dir>
export APACHEROOT
```

Platform-Independent Binaries

The platform-independent binary under the “Suites” package includes all Totem-supported platforms, with transparent access without regard to machine. If you want to use a relative invocation path, a PATH variable must be defined for the desired binary location, but the $APACHEROOT variable is not required, and will be ignored if incorrect. The correct binary installation for the machine is found on Totem invocation.

Invocation

In order to test the installation, use the following command in the application directory:

```
Totem
```

The Totem GUI should appear, as shown in Figure A-1 below. The data files can now be loaded and Totem run.

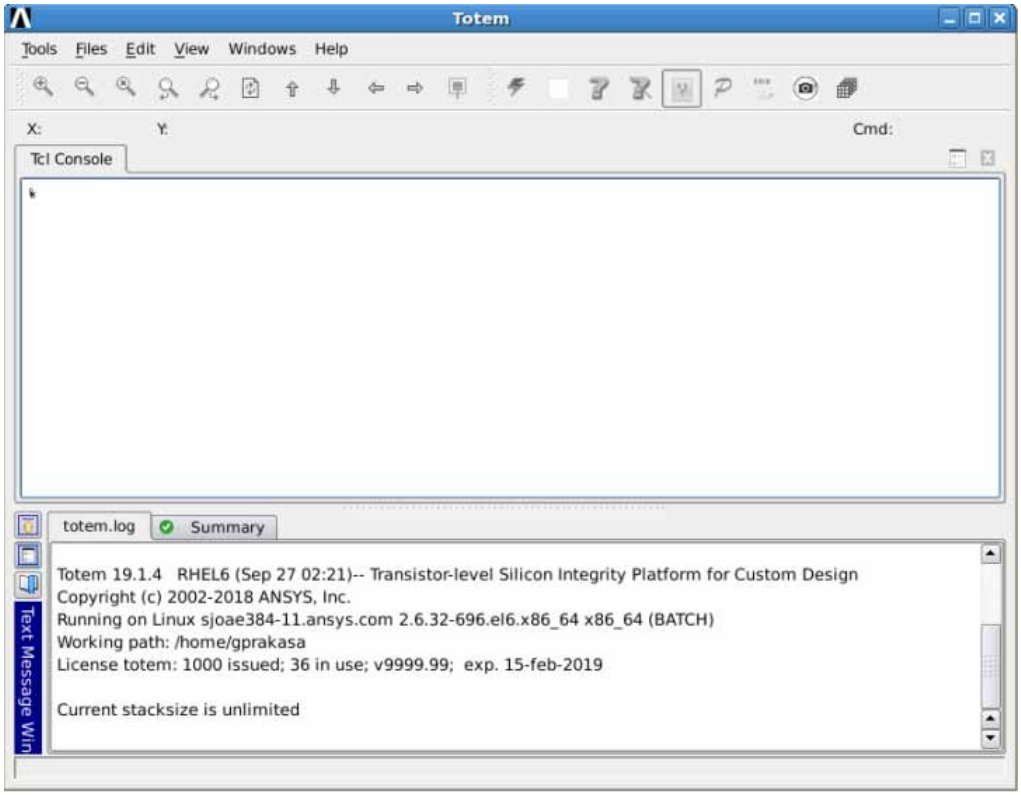


Figure A-1 Initial GUI of the Ansys Totem program

Appendix B1
Power Integrity GUI and Menus

Introduction

This appendix describes the graphic user interface and menus available for running Totem Power Integrity Analysis. For TCL invocation and command descriptions, see Appendix D, "TCL Command Summary".

For descriptions of Signal EM analysis menus, see Appendix B2, "Signal EM GUI and Menus".

For descriptions of ESD analysis menus, see Appendix B3, "ESD GUI and Menus".

Totem Graphic User Interface Description

The Totem GUI functional areas are shown in Figure B1-1 below.

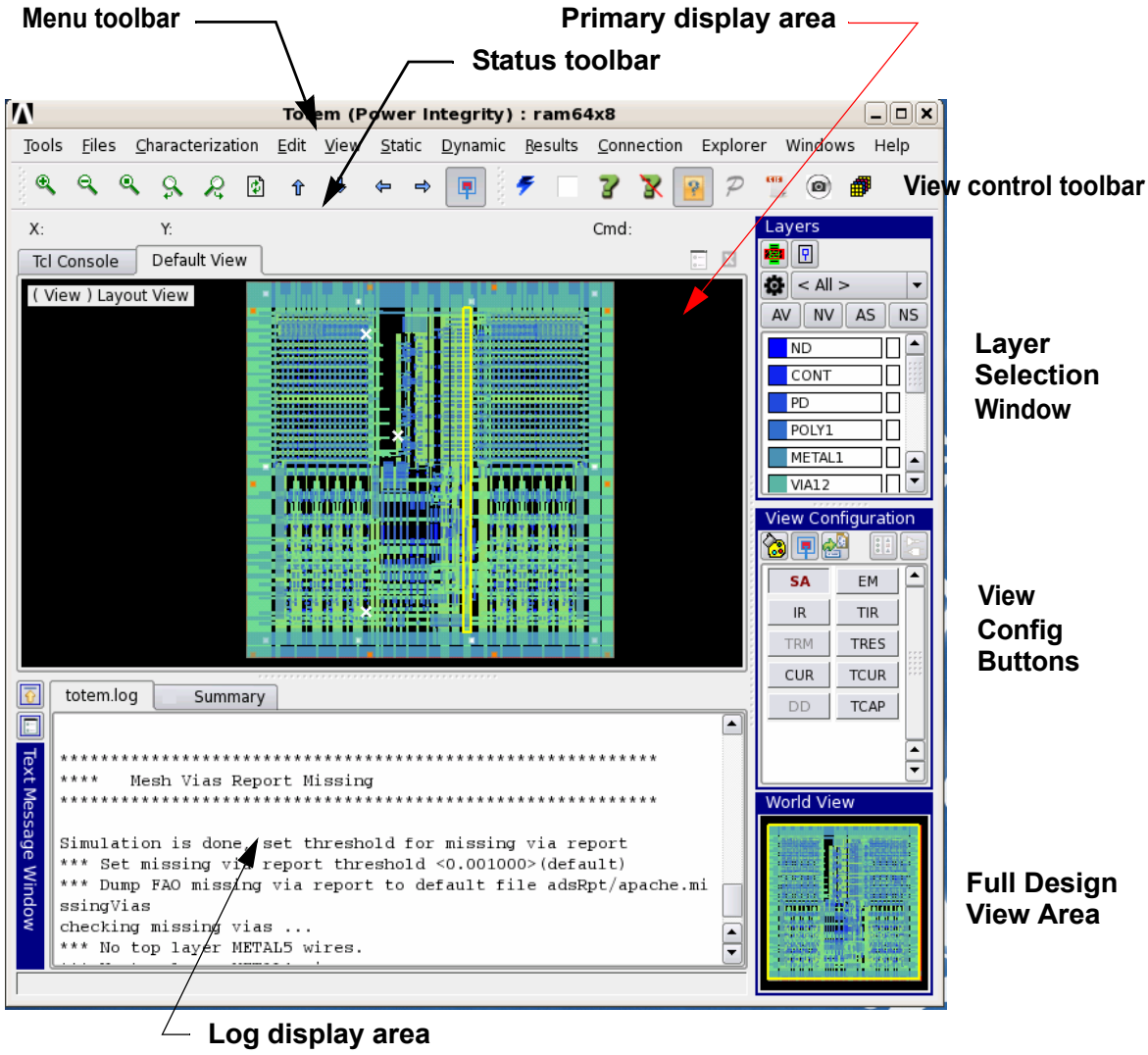


Figure B1-1 Totem graphical user interface

The Totem GUI provides a configurable multi-window, multi-page, multi-frame debugging environment. You can combine different color maps and reports on the same GUI page to debug design weaknesses and examine the source of design issues much more efficiently. A few key features are:

- Switch view menu based on analysis tool to only highlight product specific features
- Configurable multi-frame multi-page interface to allow user configure his/her own preferred debugging environment
- Pre-select and status bar helps user query design information in a much more efficient way
- Embedded GDS viewer to allow user cross-probe between analysis layout view and color maps with GDS viewer for easy debugging

The following figure is an example of how you can use the Totem GUI to configure a multi-frame page for debugging:

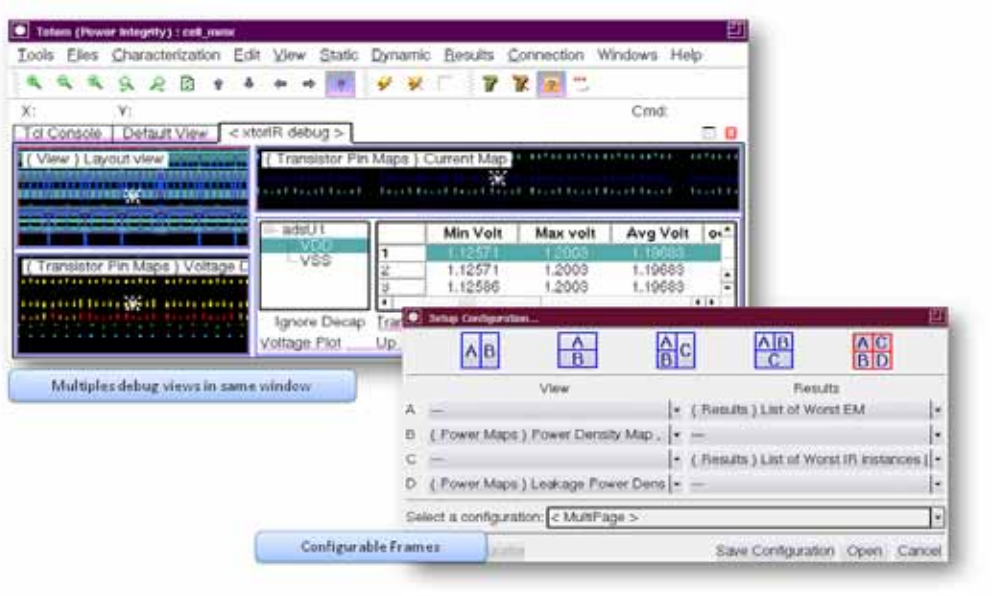


Figure B1-2 Totem multi-frame GUI

General GUI Features

GUI Look and Feel Preference Setting

The *Totem.ini* file can be used to control the GUI menu, dialogs, GDS View, Waveform Viewer, multi-frame setup and other common options, such as font type, font size, and highlight preferences. A template of the *Totem.ini* file is in the *lib/guicfg* directory in the default installation package (*$APACHEROOT/lib/guicfg/Totem.ini*).

The priority of available *Totem.ini* files is as follows.

1. From the local Totem run directory *adsGui/Totem.ini.*
2. From user's home directory *~/.totem/Totem.ini.*
3. From installation *$APACHEROOT/lib/guicfg/Totem.ini.*

If files in locations 1 and 2 are not available, then the default GUI preference settings from 3 are applied. Or you can use “F10” button to bring up the Preference dialog and click the “Reset” button on the bottom of the dialog to reset to Totem default settings.

Layers Window

The **Layers** window (see Figure B1-3) allows viewing information related to layers. The **Layers** window functions are :

- AV - All Visible
- NV - None Visible
- AS - All Selectable
- NS - None Selectable

Note that when **SA** is selected in the **View Configuration** window, and the “Configure Color Maps” button is clicked, a Layer Select dialog box is displayed (see Figure B1-5), which allows changing the display features of each layer. Individual layer display can be changed using the check boxes. Click on the column headers to change the display

characteristics for groups of layers: Vis (visibility), Fill/Line (object display type), Sel (select metal/via layers), Pin (select metal/via layer pins).

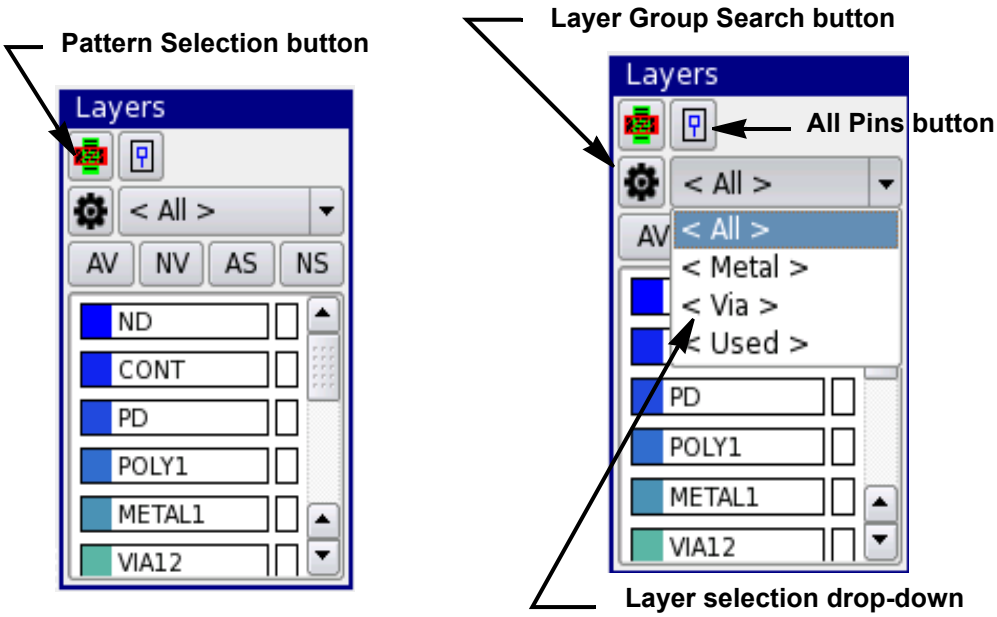


Figure B1-3 Layers selection window

In the **Layers** window, use the left top button to switch between **Pattern Brush** and **Transparent Drawing** modes, as shown in Figure B1-3 and Figure B1-4.Other capabilities of the Layer Selection dialog are.

- support for layer search and filtering using the Layer Group Search button
- display only “used layers”, which is turned on by default
- option for name-based search of layer-names, which also supports wildcards
- the All Pins button turns On/Off transistor pins for all layers in the SA view, as shown in Figure B1-3.

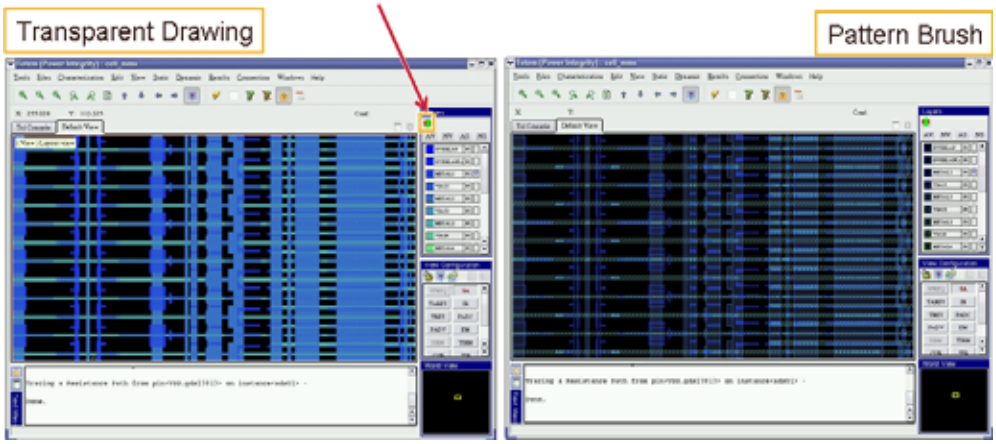


Figure B1-4 Pattern brush & transparent drawing displays

Configuration Window

At the top of the **View Configuration** window, as shown in Figure B1-5, are icons for opening a “Set Color Range” dialog, executing “Show Power Pad”, opening a “Configure Color Maps” dialog, opening a “View GDS” dialog, and opening a “View Netlist” dialog.

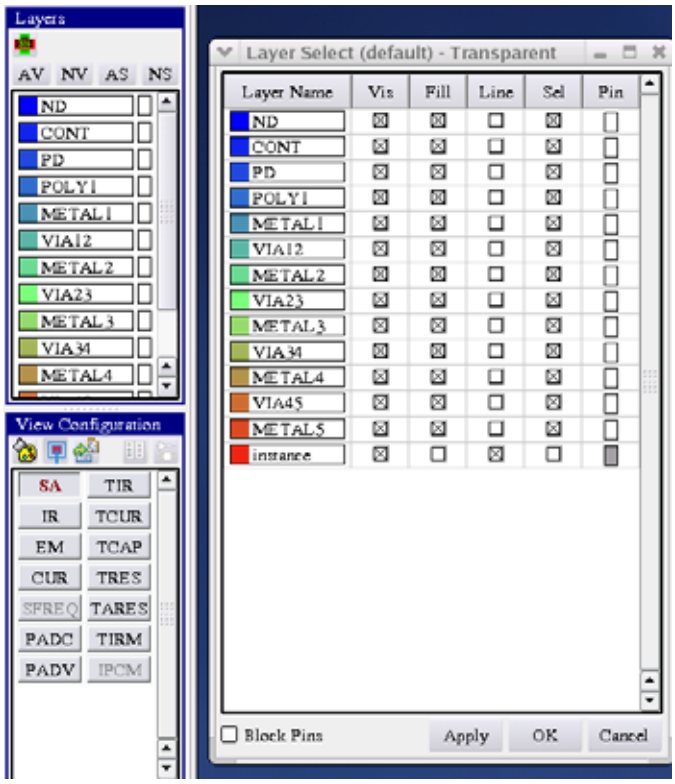


Figure B1-5 Layer Select and View Configuration windows

- Set Color Range” (left Configuration icon)

  Displays a Color Map dialog box based on the ‘View Results’ or **View Configuration** window selection.

Note: Voltage Drop Color Map displays for multiple Vdd designs indicate the worst % drop of all nets selected in the View Nets dialog box. So multi-Vdd instances have colors indicating the domain with the worst % voltage drop. The right-hand ‘mv’ column in the Voltage Drop Color Map dialog indicates only the % of the ideal voltage selected in the ‘VDD Domain’ drop down box. Hence selecting a different VDD Domain value changes the values in the ‘mv’ column, but does not change the worst % voltage drop color map.

Color maps may display either objects, a high parameter value, or a parameter density by tiled grid. Density displays use a grid composed of “tiles” that are 30 microns square by default. The size/number of the tiles can be changed using the ‘Grid Num’ or ‘Grid Size’ boxes on the dialog.

One of up to 20 colors can be assigned to specific ranges of parameter values, such as static or dynamic voltage drop, power, EM severity, current, frequency, or capacitance. Display colors can be changed by clicking on the color button and then selecting a new color from the ‘Pick a Color’ dialog box that appears. Ranges

of parameter values also can be turned on or off independently with check boxes. The **Default** button allows easy return to the original color display values.

- “Show Power Pad” (center left Configuration icon) - toggles the display of power pads on and off (on by default). VDD pads are shown in orange and VSS pads in white.
- “Configure Color Maps” (center Configuration icon) - in general the specific color map dialog box for each of the following functions in the View Configuration window can be displayed by clicking on the ‘Set Color Range’ configuration button described above.

  You can change color settings between 5 to 20 in the Color map dialog for wires and vias, to allow more color resolution in customizing color maps and movies, as shown in Figure B1-6.

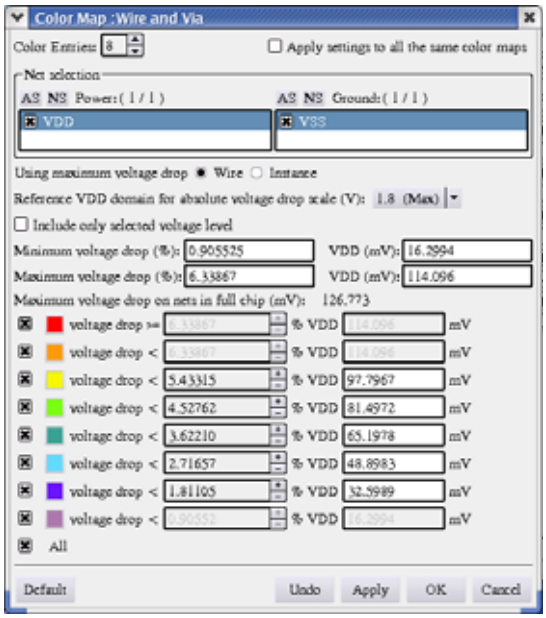


Figure B1-6 Color map color setting dialog

- View GDS (center right Configuration icon) - opens an “Import GDS ...” dialog, allowing you to specify a GDS file, include a Top Cell and Layer Map, as well as Third Party Technology Data files for importing.
- View netlist (right Configuration icon) - opens an “Import Netlist ...” dialog, allowing you to specify a Netlist File and a Top Cell for importing.

Configuration buttons

The buttons displayed in the **Configuration** window are customizable, and can be modified clicking on the “Configure Color Maps” button when **SA** is selected (see Figure B1-7).

You then can select entries from the left side table and use the right arrow button to add new maps in the panel or select entries from the right table and use the left arrow button to delete maps from the fast access panel.

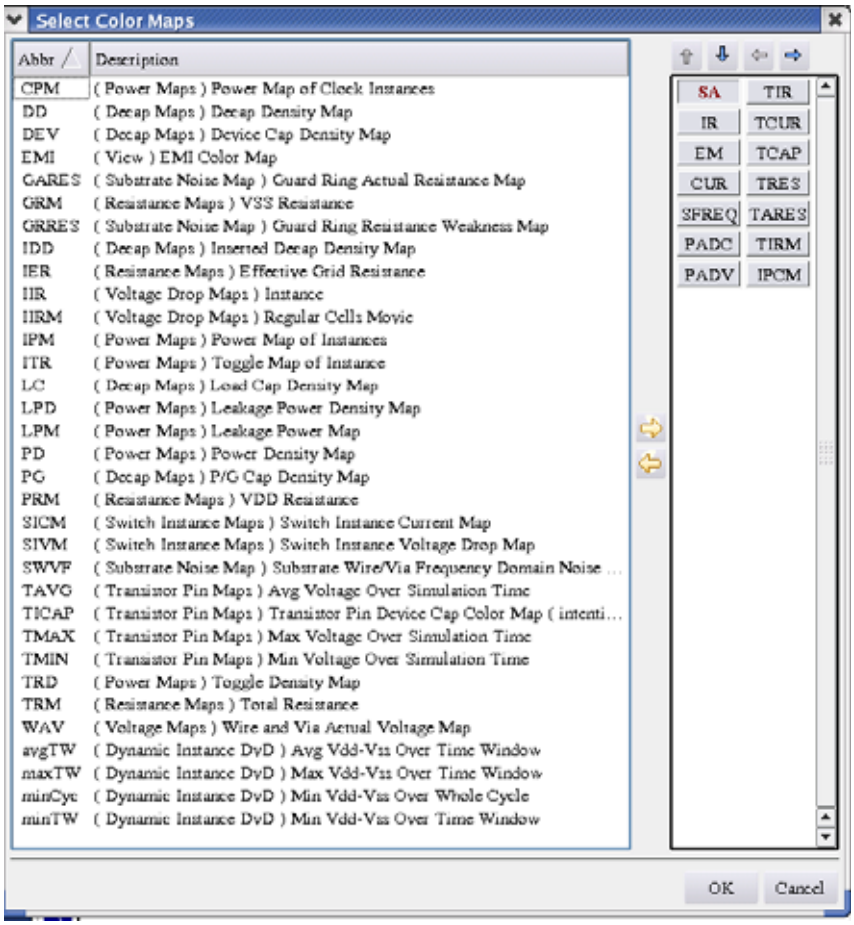


Figure B1-7 Select Color Maps dialog

The default View Configuration buttons are described following:

First column of 'View Configuration' buttons (default)

- **SA** - "Show all" of layout - displays some or all objects in the design layout (individual layer grids, instances, vias and pins per layer), based on the selections in the 'Layers' dialog box. The dialog allows a choice of colors for displaying each type of object, as well as showing objects in either solid or outline representations. By default pins are turned off, layer and instance objects are displayed solid.
- **IR** - "IR Map" - displays the latest analysis results, static IR voltage drop or dynamic voltage drop analysis, based on view selections in the 'Layers' dialog box and the color ranges selected in the 'Voltage Color Map" dialog. Instances are shown in Fill representation by default, while layers and vias are always shown in Fill representation, in the color chosen to indicate their voltage drop category. At high magnifications high voltage drop values on individual instances can be observed. By default the ranges assigned to each color are equally divided in six steps between the 'Maximum' voltage drop and 'Minimum' voltage drop values specified.
- **CUR** - "Current Map" - displays results of maximum current by node for static or dynamic conditions, based on the color ranges selected in the 'Current Color Map" dialog. Individual colors represent a selected percent of maximum current in the design. Instances are not displayed by default. If displayed, instances are shown in their selected object color.

Second column 'View Configuration' buttons

- **EM -** "Electromigration Map - Peak" - displays the peak values of EM, in colors identifying the severity of the EM values according to the color map table
- **TIR** - "Transistor Voltage Drop Map" - displays the worst transistor pin voltage drop over the simulation time, in colors identifying the severity of the voltage drop.
- **TRES** - "Transistor P/G Pin Weakness Map" - displays a P/G pin weakness ranking colormap, in colors indicating the relative resistance of all transistor pins from highest to lowest. Click on the 'Set Color Range' button in the Configuration group to obtain a 'Transistor Pin PG Weakness Color Map' dialog, which allows you to select colors for a range of resistance values as a percentage of the resistance gradient, set the maximum and minimum gradient percentages, and also increase the size of the pins for ease of viewing using the 'Pin Size Scale' window.
- **TCUR** - "Transistor Current Map" - displays transistor pins in colors identifying their effective current values. The effective current is the average current within the first cycle of the spcurrent profile, or the average current from BLOCK_POWER_FOR_SCALING. Click on the 'Set Color Range' button in the Configuration group to obtain a 'Transistor Pin Current Color Map' dialog, which allows you to select colors for a range of pin current values, set the current range displayed, and also increase the size of the pins for ease of viewing using the 'Pin Size Scale' window.
- **TCAP** - "Transistor Decap Map" - displays transistor pins in colors identifying their device capacitance values. Click on the 'Set Color Range' button in the Configuration group to obtain a 'Transistor Pin Decap Color Map' dialog, which allows you to select colors for a range of device capacitance values, set the capacitance range displayed, and also increase the size of the pins for ease of viewing using the 'Pin Size Scale' window.

GUI View Control and Query Toolbars

Right-clicking at the right end of the Menu toolbar brings up a dialog box to display the View Control and Query toolbars. GUI functions available from the View Control Toolbar and Query Toolbar are described below. See the key in Figure B1-8 :



Figure B1-8 View control and Query toolbar functions key

View Control Toolbar functions are (see Figure B1-8):

1. Zoom In
2. Zoom Out
3. Fit All
4. Last View
5. Next View
6. Redraw
7. Up
8. Down
9. Left
10. Right

11. Pad Display Enable/Disable

Query Toolbar functions are (see Figure B1-8):

12. Blinking Enable/Disable

13. Blinking Color setting dialog

14. Query Object - displays a 'Query Objects' dialog box that allows searching for design objects, specified either by name or by x,y location. Also allows highlighting multiple instances of objects that are listed in a specified file. The highlight color for objects to be found also can be selected.

You can find information about an object such as a transistor in the query dialog and highlight the desired object in the GUI. Bring up the dialog by clicking on the 'Query Object' button. The Query dialog is shown in Figure B1-9 below:

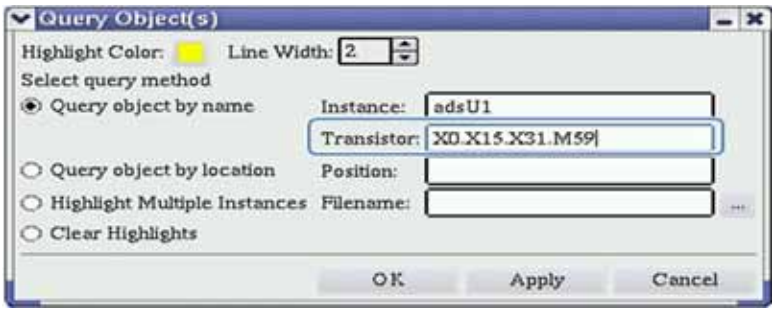


Figure B1-9 Transistor query by name

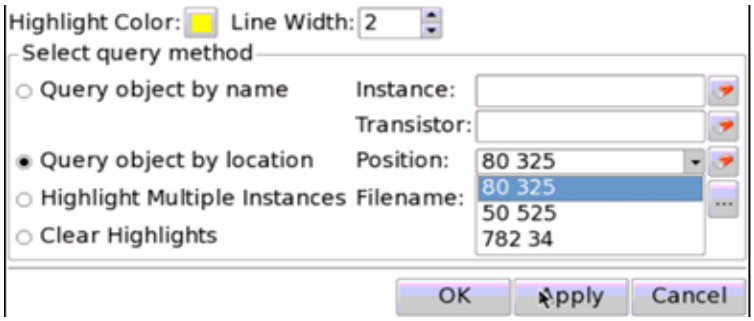


Figure B1-10 Multiple-query object by location

The last three position queries are saved and displayed on a pull down sub-menu, identifying the recently-queried positions, as shown in Figure B1-10. You can select any of the positions from the pull down menu, and then click on Apply or OK to zoom the Totem display to that region.

15. Clear Highlight - clears all highlights and rulers in the display

16. Preselect Query Enable/Disable

17. Instance Property - for multiple-instance SOC designs, displays a dialog showing key properties for a selected instance.

18. Snapshot - takes a screen shot of the full Totem window and saves it in a GIF file in the selected filename in the dialog box that is displayed. GIF screen shots can be displayed using the 'xv' utility in the Linux OS.

19. Switch LSW GDS Layer Display - switches LSW layer display

Status Toolbar

The Status toolbar displays cursor location, DvD, current, or EM information for a pre-selected transistor, wire, or via geometry as you move the cursor on the colormap, as shown in Figure B1-11.

Also the Status toolbar allows configuration of each frame for easy debugging. You can enable/disable Ruler Bar, Scroll Bar, and Frame Title, Sync all views, or only the current view.

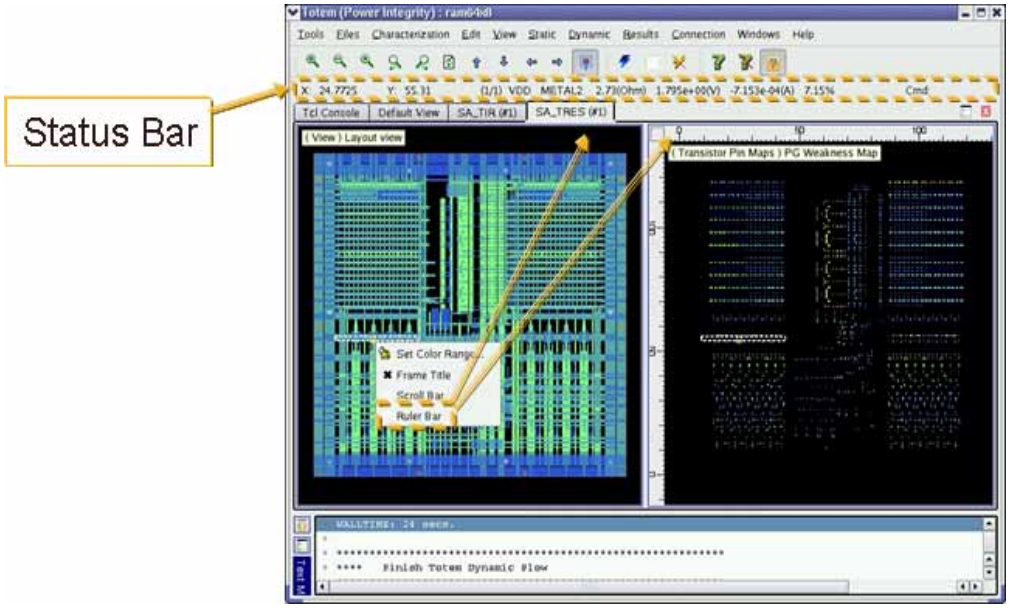


Figure B1-11 Easy frame and Status toolbar usage

Advanced Filter Function for Table Dialogs

Many table dialogs in Totem have a yellow filter wizard icon in the upper right corner, which displays a filter rule setting frame, and allows you to set up multiple advanced filtering rules to investigate and fix design issues more easily and quickly.

You can also set filtering rules column-wise. Each column has a different operation model. For example, for a Netlist (X,Y) column you can set “include/exclude” regions and widths, and the length columns allow you to set arithmetic >, >=, =, <, <= conditions.

After finishing setup, use the “Save As” button to set the filter rule name. Then click the “OK” button to create the filter rule and apply it.

Multiple selection and multiple highlighting

Multiple selections from a dialog, such as **List of Worst Transistor Pin Voltages,** can be performed using **Shift** or **Ctrl** + left mouse button, which highlights the selected pins, as shown in Figure B1-12.

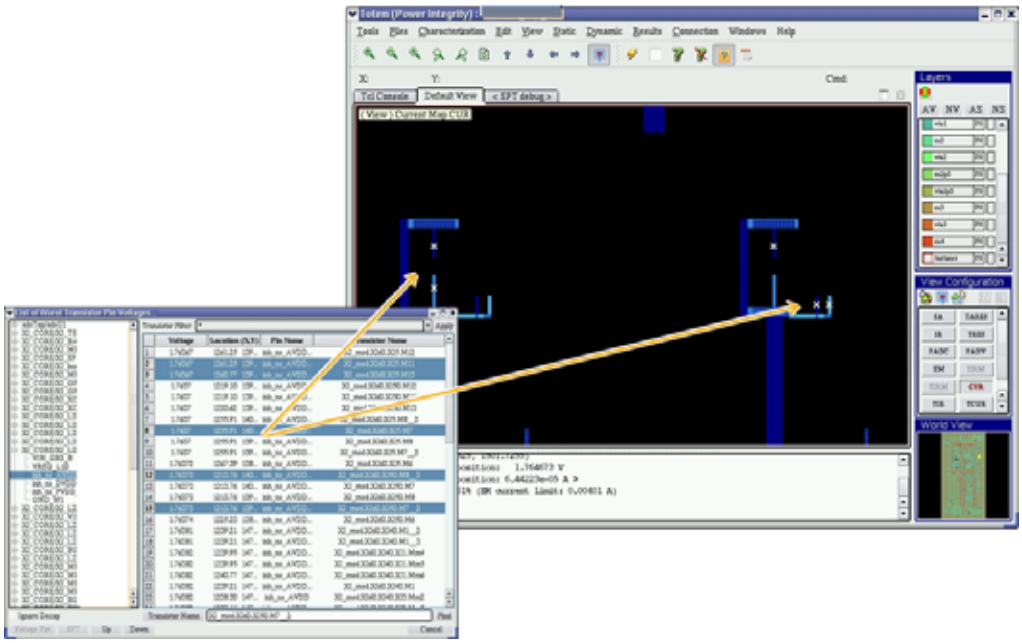


Figure B1-12 Multiple selection and multiple highlighting

GDS pre-selected cross-probing

Using the toolbar you can pre-select and select GDS cross-checking in the Totem layout view, as shown in Figure B1-13 below.

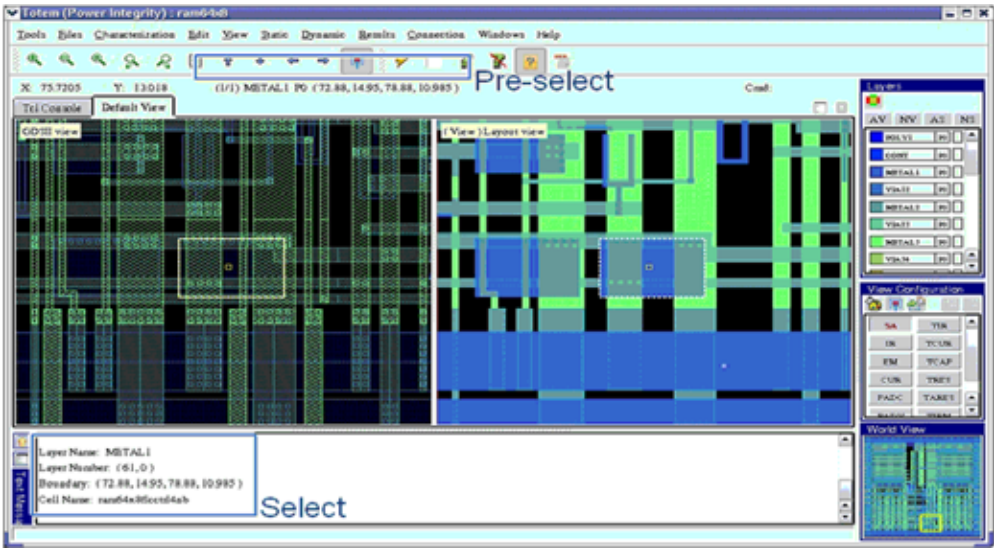


Figure B1-13 Preselection and selection of cross-checking

Customizable forms for all GUI table dialogs

The Totem GUI supports customizable GUI forms for all “List of **” dialogs. The following capabilities are available:

1. “Sort” function - click the header of each column to sort items in the column.
2. “Show/Hide” function - use a right mouse button to click on the header to pop up a menu to select Hide or Show function.
3. “Move” function: use the mouse to drag and drop the header of the column to a new position.
4. “Bookmark” function - use the bookmark function to show or filter bookmark entries.
5. “Up” and “Down” function - go “Up” and “Down” to browse through the table entries.
6. Auto Replay function - supports auto replay function for all GUI form changes.

SPT debugging from any pin-based colormap

The Shortest Path Tree (SPT) capability enables you to highlight the most resistive path from the selected transistor pin to the pad. You can examine the SPT in the GUI to determine any weaknesses in the design, such as missing vias. Use SPT debugging by selecting transistor pins in map view, clicking the middle mouse button, and selecting T**race SPT**, as shown in Figure B1-14.

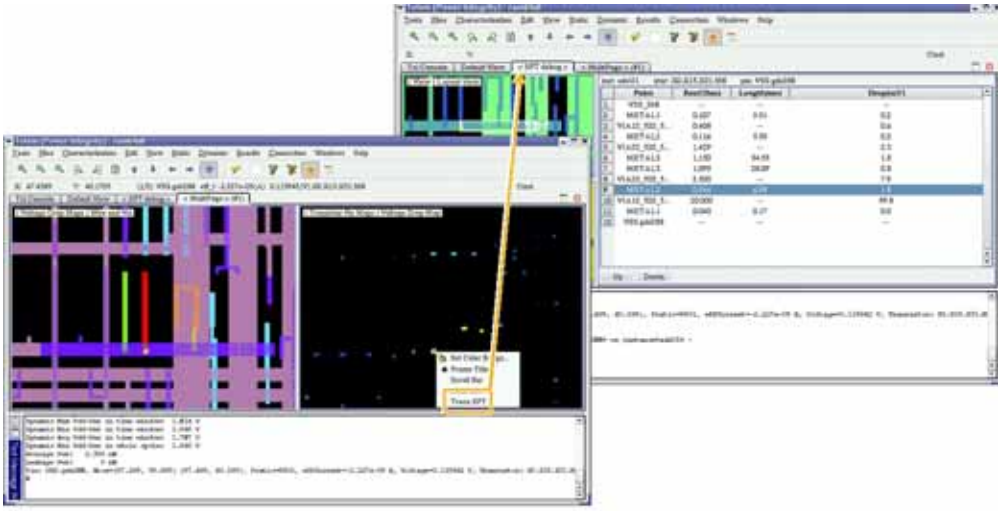


Figure B1-14 SPT debugging from any pin-based colormap

Net selection

You can exclude unselected nets, or include all nets in the design, directly from layout view window when debugging certain EM violations of selected geometry. Select the geometry and then click the middle mouse button to bring up a popup menu, then use the **Display Nets** option to choose either **Exclude unselected nets** or **Include all nets**, as shown in Figure B1-15. You can also use the bind keys **Ctrl** + **N** and **Shift** + **N** to achieve the same results.

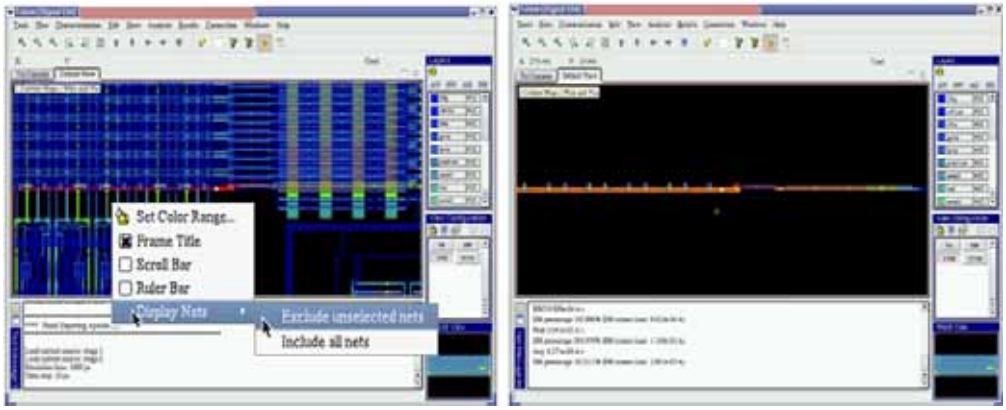


Figure B1-15 Net selection enhancement

Displaying maps of switch current and voltage

You can plot current and voltage maps across power switches in the design with different colors, based on the switch current and voltage values. Because this has native support, you do not have to create the maps using the 'select add' command. The TCL commands 'show sivm' and 'show sicm' allow you to prepare the maps quickly to see the voltage drop and current across the switches. The color range dialog box can be used to change the colors as desired. An example SICM command map display is shown in Figure B1-16:

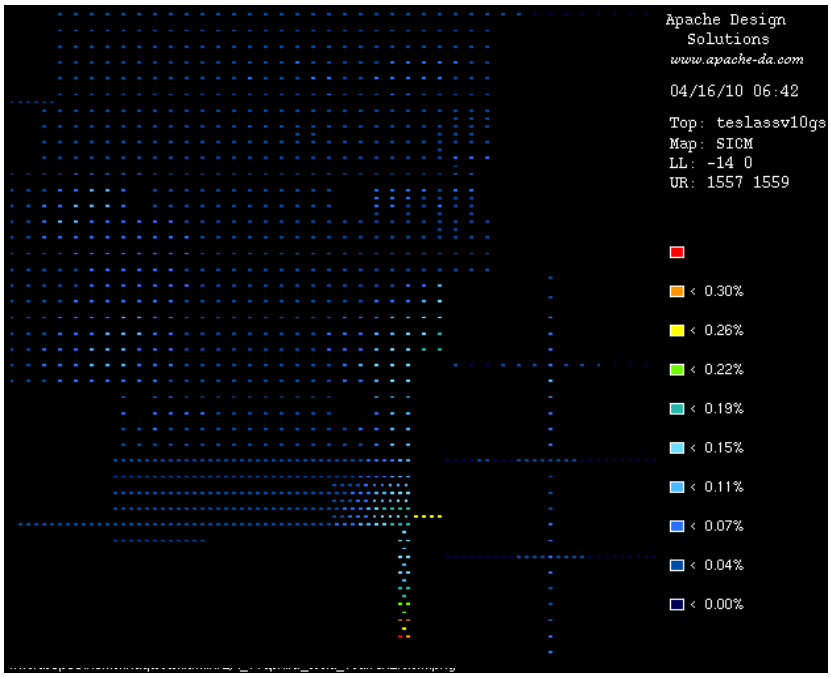


Figure B1-16 Switch current map

TCL Command Line

To use the TCL command line, click on the **Tcl Console** tab to the left above the Layout View window.

Text Message and Log Window

The text message and log window is below the Layout View window and displays all outputs from Totem.

Exporting Marker Coordinates from “List of ...” Dialogs

You can export marker coordinates of selected bookmark entries in a “List of ...” dialog to a Tcl command file, as shown in Figure B1-17. Then the marker Tcl command file later can be sourced in the Totem Tcl console to highlight the exported markers in GUI display. The procedure is as follows:

1. In the dialog box, click on the line number column to bookmark the entries to be exported.
2. Once all entries are bookmarked, click on the “TCL” icon in the menu bar. This pops up the export file dialog to allow you to define the export Tcl command file name.
3. Click the “Save” button to generate the marker coordinate Tcl command file.
4. Later source this marker Tcl command file in the Totem Tcl console to highlight the bookmarked positions.

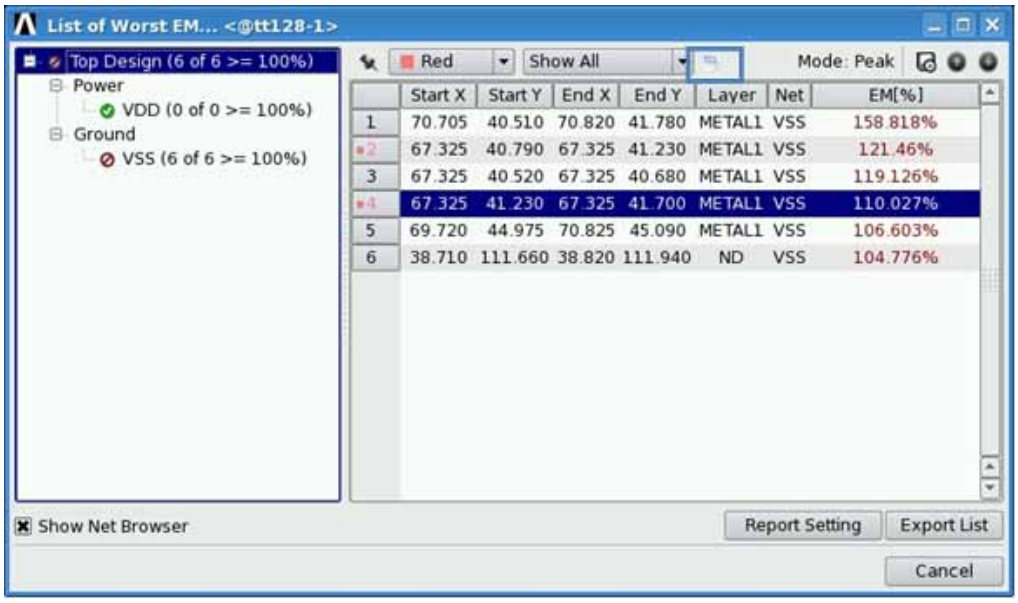


Figure B1-17 Exporting a marker coordinates file from a list dialog

Other GUI Support Features

Middle Mouse Button/wheel Functions

Pressing the middle mouse button/wheel with the cursor within the design brings up a menu with the following functions available:

- **Set Color Range** - see section "Set Color Range” (left Configuration icon)", page B1-553.
- **Frame Title** - displays View type at top of frame (default)
- **Scroll Bar** - displays the scroll bar

- **Ruler Bar** - overlays a 10 um x,y grid system on the design view
- **Display Nets -** choose “Exclude unselected nets” or “Include all nets”.
- **P2P Resistance Calculation** - displays a P2P Resistance Calculation dialog box to allow choosing ‘From’/’To’ points and other test parameters. See section "P2P Resistance Calculation", page B1-587, for more details.

Bindkey Support (Virtuoso^{tm}-like behavior)

Totem bindkey operation matches Virtuoso bindkey behavior. See section "Bindkey Functions", page B1-623, for a summary and information on reconfiguring bindkeys.

Bindkey behavior relative to the original Totem GUI are:

- Right mouse button zoom
  - Zoom IN - depressing the right mouse button and dragging from *left to right* creates a rectangular area that is zoomed in.
  - Zoom OUT - depressing the right mouse button and dragging from *right to left* creates a rectangular area that is zoomed out.
- Ruler control enhancement
  - Press preassigned bindkey **K** to enable ruler drawing.
  - Press left mouse button to start the ruler (no need to keep button depressed).
  - Move mouse in 45-degree angle to do 45-degree drawing.
  - Draw multiple connected straight-line segments by left clicking the mouse to end one segment and start a new one. This allows total length measurement of multiple segments.
  - End ruler drawing by double clicking the left mouse button, or single clicking the right mouse button.
  - While in ruler drawing, normal Zoom operation is available using the right mouse button, to improve ruler resolution or scope.
  - Use bindkey **Shift+K** to clear all ruler drawing.

Special GUI support

- **Edit ->Undo** and **->Redo**
  - **Undo** reverses the action performed by the previous command invoked from the **Edit** menu. If the **Undo** command cannot be performed, a message is displayed. **Redo** reverses the last **Undo** command performed.
- **GDS view** and **Netlist View** displays

  See section "View -> GDSII View", page B1-572 and section "View -> Netlist Hierarchy", page B1-590.
- **Windows** menu features
  - Displays Multiple Frames

    Allows setting up multi-frame configurations. See Figure B1-2.
  - Preferences

    Allows setting up GUI display preferences.
  - Toolbars

    Allows you to enable/disable “View Control Bar” and “Query Bar”.
  - Bring main window to top
  - Detach/Attach Tcl console

    Allows you to invoke TCL command and view GUI at the same time.
  - Detach/Attach LSW

Allows you to detach the layer selection panel (see Figure B1-3).

- **Preference** dialog box
  - Use bindkey **F10** to bring up the Preference dialog, with the following features:

Common Options

- GUI Font: (Sans Serif 9 Normal default)
- Text Message Font : (Sans Serif 10 Normal default)
- Tcl Console Font : (Courier 10 Normal default)
- Totem Explorer Font: (Sans Serif-8-Normal default)
- Highlight : Blinking or Color (default)
- Highlight Sync : Current View or All Views (default)
- Highlight Method : Auto Zoom (default) or User defined.

  Auto Zoom: zoom and center at the selected location.

  User defined: highlight the select location (marker), but not auto center, if the selected location is within the current zoom canvas. If the new selected point is outside current zoom view, the GUI uses current fixed zoom ratio and centers at the new selected point.
- Zoom sync : Current View or All Views (default)
- Waveform Viewer : xgraph or Totem Waveform Viewer (default)
- Waveform Viewer Integral Method : Left-Rectangle (default) or Trapezoid
- Color Maps : Smooth Color Spectrum or Disable GDS Instance in Color Map (default)
- Image background for printing: sets color of background for image printing (default black)
- Scroll Bar for Color Entries : 12 (default) - sets maximum number of color ranges before scroll bar is used.
- Editing Mode : Mode1 (default), or Mode2, which allows Virtuoso-style “what-if” editing functions to support via layers editing, such as adding and deleting vias.
- Auto Region Filter: Enabled (default) or Disabled

  Follow GSR: MULTI_THREADS” Setting checkbox

  Using Multi-threads Mechanism - you can choose to either “Follow GSR: MULTI_THREADS Setting” (default), or else choose to override the GSR keyword value using the “Enable Thread Number” checkbox to set a maximum number of threads to use.
- Text Message Window Options: Provides a ‘Show Summary Page’ check box for a summary of Process steps completed in the log area below the design display (default On).

GDSII Display Options

- Display Controls : “GDS Text” and “Path Borders” (both on by default)
- Array Display : Full (default), Border, or Source
- Filter Size : number of pixels (default 1)

Grid Controls

- Type : None or Dots (default)
- Minor Spacing: 0.002 (default)
- Major Spacing : 0.01 (default)
- Spacing Mode: User Specify (default), or Automation Detection

- X snap Spacing : 0.001(default)
- Y snap Spacing : 0.001(default)

User-preferred GUI settings

You can import your preferred default GUI settings by placing the Totem resource file *~/.totemrc* in your home directory, and then including the GUI configuration file in the Totem resource file. You can create the GUI configuration file by exporting it from an existing Totem GUI session using the 'export guiconf' TCL command. Then include the 'import guiconf <gui_configuration_file>' command in the *~/.totemrc* file, and place the *~/.totemrc* file in your home directory.

Power Integrity GUI Menus

The Totem GUI menu functions are summarized in the following sections. Totem Integrates Power integrity MMX, Signal EM, and ESD functions in the Totem platform, providing dynamic switching using the **Tools** menu to a product-specific set of GUI menus, as shown in Figure B1-18.

Tools -> Power Integrity

Selects power integrity setup and analysis mode, whose GUI features and menus are described in this appendix chapter.

Tools -> Signal EM

Selects signal EM setup and analysis mode. See GUI and menu descriptions in Appendix B2, "Signal EM GUI and Menus".

Tools -> ESD

Selects Electrostatic Discharge setup and analysis mode. See GUI and menu descriptions in Appendix B3, "ESD GUI and Menus".

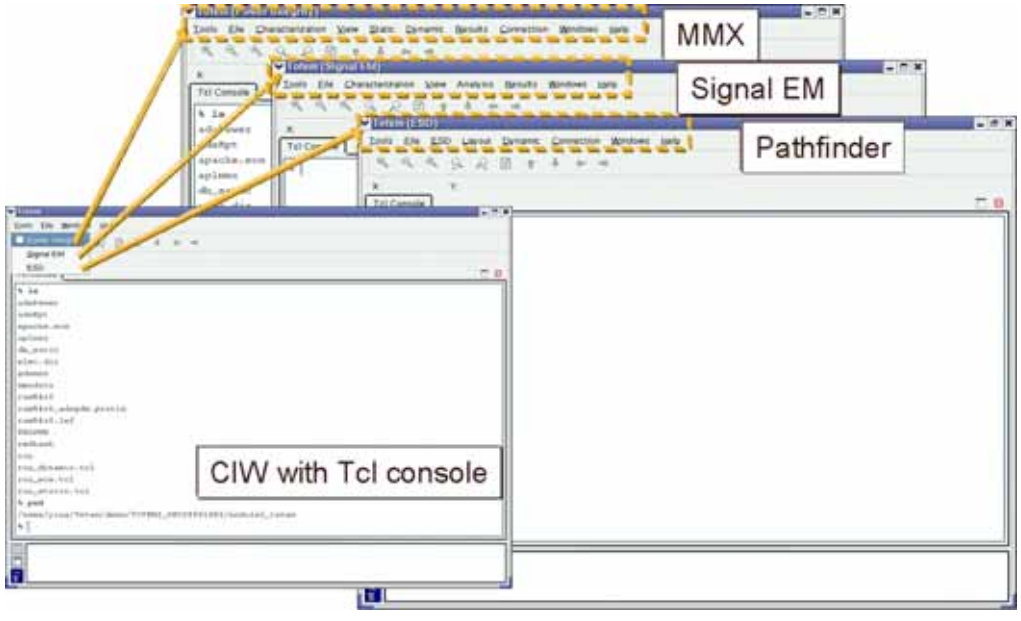


Figure B1-18 Product-specific menus

Files -> Import GDS

Displays an ‘Import GDS’ dialog box to specify the GDS file to be imported, as well as the Map File and the Top Cell. The dialog box also has fields for specifying Dictionary, Technology File and Display File.

Importing GDS View through Tcl interface

Totem also allows you to import the GDS file and display it in a GDS View using a TCL command, so you can set up batch TCL commands to import GDS and load the GDS View without using the GUI interface. The procedure to do this is as follows.

From the Totem GUI command line:

```
import_gds -gds <gds_file> [ -map <map_file> -top <cell>|
     -tf <Virtuoso_tech_file> -drf <Virtuoso_display_file>
    -vmap <Virtuoso_layer_map_file> ]
```

or

```
import_gds -cfg <configuration_file>
```

where the configuration file contains the TCL arguments

-gds <gds_file>

-tf <Virtuoso_tech_file>

-drf <Virtuoso_display_file>

…

open_gds -lib <lib_name> -cell <cell_name> -view <view_name>

From a Tcl batch file:

```
gui {
import_gds -gds <gds_file> [ -map <map_file> -top <cell> |
    -tf <Virtuoso_tech_file> -drf <Virtuoso_display_file>
    -vmap <Virtuoso_layer_map_file>  ]
open_gds -lib <lib_name> -cell <cell_name> -view <view_name>
}
```

For example:

```
gui  {
    import_gds -gds AAA.gds -map AAA.gds.map
    -tf AAA.tf -drf AAA.drf
  open_gds -lib AAA -cell top -view layout
}
```

Files -> Import Netlist

Displays a dialog from which you can select a netlist file to be imported.

The integration of Netlist Hierarchy View with Cross-Probing provides easy access to netlist information for debugging, and enables cross-probing with Layout View and other colormaps, as shown in Figure B1-19. When you select an entry in the Netlist Hierarchy view, the corresponding transistor pin is highlighted in the colormap. You can also select a transistor pin in the transistor pin map or SA view with the ‘pinInst’ check box enabled. To enable the pinInst check box, refer to Figure B1-3 on using the Layer Selection Window.

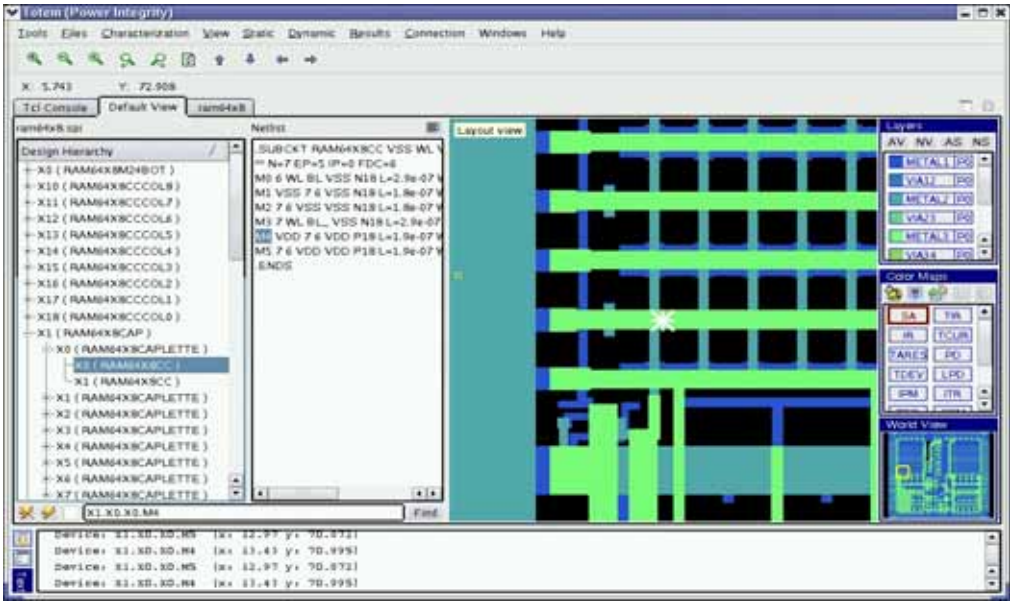


Figure B1-19 Netlist view and cross-probing

To import a netlist and display the netlist hierarchy view, use the command **Files -> Import Netlist**, and fill in the netlist filename, or you can click on **View Netlist** in the 'View Configuration' panel (first button from the right), and specify the netlist filename, which also brings up the netlist view. You can import a CDL format netlist in Netlist Hierarchy View to cross-probe with the Totem database and the Virtuoso Schematic Composer, or the Layout editor.

Files -> Import Design Data

Displays an 'Import Design Data' dialog box to specify the design data files to be imported. The dialog box has fields for specifying GSR, LEF, LIB, and Apache Tech files for the design. Package definition files PCELL, PLOC, and PAD can also be specified for import. If the correct path to the GSR or LEF files are specified, and the other files have the same path, click the button 'Use Same Prefix' to insert the same path in all file specification boxes. Selecting 'OK' then loads all specified design files.

This menu is grayed out after design setup.

Files -> Import Database

Displays an 'Import ADB' dialog box to specify the database file to be imported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as 'Create Dir', 'Delete File', and 'Rename File'.

This menu is grayed out after a database is imported.

Files -> Import ECO...

Displays an 'Import ECO' dialog box to specify the ECO file to be imported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as 'Create Dir', 'Delete File', and 'Rename File'.

Files -> Export Flatten GDS ...

Displays a dialog to allow exporting a flattened GDS layout to a specified GDS output file.

Files -> Export Database ...

Displays an ‘Export ADB’ dialog box to specify the database file to be exported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Export ECO ...

Displays an ‘Export ECO’ dialog box to specify the ECO file to be exported. The dialog also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Import Third Party Technology Data ...

This command allows easy import of third party technology data for cross probing. For example, in addition to Virtuoso Tech file and display file support, you also can import Virtuoso layer map files. With these three files from the Virtuoso setup environment there is no ambiguity when trying to map the Totem Layer Select Window configuration with Virtuoso to display the same color and drawing pattern. Figure B1-20 below shows the setup dialog.

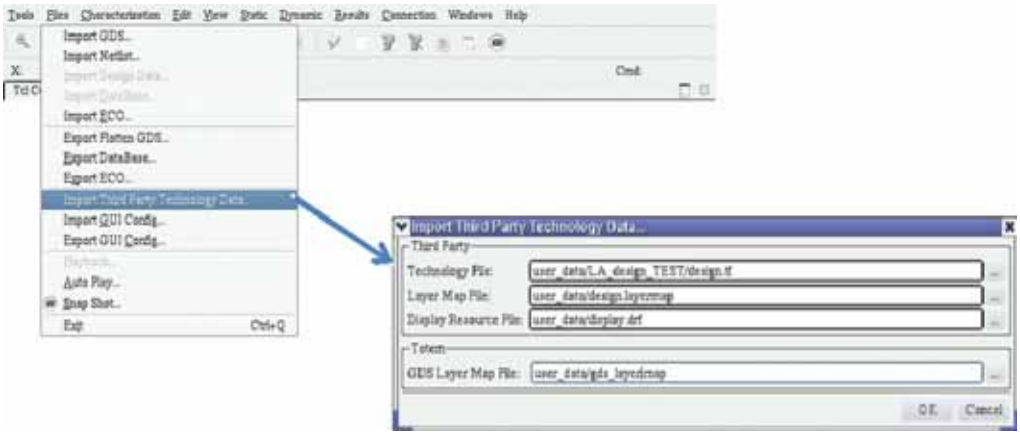


Figure B1-20 Displaying Virtuoso layermap files

Files -> Import GUI Config ...

Displays an ‘Import Configuration’ dialog box to specify the configuration file to be imported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Export GUI Config ...

Displays an ‘Export Configuration’ dialog to specify a configuration file to be exported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Playback ...

Displays a ‘Load Playback File’ dialog to specify the TCL command file of a previous Totem session to be loaded. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

The menu is grayed out after design setup.

Files -> Auto Play ...

Displays a dialog that allows you to specify a a Totem run file to play rerun a previous session in the GUI. Allows adjustment of the timing of GUI operations. Replay script files are located in the *adsRpt/Replay* directory.

Files -> Snap Shot ...

Takes a screen shot of the full Totem window and saves it in a GIF file with the filename specified in the dialog. GIF screen shots can be displayed using the ‘xv’ utility in the Linux OS.

Files -> Exit

Exits Totem session and saves all associated files.

Edit -> Undo

Removes the previous edit step.

The menu is grayed out until editing is performed.

Edit -> Redo

Removes the previous undo step.

The menu is grayed out until an undo step is performed.

Edit -> Add Pad

Displays an ‘Add Power/Ground Pads’ dialog box to specify new power and ground pads. You must select a Power or Ground pad type, the appropriate layer, and click on one or more legal pad locations to add pads.

Edit -> Delete Pad

Displays an ‘Delete Pad’ dialog box with a list of existing power and ground pads, their location and the layer they are on. Specify which power or ground pads on the list should be deleted, and then click on ‘Delete Pad’.

Edit -> Add Strap

Displays an ‘Add Strap’ dialog box to specify new power or ground straps. You can specify the power net (‘Available Nets for power’, ‘Available Nets for ground’), ‘Strap input method’ (‘Add strap by text input’), ‘Horizontal’ or ‘Vertical’ orientation, ‘Strap start’ x,y location, ‘Strap end’ x,y location, and ‘Strap width’. You can add multiple straps by x,y range and pitch, and specify the appropriate metal layers and associated vias.

Edit -> Edit Strap

Displays an ‘Edit Strap’ dialog box to specify modifications to existing power or ground straps. After clicking on the strap to be modified, information about it appears in the dialog box: ‘Net’, ‘Wire’ (layer), ‘Length’, ‘Width’, ‘Strap left lower x’ (x location at lower left end of strap), ‘Strap left lower y’ (y location at lower left end of strap) and ‘Resistance’. When the correct information on the strap to be edited is displayed, the strap can be modified using the ‘Delete’ or ‘Change’ buttons.

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To stretch a rectangle (supports GUI operator “8 control points stretch” component):

a. Select a rectangle and choose **Edit -> Edit Strap**, or use bindkey 's'.

b. The editing coordinate resolution is set based on “X Snap Spacing” and “Y Snap Spacing” settings in the “Preference” dialog.

c. Once the change is done, click on “Change” button in “Edit Strap” dialog to commit the change.

d. Use the “ESC” key to escape the edit command.

e. Bring back previous edit command mode by using right-mouse-button.

f. Once the change is done, click on the “**Change**” button in “Edit Strap” dialog to make the change.

Edit -> Add Via

Displays an ‘Add Via’ dialog box to specify new vias to be added. After clicking on the location for a new via, information about the layers at that location appears in the ‘Select Metal Layers’ panel. After selecting the appropriate layers for the via, the available vias for the selected two layers appear in the ‘Available Via Type Lists’ panel. Select the appropriate via type and then execute the addition.

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To add a via:

a. Select the via layer to edit in the LSW.

b. Use menu command **Edit -> Add Via**. A list of available via models on the selected via layer is shown in the **Add Via** dialog.

c. Select one of the via models; the dashed line bounding box of the selected via model is displayed in GUI display.

d. Use the left mouse button to place the selected via model in the GUI layout window to drop the via.

e. You can select the **Fill** option in the **Add Via** dialog to automatically fill the entire top metal and bottom metal overlapping area by selected a via model.

Edit -> Delete Via

Displays a ‘Delete Via’ dialog box to specify existing vias to be removed from the design. Displaying on vias on particular layers using the ‘View Layers’ button will help select the vias desired. After selecting the via to be deleted, its Net and Via Name are identified in the dialog box. If this is the desired via to be deleted, click on the **Delete** button to execute the command.

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To delete a via:

a. Select the via layer to edit in the LSW (Layer Selection Window).

b. Use the menu command **Edit -> Delete Via**.

c. Select the via geometry in the GUI layout window using the left mouse button. The selected via geometry is displayed in the **Delete Via** dialog box.

d. Click the **Delete** button to delete the selected via.

Edit -> Add Decap Cell

Displays an ‘Add Decap Cell’ dialog box to specify new decoupling capacitors to be added. After selecting the power or ground net, click on the location for a new decap, and information about the layers at that location appears in the ‘Select Vdd’ or Select Gnd’ panel. The available decap cells appear in the ‘Available Decap Cells List’ panel. Select the appropriate decap cell type and then execute the addition using the ‘Commit Adding’ button.

Edit -> Add/Delete Probe

Displays an ‘Add/Delete Probe’ dialog, which shows probes already specified in the GSR PROBE_NODE_FILE. By clicking on a probe item, you can see the position highlighted in the GUI (multi-select and multi-highlight is also possible). To add new probes, the co-ordinates can be input by selecting ‘Add’, selecting the layer in the ‘Add Power/Ground Probe’ dialog, and then clicking on a valid probe location in the design window. Also, unneeded probes can be deleted by clicking on the delete icon next to the probe entry to be deleted.

Edit -> Add Rectangle

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To draw a new rectangle:

a. Select a metal layer in LSW window to activate a working layer.

b. Choose one net from the “Net” selection menu of the “Add Rectangle” dialog.

c. Click the left-mouse button twice to specify a corner of the rectangle.

d. Press and hold the left-mouse button, tracing a box, then release the button.

e. Display and hide the “Add Rectangle” dialog using bind key “F3”.

f. Use the Delete (Backspace) key to reset a clicked point.

g. Use the “ESC” key to exit the edit command.

h. The editing coordinate resolution is set using the “X Snap Spacing” and “Y Snap Spacing” settings in the “Preference” dialog.

i. Bring back the default edit command mode using the right mouse button.

This menu is grayed out in Mode1.

Edit -> Delete Objects

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To delete a rectangle (both vias and wires can be deleted):

a. Select rectangle bounding box to be deleted.

b. Press Delete.

c. Use the “ESC” key to exit the edit command.

d. Bring back the previous edit command mode by using right-mouse-button.

This menu is grayed out in Mode1.

Edit -> Create Ruler

After turning on the ruler function, clicking the left mouse button on an x,y location on the design and dragging it to a second point on the design creates an x,y axis with lengths in microns. For easily measuring long distances on a design with precision, press the SHIFT key and click the left mouse button to define the start point and the end point of the measurement. The location of the intersection of the x,y axis and the end point are displayed on screen.

Edit -> Clear All Rulers

All drawn rulers remain displayed until the ‘Clear All Rulers’ button is selected.

View -> GDSII View

After specifying GDS setup in the GSR file, this command brings up a **GDSII View** dialog**.** In the dialog select the Library in the **Library** section, and a list of cells is displayed in the **Cell** column. After selecting a cell to display and the **Show Library Information** box, and Open to display the GDSII view dialog.

If you specify the top cell when loading the GDS View setup dialog, the GDS View library, cell and view selection dialog setup step is automatically skipped and GDS View is displayed directly.

Note that The GDS layer map file input is optional, so you do not need to prepare a GDS layer map file when using Totem as a standalone GDS Viewer.

The viewer also provides LSW cross-probing between Apache Tech layers and GDS layers if you import a GDS layer map file during setup.

The GDS viewer converts GDSII to GDS view with GDS layer selection capability, as shown in Figure B1-21.

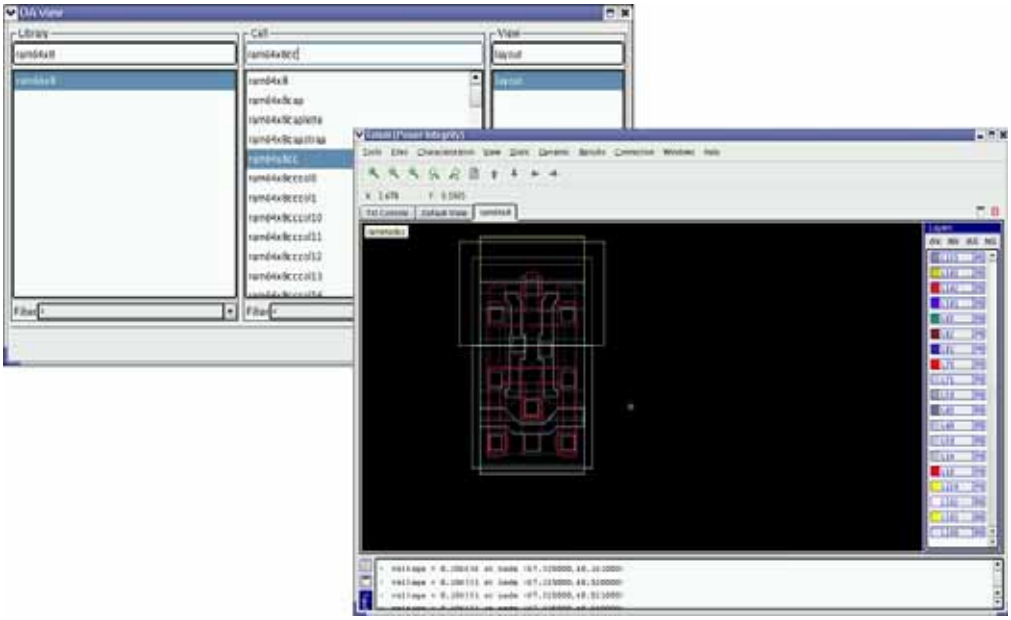


Figure B1-21 GDS view

You can change the default GDS view setting by bringing up the ‘Preference’ dialog (F10 key).

The viewer also provides LSW cross-probing between Apache Tech layers and GDS layers if you import a GDS layer map file during setup. See examples of these special GDS Viewer functions in Figure B1-22 through Figure B1-29.

GDS hierarchy browser

The GDS Viewer has a GDS Hierarchy Browser to allow you to cross-check GDS hierarchies. In the GDS View, click the middle mouse button to invoke the GDS Hierarchy Browser, as shown in Figure B1-22.

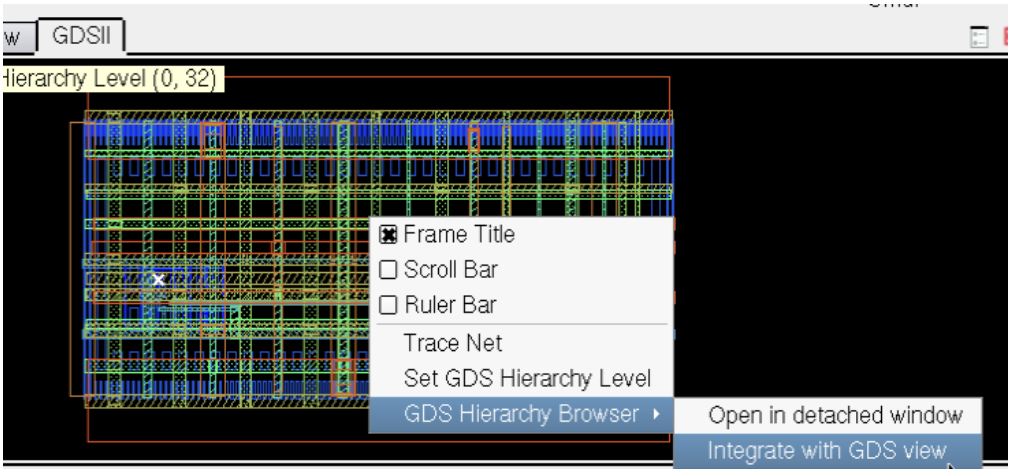


Figure B1-22 Bringing up the hierarchy viewer

You can choose to either integrate the GDS hierarchy browser with the GDS view or view it as a standalone dialog.

The standalone hierarchy display is shown in Figure B1-23.

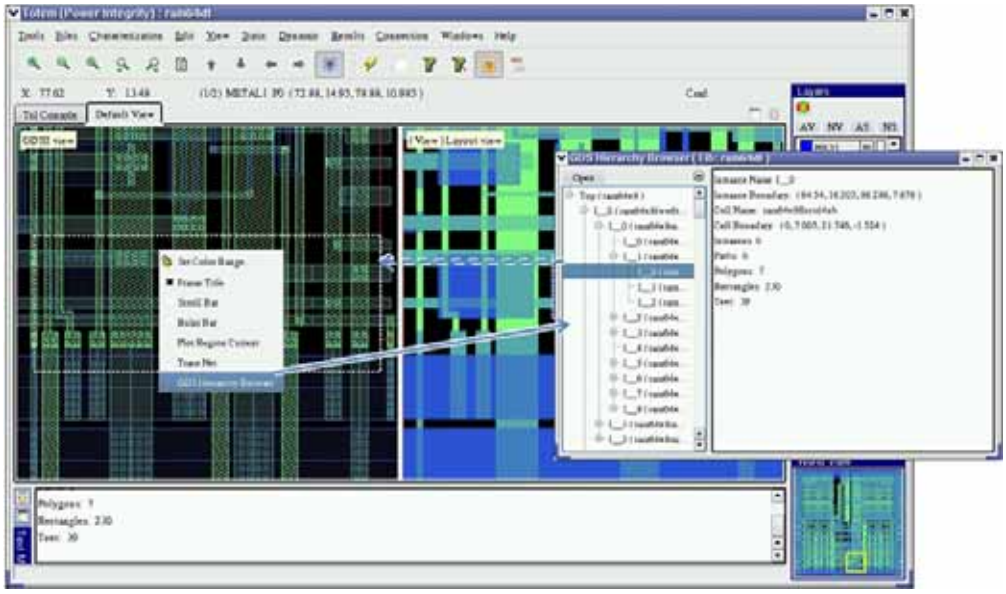


Figure B1-23 GDS hierarchy browser

Figure B1-24 shows the integrated hierarchy browser.

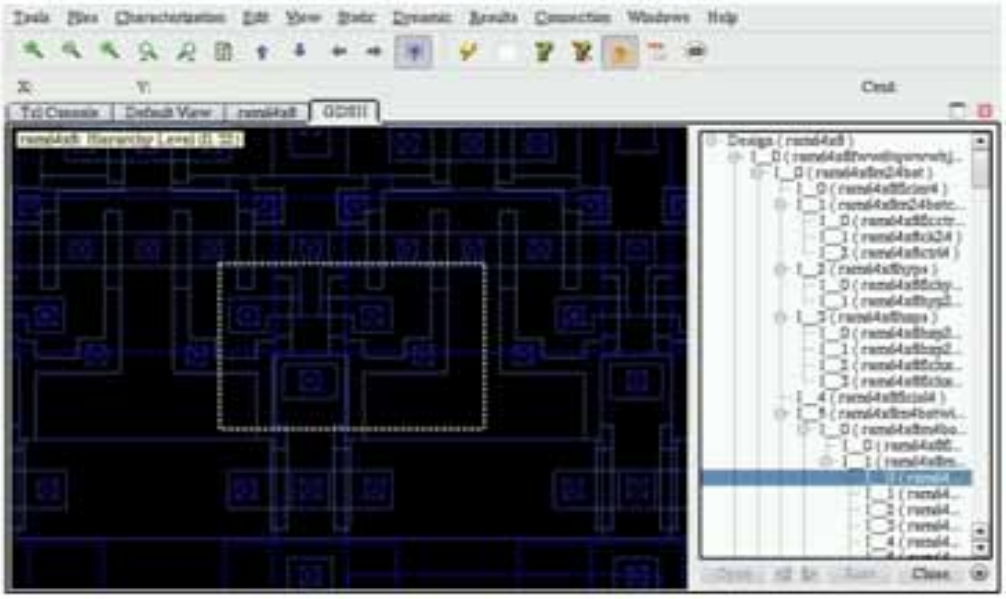


Figure B1-24 Integrated GDS view and hierarchy browser

GDS hierarchy boundary display

To make GDS debugging easier, the Totem GUI allows you to highlight GDS instance hierarchy boundaries. You can then use the bindkey combination **Shift + F** to open the full flattened GDS hierarchy in the GDS view. Use **Shift + V** keys to go down one hierarchy and **Ctrl + V** keys to go up one hierarchy. A display is shown in Figure B1-25 below
:

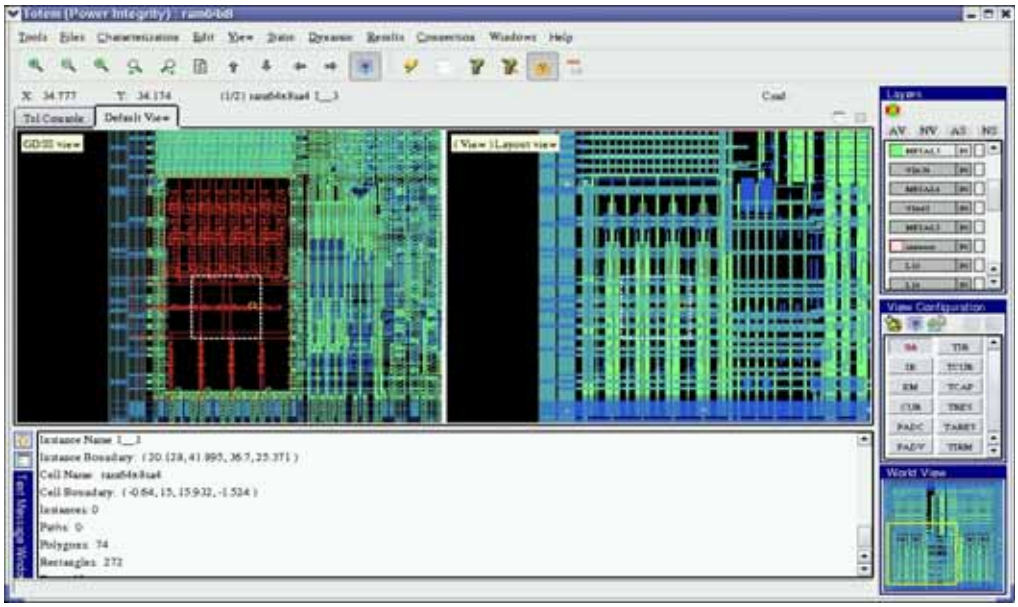


Figure B1-25 GDS hierarchy boundary display

GDS View-based P2P Resistance Checking

You can directly perform point-to-point resistance checking from GDS View, without going through a detailed GSR and TCL command setup, using the following steps:

1. Without doing design setup, go to **Files -> Import GDS,** to load the GDS file and GDS layer map file.
2. Go to **View -> GDSII View** to open GDS View.
3. After GDS View is loaded in the Totem GUI, use the middle mouse button to bring up the dialog box and select the **GDS P2P Resistance Calculation** entry.
4. The **GDS P2P Resistance Calculation** dialog is shown on the screen to allow you to set up a GDS file, GDS layer map file, Apache Technology file, Net name, and **From**, and **To** resistance end points, as shown in Figure B1-26. You can also define a resistance threshold constraint to be checked.
5. You can either manually define the **From** and **To** points in the dialog, or use the mouse to click the geometry objects directly in the GDS View to define the points.
6. Then click the **Apply** button to start the calculation.
7. Totem automatically sets up the GDS configuration file, GSR, and Tcl commands based on your input in the previous steps, and submits a child process to perform GDS2DB tracing, setup design, extraction and resistance calculation.

   Similar to the Save/Reload flow, the same license is shared between child process (setup design to perform resistance calculation) and parent process (GDS View). And all the child process setup is done in a separate sub-directory, under *<working_directory>/p2p<id>*, so you may check all tcl, GSR, configuration setup in the directory to debug.
8. After the resistance calculation is done, you can see the minimum resistance path GUI display between the two selected points. If the resistance constraint is defined during setup, both the effective resistance results and constraint are displayed at the header of the SPT window. If there is a resistance constraint violation, you can continue to use the child process GUI to debug the existing resistance path to identify if there are any design issues.
9. If you want to perform another P2P resistance check on the same net, continue to use the child process GUI to select the next “From/To” points using the middle mouse button pop-up menu, and repeat the process using steps 4 through 8. This can save time in avoiding the GDS trace design setup and extraction.
10. If you are finished with the results and want to quit the child process GUI, Totem automatically returns to parent process GDS View setup.
11. You can also switch to the **Result** page of the **GDS P2P Resistance Calculation** dialog to check the previous P2P resistance check results, or re-submit the previous setup.

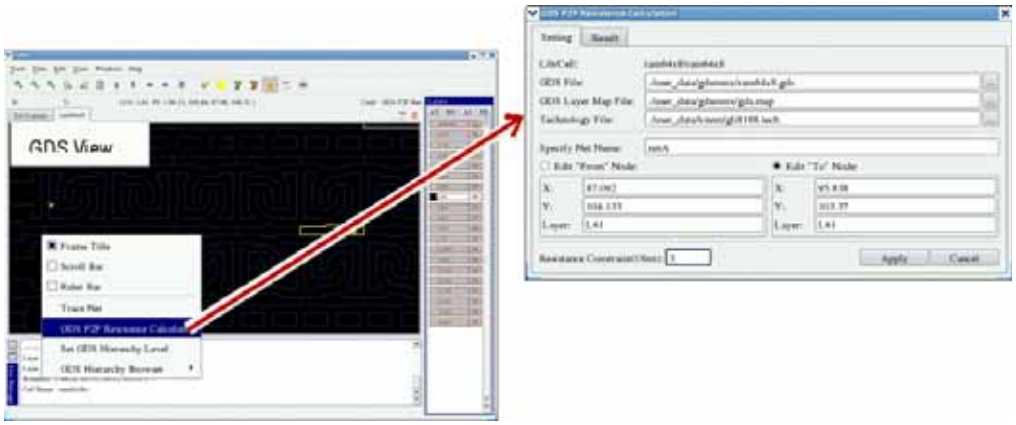


Figure B1-26 P2P Resistance calculation

GDS View integration with Layout Cross-Probing

Allows checking signal routing and cross-probing with GDS view for easy debugging, as shown in Figure B1-27.

You can import GDS after the ‘setup design’ step with one of the following methods:

1. a. Select the menu command **Files -> Import GDS.**

   or

   b. Click on the **View GDS** button in the **View Configuration** panel (second button from the right).

2. A dialog box is displayed. Fill in the GDS filename, the GDS layer map filename, and the top cell name.

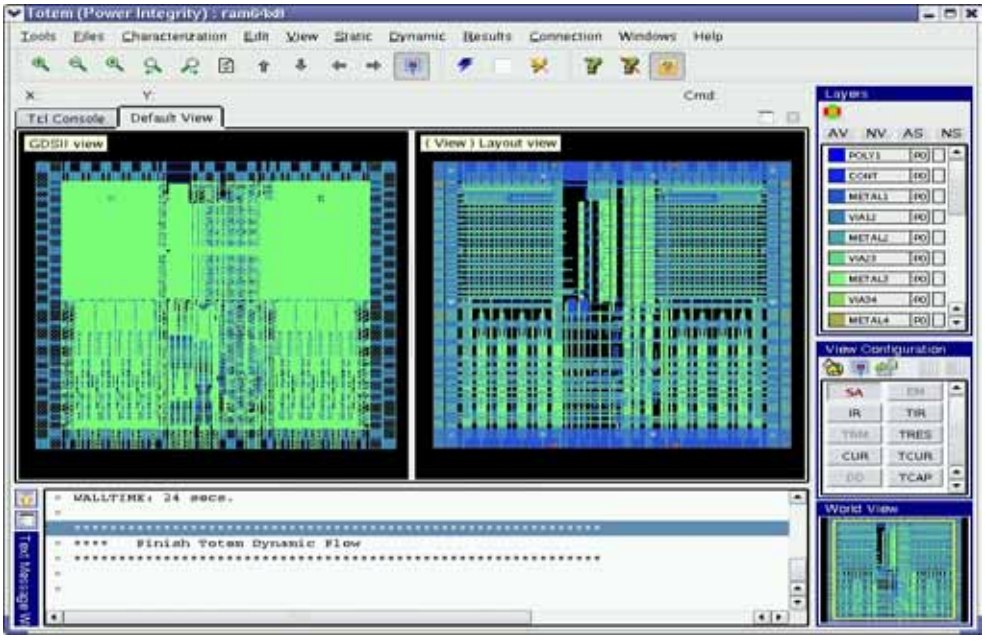


Figure B1-27 GDS and layout view cross-probing

GDS viewer text search and net tracing

To find the GDS “start tracing” point and debug GDS tracing issues, Totem has text search and net trace capabilities. You can click **Trace Net** on the middle mouse pop-up menu to invoke the Trace Net dialog to search text, or set up GDS layer mapping and trace rules. An example is shown in Figure B1-28 below.

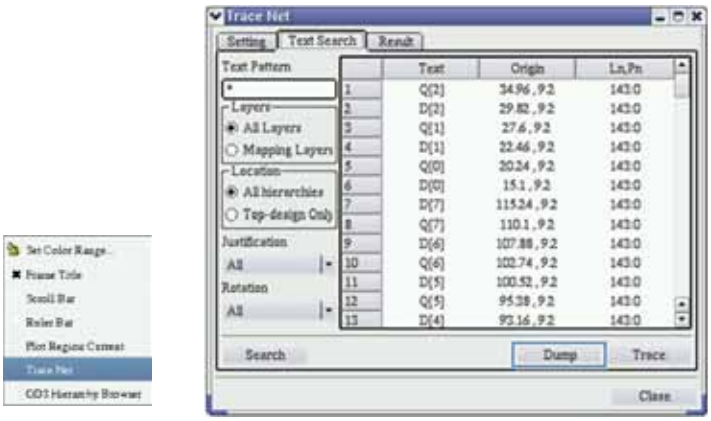


Figure B1-28 Text search and net tracing

You can select text entries in the text search table and click the **Dump** button to stream out GDS “start tracing” points in an output file. Then you can copy and paste the contents of the file into the gdsmmx/gds2db configuration file in the VDD_NETS and GND_NETS keywords, to define the selected “start tracing” points.

You can also click on the **Trace** button to start GDS tracing from selected text entries in the table, or click on any geometry in the GDS view to start tracing. The trace result is then displayed on the Result page of the Trace Net dialog, including Visible control, Color control, Delete Control, Trace Progress display and trace stop functions, as shown in Figure B1-29.



Figure B1-29 Net tracing results display

View -> OA Database View

You can use the GUI to view an OASIS database library layout view, the same as the GDS view by selecting **View -> OA Database View**, clicking on a *lib DEF* OA library header file from the list in the dialog, and then selecting one cell to check its layout view, as shown in Figure B1-30.

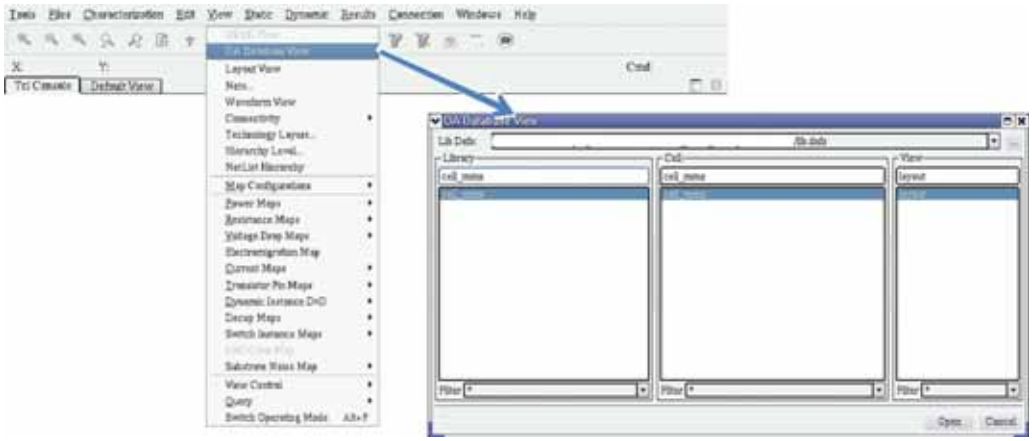


Figure B1-30 OA library layout view

View -> Layout View

Performs the same “show layout view” operation as the **SA** button in the ‘Configuration buttons’ panel. Does not change the area of the design viewed.

View -> Net Display Map

Displays all nets in different colors.

View -> Nets...

Displays a ‘Nets’ dialog box with lists of VDD and VSS nets. The dialog allows you to select and highlight particular nets by name or by type: VDD, VSS, Clock, or Signal (if extracted). Nets of a selected type also can be chosen by name to assist in finding a desired net in a very long list. Also displays instances and pins if they are selected in the ‘Layers’ dialog.

In addition to viewing regular nets in the **View -> Nets** dialog, if the design includes metal filler, you can select the check box **Exclude Metal filler** to exclude them, or go to any color view and use the middle mouse pop-up menu to select the **Display Nets -> Exclude metal fill net** check box.

You can also set generic name-based net filtering in the **View -> Nets** dialog to switch “Enable Net By Name” or “Exclude Net By Name” functions to include or exclude net names. Wildcard names are supported.

The check box “GSR only” allow you to only display nets defined in the GSR. Also, internal nets are highlighted in dark blue within the “Nets” selection dialog box to help distinguish internal nets.

View -> Waveform View

In standalone mode displays the waveform viewer from CIW dialog and imports a waveform file without loading the database. The command supports multi-page, multi-frame displays to provide a more user-friendly debug environment for cross-probing. The waveform viewer supports *.wdb, taO, and *.tr0 files in standalone mode to display and

perform waveform operations, as well as case insensitive searching in the signal browser. Copy and paste functionality in the signal browser of the waveform viewer is also supported.

Figure B1-31 below shows the standalone waveform viewer.

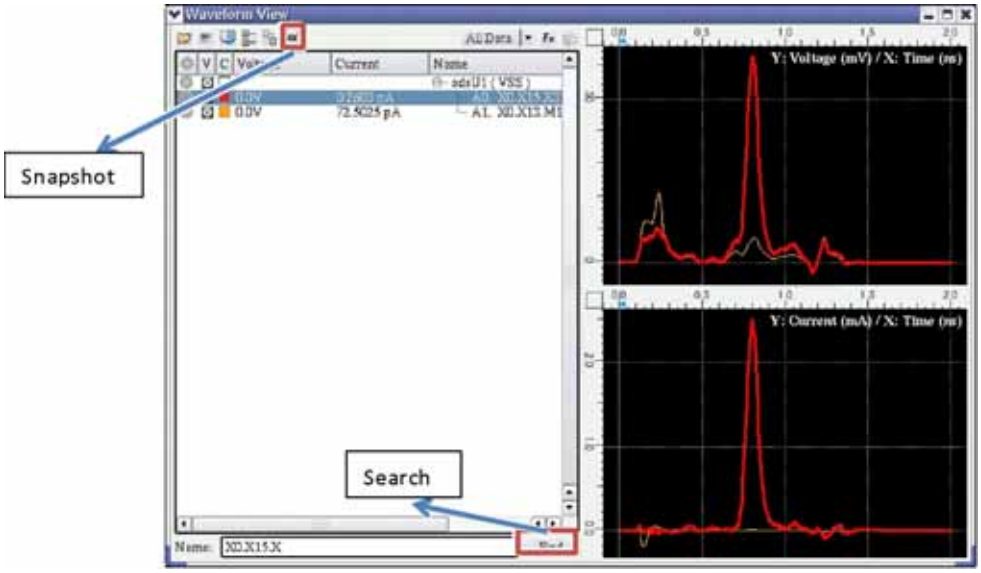


Figure B1-31 Standalone waveform viewer

The integrated waveform mode helps debug a hot spot or other design issue more easily. The integrated waveform viewer dialog allows you to cross-probe transistor pin voltage/ current waveforms with other DvD, IR, and EM maps, and movie results, as shown in Figure B1-32.

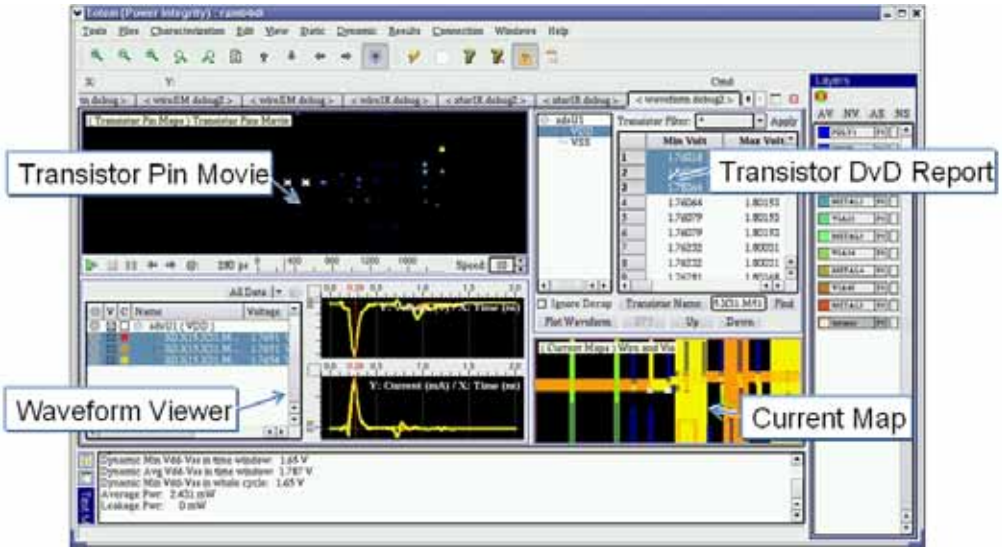


Figure B1-32 Integrated waveform viewer

You can plot two types of waveform views, transistor voltage and current waveforms, and also regional current waveforms, and they can be executed in different ways.

The functions of the waveform viewer are described in the following section.

"Add Page" function

Click on the green "**+**" icon on the left side of the Waveform Viewer dialog to add a new page.

"Delete Page" function

Click on the red "**x**" icon on the left side of the Waveform Viewer dialog to delete the current active page. The default page cannot be deleted.

"Add Frame" function

Except for the default page, you can add new frames on a non-default page. The blue frame icon on the left side of Waveform Viewer dialog allows adding a new frame to the current active page.

"Delete Frame" function

In the frame you want to delete, click the middle mouse button to bring up a pop-up menu and select "**Delete Current Frame**".

Adding a signal in a frame

You can drag a signal from the Signal Browser on the left side of the Waveform Viewer and place it in a selected frame. Newly-added signals are displayed in the header of the frame. If there is more than one signal in the frame, only the symbol of the signal (A0, A1, C0, C1...) is shown in the header. Totem only allows one type of signal (voltage or current) inside a frame.

To select the voltage waveform of given transistor from signal browser, place the cursor at the top of the Voltage column of the Signal Browser to drag. Similarly, you can select a current waveform by putting the cursor at the top of the Current column of the Signal Browser to drag.

To sync the active view page with the signal browser

Click on the "**[View All | View Page]**" button at the top of Signal Browser. You can switch these two modes to see all, or see just the active page signals, in the Signal Browser.

To show, hide, or delete signals

You can select a visible icon in the Signal Browser to show, hide or delete.

Case sensitive/insensitive search

You can toggle the "**[Insensitive Name | Sensitive Name]**" button to control the case sensitivity of the transistor name search function.

To copy the signal name

You can click on the signal in the Signal Browser and use the right mouse button to bring up the pop-up menu and select "**Copy Signal Name**".

Calculation Function

You can use the calculation function in the waveform viewer to perform statistical measurements, such as calculation of average, RMS (root mean square), and peak current within a selected time range, as shown in Figure B1-33. Or you can use the integral function to calculate total charge, as well as perform linear or formula-based waveform operations.

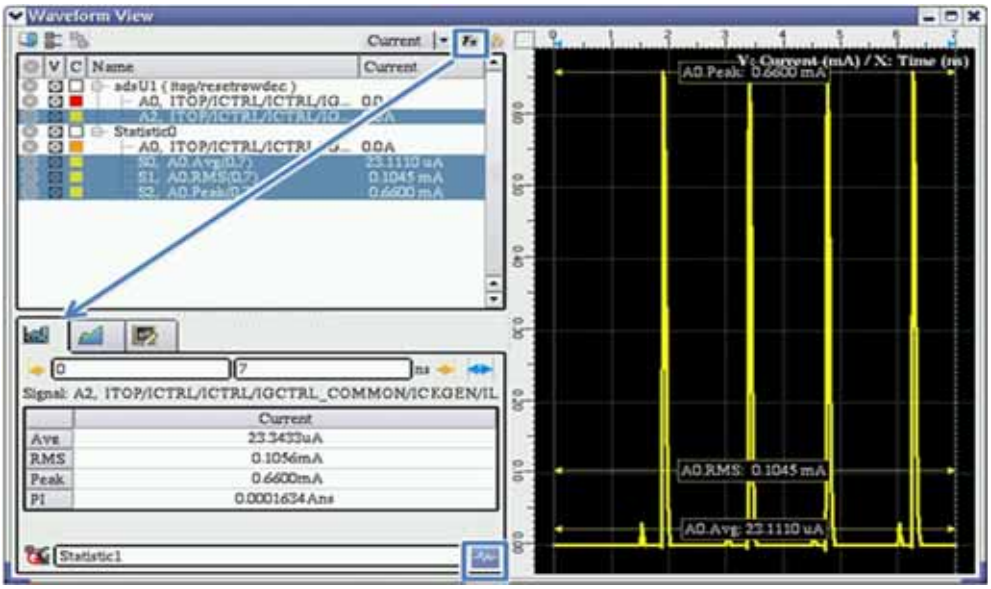


Figure B1-33 Waveform viewer calculation function

To invoke the waveform viewer calculation function, click on the **Fx** button There are three additional sub-pages displayed in the Waveform Viewer, as follows:

Statistics

- To calculate average, RMS, peak, or an integral of the selected waveform within a specified time window.
- To change the start time of the time window, use the left mouse button to click on the time axis, or directly input the value beside the left yellow arrow.
- To change the end time of the time window, use the right mouse button to click on the time axis, or directly input the value beside the right yellow arrow.

You also can click on the **Vis** column to control which measurement functions are enabled. And you can click on waveform **Apply** button in the lower right corner of the page, which displays the measurement results in the waveform view.

Linear

- To apply linear equations to selected signals.
- You can click the **Apply** button in the lower right corner of the page to apply the linear equation and the resulting waveform is displayed in the waveform view.

Formula

- To apply a user-defined formula to selected signals.
- You can click on the **Calculator** button in the top right corner of the page to bring up calculator view, or directly type in the formula in the center input box.
- You can click on the **Apply** button in the lower right corner of the page and resulting waveform from the formula calculation is displayed in the waveform view.

FFT Function

To use the FFT function (see Figure B1-34):

1. Select a signal in the signal browser.
2. Select the **FFT** icon on the top right of the signal browser.
3. Input the start time and end time in the pop-up dialog to define window range for FFT transformation.
4. Adjust the sampling rate and maximum frequency range.
5. Click **OK** to start FFT transformation.
6. Once FFT transformation is done, a new waveform viewer window pops up to display the frequency domain waveform.

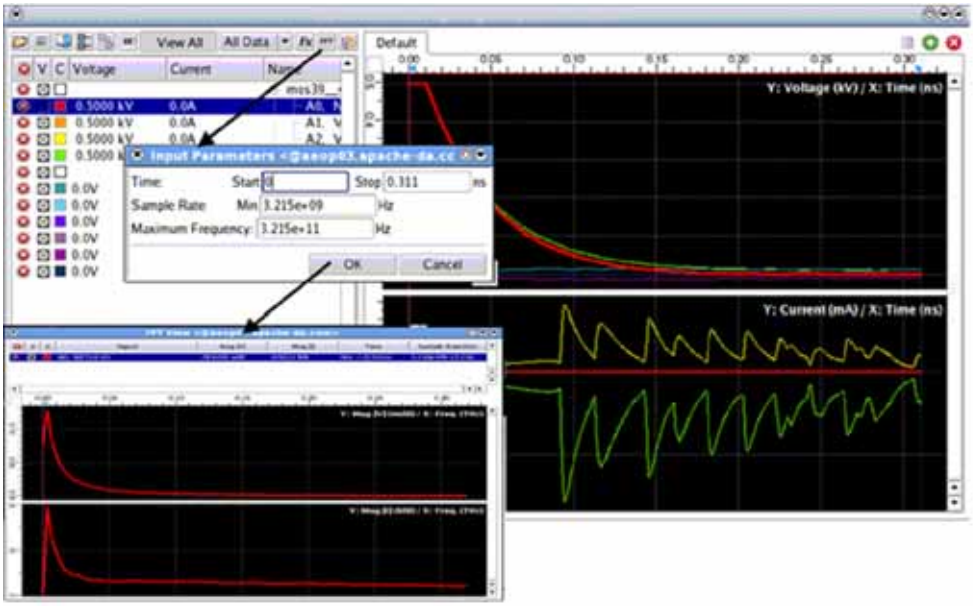


Figure B1-34 FFT function

Examining Waveform Views

Zoom Full. Use the middle mouse button to pop up a menu inside the Waveform Viewer and select Zoom Full, which zooms out to the maximum. You can also use the '**F'** bindkey in the waveform viewer to zoom to Full if the waveform view is not in multi-frame page mode.

Zoom In. Use the right mouse button to drag from left to right, creating a rectangular box defining a zoom-in area, the same as in the regular layout window.

Waveform Zoom-In X-dimension only. Use the right mouse button to draw a horizontal line from left to right indicating the desired zoom range.

Waveform Zoom-In Y-dimension only. Use the right mouse button to draw a vertical line from top down indicating the desired zoom range.

Zoom Out. Use the right mouse button to drag from right to left, creating a rectangular box defining a zoom-out view (zooming out by the ratio of the view size to the drawn rectangle size), the same as in the regular layout window.

Getting Waveform Details. Use the left mouse button to click on the waveform view at a location of interest. A red vertical line is displayed, along with the voltage and current at that time in the left side waveform signal browser. You can click on the header of the Current and Voltage columns to choose maximum or minimum value sorting modes.

Cross-probing Between Layout View and Results Table. When you select a signal inside the signal browser on the lefthand side of the Waveform Viewer, the layout view zooms to the selected region or transistor pin location, and the waveform is highlighted.

Manipulating Views in the Signal Browser. You can change waveform colors, make the waveform invisible, or delete a signal in the signal browser using the available check boxes in the browser. You can delete all signals by clicking on the header of the delete column (“X”). A popup dialog asks for confirmation of the deletion request.

Set x-axis and y-axis range. Resets the waveform view x-axis and y-axis range to help you zoom in or zoom out. You can use the middle mouse button in the waveform view to bring up a popup menu to check this feature.

Restore range to default. Restores the original default x-axis and y-axis range settings

Change waveform curve width, style, symbol. You can use the right mouse button in the signal browser of the waveform viewer to bring up a dialog to reset width, style, and symbol for selected signals.

Plotting region-based current

In any color map or movie, use the middle mouse button to pop up the menu and select **Plot Region Current**. You can either Use Existing Zoom Area, or Define Region by Left Mouse Button. Then plot region current either by using the existing zoom area in the layout window or by using the left mouse button to define a region to plot the current waveforms, as shown in Figure B1-35.

Note that the region current waveform is the sum of all current sources within the chosen region, *not* the current on the wire inside the layout window.

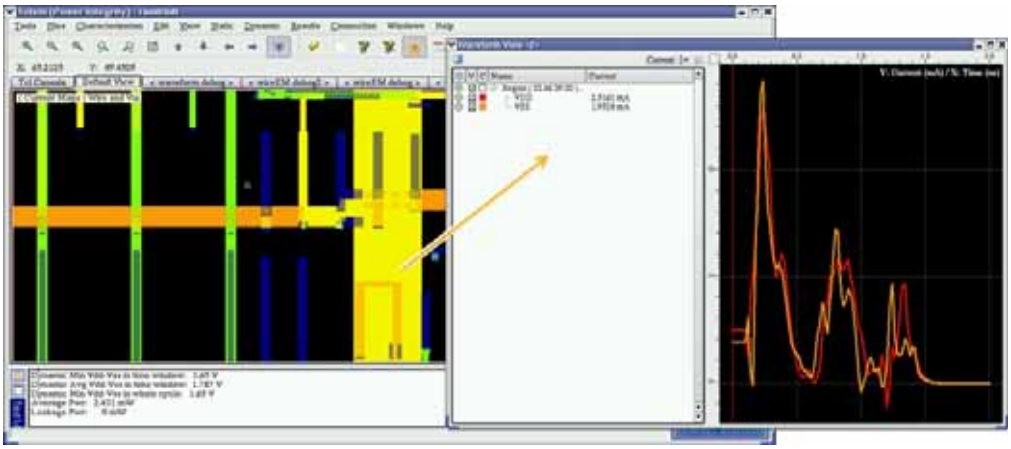


Figure B1-35 Plotting regional current

View -> Connectivity ->

Highlight Disconnected Instances

Highlights disconnected instances in the design.

This menu is grayed out when there are no identified disconnected instances.

Highlight Disconnected Wire and Via

Highlights disconnected wires and vias in the design.

This menu is grayed out when there are no identified disconnected wires and vias.

Disconnected Instances

Turns on highlighting of disconnected instances in the **SA** view, which is off by default. The highlights can be cleared using the same menu command, the **CLR** menu button, or the TCL command 'select clearall'.

Disconnected Wires and Vias

Turns on highlighting of disconnected wires and vias in the **SA** view, which is off by default. The highlights can be cleared using the same menu command, the **CLR** menu button, or the TCL command 'select clearall'.

Disconnected Transistor Pins...

To capture potential design problems in MMX blocks, provides a list of all transistors disconnected from the power and ground grid.

List of PG Weakness -> Instances

For multiple-instance designs, displays a ‘Report of PG Weakness Instances’ dialog box, displaying a sorted list of Power/Ground Weakness instances, with following information on each instance:

Total Resistance: the total normalized P/G resistance seen by the instance, relative to the highest normalized P/G resistance (Rinst - Rmin)/(Rmax - Rmin), where Rinst is the sum of the effective VDD and VSS resistances for the instance, and Rmin and Rmax are the minimum and maximum Rinst values in the design

VDD%: the VDD percentage of the normalized instance resistance (Rvdd*100/ Rinst)

VSS%: the VSS percentage of the normalized instance resistance (Rvss*100/ Rinst)

X,Y Location: the x,y location of the instance

Instance Name: the name of the instance

You can also highlight and zoom to any of the critical instances by using the ‘Go to Location’ button, and you can sort the table by Total Resistance, Vdd, or Vss order criteria.

This menu is grayed out when the design consists of a single MMX instance.

List of PG Weakness -> Transistor Pins

Displays a ‘Report of Gridcheck Weakness for Transistor Pins’ dialog box, displaying a sorted list of normalized grid resistance by pin, with following information on each :

Resistance: the total normalized P/G resistance seen by the pin, relative to the highest normalized P/G resistance (Rinst - Rmin)/(Rmax - Rmin), where Rinst is the sum of the effective VDD and VSS resistances for the pin, and Rmin and Rmax are the minimum and maximum Rinst values in the design.

X,Y Location: the x,y location of the instance

Instance Name: the name of the instance of the pin

Pin Name: name of the pin

You can also select and zoom to any of the critical pins by using the ‘Go to Location’ button. The next 1000 highest resistance transistor pins on the list can be viewed by using the ‘Next 1000’ button.

With this command you can set up a filter function to filter either instance name or pin name in the **Transistor Pin** dialog box. Instance name filtering is mainly for design setup with multiple instances. Pin name filtering allows checking pins on a domain basis or exact pin name searching for quick access.

List of Guard Rings Weakness ...

Displays a list of guard rings for which resistance is relatively high, representing potentially low substrate noise protection.

This menu is grayed out unless the design analysis includes substrate.

Missing Vias...

Displays a ‘Report of Missing Via Area’ dialog listing all P/G mesh intersections that represent possible missing via locations, including those on 45-degree geometry, compiled by surveying all grid intersections on all layers and noting the intersections that are not connected by vias. It also lists the voltage difference between upper and lower metal layers at potential via sites. The dialog allows you to step up or down the list and select a missing via location. Clicking on the 'Include GDS cell' button and the check box allows you to include a GDS cell. Clicking on the ‘Go to Location’ button highlights the missing via location for closer inspection. The missing vias list is sorted by voltage gradient---the higher the voltage difference between the metal layers above and below the via, the more impact adding this via would have on improving the voltage drop.

Creates and filters the entries in the missing via report dialog to focus on the most critical violations. You can choose to ignore stack vias, only display hot instances, or even set up a differential voltage threshold across the missing vias to reduce the vias in the list. You can also sort the missing via report using X-coordinates or Y-coordinates. To do this, you must select the X/Y sorting function in the “Location” column, as shown in Figure B1-36, then use the following steps,:

1. Press the Right-Mouse-Button on “Location (X,Y)” heading.
2. Click
3. the Left-Mouse-Button on “Location Sort By X” or “Location Sort by Y”. Click Apply.

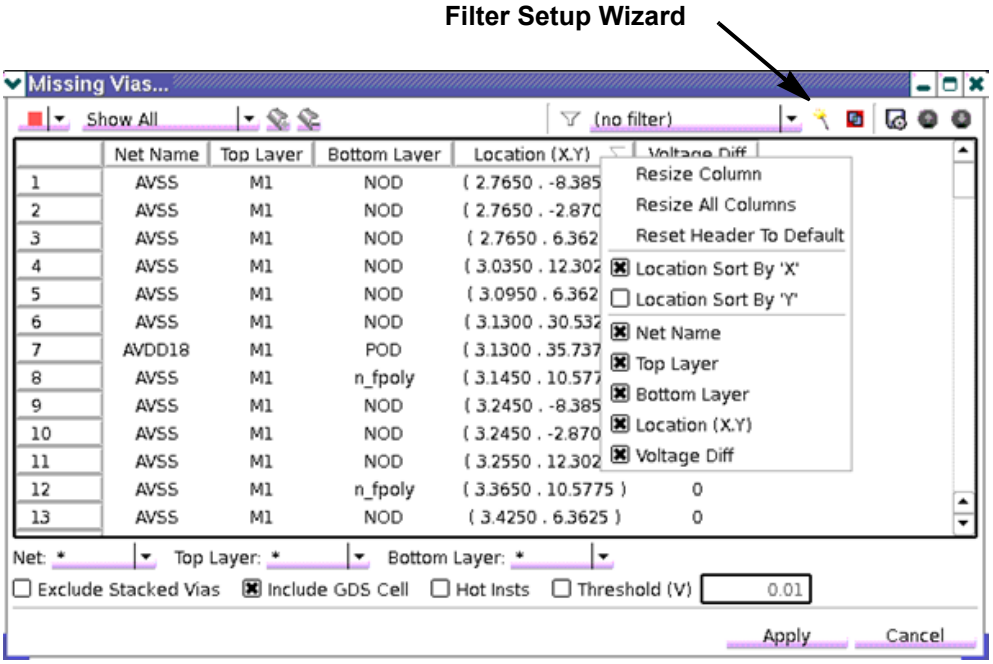


Figure B1-36 Missing Vias dialog

You can also exclude user-defined regions for Missing Via Reports using a file, or by zooming to the region of interest and clicking the “Get Screen Area” option. The GUI zoom-based approach is outlined in the steps below.

1. Click “Filter Setup Wizard” in the Missing Via Report window.
2. Zoom to the region of interest and click “Get Screen Area”, as shown in Figure B1-37.
3. Zoom to the region desired and click the blank square next to the Exclude box, “Get Screen Area”.
4. Click on the “Save Current Setting as New Filter Rule” icon, and select a filter number in the dialog, to save the marked region for future reference, then click “OK”.

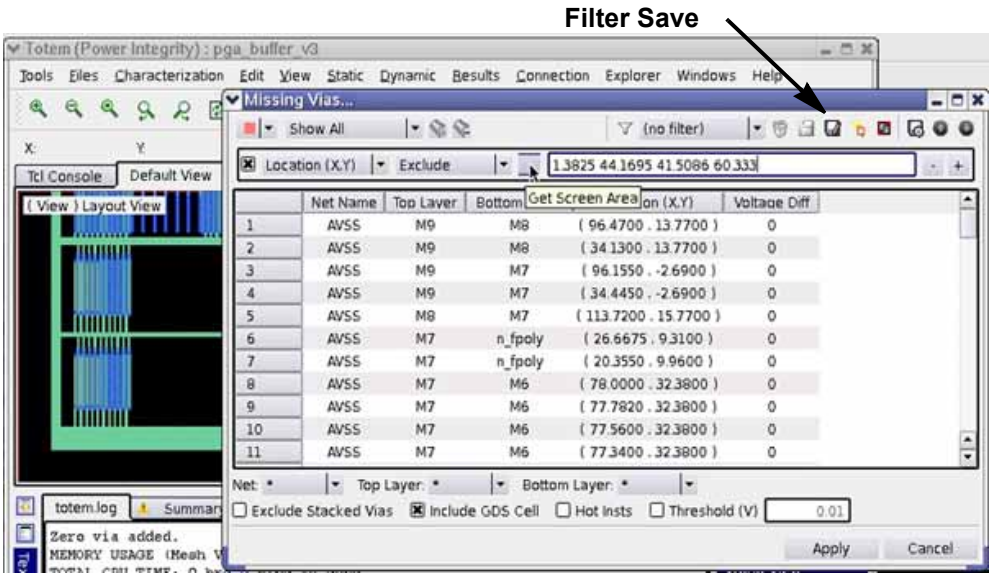


Figure B1-37 Missing Via Exclusion steps

You can easily select discrete entries on the list using the CTRL key + left mouse button. Also using the Shift key + left mouse button you can select a continuous group of listings.by clicking on the first and last entry desired. All selected entries are highlighted in the Totem display. User can specify marker layers in gds2db cci flow and tool will automatically exclude the regions for missing via check (it is reported in adsRpt/GDS/ missing_via_exclude_region_list.rpt)

List of Overlapping Substrate Wires

To view overlaps in substrate layers detected by CSE. User can identify bad block placement if the overlap errors are highlighted along edges of a block.

Shorts ...

Displays a ‘Report of Shorts Area’ dialog listing all P/G wires that may be improperly shorted. The dialog allows you to step up or down the list and select a short location. Clicking on the ‘Go to Location’ button zooms to and highlights the potential short for closer inspection.

GDS Shorts ...

Displays information on GDS shorts. **This option also allows to** highlight min_res_path between two shorts points. Multiple selections can be performed using Shift + left mouse button. This menu is grayed out if no GDS shorts have been identified.

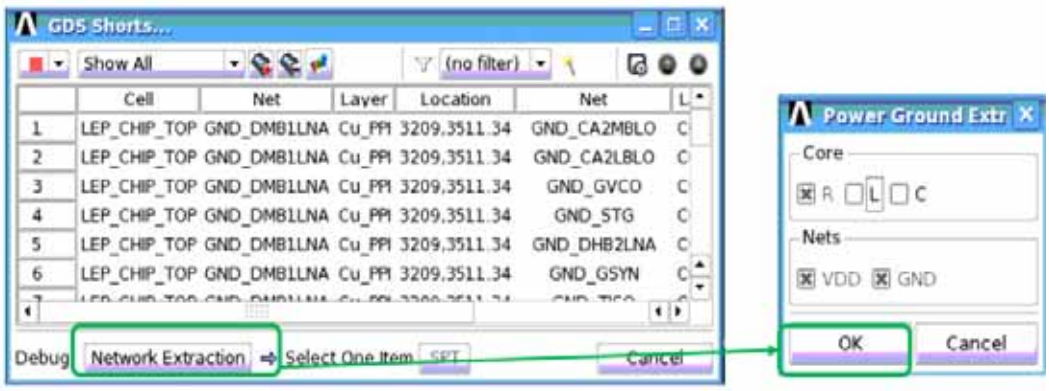


Figure B1-1 GDS Shorts in GUI

P2P Resistance Calculation

Executes point-to-point resistance calculations directly by bringing up a setup dialog, as shown in Figure B1-2. Use the left mouse button to click on any geometry in GUI layout or color map view to set up the “From” and “To” nodes for P2P resistance calculation. You also can click on the **Perform Min Res Path** box at the same time to debug design resistance weakness or for ESD analysis.

When a specified point has no node in the same location, you can take the nearest available node within the search radius specified (default 1um), or redo extraction by including nodes at the given point. Totem continues to look outside the radius given by 4N (N being radius specified).

A red region highlighting the search area can be seen in the GUI, along with a yellow marker highlighting the node found (within the 4N radius). If there is no node found within the 4N radius, the option “Use existing extra nodes” is disabled.

A list of P2P resistance calculations matching the search criteria is then displayed.

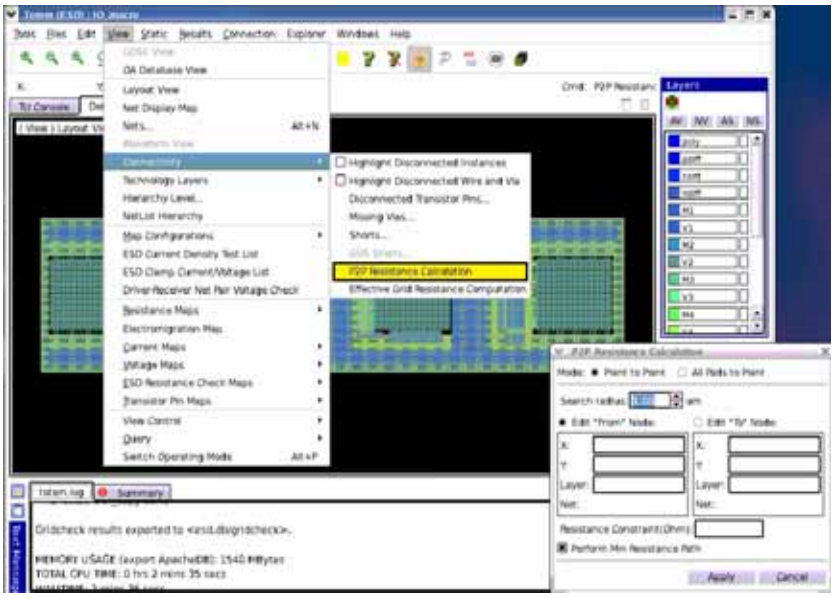


Figure B1-2 Resistance calculation setup

Effective Grid Resistance Computation

Displays an ‘Effective Grid Resistance Computation’ dialog, to select options for executing the grid resistance calculation command.

The computation Modes available are

Arc - one worst-case node per net included (default)

Loop - one node per net, but includes both power and ground nodes

All Points - includes all nodes

All Pins - for MMX designs, also includes all pins internal to transistors (but only one node per pin)

Input data includes:

Inst/Node Count - maximum number of worst-case instances or nodes to be included (default 1000)

Thread Count - number of threads used for parallel R computation

Net Names - allows selection of a limited group of nets to be included. The Customize button allows selection of nets from a dialog box list.

Layer Names - allows selection of a limited group of Layers to be included. The Customize button allows selection of layers from a dialog box list.

Instance Names - allows selection of a limited group of instances to be included.

Cell Names - allows selection of a limited group of cells to be included.

Instance File - specifies a file that contains the list of instances to be included in the computation. The Browse button allows file selection from a dialog box list.

Cell File - specifies a file that contains the list of cells to be included in the computation. The Browse button allows file selection from a dialog box list.

Output File - specifies an output file other than the default, *adsRpt/<designName>.res_calc*. Two types of output can be selected: ”Incremental” means that the new resistance computation does not include nodes previously computed, while “Appended” means that the new output results are appended to the previous output results file.

View -> Technology Layers (Metal and Via) ...

Displays a graphic showing the wire and via layers in the design and the key characteristics of each, including dielectric constant (e), metal thickness and full layer thickness (h). If you use the right mouse button to create a box over one or more layers of particular interest, the view zooms to include just those layers. Clicking on the ‘Full View’ button then returns the view to the full view of all layers. Figure B1-3 shows an example Technology view.

View -> Technology Layers (Substrate) ...

Displays a substrate Technology View to support SOI/BCD advanced processes for substrate analysis separate from the metal and via technology view. You can use the this command to see “Thickness” and “Depth” attributes. You also can sync the color of the layer with the LSW (Layer Select Window) and expand or collapse the sub-layers for easy viewing and debugging, as shown in Figure B1-4.

This menu is grayed out unless the design analysis includes substrate.

.

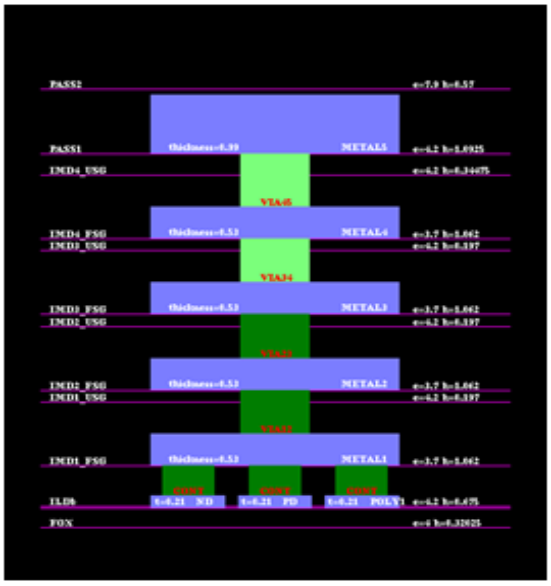


Figure B1-3 Layer characteristics display in the technology view

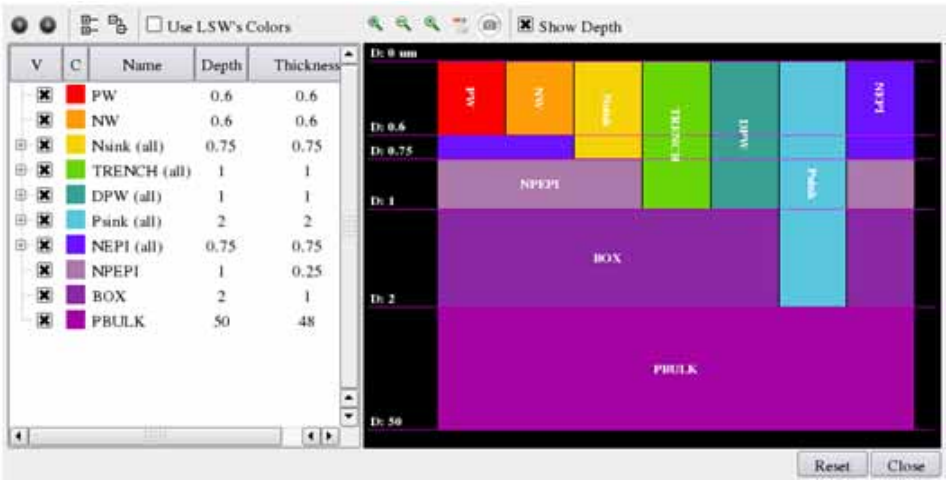


Figure B1-4 Substrate technology layer viewer

View -> Hierarchy Level...

Displays a ‘Hierarchy Level’ dialog box that allows you to select the number of hierarchy levels for which design details are displayed. Hierarchy levels below that specified are shown as a “black box.”

View -> Netlist Hierarchy

Displays a Netlist dialog box showing the netlist hierarchy.

The integration of Netlist Hierarchy View with Cross-Probing Provides easy access to netlist information for debugging, and enables cross-probing with Layout View and other colormaps, as shown in Figure B1-5. When you select an entry in the Netlist Hierarchy view, the corresponding transistor pin is highlighted in the colormap. You can also select a transistor pin in the transistor pin map or SA view with the ‘pinInst’ check box enabled. To enable the pinInst check box, refer to Figure B1-3 on using the Layer Selection Window.

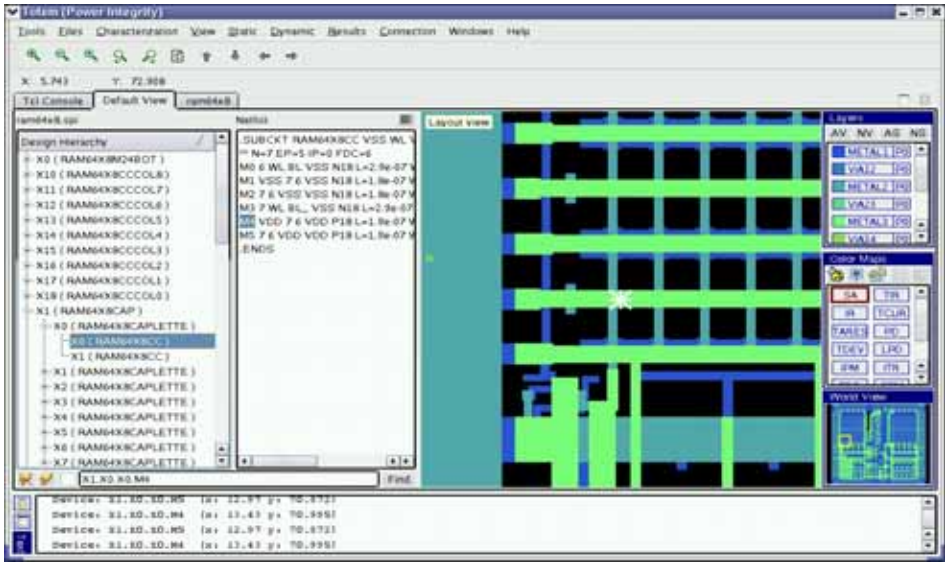


Figure B1-5 Netlist view and cross-probing

To import a netlist and display the netlist hierarchy view, use the command **Files -> Import Netlist**, and fill in the netlist filename, or you can click on **View Netlist** in the 'View Configuration' panel (first button from the right), and specify the netlist filename, which also brings up the netlist view. You can import a CDL format netlist in Netlist Hierarchy View to cross-probe with the Totem database and the Virtuoso Schematic Composer, or the Layout editor.

View -> Map Configurations ->

Layers Color Map

Displays a 'Layers' dialog box, the same as the 'View Layers' button in the 'Configuration' control panel. The dialog allows you to turn on/off and select color and outline or fill characteristics for display of all metal and via layers and the associated pins, as well as instances, in the design.

You can import the Virtuoso technology file, default display color file and a layer mapping rule using the **Files -> Import GDS** menu command to help cross-probe debugging between Totem GDS view and SA layout view. Then click on **View -> Map Configurations -> Layers Color Map**. After loading the Virtuoso tech and display file, the Virtuoso-like Layer Selection Window displays grouping information (see Figure B1-6), and GDS and SA views show the same color scheme, as shown in Figure B1-7.

The Layer Select dialog columns allow you to select **Bru** (Brush), **Vis** (Visible), **Fill** (Fill), **Line** (Outline), **Sel** (Select), and **Pin** (PinInst) layer representations. For example, select **Bru** to choose brush layer drawing mode instead of transparent drawing mode. To control Metal/Via layer display, click on the header of each column, as shown in Figure B1-6. Also you can click on the **Vis** column (Visible) to get a popup menu and choose to show all or hide all layers, or you can show all metal or all via layers. The same procedure applies to changing layer display characteristics using the other column headers.

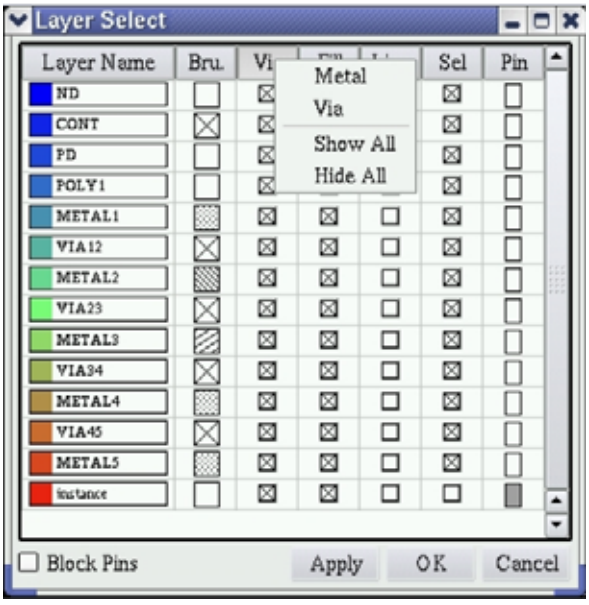


Figure B1-6 Layer select window

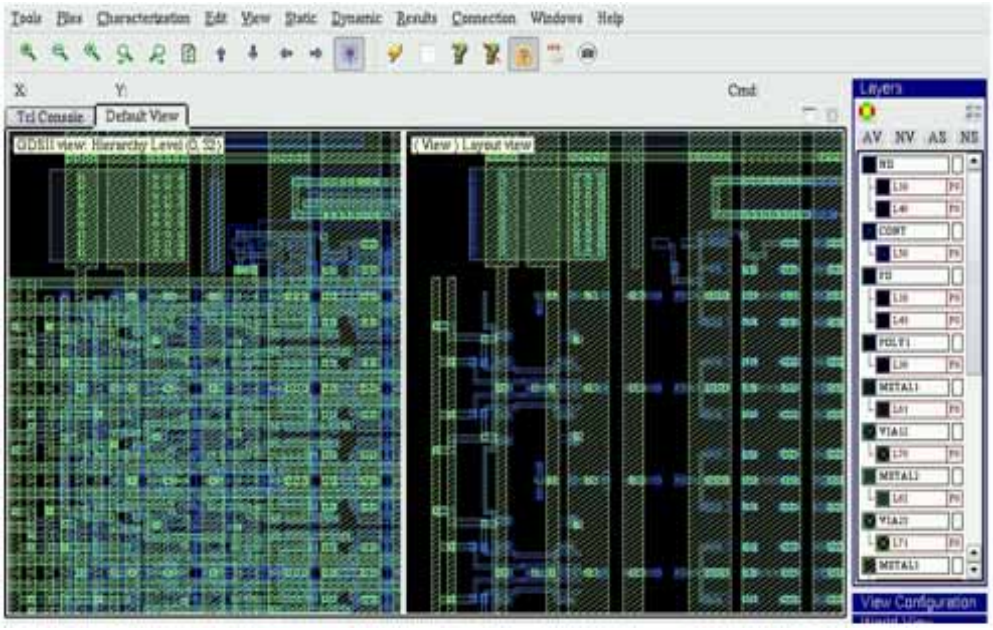


Figure B1-7 GDS view and layout view cross probing

IR Drop Color Map

Displays a static IR ‘Voltage Drop Color Map’ dialog box that allows you to select colors for a set of static voltage drop ranges in the design. You can choose to use the ‘maximum instance voltage drop’ in the design by clicking on the button. The default display divides the ranges equally between the specified minimum and maximum voltage drop settings.

For DvD analysis, a “Color Map: Wire and Via” dialog is displayed, with only the voltage values for selected nets under the “Reference VDD domain” drop-down menu. Voltage values are shown in descending voltage order. When you select only ground nets, voltage values for the associated power nets are used if the pg arc is defined; otherwise all voltage values are displayed.

You can select the colors and voltage ranges to be displayed, as well as absolute or percentage voltage presentation methods.

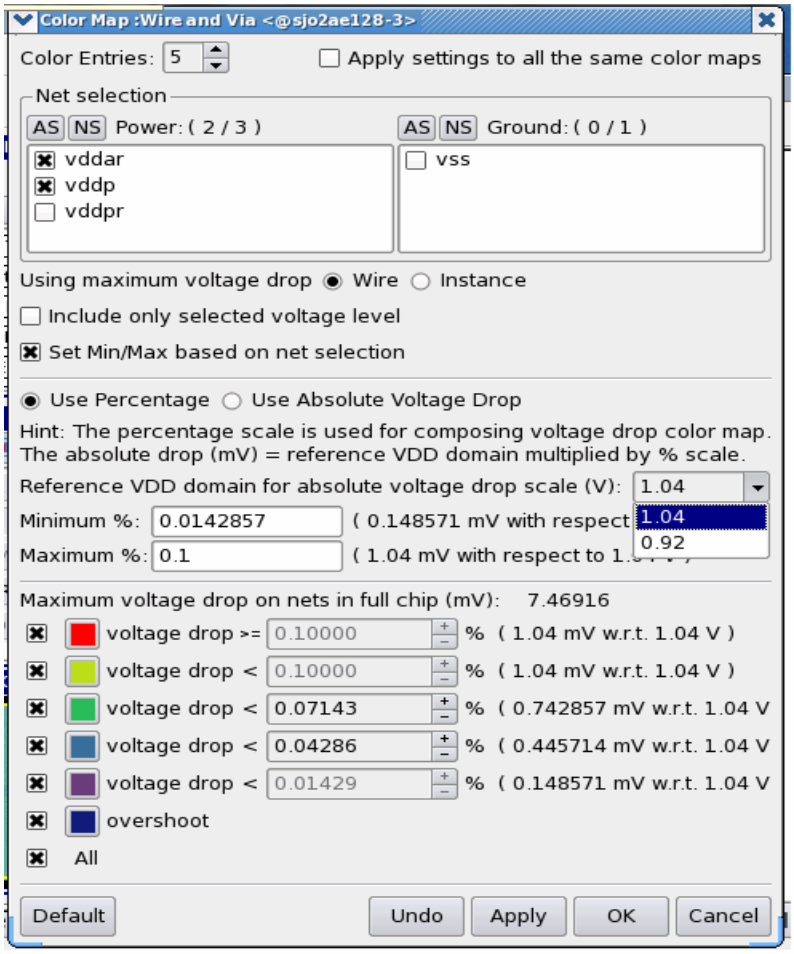


Figure B1-8 Wire and Via Color Map dialog

Current Color Map

Displays a ‘Current Color Map’ dialog box that allows you to select colors for a set of current values from minimum to maximum in the design. The default display divides the ranges equally between the minimum and maximum current.

Power Density Color Map

Displays a ‘Power Density Distribution Color Map’ dialog box that allows you to select colors for a set of power density values as a percentage of maximum power density. You can change the number/size of the tiles using the Grid Num and Grid Size windows to get more or less resolution in the density information.

This menu is grayed out when the design consists of a single MMX instance.

Leakage Power Density Color Map

Displays a ‘Leakage Power Density Color Map’ dialog box that allows you to select colors for a range of leakage power density values as a percentage of maximum leakage power density. You can change the number/size of the tiles using the Grid Num and Grid Size windows to get more or less resolution in the density information.

Instance Power Color Map

Displays an ‘Instance Power Density Color Map’ dialog box that allows you to select colors for a range of instance power values as a percentage of maximum instance power.

This menu is grayed out when the design consists of a single MMX instance.

EM Color Map

Displays the ‘ElectroMigration Color Map’ dialog box that allows you to select colors for a range of EM values as a percentage of EM maximum tolerance.

Toggle Density Color Map

Displays the Toggle Density Color Map dialog box, which allows you to select colors for a range of average toggle rates per tile as a percent of maximum toggle density. You can change the number/size of the tiles using the ‘Grid Num’ and ‘Grid Size’ windows to get more or less resolution in the density information.

This menu is grayed out when the design consists of a single MMX instance.

Instance Toggle Rate Color Map

Displays the ‘Instance Toggle Rate Color Map’ dialog, which allows you to select colors to display toggle rates per instance.

This menu is grayed out when the design consists of a single MMX instance.

View -> EM Mode

Avg - RMS - Peak

Displays Average, RMS or Peak EM results. Peak EM values are displayed by default in power integrity analysis.

View -> Power Maps ->

This menu is enabled when design has digital place and route blocks.

Note: The Layer Map dialog does not control map displays. Since instances are the only objects displayed it can only turn instance display on and off. Also, you can change the colors, scaling and tile grid size of the display of the following map commands by using the View -> Map Configurations command for the display.

Power Density Map

Performs the same operation as the **PD** button in the ‘Configuration buttons’ panel.

This menu is grayed out when the design consists of a single MMX instance.

Power Map of Instances

Performs the same operation as the **IPM** button in the ‘Configuration buttons’ panel. See description above.

This menu is grayed out when the design consists of a single MMX instance.

Power Map of Clock Instances

Performs the same operation as the **CPM** button in the ‘Configuration buttons’“panel.

This menu is grayed out when the design consists of a single MMX instance.

Leakage Power Map

Performs the same operation as the **LPM** button in the ‘Configuration buttons’ panel. See description above.

This menu is grayed out when the design consists of a single MMX instance.

Leakage Power Density Map

Displays a color map of leakage power density if leakage power data is available.

This menu is grayed out when the design consists of a single MMX instance.

Toggle Density Map

Displays a color map indicating the average toggle rate per tile area in the design.

This menu is grayed out when the design consists of a single MMX instance.

Toggle Map of Instances

Displays a color map indicating the toggle rate by instance in the design.

This menu is grayed out when the design consists of a single MMX instance.

Pad Voltage Map

Displays a color map indicating the pad voltage in the design. The pad colors represent the pad voltage for static IR analysis and absolute peak voltage for dynamic analysis. A white pad outline indicates a ground pad and an orange pad outline represents a power pad. Click on the ‘Set Color Range’ configuration button to display the Pad Voltage Color Map dialog that indicates the pad voltage values and allows you to change the colors, and also scale the size of the pads for better viewing.

View -> Resistance Maps ->

This menu is enabled when design has digital place and route blocks, and displays resistance for multiple-instance designs.

Total Path Resistance

For multiple-instance designs, displays a color map indicating the relative strength of the Vdd and GND grid connections to each instance, computed as a normalized “gradient”. The Total Path Resistance gradient for an instance is calculated as (Rinst - Rmin)/(Rmax - Rmin), where Rmin and Rmax indicate the best and worst grid connections among all instances in the design, and Rinst is the sum of the Vdd and Vss resistances connected to the instance. For instances with multiple Vdd/Vss connections, the sum of the highest Vdd resistance plus the highest Vss resistance connected to the instance is selected as Rinst. In multiple Vdd/Vss designs use the **View-> Nets** command to select the nets to be included in the display. Note that turning off a net that does not represent the maximum Rinst value does not cause a change in the color display. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Resistance Color Map’ dialog, which allows you to select colors for the range of grid strengths as a percentage of Rmax-Rmin. For more details on the Gridcheck report, see the “*RedHawk User Manual”*. (Note that this command previously was called “Total Resistance”.)

VDD Path Resistance

For multiple-instance designs, displays a color map indicating the relative strength of the VDD grids connected to each instance, as the instance VDD resistance gradient. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Resistance Color Map’ dialog, which allows you to select colors for a range of resistance gradient values as a percentage of the global Rmax - Rmin. (Note that this command previously was called “VDD Resistance”.)

GND Path Resistance

For multiple-instance designs, displays a color map indicating the relative strength of the VSS grids connected to each instance, as the instance VSS resistance gradient. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Resistance Color Map’ dialog, which allows you to select colors for a range of resistance gradient values as

a percentage of the global Rmax - Rmin. (Note that this command previously was called “VSS Resistance”.)

Pin Path Resistance

For multiple-instance designs, displays a color map of pin path resistance (grid check) for every instance pin. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Pin Path Resistance’ color map dialog, which allows you to select colors for a range of resistance values.

Wire/Via Path Resistance

Displays a color map of path resistance (grid check) for every wire/via. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Wire/Via Path Resistance’ color map dialog, which allows you to select colors for a range of resistance values.

Wire/Via Extractor Resistance

Displays a color map of extractor resistance for every wire or via. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Wire/Via Extractor Resistance’ color map dialog, which allows you to select colors for a range of resistance values.

Instance Actual Resistance

For multiple-instance designs, displays Wire/Via Actual Resistance maps to improve visual checking of wire and via effective resistance from the pads, for debugging design weakness. The color map displays the area in which instance resistance calculation has been computed, and continues updating as the res_calc tcl command is executes. You can also query the actual resistance value of any wire/via by clicking on the geometry in the GUI color map, and the log message window displays the actual resistance value. The actual resistance ranking report is also available under **Results -> List of Wire / Via Actual Resistance** after the map is generated.

(Note that this command previously was called “Effective Grid Resistance”.)

View -> Voltage Drop Maps

Wire and Via

Displays the latest analysis results, static IR voltage drop or dynamic voltage drop analysis, for wires and vias. Performs the same operation as the **IR** button in the

'"Configuration buttons' panel. Click on the 'Set Color Range' button in the View Configuration window to obtain a 'Voltage Drop Color Map' dialog (see Figure B1-9).

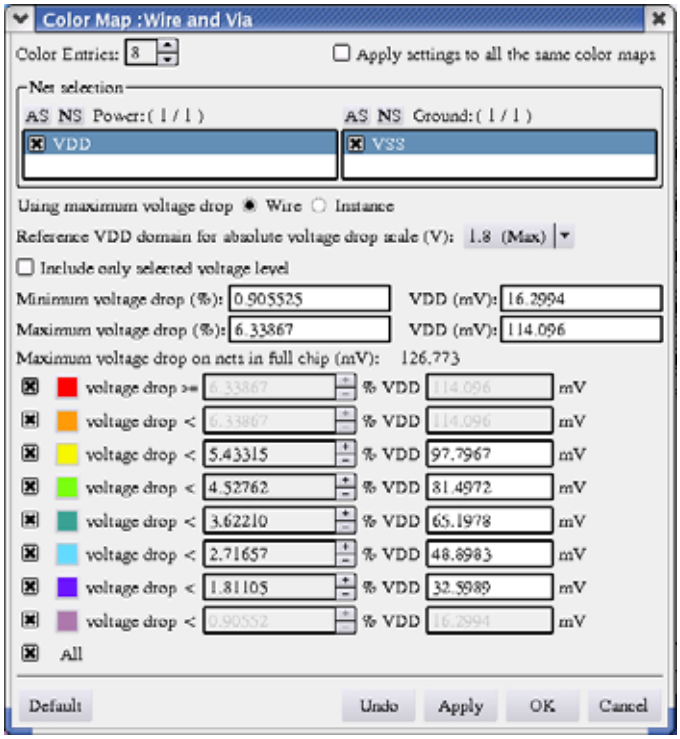


Figure B1-9 Voltage Drop Color Map Dialog

The Voltage Drop Color Map dialog allows you to modify the GUI display in several ways:

- select the number of colors to be used in the display (default 8).
- select the power and ground net groups to be displayed using the 'Net selection' box.
- select the type of voltage drop display with the button 'Using maximum voltage drop' for either **Wires** or **Instances**. Instance voltages are Vdd-Vss.
- select the Reference Vdd domain from which to scale the voltage drop display ranges.
- select the Minimum and Maximum voltage drop values to display, if other than those calculated by default.
- select different colors or voltage drop ranges as a percentage of maximum DvD value
- turn off one or more voltage range displays using the left column of buttons
- set the maximum and minimum DvD percentages to be displayed

When creating a DvD movie, set the proper thresholds with the 'Using maximum voltage drop' button for instances.

Instance

Displays the latest analysis results, static IR voltage drop or dynamic voltage drop analysis, for instances only. The display is the same as described for 'Wire and Via' previously, except that the voltages are "effective" values, Vdd-Vss.

This menu is grayed out when the design consists of a single MMX instance.

Regular Cells Movie

Replays design analysis steps for multiple-cell designs, based on waveform storage. Frames can be stopped for more detailed analysis and DB cross-probing.

This menu is grayed out when the design consists of a single MMX instance.

View -> Electromigration Map

Displays results of electromigration analysis for static or dynamic current conditions, the same operation as the **EM** button in the 'Configuration buttons' panel.

View -> Current Maps

Wire and Via

Displays a color map indicating the wire and via currents in the design, the same as using the **CUR** button in the 'Configuration buttons' panel.

Pad Current Map

Displays a color map indicating the pad currents in the design. The pad colors represent the absolute average current for static IR analysis and absolute peak current for dynamic analysis. A white pad outline indicates a ground pad and an orange pad outline represents a power pad. Click on the 'Set Color Range' configuration button to display the Pad Current Map dialog that indicates the pad current values and allows you to change the colors for each range. The Pad Size Scaling Factor panel allows you to scale (multiply) the size of the pads for better viewing. The Color Entries panel allows you to expand or reduce the number of color steps used to display pad current (default is 10 steps).

Instance Peak Current Map

Displays the instance peak current distribution in the design in microAmps. Click on the 'Set Color Range' configuration button to display the Instance Peak Current Map dialog that indicates the peak current values and allows you to change the colors for each range. The Color Entries panel allows you to expand or reduce the number of color steps used to display pad current (default is 10 steps).

This menu is grayed out when the design consists of a single MMX instance.

View -> Voltage Maps

Wire and Via Actual Voltage Map

Displays a map of actual voltages to help designers evaluate absolute voltage levels rather than voltage drop values. You can control the net selection in the color control dialog, and the GUI automatically sets the minimum and maximum ranges based on the net selection.

Wire and Via Peak2Peak Noise Map

Allows monitoring peak-to-peak wire and via voltage variation. You must set the GSR keyword "DYNAMIC_SELECTIVE_SAVE 0" before running dynamic analysis to ensure that there are no waveform data missing during the GUI waveform post-processing step. This map is very helpful for analog designs or for substrate noise analysis for designers who want to understand the peak variation of the rail instead of just the voltage drop or the bounce. You also can add WVP2P to the default list of maps by picking it from "Configure Color Map" button. Also, you can use the TCL command "show WVP2P".

Note that this GUI feature invokes waveform post-processing and hence causes an increase in runtime in large designs.

View -> Transistor Pin Maps ->

Avg Voltage Over Simulation Time

Max Voltage Over Simulation Time

Min Voltage Over Simulation Time

Displays the average, maximum or minimum transistor pin voltage over the simulation time in colors identifying their voltage values. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Transistor Pin Voltage Color Map’ dialog, which allows you to select colors for a range of voltage drop values as a percentage of maximum DvD, set the maximum and minimum DvD percentages, and also increase the size of the pins for ease of viewing using the ‘Pin Size Scale’ window.

Voltage Drop Map

Displays the worst transistor pin voltage drop over the simulation time in colors identifying the severity of the voltage drop.

Transistor Pins Movie

Replays design analysis steps transistor pins, based on waveform storage. Frames can be stopped for more detailed analysis and DB cross-probing.

Actual Resistance Map

After the ‘perform extraction’ command, generates a total resistance color map for all transistor pins in MMX/ESD analysis. The resistance report is generated in the file *adsRpt/MMX/<design>.mmx.res*. When this command is executed, a ‘Setup resistance map limit’ dialog is displayed showing to limit on number of transistor pins to be checked in the TARES map (10,000 by default). To change the maximum number calculated, you can also use the GSR keyword ‘MMX_RES_MAP_LIMIT <limit>’. When OK is clicked in this dialog, the Transistor pin Actual Resistance map is displayed, in colors identifying the actual pin resistance values. If this map is not one of the default buttons in the View Configuration panel, click on the ‘Configure Color Maps’ icon and select the TARES map from the list to be displayed. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Transistor Pin Actual Resistance Color Map’ dialog, which allows you to select colors for a range of resistance values as a percentage of the resistance gradient, set the maximum and minimum resistance gradient percentages, and also increase the size of the pins for ease of viewing using the ‘Pin Size Scale’ window.

The bind key ‘Ctrl+Shift+A’ also can be used to bring up the transistor pin limit dialog for the TARES map. You can also invoke incremental mode to append new results in addition to the results from previous runs. This function is designed to allow you to get a portion of large designs checked more quickly. If you do not click on the ‘OK’ button within 60 seconds, **Totem** resumes with checking Transistor PG pin resistance based on default settings.

PG Weakness Map

Displays a P/G pin weakness ranking colormap, in colors indicating the relative resistance of all transistor pins from highest to lowest. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Transistor Pin PG Weakness Color Map’ dialog, which allows you to select colors for a range of resistance values as a percentage of the resistance gradient, set the maximum and minimum gradient percentages, and also increase the size of the pins for ease of viewing using the ‘Pin Size Scale’ window.

Current Map

Displays transistor pins in colors identifying their resistance values. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Transistor Pin Current Color Map’ dialog, which allows you to select colors for a range of pin current values, set the current range displayed, and also increase the size of the pins for ease of viewing using the ‘Pin Size Scale’ window.

Also, you can adjust the display settings by clicking on the left “Set Color Range” **View Configuration** button to display the **Color Map: Current Map** dialog. This dialog allows

you to choose the nets to be displayed, and the maximum and minimum current You can also select the number of colors and the colors to use, to help the current range of interest and define the hot spots more easily.

Transistor Pin Device Cap Color Map

Displays transistor pins in colors identifying their device capacitance values. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Transistor Pin Decap Color Map’ dialog, which allows you to select colors for a range of device capacitance values, set the capacitance range displayed, and also increase the size of the pins for ease of viewing using the ‘Pin Size Scale’ window.

Transistor Pin Device Cap Color Map (intentional)

Displays only the intentional decap transistor pins, as shown in the example of an intentional decap color map below, Figure B1-10. This allows checking where the intentional decap transistors are located, and if they are effectively placed in the correct hotspot.

You can also add a “TICAP” map button to the View Configuration window to display Transistor Pin Device Cap (intentional) color maps.

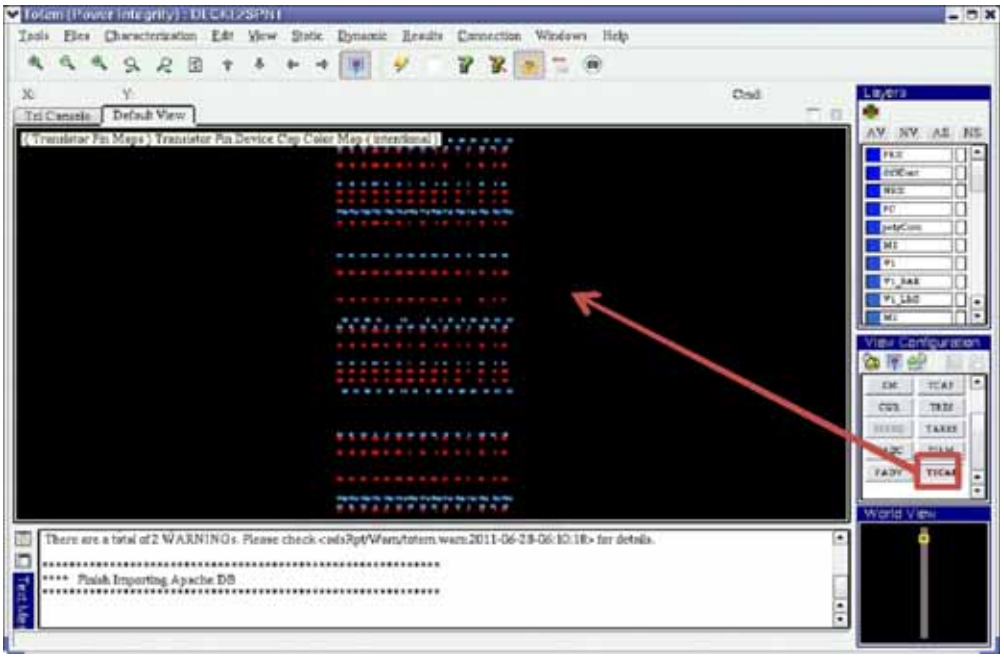


Figure B1-10 Transistor pin map

View-> Dynamic Instance DvD ->

Max Vdd-Vss Over Time Window

Displays the maximum dynamic Vdd - Vss differential over the time window, for the nets selected in **View Nets**. Display is based on view selections in the ‘Voltage Color Map” dialog. Instances are shown in Fill representation in the colors chosen to indicate their voltage drop category. At high magnifications the maximum Vdd - Vss difference values for individual instances can be observed. You can toggle between the Maximum Instance Voltage Drop (differential) and the Maximum Wire Voltage Drop (differential). Note that this is a *best case*, showing the maximum difference between Vdd and Vss over the time window.

This menu is grayed out when the design consists of a single MMX instance.

Min Vdd-Vss Over Time Window

Displays the minimum dynamic Vdd - Vss differential over the time window, for the nets selected in **View Nets**. Display is based on view selections in the ‘Voltage Color Map” dialog. Instances are shown in Fill representation in the colors chosen to indicate their voltage drop category. At high magnifications the maximum Vdd - Vss difference values for individual instances can be observed. You can toggle between the Maximum Instance Voltage Drop (differential) and the Maximum Wire Voltage Drop (differential). This shows the worst case minimum Vdd - Vss difference over the time window.

This menu is grayed out when the design consists of a single MMX instance.

Avg Vdd-Vss Over Time Window

Displays the average dynamic Vdd - Vss differential over the time window, for the nets selected in **View Nets**. Display is based on view selections in the ‘Voltage Color Map” dialog. Instances are shown in Fill representation in the colors chosen to indicate their voltage drop category. At high magnifications the average Vdd - Vss difference values for individual instances can be observed. You can toggle between the Maximum Instance Voltage Drop (differential) and the Maximum Wire Voltage Drop (differential).

This menu is grayed out when the design consists of a single MMX instance.

Min Vdd-Vss Over Whole Cycle

Displays the minimum dynamic Vdd - Vss differential over the whole cycle, for the nets selected in **View Nets**. Display is based on view selections in the ‘Voltage Color Map” dialog. Instances are shown in Fill representation in the colors chosen to indicate their voltage drop category. At high magnifications the minimum Vdd - Vss difference values for individual instances can be observed. You can toggle between the Maximum Instance Voltage Drop (differential) and the Maximum Wire Voltage Drop (differential). This shows the worst case Vdd - Vss difference over the whole cycle, generally very similar to the map for DvD.

This menu is grayed out when the design consists of a single MMX instance.

View -> Decap Maps ->

Note: You can change the colors, scaling and tile grid size of the display of the following map commands by using the View -> Map Configurations -> Cap Density Color Map command for the display, or click on the ‘Set Color Range’ Configuration button.
These menus are grayed out when the design consists of a single MMX instance.

Decap Density Map

Performs the same operation as the **DD** button in the ‘Configuration buttons’ panel.

Device Cap Density Map

Performs the same operation as the **DEV** button in the ‘Configuration buttons’ panel.

Load Cap Density Map

Performs the same operation as the **LC** button in the ‘Configuration buttons’ panel.

P/G Cap Density Map

Performs the same operation as the **PG** button in the ‘Configuration buttons’ panel.

Inserted Decap Density Map

Performs the same operation as the **IDD** button in the ‘Configuration buttons’ panel.

View -> Switch Instance Maps

Switch Instance Voltage Drop Map

Displays a voltage drop color map for a selected instance.

This menu is grayed out when the design has no power switches.

Switch Instance Current Map

Displays a current color map for a selected instance.

This menu is grayed out when the design has no power switches.

View -> EMI Color Map

When EMI analysis is performed and results available, displays a tile-based color map of EMI noise.

View -> Substrate Noise Map

These menus are grayed out when the design analysis does not include substrate.

Substrate Frequency Domain Density Map

Displays a substrate frequency domain noise density map (SFREQ). You can zoom into a smaller area to see the noise level (dBV) in each tile to get more detailed information.

Substrate Wire/Via Frequency Domain Noise Map

Displays a frequency domain noise map for wires and vias directly (instead of using the density map from SFREQ). You can control the generation of this map for selected layers to improve debugging of noise propagation. The default layer in this map is Pwell and Nwell only.

Also, from the Tcl interface you can use the “generate swvf” command to generate the “Substrate Wire/Via Frequency Domain Noise Map” (see section "generate", page D-942).

Guard Ring Resistance Weakness Map

For substrate coupling noise analysis, displays a map of guard ring weakness checking results, which ranks the resistance strength of all guard ring structures in the design, or within a selected region, from the weakest to strongest, to help designers discover any weaknesses in the power delivery network connections to the guard ring.

Guard Ring Actual Resistance Map

Displays a guard ring actual resistance map to evaluate potential latch-up problems due to weak guard ring resistance connections. An actual resistance ranking report is also generated, which is also available under **Results -> List of Guard Ring Actual Resistance** after the map is generated. The guard ring actual resistance map and ranking report are shown in multi-frame page format in Figure B1-11.

View -> Temperature Map

Thermal Profile Map

After thermal analysis, displays a self-heat thermal profile map of the design.

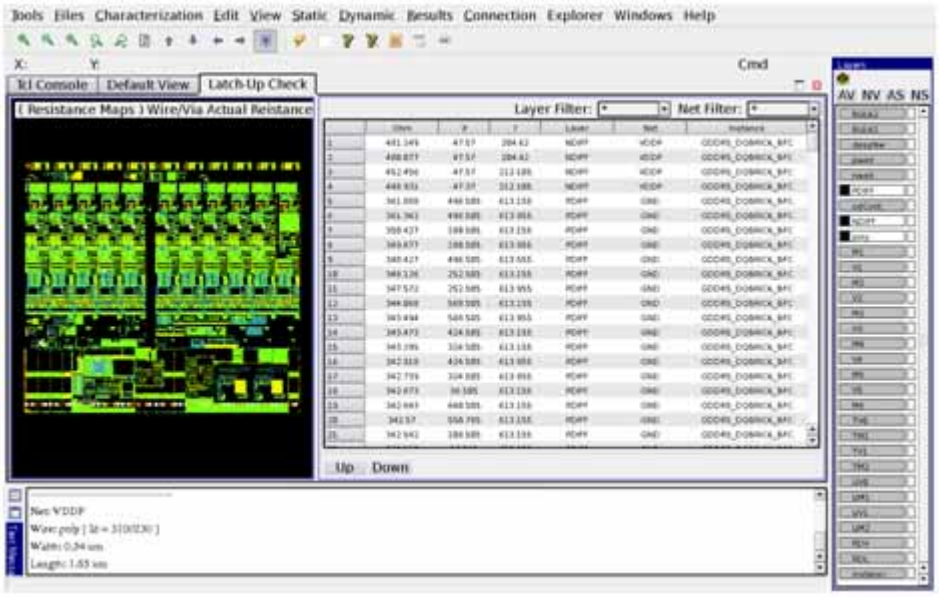


Figure B1-11 Guard Ring Actual Resistance

Totem can find the SPT (minimum resistance path) through substrate layers. If the lowest resistance path goes through a non-optimized routing path, there is potential design weakness. Note that the resistance value shown in the substrate geometry in the SPT report just represents one resistor branch among the entire substrate 3D mesh, so the resistance value is high. If the SPT path goes through substrate layers, then the SPT text and GUI report displays the proper path.

View -> View Control

Zoom In by 2

Zooms view in by a factor of 2 (dimensions increase 2 times).

Zoom Out by 2

Zoom view out by a factor of 2 (dimensions decrease by 1/2)

Fit All

Displays full design

Last View

Displays previous view.

Next View

Displays view before previous view.

Redraw

Refreshes present view.

Scroll Up

Moves view up.

Scroll Down

Moves view down.

Scroll Left

Moves view to left.

Scroll Right

Moves view to right.

View 0

GDS Viewer view of top hierarchy level.

View 32

GDS Viewer flattened view of up to 32 hierarchy levels below present view.

View -1

GDS Viewer flattened view of one hierarchy level above present view.

View +1

GDS Viewer flattened view of one hierarchy level below present view.

Display Selected Nets

Displays all nets that are selected.

Display All Nets

Displays all nets.

Show Power Pad

Highlights power pads.

View -> Query

Switch Highlight Mode

Switches highlight mode between blinking and non-blinking.

Set Highlight Color

Displays a dialog box in which to choose a highlight color. Default: white.

Query Object ...

Displays a dialog box in which characteristics of the selected object are listed.

Clear Highlight

Clears highlights from displayed objects.

Preselect Query

Displays object information in the Status box when cursor hovers over an object. Use the space bar to cycle through information for other objects at the same coordinates.

Instance Property

For multiple-instance SOC designs, displays a dialog showing key properties of a selected instance.

Dump Gif

Saves a GIF file of the current layout view in the selected directory.

View -> Switch Operating Mode

Switches GUI style between the new GUI style format and the “original” Totem GUI display format (Alt - p). Note that some available menu commands change when switching between styles, such as the **Windows -> Toolbars** command.

Job Monitor

Brings up a Job Monitor dialog to provide status on running jobs at selected intervals.

Static-> Power

Block Power Assignment

Allows you to perform a number of functions related to BPA and Block Power Master Cell operations: (see Figure B1-12).

- Import or set up the BPA in the Block Power Assignment setup dialog.
- Select an entry in the setup dialog and click the “Edit” button.
- Use the mouse to stretch the 8 control points of the highlighted region bounding box.
- Create/Modify/Delete a Block Power Master Cell.
- Append a subarea to a Block Power Master Cell.
- Edit a BPA rule with the newly-defined Master Cell.

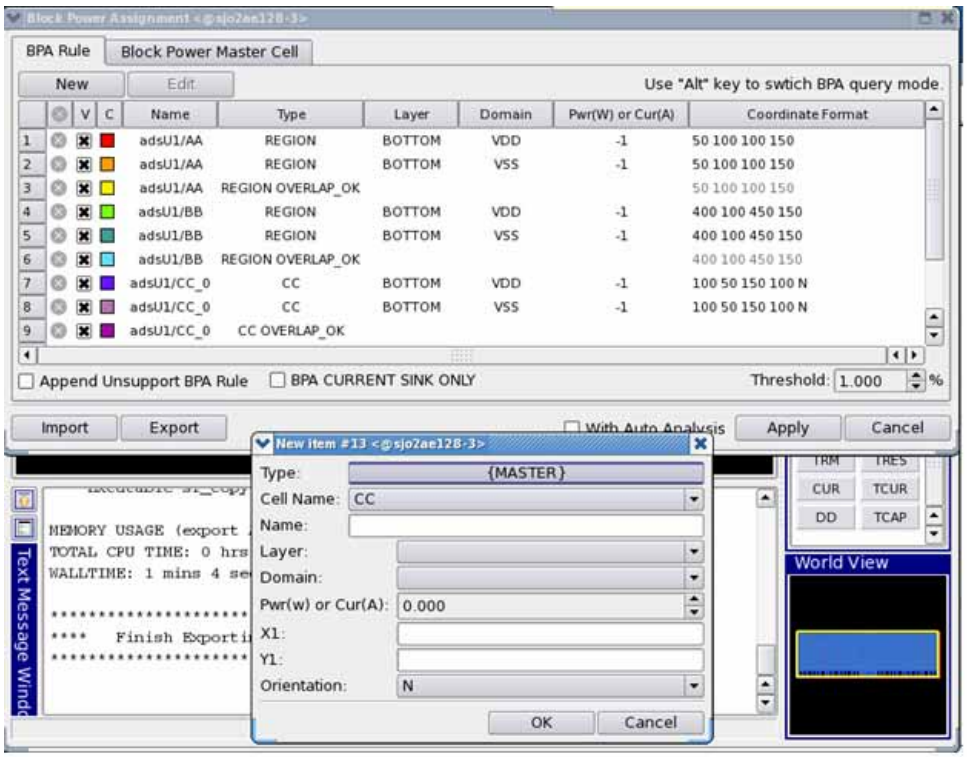


Figure B1-12 BPA master cell handling

You can also define rectilinear regions for BPA for non-MMX cell setup (MMX_PIN and MMX_REGION cannot be used). The steps are illustrated in the following four figures:

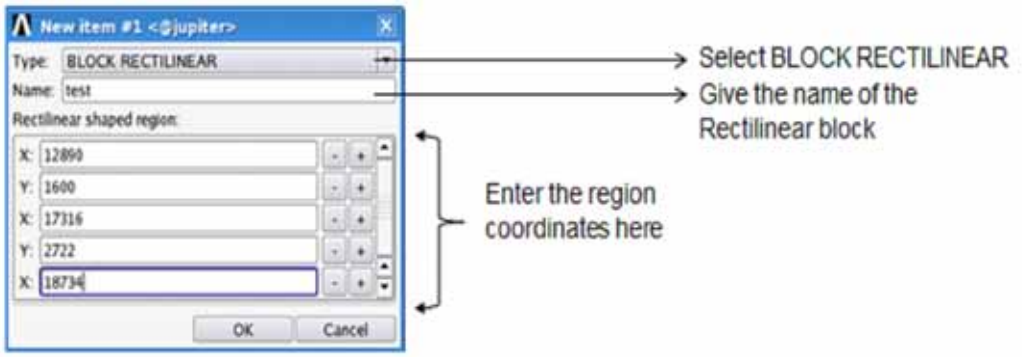


Figure B1-13 Block Rectilinear BPA dialog

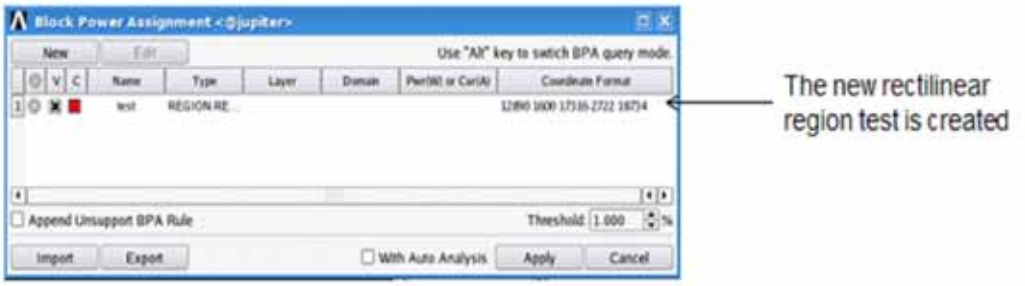


Figure B1-14 Rectilinear region power definition - step 1



Figure B1-15 Rectilinear region power definition - step 2

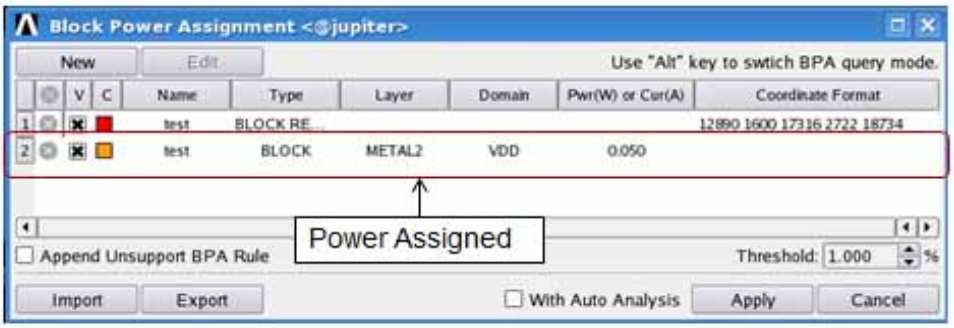


Figure B1-16 Rectilinear region power definition - step 3

Calculate

Executes power calculation for the static IR voltage drop analysis flow.

Import

Displays an ‘Import’ dialog box to specify a previously-executed power calculation directory to be imported.

Static -> Network Extraction

Displays a ‘Power Ground Extraction’ dialog box to specify the core R,L,C parameters to extract for VDD, GND, CLK and SIGNAL nets.

Static -> Package Constraints

Displays a ‘Package Constraints’ dialog box to specify the key R, L, C parameters for pads, wirebond/bumps, and package power and ground subcircuit models.

Static -> Static IR-drop Analysis

Displays a ‘Low Power Ramp Up Analysis’ dialog box that allows selection of the desired simulation parameters ‘Piecewise Cap Model’ and Simulation Time’ for ramp-up analysis.

Static -> EM Check

Uses current from static simulation analysis to perform the EM check command, the same as the Tcl command ‘perform em-check’, and displays key results in the log file.

Dynamic -> Power

Calculate

Executes power calculation for power integrity dynamic analysis flow.

Import

Displays an ‘Import’ dialog box to specify a previously-executed power calculation directory to be imported.

Dynamic -> Network Extraction

Displays a ‘Power Ground Extraction’ dialog box to specify the core R,L,C parameters to extract for VDD, GND, CLK and SIGNAL nets.

Dynamic -> Package Constraints

Displays a ‘Package Constraints’ dialog box to specify the key R, L, C parameters for pads, wirebond/bumps, and package power and ground subcircuit models.

Dynamic -> Vectorless Dynamic Voltage Drop Analysis

Executes vectorless dynamic voltage drop analysis.

Dynamic -> VCD-based Dynamic Voltage Drop Analysis

Executes dynamic voltage drop analysis using VCD file.

Dynamic -> EM Check

Uses current from dynamic simulation analysis to perform the EM check command, the same as the Tcl command ‘perform em-check’, and displays key results in the log file.

Results -> Log Message Viewer ...

Provides access to four different report viewers described in the following paragraphs: ‘Errors and Warnings Summary’ (default), ‘CPU/Memory Usage’, ‘Setup Design’, ‘Power’ and ‘Results’.

Errors/Warnings Summary

Displays a dual window showing a summary of the total messages generated during the analysis by type (Error, Warning, Info) in the upper window and a selected individual message in the lower window.

CPU/Memory Usage

Displays a table showing maximum Memory use, Total CPU Time, and Wall Time for each design stage from importing files through dynamic analysis. Note that the CPU time for the simulation step is *not* displayed as cumulative for convenience; the following step and all other steps are shown as cumulative CPU time.

Setup Design

Displays a list of files and their paths that have been imported in the design.

Power

Displays a dual window showing a summary of design power information by frequency and by domain in the upper window, and power related warning messages by instance in the lower window.

Results

Displays a simulation results report, such as Worst Dynamic Voltage Drop summary report for dynamic analysis.

Results -> Design Summary Report

Generate

Displays a ‘Report Result’ dialog box to allow specification of a Constraint File or choose the Sample, as well as the Report path. When **OK** is selected the report is generated and the Design Summary dialog box displayed. See section "MMX Design Summary Report", page 6-224, for more information about creating the Constraint File for defining the desired Design Summary Report.

View

Displays the Design Summary Report specified in the **Generate** command. See section "MMX Design Summary Report", page 6-224, for more information about Design Summary Reports.

This menu is grayed out until a Design Summary Report is generated.

Results -> List of ECOs ...

Displays a list of ECO changes made using either “What-if” analysis or FAO commands to allow you to do cross-probe checking and editing using a third-party layout editor. See section "Approach 6: Find ESD/Fuse cells that cause intentional shorts", page 3-150.

Results -> List of Pads ...

Displays a “List of Pads” dialog box showing Pin Name, x,y location, Layer, domain name of all pads in the design, and after analysis the Voltage, and Current values. The peak current and the time when peak current occurred after dynamic analysis is reported. Also, a “Plot Waveform” button allows you to plot the current waveforms of selected pads.

Results -> List of Transistor Information ...

By default, displays a dialog with a list of all transistors in the design, including instance name, net, pin name, average current, PRatio, x/y location, and transistor name. The dialog provides the following functions for selected transistors, using buttons at the bottom of the frame:

- **Plot Waveform**: plots current waveforms of 'Instance/Net/Pin' individually or region-based (using the 'Filter' icon at the top of the dialog).

  Whenever users plot region current (through tcl or GUI), advanced filtering is triggered to filter out all xtors outside the region plot. Users can perform this multiple times and all of the regions are recorded until the dialog box is closed. This “List of Transistor Information” dialog must be open during GUI or tcl plot current -region command in order to trigger advanced region-based filtering.
- **SPT** : minimum resistance calculation is executed when a single transistor is selected.

Results -> List of Transistor Resistance ...

Calculates resistance and displays a table reporting both “Transistor Resistance Strength” (TRES) and “Transistor Actual Resistance” (TARES), as shown in Figure B1-17

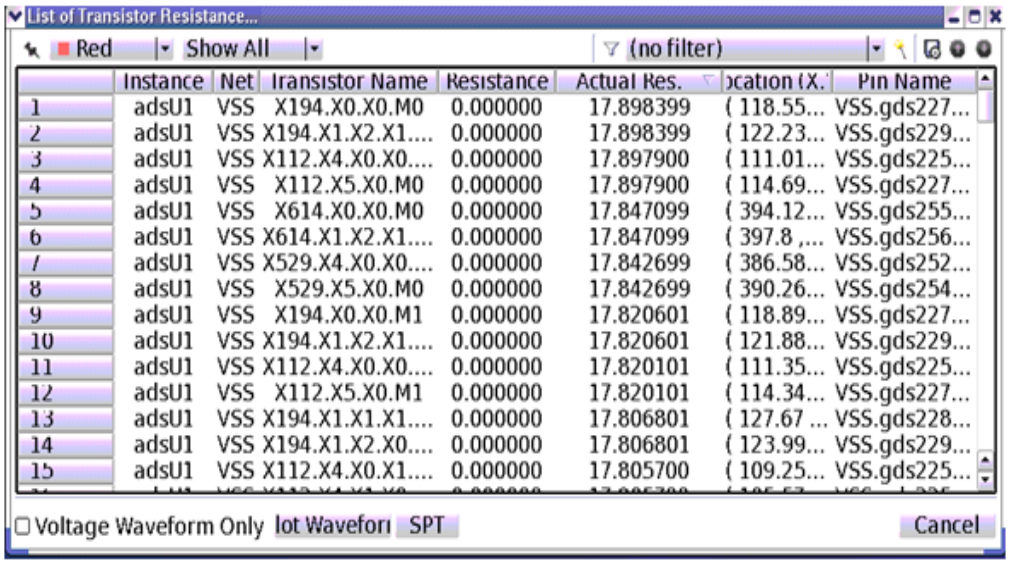


Figure B1-17 List of Transistor Resistance dialog

SPT can be executed when a single transistor is selected, and the **Plot Waveform** option can be used to plot voltage and current waveforms of selected transistor pins. You also can apply the advanced filter function to specific regions for ease of evaluation.

Results -> List of Resistance Constraint Checks ...

Displays a dialog showing directories from which to select a *<design>.res_constraint_check.rpt* file to get constraint check results.

Results -> List of Guard Ring Actual Resistance ...

Displays a substrate guard ring actual resistance ranking report dialog based on the guard ring actual resistance map (see section "Actual Resistance Map", page B1-599) to evaluate potential latch-up problems due to weak guard ring resistance connections.

Results -> List of Worst EM ...

Displays an ordered list of worst-case high electromigration locations, with the wire x,y starting and ending locations for each occurrence. You can also highlight and zoom to any of the critical EM locations by using the ‘Go to Location’ button, and also step through the various locations using the ‘Prev’ and ‘Next’ buttons.

The latest EM mode or ESD rule name is displayed in the “List of Worst EM” dialogs after EM or ESD analysis. When you switch to a different EM mode or ESD rule, the new EM mode or ESD rule name is displayed in color maps and reports so it is clear which EM mode or ESD rule is displayed.

Results -> List of Worst IR for Wires and Vias ...

Displays a ‘Report of Worst IR for Wire and Via’ dialog box, with an ordered list of worst-case voltage drops at Power and Ground nodes, with the x, y Location and Layer of each. By default the worst 1000 IR drops from all P/G nets are listed. You can also select the Worst N (100 by default) IR drops per layer of each net. A 5% IR drop threshold is used to select the worst nodes by default. The list can be shortened by selecting a larger % drop threshold in the ‘Threshold’ box.

Layer-based filtering for worst IR drop has four options to include desired layers and four options to exclude desired layers (options starting with “!”). After setting these filters, you

must save using the “Save” button. The results also can be exported using the ‘Export File’ option

Results -> List of Worst IR Instances (Static) ...

Displays a ‘Report of Worst IR for Wire & Via’ dialog, with an ordered table of worst-case high IR drop instances, with following information on each instance: Vdd-Vss, Vdd drop, Vss bounce, Ideal Vdd, and the x,y Location of the node. You can select the domain to show for multiple-domain designs. You can also zoom to any of the critical instances and highlight them by using the ‘Go to Location’ button.

This menu is grayed out when the design consists of a single MMX instance.

Results -> List of Worst Instance DVD ...

Displays a ‘Report of Worst Dynamic Voltage Drop Instances’ dialog box, with an ordered table of worst-case high DvD instances, with following information on each instance: Ideal (nominal) Vdd, Ave DV (average dynamic voltage), Max DV, Min DV, Min DV WC (worst case condition), and the x,y Location and instance Name of each. Using the ‘Sort By’ box at the bottom of the dialog, you can sort the list of critical instances by Average, Maximum or Minimum VDD minus VSS differential. You can also zoom to any of the critical instances and highlight them by selecting an instance in the list and clicking on the Zoom button. The ‘DvD Plot’ button displays an XGraph voltage drop waveform of a selected instance. By default a 5% DvD threshold is used to select the worst instances, based on the criteria selected (‘Average Vdd-Vss’ by default). The list can be shortened by selecting a larger % drop threshold in the ‘Threshold’ box, such as 10% or 20%. You can navigate a large list of instances using the ‘First’, ‘Last’, ‘Prev 1000’, and ‘Next 1000’ buttons at the bottom of the dialog.

This menu is grayed out when the design consists of a single MMX instance.

Results -> List of Worst Transistor Pin Voltages...

Displays a dialog listing the voltage domains in the left side. After selecting a voltage domain, the Report of Worst Transistor Pin Voltages is displayed, giving the Min Volt’, ‘Max Volt’ and ‘Avg Volt’, pin x,y location, pin name, and transistor name. You can click on the ‘Min Volt’, ‘Max Volt’ and ‘Avg Volt’ column headers to sort the worst transistor pin voltage entries.You can also select and zoom to any of the critical pins by using the ‘Zoom to Location’ button, and view an XGraph plot of the voltage using the ‘Voltage Plot’ button.

- Method 1: From the **Results -> List of Worst Transistor Pin Voltages**”, select the desired transistor pin entries and click the **Plot Waveform** button.
- Method 2: From any transistor pin map, select a transistor pin by using the left mouse button (use Shift + left mouse button to incrementally add new multi-selections in the color map, or use Ctrl + left mouse button to remove pins from multi-selection). Then use the middle mouse button to pop up the menu and select the **Plot Waveform** entry to invoke the waveform viewer. A sample display is shown in Figure B1-18.

The TCL command for plotting transistor current is:

```
plot current -region -xtorname <device> ?-pinname <pin_name>?
```

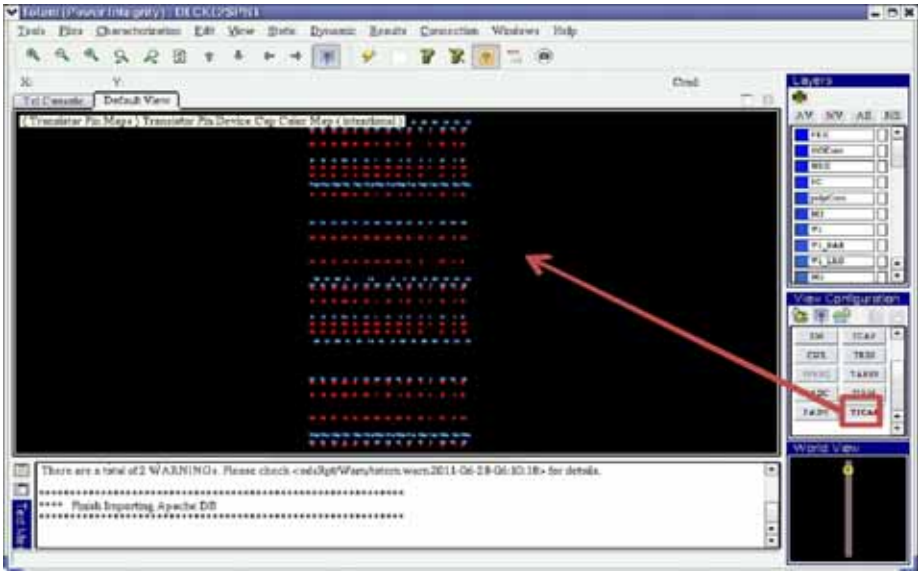


Figure B1-18 Transistor waveform plotting

Sorting worst transistor pin voltages. From a pre-defined multi-frame configuration use “waveform debug”, “waveform debug2”, and “waveform debug3” to bring up the multi-frame cross-probing debug interface. One example of “waveform debug2” is shown in Figure B1-32.

The browser allows filtering entries by transistor name in the “List of Worst Transistor Pin Voltages” dialog, as shown in Figure B1-19.

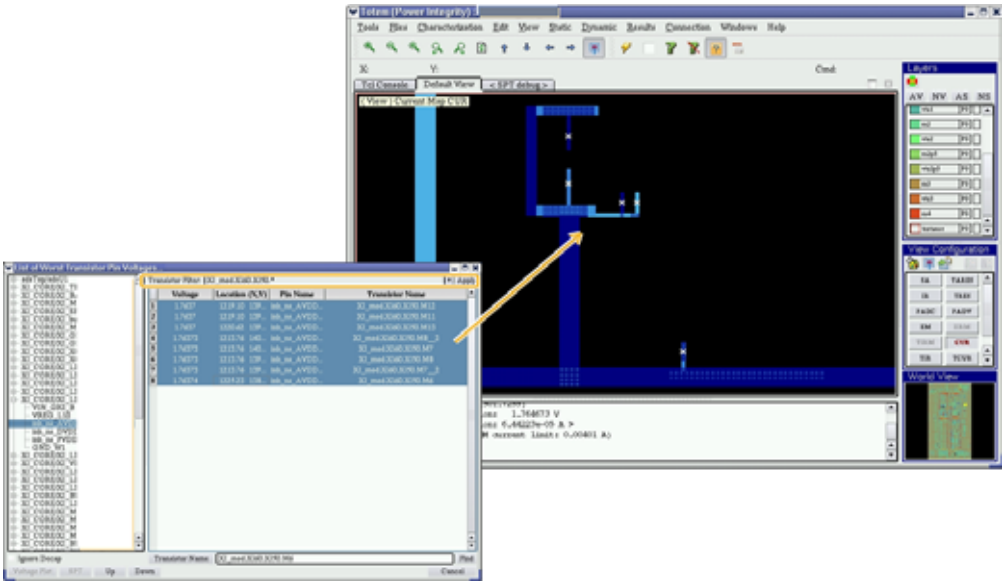


Figure B1-19 Transistor name filtering

You can also change the arrangement of entries in the dialog from instance-based to net-based. This lists the transistor pins of all instances in the design in single ranking table to

identify the worst transistor pins, without going through instance-by-instance to find the worst transistor pin or transistor pins with violations.

Also, in the **List of Worst Transistor Pin Voltages** dialog, you can select a transistor pin that you want to examine for a minimum resistance path (SPT). Once a path is selected, click the ‘SPT’ button at the lower left corner of the dialog to bring up an ‘SPT debug’ window to examine design weaknesses, as shown in Figure B1-20.

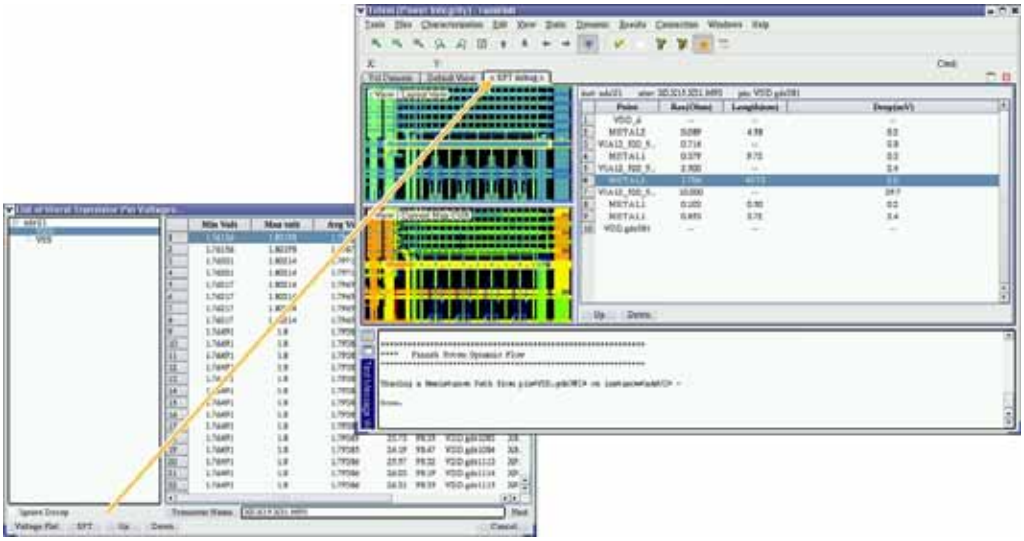


Figure B1-20 SPT debugging from list of worst transistor pin voltages

Results -> List of Worst Transistor Pin Current ...

Displays a list the worst transistor pin currents in the Power EM flow, including the pin name, x,y location, transistor name and average, RMS, and Peak current.

Results -> List of Highest Power Instances ...

Displays a ‘Highest Power Instance Report’ dialog box, with an ordered list of worst-case high static power consumption instances, with the X,Y location and instance name of each occurrence. You can also highlight and zoom to any of the critical instances by using the ‘Zoom to Location’ button.

This menu is grayed out when the design consists of a single MMX instance.

Results -> Supply Current Waveforms ...

Brings up a ‘Current Waveform Viewer’ dialog to choose current waveforms to be displayed. Both “battery” and “demand” current types can be plotted for user-selected nets.

- If only **Demand Current** is selected:
  - If all power nets are selected, total demand current is displayed.
  - If one or more of the power nets, but not all, are selected, then net-specific demand current waveforms are displayed. Ground net selections are treated the same.
- If only **Battery Current** is selected:
  - If all power nets are selected, total battery current is displayed.
  - If one or more of the power nets, but not all, are selected, then net-specific battery current waveforms are displayed. Ground net selections are treated the

same.

- If both **Battery Current** and **Demand Current** are selected:
  - If all power nets are selected, total demand current and total battery current are displayed. Ground net selections are treated the same.
  - If one or more of the power nets, but not all, are selected, then net-specific battery and demand current waveforms are displayed. Ground net selections are treated the same.

The presimulation time period also can be displayed if the **Show Presim** button is clicked.

Results -> Analysis Histograms...

Displays an ‘Analysis Histogram’ dialog box to specify what parameters to display in the histogram: Dynamic Voltage Drop, Static IR, Static EM, or Dynamic EM. Histograms can be constructed ‘By Layer’ selection, or ‘By Instance’ for criteria

- Average/Max/Min DvD over Timing Window,
- Minimum DvD in Simulation Cycle,
- Max Vdd Drop, or
- Max Vss Bounce

For multiple Vdd/Vss domain designs, histograms can be displayed by domain Net Name.

‘Limits’ boxes provide automatic selection of ‘Lower’ and ‘Upper’ mv limits, as well as a choice of bin size or number, or values for all these display parameters can be selected.

To display analysis data for elements only within a specified bounding box area, use the TCL command 'condition set -xy <x1 y1 x2 y2>', where x1,y1 and x2,y2 are the corners of the desired bounding box to be analyzed.

Results -> Movie ->

Make

Displays a dialog box to select the file for storing the simulation movie, and also chose whether you want a Full Chip Movie or a Partial Chip at Current Zoom.

Show

Displays a window to show the color-coded results of dynamic simulation in steps, showing the step number and simulation time for each step. You can use the normal video controls, such as Power (dismisses window), Start, Stop and Pause, or you can change the frames per second speed, or manually step through the simulation.

Connection -> Connect to layout editor ...

Sets up connection to a third-party layout editor, as described in section "Approach 6: Find ESD/Fuse cells that cause intentional shorts", page 3-150.

Totem supports cross-probing to Cadence Virtuoso and Springsoft Laker layout editors and Virtuoso Schematic viewers. To set up these capabilities, follow the procedures described below:

Cadence Virtuoso cross-probe setup

1. Virtuoso *.cdsinit* file setup:
   a. Edit the *.cdsinit* file to load Apache skill code program.
   b. Append file *$APACHEFILE/lib/crossprobe/cdsinit-sample* in your *.cdsinit* file. The *.cdsinit* file could be in the (1) current working directory, (2) user's home directory, or (3) Virtuoso installed/setup directory.
2. Source the Virtuoso resource file *cadence.rc* or *virtuoso.rc.*

3. Edit and source Totem resource file *totem.rc.* You can find the *totem.rc* template file under *$APACHEROOT/lib/crossprobe/totem.rc*.

A cross-probing icon on the View Control toolbar turns Virtuoso cross-probing On and Off. This icon is only enabled and displayed when the Virtuoso-to-Totem proxy cross-probing mode is used.

Totem-Virtuoso cross-probing/analysis can be invoked on a different machine for performance/capacity improvement using industry standard interfaces. Currently only LSF and SSH interface is supported. The usage procedure is:

1. Set the environment variable `TOTEM_SETUP_WIZARD_DIR` to your copy of the Totem wizard configuration directory.
2. Add entries to the job submission configuration file at:
   `$TOTEM_SETUP_WIZARD_DIR/wizard/wrapper/wrapper.ini`
3. Set the environment variable APACHE_CROSS_PROBING_WRAPPER_SET to the user-defined entry.

Springsoft Laker layout editor cross-probe setup

1. Source the Laker resource file *laker.rc*
2. Edit and source the Totem resource file *totem.rc.* You can find the *totem.rc* template file under *$APACHEROOT/lib/crossprobe/totem.rc.*

Required environment variables

Inside the *totem.rc* template file, you must set up the following global environment variables:

A. Cross probing

```
setenv APACHE_CROSS_PROBING_LAYOUT <layout_editor_binary_name>
setenv APACHE_CROSS_PROBING_SCHEMATIC <schematic_binary>
```

Specify the 3rd-party binary name.

B. Design setting

```
setenv APACHE_CROSS_PROBING_DBPATH <design_database_path>
```

Specifies the directory containing *cds.lib* (Virtuoso library header file) or Laker layout database directory, which contains l*aker.rc* file.

```
setenv APACHE_CROSS_PROBING_TOPDESIGN <top_cell_name>
```

Specifies the design hierarchy/cell name in this variable to allow Totem to find the correct hierarchy in the layout design database.

```
setenv APACHE_CROSS_PROBING_LIBRARY <library_name>
```

Specifies the layout design library name to allow Totem to invoke the 3rd party layout editor and load the correct design library (there may be multiple design libraries for the same hierarchy).

Replay mode setup variable

```
setenv APACHE_CROSS_PROBING_REPLAY_MODE y
```

Forces automatic entering of the replay mode for connection setup. This variable is *required* for Laker, and optional but recommended for Virtuoso.

Cross-probing port setting variables

```
setenv APACHE_CROSS_PROBING_WAITTIME <connection_timeout_limit>
```

Specifies connection time-out limit to 3rd party layout editor. Default time is 30 seconds.

```
setenv APACHE_CROSS_PROBING_STARTPORT <startport_id>
setenv APACHE_CROSS_PROBING_PORTRANGE <range_number>
```

Specify the connection socket port range. Totem starts to search available ports in order from <startport_id> and stops searching when reaching a port ID larger than (<startport_id> + <range_number>). Default ports are 11000 for APACHE_CROSS_PROBING_STARTPORT and 5000 for APACHE_CROSS_PROBING_PORTRANGE

After setting up cross-probing with the above procedure, use the following steps to perform cross-probing investigations in Totem.

1. Choose the GUI menu command **Connection -> Connect to layout editor**. Totem displays a dialog window to allow you to modify layout design variable settings, as shown in Figure B1-21.
2. After clicking the **OK** button, Totem attempts to connect to the selected 3rd party layout editor. If the socket connection can be made within the time-out limit, the 3rd party layout editor GUI is displayed.

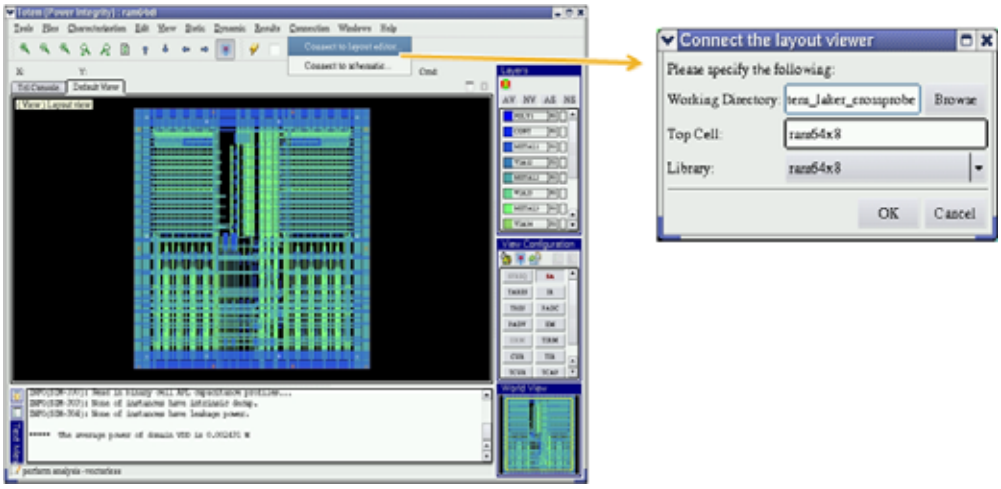


Figure B1-21 Making the cross-probing connection

Totem layout cross-probing capabilities include:

- Zoom-In / Zoom-Out
- Highlighting geometry in the layout window
- Zooming to a selected transistor location from a GUI list of key transistor locations (see Figure B1-22 and Figure B1-23). Use the command **Results -> List of Worst Transistor Pin Voltages.**

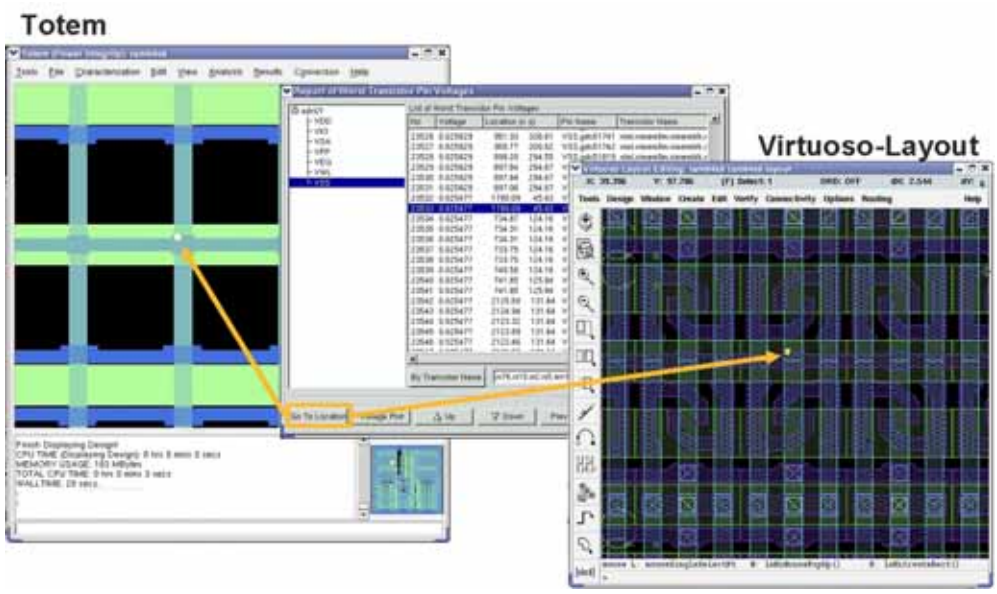


Figure B1-22 Cross-probing to Virtuoso layout

- Zooming to a selected EM violation location from a list or critical EM locations, use the command **Results -> List of Worst EM.**
- for table look-up of a selected ECO using “what-if “analysis, use the command **Results -> List of ECO**

The 3rd-party layout editor displays the items identified in Totem.

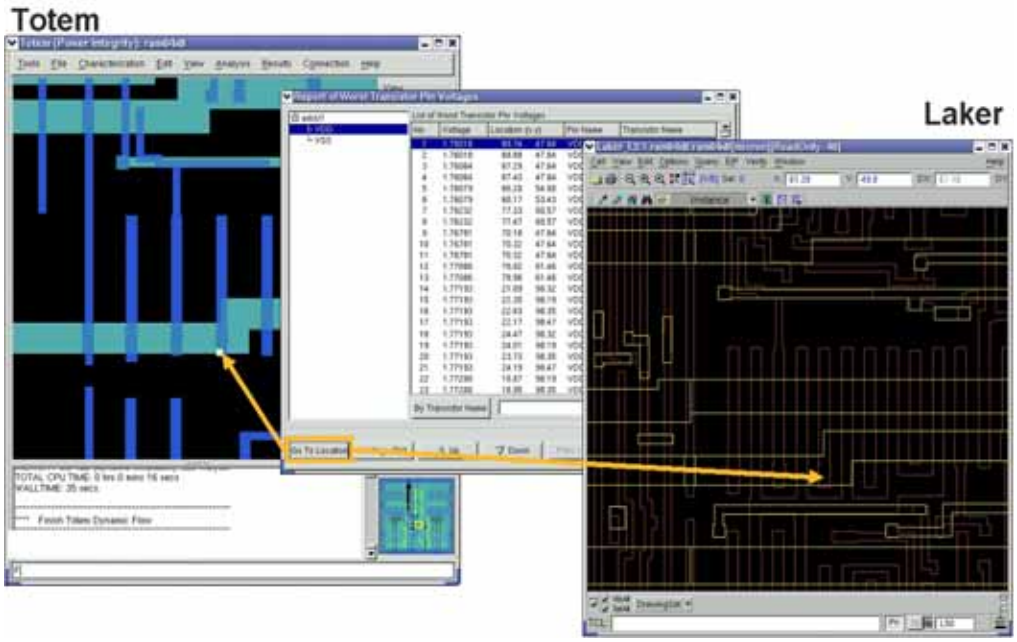


Figure B1-23 Cross-probing to Laker layout

Connection -> Connect to schematic ...

See setup in section "Cadence Virtuoso cross-probe setup", page B1-614

Uses the Totem cross-probing function to connect to Virtuoso Composer to view the schematic, as shown in Figure B1-24. This can help debug EM / IR / DvD problems that may be caused by design issues in the circuit.

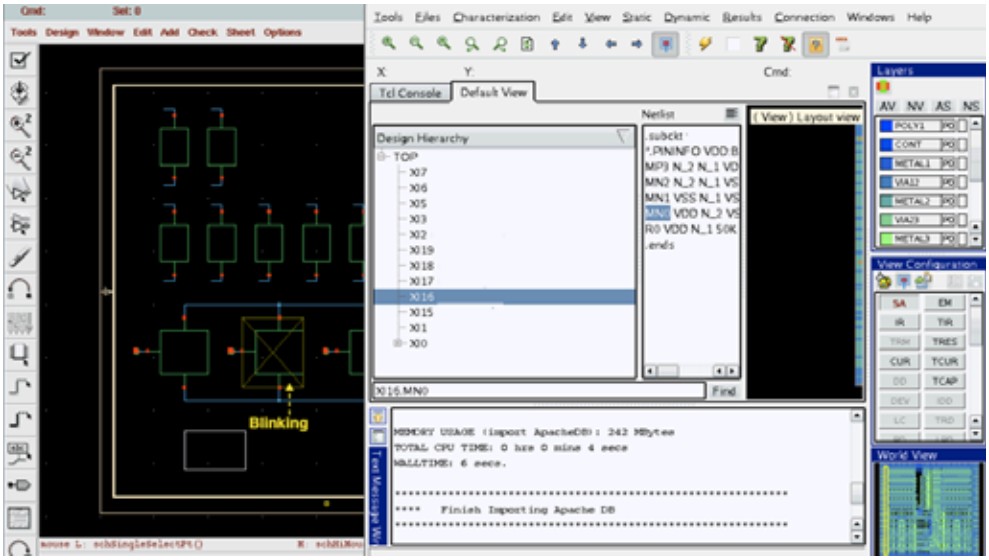


Figure B1-24 Cross-probing in Virtuoso Schematic Composer

Use of schematic device names in the DB for Virtuoso cross-probing

The default device names stored in the Totem database change from the name in DSPF netlist to the name in the simulation netlist, as follows:

- in the SPICE_NETLIST + DSPF_NETLIST flow: the device name from the SPICE_NETLIST
- in the DSPF-only flow: the device name in the instance section in the DSPF_NETLIST file

This helps set up Totem-to-Virtuoso composer schematic netlist cross-probing.

Totem Explorer

Explorer -> Generate

Displays a dialog box to allow you choose the Data Integrity, Design Weakness and Hot Spot results that you want, and the threshold values controlling the data to be displayed. Data Integrity checks available include:

- CMM Validation
- APLMMx Current Consistency
- APL Current
- APL Cap
- APL Piecewise Cap
- LIB
- LEF
- GDS2DEF

- GDS2DB
- DEF
- SPEF
- STA
- VCD
- GSR Tcl
- Package Settings
- Pad File

Design Weakness checks available include:

- Pad Placement Quality
- Package Drop Contribution
- PG Resistance Distribution
- PG Resistance Imbalance
- Switch Placement Quality
- Decap Distribution
- Power Distribution Quality
- Clock Buffer Clustering
- Peak Current
- Simultaneous Switching

The Hot Spot checks available are:

- Dynamic Voltage Drop
- MMX Cell-Based
- Static IR
- Power EM

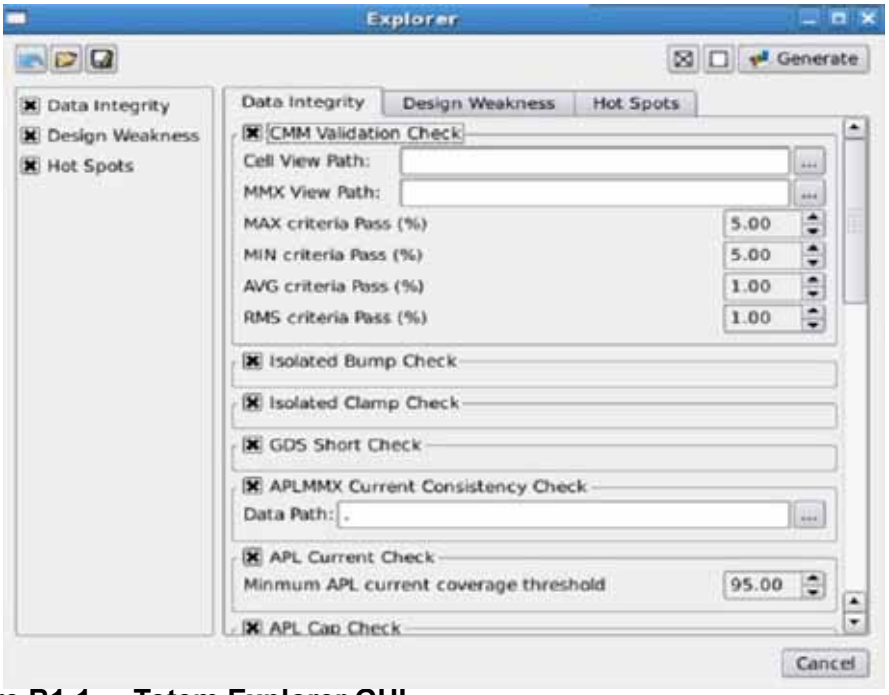


Figure B1-1 Totem Explorer GUI

After the desired results are selected under each tab, the Explorer Summary dialog is displayed, and the results can be analyzed.

Explorer -> View

Displays the Summary of results selected and created with the **Generate** command.

Windows -> Display Multiple Frames

Setup Configuration...

Allows setting up a multi-frame configuration, based on your debug preferences, using the menu selection **Setup Configuration**, as shown in Figure B1-2. You can **Save / Reload** previous configurations.

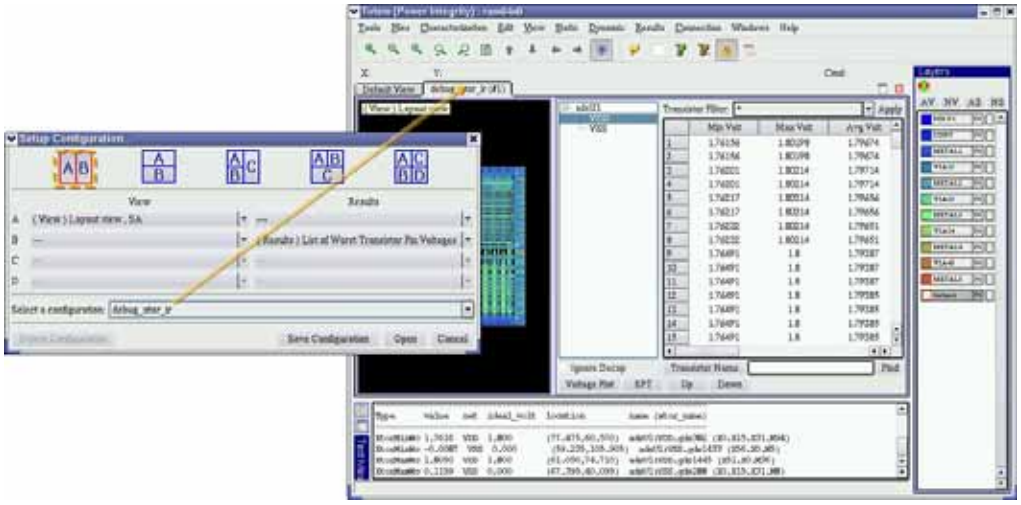


Figure B1-2 Multi-frame configuration setup

There are many types of preconfigured multiple-frame views for power analysis, to make setting up the debug environment more convenient, as shown in Figure B1-3.

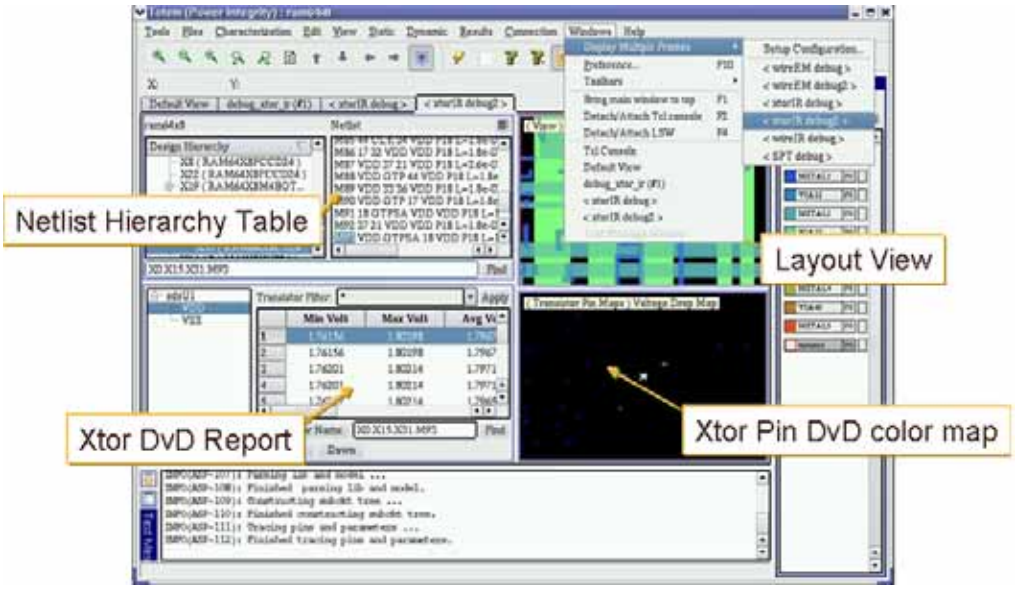


Figure B1-3 Preconfigured multiframes

Per frame net selection allows selection of different nets in different GUI frames to allow cross-checking of different nets at the same time, as shown in Figure B1-4.

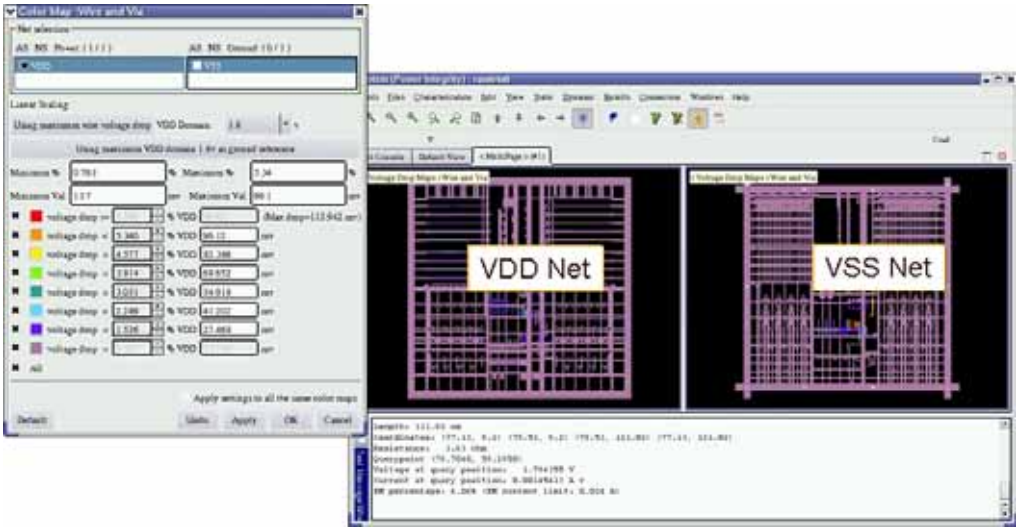


Figure B1-4 Per frame net display and selection

Windows -> Preferences ...

Brings up a Preferences dialog box, with the following option selections:

Common options

- GUI Font: (Sans Serif 9 Normal default)
- Text Message Font : (Courier 10 Normal default
- Tcl Console Font : (Courier 10 Normal default)

- Totem Explorer Font: (Sans Serif-8-Normal default)
- Highlight : Blinking or Color (default)
- Highlight Sync : Current View or All Views (default)
- Highlight Method : Auto Zoom (default) or User defined.

  Auto Zoom: zoom and center at the selected location.

  User defined: highlight the select location (marker), but not auto center, if the selected location is within the current zoom canvas. If the new selected point is outside current zoom view, the GUI uses current fixed zoom ratio and centers at the new selected point.
- Zoom sync : Current View or All Views (default)
- Waveform Viewer : xgraph or Totem Waveform Viewer (default)
- Waveform Viewer Integral Method : Left-Rectangle (default) or Trapezoid
- Color Maps : Smooth Color Spectrum or Disable GDS Instance in Color Map (default) The “Disable GDS Instance in Color Map” box allows disabling the instance IR map of GDS_CELLS to view the leaf instances underneath.
- Image background for printing: sets color of background for image printing (default black)
- Scroll Bar for Color Entries : 12 (default) - sets maximum number of color ranges before scroll bar is used.
- Editing Mode : Mode1 (default), or Mode2, which allows Virtuoso-style “what-if” editing functions to support via layers editing, such as adding and deleting vias.
- Auto Region Filter: Enabled (default) or Disabled
- Using Multi-threads Mechanism - “Follow GSR: MULTI_THREADS Setting” (default), or else override the GSR keyword value using the “Enable Thread Number” checkbox to set a maximum number of threads to use.
- Text Message Window Options: Provides a ‘Show Summary Page’ check box for a summary of Process steps completed in the log area below the design display (default On).
- Min Resistance Path (SPT) Analysis: By default “Support Multiple” paths is provided.

GDSII Display options

- Display Controls : selection of “GDS Text” and “Path Borders” (both default On)
- Array Display : Full (default), Border, or Source
- Filter Size : number of pixels (default 1)

Grid Controls

- Type : None or Dots (default)
- Minor Spacing: 0.002 (default)
- Major Spacing : 0.01 (default)
- Spacing Mode: “User Specify” (default) spacing, or “Automation Detection”
- X snap Spacing : 0.001(default)
- Y snap Spacing : 0.001(default)

Windows -> Toolbars

Note that this command is not available when the original GUI operating mode style is selected (Alt - p).

Show all toolbars ->

Displays both View Control and Query toolbars.

Hide all toolbars

Hides both toolbars.

View Control Bar

Check box to control display of the View Control toolbar (on by default)

Query Bar

Check box to control display of the Query toolbar (on by default)

Windows -> Bring main window to top

Displays main design view on top.

Windows -> Detach/Attach Tcl console

Displays separate TCL terminal window.

Windows -> Detach/Attach LSW

Displays separate Layers, View Configuration, and World View windows.

Windows -> Tcl Console

Displays TCL terminal window as primary view.

Windows -> Default View

Displays full design view.

Windows -> Text Message Window

Brings detached Text Message window to top.

Help -> About...

Displays information about the Totem software release being utilized.

Help -> Manual

Provides a link to the latest “Totem User Manual” in PDF format.

Bindkey Functions

Functional Summary

- Functional bindkeys (Virtuoso-style behavior)
  - **F1** - Brings main window to top
  - **F2** - Detaches / attaches TCL console
  - **F4** -Detaches / attaches LSW panel
  - **F10** - Brings up ‘Preference’ dialog
- View Control bindkeys
  - **Ctrl** + **Z** - Zoom in by 2
  - **Shift** + **Z** - Zoom out by 2
  - **F** - Fit All

W - Last view
Shift + **W** - Next view
Ctrl + **R** - Redraw
Up - Scroll up
Down - Scroll down
Left - Scroll left
Right - Scroll Right
Ctrl + F - Display only top hierarchy in GDS view
Shift + F - Flatten all hierarchies in GDS view
Ctrl + P - Show power pad
Shift + V - Go down one hierarchy in GDS view
Ctrl + V - Go up one hierarchy in GDS view
Ctrl + **N** - Display selected nets / exclude unselected nets
Shift + **N** - Display all nets

- GUI Command bindkeys

  Ctrl + **Q** - Exit GUI
  K - Creates ruler
    - Supports 45-degree ruler drawing
    - Supports multiple-segment ruler drawing
  Shift + K - Clears all rulers
  Alt + P - Switches between the RedHawk-like GUI and the enhanced Totem GUI

Reconfiguring Bindkeys

You can reconfigure bindkey settings using the following steps

1. Export the existing bindkey settings supported in the Totem GUI to a text configuration file using the TCL command:

   ```
   export keybinding <keybinding_filename>
   ```

   Note that this command is only available in the GUI, and not in batch or interactive modes.

2. Edit the bindkey configuration file created in step 1 to establish the new bindkey settings desired.

   The format of the bindkey configuration file is as follows:

   ```
   # <description of bindkey >
   <bindkey name> <bindkey definition>
   ```

   Bindkey definitions can be assigned to any combination of single letter, **Ctrl**, **Shift**, and **Alt** buttons.

   Note that no spaces are allowed in bindkey definitions.

3. The newly-defined bindkeys can then be imported using one of the following methods:

   a. In GUI mode, use the command

      ```
      import keybinding <keybinding_filename>
      ```

   b. Rename the keybinding configuration file '*keybind.cfg*' and put it in
      - your working directory, or
      - the directory *$HOME/.totem/*

If there is a *keybind.cfg* file in both of the directories above, the one in your working directory is used.

Appendix B2
Signal EM GUI and Menus

Introduction

This appendix describes the graphic user interface and menus available for running Totem Signal EM Analysis. For TCL invocation and command descriptions, see Appendix D, "TCL Command Reference".

For descriptions of Power Integrity analysis menus, see Appendix B1, "Power Integrity GUI and Menus"

For descriptions of ESD analysis menus, see Appendix B3, "ESD GUI and Menus".

Totem Signal EM GUI Description

Introduction

The Totem GUI functional areas are shown in Figure B2-1 below.

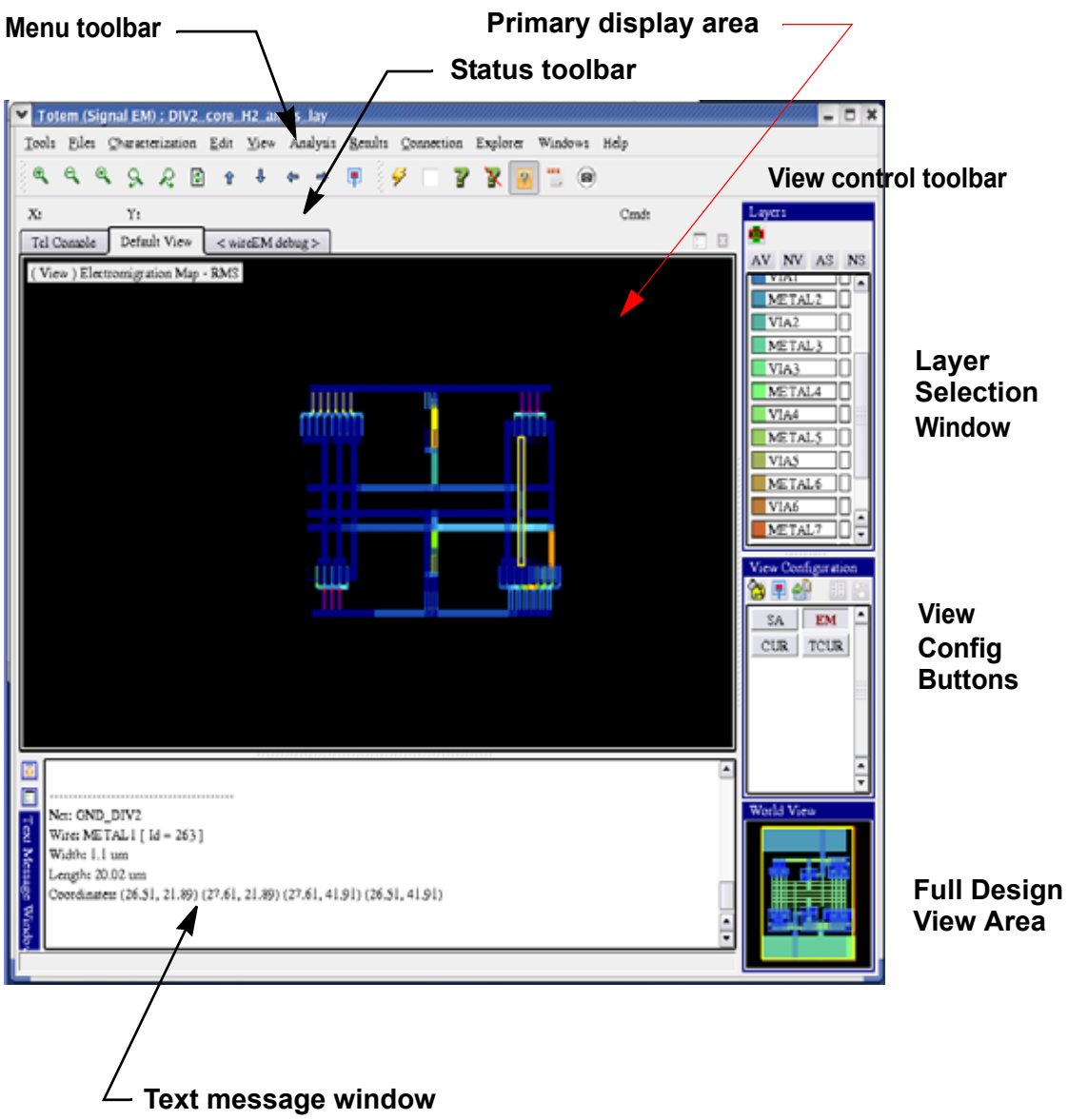


Figure B2-1 Totem signal EM graphical user interface

The Totem GUI provides a configurable multi-window, multi-page, multi-frame debugging environment. You can combine different color maps and reports on the same GUI page to debug design weaknesses and examine the source of design issues much more efficiently. A few key features are:

- Switch view menu based on analysis tool to only highlight product-specific features
- Configurable multi-frame multi-page interface to allow configuring the preferred debugging environment
- Pre-select and status bar helps query design information in a much more efficient way
- Embedded GDS viewer to allow cross-probing between analysis layout view and color maps with GDS viewer for easy debugging

The following figure is an example of how you can use the Totem GUI to configure a multi-frame page for debugging:

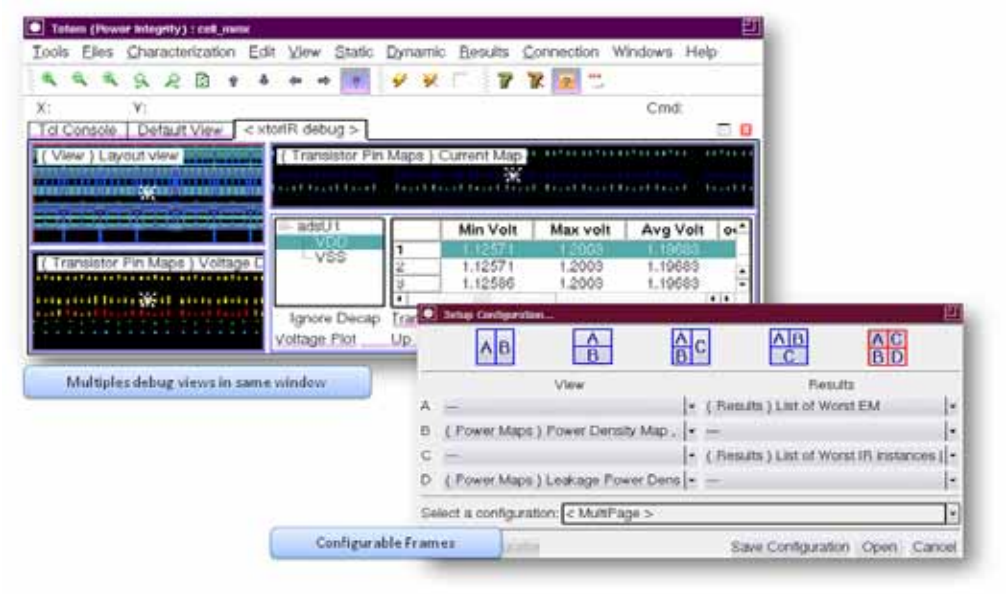


Figure B2-2 Totem multi-frame GUI

General GUI Features

GUI Look and Feel Preference Setting

The *Totem.ini* file can be used to control the GUI menu, dialogs, GDS View, Waveform Viewer, multi-frame setup and other common options, such as font type, font size, and highlight preferences. A template of the *Totem.ini* file is in the *lib/guicfg* directory in the default installation package (*$APACHEROOT/lib/guicfg/Totem.ini*).

The priority of available *Totem.ini* files is as follows.

1. From the local Totem run directory *adsGui/Totem.ini.*
2. From user's home directory *~/.totem/Totem.ini.*
3. From installation *$APACHEROOT/lib/guicfg/Totem.ini.*

If files in locations 1 and 2 are not available, then the default GUI preference settings from 3 are applied. Or you can use “F10” button to bring up the Preference dialog and click the “Reset” button on the bottom of the dialog to reset to Totem default settings.

Layers Window

The **Layers** window (see Figure B2-3) allows viewing information related to layers. The **Layers** window functions are:

- AV - All Visible
- NV - None Visible
- AS - All Selectable
- NS - None Selectable

Note that when **SA** is selected in the **View Configuration** window, and the “Configure Color Maps” button is clicked, a Layer Select dialog box is displayed (see Figure B2-5),

which allows changing the display features of each layer. Individual layer display can be changed using the check boxes. Click on the column headers to change the display characteristics for groups of layers: Vis (visibility), Fill/Line (object display type), Sel (select metal/via layers), Pin (select metal/via layer pins).

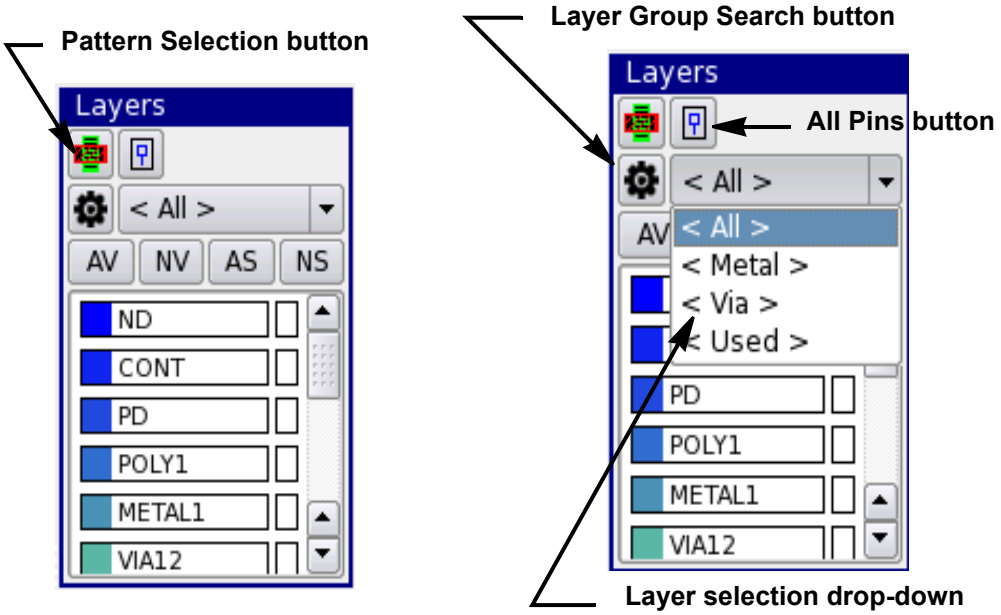


Figure B2-3 Layers selection window

In the **Layers** window, use the left top button to switch between **Pattern Brush** and **Transparent Drawing** modes, as shown in Figure B2-3 and Figure B2-4.Other capabilities of the Layer Selection dialog are.

- support for layer search and filtering using the Layer Group Search button
- display only “used layers”, which is turned on by default
- option for name-based search of layer-names, which also supports wildcards

the All Pins button turns On/Off transistor pins for all layers in the SA view, as shown in Figure B2-3.

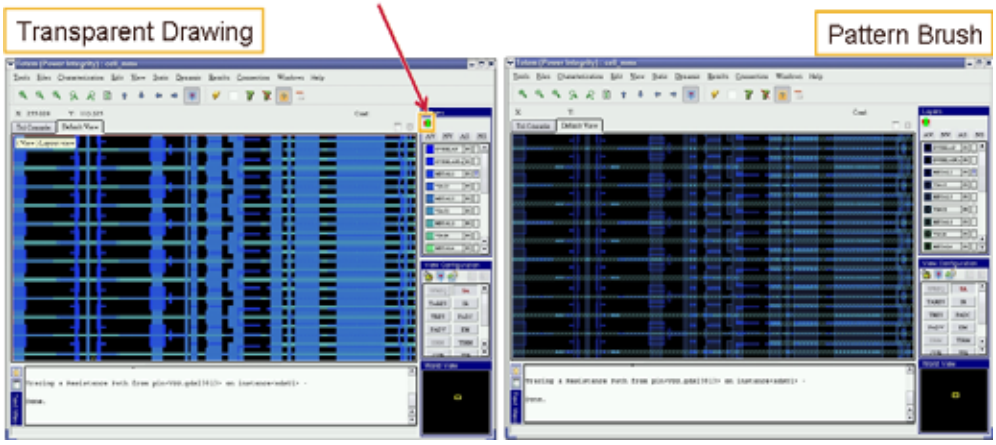


Figure B2-4 Pattern brush & transparent drawing displays

View Configuration Window

At the top of the **View Configuration** window, as shown in Figure B2-5, are buttons for opening a “Set Color Range” dialog, executing “Show Power Pad”, and opening a “Configure Color Maps” dialog.

The icons on the left side provide functions to “View GDS”, which brings up an **Import GDS...** dialog, and “View Netlist”, which brings up an **Import Netlist..**. dialog.

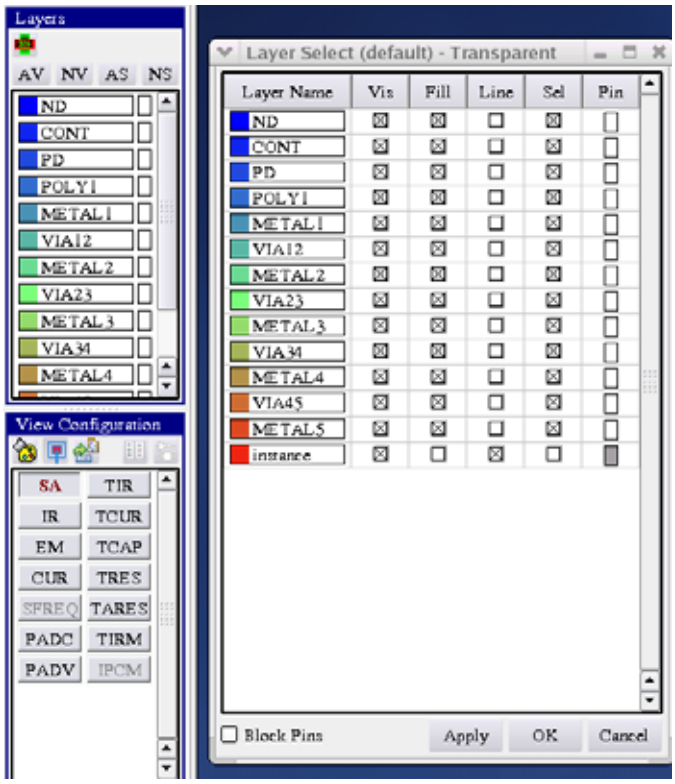


Figure B2-5 Layer Select and View Configuration windows

- “Set Color Range” (left Configuration icon)

  Displays a Color Map dialog box based on the ‘View Results’ or **View Configuration** window selection.

  Color maps may display either objects, a high parameter value, or a parameter density by tiled grid. Density displays use a grid composed of “tiles” that are 30 microns square by default. The size/number of the tiles can be changed using the ‘Grid Num’ or ‘Grid Size’ boxes on the dialog.

  One of up to 20 colors can be assigned to specific ranges of parameter values, such as static or dynamic voltage drop, power, EM severity, current, frequency, or capacitance. Display colors can be changed by clicking on the color button and then selecting a new color from the ‘Pick a Color’ dialog box that appears. Ranges of parameter values also can be turned on or off independently with check boxes. The **Default** button allows easy return to the original color display values.

- “Show Power Pad” (center left Configuration icon) - toggles the display of power pads on and off (on by default). VDD pads are shown in orange and VSS pads in white.
- “Configure Color Maps” (center Configuration icon) - in general the specific color

map dialog box for each of the following functions in the View Configuration window can be displayed by clicking on the ‘Set Color Range’ configuration button described above.

You can change color settings between 5 to 20 in the Color map dialog for wires and vias, to allow more color resolution in customizing color maps and movies, as shown in Figure B2-6.

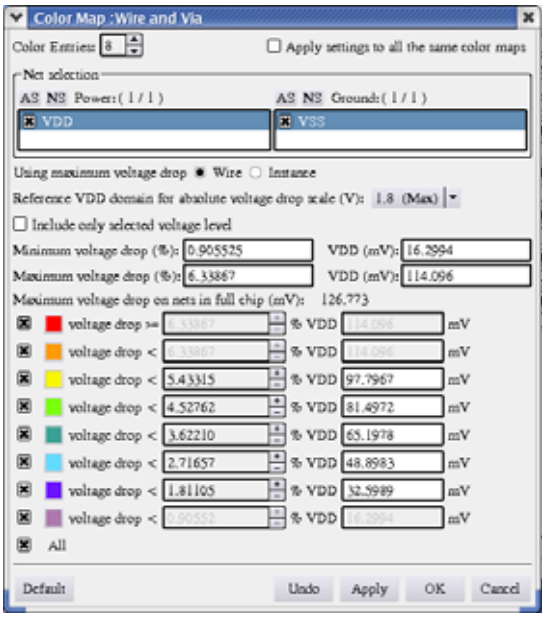


Figure B2-6 Color map color setting dialog

- View GDS (center right Configuration icon) - opens an “Import GDS ...” dialog, allowing you to specify a GDS file, include a Top Cell and Layer Map, as well as Third Party Technology Data files for importing.
- View netlist (right Configuration icon) - opens an “Import Netlist ...” dialog, allowing you to specify a Netlist File and a Top Cell for importing.

Configuration buttons

The buttons displayed in the **Configuration** window are customizable, and can be modified clicking on the “Configure Color Maps” button when **SA** is selected (see Figure B2-7).

You then can select entries from the left side table and use the right arrow button to add new maps in the panel or select entries from the right table and use the left arrow button to delete maps from the fast access panel.

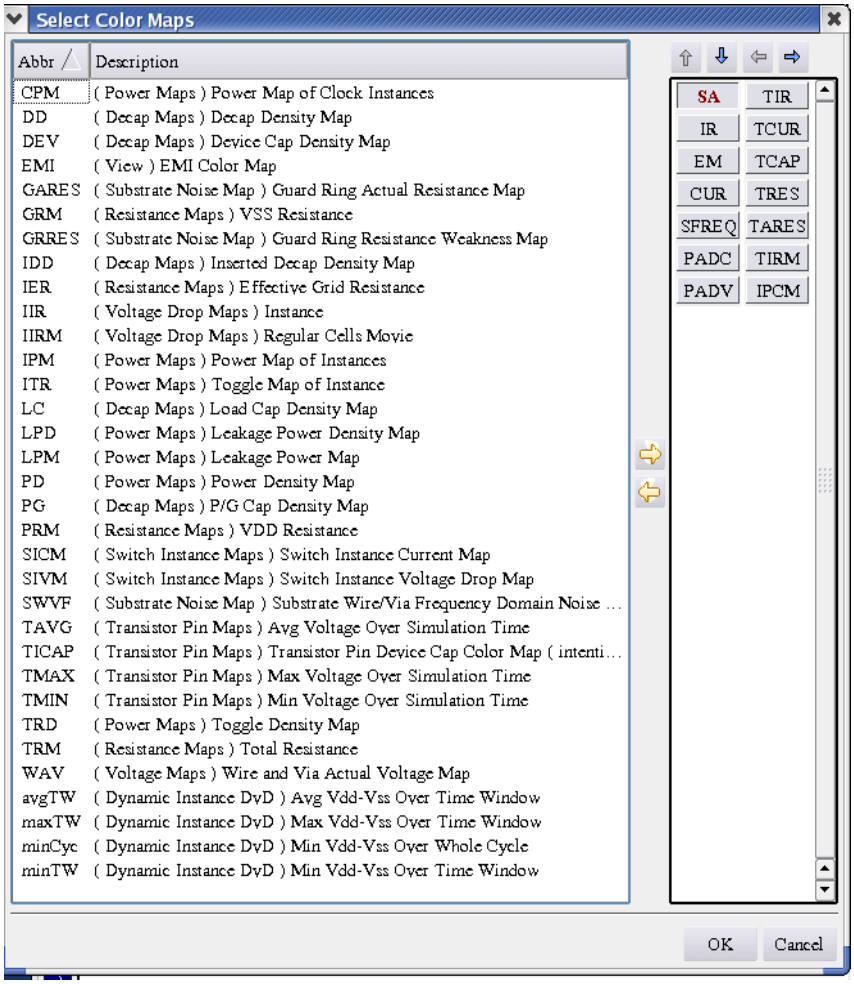


Figure B2-7 Select Color Maps dialog

The default View Configuration buttons are described following:

First row of 'View Configuration' buttons (default)

- **SA** - "Show all" of layout - displays some or all objects in the design layout (individual layer grids, instances, vias and pins per layer), based on the selections in the 'Layers' dialog box. The dialog allows a choice of colors for displaying each type of object, as well as showing objects in either solid or outline representations. By default pins are turned off, layer and instance objects are displayed solid.
- **CUR** - "Current Map" - displays results of maximum current by node for static or dynamic conditions, based on the color ranges selected in the 'Current Color Map" dialog. Individual colors represent a selected percent of maximum current in the design. Instances are not displayed by default. If displayed, instances are shown in their selected object color.

Second row of 'View Configuration' buttons

- **EM** - "ElectroMigration Map" - displays results of electromigration analysis for static or dynamic current conditions, based on object view selections in the 'Layers' dialog box and the color ranges selected in the 'ElectroMigration Color Map" dialog. (Instances are turned off by default, and are shown in selected object colors when turned on.)

The value displayed is the EM current tolerance, which is the computed fraction of current density in the wire relative to the maximum acceptable EM current density specified in the 'metal' keyword ('EM' parameter) in the **.tech* file. This value must be set for either static or dynamic condition of interest.

- **TCUR** - “Transistor Current Map” - displays transistor pins in colors identifying their effective current values. The effective current is the average current within the first cycle of the sp current profile, or the average current from BLOCK_POWER_FOR_SCALING. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Transistor Pin Current Color Map’ dialog, which allows you to select colors for a range of pin current values, set the current range displayed, and also increase the size of the pins for ease of viewing using the ‘Pin Size Scale’ window.”

GUI View Control Toolbar

Right-clicking at the right end of the menu toolbar brings up a dialog box to display the View Control and Query toolbars. GUI functions available from the View Control Toolbar and Query Toolbar are described below. See the key in Figure B2-8:



Figure B2-8 View control and Query Toolbar functions key

View Control Toolbar functions are (see Figure B2-8).

1. Zoom In
2. Zoom Out
3. Fit All
4. Last View
5. Next View
6. Redraw
7. Up - moves the view up
8. Down - moves the view position down
9. Left - moves the view position left
10. Right - moves the view position right
11. Pad Enable/Disable
12. Blinking Enable/Disable
13. Blinking Color setting
14. Query Object - displays a ‘Query Objects’ dialog box that allows searching for design objects, specified either by name or by x,y location. Also allows highlighting multiple instances of objects that are listed in a specified file. The highlight color for objects to be found also can be selected.

You can find information about an object such as a transistor in the query dialog and highlight the desired object in the GUI. Bring up the dialog by clicking on the 'Query Object' button. The Query dialog is shown in Figure B2-9 below:

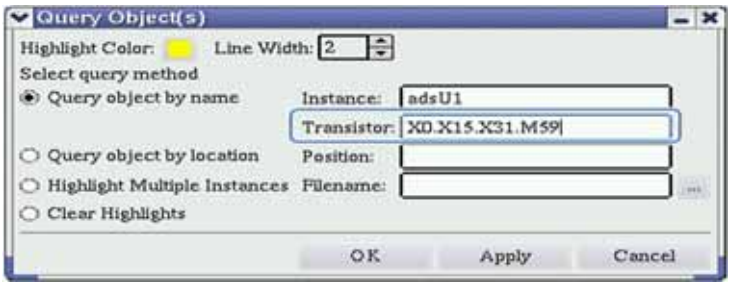


Figure B2-9 Transistor query by name

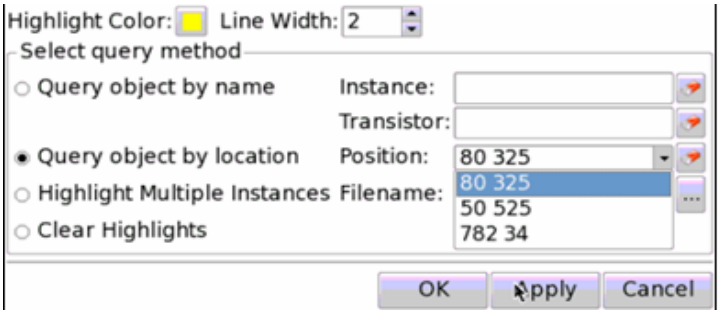


Figure B2-10 Multiple-query object by location

The last three position queries are saved and displayed on a pull down sub-menu, identifying the recently-queried positions, as shown in Figure B2-10. You can select any of the positions from the pull down menu, and then click on Apply or OK to zoom the Totem display to that region.

15. Clear Highlight - clears all highlights and rulers in the display
16. Pre-select Enable/Disable
17. Dump GIF
18. Snapshot - Takes a screen shot of the full Totem window and saves it in a GIF file in the selected filename in the dialog box that is displayed. GIF screen shots can be displayed using the 'xv' utility in the Linux OS.
19. Switch LSW GDS Layer Display - switches LSW layer display

Status Toolbar

The Status toolbar displays cursor location, cell name, DvD, current, resistance, or EM information for a selected transistor, wire, or via geometry as you move the cursor on the colormap, as shown in Figure B2-11.

For multiple frame viewing (set up with the **Windows** menu) the Status toolbar allows configuration of each frame for easy debugging. You can enable/disable Ruler Bar, Scroll Bar, and Frame Title, Sync all views, or only the current view.

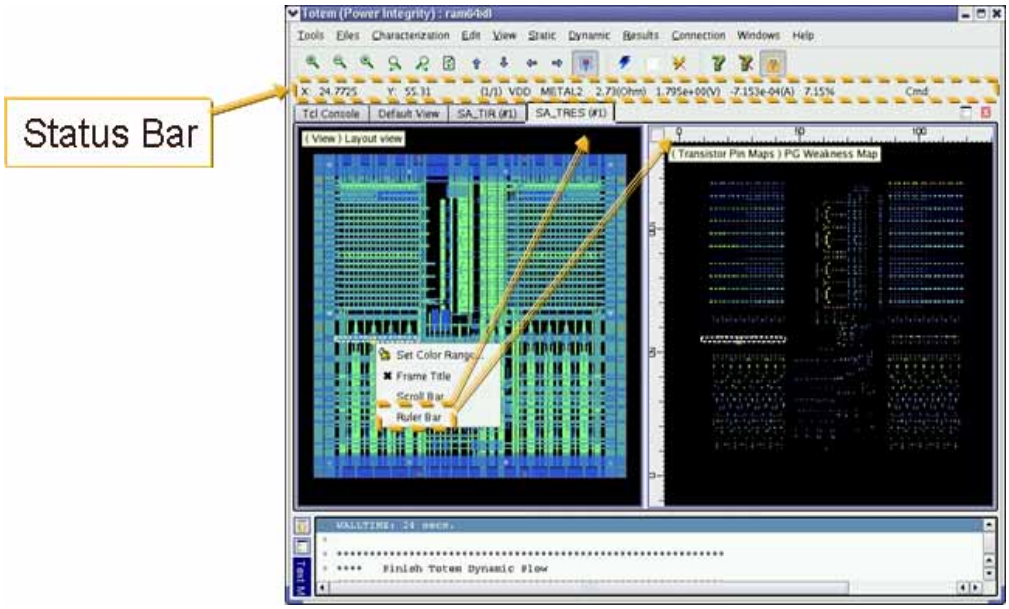


Figure B2-11 Easy frame and Status toolbar usage

Multiple selection and multiple highlighting

Multiple selections from a dialog, such as **List of Worst Transistor Pin Voltages,** can be performed using **Shift** or **Ctrl** + left mouse button, which highlights the selected pins, as shown in Figure B2-12.

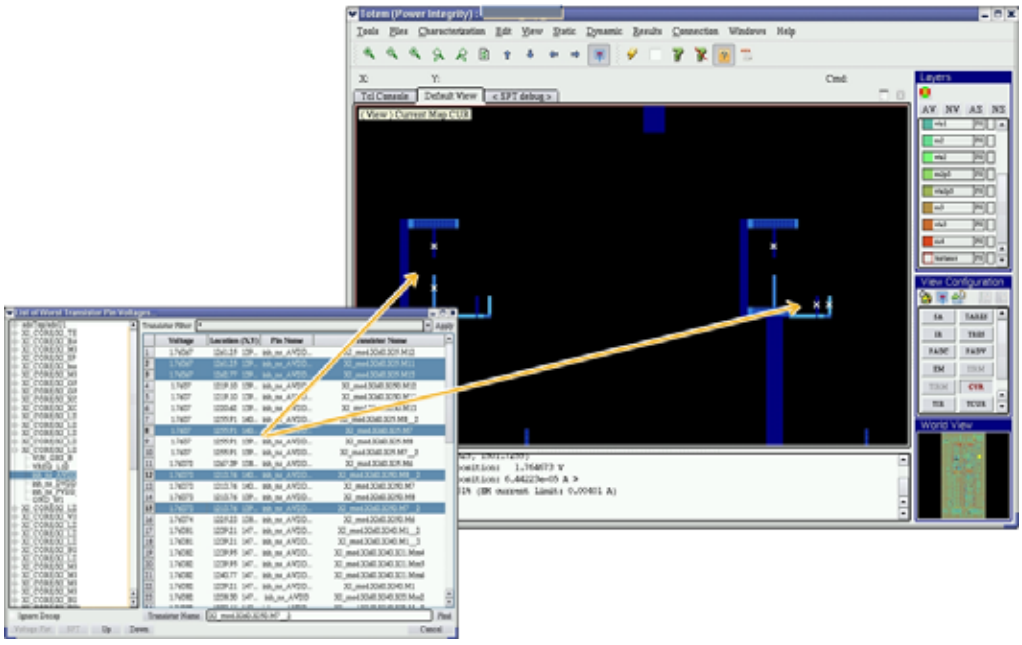


Figure B2-12 Multiple selection and multiple highlighting

Advanced Filter Function for Table Dialogs

Many table dialogs in Totem have a yellow filter wizard icon in the upper right corner, which displays a filter rule setting frame, and allows you to set up multiple advanced filtering rules to investigate and fix design issues more easily and quickly.

You can also set filtering rules column-wise. Each column has a different operation model. For example, for a Netlist (X,Y) column you can set “include/exclude” regions and widths, and the length columns allow you to set arithmetic >, >=, =, <, <= conditions.

After finishing setup, use the “Save As” button to set the filter rule name. Then click the “OK” button to create the filter rule and apply it.

GDS pre-selected cross-probing

Using the toolbar you can pre-select and select GDS cross-checking in the Totem layout view, as shown in Figure B2-13 below.

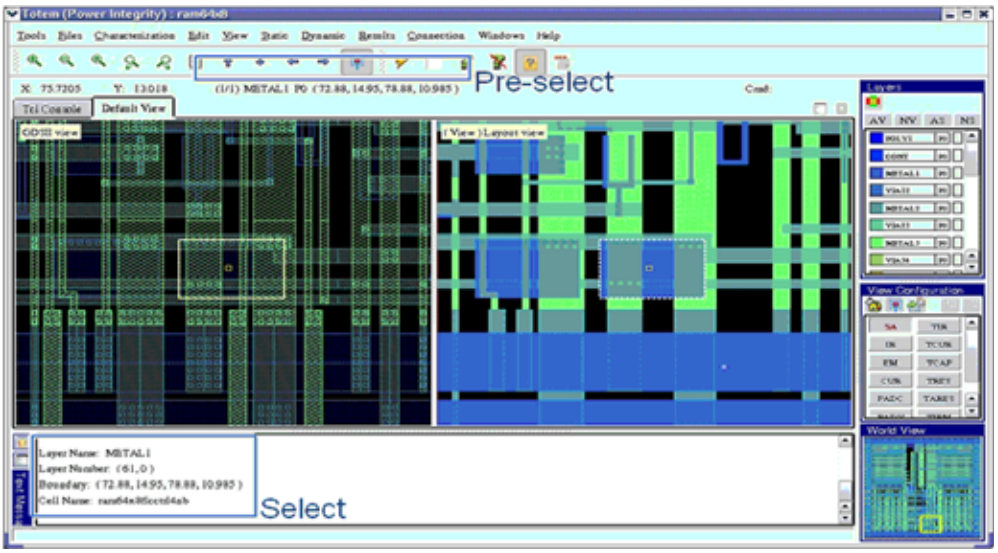


Figure B2-13 Pre-selection and selection of cross-checking

Customizable forms for all GUI table dialogs

The Totem GUI supports customizable GUI forms for all “List of XXX” dialogs. The following capabilities are available:

1. “Sort” function - click the header of each column to sort items in the column.
2. “Show/Hide” function - use a right mouse button to click on the header to pop up a menu to select Hide or Show function.
3. “Move” function: use the mouse to drag and drop the header of the column to a new position.
4. “Bookmark” function - use the bookmark function to show or filter bookmark entries.
5. “Up” and “Down” function - go “Up” and “Down” to browse through the table entries.
6. Auto Replay function - supports auto replay function for all GUI form changes.

SPT debugging from any pin-based colormap

The Shortest Path Tree (SPT) capability enables you to highlight the most resistive path from the selected transistor pin to the pad. You can examine the SPT in the GUI to determine any weaknesses in the design, such as missing vias. Use SPT debugging by selecting transistor pins in map view, clicking the middle mouse button, and selecting T**race SPT**, as shown in Figure B2-14.

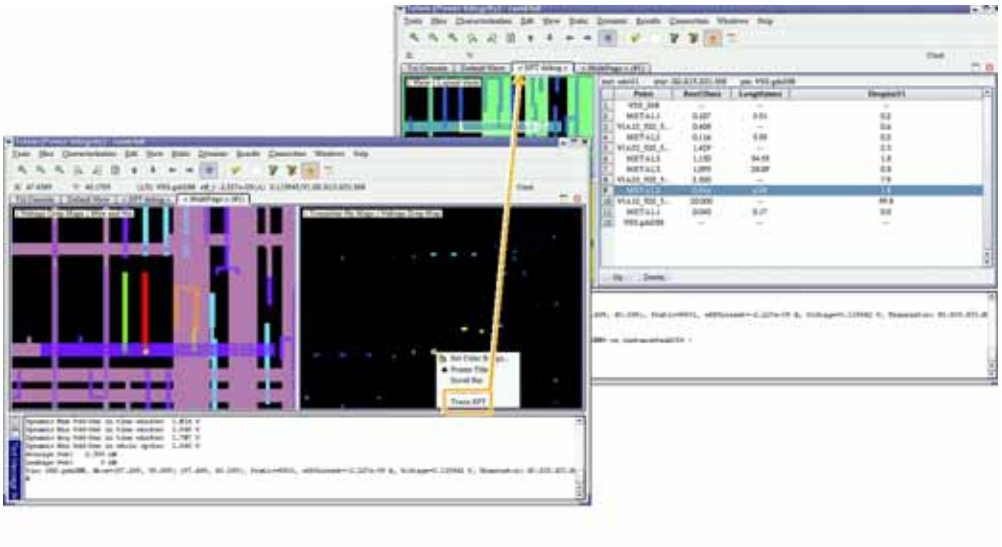


Figure B2-14 SPT debugging from any pin-based colormap

Net selection

You can exclude unselected nets, or include all nets in the design, directly from layout view window when debugging certain EM violations of selected geometry. Select the geometry and then click the middle mouse button to bring up a popup menu, then use the **Display Nets** option to choose either **Exclude unselected nets** or **Include all nets**, as shown in Figure B2-15. You can also use the bind keys **Ctrl** + **N** and **Shift** + **N** to achieve the same results.

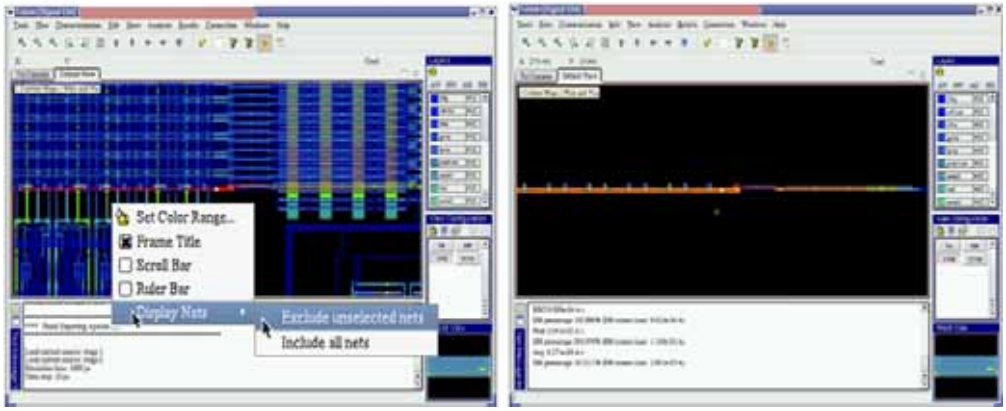


Figure B2-15 Net selection

TCL Command Line

To use the TCL command line, click on the **Tcl Console** tab to the left above the Layout View window.

Text Message and Log Window

The text message and log window is below the Layout View window and displays all outputs from Totem.

Exporting Marker Coordinates from “List of ...” Dialogs

You can export marker coordinates of selected bookmark entries in a “List of ...” dialog to a Tcl command file, as shown in Figure B2-16. Then the marker Tcl command file later can be sourced in the Totem Tcl console to highlight the exported markers in GUI display. The procedure is as follows:

1. In the dialog box, click on the line number column to bookmark the entries to be exported.
2. Once all entries are bookmarked, click on the “TCL” icon in the menu bar. This pops up the export file dialog to allow you to define the export Tcl command file name.
3. Click the “Save” button to generate the marker coordinate Tcl command file.
4. Later source this marker Tcl command file in the Totem Tcl console to highlight the bookmarked positions.

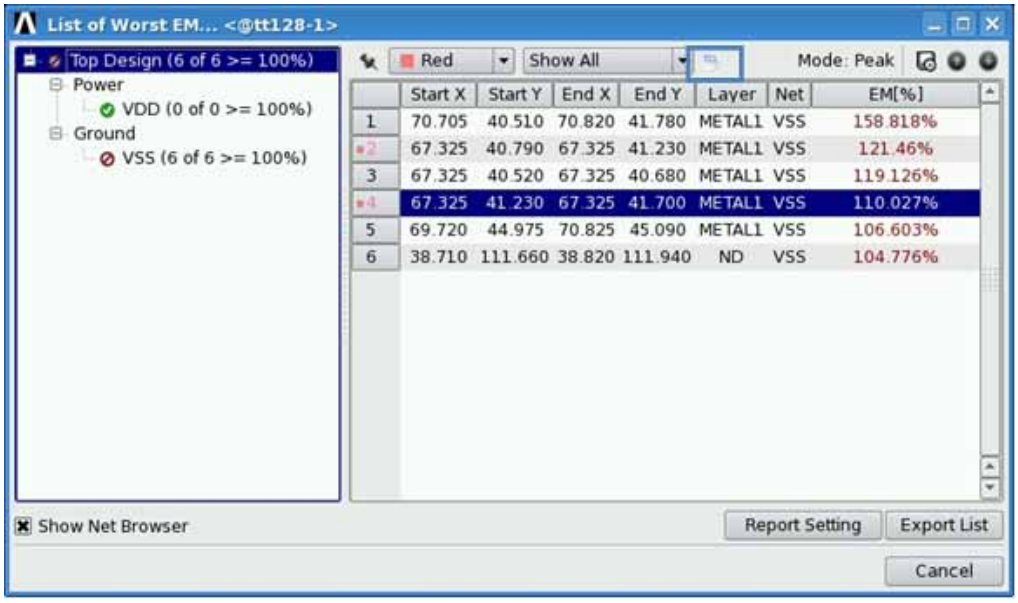


Figure B2-16 Exporting a marker coordinates file from a list dialog

Other GUI Support Features

Middle Mouse Button/wheel Functions

Pressing the middle mouse button/wheel with the cursor within the design brings up a menu with the following functions available:

- **Set Color Range** - see “Set Color Range” (left Configuration icon).
- **Frame Title** - displays View type at top of frame (default)
- **Scroll Bar** - displays the scroll bar
- **Ruler Bar** - overlays a 10 um x,y grid system on the design view
- **Display Nets -** choose “Exclude unselected nets” or “Include all nets”.

Bindkey Support (Virtuoso^{tm}-style behavior)

Totem bindkey operation matches Virtuoso bindkey behavior. See section "Bindkey Functions", page B2-678, for a summary and information on reconfiguring bindkeys.

Bindkey behavior relative to the original Totem GUI are:

- Right mouse button zoom
  - Zoom IN - depressing the right mouse button and dragging from *left to right* creates a rectangular area that is zoomed in.
  - Zoom OUT - depressing the right mouse button and dragging from *right to left* creates a rectangular area that is zoomed out.
- Ruler control enhancement
  - Press preassigned bindkey **K** to enable ruler drawing.
  - Press left mouse button to start the ruler (no need to keep button depressed).
  - Move mouse in 45-degree angle to do 45-degree drawing.
  - Draw multiple connected straight-line segments by left clicking the mouse to end one segment and start a new one. This allows total length measurement of multiple segments.
  - End ruler drawing by double clicking the left mouse button, or single clicking the right mouse button.
  - While in ruler drawing, normal Zoom operation is available using the right mouse button, to improve ruler resolution or scope.
  - Use bindkey **Shift**+**K** to clear all ruler drawing.

User-preferred GUI settings

You can import your preferred default GUI settings by placing the Totem resource file *~/.totemrc* in your home directory, and then including the GUI configuration file in the Totem resource file. You can create the GUI configuration file by exporting it from an existing Totem GUI session using the ‘export guiconf’ TCL command. Then include the ‘import guiconf <gui_configuration_file>’ command in the *~/.totemrc* file, and place the *~/.totemrc* file in your home directory.

Signal EM GUI Menus

The Totem GUI menu functions for signal EM analysis are summarized in the following sections. Totem Integrates MMX, Signal EM, and ESD functions in the Totem platform, providing dynamic switching using the **Tools** menu to a product-specific set of GUI menus, as shown in Figure B2-17.

Tools -> Power Integrity

Selects power integrity setup and analysis mode. See GUI and menu descriptions in Appendix B1, "Power Integrity GUI and Menus".

Tools -> Signal EM

Selects signal EM setup and analysis mode.

Tools -> ESD

Selects Electrostatic Discharge setup and analysis mode. See GUI and menu descriptions in Appendix B3, "ESD GUI and Menus".

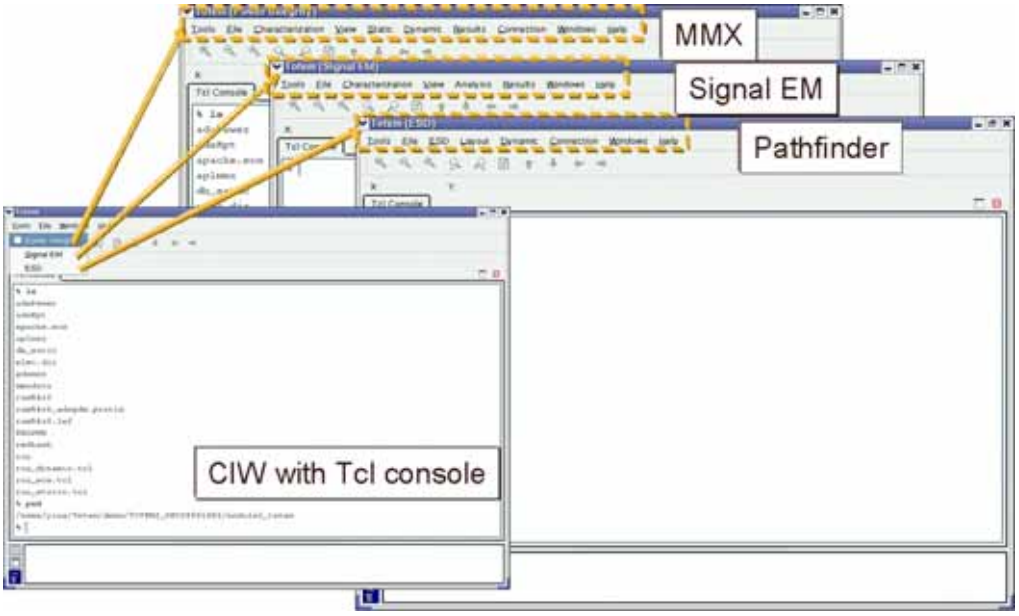


Figure B2-17 Product-specific menus

Files -> Import GDS...

Displays an 'Import GDS' dialog box to specify the GDS file to be imported, as well as the Map File and the Top Cell. The dialog box also has fields for specifying Dictionary, Technology File and Display File.

Importing GDS View through Tcl interface

Totem also allows you to import the GDS file and display it in a GDS View using a TCL command, so you can set up batch TCL commands to import GDS and load the GDS View without using the GUI interface. The procedure to do this is as follows.

From the Totem GUI command line:

```
import_gds -gds <gds_file> [ -map <map_file> -top <cell>|
     -tf <Virtuoso_tech_file> -drf <Virtuoso_display_file>
    -vmap <Virtuoso_layer_map_file> ]
```

or

```
import_gds -cfg <configuration_file>
```

where the configuration file contains the TCL arguments

-gds <gds_file>

-tf <Virtuoso_tech_file>

-drf <Virtuoso_display_file>

…

open_gds -lib <lib_name> -cell <cell_name> -view <view_name>

From a Tcl batch file:

```
gui {
import_gds -gds <gds_file> [ -map <map_file> -top <cell> |
    -tf <Virtuoso_tech_file> -drf <Virtuoso_display_file>
    -vmap <Virtuoso_layer_map_file>  ]
open_gds -lib <lib_name> -cell <cell_name> -view <view_name>
}
```

For example:

```
gui  {
    import_gds -gds AAA.gds -map AAA.gds.map
    -tf AAA.tf -drf AAA.drf
  open_gds -lib AAA -cell top -view layout
}
```

GDS View integration with Layout Cross-Probing

Allows checking signal routing and cross-probing with GDS view for easy debugging, as shown in Figure B2-18.

You can import GDS after the ‘setup design’ step with one of the following methods:

1. a. Select the menu command **Files -> Import GDS.**

   or

   b. Click on the **View GDS** button in the **View Configuration** panel (second button from the right).

2. A dialog box is displayed. Fill in the GDS filename, the GDS layer map filename, and the top cell name.

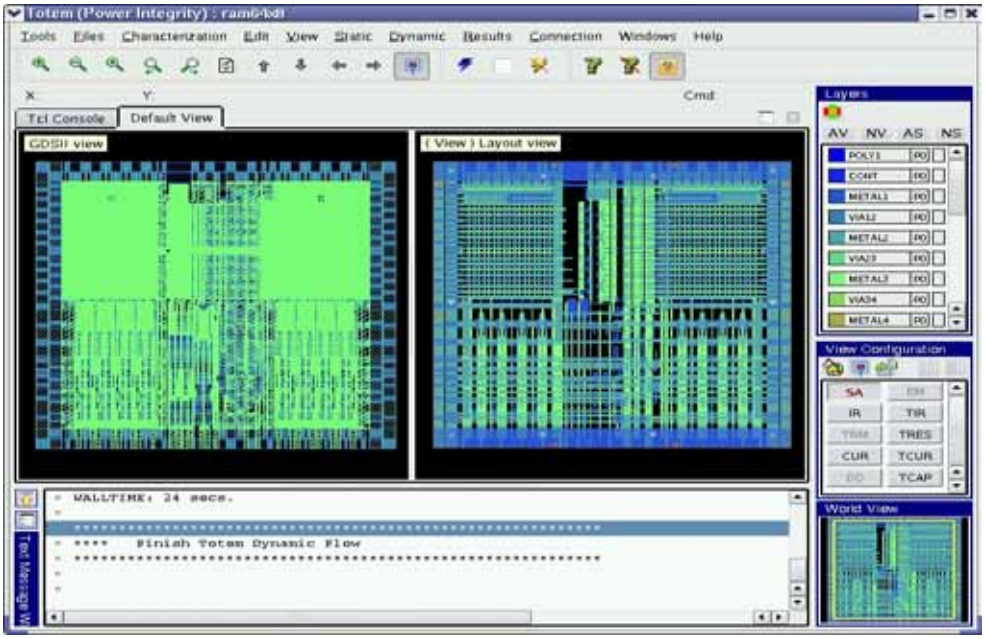


Figure B2-18 GDS and layout view cross-probing

The GDS viewer converts GDSII to GDS view with a GDS layer selection capability, as shown in Figure B2-19.

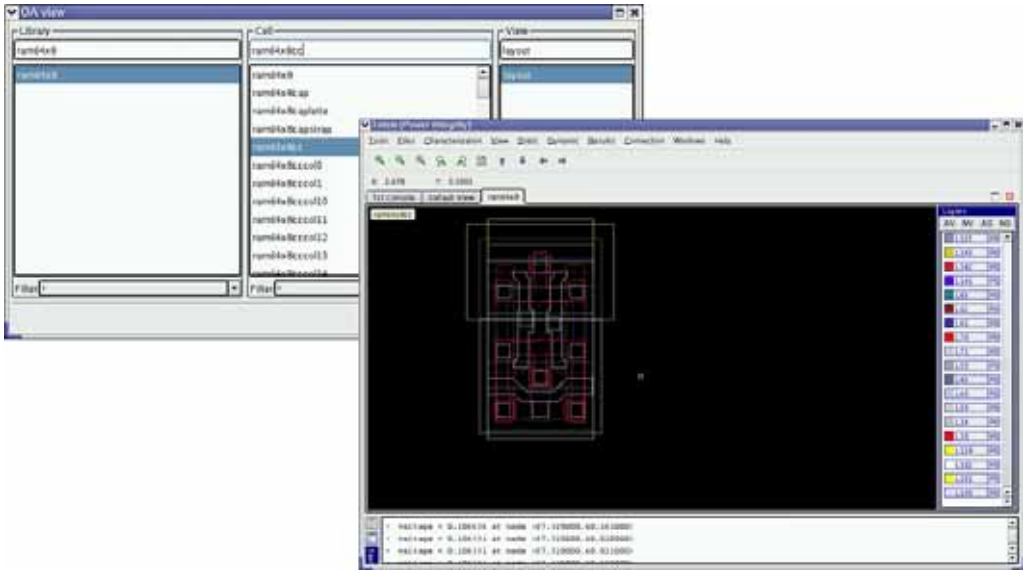


Figure B2-19 GDS view

To import a GDS file, you can select the file from the dialog box displayed after using the menu command **Files -> Import GDS**. Then you can use the command **View ->GDSII View** to bring up a dialog to select a GDS hierarchy to load in the GUI window. You can change the default GDS view setting by bringing up the ‘Preference’ dialog (F10 key).

Files -> Import Netlist ...

Displays a dialog from which you can select a netlist file to be imported.

The integration of Netlist Hierarchy View with Cross-Probing provides easy access to netlist information for debugging, and enables cross-probing with Layout View and other colormaps, as shown in Figure B2-20. When you select an entry in the Netlist Hierarchy view, the corresponding transistor pin is highlighted in the colormap. You can also select a transistor pin in the transistor pin map or SA view with the ‘pinInst’ check box enabled. To enable the pinInst check box, refer to Figure B2-3 on using the Layer Selection Window.

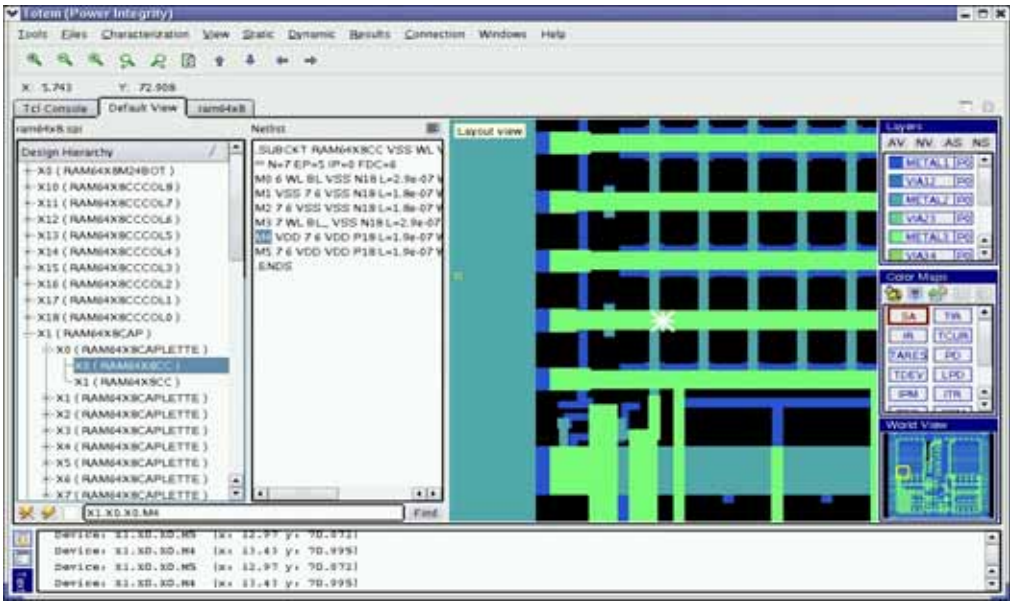


Figure B2-20 Netlist view and cross-probing

To import a netlist and display the netlist hierarchy view, use the command **Files -> Import Netlist**, and fill in the netlist filename, or you can click on **View Netlist** in the 'View Configuration' panel (first button from the right), and specify the netlist filename, which also brings up the netlist view. You can import a CDL format netlist in Netlist Hierarchy View to cross-probe with the Totem database and the Virtuoso Schematic Composer, or the Layout editor.

Files -> Import Design Data ...

Displays an 'Import Design Data' dialog box to specify the design data files to be imported. The dialog box has fields for specifying GSR, LEF, LIB, and Apache Tech files for the design. Package definition files PCELL, PLOC, and PAD can also be specified for import. If the correct path to the GSR or LEF files are specified, and the other files have the same path, click the button 'Use Same Prefix' to insert the same path in all file specification boxes. Selecting 'OK' then loads all specified design files.

The menu is grayed out after design setup.

Files -> Import Database ...

Displays an 'Import ADB' dialog box to specify the database file to be imported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as 'Create Dir', 'Delete File', and 'Rename File'.

The menu is grayed out after a database is imported.

Files -> Import ECO ...

Displays an 'Import ECO' dialog box to specify the ECO file to be imported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as 'Create Dir', 'Delete File', and 'Rename File'.

Files -> Export Flatten GDS ...

Displays a dialog to allow exporting a flattened GDS layout to a specified GDS output file.

Files -> Export Database ...

Displays an ‘Export ADB’ dialog box to specify the database file to be exported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Export ECO ...

Displays an ‘Export ECO’ dialog box to specify the ECO file to be exported. The dialog also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Import Third Party Technology Data ...

This command allows easy import of third party technology data. For example, in addition to Virtuoso Tech file and display file support, you also can import Virtuoso layer map files. With these three files from the Virtuoso setup environment there is no ambiguity when trying to map the Totem Layer Select Window configuration with Virtuoso to display the same color and drawing pattern. Figure B2-21 below shows the setup dialog.

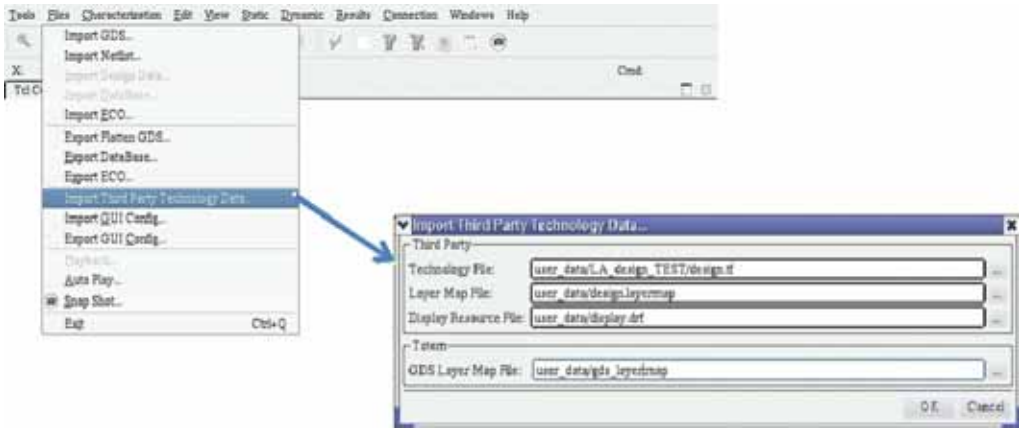


Figure B2-21 Displaying Virtuoso layermap file

Files -> Import GUI Config ...

Displays an ‘Import Configuration’ dialog box to specify the configuration file to be imported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Export GUI Config ...

Displays an ‘Export Configuration’ dialog box to specify the configuration file to be exported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Playback ...

Displays a ‘Load Playback File’ dialog to specify the TCL command file of a previous Totem session to be loaded. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

This menu is grayed out after design setup.

Files -> Auto Play ...

Displays a dialog that allows you to specify a a Totem run file to play rerun a previous session in the GUI. Allows adjustment of the timing of GUI operations. Replay script files are located in the *adsRpt/Replay* directory.

Files -> Snap Shot ...

Takes a screen shot of the full Totem window and saves it in a GIF file in the selected filename in the dialog box that is displayed. GIF screen shots can be displayed using the 'xv' utility in the Linux OS.

Files -> Exit

Exits Totem session and saves all associated files.

Edit -> Undo

Removes the previous edit step.

The menu is grayed out until editing is performed.

Edit -> Redo

Removes the previous undo step.

The menu is grayed out until an undo step is performed.

Edit -> Add Strap

Displays an 'Add Strap' dialog box to specify new power or ground straps. You can specify the power net ('Available Nets for power', 'Available Nets for ground'), 'Strap input method' ('Add strap by text input'), 'Horizontal' or 'Vertical' orientation, 'Strap start' x,y location, 'Strap end' x,y location, and 'Strap width'. You can add multiple straps by x,y range and pitch, and specify the appropriate metal layers and associated vias.

Edit -> Edit Strap

Displays an 'Edit Strap' dialog box to specify modifications to existing power or ground straps. After clicking on the strap to be modified, information about it appears in the dialog box: 'Net', 'Wire' (layer), 'Length', 'Width', 'Strap left lower x' (x location at lower left end of strap), 'Strap left lower y' (y location at lower left end of strap) and 'Resistance'. When the correct information on the strap to be edited is displayed, the strap can be modified using the 'Delete' or 'Change' buttons.

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **"**Editing Mode**"** to **"**Mode2**"**

To stretch a rectangle (supports GUI operator "8 control points stretch" component):

a. Select a rectangle and choose **Edit -> Edit Strap**, or use bindkey 's'.

b. The editing coordinate resolution is set based on "X Snap Spacing" and "Y Snap Spacing" settings in the "Preference" dialog.

c. Once the change is done, click on "Change" button in "Edit Strap" dialog to commit the change.

d. Use the "ESC" key to escape the edit command.

e. Bring back previous edit command mode by using right-mouse-button.

f. Once the change is done, click on the "**Change**" button in "Edit Strap" dialog to make the change.

Edit -> Add Via

Displays an ‘Add Via’ dialog box to specify new vias to be added. After clicking on the location for a new via, information about the layers at that location appears in the ‘Select Metal Layers’ panel. After selecting the appropriate layers for the via, the available vias for the selected two layers appear in the ‘Available Via Type Lists’ panel. Select the appropriate via type and then execute the addition.

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To add a via:

a. Select the via layer to edit in the LSW.

b. Use menu command **Edit -> Add Via**. A list of available via models on the selected via layer is shown in the **Add Via** dialog.

c. Select one of the via models; the dashed line bounding box of the selected via model is displayed in GUI display.

d. Use the left mouse button to place the selected via model in the GUI layout window to drop the via.

e. You can select the **Fill** option in the **Add Via** dialog to automatically fill the entire top metal and bottom metal overlapping area by selected a via model.

Edit -> Delete Via

Displays a ‘Delete Via’ dialog box to specify existing vias to be removed from the design. Displaying on vias on particular layers using the ‘View Layers’ button will help select the vias desired. After selecting the via to be deleted, its Net and Via Name are identified in the dialog box. If this is the desired via to be deleted, click on the **Delete** button to execute the command.

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To delete a via:

a. Select the via layer to edit in the LSW (Layer Selection Window).

b. Use the menu command **Edit -> Delete Via**.

c. Select the via geometry in the GUI layout window using the left mouse button. The selected via geometry is displayed in the **Delete Via** dialog box.

d. Click the **Delete** button to delete the selected via.

Edit -> Add Rectangle

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To draw a new rectangle:

a. Select a metal layer in LSW window to activate a working layer.

b. Choose one net from the “Net” selection menu of the “Add Rectangle” dialog.

c. Click the left-mouse button twice to specify a corner of the rectangle.

d. Press and hold the left-mouse button, tracing a box, then release the button.

e. Display and hide the “Add Rectangle” dialog using bind key “F3”.

f. Use the Delete (Backspace) key to reset a clicked point.

g. Use the “ESC” key to exit the edit command.

h. The editing coordinate resolution is set using the “X Snap Spacing” and “Y Snap Spacing” settings in the “Preference” dialog.

i. Bring back the default edit command mode using the right mouse button.

j. “Add Rectangle” does not support via layers. If a via layer is chosen for editing, an ERROR pop-up dialog is displayed.

This menu is grayed out in Editing Mode1.

Edit -> Delete Objects

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To delete a rectangle (both vias and wires can be deleted):

a. Select rectangle bounding box to be deleted.

b. Press Delete.

c. Use the “ESC” key to exit the edit command.

d. Bring back the previous edit command mode by using right-mouse-button.

This menu is grayed out in Editing Mode1.

Edit -> Create Ruler ->

After turning on the ruler function, clicking the left mouse button on an x,y location on the design and dragging it to a second point on the design creates an x,y axis with lengths in microns. For easily measuring long distances on a design with precision, press the SHIFT key and click the left mouse button to define the start point and the end point of the measurement. The location of the intersection of the x,y axis and the end point are displayed on screen.

Edit -> Clear All Rulers

All drawn rulers remain displayed until the ‘Clear All Rulers’ button is selected.

View -> GDSII View *

After specifying GDS setup in the GSR file, this command brings up a **GDSII View** dialog**.** In the dialog select the Library in the **Library** section, and a list of cells is displayed in the **Cell** column. After selecting a cell to display and the **Show Library Information** box, and Open to display the GDSII view dialog.

If you specify the top cell when loading the GDS View setup dialog, the GDS View library, cell and view selection dialog setup step is automatically skipped and GDS View is displayed directly.

Note that The GDS layer map file input is optional, so you do not need to prepare a GDS layer map file when using Totem as a standalone GDS Viewer.

The viewer also provides LSW cross-probing between Apache Tech layers and GDS layers if you import a GDS layer map file during setup.

The GDS viewer converts GDSII to GDS view with GDS layer selection capability, as shown in Figure B2-22.

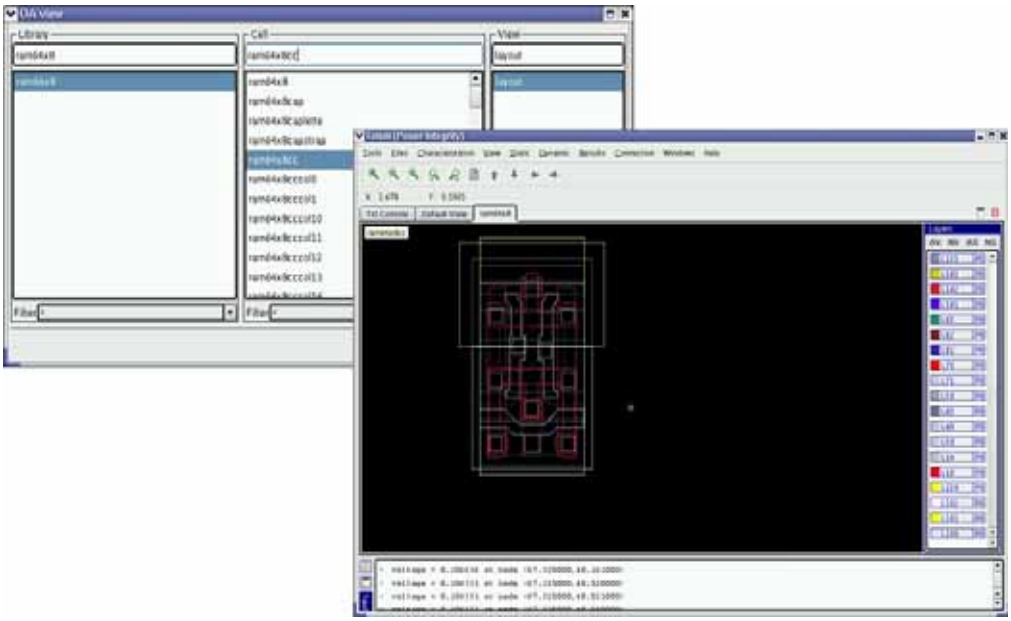


Figure B2-22 GDS view

You can change the default GDS view setting by bringing up the ‘Preference’ dialog (F10 key).

The viewer also provides LSW cross-probing between Apache Tech layers and GDS layers if you import a GDS layer map file during setup. See examples of these special GDS Viewer functions in Figure B2-23 through Figure B2-29.

GDS hierarchy browser

The GDS Viewer has a GDS Hierarchy Browser to allow you to cross-check GDS hierarchies. In the GDS View, click the middle mouse button to invoke the GDS Hierarchy Browser, as shown in Figure B2-23.

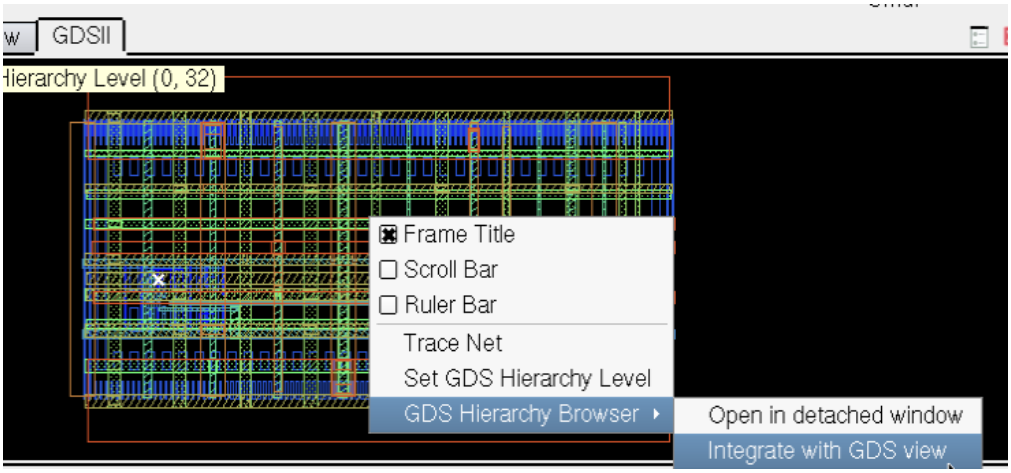


Figure B2-23 Bringing up the hierarchy viewer

You can choose to either integrate the GDS hierarchy browser with the GDS view or view it as a standalone dialog.

The standalone hierarchy display is shown in Figure B2-24.

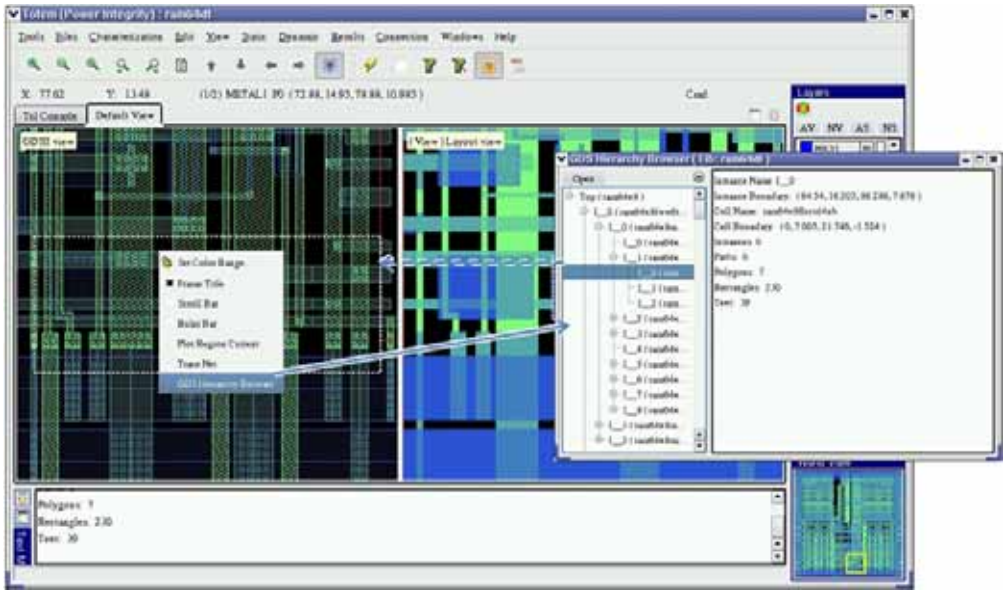


Figure B2-24 GDS hierarchy browser

Figure B2-25 shows the integrated hierarchy browser.

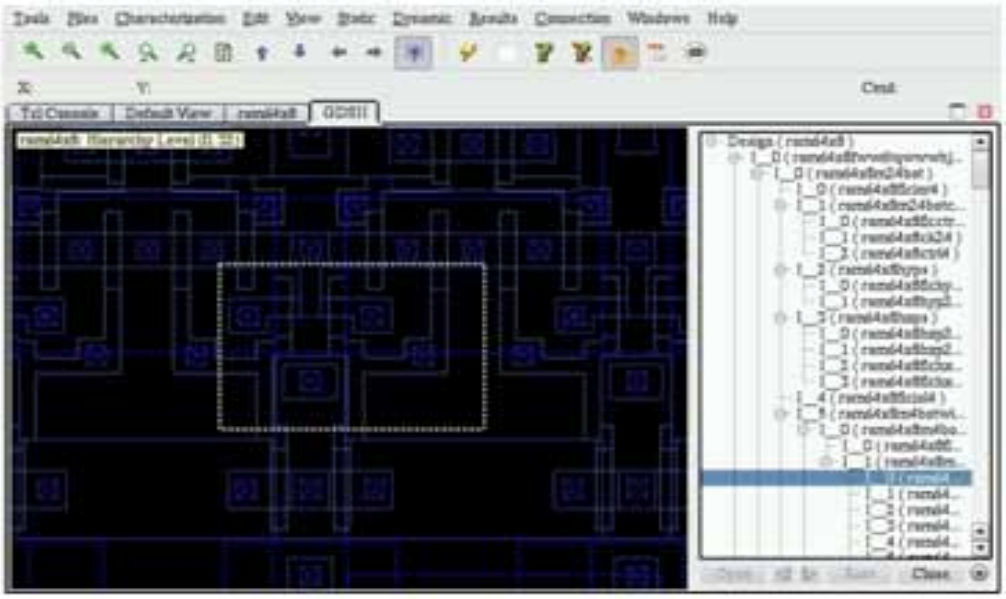


Figure B2-25 Integrated GDS view and hierarchy browser

GDS hierarchy boundary display

To make GDS debugging easier, the Totem GUI allows you to highlight GDS instance hierarchy boundaries. You can then use the bindkey combination **Shift + F** to open the full flattened GDS hierarchy in the GDS view. Use **Shift + V** keys to go down one hierarchy and **Ctrl + V** keys to go up one hierarchy. A display is shown in Figure B2-26 below

:

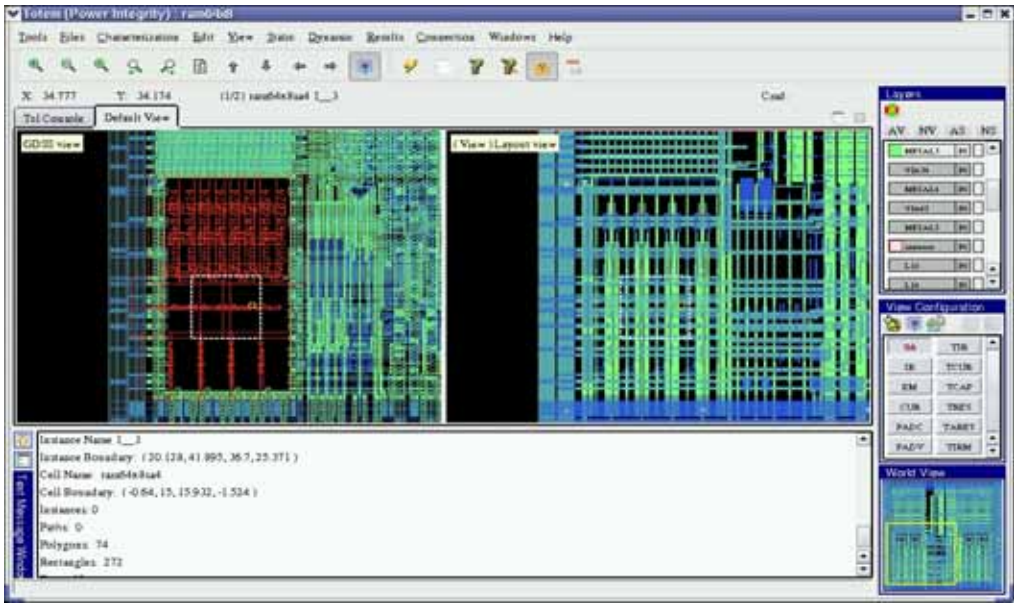


Figure B2-26 GDS hierarchy boundary display

GDS View-based P2P Resistance Checking

You can directly perform point-to-point resistance checking from GDS View, without going through a detailed GSR and TCL command setup, using the following steps:

1. Without doing design setup, go to **Files -> Import GDS,** to load the GDS file and GDS layer map file.
2. Go to **View -> GDSII View** to open GDS View.
3. After GDS View is loaded in the Totem GUI, use the middle mouse button to bring up the dialog box and select the **GDS P2P Resistance Calculation** entry.
4. The **GDS P2P Resistance Calculation** dialog is shown on the screen to allow you to set up a GDS file, GDS layer map file, Apache Technology file, Net name, and **From**, and **To** resistance end points, as shown in Figure B2-27. You can also define a resistance threshold constraint to be checked.
5. You can either manually define the **From** and **To** points in the dialog, or use the mouse to click the geometry objects directly in the GDS View to define the points.
6. Then click the **Apply** button to start the calculation.
7. Totem automatically sets up the GDS configuration file, GSR, and Tcl commands based on your input in the previous steps, and submits a child process to perform GDS2DB tracing, setup design, extraction and resistance calculation.

   Similar to the Save/Reload flow, the same license is shared between child process (setup design to perform resistance calculation) and parent process (GDS View). And all the child process setup is done in a separate sub-directory, under *<working_directory>/p2p<id>*, so you may check all tcl, GSR, configuration setup in the directory to debug.
8. After the resistance calculation is done, you can see the minimum resistance path GUI display between the two selected points. If the resistance constraint is defined during setup, both the effective resistance results and constraint are displayed at

the header of the SPT window. If there is a resistance constraint violation, you can continue to use the child process GUI to debug the existing resistance path to identify if there are any design issues.

9. If you want to perform another P2P resistance check on the same net, continue to use the child process GUI to select the next “From/To” points using the middle mouse button pop-up menu, and repeat the process using steps 4 through 8. This can save time in avoiding the GDS trace design setup and extraction.
10. If you are finished with the results and want to quit the child process GUI, Totem automatically returns to parent process GDS View setup.
11. You can also switch to the **Result** page of the **GDS P2P Resistance Calculation** dialog to check the previous P2P resistance check results, or re-submit the previous setup.

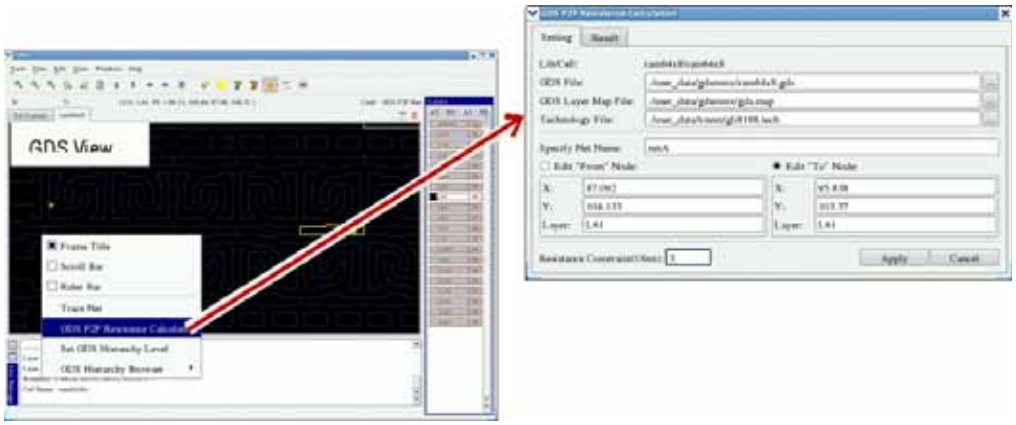


Figure B2-27 P2P Resistance calculation

GDS viewer text search and net tracing

To find the GDS “start tracing” point and debug GDS tracing issues, Totem has text search and net trace capabilities. You can click **Trace Net** on the middle mouse pop-up menu to invoke the Trace Net dialog to search text, or set up GDS layer mapping and trace rules. An example is shown in Figure B2-28 below.

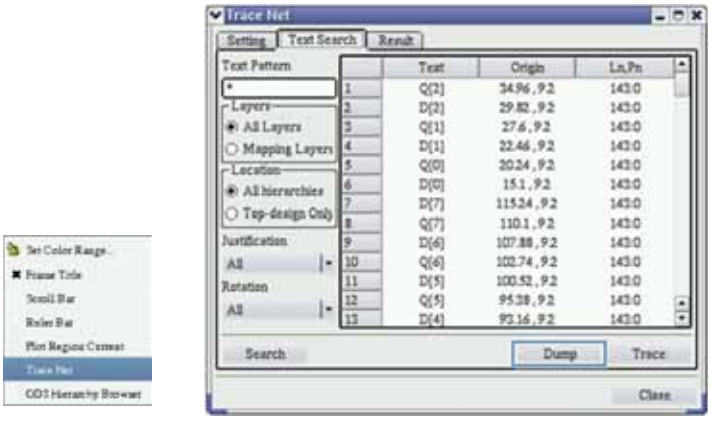


Figure B2-28 Text search and net tracing

You can select text entries in the text search table and click the **Dump** button to stream out GDS “start tracing” points in an output file. Then you can copy and paste the contents of the file into the gdsmmx/gds2db configuration file in the VDD_NETS and GND_NETS keywords, to define the selected “start tracing” points.

You can also click on the **Trace** button to start GDS tracing from selected text entries in the table, or click on any geometry in the GDS view to start tracing. The trace result is then displayed on the Result page of the Trace Net dialog, including Visible control, Color control, Delete Control, Trace Progress display and trace stop functions, as shown in Figure B2-29.



Figure B2-29 Net tracing results display

View -> OA Database View

You can use the GUI to view an OASIS library layout view, the same as the GDS view by selecting **View -> OA Database View**, clicking on a lib DEF OA library header file from the list in the dialog, and then selecting one cell to check its layout view, as shown in Figure B2-30.

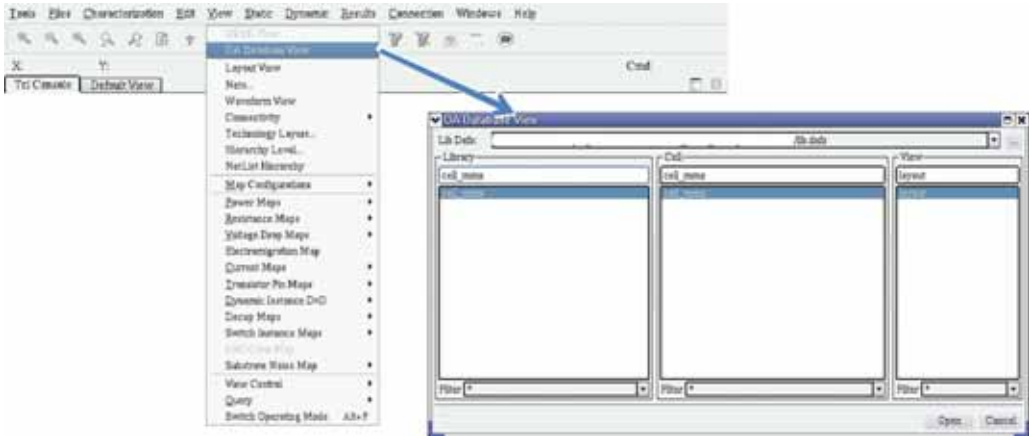


Figure B2-30 OA library layout view

View -> Layout View

Performs the same “show layout view” operation as the **SA** button in the ‘Configuration buttons’ panel. See description above. Does not change the area of the design viewed.

View -> Net Display Map

Displays all nets in different colors.

View -> Nets ...

Displays a ‘Nets’ dialog box with lists of VDD and VSS nets. The dialog allows you to select and highlight particular nets by name or by type: VDD, VSS, Clock, or Signal (if extracted). Nets of a selected type also can be chosen by name to assist in finding a desired net in a very long list. Also displays instances and pins if they are selected in the ‘Layers’ dialog.

In addition to viewing regular nets in the **View -> Nets** dialog, if the design includes metal filler, you can select the check box **Exclude Metal filler** to exclude them, or go to any color view and use the middle mouse pop-up menu to select the **Display Nets -> Exclude metal fill net** check box.

You can also set generic name-based net filtering in the **View -> Nets** dialog to switch “Enable Net By Name” or “Exclude Net By Name” functions to include or exclude net names. Wildcard names are supported.

The check box “GSR only” allow you to only display nets defined in the GSR. Also, internal nets are highlighted in dark blue within the “Nets” selection dialog box to help distinguish internal nets.

View -> Waveform View

In standalone mode displays the waveform viewer from CIW dialog and imports a waveform file without loading the database. The command supports multi-page, multi-frame displays to provide a more user-friendly debug environment for cross-probing. The waveform viewer supports *.wdb, taO, and *.tr0 files in standalone mode to display and perform waveform operations, as well as case insensitive searching in the signal browser. Copy and paste functionality in the signal browser of the waveform viewer is also supported.

Figure B2-31 below shows the standalone waveform viewer.

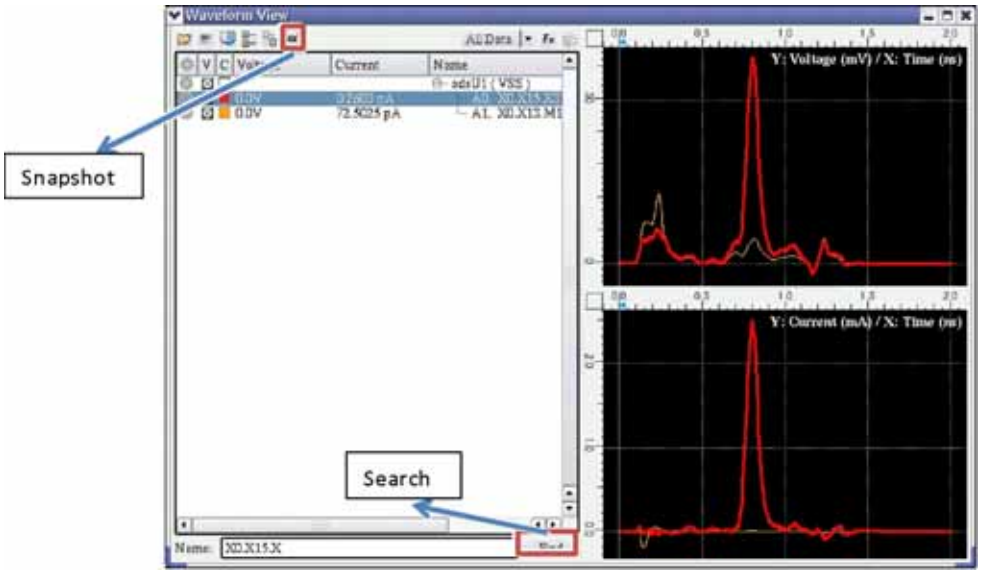


Figure B2-31 Standalone waveform viewer

To help debug a hot spot or other design issue more easily, the integrated waveform viewer dialog allows you to cross-probe transistor pin voltage/current waveforms with other DvD, IR, and EM maps, and movie results, as shown in Figure B2-32.

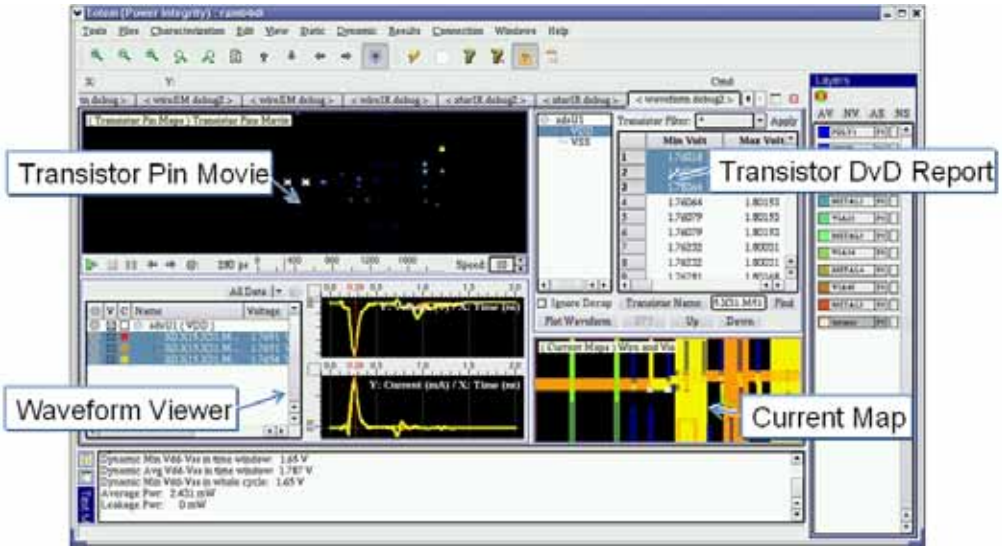


Figure B2-32 Integrated waveform viewer

You can plot two types of waveform views, transistor voltage and current waveforms, and also regional current waveforms, and they can be executed in different ways.

The functions of the waveform viewer are described in the following section.

"Add Page" function

Click on the green "**+**" icon on the left side of the Waveform Viewer dialog to add a new page.

"Delete Page" function

Click on the red "**x**" icon on the left side of the Waveform Viewer dialog to delete the current active page. The default page cannot be deleted.

"Add Frame" function

Except for the default page, you can add new frames on a non-default page. The blue frame icon on the left side of Waveform Viewer dialog allows adding a new frame to the current active page.

"Delete Frame" function

In the frame you want to delete, click the middle mouse button to bring up a pop-up menu and select "**Delete Current Frame**".

Adding a signal in a frame

You can drag a signal from the Signal Browser on the left side of the Waveform Viewer and place it in a selected frame. Newly-added signals are displayed in the header of the frame. If there is more than one signal in the frame, only the symbol of the signal (A0, A1, C0, C1...) is shown in the header. Totem only allows one type of signal (voltage or current) inside a frame.

To select the voltage waveform of given transistor from signal browser, place the cursor at the top of the Voltage column of the Signal Browser to drag. Similarly, you can select a current waveform by putting the cursor at the top of the Current column of the Signal Browser to drag.

To sync the active view page with the signal browser

Click on the "**[View All | View Page]**" button at the top of Signal Browser. You can switch these two modes to see all, or see just the active page signals, in the Signal Browser.

To show, hide, or delete signals

You can select a visible icon in the Signal Browser to show, hide or delete.

Case sensitive/insensitive search

You can toggle the "**[Insensitive Name | Sensitive Name]**" button to control the case sensitivity of the transistor name search function.

To copy the signal name

You can click on the signal in the Signal Browser and use the right mouse button to bring up the pop-up menu and select "**Copy Signal Name**".

Calculation Function

You can use the calculation function in the waveform viewer to perform statistical measurements, such as calculation of average, RMS (root mean square), and peak current within a selected time range, as shown in Figure B2-33. Or you can use the integral function to calculate total charge, as well as perform linear or formula-based waveform operations.

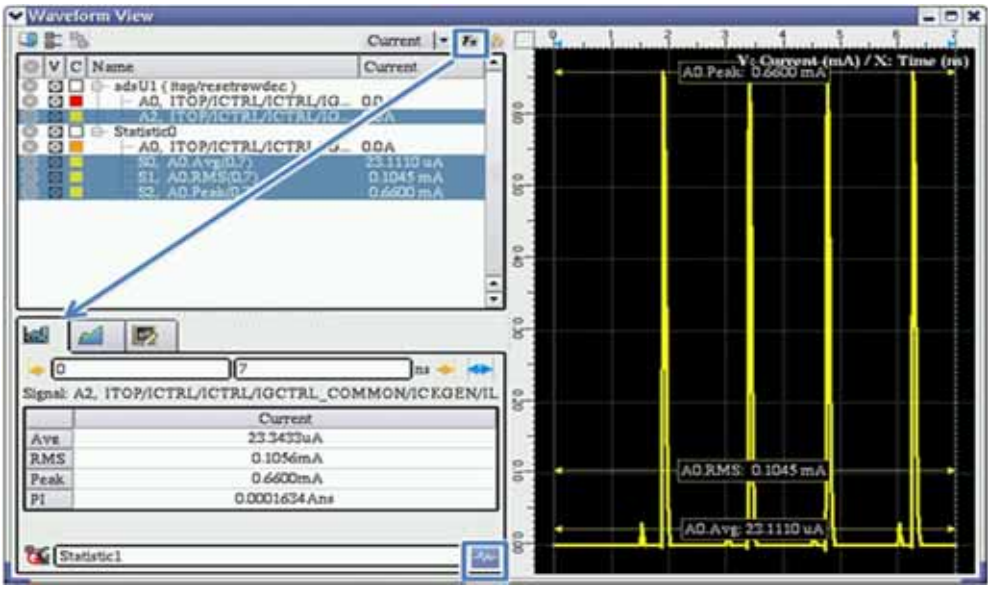


Figure B2-33 Waveform viewer calculation function

To invoke the waveform viewer calculation function, click on the **Fx** button There are three additional sub-pages displayed in the Waveform Viewer, as follows:

Statistics

- To calculate average, RMS, peak, or an integral of the selected waveform within a specified time window.
- To change the start time of the time window, use the left mouse button to click on the time axis, or directly input the value beside the left yellow arrow.
- To change the end time of the time window, use the right mouse button to click on the time axis, or directly input the value beside the right yellow arrow.

You also can click on the **Vis** column to control which measurement functions are enabled. And you can click on waveform **Apply** button in the lower right corner of the page, which displays the measurement results in the waveform view.

Linear

- To apply linear equations to selected signals.
- You can click the **Apply** button in the lower right corner of the page to apply the linear equation and the resulting waveform is displayed in the waveform view.

Formula

- To apply a user-defined formula to selected signals.
- You can click on the **Calculator** button in the top right corner of the page to bring up calculator view, or directly type in the formula in the center input box.
- You can click on the **Apply** button in the lower right corner of the page and resulting waveform from the formula calculation is displayed in the waveform view.

FFT Function

To use the FFT function (see Figure B2-34):

1. Select a signal in the signal browser.
2. Select the **FFT** icon on the top right of the signal browser.
3. Input the start time and end time in the pop-up dialog to define window range for FFT transformation.
4. Adjust the sampling rate and maximum frequency range.
5. Click **OK** to start FFT transformation.
6. Once FFT transformation is done, a new waveform viewer window pops up to display the frequency domain waveform.

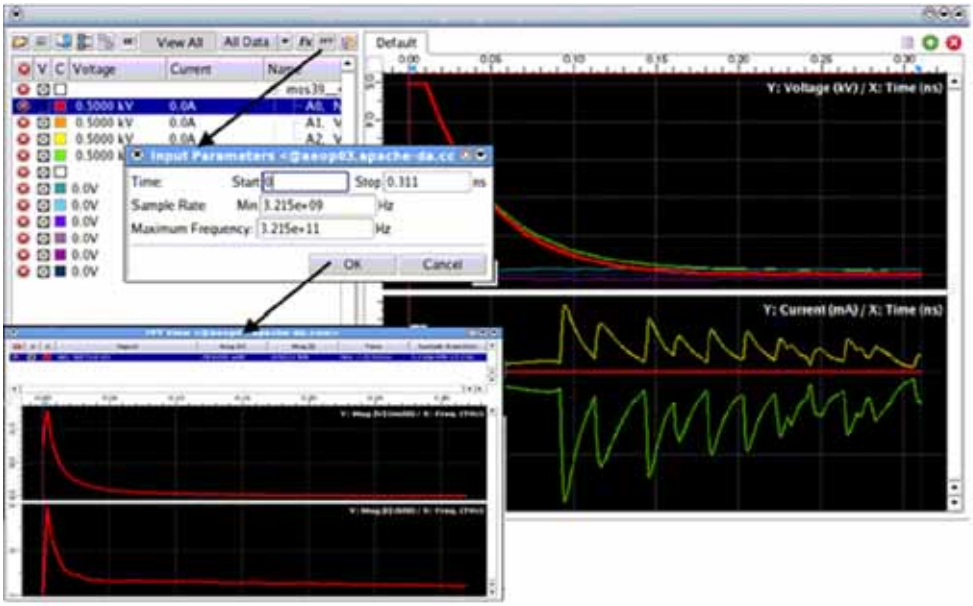


Figure B2-34 FFT function

Examining Waveform Views

Zoom Full. Use the middle mouse button to pop up a menu inside the Waveform Viewer and select Zoom Full, which zooms out to the maximum. You can also use the '**F'** bindkey in the waveform viewer to zoom to Full if the waveform view is not in multi-frame page mode.

Zoom In. Use the right mouse button to drag from left to right, creating a rectangular box defining a zoom-in area, the same as in the regular layout window.

Waveform Zoom-In X-dimension only. Use the right mouse button to draw a horizontal line from left to right indicating the desired zoom range.

Waveform Zoom-In Y-dimension only. Use the right mouse button to draw a vertical line from top down indicating the desired zoom range.

Zoom Out. Use the right mouse button to drag from right to left, creating a rectangular box defining a zoom-out view (zooming out by the ratio of the view size to the drawn rectangle size), the same as in the regular layout window.

Getting Waveform Details. Use the left mouse button to click on the waveform view at a location of interest. A red vertical line is displayed, along with the voltage and current at that time in the left side waveform signal browser. You can click on the header of the Current and Voltage columns to choose maximum or minimum value sorting modes.

Cross-probing Between Layout View and Results Table. When you select a signal inside the signal browser on the lefthand side of the Waveform Viewer, the layout view zooms to the selected region or transistor pin location, and the waveform is highlighted.

Manipulating Views in the Signal Browser. You can change waveform colors, make the waveform invisible, or delete a signal in the signal browser using the available check boxes in the browser. You can delete all signals by clicking on the header of the delete column (“X”). A popup dialog asks for confirmation of the deletion request.

Set x-axis and y-axis range. Resets the waveform view x-axis and y-axis range to help you zoom in or zoom out. You can use the middle mouse button in the waveform view to bring up a popup menu to check this feature.

Restore range to default. Restores the original default x-axis and y-axis range settings

Change waveform curve width, style, symbol. You can use the right mouse button in the signal browser of the waveform viewer to bring up a dialog to reset width, style, and symbol for selected signals.

View -> Connectivity ->

Highlight Disconnected Instances

Turns on highlighting of disconnected instances in the **SA** view, which is off by default. The highlights can be cleared using the same menu command, the **CLR** menu button, or the TCL command 'select clearall'.

This menu is grayed out for designs that have only a single MMX instance.

Highlight Disconnected Wires and Vias

Turns on highlighting of disconnected wires and vias in the **SA** view, which is off by default. The highlights can be cleared using the same menu command, the **CLR** menu button, or the TCL command 'select clearall'.

This menu is grayed out when there are no identified disconnected wires and vias.

Disconnected Transistor Pins ...

To capture potential design problems in MMX blocks, provides a list of all transistors disconnected from the power and ground grid.

Missing Vias ...

Displays a ‘Report of Missing Via Area’ dialog listing all P/G mesh intersections that represent possible missing via locations, including those on 45-degree geometry, compiled by surveying all grid intersections on all layers and noting the intersections that are not connected by vias. It also lists the voltage difference between upper and lower metal layers at potential via sites. The dialog allows you to step up or down the list and select a missing via location. Clicking on the 'Include GDS cell' button and the check box allows you to include a GDS cell. Clicking on the ‘Go to Location’ button highlights the missing via location for closer inspection. The missing vias list is sorted by voltage gradient---the higher the voltage difference between the metal layers above and below the via, the more impact adding this via would have on improving the voltage drop.

Creates and filters the entries in the missing via report dialog to focus on the most critical violations. You can choose to ignore stack vias, only display hot instances, or even set up a differential voltage threshold across the missing vias to reduce the vias in the list.

You can also sort the missing via report using X-coordinates or Y-coordinates. To do this, you must select the X/Y sorting function in the “Location” column, as shown in Figure B2-35, then use the following steps,:

1. Press the Right-Mouse-Button on “Location (X,Y)” heading.

2. Click
3. the Left-Mouse-Button on “Location Sort By X” or “Location Sort by Y”. Click Apply.

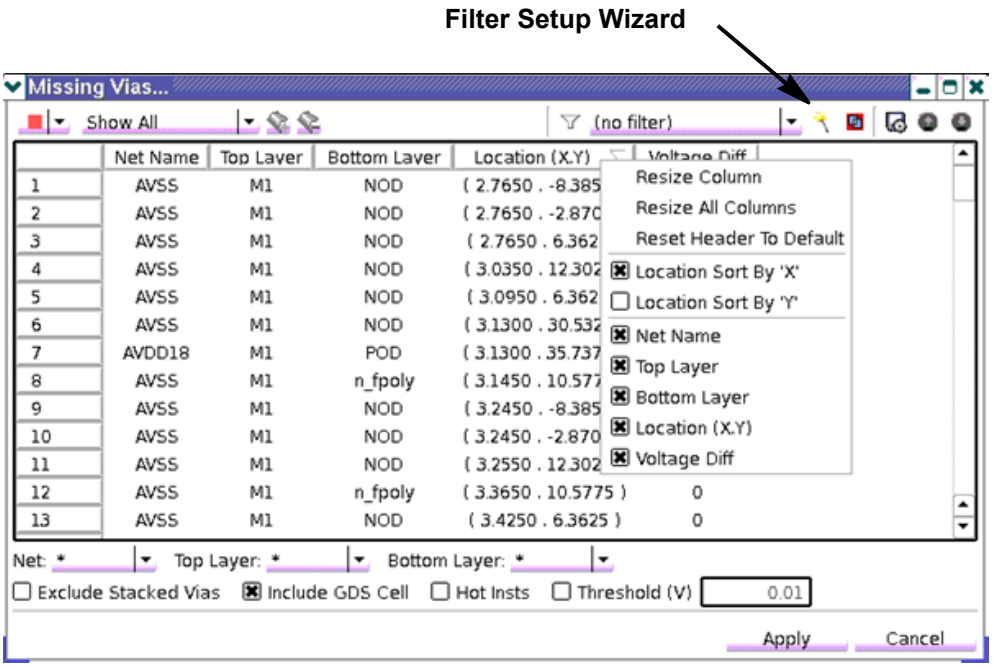


Figure B2-35 Missing Vias dialog

You can also exclude user-defined regions for Missing Via Reports using a file, or by zooming to the region of interest and clicking the “Get Screen Area” option. The GUI zoom-based approach is outlined in the steps below.

1. Click “Filter Setup Wizard” in the Missing Via Report window.
2. Zoom to the region of interest and click “Get Screen Area”, as shown in Figure B2-36.
3. Zoom to the region desired and click the blank square next to the Exclude box, “Get Screen Area”.
4. Click on the “Save Current Setting as New Filter Rule” icon, and select a filter number in the dialog, to save the marked region for future reference, then click “OK”.

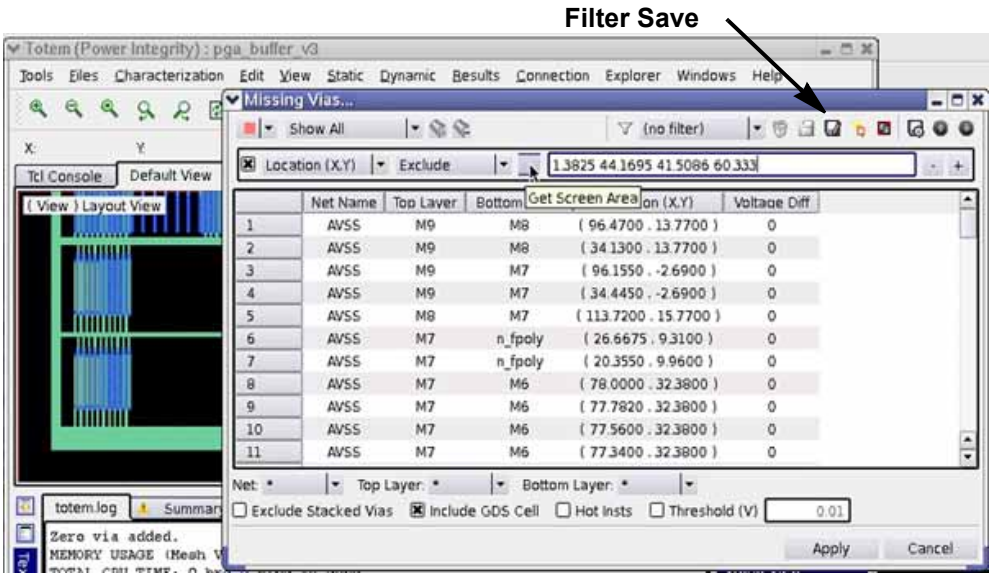


Figure B2-36 Missing Via Exclusion steps

You can easily select discrete entries on the list using the CTRL key + left mouse button. Also using the Shift key + left mouse button you can select a continuous group of listings.by clicking on the first and last entry desired. All selected entries are highlighted in the Totem display.

Shorts ...

Displays a ‘Report of Shorts Area’ dialog listing all P/G wires that may be improperly shorted. The dialog allows you to step up or down the list and select a short location. Clicking on the ‘Go to Location’ button zooms to and highlights the potential short for closer inspection.

GDS Shorts ...

Displays information on GDS shorts.

Effective Grid Resistance Computation

Displays an ‘Effective Grid Resistance Computation’ dialog, to select options for executing the grid resistance calculation command.

The computation Modes available are

Arc - one worst-case node per net included (default)

Loop - one node per net, but includes both power and ground nodes

All Points - includes all nodes

All Pins - for MMX designs, also includes all pins internal to transistors (but only one node per pin)

Input data includes:

Inst/Node Count - maximum number of worst-case instances or nodes to be included (default 1000)

Thread Count - number of threads used for parallel R computation

Net Names - allows selection of a limited group of nets to be included. The Customize button allows selection of nets from a dialog box list.

Layer Names - allows selection of a limited group of Layers to be included. The Customize button allows selection of layers from a dialog box list.

Instance Names - allows selection of a limited group of instances to be included.

Cell Names - allows selection of a limited group of cells to be included.

Instance File - specifies a file that contains the list of instances to be included in the computation. The Browse button allows file selection from a dialog box list.

Cell File - specifies a file that contains the list of cells to be included in the computation. The Browse button allows file selection from a dialog box list.

Output File - specifies an output file other than the default, *adsRpt/<designName>.res_calc*. Two types of output can be selected: ”Incremental” means that the new resistance computation does not include nodes previously computed, while “Appended” means that the new output results are appended to the previous output results file.

View -> Technology Layers

Technology Layers (Metal and Via)...

Displays a graphic showing the layers in the design and the key characteristics of each, including dielectric constant (e), metal thickness and full layer thickness (h). If you use the right mouse button to create a box over one or more layers of particular interest, the view zooms to include just those layers. Clicking on the ‘Full View’ button then returns the view to the full view of all layers.

. Figure B2-37 shows an example Technology view.

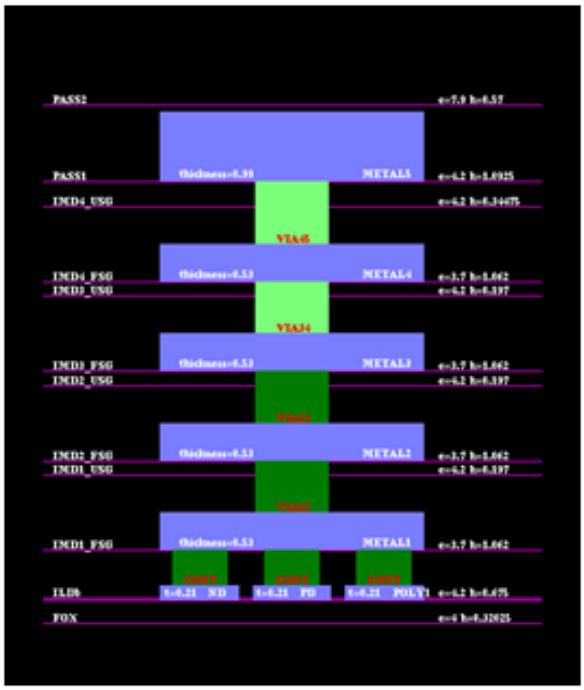


Figure B2-37 Layer characteristics display in the technology view

Technology Layers (Substrate) ...

Displays a substrate Technology View to support SOI/BCD advanced processes for substrate analysis separate from the metal and via technology view. You can use the this command to see “Thickness” and “Depth” attributes. You also can sync the color of the

layer with the LSW (Layer Select Window) and expand or collapse the sub-layers for easy viewing and debugging, as shown in Figure B2-38.

This menu is grayed out unless the design analysis includes substrate.

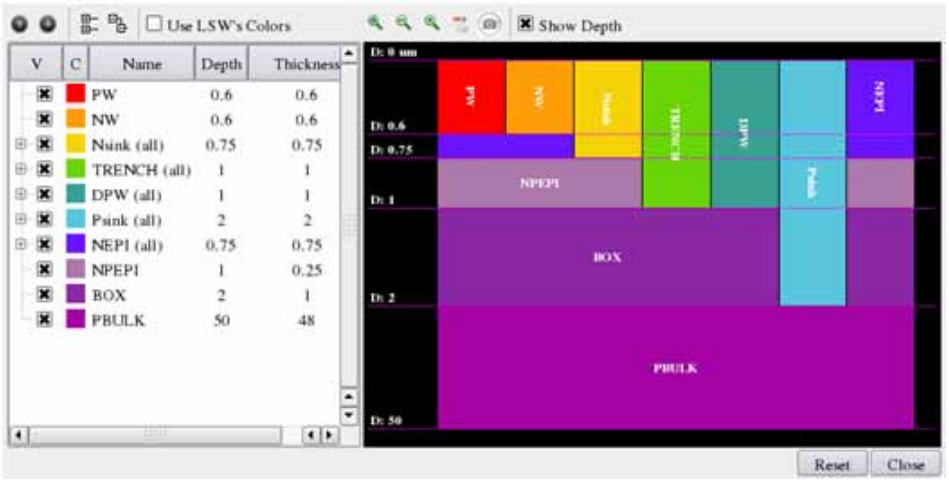


Figure B2-38 Substrate technology layer viewer

View -> Hierarchy Level ...

Displays a ‘Hierarchy Level’ dialog box that allows you to select the number of hierarchy levels for which design details are displayed. Hierarchy levels below that specified are shown as a “black box.”

View -> Netlist Hierarchy

Displays a Netlist dialog box showing the netlist hierarchy.

The integration of Netlist Hierarchy View with Cross-Probing Provides easy access to netlist information for debugging, and enables cross-probing with Layout View and other colormaps, as shown in Figure B2-39. When you select an entry in the Netlist Hierarchy view, the corresponding transistor pin is highlighted in the colormap. You can also select a transistor pin in the transistor pin map or SA view with the ‘pinInst’ check box enabled. To enable the pinInst check box, refer to Figure B2-3 on using the Layer Selection Window.

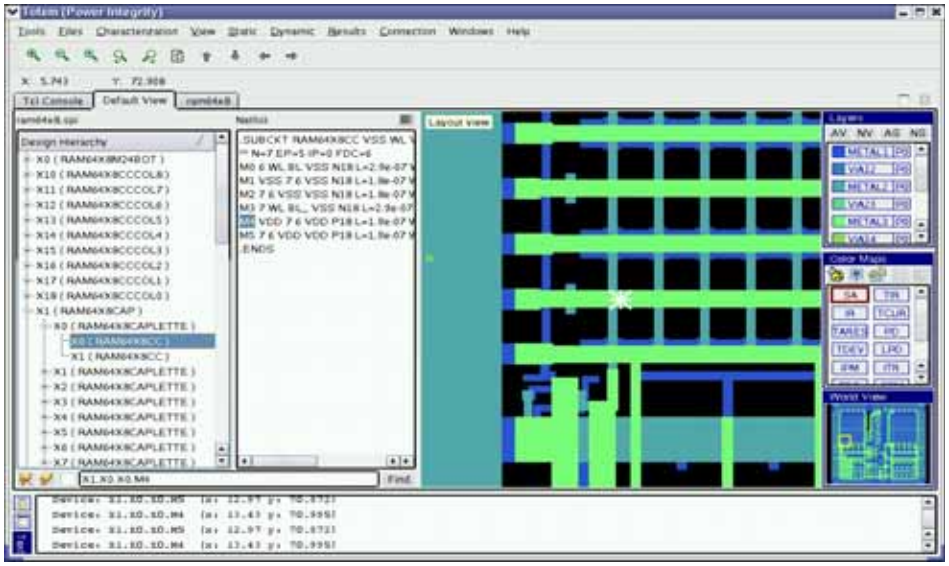


Figure B2-39 Netlist view and cross-probing

To import a netlist and display the netlist hierarchy view, use the command **Files -> Import Netlist**, and fill in the netlist filename, or you can click on **View Netlist** in the 'View Configuration' panel (first button from the right), and specify the netlist filename, which also brings up the netlist view. You can import a CDL format netlist in Netlist Hierarchy View to cross-probe with the Totem database and the Virtuoso Schematic Composer, or the Layout editor.

View -> Map Configurations ->

Layers Color Map

Displays a 'Layers' dialog box, the same as the 'View Layers' button in the 'Configuration' control panel. The dialog allows you to turn on/off and select color and outline or fill characteristics for display of all metal and via layers and the associated pins, as well as instances, in the design.

You can import the Virtuoso technology file, default display color file and a layer mapping rule using the **Files -> Import GDS** menu command to help cross-probe debugging between Totem2 GDS view and SA layout view. Then click on **View -> Map Configurations -> Layers Color Map**. After loading the Virtuoso tech and display file, the Virtuoso-like Layer Selection Window displays grouping information (see Figure B2-40), and GDS and SA views show the same color scheme, as shown in Figure B2-41.

The Layer Select dialog columns allow you to select **Bru** (Brush), **Vis** (Visible), **Fill** (Fill), **Line** (Outline), **Sel** (Select), and **Pin** (PinInst) layer representations. For example, select **Bru** to choose brush layer drawing mode instead of transparent drawing mode. To control Metal/Via layer display, click on the header of each column, as shown in Figure B2-40. Also you can click on the **Vis** column (Visible) to get a popup menu and choose to show all or hide all layers, or you can show all metal or all via layers. The same procedure applies to changing layer display characteristics using the other column headers.

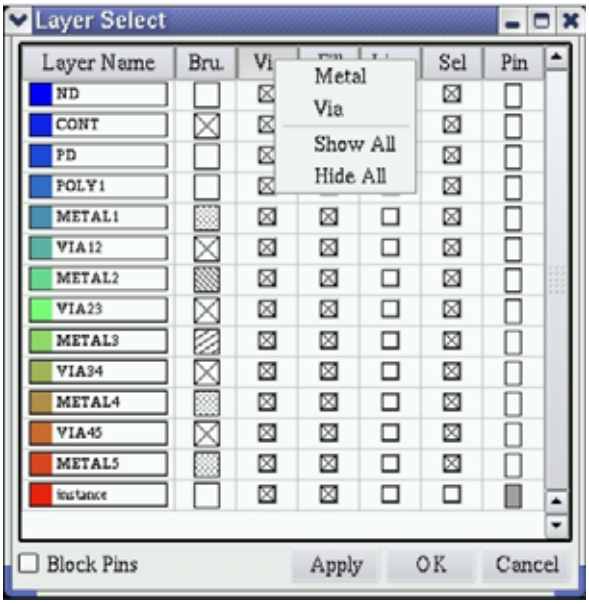


Figure B2-40 Layer select window

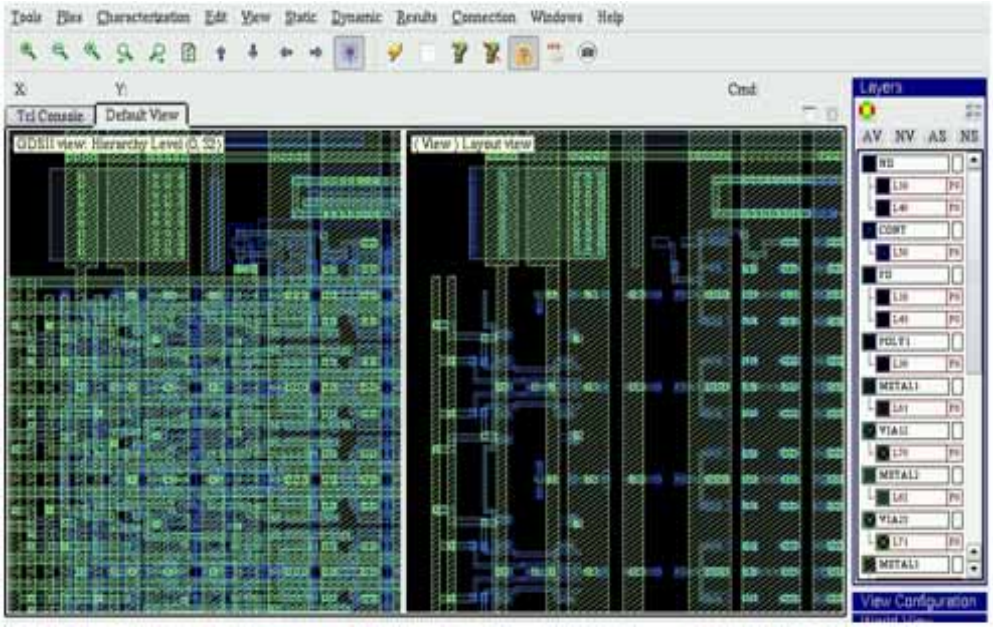


Figure B2-41 GDS view and layout view cross probing

Current Color Map

Displays a ‘Current Color Map’ dialog box that allows you to select colors for a set of current values from minimum to maximum in the design. The default display divides the ranges equally between the minimum and maximum current.

EM Color Map

Displays the 'ElectroMigration Color Map' dialog box that allows you to select colors for a range of EM values as a percentage of EM maximum tolerance.

View -> EM Mode

Avg - RMS - Peak

Displays Average, RMS or Peak EM results. RMS EM values are displayed by default in signal EM analysis.

View -> Electromigration Map

Displays results of electromigration analysis for static or dynamic current conditions, the same operation as the **EM** button in the 'Configuration buttons' panel.

View -> Current Maps

Wire and Via

Displays a color map indicating the wire and via currents in the design, the same as using the **CUR** button in the Configuration buttons' panel.

View -> Transistor Pin Maps ->

Current Map

Displays transistor pins in colors identifying their resistance values. Click on the 'Set Color Range' button in the Configuration group to obtain a 'Transistor Pin Current Color Map' dialog, which allows you to select colors for a range of pin current values, set the current range displayed, and also increase the size of the pins for ease of viewing using the 'Pin Size Scale' window.

Also, you can adjust the display settings by clicking on the left "Set Color Range" **View Configuration** button to display the **Color Map: Current Map** dialog. This dialog allows you to choose the nets to be displayed, and the maximum and minimum current You can also select the number of colors and the colors to use, to help the current range of interest and define the hot spots more easily.

View -> Temperature Map

Thermal Profile Map

Displays a color-coded map of regional temperature for the layer specified.

View -> View Control

Zoom In by 2

Zooms view in by a factor of 2 (dimensions increase 2 times).

Zoom Out by 2

Zoom view out by a factor of 2 (dimensions decrease by 1/2)

Fit All

Displays full design

Last View

Displays previous view.

Next View

Displays view before previous view.

Redraw

Refreshes present view.

Scroll Up

Moves view up.

Scroll Down

Moves view down.

Scroll Left

Moves view to left.

Scroll Right

Moves view to right.

View 0

GDS Viewer view of top hierarchy level.

View 32

GDS Viewer flattened view of up to 32 hierarchy levels below present view.

View -1

GDS Viewer flattened view of one hierarchy level above present view.

View +1

GDS Viewer flattened view of one hierarchy level below present view.

Display Selected Nets

Displays all nets that are selected.

Display All Nets

Displays all nets.

Show Power Pad

Highlights power pads.

View -> Query

Switch Highlight Mode

Switches highlight mode between blinking and non-blinking.

Set Highlight Color

Displays a dialog box in which to choose a highlight color. Default: white.

Query Object ...

Displays a dialog box in which characteristics of the selected object are listed.

Clear Highlight

Clears highlights from displayed objects.

Preselect Query

Displays object information in the Status box when cursor hovers over an object. Use the space bar to cycle through information for other objects at the same coordinates.

Instance Property

For multiple-instance SOC designs, displays a dialog showing key properties of a selected instance.

Dump Gif

Saves a GIF file of the current layout view in the selected directory.

View -> Switch Operating Mode

Switches GUI style between the new GUI style format and the “original” Totem GUI display format (Alt - p). Note that some available menu commands change when switching between styles, such as the **Windows -> Toolbars** command.

View -> Job Monitor

Brings up a Job Monitor dialog to provide status on running jobs at selected intervals.

Analysis -> Static Signal EM Analysis

Runs static signal EM analysis.

Analysis -> Dynamic Signal EM Analysis

Runs dynamic signal EM analysis.

Analysis -> EM Check

After performing signal EM analysis, uses signal current results to run the EM check command, the same as the Tcl command ‘perform em_check’.

Results -> Log Message Viewer...

Provides access to four different report viewers, ‘Errors and Warnings Summary’ (default), ‘CPU/Memory Usage’, ‘Setup Design’, ‘Power’ and ‘Results’, described in the following paragraphs:

Errors/Warnings Summary

Displays a dual window showing a summary of the total messages generated during the analysis by type (Error, Warning, Info) in the upper window and a selected individual message in the lower window.

CPU/Memory Usage

Displays a table showing maximum Memory use, Total CPU Time, and Wall Time for each design stage from importing files through dynamic analysis. Note that the CPU time for the simulation step is *not* displayed as cumulative for convenience; the following step and all other steps are shown as cumulative CPU time.

Setup Design

Displays a list of files and their paths that have been imported in the design.

Power

Displays a dual window showing a summary of design power information by frequency and by domain in the upper window, and power related warning messages by instance in the lower window.

Results

Displays a simulation results report, such as Worst Dynamic Voltage Drop summary report for dynamic analysis.

Results -> Design Summary Report

Generate

This feature is not available in Signal EM analysis.

View

This feature is not available in Signal EM analysis.

Results -> List of Worst EM ...

Displays an ordered list of worst-case high electromigration locations, with the wire x,y starting and ending locations for each occurrence. You can also highlight and zoom to any of the critical EM locations by using the ‘Go to Location’ button, and also step through the various locations using the ‘Prev’ and ‘Next’ buttons.

The latest EM mode name is displayed in the “List of Worst EM” dialogs after EM analysis. When you switch to a different EM mode, the new EM mode is displayed in color maps and reports so it is clear which mode is displayed.

Results -> List of Highest Wire Temperature...

Displays a ranked list of per wire temperature table from the final emcheck step in a pop-up window in CTM flow.

An initial analysis is done to calculate a new operating temperature for every wire in the design and these temperature values are then read back into RedHawk/Totem. The per-wire (or per-wire-segment) temperatures are then used to re-extract resistor values in the design and to compute new EM limits. Then perform a new analysis and finally a new emcheck.

This supports all Tools - Power Integrity, Signal EM, ESD. The GSR keyword "TEMPERATURE_REPORT_LINE_NUMBER" controls the line number in the report file (default 1000 lines). The following are the features of this table:

1. Supports zoom in, preselect highlight and message hint functions
2. Supports default debug frame < Highest WT debug > using *Windows -> Display Multiple Frames -> < Highest WT debug >*

Results -> List of Signal Nets Transistor Pin Currents

After Transient signal EM analysis, displays a signal net browser with a net name filter, as well as a net name search function. By selecting signal nets in the browser, all transistor pins of the selected nets are displayed on the right side list, showing transistor name, and the associated average, RMS, and peak currents. You can also use the “Plot Waveform” button to display the current waveform of selected transistor pins in the waveform viewer.

Results -> Analysis Histograms ...

Displays an ‘Analysis Histogram’ dialog box to specify what parameters to display in the histogram output:

- Layer
- EM Mode
- Signal Net

The Generate Histogram Distribution check box specifies the output file.

'Limits' box provides automatic selection of 'Lower' and 'Upper' % limits, or the limits may be specified manually. There is also a choice of bin size or number of bins.

To display analysis data for elements only within a specified bounding box area, use the TCL command 'condition set -xy <x1 y1 x2 y2>', where x1,y1 and x2,y2 are the corners of the desired bounding box to be analyzed.

Connection -> Connect to layout editor...

Sets up connection to a third-party layout editor, as described in section "Approach 6: Find ESD/Fuse cells that cause intentional shorts", page 3-150.

Totem supports cross-probing to Cadence Virtuoso and Springsoft Laker layout editors and Virtuoso Schematic viewers. To set up these capabilities, follow the procedures described below:

Cadence Virtuoso cross-probe setup

1. Virtuoso *.cdsinit* file setup:
   a. Edit the *.cdsinit* file to load Apache skill code program.
   b. Append file *$APACHEFILE/lib/crossprobe/cdsinit-sample* in your *.cdsinit* file. The *.cdsinit* file could be in the (1) current working directory, (2) user's home directory, or (3) Virtuoso installed/setup directory.
2. Source the Virtuoso resource file *cadence.rc* or *virtuoso.rc.*
3. Edit and source Totem resource file *totem.rc.* You can find the *totem.rc* template file under *$APACHEROOT/lib/crossprobe/totem.rc*.

A cross-probing icon on the View Control toolbar turns Virtuoso cross-probing On and Off. This icon is only enabled and displayed when the Virtuoso-to-Totem proxy cross-probing mode is used.

Totem-Virtuoso cross-probing/analysis can be invoked on a different machine for performance/capacity improvement using industry standard interfaces. Currently only LSF and SSH interface is supported. The usage procedure is:

1. Set the environment variable `TOTEM_SETUP_WIZARD_DIR` to your copy of the Totem wizard configuration directory.
2. Add entries to the job submission configuration file at:
   `$TOTEM_SETUP_WIZARD_DIR/wizard/wrapper/wrapper.ini`
3. Set the environment variable APACHE_CROSS_PROBING_WRAPPER_SET to the user-defined entry.

Springsoft Laker layout editor cross-probe setup

1. Source the Laker resource file *laker.rc*
2. Edit and source the Totem resource file *totem.rc.* You can find the *totem.rc* template file under *$APACHEROOT/lib/crossprobe/totem.rc.*

Required environment variables

Inside the *totem.rc* template file, you must set up the following global environment variables:

A. Cross probing

```
setenv APACHE_CROSS_PROBING_LAYOUT <layout_editor_binary_name>
setenv APACHE_CROSS_PROBING_SCHEMATIC <schematic_binary>
```

Specify the 3rd-party binary name.

B. Design setting

```
setenv APACHE_CROSS_PROBING_DBPATH <design_database_path>
```

Specifies the directory containing *cds.lib* (Virtuoso library header file) or Laker layout database directory, which contains l*aker.rc* file.

```
setenv APACHE_CROSS_PROBING_TOPDESIGN <top_cell_name>
```

Specifies the design hierarchy/cell name in this variable to allow Totem to find the correct hierarchy in the layout design database.

```
setenv APACHE_CROSS_PROBING_LIBRARY <library_name>
```

Specifies the layout design library name to allow Totem to invoke the 3rd party layout editor and load the correct design library (there may be multiple design libraries for the same hierarchy).

Replay mode setup variable

```
setenv APACHE_CROSS_PROBING_REPLAY_MODE y
```

Forces automatic entering of the replay mode for connection setup. This variable is *required* for Laker, and optional but recommended for Virtuoso.

Cross-probing port setting variables

```
setenv APACHE_CROSS_PROBING_WAITTIME <connection_timeout_limit>
```

Specifies connection time-out limit to 3rd party layout editor. Default time is 30 seconds.

```
setenv APACHE_CROSS_PROBING_STARTPORT <startport_id>
setenv APACHE_CROSS_PROBING_PORTRANGE <range_number>
```

Specify the connection socket port range. Totem starts to search available ports in order from <startport_id> and stops searching when reaching a port ID larger than (<startport_id> + <range_number>). Default ports are 11000 for APACHE_CROSS_PROBING_STARTPORT and 5000 for APACHE_CROSS_PROBING_PORTRANGE

After setting up cross-probing with the above procedure, use the following steps to perform cross-probing investigations in Totem.

1. Choose the GUI menu command **Connection -> Connect to layout editor**. Totem displays a dialog window to allow you to modify layout design variable settings, as shown in Figure B2-42.

2. After clicking the **OK** button, Totem attempts to connect to the selected 3rd party layout editor. If the socket connection can be made within the time-out limit, the 3rd party layout editor GUI is displayed.

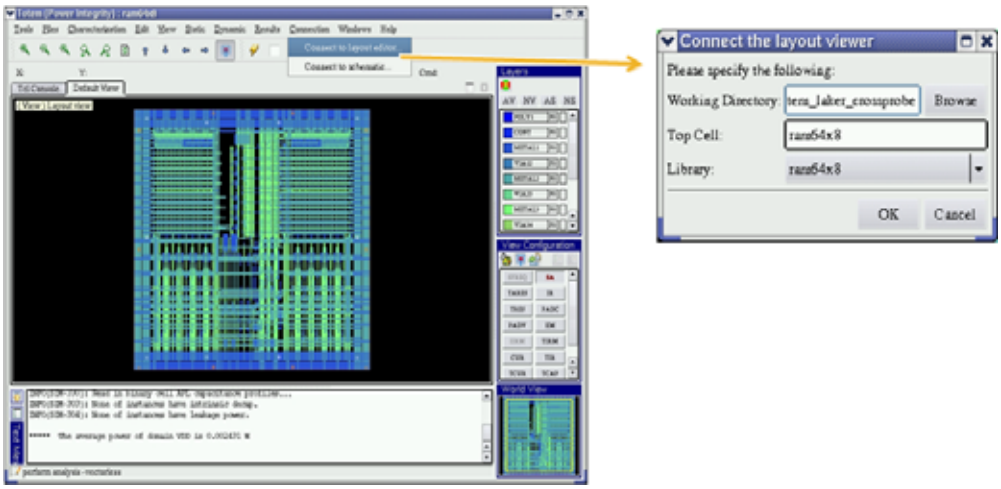


Figure B2-42 Making the layout editor cross-probing connection

Totem layout cross-probing capabilities include:

- Zoom-In / Zoom-Out
- Highlighting geometry in the layout window
- Zooming to a selected transistor location from a GUI list of key transistor locations (see Figure B2-43 and Figure B2-44). Use the command **Results -> List of Worst Transistor Pin Voltages.**

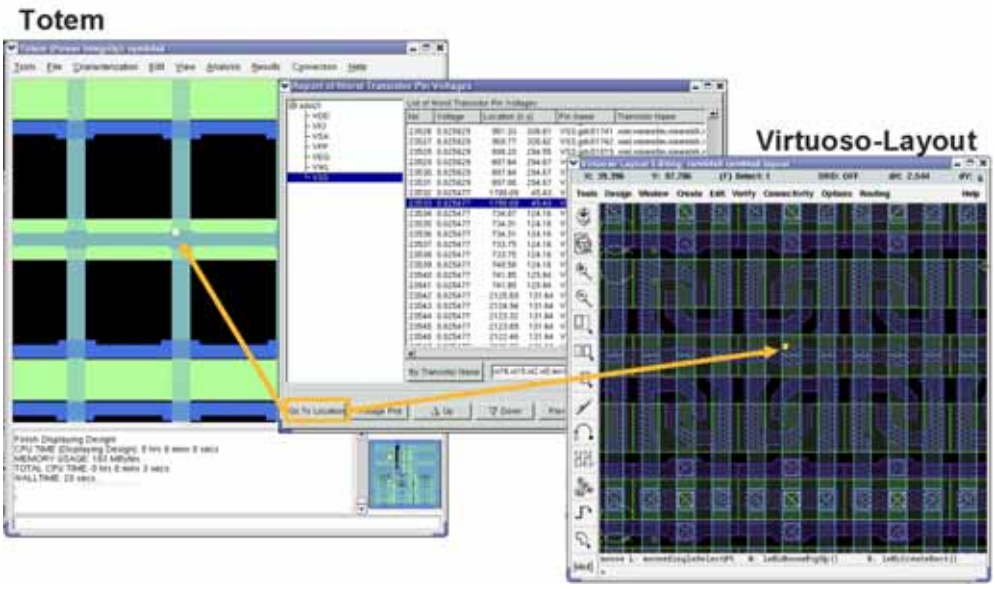


Figure B2-43 Cross-probing to Virtuoso layout

- Zooming to a selected EM violation location from a list or critical EM locations, use the command **Results -> List of Worst EM.**

The 3rd-party layout editor displays the items identified in Totem.

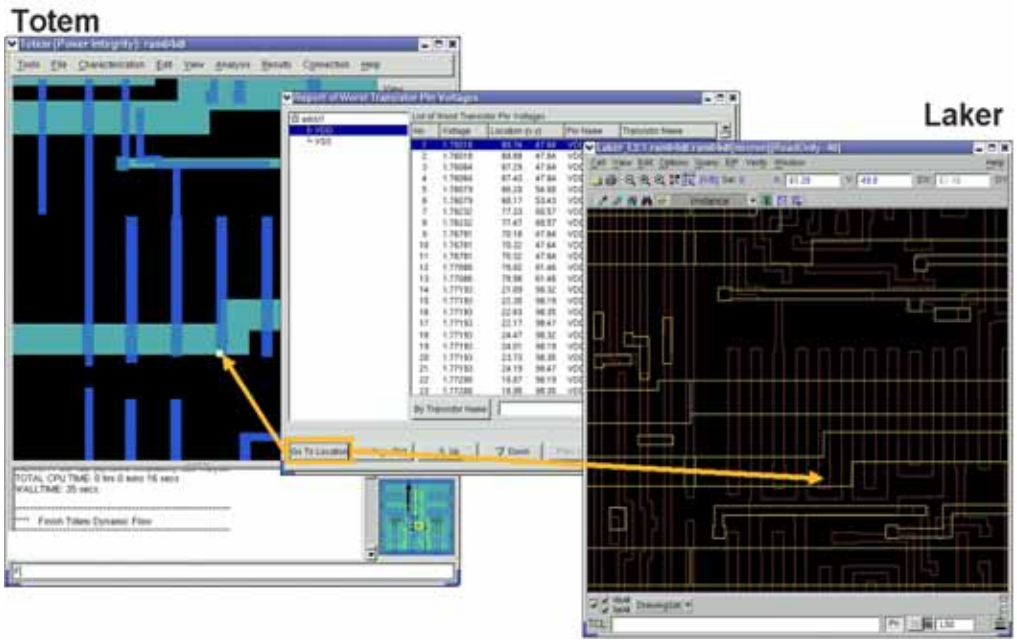


Figure B2-44 Cross-probing to Laker layout

Connection -> Connect to schematic ...

See setup in section "Cadence Virtuoso cross-probe setup", page B2-669

Uses the Totem cross-probing function to connect to Virtuoso Composer to view the schematic, as shown in Figure B2-45. This can help debug EM / IR / DvD problems that may be caused by design issues in the circuit.

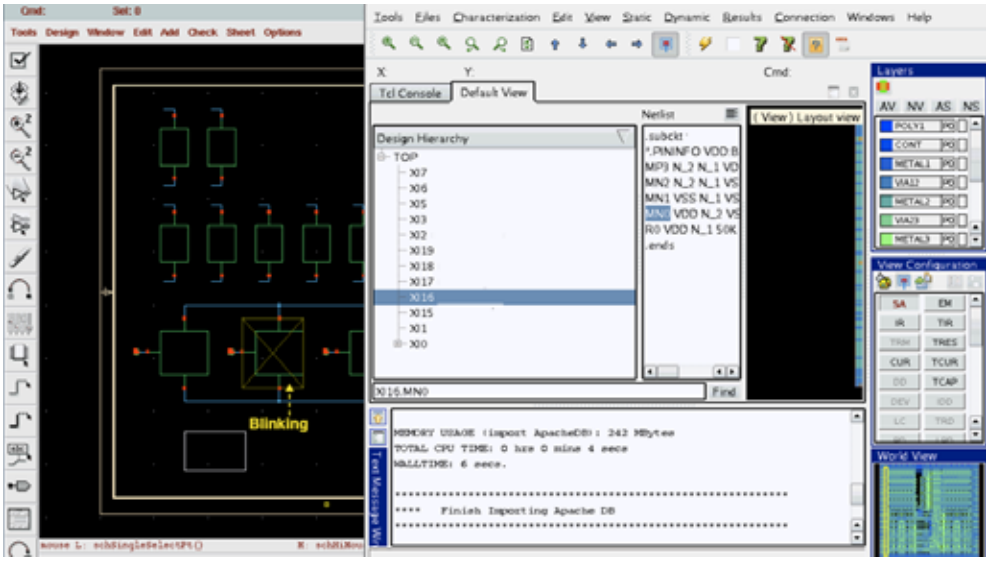


Figure B2-45 Cross-probing in the Virtuoso Schematic Composer

Use of schematic device names in the DB for Virtuoso cross-probing

The default device names stored in the Totem database change from the name in DSPF netlist to the name in the simulation netlist, as follows:

- in the SPICE_NETLIST + DSPF_NETLIST flow: the device name from the SPICE_NETLIST
- in the DSPF-only flow: the device name in the instance section in the DSPF_NETLIST file

This helps set up Totem-to-Virtuoso composer schematic netlist cross-probing.

Totem Explorer

Explorer -> Generate

Displays a dialog box to allow you choose the Data Integrity, Design Weakness and Hot Spot results that you want, and the threshold values controlling the data to be displayed. Data Integrity checks available include:

- CMM Validation
- APLMMx Current Consistency
- APL Current
- APL Cap
- APL Piecewise Cap
- LIB
- LEF
- GDS2DEF
- GDS2DB
- DEF
- SPEF
- STA
- VCD
- GSR Tcl
- Package Settings
- Pad File

Design Weakness checks available include:

- Pad Placement Quality
- Package Drop Contribution
- PG Resistance Distribution
- PG Resistance Imbalance
- Switch Placement Quality
- Decap Distribution
- Power Distribution Quality
- Clock Buffer Clustering
- Peak Current
- Simultaneous Switching

The Hot Spot checks available are:

- Dynamic Voltage Drop
- MMX Cell-Based

- Static IR
- Power EM

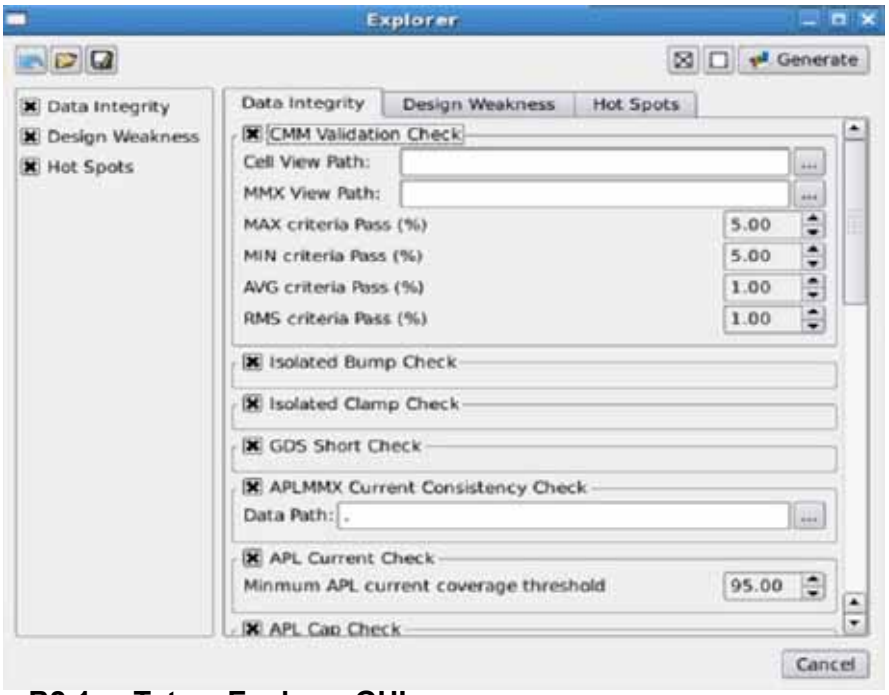


Figure B2-1 Totem Explorer GUI

After the desired results are selected under each tab, the Explorer Summary dialog is displayed, and the results can be analyzed.

Explorer -> View

Displays the Summary of results selected and created with the **Generate** command.

Windows -> Display Multiple Frames

Setup Configuration ...

Allows setting up a multi-frame configuration, based on your debug preferences, using the menu selection **Setup Configuration**, as shown in Figure B2-2. You can **Save / Reload** previous configurations.

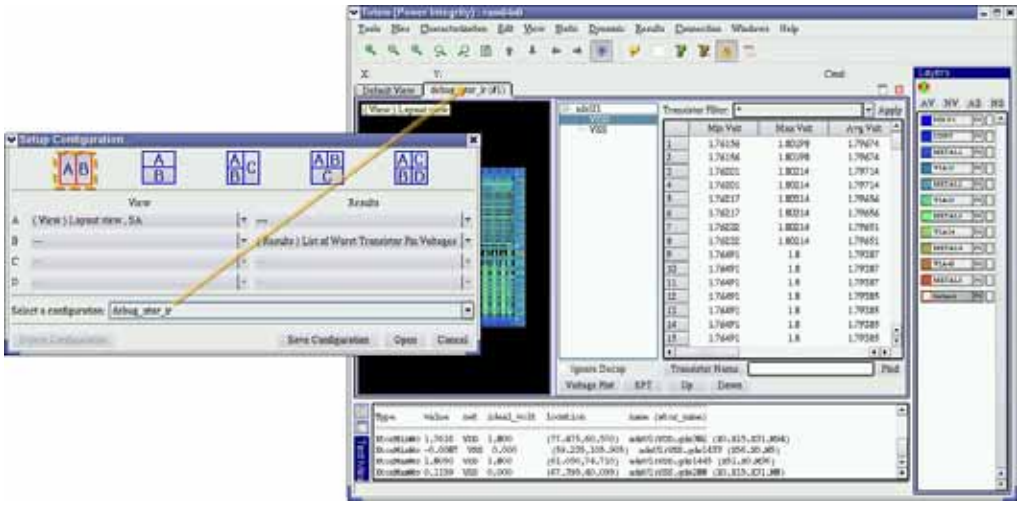


Figure B2-2 Multi-frame configuration setup

There are other types of preconfigured multi-frames for power analysis and signal EM analysis, to make setting up the debug environment more convenient, as shown in Figure B2-3.

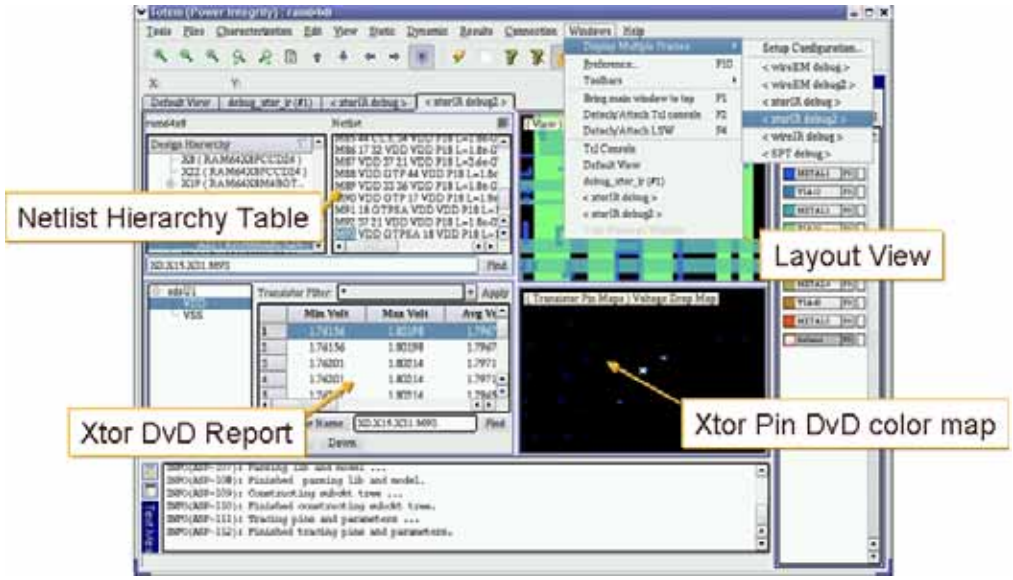


Figure B2-3 Preconfigured multiframes

Per frame net selection allows selection of different nets in different GUI frames to allow cross-checking of different nets at the same time, as shown in Figure B2-4.

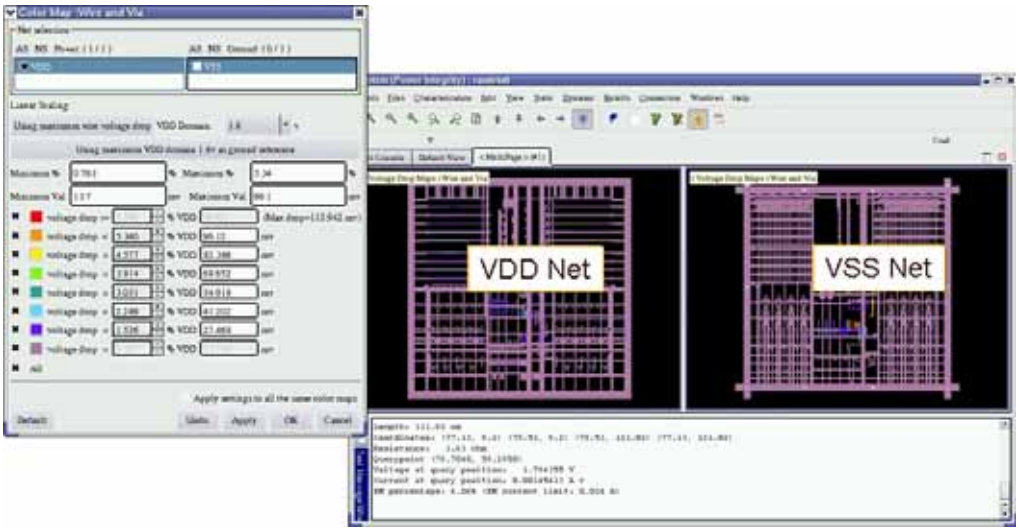


Figure B2-4 Per frame net display and selection

Windows -> Preferences ...

Brings up a **Preference** dialog box, with the following option selections:

Common options

- GUI Font: (Sans Serif 9 Normal default)
- Text Message Font : (Sans Serif 9 Normal default)
- Tcl Console Font : (Sans Serif 9 Normal default)
- Totem Explorer Font: (Sans Serif-8 Normal default)
- Highlight : Blinking or Color (default white)
- Highlight Sync : Current View or All Views (default)
- Highlight Method : Auto Zoom (default) or User Defined.

  Auto Zoom: zoom and center at the selected location.

  User Defined: highlight the select location (marker), but not auto center, if the selected location is within the current zoom canvas. If the new selected point is outside current zoom view, the GUI uses current fixed zoom ratio and centers at the new selected point.
- Zoom sync : Current View or All Views (default)
- Waveform Viewer : xgraph or Totem Waveform Viewer (default)
- Waveform Viewer Integral Method : Left-Rectangle (default) or Trapezoid
- Color Maps : Smooth Color Spectrum, or Disable GDS Instance in Color Map (default) The “Disable GDS Instance in Color Map” box allows disabling the instance IR map of GDS_CELLS to view the leaf instances underneath.
- Image background for printing: sets color of background for image printing (default black)
- Scroll Bar for Color Entries : 12 (default) - sets maximum number of color ranges before scroll bar is used.
- Editing Mode : Mode1 (default), or Mode2, which allows Virtuoso-style “what-if” editing functions to support wire and via editing

- Auto Region Filter: Enabled (default) or Disabled
- Using Multi-threads Mechanism - you can choose to either “Follow GSR: MULTI_THREADS Setting” (default), or choose to override the GSR keyword value using the “Enable Thread Number” checkbox to set a maximum number of threads to use.
- Text Message Window Options: provides a ‘Show Summary Page’ check box for a summary of Process steps completed in the log area below the design display (default On).
- Min Resistance Path (SPT) Analysis: By default “Support Multiple” paths is provided.

GDSII Display Options

- Display Controls : choose GDS Text or Path Borders (default)
- Array Display : Full (default), Border, or Source
- Filter Size : number of pixels (default 1)

Grid Controls

- Type : None or Dots (default)
- Minor Spacing: 0.002 (default)
- Major Spacing : 0.01 (default)
- Spacing Mode: User Specify spacing, or Automation Detection (default)
- X snap Spacing : no default
- Y snap Spacing : no default

Windows -> Toolbars

Note that this command is not available when the original GUI operating mode style is selected (Alt - p).

Show all toolbars

Displays both View Control and Query toolbars (default)

Hide all toolbars

Hides View Control and Query toolbars.

View Control Bar

Check box to control display of the View Control toolbar (default On).

Query Bar

Check box to control display of the Query toolbar (default On).

Windows -> Bring main window to top F1

Displays main design view on top.

Windows -> Detach/Attach Tcl console F2

Displays separate TCL terminal window.

Windows -> Detach/Attach LSW F4

Displays separate Layers, View Configuration, and World View windows.

Windows -> Tcl Console

Displays TCL terminal window as primary view.

Windows -> Default View

Displays full design view.

Windows -> Text Message Window

Brings detached Text Message window to top.

Help -> About...

Displays information about the Totem software release being utilized.

Help -> Manual

Provides a link to the latest “Totem User Manual” in PDF format.

Bindkey Functions

Functional Summary

- Functional bindkeys (Virtuoso-style behavior)
  - **F1** - Brings main window to top
  - **F2** - Detaches / attaches TCL console
  - **F4** -Detaches / attaches LSW panel
  - **F10** - Brings up ‘Preference’ dialog
- View Control bindkeys
  - **Ctrl** + **Z** - Zoom in by factor of 2
  - **Shift** + **Z** - Zoom out by factor of 2
  - **F** - Fit All
  - **W** - Last view
  - **Shift** + **W** - Next view
  - **Ctrl** + **R** - Redraw
  - **Up** - Scroll up
  - **Down** - Scroll down
  - **Left** - Scroll left
  - **Right** - Scroll Right
  - **Ctrl + F -** Display only top hierarchy in GDS view
  - **Shift + F** - Flatten all hierarchies in GDS view
  - **Ctrl + P** - Show power pad
  - **Shift + V** - Go down one hierarchy in GDS view
  - **Ctrl + V** - Go up one hierarchy in GDS view
  - **Ctrl** + **N** - Display selected nets / exclude unselected nets
  - **Shift** + **N** - Display all nets
- GUI Command bindkeys
  - **Ctrl** + **Q** - Exit GUI
  - **K** - Creates ruler

- Supports 45-degree ruler drawing
- Supports multiple-segment ruler drawing

Shift + K - Clears all rulers

Alt + P - Switches between the RedHawk-like GUI and the enhanced Totem2 GUI

Reconfiguring Bindkeys

You can reconfigure bindkey settings using the following steps

1. Export the existing bindkey settings supported in the Totem2 GUI to a text configuration file using the TCL command:

   ```
   export keybinding <keybinding_filename>
   ```

   Note that this command is only available in the GUI, and not in batch or interactive modes.

2. Edit the bindkey configuration file created in step 1 to establish the new bindkey settings desired.

   The format of the bindkey configuration file is as follows:

   ```
   # <description of bindkey >
   <bindkey name> <bindkey definition>
   ```

   Bindkey definitions can be assigned to any combination of single letter, **Ctrl**, **Shift**, and **Alt** buttons.

   Note that no spaces are allowed in bindkey definitions.

3. The newly-defined bindkeys can then be imported using one of the following methods:

   a. In GUI mode, use the command

      ```
      import keybinding <keybinding_filename>
      ```

   b. Rename the keybinding configuration file '*keybind.cfg*' and put it in

      - your working directory, or
      - the directory *$HOME/.totem/*

   If there is a *keybind.cfg* file in both of the directories above, the one in your working directory is used.

Appendix B3

ESD GUI and Menus

Introduction

This appendix describes the graphic user interface and menus available for running Totem ESD Analysis. For TCL invocation and command descriptions, see Appendix D, "TCL Command Reference".

For descriptions of Power Integrity analysis menus, see Appendix B1, "Power Integrity GUI and Menus"

For descriptions of Signal EM analysis menus, see Appendix B2, "Signal EM GUI and Menus".

Totem ESD GUI Description

Introduction

The Totem ESD GUI functional areas are shown in Figure B3-1 below.

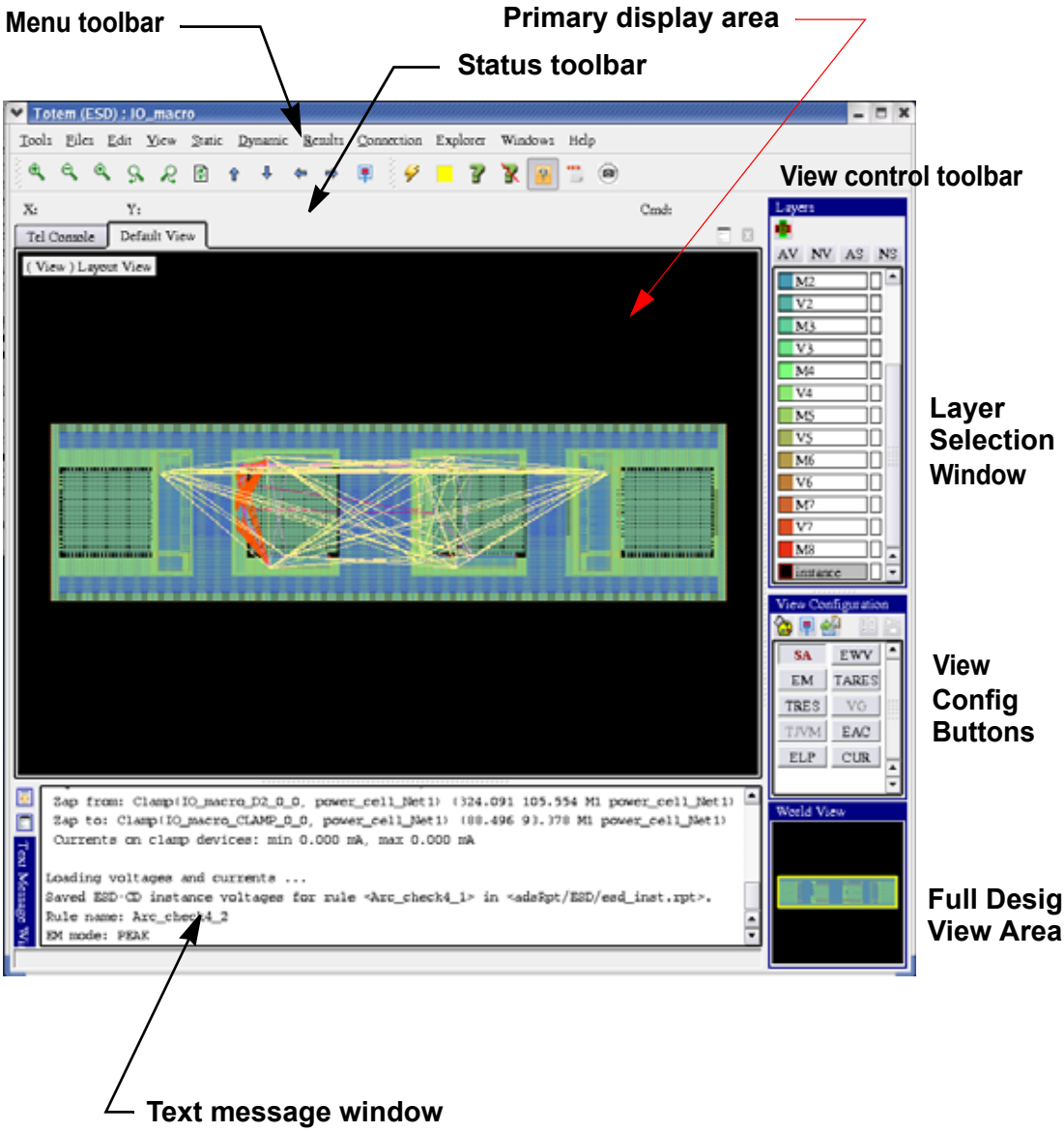


Figure B3-1 Totem ESD graphical user interface

The Totem GUI provides a configurable multi-window, multi-page, multi-frame debugging environment. You can combine different color maps and reports on the same GUI page to debug design weaknesses and examine the source of design issues much more efficiently. A few key features are:

- Switch view menu based on analysis tool to only highlight product specific features
- Configurable multi-frame multi-page interface to allow user configure his/her own preferred debugging environment
- Pre-select and status bar helps user query design information in a much more efficient way

- Embedded GDS viewer to allow user cross-probe between analysis layout view and color maps with GDS viewer for easy debugging

The following figure is an example of how you can use the Totem GUI to configure a multi-frame page for debugging:

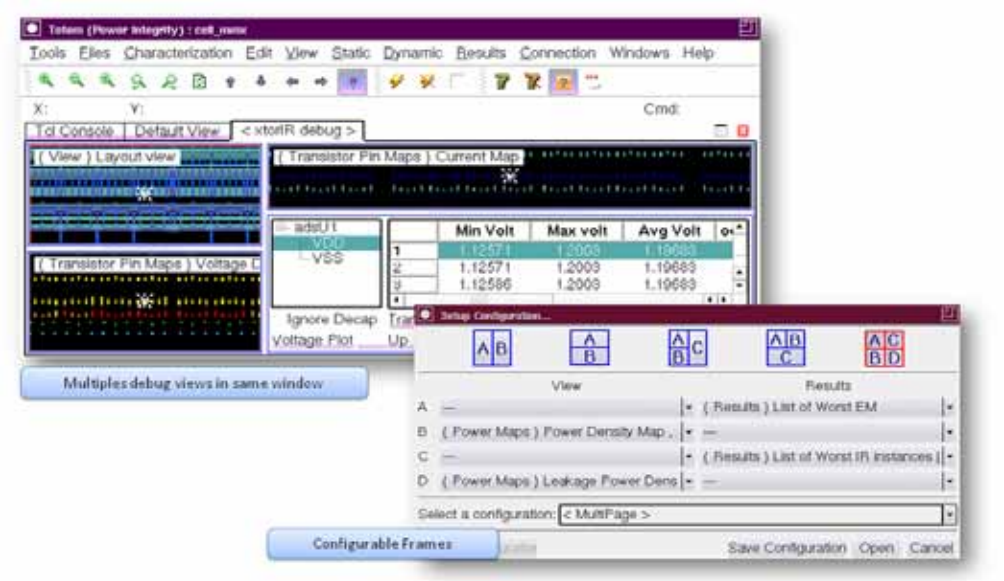


Figure B3-2 Totem multi-frame GUI

General GUI Features

GUI Look and Feel Preference Setting

The *Totem.ini* file can be used to control the GUI menu, dialogs, GDS View, Waveform Viewer, multi-frame setup and other common options, such as font type, font size, and highlight preferences. A template of the *Totem.ini* file is in the *lib/guicfg* directory in the default installation package (*$APACHEROOT/lib/guicfg/Totem.ini*).

The priority of available *Totem.ini* files is as follows.

1. From the local Totem run directory *adsGui/Totem.ini.*
2. From user's home directory *~/.totem/Totem.ini.*
3. From installation *$APACHEROOT/lib/guicfg/Totem.ini.*

If files in locations 1 and 2 are not available, then the default GUI preference settings from 3 are applied. Or you can use “F10” button to bring up the Preference dialog and click the “Reset” button on the bottom of the dialog to reset to Totem default settings.

Layers Window

The **Layers** window (see Figure B3-3) allows viewing information related to layers. The **Layers** window functions are:

- AV - All Visible
- NV - None Visible
- AS - All Selectable
- NS - None Selectable

Note that when **SA** is selected in the **View Configuration** window, and the “Configure Color Maps” button is clicked, a Layer Select dialog box is displayed (see Figure B3-5), which allows changing the display features of each layer. Individual layer display can be changed using the check boxes. Click on the column headers to change the display characteristics for groups of layers: Vis (visibility), Fill/Line (object display type), Sel (select metal/via layers), Pin (select metal/via layer pins).

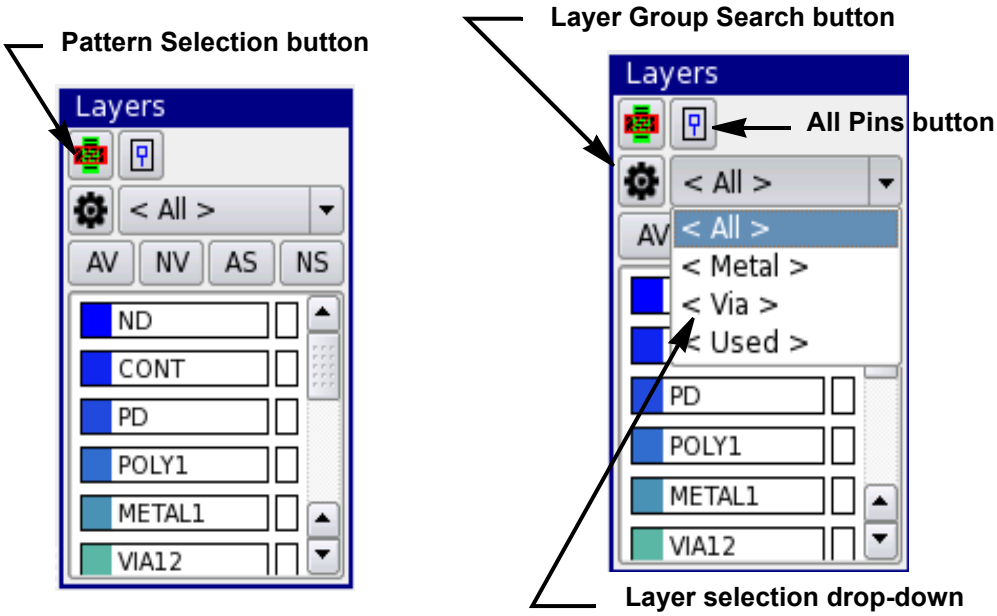


Figure B3-3 Layers selection window

In the **Layers** window, use the left top button to switch between **Pattern Brush** and **Transparent Drawing** modes, as shown in Figure B3-3 and Figure B3-4.Other capabilities of the Layer Selection dialog are.

- support for layer search and filtering using the Layer Group Search button
- display only “used layers”, which is turned on by default
- option for name-based search of layer-names, which also supports wildcards

the All Pins button turns On/Off transistor pins for all layers in the SA view, as shown in Figure B3-3.

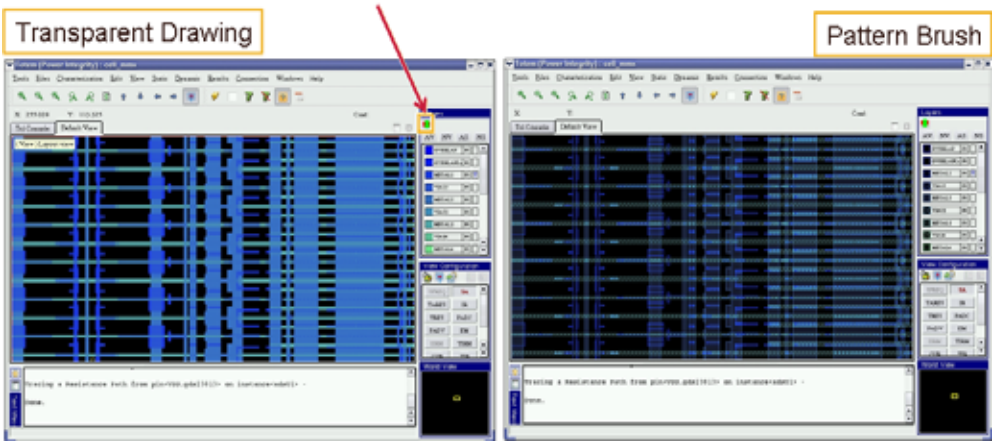


Figure B3-4 Pattern brush & transparent drawing displays

View Configuration Window

At the top of the **View Configuration** window, as shown in Figure B3-5, are buttons for opening a “Set Color Range” dialog, executing “Show Power Pad”, and opening a “Configure Color Maps” dialog.

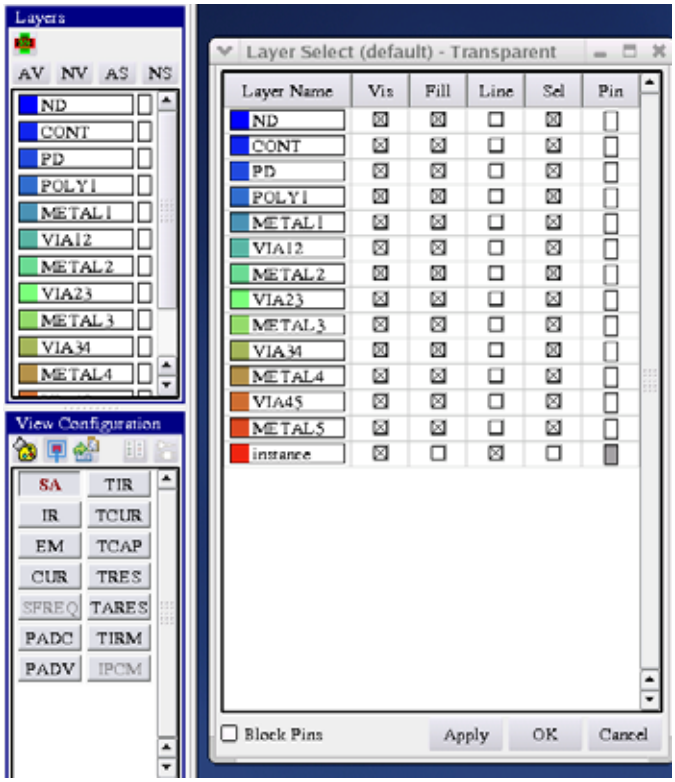


Figure B3-5 Layer Select and View Configuration windows

- “Set Color Range” (left Configuration icon)

Displays a Color Map dialog box based on the ‘View Results’ or **View Configuration** window selection.

Color maps may display either objects, a high parameter value, or a parameter density by tiled grid. Density displays use a grid composed of “tiles” that are 30 microns square by default. The size/number of the tiles can be changed using the ‘Grid Num’ or ‘Grid Size’ boxes on the dialog.

One of up to 20 colors can be assigned to specific ranges of parameter values, such as static or dynamic voltage drop, power, EM severity, current, frequency, or capacitance. Display colors can be changed by clicking on the color button and then selecting a new color from the ‘Pick a Color’ dialog box that appears. Ranges of parameter values also can be turned on or off independently with check boxes. The **Default** button allows easy return to the original color display values.

- “Show Power Pad” (center left Configuration icon) - toggles the display of power pads on and off (on by default). VDD pads are shown in orange and VSS pads in white.
- “Configure Color Maps” (center Configuration icon) - in general the specific color map dialog box for each of the following functions in the View Configuration window can be displayed by clicking on the ‘Set Color Range’ configuration button described above.

  You can change color settings between 5 to 20 in the Color map dialog for wires and vias, to allow more color resolution in customizing color maps and movies, as shown in Figure B3-6.

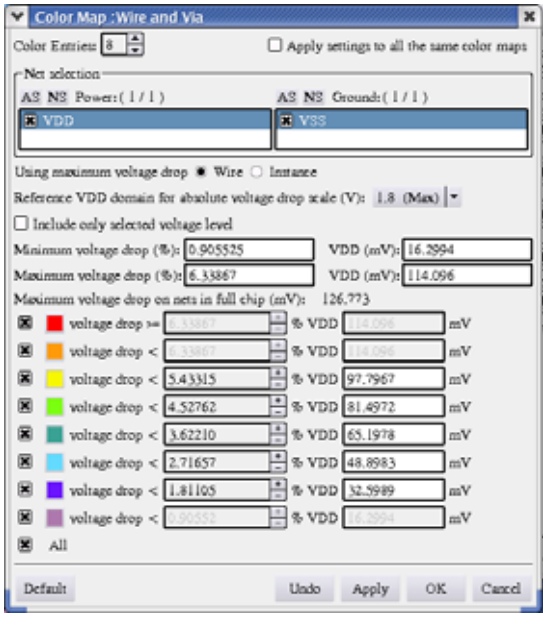


Figure B3-6 Color map color setting dialog

- View GDS (center right Configuration icon) - opens an “Import GDS ...” dialog, allowing you to specify a GDS file, include a Top Cell and Layer Map, as well as Third Party Technology Data files for importing.
- View netlist (right Configuration icon) - opens an “Import Netlist” dialog, allowing you to specify a Netlist File and a Top Cell for importing.

Configuration buttons

The buttons displayed in the **Configuration** window are customizable, and can be modified clicking on the “Configure Color Maps” button when **SA** is selected (see Figure B3-7).

You then can select entries from the left side table and use the right arrow button to add new maps in the panel or select entries from the right table and use the left arrow button to delete maps from the fast access panel.

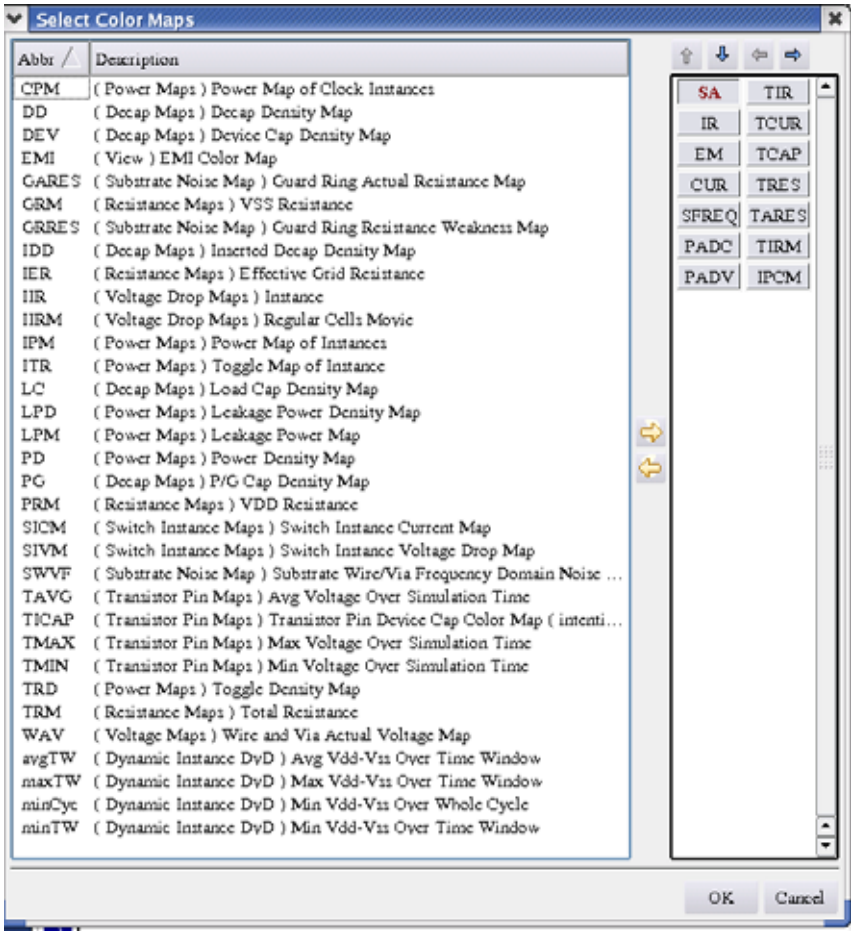


Figure B3-7 Select Color Maps dialog

The default View Configuration buttons are described following:

First column of ‘View Configuration’ buttons (default)

- **SA** - “Show all” of layout - displays some or all objects in the design layout (individual layer grids, instances, vias and pins per layer), based on the selections in the ‘Layers’ dialog box. The dialog allows a choice of colors for displaying each type of object, as well as showing objects in either solid or outline representations. By default pins are turned off, layer and instance objects are displayed solid.
- **EM** - “ElectroMigration Map” - displays results of electromigration analysis for static or dynamic current conditions, based on object view selections in the ‘Layers’ dialog box and the color ranges selected in the ‘ElectroMigration Color Map” dialog. (Instances are turned off by default, and are shown in selected object colors when turned on.)
  The value displayed is the EM current tolerance, which is the computed fraction of

current density in the wire relative to the maximum acceptable EM current density specified in the 'metal' keyword ('EM' parameter) in the **.tech* file. This value must be set for either static or dynamic condition of interest.

- **TRES** - “Transistor P/G Pin Weakness Map” - displays a P/G pin weakness ranking colormap, in colors indicating the relative resistance of all transistor pins from highest to lowest. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Transistor Pin PG Weakness Color Map’ dialog, which allows you to select colors for a range of resistance values as a percentage of the resistance gradient, set the maximum and minimum gradient percentages, and also increase the size of the pins for ease of viewing using the ‘Pin Size Scale’ window.
- **TJVM** - inactive - “Transistor Pin Junction Voltage Movie”
- **ELP** - “ESD Loop Resistance Check for Instances

Second column ‘View Configuration’ buttons

- **EWV** - ‘‘Wire and Via Voltage Map”
- **TARES** - “Transistor Actual Resistance Pin Map”
- **VG** - inactive - “Voltage Gradient Map”
- **EAC** - “ESD Arc Resistance Check Map for Instances”
- **CUR** - “Current Map” - displays results of maximum wire and via current by node for static or dynamic conditions, based on the color ranges selected in the ‘Current Color Map” dialog. Individual colors represent a selected percent of maximum current in the design. Instances are not displayed by default. If displayed, instances are shown in their selected object color.

GUI View Control Toolbar

Right-clicking at the right end of the Menu toolbar brings up a dialog box to display the View Control and Query toolbars. GUI functions available from the View Control Toolbar and Query Toolbar are described below. See the key in Figure B3-8:



Figure B3-8 View control functions key

View Control Toolbar functions are (see Figure B3-8).

1. Zoom In
2. Zoom Out
3. Fit All
4. Last View
5. Next View
6. Redraw
7. Up
8. Down
9. Left
10. Right

11. Pad Enable/Disable
12. Blinking Enable/Disable
13. Blinking Color setting
14. Query Object - displays a ‘Query Objects’ dialog box that allows searching for design objects, specified either by name or by x,y location. Also allows highlighting multiple instances of objects that are listed in a specified file. The highlight color for objects to be found also can be selected.

    You can find information about an object such as a transistor in the query dialog and highlight the desired object in the GUI. Bring up the dialog by clicking on the ‘Query Object’ button. The Query dialog is shown in Figure B3-9 below:

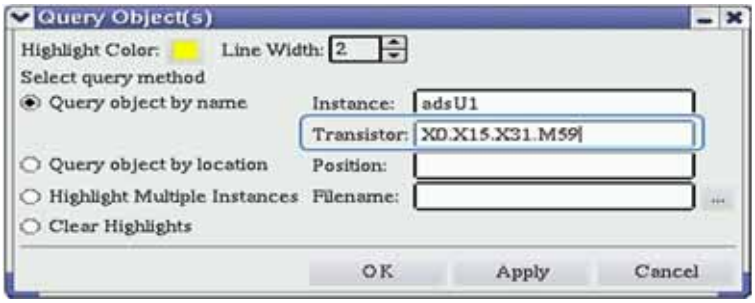


Figure B3-9 Transistor query by name

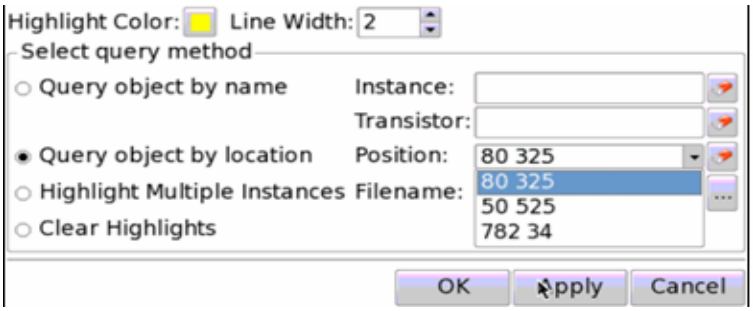


Figure B3-10 Multiple-query object by location

The last three position queries are saved and displayed on a pull down sub-menu, identifying the recently-queried positions, as shown in Figure B3-10. You can select any of the positions from the pull down menu, and then click on Apply or OK to zoom the Totem display to that region.

15. Clear Highlight - clears all highlights and rulers in the display
16. Pre-select Enable/Disable
17. Dump GIF
18. Snapshot - Takes a screen shot of the full **Totem** window and saves it in a GIF file in the selected filename in the dialog box that is displayed. GIF screen shots can be displayed using the ‘xv’ utility in the Linux OS.
19. Switch LSW GDS Layer Display - switches LSW layer display

GDS pre-selected cross-probing

Using the toolbar you can pre-select and select GDS cross-checking in the Totem layout view, as shown in Figure B3-11 below.

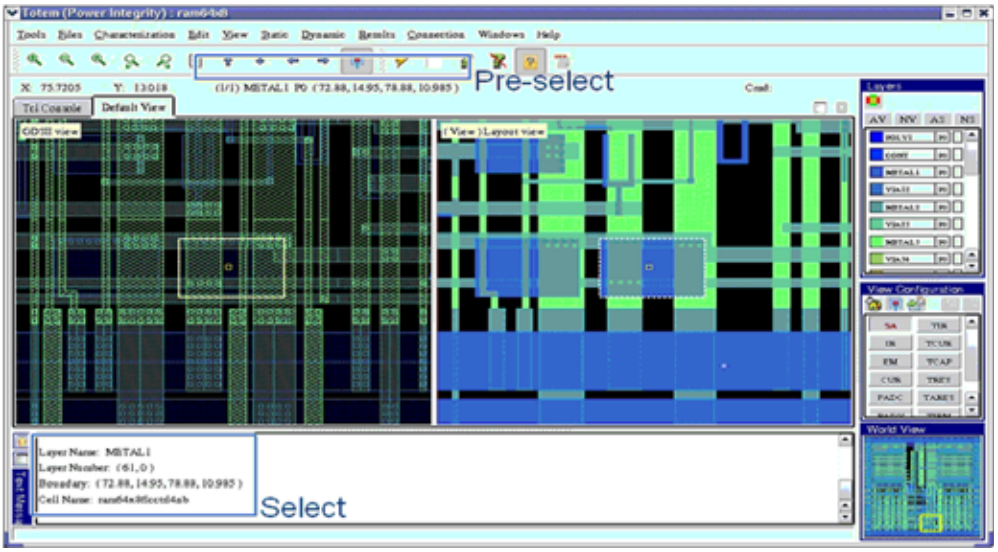


Figure B3-11 Preselection and selection of cross-checking

Status Toolbar

The Status toolbar displays cursor location, DvD, current, or EM information for a pre-selected transistor, wire, or via geometry as you move the cursor on the colormap, as shown in Figure B3-12.

Also the Status toolbar allows configuration of each frame for easy debugging. You can enable/disable Ruler Bar, Scroll Bar, and Frame Title, Sync all views, or only the current view.

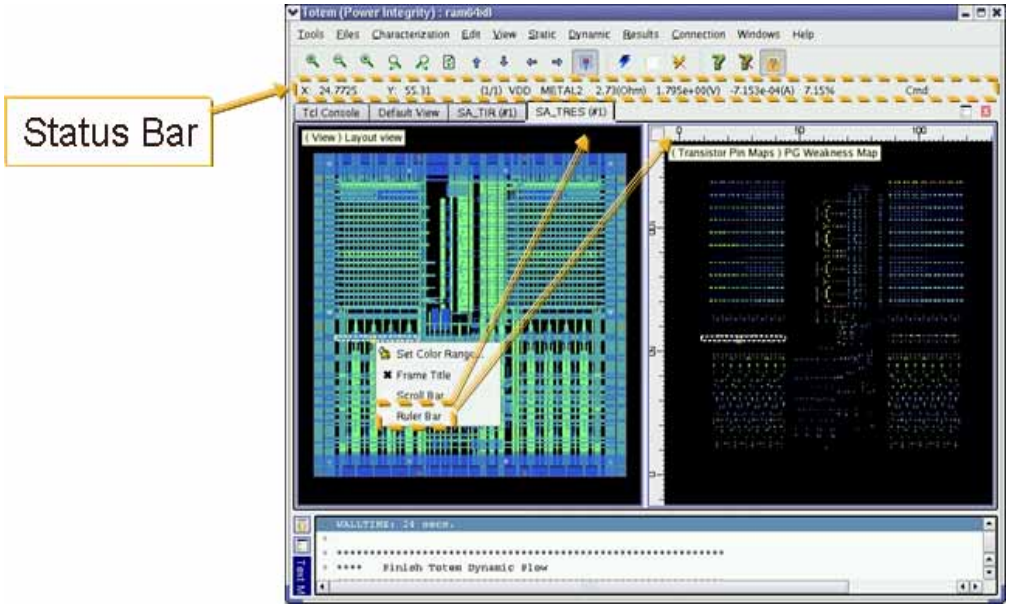


Figure B3-12 Easy frame and Status toolbar usage

Advanced Filter Function for Table Dialogs

Many table dialogs in Totem have a yellow filter wizard icon in the upper right corner, which displays a filter rule setting frame, and allows you to set up multiple advanced filtering rules to investigate and fix design issues more easily and quickly.

You can also set filtering rules column-wise. Each column has a different operation model. For example, for a Netlist (X,Y) column you can set “include/exclude” regions and widths, and the length columns allow you to set arithmetic >, >=, =, <, <= conditions.

After finishing setup, use the “Save As” button to set the filter rule name. Then click the “OK” button to create the filter rule and apply it.

TCL Command Line

To use the TCL command line, click on the **Tcl Console** tab to the left above the Layout View window.

Text Message and Log Window

The text message and log window is below the Layout View window and displays all outputs from Totem.

Exporting Marker Coordinates from “List of ...” Dialogs

You can export marker coordinates of selected bookmark entries in a “List of ...” dialog to a Tcl command file, as shown in Figure B3-13. Then the marker Tcl command file later can be sourced in the Totem Tcl console to highlight the exported markers in GUI display. The procedure is as follows:

1. In the dialog box, click on the line number column to bookmark the entries to be exported.

2. Once all entries are bookmarked, click on the “TCL” icon in the menu bar. This pops up the export file dialog to allow you to define the export Tcl command file name.
3. Click the “Save” button to generate the marker coordinate Tcl command file.
4. Later source this marker Tcl command file in the Totem Tcl console to highlight the bookmarked positions.

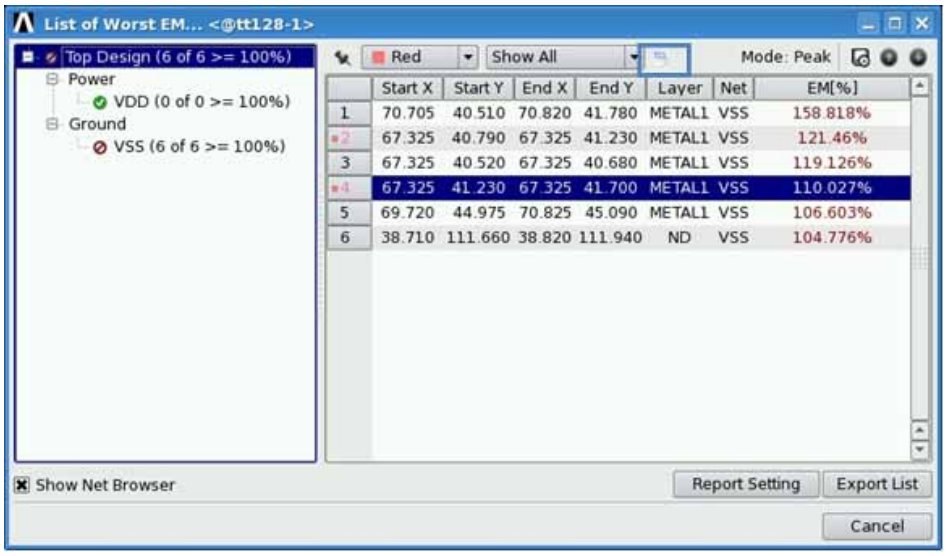


Figure B3-13 Exporting a marker coordinates file from a list dialog

Other GUI Support Features

Middle Mouse Button/wheel Functions

Pressing the middle mouse button/wheel with the cursor within the design brings up a menu with the following functions available:

- **Set Color Range** - see section "“Set Color Range” (left Configuration icon)", page B3-684.
- **Frame Title** - displays View type at top of frame (default)
- **Scroll Bar** - displays the scroll bar
- **Ruler Bar** - overlays a 10 um x,y grid system on the design view
- **Display Nets -** choose “Exclude unselected nets” or “Include all nets”.
- **P2P Resistance Calculation** - displays a P2P Resistance Calculation dialog box to allow choosing ‘From’/’To’ points and other test parameters. See section "P2P Resistance Calculation", page B3-715, for more details.

Bindkey Support (Virtuoso^{tm}-style behavior)

Totem bindkey operation matches Virtuoso bindkey behavior. See section "Bindkey Functions", page B3-741, for a summary and information on reconfiguring bindkeys.

Bindkey behavior relative to the original Totem GUI are:

- Right mouse button zoom
  - Zoom IN - depressing the right mouse button and dragging from *left to right*

creates a rectangular area that is zoomed in.

- Zoom OUT - depressing the right mouse button and dragging from *right to left* creates a rectangular area that is zoomed out.

- Ruler control enhancement
  - Press preassigned bindkey **K** to enable ruler drawing.
  - Press left mouse button to start the ruler (no need to keep button depressed).
  - Move mouse in 45-degree angle to do 45-degree drawing.
  - Draw multiple connected straight-line segments by left clicking the mouse to end one segment and start a new one. This allows total length measurement of multiple segments.
  - End ruler drawing by double clicking the left mouse button, or single clicking the right mouse button.
  - While in ruler drawing, normal Zoom operation is available using the right mouse button, to improve ruler resolution or scope.
  - Use bindkey **Shift+K** to clear all ruler drawing.

User-preferred GUI settings

You can import your preferred default GUI settings by placing the Totem resource file *~/.totemrc* in your home directory, and then including the GUI configuration file in the Totem resource file. You can create the GUI configuration file by exporting it from an existing Totem GUI session using the 'export guiconf' TCL command. Then include the 'import guiconf <gui_configuration_file>' command in the *~/.totemrc* file, and place the *~/.totemrc* file in your home directory.

ESD GUI Menus

The Totem GUI menu functions are summarized in the following sections. Totem Integrates Power integrity MMX, Signal EM, and ESD functions in the Totem platform, providing dynamic switching using the **Tools** menu to a product-specific set of GUI menus, as shown in Figure B3-14.

Tools -> Power Integrity

Selects power integrity setup and analysis mode. See GUI and menu descriptions in Appendix B1, "Power Integrity GUI and Menus".

Tools -> Signal EM

Selects signal EM setup and analysis mode. See GUI and menu descriptions in Appendix B2, "Signal EM GUI and Menus".

Tools -> ESD

Selects Electrostatic Discharge setup and analysis mode, whose GUI features and menus are described in this appendix chapter.

Totem Integrates MMX, Signal EM, and ESD functions in the Totem platform, providing dynamic switching using the **Tools** menu to a product-specific set of GUI menus, as shown in Figure B3-14.

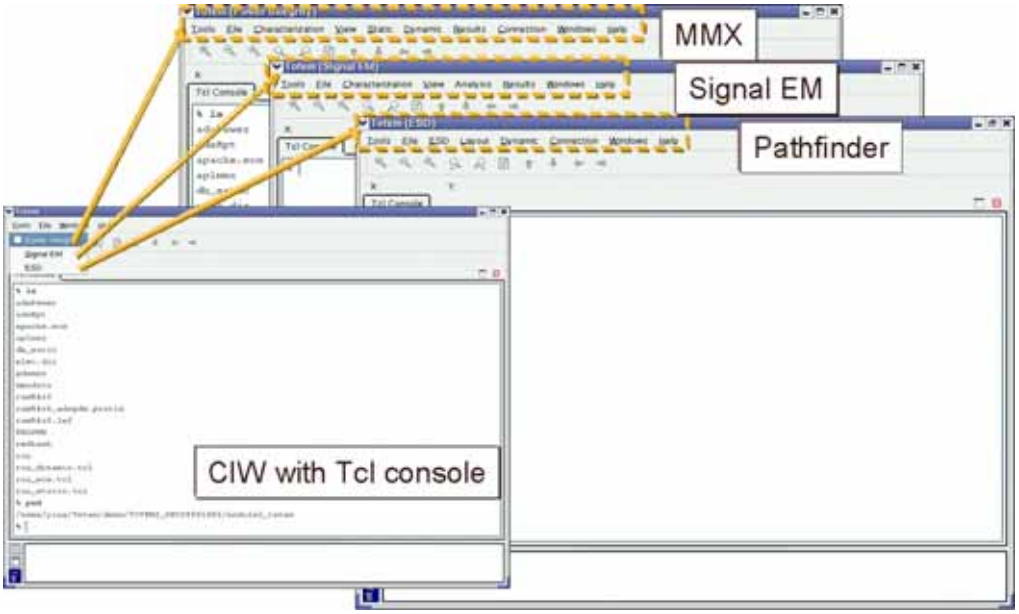


Figure B3-14 Product-specific menus

Files -> Import GDS...

Displays an 'Import GDS' dialog box to specify the GDS file to be imported, as well as the Map File and the Top Cell. The dialog box also has fields for specifying Dictionary, Technology File and Display File.

Importing GDS View through Tcl interface

Totem also allows you to import the GDS file and display it in a GDS View using a TCL command, so you can set up batch TCL commands to import GDS and load the GDS View without using the GUI interface. The procedure to do this is as follows.

From the Totem GUI command line:

```
import_gds -gds <gds_file> [ -map <map_file> -top <cell>|
     -tf <Virtuoso_tech_file> -drf <Virtuoso_display_file>
    -vmap <Virtuoso_layer_map_file> ]
```

or

```
import_gds -cfg <configuration_file>
```

where the configuration file contains the TCL arguments

-gds <gds_file>

-tf <Virtuoso_tech_file>

-drf <Virtuoso_display_file>

…

open_gds -lib <lib_name> -cell <cell_name> -view <view_name>

From a Tcl batch file:

```
gui {
import_gds -gds <gds_file> [ -map <map_file> -top <cell> |
    -tf <Virtuoso_tech_file> -drf <Virtuoso_display_file>
    -vmap <Virtuoso_layer_map_file>  ]
open_gds -lib <lib_name> -cell <cell_name> -view <view_name>
```

```
        }
```

For example:

```
        gui  {
            import_gds -gds AAA.gds -map AAA.gds.map
            -tf AAA.tf -drf AAA.drf
          open_gds -lib AAA -cell top -view layout
        }
```

GDS View integration with Layout Cross-Probing

Allows checking signal routing and cross-probing with GDS view for easy debugging, as shown in Figure B3-15.

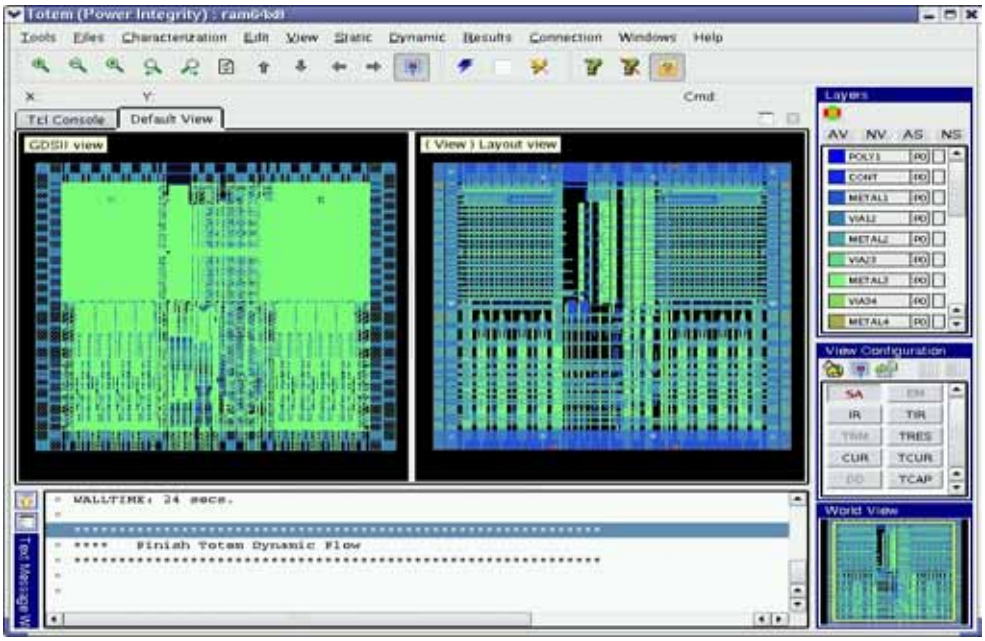


Figure B3-15 GDS and layout view cross-probing

You can import GDS after the ‘setup design’ step with one of the following methods:

1. a. Select the menu command **Files -> Import GDS.**

   or

   b. Click on the **View GDS** button in the **View Configuration** panel (second button from the right).

2. A dialog box is displayed. Fill in the GDS filename, the GDS layer map filename, and the top cell name.

The GDS viewer converts GDSII to GDS view with GDS layer selection capability, as shown in Figure B3-16.

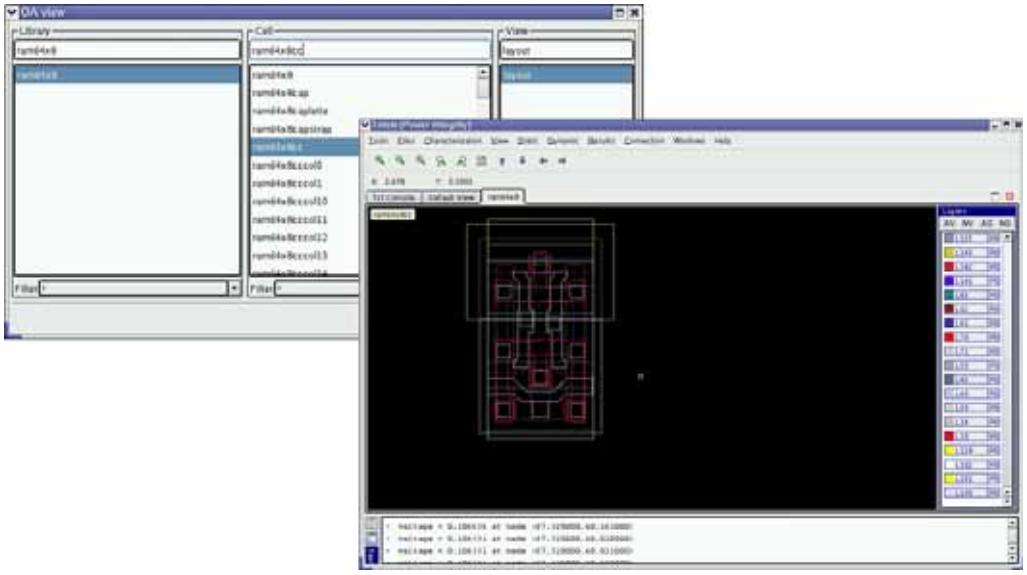


Figure B3-16 GDS view

To import a GDS file, you can select the file from the dialog box displayed after using the menu command **Files -> Import GDS**. Then you can use the command **View ->GDSII View** to bring up a dialog to select a GDS hierarchy to load in the GUI window. You can change the default GDS view setting by bringing up the ‘Preference’ dialog (F10 key).

Files -> Import Netlist...

Displays a dialog from which you can select a netlist file to be imported.

The integration of Netlist Hierarchy View with Cross-Probing Provides easy access to netlist information for debugging, and enables cross-probing with Layout View and other colormaps, as shown in Figure B3-17. When you select an entry in the Netlist Hierarchy view, the corresponding transistor pin is highlighted in the colormap. You can also select a transistor pin in the transistor pin map or SA view with the ‘pinInst’ check box enabled. To enable the pinInst check box, refer to Figure B3-3 on using the Layer Selection Window.

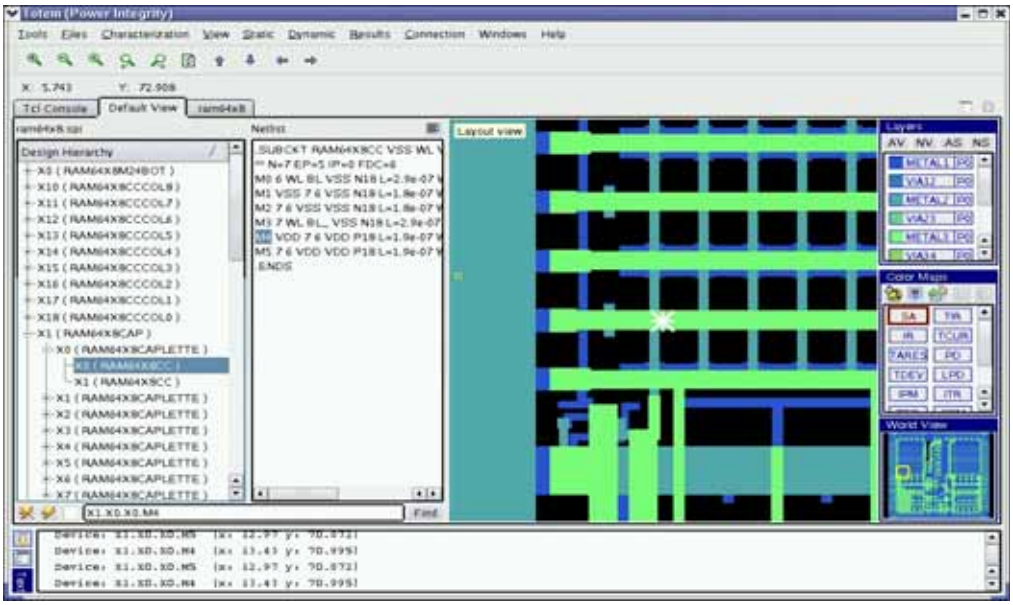


Figure B3-17 Netlist view and cross-probing

To import a netlist and display the netlist hierarchy view, use the command **Files -> Import Netlist**, and fill in the netlist filename, or you can click on **View Netlist** in the 'View Configuration' panel (first button from the right), and specify the netlist filename, which also brings up the netlist view. You can import a CDL format netlist in Netlist Hierarchy View to cross-probe with the Totem database and the Virtuoso Schematic Composer, or the Layout editor.

Files -> Import Design Data...

Displays an 'Import Design Data' dialog box to specify the design data files to be imported. The dialog box has fields for specifying GSR, LEF, LIB, and Apache Tech files for the design. Package definition files PCELL, PLOC, and PAD can also be specified for import. If the correct path to the GSR or LEF files are specified, and the other files have the same path, click the button 'Use Same Prefix' to insert the same path in all file specification boxes. Selecting 'OK' then loads all specified design files.

The menu is grayed out after design setup.

Files -> Import Database...

Displays an 'Import ADB' dialog box to specify the database file to be imported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as 'Create Dir', 'Delete File', and 'Rename File'.

The menu is grayed out after a database is imported.

Files -> Import ESD DB ...

Displays an 'Import ESD DB' dialog box to specify the database file to be imported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as 'Create Dir', 'Delete File', and 'Rename File'.

Files -> Import ECO ...

Displays an ‘Import ECO’ dialog box to specify the ECO file to be imported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Export Flatten GDS ...

Displays a dialog to allow exporting a flattened GDS layout to a specified GDS output file.

The menu is grayed out when there is no flattened view.

Files -> Export Database ...

Displays an ‘Export DB’ dialog box to specify the database file to be exported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Export ESD DB ...

Displays an ‘Export ESD DB’ dialog box to specify the database file to be exported. The dialog box also has fields for browsing directories and specifying database files.

Files -> Export ECO ...

Displays an ‘Export ECO’ dialog box to specify the ECO file to be exported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Import Third Party Technology Data ...

This command allows easy import of third party technology data. For example, in addition to Virtuoso Tech file and display file support, you also can import Virtuoso layer map files. With these three files from the Virtuoso setup environment there is no ambiguity when trying to map the Totem Layer Select Window configuration with Virtuoso to display the same color and drawing pattern. Figure B3-18 below shows the setup dialog.

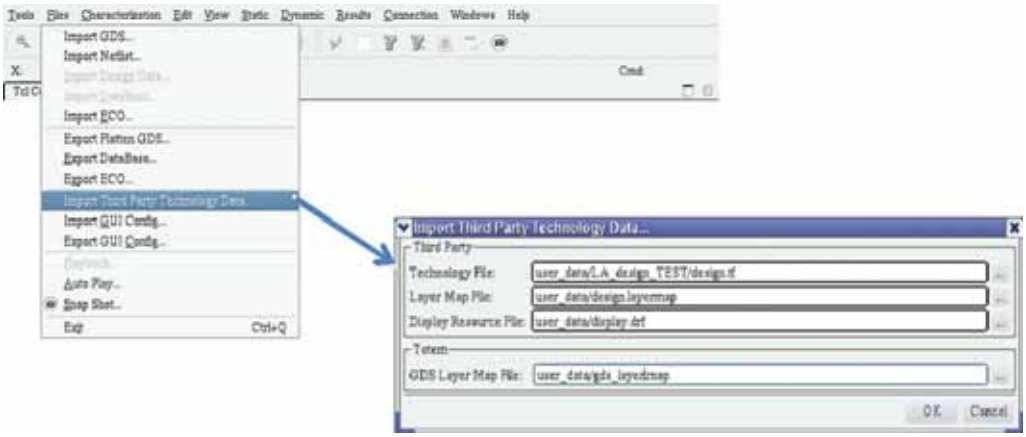


Figure B3-18 Displaying Virtuoso layermap file

Files -> Import GUI Config ...

Displays an ‘Import Configuration’ dialog box to specify the configuration file to be imported. The dialog box also has fields for browsing directories and specifying database

files, and also buttons for directory and file management such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Export GUI Config ...

Displays an ‘Export Configuration’ dialog box to specify the configuration file to be exported. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

Files -> Playback ...

Displays a ‘Load Playback File’ dialog box to specify the Totem session playback file to be loaded. The dialog box also has fields for browsing directories and specifying database files, and also buttons for directory and file management, such as ‘Create Dir’, ‘Delete File’, and ‘Rename File’.

This menu is grayed out after design setup.

Files -> Auto Play ...

Displays a dialog that allows you to specify a a Totem run file to play rerun a previous session in the GUI. Allows adjustment of the timing of GUI operations. Replay script files are located in the *adsRpt/Replay* directory.

Files -> Snap Shot ...

Takes a screen shot of the full Totem window and saves it in a GIF file in the selected filename in the dialog box that is displayed. GIF screen shots can be displayed using the ‘xv’ utility in the Linux OS.

Files -> Exit

Exits Totem session and saves all associated files.

Edit -> Undo

Removes the previous edit step.

The menu is grayed out until editing is performed.

Edit -> Redo

Removes the previous undo step.

The menu is grayed out until an undo step is performed.

Edit -> Add Pad

Displays an ‘Add Power/Ground Pads’ dialog box to specify new power and ground pads. You must select a Power or Ground pad type, the appropriate layer, and click on one or more legal pad locations to add pads.

Edit -> Delete Pad

Displays an ‘Delete Pad’ dialog box with a list of existing power and ground pads, their location and the layer they are on. Specify which power or ground pads on the list should be deleted, and then click on ‘Delete Pad’.

Edit -> Add Strap

Displays an ‘Add Strap’ dialog box to specify new power or ground straps. You can specify the power net (‘Available Nets for power’, ‘Available Nets for ground’), ‘Strap input method’ (‘Add strap by text input’), ‘Horizontal’ or ‘Vertical’ orientation, ‘Strap start’ x,y location, ‘Strap end’ x,y location, and ‘Strap width’. You can add multiple straps by x,y range and pitch, and specify the appropriate metal layers and associated vias.

Edit -> Edit Strap

Displays an ‘Edit Strap’ dialog box to specify modifications to existing power or ground straps. After clicking on the strap to be modified, information about it appears in the dialog box: ‘Net’, ‘Wire’ (layer), ‘Length’, ‘Width’, ‘Strap left lower x’ (x location at lower left end of strap), ‘Strap left lower y’ (y location at lower left end of strap) and ‘Resistance’. When the correct information on the strap to be edited is displayed, the strap can be modified using the ‘Delete’ or ‘Change’ buttons.

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To stretch a rectangle (supports GUI operator “8 control points stretch” component):

a. Select a rectangle and choose **Edit -> Edit Strap**, or use bindkey 's'.

b. The editing coordinate resolution is set based on “X Snap Spacing” and “Y Snap Spacing” settings in the “Preference” dialog.

c. Once the change is done, click on “Change” button in “Edit Strap” dialog to commit the change.

d. Use the “ESC” key to escape the edit command.

e. Bring back previous edit command mode by using right-mouse-button.

f. Once the change is done, click on the “**Change**” button in “Edit Strap” dialog to make the change.

Edit -> Add Via

Displays an ‘Add Via’ dialog box to specify new vias to be added. After clicking on the location for a new via, information about the layers at that location appears in the ‘Select Metal Layers’ panel. After selecting the appropriate layers for the via, the available vias for the selected two layers appear in the ‘Available Via Type Lists’ panel. Select the appropriate via type and then execute the addition.

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To add a via:

a. Select the via layer to edit in the LSW.

b. Use menu command **Edit -> Add Via**. A list of available via models on the selected via layer is shown in the **Add Via** dialog.

c. Select one of the via models; the dashed line bounding box of the selected via model is displayed in GUI display.

d. Use the left mouse button to place the selected via model in the GUI layout window to drop the via.

e. You can select the **Fill** option in the **Add Via** dialog to automatically fill the entire top metal and bottom metal overlapping area by selected a via model.

Edit -> Delete Via

Displays a ‘Delete Via’ dialog box to specify existing vias to be removed from the design. Displaying on vias on particular layers using the ‘View Layers’ button will help select the vias desired. After selecting the via to be deleted, its Net and Via Name are identified in the dialog box. If this is the desired via to be deleted, click on the **Delete** button to execute the command.

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **“**Editing Mode**”** to **“**Mode2**”**

To delete a via:

a. Select the via layer to edit in the LSW (Layer Selection Window).

b. Use the menu command **Edit -> Delete Via**.

c. Select the via geometry in the GUI layout window using the left mouse button. The selected via geometry is displayed in the **Delete Via** dialog box.

d. Click the **Delete** button to delete the selected via.

Edit -> Add Rectangle

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **"**Editing Mode**"** to **"**Mode2**"**

To draw a new rectangle:

a. Select a metal layer in LSW window to activate a working layer.

b. Choose one net from the "Net" selection menu of the "Add Rectangle" dialog.

c. Click the left-mouse button twice to specify a corner of the rectangle.

d. Press and hold the left-mouse button, tracing a box, then release the button.

e. Display and hide the "Add Rectangle" dialog using bind key "F3".

f. Use the Delete (Backspace) key to reset a clicked point.

g. Use the "ESC" key to exit the edit command.

h. The editing coordinate resolution is set using the "X Snap Spacing" and "Y Snap Spacing" settings in the "Preference" dialog.

i. Bring back the default edit command mode using the right mouse button.

j. "Add Rectangle" does not support via layers. If a via layer is chosen for editing, an ERROR pop-up dialog is displayed.

This menu is grayed out in Editing Mode1.

Edit -> Delete Objects

To use Virtuoso-style editing features, open the **Preference** dialog (**F10** button, or **Windows -> Preference**) and switch **"**Editing Mode**"** to **"**Mode2**"**

To delete a rectangle (both vias and wires can be deleted):

a. Select rectangle bounding box to be deleted.

b. Press Delete.

c. Use the "ESC" key to exit the edit command.

d. Bring back the previous edit command mode by using right-mouse-button.

This menu is grayed out in Editing Mode1.

Edit -> Clamp Pin Setup

Add

Displays a Clamp Pin Setup dialog box to allow selection of options for the 'pfs add clamp_pin' command. The 'pfs add' command creates shorted parallel clamp pin regions. Note that extraction must be run after adding or deleting shorted regions for the actions to take place. See the TCL command and options description, page D-969.

You can also add clamp pins using the mouse to select the desired node geometries. With the Clamp Pin Setup dialog open you can either

- Click two corner points with the left mouse button to include the desired pin geometries, or
- Click and drag with the left mouse button to include the desired pin geometries

Both of these pin node selection methods eliminates the need to specify pin node locations one-by-one, and creates a list of nodes that becomes a set of pin statements in the clamp cell syntax that can be mapped to all instances of the cell.

Delete

Displays a Clamp Pin Setup dialog box to allow selection of options for the ‘pfs delete clamp_pin’ command. The ‘pfs delete’ command deletes previous esdcheck results from the database, and also deletes previously-created shorted clamp pin regions. See the TCL command and options description, page D-971.

Highlight

Displays a Clamp Pin Setup dialog box to allow selection of options for the ‘pfs show clamp_pin’ command. The ‘pfs show’ command displays clamp pin shorting information. See the TCL command and options description, page D-972.

Import

Displays a Clamp Pin Setup dialog box to allow selection of options for the ‘pfs import clamp_pin’ command. The 'pfs import’ command is used to import the specified shorting clamp description file that has been previously exported using the ‘pfs export’ command. See the TCL command and options description, page D-972.

Export

Displays a Clamp Pin Setup dialog box to allow selection of options for the ‘pfs export clamp_pin’ command. See the TCL command and options description, page D-971.

Edit -> ESD Clamp ECO

Clamp Cell

Displays an ESD Clamp Cell dialog box in which a cell can be selected and parameters specified for editing. Provides a method of performing the TCL command ‘eco add/delete clamp_cell’ in the GUI interface.

Clamp Instance

Displays an ESD Clamp Instance dialog box in which an instance can be selected and parameters specified for editing. Provides a method of performing the TCL command ‘eco add/delete clamp_instance’ in the GUI interface.

Edit -> Create Ruler ->

After turning on the ruler function, clicking the left mouse button on an x,y location on the design and dragging it to a second point on the design creates an x,y axis with lengths in microns. For easily measuring long distances on a design with precision, press the SHIFT key and click the left mouse button to define the start point and the end point of the measurement. The location of the intersection of the x,y axis and the end point are displayed on screen.

Edit -> Clear All Rulers

All drawn rulers remain displayed until the ‘Clear All Rulers’ button is selected.

View -> GDSII View

After specifying GDS setup in the GSR file, this command brings up a **GDSII View** dialog**.** In the dialog select the Library in the **Library** section, and a list of cells is displayed in the **Cell** column. After selecting a cell to display and the **Show Library Information** box, and Open to display the GDSII view dialog.

If you specify the top cell when loading the GDS View setup dialog, the GDS View library, cell and view selection dialog setup step is automatically skipped and GDS View is displayed directly.

Note that The GDS layer map file input is optional, so you do not need to prepare a GDS layer map file when using Totem as a standalone GDS Viewer.

The viewer also provides LSW cross-probing between Apache Tech layers and GDS layers if you import a GDS layer map file during setup.

The GDS viewer converts GDSII to GDS view with GDS layer selection capability, as shown in Figure B3-19.

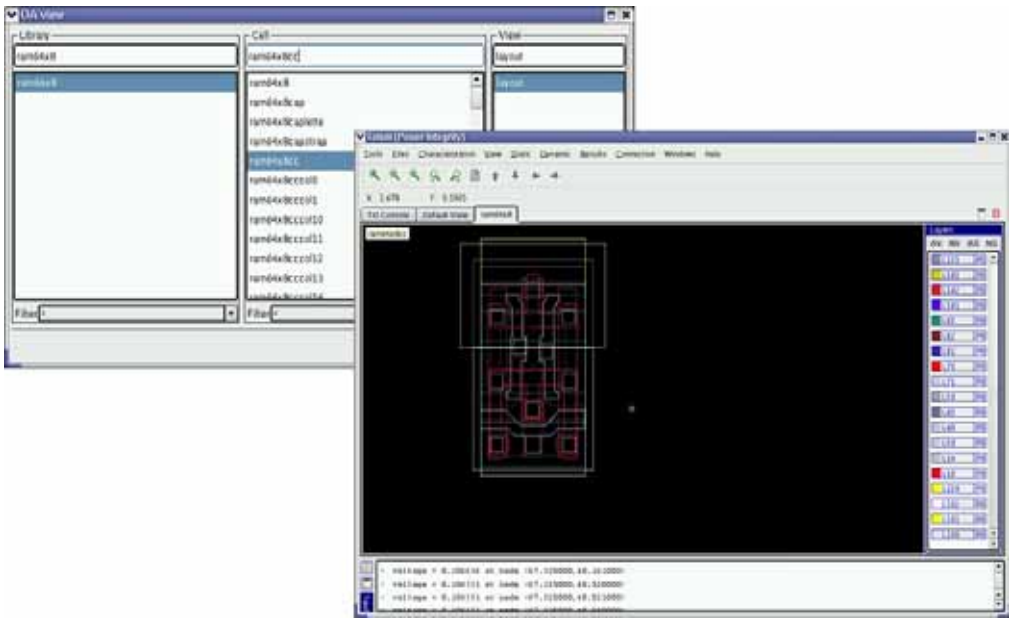


Figure B3-19 GDS view

You can change the default GDS view setting by bringing up the ‘Preference’ dialog (F10 key).

The viewer also provides LSW cross-probing between Apache Tech layers and GDS layers if you import a GDS layer map file during setup. See examples of these special GDS Viewer functions in Figure B3-20 through Figure B3-26.

GDS hierarchy browser

The GDS Viewer has a GDS Hierarchy Browser to allow you to cross-check GDS hierarchies. In the GDS View, click the middle mouse button to invoke the GDS Hierarchy Browser, as shown in Figure B3-20.

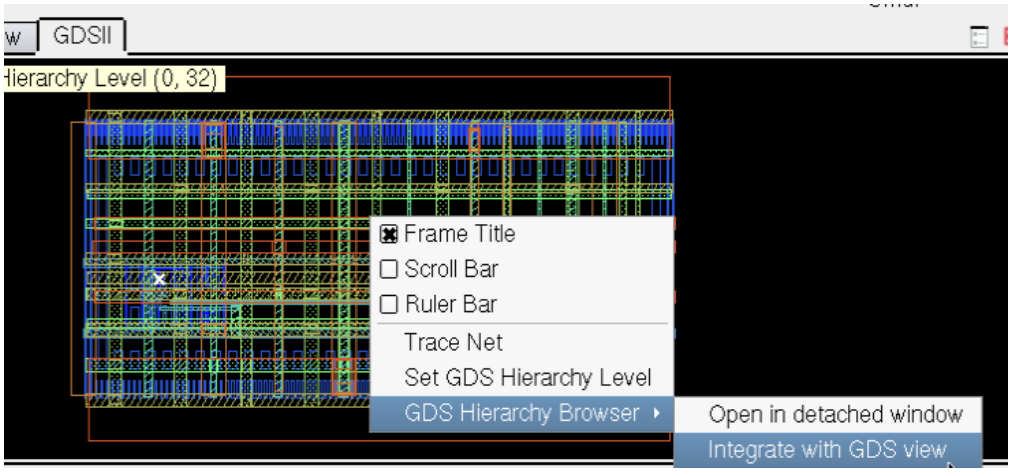


Figure B3-20 Bringing up the hierarchy viewer

You can choose to either integrate the GDS hierarchy browser with the GDS view or view it as a standalone dialog.

The standalone hierarchy display is shown in Figure B3-21.

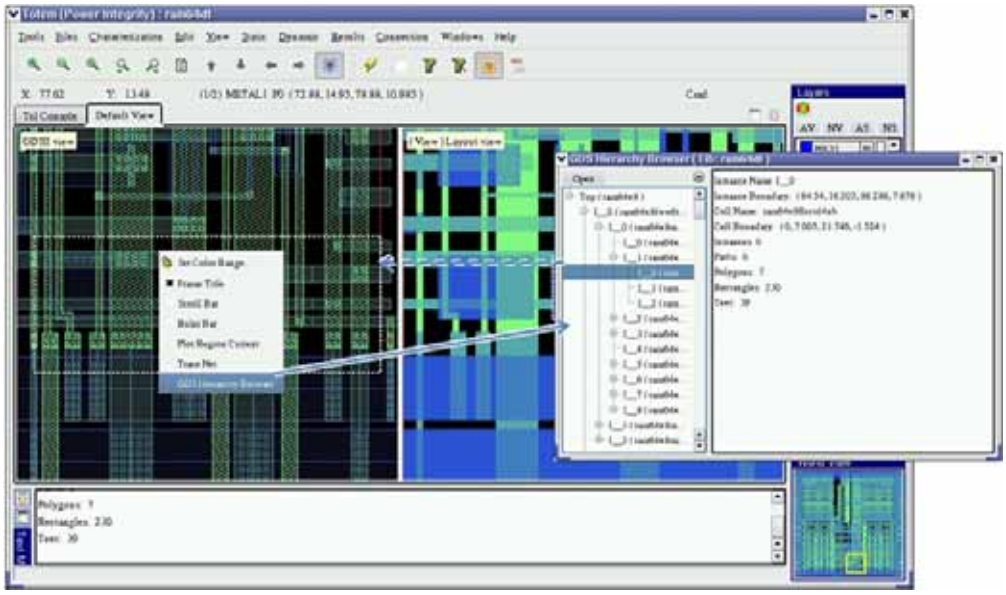


Figure B3-21 GDS hierarchy browser

Figure B3-22 shows the integrated hierarchy browser.

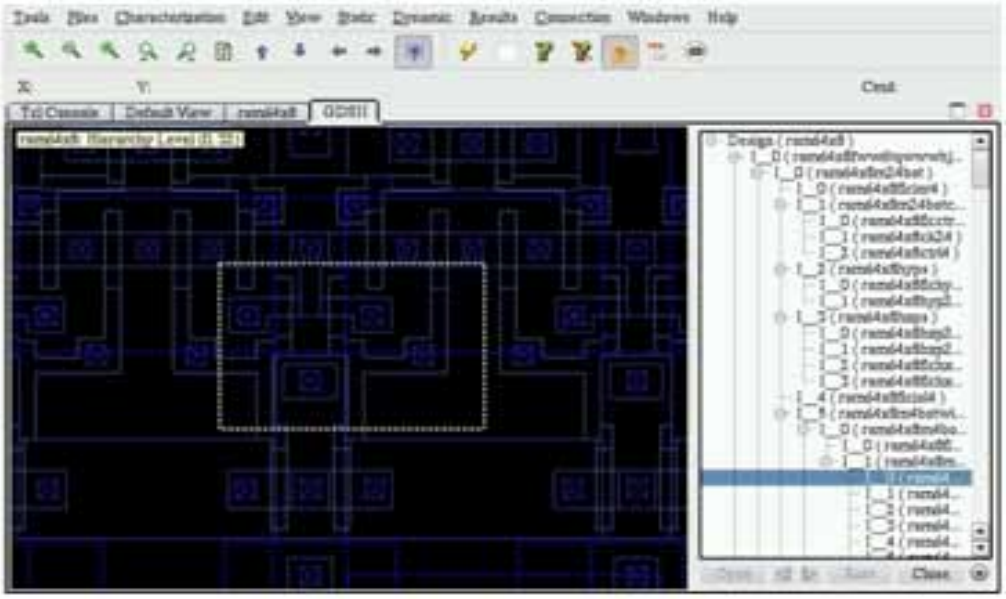


Figure B3-22 Integrated GDS view and hierarchy browser

GDS hierarchy boundary display

To make GDS debugging easier, the Totem GUI allows you to highlight GDS instance hierarchy boundaries. You can then use the bindkey combination **Shift + F** to open the full flattened GDS hierarchy in the GDS view. Use **Shift + V** keys to go down one hierarchy and **Ctrl + V** keys to go up one hierarchy. A display is shown in Figure B3-23 below
:

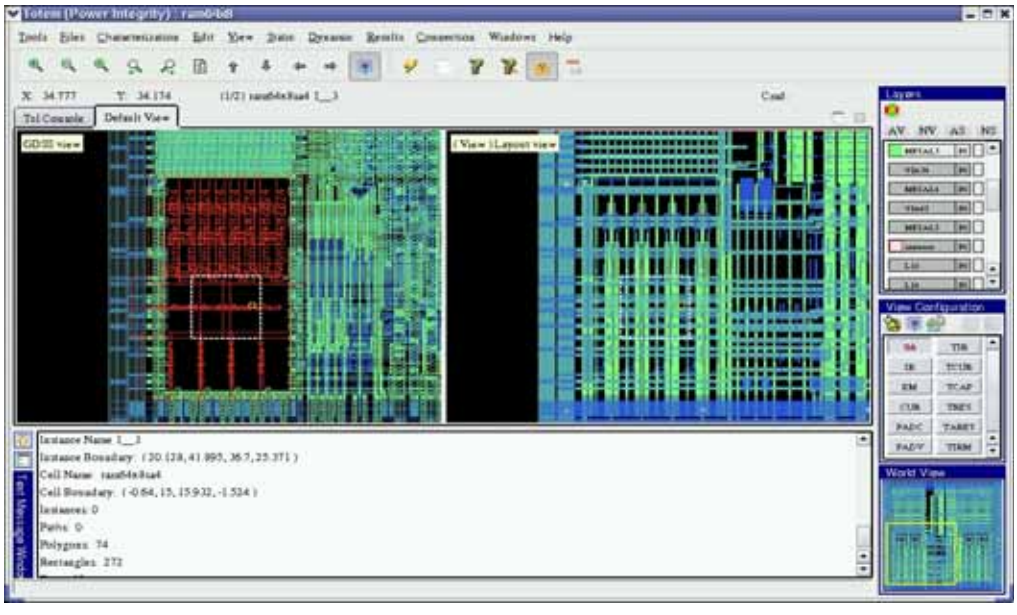


Figure B3-23 GDS hierarchy boundary display

GDS View-based P2P Resistance Checking

You can directly perform point-to-point resistance checking from GDS View, without going through a detailed GSR and TCL command setup, using the following steps:

1. Without doing design setup, go to **Files -> Import GDS,** to load the GDS file and GDS layer map file.
2. Go to **View -> GDSII View** to open GDS View.
3. After GDS View is loaded in the Totem GUI, use the middle mouse button to bring up the dialog box and select the **GDS P2P Resistance Calculation** entry.
4. The **GDS P2P Resistance Calculation** dialog is shown on the screen to allow you to set up a GDS file, GDS layer map file, Apache Technology file, Net name, and **From**, and **To** resistance end points, as shown in Figure B3-24. You can also define a resistance threshold constraint to be checked.
5. You can either manually define the **From** and **To** points in the dialog, or use the mouse to click the geometry objects directly in the GDS View to define the points.
6. Then click the **Apply** button to start the calculation.
7. Totem automatically sets up the GDS configuration file, GSR, and Tcl commands based on your input in the previous steps, and submits a child process to perform GDS2DB tracing, setup design, extraction and resistance calculation.

   Similar to the Save/Reload flow, the same license is shared between child process (setup design to perform resistance calculation) and parent process (GDS View). And all the child process setup is done in a separate sub-directory, under *<working_directory>/p2p<id>*, so you may check all tcl, GSR, configuration setup in the directory to debug.
8. After the resistance calculation is done, you can see the minimum resistance path GUI display between the two selected points. If the resistance constraint is defined during setup, both the effective resistance results and constraint are displayed at the header of the SPT window. If there is a resistance constraint violation, you can continue to use the child process GUI to debug the existing resistance path to identify if there are any design issues.
9. If you want to perform another P2P resistance check on the same net, continue to use the child process GUI to select the next “From/To” points using the middle mouse button pop-up menu, and repeat the process using steps 4 through 8. This can save time in avoiding the GDS trace design setup and extraction.
10. If you are finished with the results and want to quit the child process GUI, Totem automatically returns to parent process GDS View setup.
11. You can also switch to the **Result** page of the **GDS P2P Resistance Calculation** dialog to check the previous P2P resistance check results, or re-submit the previous setup.

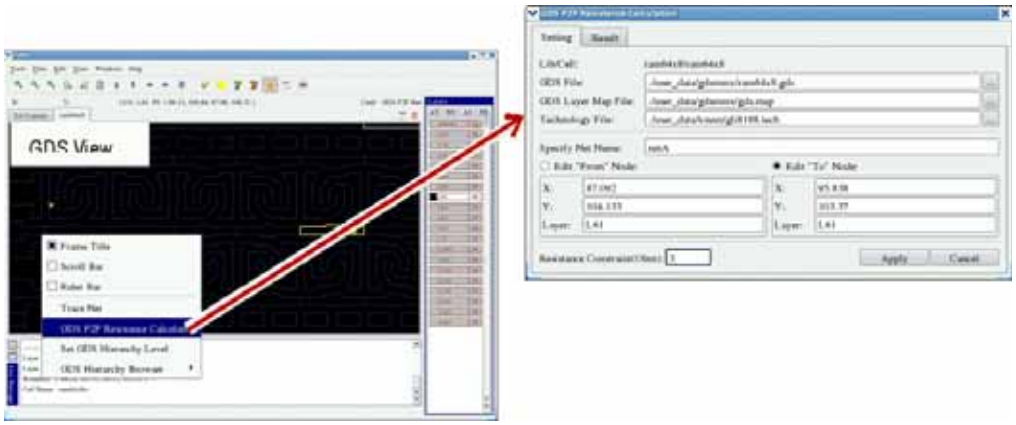


Figure B3-24 P2P Resistance calculation

GDS viewer text search and net tracing

To find the GDS “start tracing” point and debug GDS tracing issues, Totem has text search and net trace capabilities. You can click **Trace Net** on the middle mouse pop-up menu to invoke the Trace Net dialog to search text, or set up GDS layer mapping and trace rules. An example is shown in Figure B3-25 below.

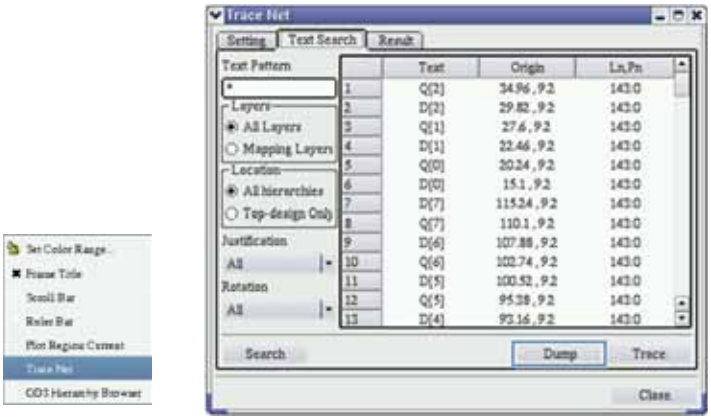


Figure B3-25 Text search and net tracing

You can select text entries in the text search table and click the **Dump** button to stream out GDS “start tracing” points in an output file. Then you can copy and paste the contents of the file into the gds2db configuration file in the VDD_NETS and GND_NETS keywords, to define the selected “start tracing” points.

You can also click on the **Trace** button to start GDS tracing from selected text entries in the table, or click on any geometry in the GDS view to start tracing. The trace result is then displayed on the Result page of the Trace Net dialog, including Visible control, Color control, Delete Control, Trace Progress display and trace stop functions, as shown in Figure B3-26.



Figure B3-26 Net tracing results display

View -> OA Database View

You can use the GUI to view an OASIS library layout view, the same as the GDS view by selecting **View -> OA Database View**, clicking on a lib DEF OA library header file from the list in the dialog, and then selecting one cell to check its layout view, as shown in Figure B3-27.

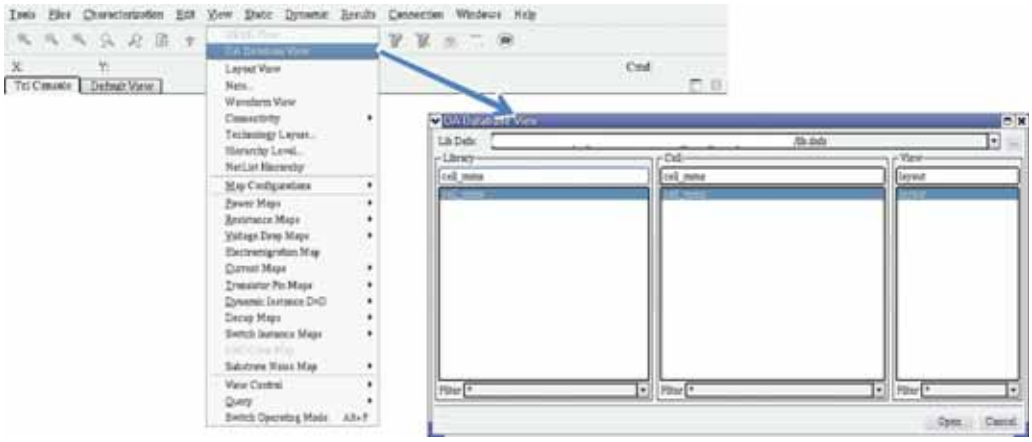


Figure B3-27 OA library layout view

View -> Layout View

Performs the same “show layout view” operation as the **SA** button in the ‘Configuration buttons’ panel. See description above. Does not change the area of the design viewed.

View -> Net Display Map

Displays all nets in different colors.

View -> Nets ...

Displays a ‘Nets’ dialog box with lists of VDD and VSS nets. The dialog allows you to select and highlight particular nets by name or by type: VDD, VSS, Clock, or Signal (if extracted). Nets of a selected type also can be chosen by name to assist in finding a desired net in a very long list. Also displays instances and pins if they are selected in the ‘Layers’ dialog.

In addition to viewing regular nets in the **View -> Nets** dialog, if the design includes metal filler, you can select the check box **Exclude Metal filler** to exclude them, or go to any color view and use the middle mouse pop-up menu to select the **Display Nets -> Exclude metal fill net** check box.

You can also set generic name-based net filtering in the **View -> Nets** dialog to switch “Enable Net By Name” or “Exclude Net By Name” functions to include or exclude net names. Wildcard names are supported.

The check box “GSR only” allow you to only display nets defined in the GSR. Also, internal nets are highlighted in dark blue within the “Nets” selection dialog box to help distinguish internal nets.

View -> Waveform View

In standalone mode, displays the waveform viewer from CIW dialog and imports a waveform file without loading the database. The command supports multi-page, multi-frame displays to provide a more user-friendly debug environment for cross-probing. The waveform viewer supports *.wdb, taO, and *.tr0 files in standalone mode to display and perform waveform operations, as well as case insensitive searching in the signal browser. Copy and paste functionality in the signal browser of the waveform viewer is also supported.

Figure B3-28 below shows the standalone waveform viewer.

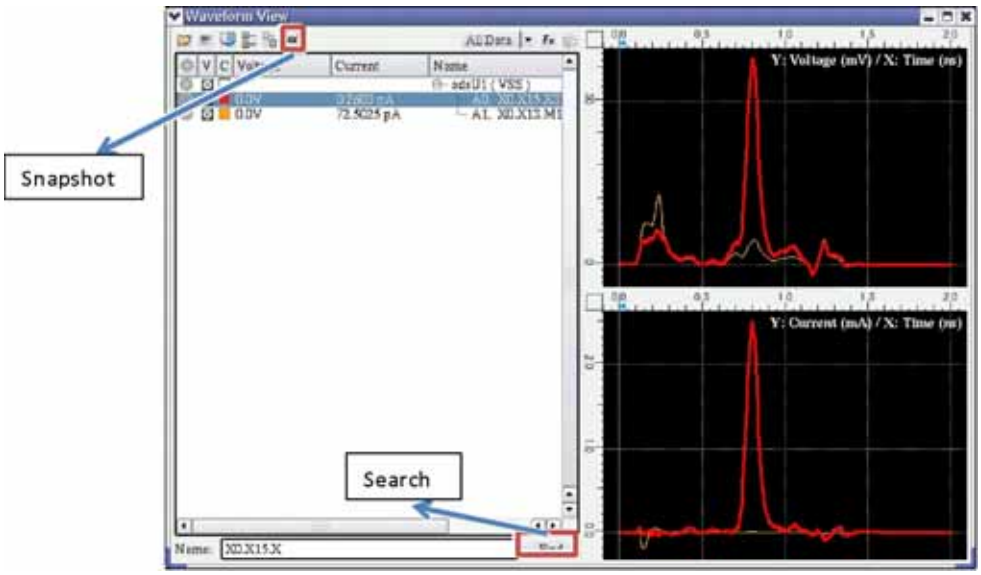


Figure B3-28 Standalone waveform viewer

To help debug a hot spot or other design issue more easily, the integrated waveform viewer dialog allows you to cross-probe transistor pin voltage/current waveforms with other DvD, IR, and EM maps, and movie results, as shown in Figure B3-29.

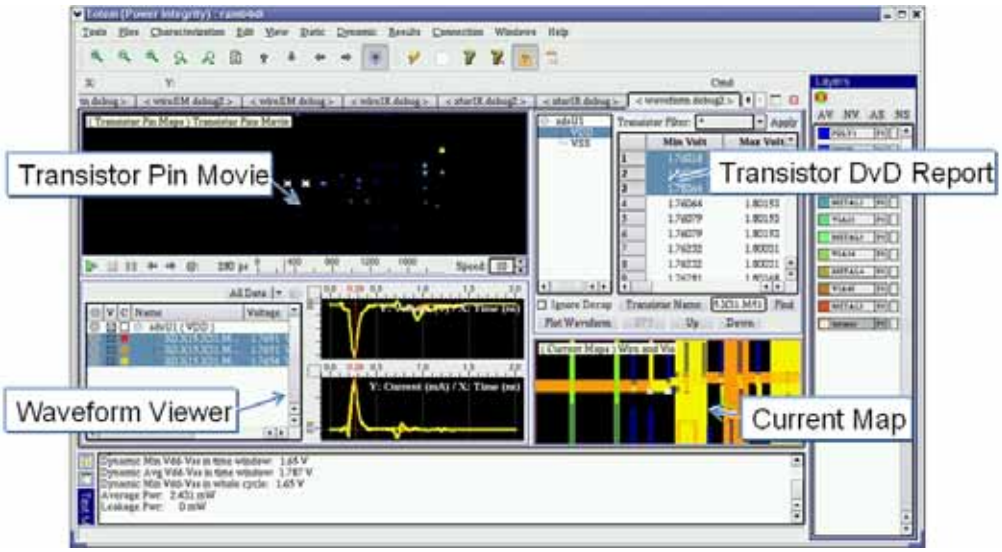


Figure B3-29 Integrated waveform viewer

You can plot two types of waveform views, transistor voltage and current waveforms, and also regional current waveforms, and they can be executed in different ways.

The functions of the waveform viewer are described in the following section.

"Add Page" function

Click on the green "**+**" icon on the left side of the Waveform Viewer dialog to add a new page.

"Delete Page" function

Click on the red "**x**" icon on the left side of the Waveform Viewer dialog to delete the current active page. The default page cannot be deleted.

"Add Frame" function

Except for the default page, you can add new frames on a non-default page. The blue frame icon on the left side of Waveform Viewer dialog allows adding a new frame to the current active page.

"Delete Frame" function

In the frame you want to delete, click the middle mouse button to bring up a pop-up menu and select "**Delete Current Frame**".

Adding a signal in a frame

You can drag a signal from the Signal Browser on the left side of the Waveform Viewer and place it in a selected frame. Newly-added signals are displayed in the header of the frame. If there is more than one signal in the frame, only the symbol of the signal (A0, A1, C0, C1...) is shown in the header. Totem only allows one type of signal (voltage or current) inside a frame.

To select the voltage waveform of given transistor from signal browser, place the cursor at the top of the Voltage column of the Signal Browser to drag. Similarly, you can select a current waveform by putting the cursor at the top of the Current column of the Signal Browser to drag.

To sync the active view page with the signal browser

Click on the "**[View All | View Page]**" button at the top of Signal Browser. You can switch these two modes to see all, or see just the active page signals, in the Signal Browser.

To show, hide, or delete signals

You can select a visible icon in the Signal Browser to show, hide or delete.

Case sensitive/insensitive search

You can toggle the "**[Insensitive Name | Sensitive Name]**" button to control the case sensitivity of the transistor name search function.

To copy the signal name

You can click on the signal in the Signal Browser and use the right mouse button to bring up the pop-up menu and select "**Copy Signal Name**".

Calculation Function

You can use the calculation function in the waveform viewer to perform statistical measurements, such as calculation of average, RMS (root mean square), and peak current within a selected time range, as shown in Figure B3-30. Or you can use the integral function to calculate total charge, as well as perform linear or formula-based waveform operations.

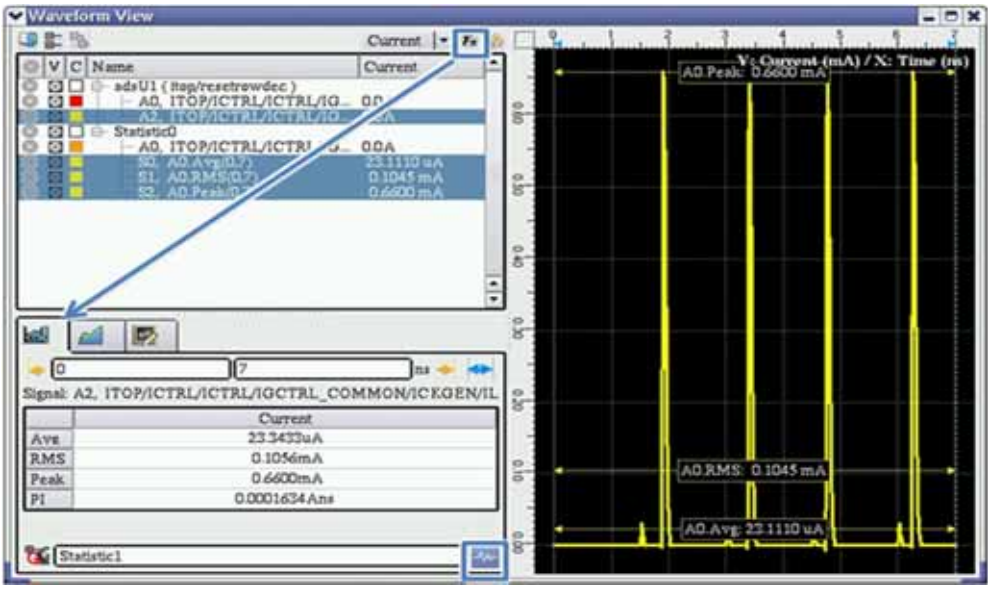


Figure B3-30 Waveform viewer calculation function

To invoke the waveform viewer calculation function, click on the **Fx** button There are three additional sub-pages displayed in the Waveform Viewer, as follows:

Statistics

- To calculate average, RMS, peak, or an integral of the selected waveform within a specified time window.
- To change the start time of the time window, use the left mouse button to click on the time axis, or directly input the value beside the left yellow arrow.

- To change the end time of the time window, use the right mouse button to click on the time axis, or directly input the value beside the right yellow arrow.

You also can click on the **Vis** column to control which measurement functions are enabled. And you can click on waveform **Apply** button in the lower right corner of the page, which displays the measurement results in the waveform view.

Linear

- To apply linear equations to selected signals.
- You can click the **Apply** button in the lower right corner of the page to apply the linear equation and the resulting waveform is displayed in the waveform view.

Formula

- To apply a user-defined formula to selected signals.
- You can click on the **Calculator** button in the top right corner of the page to bring up calculator view, or directly type in the formula in the center input box.
- You can click on the **Apply** button in the lower right corner of the page and resulting waveform from the formula calculation is displayed in the waveform view.

FFT Function

To use the FFT function (see Figure B3-31):

1. Select a signal in the signal browser.
2. Select the **FFT** icon on the top right of the signal browser.
3. Input the start time and end time in the pop-up dialog to define window range for FFT transformation.
4. Adjust the sampling rate and maximum frequency range.
5. Click **OK** to start FFT transformation.
6. Once FFT transformation is done, a new waveform viewer window pops up to display the frequency domain waveform.

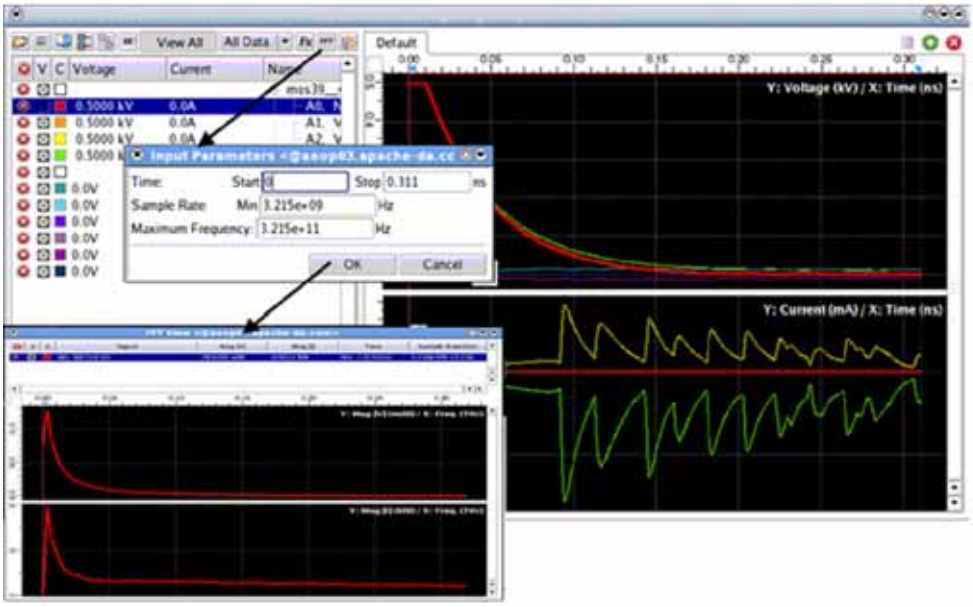


Figure B3-31 FFT function

Examining Waveform Views

Zoom Full. Use the middle mouse button to pop up a menu inside the Waveform Viewer and select Zoom Full, which zooms out to the maximum. You can also use the '**F'** bindkey in the waveform viewer to zoom to Full if the waveform view is not in multi-frame page mode.

Zoom In. Use the right mouse button to drag from left to right, creating a rectangular box defining a zoom-in area, the same as in the regular layout window.

Waveform Zoom-In X-dimension only. Use the right mouse button to draw a horizontal line from left to right indicating the desired zoom range.

Waveform Zoom-In Y-dimension only. Use the right mouse button to draw a vertical line from top down indicating the desired zoom range.

Zoom Out. Use the right mouse button to drag from right to left, creating a rectangular box defining a zoom-out view (zooming out by the ratio of the view size to the drawn rectangle size), the same as in the regular layout window.

Getting Waveform Details. Use the left mouse button to click on the waveform view at a location of interest. A red vertical line is displayed, along with the voltage and current at that time in the left side waveform signal browser. You can click on the header of the Current and Voltage columns to choose maximum or minimum value sorting modes.

Cross-probing Between Layout View and Results Table. When you select a signal inside the signal browser on the lefthand side of the Waveform Viewer, the layout view zooms to the selected region or transistor pin location, and the waveform is highlighted.

Manipulating Views in the Signal Browser. You can change waveform colors, make the waveform invisible, or delete a signal in the signal browser using the available check boxes in the browser. You can delete all signals by clicking on the header of the delete column ("X"). A popup dialog asks for confirmation of the deletion request.

Set x-axis and y-axis range. Resets the waveform view x-axis and y-axis range to help you zoom in or zoom out. You can use the middle mouse button in the waveform view to bring up a popup menu to check this feature.

Restore range to default. Restores the original default x-axis and y-axis range settings

Change waveform curve width, style, symbol. You can use the right mouse button in the signal browser of the waveform viewer to bring up a dialog to reset width, style, and symbol for selected signals.

Plotting region-based current

In any color map or movie, use the middle mouse button to pop up the menu and select **Plot Region Current**. You can either Use Existing Zoom Area, or Define Region by Left Mouse Button. Then plot region current either by using the existing zoom area in the layout window or by using the left mouse button to define a region to plot the current waveforms, as shown in Figure B3-32.

Note that the region current waveform is the sum of all current sources within the chosen region, *not* the current on the wire inside the layout window.

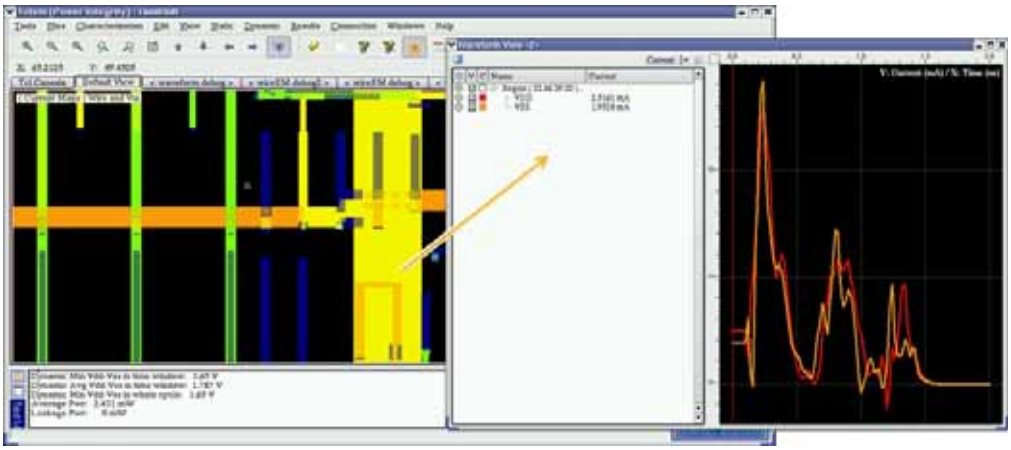


Figure B3-32 Plotting regional current

View -> Connectivity ->

Highlight Disconnected Instances

Turns on highlighting of disconnected instances in the **SA** view, which is off by default. The highlights can be cleared using the same menu command, the **CLR** menu button, or the TCL command 'select clearall'.

This menu is grayed out for designs that have only a single MMX instance.

Highlight Disconnected Wires and Vias

Turns on highlighting of disconnected wires and vias in the **SA** view, which is off by default. The highlights can be cleared using the same menu command, the **CLR** menu button, or the TCL command 'select clearall'.

This menu is grayed out when there are no identified disconnected wires and vias.

Disconnected Transistor Pins ...

To capture potential design problems in MMX blocks, provides a list of all transistors disconnected from the power and ground grid.

Missing Vias ...

Displays a ‘Report of Missing Via Area’ dialog listing all P/G mesh intersections that represent possible missing via locations, including those on 45-degree geometry, compiled by surveying all grid intersections on all layers and noting the intersections that are not connected by vias. It also lists the voltage difference between upper and lower metal layers at potential via sites. The dialog allows you to step up or down the list and select a missing via location. Clicking on the 'Include GDS cell' button and the check box allows you to include a GDS cell. Clicking on the ‘Go to Location’ button highlights the missing via location for closer inspection. The missing vias list is sorted by voltage gradient---the higher the voltage difference between the metal layers above and below the via, the more impact adding this via would have on improving the voltage drop.

The function creates and filters the entries in the missing via report dialog to focus on the most critical violations. You can choose to ignore stack vias, only display hot instances, or even set up a differential voltage threshold across the missing vias to reduce the vias in the list. You can also sort the missing via report using X-coordinates or Y-coordinates. To

do this, you must select the X/Y sorting function in the “Location” column, as shown in Figure B3-33, then use the following steps,:

1. Press the Right-Mouse-Button on “Location (X,Y)” heading.
2. Click
3. the Left-Mouse-Button on “Location Sort By X” or “Location Sort by Y”. Click Apply.

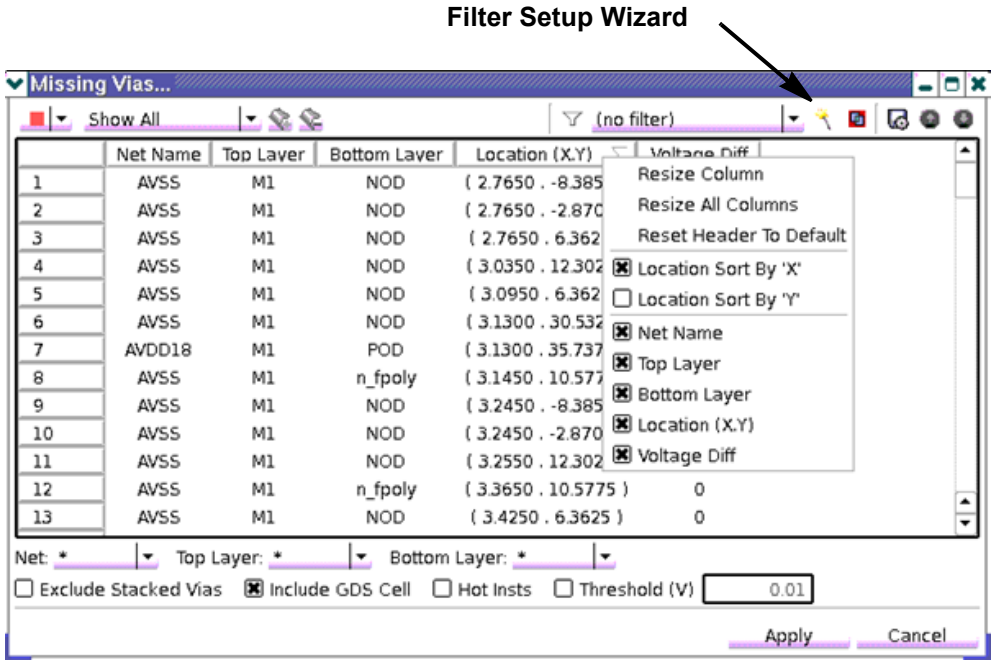


Figure B3-33 Missing Vias dialog

You can also exclude user-defined regions for Missing Via Reports using a file, or by zooming to the region of interest and clicking the “Get Screen Area” option. The GUI zoom-based approach is outlined in the steps below.

1. Click “Filter Setup Wizard” in the Missing Via Report window.
2. Zoom to the region of interest and click “Get Screen Area”, as shown in Figure B3-34.
3. Zoom to the region desired and click the blank square next to the Exclude box, “Get Screen Area”.
4. Click on the “Save Current Setting as New Filter Rule” icon, and select a filter number in the dialog, to save the marked region for future reference, then click “OK”.

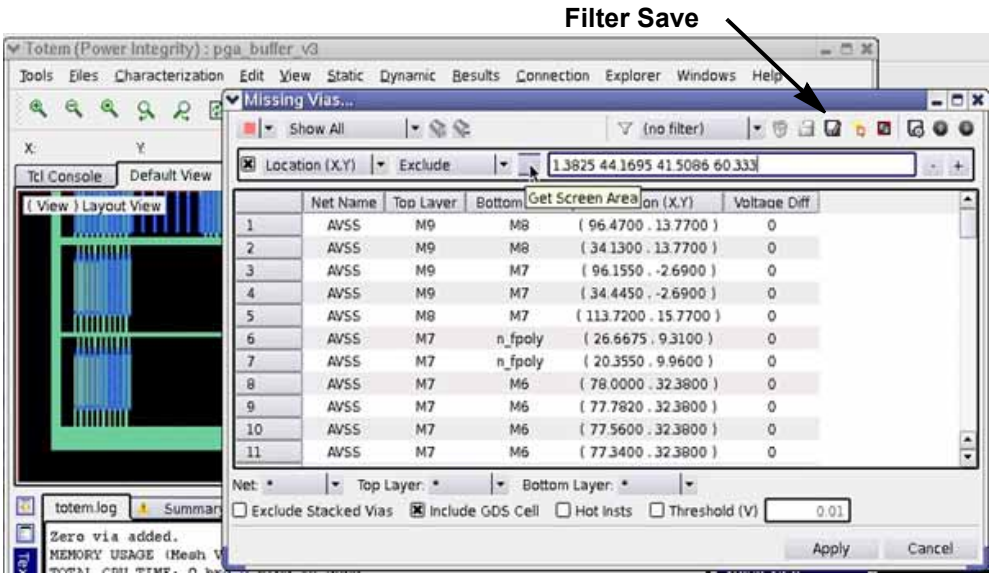


Figure B3-34 Missing Via Exclusion steps

You can easily select discrete entries on the list using the CTRL key + left mouse button. Also using the Shift key + left mouse button you can select a continuous group of listings.by clicking on the first and last entry desired. All selected entries are highlighted in the Totem display

Shorts ...

Displays a ‘Report of Shorts Area’ dialog listing all P/G wires that may be improperly shorted. The dialog allows you to step up or down the list and select a short location. Clicking on the ‘Go to Location’ button zooms to and highlights the potential short for closer inspection.

GDS Shorts ...

Displays information on GDS Shorts.

P2P Resistance Calculation

Executes point-to-point resistance calculations directly by bringing up a setup dialog, as shown in Figure B3-35. Use the left mouse button to click on any geometry in GUI layout or color map view to set up the “From” and “To” nodes for P2P resistance calculation. You also can click on the **Perform Min Res Path** box at the same time to debug design resistance weakness or for ESD analysis.

When a specified point has no node in the same location, you can take the nearest available node within the search radius specified (default 1um), or redo extraction by including nodes at the given point. Totem continues to look outside the radius given by 4N (N being radius specified).

A red region highlighting the search area can be seen in the GUI, along with a yellow marker highlighting the node found (within the 4N radius). If there is no node found within the 4N radius, the option “Use existing extra nodes” is disabled.

A list of P2P resistance calculations matching the search criteria is then displayed.

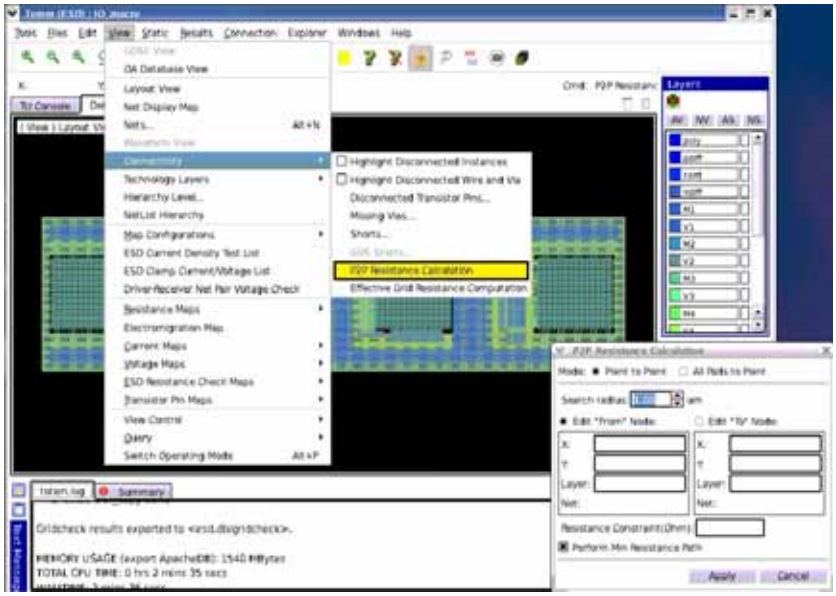


Figure B3-35 Resistance calculation setup

Effective Grid Resistance Computation

Displays an ‘Effective Grid Resistance Computation’ dialog, to select options for executing the grid resistance calculation command.

The computation Modes available are

Arc - one worst-case node per net included (default)

Loop - one node per net, but includes both power and ground nodes

All Points - includes all nodes

All Pins - for MMX designs, also includes all pins internal to transistors (but only one node per pin)

Input data includes:

Inst/Node Count - maximum number of worst-case instances or nodes to be included (default 1000)

Thread Count - number of threads used for parallel R computation

Net Names - allows selection of a limited group of nets to be included. The Customize button allows selection of nets from a dialog box list.

Layer Names - allows selection of a limited group of Layers to be included. The Customize button allows selection of layers from a dialog box list.

Instance Names - allows selection of a limited group of instances to be included.

Cell Names - allows selection of a limited group of cells to be included.

Instance File - specifies a file that contains the list of instances to be included in the computation. The Browse button allows file selection from a dialog box list.

Cell File - specifies a file that contains the list of cells to be included in the computation. The Browse button allows file selection from a dialog box list.

Output File - specifies an output file other than the default, *adsRpt/<designName>.res_calc*. Two types of output can be selected: ”Incremental” means that the new resistance computation does not include nodes previously computed, while “Appended” means that the new output results are appended to the previous output results file.

View -> Technology Layers (Metal and Via)...

Displays a graphic showing the layers in the design and the key characteristics of each, including dielectric constant (e), metal thickness and full layer thickness (h). If you use the right mouse button to create a box over one or more layers of particular interest, the view zooms to include just those layers. Clicking on the ‘Full View’ button then returns the view to the full view of all layers.

Figure B3-36 shows an example Technology view.

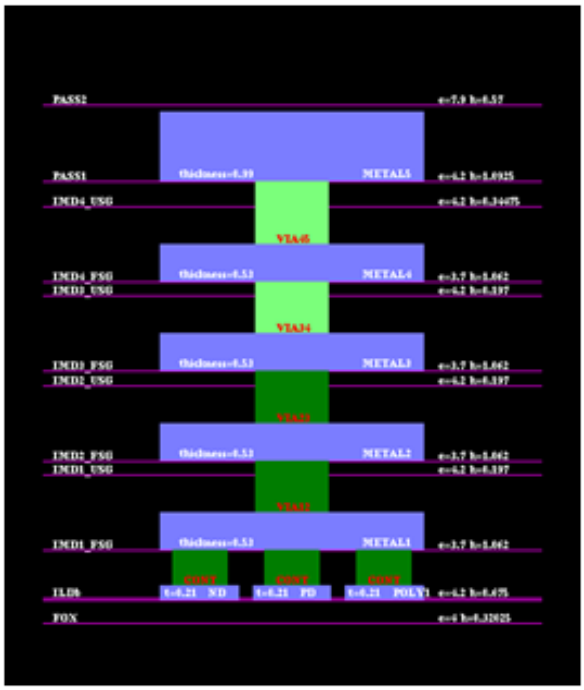


Figure B3-36 Layer characteristics display in the technology view

View -> Technology Layers (Substrate) ...

Displays a substrate Technology View to support SOI/BCD advanced processes for substrate analysis separate from the metal and via technology view. You can use the this command to see “Thickness” and “Depth” attributes. You also can sync the color of the layer with the LSW (Layer Select Window) and expand or collapse the sub-layers for easy viewing and debugging, as shown in Figure B3-37.

This menu is grayed out unless the design analysis includes substrate.

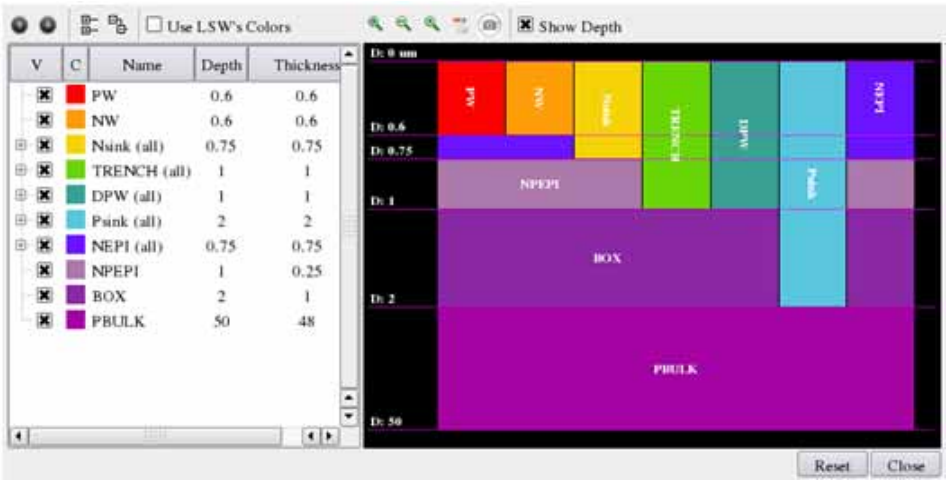


Figure B3-37 Substrate technology layer viewer

View -> Hierarchy Level ...

Displays a ‘Hierarchy Level’ dialog box that allows you to select the number of hierarchy levels for which design details are displayed. Hierarchy levels below that specified are shown as a “black box.”

View -> Netlist Hierarchy

Displays a Netlist dialog box showing the netlist hierarchy.

The integration of Netlist Hierarchy View with Cross-Probing Provides easy access to netlist information for debugging, and enables cross-probing with Layout View and other colormaps, as shown in Figure B3-38. When you select an entry in the Netlist Hierarchy view, the corresponding transistor pin is highlighted in the colormap. You can also select a transistor pin in the transistor pin map or SA view with the ‘pinInst’ check box enabled. To enable the pinInst check box, refer to Figure B3-3 on using the Layer Selection Window.

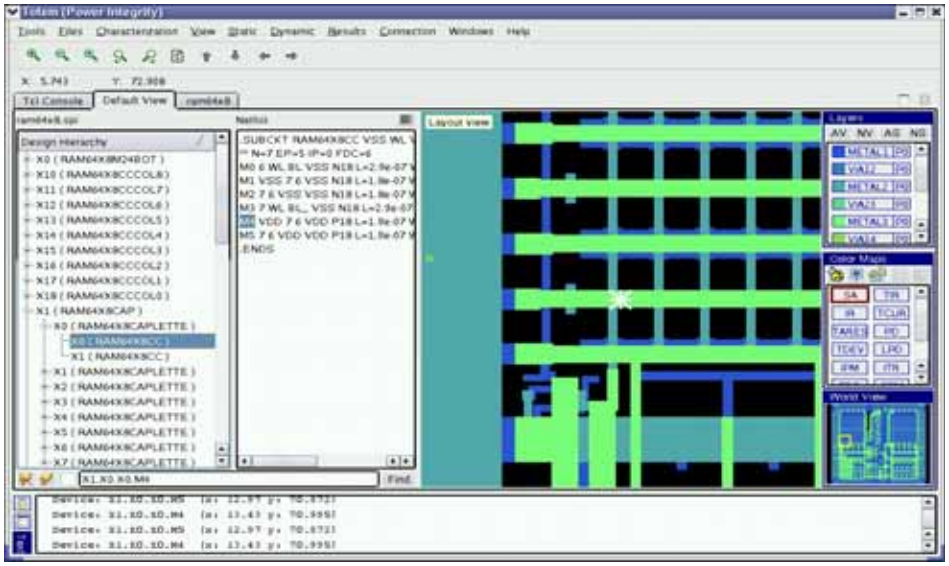


Figure B3-38 Netlist view and cross-probing

To import a netlist and display the netlist hierarchy view, use the command **Files -> Import Netlist**, and fill in the netlist filename, or you can click on **View Netlist** in the 'View Configuration' panel (first button from the right), and specify the netlist filename, which also brings up the netlist view. You can import a CDL format netlist in Netlist Hierarchy View to cross-probe with the Totem database and the Virtuoso Schematic Composer, or the Layout editor.

View -> Map Configurations ->

Layers Color Map

Displays a 'Layers' dialog box, the same as the 'View Layers' button in the 'Configuration' control panel. The dialog allows you to turn on/off and select color and outline or fill characteristics for display of all metal and via layers and the associated pins, as well as instances, in the design.

You can import the Virtuoso technology file, default display color file and a layer mapping rule using the **Files -> Import GDS** menu command to help cross-probe debugging between Totem2 GDS view and SA layout view. Then click on **View -> Map Configurations -> Layers Color Map**. After loading the Virtuoso tech and display file, the Virtuoso-like Layer Selection Window displays grouping information (see Figure B3-39), and GDS and SA views show the same color scheme, as shown in Figure B3-40.

The Layer Select dialog columns allow you to select **Bru** (Brush), **Vis** (Visible), **Fill** (Fill), **Line** (Outline), **Sel** (Select), and **Pin** (PinInst) layer representations. For example, select **Bru** to choose brush layer drawing mode instead of transparent drawing mode. To control Metal/Via layer display, click on the header of each column, as shown in Figure B3-39. Also you can click on the **Vis** column (Visible) to get a popup menu and choose to show all or hide all layers, or you can show all metal or all via layers. The same procedure applies to changing layer display characteristics using the other column headers.

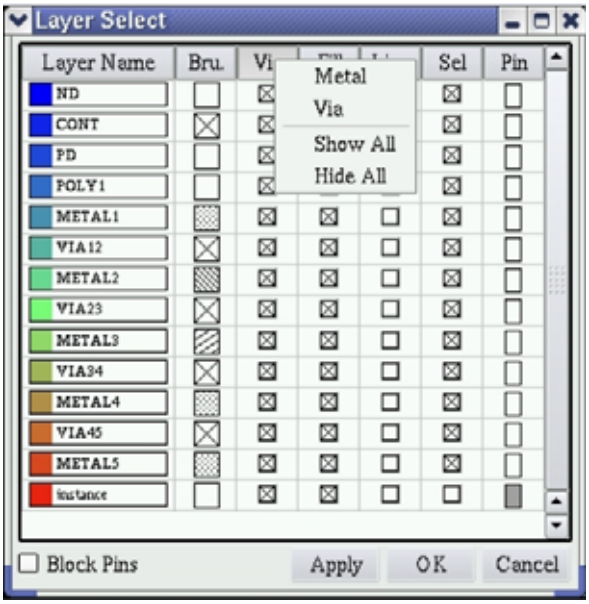


Figure B3-39 Layer select window

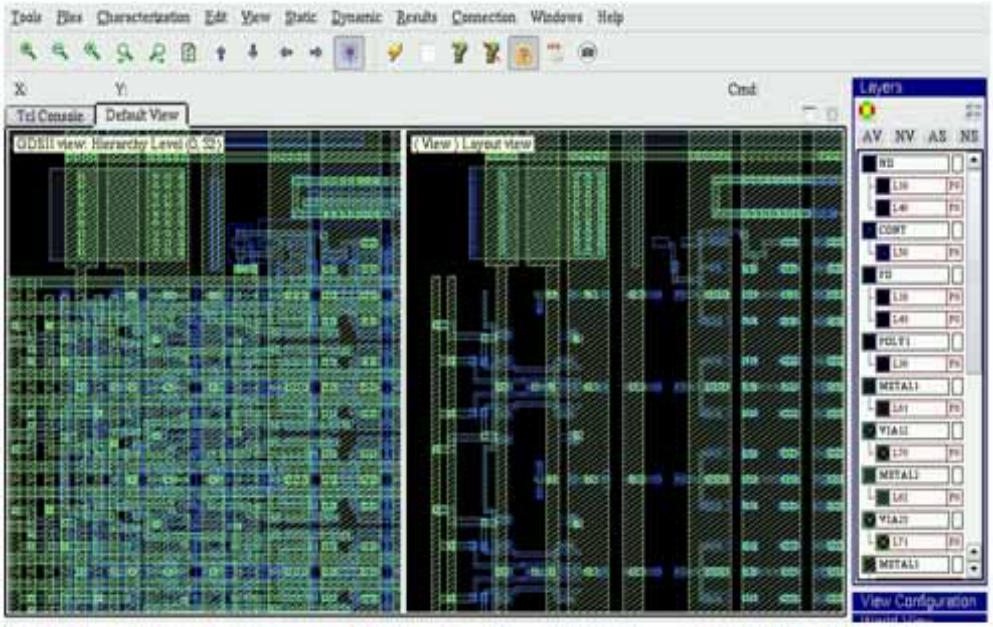


Figure B3-40 GDS view and layout view cross probing

Current Color Map

Displays a ‘Current Color Map’ dialog box that allows you to select colors for a set of current values from minimum to maximum in the design. The default display divides the ranges equally between the minimum and maximum current.

EM Color Map

Displays the ‘ElectroMigration Color Map’ dialog box that allows you to select colors for a range of EM values as a percentage of EM maximum tolerance.

View -> ESD Current Density Test List

Displays a “ESD Current Density Test List.” dialog, including From/To information. This provides better information without going to the reports to see the bump pairs. This also displays all the zaps performed under a Current Density rule. Results from multiple tests can be merged by selecting multiple ESD rule results and then clicking the **Load** button to load the merged results.

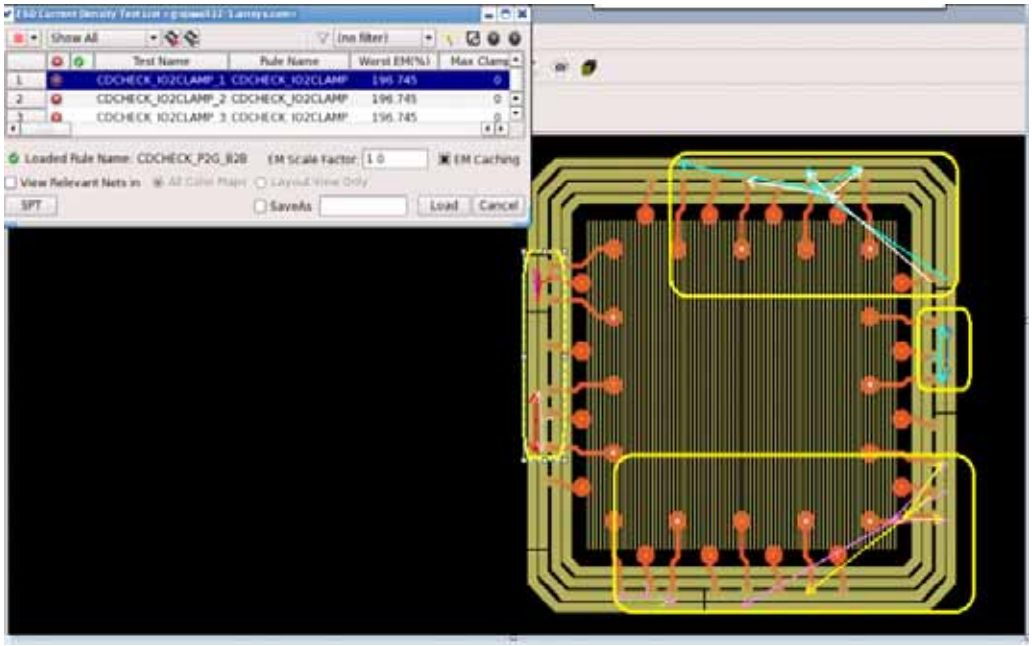


Figure B3-41 View -> ESD Current Density Test List

View -> ESD Clamp Current/Voltage List

Displays an ESD Clamp Current/Voltage List, based of current density B2B checks of clamp devices. Based on the IMAX and VMAX values specified in the ESD rule file, the list classifies clamps as either PASS or FAIL for the specified limits, and also lists From/To pins involved in the checks.

View -> Driver-Receiver Net Pair Voltage Check

Checks and displays a list of driver/receiver net pairs that exceed voltage thresholds checks for ESD current density tests, as shown in Figure B3-42. The dialog allows specifying the maximum allowed voltage difference and the nets to be checked. Node pairs that exceed the specified voltage threshold f are listed. You can then highlight the instances of selected node pairs and draw F-lines between nodes.

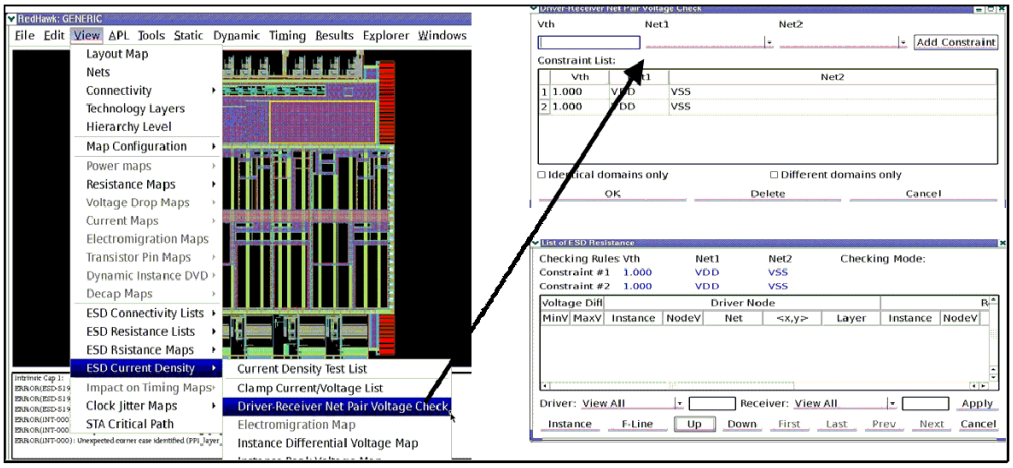


Figure B3-42 Driver-receiver net pair voltage checking

View -> Resistance Maps

This menu is enabled when design has digital place and route blocks for checking grid resistance for instances.

Total Path Resistance

Displays a color map indicating the relative strength of the Vdd and GND grid connections to each instance, computed as a normalized “gradient”. The Total Path Resistance gradient for an instance is calculated as (Rinst - Rmin)/(Rmax - Rmin), where Rmin and Rmax indicate the best and worst grid connections among all instances in the design, and Rinst is the sum of the Vdd and Vss resistances connected to the instance. For instances with multiple Vdd/Vss connections, the sum of the highest Vdd resistance plus the highest Vss resistance connected to the instance is selected as Rinst. In multiple Vdd/Vss designs use the **View-> Nets** command to select the nets to be included in the display. Note that turning off a net that does not represent the maximum Rinst value does not cause a change in the color display. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Resistance Color Map’ dialog, which allows you to select colors for the range of grid strengths as a percentage of Rmax-Rmin. For more details on the Gridcheck report, see the “*RedHawk User Manual”*. (Note that this command previously was called “Total Resistance”.)

VDD Path Resistance

Displays a color map indicating the relative strength of the VDD grids connected to each instance, as the instance VDD resistance gradient. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Resistance Color Map’ dialog, which allows you to select colors for a range of resistance gradient values as a percentage of the global Rmax - Rmin value. (Note that this command previously was called “VDD Resistance”.)

GND Path Resistance

Displays a color map indicating the relative strength of the ground grids connected to each instance, as the instance ground resistance gradient. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘GND Path Resistance’ color map dialog, which allows you to select colors for a range of resistance gradient values as a percentage of the global Rmax - Rmin. (Note that this command previously was called “VSS Resistance”.)

Pin Path Resistance

Displays a color map of pin path resistance (grid check) for every instance pin. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Pin Path Resistance’ color map dialog, which allows you to select colors for a range of resistance values.

Wire/Via Path Resistance

Displays a color map of path resistance (grid check) for every wire/via. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Wire/Via Path Resistance’ color map dialog, which allows you to select colors for a range of resistance values.

Wire/Via Extractor Resistance

Displays a color map of extractor resistance for every wire or via. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Wire/Via Extractor Resistance’ color map dialog, which allows you to select colors for a range of resistance values.

Instance Actual Resistance

For ESD analyses Totem displays Wire/Via Actual Resistance maps to improve visual checking of wire and via effective resistance from the pads, for debugging design weakness. The color map displays the area in which instance resistance calculation has been computed, and continues updating as the res_calc tcl command is executes. You can also query the actual resistance value of any wire/via by clicking on the geometry in the GUI color map, and the log message window displays the actual resistance value. The actual resistance ranking report is also available under **Results -> List of Wire / Via Actual Resistance** after the map is generated.

(Note that this command previously was called “Effective Grid Resistance”.)

The computation Modes available are

- Arc - one worst-case node per net included (default)
- Loop - one node per net, but includes both power and ground nodes
- All Points - includes all nodes
- All Pins - for MMX designs, also includes all pins internal to transistors (but only one node per pin)

Input data includes

- Inst/Node Count - maximum number of worst-case instances or nodes to be included (default 1000)
- Thread Count - number of threads used for parallel R computation
- Net Names - allows selection of a limited group of nets to be included. The Customize button allows selection of nets from a dialog box list.
- Layer Names - allows selection of a limited group of Layers to be included. The Customize button allows selection of layers from a dialog box list.
- Instance Names - allows selection of a limited group of instances to be included.
- Cell Names - allows selection of a limited group of cells to be included.
- Instance File - specifies a file that contains the list of instances to be included in the computation. The Browse button allows file selection from a dialog box list.
- Cell File - specifies a file that contains the list of cells to be included in the computation. The Browse button allows file selection from a dialog box list.

Wire/Via Actual Resistance

Displays Wire/Via Actual (effective) Resistance maps to improve visual checking of wire and via effective resistance from the pads, for debugging design weakness. The color

map displays the area in which effective resistance calculation has been computed, and continues updating as the res_calc tcl command is executes. You can also query the actual effective resistance value of any wire/via by clicking on the geometry in the GUI color map, and the log message window displays the effective resistance value.

View -> Electromigration Map

Displays results of electromigration analysis for static or dynamic current conditions, the same operation as the **EM** button in the 'Configuration buttons' panel.

View -> Current Maps

Wire and Via

Displays the Wire and Via current values in the GUI as a color coded map. To see the Wire and Via current values, click on the Color map Configuration button to display the Current Color Map dialog. You can change the color display and the current tolerance ranges with this dialog. Individual color ranges can be turned off using the left side check boxes. Provides the same display as the **CUR** button in the Configuration buttons panel.

Pad Current Map

Displays a color map indicating the pad currents in the design. The pad colors represent the absolute average current for static IR analysis and absolute peak current for dynamic analysis. A white pad outline indicates a ground pad and an orange pad outline represents a power pad. Click on the 'Set Color Range' configuration button to display the Pad Current Map dialog that indicates the pad current values and allows you to change the colors for each range. The Pad Size Scaling Factor panel allows you to scale (multiply) the size of the pads for better viewing. The Color Entries panel allows you to expand or reduce the number of color steps used to display pad current (default is 10 steps).

View -> Voltage Maps

Wire and Via Voltage

Displays actual voltages for wires/vias after ESD current density checking. You can also zoom in and select a particular wire/via/clamp to see detailed current/voltage values displayed in the Log window.

Instance Peak Voltage Map

Displays the Peak voltage values for instances in the GUI as a color coded map. To see the Peak voltage color values, click on the Color map Configuration button to display the ESD Instance Peak Voltage Color Map dialog. You can change the color display and the voltage tolerance ranges with this dialog. Individual color ranges can be turned off using the left side check boxes.

Instance Differential Voltage Map

Displays the Differential voltage values for instances in the GUI as a color coded map. To see the Differential voltage color values, click on the Color map Configuration button to display the ESD Instance Differential Voltage Color Map dialog. You can change the color display and the voltage tolerance ranges with this dialog. Individual color ranges can be turned off using the left side check boxes.

View -> ESD Resistance Check Maps

Displays color maps showing ESD characteristics, integrated with the list of ESD resistances, so the map varies according to the test selected. In addition, Totem optionally draws flight lines and traces the minimum resistance path for the ESD path selected. You can also change the threshold value of the ESD checks on the fly and then analyze the corresponding ESD map. For each type of color map click on the 'Set Color Range' configuration button to display the associated ESD Color Map dialog, which specifies and allows you to change the colors, the resistance ranges, and the resistance thresholds. The loop count can also be changed, which specifies the maximum number of clamps to traverse between bump pairs.

ESD arc resistance for instance

Displays a color map indicating the smallest clamp to instance arc resistance, with flight lines for each arc.

ESD loop resistance for instance

Displays a color map indicating the smallest clamp to instance loop resistance.

Bump to Inst Resistance

Displays a color map and flight lines indicating the maximum resistance for bump to instance paths, and indicates by color those paths that are less than or exceed the specified resistance threshold (green and red by default).

This menu is grayed out for designs with a single MMX instance.

Bump to Bump Resistance

Displays a color map and flight lines indicating the resistance between bump pairs, and indicates by color those paths that are less than or exceed the specified resistance threshold (green or red by default). The B2B resistance color maps and the associated color map dialog allows you to change the way the B2B resistance check information is displayed.

Bump to Clamp Resistance

Displays a color map and flight lines indicating the resistance for bump to clamp paths, and indicates by color those paths that are less than or exceed the specified resistance threshold (green and red by default).

Clamp to Clamp Resistance

Displays a color map and flight lines indicating the resistance for clamp to clamp paths, and indicates by color those paths that are less than or exceed the specified resistance threshold (green and red by default).

See also section "Resistance Checking for B2B, B2C, and C2C Rules", page 12-369, for more details on these checks.

View -> Transistor Pin Maps ->

Actual Resistance Map

After the 'perform extraction' command, generates a total resistance color map for all transistor pins in MMX/ESD analysis. The resistance report is generated in the file *adsRpt/MMX/<design>.mmx.res*. When this command is executed, a 'Setup resistance map limit' dialog is displayed showing to limit on number of transistor pins to be checked in the TARES map (10,000 by default). To change the maximum number calculated, you can also use the GSR keyword 'MMX_RES_MAP_LIMIT <limit>'. When OK is clicked in

this dialog, the Transistor pin Actual Resistance map is displayed, in colors identifying the actual pin resistance values. If this map is not one of the default buttons in the View Configuration panel, click on the ‘Configure Color Maps’ icon and select the TARES map from the list to be displayed. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Transistor Pin Actual Resistance Color Map’ dialog, which allows you to select colors for a range of resistance values as a percentage of the resistance gradient, set the maximum and minimum resistance gradient percentages, and also increase the size of the pins for ease of viewing using the ‘Pin Size Scale’ window.

The bind key ‘Ctrl+Shift+A’ also can be used to bring up the transistor pin limit dialog for the TARES map. You can also invoke incremental mode to append new results in addition to the results from previous runs. This function is designed to allow you to get a portion of large designs checked more quickly. If you do not click on the ‘OK’ button within 60 seconds, **Totem** resumes with checking Transistor PG pin resistance based on default settings.

PG Weakness Map

Displays a P/G pin weakness ranking colormap, in colors indicating the relative resistance of all transistor pins from highest to lowest. Click on the ‘Set Color Range’ button in the Configuration group to obtain a ‘Transistor Pin PG Weakness Color Map’ dialog, which allows you to select colors for a range of resistance values as a percentage of the resistance gradient, set the maximum and minimum gradient percentages, and also increase the size of the pins for ease of viewing using the ‘Pin Size Scale’ window.

Current Map

Not available in ESD analysis.

View -> View Control

Zoom In by 2

Zooms view in by a factor of 2 (dimensions increase 2 times).

Zoom Out by 2

Zoom view out by a factor of 2 (dimensions decrease by 1/2)

Fit All

Displays full design

Last View

Displays previous view.

Next View

Displays view before previous view.

Redraw

Refreshes present view.

Scroll Up

Moves view up.

Scroll Down

Moves view down.

Scroll Left

Moves view to left.

Scroll Right

Moves view to right.

View 0

GDS Viewer view of top hierarchy level.

View 32

GDS Viewer flattened view of up to 32 hierarchy levels below present view.

View -1

GDS Viewer flattened view of one hierarchy level above present view.

View +1

GDS Viewer flattened view of one hierarchy level below present view.

Display Selected Nets

Displays all nets that are selected.

Display All Nets

Displays all nets.

Show Power Pad

Highlights power pads.

View -> Query

Switch Highlight Mode

Switches highlight mode between blinking and non-blinking.

Set Highlight Color

Displays a dialog box in which to choose a highlight color. Default: white.

Query Object ...

Displays a dialog box in which characteristics of the selected object are listed.

Clear Highlight

Clears highlights from displayed objects.

Preselect Query

Displays object information in the Status box when cursor hovers over an object. Use the space bar to cycle through information for other objects at the same coordinates.

Instance Property

For multiple-instance SOC designs, displays a dialog showing key properties of a selected instance.

Dump Gif

Saves a GIF file of the current layout view in the selected directory.

View -> Switch Operating Mode

Switches GUI style between the new GUI style format and the “original” Totem GUI display format (Alt - p). Note that some available menu commands change when switching between styles, such as the **Windows -> Toolbars** command.

View -> Job Monitor

Brings up a Job Monitor dialog to provide status on running jobs at selected intervals.

Static -> Network Extraction

Displays a ‘Power Ground Extraction’ dialog box to specify the core R,L,C parameters to extract for VDD, GND, CLK and SIGNAL nets.

Static -> P2P Current Density Check

Displays a P2P Current Density Check dialog, from which you can specify a ‘from Node” / to ‘To Node’ current density check on a particular layer, using a specified Rule and Zap Current.

Static -> Resistance and Current Density Check

Displays a Resistance and Current Density Check dialog from which you must specify a ESD Rule File and optionally can specify a Clamp Rule File and an Output Directory. The Number of Threads and Job Count can also be specified.

The output type can also be selected by check box, as follows:

Append Mode : appends results to the output files instead of overwriting the previous output

Incremental Mode : performs incremental instance and macro point selection,

Number of threads : selects number of parallel jobs to be run.

A check box allows the Clamp DB to be saved.

Results -> Log Message Viewer ...

Provides access to four different report viewers described in the following paragraphs: ‘Errors and Warnings Summary’ (default), ‘CPU/Memory Usage’, ‘Setup Design’, ‘Power’ and ‘Results’.

Errors/Warnings Summary

Displays a dual window showing a summary of the total messages generated during the analysis by type (Error, Warning, Info) in the upper window and a selected individual message in the lower window.

CPU/Memory Usage

Displays a table showing maximum Memory use, Total CPU Time, and Wall Time for each design stage from importing files through dynamic analysis. Note that the CPU time for the simulation step is *not* displayed as cumulative for convenience; the following step and all other steps are shown as cumulative CPU time.

Setup Design

Displays a list of files and their paths that have been imported in the design.

Power

Displays a dual window showing a summary of design power information by frequency and by domain in the upper window, and power related warning messages by instance in the lower window.

Results

Displays a simulation results report, such as Worst Dynamic Voltage Drop summary report for dynamic analysis.

Results -> Design Summary Report

Generate

Displays a ‘Report Result’ dialog box to allow specification of a Constraint File or choose the Sample, as well as the Report path. When **OK** is selected the report is generated and the Design Summary dialog box displayed. See section "MMX Design Summary Report", page 6-224, for more information about creating the Constraint File for defining the desired Design Summary Report.

View

Displays the Design Summary Report specified in the **Generate** command. See section "MMX Design Summary Report", page 6-224, for more information about Design Summary Reports.

This menu is grayed out until a Design Summary Report is generated.

Results -> List of Effective Grid Resistance

After performing the ‘perform res_calc -loopMode’ command to compile effective grid resistance and effective instance resistance, this command displays a list of Grid Resistance results.

Results -> List of Effective Instance Resistance

After performing the ‘perform res_calc -loopMode’ command to compile the effective grid resistance and effective instance resistance, this command displays a list of Instance Resistance results.

Results -> List of Pads ...

Displays a List of Pads dialog, presenting a list of pad pin information by name, x/y location, layer, and P/G net. The peak current and the time when peak current occurred after dynamic analysis is reported. Also, a “Plot Waveform” button allows you to plot the current waveforms of selected pads.

Results -> List of ESD Clamps

Displays a List of ESD Clamps dialog, with data on each power and ground clamp by Inst name, Cell name, Loc IDs, x/y locations, Layers, Net names, and Pin names.

Results -> ESD Connectivity Lists

Unconnected Clamp Devices

Displays an Unconnected Clamp Devices dialog showing Cell Name, Pin, x/y location, LocID and Instance Name for unconnected clamp devices. This allows you to examine clamp devices that have connectivity issues.

Clamp Connectivity

Displays a “Clamp Connectivity” dialog, which allows you to review clamp connectivity information, such Cell name, Pin, x/y location, Layer, Loc ID, and Instance Name.

Net Connectivity

Displays a “Net Connectivity” dialog, which allows you to set up net connectivity checks using the GUI interface. You can select Nets to be checked, number of Stages, the Filtering Rule File, and the Output File.

Isolated Bumps

Displays an Isolated Bumps dialog, which lists Bumps not connected to any clamp device, and their x/y Location, Layer and Net.

Results -> List of Wire /Via Actual Resistance...

Displays a List of Wire/Via Actual Resistance dialog after executing the **View -> Resistance Maps -> Actual Wire/Via Resistance** command. The dialog displays a list of wires and vias with their associated layer, net name, and resistance.

Results -> List of Worst EM...

Displays a List of Worst EM dialog showing a net hierarchy browser on the left side (default), and also a list of worst EM calculations, with wire x/y Start and End locations, Layer, Net, and EM%. The order of the EM items can be changed with the up and down arrows. The ESD Rule used for checking is also displayed.

Results -> List of ESD Static Rule Checks

Displays a dialog to select B2B resistance check results. Choose “Bump Centric” or “Clamp Centric” from the top of the dialog box, as shown in. When you select “Bump Centric”, the B2C result table shows the report on a per bump basis and displays the best arc value for each bump. You can click on each bump entry to expand all arcs related to this bump in the lower table.

Similarly, when you select “Clamp Centric”, the B2C result table shows the report on a per clamp basis and displays the best arc value of each clamp. Users can click each clamp entry to expand all arcs related to this clamp in the lower table.

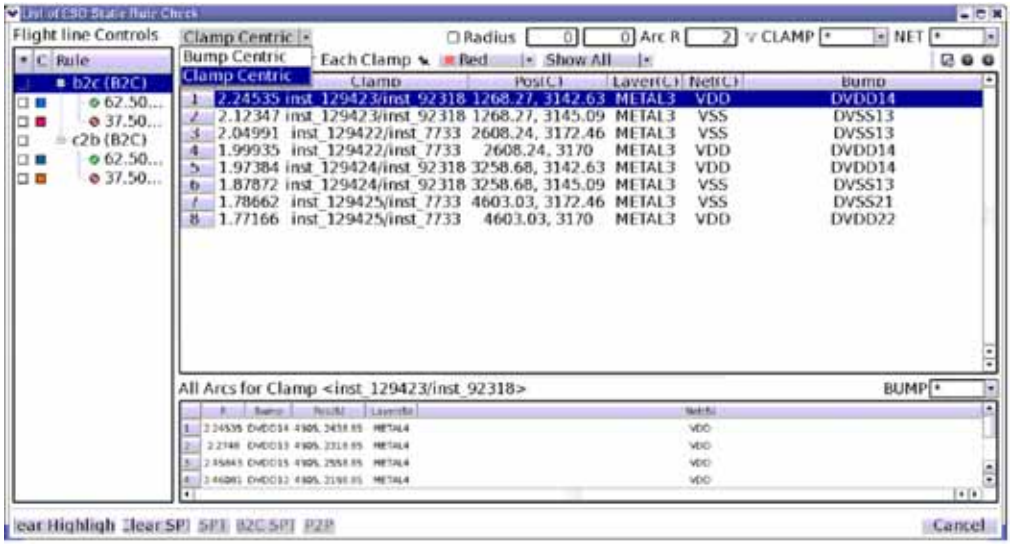


Figure B3-43 List of ESD Static Rule Check dialog

Bring up the color range setup dialog, or click on the color setting check box of the B2B ESD rule in the left side browser. You also can assign different colors for flight lines based on resistance threshold, which allows you more detail about the margin of violations. Go to Pad and Flight Line color Control frame to change the display from **Pass/Fail Drawing**

to **Spectrum Drawing,** as shown in Figure B3-44, and then fine tune the resistance range for each color.

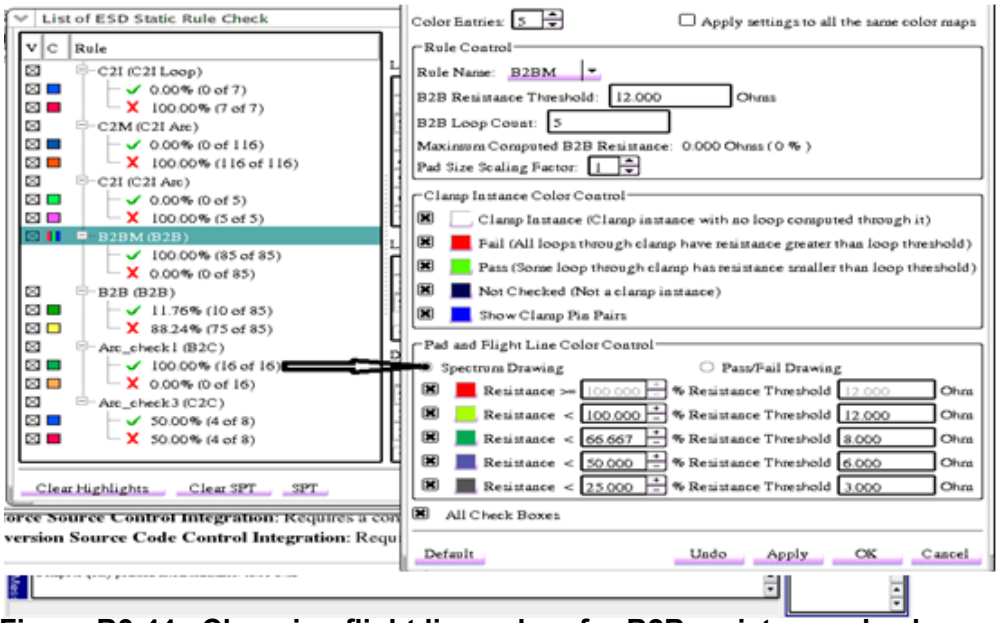


Figure B3-44 Changing flight line colors for B2B resistance checks

Results -> List of ESD P2P Resistance

Displays a list of point-to-point resistance checks when you specify an ESD P2P resistance check using a “Type R” rule. The arc resistances and the From and To locations are then displayed in a list based on the ESD rules specified (see Figure B3-45).

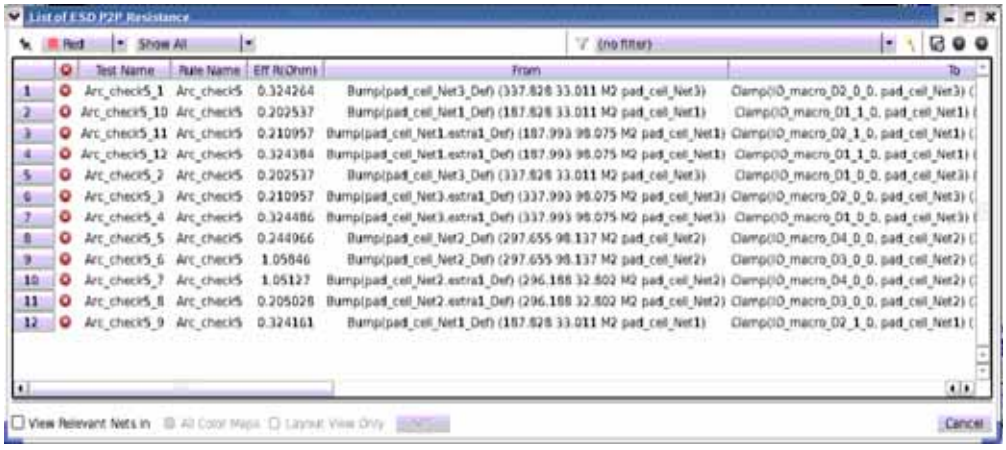


Figure B3-45 List of P2P Resistances

Results -> Analysis Histograms

Displays an ‘Analysis Histogram’ dialog box to specify what parameters to display in the histogram: Dynamic Voltage Drop, Static IR, Static EM, or Dynamic EM. Histograms can be constructed ‘By Layer’ selection, or ‘By Instance’ for criteria

- Average/Max/Min DvD over Timing Window,
- Minimum DvD in Simulation Cycle,
- Max Vdd Drop, or
- Max Vss Bounce

For multiple Vdd/Vss domain designs, histograms can be displayed by domain Net Name.

‘Limits’ boxes provide automatic selection of ‘Lower’ and ‘Upper’ mv limits, as well as a choice of bin size or number, or values for all these display parameters can be selected.

To display analysis data for elements only within a specified bounding box area, use the TCL command 'condition set -xy <x1 y1 x2 y2>', where x1,y1 and x2,y2 are the corners of the desired bounding box to be analyzed.

Connection -> Connect to layout editor ...

Sets up connection to a third-party layout editor, as described in section "Approach 6: Find ESD/Fuse cells that cause intentional shorts", page 3-150.

Totem supports cross-probing to Cadence Virtuoso and Springsoft Laker layout editors and Virtuoso Schematic viewers. To set up these capabilities, follow the procedures described below:

Cadence Virtuoso cross-probe setup

1. Virtuoso *.cdsinit* file setup:
   a. Edit the *.cdsinit* file to load Apache skill code program.
   b. Append file *$APACHEFILE/lib/crossprobe/cdsinit-sample* in your *.cdsinit* file. The *.cdsinit* file could be in the (1) current working directory, (2) user's home directory, or (3) Virtuoso installed/setup directory.
2. Source the Virtuoso resource file *cadence.rc* or *virtuoso.rc.*
3. Edit and source Totem resource file *totem.rc.* You can find the *totem.rc* template file under *$APACHEROOT/lib/crossprobe/totem.rc*.

A cross-probing icon on the View Control toolbar turns Virtuoso cross-probing On and Off. This icon is only enabled and displayed when the Virtuoso-to-Totem proxy cross-probing mode is used.

Totem-Virtuoso cross-probing/analysis can be invoked on a different machine for performance/capacity improvement using industry standard interfaces. Currently only LSF and SSH interface is supported. The usage procedure is:

1. Set the environment variable TOTEM_SETUP_WIZARD_DIR to your copy of the Totem wizard configuration directory.
2. Add entries to the job submission configuration file at:
   `$TOTEM_SETUP_WIZARD_DIR/wizard/wrapper/wrapper.ini`
3. Set the environment variable APACHE_CROSS_PROBING_WRAPPER_SET to the user-defined entry.

Springsoft Laker layout editor cross-probe setup

1. Source the Laker resource file *laker.rc*
2. Edit and source the Totem resource file *totem.rc.* You can find the *totem.rc* template file under *$APACHEROOT/lib/crossprobe/totem.rc.*

Required environment variables

Inside the *totem.rc* template file, you must set up the following global environment variables:

A. Cross probing

```
setenv APACHE_CROSS_PROBING_LAYOUT <layout_editor_binary_name>
setenv APACHE_CROSS_PROBING_SCHEMATIC <schematic_binary>
```

Specify the 3rd-party binary name.

B. Design setting

```
setenv APACHE_CROSS_PROBING_DBPATH <design_database_path>
```

Specifies the directory containing *cds.lib* (Virtuoso library header file) or Laker layout database directory, which contains l*aker.rc* file.

```
setenv APACHE_CROSS_PROBING_TOPDESIGN <top_cell_name>
```

Specifies the design hierarchy/cell name in this variable to allow Totem to find the correct hierarchy in the layout design database.

```
setenv APACHE_CROSS_PROBING_LIBRARY <library_name>
```

Specifies the layout design library name to allow Totem to invoke the 3rd party layout editor and load the correct design library (there may be multiple design libraries for the same hierarchy).

Replay mode setup variable

```
setenv APACHE_CROSS_PROBING_REPLAY_MODE y
```

Forces automatic entering of the replay mode for connection setup. This variable is *required* for Laker, and optional but recommended for Virtuoso.

Cross-probing port setting variables

```
setenv APACHE_CROSS_PROBING_WAITTIME <connection_timeout_limit>
```

Specifies connection time-out limit to 3rd party layout editor. Default time is 30 seconds.

```
setenv APACHE_CROSS_PROBING_STARTPORT <startport_id>
setenv APACHE_CROSS_PROBING_PORTRANGE <range_number>
```

Specify the connection socket port range. Totem starts to search available ports in order from <startport_id> and stops searching when reaching a port ID larger than (<startport_id> + <range_number>). Default ports are 11000 for APACHE_CROSS_PROBING_STARTPORT and 5000 for APACHE_CROSS_PROBING_PORTRANGE

After setting up cross-probing with the above procedure, use the following steps to perform cross-probing investigations in Totem.

1. Choose the GUI menu command **Connection -> Connect to layout editor**. Totem displays a dialog window to allow you to modify layout design variable settings, as shown in Figure B3-46.

2. After clicking the **OK** button, Totem attempts to connect to the selected 3rd party layout editor. If the socket connection can be made within the time-out limit, the 3rd party layout editor GUI is displayed.

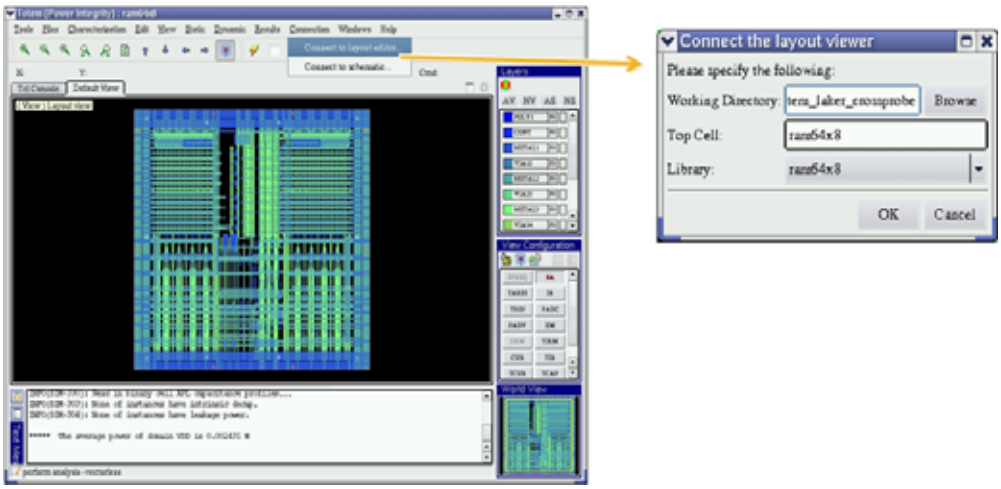


Figure B3-46 Making the cross-probing connection

Totem layout cross-probing capabilities include:

- Zoom-In / Zoom-Out
- Highlighting geometry in the layout window
- Zooming to a selected transistor location from a GUI list of key transistor locations (see Figure B3-47 and Figure B3-48). Use the command **Results -> List of Worst**

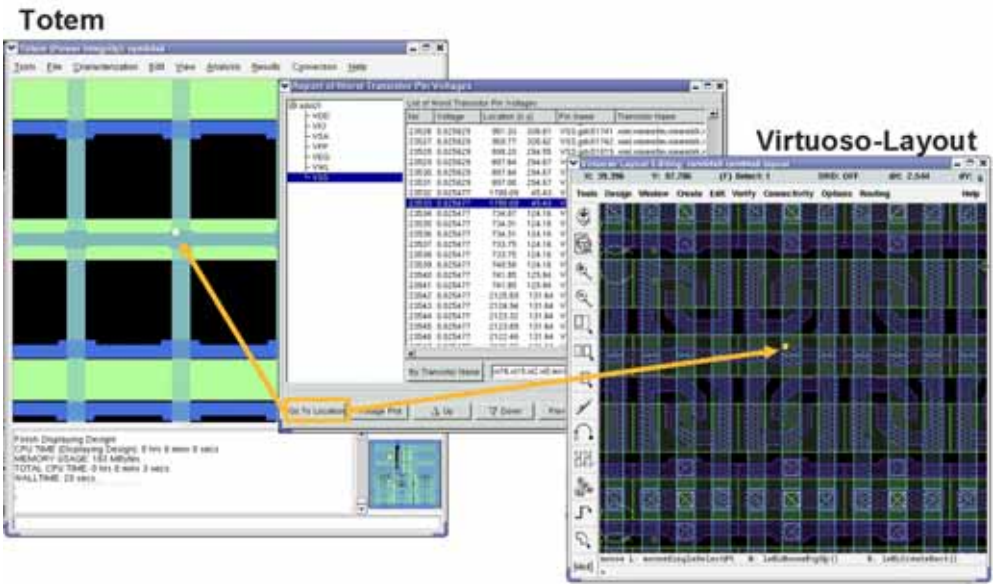


Figure B3-47 Cross-probing to Virtuoso layout

- Zooming to a selected EM violation location from a list or critical EM locations, use the command **Results -> List of Worst EM.**

The 3rd-party layout editor displays the items identified in Totem.

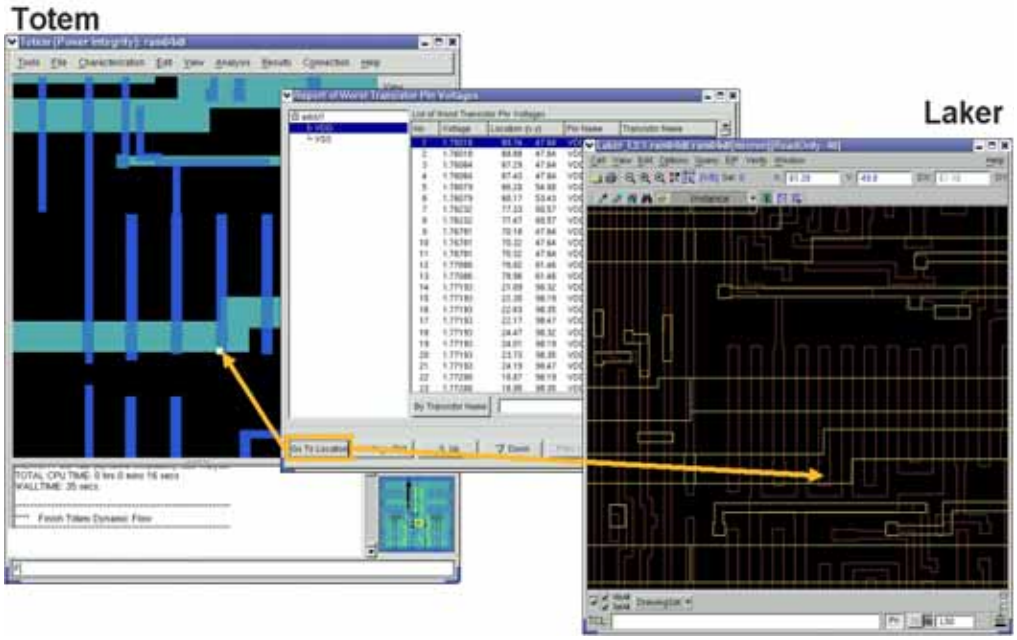


Figure B3-48 Cross-probing to Laker layout

Connection -> Connect to schematic

See setup in section "Cadence Virtuoso cross-probe setup", page B3-732

Uses the Totem cross-probing function to connect to Virtuoso Composer to view the schematic, as shown in Figure B3-49. This can help debug EM / IR / DvD problems that may be caused by design issues in the circuit.

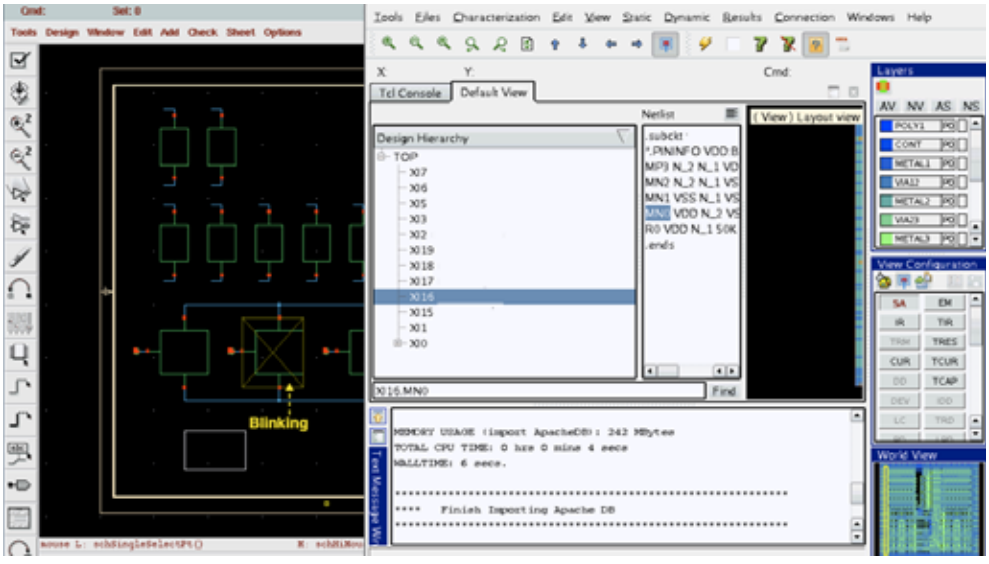


Figure B3-49 Cross-probing in the Virtuoso Schematic Composer

Use of schematic device names in the DB for Virtuoso cross-probing

The default device names stored in the Totem database change from the name in DSPF netlist to the name in the simulation netlist, as follows:

- in the SPICE_NETLIST + DSPF_NETLIST flow: the device name from the SPICE_NETLIST
- in the DSPF-only flow: the device name in the instance section in the DSPF_NETLIST file

This helps set up Totem-to-Virtuoso composer schematic netlist cross-probing.

Totem Explorer

Explorer -> Generate

Displays a dialog box to allow you choose the Data Integrity, Design Weakness and Hot Spot results that you want, and the threshold values controlling the data to be displayed. Data Integrity checks available include:

- CMM Validation
- APLMMx Current Consistency
- APL Current
- APL Cap
- APL Piecewise Cap
- LIB
- LEF
- GDS2DEF
- GDS2DB
- DEF
- SPEF
- STA
- VCD
- GSR Tcl
- Package Settings
- Pad File

Design Weakness checks available include:

- Pad Placement Quality
- Package Drop Contribution
- PG Resistance Distribution
- PG Resistance Imbalance
- Switch Placement Quality
- Decap Distribution
- Power Distribution Quality
- Clock Buffer Clustering
- Peak Current
- Simultaneous Switching

The Hot Spot checks available are:

- Dynamic Voltage Drop
- MMX Cell-Based
- Static IR

- Power EM

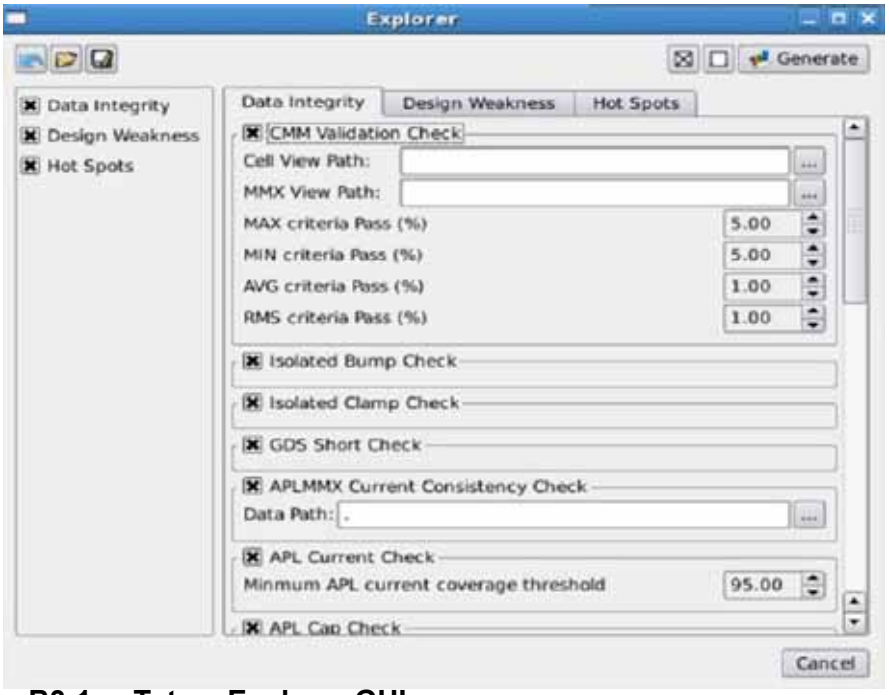


Figure B3-1 Totem Explorer GUI

After the desired results are selected under each tab, the Explorer Summary dialog is displayed, and the results can be analyzed.

Explorer -> View

Displays the ESD Explorer (Beta Version) GUI that allows GUI level debug and root-causing and also displays the summary of results selected and created with the **Generate** command.

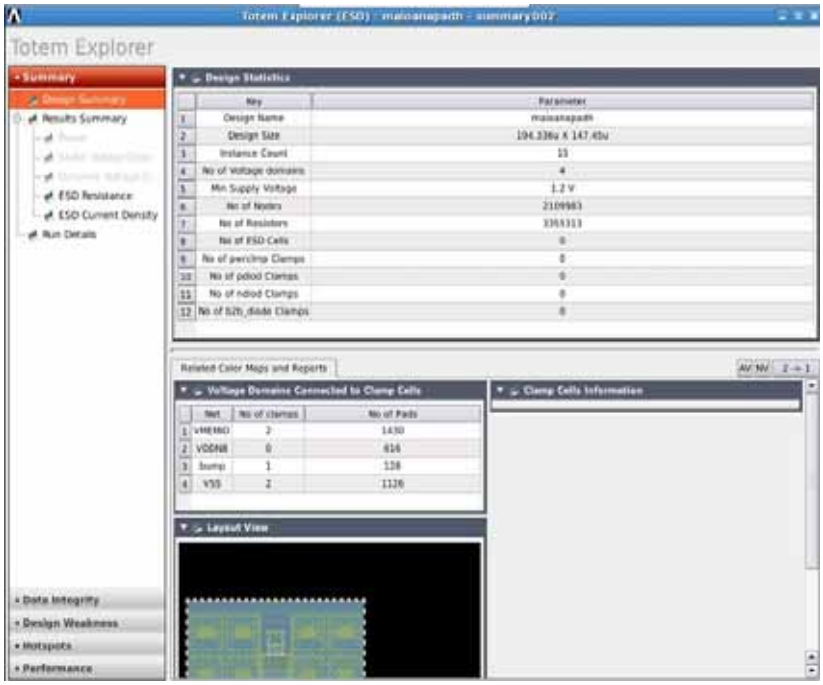


Figure B3-1 ESD Explorer GUI (Beta Version)

Windows -> Display Multiple Frames

Setup Configuration...

Allows setting up a multi-frame configuration, based on your debug preferences, using the menu selection **Setup Configuration**, as shown in Figure B3-2. You can **Save / Reload** previous configurations.

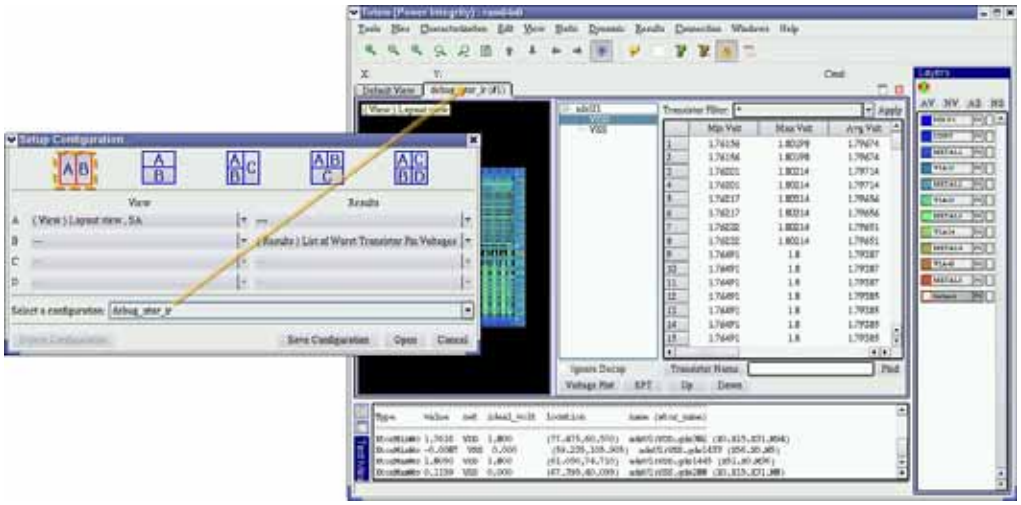


Figure B3-2 Multi-frame configuration setup

There are several types of preconfigured multiple-frames for ESD analysis, to make setting up the debug environment more convenient, as shown in Figure B3-3.

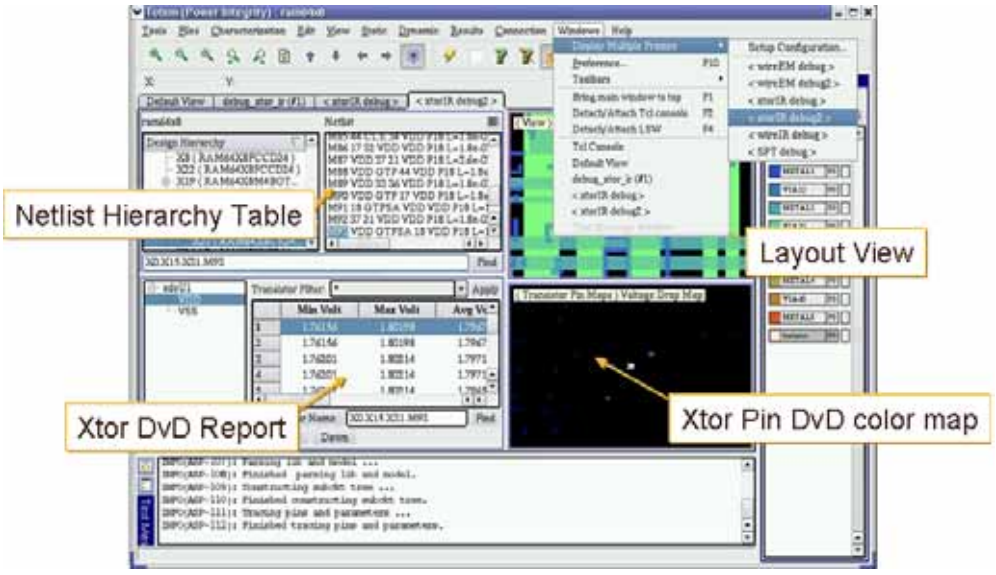


Figure B3-3 Preconfigured multiframes

Per frame net selection allows selection of different nets in different GUI frames to allow cross-checking of different nets at the same time, as shown in Figure B3-4.

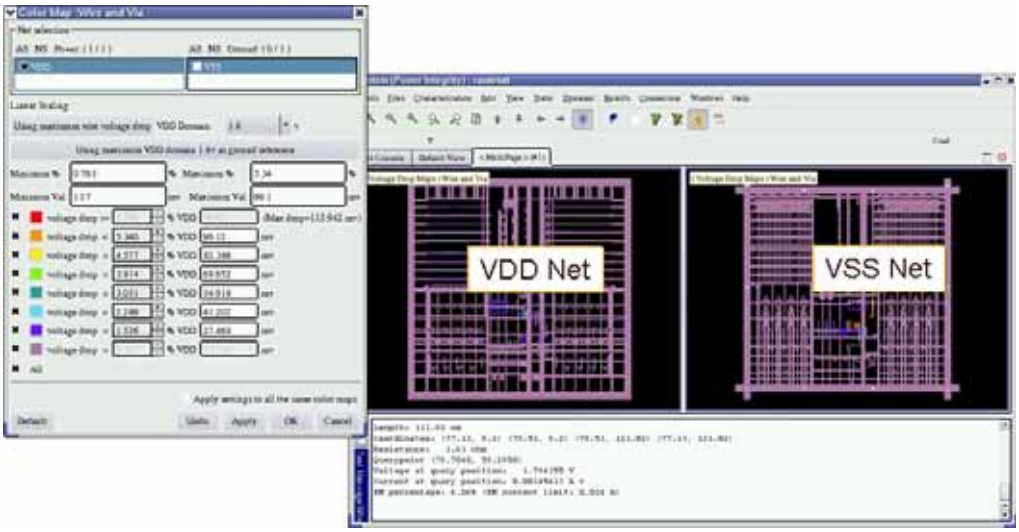


Figure B3-4 Per frame net display and selection

Various types of ESD frames are available for simultaneous display.

Windows -> Preferences ...

Brings up a Preference dialog box, with the following option selections:

Common options

- GUI Font: (Sans Serif 9 Normal default)
- Text Message Font : (Courier 10 Normal default)
- Tcl Console Font : (Courier 10 Normal default)
- Totem Explorer Font: (Sans Serif-8-Normal default)
- Highlight : Blinking or Color (default)
- Highlight Sync : Current View or All Views (default)
- Highlight Method : Auto Zoom (default) or User Defined

  Auto Zoom: zoom and center at the selected location.

  User defined: highlight the select location (marker), but not auto center, if the selected location is within the current zoom canvas. If the new selected point is outside current zoom view, the GUI uses current fixed zoom ratio and centers at the new selected point.
- Zoom sync : Current View or All Views (default)
- Waveform Viewer : xgraph or Totem Waveform Viewer (default)
- Waveform Viewer Integral Method : Left-Rectangle (default) or Trapezoid
- Color Maps : Smooth Color Spectrum or Disable GDS Instance in Color Map (default) The “Disable GDS Instance in Color Map” box allows disabling the instance IR map of GDS_CELLS to view the leaf instances underneath.
- Image background for printing: sets color of background for image printing (default black)
- Scroll Bar for Color Entries : 12 (default) - sets maximum number of color ranges

before scroll bar is used.

- Editing Mode : Mode1 (default), or Mode2 - which allows Virtuoso-style “what-if” editing function to support via layers editing, such as adding and deleting vias.
- Auto Region Filter: Enabled (default) or Disabled
- Using Multi-threads Mechanism - you can choose to either “Follow GSR: MULTI_THREADS Setting” (default), or override the GSR keyword value using the “Enable Thread Number” checkbox to set a maximum number of threads to use.
- Text Message Window Options: Provides a ‘Show Summary Page’ check box for a summary of Process steps completed in the log area below the design display (default On).
- Min Resistance Path (SPT) Analysis: By default “Support Multiple” paths is provided.

GDSII Display Options

- Display Controls : GDS Text and Path Borders (both default On)
- Array Display : Full (default), Border, or Source
- Filter Size : number of pixels (default 1)

Grid Controls

- Type : None or Dots (default)
- Minor Spacing: 0.002 (default)
- Major Spacing : 0.01 (default)
- Spacing Mode: User Specify spacing, or Automation Detection (default)
- X snap Spacing : no default
- Y snap Spacing : no default

Windows -> Toolbars

Note that this command is not available when the original GUI operating mode style is selected (Alt - p).

Show all toolbars

Displays both View Control and Query toolbars (default).

Hide all toolbars

Hides View Control and Query toolbars.

View Control Bar

Check box to control display of the View Control toolbar (default On).

Query Bar

Check box to control display of the Query toolbar (default On).

Windows -> Bring main window to top F1

Displays main design view on top.

Windows -> Detach/Attach Tcl console F2

Displays separate TCL terminal window.

Windows -> Detach/Attach LSW F4

Displays separate Layers, View Configuration, and World View windows.

Windows -> Tcl Console

Displays TCL terminal window as primary view.

Windows -> Default View

Displays full design view.

Windows -> Text Message Window

Brings detached Text Message window to top.

Help -> About...

Displays information about the Totem software release being utilized.

Help -> Manual

Provides a link to the latest “Totem User Manual” in PDF format.

Bindkey Functions

Functional Summary

- Functional bindkeys (Virtuoso--style behavior)
  - **F1** - Brings main window to top
  - **F2** - Detaches / attaches TCL console
  - **F4** -Detaches / attaches LSW panel
  - **F10** - Brings up ‘Preference’ dialog
- View Control bindkeys
  - **Ctrl** + **Z** - Zoom in by 2
  - **Shift** + **Z** - Zoom out by 2
  - **F** - Fit All
  - **W** - Last view
  - **Shift** + **W** - Next view
  - **Ctrl** + **R** - Redraw
  - **Up** - Scroll up
  - **Down** - Scroll down
  - **Left** - Scroll left
  - **Right** - Scroll Right
  - **Ctrl + F -** Display only top hierarchy in GDS view
  - **Shift + F** - Flatten all hierarchies in GDS view
  - **Ctrl + P** - Show power pad
  - **Shift + V** - Go down one hierarchy in GDS view
  - **Ctrl + V** - Go up one hierarchy in GDS view
  - **Ctrl** + **N** - Display selected nets / exclude unselected nets
  - **Shift** + **N** - Display all nets
- GUI Command bindkeys
  - **Ctrl** + **Q** - Exit GUI
  - **K** - Creates ruler

- Supports 45-degree ruler drawing
- Supports multiple-segment ruler drawing

Shift + K - Clears all rulers

Alt + P - Switches between the RedHawk-like GUI and the enhanced Totem GUI

Reconfiguring Bindkeys

You can reconfigure bindkey settings using the following steps

1. Export the existing bindkey settings supported in the Totem GUI to a text configuration file using the TCL command:

   ```
   export keybinding <keybinding_filename>
   ```

   Note that this command is only available in the GUI, and not in batch or interactive modes.

2. Edit the bindkey configuration file created in step 1 to establish the new bindkey settings desired.

   The format of the bindkey configuration file is as follows:

   ```
   # <description of bindkey >
   <bindkey name> <bindkey definition>
   ```

   Bindkey definitions can be assigned to any combination of single letter, **Ctrl**, **Shift**, and **Alt** buttons.

   Note that no spaces are allowed in bindkey definitions.

3. The newly-defined bindkeys can then be imported using one of the following methods:

   a. In GUI mode, use the command

      ```
      import keybinding <keybinding_filename>
      ```

   b. Rename the keybinding configuration file '*keybind.cfg*' and put it in

      - your working directory, or
      - the directory *$HOME/.totem/*

   If there is a *keybind.cfg* file in both of the directories above, the one in your working directory is used.

Appendix C
File Definitions

Introduction

This appendix describes the format and contents of internal input and output files supported by Totem.

Input Files

1. Technology file (**.tech*)
2. Global Switching Configuration file (*.gsc)
3. Global System Requirements file (**.gsr*)
4. Pad instance, cell, or location files (**.pad, *.pcell, *.ploc*)

The information from the following files is required. The file reference information should be included in the GSR file.

5. Library technology files
6. Design netlist files
7. Synopsys library files

Totem input files can be prefixed by the design name. The symbol “#” preceding a line is treated as a comment for all these files.

NOTE: Totem can read LEF, DEF, STA, SPEF, VCD, FSDB and GDS files in compressed **.gz* (gzip) format.

Output Files

Results are generated in each phase of the analysis process. See details of report files and graphic displays available in the individual subject chapters and in Chapter 6, "Results Evaluation", for more details.

Keyword Syntax Conventions

The syntax conventions used for defining Totem commands, options and keywords are as follows:

< x >	x describes a variable or value to be specified
[a | b | c]	one of a, b, or c must be selected
? x ?	x is an optional parameter

a b c d	a, b, c, and d parameters all must be specified
<x> ...	elements of the same type as <x> can be added
{ {a b c ...} {j k l ...} ...}	similar sets of elements may be added. Note that if only one set of elements is included, two sets of brackets may still be required
+ - * /	standard arithmetic operators for add, subtract, multiply, divide

Apache Technology File (*.tech)

The Totem technology file (*.tech*) provides technology information about the process and specifies parameters for each metal layer and type of via. A separate technology file is needed for each IC process. The following technology information is specified:

- Conductor name of each metal layer from the *.lef* file
- Thickness, resistivity, resistance temperature coefficient, and EM limit for each metal layer
- Via name, resistance per via, and via EM limit
- Resistance, inductance, and capacitance of the wire-bond or flip-chip solder bumps
- Thickness, height, and dielectric constant of dielectric layer
- Thickness and resistivity of the substrate layers

When specifying values in the *.tech* file, Totem supports the following units and prefixes. Unit is case sensitive.

Unit prefix conventions:

- terra = t = 1e+12
- giga = g = 1e+9
- mega = M or me = 1e+6
- kilo = k = 1e+3
- milli = m = 1e-3
- micro = u = 1e-6
- nano = n = 1e-9
- pico = p = 1e-12
- femto = f = 1e-15

Example: 1.3p = 1.3e-12. There is no space between the number and unit symbol.

Unit length conversions

- 1 mil = 0.001 inch = 2.54e-5 meters
- 1 inch = 2.54e-2 meters
- 1 micron = 1.0e-6 meters

Encrypting and Decrypting a Tech File

Two utility programs in the *bin* directory allow a Totem ASCII Tech file to be encrypted using a password, and Totem can then run the file in encrypted form. As needed, the encrypted file then can be decrypted with the correct password.

1. To encrypt a tech file, on a UNIX command line run the command:

```
techEncrypt <tech_filename>
```

2. Provide a password at the prompt.
3. An encrypted file, <*tech_filename*>*.enc.tech*, is created.
4. The encrypted tech file now can be used in Totem the same as a non-encrypted tech file.
5. To decrypt a tech file, on a UNIX command line run the command:
   ```
   techDecrypt -i <encrypted_tech_file> -o <tech_file>
   ```
6. Provide the correct password at the prompt.
7. A decrypted ASCII file, <*tech_file*>, is created.

Partial File Encryption

Totem supports encryption/decryption of specified sections of the tech file, so that sensitive process information can be hidden from end users. The main RC tech file has two sections; the first section has metal/vias/dielectric information that is not encrypted and is visible in plain text. The next section has entries for some metals/vias/dielectrics that is encrypted. This section may only contain the values related to RC, heights, or dielectric constants for the same metals/dielectric that is specified in unencrypted section. The 'start' and 'end' encryption lines are comment lines, so if there are "#ENCRYPT_START" and "#ENCRYPT_END" lines in a tech file, the designated section between the lines is automatically encrypted as specified.

Technology File Keywords

NOTE: Keyword and option names make no distinction between upper and lower case.

DIELECTRIC

Defines the dielectric layer parameters. Values for all layers must be defined when capacitance and inductance extraction is performed. By default, dimensions are in microns. *Optional; default: None*

Syntax:

```
DIELECTRIC <dielectric_layer_name> {
    constant <ER_value>
    thickness <value>
    [ Height <value> |
    Above <dielectric_layer_name> ]
    ER_VS_SI_SPACING {
     (<spacing_1>, <ER1>)
     ...
     (<spacing_n>, <ERn>)
    }
}
```

where

dielectric <dielectric_layer_name> : specifies name of dielectric layer

constant <ER_value> : specifies the value of dielectric constant (relative permittivity for the layer, except as otherwise defined (no units).

thickness <value> : specifies the thickness of the dielectric layer

Height <value> : specifies the height above defined base layer (0)

Above <dielectric_layer_name> : specifies that the layer is just above the defined layer

ER_VS_SI_SPACING: specifies the relative permittivity of dielectric layers between conductors as a function of spacing. Note that you must create the tech file from the ITF file using rhtech, and then run the 'captab' utility to update the tech file:

```
captab <tech_file> -layer all -all
```

Also note that the spacing specified is the on-silicon value.

Example:

```
dielectric IMD3b {
  thickness 0.22000
  constant 2.9
  ER_VS_SI_SPACING {
    { 0.2 2.8 }
     { 0.21 2.9 }
     { 0.23 2.99 }
     { 0.25 2.6 }
     { 0.26 3.0 }
   }
  above IMD3a
}
```

EM_RULE_SET

This keyword describes and names a set of EM rules for a desired EM mode or condition in the EM tech file, and should contain rules for all metal/via layers of interest. The EM tech file can contain any number of rule set entries.

Syntax:

```
EM_RULE_SET <rule_set_name1>
metal <layer1> {
    <layer 1 EM rules>
...
}
metal <layer2> {
    <layer 2 EM rules>
...
}
...
```

Example:

```
EM_RULE_SET  typical_avg
metal M3 {
    <M3 EM rules for typical_avg>
   ...
}
metal M2 {
    <M2 EM rules for typical_avg>
   ...
EM_RULE_SET  worst_peak
metal M3 {
    <M3 EM rules for worst_peak>
...
}
```

```
metal M2
 ...
```

HALF_NODE_SCALE_FACTOR

Specifies the scaling factor for RC extraction parameters and geometry, which is taken by the *rhtech* utility from the same keyword in the *itf/.nxtgrd* file used by the foundry, or the *ircx2tech* utility and the *iRCX* file can be used. This scaling affects extracted RC parameter values and all parameters in the *itf/nxtgrd* file as if dimensions were reduced by the specified factor, but it does not scale parameter values in the Tech file, or actual dimensions in the Totem layout view or GUI. All geometry reporting (location, metal width/length) keeps the original dimensions so that you can fix/cross probe from the original database.

When the tool calculates the width of a piece of metal, it takes the drawn width and multiplies it with the HALF_NODE_SCALE_FACTOR value, such as 0.9. For EM, the EM limits in the tech file are for post-shrink values.

The associated APL model scaling is performed in the BSIM model using an equivalent keyword item '.option geoshrink=<factor>'. As an alternative, you can use GSR keywords DEF_SCALING_FACTOR and LEF_SCALING_FACTOR, for which the layout dimensions *are* scaled. *Optional; Default: no scaling.*

Syntax:

```
HALF_NODE_SCALE_FACTOR <factor>
```

Example:

```
HALF_NODE_SCALE_FACTOR 0.8
```

METAL

Defines the physical characteristics of each metal layer, and the effective size of the wires in them. Resistivity must be defined, using either a constant or a process parameter-based calculation method. If more than one value is defined, the more accurate process-based values are used. Totem records which method is used in the session log file. Many of the process parameter-based keywords and values are taken from standard technology files and are not incorporated or edited directly.

An important aspect in determining many of the parameter values for EM is the *wire width* to be used. There are two width definitions for wires, *drawn width,* W(D), and *silicon width,* W(S). If the option WIDTH_SI_VS_WIDTH_AND_SPACING is defined, then:

W(S) = w (looked up from the table using W(D)) – EM_ADJUST.

Otherwise, if options RESISTIVE_ONLY_ETCH or ETCH_VS_WIDTH_AND_SPACING are defined, then

W(S) = W(D) – etchL – etchR – EM_ADJUST,

where 'etchL' and 'etchR' are the left (down) and right (up) side etching effects calculated from the etch table for the wire. If none of above options are defined, then 'W(S) = W(D) – EM_ADJUST', where EM_ADJUST is the keyword for each layer.

For half-node scaling, when the tool calculates the width of a piece of metal, it takes the drawn width and multiplies it with the HALF_NODE_SCALE_FACTOR, such as 0.9. For EM, the EM limits in the tech file are for post-shrink values.

There are also two widths used in EM calculation, 'w(lookup)' is the width used to look up the value in the EM rule table, and 'w' is the width used to multiply current density to get EM current. There are two GSR keywords to control the type of width used, as shown in the following table. Note that the default is foundry-dependent.

Defining Wire Width for EM Calculation	USE_DRAWN_WIDTH_ FOR_EM_LOOKUP = 1	USE_DRAWN_WIDTH_ FOR_EM_LOOKUP = 0
USE_DRAWN_WIDTH_ FOR_EM = 1	W(lookup) = W(D) W = W(D)	N/A
USE_DRAWN_WIDTH_ FOR_EM = 0 (default)	W(lookup) = W(D) W = W(S)	W(lookup) = W(S) W = W(S)

Required for all metal layers, except as noted.

Note: The first part of this definition includes basic metal specifications, including the effective wire width to be used in EM calculations. See the following sections for advanced metal resistance and capacitance specification methods based on process parameter relationships for: a. temperature, b. thickness, c. width and spacing, d. etch geometry.

Syntax:

```
METAL <metal_layer_name> {
    ? MINWIDTH <drawn_wire_width_um> ?
    ? MINSPACE <wire_spacing_um> ?
    WIDTH_ADJUST <adjustment>
        ? WIDTH-SPACE {
        {<w1> <spacing1>}
        {<w2> <spacing2>}
            ...
        }?
    Pitch <value of length>
    Thickness <t_value>
    Resistance <rpsq>
    RHO <value_Ohm-microns>
    CAP_DENSITY <value_fF/um^2> <associated_layer>
    EM <max_wire_current_density>
    ? WIDTH_SI_VS_WIDTH_AND_SPACING ?
    ? T_EM <final_temp> ?
     TNOM_EM <nom_temp>
     EM_TEMP_RATING {
        { <temp1> <derating_factor1> }
        { <temp2> <derating_factor2> }
        ...
        }
    }
    ? EM_adjust <wire_width_adj_um> ?
    ? EM_thickAdjust <wire_thickness_adj_um>?
    ? WIDTH_BASED_EM {
       width { <silicon_width1> ... <silicon_widthN>}
       em {<limit_per_u-sil_width1> ... <limit_per_u-sil_widthN+1>}
        }?
    ? BLECH_JLC <Blech_product>
    [ Height <length-value> |
     Above <dielectric_layer_name_below_metal> ]?
        POLYNOMIAL_BASED_EM_[DC | PEAK | RMS ] {
        COND_RULE {<cond_rule1> ... ? AND (UPSTREAM)? }
```

```
            EM_POLYNOMIAL { <polynomial expression> }
            COND_RULE {<cond_ruleA> ... ? AND (DOWNSTREAM)? )
            EM_POLYNOMIAL { <polynomial expression> }
            EM_POLYNOMIAL { ANTI_DERATE (polynom expression)
                IMAX ( DERATE ( MIN ((polynom expression ))))}
            ...
        }
```

Example:

```
        POLYNOMIAL_BASED_EM_DC {
            COND_RULE { L>10 AND w>0.18 AND (UPSTREAM) }
                EM_POLYNOMIAL { 0.02*w IMAX sqrt(3*(w -0.01)^2)
            COND_RULE { L>10 AND w>0.18 AND (DOWNSTREAM) }
                EM_POLYNOMIAL { 0.04*w IMAX sqrt(4*(w -0.01)^2)
    }
        POLYNOMIAL_BASED_EM_PEAK {
            COND_RULE { W/L > 2 AND W/L < 10 }
                EM_POLYNOMIAL { 10 * ( w - 0.1 ) }
        }
        POLYNOMIAL_BASED_EM_RMS {
            COND_RULE {L <= 10 }
            EM_POLYNOMIAL { sqrt( 1 * delta_T *( w - 0.1 ) ^ 2 )
    }
            COND_RULE { L >= 20.0 AND w > 0.18 }
            EM_POLYNOMIAL {1.3*w IMAX DERATE(min((w* 43.6/l),
                ( 0.06*l/25 )) )
            EM_POLYNOMIAL { ANTI_DERATE ( 0.01*w ) IMAX (
                DERATE ( MIN (( 0.001 + 0.00223086*w ),
                ( 0.032/w )))
        }
    }
```

where

MINWIDTH <drawn_wire width_um> / MINSPACE <wire_spacing_um> : specifies the minimum wire width and minimum spacing between wires for DRC, based on the WMIN and SMIN values in the ITF file, respectively. Note that the LEF WIDTH parameter is used first for specifying metal wire width when creating DEF net wires.

WIDTH_ADJUST <adjustment>: specifies an adjustment to width spec such that effective wire width for analysis = original width minus <adjustment>

WIDTH-SPACE <w1> <spacing1> : specifies pairs of drawn wire widths and associated spacing values for wires present in the design.

Pitch <distance> : specifies the distance between wire centers for the grid (um)

Thickness <t_value> : specifies metal layer thickness (um)

Resistance <rpsq> : specifies the metal sheet resistance, in Ohms per square. Effective resistance is calculated as rpsq * length / width_si, where length and width_si are length and silicon width of the metal segment, respectively.

RHO <value_Ohm-microns> : specifies the global value of resistivity for wires

CAP_DENSITY: user-specified capacitance density (fF/um^2) between the given layer and the specified <associated_layer> (instead of extracting cap value)

EM <max_wire_current_density> : max current density allowable for acceptable EM conditions, in current per wire width (units specified in 'Units' section) (default: 1.0)

WIDTH_SI_VS_WIDTH_AND_SPACING: specifies W(S) based on width and spacing data (taken from appropriate vendor data)

T_EM <final_temp> : specifies final operating temperature used for EM calculations. Overrides GSR TEMPERATURE value for metal if values are different.

TNOM_EM <nom_temp>: nominal temperature used for EM calculations

EM_TEMP_RATING <temp1> <derating_factor1> : specifies the ratio of maximum current density between <temp1> and TNOM_EM. So the maximum current density limit at <temp1> can be calculated by multiplying the <derating_factor1> by TNOM_EM.

EM_TEMP_DERATE : When set to 1 within POLYNOMIAL_BASED_EM_RMS or POLYNOMIAL_BASED_EM_PEAK, user can specify separate EM_TEMP_RATING for RMS and Peak limits within the individual rule section.

EM_Adjust : adjustment to wire width.
If specified, calculated current density = true current in wire / (drawn width of wire - EM_Adjust)

EM_thickAdjust : adjustment to actual wire thickness for Blech JLC length calculation

WIDTH_BASED_EM {...} : specifies EM limit values per micron for metal segments based on their *silicon* width (by table lookup). So for width <silicon_width1>, use EM value <limit_per_u-sil_width1>, up to <silicon_widthN>, use EM value <limit_per_u-sil_widthN>, and for widths greater than <silicon_widthN>, use EM value <limit_per_u-sil_widthN+1>. If specified, EM_ADJUST values are applied to width values. No interpolation or extrapolation is performed.

BLECH_JLC <Blech_product> : specifies a maximum product (mA/um) of worst case wire length and current density for groups of connected and active segments on a layer. This feature is turned on by setting the ENABLE_BLECH GSR keyword to 1. The EM limit calculation is:

EM limit = Blech_JLC*(wire_thickness/length)* width

Height <value> : specifies the height above defined base metal layer (0)

Above <dielectric_layer_name_below_metal> : specifies that metal layer is just above defined dielectric layer. The bottom of the metal is coincident with the top of the dielectric.

POLYNOMIAL_BASED_EM_* : defines tables of relationships for DC (or AVG), RMS, and PEAK maximum EM current, based on specified values for LENGTH_RANGES and WIDTH_RANGES. EM_POLYNOMIAL entries define an equation for maximum EM current as a function of metal width and length for each combination of width range and length range. If WIDTH_RANGES is specified before LENGTH_RANGES, then WIDTH_RANGES is used as the primary table index. Units: mA.

If there is only one polynomial that is applied to all lengths for a condition, the range can be represented by any very large number.

COND_RULE < >: defines a conditional rule limit, such as "L >= <length> AND w <= <width>". This option can be defined using equations.

UPSTREAM/DOWNSTREAM : optionally defines conditional rules that are different for opposite current flow directions, that is, via electron flow toward higher layers (UPSTREAM) and toward lower layers (DOWNSTREAM)

IMAX <Imax polyn> entry : specifies an additional polynomial for the maximum allowed current (Imax) in the polynomial-based EM equation, which can be applied selectively for a desired length/width range. Current computed from the regular polynomial is checked against Imax, and limited to the Imax value if exceeded. Temperature derating is applied only to regular polynomials and not to Imax.

ANTI_DERATE: turns off default temperature and EM_MISSION_PROFILE derating for specified parts of the polynomial. Should not be applied to IMAX polynomials.

DERATE : the IMAX clause defines the absolute maximum limit, so no derating factors are applied to it. DERATE is used in the IMAX equation to specify that the enclosed value or equation needs to have temperature and EM_MISSION_PROFILE derating factors applied to it. Should not be applied to non-IMAX polynomials.

Note: Max peak current used for EM check is the value obtained from above current rules, divided by sqrt(r), where r is the duty cycle. For PG EM, r = 1. Note that this keyword is supported only for signal EM analyses.

Example 1:

```
METAL Metal7
{
    THICKNESS   0.7
    RESISTANCE  0.025
    EM          2.4
    EM_ADJUST   0.02
    BLECH_JLC   250
    ABOVE       diel
}
```

Example 2:

```
...
   POLYNOMIAL_BASED_EM_RMS {
     LENGTH_RANGES { 1e38 }
     EM_POLYNOMIAL {sqrt(15.00*delta_T*(w - 0.015)^2*
        (w - 0.015 + 0.155 )/(w - 0.015 + 0.055)) }
     }
```

where the ‘delta_T’ parameter is defined by the GSR keyword DELTA_T_RMS_EM.

Example 3:- Using EM_POLYNOMIAL format

```
...
POLYNOMIAL_BASED_EM_DC {
    LENGTH_RANGES { 5.00 20.00 }
    WIDTH_RANGES { 0.05 0.15 }
# For L<5
    EM_POLYNOMIAL { 4*0.388*( w -0.016) }
    EM_POLYNOMIAL { 4*0.388*( w -0.016) }
    EM_POLYNOMIAL { 4*1.388*( w -0.016) }
    EM_POLYNOMIAL { 4*1.388*( w -0.016) }
    EM_POLYNOMIAL { 4*2.388*( w -0.016) }
# For L=5
    EM_POLYNOMIAL { 4*0.388*( w -0.016) }
```

```
    EM_POLYNOMIAL { 4*0.388*( w -0.016) }
    EM_POLYNOMIAL { 4*1.388*( w -0.016) }
    EM_POLYNOMIAL { 4*1.388*( w -0.016) }
    EM_POLYNOMIAL { 4*2.388*( w -0.016) }
# For 5<L<20
    EM_POLYNOMIAL { (20/L)*0.388*( w -0.016) }
    EM_POLYNOMIAL { (20/L)*0.388*( w -0.016) }
    EM_POLYNOMIAL { (20/L)*1.388*( w -0.016) }
    EM_POLYNOMIAL { (20/L)*1.388*( w -0.016) }
    EM_POLYNOMIAL { (20/L)*2.388*( w -0.016) }
# For L=20
    EM_POLYNOMIAL { (20/L)*0.388*( w -0.016) }
    EM_POLYNOMIAL { (20/L)*0.388*( w -0.016) }
    EM_POLYNOMIAL { (20/L)*1.388*( w -0.016) }
    EM_POLYNOMIAL { (20/L)*1.388*( w -0.016) }
    EM_POLYNOMIAL { (20/L)*2.388*( w -0.016) }
# For L>20
    EM_POLYNOMIAL { 0.388*( w -0.016) }
    EM_POLYNOMIAL { 0.388*( w -0.016) }
    EM_POLYNOMIAL { 1.388*( w -0.016) }
    EM_POLYNOMIAL { 1.388*( w -0.016) }
    EM_POLYNOMIAL { 2.388*( w -0.016) }
}
```

Example 4 - Using EM_POLYNOMIALS format:

```
...
POLYNOMIAL_BASED_EM_DC {
    LENGTH_RANGES { 5.00 20.00 }
    WIDTH_RANGES { 0.05 0.15 }
# For L<5
    EM_POLYNOMIALS {
        { 4*0.388*( w -0.016) }
        { 4*0.388*( w -0.016) }
        { 4*1.388*( w -0.016) }
        { 4*1.388*( w -0.016) }
        { 4*2.388*( w -0.016) }
}
# For L=5
    EM_POLYNOMIALS {
        { 4*0.388*( w -0.016) }
        { 4*0.388*( w -0.016) }
        { 4*1.388*( w -0.016) }
        { 4*1.388*( w -0.016) }
        { 4*2.388*( w -0.016) }
}
...
```

Process parameter-based resistance-capacitance modeling

Users that have statistical process variation data in StarRC-XT, **.itf* and **.nxtgrd* tech files can bring the more accurate tech file parameters and equations into the technology file using the *rhtech* translation utility. Process variation adjustments for combinations of wire width, wire thickness, wire spacing, resistivity, and operating temperature are accommodated in the Totem tech file. The values of wire

resistance and capacitance used by Totem during simulation is then the result of these process variation adjustments available from computations using the new tech file equations and coefficients.

a. Resistance vs. temperature

Totem has the capability of modeling temperature-dependent resistance for conducting layers and vias. Temperature is specified in degrees centigrade. Resistance is modeled in the following way, the same method as is used in Spice:

```
R_eff= R_nominal*[1+ Coeff_RT1* (T-T0) + Coeff_RT2* (T-T0) ^2]
```

R_nominal is the resistance value at the nominal temperature T0. The syntax for specifying the necessary temperature parameters to calculate resistance is as follows:

```
?[ T <operating temp-C>
        Tnom <nominal temp-C>
        Coeff_RT1 <R variation_first order coeff>
        Coeff_RT2 <R variation_second order coeff>]?
```

where

T <operating temp-C>, Tnom <nominal temp-C>: optionally specifies the chip operating temperature and nominal temperature in degrees C. If a temperature model is used using the following temp coefficients, both T and Tnom must be specified.

Note: the GSR TEMPERATURE value takes priority over this tech file specification.

Coeff_RT1 <R variation-first order coeff> : specifies the first order metal resistance temperature coefficient, in Ohms per degree C. (default: 0)

Coeff_RT2 <R variation-second order coeff> : specifies the second order metal resistance temperature coefficient, in Ohms per degree C.

Coeff_RT1 and Coeff_RT2 are linear and quadratic temperature coefficients, and R_eff is the modeled resistance at the operating temperature T. Notice that the calculated resistance R_eff exactly equals the nominal resistance R_nominal if T=T0, or if both Coeff_RT1=0 and Coeff_RT2=0.

The coefficient options Coeff_RT1, Coeff_RT2, T and T0 can be specified in the Apache Technology file on a per-layer basis. The default values for Coeff_RT1 and Coeff_RT2 should be set to zero. Values for nominal temperature (Tnom or T0) and operating temperature T must be specified for all layers.

A sample tech file entry for temperature variation is shown below.

```
metal m1
{
  Thickness   0.2046
  T           125
  Tnom        25
  Coeff_RT1   0.00265
  Coeff_RT2   -2.641e-07
  ...
}
```

To specify resistance-temperature coefficients as a function of silicon wire width, the following syntax is used:

```
CRT_VS_SI_WIDTH {
    {w1 <Coeff_RT1_1> <Coeff_RT2_1> }
    {w2 <Coeff_RT1_2> <Coeff_RT2_2> }
```

```
    ...
  }
```

where

w* : specifies silicon wire widths

<Coeff_RT1_*> <Coeff_RT2_*> : specifies the first and second order temperature coefficients of the resistance equation associated with wire width w*. Interpolation used to obtain values.

b. Adjustments for Thickness

A number of process-based adjustments to wire thickness are available for improving the accuracy of resistance and capacitance values. The syntax for these adjustments is described in the following paragraphs.

```
THICKNESS_VS_WIDTH_AND_SPACING [RESISTIVE_ONLY|CAPACITIVE_ONLY] {
 ? SPACINGS { s1 s2 s3 ... } ?
 ? WIDTHS { w1 w2 w3 ... } ?
 VALUES {
    { t11 t12 t13 ... }
    { t21 t22 ... }
    { ... }
}
}
```

where

Resistive_only/Capacitive_only : by default, thickness values are used for both resistance and capacitance calculation

RESISTIVE_ONLY: thickness values used for resistance calculation only.

CAPACITIVE_ONLY: thickness values used for capacitive calculation only.

WIDTHS { w1 w2 ... } : specifies the widths of the wires in the design

SPACINGS { s1 s2 ...) : specifies the spacings between adjacent wires associated with specific wire widths

VALUES : specifies the thickness change txy corresponding to the proper width and/or spacing values in the WIDTHS/SPACINGS array (at least one parameter array must be provided). The new thickness T = old_T*(1+ t), where t is the value from VALUES table. In the VALUES table, column headers are SPACINGS, and row titles are WIDTHS. Interpolation is used to obtain intermediate values.

```
THICKNESS_VS_DENSITY [RESISTIVE_ONLY | CAPACITIVE_ONLY] {
 {d1 t1} {d2 t2} ...
 }
```

where

Resistive_only/Capacitive_only :by default, thickness values are used for both resistance and capacitance calculation

RESISTIVE_ONLY: thickness values used for resistance calculation only.

CAPACITIVE_ONLY: thickness values used for capacitive calculation only.

{ d1 t1 } : specifies the thickness change tx corresponding to the metal density dx for particular area of the chip. The new thickness T = old_T*(1+t), where t is the value from the density table. Totem uses a 100um X 100um bounding box to calculate density. Interpolation used to obtain values.

DENSITY_BOX_WEIGHTING_FACTOR {
{s1 w1} {s2 w2} {s3 w3} ...
}

where

sn : specifies size (in um) of a set of inclusive square bounding boxes for density calculation, where s1 is included in s2, and s2 is included in s3 ... up to sn.

wn : specifies the density weighting factor associated with square sn (from -10 to 10). By default, a table with one 100um X 100um sized box and weighting factor 1.0 is used.

This keyword is used with THICKNESS_VS_DENSITY to calculate the effective thickness by applying the effective density, where the effective density = SUM (density-of-square-sn* wn). Can be used for resistance or capacitance calculation, depending on whether THICKNESS_VS_DENSITY applies to resistance and/or capacitance. No interpolation/extrapolation.

BOTTOM_THICKNESS_VS_SI_WIDTH {
{w1 t1} {w2 t2} ...
}

where

{ w1 t1}: specifies the thickness change tx corresponding to the silicon width at the lower plane of the wire. The new T = old_T*(1+ t), where t is the change in thickness from the width table. Note that the thickness change is made to the bottom of the metal, so the thickness of the dielectric below the metal and distance between metal layers also change correspondingly, which affects capacitance extraction. Interpolation used to obtain values.

POLYNOMIAL_BASED_THICKNESS_VARIATION

Modeling methodology for thickness has significantly changed from 90nm to 65nm technology. In 90 nm technology, the thickness of metal layers, Tsi, is modeled by a linear equation that is a function of only the actual (“silicon”) area density, D, of each metal layer:

```
Tsi = aD + b
```

and

```
Density,D = (AreaSi-all polygons )/( Area of meas window)
```

In 65 nm technology, the thickness is modeled by a polynomial equation that is a function of both actual metal area density and drawn width:

```
Tsi /T_minW_minS = a * D^4 + b *D^3 + c * D ^2 + d * D + e +1
```

where coefficients a, b, c, d and e are also polynomials and functions of drawn width, and 'T_minW_minS' is the thickness when the width and spacing are minimum values.

Thickness has a similar polynomial relationship as a function of drawn width. The form of the Totem Tech file that supports this more accurate thickness modeling is shown in the following example:

POLYNOMIAL_BASED_THICKNESS_VARIATION {
DENSITY_POLYNOMIAL_ORDERS { 4, 3, 2, 1, 0 }

```
 WIDTH_POLYNOMIAL_ORDERS { 4, 3, 2, 1, 0 }
 WIDTH_RANGES {0.18 12.0000}
 POLYNOMIAL_COEFFICIENTS {
     {0  7.18007E+03  -3.74408E+03  6.25290E+02   3.33498E+01 }
     {0 -8.41826E+03   4.36173E+03 -7.20647E+02   3.75707E+01 }
     {0  3.01180E+03  -1.56096E+03  2.57063E+02  -1.31704E+01 }
     {0 -3.69306E+02   1.93110E+02 -3.19649E+01   1.58144E+00 }
     {0 -2.73935E+00  -9.12962E-01  4.76445E-01   5.41430E-03 }
   }
POLYNOMIAL_COEFFICIENTS {
 {-3.5524E-03  1.3434E-02 -3.7701E-01  2.1844E+00 -1.1489E+00 }
 { 8.6884E-03  6.6935E-02  3.6091E-03 -3.6206E+00  1.9171E+00 }
 {-1.0863E-02 -1.2601E-01  8.4772E-01  1.4232E+00 -9.2238E-01 }
 { 7.8418E-03  4.6979E-02 -5.8063E-01  1.1264E-01  6.4299E-02 }
 {-2.0450E-03 -3.3022E-03  1.2592E-01 -1.7997E-01  1.0073E-01 }
     }
POLYNOMIAL_COEFFICIENTS {
       {0    0    0    0    0 }
       {0    0    0    0    0 }
       {0    0    0    0    0 }
       {0    0    0    0    0 }
       {0    0    0    0   -0.001955 }
     }
}
```

In the table above the columns represent the coefficients for the density polynomial orders 4,3,2,1,0 from left to right. The rows represent the coefficients for the width polynomial orders 4,3,2,1,0 from the first through fifth rows. And the three coefficient tables represent, from top to bottom, a width range of (1) <0.18, (2) 0.18 to 12.0, and (3) >12.0 microns.

Using the example above, the calculation of the ratio of silicon thickness to nominal thickness (ST/NT) is performed as follows:

```
ST/NT = alpha*D^4 + beta*D^3 + gamma*D^2 + delta*D + epsilon + 1
```

where D = density and coefficients alpha, beta, gamma, delta, epsilon are calculated from the POLYNOMIAL_COEFFICIENTS table:

```
{ a1 b1 c1 d1 e1}
{ a2 b2 c2 d2 e2}
{ a3 b3 c3 d3 e3}
{ a4 b4 c4 d4 e4}
{ a5 b4 c5 d5 e5}
```

as:

```
alpha = a1*W^4 + b1*W^3 + c1*W^2 + d1*W + e1
beta = a2*W^4 + b2*W^3 + c2*W^2 + d2*W + e2
gamma = a3*W^4 + b3*W^3 + c3*W^2 + d3*W + e3
delta = a4*W^4 + b4*W^3 + c4*W^2 + d4*W + e4
epsilon = a5*W^4 + b5*W^3 + c5*W^2 + d5*W + e5
```

and W = scaled Width.

For W=2 and D=0.4 use the second (middle) POLYNOMIAL_COEFFICENTS table:

```
{-3.5524E-03 1.3434E-02 -3.7701E-01 2.1844E+00 -1.1489E+00 } => alpha =
    1.7624936
{ 8.6884E-03 6.6935E-02 3.6091E-03 -3.6206E+00 1.9171E+00 } => beta =
```

```
    -4.6351692
{-1.0863E-02 -1.2601E-01 8.4772E-01 1.4232E+00 -9.2238E-01 } => gamma =
    4.133012
{ 7.8418E-03 4.6979E-02 -5.8063E-01 1.1264E-01 6.4299E-02 } => delta =
    -1.5316402
{-2.0450E-03 -3.3022E-03 1.2592E-01 -1.7997E-01 1.0073E-01 } => epsilon =
    0.1853324
```

Therefore, in this example the ST/NT ratio = 0.9824272474.

c. Resistivity and resistance vs. wire width and spacing

If more than one of the following parameters for metal resistance are specified, only one is used, in the following order of priority:

- RHO_VS_SI_WIDTH_AND_THICKNESS
- RHO_VS_WIDTH_AND_SPACING
- RPSQ_VS_WIDTH_AND_SPACING
- RPSQ_VS_SI_WIDTH
- RESISTANCE (wires)

c1. Resistivity

Resistivity, rho (Ohm-microns), resistance, and capacitance are functions of actual metal width and thickness values, which are specified in an M-by-N matrix. For the appropriate width and thickness values, the value of rho is picked from the matrix by Totem, as shown in the syntax for the table below. Columns represent width values left to right and rows represent thickness values top to bottom. Linear interpolation is used if a width or thickness is not found in the table.

```
RHO_VS_SI_WIDTH_AND_THICKNESS {
     ? WIDTH { W1 W2 W3 ... Wm }?
     ? THICKNESS { t1 t2 t3 ... tn } ?
     VALUES  {
        {rho_11 rho_12 rho_13 ... rho_1m }
        {rho_21 rho_22 rho_23 ... rho_2m }
        {rho_31 rho_32 rho_33 ... rho_3m }
        {rho_41 rho_42 rho_43 ...         }
        {rho_51 rho_52 rho_53 ...         }
        ....
        {rho_n1 ...                       }
       }
```

where

WIDTHS : specify the wire widths w*

THICKNESS: specifies the thicknesses of the metal t*

VALUES are rho values rho_* corresponding to the width and/or thickness values in the WIDTHS and THICKNESS table. At least one parameter array must be provided. In the 2-D VALUES table, row titles are THICKNESS, column headers are WIDTH. Linear interpolation is used to obtain intermediate values.

Example:

```
RHO_VS_SI_WIDTH_AND_THICKNESS {
   WIDTH { 0.5 1.0 2.0 5.0 10.0 }
```

```
        THICKNESS { 0.875 1.25 1.5 1.625 }
        VALUES {
          { 0.0222 0.0209 0.02 0.02 0.02 }
          { 0.0221 0.0218 0.0215 0.0213 0.0211 }
          { 0.022 0.0217 0.0214 0.0212 0.021 }
          {0.02 0.0199 0.0199 0.0199 0.0199 }
        }
      }
```

Resistivity also is a function of actual metal width and spacing values, which are specified in an MxN matrix. For the appropriate width and spacing values, the rho value is picked from the matrix by Totem, as shown in the syntax for the table below. Columns represent spacing values left to right and rows represent width values top to bottom.

```
RHO_VS_WIDTH_AND_SPACING {
    ? SPACINGS { S1 S2 S3 ... Sm }?
    ? WIDTHS   { W1 W2 W3 ... Wn }?
    VALUES  {
         {rho_11 rho_12 rho_13 ... rho_1m }
         {rho_21 rho_22 rho_23 ... rho_2m }
         {rho_31 rho_32 rho_33 ... rho_3m }
        ....
         {rho_n1 ...                      }
        }
    }
```

where

SPACINGS : specifies the spacings s* between metal wires

WIDTHS : specifies the associated silicon wire widths w*

VALUES are rho values rho_* corresponding to the width and/or spacing values in the WIDTHS and SPACINGS table. In the 2-D VALUES table, column headers are SPACINGS and row titles are WIDTHS. Linear interpolation is used to obtain intermediate values.

c2. Resistance

For advanced processes with very small wire dimensions, the sheet resistance for metal lines changes with their width and spacing. Totem estimates resistance for metal lines based on their width and/or spacing relative to neighboring metal lines. Totem determines the appropriate sheet resistance value to use for a metal line in one of the following ways:

- Using nominal width-spacing pairs specified by the WIDTH-SPACE array, Totem associates a particular spacing to a wire based on specified widths. It takes the appropriate sheet resistance value based on that width-spacing value.
- If a nominal WIDTH-SPACE array is not specified, then Totem partitions the design into several tiles and determines the routing density for each metal layer in every tile. Based on the routing density in a tile, Totem associates a spacing for all metal routes in that layer within a tile. It then determines the sheet resistance to be used based on the spacing value identified for every metal route.

The syntax for this type of resistance model is as follows:

```
? RPSQ_VS_WIDTH_AND_SPACING {
    ? SPACINGS { s1 s2 ...} ?
    ? WIDTHS { w1 w2 ...} ?
```

```
        VALUES {
            { RPSQ11 RPSQ12... }
            { RPSQ21 }
            { ... }
        }?
        }
    ? RPSQ_VS_SI_WIDTH {
        {w1 rpsq1} {w2 rpsq2}
      }?
```

where

RPSQ_VS_WIDTH_AND_SPACING : defines a table of resistance values, in Ohms per square, for looking up sheet resistance. SPACINGS (s1,s2, ...) define the column headers for the table. Ideal silicon WIDTHS (w1, w2, ...) define the row titles for the table. VALUES are the sheet resistance values (RPSQxy ...) in the WIDTHS/SPACINGS array (at least one parameter array must be provided). Width-Spacing rules:

- unit values are defined in the units section
- the width array must contain at least 1 value, and if more, must be in increasing order. Hence the first element is the minimum width.
- the spacing array must contain at least 1 value, and if more, must be in increasing order. Hence the first element is the minimum spacing. Linear interpolation is used to obtain intermediate values.

RPSQ_VS_SI_WIDTH: specifies the sheet resistance rpsq* corresponding to silicon widths w*. Interpolation used to obtain intermediate values.

Example:

```
METAL metal1
{
MINWIDTH 0.2
MINSPACE 0.3
PITCH 0.5
THICKNESS 0.4
RESISTANCE 0.091
EM 0.91
BLECH_JLC 10.0
HEIGHT 1
ABOVE DIEL1
RPSQ_VS_WIDTH_AND_SPACING {
        WIDTHS { 0.42 0.84 1 2 }
    VALUES {
        { 0.02 }
        { 0.03 }
        { 0.04 }
        { 0.041 }
    }
}
```

Simple wire resistance is specified using the syntax:

Resistance <rpsq>

where

<rpsq> specifies the metal sheet resistance, in Ohms per square. Effective resistance is calculated as (rpsq * length) / width_si, where length and width_si are length and silicon width of the metal segment, respectively.

d. Wire Resistance and Capacitance vs. Etch Width and Spacing

Several process variation adjustments for etch adjustment are made for wire resistance and capacitance calculation using the parameter formula:

```
Silicon_width = drawn_width - etch_adjust
```

The syntax for providing etch adjustments to width for both resistance and capacitance calculations are given below in priority order. If more than one of the keywords is provided for resistance or capacitance, the higher priority keyword value is used.

RESISTIVE_ONLY_ETCH : Resistance_only
ETCH_VS_WIDTH_AND_SPACING : R and C
CAPACITIVE_ONLY_ETCH : Capacitance only
SIDE_TANGENT: Capacitance only
ETCH : R and C

These parameters are defined in the following pages.

RESISTIVE_ONLY_ETCH <silicon_width_microns>

Specifies an etch adjustment to resistance for wires throughout the chip.

```
ETCH_VS_WIDTH_AND_SPACING ?[ RESISTIVE_ONLY| CAPACITIVE_ONLY ] ?
    ? [PERPENDICULAR_TO_REFERENCE |
       PARALLEL_TO_REFERENCE ]?
    {
    ? SPACINGS { s1 s2 s3 ... } ?
    ? WIDTHS { w1 w2 ... } ?
    VALUES {
    { e11 e12 e13 ...}
    { e21 e22 ... }
    { ... }
    }
    }
```

where

RESISTIVE_ONLY/CAPACITIVE_ONLY : specify etch adjustments to width for either resistance or capacitance calculations only. Without RESISTIVE_ONLY or CAPACITIVE_ONLY, etch values apply to both. For RESISTIVE_ONLY and CAPACITIVE_ONLY, etching on top and bottom surfaces of the metal is the same amount, that is, W_top = W - 2*e and W_bot = W - 2*e, where W is the original drawn width, and W_top and W_bot are top and bottom silicon wire widths after etching.

Note: the metal wire still has a rectangular cross section

PERPENDICULAR_TO_REFERENCE : when the tech file keyword REFERENCE_DIRECTION is set to VERTICAL, the parasitic calculations for all vertically-oriented wires use the PARALLEL_TO_REFERENCE ETCH_VS_WIDTH_AND_SPACING data, and for horizontally-oriented wires the PERPENDICULAR_TO_REFERENCE data are used.

PARALLEL_TO_REFERENCE : when the tech file keyword REFERENCE_DIRECTION is set to PARALLEL, the parasitic calculations for all vertically-oriented wires use the PERPENDICULAR_TO_REFERENCE ETCH_VS_WIDTH_AND_SPACING data, and for horizontally-oriented wires the PARALLEL_TO_REFERENCE data are used.

Note: If any of the three tech file keywords are missing (REFERENCE_DIRECTION, PARALLEL_TO_REFERENCE or PERPENDICULAR_TO_REFERENCE), wire orientation-related etch rate effects are ignored and no etch compensation is done.

SPACINGS : specifies the spacing dimension between wires

WIDTHS : specifies drawn widths of the metal wires in the design

VALUES are etch values corresponding to the width or spacing in the WIDTHS or SPACINGS tables. In the VALUES table, columns headers are SPACINGS and row titles are WIDTHS. Interpolation used to obtain values.

Example 1: In the example below spacing is defined by columns left to right and widths are defined by rows top to bottom:

```
ETCH_VS_WIDTH_AND_SPACING RESISTIVE_ONLY {
SPACINGS { 0.378 0.42 0.5 0.84 0.9 1 2 3 }
WIDTHS   { 0.42 0.84 1.68 2.52 }
VALUES   {
    { 0.0218 0.0217 0.0214 0.0206 0.0205 0.0203 0.0196 0.0194 }
    { 0.0233 0.0231 0.0228 0.0221 0.0220 0.0218 0.0209 0.0205 }
    { 0.0242 0.0241 0.0239 0.0233 0.0232 0.0230 0.0220 0.0215 }
    { 0.021  0.0245 0.0243 0.0238 0.0237 0.0236 0.0227 0.0221 }
}
}
```

Example 2:

```
metal M3 {
 ETCH_VS_WIDTH_AND_SPACING CAPACITIVE_ONLY PERPENDICULAR_TO_REFERENCE
     <width and spacing data>
 ETCH_VS_WIDTH_AND_SPACING CAPACITIVE_ONLY PARALLEL_TO_REFERENCE
     <width and spacing data>
 ETCH_VS_WIDTH_AND_SPACING RESISTIVE_ONLY PARALLEL_TO_REFERENCE
     <width and spacing data>
 ETCH_VS_WIDTH_AND_SPACING RESISTIVE_ONLY PERPENDICULAR_TO_REFERENCE
     <width and spacing data>
 }
```

ETCH_VS_WIDTH_AND_SPACING ETCH_FROM_TOP (capacitance only)

For ETCH_FROM_TOP, the etching occurs in opposite directions on the top and bottom surfaces of the wires. That is, for positive etch from the top, W_top = W - 2*e and W_bot = W + 2*e, and the negative etch from the top case is the reverse. After etching, the metal has a trapezoidal cross section, as shown in Figure C-1 below.

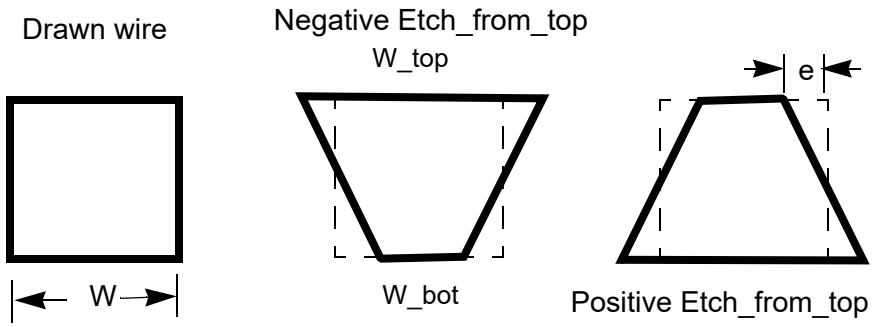


Figure C-1 Etch_from_top parameters

The syntax for the etch from the top capacitance correction is:

```
ETCH_VS_WIDTH_AND_SPACING ETCH_FROM_TOP
    ? SPACINGS { s1 s2 s3 ... } ?
    ? WIDTHS { w1 w2 w3 ... } ?
    VALUES {
    { e11 e12 e13 ... }
    { e21 e22 ... }
    {... }
    }
    }
```

where

WIDTHS : specifies drawn widths of the metal wires in the design

SPACINGS : specifies the spacing between wires

VALUES are etch values corresponding to the width and spacing in the WIDTHS and SPACINGS table. In the VALUES table, columns headers are SPACINGS and row titles are WIDTHS. Interpolation used to obtain values.

Note that only WIDTHS ***or*** SPACINGS parameters can be provided, so the VALUES array is one-dimensional.

`CAPACITIVE_ONLY_ETCH <silicon_width_microns>`

Specifies an etch adjustment for wire capacitance calculation throughout the chip.

`SIDE_TANGENT (capacitance only)`

Side_tangent defines the angle of the sides of the wires after etching, relative to the top surface, as shown in Figure C-2. In this type of etch specification, v is the angle from vertical of the conductor sides. After applying this keyword, W_top = W + v*t and W_bot = W - v*t, where W is the original drawn width, W_top is the top width after applying SIDE_TANGENT, and t is the thickness of the metal. The metal wire has a trapezoidal cross section.

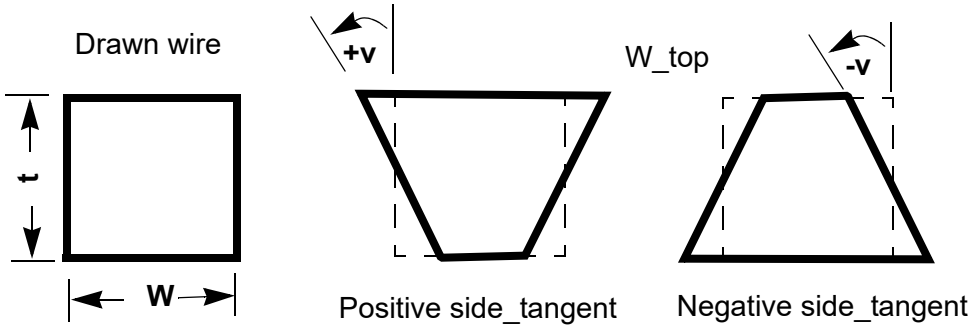


Figure C-2 Side_tangent parameters

The syntax for the side tangent capacitance correction is:

```
SIDE_TANGENT <v_side_angle>
```

where

<v_side_angle> : specifies the angle of the side of the wire from vertical after etching

ETCH <silicon_width_microns>

Specifies an etch adjustment for wire resistance or capacitance calculation throughout the chip.

e. Metal Fill-Related Parameters (capacitance effect)

Several parameters are used to specify non-circuit metal fill topology for adjusting capacitance effect. They are FILL_RATIO, FILL_SPACING, FILL_WIDTH, and FILL_TYPE. When FILL_RATIO is specified for a layer, any empty space encountered during extraction is modeled as though it were filled with floating metal of the same layer. This affects only the vertical capacitance. The keyword syntax is described below.

FILL_RATIO <ratio>

Specifies the ratio of metal fill allowed relative to the legal space available, as constrained by the FILL_SPACING and FILL_WIDTH parameters. The range is 0 to 1.0.

FILL_SPACING <spacing>

Specifies the fixed lateral spacing separating P/G nets and filled objects, in um. Required if FILL_RATIO is specified.

FILL_WIDTH <width>

Specifies the fixed width of metal fill objects, in um. Required if either FILL_SPACING or FILL_RATIO is specified.

FILL_TYPE [GROUNDED| FLOATING]

Specifies whether the filled metal segments are treated as grounded or floating. Default: grounded.

f. Conductor Cladding Parameters

Three keywords are used for specifying cladding effect on conductor resistance, DROP_FACTOR, CLAD_RPSQ and CLAD_THICKNESS. They are described below.

DROP_FACTOR <vert_ht>

where

vert_ht : specifies the absolute height (in um) of the decrease in base height of all conductors above the current conductor that do not have geometries in a given layout area (default 0).

CLAD_RPSQ <rpsq>

where

rpsq : specifies the effective resistance per square of cladding on a conductor, in Ohms/square. This keyword must be used with CLAD_THICKNESS.

CLAD_THICKNESS <thickness>

where

t : specifies the effective thickness of cladding on a conductor, in um.

SUBSTRATE

Defines the thermal conductivity (W/m-degree K), thickness (microns), resistivity (Ohm-cm), and capacitance (pF) for the four layers comprising the substrate. *Optional. Default: none*

Syntax:

```
SUBSTRATE [ BULK | EPI | PWELL | NWELL ]
{
? THERMAL_CONDUCTIVITY <value>?
? THICKNESS <value>?
? RESISTIVITY <value>?
? CAPACITANCE <value>?
}
```

Example:

```
SUBSTRATE BULK
{
THERMAL_CONDUCTIVITY 120
THICKNESS 500
RESISTIVITY 0.012
}
```

UNITS

Defines units for parameters in the technology file. *Optional; defaults: shown in table below.*

Syntax

```
units
{
capacitance    <value> (default: 1 pF;  1e-12 Farad)
inductance     <value> (default: 1 nH;  1e-9 Henry)
resistance     <value> (default: 1 Ohm)
length         <value> (default: 1 um;  1e-6 meter)
current        <value> (default: 1 mA;  1e-3 Ampere)
voltage        <value> (default: 1 Volt)
power          <value> (default: 1 Watt)
time           <value> (default: 1 ns;  1e-9 second)
frequency      <value> (default: 1 MHz; 1e+6 Hertz)
}
```

Example:

```
units
{
 capacitance 1p
 inductance  1n
 resistance  1
 length      1u
 current     1m
 voltage     1
 power       1
 time        1n
 frequency   1M
}
```

VIA <via_name>

Specifies the via name and physical parameters defining the physical characteristics of each via. Note that multiple vias can be defined under different via names. Note that Via parameter definitions are taken first from the Apache Tech file, but for cases in which data are not present in the Tech file, then Technology LEF file data is used. *Required for all vias.*

Syntax:

```
VIA <via_name> {
    Resistance <resistance_value>
    UpperLayer <metal_layer_above>
    LowerLayer <metal_layer_below> ? <metal_layer_name2>?
    Coeff_RT1 <R variation-first order coeff>
    Coeff_RT2 <R variation-second order coeff>
    TNOM <nominal_temp>
    ...
    ? RPV_VS_AREA {
    {<min-area_1> <resistance-1>
    ...
    <max-area_n> <resistance-n>
    ...
    }
    } ?
    WIDTH {<via_width>}
    AREA <via_area>
    RPV_VS_WIDTH_AND_LENGTH {
      LENGTHS { L1, L2, ...}
      WIDTHS { W1, W2, ...}
      VALUES {
        { W1L1  W1L2 ... }
        { W2L1  W2L2 ...}
    }
    WIRE_R_COMPENSATION [0|1]
    RPV_VS_NUM {
        {<num_cuts1>, <resistance1>} {<num_cuts2>, <resistance2>}
        ...
    }
    EM <value of EM current limit>
```

```
    ? CRT_VS_AREA {{ <Area1> c11 c12 } {<Area2> c21 c22}
    {<Area3> c31 c32 } ... } ?
    ? TOPOLOGY_BASED_EM {
# number of cuts-based calculation
    [ downflow  {
        via_number { <n1> <n2> ...}
        metal_width { <w1> <w2> ...}
        em {
           { <em_n1w1> <em_n1w2> <em_n1w3> ...}
           { <em_n2w1> <em_n2w2> <em_n2w3> ...}
        }
     }
     upflow {
        ...
    } |
# area of cuts-based calculation
    downflow  {
        via_area ( <a1> <a2> ... }
        metal_width { <w1> <w2> ...}
        em {
            { <em1> }
            { <em2> }
            ...
        }
     }
     upflow {
    ...
    } ]
    }

    EMV_VS_AREA {<area1> <arith_expr_for_EM_limit1>
        <area2> <arith_expr_for_EM_limit2>
        ...
    }
    }
    EM_TEMP_RATING {
        downflow {
        { <temp1> <derating_factor1> }
        { <temp2> <derating_factor2> }
        ...
        )
        upflow {
        { <temp1> <derating_factor3> }
        { <temp2> <derating_factor4> }
        ...
        }
    }
    DIRECTION_BASED_EM_TEMP_RATING {
        downflow {
        { <temp1> <derating_factor1> }
        { <temp2> <derating_factor2> }
```

```
            ...
          }
           upflow {
           { <temp1> <derating_factor3> }
           { <temp2> <derating_factor4> }
            ...
           }
       }
       } ?
```

Note: when Upflow and Downflow derating values are equal, then the syntax can be:

```
    EM_TEMP_RATING {
           { <temp1> <derating_factor1> }
           { <temp2> <derating_factor2> }
           ...
           )


       POLYNOMIAL_BASED_EM_[DC | RMS | PEAK ] {
           LENGTH_RANGES { <L0> ... <Lx> ... <Ln> }
           EM_POLYNOMIAL {<polynom for EM current for L <= L0 >}
           ...
           EM_POLYNOMIAL {<polynom for EM current for Lx-1< L <= Lx >}
           ...
           EM_POLYNOMIAL {<polynom for EM current for L > Ln >}
           COND_RULE { (10.0 >= Lu >= 4.9 AND Wb <= 0.18) AND (
               Lu > Lb) AND DOWNSTREAM }
           EM_POLYNOMIAL {1*0.187 WIRE_MAX
               (10/Lu)*0.57*(Wb-0.0085) }

       }
```

Note: from version 11.2 and on the following format is available for specifying EM polynomials for via layers:

```
       ? POLYNOMIAL_BASED_EM_[DC | RMS | PEAK ] {
           COND_RULE { <Boolean description of variable range 1> }
               EM_POLYNOMIAL { <polynomial equation 1> }
           COND_RULE { <Boolean description of variable range 2> }
               EM_POLYNOMIAL { <polynomial equation 2> }
           ...
       }
```

where

<via_name>: name of via

Resistance <value of resistance> : specifies via resistance in Ohms for a cut with the minimum width w1 from the width table, cross-sectional area $w1^2$, and no TEMPERATURE scaling.

UpperLayer : specifies the name of the metal layer above the via

LowerLayer: specifies the name(s) of one or more metal layers below the via

Coeff_RT1 (or _RT2) : specifies the first/second order metal resistance temperature coefficient, in Ohms per degree C. (default: 0)

TNOM: specifies nominal temperature in degrees C

RPV_VS_AREA {a1 r1} : the resistance of a via r* (Tech file resistance units) depends on its area, listed in order of increasing area. Interpolation is performed if the specific area is not in the table, but no extrapolation.

WIDTH <via_width>: width of via (square)

AREA <via_area> : specifies basic via area in um^2 (from appropriate vendor file). Note that vias on same layer with different cut areas and properties can be defined under different via names.

Note: either WIDTH or AREA must be defined for R and EM calculation.

RPV_VS_WIDTH_AND_LENGTH : models the resistivity of virtual vias in trench contacts according to the via width and length.

WIRE_R_COMPENSATION: By default extraction always includes the resistance of wire segments directly below and above via cuts (usually referred as metal-overhang). Set this keyword to 1 if you want to exclude resistance of this type from R extraction for a particular via, and use only the vertical via resistance. *Default: 0 (include).*

Note: The global WIRE_R_COMPENSATION value is set by the GSR keyword, but does not include pseudo-vias, which must be set in this tech file VIA section. Also, if this keyword is not set in the GSR file, it must be specified in every VIA section in the tech file.

RPV_VS_NUM : specifies the via resistance (Ohms or standard units, unless specified) based on the number of via cuts

EM <value of EM current limit> : EM current limit in mA for a cut with the minimum width w1 from the width table and cross-sectional area $w1^2$

CRT_VS_AREA: specifies area-dependent temperature coefficients of resistance in the Tech file for vias having different cut areas. <Area1> is one via cut area, c11 is first order temperature coefficient of resistance for <Area1> and c12 is second order temperature coefficient of resistance for <Area1>. Values for specific areas not listed are interpolated.

TOPOLOGY_BASED_EM : can be specified using either a number-of-cuts topology based EM limit, or using a total via area topology-based EM limit, as described in the syntax below.

Number of cuts-based calculation: EM limit values are the total EM limit for the number of cuts specified. *One exception: for certain foundry rule decks (as identified by the IRCX "# DRM" comment line in the tech file), EM limit values are per-cut, with no interpolation or extrapolation.* For the number of cuts and metal width of the via, the EM limit is picked from the matrix by Totem. Columns represent width values left to right, and rows represent the number of cut values top to bottom

downflow: specifies the EM value for vias in which the predominant electron flow direction is from the upper to the lower layer.

upflow: specifies the EM value for vias in which the predominant electron flow direction is from the lower to the upper metal layer.

via_number {<n1> <n2> ... }: specifies the number of cuts in the via

metal_width { w1 w2 ...}: list of widths of the metal wires in the flow direction for the layer that electrons are flowing *from.* If electron flow direction is upward, use the lower metal width, otherwise use the upper layer width.

em_n1w1 em_n1w2 ... em_nxwx } : specifies the maximum EM value for specified combinations of via cuts and specified metal_widths.

Area-of-cuts-based calculation: EM limit values are specified per-cut. For each total via area, the specified maximum EM current is picked from the list by Totem. Interpolation and extrapolation are used between limit values.

downflow: specifies the EM value for vias in which the predominant current flow direction is from the upper to the lower layer.

upflow: specifies the EM value for vias in which the predominant current flow direction is from the lower to the upper metal layer.

via_area (<a1> <a2> ...}: list of total area for vias in design

metal_width { w1 w2 ...}: list of widths of the metal wires in the flow direction for the layer that current is flowing *from.* If current flow direction is upward, use the lower metal width, otherwise use the upper layer width.

em { <em1> <em2> ... }: specifies maximum EM current associated with each via area: current <em1> for area <a1>, and so on. Units: mA.

EMV_VS_AREA : specifies the maximum EM current for area of the via cut. Interpolation is used for missing areas, but not extrapolation. If multiple cuts exists, max current is multiplied by the number of cuts. Units: mA. Supports regular arithmetic operations to calculate EM limits based on area. Note that there should be no spaces between operators and numbers in representing a limit. This keyword does not come from rhtech because Star-RCXT does not include it. Like the 'EM' option, you must manually add it to the technology file. See the use of the GSR keyword MERGE_ABUTTED_CUTS to combine abutted cuts into one (section "MERGE_ABUTTED_CUTS", page C-835). This option is also used to support slot (non-square) vias in newer technology designs.

EM_TEMP_RATING <temp1> <derating_factor1> : specifies the ratio of maximum current density between <temp1> and TNOM_EM. So the maximum current density limit at <temp1> can be calculated by multiplying the <derating_factor1> by TNOM_EM. Different ratings for upflow and downflow current directions can be specified. *Note that this setting only affects Average EM limits, and not RMS and PEAK limits.*

DIRECTION_BASED_EM_TEMP_RATING: specifies the ratio of maximum current density between <temp1> and TNOM_EM, for derating that is different in the UPFLOW and DOWNFLOW directions. The maximum current density at <temp1> is calculated by multiplying the <derating_factor1> by TNOM_EM. *Note that this setting only affects Average EM limits, and not RMS and PEAK limits.* Also note that this is outside of the TOPOLOGY_BASED_EM keyword, and that

via_area (<a1> <a2> ...}: specifies total areas for vias in design

em { <em1> <em2> ... }: specifies maximum EM current associated with each via area: current <em1> for area <a1>, and so on. Units: mA.

POLYNOMIAL_BASED_EM_* : defines tables for maximum DC (or AVG), RMS, and PEAK EM current, based on the lengths and widths of the attached metal, and EM_POLYNOMIALS entries define maximum EM limit as functions of length and width for each length and width range. The larger of the lengths of attached Upper and Lower metal layer segments is used in the via EM calculation equation (units: mA).

Each group of EM_POLYNOMIALS represents a specified LENGTH_RANGE, and each line represents a specified WIDTH_RANGE. If WIDTH_RANGES is specified before LENGTH_RANGES, then WIDTH_RANGES is used as the primary table index.

Temperature derating only for DC (AVG). Use one very large length entry to indicate no length dependency. For the RMS polynomial relationship, delta_T can be included to represent the expected temperature change due to Joule heating (set the delta_T value with the GSR keyword DELTA_T_RMS_EM).

VIAMODEL

Required for Totem if WIDTH sub-keyword is not defined in VIA keyword. This information is usually available from the LEF file.

Syntax:

```
VIAMODEL <model_name>
{
<metal_layer_name>  <X_loc_lower_left> <Y_loc_lower_left>
<X_loc_upper_right> <Y_loc_upper_right>
<via_layer_name>    <X_loc_lower_left> <Y_loc_lower_left>
<X_loc_upper_right> <Y_loc_upper_right>
<metal_layer_name>  <X_loc_lower_left> <Y_loc_lower_left>
<X_loc_upper_right> <Y_loc_upper_right>
}
```

Example:

```
VIAMODEL via1
{
metal2  -0.2 -0.2 0.2 0.2
v1      -0.095 -0.095 0.095 0.095
metal1  -0.21 -0.21 0.21 0.21
}
```

Global Switching Configuration (GSC) File

The Global Switching Configuration file specifies the switching status of blocks, instances, and voltage domains in the design for both power calculation and simulation steps, using the fixed state keywords or multi-state definitions from SIM2IPROF, AVM, or APLMMX.

You can simulate multiple states in a vectorless simulation for macros with a multiple-state current model, in a GSC-based vectorless flow. You must ensure that the multiple states are captured in the current file as defined with the CUSTOM_STATE_SIM_TIME configuration file keyword in SIM2IPROF or APLMMX. Use predefined and user-defined custom states in the GSC file. With extended simulation time the same sequence is repeated.

Note the following GSC behavior for different design conditions:

- If all GSC setting values are for a single cycle, Totem repeats the state over the full simulation time.
- If any GSC setting includes multiple states, Totem triggers multi-cycle GSC flow.
- During multi-cycle GSC flow, for cells with multiple state settings in the GSC, Totem repeats the pattern.
- For cells with a single state setting in the GSC, Totem honors the first cycle state and randomly picks states for succeeding cycles.
- If any instance is repeated in multiple GSC files, values that are changed in a later GSC file are used, and warning GSC-028 is displayed to indicate that previous values for the instance are overwritten. Unchanged values are retained in a later GSC file .

The name of the GSC file is specified in the GSR file (see section "GSR File Keywords", page C-773) and the state definitions are in the GSC config file:

Syntax (in GSR file):

```
GSC_FILES <gsc_FilePathName>
```

Syntax (in GSC file):

```
[<blockName> | <instanceName> ] ? <domain_name>?
```

```
<state1> ? <state2> ... ?
```

where <stateN> can be

[UNDECIDED | TOGGLE |HIGH | LOW | STANDBY | DISABLE | ENABLE | OFF | <custom_state_name>]

Wildcards (*) can be used for defining sets of blocks and instances. The default state of an instance or block is UNDECIDED unless defined otherwise.

NOTE: Reading large GSC files sometimes can cause long run times because of iteration to map hierarchical names, particularly if some are not in the design. For GSC files that contain only instance names (that is, no wildcard or block names in the GSC file), you may add a comment line in GSC file

```
# EXACT_INSTANCE_NAME_MATCH
```

which instructs Totem to do faster exact and direct name mapping.

Following any section of exact name matching, a comment line "# END_EXACT_INSTANCE_NAME_MATCH" must be inserted to differentiate exact matches from any subsequent normal wild card entries. If an "# END_EXACT_INSTANCE_NAME_MATCH" comment is missing in the GSC file, it will automatically be appended at the end of the GSC file.

The following keywords are used to define the state of cells (instances of standard cells, memories, I/Os, hard macros), blocks, or domains during simulation:

<blockName>/<instanceName> : specifies the name of an instance or block for which you want to set the state

<domain_name>: optionally specifies a domain net name in <blockName>/<instanceName>, for which you want to set the state. Note that <domain_name> can be used without <blockName>/<instanceName>

UNDECIDED (default): the state of the instance/block is determined by Totem during simulation.

TOGGLE: specifies that the instance *will* toggle during the simulation, and Totem determines its switching direction. If a block is specified, *all instances in the block's hierarchy toggle during simulation.* This state is not recommended for realistic analysis.

HIGH: identifies the high switching charge condition, which could be either Write or Read for memory, or a 0->1 change for a single output, or the representative output.

Note: HIGH and LOW designate the order of APL characterization, not necessarily the power level.

LOW: identifies the low switching charge condition, which could be either Write or Read for memory, or a 1->0 change for a single output, or the representative output.

DISABLE: specifies that the block or instance is inactive. However, instances in this state consume leakage power.

ENABLE: specifies that the block or instance is active.

OFF: specifies that the switch instance or power-gated block/instance is in the OFF state. Instances in this state consume zero leakage power.

<custom_state_name>: specifies a multi-state definition provided in SIM2IPROF, AVM or APLMMX.

<state2> ... : added state definitions for cycle 2 and after, which can be any of the predefined or user-defined custom states; the first state defines cycle 1, <state2> defines cycle 2, and so on.

Example:

```
GSC_FILES DesignABC/gl_sw2.gsc
```

Then in the GSC file *DesignABC/gl_sw2.gsc*, the contents could be:

```
block2C toggle
instB* standby
instC1 LOW StateB2 HIGH StateD1
ABC VDD1 High
```

Note that to read in a GSC file on the fly, use the TCL command:

```
import gsc ABC.gsc
```

where *ABC.gsc* is the GSC filename.

You ***cannot use the command:***

```
gsr set gsc ABC.gsc
```

Global System Requirements File (*.gsr)

The Global System Requirements (GSR) file contains the input design file specifications, operating conditions for chip for power calculation, static IR drop and EM analysis, and dynamic voltage drop analysis.

Totem can read shell environmental variables from GSR file. However, this is not supported for gds config file. For example,

```
APL_FILES {
    $PWD/$DESIGN\.spcurrent current
    $PWD/$DESIGN\.cdev cdev
}
GSC_FILE `pwd`/$DESIGN\.gsc
```

When specifying values in the GSR file, Totem supports the following units and prefixes. Units are case sensitive.

Unit prefixes

- terra = t = 1e+12
- giga = g = 1e+9
- mega = M or me = 1e+6
- kilo = k = 1e+3
- milli = m = 1e-3
- micro = u = 1e-6
- nano = n = 1e-9
- pico = p = 1e-12
- femto = f = 1e-15

Example: 1.3p = 1.3e-12 (there is no space between the number and the unit symbol)

Unit length conversions

- 1 mil = 0.001 inch = 2.54e-5 meters
- 1 inch = 2.54e-2 meters
- 1 micron = 1.0e-6 meters

GSR File Keywords

NOTE: Keyword names make no distinction between upper and lower case.

Input Data Keywords

ADD_LEF_CELL_FOR_POWER

By default Totem assigns zero power to instances with no LIB data in the design. By setting ADD_LEF_CELL_FOR_POWER to 1, Totem computes switching power even for instances without LIB data. Internal power and leakage power are still ignored. *Optional. Default: 0 (off).*

Syntax:

```
ADD_LEF_CELL_FOR_POWER [ 0 | 1 ]
```

Example:

```
ADD_LEF_CELL_FOR_POWER 1
```

AUTO_PAD_CONNECTION_LAYERS

Creates pad instance connection plocs automatically. Totem searches within the specified range of each pad cell by layer and generates one P/G source for each pad cell at the closest location on the specified layers. Note that you cannot specify net names in the PLOC when using AUTO_PAD_CONNECTION_LAYERS or an error message is generated. *Optional. Default: none.*

Note: this keyword replaces previously-used keyword PLOC_SNAP_DISTANCE.

Syntax:

```
AUTO_PAD_CONNECTION_LAYERS {
    <layerA> <snap_range>
    <layerB> <snap_range>
    ...
    DEFAULT <default_snap_range>
}
```

where

If a layer has no range specified, but the DEFAULT range is set, the default value is used.

If no range is set for a layer and no DEFAULT range is set, a pin is not snapped for the layer.

Example:

```
AUTO_PAD_CONNECTION_LAYERS {
    M1 10.0
    M2
    M3 15.0
    DEFAULT 5.0
}
```

BLOCK_INSTANCE_POWER_FILE

To provide cell type (master cell for block) wise power number so that tool can assign the same power for all its instantiations.

Syntax:

```
BLOCK_INSTANCE_POWER_FILE {
```

```
        FULLCHIP <block_name> <file_name>
        CELLTYPE <block_cell_name> <file_name>
    }
```

where

<file_name> gives the detailed per instance power under the block, unspecified instance will be assigned zero power.

<block_cell_name> is the master block name which is instantiated multiple times in the design.

Please note:

a. Blocks covered under FULLCHIP IPF file, will be filtered out to assign the user given power specified by CELLTYPE.

b. If both INSTANCE_POWER_FILE and BLOCK_INSTANCE_POWER_FILE are set for an instance, priority will be given to INSTANCE_POWER_FILE power number.

BLOCK_POWER_ASSIGNMENT

For designs that are in early stages of development, and do not have complete placement and routing information, the BLOCK_POWER_ASSIGNMENT keyword allows designers to evaluate the grid response to power assignments within unfinished blocks or sub-areas of the chip. These areas for analysis can be without assigned instances or they may contain instances that do not have power information yet. So in the early stages of design, you can make power assignment assumptions about a block or region and then analyze the grid response at nodes in the assignment area in terms of voltage drop, current, and EM values. Totem automatically assign VSS current based on VDD power assignment and P/G arc information.

There are four primary ways in which BPA is used to assign power for analysis:

a. pin-based block power assignment

b. use an existing defined block using the BLOCK/PIN options

c. create a new block definition using FULLCHIP/REGION options, or

d. define a modified area within a block for analysis

Each of these types of analysis has its own syntax and usage, which are described in the following sections. *Required for early design analysis. Default: None.*

a. Define pin-based block power assignment

MMX pin-based region power assignment is supported for MMX instances that have many P/G pins inside to represent transistors using keywords MMX_PIN and MMX_REGION. So the power is only distributed to P/G pins, while regular region-based BPA assigns power to nodes on the given layer within the region. This feature is used to change the power distribution inside a MMX instance for static analysis only. To assign power to a DEF instance, or a region of a mixed signal design, the syntax is:

```
BLOCK_POWER_ASSIGNMENT {
    [<DEF_inst_name> MMX_PIN ALL <NetName>
        [<power_W>|<GndCurrentA> ]
    ...
    [<DEF_inst_name>/<BPA_region_name>/<subregion_name>/...
        MMX_REGION ALL <NetName>
        [<powerW>|<GndCurrentA> ] <llx> <lly> <urx> <ury> ]
```

```
        ...
        <cellName> MMX_CELLTYPE ALL <pinName>
            [<powerW>|<GndcurrentA>]
        ...
    }
```

where

<DEF_inst_name> MMX_PIN ALL : specifies the name of an entire DEF instance to have power assigned

<NetName>: specifies the name of net to be analyzed

<DEF_inst_name>/<BPA_region_name>/<subregion_name> : specifies the hierarchy of a defined and named region to have power assigned

MMX_REGION ALL : indicates a sub-area of the full instance is to be defined and considered

llx> <lly> <urx> <ury> : specifies the lower left and upper right x,y coordinates of the region to be analyzed

<cellName> MMX_CELLTYPE : specifies pin power for a given MMX cell name and for all MMX instances of the same cell.

Example - :MMX Pin-based Region Power Assignment

Using the MMX_PIN and MMX_REGION keywords, power can be distributed to MMX P/G pins, while regular region-based BPA assigns power to nodes on the given layer within the region. This feature is used to change the power distribution inside a MMX instance for static analysis only.

You can assign power :

- to transistor pins inside MMX instances using the MMX_PIN keyword in BPA.
- to regions inside MMX instances using MMX_REGION keyword and a “/” character to indicate hierarchy in the first column.
- hierarchically by defining a region inside another region using the MMX_REGION keyword. Use “/” character to indicate hierarchy.

For example:

```
BLOCK_POWER_ASSIGNMENT {
  BlockABC/ram64/adsU1 MMX_PIN all VDD 0.18
  BlockABC/ram64/adsU1 MMX_PIN all VSS 0.1
  BlockABC/ram64/adsU1/R1 MMX_REGION all VDD 0.04 1789 980 1822 1031
  BlockABC/ram64/adsU1/R1 MMX_REGION all VSS 0.01 1789 980 1822 1031
  BlockABC/ram64/adsU1/R10 MMX_REGION all VSS 0.03 1842 934 1888 966
  BlockABC/ram64/adsU1/R10/R11 MMX_REGION all VSS 0.02 1843 935 1863 965
  BlockABC/ram64/adsU1/RR REGION all VDD 0.09 0 0 40 40
  BlockABC/ram64/adsU1/RR REGION all VSS 0.05 0 0 40 40
}
```

In general, block power is assigned hierarchically, so power of a child block of the same type as the parent is considered part of the parent’s power assignment.

In the above example, the final power for this instance is 0.18W for VDD and 0.1A for VSS. Power/current in region R1 is 0.04W for VDD and 0.01A for VSS. Current in region R10 is 0.03A for VSS. Current in region R10/R11 is 0.02A for VSS.

The last two lines in this BPA setup example are regular metal/via region-based BPA settings. Instance *BlockABC/ram64/adsU1/RR* is created and is assigned 0.09W for VDD and 0.05A for VSS. However, since the */RR region is not type MMX, its power is not included in the MMX region assignments.

b. Existing block power assignment

To assign power to an existing block/instance from LEF/DEF, the syntax is:

```
BLOCK_POWER_ASSIGNMENT {
    [ <instName> [BLOCK |PIN ] |
     [ <layer_name> | ALL | TOP | BOTTOM ]
    <netName>?:<pinName>?
    [ <powerW> |<GndCurrentA> | -1 ]
    PER_NODE
    ...
    }
```

Or for master cell-based BPA:

```
BLOCK_POWER_ASSIGNMENT {
     <master_cellName> CELLTYPE
    [ <layer_name> | ALL | TOP | BOTTOM ] <pinName>
    [ <powerW> |<GndCurrentA> | -1 ]
    ...
    }
```

where

<instName>: name of existing instance

<layer name> : specifies the layername for assignment

ALL : includes all layers. Multiple layer specifications can be provided for the same instance and net, in which case the power or current is added and distributed equally between nodes on the P/G grid on the specified layers. If ALL layers are specified, all nodes on the grid are considered.

TOP/BOTTOM : includes either the highest or the lowest mask layers overlapping the BPA instance. When layername, TOP, or BOTTOM specifications are used, *all* power/current is assigned to the BPA blocks on the top, bottom or the specified metal layer in the layout.

<netName>: specifies the power/ground net to have power assigned. All nets to be included must be specified on a separate line (switch sub-nets are included automatically)

<pinName>: optionally, power for a particular pin can be specified

<powerW>/<GndCurrentA> : specifies either the power in Watts for power nets, the ground current in Amps for ground nets,

'-1' : allows power calculation to determine the appropriate power/current based on other information available, such as toggle rate, BPFS and APL characterization. This can be used for either cell instances (blocks) or regions. You must provide correct and complete power calculation data so that power for the block can be calculated accurately. See section "Power Calculation", page 4-149, for details. Use one line in the BPA specification for every domain (net) in the block. Note that if one net in the block is assigned '-1' for power/current in BPA, power/current for *all* domains in the block is determined by the power calculation engine.

PER_NODE : assigns power/current per VIA/CONTACT inside a block instead of assigning to the total block. To ensure single node is created per VIA/CONTACT set the keyword SPLIT_SPARSE_VIA_ARRAY accordingly.

<master_cellName> CELLTYPE : specifies assigning power by master cell type, where layer and pin names must be provided

c. Create a new block definition for power assignment

c1. To assign power to a user-defined block/instance and assign power, the syntax is:

```
BLOCK_POWER_ASSIGNMENT {
    <BPA_instName> REGION
    [ <layer name> | ALL | TOP | BOTTOM ]
    <netName>?:<pinName>?
    [ <powerW>|<GndCurrentA> ] <llx> <lly> <urx> <ury>
    ...
    }
```

where

<BPA_instName> REGION : specifies a user-defined name for the region to have power assigned, which *cannot* be the design name, nor the name of an existing cell or instance. This also means <BPA_instName> becomes an instance in the design and must be used in other file references also, such as STA and GSC.
For backward compatibility, note that for prior-release GSR files that have used the former FULLCHIP ..." syntax in B_P_A, Totem substitutes the region name “adsFULLCHIP” to allow static analysis to proceed. However, you must use the “adsFULLCHIP” <BPA_instName> also in STA, USER_STA and GSC files to perform dynamic analysis.

<llx> <lly> <urx> <ury>: specifies lower left and upper right corner coordinates of defined region

c2. To create a new instance for cells defined in DECAP_CELL, or to use the BLOCK_POWER_MASTER_CELL keyword (see section "BLOCK_POWER_MASTER_CELL", page C-782), and also allow overlaps, the syntax is:

```
BLOCK_POWER_ASSIGNMENT {
    <BPA_instName> <cellName>
    [ <layer name> | ALL | TOP | BOTTOM ] <netName>
    [<powerW>|<GndCurrentA>]
    <llx> <lly> <orientation>
    ...
    ?<BPA_instName> <cellName> OVERLAP_OK?
    }
```

where

<cellName> : cell name specified in GSR keyword DECAP_CELL or BLOCK_POWER_MASTER_CELL

<llx>,<lly> : specifies lower left corner of cell

<orientation> : specifies orientation of cell

OVERLAP_OK : for instances that overlap defined BPA regions, ensures that the instance is not deactivated in such cases. The BPA regions that overlap any instance other than the parent instance must be defined as 'OVERLAP_OK'.

c3. To assign power in a region that includes the entire design, the syntax is:

```
BLOCK_POWER_ASSIGNMENT {
    <BPA_instName> FULLCHIP
    [ <layer name> | ALL | TOP | BOTTOM ]
    <netName>?:<pinName>?
    [<powerW>|<GndCurrentA>]
    ...
```

```
    }
```

where

FULLCHIP: specifies that the whole design is included in the <BPA_instName>

c4. For mmx-level power distribution, using GDS keyword WHITE_BOX_CELLS, for hierarchical static analysis use syntax as follows:

```
BLOCK_POWER_ASSIGNMENT {
    <mmx_cellname> MMX_CELLTYPE ALL <net_name>
[<power>|<current>]
    ...
}
```

Then in the *gds.config* file (GDSII_FILE) set:

```
WHITE_BOX_CELLS {
...
}
```

c5. For BPA REGION assignments with multiple power/ground nets, you can define pins and P/G arcs for the master cell of the BPA region in the custom library file, and specify net-pin connections in BPA with a '<netName>:<pinName>' format.

For example, for a BPA region *TOP/region1* in power domains VDD1 and VDD2, an example custom lib file would look as follows:

```
cell TOP_region1 {
  pin vdd1 {
    type vdd
  }
  pin vdd2 {
    type vdd
  }
  pin vss {
    type vss
  }
  pgarc {
    vdd1  vss
    vdd2  vss
  }
}
```

And the associated example GSR would be as follows:

```
BLOCK_POWER_ASSIGNMENT {
    TOP/region1 REGION M1 VDD1:vdd1 0.1 100 100 300 450
    TOP/region1 REGION M1 VDD2:vdd2 0.1 100 100 300 450
    TOP/region1 REGION M1 VSS:vss 0.2 100 100 300 450
}
```

You can also exclude regions from power assignment in early analysis for designs at a stage where place and route is complete for some hierarchical blocks, and you want to use power calculation data for the completed blocks and use power assignment for rest of the design.

For power/current assigned to hierarchical elements in a design, power/current assigned to child elements are included in the parent's assignment. So a parent block's power assignment is the sum of the power assigned specifically to its children and also the power assigned to areas that are outside of its children.

Totem reports the BPA region instances in the file *<design>.inst.pin* and in files *<design>.inst.arc* and in *<design>.inst.worst*.

d. Define a modified area within a block for power assignment

a. To define sub-regions inside an existing BPA instance (multiple specifications on the same instance are allowed) for power assignment, the syntax is:

```
BLOCK_POWER_ASSIGNMENT (
    <BPA_instName> BLOCK AREA <llx> <lly> <urx> <ury>
    <BPA_instName> BLOCK RECTILINEAR <x1 y1 x2 y3 x4 y5 ..>
    <BPA_inst_name> BLOCK CURRENT_DENSITY <node_spacing>
    [<powerW>|<GndCurrentA> ]
    ...
    }
```

where

BLOCK AREA <llx> <lly> <urx> <ury>: specifies a rectangular area to be assigned power with x,y coordinates

BLOCK RECTILINEAR <x1 y1 x2 y3 x4 y5 ... >: specifies corners of the rectilinear in order (either direction). The sides must be either horizontal or vertical. To avoid repetition of the same coordinate value, you can use a reduced format for providing the coordinates, in which a repeated x or y value can be dropped, such as “x1 y1 x2 y3 x4 y5 ...”, where y2 x3 y4 x5 … are repeated values and do not need to be re-entered. Also, to assist in keeping track of x,y values for the reduced format, the following format is allowed “20x 15y 190x 30y 150x 80y 70x 20y 20x”, in which case the letters are ignored. However, for ease of use in understanding and reading back the x and y values, the full syntax with repeated x and y coordinates is often preferred.

BLOCK CURRENT_DENSITY <node_spacing> : assigns current sinks and sources with different spacing for different BPA instances (microns)

b. To define and exclude a region from BPA power assignment, the syntax is:

```
BLOCK_POWER_ASSIGNMENT (
    <regionName> REGION EXCLUDE <llx> <lly> <urx> <ury>
    [<powerW>|<GndCurrentA> ]
    ...
    }
```

The INCLUDE and EXCLUDE capabilities make it easier for you to define areas that are not complete rectangles to be specified for power assignment. The EXCLUDE option allows you to define rectangular regions to be excluded from areas occupied by BPA blocks/regions, and the INCLUDE option then can *add back* areas to be included *within* EXCLUDE areas.

To define and exclude an overlapping instance in the design from BPA power assignment, or exclude certain cellnames, the syntax is:

```
BLOCK_POWER_ASSIGNMENT (
    [ <instName> BLOCK | <cellname> CELLTYPE ] EXCLUDE
    ...
    }
```

To modify an area defined by 'EXCLUDE' to form rectilinear regions, the syntax is:

```
BLOCK_POWER_ASSIGNMENT (
    <regionName> REGION INCLUDE <llx> <lly> <urx> <ury>
    [<powerW>|<GndCurrentA> ]
    ...
```

```
}
```

To use the boundary box of an existing instance to do the same as described above, add the line:

```
<instName> BLOCK INCLUDE
```

c. To exclude macro instances overlapping a BPA instance for power assignment, add the line:

```
<BPA_instName> BLOCK EXCLUDE_MACRO
```

To exclude instances defined in the BLOCK_POWER_FOR_SCALING keyword, and which overlap the BPA instance for power assignment, add the line:

```
<instName> BLOCK EXCLUDE_BPFS
```

The BLOCK EXCLUDE_MACRO/BPFS options specify that areas under any macro that overlap the specified block are ignored, and the power assigned to the block is not distributed to the excluded macros/memories. This is applicable to blocks defined as FULLCHIP, BLOCK or REGION.

d. To exclude an area occupied by all instances overlapping the BPA instance, add the line:

```
<BPA_instName> BLOCK OVERLAP_OK}
```

Example:

```
BLOCK_POWER_ASSIGNMENT {
    65NM_ABC FULLCHIP M6 VDD 0.6
    region1 REGION via3 VDDC 0.3 120.0 450.0 200.5 530.5
    INST1 BLOCK EXCLUDE
    regionNotForBPA REGION EXCLUDE 1000 1000 4000 4000
    INST1/I2 BLOCK INCLUDE
    subRegionForBPA REGION INCLUDE 1500 1500 2000 2000
    X0/adsU1/RR BLOCK OVERLAP_OK
    }
```

In the above example, areas inside excluded regions, as occupied by instance INST1/I2, and the rectangle (1500 1500 2000 2000) are included and used for power assignment. Note that regions to be included or excluded can be specified either by coordinates, or by instance name, and that INCLUDE statements can only be applied to and modify EXCLUDED areas.

e. To assign per transistor power/current, using MMX_XTOR constraints in BLOCK_POWER_ASSIGNMENT settings. Transistor-based BPA is ignored if the current due to assigned power exceeds the IDSAT current for the transistor. The syntax is:

```
BLOCK_POWER_ASSIGNMENT {
    <instance_hierarchy/<device_name> MMX_XTOR all
    <net_name> [<power>|<current>]
}
```

For example,

```
BLOCK_POWER_ASSIGNMENT {
    adsU1/X0.X1 MMX_XTOR all VDD 0.2
    adsU1/X0.X1.M0 MMX_XTOR all VDD 0.001
}
```

e. Assigning power in White Box Cells with Block Power Assignment

The Totem White Box Cell + BPA flow can accept an input BPA file along with the cell names from GDS to assign power using BPA option MMX_CELLTYPE to create MMX_REGION data for those cell names. You are not required to white box any of the cell names provided in the BPA file. The syntax is:

```
BLOCK_POWER_ASSIGNMENT {
    <cell_name> MMX_CELLTYPE ALL <net_name>
    [<power_Watts | Gnd_current_Amps>]
    ...
}
```

In the following example, 0.022 W is assigned to white box cell *x1adda* for net VDD:

```
BLOCK_POWER_ASSIGNMENT {
    adsU1 MMX_PIN ALL VDD 0.0001
    x1adda MMX_CELLTYPE ALL VDD 0.022
}
```

BLOCK_POWER_ASSIGNMENT_FILE

Specifies one or more files for defining parameters for early power analysis. User can set/append values of this keyword using 'gsr set'/'gsr set'/'gsr append' TCL commands. These commands can be executed at any stage of RedHawk run, even before setup design stage. *Optional. Default: none.*

Syntax:

```
BLOCK_POWER_ASSIGNMENT_FILE <file1>
BLOCK_POWER_ASSIGNMENT_FILE <file2>
...
```

or

```
BLOCK_POWER_ASSIGNMENT_FILE {
<file1>
...
}
```

Example:

```
BLOCK_POWER_ASSIGNMENT_FILE {
bpa.txt
BPA_xyz.txt
}
```

The syntax and sample contents of a block power assignment file is as follows:

```
BLOCK_POWER_MASTER_CELL {
<BPMC_parameter_values>
...
}

BLOCK_POWER_ASSIGNMENT {
<BPA parameter_values>
...
}

BLOCK_POWER_MASTER_CELL {
CELL2 0 0 1980 1720
}
BLOCK_POWER_ASSIGNMENT {
INST1 CELL2 METAL4 vdd_i 0.01 1000 2000 N
```

```
INST1 CELL2 METAL4 vss 0.00920 1000 2000 N

REGION2 REGION METAL6 vdd_e 0.00500 14250 1500 16750 3500
REGION2 REGION METAL6 vss 0.005259 14250 1500 16750 3500
}
```

The specified values in the BPA section of the GSR are recorded in the output file *adsRpt/apache.blockPowerAssign*.

BLOCK_POWER_MASTER_CELL

Specifies multiple regions, and optionally rectilinear areas inside BPA REGIONs or in LEF/DEF blocks, that share a common master cell. Rectilinear area definitions enable you to distribute power to specified sub-areas within a rectilinear block. Power/current assigned to REGIONs/blocks is hooked up within the user-defined rectilinear areas. You must define the common master cells and the rectilinear regions within it using a BLOCK_POWER_MASTER_CELL definition and the BLOCK_POWER_ASSIGNMENT keyword.

For each master cell the bboxes inside the brackets {} define rectilinear subareas inside the cell for power assignment. You must make sure that the bboxes inside each cell fall within the bbox of the cell; areas outside of the master cell bbox are discarded. *Optional. Default: none.*

Syntax:

```
BLOCK_POWER_MASTER_CELL {
  <master_cell_name1> <BB_llx> <BB_lly> <BB_urx> <BB_ury> {
    ? AREA <llx> <lly> <urx> <ury> ?
    ...
   ? RECTILINEAR <x1 y1 x2 y3 x4 y5 ...>?
    ...
  }
}
```

Example:

```
BLOCK_POWER_MASTER_CELL {
rcell 0 0 400 200
myRegion 0 0 200 100 {
  0 0 100 100
  100 0 200 50
 }
 defCell 0 0 200 300
 {
  0 0 200 100
  0 100 100 300
 }
}

BLOCK_POWER_ASSIGNMENT {
regionA rcell MET3 vdd 0.12 100 100 N
regionA rcell MET3 vss 0.12 100 100 N
regionB rcell MET1 vdd 0.1 500 100 E
regionB rcell MET1 vss 0.1 500 100 E
REGION1 myregion MET5  VDDC 1 1850 1030 N
REGION1 myregion MET5  VSS 1 1850 1030 N
```

```
INST2 BLOCK  MET5 VDDC 0.5
INST2 BLOCK  MET5 VSS 0.5
}
```

In this example, regions ‘regionA’ and ‘regionB’ share the same master cell, ‘rcell’, and ‘defCell ‘ is the master cell of ‘INST2’.

BLOCK_SAIF_FILES

Supports the use of hierarchical block SAIF activity files, and allows you to specify for hierarchical blocks the SAIF file path, the root instance path, and a substitute instance path. *DMP compatible. Optional. Default: none.*

Syntax:

```
BLOCK_SAIF_FILES {
   SAIF_FILES {
        <block name> <SAIF_file_path>
        ROOT_INSTANCE <root_instance_path>
        SUBSTITUTE_PATH substitute_instance_path>
  }
  }
```

Example:

```
BLOCK_SAIF_FILES {
  SAIF_FILE {
    green_core3 green.saif
    ROOT_INSTANCE green_tst/inst1/
    SUBSTITUTE_PATH green_core3/
  }
  ...
}
```

BLOCK_STA_FILES

Running STA non-hierarchically is the most accurate and recommended method. However, BLOCK_STA_FILES allows hierarchical STA files to be used by Totem in analysis. The hierarchical STA files are compiled into a composite file as if the STA data was flat. Multiple instantiations of a hierarchical STA block are allowed, but for best results every block specified should have its own STA data with correct boundary conditions. Using BLOCK_STA_FILES for a top level STA file is also acceptable.

Hierarchical STA works best under the following conditions:

- Blocks have donut style designs, with all paths having FFs on both input and output.
- All instances driven by block specific FFs have updated timing windows.
- Instances of the same block have symmetrical instantiations.
- If there is a top level timing block, it should be specified in BLOCK_STA_FILES

Conditions in which hierarchical STA analysis is *not recommended* are:

- Many feedthrough paths in the block instances.
- Boundary conditions are very different between blocks.
- Blocks have different STA modes and need to be simulated as such.

FREQ_OF_MISSING_INSTANCES <frequency_in_Hertz> specifies the operating frequency for instances not in the STA file, but in the design (the default is 0).

INSTTYPE is optional with STA file of top block and instances.

CELLTYPE should be used with STA file of any hierarchical CELL. Each instance of this cell is annotated with the given STA file.

Note: If both BLOCK_STA_FILES and STA_FILES are used, then BLOCK_STA_FILES is ignored, and if a USER_STA_FILE is also used, any common data in the specified USER_STA_FILE overrides data in either of the other files.

See section "pt2timing", page E-1009, for more information on how to use hierarchical STA blocks. *DMP compatible. Optional. Default: none.*

Syntax:

```
BLOCK_STA_FILES {
    FREQ_OF_MISSING_INSTANCES <frequency_in_hertz>
    [<top_block> | <Hierarchy_path/instance_name>]
        <filename> ? INSTTYPE ?
    ...
    <cell_name> <filename>  CELLTYPE
    ...
}
```

Example:

```
BLOCK_STA_FILES {
    Top_level_block top.timing
    InstA1  abc12.timing
    InstB2  efc23.timing
    InstC3  xyz56.timing
    InstC3/instd1 ghi78.timing INSTTYPE
    BlockZ1 blockz1.timing CELLTYPE
}
```

BPA_BY_CURRENT

When set, keywords BLOCK_POWER_ASSIGNMENT and BLOCK_POWER_FOR_SCALING accept current values for both power and ground domains, in Amperes. *Optional. Default: 0.*

Syntax:

```
BPA_BY_CURRENT [ 0 | 1 ]
```

Example:

```
BPA_BY_CURRENT 1
BLOCK_POWER_ASSIGNMENT {
    adsU1/R1 REGION ALL vddcp 0.02 75.849 265.795 185.445 642.312
    adsU1/R1 REGION ALL gnda 0.0111 75.849 265.795 185.445 642.312
}
```

In this case a 0.02 Amp current is assigned to domain vddcp, and 0.0111 A to domain gnda.

BPA_BY_LAYER

By default Totem distributes power assigned using BPA equally among the nodes of all layers specified in the BPA statement, without taking into consideration per layer breakup. This keyword allows power assignment per layer, and Totem honors the power/current assigned to each BPA statement for each layer, and does *not* uniformly distribute power among all nodes in the net. However, note that if a BPA block that has nets or layers assigned a power or current of -1, 'pwrcalc' computes

the power for the block and the power is distributed uniformly for all nodes on different layers as before. *Optional. Default: 0 (off)*.

Syntax:

```
BPA_BY_LAYER [ 0 | 1 ]
```

BPA_CONN_MARGIN

Defines a margin to be extended beyond the original bounding box of a BPA instance, to enlarge the area where current sinks on nodes can be selected for power/current assignment. This is important for cases in which the nodes are at or very near the BPA boundary, and where small inaccuracies or roundoff in co-ordinate value can cause the node to lie outside of the BPA boundary and cause disconnects. *Optional. Default: 0.0 (uses the original BPA bbox).*

Syntax:

```
BPA_CONN_MARGIN <distance_um>
```

BPA_CONN_DISTANCE

Defines a distance around the original bounding box of a BPA instance to search for a node to be connected to the BPA instance for current assignment. Note that **just one** nearest node outside of the bbox is selected using this keyword. This keyword is suitable for cases when specified region is very small and no nodes are found in it, so it searches for the nearest node within specified distance to the region and creates a current sink. Or it can be used when you want the BPA region to perform as a single current sink point and the BPA area is too small to accurately specify a bbox that is small enough to cover just one node. *Default: 0.0 (uses the original BPA bbox).*

Syntax:

```
BPA_CONN_DISTANCE <distance_um>
```

BPA_CURRENT_DENSITY

Allows power/current sinks to be distributed uniformly on metal layers, not based on via density, but on the global pitch defined by this keyword. This early analysis scenario simulates a condition in which standard cells are already placed. *DMP compatible. Optional. Default: 0.0 (no extra nodes added).*

Syntax:

```
BPA_CURRENT_DENSITY <value>
```

where '<value>' is the pitch in microns (>= 1).

CAP_BLOCKING_LAYERS_TO_OTHER_DIES

Specify layers to avoid extracting the coupling cap to 2ND_GRND or PRTL_2ND_GRND layers. Layers listed in this GSR are blocked from producing coupling capacitances to 2ND_GRND and PARTIAL_2ND_GRND layers, which are layers that represent 3DIC adjacent-die substrate ground planes in single-die flows.

Syntax:

```
CAP_BLOCKING_LAYERS_TO_OTHER_DIES {
<layer1>
<layer1>
..
}
```

CELL_GEOMETRIES_AS_MFILL

Provides control over MFILL pushing of LEF cell geometries to achieve high accuracy in TSMC N10 resistance correlation cases, by allowing obstruction and signal pin geometries to have the most influence on metal etch calculations, including color mask information and exact spacing. This keyword interacts with the GSR keyword READ_LEF_OBS in that setting CELL_GEOMETRIES_AS_MFILL either to OBS or ALL disables the other average spacing flow for obstruction impact on etching. To use the older flow for obstruction modeling, this keyword must be set to [NONE/0] or SIGNAL. *DMP compatible. Optional. Default: 0.*

Syntax:

```
CELL_GEOMETRIES_AS_MFILL { OBS | SIGNAL | ALL/1 | NONE/0 ]
```

where

OBS: pushes all LEF obstructions to metal wires on the MFILL net

SIGNAL: pushes all signal pin instances to metal wires on the MFILL net

[ALL|1]: pushes both obstruction and signal pin geometries to wires on MFILL

[NONE|0]: no geometries are pushed (default)

CELL_PIN_FILE

Specifies a cell pin file that helps reduce extraction runtime in designs involving cells that have many overlapping pins in different metal layers, such as metal1 and metal2. Using this keyword you can control which metal layer should be used for hooking up the current sinks in the analysis. You can specify the cell name, or use the option STD_CELL to cover all the standard cells. *Optional. Default: None*

Syntax:

```
CELL_PIN_FILE <filename>
```

The internal file syntax is as follows:

```
CELL [ STD_CELL | <cell_name> ]
PIN [ <Vdd_pin_name> | <Gnd_pin_name> ]
LAYER <metal_layer>
END PIN
END CELL
```

Pin file content example:

```
CELL STD_CELL
PIN VDD1
LAYER  M1
END PIN
PIN VSS1
LAYER  M1
END PIN
END CELL
```

DEF_FILES

Required keyword that specifies the Database Exchange Files (*.*def*) to be used for the design. The last file listed must be the “top” file for the design.

Note: if different nets connect to the same instance pin, this creates an unacceptable connectivity (connection) collision, and Totem can only pick one of the nets. The chosen net depends on the format of the instance name. The DEF file SPECIALNETS section supports wildcards (*) in instance names, so the priority of net connection selection is as follows:

1. exact instance name, such as *MAC/VCCL_MACRO (MACRO/inst01 DVDD)*
2. wildcard instance name, such as *MAC/VCCK_MACRO (MACRO/* DVDD), or* VCCK (* DVDD)

So if there is a net name collision for an instance whose name has a wildcard, the exact name is kept and Totem issues WARNING (DEF-012) and continues; that is, the wildcard entry has a lower priority in net connectivity. When a net name collision occurs for an exact instance name (no wildcard), Totem issues WARNING (PIS-605) and replaces the first name encountered with the later one. *DMP compatible.*

Note that DEF files should be imported directly using this keyword. Required. Default: None.

Syntax:

```
DEF_FILES {
    <def_FilePathName-1>.def block
    ...
    <def_FilePathName-n>.def top
}
```

where

<def_FilePathName-1>.def : specifies the path and filename for a block; the last *.def* file should be the top-level DEF file.

Example:

```
DEF_FILES {
../design_data/ABCDE.def top
}
```

DEF_IGNORE_LAYERS

Specifies layers in the DEF file to be ignored during design data input for Totem analysis. *DMP compatible. Optional. Default: None.*

Syntax:

```
DEF_IGNORE_LAYERS [
<layer_name1>
...
<layer_nameN>
}
```

Example:

```
DEF_IGNORE_LAYERS {
m4
m6
}
```

DEF_IGNORE_PIN_LAYERS

Specifies PIN layers in the DEF file to be ignored during design data input for Totem analysis. *DMP compatible. Optional. Default: None.*

Syntax:

```
DEF_IGNORE_PIN_LAYERS [
<layer_name1>
...
<layer_nameN>
}
```

Example:

```
DEF_IGNORE_PIN_LAYERS {
metal3
metal5
}
```

DEF_IGNORE_SPECIFIC_LAYERS

For LEF/DEF importing, allows you to ignore specified layers for specified blocks. *DMP compatible. Optional. Default: None*

Syntax:

```
DEF_IGNORE_SPECIFIC_LAYERS  {
    <def_block1_path> "<ignored_layer1> ..."
    <def_block2_path> "<ignored_layer2> ..."
    ...
  }
```

where '<def_blockN_path>' is a specified block path, followed by layers to be ignored.

DEF_IMPORT_REGION

Defines specific rectangular regions of particular def to be imported and retained, even if they are not placed (placement coordinate) inside the region imported from top block. With GSR keyword `IMPORT_REGION,` if a block is not placed within the region imported by top def, that block is completely thrown away. However, with `DEF_IMPORT_REGION,` all the blocks defined will be imported and flattened. This new method will be useful in cases like, when user wants to do a block level analysis, the RDL is placed in the TOP level design at a point outside the tile.*Optional. Default: None.*

Syntax:

```
DEF_IMPORT_REGION {
<Design name in the def> {
<low_left_x> <low_left_y> <up_right_x> <up_right_y>...
       }
}
```

DEF_READ_ALL_IO_NETS

When set, enables importing all primary I/O nets in all DEF blocks in the design hierarchy, so you *do not need to* specify signal net names in each DEF block individually in ESD_SIGNAL_NETS in order to analyze them. Totem automatically traverses all connected metals from all hierarchies of DEF, irrespective of the names in lower level DEFs. *Optional. Default: 0.*

Syntax:

```
DEF_READ_ALL_IO_NETS [0|1 ]
```

DSPF_MAP

Specifies the metal layer mapping configuration file, to identify metal layers both in DSPF and GDS. This mapping is required by Totem to identify what layer is present in the DSPF for the sub-node section. The layer specification in the DSPF file typically can be found in the header of the LAYER_MAP section. Supports tracing multiple layer numbers mapped to one layer name. The format of this map file is as follows:

```
?#DSPF_PAR_METAL [ LVL | LAYER | <other_layer_parameter> ]?
?#USE_DSPF_LAYER_NAME ?
```

```
<tech_layer_name> <dspf_layer_param1> <dspf_layer_param2>
    ... ? trace?
```

where

trace: can be optionally specified if the layers is used for GDS2DB geometry tracing. If none of the layers are specified, APLMMX uses the lowest metal layer from TECH_FILES for GDS2DB geometry tracing.

DSPF_PAR_METAL < > : optional and need not be specified if the value is “lvl” or “layer”, since these two-layer parameters are automatically detected.

Note that DSPF_PAR_METAL is not required if the TRACE_FROM_ILINE keyword is set in the configuration file. APLMMX automatically uses the DSPF *|I line of the MOS transistor as the GDS2DB trace point, so there is no need to provide DSPF metal layer trace setup information.

Note that multiple DSPF layer IDs can be mapped to the same tech file layer.

USE_DSPF_LAYER_NAME : defines a DSPF layer name instead of layer ID in the file, because layer ID in DSPF file may vary when the process node migrates, but the layer name usually is not changed. To specify layer name instead of layer ID, a comment must be put into the file to introduce it, as follows:

```
#USE_DSPF_LAYER_NAME
<Tech_file_layer_name> <DSPF_layer_name> ? trace?
```

Example DSPF layer name specification:

```
#USE_DSPF_LAYER_NAME
M1 metal1  trace
M2 metal2  trace
...
```

where ‘metal1’ and ‘metal2’ are layer names defined in the LAYER_MAP section in the DSPF file.

Syntax:

```
DSPF_MAP <filename>
```

EM_DISABLE_SLICED_MINW_CHECK

When set to 1, turn off the automatic min width sliver check for wires that are cut by WIRE_SLICE_WIDTH to a width less than minWidth. It improves performance and helps in better mesh modeling. Default:0

Syntax:

```
EM_DISABLE_SLICED_MINW_CHECK [1|0]
```

FAST_DEF_READ

Specifies the number of additional processes to be used for importing DEFs, which is recommended for use in the DMP flow. *Optional. Default: 0.*

Syntax:

```
FAST_DEF_READ <num_proc>
```

GDSII_FILE

Specifies the GDS cell name, the configuration filename, and the GDS mode. *Required. Default: None.*

Syntax:

```
GDSII_FILE {
  <cell_name> <gds2db_config_file>
```

```
    [REDUCED | EXTENDED | FULL ] ? TOP ?
  }
```

where

REDUCED : indicates a GDS2DEF model

EXTENDED : indicates a GDS2DEF -m model

FULL: specifies a detailed transistor view (GDSMMX model)

TOP : option indicating that <cell_name> is the top of the hierarchy

Example

```
GDSII_FILE {
  ram26-b gds2db.config full top
}
```

GDS_CELLS

Specifies a set of GDS cells converted by *gds2db* or *gds2db -m*, and where the converted LEF, DEF, and LIB files are located. Note that in the GDS2DB flow, you can use the GDSII_FILE keyword for top level specification and GDS_CELLS for child block specifications.

The switch model file of the sub-block is automatically loaded in the same directory as the LEF/DEF/PRATIO files when you specify the “mmx” option.

Usage Notes:

a. If you use GDS_CELLS input for *gds2db* or *gds2db -m* cells in the design, you must use the LEF_FILES, DEF_FILES, and LIB_FILES GSR keywords for input of any additional LEF, DEF, and LIB files (not converted by *gds2db/ gds2db -m*). In other words, when using GDS_CELLS input, do not use the ‘`import`’ command for any cell files.

b. If there are multiple cells from GDS conversion defined in GDS_CELLS, the top-level hierarchy should be the last entry.

DMP compatible. Optional. Default: none.

Syntax:

```
GDS_CELLS
{
<GdsCellName-1> <path_to_cell-1> ? mmx ?
...
<GdsCellName-n> <path_to_cell-n> ? mmx ?
}
```

where

<GdsCellName-1> <path_to_cell-1> : specifies the cell name and path to the file.

mmx : option that identifies a mixed mode transistor model rather than a detailed memory model

Example:

```
GDS_CELLS
{
mem_cell1 ./user_data/gds2def
}
```

GSC_FILES

Specifies the Global Switching Configuration file to be used and the status of any instances or blocks in the design where power switching occurs. For details of the contents and syntax of the GSC file, see section "Global Switching Configuration (GSC) File", page C-770. Note that to read in a GSC file on the fly, use the TCL command

```
import gsc ABC.gsc
```

where *ABC.gsc* is the GSC filename. You now cannot use the command

```
gsr set gsc ABC.gsc
```

Also, if a VCD file setting exists, the GSC value overrides the VCD for any instances specified. *DMP compatible. Optional. Default: None.*

Syntax in GSR file:

```
GSC_FILES <gsc_FilePathName>
```

Syntax in GSC file:

```
[<instanceName> | <blockName> ] [ TOGGLE | HIGH | LOW |
POWERUP | STANDBY | DISABLE | OFF | UNDECIDED ]
...
```

Example in GSR file:

```
GSC_FILES DesignABC/gl_sw2.gsc
```

Example in GSC file DesignABC/gl_sw2.gsc:

```
blockABC Powerup
block2C toggle
instB* standby
instC1 off
```

HONOR_LEF_PIN_TYPE

When set, disables “name-based” P/G pin/net type setting (default). When Off, LEF nets having “VDD”, “vdd”, “VCC”, or “vcc” as part of their names are considered as power nets, and nets having “VSS”, “vss”, or “gnd” as part of their names are considered ground nets. Used along with keyword PGNET_HONOR_DEF_TYPE for DEF name handling. *DMP compatible. Optional. Default: 1.*

Syntax

```
HONOR_LEF_PIN_TYPE [0|1]
```

IGNORE_PLOC_INTERNALNETS

When set to 0, Totem places plocs on internal nets. By default (1), any plocs placed on internal nets are ignored. *DMP compatible. Optional. Default: 1.*

Syntax:

```
IGNORE_PLOC_INTERNALNETS [ 0 |1 ]
```

IGNORE_PRECHECK_ERROR

If this keyword is turned On, data processing continues, even if a data precheck error is found. *Optional; default: 0 (Off).*

Syntax:

```
IGNORE_PRECHECK_ERROR [ 0|1 ]
```

IMPORT_NETS

When specified, only the net(s) specified are imported. Note that the net names should be full path names after flattening, and also that this keyword works only when the FAST_DEF_READ GSR keyword is set. *Optional. Default: none.*

Syntax:

```
IMPORT_NETS {
<net1_name>
<net2_name>
...
}
```

IMPORT_NETS_FILE

When set, specifies filenames containing signal nets to be selectively imported to reduce time required to the GSR when there are a large number of imported nets. *Optional. Default: none.*

Syntax:

```
IMPORT_NETS_FILE <signal_nets_file>
```

INSTANCE_POWER_FILES

Defines the absolute or relative path from the Totem run directory to the power file, which contains a list of instances and their power consumption, as well as hierarchical prefixes to add the instance names in the IPF files to match them to the design instances. For instances not specified in the power file, the power is assumed to be zero. Therefore, all significant power consumers should be specified, or none. Note that filler and tap cells are ignored when checking IPF coverage. For mixed mode analysis, you can use VCD for block power and INSTANCE_POWER_FILES to assign power to the top level instances (excluding the block). For non-mixed mode flows, INSTANCE_POWER_FILES still takes the highest priority and overrides all other keywords defining power/activity for instances. *Optional; default: None.*

Syntax:

```
INSTANCE_POWER_FILES [ <inst_power_file_path> |
    { <inst1_power_file_path1>
     <inst2_power_file_path2>
     <inst3_power_file_path3> <inst3_hierpath>
     <inst4_power_file_path4> <inst4_hierpath>
     ...
} ]
```

where

<instN_power_file_pathN> : specifies the paths to the IPF files

<instN_hierpath> : specifies the hierarchical prefix to add the instance names in the IPF files to match them to the design instances. If the hierarchical path prefix is not specified, the <instN_power_file_pathN> should specify the instance name with complete hierarchy. If the hierarchical path prefix is specified, each instance name in <instN_power_file_pathN> is prefixed with the <instN_hier_pathN> to match them to the design instances in hierarchy.

Example:

```
INSTANCE_POWER_FILES
    ( designABC/abc/instPowerFileABC HIER_CD
)
```

The format for specifying pin power/current information in the Instance Power File, for multiple Vdd/Vss designs, one pin per line, is:

```
<instance_name> <pin_power_W> <Vdd_pin_name>
<instance_name> <pin_current_A> <Vss_pin_name>
...
```

To specify pin power for single Vdd/Vss designs, the format is:

```
<instance_name> <total power_W>
...
```

KEEP_POWER_DATA

Allows control of unnecessary power data generation. If turned Off, regenerates Totem data for power calculation for each power calculation. By default Totem power data is retained. *Optional. Default: On.*

Syntax:

```
KEEP_POWER_DATA [ 0 |1 ]
```

LEF_FILES

Required to specify the physical Library Exchange Files (*.*lef*) to be used in the design. *Note that LEF files should be imported directly using this keyword. DMP compatible. DMP compatible. Required. Default: None.*

Syntax:

```
LEF_FILES {
    <lef_FilePathName-1>.lef
    ...
    <lef_fFilePathName-n>.lef
}
where
```

<lef_FilePathName-1>.lef : specifies the path and name of the LEF file; the first *.lef* file should contain the technology information.

Example:

```
LEF_FILES {
../design_data/ABCD.lef
}
```

LEF_IGNORE_PIN_LAYERS

Specifies PIN layers in the LEF file to be ignored during design data input for Totem analysis. *DMP compatible. Optional. Default: None.*

Syntax:

```
LEF_IGNORE_PIN_LAYERS [
<layer_name1>
...
<layer_nameN>
}
```

Example:

```
LEF_IGNORE_PIN_LAYERS {
M4
M5
}
```

LIB_FILES

Required to specify Synopsys library files (*.*lib*) or one custom lib file to be used in the design. If a directory is specified, all files in the directory will be selected.

Note that if there are multiple power/ground domains in the design you must include pgarc specifications in a custom lib file. If P/G arcs are not defined for designs with multiple power/ground domains, an error message is displayed. *Also note that LIB files should be imported directly using this keyword. DMP compatible. Required. Default: None.*

Syntax:

```
LIB_FILES {
[ <lib_filename> | <lib_file_dir> ] ? CUSTOM ?
...
}
```

where

<lib_filename> : specifies library filename

<lib_file_dir> : specifies library directory

CUSTOM : specifies a single custom library file

Example:

```
LIB_FILES {
    special/custom.lib CUSTOM
    dirAB/memory/mem.lib
}
```

With multiple power/ground domains the contents of the *custom.lib* file could be:

```
cell ram256x8 {
    type memory
    pgarc {
     VDDA_1ST VSSA_1ST
     VDDA_2ND VSSA_2ND
     VDDA_SH VSSA_SH
  }
}
```

LIB_IGNORE_IO_VOLTAGE

Controls reading of Vimax, Vimin, Vih, Vil, Vol, Voh, Vomin, and Vomax values from LIB files in Totem when encountering Liberty files with unsupported Vi*/Vo* formats. The default value 0 allows reading in these values from LIB, while setting it to 1 ignores the values. . *Optional; default: 0.*

Syntax:

```
LIB_IGNORE_IO_VOLTAGE [ 1| 0]
```

Example:

```
LIB_IGNORE_IO_VOLTAGE 0
```

LIB_PIN_CAP

When set to max/min, Totem uses the maximum/minimum lib value between the rise/fall_capacitance_range and the pin's capacitance. When this keyword is not set, the typical value of pin capacitance is used. *Optional; default: typical capacitance value.*

Syntax:

```
LIB_PIN_CAP [min|max|typ]
```

MACRO_POWER_FILES

Specifies one or more files in which the regions of memories or other macros are described, including their types, power pins, instantiations, current consumption, and bounding boxes. In cases when detailed GDS modeling is unnecessary and requires excessive resources, the MACRO_POWER_FILES keyword can be used to efficiently model macros. Current can be distributed non-uniformly among LEF pins such that different functional portions of the macro are accounted for. For example, the total current can be distributed among the sense amp, X and Y decoder and the memory array regions of a memory cell, depending on the current sourced in that particular functional block. In this way a better granularity of current distribution over the memory/macros is achieved, without having to use a full GDS (*gds2def*) flow. The LEF pins of the macro are grouped into appropriate regions and assigned the appropriate power. The LEF pins may be split into multiple pins in the case where a pin is a stripe that crosses regions. *Optional; default: None.*

Syntax:

```
MACRO_POWER_FILES {
   <memory_macro_filename>
    ...
}
```

Example:

```
MACRO_POWER_FILES {
   memory_macroMN.pwr
}
```

The specified macro definition file has a format as in the following example:

```
   LEF_MACRO rf16x72cm1bw {
   REGION_TYPE {
<type-name>< Vdd-pin> <Vss-pin> <current_fraction>
    senseamp     VDD      VSS      .6
    predecode    VDD      VSS      .10
    memarray     VDD      VSS      .13
    xdecode      VDD      VSS      .02
   }
REGION_BBOXES {
<region-bbox-name> <region-name> <ll_x> <ll_y> <ur_x>  <ur_y>
    sensemap1        senseamp     0.0    0.0    40.0     10.0
    sensemap2        senseamp     60.0   0.0    100.0    10.0
    predecode        predecode    40.0   0.0    60.0     10.0
    memarray1        memarray     0.0    10.0   40.0     50.0
    memarray2        memarray     60.0   10.0   100.0    50.0
    decode           xdecode      40.0   10.0   60.0     50.0
    }
    }
```

The values on each line in the REGION_TYPE section specify the region type, the connected Vdd/Vss nets defined for each region type, and the fraction of the total current consumed by the region. Values on each line in the REGION_BBOXES section specify the instantiations of each region for the IP/Memory LEF_MACRO, and the corner coordinates for the bounding box of each region instantiation.

An extracted memory/IP model using the user-specified region and the associated current distributions will be created for Totem after import of DEF for each LEF_MACRO specified.

NET_NAME_CASE_SENSITIVE

Specifies DSPF parsing to be case-sensitive for nets, to improve GDS2DEF extraction and prevent duplicate name errors with case-insensitive processing.

Supports case-sensitive settings in the probe file header, so the output **.aplmmx* file in the power EM flow, or **.probe* in the signal EM flow does not show the "insensitive=yes" in the header for Spectre flow, indicating that the probe file is case sensitive. *Optional. Default:0 (off).*

Syntax:

```
NET_NAME_CASE_SENSITIVE [0 |1 ]
```

NATIVE_GDS_SUPPORT

Enables native GDS support (GDS2DB) in Totem. Required to run the GDS2DB flow. Used with the GDSII_FILE GSR keyword. Native GDS support is provided based on the following settings:

- 0 - no native GDS support; GDS2DB not invoked (runs GDS2DEF/GDSMMX)
- 1 - native GDS support provided; GDS2DB normal execution
- 2 - GDS2DB operation optimized for speed and memory use

Default: 2.

Syntax:

```
NATIVE_GDS_SUPPORT [ 0 | 1 | 2]
```

OASIS_INPUT_FILE

Specifies GDS input data in OASIS format. No default.

Syntax:

```
OASIS_INPUT_FILE {
  <cell_name> <OASIS_config_file>
    [REDUCED | EXTENDED | FULL ] ? TOP ?
}
```

where

REDUCED : indicates a GDS2DEF model

EXTENDED : indicates a GDS2DEF -m model

FULL: specifies a detailed transistor view (GDSMMX model)

TOP : an option indicating that <cell_name> is the top of the hierarchy

PACKAGE_SPICE_SUBCKT_INFO

Defines the package Spice subcircuit model to be used in simulation. When used, the TCL commands of 'setup pad', 'setup wirebond', and 'setup package' are not in effect. However, if the TCL command 'setup pss' is used, the Spice subcircuit specified in the '-subckt' option replaces the previous value of this keyword. This keyword replaces the previous keyword, PACKAGE_SPICE_SUBCKT, and now allows hierarchical Spice decks. *Optional. Default: None.*

Syntax:

```
PACKAGE_SPICE_SUBCKT_INFO {
    PATH <path_package_Spice_file>
```

```
    ? TOP <top subckt name> ?
}
```

Note that the PATH option must be specified. If TOP is not specified, then the first subckt is treated as the top subckt.

PAD_FILES

Specifies one or more pad files to be used in the design, of format type *.pad, .pcell*, or *.ploc*.

For substrate analysis, you can specify PLOCs on the bottom of the wafer or any other substrate layer to model the backside connections to the package. The PLOCs are specified similar to any other metal layer PLOC. Note that power net PLOCs are honored only if they lie on the same net substrate wire, but ground net PLOCs can be shorted to a different ground net wire through the substrate ground network. *DMP compatible. Optional; .pad, .pcell, or .ploc files can be imported separately through <design>.pad, <design>.pcell, or <design.ploc> files. Default: none.*

Syntax:

```
PAD_FILES
{
<padFilePathName-1>
...
<padFilePathName-n>
}
```

where

<padFilePathName-n> : specifies the path and filename for the pad file

Example:

```
PAD_FILES
{
MNOPQ.ploc
MNOPQ.pad
MNOPQ.pcell
}
```

For a detailed description of the syntax for the *.ploc, .pad,* and *.pcell* files, please see the section "Pad, Power/Ground and I/O Definition Files", page C-911.

PGNET_HONOR_DEF_TYPE

Ignores name-based net type assignments, and honors the DEF net type assignment regardless of name. When set to 0, a net name that includes the strings 'VDD', 'vdd' 'VCC', 'vcc', 'GND', 'gnd', 'VSS' or 'vss' are considered P/G nets, no matter what type is specified in DEF. Used with keyword HONOR_LEF_PIN_TYPE for handling LEF names. *DMP compatible. Optional. Default: 1 (On)*

Syntax:

```
PGNET_HONOR_DEF_TYPE [ 0 | 1 ]
```

POWER_MODE

Specifies the primary data source for internal/switching power and leakage power calculation analysis. *Optional. Default: Mixed.*

Syntax:

```
POWER_MODE [ APL | LIB | MIXED | APL_PEAK | APL_PEAK1 ]
```

where

APL : specifies primary use of APL power data for internal/switching (*cell.ifprof*) and leakage power (*<cell>.cdev*). Where APL data are not available, *.lib* data are used.

LIB : specifies use of *.lib* power consumption data; cells without power data in *.lib* will *not* have internal power consumption data, but will have switching power information from Totem.

MIXED : specifies primary use of *.lib* power data; for cells without power components (internal power consumption or leakage power) in *.lib*, APL data are used. For MMX cells, APL has priority over .lib for internal power calculation.

APL_PEAK: uses the peak charge from APL in power calculation for every cell in the design, and the current is derived from the charge.

APL_PEAK1: uses the APL peak current values for every cell in the in power calculation. Power for each instance is computed as Power = (peak current) *(supply voltage)*(toggle rate). Using peak current leads to a higher power value if a more conservative model is desired.

Example:

```
POWER_MODE APL
```

POWER_IGNORE_ASYNCH_PIN

When set, power calculation automatically identifies asynchronous input power pins of memory cells and ignores them when calculating power. *Optional. Default: 1 (On)*

Syntax:

```
POWER_IGNORE_ASYNC_PIN [ 0 | 1 ]
```

POWER_LEAKAGE_SCALING_FACTOR

When set, all leakage power values in Lib/APL are multiplied by the specified factor. *Optional. Default: 1.0.*

Syntax:

```
POWER_LEAKAGE_SCALING_FACTOR <factor>
```

POWER_LEAKAGE_SCALING_FILE

Specifies a file defining per-cell leakage power scaling, and also per domain if desired. Wildcards are also supported for cell names. For one cell, multiple pins can be defined in the file. All leakage power values in Lib/APL are multiplied by the specified factor for the cell.*Optional. Default: none.*

Syntax:

```
POWER_LEAKAGE_SCALING_FILE <leakage_scaling_filename>
```

The file format is as follows:

```
<cell_name1> <leakage_scaling_factor1> ? <domain>?
...
```

PRINT_ONE_PLOC_PER_PADINST

When set to 1, Totem creates a special *adsRpt/PG_simple.ploc* file, in which each pad instance has just one ploc specified (instead of multiple plocs in the *PG.ploc* file), and includes an RLC declaration for each bump. This file facilitates PSS node hook-up with the PSS *.ploc* file. Users need to manually change the

PG_simple.ploc file, adding the PSS subckt nodes, and then re-run Totem with this modified *ploc* file. *DMP compatible. Optional. Default: 0 (off)*

Syntax:

```
PRINT_ONE_PLOC_PER_PADINST [ 0|1 ]
```

Example:

```
PRINT_ONE_PLOC_PER_PADINST 1
```

RDL_CELL

Redistribution layer geometries for flip chip designs are typically available only in GDS format. To include RDL data in Totem analysis, two steps are required:

a. Conversion of RDL GDS to DEF form with the '*gds2def*' utility

b. Instantiating this DEF from the top level design

When the RDL_CELL keyword is correctly specified, Totem automatically reads in the RDL master cell DEF (after importing top DEF), creates the RDL instance, places it according to the LOCATION specification, and connects all pins listed to top level nets. Note that all RDL- related information is needed only in the RDL_CELL keyword specification. If there are multiple definitions for OFFSET, LOCATION, and ORIENTATION, the last values override previous ones. Note that undefined child instances are ignored. *Required for RDL cells. Default: none.*

To correctly use this GSR keyword, use the following guidelines:

- Prepare the RDL master cell DEF
- Make sure all fields in RDL_CELL are correctly specified, including the master cell file path, and the instance LOCATION relative to the top DEF coordinate system.
- The RDL DEF file path is not needed in DEF_FILES in the GSR.
- The RDL instance is not needed in a GDS_CELLS definition, nor as a component of the top DEF.

Syntax

```
RDL_CELL {
    <RDL_inst_name> <RDL_DEF_master_cell_name>
        <path_RDL_master_DEF_file>
    LOCATION <x_loc> <y_loc>
    ? OFFSET <x_offset> <y_offset>
    ? ORIENTATION <orientation> <dummy> ?
    NET <Vdd_net_name> <power_pin_name>
    NET <Vss_net_name> <ground_pin_name>
    ...
}
```

where

<RDL inst_name>: instance name of RDL cell

<RDL_master_cell_name>: master cell name

<path_RDL_master_DEF_file> : path to RDL master cell defined in DEF

LOCATION <x_loc> <y_loc>: x,y location relative to top DEF coordinate system (microns)

OFFSET : specifies x,y offset distance relative to top DEF coordinate system Default: 0,0. (microns)

ORIENTATION : orientation specification using the same nomenclature as in the top DEF. Default: N. A third entry is required on this line that is ignored (dummy).

NET <Vdd/Vss_net_name> <power/ground_pin_name>: describe the RDL instance's pin connections to the top level nets. Each line describes only one pin. Multiple such lines may occur in the block.

Example

```
RDL_CELL {
rdl_inst chip_rdl  /home/project/data/def/chip_rdl.def
LOCATION 500 250
OFFSET -25 -25
ORIENTATION FS dummy
NET VDD VDD
NET VSS VSS
}
```

READ_LEF_OBS

When this keyword is turned On, Totem reads the specified parts of the “OBS” section in LEF to determine the regions to be assigned power for early stage analysis. The only parts of the OBS data read in are LAYERs defined as OVERLAP and shapes defined as RECT. All other items defined in the LEF OBS section are ignored. *Optional. Default: off : 0 .*

Syntax:

READ_LEF_OBS 1

Example: OBS section items in LEF included in power assignment:

```
OBS
    LAYER OVERLAP ;
    RECT 0.0 0.0 800.0 400.0 ;
    RECT 0.0 400.0 400.0 600.0 ;
END
```

REMOVE_PARENTLEF_GEOS

When set, drops the LEF of cells from GDS2DEF blocks--the parent (original) LEF. *Optional. Default: off, 0.*

Syntax

```
REMOVE_PARENTLEF_GEOS [ 0 | 1 ]
```

REPORT_MAXCAP_VIOLATION

Reports LIB max cap violations to the file *adsRpt/<design>.maxCapViolation*. When set to -1, all violations are reported. When set to a positive integer, n, the number of violations reported is limited to n. The output report format is:

```
<inst_name:pin> <cell_name> <library> <max_cap> <spef_cap>
```

When both 'USE_LIB_MAX_CAP 1' and 'REPORT_MAXCAP_VIOLATION <limit>' keywords are set, the report is prepared and the maxcap limit is honored in LIB. When only REPORT_MAXCAP_VIOLATION is set, only the max-cap violations are reported, and SPEF/Steiner tree estimates (or whatever is available) are used. *Optional. Default: 0 (off)*.

Syntax

```
REPORT_MAXCAP_VIOLATION [ 0 | n | -1 ]
```

Example

```
REPORT_MAXCAP_VIOLATION 150
```

SAIF_FILE

Specifies the root instance name and the Switching Activity Interface File that contains design toggle information. *Optional. Default: none.*

Syntax:

```
SAIF_FILE {
    <block_name> <SAIF_filename>
    ROOT_INSTANCE <root_instance_name>
    SUBSTITUTE_PATH <substitute_instance_path>
}
```

where

<block_name>: can be <design name>, <block name>, or FULLCHIP

<SAIF_filename>: specifies the full path to the SAIF filename

ROOT_INSTANCE: specifies the hierarchical path to the root instance; the <root_instance_name> is the mapping between the SAIF names and the DEF names.

For an example SAIF file as follows:

```
(SAIFILE
(SAIFVERSION "2.0")
    ...
(INSTANCE tb
  (INSTANCE ldo_bench
    (NET
      (A\[0\]
        (T0 70000) (T1 0) (TX 0)
        (TC 0) (IG 0)
      )
      ...
      )
   )
)
```

the top DEF design name is : l*do_design*, and the GSR setting would be:

```
SAIF_FILE {
    ldo_design user_data/ldo_design.saif
    ROOT_INSTANCE  tb/ldo_bench
}
```

STA_CRITICAL_PATH_FILE

To perform fixing and optimization for timing of instances on critical paths in a design (‘FAO_OBJ PATH’), you must specify the critical path report using this keyword. *Optional. Default: none.*

Syntax

```
STA_CRITICAL_PATH_FILE
{
<critical path report path>
}
```

Example

```
STA_CRITICAL_PATH_FILE
{
```

```
Totem/STA/TYP_S_INTERNALS.path
}
```

STA_FILES

If the keyword is defined, the power for each net is calculated from the transition times (“slew”) of each instance’s input/output pins, using the clock frequency domain for all the instances that are specified in the STA output file, instead of deriving its value from the CLOCK_ROOTS keyword.

For instances not defined in the STA output file or VCD file, the frequency of the instance is set to zero. Use the frequency specified by FREQ_OF_MISSING_INSTANCES keyword for those instances that are not covered in the STA file. FREQ_OF_MISSING_INSTANCES is not recommended to be specified unless you know that STA does not cover the design well. If the EXTRACT_CLOCK_NETWORK option is specified as 1 (default 0), clockroots defined in the clock section of the STA output file are used for clockroot tracing, instead of the STA-identified clock instances.

Note: If both BLOCK_STA_FILES and STA_FILES are used, then BLOCK_STA_FILES is ignored, and if a USER_STA_FILE is also used, any common data in the specified USER_STA_FILE overrides data in either of the other files.

The sta_output_file_name is derived from running the *pt2timing.pl* script using STA tools, such as PrimeTime. For the use of *pt2timing.pl,* please refer to section "pt2timing", page E-1009. *DMP compatible. Required for dynamic analysis; Optional for static analysis.*

Syntax:

```
STA_FILES
{
    FREQ_OF_MISSING_INSTANCES <frequency in hertz>
    EXTRACT_CLOCK_NETWORK [ 0 |1 ]
    <top_level_block_name> <sta_output_file_name>
}
```

where

FREQ_OF_MISSING_INSTANCES <frequency in hertz> : specifies the operating frequency for instances not in the STA file but in the design. Default: 0.

EXTRACT_CLOCK_NETWORK : when set to 1, specifies that the clock network is extracted from clock roots defined in STA

<top_level_block_name>: the top level block name of the chip

<sta_output_file_name> : the file generated from running *pt2timing.tcl* when running STA analysis (e.g., PrimeTime)

Example:

```
STA_FILES
{
top_level_block block1.sta
}
```

STANDARD_CELL_HEIGHT

Specifies allowable heights for all standard cells in the design (microns). *DMP compatible. Optional. Default: none.*

Syntax

```
STANDARD_CELL_HEIGHT {
    <cell_height1>
    <cell_height2>
    ...
}
```

TEMPERATURE

Specifies actual design-dependent operating temperature in degrees C. Nominal temperature is the process-related, design-independent value specified in the Tech file. This TEMPERATURE specification overrides the value of T in the Tech file for resistance calculations. For EM analysis, this value is overridden by the T_EM value in the Tech file. *DMP compatible. Optional. Default: none.*

Syntax:

```
TEMPERATURE <actual_temperature_°C>
```

Example:

```
Temperature 100
```

TECH_CONNECTIVITY_FILE

Specifies a file containing LVS-style connectivity statements of the form:

```
CONNECT <layer1> <layer2> ?BY <via>?
...
```

Totem uses this information, along with the layer heights and thicknesses, to block connections between all layers that touch or overlap vertically, but have no CONNECT statement saying they should be connected during extraction. The effect is identical to the *ircx2tech* ircx-to-tech file conversion flow with LVS connection file information input to *ircx2tech* using the '-ct <LVS_connect_Info_File>' option, except that there is no facility for mapping the LVS names to Totem names. All names in the TECH_CONNECTIVITY_FILE are assumed to be tech file names. This allows foundries to release the LVS connectivity information separately from the core tech file. *Optional; default: none.*

```
TECH_CONNECTIVITY_FILE <filename>
```

TECH_FILES

Required to specify the Totem technology file (*.*tech*) to be used in the design. *DMP compatible. Required. Default: none.*

Note: the tech file should be imported directly using this keyword and not in a separate 'import' command.

Syntax:

```
TECH_FILES <techFilePathName>.tech
```

Example:

```
TECH_FILES abcDesign4.tech
```

THERMAL_PROFILE

With the THERMAL_PROFILE keyword and the 'perform pwrcalc' command, the chip thermal model file is generated for thermal analysis, which maps tile and layer-based temperatures in the thermal profile to instances in the design, with the following format:

<inst name> <Tavg_C> <Tmax_C> <Tmin_C>

If the instance is in the intersection of the grid defined in <thermal_profile_file>, the max temperature is used from the intersected grid. The wire/via temperature values are based on the per layer tile-based thermal profile file generated by Sentinel. The EM limit is modified by the temperature data specified in the tech file keyword EM_TEMP_RATING *(only affects Average EM limits, and not RMS and PEAK limits). Optional; default: none.*

Syntax:

```
THERMAL_PROFILE {
      FILE <thermal_profile_file>
}
```

where the <thermal_profile_file> is from Sentinel-TI package modeling.

TOGGLE_RATE_RATIO_COMB_FF

Defines the ratio of the toggle rate of the nets driven by combinational cells relative to the toggle rate of the nets directly driven by flip-flops and latches. Depending on the depth of the combinational logic, the ratio can be from 0.3 to 0.75. Only the toggle rate of the combinational instances defined from TOGGLE_RATE, BLOCK_TOGGLE_RATE, BLOCK_TOGGLE_RATE_FILE, INSTANCE_TOGGLE_RATE, and INSTANCE_TOGGLE_RATE_FILES are affected by this keyword. The toggle rate defined by the VCD_FILE keyword is *not* affected. *Optional; default: 1.0.*

Syntax:

```
TOGGLE_RATE_RATIO_COMB_FF <TR_ratio_combin_re_FF>
```

Example:

```
TOGGLE_RATE_RATIO_COMB_FF 0.7
```

USE_DEF_VIARULE

Allows you to choose how via rules are handled. If this keyword is set to 1, the DEF parser ignores the LEF via rule and uses the DEF via rule. When set 0, the DEF parser uses the LEF via rule. *DMP compatible. Optional. Default: 0.*

Syntax

```
USE_DEF_VIARULE [ 0 | 1 ]
```

USER_STA_FILE

USER_STA_FILE allows you to specify the essential timing and slew data needed by Totem, such as instance timing windows defined by switching times, and whether the instance is part of a signal or clock network. Data in this file supersedes the same data in the regular STA file, so it can be easily used for “what-if” analysis during power ramp-up analysis. For example, if you want to incrementally set the timing window for instances, use the following steps:

```
gsr set USER_STA_FILE <file name>
import sta
```

USER_STA_FILE is intended to be a much more understandable format/syntax to use than the normal STA file.

Note: If BLOCK_STA_FILES or STA_FILES is also used, any common data in the specified USER_STA_FILE overrides data in either of the other files. *DMP compatible. Optional. Default: none.*

Syntax

```
USER_STA_FILE <filename>
```

The allowed format of information in the specified USER_STA file is different depending on whether a signal/clock net or a switch is specified, as follows

a. Signal or clock timing window

```
<inst_name> TW <min_arriv_time> <max_arriv_time> ?<Freq>? ?[ s | c ]?
```

Note: pin name is not supported

b. Switch timing window

```
<inst_name/pin_name> SW <min_arriv_time> <max_arriv_time> ?<Freq>?
```

Note: the "/" is required to specify pin name

c. Slew

```
<inst_name> SL <rise_trans_time> <fall_trans_time>
<inst_name>/<pin_name> SL <rise_trans_time> <fall_trans_time>
...
```

where

<inst_name>: name of instance. Note that wildcards can be used to designate a set of instances to be specified together, such as "instABC12*".

<inst_name>/<pin_name> : if a pin_name specification is allowed, transition times are applied to that pin; otherwise data is applied to the instance

TW/SL/SW: identifies a Timing Window, Slew, or Switch data line

<min_arriv_time> <max_arriv_time>: specifies minimum and maximum arrival times in seconds

<Freq> : (optional) operating frequency of instance.
If <Freq> is *not* specified:

(1) if the instance is in the STA file, use frequency defined there.

(2) if the instance is not covered in STA, or the frequency is not given, use the FREQ_OF_MISSING_INSTANCES value defined in STA_FILES section in GSR. If FREQ_OF_MISSING_INSTANCES value is not specified, the frequency is assumed to be 0.

[s | c] : optional specification of signal type, signal/data or clock

<rise_trans_time> <fall_trans_time> : specifies rising and falling transition times in seconds

Example of User STA file contents:

```
top/block/inst1       SL 0.1e-9 0.1e-9
top/block/inst1       TW 4e-9 4e-9 100e6 c
top/block/inst1/CLK   SL 0.01e-9 0.01e-9 100e6
top/block/inst2*      TW 5.1e-9 5.2e-9 200e6
top/power_gate/SW1/E  SW 1.0e-9 1.0e-9
```

VCD_FILE

The VCD_FILE keyword reads in original VCD files directly for power calculation purposes. Similar to the standalone *vcdscan* utility, some parameters are needed to process the VCD file. The utility *vcdtrans* can be called to generate the toggle file also needed for power calculation. *DMP compatible. Optional. Default: None.*

Syntax:

```
VCD_FILE {
  <top_block_name> <VCD filepath>
  ? FILE_TYPE [ VCD | FSDB | RTL_VCD | RTL_FSDB ]?
  ? VCD_DRIVEN [0|1]?
```

```
    ? FRONT_PATH <“string”> ?
    ? SUBSTITUTE_PATH <“string”> ?
    ? SELECT_RANGE <start_time> <end_time> ?
    ? SELECT_TYPE [ WORST_POWER_CYCLE | WORST_DPDT_CYCLE ]?
    ? START_TIME <start> ? ? END_TIME <end> ?
    ? TRUE_TIME [0|1] ?
    ? MAPPING <map_file> ?
    ? STA_VCD_FREQ_RATIO <ratio>?
  }
```

where

<top_block_name> : specifies name of top block

<VCD filepath> : specifies path to VCD file

FILE_TYPE [VCD | FSDB | RTL_VCD | RTL_FSDB] : selects file type (default: VCD)

VCD_DRIVEN : turns VCD-driven state propagation based power calculation on and off. Default is off.

FRONT_PATH “<redundant_path_string>” : specifies the string describing the VCD file hierarchical path of the instance. This string is then replaced by the SUBSTITUTE_PATH string in order to match the path of the instance in the DEF.

SUBSTITUTE_PATH “<substitute_path_string>”: specifies the string describing the DEF file hierarchical path of the instance. This is used internally to substitute the VCD path (specified by FRONT_PATH) with the DEF path in order for Totem to properly identify the equivalent instances and nets in the VCD and DEF.

SELECT_RANGE: specifies the start_time and end_time for performing automatic critical cycle selection during power calculation. If START_TIME and END_TIME values are set to -1, the full VCD period is included in cycle selection.

SELECT_TYPE : default value WORST_POWER_CYCLE ranks cycles based on highest cycle power. Set to WORST_DPDT_CYCLE invokes DPDT (delta power/delta time) ranking of cycle selection, since large cycle-to-cycle changes in power level (high DPDT) may cause very high di/dt values, leading to large dynamic voltage drops.

START_TIME <time> : specifies VCD or FSDB file start time for power calculation. If both START_TIME and SELECT_RANGE are specified, SELECT_RANGE is ignored, so cycle selection is skipped. Default start time = 0. If the specified START_TIME is within the dump-off range (where activity is not considered), the START_TIME is automatically reset to the start of the next dump-on time.

END_TIME <time> : specifies VCD or FSDB file end time for VCD-driven state propagation-based power calculation. For other flows, END_TIME is ignored, and it is determined by START_TIME + DYNAMIC_SIMULATION_TIME. Default end time is the end of the VCD/FSDB file.

TRUE_TIME : if set to 0, uses STA timing data and assumes no glitches; if set to 1, uses VCD switching and timing data; default = 0.

MAPPING <filename> : name of map file with RTL VCD instance names and corresponding DEF instance name

STA_VCD_FREQ_RATIO - matches the VCD frequency to the STA frequency (the specified <ratio> = STA_freq/VCD_freq). When Totem reads VCD data to select cycles, it uses the clock frequency defined in the STA file to cut

cycle frames from VCD. If this option is used, VCD data is processed according to the STA frequency and the specified ratio. Note that this option does not affect the frequency selection in simulation.

Example:

```
VCD_FILE {
    top top.vcd
    FILE_TYPE FSDB
    FRONT_PATH “test/setup/”
    SUBSTITUTE_PATH “”
    START_TIME 100000
    END_TIME   200000
    TRUE_TIME 1
}
```

VCD_CONSISTENT_SCENARIO_FILTER

VCD Parsing Stage will have consistent results between power calculation and simulation when this GSR keyword is ON along with 'TRUE_TIME 1'. This feature is supported in averaged mode of power analysis also (enabled by setting POWER_ANALYSIS_MODE averaged) *Default : 1 if* 'TRUE_TIME 1' is set in GSR

Syntax:

```
VCD_CONSISTENT_SCENARIO_FILTER [ 0 | 1 ]
```

Parameter Keywords

ADD_PLOC_FROM_DEF

Allows specifying an additional DEF file that defines PLOCs, in addition to the top DEF file. In this case Totem only uses PIN information from this DEF and does not read any other info from it. The DEF file pathname specified in the keyword 'must be the same as specified in the DEF_FILES keyword, whether an absolute or a relative path. *Optional. Default: none.*

Syntax:

```
ADD_PLOC_FROM_DEF {
   <DEF file path>
 }
```

ADD_PLOC_FROM_TOP_DEF

If set to 1, picks plocs from all layers using the PINS section in the top level DEF, provided that power and ground pin geometries in the PINS section are well-defined with physical shapes for connection to top level. You can also select metal layers by name from which the plocs are created. This option is most useful in block level analysis, where block level pins are defined in DEF. Note that Totem exits if a script contains both import pad commands and ‘ADD_PLOC_FROM_TOP_DEF 1’. (The keyword ‘ADD_PAD_LOC_FROM_TOP_DEF’ is the same function, but is now obsolete.) *DMP compatible. Optional; default: 0 (no automatic copying from DEF).*

Syntax:

```
ADD_PLOC_FROM_TOP_DEF [ 0|1] { <layerName1> <layerName2> ... }
```

APACHE_DB_OVERWRITE

Specifies whether the previous version of the Apache database is overwritten by new analysis data or not. *DMP compatible. Optional. Default: 1 (overwrite).*

Syntax:

```
APACHE_DB_OVERWRITE [ 1 | 0 ]
```

APACHE_FILES

Specifies the amount of internal backup files to be saved on exit from a Totem run. Values are:

- normal - some files removed
- clean - most files removed

DMP compatible. Optional. Default: normal.

Syntax:

```
APACHE_FILES [ normal | clean ]
```

Example:

```
APACHE_FILES clean
```

CACHE_DIRECTORY

Specifies the directory where data is cached. Should be a disk on the machine where Totem is running for efficient operation. *DMP compatible. Optional. Default: current working directory.*

Syntax:

```
CACHE_DIR <path to cache directory>
```

Example:

```
CACHE_DIR /local/cache
```

CACHE_MODE

Enables adaptive disk caching in Totem. Using this “smart” disk caching is recommended whenever a run requires more than the available physical memory, to reduce the need for generic system memory swapping and to make memory use much more efficient. Make sure that there is local disk space to use for disk caching. *DMP compatible. Optional. Default: 0 (off).*

Syntax:

```
CACHE_MODE [ 0 | 1 ]
```

COMPLEX_PIN_MAX_RECT_NUM

When set to 0, turns off complex pin reduction, which ignores pins with at least one dimension less than 0.05 microns. You can change the maximum number of pin geometries without pin reduction using the <max_num_pins> option. Also see the COMPLEX_PIN_CHECK keyword. *Optional. Default:* 5000.

Syntax:

```
COMPLEX_PIN_MAX_RECT_NUM [0|<max_num_pins>]
```

DEF_PG_NETS_FILE

Allows clock and signal nets (defined in SPECIALNETS in DEF) to be defined as power/ground nets for purposes of EM analysis, using a file specified with this keyword. The specified file should use the following format:

```
<design> <net> [ power| ground| signal |clock]
```

For example, an entry in the specified file would be:

```
blockABC2 clockAB power
```

In this example, net 'clockABC' is treated as a power net for EM analysis. This keyword also can be used for ESD analysis to run current density checks for bump-to-bump checks from pad-Vdd and pad-Vss. *DMP compatible. Optional. Default: none.*

Syntax:

```
DEF_PG_NETS_FILE <filename>
```

DEF_SCALING_FACTOR

Specifies the scaling factor to be applied to the length units used in each *.def* file. This allows shrinking a design DB from an older process technology (for example, 0.15um) to a newer process technology (for example, 0.11um).

For the top cell, you should specify the file path name of the top DEF similar to specifying blocks. The keyword option 'all' is valid to specify all blocks should be scaled, including the top cell. *DMP compatible. Optional. Default: 1.0.*

Note: If 'All' or the top cell scaling factor is not specified, make sure that the *.ploc* locations correspond to the modified-dimension layout. Otherwise, you may get an ERROR message that the net has no driver, and the simulation may fail.

Syntax:

```
DEF_SCALING_FACTOR
{
[ All <scale_factor> |
<DEF_top_block_path-1> <scale_factor>
...
<DEF_top_block_path-n> <scale_factor> ]
}
```

where

All <value> : specifies the length scaling factor applied to all DEF files.

<DEF_top_block_path-n> <value> : specifies the DEF top block file path and the length scaling factor to use on the file.

Examples:

```
DEF_SCALING_FACTOR
{
INVERT_ABC 0.85
all 0.75
}

DEF_SCALING_FACTOR
{
user_data/LEFDEF/INST-Y0.def 0.75
}
```

DYNAMIC_FF_ADJUST_NTRIG

When set, applies negative trigger switching (StandByNtrig C11 state) for all cycles on clock falling edges for FF/MEM instances (On by default in 14.2). *Optional. Default: 0 (Off).*

```
DYNAMIC_FF_ADJUST_NTRIG [0 | 1]
```

DYNAMIC_GENERATE_BRANCH_WAVEFORM

When set, supports the 'report em_state' command in Power EM analysis for multi-state multi-cycle (MSMC) setup, or single-state single-cycle cases by generating current waveforms for all branches. If analysis is performed with this keyword turned on, then waveforms are available to use the command

```
report em_state ?-state { <state1> <state2> ... <stateN> }?
```

to switch current among states. *Optional. Default: 0 (off).*

```
DYNAMIC_GENERATE_BRANCH_WAVEFORM [0 | 1]
```

DYNAMIC_MSTATE_FILTER

When turned off, does not filter out events with different trigger pins for custom state cells during simulation, which is useful when custom state cells are included in the design. *DMP compatible. Optional. Default: 1 (on).*

Syntax:

```
DYNAMIC_MSTATE_FILTER [0 | 1]
```

DYNAMIC_PEAK_CURRENT_AWARE

When set, selects peak currents from APL for cells specified in the cell list. *Optional. Default: none.*

Syntax:

```
DYNAMIC_PEAK_CURRENT_AWARE <cell_list>
```

DYNAMIC_PRECHECK

By default this keyword allows automatic pre-checking of key simulation data integrity issues. *Optional. Default: 1 (On).*

Syntax:

```
DYNAMIC_PRECHECK [ 0| 1]
```

DYNAMIC_SAVE_WAVEFORM

When set, controls creation of waveform plots for all Vdd/Vss nodes of all instances (or all transistors for MMX designs). *Optional. Default: 1.*

Syntax:

```
DYNAMIC_SAVE_WAVEFORM [1 | 0 ]
```

DYNAMIC_SELECTIVE_SAVE

When turned off (0) saves all dynamic waveform files. By default (1) saves only waveforms for pad and pin nodes, and switch nodes for low power designs, which reduces disk space use for intermediate files and processing time. *Optional. Default: 1*.

Syntax:

```
DYNAMIC_SELECTIVE_SAVE [ 0 | 1 ]
```

EVA_PG_AWARE

Allows consideration of anomalous switching due to power and ground design weaknesses during Totem vectorless DvD analysis. *DMP compatible. Optional. Default: 0 (off, do not consider P/G weakness).*

Syntax:

```
EVA_PG_AWARE [ 1 | 0 ] ?<PG_switch_ratio>?
```

where

<PG_switch_ratio> : optionally specifies the ratio defined as: the maximum number of instances that will switch due to P/G weakness divided by the total number of instances in the design. For example, if the ratio is set to 0.2, and TOGGLE_RATE is set to 0.0, not more than 20% of the total number of instances will switch, all due to the ranking of P/G weakness. It is recommended that TOGGLE_RATE be set to 0.0 when EVA_PG_AWARE is set to 1 to focus on just the switching due to P/G weakness. (Default ratio: 0.0009)

Example:

```
EVA_PG_AWARE 1 0.2
```

GDSII_SCALING_FACTOR

Specifies the scaling factor to be applied to the length units used in each *.gds* file. This allows shrinking a design DB from an older process technology (for example, 0.15um) to a newer process technology (for example, 0.11um).

For the top cell, you should specify the file path name of the top GDS, similar to specifying blocks. The keyword option 'all' is valid to specify all blocks should be scaled, including the top cell. Also supports bridge via handling. *Optional. Default: 1.0.*

Note: If 'All' or the top cell scaling factor is not specified, make sure that the *.ploc* locations correspond to the modified-dimension layout. Otherwise, you may get an ERROR message that the net has no driver, and the simulation may fail. *Optional. Default: 1.0.*

Syntax:

```
GDSII_SCALING_FACTOR
{
[ All <scale_factor> |
<GDS_top_path-1> <scale_factor>
...
<GDS_top_path-n> <scale_factor> ]
}
```

where

All <value> : specifies the length scaling factor applied to all DEF files.

<GDS_top_path-n> <value> : specifies the GDS top file path and the length scaling factor to use on the file.

GENERATE_CPM

When set, invokes a special flow for CPM generation, which is distinct from the dynamic voltage drop analysis flow. **This keyword must be set for CPM generation.** *DMP compatible. Optional. Default: 0.*

Syntax:

```
GENERATE_CPM [ 0 | 1 ]
```

GND_NETS

Specifies the voltage for Vss nets, and also defines equivalent ground domain net names in DEF, which are merged in Totem. The ground domain names are defined

in the DEF file using the SPECIALNETS keyword. *DMP compatible. Optional. Default: all the SPECIALNETS are designated as USE GROUND and set to 0 volts.*

Note: A net can only be merged into one domain (the first one specified). There can be no net hierarchy.

Syntax:

```
GND_NETS
<Vss_domain_net_name> <value_Volts> {
    <equiv_Vss_net_name1>
    ...
    }
 ...
}
```

where

<Vss_domain_net_name> : specifies DEF name for ground domain net, such as VSS for the core power domain.

<value_Volts> : nominal voltage

<equiv_Vss_net_name1> : equivalent ground net name in same domain

Example:

```
GND_NETS
{
GND 0
VSS 0
}
```

LEF_SCALING_FACTOR

Specifies the scaling factor to be applied to the length units used in the *.lef* file. This is for shrinking the design DB from an older process technology (e.g., 0.15um) to a newer process technology (e.g., 0.11um). *DMP compatible. Optional. Default: 1.0.*

Syntax:

```
LEF_SCALING_FACTOR
{
[ All <value> |
<LEF_FilePathName-1> <value>
...
<LEF_FilePathName-n> <value> ]
}
```

where

All <value> : specifies the length scaling factor applied to all the LEF files.

<LEF_FilePathName-1> <value> : specifies the path and filename of the LEF file and the length scaling factor to use on the design.

Example:

```
LEF_SCALING_FACTOR
{
lefs/MNOPQ.lef 0.8
}
```

LICENSE_WAIT

Allows a Totem run that does not have an available Totem license to wait until one is available, instead of exiting. This is useful after an initial static analysis run, when

another special Totem license is needed in the session and the option '`-lmwait`' was not used. *DMP compatible. Optional. Default: 0 (off)*

Note: The process is controlled by the license manager and hence appears to be hung while in waiting mode.

Syntax:

```
LICENSE_WAIT [ 0 | 1 ]
```

MMX_DVD_GLITCH_FILTER_FILE

Specifies a file a which defines the criteria for filtering the dvd glitch. Details of the filtered glitches are reported in adsRpt/Dynamic/cell_mmx.mmx_pin_filtered file.

Syntax:

```
MMX_DVD_GLITCH_FILTER_FILE <file_path>
```

Format of filter file:

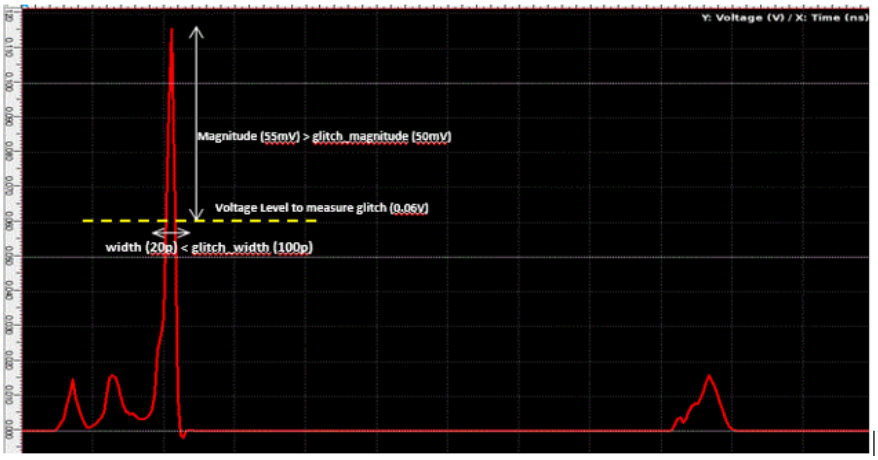
```
#<domain_name> <glitch_width_ps> <voltage_level_volts>
<glitch magnitude_mv>
```

Sample filter file:

```
#<domain_name> <glitch_width_ps> <voltage_level_volts>
<glitch magnitude_mv>
####### Filter out glitch for VDD domain
VDD 90 0.96 40
##### Filter out glitch for VSS domain
VSS 100 0.06 50
```

Where,

- 'voltage_level' is where the glitch width and glitch magnitude/height are measured.
- 'glitch magnitude' is measured from the specified voltage level to the glitch peak.
- a glitch is filtered if
  - glitch width is smaller than the glitch width threshold AND
  - glitch magnitude/height is larger than the specified glitch magnitude



MMX_RES_MAP_LIMIT

The **Transistor Pin Actual Resistance Map** menu command generates a color map of the actual resistance for all transistor pins in the MMX analysis. For run-time reasons only the highest 10,000 pin resistances are calculated by default, based on P/G grid weakness computations. To change the maximum number of resistances calculated, use this keyword. *Optional. Default: 10000.*

Syntax:

```
MMX_RES_MAP_LIMIT <pin_limit>
```

MULTI_THREADS

Provides control of the number of threads that Totem uses, which can provide a significant reduction in run time in power calculation and simulation by using multiple CPUs, compared to single-core use, during transient simulation and results post-processing for large designs during the verification process. By default all available CPUs are used. *DMP compatible. Optional. Default: 1.*

Syntax:

```
MULTI_THREADS [On=1|Off=0] ?<max_threads>?
```

Examples:

```
MULTI_THREADS 1 (uses all available CPUs - default)
MULTI_THREADS 0 (uses single-thread--one CPU)
MULTI_THREADS 1 1 (uses single-thread)
MULTI_THREADS 1 4 (uses multi-thread with four CPUs)
```

PERFORMANCE_IMPACT_REPORT

When set to 1, Totem reports and highlights asymmetric metal layer node distribution, in which one or more layers has a number of nodes that exceeds 50 percent of the average node count of all layers. This can be caused by an imbalance from the use of multiple metal layers inside LEF cells, or by disjointed via arrays.

Syntax:

```
PERFORMANCE_IMPACT_REPORT [ 0 | 1 ]
```

PGPLOC_DEBUG

Generates domain information for bumps in the PLOC file. You can get power/ground domain names for each bump in Totem-generated PLOC file, *adsRpt/PG.ploc*. Also displays RLC data for each bump in the *adsRpt/PG_simple.ploc* file if the GSR keyword setting “PRINT_ONE_PLOC_PER_PADINST 1” is used. *DMP compatible. Optional. Default: 0 (Off).*

Syntax:

```
PGPLOC_DEBUG [ 0 | 1 ]
```

PRINT_EM_VIA_BOX

Specifies that via_cut_bounding_box_coords and current_direction be included in the *.em report of EM and current data, as requested using the EM_DUMP_PERCENTAGE GSR keyword (see details on page C-828). *DMP compatible. Optional. Default: 0.*

Syntax:

```
PRINT_EM_VIA_BOX [ 0 | 1 ]
```

PRINT_EM_VIA_INFO

When set, prints complete via EM information in the EM worst file. *DMP compatible. Optional. Default: 0.*

Syntax:

```
PRINT_EM_VIA_INFO [ 0 | 1 ]
```

REPORT_DISCONN_MIN_LENGTH

Specifies the minimum wire length above which disconnected-wire reports are created. *Optional. Default: None.*

Syntax:

```
REPORT_DISCONN_MIN_LENGTH <min_length_microns>
```

REPORT_FLATTEN_LOG

When set, records debug data in the flatten stage in the file *adsRpt/flatten.rpt*. Logical connectivity issues can cause certain nets to be merged and incorrectly renamed. To resolve such cases, you can use this keyword to create a file that lists all the nets that are merged and renamed when the design is flattened. *DMP compatible. Optional. Default: 0.*

Syntax:

```
REPORT_FLATTEN_LOG [0 |1]
```

REPORT_PEAK_MEMORY

When set, reports peak memory usage for each stage in the log file. By default memory use at the end of each stage is reported. *DMP compatible. Optional. Default: 0ff.*

Syntax:

```
REPORT_PEAK_MEMORY [0|1]
```

REPORT_REDUCTION

This keyword controls the number of output reports generated and the contents of the *adsRpt* folder, depending on the value set:

off/ 0: complete information (default)

normal/ 1: files that can be created from the database using the 'report' TCL command (currently only files *adsRpt/.power.rpt*), *adsRpt/Dynamic/<>.dvd.mmx*, and *adsRpt/Static/<>.ir.mmx*) are *not* generated. To generate the **.power.rp*t file, use the command 'report power -all ?-o <output_file>?'. You can invoke on the fly the following commands to get reports *adsRpt/Static/<>.ir.mmx* and *adsRpt/Dynamic/<>.dvd.mmx*:

after static analysis,

```
report ir  -mmxpin ?-o <file>? ?-limit <num_per_domain>?
```

after dynamic analysis.

```
report dvd -mmxpin ?-o <file>? ?-limit <num_per_domain>?
```

If '-o <file>' is not specified, the default output filename is used.

max (maximum reduction mode)/ 2: same as ‘normal’, and in addition, files that cannot be retrieved from the database (*adsRpt/Static/<design>.inst.arc* and *adsRpt/Static/<design>.inst.pin*), are gzipped.

DMP compatible. Optional. Default: 0ff.

Syntax:

```
REPORT_REDUCTION [off |normal |max ]
```

SPLIT_VDD_EXTRACT_LP

SPLIT_VDD_EXTRACT_LP_FSIZE

For the 'plot voltage' command, if the number of nodes is greater than 5M , when the 'plot voltage' command is executed the first time, the vdd file is split into multiple

vdd files, with 1M nodes in each vdd file. From second time the command is executed onwards, the 'plot voltage' command reads the split vdd files. The default 5M node threshold at which point the files are split can be adjusted using this keyword. *Optional. Default: 5 M.*

Syntax:

```
SPLIT_VDD_EXTRACT_LP <value>
```

The number of nodes in each split vdd file can be adjusted using the keyword 'SPLIT_VDD_EXTRACT_LP_FSIZE <value>'. *Optional. Default: 1 M.*

Syntax:

```
SPLIT_VDD_EXTRACT_LP_FSIZE <value>
```

STD_CELL_SINGLE_NOMINAL_VOLTAGE_ONLY

If set to 1, checks the value of the APL nominal voltage keyword VDD against the one or more voltage values specified in the GSR keyword VDD_NETS. All standard cells have only one nominal voltage. Any discrepancy between voltage values specified in VDD_NETS and APL nominal voltage values are flagged with an Error and APL importing is halted. *Optional. Default: 0.*

Syntax:

```
STD_CELL_SINGLE_NOMINAL_VOLTAGE_ONLY [ 0 | 1 ]
```

SUPPLY_NETS

This keyword allows you to analyze designs with power-only or ground-only nets, and also specify negative voltages for a net. When set, this keyword automatically enables the keyword “TOTEM_MMX_VDD_OR_GND_ONLY”, which allows Totem to take designs with power-only or ground-only net setups. Some details about using this keyword are described below:

a. You can put a net in the GND_NETS section only if it connects to a “0” ideal ground at block level, as well as at top-level. If the net can be connected to “0” ideal ground at block-level and *non-zero* at top-level, put this net in SUPPLY_NETS instead of GND_NETS.

b. For the ground-only net case, you must set up ACE characterization outside of APLMMX and run ACE first, before APLMMX, to get a *<design>.ace.mmx* file, and then import this file back into APLMMX using the ACE_OUTPUT_FILE keyword. You must define a dummy power net in EXTERNAL_POWER_NETS and VDDVALUE to help ACE understand the voltage level of the battery in order to get accurate characterization results.

c. APLMMX also creates a dummy GND_NETS section in the internally-created GDS configuration template file, which you can pass to Totem GDS2DB.

d. If used, the SUPPLY_NETS keyword must be in both APLMMX configuration file and in the GSR, and in this case the VDD_NETS keyword cannot also be specified in these files.

e. If a negative or zero ideal voltage is specified, the Totem GUI checks the ideal voltage level for each domain. If one is zero or negative, the GUI automatically switches to “MAX” mode to take the maximum voltage to calculate the bounce, and display the Transistor Pin Movie.

Syntax:

```
SUPPLY_NETS {
    <net1> <voltage>
    <net2> <voltage>
    ...
```

```
        <netN> <voltage>
    }
```

Example:

```
    SUPPLY_NETS {
        Vdd -1.0
    }
```

or

```
    SUPPLY_NETS {
        Vss 0.0
    }
```

TEMPERATURE_REPORT_LINE_NUMBER

When set, controls the maximum number of lines in the temperature report file. Totem calculates an operating temperature for every wire in the design, and these temperature values are then read back into Totem. The per-wire (or per-wire-segment) temperatures are then used to re-extract resistor values in the design, and to compute new EM limits. This is performed for Power Integrity, Signal EM, and ESD analyses, and a temperature report created in each case. *Optional; default :1000*

Syntax:

```
TEMPERATURE_REPORT_LINE_NUMBER <max_lines>
```

TOTEM_IR_REPORT_LINE_NUMBER

Specifies the maximum number of worst wire and via values to be listed for power and ground domains by the display command **Results-> List of Worst IR for Wire and Via**, as well as in the corresponding text report. It is possible to specify up to maximum of 100,000 lines. Note that this feature is only available in the Totem2 interface. *Optional; default :1000 power and 1000 ground values.*

Syntax:

```
TOTEM_IR_REPORT_LINE_NUMBER <num_lines_power/ground>
```

VDD_NETS

Specifies the voltage for Vdd nets, and also defines equivalent power domain net names in DEF, which are merged in Totem. The power domain names are defined in the DEF file using the SPECIALNETS keyword. See also the SUPPLY_NETS keyword for handling special supply cases. *DMP compatible. Required. Default: none.*

Note: A net can only be merged into one domain (the first one specified). There can be no net hierarchy.

Syntax:

```
VDD_NETS
{
 <Vdd_domain_net_name> <value_Volts> {
    <equiv_Vdd_net_name1>
    ...
    }
 ...
}
```

where

<Vdd_domain_net_name> : specifies DEF name for power domain net, such as VDD for the core power domain, and VDDQ for the I/O power ring.

<value_Volts> : nominal voltage

<equiv_Vdd_net_name1> : equivalent power net name in same domain

Example:

```
VDD_NETS
{
 VDD 1.1 {
    VDD1CORE
}
 VDDQ 3.3 {
    VDDQ1
}
}
```

VIA_IR_REPORT

When turned on, generates a Via Voltage Drop and Current report for both static and dynamic analysis. The report contains voltage drop and current values through the vias, up to a limit of 1000 lines. Also, you can use the TCL command 'report [ir | dvd] -via -o <output_file>' to generate this report. The default output files are:

- Static: *adsRpt/Static/< >.via.ir.worst*
- Dynamic: *adsRpt/Dynamic/< >.via.ir.worst*

Optional. Default: 0 (no report).

Syntax:

```
VIA_IR_REPORT [ 0 | 1 ]
```

Custom Cell Modeling Keywords

CMM_CELLS

Specifies the available Custom Macro Model (CMM) cells to be used in the design, to speed up handling of hierarchical memory blocks and other macros used multiple times in a design. The binary ‘optimize’ option reduces the RLC network within CMM instances, resulting in fewer nodes and resistors, and thus reducing simulation memory use and runtime. If desired, you can use the full binary model by specifying the ‘original’ option. The ‘raw’ model is not the original input design data, such as a Spice netlist and GDS; it contains the processed LEF and DEF electrical and physical data, including a GSR and TCL template, which allows you to regenerate a CMM. The raw model can solve issues with different release versions, and allow regenerating a CMM with a different technology file to resolve a CMM version incompatibility, or a technology layer stacking consistency limitation. The present values can be displayed using ‘gsr get CMM_CELLS’. Note that run exits if the cell name specified within CMM_CELLS keyword mismatches with that in the CMM model. *DMP compatible. Optional. Default: raw.*

Syntax:

```
CMM_CELLS {
<cell_name> <model_cell_path> [ raw | original | optimize ]
...
}
```

CMM_CREATE_PINS

Totem creates surface pins that cover the entire layer geometry for the top two routing layers by default, and in lower metal layers it creates boundary pins on the edges of wire shapes that touch the bounding box of cells. These small pins are created to connect to other geometries at the top level by abutment. The boundary pins are not created if the geometry does not touch the boundary of the cell. See the related keyword, CMM_PIN_BOUNDARY_LIMIT for setting pin distance from the cell boundary. Setting the value to 0 turns off pin creation. *DMP compatible. Optional. Default: 1 (On).*

Syntax:

```
CMM_CREATE_PINS [ 0 | 1 ]
```

CMM_CONSTRAINT_FILE

Specifies a file containing design parameter constraints for the IP/blocks in the design. The file is stored in the CMM model. When the CMM is loaded SOC analysis, extra post-processing steps are taken to check the specified constraints generate a constraint violation report in the top-level assembly stage after analysis indicating compliance with the constraints provided by the IP/block designer. For descriptions of output reports on CMM constraints, see section "CMM Reports", page 6-228. *Optional. Default: None*

Syntax:

CMM_CONSTRAINT_FILE <filename>

Supported constraint types are:

a. min/max voltages in dynamic analysis and static IR checks for block to top-level connection points

b. min/max voltages in dynamic analysis and static IR checks for pins inside the blocks

The CONSTRAINT FILE content format is as follows:

```
<cell <cellname>
TOP_CONNECTION_MAX_VOLTAGE {
    <pg_domain> <value>
}
TOP_CONNECTION_MIN_VOLTAGE {
    <pg_domain> <value>
}
TOP_CONNECTION_IR {
    <pg_domain> <value>
}
PIN_MAX_VOLTAGE {
    <pg_domain> <value>
}
PIN_MIN_VOLTAGE {
    <pg_domain> <value>
}
PIN_IR {
    <pg_domain> <value>
}
```

Note 1: If a domain has no specification in some category, top-level analysis does not check that domain for that category.

Note 2: <value> can be specified in units of Volts (or as drop percentage for power domains only).

Note 3: TOP_CONNECTION_IR and PIN_IR are checked for Static analysis only, while all others are for dynamic analysis.

Constraint file example:

```
#cell ram64x8
TOP_CONNECTION_MAX_VOLTAGE {
    vdd2 1.085
    gnd 0.05
}
TOP_CONNECTION_MIN_VOLTAGE {
    vdd1 1.07
    vdd2 1.07
}
TOP_CONNECTION_IR {
    vdd1 5%
    vdd2 5%
    gnd 0.05
}
PIN_MAX_VOLTAGE {
    gnd 0.1
}
PIN_MIN_VOLTAGE {
    vdd1 1.065
    vdd2 1.065
}
PIN_IR {
    vdd1 7%
    vdd2 7%
    gnd 0.07
}
```

CMM_EXCLUDE_FILES

By default, all available files in CMM models are imported at top level when importing the CMM. To exclude some of these files by CMM model and file type, this keyword can be used. *Optional. Default: none.*

Syntax:

```
CMM_EXCLUDE_FILES {
    <modelname1> <file_type1> <file_type2> ...
    <modelname2> <file_type1> <file_type2> ...
...
}
```

where

<modelname1>: CMM model name

<file_type> : type of file to be excluded

Example:

```
CMM_EXCLUDE_FILES {
    M1 ipf vcd spf sta ...
    M2 bpfs gsc
```

```
}
```

So in the example, file types IPF, VCD, SPF and STA should not be imported for CMM model M1, and BPFS and GSC information should not be imported for model M2. Note that BLOCK_POWER_ASSIGNMENT (BPA) *cannot be excluded* from the CMM.

Extra caution is needed when importing any instance power file (IPF), since any instance not covered by IPF is assigned a power of 0, which may not be desirable. So if any IPF file is imported, a warning is displayed that an IPF was imported for some CMM models, but not for top level or other CMM.

CMM_EXPAND_PINS_AT_TOP

Turning on this keyword expands DEF pins that are smaller than the shapes of wire geometries that they overlap, to eliminate CMM model connectivity issues at the top level of the design. *Optional. Default: 0.*

Syntax:

```
CMM_EXPAND_PINS_AT_TOP [ 0 | 1 ]
```

CMM_IGNORE_BLOCK_GSC

When set, honors the top level GSC file for defining instance switching when the top level GSC file contains wildcard characters. *Optional. Default: 0.*

Syntax:

```
CMM_IGNORE_BLOCK_GSC [0 | 1 ]
```

CMM_INCLUDE_APL

By default generated CMM models include all APL files that are specified in the APL_FILES keyword. Then the APL files used in CMM generation can be specified in GSR file in the top level run. *Optional. Default: 1.*

Syntax:

```
CMM_INCLUDE_APL [ 0 | 1 ]
```

CMM_INSTANCES

This keyword can be used to select either the 'optimize' or 'original' RLC network model for a specific CMM instance, rather than specifying the model at the CMM cell level. You can specify either 'optimize', 'original' or 'compact' model for a CMM cell using the 'CMM_CELLS' keyword.

For CMM instances not listed in the CMM_INSTANCES keyword, or the CMM_INSTANCES keyword is absent, then the instance use the RLC view as defined at cell level in CMM_CELLS. Note the interaction between the CMM_CELLS and CMM_INSTANCES keywords:

a. If optimization is needed for a CMM cell based on the CMM_CELLS keyword value, then the optimized model is generated, regardless of its actual use by any CMM Instance.

b. If a CMM instance is specified to use the optimized model, but no optimized model is generated for the CMM cell (due to cell level options), then the CMM instance uses the original model.

You can verify which model was used for an instance in the output a*dsRpt/apache.CMM.rpt* file. *Optional. Default: None.*

Syntax:

```
CMM_INSTANCES {
   <cmm_instance_path> {original | optimize}
    ...
}
```

CMM_LAYER_MAP_FILES

When specified, provides layer mapping when the technology and LEF layers are named differently in the CMM and in the top level design tech files. This feature is used in the top level run for a design containing CMMs. Layer maps are CMM cell specific; their layer names are mapped to layer names in top level design using the specified mapping files, which should specify the full name translation from layer names in CMM to top level layer names. The layer stacking order and matching layers' properties must be the same. The format of a CMM layer map file is:

```
<CMM_layer_name> <top_layer_name>
...
```

If you have a layer used in CMM but not in the top level, you should specify the relevant top tech layer as “UNUSED”, such as in the following file example:

```
pdiff UNUSED
co UNUSED
ndiff UNUSED
po UNUSED
METAL0 M0
VIA0 V0
Metal1 M1
...
```

The AUTO option creates a layer mapping file automatically by layer order for the CMM cell, assuming cells have the same layer order and position in both CMM and top level design. So for example, if technology file 1 has a fourth layer with name “m1”, and technology file 2 has a fourth layer named “metal1”, then if the ‘AUTO’ option is used, layer m1 is mapped to metal1, since both are fourth layers in their own tech files. *DMP compatible. Optional. Default: none.*

Note: if neither a layer map file nor AUTO is specified for a block, its layers are merged automatically based on layer height.

Syntax:

```
CMM_LAYER_MAP_FILES {
    <CMM_cellname> [<filename> | AUTO | UNUSED]
    ...
}
```

Example 1 - How the file is created :

Assume that your CMM tech file has 12 layers , C1 to C12, and your top tech file has 16 layers, T1 to T16. In the layer map file, you would specify layers C1 to C4 as “UNUSED”, then layers C5 to C12 map from T1 to T8 (these must be in stack order in one-to-one mapping, so C5 <-> T1, C6<->T2, ..., C12<->T8. The combined tech file for the top then is C1, C2, C3, C4, T1, T2, ..., T16. The layer properties for C1 to C4 layers in top tech are copied from the CMM tech file, the C5 to C12 layers in the CMM tech are dropped, and T1 and T8 layer properties from top tech are used for the layers in CMM. Totem does not check the property of layers C5 to C12 to see if they match T1 to T8; Totem only checks that the temperatures used for extraction for layer C5 to C12 matches the temperatures of layers T1 to T8.

Example 2:

For three CMM blocks in a top level run, CMM_LAYER_MAP_FILES could describe the layers as follows:

```
CMM_LAYER_MAP_FILES {
  dpL1DDATARAM ./myLayerMapFile
  dpIDATARAM AUTO
  #  dpLSReqQueCam
}
```

So for CMM block ‘dpL1DDATARAM’, layers are merged based on the layer map file “myLayerMapFile”. For ‘dpIDATARAM’, first a layer map file is created automatically by layer order and then the tech files are merged. For ‘dpLSReqQueCam’, neither a layer map file nor AUTO are specified, so its layers are merged automatically based on layer height.

CMM_MODEL_CREATION

When this flag is set, Totem prepares to create Custom Macro Models (CMM) for hierarchical memory blocks and other macros (this keyword was formerly called CHARACTERIZE_BLOCK). The models are created after extraction, when the design is saved:

```
save design -o <CMM_model_dir_path>
```

Required for CMM generation. Default: 0 (off).

Syntax:

```
CMM_MODEL_CREATION [ 1| 0]
```

CMM_PIN_BOUNDARY_LIMIT

Allows CMM pin creation only within a specified minimum distance from the cell boundary. The CMM_CREATE_PINS keyword creates surface pins that cover the entire layer geometry for the top two routing layers by default, and in lower metal layers small boundary pins are created on the edges of wire shapes that touch the cell bounding box to connect to other geometries at the top level by abutment. The boundary pins are not created if the geometry does not touch the boundary of the cell. The keyword CMM_PIN_BOUNDARY_LIMIT can be used to set the distance from the boundary so that the pins can be created within the specified distance from the cell boundary. *Optional. Default: 0.01 um*

Syntax:

```
CMM_PIN_BOUNDARY_LIMIT <distance_um>
```

CMM_POWER_OVERRIDE_IPF

When set to 1, honors CMM Raw model BPA/BPFS instead of instance power file (IPF). *Default: 0.*

Syntax:

```
CMM_POWER_OVERRIDE_IPF [ 0 | 1 ]
```

CMM_PROCESS

Adds information in the text header in the CMM DB model that contains PVT and tech file information, for reporting at the top level. The header file contains process, voltage, power and ground nets data and operating temperature for the CMM model. Totem gets the domain-specific voltage and temperature data from this keyword and embeds it in the header. *Optional. Default: None.*

Syntax:

```
CMM_PROCESS  <string>
```

where <string> can be just the process specification to be put into the header. The following is an example of GSR entries that are then stored in the header file:

```
VDD_NETS {
    Vcc1 1.1
    Vcc3p3 3.3
}
GND_NETS {
    Vss 0
    VssA 0
}
TEMPERATURE 110
CMM_PROCESS ff_hvt
```

For more information on the CMM, you can use the command

```
perform cmmcheck <path_to_CMM>
```

and get the PVT information stored in the header file.

CMM_VIA_MODEL_LAYER

This keyword enables the user to provide a via layer and decide whether the current will be summed up (i.e., current sinks being created) to metal layer above/ below this via layer based on the "above/below" arguments By default, current sinks (*gds.XX pins) are created on the layer above. For instance, if via3 is specified, by default the current sinks would be generated on metal4. *Optional. Default: above*

Syntax:

```
CMM_VIA_MODEL_LAYER <via_layer> [ above | below ]
```

IP_MODELS

This keyword maps the IP name and the path to its unified model directory. Each IP directory include multiple views, and all the available views for the same IP are imported. User can choose the specific view type of model for different instance of the same IP. Only CMM flavors are supported. If both IP_MODELS and CMM_CELLS are specified for the same master cell, IP_MODELS will be given priority and CMM_CELLS will be ignored. *Optional. Default: None*

Syntax:

```
IP_MODELS {
    <master_cell_name> <IP Model Directory>
}
```

IP_MODEL_INST_MAP

Specifies those instances of IP using a different view, rather than the default view, to expand the usage of keyword IP_MODELS. Must be used with keywords IP_MODELS and IP_MODEL_CELL_MAP. *DMP compatible. Optional. Default: None.*

Syntax

```
IP_MODEL_INST_MAP {
<inst_name> <cellname> <specified_instance_view>
...
}
```

IP_MODEL_CELL_MAP

Specifies that the default view of the model is to be used for all instances of an IP. Must be used with the IP_MODELS keyword. *DMP compatible. Optional. Default: None.*

Syntax

```
IP_MODEL_CELL_MAP {
<cell name> <default view name>
}
```

Electromigration Keywords

ANALYZE_NETS_FILE

An extension to the keyword ANALYZE_NETS, this keyword specifies a file name that lists signal nets to be extracted, simulated and checked with EM rules in the signal EM analysis flow. Note that this keyword *cannot* be set using the TCL command “gsr set ANALYZE_NETS_FILE { }”. *Optional. Default: none.*

Syntax

```
ANALYZE_NETS_FILE <file_with_net_names>
```

CONFIGURABLE_REPORT_FILE

Specifies the configurable Signal/Power EM report filename, which specifies the data desired in the report, and their order. The default list of items that can be selected to be included in the configuration file are as follows:

Report file for power EM:

```
EM_WORST_POWER
WIRESEGMENT $LAYER $STARTX $STARTY $ENDX $ENDY $EM_RATIO $NET_NAME $WIDTH
    $CURRENT $BLECH_LENGTH
VIA $VIA_NAME $CX $CY $EM_RATIO $NET_NAME $CURRENT $BLECH_LENGTH
OPTIONS $GROUP_BY_FIRST_COL
OPTIONS $USE_OLD_FORMAT
```

Report file for signal EM:

```
EM_WORST_SIGNAL
WIRESEGMENT $LAYER $STARTX $STARTY $ENDX $ENDY $EM_RATIO $NET_NAME $WIDTH
    $CURRENT $EM_LIMIT $CURRENT_DIR $FREQUENCY
VIA $VIA_NAME $CX $CY $EM_RATIO $NET_NAME $CURRENT $EM_LIMIT $CURRENT_DIR
$FREQUENCY
OPTIONS $GROUP_BY_FIRST_COL
OPTIONS $USE_OLD_FORMAT
```

The keyword “OPTIONS $GROUP_BY_FIRST_COL” in the configurable report file allows grouping values for any parameter specified in the first column together. So when this keyword is used, whatever parameter is listed in the first column of the configurable report file, the entries are grouped together in the output reports. In the case of $NET_NAME in the first column, for example, the generated *.em.worst* files are organized with wire segment EM values together. The option “USE_OLD_FORMAT” can be used if user wants to retain the original/default header of the ‘dump res_network’ when options such as EM_NETS_ONLY or INCLUDE_WIRE_STUBS are used.

Optional. Default: none.

Syntax:

```
CONFIGURABLE_REPORT_FILE <config_filename>
```

DELTA_T_RMS_EM

Specifies the expected wire temperature increase due to Joule heating when calculating peak RMS EM current in tech file 'metal' section, using the 'delta_T' option in POLYNOMIAL_BASED_EM_RMS. *DMP compatible. Optional. Default: none.*

Syntax:

```
DELTA_T_RMS_EM <temp_increase_deg C>
```

DETAILED_CONN_ISSUE_RPT

When set, reports nets that are unconnected to the receiver instance pin, or the receiver LEF is missing, in the *adsRpt/SignalEM/connectivity_issue.rpt* file. *Optional. Default: 0.*

Syntax:

```
DETAILED_CONN_ISSUE_RPT [ 0 | 1 ]
```

DRIVER_RECEIVER_IMBALANCE_CHECK

To check current consistency between total average current flowing into and out of the net. Below two checks are done with this keyword:

a. Check the difference between avg driver and receiver current against a certain threshold for vector-based analysis. A warning is issued if it is above threshold, and all violating nets are listed under `'adsRpt/apache.curConsistency.rpt'` file.

b. Check if there is only one pin for the net. If yes, a warning is issued and such nets are reported in the file`'adsRpt/apache.sigem.onePinNet'`.

Default:1 ; uses threshold of 10uA current.

Syntax:

```
DRIVER_RECEIVER_IMBALANCE_CHECK <0|1> <differential
current>
```

DYNAMIC_EM_FRAME

When specified in the dynamic power EM analysis flow, Totem calculates the AVG/RMS current over your defined time frame or window, instead of over the entire simulation time. This allows you to perform multiple-cycle simulations in a single Totem dynamic simulation, but still calculate the AVG and RMS current to check EM violations based on the clock cycle boundary. *Optional. Default: Off. If turned on, default frame size is 1/FREQ.*

Syntax:

```
DYNAMIC_EM_FRAME [0|1] <Frame_length>
```

Example:

```
DYNAMIC_EM_FRAME 1 4n
```

DYNAMIC_TIME_STEP

When set, allows you to set up time step controls for vector-based mode signal EM analyses. For mixed mode, you can also specify SEM_VECTORLESS_TIME_STEP to control time steps for vectorless nets. *Optional. Default 10ps.*

Syntax:

```
DYNAMIC_TIME_STEP <time_step>
```

DYNAMIC_VOLTAGE_CHECK

By default, voltage checking for node voltages greater than 8*(ideal VDD) are performed. If violating node voltages are found, the simulation run is terminated with a SIM-436 warning in the log file. To turn off these voltage checks, set this keyword to 0. Note that the substrate noise injection flow, using the 'perform analysis -inject_noise' command, automatically turns off this check to avoid false errors. *Optional. Default: 1.*

Syntax:

```
DYNAMIC_VOLTAGE_CHECK [ 0 | 1]
```

EM_CHECK_2D

Controls perpendicular direction EM checking. When set, checks current in both directions. For dump resistor network values to be consistent with the GUI and EM reporting, set 'EM_CHECK_2D 0' so that there is only one EM value for each wire. *Optional. Default: 1, On.*

Syntax:

```
EM_CHECK_2D [ 0 | 1 ]
```

EM_CUSTOM_CURRENT_FILE

You can use EM_CUSTOM_CURRENT_FILE to create a custom current file to specify the current at signal net driving points. Using the custom current file you can also limit EM analysis to those nets that are specified in the file. Only nets specified in file are reported if you also set the GSR keyword 'EM_CCF_ONLY' to 1.

The format of the custom current file is as follows:

```
<net_name> INST <inst_name> <pin_name> <avg> <rms> <peak>
...
<net_name> CELL <cell_name> <pin_name> <avg> <rms> <peak>
...
```

where

net_name: specifies the net name

INST <inst_name>: specifies the instance name

pin_name: specifies the pin name

<avg> <rms> <peak>: specifies the current value for each type of current, in Amps. Current types that are not to be set should be given values of '-1'.

Example custom current file contents:

```
net1 INST ANA1 VREGO -1 -1  1e-3
net2 INST ANA1 SIGPIN -1 -1 2e-3
```

The example sets PEAK current to '1e-3' for net1 and '2e-3' for net2 at driving points VREGO and SIGPIN, respectively. Other current values are unspecified. *Optional. Default: None*

Syntax:

```
EM_CUSTOM_CURRENT_FILE <filename>
```

Example:

```
EM_CUSTOM_CURRENT_FILE abc2.ccf
```

EM_CCF_ONLY

When specifying an EM_CUSTOM_CURRENT_FILE, only nets specified in the file are reported if you also set 'EM_CCF_ONLY' to 1. *Optional. Default: 0 (off).*

Syntax:

```
EM_CCF_ONLY [ 1 | 0 ]
```

Example:

```
EM_CCF_ONLY 1
```

EM_DUMP_PERCENTAGE

EM_DUMP_PERCENTAGE specifies the range of EM ratio percentages to be reported in the results files **em.rms, *.em.avg* and **.em.peak* reports. where EM_Ratio = actual_current_density/ current_density_limit. The **.em.** file reports EM-related parameters for wires and vias, but is not generated if EM_DUMP_PERCENTAGE is not specified. *Optional. Default: None*

Syntax:

```
EM_DUMP_PERCENTAGE <min_percent> <max_percent>
```

where <min_percent> defines the minimum EM ratio percentage to be reported and <max_percent> is the maximum EM ratio percentage to be reported. If only <min_percent> is specified, all EM ratio percentages higher than <min_percent> are reported. Values of <min_percent> and <max_percent> are integers.

Example:

```
EM_DUMP_PERCENTAGE 0-20
```

For wires, the line format for EM data in the **.em* file is as follows:

```
<layer> <segment_end_coords> <EM_Ratio> <Current_value> <netname> <width>
```

For vias, the line format for the EM data is:

```
<via_name> <x-y_coord> <EM_Ratio> <current_value> <netname>
    <via_cut_bounding_box_coords> <current_direction>
```

Current values are in Amps, and coordinates and dimensions are in um. To include values of <via_cut_bounding_box_coords> and <current_direction> in the report, users must set the GSR keyword PRINT_EM_VIA_BOX to 1.

EM_DUTY_RATIO_ROOT

Allows you to specify the root value of the expression for duty ratio. The duty ratio is defined as r = signal duration / (1/(f*TR)), and Ipeak_new = I_peak * r^(1/root), where f is the clock frequency and TR is the signal toggle rate. *Optional. Default:* 2.

Syntax:

SEM_DUTY_RATIO_ROOT <root_value>

Example:

```
SEM_DUTY_RATIO_ROOT 3
```

In this example, Ipeak_new = I_peak * r^(1/3)

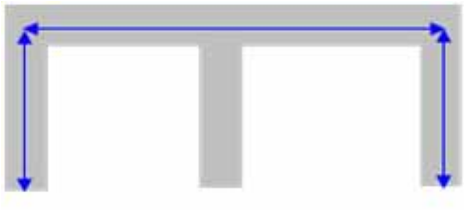
EM_LENGTH_USE_MAX_LENGTH

When set, specifies wire length for EM rule lookup is defined based on all connected wire segments and terminating at a via, using the segment centerline. The calculation rules are:

- When set to “0”, Totem uses the total metal length for EM rule lookup, that is, all connected segments (default).
- When set to “1”, Totem uses the segments representing the longest metal

length for EM rule lookup, as shown in the figure below.

DMP compatible. Optional. Default:0.



Syntax:

```
EM_LENGTH_USE_MAX_LENGTH [0|1]
```

EM_MISSION_PROFILE

Specifies a file used to define temperature-dependent coefficients to relax EM limits for metal and vias. By default all coefficients are set to 1, in which case no scaling takes place. Supports also two-dimension EM mission tables corresponding to Temperature and Life_Cycle for defining coefficients to relax EM limits for metal wires and vias. There is no interpolation (must match exactly) for Temperature/ Life_Cycle values. If the option "REGULAR_TEMP_OFF" is specified in EM_Mission profile, then the temperature derating is disabled and EM Mission Temperature / Life_Cycle / Fail_rate factor is applied to EM limits. *DMP compatible. none.*

Syntax:

```
EM_MISSION_PROFILE <path_to_file>
```

Contents of a sample mission profile are shown below:

```
via VIA12 {
    { 0 5 }
    { 25 10 }
    { 105 100 }
    { 110 0.697 }
    { 125 0.25 }
    { 150 0.053 }
}
metal METAL1 {
    { 0 5 }
    { 25 11 }
    { 105 111 }
    { 110 0.697 }
    { 125 0.20 }
    { 150 0.053 }
}
```

In the above example, at a temperature of 125° C, the EM limit for METAL1 is scaled by a factor of 0.20.

Two dimensional example mission profile:

```
metal M1 {
  Temperature { 0 25 50 100 125 }
  Life_Cycle { 1000 2000 3000 }
  VALUES {
```

```
    { 1 2 3 4 5 }
    { 1 2 3 4 5 }
    { 1 2 3 4 5 }
}
```

EM_MODE

Specifies the type of EM results to be displayed from dynamic analysis. The true average, root-mean-squared, or peak value of current over the simulation time is compared to the specified EM limit. This keyword is not applicable to static analysis, which provides true average current values only. Note that tech file keywords 'EM_RULE_SET_*' and 'EM_RULE_SET <rule name>' (see section "EM_RULE_SET", page C-746) can be used to define customized EM rules. *DMP compatible. Optional. P/G EM analysis default: peak; Signal EM analysis default: rms.*

Syntax:

```
EM_MODE [ avg | rms | peak ]
```

Example:

```
EM_MODE peak
```

EM_NET_INFO

Defines a file specifying transition times or rising/falling times, toggle rates, unidirectional and bidirectional current scaling factors, extra receiver capacitance ("extra-cap") per signal net, and also the slew (transition time) range for all nets, including nets that are set as CONST in the STA file. These values override the global values of TOGGLE_RATE and EM_SLEW_* keywords, as well as any toggle rate values set in power calculation or transition times in the STA file. Note that APLMMX automatically applies EM_NET_INFO settings to nets connected using the METAL_RESISTOR configuration file keyword.

You can input the slew range as a comment in the header, such as "# slew_range <value>", where <value> is any floating point number between 0 and 1. Columns 3 ~ 7 values are optional, but all columns to the left of a specified value must have at least a "-" column placeholder. The last column assigns a voltage to primary input signal nets (optional) for signal EM analysis. Wild cards for <net_name> are allowed. With accurate separated rise and fall input transition times and setting "SEM_SLEW_OPTIMIZATION 4", vectorless signal EM can achieve accurate EM correlation with Spice. The content format is:

```
<netname> <rising_time_sec>/<falling_time_sec> <toggle_rate>
   <freq> <uni-dir_scale> <bi-dir_scale> <extra_cap>
   ?volt{<voltage>}?
```

A example file follows:

```
# slew_range 0.8
net1 1e-13 2 2e09 0.7 - 1e-14
net2 10e-12/20e-12  2 2e9 - - - volt{1.3}
...
```

In the example above, 'net1' is assigned a transition time of 1e-13 seconds, within the slew range of 80%. It's also assigned a toggle rate of 2 and frequency of 2 GHz. Its uni-directional currents are scaled by 0.7. No bi-directional scaling is used, but an extra receiver side load pin cap of 1e-14 F is applied. The 'net2' line indicates rising/falling times, as well as the toggle rate, frequency, and primary voltage.

You also can roll-up or propagate slew/toggle-rates from assigned nets in the EM_NET_INFO file to unassigned nets identified via CCR tracing *Optional. Default: none.*

Syntax:

```
EM_NET_INFO <filename> ?[0|1]?
```

where

0 : default, no propagation

1 : turns on slew/toggle propagation

Sample scenarios:

- If a solely bi-directional net has a single slew assigned in EM_NET_INFO, the same is propagated to pull-up and pull-down nets in CCR. If the bi-directional net has rise and fall-slews, the pull-up nets are assigned a rise-slew and pull-down nets are assigned a fall-slew.
- If different slews/toggle-rates are assigned on multiple pull-up/pull-down branches driving a common net, minimum slew and maximum toggle-rates are propagated to the common net and subsequent nets that can be traced from it using CCR.
- If a uni-directional current scaling factor is assigned for one branch, use the same scale factor for other uni-directional branches of that CCR.
- When set, uses effective pulse width calculations in peak EM rules. The pulse width is scaled as follows:

  a) vector-based: pulse_width_effective = pulse_width*period/simulation

  b) vectorless: pulse_width_effective = pulse_width*true_toggle_rate

  APLMMX (both vector-based and vectorless) takes EM_NET_INFO and stores frequency for each net in the *dspf.rh* file. When the net can get the frequency, pulse_width_effective is calculated automatically. The result of this is :
  - No change if EM_NET_INFO or FREQ is not used for vector-based analysis.
  - For the vectorless flow, there can be a change in final effective pulse width because true_toggle_rate is used for scaling.
- The order or priority for slew/freq/TR assignment is as follows (highest to lowest)
  a. Use explicitly specified net info for the net in EM_NET_INFO file
  b. If (a) not available, then a rolled-up info will be used (if it applies for that net). Rolled up info can be from multiple branches. Worst one will be used.
  c. If both (a) and (b) do not apply, then default values will be assigned from wildcard characters.

EM_REPORT_MODE_ONLY

The speed of signal EM analysis can be increased by generating only the report desired using EM_REPORT_MODE_ONLY. The EM_MODE keyword determines the type of report displayed: average, rms (default), or peak, but all three types of reports are generated. If you turn EM_REPORT_MODE_ONLY On (1), only the selected report is generated, which for 'rms' means that only the file *<design>.rms_em.worst* is generated (under the directory *adsRpt/SignalEM*). *Optional. Default: 0 (off)*

Syntax:

```
EM_REPORT_MODE_ONLY [ 0 | 1 ]
```

EM_REPORT_PERCENTAGE

Specifies the portion of calculated EM current ratios that should be reported in the *esd_em.rpt* file. For each wire segment and via Totem reports the ratio of the current in it relative to the “critical” current value set by the user in the *.tech* file, which is the “EM percentage”. So if the EM_Report_Percentage is set to 50, all calculated EM current ratios that exceed 50 percent are reported (up to the specified Report_Line_Number limit). However, note that at least the worst EM percentage value per wire or via is reported when the worst EM percentage values are less than the specified EM percentage limit. If set to -1, all percentage values are reported. *DMP compatible. Optional. Default: 100 (percent)*

Syntax:

```
EM_REPORT_PERCENTAGE <percent_violation_thresh>
```

Example:

```
EM_REPORT_PERCENTAGE 80
```

EM_REPORT_PERCENTAGE_BY_LAYER

Specifies the portion of calculated EM current ratios that should be reported by specified layer(s). For each specified layer, Totem reports the ratio of the current in wire segments and vias relative to the “critical” current value set by the user in the .tech file, which is the “EM percentage” in %. If the layer is not included in the block, then the global value specified by the keyword “EM_REPORT_<mode>_PERCENTAGE <percent_of_limit> is used. If there is no mode specified, the specified percent of limit values are applied to all three modes. *Optional. Default: none.*

Syntax:

```
EM_REPORT_PERCENTAGE_BY_LAYER  ?[AVG| RMS| PEAK] ? {
    <layerName1> <percent_of_limit1>
    <layerName2> <percent_of_limit2>
    ...
}
```

EM_REPORT_<mode>_PERCENTAGE

Supports different EM percentage limits for AVG, RMS, and PEAK modes, which creates three files listing the EM limit violations:

<cell>.em.worst.avg

<cell>.em.worst.rms

<cell>.em.worst.peak

The three output reports list EM values exceeding the specified limits for each mode. Separate EM limits can be set for each mode with these three keywords. Individual limits that are not specified default to the limit in the EM_REPORT_PERCENTAGE keyword. *DMP compatible. Optional. Default: none.*

Syntax:

```
EM_REPORT_DC_PERCENTAGE <percent_of_limit>
EM_REPORT_RMS_PERCENTAGE <percent_of_limit>
EM_REPORT_PEAK_PERCENTAGE <percent_of_limit>
```

EM_REPORT_LINE_NUMBER

Specifies the maximum number of lines in the output EM current ratio report, based on the EM_REPORT_PERCENTAGE keyword specification, or no limit (-1). *DMP compatible. Optional. Default: 1000.*

Syntax:

```
EM_REPORT_LINE_NUMBER [<Max_line> | -1]
```

Example:

```
EM_REPORT_LINE_NUMBER -1
```

EM_RULE_SET_DC

EM_RULE_SET_PEAK

EM_RULE_SET_RMS

Specifies the name of a set of customized EM rules to be used for AVG, PEAK and RMS analyses, along with tech file keyword EM_RULE_SET, which names and defines the different rules. This allows different rule sets to be defined prior to running and then easily specified or changed for each EM analysis desired. *Optional. Default: None.*

Syntax:

```
EM_RULE_SET_DC <rule_set_name>
EM_RULE_SET_PEAK <rule_set_name>
EM_RULE_SET_RMS <rule_set_name>
```

EM_SLEW_NO_STA

Specifies the behavior of signal EM analysis when

- a net is driven by a cell, and
- the net has no STA statistical timing information.

The GSR EM_SLEW_NO_STA keyword has three arguments representing three different ways of modeling STA data:

- 'ideal' (default) - means that the driver is considered ideal, with no internal resistance, so the transition time is very short. So if a net has a significant load (large capacitance) this generates large currents, particularly in PEAK current values.
- 'derived' - means that the transition time is computed the same as if there were no driver driving the net. So any values set by other related GSR keywords-- for example, EM_SLEW_SIG_TIME and EM_SLEW_SIG_PERCENTAGE-- are taken without any changes.
- 'input_transition' - means that the argument from GSR keyword INPUT_TRANSITION (which has a default of 100ps) is used as the value for the missing STA.data. Note the difference in this option compared to 'derived'; the 'derived' value is NOT considered a replacement for STA, but is taking the time argument directly. Fundamentally, the 'input_transition' value still takes the actual net load into account and computes from both the net load and the given INPUT_TRANSITION number its own transition time in a non-linear fashion.

(Note that signal nets that are not driven at all can also be considered, but this keyword is not applicable to them.) *DMP compatible. Optional. Default: ideal.*

Syntax:

```
EM_SLEW_NO_STA [ ideal | derived | input_transition ]
```

Example:

```
EM_SLEW_NO_STA derived
```

EM_TECH_FILE

Specifies a file that includes definitions of all needed EM rules for different conditions, using multiple EM_RULE_SET tech file keywords. The EM tech file can

contain any number of rule sets descriptions, each containing rules for all metal/via layers of interest.

Syntax:

```
EM_TECH_FILE <rule set filename>
```

EM_TECH_DC

Specifies a file that defines the average current density EM limits by layer to be used in calculating potential EM violations. The data in the specified EM file has the same syntax as the corresponding EM-related ‘metal’ and ‘via’ keywords in the tech file. If this keyword is not defined, the EM limit values are taken from the tech file. *DMP compatible. Optional. Default: none*

Syntax:

```
EM_Tech_DC <EM_average_filename>
```

Example:

```
EM_Tech_DC EM_dc_limit.mno
```

EM_TECH_PEAK

Specifies a file that defines the peak current density EM limits by layer to be used in calculating potential EM violations. The data in the specified EM file has the same syntax as the corresponding EM-related ‘metal’ and ‘via’ keywords in the tech file. If this keyword is not defined, the EM limit values are taken from the tech file. *DMP compatible. Optional. Default: none*

Syntax:

```
EM_Tech_Peak <EM_peak_filename>
```

Example:

```
EM_Tech_Peak EM_peak_limit.abc
```

EM_TECH_RMS

Specifies a file that defines the RMS current density EM limits by layer to be used in calculating potential EM violations. The data in the specified EM file has the same syntax as the corresponding EM-related ‘metal’ and ‘via’ keywords in the tech file. If this keyword is not defined, the EM limit values are taken from the tech file. *Optional. Default: none*

Syntax:

```
EM_Tech_RMS <EM_RMS_filename>
```

Example:

```
EM_Tech_RMS EM_rms_limit.efg
```

EM_TOPOLOGY_USE_ELECTRON_FLOW

Changes which wire width is used for EM calculation by the Topology-Based EM rule for vias. By default, Totem uses the width of the wire that *current* flows to for the calculation. When set to 1, Totem uses the width of the wire that *electrons* flow to for the calculation. *Optional. Default: 0.*

Syntax:

```
EM_TOPOLOGY_USE_ELECTRON_FLOW [ 0 | 1 ]
```

ENABLE_AUTO_EM

When set to 1, automatically performs EM checking during regular post-simulation processing. *Starting in v13.2,* by default you must execute a separate 'perform emcheck' step. *Optional; Default: 0.*

Syntax

```
ENABLE_AUTO_EM [ 0 | 1 ]
```

ENABLE_BLECH

By default the Blech length filtering function is on, which considers potential EM violations for circuit segments that have a Blech product less than the critical BLECH_JLC product value specified in the tech file. Totem uses a conservative approach in this calculation, using the worst case current density J and worst case sum of wire lengths for each group of connected and active wire segments on a layer. That is, the wires considered are all on the same layer directly connected to each other, and in an active part of the subnet (no stubs). If these conditions on the wires in the group are satisfied, their EM values are filtered out. *DMP compatible. Optional. Default: 1(On).*

Syntax:

```
ENABLE_BLECH [ 0 | 1 ]
```

ENABLE_POLYNOMIAL_EM

Supports equation-based EM without using the Blech effect, which enables you to use polynomial-based EM when ENABLE_BLECH is set to 0. EM values are still calculated based on the polynomials defined in the tech file. The different setting conditions are shown below. *DMP compatible. Optional. Default: 0*:

ENABLE_BLECH	ENABLE_POLYNOMIAL_EM	Polynomial EM used
0	0	NO
0	1	YES
1	don't care	YES

Syntax:

```
ENABLE_POLYNOMIAL_EM [ 0| 1 ]
```

IGNORE_HALF_NODE_SCALE_FOR_EM

When set, uses original design dimensions and ignores half node scale factor for computing EM limit violations. *Optional. Default: 0 (Off)*

Syntax:

```
IGNORE_HALF_NODE_SCALE_FOR_EM [ 0| 1 ]
```

MERGE_ABUTTED_CUTS

For designs with via cuts touching each other, Totem by default applies EM limits per via cut based on area, but you can use MERGE_ABUTTED_CUTS to merge these cuts into a single cut to obtain a common EM limit. To use the keyword, provide values in the EMV_VS_AREA table in the Tech file, and then set 'MERGE_ABUTTED_CUTS on'. *Optional. Default: 0ff.*

Syntax:

```
MERGE_ABUTTED_CUTS [on | off ]
```

NEW_MERGE_WIRE

When set, improves signal EM analysis with better geometry handling of jog patterns in EM checking. *Optional. Default: 0 - 0ff.*

Syntax:

```
NEW_MERGE_WIRE [ 0 | 1 ]
```

SEM_ANALYZE_NET_ONLY

The speed of signal EM analysis can be increased by limiting the number of EM objects included and generating only the type of analysis desired. This option only impacts signal net current calculation and EM checking. It does not affect the extraction stage. The default value ‘ALL’ analyzes all signal nets. ‘LAYER’ and ‘VIA’ options provide a means of filtering analysis results. Only one option value may be specified at a time. (Note that previous name EM_ANALYZE_NET_ONLY still is functional, but will be phased out in a future release.) *Optional. Default: ALL*

Syntax:

```
SEM_ANALYZE_NET_ONLY [CLOCK|SIGNAL|LAYER|VIA|ALL]
```

where

CLOCK: limits the EM report to signal nets that are of type 'clock' (part of a clock tree)

SIGNAL: limits the EM report to signal nets that are not of type 'clock'

LAYER: limits the EM report to signal net metal layers, but not vias

VIA: limits the EM report to signal net vias, but not metal layers

ALL: reports EM results on all signal nets

Example:

```
SEM_ANALYZE_NET_ONLY LAYER
```

SEM_ENABLE_SHORTS_REPORT

When set, creates a report of signal net shorts in the design in the file *adsRpt/shorts.rpt. Optional. Default: 0.*

Syntax:

```
SEM_ENABLE_SHORTS_REPORT [ 0 | 1 ]
```

SEM_NET_CHECK_SHORT

When set, checks signal EM nets for driver-receiver shorts. Nets with shorts are dropped from analysis. *Optional. Default: 0.*

Syntax:

```
SEM_NET_CHECK_SHORT [ 0 | 1 ]
```

SEM_NET_TOGGLE_RATE

Specifies exclusively the toggle rate of each signal nets. Note that keywords SEM_NET_TOGGLE_RATE_FILE and SEM_NET_INFO can also specify the toggle rate of signal nets.The values in SEM_NET_TOGGLE_RATE_FILE has priority over SEM_NET_INFO, and SEM_NET_TOGGLE_RATE overrides both values.*Optional. Default: None*

Syntax:

```
SEM_NET_TOGGLE_RATE {
      <net_name1> <toggle_rate1>
      <net_name2> <toggle_rate2>
```

```
    ...
}
```

Example:

```
SEM_NET_TOGGLE_RATE {
    net1   0.1
    net2   0.2
    ...
}
```

SEM_NET_TOGGLE_RATE_FILE

Specifies a file containing the toggle rate values for signal nets.*Optional. Default: None*

Syntax:

```
SEM_NET_TOGGLE_RATE_FILE <filename>
```

The format of the file is as shown below:

```
<net_name> <toggle_rate>
```

SEM_RECOVERY_FACTOR

Allows calculation of the effective signal EM current as a specified relationship between the charging current and discharging current, by specifying a discharge current recovery factor, RF. By default, Totem calculates the numeric average current from net charging and discharging simulation waveforms (R=1). When a SEM_RECOVERY_FACTOR is specified, the average total current including recovery is calculated as follows:

$I_{ave1} = I_{charge} - RF*I_{discharge}$

$I_{ave2} = I_{discharge} - RF*I_{charge}$

Then I_{ave} = the larger of I_{ave1} or I_{ave2}. The recovery factor RF can be set to any value from -1.0 to 1.0. If RF is set to a negative number, some or all of the absolute value of EM current is used in the calculation, instead of using the numerical average value. A setting of RF=-1 adds the full discharge portion of the waveform to the average EM current calculation, rather than subtracting it. *DMP compatible. Optional. Default: 1.0.*

Syntax:

```
SEM_RECOVERY_FACTOR <RF>
```

Example:

```
SEM_RECOVERY_FACTOR 0.2
```

SEM_SLEW_OPTIMIZATION

Controls the accuracy of slew optimization adjusted for load, to provide better current correlation with respect to Spice. When set in vectorless signal EM, this keyword considers the resistance shielding effect in slew optimization to improve waveform construction and obtain more accurate RMS and PEAK current results.

Note that slew optimization mode 4 is specifically designed for Totem transistor-level vectorless signal EM analysis, and no other slew optimization mode should be used.

Syntax:

```
SEM_SLEW_OPTIMIZATION 4
```

SEM_VECTORLESS_TIME_STEP

When set, allows you to specify different time steps for vectorless signal EM analyses. For mixed mode, you can also specify DYNAMIC_TIME_STEP to control time steps for both vectorless and vector-based nets. *Optional. Default none.*

Syntax:

```
SEM_VECTORLESS_TIME_STEP <time_step>
```

To auto-adjust vectorless Signal EM simulation time step for very fast nets (very small transition time), user can specify option 4 after specifying preferred default time step in the keyword.

Example:

```
SEM_VECTORLESS_TIME_STEP 2ps 4
```

TEMPERATURE_EM

Specifies global operating temperature for calculating static and dynamic EM effects. *DMP compatible. Optional. Default: see* section "Temperature Setting", page 4-170.

Syntax:

```
TEMPERATURE_EM <temp ºC>
```

TEMPERATURES_EM

Specifies per layer operating temperatures for calculating static and dynamic EM effects. Optional. Default: see section "Temperature Setting", page 4-170.

Syntax:

```
TEMPERATURES_EM {
    <layer_1> <temp_1 ºC>
    ...
    <layer_n> <temp_n ºC>
}
```

THERMAL_COUPLING_PITCH

Performs RMS hot spot filtering in signal EM flow using TCL command `'perform emcheck -hotspot_filter'`. *Default: 15 for all metal layer.*

Syntax:

```
THERMAL_COUPLING_PITCH {
    <ALL | metal layer> [spacing as a multiplier to metal
pitch]
    <metal layer> [spacing as a multiplier to metal pitch]
    <metal layer> [spacing as a multiplier to metal pitch]
    . . .
}
```

Example:

```
THERMAL_COUPLING_PITCH {
    ALL 20 #assign 20 as multiplier for metal layer
    METAL1 10 #metal 1 now gets 10 as pitch multiplier
    fpoly 0
}
```

USE_DRAWN_WIDTH_FOR_EM

When set, RedHawk uses the drawn width to calculate the current density of wires. When this option is turned off (set to 0), silicon width is used for current density calculation. DMP compatible. Optional. Default: the default setting for this keyword is foundry-dependent. Please consult ANSYS Support for appropriate usage.

Syntax:

```
USE_DRAWN_WIDTH_FOR_EM [ 0| 1]
```

	USE_DRAWN_WIDTH_FOR_EM_LOOKUP 1	USE_DRAWN_WIDTH_FOR_EM_LOOKUP 0
USE_DRAWN_WIDTH_FOR_EM 1	W (lookup) = W(D) W= W(D)	Not Supported
USE_DRAWN_WIDTH_FOR_EM 0	W (lookup) = W(D) W=W(S)	W (lookup)=W(S) W=W(S)

Some foundries require EM to be computed based on silicon width, but lookup based on drawn width for 20 nm and lower nodes. This flow is enabled by setting the two keywords as shown in the table above.

USE_DRAWN_WIDTH_FOR_EM_LOOKUP

When set to 1, RedHawk uses the drawn wire width for EM rule lookup table. And when set to 0, the silicon width is used. DMP compatible. Optional. *Default: the default setting for this keyword is foundry-dependent. Please consult ANSYS Support for appropriate usage.*

Syntax:

```
USE_DRAWN_WIDTH_FOR_EM_LOOKUP [ 0 | 1 ]
```

VIA_COMPRESS

Can be used to control compression (grouping) of single cut vias in cases when run times are too long. When turned off, Totem leaves single cut vias as they are. When turned on, Totem groups single cut vias into arrays whose maximum size is the lesser of the value specified in the GSR keyword SPLIT_SPARSE_VIA_ARRAY (see page C-853) and a criteria allowing a small decrease in resistance accuracy, which limits array size. If the SSVA keyword is not set, the via grouping is based on the resistance accuracy criteria alone. Turning off via compression alters signal EM analysis violation results typically less than 0.1%, because of small changes to the underlying resistor modeling. *DMP compatible. Optional. Default: 1 (On).*

Syntax:

```
VIA_COMPRESS [1 | 0 ]
```

Extraction and Netlisting Keywords

ACCURATE_METAL_DENSITY

Provides more accurate metal density calculation for designs that contain overlapping or redundant metal geometries. This keyword was off by default prior to v13.2. *DMP compatible. Optional. Default: 1 (On).*

Syntax:

```
ACCURATE_METAL_DENSITY [1 | 0 ]
```

ALLOW_SEPARATED_METAL_PSEUDO_VIAS

Specifies when pseudo vias are inserted, based on what type of tech file is being used. By default (AUTO), when a particular foundry tech file for a 20nm or smaller node is detected, pseudo-vias are only formed between layers that either touch vertically or have an RCSD resistance table defined. This blocks pseudo vias from being extracted except between base layers for particular foundry tech files (contact your Apache AE for details on use). The default setting for all other tech files connects all non-touching vertically coincident layers with pseudo vias. *Optional. Default: AUTO.*

Syntax:

```
ALLOW_SEPARATED_METAL_PSEUDO_VIAS [ AUTO | 0 | 1 ]
```

where

AUTO (default) - as described above, depending on what foundry tech file is used. The setting for this keyword is foundry-dependent. Please consult Apache Support for appropriate usage.

0 - no via connections between non-touching vertically-coincident layers

1 - inserts pseudo vias between all non-touching vertically-coincident layers

BRIDGE_WIRE_CONNECTION

When set, Totem honors bridge connection wires (do not touch wires on the same layer, but connect to wires on other layers through vias). *Default: 0*

Syntax:

```
BRIDGE_WIRE_CONNECTION [ 0 | 1 ]
```

CELL_CURRENT_DIST_FILE

Specifies a file that contains a list of cells that receive equal current distribution on all nodes on each pin, while all other cell pin current is assigned to only one of the nodes connected to each pin instance. If there are no cells specified in this file, pin current distribution is controlled by the keyword INST_CURRENT_DIST_MODE. *DMP compatible. Optional. Default: none.*

Syntax:

```
CELL_CURRENT_DIST_FILE <filename>
```

CEXTRACTION_SPEF_LAYER_MAP

Specifies the filename for performing layer mapping if the layer names in the DSPF file are different from the tech file. *Optional. Default: none.*

Syntax:

```
CEXTRACTION_SPEF_LAYER_MAP <fileName>
```

CEXTRACTION_USE_SPEF

When set, Totem Signal EM analysis flow uses DSPF back-annotated capacitances, instead of its own extraction. The DSPF netlist is read in the same way as a SPEF file, but, Totem automatically recognizes the file format and processes it accordingly. *Optional. Default: 0.*

Syntax:

```
CEXTRACTION_USE_SPEF [ 0 | 1 ]
```

CLEAN_VIAS_AFTER_WIRES

When set, interchanges the via compression step with the wire merge step, so that compression occurs after merging, and also allows wires that come from pushed LEF pins, to improve via compression. This deals with the special cases in which metal1 is only in LEF, and via1 and metal2 are only in DEF, and generally allows more freedom for via compression. *Default: 0 (1 by default in v14.2) .*

Syntax:

```
CLEAN_VIAS_AFTER_WIRES [ 0 | 1 ]
```

CMM_RIVETED_CONN

This keyword improves the accuracy of CMM voltage drops when there are wire geometries at the top level overlapping CMM pin geometries, such as metal4 CMM pins connecting to overlaying metal4 wire geometries. In such cases Totem could identify duplicate metal4 shapes, and create parallel resistive paths that reduce the accuracy of the connections. Using this keyword removes the duplicate paths and improves accuracy. *DMP compatible. Optional. Default: On (1).*

Syntax:

```
CMM_RIVETED_CONN [ 0 | 1 ]
```

CONNECT_SWITCH_PINS

For switch cells, allows shorting of different combinations of internal and external pins. *Optional. Default: 1.*

Syntax:

```
CONNECT_SWITCH_PINS [ 0 | 1 | 2 |3 ]
```

where

- 0 : shorts only INT switch pin instance shapes
- 1 : shorts both INT and EXT switch pin instance shapes (default)
- 2 : shorts neither INT nor EXT switch pin instance shapes. Note that for designs with switches described as MACRO cells, this setting allows current propagation in the ring, as internal and external pins are not merged.
- 3 : shorts only EXT switch pin instance shapes

COUPLEC

If set to 1, extracts coupling capacitance between different nets in power and ground domains. *Optional. Default: 0 (off).*

Syntax:

```
COUPLEC [ 0 | 1 ]
```

DO_PININST_INTERNAL_CONN

When set, models P/G pins on all instances as internally connected to each other per domain, even though the LEF definition does not show that. *Optional. Default: 0 (off).*

Syntax:

```
DO_PININST_INTERNAL_CONN [ 0 | 1 ]
```

ENABLE_AUTO_SCANLINE

When set to 1, tool intelligently picks up layers in SCANLINE_MERGE_LAYERS keyword for advanced technology nodes. *Default: 0.*

Syntax:

```
ENABLE_AUTO_SCANLINE [ 0 | 1 ]
```

EXPAND_CELL_PIN_FILE

Supports modifying connections of switch internal/external pins to the rest of the power grid, for cases in which real physical connectivity is missing in DEF. This feature is particularly useful when switch cells are used in early stage analysis.

The syntax for specifying cell/ pin switch modifications in an “expand file” is as follows (distance units are microns):

```
VERSION <1.0>
CELL <cellname>
PIN <pin_name>
COPY_PIN_LAYER FROM <layer_name> TO <layer_name1> ...
LAYER <layer_name>
EXPAND <dist_left> <dist_right> <dist_up> <dist_down> FROM PIN
...
END PIN
EXPAND <dist_left> <dist_right> <dist_up> <dist_down> FROM CELL
...
END CELL
```

An example cell/pin expand file for pins would look as follows:

```
VERSION 1.0
CELL SWITCH_CELL_1A
PIN VDD_EXT
COPY_PIN_LAYER FROM metal4 TO metal5 metal6 metal7
LAYER metal7
EXPAND 20 40 20 40 FROM PIN
END PIN
PIN VDD_INT
COPY_PIN_LAYER FROM metal4 TO metal5 metal6 metal7
LAYER metal7
EXPAND 20 40 20 40 FROM PIN
END PIN
PIN VSS
COPY_PIN_LAYER FROM metal4 TO metal5 metal6 metal7
LAYER metal7
EXPAND 20 40 20 40 FROM PIN
END PIN
END CELL
```

The COPY_PIN_LAYER option can be used before the PIN definition also.

Syntax:

```
EXPAND_CELL_PIN_FILE <expand_filename>
```

EXTRACTION_INC

If ECO changes have been made to decaps, vias, or pins, and extraction must be performed again, with this keyword set to 1, an incremental extraction is performed only in the areas of ECO changes. Full design extraction is always performed for changes to wires. *Optional. Default: 0 (off).*

Syntax:

```
EXTRACTION_INC [0 | 1]
```

EXTRACT_PIN_VOL_INSTS

Allows boundary pins for blocks to be extracted for use in static and dynamic block-only runs. (Blocks are hierarchical instances that contain child instances.) To extract a block's pins in a full-chip top-down analysis, EXTRACT_PIN_VOL_INSTS specifies the post-flattened block names, and captures the "minimum" worst-case voltage seen at the block boundary pins during the top level dynamic simulation. This allows you to perform block level analysis with worst-case dynamic boundary voltages.

With the block definitions Totem extracts the pins for each specified block during '`setup design`', based on P/G net connections between the block and top-level routes. The boundary pins and their voltages are available after full chip simulation. These block pin voltages are then available to run simulation on individual blocks. *Optional. Default: None.*

Syntax:

```
EXTRACT_PIN_VOL_INSTS {
<hierarchical_path>/<block_inst1>
...
}
```

Example:

```
EXTRACT_PIN_VOL_INSTS {
abcd3560/ifd_top_inst
abcd3560/vdec_pri_inst
abcd3560/ter_eq1
abcd3560/test_mips_inst
}
```

This example creates a probe for each power/ground pin specified in DEF files describing these hierarchical blocks. To see the locations of these probes (block pins) use the menu command **View -> Connectivity -> Show Block Pins**. The names of the block pins that are created are reported, along with their measured voltages, in the file *adsRpt/fullchip_PV.rpt*.

For each block that is specified in the GSR file, a voltage source file (PLOC) is created. To perform a block level analysis, in a block-level GSR file, specify the corresponding *ploc* file generated from the top-level dynamic run. Then run block level static or dynamic analysis.

EZ_MERGE_NON_RECT_WIRE

Used to turn on 45-degree wire merging to correctly handle 45-degree wire segments with overlapping geometries. *Default : 0 (off).*

Syntax:

```
EZ_MERGE_NON_RECT_WIRE [ 0 | 1 ]
```

EZ_MERGE_NON_RECT_WIRE_MAX_LENGTH

Used to control the maximum wire length in the 45-degree wire merging process. *Default: maximum length 100 um.*

Syntax:

```
EZ_MERGE_NON_RECT_WIRE_MAX_LENGTH <max_length_um>
```

IGNORE_DUMMY_PGNET

When set, turns on checking of dummy nets specified in GSR keywords VDD_NETS and GND_NETS. *DMP compatible. Optional. Default: 0*.

Syntax:

```
IGNORE_DUMMY_PGNET [ 0 | 1 ]
```

IGNORE_OPC_METAL

When set, ignores all isolated wires with a length or width smaller than the specified dimension (microns.) *Default : 0 (off - do not ignore any wires).*

Syntax:

```
IGNORE_OPC_METAL <threshold_dimension>
```

IGNORE_THICKNESS_VARIATION

When set, ignores metal thickness variation due to metal density during resistance extraction. The net thickness, and therefore its resistance, is impacted by the local metal density (if the technology file includes a thickness variation description). If the metal density value is not reliable because of data input issues or early analysis, you can ignore its impact by setting this keyword to 1. *Optional. Default: 0.*

Syntax:

```
IGNORE_THICKNESS_VARIATION [ 0 | 1 ]
```

INST_CURRENT_DIST_MODE

When set to 1, equally distributes instance current to all nodes located in the pin instance region.

When set to -1 and when the number of instances in the design is greater than 100k, current is distributed equally to all nodes located in the pin instance region for cells defined in CELL_CURRENT_DIST_FILE. And when the instance count is less than 100k with INST_CURRENT_DIST_MODE set to -1, pin current distribution is turned on for all instances.

When INSTANCE_CURRENT_DIST_MODE is set to 0, current distribution on the pins are turned off, even if CELL_CURRENT_DIST_FILE contains some cell definitions.*DMP compatible. Optional. Default: Off (0).*

Syntax:

```
INST_CURRENT_DIST_MODE [ 0 | 1 | -1]
```

INTERNAL_CONNECT_PIN_CELLS_FILES

Totem is accurate in modeling LEF definitions of a cell because power/ground pins defined in the LEF are pushed to the upper level of hierarchy as wires, and treated the same as other geometries in the design; there is no assumption of internal

connection. However, in some cases, such as when pins need to be modeled as internally connected to each other (pin-node-to-pin-node connections with user-defined resistor values, for example), even though the LEF definition does not show that, the modeling can be changed for specified cells by using this keyword. In this case, the specified <cell_list_file> must list all cells to be modeled. *Optional; default: None.*

Syntax:
```
INTERNAL_CONNECT_PIN_CELLS_FILES <cell_list_file>
```
Example:
```
INTERNAL_CONNECT_PIN_CELLS_FILES Int_conn_cells
```

LEXTRACTION_FREQ

Specifies the inductance extraction frequency. *DMP compatible. Optional. Default: 10G.*

Syntax:
```
LEXTRACTION_FREQ <value-Hz>
```
Example:
```
LEXTRACTION_FREQ 5G
```

LEXTRACTION_MODE

Specifies the mode for running inductance extraction. A choice of 'Fast' or 'Accurate' for power net extraction is available, or 'Signal' net extraction. If using Accurate, there is no L reduction and this will result in tremendous slow-down in performance. The accuracy between Fast and Accurate modes for L extraction are less than 3% in DvD results. *Optional. Default: FAST.*

Syntax:
```
LEXTRACTION_MODE [ Fast | Accurate | Signal ]
```
Example:
```
LEXTRACTION_MODE Accurate
```

LONG_WIRE_RES_CALC

When set, wire resistance calculation uses the density for each resistive section of the wire, rather than the average density of the whole wire, which produces a more accurate resistance calculation. The density variation is only relevant in cases in which metal density information in THICKNESS_VS_DENSITY and POLYNOMIAL_BASED_THICKNESS_VARIATION tech file keyword tables is used to look up factors that determine the final wire thickness. *Optional. Default: Off (0).*

Syntax:
```
LONG_WIRE_RES_CALC [ 0 | 1 ]
```

LOWEST_METAL

Specifies the lowest metal layer name (defined in the *.tech* file) for capacitance extraction. *DMP compatible. Optional. Default: None.*

Syntax:
```
LOWEST_METAL  <metal_layer_name>
```
Example:
```
LOWEST_METAL Metal1
```

MERGE_ABUTTED_ASYM_CUTS

When set, merges abutted asymmetric via cuts, and thereby avoids false EM violations. *Optional. Default: Off.*

Syntax:

```
MERGE_ABUTTED_ASYM_CUT [ 0 | 1 ]
```

MERGE_WIRE

MERGE_WIRE is On by default (unless there are 45-degree wires in the design, when wire merging is disabled), and tries to merge all wires that overlap. Totem automatically drops overlapping nets in analysis if there are any wires that MERGE_WIRE cannot merge. These nets are reported in the file *adsRpt/SignalEM/topcell.droppedSignalNets*. Note that you can force wire merging setting 'MERGE_WIRE 1' for designs with 45-degree wires. *DMP compatible. Optional. Default: 1 (On).*

Syntax:

```
MERGE_WIRE [ 0 | 1 ]
```

MESH_VIAS_FILE

Specifying the filename *ecoRouting.tcl* with this keyword enables Totem to do ECO routing at the beginning of the 'setup design' step. Normal ECO routing, as specified in the command file, is after setup design, in which case some cell pin instance geometries may not be pushed out when they depend on ECO routing geometries. *Optional. Default: none.*

Syntax:

```
MESH_VIAS_FILE ecoRouting.tcl
```

METAL_DENSITY_BOUNDS

When set, restricts the computed metal density value between specified minimum and maximum values per metal layer. The metal “density” is the fraction of the layer covered by metal, a value between 0.0 and 1.0. This keyword is useful for early analysis, or when the data is not mature enough for accurate density computation. The min and max density values can usually be obtained from the foundry. Metal density impacts metal thickness consideration during resistance extraction, depending on the technology file description. *Optional. Default: none.*

Syntax:

```
METAL_DENSITY_BOUNDS {
    <layer> <min_density> <max_density >
}
```

Example:

```
METAL_DENSITY_BOUNDS {
 M4 0.1 0.3
}
```

In this example the computed metal area density for M4 layer metal is restricted between 0.1 and 0.3.

MINWIDTH_FROM_LEF

Specifies that the minimum wire width should be taken from the LEF WIDTH parameter. *Optional. Default: 1 (On).*

Syntax:

```
MINWIDTH_FROM_LEF [ 0 | 1 ]
```

MPR_MODE

Provides control of power grid extraction and modeling using Totem's advanced Mesh Pattern Recognition technology. MPR uses both circuit element pattern recognition and network reduction to increase analysis performance by reducing memory requirements and run time for designs with dense P/G grids. There are three active modes, with different amounts of network reduction:

0 - no MPR (default)

1 - uses pattern recognition, but no network reduction, which may improve Totem memory requirements and run time, but not simulation. This setting is recommended for IR/EM sign-off.

2 - uses pattern recognition and moderate network reduction, which may improve Totem memory requirements, but not as much in simulation as mode 3. However, this mode has better accuracy than mode 3.

3 - uses pattern recognition and aggressive network reduction. For designs with dense P/G grids, memory requirements may be decreased by 40% and run time by 50% for dynamic simulation where there are capacity bottlenecks. This setting is recommended for DvD/CPM analysis for large designs with capacity bottlenecks.

You can change the MPR mode between static and dynamic runs using the command:

```
gsr set MPR_MODE [ 1 | 2 |3 ]
```

Also, MPR supports Shortest Path Tracing (minimum resistance path tracing).

DMP compatible. Optional. Default : 0 (off).

Syntax:

```
MPR_MODE [ 1 | 2 |3 ]
```

MPR_PARTITION_REDUCTION

When set, MPR_PARTITION_REDUCTION reduces the resistor/node ratio of the design and provides partition reduction in the flow (if the pattern found is sufficiently large), and consequently improves runtime and memory use. *Optional. Default:* 0.

Syntax:

```
MPR_PARTITION_REDUCTION [ 0 | 1 ]
```

MPR_POWER_LIMIT_FOR_RED

When set, improves the accuracy of MPR reduction mode by triggering non-uniform network reduction if tile power is less than the specified value. That is, any patterned network covered by the tile area is not reduced if the tile power is less than the specified value. The recommended value is 0.001W or above. *Optional. Default: None.*

Syntax

```
MPR_POWER_LIMIT_FOR_RED <power_W>
```

NET_LOAD_FILE

Specifies a file that describes the load capacitance for a list of nets, in the format:

```
<net_name1> <load_cap1>
<net_name2> <load_cap2>
```

```
...
```

Also, the specified file can enable a scaling factor function to adjust the Totem-calculated Instance Cload (for Dynamic-IR simulation) and Instance Power (for Static-IR simulation). This function permits bounding the IR-drop calculations by adding a percentage error (derived from Spice simulations) as a scaling factor. To enable this enhancement, in the Net Load File set the keyword "OUTPUT_PIN_CAP_SCALE 1", and provide the desired scaling factors in the format :

```
<Inst_name>/<Pin_name> <scaling_factor>
...
```

Example Net Load File scaling:

```
$OUTPUT_PIN_CAP_SCALE 1
ucpu0/clk_drvr_13/Y 1.2
ucpu1/clk_drvr_13/Y 1.2
```

DMP compatible. Optional. Default: None.

Syntax

```
NET_LOAD_FILE <filename>
```

NET_NAME_CASE_SENSITIVE

Enables DSPF parsing to be case-sensitive for nets to improve GDS2DEF extraction. in the GSR file. *DMP compatible. Optional. Default: 0*

Syntax:

```
NET_NAME_CASE_SENSITIVE [ 1|0 ]
```

PIN_SLICE_CELL_LIST_FILE

Specifies a file containing a unique list of cells for which the keyword PIN_SLICE_LIMIT should apply. *Optional. Default: none.*

Syntax:

```
PIN_SLICE_CELL_LIST_FILE <cell_list_file>
```

PIN_SLICE_LIMIT

Globally splits large single transistor pins into many pins based on the specified distance factor (pitch), unless the keyword PIN_SLICE_CELL_LIST_FILE is set, when it only applies to cells listed in this file. This approach enables the distribution of currents on a large transistor pins over the entire area of the pin. The specified pitch is the distance between the new split pins.

When set, if a pin is created that is longer than the limit, additional nodes are created, starting at the left/bottom, at intervals specified in microns. Note that existing nodes for wire or via connections are not considered. *Optional. Default: none.*

Syntax:

```
PIN_SLICE_LIMIT <spacing_um>
```

Example:

```
PIN_SLICE_LIMIT 5
```

This setting creates pin nodes every 5 microns.

PLOC_SHORT_INTERNAL_NET

In some designs internal net routing goes through the package along with die. When set, this keyword provides a lower resistive path for those instances that are sitting far away from the switch, which mimics the package effect in these cases when a package file is not available, it can be achieved by shorting all bumps on the required internal net. You must specify internal net names for bumps that need to be shorted together, as well as RLC values as appropriate. If RLC values are not provided, pads on internal nets are shorted to the simulated node voltage (not an ideal voltage source). *Optional; default : none.*

Syntax:

```
PLOC_SHORT_INTERNAL_NET {
  <Net1> ? <R1> <L1> <C1> ?
  <Net2> ? <R2> <L2> <C2> ?
  ...
}
```

POWERUP_RANDOM_TOGGLE

When set, Totem starts random switching of instances in low-power domains when ramp-up voltage is more than half of ideal value. *DMP compatible. Optional. Default 0ff (no switching).*

Syntax

```
POWERUP_RANDOM_TOGGLE [0|1]
```

PPI_STD_IGNORE_TOUCH_VIA_MET

When turned on, touching pin geometries with vias are not considered and dropped from analysis (not pushed out). *Optional. Default: Off.*

Syntax:

```
PPI_STD_IGNORE_TOUCH_VIA_MET [0 | 1 ]
```

PPI_CELL_EDGE_MAX_NM_THRESHOLD

For a pin instance geometry that meets the threshold percent criteria to be pushed out (see PPI_CELL_EDGE_THRESHOLD_PERCENT keyword following), its absolute length must also be not less than the cell box length by the specified value (default is 50 um).

Syntax:

```
PPI_CELL_EDGE_MAX_NM_THRESHOLD <max_length>
```

Example

(See example for next keyword)

PPI_CELL_EDGE_THRESHOLD_PERCENT

When a pin instance geometry has a length (in the cell rail direction) that is not less than the cell bounding box length by more than the specified percentage, it is pushed out as a routing wire. Note that the absolute length criteria (see previous PPI_CELL_EDGE_MAX_NM_THRESHOLD keyword) also must be satisfied. *Optional. Default: 1%.*

Syntax:

```
PPI_CELL_EDGE_THRESHOLD_PERCENT <percent>
```

Example

For the case in which the cell boundary box length along rail direction is 1000 nm:

```
PPI_CELL_EDGE_THRESHOLD_PERCENT 20
PPI_CELL_EDGE_MAX_NM_THRESHOLD 500
```

So 20% of 1000 nm = 200 nm, and 200 nm < 500nm, so the limit set is 200 nm. Then the pin instance geometries are handled as shown below:

- pin instance geometries with length 1003 are pushed out.
- pin instance geometries with length < 800 are not pushed out.

PROBE_NODE_FILE

Specifies a file containing a list of named probe locations to query for current and voltage. The format of file contents is as follows:

```
<x_location> <y_location> <metal_layer> <probe_name>
[<net> [<current>]]
```

Sample PROBE_NODE_FILE contents:

```
1023.2 3457.1   metal1   nodeAB
1045.6 2828.1   metal2   nodeCD
1980.2 1453.7   metal3   nodeEF
```

Here, when <current> value is defined, a current sink will be added at the specified location during static IR drop analysis. Also, during resistance calculation, if a node computed for resistance is a probe node, then the name of that node will be reported in a new column ‘ProbeName’ in the res_calc report.

Note that the ‘probe’ TCL command also can be used to specify probes, and the command 'plot current -probe -name <name>' displays the current waveform. *Optional. Default: none.*

Syntax:

```
PROBE_NODE_FILE <path_to_file>
```

PUSH_PININST_CELLS_FILES

Specifies a file that lists all cells whose pin instance geometries should be pushed out unconditionally during extraction. In some types of designs most cells in the design would have to be added to the <ppi_cell_file> to catch all desired intra-cell connections. The file also can be used to specify shorted pin nodes within a single layer and within certain bounding box regions inside a cell. Wild card characters in the cell names and pin names are allowed. The types of syntax allowed in the PPI file are given below:

Original format:

```
<cellName> [ All |<pinName1> { <layer1> <layer2> ...} <pinName2> {...} ...]
```

Newer format:

```
<cellName>[All|PIN {<pinName1> ...} LAYER {<layerName1> ...} {BBOX <bbox1>}]
...
<cellName>[All|PIN {<pinName1> ...} LAYER {<layerName1> ...} {BBOX <bboxN>}]
```

For both types of syntax above, wildcards can be used to specify both <cellName> and <pinName>, using the single-character wildcard “?”, and the multiple-character wildcard “*”. The <bbox> is specified as the x,y coordinates of the lower left and upper right corners of the region. BBoxes and layers are the same for all wildcard members. Multiple lines with the same cellname and different boundary boxes can be specified. The layername and bbox parameters are optional. A sample push pininst cells file is shown below:

```
AB2 PIN { VDDS } LAYER { MET1 } BBOX {0 0 5 5}
```

```
AB2 PIN { VDDS } LAYER { MET1 } BBOX {10 10 20 20}
```

In the above example, all the nodes in the region {0 0 5 5} for cell AB2 on layer MET1 for pin VDDS are shorted together. Similarly, for the region {10 10 20 20} for the cell AB2 on layer MET5 for pin VDDS, all nodes are shorted together. However, these two regions are not shorted. NOTE: Only pins of the same layer and of the same net can be shorted together. *DMP compatible. Optional. Default: None.*

Syntax:

```
PUSH_PININST_CELLS_FILES <ppi_cell_file>
```

PUSH_PG_PININST

When set, extracts and processes P/G pin instances in standard cells even if they are not on a boundary, since in some low-power designs, they are important and need to be extracted. When PUSH_PG_PININST is turned On :

- If cell is not a standard cell, all pins are extracted
- If cell is a standard cell (it has rail pins), and
  - it has P/G pins on the boundary (rail pins), pins are extracted
  - it has P/G pins not on the boundary, pins are extracted
  - for non-P/G pins, pins not extracted

The default behavior is that standard cell pins not on a boundary are not extracted. *DMP compatible. Optional. Default: 0, off.*

Syntax:

```
PUSH_PG_PININST [ 0 | 1 ]
```

REPORT_ALL_UNCONNECT_PORTS

Specifies how to report pins containing one or more unconnected groups of geometries in the file *adsRpt/<topcellname>.PinInst.unconnect.md* (a “group” is a set of geometries connected together, although not necessarily to the pin). which has the following content syntax:

```
PinInst<multi_port_macro_1:gnd> (total groups 5)
1/5 at ( 32.000000 27.000000 ) M2
```

This shows that one group of five associated with the pin is unconnected, and indicates one x,y location identifying the unconnected group.

Optional. Default: Off.

Syntax:

```
REPORT_ALL_UNCONNECT_PORTS [ off | all | macro_only |
macro_detail | macro_std_all ]
```

where

off : if all pin geometries are unconnected, the instance is reported as unconnected (default). Used for standard cells.

all: if any pin geometries are unconnected, the instance is reported as unconnected. Used for other than standard cells.

macro_only: standard cell macros are reported as in the 'off' option; non standard cell macros are reported as in the 'all' option.

macro_detail: a list of macro instances with at least one pin disconnected is reported in the file *adsRpt/<topcellname>.PinInst.unconnect.md.*

macro_std_all : provides detailed reporting for all standard cell disconnects and macro pinInst disconnects in the file *adsRpt/<design name>.PinInst.unconnect.md.*

SCANLINE_MERGE_LAYERS

Improves accuracy of 45-degree wire extraction when cleaning wire geometries on the layers specified, including substrate layers, or ALL layers. *DMP compatible. Optional. Default: none.*

Syntax:

```
SCANLINE_MERGE_LAYERS {
[ ALL | <layer1> <layer2> ...]
}
```

SINGLE_CUT_SLICE_LAYERS

Specifies layers on which to slice multiple-cut via geometries into separate single cuts for cases in which compression of multi-cut vias is needed. (Totem does not compress multi-cut via groups, so using this keyword changes the input vias to single cuts, allowing compression.) *Optional. Default: none.*

Syntax:

```
SINGLE_CUT_SLICE_LAYERS {
    <layer1>
    <layer2>
    ...
}
```

SIZE_BASED_PININST_CURRENT_DISTRIBUTION

When set, Totem distributes current according to pin geometry size when there are many nodes in the same section of a pin. *DMP compatible. Optional. Default: Off.*

Syntax:

```
SIZE_BASED_PININST_CURRENT_DISTRIBUTION [ 0 | 1 ]
```

SKIP_CONNECTIVITY_CHECK

When set, skips the connectivity checking step when performing extraction, to speed up extraction without any significant loss of accuracy. *Optional. Default: 0.*

Syntax:

```
SKIP_CONNECTIVITY_CHECK [ 0 | 1 ]
```

SKIP_LAYERS_FILE

Specifies a file containing a list of layer names, one per line, as follows:

```
# comment
<layer1>
<layer2>
...
```

The specified layers have minimum (effectively zero) resistance extracted for them, whether they are metal, via or pseudo-via layers. The typical use of this keyword is to run an extracted model that gives gridcheck results that only includes resistance for particular layers, generally with the layers below metal1 removed. This is useful for examining the effect of metal1 and above wiring separately. *Optional. Default: none.*

Syntax:

```
SKIP_LAYERS_FILE <filename>
```

SPLIT_SPARSE_CONTACT_VIA_ARRAY

In the GDS2DB flow, operates similarly to SPLIT_SPARSE_VIA_ARRAY, but only splits contact via arrays of the specified length and not regular routing via arrays. *Optional. Default: none.*

Syntax:

```
SPLIT_SPARSE_CONTACT_VIA_ARRAY <max_length_microns>
```

SPLIT_SPARSE_VIA_ARRAY

Improves via compression (grouping) in defined via arrays by setting a maximum length for arrays, without losing accuracy in resistance extraction. If this keyword is specified, via arrays longer than the specified maximum length L are split into individual arrays of length L or less for circuit modeling purposes. This can be used to achieve more accurate voltage drop numbers. For asymmetric via arrays (via cuts that have different sizes), extraction splits asymmetric via arrays into individual vias to improve accuracy. Note that this keyword is overridden by SPLIT_VIA_ARRAY if both of them are set. *DMP compatible. Optional. Default: none.*

Syntax:

```
SPLIT_SPARSE_VIA_ARRAY <max_length_microns>
```

SPLIT_VIA_ARRAY

Allows per layer specifications of the via split threshold (in um). Note that keyword SPLIT_SPARSE_VIA_ARRAY is overridden by SPLIT_VIA_ARRAY if both of them are set. *DMP compatible. Optional. Default: none.*

Syntax:

```
SPLIT_VIA_ARRAY {
    <layer1> <split_threshold1>
    <layer2> <split_threshold2>
    ...
}
```

STATIC_CONNECT_INST_FLOAT_GND

When set, excludes instances with floating ground pins from static analysis. Such instances are removed from output reports and in IR maps they are shown in red. By default instances with floating grounds are a part of static simulation. *Optional. Default: 1.*

Syntax:

```
STATIC_CONNECT_INST_FLOAT_GND [0|1]
```

STATIC_IR_HIDE_DISCON_INST

When set, hides disconnected instances in static IR instance maps. *Optional. Default:* 0.

Syntax:

```
STATIC_IR_HIDE_DISCON_INST [ 0 | 1 ]
```

USE_INTERP_ETCH_FOR_R

When set to 1, correctly interpolates.tech file curve flatness checks of the "ETCH_VS_WIDTH_AND_SPACING" table. *Optional. Default: 0.*

Syntax:

```
USE_INTERP_ETCH_FOR_R [ 0 | 1 ]
```

USE_MVM_PIN_MODEL

For designs with standard cells having both MET1 and MET2 parallel rails with vias between, faster runtimes and better memory use can be achieved by turning on this keyword, which also improves performance for standard cells having pin geometries with "fish-bone" structures. More than 50% reduction in node count can be achieved in most of these cases. *DMP compatible. Optional. Default: none.*

Syntax:

```
USE_MVM_PIN_MODEL [ 0 | 1 ]
```

USE_ZERO_WIRE_R_COMP

Controls zero-Ohm R compensation on via wires. By default Totem uses a resistance of 1.01*MIN_RESISTANCE for via wire R compensation, which prevents nodes associated with the zero-Ohm assumption from being ignored. You can revert to previous zero-Ohm assumption by setting this keyword to 1. *Optional. Default: 0.*

Syntax:

```
USE_ZERO_WIRE_R_COMP [ 0 | 1 ]
```

WIRE_R_COMPENSATION

By default extraction always includes the resistance of wire segments directly below and above via cuts (usually referred as metal-overhang). Set this keyword if you want to exclude all resistance of this type from R extraction, and use only the vertical via resistance.

Note: Compensation for pseudo-vias is not turned on by this keyword, and is only on if the appropriate tech file VIA section has WIRE_R_COMPENSATION set. Also, if this keyword is set only in the tech file, it must be specified in every VIA section in the tech file.

Optional. Default: 0 (include).

Syntax:

```
WIRE_R_COMPENSATION [ 0 | 1 ]
```

Characterization Keywords

APL_FILES

APL_FILES can be used instead of the '`import apl`' command to specify multiple APL input files and directories at a time (a single '`import apl`' command cannot support multiple files or any directory imports). The files are listed in a block in which the first column specifies a filename or directory and the second column specifies what type of file it is, such as current, device capacitance, pwcap, or avm. At the end of the '`setup design`' stage, all files/directories in the specified list are processed and imported. If there is a TCL command '`import apl <current-file>`' or '`import apl -c <cap-file>`' in the Totem command file, then that command is processed and the files/directories in the APL_FILES keyword are

ignored. If there is a keyword 'PIECEWISE_CAP_FILES <pwc-file>' in the GSR file, then this piecewise file specification supersedes those listed in the APL_FILES list.

Totem supports transistor-pin based voltage derating when the spcurrent file has currents at multiple voltages, and also Totem automatically performs electrical and physical model consistency checking. So if the number of pins from APL_FILES **.spcurrent* and **.cdev* do not match the number of pins in the physical database created by GDS2DB/GDSMMX, Totem aborts to avoid incorrect input data that would generate incorrect results. *DMP compatible. Optional. Default: None.*

Syntax:

```
APL_FILES {
   <input_file> [ current | cdev | pwcap | avm |
       current_avm | cap_avm]
  ...
  ? <input_dir> [ current | cdev | pwcap ]?
  ...
}
```

Example:

```
APL_FILES {
    aaa.spcurrent  current
    bbb.cdev cdev
    ccc.pwcdev pwcap
    avm.config avm
    vmemory.current current_avm
    vmemory.cdev cap_avm
    aaa_current_dir  current
    bbb_cdev_dir cdev
    ccc_pwcap_dir pwcap
}
```

DYNAMIC_APL_RESAMPLE

When set to 1, Totem resamples the APL current during simulation instead of using raw APL current. Please note, this will increase the memory usage. *Optional. Default:0*

Syntax:

```
DYNAMIC_APL_RESAMPLE [ 0 | 1 ]
```

FAST_IMPORT_APL_MODE

When set, APL import is optimized for parallel interpolation data preparation and peak memory usage to reduce APL import time. Also, the parallel intermediate directory after interpolation is removed. *Optional. Default: 0.*

Syntax:

```
FAST_IMPORT_APL_MODE [ 0 | 1 ]
```

IGNORE_APL_CHECK

When importing APL data into Totem, 'aplchk' checks the quality of the data, which determines the quality of voltage drop results. When this keyword is turned on, this data checking is disabled. *DMP compatible. Optional. Default: 0 (do not ignore).*

Syntax:

```
IGNORE_APL_CHECK [ 0 | 1 ]
```

IGNORE_APL_CHECK_SWITCH

When set, APL ignores incorrect switch cell counts or multiple headers in switch models, since in some cases files are manually modified to merge multiple switch models into one, resulting in errors. With this keyword set, when APL switch cell counts do not match the number of cells in the switch cell file, **Totem** continues without an error. *DMP compatible. Optional. Default: 0 (do not ignore).*

Syntax:

```
IGNORE_APL_CHECK_SWITCH [ 0 | 1 ]
```

IGNORE_APL_PROCESS_CORNER

When turned on, supports reading of APL data for different process corners that have been merged in the same run. To merge data from different process corners in APL, use the 'aplmerge' command:

```
aplmerge -ignore_corner <filename1> <filename2> -o abc
```

DMP compatible. Optional. Default: 0 (off).

Syntax

```
IGNORE_APL_PROCESS_CORNER [0|1]
```

Note: importing different corner APL data is not recommended in most cases, since it may create incorrect Totem results.

LIB2AVM

If APL characterizations are not available for some memory cells, for the dynamic flow Totem can automatically collect the necessary data from the design, LIB files and the GSR, and create an AVM (Apache Virtual Model) configuration file in *adsRpt/avm.conf*, which describes parameters such as read/write/standby mode, power consumption, access time, setup time, load information, and current waveform type, for each memory type. Totem internally uses this config file to generate and use rule-based current profiles, leakage current and decoupling capacitance for memory blocks in dynamic analysis. The AVM utility then creates current profiles for these memory cells that have no APL data.

By default (1), a triangular-shaped profile is provided (composed of multiple triangles for complicated profiles). When set to 2, a trapezoidal-shaped current profile is provided. Any available APL memory characterization data overrides internally-generated AVM data. A value of 0 turns off automated AVM config file generation. Memories are identified based on the following attributes in `*.lib` for AVM generation: '`memory ( )`', or '`timing( ){mode...}`'. *DMP compatible. Optional. Default: 1 (triangular waveform).*

Syntax

```
LIB2AVM [ 0/OFF | 1/TRIANGULAR | 2/TRAPEZOIDAL ]
```

Example:

```
LIB2AVM 0
```

Clock, Toggle Rate and Power Scaling Keywords

Note that you can specify separate toggle rates for clock and non-clock portions of the design using several GSR keywords: TOGGLE_RATE, INSTANCE_TOGGLE_RATE, INSTANCE_TOGGLE_RATE_FILES, BLOCK_TOGGLE_RATE, and

BLOCK_TOGGLE_RATE_FILES. In each case the clock net toggle rate is optional and has a default value of 2 if it is not specified.

BLOCK_POWER_FOR_SCALING

Specifies user-known block and instance power consumption values per block or instance, and for multiple Vdd/Vss designs, per pin, which Totem uses to scale individual component toggle rates to achieve more accurate analysis. The power consumption data is used to scale toggle rates for instances in individual blocks and instances to match the total power consumption for the chip. The top-level block is specified for a flat design. The full hierarchical path must be supplied for all blocks and instances. Note that for MVDD specifications by pin, the VSS pin *current* is specified, not the power.

For multiple Vdd/Vss designs, power consumption is assigned and scaled by pin. If Vdd pin names are *not* specified, all VDD pin power is proportionally scaled with a common toggle rate. Note that this is supported only for leaf level cells for MVdd; hierarchical blocks are currently not supported.

You can also control power assignment on a “celltype” basis, using the ‘CELLS’ option. User can also set/append values of this keyword using ‘gsr set’/’gsr set’/’gsr append’ TCL commands. These commands can be executed at any stage of RedHawk run, even before setup design stage. *DMP compatible. Optional. Default: none.*

Syntax (GSR):

```
BLOCK_POWER_FOR_SCALING {
    CELLTYPE <cell_name> [<pwr_W> ?<Vdd_pin>? |
                <cell_current_A> ?<Vss_pin> ? ]
    ...
    [FULLCHIP | BLOCK | <top_block_name>] <full_leaf_inst_path>
        <pwr_W>
    [FULLCHIP | BLOCK |<top_block_name>] <full_block_path>
        <pwr_W> ? <domain> ? ? <pin> ?
    ...
    [FULLCHIP | BLOCK | <top_block_name>]
        full_mVdd_inst_path> [<pwr_W> ?<VDD_pin>? |
        <current_A> ?<Vss_pin>?]
    ...
    CELLS [ comb | ff_latch | mem | clockinst | io ] <pwr_W>
    ...
}
```

Example (GSR):

```
BLOCK_POWER_FOR_SCALING
{
  CELLTYPE cellA1 0.01
  CELLTYPE cell3 0.001
  FULLCHIP designABC 1.5
  BLOCK block1 0.5
  FULLCHIP block1/instance2 0.00322
  BLOCK block1/sub_blockA 0.1
  FULLCHIP block2/AN210J 0.001 VDD
  FULLCHIP ia32/nand342 0.005 VDD5
  CELLS comb  0.046
  CELLS ff_latch 0.0073
  CELLS mem 0.011
```

```
    CELLS clockinst 0.004
  }
```

where

CELLTYPE <cell_name> : specifies individual cell name

<pwr_W>: power consumption for associated block, instance, or pin, in Watts

FULLCHIP | BLOCK | <top_block_name> : specifies power for the chip or block, or per domain or pin in the block. The sub-keyword or <top block name> must be defined in DEF after running Totem through power calculation.

<full_block_path> : specifies full hierarchical path for the block

<full_leaf_inst_path> : specifies full hierarchical path for the leaf instance

<full_mVdd_inst_path> <power_Watts>: full hierarchical path to multiple Vdd/Vss design instance and pin-based power consumption listing.

<VDD_pin> : optional - VDD pin name associated with specified power. If '< VDD pin name>' is *not* specified, all VDD pin power is proportionally scaled with a common toggle rate.

<VSS_pin> : optional - VSS pin name associated with specified current. If '< VSS pin name>' is *not* specified, all VSS pin current is proportionally scaled with a common toggle rate.

CELLS [comb | ff_latch | mem | clockinst | io]: assigns power to all cells of a particular celltype

Example (in a Block_Power_For_Scaling file):

```
CELLTYPE cellA1 0.01
CELLTYPE cell3 0.001
FULLCHIP designABC 1.5
BLOCK block1 0.5
FULLCHIP block1/instance2 0.00322
BLOCK block1/sub_blockA 0.1
FULLCHIP block2/AN210J 0.001 VDD
FULLCHIP block2/AN210J 0.003 VDDC
FULLCHIP ia32/nand231 0.001 VDDQ
FULLCHIP ia32/nand342 0.005 VDD5
```

where

"designABC" is the name of the top level block. Since Totem flattens the design, the designABC is the full-chip.

"cellA1 0.01" and "cell3 0.001", power of .01W is assigned any instance that uses cellA1as master cell and .001W is assigned to any instance that uses cell3.

The total power defined by "FULLCHIP block1 0.5", includes power defined for <full_block_instance_path>/block1.

"block1/instance2 0.00322", power is assigned to instance2, which is an instance in block1. For "block1/sub_blockA 0.1", power is assigned to sub_blockA, which can be a clustered block inside block1.

The last four items are pin-based power specifications for mVdd.

BLOCK_POWER_FOR_SCALING_FILES

Specifies absolute or relative path from Totem run directory that contains the power specification, as in BLOCK_POWER_FOR_SCALING keyword. *DMP compatible. Optional; default: None*

Syntax:

```
BLOCK_POWER_FOR_SCALING_FILES
```

```
{
<Block_Power_FilePathName>
}
```

Example:

```
BLOCK_POWER_FOR_SCALING_FILES
{
designABC/block1-a_scaling
}
```

BLOCK_TOGGLE_FILES

Note: Recommend using keywords VCD_FILE and BLOCK_VCD_FILES instead of this keyword.

Specifies the block toggle file for each block. The specified toggle file defines the average toggle rate for each net that is used to calculate power data.

The <toggle_file_name> is generated by running the *vcdtrans* utility on the respective VCD (Vector Change Dump) files and can be specified for each block. For more details on using *vcdtrans*, please refer to section "fsdbtrans", page E-995. If VCD is available, use VCD_FILE (defined below) for power calculation. *DMP compatible. Optional. Default: None.*

Syntax:

```
BLOCK_TOGGLE_FILES
{
<block_name_N> <toggle_file_name_N>
    ...
}
```

where

<block_name_N> : name of block

<toggle_file_name_N> : specifies toggle file for <block_name_N>

Example:

```
BLOCK_TOGGLE_FILES
{
    top_block top_block.toggle
}
```

BLOCK_TOGGLE_RATE

Defines the default toggle rate of the nets in a user-specified block or instance that are not otherwise specified and optionally also any associated clock nets. The blocks or instances should be defined in the DEF files. When BLOCK_TOGGLE_RATE is defined for a specific block, it supersedes the TOGGLE_RATE defined for the whole chip, but not a specified INSTANCE_TOGGLE_RATE for the block. The <clock_network_TR> entry for a block applies to clock pins and clock buffer outputs. For an instance, whether a clock network instance or not, the first TR entry applies to the network toggle rate, and the second, if present, applies to the clock buffer and clock pin toggle rate. The <block_inst_name>/<inst_name> specifications can take a wildcard '*'. For example, ABC* matches all block instance/instance names starting with ABC. *DMP compatible. Optional. Default: clock rate: 2.0.*

Syntax:

```
BLOCK_TOGGLE_RATE
{
```

```
    <block_inst_name/inst_name> <output_TR> ?<clock_network_TR>?
    <non-clock_inst_name> <output_TR> ?<clock_network_TR>?
    <clock_network_inst_name> <output_TR> ?<clock_network_TR>?
    ...
  }
```

Example:

```
  BLOCK_TOGGLE_RATE
  {
  block1 0.5 1.0
  block2 0.4
  block3* 0.2
  }
```

BLOCK_TOGGLE_RATE_FILES

Specifies the file containing toggle rates for blocks/instances defined in the *.def* file. The toggle rate settings in this file supersede the TOGGLE_RATE defined for the whole chip, but not a specified INSTANCE_TOGGLE_RATE for a block. The format of the file is as described for the BLOCK_TOGGLE_RATE keyword syntax. *DMP compatible. Optional. Default: None.*

Syntax:

```
BLOCK_TOGGLE_RATE_FILES <toggle_rate_file>
```

CELL_TOGGLE_RATE

This keyword allows specifying individual toggle rates by leaf cell name. *Optional. Default: none.*

Syntax:

```
CELL_TOGGLE_RATE {
  <leaf_cell_name> <toggle_rate> ?<clock_toggle_rate>?
...
}
```

CELL_TOGGLE_RATE_FILE

Specifies a file or files containing specific toggle rates for cells defined in the *.def* file. The toggle rate settings in this file supersede the TOGGLE_RATE defined for the whole chip. *Required. Default: none.*

Syntax:

```
CELL_TOGGLE_RATE_FILE <toggle_rate_file>
```

or

```
CELL_TOGGLE_RATE_FILE {
    <toggle_rate_file1>
    <toggle_rate_file2>
    ...
}
```

FREQUENCY

Defines the dominant operating frequency on the chip, or the lowest frequency that includes a majority of the power consumption on the chip. More specifically, for a design in which there are several frequencies that consume significant power, the

frequency to be specified is the frequency at which less than 10% of the chip power is consumed at *lower* frequencies.

For example, assume that there are three significant frequencies on the chip, and they consume the following power: 100 MHz (5mw), 200 MHz (20mw), and 400 Mz (70mw). The FREQUENCY value to be specified in this case would be 200 MHz, even though a majority of the power is consumed at 400 MHz. *Required. Default: none.*

Syntax:

```
FREQUENCY <value in Hertz>
```

Example:

```
FREQUENCY 160e6 (or 160M, 160me)
```

Note that ‘me’ represents mega.

GSC_OVERRIDE_IPF

When set to 1, overrides settings of BLOCK_POWER_FOR_SCALING and BLOCK_POWER_FOR_SCALING_FILE, INSTANCE_TOGGLE_RATE, BLOCK_TOGGLE_RATE, and TOGGLE_RATE, and the toggle rate value set in the GSC is honored. *DMP compatible. Optional. Default: 0.*

Syntax:

```
GSC_OVERRIDE_IPF [ 0 | 1 ]
```

INACTIVE_NETS

Specifies the running frequency of clock nets that are not specified by CLOCK_ROOTS, or for special nets such as Reset and Scan. The frequency of these nets is fixed throughout the analysis of power calculation and IR drop. Note that STA_FILES specifications override both INACTIVE_NETS and CLOCK_ROOTS specifications. *DMP compatible. Optional. Default: None*

Syntax:

```
INACTIVE_NETS
{
    <net_name> <frequency>
        ...
}
```

Example:

```
INACTIVE_NETS
{
    test 10e6
    reset 0
    ...
}
```

INSTANCE_TOGGLE_RATE

Specifies average toggle rates for instances in the design. If there are a lot of instances in the chip, using this keyword is recommended, rather than using BLOCK_TOGGLE_RATE or BLOCK_TOGGLE_RATE_FILE keywords. Wildcard characters are supported. Whether a clock network instance or not, the first TR entry applies to the network toggle rate, and the second, if present, applies to the clock buffer or clock pin toggle rate. If only one TR value is specified, it is used for the output/signal toggle rates associated with the instance*. DMP compatible. Optional. Default: clock: 2.0.*

Syntax:

```
INSTANCE_TOGGLE_RATE {
    <non-clock_inst_name> <output_TR> ?<clock_network_TR>?
    <clock_net_inst_name> <output_TR> ?<clock_network_TR>?
    ...
}
```

Example:

```
INSTANCE_TOGGLE_RATE
{
 instance1 0.5 1.5
 instance2 0.3
}
```

INSTANCE_TOGGLE_RATE_FILES

Specifies the instance toggle rate file, which provides toggle rates for instances on the chip. Specifies the instance toggle rate file, which provides toggle rates for instances on the chip. The format of the contents of the file is:

```
<non-clock_inst_name> <output_TR> ?<clock_network_TR>?
```

A message is displayed and recorded in the log file showing the number of instances listed in the INSTANCE_TOGGLE_RATE_FILES successfully read in and also the number of listed instances not found in the design or floating. Wildcard characters are supported. *DMP compatible. Optional. Default: None.*

Syntax:

```
INSTANCE_TOGGLE_RATE_FILES <instance_toggle_rate_file>
```

Example:

```
INSTANCE_TOGGLE_RATE_FILES ABCDE_instance_TR_file
```

PARA_CALC_POWER

When set (default), performs power calculation and extraction in parallel threads, which improves performance. *DMP compatible. Optional. Default: 1.*

Syntax

```
PARA_CALC_POWER [ 0 | 1 ]
```

POWER_DISABLE_SWITCH

When set, the toggle rate for all switch cells is set to zero. *Optional. Default: 0.*

Syntax:

```
POWER_DISABLE_SWITCH [ 0 | 1 ]
```

POWER_DRIVER_TOGGLE_RATE

Propagates the instance toggle rate to driver nets to insure consistency between the instance toggle rate and the driver net toggle rate. *Optional. Default: 1 (On).*

Syntax:

```
POWER_DRIVER_TOGGLE_RATE [ 0 | 1 ]
```

POWER_MISSING_IPF_POWER

For instances with missing INSTANCE_POWER_FILES values, this keyword by default (0ff) sets missing power values to 0. If the keyword is set to 1, power

calculation uses the global toggle rate to compute the power values for instances without IPF values. *DMP compatible. Optional. Default: 0.*

Syntax:

```
POWER_MISSING_IPF_POWER [0 | 1 ]
```

POWER_STATE_DEPENDENT_LEAKAGE

When turned On, calculates state-dependent leakage power in vectorless and VCD analyses. *DMP compatible. Optional. Default: 0.*

Syntax:

```
POWER_STATE_DEPENDENT_LEAKAGE [0|1]
```

POWER_TRANSIENT_ANALYSIS

Power Transient Analysis, also known as “Variable Power”, provides more flexibility to simulate frame-by-frame power/current transient behavior. The POWER_TRANSIENT_ANALYSIS keyword allows you to specify a set of configuration files that each defines parameters for analysis of a separate frame. The following power control GSR keywords are supported by Power Transient Analysis:

GSC_FILES

INSTANCE_POWER_FILES

GSC_OVERRIDE_IPF

BLOCK_POWER_FOR_SCALING

STATE_PROPAGATION

TOGGLE_RATE_RATIO_COMB_FF

INSTANCE_TOGGLE_RATE

BLOCK_TOGGLE_RATE TOGGLE_RATE

Note that Power Transient Analysis works only in vectorless mode. *Optional. Default: None.*

Syntax:

```
POWER_TRANSIENT_ANALYSIS {
   <duration_frame1_sec> <config_file1>
   <duration_frame2_sec> <config_file2>
    ...
}
```

POWER_VCD_IGNORE_ERROR

When set to 1, if encountering an incorrect block name, Totem skips the VCD file and continues with power calculation. By default Totem errors out when a block name is not specified correctly. *Optional. Default: 0.*

Syntax:

```
POWER_VCD_IGNORE_ERROR [ 0 | 1 ]
```

POWER_VCD_NUM_PROCESS

Specifies the number of processors to use for VCD analysis. The multi-threading function is On by default, so Totem automatically decides how many VCDs are read in parallel. *DMP compatible*. *Optional. Default: -1 .*

Syntax:

```
POWER_VCD_NUM_PROCESS <num_proc>
```

POWER_VCD_REUSE_EVENT

For multiple VCD file flows, if there are multiple blocks using the same VCD/FSDB file with the same VCD_FILE options FRONT_PATH and START_TIME, this keyword when set to 1, applies first block events to other blocks using the same VCD file. That RedHawk reads the VCD/FSDB only once, checks if the global nets are reusable and reuses the VCD for other blocks, to speed up VCD analysis. This feature can be used in 'averaged' mode also. If this keyword is set to 2, RedHawk reuses VCD without performing check for global nets reusability. *DMP compatible. Optional. Default: 1 (On).*

Syntax:

```
POWER_VCD_REUSE_EVENT [ 0 | 1 | 2 ]
```

where

0: turn off

1: default value, will check if global nets are reusable and keeps accuracy

2: without global nets, if many VCD's have same front path and other options, then it will speed up the run.

PS_IN_FLOW_APL

When set, the APL import flow is merged with power calculation to improve run time with large amounts of APL data. In particular, it correctly handles state- dependent pratios in Raw cell_view models. *DMP compatible. Optional. Default: 1(On).*

Syntax

```
PS_IN_FLOW_APL [0|1]
```

SCALE_CLOCK_POWER

Includes scaling of toggle rates for clock network components in the BLOCK_POWER_FOR_SCALING calculation. If set to zero, clock net toggle rates are not scaled. *DMP compatible. Optional. Default: 1.*

Syntax:

```
SCALE_CLOCK_POWER [0|1]
```

STA_VCD_FREQ_RATIO

Matches the VCD frequency to the STA frequency (the specified <ratio> = STA_freq/VCD_freq). When Totem reads VCD data to select cycles, it uses the clock frequency defined in the STA file to cut cycle frames from VCD. If this keyword is used, VCD data is processed according to the STA frequency and specified ratio. *Optional. Default: None*

Syntax:

```
STA_VCD_FREQ_RATIO <ratio>
```

Example:

```
STA_VCD_FREQ_RATIO 2.0
```

In this example, with a STA clock period of 1 ns, Totem uses a 2 ns cycle for VCD to determine frame size and toggle rates.

STATE_PROPAGATION

STATE_PROPAGATION controls the operation of the state propagation analysis engine for vectorless analysis. State propagation is designed to determine the probability that instances will switch during the simulation window of interest, based on a long simulation time. If this keyword is *not* defined, state propagation is *off* and instance switching is determined by default toggle rates in the design. If STATE_PROPAGATION is defined, when the Power Calculation command is invoked, state propagation is performed first and the toggle rates obtained from state propagation are then used as the basis for calculating instance power consumption in the design. *DMP compatible.* R*equired for state propagation analysis.*

Syntax

```
 STATE_PROPAGATION {
PROPAGATION_MODE PROBABILITY
? GATED_ON_PERCENTAGE  <On_fraction> ?
? CONSTRAINT_FILE  <constraint_filename> ?
 ? GATED_CLOCK_COVERAGE [ 0| 1]?
 ? GATED_CONTROL_FILE  <control_filename> ?
?  SEQ_CELL_MODE [1|0] ?
}
```

where

PROPAGATION_MODE PROBABILITY - automatically determines switching probability for instances.

GATED_ON_PERCENTAGE <On_fraction>: specifies the fraction, between 0 and 1.0, of clock gates that are on, as follows:

All gating cells are On if GATED_ON_PERCENTAGE is 1.

All gating cells are Off if GATED_ON_PERCENTAGE is 0.

Increasing the value of GOP turns on additional clock gates, and does not turn off any that are already turned on. So, any clock gate that is on with a GOP of, for example, 0.6, is also on when GOP is set to 0.8.

CONSTRAINT_FILE : specifies a file describing the toggle rates of key instances or nets for accurate toggle rate propagation.

GATED_CLOCK_COVERAGE : when set to 1, provides better design coverage when using GATED_ON_PERCENTAGE, and allows choosing different clock gate turn-on scenarios for each frame so that overall analysis coverage is increased. Note that POWER_TRANSIENT_ANALYSIS must also be turned on.

GATED_CONTROL_FILE : specifies a file defining the On/Off status of individual gated clock control cells. The format of the gated clock control file data is as follows for both static and dynamic analyses:

```
<gated_clock_instance> [ On | Off ]
...
```

SEQ_CELL_MODE : enables logic propagation for flip-flop and latch cells (default 1)

Example

```
State_propagation {
PROPAGATION_MODE prob
CONSTRAINT_FILE StatePropConst_AB
GATED_CONTROL_FILE GCCont_XY
}
```

TOGGLE_RATE

Defines the default signal toggle rate of the nets on the chip that are not otherwise specified. The rate is the product of the probability that the nets will toggle times the actual clock toggle rate. Toggle rate is defined as the sum of the state changes from 0->1 and 1->0 within a clock cycle with respect to the net's clock domain. For example, a clock net generally has a toggle rate of 2.0 with respect to its clock domain, since the net switches once from 0->1 and once 1->0 within a clock cycle. Note that if there is no power consumption table in *.lib* the toggle rate is taken from TOGGLE_RATE, and charge is scaled to meet power. *DMP compatible. Optional. Default: non-clock: 0.3; clock: 2.0.*

Syntax:

```
TOGGLE_RATE <non_clock_TR> ?<clock_network_TR>?
```

where

<non_clock_TR> : defines the probability for nets switching during a clock cycle

<clock_network_TR>: applies to both clock pins and to clock buffer outputs-- the actual network clock toggle rate (Default: 2.0)

Example:

```
TOGGLE_RATE 0.2 1.0
```

Timing Keywords

It is recommended that you specify the CLOCK_ROOTS keyword if STA_FILES is not specified, since Totem can automatically trace all the clock nets and clock domains for each instance that is traceable from the clock root. Note that clock root tracing will *not* be performed if STA_FILES is specified.

BOUND_SLEW_TO_MAX_TRANSITION

Controls whether to apply SLEW_MIN/MAX_TRANSITION limitations on instance pin transition time values. *DMP compatible. Optional. Default: 1 (On).*

Syntax:

```
BOUND_SLEW_TO_MAX_TRANSITION [ 0 | 1 ]
```

CLOCK_ROOTS

Traces the nets from a specified clock root and finds the respective clock domain for all the nets and instances. Specifies the names of the clock roots (either net name or pin name) and the frequency. No wildcard matching is supported for this keyword. If STA_FILES is specified, CLOCK_ROOTS will not be used. *DMP compatible. Optional. Default: None.*

Syntax:

```
CLOCK_ROOTS
{
    <clock_root_name> <freq in Hz>
        ...
}
```

Example:

```
CLOCK_ROOTS
{
    clock_root_1 160e6
    clock_root_2 160e6
```

```
    clock3:CLK 140e6
}
```

INPUT_TRANSITION

Defines input transition time for the input pins of all instances. *Optional. Default: 10% of the value of the inverse of the frequency defined by the "FREQ" keyword.*

Syntax:

```
INPUT_TRANSITION <value in seconds>
```

Example:

```
INPUT_TRANSITION 0.2e-9 (or 0.2ns)
```

SLEW_MIN_TRANSITION | SLEW_MAX_TRANSITION

For assigning a range for transition times to instance pins in the STA file and USER_STA_FILE (prior to slew normalization), you can set the minimum and maximum transition times using these keywords, in seconds. Note that these keywords do not control instance rise time values. *DMP compatible. Optional. Defaults: minimum - 5e-12, maximum - 1e-9.*

Syntax:

```
SLEW_MIN_TRANSITION <min_rise_sec>
SLEW_MAX_TRANSITION <max_rise_sec>
```

Wire Load Specification Keywords

You should specify at least one of the following keywords:

- INTERCONNECT_GATE_CAP_RATIO
- CELL_RC_FILE, or
- STEINER_TREE_CAP

The type of power calculation performed depends on the keyword chosen. If all three keywords are specified, then the CELL_RC_FILE value is used for the nets in the cells/blocks that have the SPF file or DSPF file defined. For the rest of the blocks, if defined, the value of STEINER_TREE_CAP is used. If no STEINER_TREE_CAP or CELL_RC_FILE is specified, then INTERCONNECT_GATE_CAP_RATIO is used. If none of these keywords are specified, then the default value of 1 for the INTERCONNECT_GATE_CAP_RATIO is used.

INTERCONNECT_GATE_CAP_RATIO

Defines the ratio of the total interconnect capacitance of the net relative to the total gate capacitance of the input pin fanouts. If none of the following keywords, INTERCONNECT_GATE_CAP_RATIO, CELL_RC_FILE, or STEINER_TREE_CAP have specified values, power calculation uses the default value of INTERCONNECT_GATE_CAP_RATIO. *DMP compatible. Optional. Default: 1.*

Syntax:

```
INTERCONNECT_GATE_CAP_RATIO <value>
```

Example:

```
INTERCONNECT_GATE_CAP_RATIO 1.5
```

CELL_RC_FILES

Defines the SPEF/DSPF interconnect parasitics file for each cell in a flat or hierarchical design. When the EXTRACT_RC option is set to 1 (default), Totem builds a c1-r-c2 equivalent circuit for each instance output RC network specified in the SPEF file. Only the instance output driving a signal net (P/G nets are ignored) gets an equivalent C1/R/C2 when set. When set to 0, Totem uses the total output C for each instance output instead. For static IR-drop analysis, set EXTRACT_RC to 0. The CONDITION keyword allows selection of one of the capacitance value types from a three-value SPEF file. The syntax allows either listing the cellnames and paths, or listing SPEF files and then including the file the same as LEF/DEF/LIB files. The '<>.spfs' filename specifies the file containing the list of SPEF/DSPF files. This file must be the last item in the list and its name must end with *.spfs* , to indicate that the alternate format is being used. The contents of the *<>.spfs* file simply lists the cells and pathnames. *DMP compatible. Optional. Default: EXTRACT_RC: 1; CONDITION: typical.*

Syntax:

```
CELL_RC_FILES
{
    EXTRACT_RC [ 0 | 1 ]
    CONDITION [ best | typical | worst ]
    [ <cell_name> <path_to_dspf-spef_file>
        ...
    | <user_data>/abc.spfs ]
}
```

Example:

```
CELL_RC_FILES
{
    CONDITION worst
    cell1 SPEF_files/cell1.spef
    cell2 SPEF_files/cell2.spef
}
```

STEINER_TREE_CAP

When specified, Steiner Tree routing is performed and the resulting length is multiplied by the capacitance density value specified, in pF per um. *DMP compatible. Optional. Default: none.*

Syntax:

```
STEINER_TREE_CAP <cap in pF per um>
```

Example:

```
STEINER_TREE_CAP 2.0e-4
```

PRIMARY_OUTPUT_LOAD_CAPS

Specifies the capacitance of the output driver nets, usually with a fanout of 1. The option <output_driver_net_name> supports “*” wild card matching. If DEFAULT_LOAD is specified, the default Cload specified is applied to all of the primary I/O output nets, even if the connected standard cell is not defined as an I/O pad cell in the lib. *DMP compatible. Optional. Default: none.*

Syntax:

```
PRIMARY_OUTPUT_LOAD_CAPS
{
```

```
    IGNORE_FANOUT_CAP [ 1|0 ]<powerNet_name>
    DEFAULT_LOAD <output cap in Farads> <Vdd_name>
      <output_driver_net_name> <output_cap_Farad> <Vdd_name>
      ...
  }
```

where

IGNORE_FANOUT_CAP [0 | 1] <powerNet_name> : value 1 ignores fanout cap in the specified power net.

<Vdd name> : Vdd name that should be defined in the VDD_NETS keyword.

Example:

```
PRIMARY_OUTPUT_LOAD_CAPS
{
  IGNORE_FANOUT_CAP 1 VDD
  DEFAULT_LOAD 20p VDD
 PAD_g* 50p IO_VDD
}
```

Note: You can use DEFAULT_LOAD only for assigning loads to I/O cell nets. For all other nets in the design, specify the net name using the following syntax, without the DEFAULT_LOAD option (wild cards allowed):

```
PRIMARY_OUTPUT_LOAD_CAPS {
IGNORE_FANOUT_CAP 1 VDD
Clk_net123 15pF VDD
Clk* 15p VDD
}
```

The capacitance can be denoted by scientific notation (e-12), or as “p” for pico, “f” for femto, and so on. Be aware that for larger designs, assigning loads in this way to *all* nets might not be run time efficient. The priority for load cap is:

1.Pattern-matched in PRIMARY_OUTPUT_LOAD_CAPS(exact match > wild-card-match)

2.Pattern-matched in EM_NET_INFO (exact-match > wild-card-match)

3.DEFAULT_LOAD in PRIMARY_OUTPUT_LOAD_CAPS

PRIMARY_OUTPUT_LOAD_CAPS_FILE

Specifies primary output load capacitance within a file.

Syntax:

```
PRIMARY_OUTPUT_LOAD_CAPS_FILE ./<file_pointer>
```

USE_FAST_DECAP_ALG

When set (default), invokes “Special Decap Handling” (SDH), which provides a reduction in the number of nodes/resistors created for decap instance pins, and improves performance in filler and decap-dominated designs. If you have specified decaps in DECAP_CELLS, the auto-detected decap cells are appended to the user-specified decap cells. *Optional. Default: 1.*

Syntax:

```
USE_FAST_DECAP_ALG [ 0 | 1 ]
```

USE_LIB_MAX_CAP

In early design stages, when clock tree synthesis is not available, instances that drive very large fan-out, such as dummy clock buffers, may have unreasonably high calculated power and IR drop because of the high fanout. If USE_LIB_MAX_CAP is turned on, Totem uses the cell maximum capacitance value (max_cap) in the *.lib* file, instead of the calculated combination of interconnect and fanout capacitance, to achieve a more realistic early value for C_{load}. *DMP compatible. Optional. Default: 0 (off)*

Syntax:

```
USE_LIB_MAX_CAP [ 0 | 1 ]
```

Dynamic Simulation Keywords

ANALOG_ANALYSIS

When set, preserves all current profiles captured during APLMMX characterization, so the GSR frequency setting does not affect the current profiles during dynamic simulation. If ANALOG_ANALYSIS is set to 1 and the FREQ keyword is not set, Totem does not warn about a missing FREQ setting, and derives FREQ based on the GSC setting and the duration captured in the **.spcurrent* file during APLMMX characterization. DYNAMIC_SIMULATION_TIME covers the entire duration of the state defined in GSC.

Do not set the FREQ keyword when turning on ANALOG_ANALYSIS . If both ANALOG_ANALYSIS and FREQ are set, Totem uses the older method based on FREQ setting to chop current profiles. DYNAMIC_SIMULATION_TIME is set to 1/FREQ if it is not defined in the GSR.

If multi-cycle multi-states are listed in the GSC, the longest duration time of the selected states is chosen as the base for frequency setting, in order to preserve the full current profile of all selected states.

Also, when multi-cycle GSC is used with “ANALOG_ANALYSIS 1” for power EM, the dynamic simulation time will be the product of longest duration and number of states. (maximum_period_of _states * num_of_states).

Powerstream also generates a different report format when ANALOG_ANALYSIS is set; the cell_type power summary shows an entry “MMX_cell” and the clock power summary report is ignored. *Optional. Default: 1(on)*

Syntax:

```
ANALOG_ANALYSIS  [0 | 1]
```

BLOCK_PAR

Defines the ratio of peak cycle power to average power for all chip instances. Peak cycle power is defined as the power averaged over the worst cycle or the worst few cycles. The average power is defined as the power averaged over long normal operation cycles. *DMP compatible. Optional. Default: 1.0.*

Syntax:

```
BLOCK_PAR
{
PAR <ratio_peak_power_to_avg_power>
}
```

Example:

```
BLOCK_PAR
```

```
{
PAR 1.25
}
```

CONSISTENT_SCENARIO

In some cases different switching scenarios can be generated between SOC level runs for different conditions on the same or similar designs. When turned on (default), this keyword insures that the same or very similar designs have the same switching scenarios. *DMP compatible. Optional. Default: 1.*

Syntax:

```
CONSISTENT_SCENARIO [ 0 | 1 ]
```

DUMP_NODE_MIN_MAX_VOLTAGE

When set, simulation automatically caches out the minimum and maximum node voltage information during simulation and then the GUI post-processes the data during color map generation. This reduces runtime and memory usage significantly when generating IR, WVP2P maps for large designs, particularly those with negative supply nets. *Optional. Default: 0.*

Syntax:

```
DUMP_NODE_MIN_MAX_VOLTAGE [ 0 | 1 ]
```

DVD_GLITCH_FILTER

The keyword DVD_GLITCH_FILTER defines conditions for filtering (ignoring) some values of minimum DvD over the timing window (called “minTW”), based on voltage value and glitch width conditions, for all clock cells and GDS macros, as shown in the Figure C-3 diagram. Minimum DvD values can be filtered globally or using instance-specific conditions. An output report listing the included and filtered DvD values is written to the file */adsRpt/Dynamic/<designName>.minTW_filtered*. *Optional. Default: none.*

Note: This keyword should not be used in scan mode.

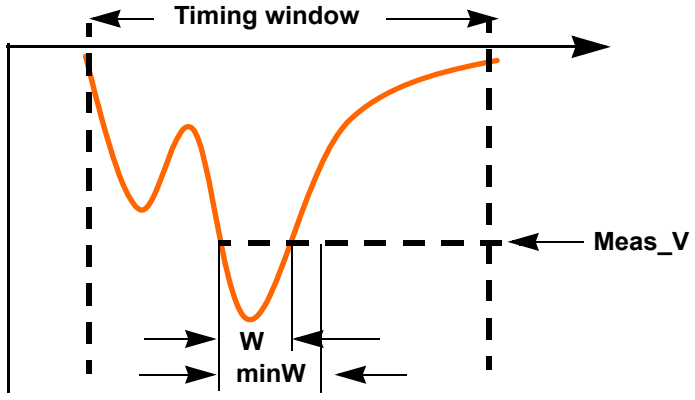


Figure C-3 Glitch width filtering with DvD_Glitch_Filter

Syntax:

```
DVD_GLITCH_FILTER {
    VOLTAGE_LEVEL <Meas_V>
    GLITCH_WIDTH <minW_sec>
```

```
        CELL_LIST_FILE <filename>
    }
```

where

VOLTAGE_LEVEL <Meas_V>: specifies the global voltage at which to measure glitch width, W

GLITCH_WIDTH <minW>: specifies the global minimum glitch width value (sec) to report minimum DvD values over the time window

CELL_LIST_FILE <filename> : specifies the file containing glitch width conditions for specific instances. The file has the format:

```
<cell_name> <meas_voltage> <local_min_glitch_width>
```

which defines specific cell names, and the associated measurement voltage and minimum glitch width to be ignored for each instance (wildcards are acceptable). The cell-specific values override the global measurement voltage and minimum glitch width settings.

Example:

```
DVD_GLITCH_FILTER {
    VOLTAGE_LEVEL 1.15
    GLITCH_WIDTH 150ps
    CELL_LIST_FILE DvD_glitch
}
```

DYNAMIC_BYPASS_SHORT

When set, Totem does not perform shorts checking during simulation. *Optional. Default: 0 (no bypass).*

Syntax:

```
DYNAMIC_BYPASS_SHORT [0 |1 ]
```

DYNAMIC_CELL_CROSS_CHECK

When set, cross checks are performed at an early stage for extra pins defined in PGARC, but not in APL. These extra pins are then ignored in simulation, and the run continues. By default simulation errors out with an “inconsistent data” message when pin definitions are different between PGARC and APL. *Default: 0.*

Syntax:

```
DYNAMIC_CELL_CROSS_CHECK [0 |1 ]
```

DYNAMIC_DETACHED_POSTPROCESS

When set, uses a separate thread to perform post-processing during simulation. *Optional. Default: 0.*

Syntax:

```
DYNAMIC_DETACHED_POSTPROCESS [ 0| 1 ]
```

DYNAMIC_FREQUENCY_AWARE

Invokes a frequency-aware vectorless flow that assists you in creating a simulation scenario in which the current demand from the circuit is modulated with respect to the frequency specified in the keyword. This scenario in turn can be used for DvD analysis or CPM creation. It is useful to create a voltage drop scenario with a resonance effect with the package and board netlists, to help understand the voltage drop issues when the die has this condition.

Syntax:

```
DYNAMIC_FREQUENCY_AWARE  <freq_Hz>
```

Example:

```
DYNAMIC_FREQUENCY_AWARE   300e6
```

In the example, the Ipwr is modulated at 300 MHz.

DYNAMIC_GROUP_WIRECAP

When set, for designs with large numbers of coupling caps, simulation groups neighboring coupling caps to reduce matrix complexity, and speeds up processing significantly. *Optional. Default: 0 (off).*

Syntax:

```
DYNAMIC_GROUP_WIRECAP [1 | 0 ]
```

DYNAMIC_LGCS_REPORT

When set, generates an LGCS report file if local ground current scaling for wire cap is specified using DYNAMIC_LOCAL_GROUND_CURRENT_SCALING 1 or 2. When DYNAMIC_LGCS_REPORT 1 is set, Totem dumps out an lgcs log file at *adsRpt/apache.inst.lgcs*. The file format is:

```
<inst_name> <scaling_factor>
```

Default: 0 (off).

Syntax:

```
DYNAMIC_LGCS_REPORT [1 | 0 ]
```

DYNAMIC_LGCS_MIN_PINCAP

When set, specifies the minimum pin cap value to use in the wirecap scaling calculation for DYNAMIC_LOCAL_GROUND_CURRENT_SCALING setting 1 or 2. The minimum pin cap of the design is reported in the *adsRpt/totem.log* file. *Default: none.*

Syntax:

```
DYNAMIC_LGCS_MIN_PINCAP <value>
```

DYNAMIC_LOCAL_GROUND_CURRENT_SCALING

Specifies ground current scaling methodology considers wirecap distribution in addition to current scaling criteria. The values have the following effects on wire cap modeling:

0: no wirecap consideration

1: standard tuning method (only considers the ratio of pinCap/wireCap)

2: considers both the pinCap/wireCap ratio and also the wireCap distribution, as well as the minimum pin cap from .lib of the design, to scale the current.

Note that the GSR keyword “DYNAMIC_LGCS_MIN_PINCAP <value>” can be used to set the min_pin_cap value to use in the scaling calculation. The minimum pin cap of the design is reported in the *adsRpt/totem.log* file. *Default: 1.*

Syntax:

```
DYNAMIC_LOCAL_GROUND_CURRENT_SCALING [0 | 1 | 2 ]
```

DYNAMIC_MBFF_MODE

When set for multi-bit flip-flop handling, aligns the starting time of the triggering clock transition and all individual bit transitions at the same time. *Default: 0.*

Syntax:
```
DYNAMIC_MBFF_MODE [1 | 0 ]
```

DYNAMIC_MCYC_TW

When turned on, DYNAMIC_MCYC_TW reports IR drop after the first toggle of the instance occurs in VCD analysis in the *mcyc_effvdd.rpt* file, which enables you to extract effective VDD values for clock cycles before an instance switches. *Optional. Default: 0 (Off).*

Syntax:
```
DYNAMIC_MCYC_TW [ 0 | 1 ]
```

DYNAMIC_PGARC_REPORT_BASE

Totem ranks and reports the worst percentage DvD values for all arcs for the selected base parameter (default avgTW). You can select a different base value, since the worst MinTW, maxTW, or minWC values from one arc may not be the worst among multiple arcs for the same instance. Using this keyword you can change the base parameter on which to base the DvD reports. *DMP compatible. Optional. Default: avgTW.*

Syntax:
```
DYNAMIC_PGARC_REPORT_BASE [avgTW | maxTW | minTW | minWC]
```
Example:
```
DYNAMIC_PGARC_REPORT_BASE minTW
```

DYNAMIC_PRESIM_TIME

Specifies the presimulation time in seconds to initialize capacitance charge and inductor current before time t=0 of simulation. You can also specify a presim Time Step Multiplier (TSM) to speed up the pre-simulation time, multiplying the length of the normal time step (set in the GSR keyword DYNAMIC_TIME_STEP). You can also select just an initial portion of the presim time to use the bigger TSM steps using the <TSM_fraction> option, with the remainder of the presim time at a normal time step. In the syntax below <presim time_ps> is the specified presimulation time, <TSM> is the Time Step Multiplier, and <TSM_fraction> is the fraction of the total presim time (0 to 1.0) at the beginning during which the longer time steps are used. If no values are set Totem automatically determines the required pre-simulation time to get accurate circuit starting conditions. The automatically-selected value of presim time is shown in the Totem log, and the TSM can be observed by looking at the waveform in the current waveform file, *adsRpt/Dynamic*. *DMP compatible. Optional. Default presim time set by Totem. (-1); default TSM=3.*

Syntax:
```
DYNAMIC_PRESIM_TIME [<presim_time_sec>| -1]  ?<TSM> ?
?<TSM_fraction>?
```
Example:
```
DYNAMIC_PRESIM_TIME 10e-09 5 0.9
```
In this example the presim time is set at 10e-09 seconds, the TSM is 5 and TSM_fraction is 0.9. This specifies a time step 5 times the length of the specified time step for the first 90% of the presim time, and a regular time step for the remaining 10% of presimulation.

DYNAMIC_REPORT_DECAP

Specifies what decap values are to be reported in the *adsRpt/Dynamic/decaps.rpt* output file. Values: 0: off (no decap report); 1: maximum current reported for all

intentional decaps as defined in the GSR; 2: in addition to decaps reported in mode 1, reports also the maximum current associated with zero-power instances. *DMP compatible. Optional. Default: 0 (off).*

Syntax:

```
DYNAMIC_REPORT_DECAP [ 0 | 1 | 2]
```

DYNAMIC_REPORT_DVD

When set, enables reporting of TW-based effective voltage drop for all switching cycles in a dynamic simulation run. By default, for basic vectorless analysis, Totem reports TW-based voltage drop for the first switching cycle, and minCyc is reported for the full simulation. For all simulations covered by VCD files, for power transient simulation, and for multiple-cycle vectorless simulation (‘perform analysis -mcyc’), Totem reports the TW-based voltage drop from all switching cycles. The output file is *mcyc_effvdd.rpt*. *DMP compatible. Optional. Default: 0 (off)*

Syntax:

```
DYNAMIC_REPORT_DVD [ 0 | 1 ]
```

DYNAMIC_SELECTIVE_SAVE

When turned off (0) saves all dynamic waveform files. By default (1) saves only waveforms for pad and pin nodes, and switch nodes for low power designs, which reduces disk space use for intermediate files. *Optional. Default:1 (only key waveforms).*

Syntax:

```
DYNAMIC_SELECTIVE_SAVE [1 | 0 ]
```

DYNAMIC_SIMULATION_TIME

If specified, dynamic simulation uses the specified start and end times. For backward compatibility, if only one number is specified, that number is interpreted as the end time, and the assumed start time is t=0. There is a maximum simulation time limit of 30us for vectorless analysis, unless it is overridden by setting the keyword IGNORE_SIMTIME_CHECK 1’. *DMP compatible. Optional. Default: 1/ Frequency.*

Syntax:

```
DYNAMIC_SIMULATION_TIME ?<start_time_sec>? <end_time_sec>
```

Example:

```
DYNAMIC_SIMULATION_TIME 3200e-12
```

is equal to

```
DYNAMIC_SIMULATION_TIME 0 3200e-12
```

DYNAMIC_SOLVER_MODE

To perform high speed circuit modeling in Totem dynamic and CPM, by default an assumption of power-ground symmetry is made, meaning that the package power and ground impedances are considered similar. Setting DYNAMIC_SOLVER_MODE to 1 allows a P/G solution without this assumption, which increases the accuracy, although it is then a more time-consuming and memory-intensive method. This solver mode is recommended for asymmetric package models and when coupling between multiple power and multiple ground networks on the chip need to be considered. *DMP compatible. Optional; default 0.*

Syntax:

```
DYNAMIC_SOLVER_MODE [ 0 | 1]
```

DYNAMIC_SORT_BY_PERCENTAGE

When turned on, analysis results for instances in the **.dvd* file can be sorted by dynamic voltage drop percentage (minimum voltage drop over timing window), rather than by absolute voltage drop values. *Optional; default 0 (off).*

Syntax:

```
DYNAMIC_SORT_BY_PERCENTAGE [ 0 | 1]
```

DYNAMIC_TIME_STEP

Specifies the dynamic simulation time step. Note that specifying a time step over 30 psec is *not recommended*. *DMP compatible. Optional. Default: 10 psec.*

Syntax:

```
DYNAMIC_TIME_STEP <time in sec>
```

Example:

```
DYNAMIC_TIME_STEP 20e-12
```

EFFECTIVE_VDD_WINDOW

The effective Vdd numbers reported in *adsRpt/Dynamic/<design>.dvd* file are based on (Vdd-Vss) values averaged over the timing window of that instance, as defined in the STA file. To obtain a valid effective Vdd, the values are averaged over several small sample windows, not over the entire timing window defined in STA file. By sliding a small window through the entire timing window of the instance, several average effective Vdd numbers are obtained. The worst of the averages computed is then reported as the effective (Vdd-Vss) voltage value. If no timing window data are available, the DvD values based on timing window are not reported.

The size of the small measurement window sliding through the full timing window is instance-specific. By default the sliding window period starts when the instance's input signal is at the 50% level and ends when the instance's output signal is at the 50% signal level (as captured in the APL library for that cell). To change the definition of the sliding measurement window, use this one or more options of this keyword. The IN2IN option can be used for very narrow windows from one level to another level on the input signal. *Optional. Default: (input: 50% V, output: 50% V, minimum width: one time step).*

Syntax

```
EFFECTIVE_VDD_WINDOW {
    ? IN2OUT <inputV_fraction> <outputV_fraction> ?
    ? IN2IN <inputV_fraction1> <inputV_fraction2> ?
    ? MINIMUM <minWidth_ps> ?
}
```

Example:

```
EFFECTIVE_VDD_WINDOW {
    IN2OUT 0.2 0.7
    MINIMUM 10
}
```

In the above example, the sliding window is the time interval between when the instance's input signal is at the 20% level to the time the instance's output signal is at the 70% level, with a minimum sliding window duration of 10ps.

MACRO_IRDROP

To output the dynamic IR drop on the framework of macro cells. When set to 1, this will display the DvD values at macro pin level on GUI DvD map, else displays that of adsU1 instance. *Default: 0*

Syntax:

```
MACRO_IRDROP [ 0 | 1 ]
```

MAX_IPROF_POINT_NUMBER

Controls the maximum number of sample points that are imported from the APL *cell.current* profile for use in simulation. *Optional; Default: 4000*.

Syntax

```
MAX_IPROF_POINT_NUMBER <max_samples>
```

MMX_ADAPTIVE_SAMPLING

Dynamic simulation during power analysis applies adaptive sampling of current profiles of MMX cells by default. When set to 0, constant step waveform sampling is used, to reduce peak memory usage. When set to 0, this is most effective for memory designs. *Optional. Default: On.*

Syntax

```
MMX_ADAPTIVE_SAMPLING [ 0 | 1 ]
```

MMX_DVD_GLITCH_FILTER_FILE

Specifies a file a which defines the criteria for filtering the dvd glitch. Details of the filtered glitches are reported in adsRpt/Dynamic/cell_mmx.mmx_pin_filtered file.

Syntax

```
MMX_DVD_GLITCH_FILTER_FILE <filter file>
```

Format of filter file:

```
#<domain_name> <glitch_width_ps> <voltage_level_volts>
<glitch magnitude_mv>
```

Sample filter file:

```
#<domain_name> <glitch_width_ps> <voltage_level_volts>
<glitch magnitude_mv>
####### Filter out glitch for VDD domain
VDD    90     0.96    40
##### Filter out glitch for VSS domain
VSS    100    .06     50
```

Where,

- ‘voltage_level’ is where the glitch width and glitch magnitude/height are measured.
- ‘glitch magnitude’ is measured from the specified voltage level to the glitch peak.
- a glitch is filtered if
  - glitch width is smaller than the glitch width threshold AND
  - glitch magnitude/height is larger than the specified glitch magnitude

MULTI_CYCLE_SCENARIO

When set, makes the charge value consistent between cycles in multi-cycle analysis, enabling matching simulation results with frame-based PowerStream power for multiple-cycle and power-transient flows. *Optional. Default: Off.*

Syntax

```
MULTI_CYCLE_SCENARIO [ 0 | 1 ]
```

PIECEWISE_SWITCH_INPUT

This keyword is used to specify the file that defines the piecewise linear waveforms applied at the controlling pin of a power switch. In the definition file, each line has the following form:

```
<switch_inst_name> <ctrl_pin_name> ( <t1> <volt1> <t2> <volt2> ... )
...
```

where times are in picoseconds and voltages are in volts. *Optional. Default: none.*

Syntax:

```
PIECEWISE_SWITCH_INPUT <file_name>
```

RTL_FAST_MODE

Specifies the RTL propagation period and also the presim time, using a period factor, n, subject to predefined maximum and minimum limits. The period factor multiplies the dominant period to define the desired simulation time. When RTL_FAST_MODE is not defined, the minimum period limit is used for RTL propagation. The period is limited by maximum and minimum values as follows:

Maximum period = 1/2 (<maximum clock period in STA file>) ps

Minimum period = 1/2 (<minimum clock period in STA file>) ps

Setting the period factor allows you to optimize the state propagation process and also adjust the number of presim cycles used for the analysis. By setting n to higher positive numbers, the state propagation process is accelerated by using fewer and longer sample periods. Setting n to a negative number has the same effect on the length of the period as using a positive n, but also reduces the number of presim cycles as the value of n is increased negatively. Note that the period factor can be set to any positive or negative real number, but analysis accuracy is decreased as n is increased either positively or negatively. *Optional. Default: none.*

Syntax

```
RTL_FAST_MODE <period_factor>
```

SIMULATION_CACHE_DIRECTORY

Specifies a separate cache directory for simulation, to prevent issues with not enough disk space. In the Signal EM flow, allows read/write of simulation cache data into the simulation directory specified, rather than into the current working directory. *Optional. Default: none.*

Syntax:

```
SIMULATION_CACHE_DIRECTORY <sim_dir_path>
```

VCD_FF_EDGE_TRIG

When turned On, supports proper handling of switching by FF/Latch edge-triggered circuits. *Optional; default 0 (off).*

Syntax

```
VCD_FF_EDGE_TRIG [ 0 | 1 ]
```

VCD_PREPARE_SCENARIO

Supports design flow when several VCD/FSDB files are used, and avoids processing the VCD/FSDB file more than once in the 'perform analysis -vcd' stage. *DMP compatible. Optional; default is 1(On).*

Syntax

```
VCD_PREPARE_SCENARIO [ 0 | 1 ]
```

VCD_SCENARIO_COMPRESS

When set, compresses the *apache.scenario* file in gzip format in both power calculation and simulation. Note that this keyword works only when the START_TIME/END_TIME options are specified using the VCD_FILE GSR keyword. *Optional. Default: 0 (off).*

Syntax

```
VCD_SCENARIO_COMPRESS [ 0 | 1 ]
```

FAO General Keywords

FAO_ACCURATE_VOLTAGE

By default, performs accurate voltage difference calculations among nets improperly shorted together, by ignoring the shorts between nets. When turned off, voltage values with the nets shorted are obtained *Optional. Default: 0 (in 15.1, default value is 1).*

Syntax:

```
FAO_ACCURATE_VOLTAGE[ 0 | 1 ]
```

FAO_ADD_STACK_VIA

When set (default), turns on stack via insertion for the FAO 'mesh'/'mesh via' commands. *Optional. Default: 1.*

Syntax

```
FAO_ADD_STACK_VIA [0|1]
```

FAO_BYPASS_CUTSIZE_CHECK

When set, turns off the cut size checks of minimum via size, which are based on the LEF VIARULE keyword setting for missing via reports. *Optional. Default: On.*

Syntax

```
FAO_BYPASS_CUTSIZE_CHECK [ 0 | 1 ]
```

FAO_HOLD_LIC

When set to 1, disables FAO license check-in and check-out, and holds the FAO license until Totem exits (default 0). This is useful when multiple FAO commands are executed in sequence.

Syntax

```
FAO_HOLD_LIC [ 0 | 1 ]
```

FAO_MISVIA_ONE_FILE

When set to 0, separates combined list of overlapping wire missing vias and stacked missing vias into two files. By default, lists are combined into one file. *DMP compatible. Optional; default 1.*

Syntax

```
FAO_MISVIA_ONE_FILE [ 0 | 1 ]
```

FAO_OBJ

FAO keyword and GSR keyword that selects the type of instance prioritization and results desired. *Optional; default: DVD*

Syntax

```
FAO_OBJ [ DVD | TIMING ?<slack threshold>? |
    PATH ?<slack threshold>? ]
```

where

DVD : instances with highest voltage drop are selected for FAO

TIMING <slack_threshold>: all instances in the design with high delta delay and delta slack values caused by DvD are selected for FAO. The default value of the slack threshold is 0.
Note: To use FAO_OBJ TIMING you must execute '`import sdf xxx.sdf`' before cell swapping, or the candidates for swapping will be zero. Also, fao_region is still honored to filter out candidates from critical path or timing/ slack info.

PATH <slack_worse than> : instances in critical timing paths with high delta delay and delta slack values caused by DvD are selected for FAO. The default value of the slack threshold is 0.

Example:

```
FAO_OBJ DVD
```

FAO_REGION

Specifies the boundary coordinates of a region for which optimization or modification of components intersecting the region should be performed, in micron units. This keyword also supports non-rectangular regions, but only with the 'decap fill', 'decap show' and 'decap remove -all' commands. Rectangular coordinates can also be used with 'mesh optimize', 'mesh fix', 'decap advise -place', and 'cell swap'. *DMP compatible. Optional; default: whole chip.*

Syntax:

```
fao_region  { <x1> <y1> ... <xn> <yn>}
```

where

<xn> <yn> are the x,y coordinates of the points bounding the region, which should be specified in clockwise order starting in the lower left corner.

Example:

FAO_REGION {140 140 140 220 220 220 220 200 160 200 160 140 }

FAO_TURBO_MODE

Specifies fast FAO computation mode if set to 1 (default), or for the most accurate mode, which requires a longer calculation time, set value to 0. *Optional; default: 1.*

Syntax:

```
fao_turbo_mode [ 1 | 0 ]
```

FIX_WINDOW

Specifies the size of the horizontal and vertical bands in which the grid widths should be modified, centered on each voltage drop hotspot. This allows you to control the area in which wire changes can occur in the design. Can extend beyond the specified analysis region. Units: microns. Use with commands: mesh fix, decap advise -place, decap fill, decap remove, cell swap. *Optional; default: 100um x 100um*

Syntax

```
fix_window  { x_dim y_dim }
```

Example:

```
fix_window { 400 400 }
```

NOISE_LIMIT

For the ordered list of worst case hot instances, the noise_limit specifies the voltage drop value below which the hot instance are not considered for mesh or decap optimization. For example, if noise_limit is set at 5%, hotspots or hot instances with less than a 5% dynamic voltage drop would not be considered for fixing. Use with commands: mesh optimize, mesh fix, and decap optimization. *Required for grid fixing; optional for decap optimization; default 0*

Syntax:

```
noise_limit <percent_voltage_drop>
```

Example:

```
noise_limit 5
```

NOISE_REDUCTION

Selects the target amount of change (decrease or increase) in voltage drop desired as a percentage of the existing voltage drop (from the last Totem run). For example, for an existing voltage drop of 100mV, to get an improvement (reduction) of 20mV requires setting this GSR keyword to 20. Use with commands: mesh optimize, mesh fix, decap optimize. *Optional; default:*

Syntax:

```
noise_reduction <percentage reduction>
```

Example:

```
noise_reduction 10
```

which targets a 10% reduction in present voltage drop

```
noise_reduction -10
```

which targets a 10% relaxation (increase) in present voltage drop

Grid Fixing and Optimization Keywords

FAO_DVD_TYPE

Specifies the type of dynamic voltage drop optimization to use for the FAO commands ‘mesh optimize/fix’. Integer values are assigned to select the following optimization choices:

1 - eff_vdd_tw : use average effective voltage (Vdd-Vss) over the timing window

2 - max_vdd_tw : use maximum effective voltage (Vdd-Vss) over the timing window

3 - min_vdd_tw : use minimum effective voltage (Vdd-Vss) over the timing window

4 - min_vdd_all : use minimum effective voltage (Vdd-Vss) over clock cycle

Optional; default: 1.

Syntax:

```
fao_dvd_type <optimize value 1-4>
```

Example:

```
FAO_DVD_TYPE 2
```

FAO_DRC_DROP_RATIO

Specifies the fraction of maximum grid width (specified in 'fao_range') to use when there are potential metal spacing DRC problems. For example, an 0.8 value allows 80% of the max grid width to be used. A grid is not changed if the width change would cause a DRC error. Use with commands mesh optimize, mesh fix. *Optional; default: 1 (100% of max width)*

Syntax:

```
fao_drc_drop_ratio <fraction_of_max_width>
```

Example:

```
FAO_DRC_DROP_RATIO 0.8
```

FAO_DYNAMIC_MODE

Specifies that FAO uses either static or dynamic mode for fix/optimize. Use with commands: mesh optimize, mesh fix. *Optional; Default: 0 (off)*

Syntax:

```
fao_dynamic_mode  [ 0 | 1 ]
```

FAO_LAYERS

Defines the layers or specific metal lines to be fixed or optimized by running FAO. Units: microns. Use with commands mesh optimize, mesh fix. *Required.*

Syntax:

```
fao_layers { { <layer name>  ?<direction>?
?<wire_size_constraints> }...} ?
```

where

<layer name>: layer name as defined in the technology file

<direction> : ver (wires running vertically) or hor (wires running horizontally)

<wire_size_constraints> : defines width allowed for specific wires. The following relational operators are allowed: >, <, >=, <=

Example:

```
fao_layers { { met4 ver > 1 < 3 } { met5 hor = 12 }}
```

Note: Even if there is only one layer item, double brackets are required.

FAO_MISVIA_DISTANCE

When set, missing vias within the specified distance of each other are reported only once. By default, all missing vias are reported. *DMP compatible. Optional; Default: 0.*

Syntax:

```
FAO_MISVIA_DISTANCE <value_um>
```

FAO_MISVIA_RPT_SHORT

When set, places vias in a shorted area if this keyword is turned on. *Optional; Default: 0 (off).*

Syntax:

```
FAO_MISVIA_RPT_SHORT [ 1 | 0 ]
```

FAO_MISVIA_RPT_STACK

By default (1) the results of missing via checks report missing stack vias in the missing vias report. *Optional; Default: 1.*

Syntax:

```
FAO_MISVIA_RPT_STACK [ 1 | 0 ]
```

FAO_NETS

Specifies the power and ground nets to be fixed or optimized. The net names must be defined in the GSR file. Use with commands mesh optimize, mesh fix. *Required.*

Syntax:

```
FAO_NETS [ <net1>| {
    <net1>
    <net2>
    ...
} ]
```

Example:

```
FAO_NETS VDD
FAO_NETS {
    VDD2
    VSS2
}
```

FAO_PRECISE_VOLTAGE

When set, voltages reported in the missing via report match the values obtained from GUI clicks at the same location. *Optional; default: 0.*

Syntax

```
FAO_PRECISE_VOLTAGE [ 0 | 1]
```

FAO_RANGE

Defines the minimum and maximum grid widths that will be considered in grid fixing, and optionally limits the direction in which wire changes can occur. Units: microns. Use with commands: mesh optimize, mesh fix. *Required.*

Syntax:

```
fao_range { { <layer> <min_width> <max_width>
? <extend_dir> ?} ... }
```

where

<extend_dir>: specifies the direction in which the width change should be made: bot/top/left/right.

Example:

```
fao_range { { met5 10 30 bot} { met6 15 40 } }
```

Note: Even if there is only one layer item, double brackets are required.

FAO_ROW_VDD_SITE

Allows FAO to handle multiple power domains simultaneously for the 'decap fill' command. The site name in a decap cell's LEF is checked against the site name of the row in DEF and the appropriate decap cells are then picked for the power domain, when the power domain and site names are specified using this command.

Syntax:

```
FAO_ROW_VDD_SITE {
<power_domain_name> <sitename1> <sitename2> ...
}
```

Example:

```
FAO_ROW_VDD_SITE {
   VDD1 CORE
   VDD2 CORE2
}
```

In the above example, DEF has two ROW sites defined: CORE that belongs to net VDD1 and CORE2 that belongs to VDD2. FAO adds decaps only in the domain(s) and sites defined by the keyword. This is currently supported only for the FAO command 'decap fill'.

FAO_SUB_GRID_NETS

Defines the wire ordering from lower/left to upper/right when creating new mesh strips. Use with command mesh sub_grid. *Optional; default: none*

Syntax:

```
fao_sub_grid_nets { <list of power/ground nets> }
```

Example:

```
fao_sub_grid_nets { VDD VSS },
```

where

{ VDD VSS} for a vertical mesh is for each pitch to create a wire for net VDD first, then to its right create a wire for a VSS net, with a pitch-to-pitch space specified by ‘fao_sub_grid_spec’.

FAO_SUB_GRID_SPEC

Defines mesh parameters for creating subgrids (could be more than one layer). Use with command ‘mesh sub_grid’. *Optional; default: none*

Syntax:

```
FAO_SUB_GRID_SPEC { { <layer> <dir> <space> <pitch>
<top_via_layer> <bot_via_layer> } ... }
```

where

<layer> is the mesh layer to create,

<dir> = h for horizontal or v for vertical mesh creation,

<space> specifies pitch-to-pitch spacing between wires of power nets specified in fao_sub_grid_nets,

<pitch> specifies center-to-center distance for wires created from the first net in fao_sub_grid_nets to the following set of power bus,

<top/bot_via_layer> specifies which top and/or bottom layer to drop (stack) vias; 0 for not dropping any vias.

Example:

```
FAO_SUB_GRID_SPEC { { MET3 h 6 12 MET4 MET2 }
    { MET2 v 5 10 0 MET1 } }
```

Note: Even if there is only one layer item, double brackets are required.

FAO_WIDTH_CNSTR

Specifies arithmetic constraint relationships between different metal layers. This GSR keyword can be used to relate the width of a particular wire to that of another mathematically. This GSR keyword is optional and intended for advanced usage. Units: microns. Use with commands: mesh optimize, mesh fix. *Optional; default: no specification*.

Syntax:

```
fao_width_cnstr { { layer1*<value> -/+ layer2*<value>
    >/</>=/<= <value> } ...}
```

Example:

```
fao_width_cnstr { { met4 - met3*2 > 0 }
    { met5 - met4*3 > 10 } }
```

Note: Even if there is only one relationship item, double brackets are required.

NUM_HOTSPOT

Defines the number of highest voltage drop hot-spots that will be targeted for fixing during mesh fixing or optimization. Use with commands: mesh optimize, mesh fix. *Optional; Default: 1*

Syntax:

```
num_hotspot <int>
```

Example:

```
num_hotspot 5
```

Decap Optimization Keywords

CAP_LIMIT

Defines an upper limit on the total amount of new decoupling capacitance that can be inserted by FAO. Units: pF. Use with commands decap advise, decap place, decap advise -place, decap fill. *Optional; default: unlimited*

Syntax:

```
cap_limit <cap_pF>
```

Example:

```
cap_limit 1000
```

which defines 1000 pf as the maximum added decap.

DECAP_CELL

Specifies the names of decap cells to be used in the design and their key physical parameters --cell width and height, effective capacitance and resistance, metal layer name, and leakage current. For information not specified in DECAP_CELL, Totem automatically extracts the needed parameter values from the LEF and APLCAP files. Information in DECAP_CELL supersedes that in APLCAP files. New decap cells can be added to the design after dynamic analysis by using the ‘gsr append DECAP_CELL’ TCL command, or existing declared cells and parameter values can be changed using ‘gsr set DECAP_CELL’, while present values can be displayed using ‘gsr get DECAP_CELL’. These commands can also be executed

even before the design is loaded in totem GUI (execution of setup design TCL command)

You can use DECAP_CELL and BPA to create new decap cells/blocks and assign decap values. These decap blocks are also considered by the 'print decap' command. Decap cells can either come from LEF or be created in the flow. BPA instantiates the existing decap cells to put decap instances in the design. *DMP compatible. Optional (required for decap insertion in what-if and FAO analysis). Default: none.*

Syntax:

```
DECAP_CELL {
<decap_cellname> ? <width_um>? ?<height_um>? ?<C_pF>?
?<R_ohm>? ?<metal_layer_name>? ?<leakage_A>?
...
}
```

Example:

```
DECAP_CELL {
decap1
decapAB 0.25 0.75 0.01 1000 MET1 1e-9
decapXY 0.75 0.75 0.03 1500 MET1 2e-9
}
```

DECAP_CELL_FILES

Allows importing multiple files that contain a list of decap cells. *DMP compatible. Optional; default: None.*

Syntax:

```
DECAP_CELL_FILES {
    <decap_file1>
    ...
}
```

Example:

```
DECAP_CELL_FILES {
  user_data/demo1.dec
  user_data/demo2.dec
  }
```

DECAP_DENSITY

Specifies a multiple of legally-placed decaps to be placed in non-overlap mode, to test for improvement in voltage drop performance. Use with commands: decap advise -place. *Optional; default: 1.0*

Syntax:

```
decap_density <integer multiple>
```

Example:

```
decap_density 3
```

DECAP_TILE_MAX

Sets the decap value per tile (display rectangle) that is displayed as red (or the chosen color) in all types of decap maps. When set with a value greater than 0 pf per tile, each decap color map has the maximum capacitance displayed in red set to the same maximum value and the decap ranges for other colors are scaled

accordingly. This allows different decap maps with different color range settings to be compared more easily. *Optional. Default: 0 (off).*

Syntax:

```
DECAP_TILE_MAX <value_pF>
```

Example:

```
DECAP_TILE_MAX 100
```

FAO_DECAP_FILL_ALG

When set to 1, allows you to maximize the amount of decap placed and minimize leakage. Provides multiple smaller decaps rather than the largest decap that fits in the available space if it provides a larger cumulative decap value. Also, if there are different types of decaps with the same decap value, but with different leakage values, decaps with lower leakage are selected. When set to 2, maintains good balance between DvD improvement and increased leakage power; it adds only enough decaps to supply the demand current for each instance. *Optional. Default: 0 (off).*

Syntax:

```
FAO_DECAP_FILL_ALG [ 0 | 1 | 2]
```

FAO_DECAP_FILL_NEW_FLOW

When turned on, 'decap fill' ignores small differences in rail wire width when snapping pins in a row or column of cells (enables acceptance of irregular rail widths). You can select the lower limit for wire width using the keyword FAO_WIRE_WIDTH_LOW_LIMIT. *Optional. Default: 0 (off).*

Syntax:

```
FAO_DECAP_FILL_NEW_FLOW [ 0 | 1 ]
```

FAO_DECAP_OVERLAP

Allows FAO to "place" decaps in non-legal placement areas (where there is not enough available space in present design), to analyze how the placement would affect dynamic voltage drop. Use with commands: decap advise -place. *Optional; Default: false.*

Syntax:

```
fao_decap_overlap [true/1 | false/0]
```

Turns On overlap if value is set to '1' or 'true', Off if value is set to '0' or 'false'.

Example:

```
fao_decap_overlap true
```

FAO_DRC_PL_OBS

When turned on, specifies that placement blockages in DEF will be checked for DRC violations when performing 'decap fill'. *Optional: default: 1 (on)*

Syntax:

```
fao_drc_pl_obs [0 | 1]
```

Example:

```
fao_drc_pl_obs 0
```

FAO_DRC_OBS

When turned on, specifies that signal net routing in DEF will be checked for DRC violations when performing 'decap fill'. *Optional: default: 0 (off)*

Syntax:

```
fao_drc_obs [0 | 1]
```

FAO_ECO_NAME

Changes the default instance name pattern for decap cells. *Optional; default: <eco_string>=SH.*

Syntax:

```
fao_eco_name <decap_cell>_<eco_string><number>
```

Example:

```
fao_eco_name decapABC_XY3
```

FAO_MAX_SHIFT

Specifies a maximum movement distance when pushing cells away from a hot instance to make space for prefilling decaps. Use with commands: decap fill -prefill and decap fill -prefill_only. *Optional: default: 5 um*

Syntax:

```
fao_max_shift <max dist to move cells um>
```

Example:

```
fao_max_shift 3
```

FAO_PLACE_GRID

Specifies an adjustment to the grid resolution ("tile" width) in microns, to get the correct placement for decaps on placement grid spacing, when the LEF file has incorrect or no SITE information. *Optional: default: none.*

Syntax:

```
fao_place_grid <grid_adjustment>
```

Example:

```
fao_place_grid 0.2
```

FAO_ROW_SITE

Specifies a particular set of "sites" (a placement grid) by its name in DEF, to avoid conflicting and overlapping decap insertions from multiple library row definitions. LEF "sites", also called "unit-tiles", are defined in LEF and used in the DEF/ROW section to create "row boxes" to allow P/R tools to create multiple clusters using different cell grid libraries, and thus may create overlapping ROW boxes. By default, without using FAO_ROW_SITE, FAO takes all ROW grids defined in DEF to create placeable decap areas. *Optional; default: all defined DEF row grids.*

Syntax:

```
FAO_ROW_SITE {
<LEF_site_name>
...
}
```

Example:

For a DEF file ROW section containing several row definitions as follows:

```
ROW CORE_ROW_37549 bonuscoreaxg 3547600 2680600 N DO 7793 BY 1 STEP 690 0 ;
```

```
ROW CORE_ROW_52855 core 8927530 2680600 N DO 2054 BY 1 STEP 230 0 ;
ROW CORE_ROW_72819 bonuscore 8927530 2680600 N DO 684 BY 1 STEP 690 0 ;
```

Using ‘FAO_ROW_SITE core’, only rows defined by the ‘core’ site name are created for FAO decap placement.

FAO_WIRE_WIDTH_LOW_LIMIT

For the ‘decap fill’ command, this keyword sets a width tolerance limit (microns) for P/G wires composed of several segments with different widths when snapping pins in a row or column of cells. Wire segments whose widths are smaller than or equal to the specified limit are not considered by 'decap fill'. Set the keyword ‘FAO_DECAP_FILL_NEW_FLOW 1' to enable the acceptance of irregular wire widths. *Optional; default: 0.*

Syntax:

```
FAO_WIRE_WIDTH_LOW_LIMIT <tolerance_u>
```

LEAKAGE_LIMIT

Specifies an upper limit on the total leakage power allowed from decaps added to the design. Units: Watts. Use with command decap advise -place. *Optional; default: infinity*

Syntax:

```
leakage_limit <max decap power leakage>
```

Example:

```
leakage_limit 10e-6
```

NUM_HOTINST

Defines the number of high voltage drop instances that are targeted for improvement during decap placement. Use with commands decap advise, decap advise -place. *Optional; default: 1*

Syntax:

```
num_hotinst <num_instances>
```

Example:

```
num_hotinst 3
```

Low-power Design Keywords

BLOCK_POWERUP_FILES

Compiles powerup data from multiple individual block analyses, and their associated time offset in seconds from simulation time t=0. Used for full-chip power analysis. *DMP compatible. Optional. Default: time offset =0.*

Syntax:

```
BLOCK_POWERUP_FILES
{
<block_name-1> <FilePathName-1> [ time_offset-1 ]
...
<block_name-n> <FilePathName-n> [ time_offset-n ]
```

```
}
```

Example:

```
BLOCK_POWERUP_FILES block_powerup.file
```

CHARGE_SWITCH

Specifies the names of switch cells used for charge switch operation in low power designs and their ramp-up threshold voltages (Volts). The RAMPUP_THRESHOLD option applies to charge switches without specific threshold values assigned. If a charge switch cell has both a charge control pin and function control pin, they must be specified. *Optional. Default: none.*

Syntax:

```
CHARGE_SWITCH {
RAMPUP_THRESHOLD <Volts>
<cell_name1> ? <Volts>?
<cell_name2> <charge_control_pin> <funct_control pin> ?<Volts>?
...
}
```

Example:

```
CHARGE_SWITCH {
RAMPUP_THRESHOLD 0.8
Charge_Sw_AB 0.7
Charge_Sw_BCD CCpin3 FCpin7
}
```

CHECK_SWITCH_POWERON

When one power switch is controlled by two enable pins, you need to make sure that the proper threshold voltage is reached when the second control pin is enabled. Using CHECK_SWITCH_POWERON checks the proper turn-on of the switch. *Optional. Default: None*

Syntax:

```
CHECK_SWITCH_POWERON {
  [ THRESHOLD <threshold_all_V> |
  <cellname> <cell_threshold_V> ]
  ...
  }
```

Example:

```
CHECK_SWITCH_POWERON {
    SW_ABC 0.2
    SW_CDE 0.1
    SW_xyz 0.15
```

DECOUPLE_LDO_GROUND

When set, avoids ground bounce effects in LDO output. Note that you also must set “DECOUPLE_LDO_GROUND 1" in the Apldo configuration file in order to eliminate the LDO ground bounce effect.

Syntax:

```
DECOUPLE_LDO_GROUND [ 0 | 1 ]
```

DYNAMIC_ADAPTIVE_RON

Specifies adjustments to switch On resistance based on the variation in the gate voltage, and sets a threshold variation value. For most accurate results the power switch should be characterized using a multi-dimensional switch model file, enabled with the APLSW keyword 'MD_PWL 1'. If the default switch model is used, Totem uses internal heuristics to scale the switch Ron. *Optional; default: 0, Off.*

Syntax:

```
DYNAMIC_ADAPTIVE_RON [0|1] <variation_threshold_%>
```

Example:

```
DYNAMIC_ADAPTIVE_RON 1  8
```

The example specifies that the threshold for Ron change is an 8% variation in power or ground voltage.

EXTRACT_INTERNAL_NET

When set, specifies that you do not need to specify the internal nets in the VDD_NETS list; Totem automatically traces the internal power nets and extracts them. *DMP compatible. Optional. Default: 1.*

Syntax:

```
EXTRACT_INTERNAL_NET [ 0 | 1 ]
```

PIECEWISE_CAP_FILES

Specifies the file that contains the piecewise linear (PWL) capacitance and leakage data for cells in a design, used for low power ramp-up analysis. *DMP compatible. Optional. Default: none.*

Syntax:

```
PIECEWISE_CAP_FILES <file name>
```

Example:

```
PIECEWISE_CAP_FILES pwl
```

POWERUP_SAVE

Specifies the file for saving powerup data for a single block, for subsequent hierarchical full-chip analysis utilizing the BLOCK_POWERUP_FILE keyword. Used for low power design analysis. *Optional. Default: none.*

Syntax:

```
POWERUP_SAVE <FilePathName>
```

Example:

```
POWERUP_SAVE powerup_save.file
```

POWERUP_OUTPUT_HIGH_PROB

Specifies the probability that the output of any instance in a low power block is high during ramp-up analysis. *Optional. Default: 0.5.*

Syntax:

```
POWERUP_OUTPUT_HIGH_PROB <probability>
```

Example:

```
POWERUP_OUTPUT_HIGH_PROB 0.7
```

SWITCH_MODEL_FILES

Specifies the location of a switch model file, as required for power-up analysis and provides the capability to update the switch cell models. You can analyze the usability and effects of different size footer/header cells in a power-gated design. The switch model file is generated by the *apIsw* utility in characterization of the switches. *DMP compatible. Optional. Default: none.*

Syntax:

```
SWITCH_MODEL_FILES
{
    <switch_model_file>
}
```

Example:

```
SWITCH_MODEL_FILES
{
    SW_MOD_AB
}
```

SWITCH_MODEL_XTR_FILES

Allows you to define transistor pairs and their relative current distribution weight using specified files. When this keyword is used, the 'USE_MF_SWITCH_MODEL' GSR keyword is automatically turned on. The syntax for the contents of a model transistor file is as follows:

```
SWITCH_CELL <cell1> {
    <ext_x1> <ext_y1> <int_x1> <int_y1> <weight1>
    <ext_x2> <ext_y2> <int_x2> <int_y2> <weight2>
    ...
}
```

where

ext_xN, ext_yN: a point on the source/drain port, connected to the external net, of the switch transistor. Units are in um.

int_xN, int_yN : a point on the source/drain port, connected to the internal net, of the switch transistor.

weightN : the relative "weight" assigned to the specified transistor location for current distribution purposes (total weight for all points is 1)

This feature requires that there is only one sink layer for the switch cell, either by defining only a sink layer port in the LEF file, or using the keyword CELL_PIN_FILE to define the sink layer for a multi-layer port pin. If there are multiple definitions of SWITCH_MODEL_XTR for the same cell, then the last definition overwrites all the previous definitions.

Note that if the SWITCH_MODEL_XTR_FILES keyword is used, Totem uses the external/internal points defined in the file to determine the port geometries for those points landed, then pairs up the geometries as the points are paired in the file. Then Totem finds the nodes on the paired geometries and selects one from each node group on the paired geometries, and adds a transistor model between the two nodes for each pair. The relative weights are assigned to the transistors so that the simulator knows how to distribute the current on each transistor. The current distribution is calculated by the weight of the external/internal point, divided by sum of all the weights of all the parallel CONNECTED transistors of the switch cell. "CONNECTED transistors" means the transistors defined by the switch cell that are actually connected to the network. Note that Totem automatically picks up the

<cell>_adsgds.sw file in GDS_CELL directory generated by GDS2DB and puts it in the SWITCH_MODEL_XTR_FILES list. *DMP compatible. Optional. Default: none.*

Syntax:

```
SWITCH_MODEL_XTR_FILES {
    <file1>
    ...
}
```

SWITCH_RES_FILES

Specifies a Switch Resistance file defining RON values for each switch, and overrides values in the switch model file. The file format is as follows:

```
<switch_instance_name1> <RON1_Ohms>
...
```

Note that after starting a run, using the TCL command 'setup switchres <filename>' also specifies a Switch Resistance file overriding switch model file values. Then using 'setup switchres clear' causes the Switch Resistance file to be ignored, and returns Ron value control to the switch model file. *Optional. Default: none.*

Syntax:

```
SWITCH_RES_FILES {
  <file name>
}
```

USE_MF_SWITCH_MODEL

Totem can create pin instance node pairs between external and internal switch nets, assign a weight for each pair, and simulate separately the characterized switch model in parallel smaller switches with the proper weight, assuming equal weight for all the pairs (see keyword SWITCH_MODEL_XTR_FILES). Automatic switch transistor recognition finds the minimum space between all the driver/receiver shapes, and assumes that the minimum space is the transistor length. Then all driver/receiver shape pairs that have a space equal to the minimum space found in the previous step are marked as switch transistor source/drain pairs. This keyword controls use of the multi-finger switch model, and removes group node shorting for switch cells if the multi-finger switch model is used.

When the design contains both multi-finger switch cells and regular switch cells, only cells with explicit multi-fingered switch information are treated as multi-fingered switch cells, and all other switch cells are treated as normal switch cells. However, you can force automatic multi-fingered switch cell treatment of all switch cells by setting this keyword to 1. *Optional. Default: 0.*

Syntax:

```
USE_MF_SWITCH_MODEL [ 0|1]
```

VP_CONTROL

Specifies the name of the Virtual Power Control file that describes the controlling pin information for switched IP macros. *Optional. Default: none.*

Syntax:

```
VP_CONTROL {
    <filename>
}
```

Example:

```
VP_CONTROL {
```

```
        vpcf_abcd4
    }
```

Information in the VPC file has the following format (all times and delays in seconds):

```
<macro name> {
POWERON <block ctrl pin_1> <int_domain1> ?<poweron_time>?
...
POWERON <block ctrl pin_n> <int_domainN> ?<poweron_time>?
```

Note: You must ensure that the timing windows for the specified pins are in the timing file for Totem in order to execute a ramp-up simulation involving the switch IP block.

ESD keywords

ESD_BSIM3_DIODES

When set, adds parasitic well diodes connecting drain to bulk and source to bulk for any instances with specified BSIM3 models, in order to correctly model reverse breakdown conditions. The area of the drain diode is 'Area of Drain' (AD) and the area of the source diode is 'Area of Source' (AS) of the BSIM3 instance. This keyword is only needed for old processes in which BSIM3 models are used rather than BSIM4 models. *Optional. Default: none.*

Syntax:

```
ESD_BSIM3_DIODES {
    bsim3modelname1   diodemodelname1
    bsim3modelname2   diodemodelname2
    ...
}
```

ESD_CLAMP_FILES

Allows you to specify one or more ESD clamp files. When executing 'perform esdcheck', if a clamp file is not specified in the command, Totem picks up the clamp files from this keyword and executes 'perform esdcheck'. *Optional. Default: none.*

Syntax:

```
ESD_CLAMP_FILE <clamp_file_name>
```

or

```
ESD_CLAMP_FILES {
  <fileName1>
  <fileName2>
  ...
}
```

ESD_CLAMP_PIN_FILE

Specifies a clamp pin settings file describing the details for shorting clamp pin regions, and those with pseudo vias. The format of the clamp pin file is as follows:

```
<clamp_cellName> [all | shortByNet] ? PIN {<pin Name>...}?
? LAYER {<layerName> ... }? ? TYPE { <clampType> } ?
? BBOX {<x1> <y1> <x2> <y2>}? BBOX {<x3> <y3> <x4> <y4>}...
? MARGIN { <distance_um> ? ? INCLUDEALL { 1| 2 } ? FINAUTO
{ 0 }
```

where

all : shorts all nodes of the same pin

shortByNet : shorts all nodes of the same net for each instance of the cell

PIN : specifies a particular set of pins to be shorted by pin name

LAYER : specifies a particular set of pins to be shorted by layer name

TYPE : specifies the clamp type

BBOX: specifies pins to be shorted in a particular BBOX. Multiple BBOX for clamp pins are supported.

FINAUTO : automatically determines the orientation of the clamp cells

MARGIN : expands the bounding box by a distance in um for node shorting.

INCLUDEALL { 1|2 }: includes wire/via nodes in addition to pin nodes for shorting, where “1” means all wire/via nodes inside the specified shorting bounding box are to be shorted, and “2” means all wire/via nodes inside the expanded MARGIN shorting bounding box, but *not inside* the basic shorting bounding box, are to be shorted.

A sample ESD_CLAMP_PIN_FILE file is shown below:

```
CellA PIN { VDD } LAYER { MET1 MET2 }
CellB all MARGIN { 0.3 } INCLUDEALL { 2 }
...
```

In the example, for CellA, all pins of VDD in MET1/MET2 are shorted together and pushed out. For CellB, all wire/via nodes inside the expanded (by 0.3 um) shorting bounding box, but not inside the basic shorting bounding box, are to be shorted. The file also can be specified using the command 'pfs import clamp_pin <file_name>' to import the setting file. Alternatively, the same results can be achieved, after extraction, on the fly using the TCL command ‘pfs add clamp_pin <cell_name> ...’ (see section "pfs", page D-969). *Optional. Default: none.*

Syntax:

```
ESD_CLAMP_PIN_FILE <file_name>
```

ESD_CLAMP_PIN_NODE_DISTANCE

Specifies the pitch (um) at which to add more nodes on each partitioned finger segment. *Optional. Default: 0 (no nodes are added).*

Syntax:

```
ESD_CLAMP_PIN_NODE_DISTANCE <distance>
```

ESD_DEF_IGNORE_LAYER

Specifies layers to be ignored in DEF files that do not contain any clamp devices, to speed up ESD analysis. *Optional. Default: none.*

Syntax:

```
ESD_DEF_IGNORE_LAYER {
    <layer1>
    <layer2>
    ...
}
```

ESD_EXTRACT_CLAMP_NET

Specifies whether to automatically promote local nets between clamp devices that are not exposed to the top level and are not extracted by default, and also whether to add a ploc for the nets extracted. For clamp cells specified in the keyword

ESD_CLAMP_PIN_FILE, this keyword (and the ESD_LOCAL_NETS keyword) control this behavior, as follows:

0 - the feature is turned off (default)

1 - adds a ploc and extracts nets that connect 2 or more clamp instances.

2 - adds a ploc and extracts all nets that are connected to any clamp instance.

Optional. Default: 0

Syntax:

```
ESD_EXTRACT_CLAMP_NET [ 0 | 1 | 2 ]
```

ESD_GSR_OPTIMIZE

When set, activates ESD capacity improvement feature. The optimized flow generates a new GSR file that selectively removes ESD-irrelevant cells/layers to reduce design size while keeping acceptable accuracy. There are three methods used to improve design capacity:

- Remove non-ESD standard cells (for B2B/B2C/C2C/CD/R checks).
- Remove layers that reduce the number of ESD instances on each DEF file (for B2B/B2C/C2C/CD/R checks).
- Remove SPLIT_VIA_ARRAY or SPLIT_SPARSE_VIA_ARRAY settings (for non-CD checks).

When ESD_GSR_OPTIMIZE is set, the Totem session terminates at the beginning of the setup design stage and a new *esdGsr.gsr* file is generated. You must then rerun Totem with *esdGsr.gsr* to get an optimized design for ESD checking.

Note that since B2I/C2I/C2M/D2R checks need to access other instances that could be anywhere in the design, there is no design reduction for B2I/C2I/C2M/D2R checks. *Optional. Default: 0.*

Syntax:

```
ESD_GSR_OPTIMIZE [ 0 | RULE_CHECK | CD_CHECK |
BUMPCLAMP_R_CHECK ]
```

where

0 : optimization not activated

RULE_CHECK (or 1) : runs rule checks based on the rules specified in the ESD_RULE_FILE keyword

CD_CHECK (or 2) : optimizes current density checking

BUMPCLAMP_R_CHECK (or 3) : optimizes B2B/B2C/C2C checking

ESD_LOCAL_NETS

Specifies which nets are subject to ploc insertion and are extracted, along with ESD_CLAMP_PIN_FILE and ESD_LOCAL_NETS keyword settings. *Optional. Default: none.*

Syntax:

```
ESD_LOCAL_NETS {
    <netName1>
    <netName2>
    ...
}
```

ESD_MERGE_NETS

Merge nets for ESD analysis. *Optional. Default: none.*

Syntax:

```
ESD_MERGE_NETS {
<prime_net> {
<sub_net1>
<sub_net2>
...
}
....
}
```

Here, Totem will merge the sub_nets into the prime_net. The net names are in terms of flattened net list similar to VDD_NETS and GND_NETS

ESD_RULE_FILES

Allows you to specify one or more ESD rule file names, so that when executing 'perform esdcheck', if a rule file is not specified in the command, Totem picks up the rules specified in this keyword and executes 'perform esdcheck'. *Optional. Default: none.*

Syntax:

```
ESD_RULE_FILE <rule_file_name>
```

or

```
ESD_RULE_FILES {
    <fileName1>
    <fileName2>
    ...
}
```

ESD_SHORT_CLAMP_PIN

When set to 1, all pin nodes are shorted. When this keyword is set to 2, Totem shorts all pin node locations in the clamp file specified in the ESD_CLAMP_FILE GSR keyword. <layer> to short the clamp nodes on the specified layer in ESD_CLAMP_FILE.

Optional. Default: 0.

Syntax:

```
ESD_SHORT_CLAMP_PIN [ 0 | 1 | 2 | <layer> ]
```

where

0 (off): no clamp pin node shorting
1 (all): short all pin nodes
2 (loc): short all pin node locations as specified in clamp file

User has an option to change node shorting any time after extraction without the need to re-extract using the following steps:

a. Modify clamp pin setting with TCL command `'pfs add/delete/import clamp_pin ...'`.

b. `pfs export clamp_pin ?-esdCell []? **?-setupClamp?** ?-shortNode {0,1,2}? ...` where '-setupClamp' '-shortNode' are new options defined as:
   -setupClamp: Generate clamp DB (as done with 'perform esdcheck -setupClamp ...'). When -setupClamp is used, -esdCell is optional, not required.)

-shortNode: Node shorting option, same as ESD_SHORT_CLAMP_PIN above.

ESD_SIGNAL_NETS

Allows you to create a list of signal nets for ESD checking. Cell name definition is mandatory when the net is outside the top block or when the net is not defined as POWER/GROUND type in LEF/DEF. Otherwise, cell name is optional for the nets in top block.*Optional. Default: none*

Syntax:

```
ESD_SIGNAL_NETS {
   <netname1> <cell_name1>
   <netname2> <cell_name2>
   ...
  }
```

ESD_SIGNAL_NET_FILE

Allows you to specify a file containing a list of signal nets for ESD checking. When the net is in top block, cell name is optional and is required if the net is not in top block. *Optional. Default: none.*

Syntax:

```
ESD_SIGNAL_NET_FILE <filename>
```

where the file contains the names of signal nets, in the format:

```
<netname1> <cell_name1>
...
```

ESD_SUBSTRATE

Specifies substrate well-diode model characterization with user-input I-V curves. Key points on the I-V curve are defined in the GSR syntax. *Optional. Default: none.*

Syntax:

```
ESD_SUBSTRATE {
    <diode_model> area=<value> iv={<i1>,<v1> <i2>,<v2> ...}
 }
```

Example:

```
ESD_SUBSTRATE {
    diodeiv area=1E-11 iv={-10,-2 0,0 1,1E-6 2,1 3,2}
 }
```

Name Mapping Keywords

BUS_DELIMITER

Defines the characters used to delimit bus bits in the Totem database and GSR. *DMP compatible. Optional. Default: [ ]*

Syntax:

```
BUS_DELIMITER <delim>
```

Example:

```
BUS_DELIMITER ()
```

BUS_DELIMITER_STA

Defines the characters used to delimit the bus bits in STA files. *DMP compatible. Optional. Default: [ ]*

Syntax:

```
BUS_DELIMITER_STA <delim>
```

Example:

```
BUS_DELIMITER_STA ()
```

HIER_DIVIDER

Defines the character used to specify the hierarchy in the Totem database and GSR. *DMP compatible. Optional. Default: /*

Syntax:

```
HIER_DIVIDER <divider>
```

Example:

```
HIER_DIVIDER /
```

HIER_DIVIDER_STA

Defines the character used to specify the hierarchy in STA files. *DMP compatible. Optional. Default: /*

Syntax:

```
HIER_DIVIDER_STA <divider>
```

Example:

```
HIER_DIVIDER_STA /
```

MMX_XTOR_HIER_DIVIDER

If the hierarchical divider in the transistor name is not "/" and ".", use this keyword to specify the hierarchical divider to help Totem identify the transistor hierarchy.

Syntax:

```
MMX_XTOR_HIER_DIVIDER  <hierarchy_string>
```

Example:

If the transistor name in the Totem database is *IswArray\|ILSB0\|M15_2__rcx*, use the following setting to help identify the hierarchy:

```
MMX_XTOR_HIER_DIVIDER \|
```

NAME_CASE_SENSITIVE

Defines the name case sensitivity. If the value is set to 1, all *.lib, lef/def, spef/dspf, vcd*, and STA filenames are assumed to be case-sensitive. *Optional. Default: 1.*

Syntax:

```
NAME_CASE_SENSITIVE [ 0 | 1 ]
```

PIN_DELIMITER

Defines a special character to separate net or instance names from pin names in Totem working files. Any character can be used that is not reserved in LEF or used in pin names. *DMP compatible. Optional. Default: :*

Syntax:

```
PIN_DELIMITER <delim>
```

Example:

```
PIN_DELIMITER @
```

PIN_DELIMITER_STA

Defines the character between instance and pin names in STA files. *DMP compatible. Optional. Default: /*

Syntax:

```
PIN_DELIMITER_STA <delim>
```

Example:

```
PIN_DELIMITER_STA :
```

Warning and Error Message Keywords

ERROR_COUNTS

Specifies the maximum number of Error messages per type that are reported to *stdout*. *DMP compatible. Optional. Default: 3.*

Syntax:

```
ERROR_COUNTS <number_mess>
```

Example:

```
ERROR_COUNTS 10
```

ERROR_LOG_COUNTS

Specifies the maximum number of Error messages per type that are reported to *adsRpt/totem.err*. *DMP compatible. Optional. Default: 100.*

Syntax:

```
ERROR_LOG_COUNTS [ <number_mess> | 0 | -1 ]
```

where

<number_mess> : specifies the maximum number of messages to report per type.

0 : no messages reported

-1 : all messages reported

Example:

```
ERROR_LOG_COUNTS 100
```

WARNING_COUNTS

Specifies the maximum number of Warning messages per type to be reported to *stdout* and *adsRpt/totem.log*. *DMP compatible. Optional. Default: 3.*

Syntax:

```
WARNING_COUNTS [ <number_mess> | 0 | -1 ]
```

where

<number_mess> : specifies the maximum number of messages to output per type.

0 : no messages output

-1 : all messages output

Example:

```
WARNING_COUNTS 10
```

WARNING_LOG_COUNTS

Specifies the maximum number of Warning messages per type that are reported to *apache.warn.xxxx*. *DMP compatible. Optional. Default: 100.*

Syntax:

```
WARNING_LOG_COUNTS [ <number_mess> | 0 | -1 ]
```

where

<number_mess> : specifies the maximum number of messages to report per type.

0 : no messages reported

-1 : all messages reported

Example:

```
WARNING_LOG_COUNTS 10
```

Ignore Function Keywords

IGNORE_BPA_ERROR

When set, Totem reports MMX BPA errors and proceeds, instead of exiting the run. *Optional. Default: 0 (Off).*

Syntax:

```
IGNORE_BPA_ERROR [1|0]
```

IGNORE_CELLS

Specifies the list of cells to be ignored during importing DEF files (they will not be imported into the design DB). For example, filler cells contribute no IR-drop and therefore can be ignored. If a DEF file path is *not* provided, cells specified are ignored in all hierarchies in the design, within the top cell and blocks. Cells specific to a block to be ignored should have the block's DEF file path provided. Wildcards are supported, with "?" representing a single character and "*" representing any number of characters. *DMP compatible. Optional. Default: none.*

Syntax:

```
IGNORE_CELLS
{
    <cell_name1> ? <DEF_file_path> ?
    ...
}
```

Example:

```
IGNORE_CELLS
{
filler_cell2 /DEF/blockCDE.def
filler_cellA17
}
```

IGNORE_CELLS_FILES

Specifies a file that contains the list of cells to be ignored by Totem during data input. The file containing the list referred to must have only one entry per line. When used in conjunction with IGNORE_CELLS above, both lists are read in. *DMP compatible. Optional. Default: none.*

Syntax:

```
IGNORE_CELLS_FILES <Exclude_cells_file_pathName>
```

Example:
```
IGNORE_CELLS_FILES /home/users/customerA/cells_exclude
```

IGNORE_DEF_ERROR

Normally if there are DEF data errors, undefined vias, or DEF blocks without power and ground pins, the Totem session is terminated with an error report. If the keyword IGNORE_DEF_ERROR is set to 1, Totem reports the DEF errors and continues. *DMP compatible. Optional. Default: 0 (perform check).*

Syntax:
```
IGNORE_DEF_ERROR [ 0 | 1 ]
```

IGNORE_ESCAPE_CHAR

If IGNORE_ESCAPE_CHAR is set to true (default), the escape character (\) in names are ignored (removed). If the keyword is set to false, escape characters are kept in names. *DMP compatible. Optional. Default: true.*

Syntax:
```
IGNORE_ESCAPE_CHAR [ true | false ]
```

IGNORE_FILE_PREPARSE

Specifies whether the file is pre-parsed for syntax errors and/or missing information, prior to running Totem. *DMP compatible. Optional. Default: 0 (do not ignore).*

Syntax:
```
IGNORE_FILE_PREPARSE [ 0 | 1 ]
```

IGNORE_FILLER_DECAP_CELL_REPORT

Ignores filler/decap cells in output reports. To speed processing of designs dominated by filler/decaps, this feature keeps filler/decap cells in extraction/ simulation to avoid continuity issues, but ignores them in post-processing and reporting. *DMP compatible. Optional. Default: 0 (do not ignore)*

Syntax:
```
IGNORE_FILLER_DECAP_CELL_REPORT [ 0 | 1 ]
```

IGNORE_FLOATING_PAD

To ignore floating plocs from GUI and PG.ploc in the analysis. *Default : 0*

Syntax:
```
IGNORE_FLOATING_PAD [ 1 | 0 ]
```

IGNORE_GDSMEM_ERROR

Specifies whether parent *.lib/.lef* files for cells that have gone through *gds2def -m* are ignored if missing. *DMP compatible. Optional. Default: 0 (perform check).*

Syntax:
```
IGNORE_GDSMEM_ERROR [ 0 | 1 ]
```

IGNORE_INSTANCES

Specifies the list of instances to be ignored by Totem during data input (they will not be imported into the design DB). For example, the filler instances contribute no IR-drop and can often be ignored. For both flattened and hierarchical designs the

ignored instance names should be the exact names from the associated DEF_FILES keyword. If a DEF file path is *not* provided, instances specified are ignored in all hierarchies in the design, within the top cell and blocks. Instances specific to a block to be ignored should have the exact block DEF file path provided. *DMP compatible. Optional. Default: none*

Syntax:

```
IGNORE_INSTANCES {
  <exact_DEF_instance_name1> ? <exact DEF_FILES path1> ?
  ...
 }
```

Example:

```
IGNORE_INSTANCES
{
    filler_instance_name1 /DEF/blockA.def
}
```

IGNORE_INSTANCES_FILES

Specifies a file that contains the list of instances to be ignored during building connectivity and reporting.The file containing the list referred to must have only one excluded instance per line. For both flattened and hierarchical designs the ignored instance names in the file should be the exact names from the associated DEF_FILES keyword. *DMP compatible. Optional. Default: none.*

Syntax:

```
IGNORE_INSTANCES_FILES <exclude_instances_file_pathname>
```

Example:

```
IGNORE_INSTANCES_FILES /home/users/customerA/nets_exclude
```

IGNORE_LEF_DEF_MISMATCH

Defines the DEF import operation. When set to 0, the DEF import operation stops when a pin instance name in the NETS section of a DEF file is not defined for the corresponding cell in the LEF file. *DMP compatible. Optional. Default: 0 (do not ignore).*

Syntax:

```
IGNORE_LEF_DEF_MISMATCH [ 0 | 1 ]
```

IGNORE_LEF_MACRO

Specifies macros for which the LEF content is ignored (that is, the pin geometries), but still imports the DEF routing data. In the syntax below <macro1> is the DEF name for the block and <macro1_APACHECELL> is the LEF macroname for the same block's only child. *DMP compatible. Optional. Default: none.*

Syntax:

```
IGNORE_LEF_MACRO
{
       <macro1>
       <macro1_APACHECELL>
       ...
}
```

Example:

```
IGNORE_LEF_MACRO
```

```
Cornerblock_LL
Cornerblock_LL_APACHECELL
```

IGNORE_LIB_CHECK

Specifies whether a semantics check for unsupported attributes and groups in the *.lib* file is performed or not. *DMP compatible. Optional. Default: 0 (perform check).*

Syntax:

```
IGNORE_LIB_CHECK [ 0 | 1 ]
```

IGNORE_NETS_FILES

Specifies the list of nets defined in the SPECIAL_NETS section of DEF files to be ignored during building connectivity. The file containing the list referred to must have only one excluded net per line. When used in conjunction with IGNORE_NETS, both list will be read in. *Optional. Default: none.*

Syntax:

```
IGNORE_NETS_FILES <ignoreNets_file_pathName>
```

Example:

```
IGNORE_NETS_FILES /home/users/customerA/nets_exclude
```

IGNORE_ROUTE

The IGNORE_ROUTE keyword allows skipping certain geometries when DEF is merged from GDS, so that for each block and the top (pointed to by <def_file_path_name>) the geometries of layer <layer_name> and above are ignored while importing data. *DMP compatible. Optional. Default: None.*

Syntax:

```
IGNORE_ROUTE {
    <def_file_path_name>  <layer_name>
}
```

Example:

```
IGNORE_ROUTE
{
 user_data/DEF/MBIST_INSTA6.def   V1
 user_data/DEF/MBIST_MACRO2.def   M3
 user_data/DEF/MBIST_INSTB3.def   V2
 user_data/DEF/INSTCS.def  V2
}
```

In the example, for block MBIST_INSTA6, geometries of layers V1 and above are ignored. (That is, only M1 and below are kept.) For the top, layers V2 and above are ignored. Note that by this setting, block MBIST_ MACRO2 will still keep its own M2 and V2, even though its parent INSTCS 's M2 and V2 are ignored.

IGNORE_SHORT

If set to true, ignores all shorts in the design. *DMP compatible. Optional. Default: 1 (on).*

Syntax:

```
IGNORE_SHORT [ 1 | 0 ]
```

IGNORE_SIMTIME_CHECK

If turned on, turns off a normal validation check that the specified simulation time is reasonable. *Optional. Default: 0 (off).*

Syntax:

```
IGNORE_SIMTIME_CHECK [ 1 | 0 ]
```

IGNORE_TECH_ERROR

Controls checking for dielectric layer data. When set to 1, the lack of dielectric data in the Tech file *does not* cause a critical failure of the run; the run continues. *DMP compatible. Optional. Default: 0 (stop for dielectric data failure)*

Syntax:

```
IGNORE_TECH_ERROR [ 0 | 1 ]
```

IGNORE_UNDEFINED_LAYER

If DEF files have layers that are not defined in the Tech file or in LEF, Totem exits at the end of DEF import. To proceed, either fix the Tech file layer data or set the keyword IGNORE_UNDEFINED_LAYER to true (1), which means wires/vias on the undefined layers will be ignored. *DMP compatible. Optional. Default: 0 (off).*

Syntax:

```
IGNORE_UNDEFINED_LAYER [ 0 | 1 ]
```

IGNORE_UPF_PGARC

When full P/G arc data is not available in the UPF, you should turn on this keyword (default off). In this case UPF P/G arc detection is disabled, and file level P/G arcs defined in the custom LIB file are used. *DMP compatible. Optional. Default: 0 (off).*

Syntax:

```
IGNORE_UPF_PGARC [ 0 | 1 ]'
```

MISSING_VIA_CHECK_IGNORE_CELLS

Allows exclusion of specified instances in missing vias reports, rather than full GDS blocks. When this keyword is used, missing vias inside instances using the specified cellnames as masters are not reported. *Optional. Default: None.*

Syntax:

```
MISSING_VIA_CHECK_IGNORE_CELLS {
  <cellname_1>
  <cellname_2>
  ...
}
```

IGNORE_VP_CONTROL_ERROR

When set, allows Totem to continue when the VPC file has errors. The VP_CONTROL file describes control pin information for switched IP macros, for example, and may include more cell types than are used in design. When the keyword is set and VPC errors are encountered, such as when there is a mismatch between the design and the VP_FILE spec, a warning (DSG-201) is displayed. *Optional. Default: Off.*

Syntax:

```
IGNORE_VP_CONTROL_ERROR [ 0 | 1 ]
```

Place and Route Block Design Keywords

BLOCK_VCD_FILES

Specifies files and parameter values for VCD-based analysis. *Optional. Default: None*

Syntax:

```
BLOCK_VCD_FILES {
    VCD_FILE
    {
        <Hierarch_path/DEF_instance_name1> <VCD file>
        VCD_DRIVEN [ 0 | 1 ]
  ? FILE_TYPE [ VCD | FSDB | RTL_VCD | RTL_FSDB ]?
  ? FRONT_PATH <“redundant_path_string”> ?
  ? SUBSTITUTE_PATH <“<substitute_path_string>”> ?
  ? SELECT_RANGE <start_time> <end_time> ?
  ? SELECT_TYPE [ WORST_POWER_CYCLE | WORST_DPDT_CYCLE ]?
  ? START_TIME <start> ? ? END_TIME <end> ?
  ? TRUE_TIME [0|1] ?
  ? MAPPING <map_file> ?
  ? STA_VCD_FREQ_RATIO <ratio>?
}
```

where

<top_block_nameN> <VCD file>: specifies the hierarchical block name that matches that identified in the DEF and the absolute or relative path to the VCD or FSDB file

VCD_DRIVEN: when set, RTL VCD State Propagation honors the END_TIME option (default 0). If END_TIME is not specified, PowerStream uses the VCD end time.

FILE_TYPE [VCD | FSDB | RTL_VCD | RTL_FSDB] : identifies the type of input

FRONT_PATH “<redundant_path_string>” : specifies the string describing the VCD file hierarchical path of the instance. This string is then replaced by a substitute path string in order to match the path of the instance in the DEF.

SUBSTITUTE_PATH “<substitute_path_string>”: specifies the string describing the DEF file hierarchical path of the instance. This option can be used internally to substitute the VCD path (specified by FRONT_PATH) with the DEF path. (This option is not required for Totem to properly identify the equivalent instances and nets in the VCD and DEF. For top level VCD, this is automatically set to “ “, and for block levels, to the block name).

SELECT_RANGE: specifies the start_time and end_time for performing automatic critical cycle selection during power calculation. If START_TIME and END_TIME values are set to -1, the full VCD period is included in cycle selection.

SELECT_TYPE : default value WORST_POWER_CYCLE ranks cycles based on highest cycle power. Set to WORST_DPDT_CYCLE invokes DPDT (delta power/delta time) ranking of cycle selection, since large cycle-to-cycle changes in power level (high DPDT) may cause very high di/dt values, leading to large dynamic voltage drops.

START_TIME <time> : specifies VCD or FSDB file start time for power calculation. If both START_TIME and SELECT_RANGE are specified, SELECT_RANGE is ignored, so cycle selection is skipped. Default start time

= 0. If the specified START_TIME is within the dump-off range (where activity is not considered), the START_TIME is automatically reset to the start of the next dump-on time.

END_TIME <time> : specifies VCD or FSDB file end time for VCD-driven state propagation-based power calculation. For other flows, END_TIME is ignored, and it is determined by START_TIME + DYNAMIC_SIMULATION_TIME. Default end time is the end of the VCD/FSDB file.

TRUE_TIME : if set to 0, uses STA timing data and assumes no glitches; if set to 1, uses VCD switching and timing data; default = 0.

MAPPING <filename> : name of map file with RTL VCD instance names and corresponding DEF instance name.

STA_VCD_FREQ_RATIO - matches the VCD frequency to the STA frequency (the specified <ratio> = STA_freq/VCD_freq). When Totem reads VCD data to select cycles, it uses the clock frequency defined in the STA file to cut cycle frames from VCD. If this option is used, VCD data is processed according to the STA frequency and the specified ratio. Note that this option does not affect the frequency selection in simulation.

Example:

```
BLOCK_VCD_FILES
{
VCD_FILE
{
  top   vcd.1
  front_path "TOP/CHIP/"
  substitute_path ""
  true_time 1
  frame_size 7000
}
VCD_FILE
{
  top2   vcd.2
  front_path "TOP/CHIP/top2/"
  substitute_path ""
  true_time 1
  frame_size 6000
}
}
```

CELL_WELL_CAP_FILE

If CELL_WELL_CAP_FILE is defined for PAD/filler cells , Totem parses and marks well cap cells and sends them to the simulation input file *apache.well* to be included in dynamic simulation. The format of the well cap file is as follows:

`<cell_name> <num_arcs> {<Vdd_name> <Vss_name> <Cap> <Eff_R>} ...`

Optional. Default: none.

Syntax:

```
CELL_WELL_CAP_FILE <filename>
```

IGNORE_INSTANCES_ON_TOP_DEF_REGIONS

Can be used to specify the corners of top DEF regions in which to ignore instances. The regions must encompass the instances you want to ignore completely, otherwise they remain in the simulation. The units are um. *Optional. Default: None.*

Syntax:

```
IGNORE_INSTANCES_ON_TOP_DEF_REGIONS {
    <ll_x>  <ll_y>  <ur_x>  <ur_y>
    ...
    }
```

Example:

```
IGNORE_INSTANCES_ON_TOP_DEF_REGIONS {
    0 0 100 100
    150 150 200 250
}
```

POWER_ALLOW_MULTIPLE_STATE

In general, PowerStream and simulation only allow one output signal pin event in each cycle, and extra events are filtered out, including FF/Latches and memories without state definitions, as well as multiple-state cells. The settings are defined as follows:

0: filter all extra events

1: only allow extra events for custom state cells; for multiple-state cells all events are dumped into the scenario file, and no events are filtered out.

2: all cells are allowed to have multiple events.

DMP compatible. Optional. Default: 1.

Syntax:

```
POWER_ALLOW_MULTIPLE_STATE [0 | 1 | 2 ]
```

POWER_ALLOW_PER_PIN_IPF

When set to 0, scales multi-domain instance power by the total power of all the domains specified in INSTANCE_POWER_FILES. By default (1), the power of individual domains entered in the IPF is scaled. *Optional. Default: 1 - On.*

Syntax:

```
POWER_ALLOW_PER_PIN_IPF [ 0 | 1 ]
```

POWER_ANALYSIS_MODE

VCD event-based state-dependent power calculation mode can be invoked using this keyword set to ‘averaged’. Only events defined in a gate VCD/FSDB file are used as a basis for power calculation in ‘averaged’ mode, although multiple-block VCDs are supported. The default ‘averaged_fast’ mode uses the state with the highest power for power calculation. *DMP compatible. Optional. Default: averaged_fast.*

Syntax:

```
POWER_ANALYSIS_MODE [ averaged | averaged_fast]
```

SCAN_CONSTRAINT_FILES

Specifies one or more scan constraint files, and the associated scan pattern. If no pattern is specified, a default pattern of “01” is assumed. When AUTO_TRACING is specified, PowerStream finds all scan chains in the design and applies the specified pattern to those chains. *DMP compatible. Optional. Default: none.*

Syntax:

```
SCAN_CONSTRAINT_FILES {
    <scan_chain_file_1> <pattern_1>
    ? <scan_chain_file_2> <pattern_2>?
    ...
    ? AUTO_TRACING <pattern>?
}
```

Example

Multiple scan chains, each has its own pattern:

```
SCAN_CONSTRAINT_FILES {
    scan_chain_1.spc
    scan_chain_2.spc 101
    scan_chain_3.spc 0011
    ...
}
```

THERMAL_MODEL

For CTM (Chip Thermal Model) generation and power calculation, this keyword specifies the way leakage power is to be handled, using values 0, 1, 2, or 3 as follows:

0 - no temperature-dependent leakage power is considered (default). When the THERMAL_PROFILE keyword is specified, the value of THERMAL _MODEL is changed to 1.

1 - no temperature-dependent leakage power is considered for cells not defined in a user-specified leakage current file (using the APL_FILES GSR keyword)

2 - temperature dependency for cells not defined in a leakage file in APL_FILES is taken from temperature-dependent scaling factors for cells included in leakage current files in APL_FILES. All cells must be defined in *<cell>.cdev* files.

3 - uses LIB leakage for base leakage power calculation and APL leak data for deriving scaling factors to compute leakage powers at different temperatures

Optional. Default: 0.

Syntax:

```
THERMAL_MODEL [ 0 | 1 | 2 | 3 ]
```

VCD_X_LOGIC_STATE

Totem has the flexibility to handle X-states in VCD to meet requirements, based on the setting of VCD_X_LOGIC_STATE, as follows:

When set to -1, logic-to-X is ignored and X-to-logic toggles are considered (default).

When set to 0, X is always treated as 0.

When set to 1, X is always treated as 1.

When set to 2, both logic-to-X and X-to-logic are considered a toggle.

When set to 3, both logic-to-X and X-to-logic toggles are ignored.

Syntax:

```
VCD_X_LOGIC_STATE [-1|0|1|2|3]
```

VECTORLESS_BLOCK

The mixed mode flow supports the scenario in which the top design has VCD, but some of the blocks need to be simulated in vectorless mode. Using this keyword you can specify block names for the VCD that are unavailable. *DMP compatible. Optional. Default: none.*

Syntax

```
VECTORLESS_BLOCK {
    <Block1>
    <Block2>
    ...
}
```

GSR Macro Keywords

DEFINE

Defines a variable value, including a complete file path, to allow ease of changing between analysis runs, providing a convenient way of reusing and editing a file path. This keyword should be defined at the top of the GSR file.

The variable also can be defined in a TCL command file, before the 'import GSR' command, such as: ‘gsr define <user_variable_name> <value>’. *Optional. Default: none.*

Syntax:

```
DEFINE <user_variable_name> <value>
```

Example

```
DEFINE AAA /my/path/design_data/lef
DEFINE BBB /my/path/design_data/lef
LEF_FILES {
       $AAA/a1.lef
       $AAA/a2.lef
       $BBB/b1.lef }
```

Or, using a command file definition:

```
run.tcl
gsr define AAA /my/path/design_data/lef
gsr define BBB /my/path/design_data/lef
```

INCLUDE

Used to include/parse contents of other GSR files as if they were included where the “INCLUDE” keyword is. *Optional. Default: none.*

Syntax:

```
INCLUDE <GSR_filename>
```

Example

If a GSR file has the contents:

```
INCLUDE <GSR_fileA>
<gsr_keyword_1> <val_1>
<gsr_keyword_2> <val_2>
...
```

and the <GSR_fileA> contents are:

```
<gsr_keywordA> <valueA>
```

This is the same as having the following GSR contents:

```
<gsr_keywordA> <valueA>
<gsr_keyword_1> <val_1>
<gsr_keyword_2> <val_2>
...
```

Substrate Analysis Keywords

See section "Substrate Noise Analysis", page 13-427.

Pad, Power/Ground and I/O Definition Files

Unified Pad Input File Format

A single input file now can be used to specify all pad, power/ground and I/O input data previously provided in separate **.pcell, *.pad* or **.ploc* files. Separate sections of the file are identified for each of the three types of information with an '*' and a unique keyword, followed by standard data lines for that type of file, as follows:

```
*PLOC
    <.ploc file data>
    ...
*PCELL
    <.pcell file data>
    ...
*PAD
    <.pad file data>
    ...
```

The unified pad input file can be imported into Totem using the '`import pad`' command:

```
import pad unified_pad.txt
```

or using the GSR PAD_FILES keyword:

```
PAD_FILES {
    unified_pad.txt
}
```

Two other file formats, the *ploc_pss* file and *pad_pss*, can also be accepted in the unified file, but their data cannot be mixed with the other three types in the same file:

```
*PLOC_PSS
    <.ploc_pss file data>
    ...
*PAD_PSS
    <.pad_pss file data>
```

The PLOC_PSS keyword is used for the *.ploc* format with a package Spice subcircuit netlist. This is different from the typical *.ploc* format where the last column has the SPICE node name that hooks up to the individual *.ploc* node.

The PAD_PSS keyword is used with the *.pad* format with a package Spice subcircuit netlist.

The following is an example of a unified file that includes all currently-supported *.ploc*, *.pcell,* and *.pad* input formats.

```
--------------------------------------------------------------------------------
## Format for .ploc files:
*PLOC
# For original .ploc format with 5 items
# <pin_name> <x_loc> <y_loc> <layer> [POWER|GROUND]
    Vdd1      60.9    1214.67  M2 POWER
    Vss1      69.6    1214.67  M2 GROUND

# For original .ploc format with P/G_net name specified, 5 items
# <pin_name> <x_loc> <y_loc> <layer> <PG_net name>
    Vdd1      60.9    1214.67  M4 VDD1
    Vdd2      69.6    1214.67  M4 VDD2
    Vss       80      1214.67  M2 VSS

# For block pin voltage flow runs with 6 items
# <pin_name> <x_loc> <y_loc> <layer> [POWER|GROUND] <pin voltage>
    TVDD:0    2292.000 5110.110       M4      POWER  1.499
    TVDD:1    2292.000 5110.110       M2      POWER  1.499
    TVDD:2    2292.000 5110.210       M6      POWER  1.497
    TVSS:0    2292.000 10.210         M6      GROUND 0.045

# For specifying bump L, R, C data - 8 items
# <pin_name> <x_loc> <y_loc> <layer> <power|ground> <L_pH> <R_Ohms> <C_pF>
    Vdd_1   15.0    25.0   metal6  power  l=10   r=0.001 c=10
    Vss_1   1050.0 23.0    metal6  ground l=20   r=0.005 c=5

# PLOC format for substrate back contact modeling (see figure below)
#<name>             <x>        <y>    <layer>     <net>   <pkgpin>
PLOC_back_contact  1000.0    1600.0      BSC      VSS      pkg_back_contact
```

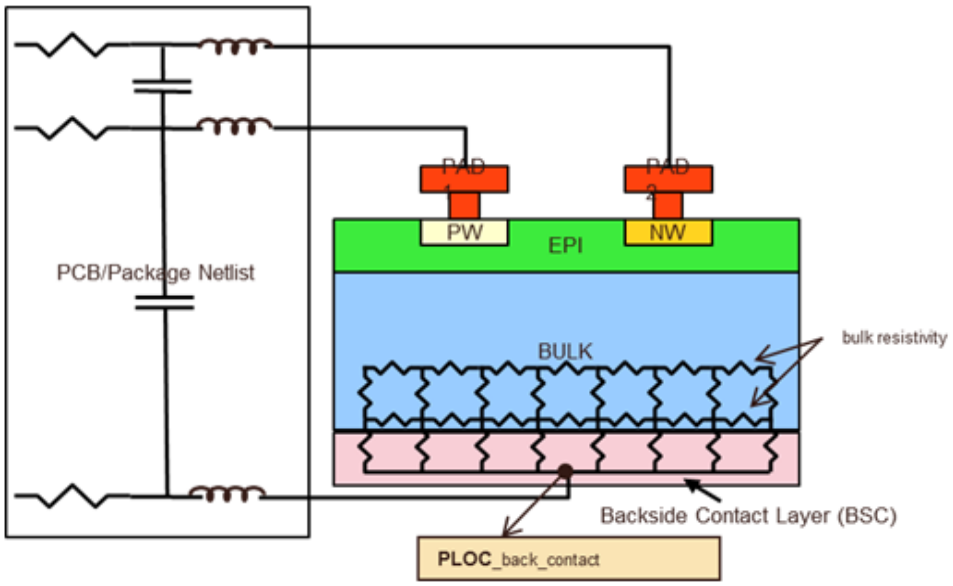


Figure C-4 Substrate back contact PLOC

```
## Format for .pcell files:
*PCELL

# Original .pcell file with 1 item
# <pad_cell macro name>
    T1VDDCOREA
    T1VSSA

# Formats that specify which pins are used in a pad macro - 2 items
# <pad_cell macro name> <macro pin name>
    VDD_PAD_MASTER vdd
    VDD_PAD_MASTER gnd
    VSS_PAD_MASTER vdd
    VSS_PAD_MASTER gnd

# Formats that specify connection locations and layers within
# a pad macro: 1-2 line format
# 1st : <pad_cell macro name> <x_macro_loc> <y_macro_loc> <layer>
# [Power|Ground]
# 2nd : <x_macro_loc> <y_macro_loc> <layer> [Power|Ground], if
# defined for the same pad_cell macro name
    PVDD1DGZ  18.0 2.4  METAL5 ground
        17.5  242.0  METAL3
    PVSS1DGZ  18.0 3.4  METAL5 POWER
        17.5  242.0  METAL3

## Format for .pad files:
*PAD

# Original .pad file format - 1 item
```

```
# <pad_instance name>
    PAD_INST1
    PAD_INST2

# To specify which pins are used in a pad instance - 2 items
# <pad_inst name> <inst_pin name>
    VDD_PAD_INST1 vdd
    VDD_PAD_INST1 gnd
    VDD_PAD_INST8 vdd
    VDD_PAD_INST5 gnd

# To specify connection loc and layers for a pad instance: 1-2 lines
# 1st: <pad_inst_name> <x_macro_loc> <y_macro_loc> <layer> [PWR|GRND]
# 2nd: <x_macro_loc> <y_macro_loc> <layer> [PWR|GRND], if defined
# for the same pad instance name
    pvdd1_io  17.50  -4.0  METAL6 POWER
        18.0  -4.0  METAL6
    pvss2_io 17.50  -4.0  METAL6 GROUND


## Format for .ploc files with a Package Spice Subckt (PSS)
# definition, in a separate input file
*PLOC_PSS
# Package-Spice-Subckt(PSS) with 6 items
# <pin_name> <x_loc> <y_loc> <layer> <PG_net name> <subckt node name>
    Vdd1      60.9    1214.67  M4 VDD  n1
    Vss1      69.6    1214.67  M4 VSS  n2
```

To set up your package parameters such that you can easily look at the effects of different package designs on power integrity, after extraction you can use the TCL command ‘setup pss’, which defines PLOC and package subcircuit files to be used in simulation, using the follow syntax:

```
setup pss -pad_file <filename> -subckt <pss_ckt_name>
```

To import pad data in a file, use the Totem command:

```
import pad <filename>
```

Notes on using the unified pad file:

1. The same format keyword may appear multiple times in a file (but grouping entries with the same format type is recommended to facilitate debugging).
2. Empty lines are ignored.
3. All existing *.ploc/.pcell/.pad* files are accepted and parsed as before.
4. For package hookup to Totem, if there are dangling nets in the package netlist, both ends need to be tied to ground through a high R, such as 10 MOhms.
5. Any file name (even extensions *.ploc/.pcell/.pad*) is allowed for unified pad input, as long as one or more of the four keywords and acceptable data are found inside the file. Otherwise, Totem displays an error during pre-parsing if it finds no *.ploc/.pcell/.pad* files and no files with the keywords *PLOC, *PCELL, or *PAD.
6. Use of PSS format excludes other formats in the same file. That is, whenever a PSS is specified with a GSR PACKAGE_SPICE_SUBCKT keyword, other pad input formats must be in a different I/O file and use a separate import command.

7. Formats for specifying a pad_cell macro or a pad_inst location within its bounding box are changed. The old format was in two lines as follows:

```
pvdd_master
17.50  -4.0  METAL6
```

The new format must have all data on one line:

```
pvdd_master  17.50  -4.0  METAL6
```

However, if additional connection locations are required for a pad_cell or pad_inst, its name is required on the first line only, and the location and layer information can be put on the second line, as in the previous format.

Individual Pad File Specification

For compatibility, individual **.pcell, *.pad* or **.ploc* files are still acceptable. If you provide the *.pad* or *.pcell* file, Totem automatically identifies the VDD/GND pad locations by determining each pad location from the *.def* file. However, if you provide the *.ploc* file, Totem can directly read in the pad locations from the file. You can also manually add power and ground pads by using the **Edit> Add Pad** command.

Note that multiple '`import pad *.pad/*.pcell`' commands are allowed.

Pad Cellname File (**.pcell*)

A. The following basic syntax can be used to describe pad cells and the associated pins in a *<design>.pcell* file:

```
<pad_cell_name_1> ?[<pin_name>| <pin_name> <layer_name>]?
...
<pad_cell_name_n> ?[<pin_name>| <pin_name> <layer_name>]?
```

Example

```
PADLZ55 VDD
PADLZ55 VSS
```

Note that if only the pad cellname is given in the above syntax, the P/G pins of each instance of cell are checked against the DEF routing. Anywhere that P/G routing touches a pin of the correct type a connection to the P/G source net is made.

B. You can specify the exact location of voltage sources within the cells. This improves current modeling accuracy, especially if the pad cells are large.

The alternative format for a *.pcell* pad file is as follows:

```
<master cell name>
<x source loc> <y source loc> <layer> <P/G pad type>
...
```

where

<master cell name>: specifies the master cell name (LEF macro name) indicating a pad cell (must be on a line by itself)

<x source loc> <y source loc> : x,y location of the master cell (referred to the macro's own coordinate system)

<layer> : source layer name

<P/G pad type> : the source P/G pad type

Example

```
PVDD1DGZ
```

```
17.5  242.0  METAL6 POWER
PVSS1DGZ
17.5  -4.0  METAL6 GROUND
```

There may be more than one set of location, layer, and type entries following a master cell name. These all imply sources specific to this pad macro cell.

For each instantiation of a master cell, its source locations are transformed with respect to the instance location. A file called *adsRpt/<design_name>.NEW.ploc* reports all such transformed sources from all instantiations.

Pad Instance Name File (*.pad)

A. The following syntax can be used to describe pad cell instances (not macro names) and the pins used in a *<design>.pad* file.

```
<pad_instance_name> ?[<pin_name> | <pin_name> <layer_name>]?
...
<pad_instance_name> ? [<pin_name> | <pin_name> <layer_name>]?
```

The following is an example of a list of pad instance name specifications (from the DEF file) in a *<design>.pad* file:

```
padring_1/ring/pad_gndc120
padring_1/ring/pad_gndc120a
padring_1/ring/pad_gndc120b
padring_1/ring/pad_vddc115
padring_1/ring/pad_vddc115a
padring_1/ring/pad_vddc115b
```

B. A *.pad* file also can use format ‘B’ as described in the previous *.*pcell* file section, by replacing the master cell name with the pad instance name. A file *adsRpt/<design_name>.NEW.ploc* is also generated.

Pad Location File (*.ploc)

If VDD/GND pad locations are already determined, they can be specified in a *<design>.ploc* file, which is formatted as follows:

```
<pad_name> <X_loc> <Y_loc> <metal name-DEF> [POWER|GROUND] <L-pH> <R-Ohms>
VSS1     6484.0 3761.0        metal4          GROUND       l=10     R=0.001
VSS2     6484.0 5141.0        metal4          GROUND       l=10     R=0.001
VDD1     320.0 1599.0         metal6          POWER        l=20     R=0.003
VDD2     6484.0 4001.0        metal6          POWER        l=20     R=0.003
VDD3     6484.0 3401.0        metal6          POWER        l=20     R=0.003
VSS3     6484.0 7961.0        metal4          GROUND       l=10     R=0.001
VSS4     320.0 3039.0         metal4          GROUND       l=10     R=0.001
VDD4     320.0 3159.0         metal6          POWER        l=20     R=0.003
VSS10    320.0 1299.0         metal4          GROUND       l=10     R=0.001
```

where <X_loc> and <Y_loc> are in microns, and L and R are specified pin/bump values, instead of general values from the ‘setup wirebond’ command. Unspecified pin/bump L/R values are taken from the ‘setup wirebond’ command values.

To facilitate PSS node hook-up with the PSS *.ploc* file, Totem creates a new file, *adsRpt/PG_simple.ploc,* in which each pad instance has just one ploc specified (instead of multiple plocs in *PG.ploc* file). You should manually change this *PG_simple.ploc* file, adding the PSS subckt nodes, and then re-run Totem with this modified *ploc* file. Note that the GSR keyword PRINT_ONE_PLOC_PER_PADINST must be set to 1 in order to create the special *adsRpt/PG_simple.ploc* file.

Note that when using the DEF_SCALING_FACTOR and LEF_SCALING_FACTOR GSR keywords, you do not need to scale ploc (x,y) values manually. Use the original unscaled PLOC file and Totem automatically places the pads in the correct locations in the scaled design.

Dual RLC specifications in PLOC

When RLC values are specified in the PLOC file as well as in the 'setup wirebond' command, the PLOC file values *override* the 'setup wirebond' values, which is indicated in the log file. Following is an example PLOC file in which the R value is given for each bump:

```
VDDC_1 561.40 3.64 M2 POWER R=100
VDDC_2 561.40 3.54 M2 POWER R=110
VSSC_1 561.40 1.40 M2 GROUND R=100
```

Following is an example of the 'setup wirebond' command:

```
setup pad -power -r 30 -ground -r 30
setup package -power -r 70 -ground -r 70
setup wirebond -power -r 50 -ground -r 50
```

For the above case, the log file contains the following messages:

```
** Pad/WireBond/Package RLC Setting Summary **
   Pad Vdd Resistance = 30 ohm
   Pad Gnd Resistance = 30 ohm
   Wirebond Vdd Resistance = 50 ohm
   Wirebond Gnd Resistance = 50 ohm
  *PIN<VDDC_1> will use the following R, I, C values.
     Vdd Resistance = 100 ohm. Vdd Induct = 0 pH. Vdd Cap = 0 pF
   *PIN<VDDC_2> will use the following R, I, C values.
      Vdd Resistance = 110 ohm. Vdd Induct = 0 pH. Vdd Cap = 0 pF
   *PIN<VSSC_1> will use the following R, I, C values.
      Gnd Resistance = 100 ohm. Gnd Induct = 0 pH. Gnd Cap = 0 pF
   Package Vdd Resistance = 70 ohms
   Package Gnd Resistance = 70 ohms
```

Pad PSS File

Hooking up distributed package SPICE subcircuit netlist nodes to the corresponding nodes on the chip can be complicated, especially if the x,y locations on the chip are unknown. In such cases, Totem can derive x,y locations from the *.pcell* file to use with the distributed package SPICE subcircuit netlist.

The unified pad file format described above can also include an enhanced format of pad cells (if no other types of pad definitions are included in the file). By listing the power/ground pin name inside the pad cell and the corresponding package subcircuit port name, the voltage sources can be hooked up appropriately. The keyword required within the unified pad file to recognize this format is *PAD_PSS. Totem automatically forms the connection without the requirement of XY location input. An example is shown below:

```
*PAD_PSS
#<pad inst name> <P/G pin name>  <package subcircuit port name>
PVDDSSTT_1 VDDC POWER1
PVSSDDRR_4 VSS GROUND4
```

The above file can have any extension and must be instantiated within the PAD_FILES { } section of GSR.

As described above, you can use the GSR keyword

```
PRINT_ONE_PLOC_PER_PADINST  1
```

along with the *.pcell* file to create a simplified *.ploc* file that contains one ploc node for every instantiation of pad cell defined in *.pcell* file. Totem creates a *adsRpt/PG_simple.ploc* file in which each pad instance has just one ploc specified. This facilitates package SPICE sub-circuit node hook-up by manually changing the file and rerunning Totem.

Library Technology Files

The Library Exchange Format, or *.lef* file, defines the IC process technology and the associated library of cell models. Each *.lef* file must be specified in the GSR keyword LEF_FILES on a separate line with its absolute or relative path from the Totem run directory. The first *.lef* file must contain technology layer and via information. The following is an example of a set of LEF files:

```
LEF/blocks.lef
LEF/abcd_13hd.lef
LEF/abcd_13hd_10_2a.lef
```

Design Netlist Files

The Design Exchange Format, or **.def* file, defines the elements of an IC design relevant to the physical layout, including the netlist and design constraints. Each *.def* file must be specified on a separate line in the GSR keyword DEF_FILES with its absolute or relative path from the Totem run directory, with the following format:

```
<path>/<blockname>.def block/top
...
```

where

block/top: indicates whether the block is a top-level block or sub-block of the top level block. The top-level block must be the last entry.

The following is an example of a set of DEF files:

```
DEF/abcd_13hd.def block
DEF/abcd_13hd_10_2a.def block
DEF/top_block.def top
```

Synopsys Library Files

The Synopsys Libraries files define the directory path for the set of Synopsys Library files (*.lib),* which are used for cell definition and power calculation.

LIB files and custom LIB files are specified using the APL configuration file keywords ‘LIB_FILES’, and ‘CUSTOM_LIBS_FILE’ and the GSR keyword ‘LIB_FILES’.

Custom LIB File Syntax

The following section shows the syntax and several examples of using a custom LIB file, which is used for overriding other LIB file data and also for providing supplementary information not included in a standard LIB file, such as definitions of P/G arcs in MVdd designs. The syntax for specifying custom library attributes is as follows (all entries are optional):

```
pgarc {
    <vdd_pin_name> <gnd_pin_name>
    ...
    }

pin <pin_name>
    {
        capacitance <pF>
        direction [ input | output | inout ]
        function "<function_type>"
        type [ clock | scan ]
        vector "<inputPinName> <clockPinName> : <pinName> [0 | 1] ..."
    }

cell <cell_name>
    {
        type [ comb | latch | ff | seq | memory ]
        subtype [input_pad | output_pad | inout_pad | clockgating | mux ]
        pin <pin_name>
        {
            capacitance <pF>
            direction [ input | output | inout ]
            function "<function_type>"
            type [ clock | scan ]
            vector "<inputPinName> <clockPinName> : <pinName> [0 | 1] ..."
        }
    }

library <library_name> {
    nomVoltage <volts>
    nomTemperature <temp>
    pin <pin_name> {
        ...
        }
    }
```

Example: In library *.lib*, pins named "CK" are of type 'clock':

```
library lib
{
    pin CK
    {
        type clock
    }
}
```

Syntax for specifying cell attributes:

```
cell <cell_name>
    {
    type <comb | latch | ff | seq | memory>
    subtype <input_pat | output_pad | inout_pad | clockgating | mux>
```

```
        pin <pin_name>
        {
            capacitance <pF> |
            direction <input | output | inout> |
            function <function_type> |
            type <clock | scan> |
            vector "<inputPinName> <clockPinName> : <pinName> [0 | 1]"
        }
    }
```

Example: In cell "A", a pin named "CK" is of type 'clock':

```
    cell A
    {
        pin CK
        {
            type clock
        }
    }
```

Example: Define vector for function "A B" of pin C:

```
    cell CEL
    {
        pin C
        {
            vector "A NA : B 0"
        }
    }
```

Syntax for specifying pin attributes:

```
    pin <pin_name>
    {
        capacitance <pF> |
        direction <input | output | inout> |
        function <function_type> |
        type [ clock | scan | signal | power | ground ]
        vector "<inputPinName> <clockPinName> : <pinName> [0 | 1]"
    }
```

Example: Pins named "CK" are of type 'clock':

```
    pin CK
    {
        type clock
    }
```

Example: specifying P/G arcs in multi-Vdd/Vss designs.

```
pgarc {      <- this statement covers all cells
   VDD VSS
   VDDL VSSA
 }
```

```
library LibABC {
  cell ram_mvdd { <- this statement is cell-specific
    pgarc {
      VDDPR VSS
      VDDAR VSS
      VDDNW VSS
   }
 }
}
```

P/G Arc Definitions in Custom LIB Files

To support cells with multiple Vdd *and* multiple Vss pins, you need to specify the P/G arcs, which defines the current path between each VDD node to the associated GND node pair. When there are multiple power domains and multiple ground domains, syntax to define the power domain arcs is required to provide the static and dynamic simulators with the current node pairs necessary to correctly assign currents.

The 'pgarc' object can be defined at the design level as follows:

```
pgarc {
    <vdd_pin_name> <vss_pin_name>
    ...
}
```

In this case the definition applies to all multiple power supply cells in the design. A pgarc can also be defined within a cell, in which case it is only applicable to that cell, as follows:

```
cell <cell_name> {
pgarc {
     <vdd_pin_name> <vss_pin_name>
    ...
      }
}
```

P/G arc definitions in a custom library are not needed for:

- 1 power and N ground pins (N = 1 or more). If one power goes to multiple grounds, the current for the power pin is assumed to split evenly between the ground arcs. Note that APL captures the VDD currents, not the VSS currents.
- N power and 1 ground when all N power pins have a profile in the current file (or power in LIB)

P/G arc definitions in a custom library *are* needed for:

- N power and M ground pins (N > 1, M > 1)
- N power and 1 ground pin, very common in memory IP, with some power pins that have no profile in the current file (or missing power in LIB), so that VDD current can be assigned realistically

For example, assume that you have a library in which most cells have three power domains, VDD, VDD2 and VDDL, and one ground domain, VSS. However, one cell, ram_mvdd, has three power domains, VDD, VDD2 and VDDL, and two ground domains, VSS and VSS2. In the ram_mvdd cell both VDD and VDDL use VSS as ground, while VDD2 has VSS2 as ground. The primary design pgarc relationships do not require a custom LIB file, since there is only one ground domain, but the pgarc relationships must

be defined in a custom LIB file since the ram_mvdd cell also has two grounds as well as three power domains. See the example custom file format following:

```
pgarc {
    VDD VSS
    VDD2 VSS
    VDDL VSS
}
cell ram_mvdd {
 pgarc {
    VDD VSS
    VDD2 VSS2
    VDDL VSS
    }
}
```

Then in the GSR file, the LIB_FILES keyword is required to specify the custom LIB filename (only one custom **.lib* file may be specified):

```
LIB_FILES {
<lib_filename> CUSTOM
...
}
```

If you are unsure as to which cells need to be specified in a custom library, then run the design through '`setup design`' and look at the data in the file *adsRpt/apache.refCell.noPGArc*. Totem detects cells with multiple ground pins that have no custom LIB file and reports them in the report *adsRpt/fapache.refCell.noPGArc*, along with all power and ground pins for each cell.

Example:

```
#<cell_name> <vdd_pin_names> <gnd_pin_names>
<SC_ANALOG> <VDD1A VDD2A> <VSS1A VSS2A>
```

Result Files

The Totem log and result files for power, EM, and IR / dynamic voltage drop are written to the *adsPower* and *adsRpt* directories. Refer to Chapter 6, "Results Evaluation", for more details on files created during Totem analysis.

Appendix D
TCL Command Reference

Introduction

This appendix describes the TCL interface and command set for running Totem. For descriptions of the GUI interface and menus, see Appendix B1, "Power Integrity GUI and Menus", Appendix B2, "Signal EM GUI and Menus", and Appendix B3, "ESD GUI and Menus".

Invoking Totem

Invoke Totem from a command line in your working directory using the UNIX syntax:

```
totem [ -b <cmnd_file>] [ -c <cmnd_file> ]
    [-f <cmnd_file>] [-i ] [-h ]
    [ -r <script> -t <time_delay_between_GUI_commands> ]
    [ -stack [<stacksize_MB> | unlimited ] ] [-lmwait]
```

where

-b <cmnd_file> : specifies a TCL script command file to run, with no Totem GUI displayed

-c <cmnd_file>: specifies a TCL script command file, and performs pre-parsing checks in the command file for both built-in and Totem-specific TCL commands, without actually executing each command, and reports warnings or errors accordingly.

-f <cmnd_file> : specifies a TCL script command file to run, with the Totem GUI displayed

-i : provides a TCL command line for text input, with no Totem GUI displayed

-stack [<stacksize_MB> | unlimited] : specifies the stack size in MB, or 'unlimited' (limited by the physical stack size available - default)

-h : displays the Totem invocation syntax on screen

-lmwait : allows a Totem run that has no available license to wait until one is available. Typically used so that invocation does not exit due to lack of a license during the initialization process. (The process is put into "sleep" mode and appears to be hung while waiting.)

-r <replay_script> : replays all TCL and GUI operations executed in a previous Totem GUI session. You can find replay scripts of previous Totem GUI sessions, which are automatically stored in the *adsRpt/Replay* directory with a timestamp of the start time identifying the different scripts. You can also use the GUI menu to bring up the replay configuration dialog using **Files -> Auto Play**.

-t : specified delay between GUI commands

When only 'totem' is invoked, the GUI is displayed by default.

TCL / Script Commands

The TCL commands in Totem enable you to run a complete session and save the results for subsequent viewing and debugging in the GUI. The TCL script file can be created either manually or by executing the desired commands and using the **Playback** menu command. The TCL script command file serves as a replay file in the GUI.

TCL Syntax Conventions

The syntax conventions used for defining Totem commands, options and keywords are as follows:

< x >	x describes a variable or value to be specified
[a | b | c]	one of a, b, or c must be selected
? x ?	x is an optional parameter
a b c d	a, b, c, and d parameters all must be specified
<x> ...	elements of the same type as <x> can be added
{ {a b c ...} {j k l ...} ...}	similar sets of elements may be added. Note that if only one set of elements is included, two sets of brackets may still be required
+ - * /	standard arithmetic operators for add, subtract, multiply, divide

TCL Command Summary

This section provides a list of the TCL commands available for running Totem from the command line, along with a summary of their syntax a brief description of each option. To see the full syntax for any command, use the 'help' command on the TCL command line.

Note: Pressing the TAB key returns the cursor to the TCL command line.

help

help

The 'help' command lists the available Totem TCL commands as follows:

cell swap *	characterize	condition	config	decap *
dump	eco	export	fao	generate
get	gsr	history	import	ircx2tech
marker	mesh	message	mmx	movie
perform	pfs	plot	print	probe
query	report	ring	route fix	save
select	setup	show	snapshot	window
zoom				

* commands only available for digital place and route blocks

The syntax and options available for any command or command group can be obtained by using the following invocation on the command line:

```
help <TCL_command_name>
```

The available Totem TCL commands are described briefly in the following section. Click on the command name in blue in the list above to jump to the description.

cell swap

cell swap <type_of_cells> <cell_name_list>

The 'cell swap' command allows high power cells to be swapped with lower power cells to improve performance in areas with a high voltage drop.

Please see Chapter 9, "Fixing and Optimizing Grid and Power Performance", for details about using this command.

characterize

characterize [apl | aplmmx] ? <options> ? <configFileName>

The 'characterize apl' command runs Apache Power Library (APL) characterization to generate dynamic Vdd current waveforms and characterize decap cells in the design.

a. The 'characterize apl' performs cell characterization using the syntax:

```
characterize apl ? -l <cellListFileName>?
            ?-o <outputFileName>? <configFileName>
```

where

-l <cellistFileName> : specifies the file containing cells to characterize.

-o <outputFileName> : specifies the results file, with default *<cell>.current*.

<confFileName> : specifies the configuration file

b. The 'characterize apl' command performs decap characterization when the following syntax is used:

```
characterize apl ?[-c | -p <decapFileName>]? ?-l <cellistFileName>?
?-o <outputFileName>? <configFileName>
```

where

-c : performs intrinsic / intentional decap characterization

-p <decapFileName> : specifies the file name containing intrinsic / intentional decap for characterization.

-l <cellistFileName> : specifies the cell list file

-o <outputFileName> : specifies the results file, with default *<cell>.cdev*.

c. 'characterize aplmmx' performs transistor-level characterization inside Totem.

```
characterize aplmmx <config file> ?[ -s | -n | -gds ]?
?-sig? ?-o <output_directory>?
```

For details on running characterization outside of Totem, see section "Characterization: Creating Electrical APLMMX Models", page 3-27.

condition

condition [set | get | unset] ? <options> ?

The 'condition' command sets, displays and unsets various conditions for post-DvD analysis (i.e., plot, print).

a. The 'condition set' command sets design parameters for post-DvD analysis:

```
condition set ?-xy <x1 y1 x2 y2>? ?-time <start end>?
    ?-type [ all |combinational| sequential| clock| memory]?
    ?-timestep <timestep>?
```

where

-xy <x1 y1 x2 y2> : specifies the x,y corner coordinates for a rectangular region (microns).
Note that if you specify 'condition set -xy', then a 'Zoom In/Out' command does *not change* the region, but if 'condition set -xy' is *not* specified, then a 'Zoom In/Out' command sets it.

-time <start> <end> : specifies the simulation start and end times, defined as a sub-range of the entire simulation (ps)

-type [all |combinational |sequential| clock| memory] : selects the instance type to be used.

-timestep <timestep> : specifies the timestep in picoseconds.

b. '`condition get`' examines conditions set previously using '`condition set`'.

```
condition get [-xy | -time | -type | -timestep ]
```

where

-xy: displays the selected rectangular region.

-time: displays the selected simulation time range,

-type: displays the selected instance type.

-timestep : displays the selected timestep

c. '`condition unset`' turns off design parameters set previously for post-DvD analysis:

```
condition unset ?-xy? ?-time? ?-type? ?-timestep? ?-all?
```

where

-xy : unsets the selected region.

-time :unsets the selected simulation time range.

-type : unsets the selected instance type.

-timestep :unsets the selected timestep.

-all : unsets all set conditions.

config

---

config [colormap | stack | viewlayer | viewnet | viewpad | viewlegend] ? <options> ?

The '`config`' command sets GUI options for reviewing, analyzing and debugging of Totem results. The individual commands are described below.

a. '`config colormap`' configures parameters for creating colormap displays

```
config colormap -map <colormmap ID> -min <value> -max <value>
    ?[ -wire | -instance ]? ?[ -percent | -absolute]?
    ?-values <list>? ?-enable [all | <list>]? ?-disable [all | <list>]?-
refvdd <value>
```

where

-map <colormap ID> : species any type of colormap. For a list of frequently-used colormap IDs, see the 'dump gif' command, page D-928.

-min <value> -max <value> : selects the minimum and maximum values to be displayed, in whatever units are used in the type of colormap selected

-wire | -instance : for IR voltage colormaps only, selects map display for wires or instances

-percent | -absolute : for IR voltage colormaps only, selects whether limits are in absolute or percentage values

-values: specifies values for every spin/entry, in fractional values (for example, specify 0.1 for 10%)

-enable: turns on the selected color (values in the list should be integers)

-disable: turns off the selected color (values in the list should be integers)

-refvdd: to config the IR map color drawing based on the specific reference voltage

Sample commands:

```
config colormap -percent -map ir -values { 0.1 0.2 0.3 0.4
    0.5 0.6 0.7 0.8 } -enable all -instance
config colormap -map IPM -values { 0.1 0.2 0.3 0.4 0.5
    0.6 .8 0.9 1 1.1 } -enable { 1 2 }
```

Note: see the example TCL script using the 'config colormap' command, page D-993.

b. `'config stack(size)'` allows resetting stack size on the fly during a Totem run, using the syntax:

```
config stack <size_in_MB>
```

where

<size_in_MB>: specifies memory available

c. `'config viewlayer'` configures parameters for viewing layers, using the syntax:

```
config viewlayer -name [all | viaonly | metalonly | <layerName>]
    -style [invisible | fill | outline]
```

where

-name : selects a layers view, either via only, metal only, or a specified layer name. Default - all.

-style : selects the style for viewing layers. Defaults - via = invisible, metal = fill, instance = outline.

d. 'config viewnet' controls display of all nets, including internal nets generally not of design interest. Using multiple commands, all nets can be turned off or on, and then specific nets can be turned on or off by net name, as desired.

```
config viewnet -name [all | <netName>] -mode [off | on]
```

where

-name : selects one of the nets to view, either all or a specified net. Default - all.

-mode : selects viewing nets or not. Default - on.

e. 'config viewpad' configures the parameters for viewing power/ground pads, using the syntax:

```
config viewpad -type [all | power |ground ] -mode [on | off]
```

where

-type : selects one of the pad types to view, or all . Default - all.

-mode : selects display of pads or not. Default - on.

f. 'config viewlegend -font and -fontsize' allows you to change the legend font and also the size between 7 and 32 in GIF images (default size is 10):

```
config viewlegend -font <family> -fontsize <size>
```

decap

decap [advise | place | fill | remove | fix | report] ? arg?

The ‘`decap`’ commands perform various types of modifications to decoupling capacitance to reduce dynamic voltage drop.

Please see Chapter 9, "Fixing and Optimizing Grid and Power Performance", for details about using these commands.

dump

dump [gif | fao | macro_pin_voltage | memory_via | metal_em_ratio | node_count | ptvoltage | res_network | sptemplate | mmx_pin_info] ? <options> ?

a. The ‘`dump gif`’ command writes out colormaps in GIF format. You may set the color mapping scale by using ‘`config viewlayer`’ and ‘`config viewnet`’ commands, or by using the ‘`import guiconf`’ command to load a previously saved GUI configuration file for a particular color map. Otherwise, it uses the system default settings. The command generally uses the same color map names as are on the GUI view and menu functions. The GIF graphic saved is exactly as shown on the display, with the directory path at the bottom and key parameters about the display conditions on the right side, as shown in Figure D-1.

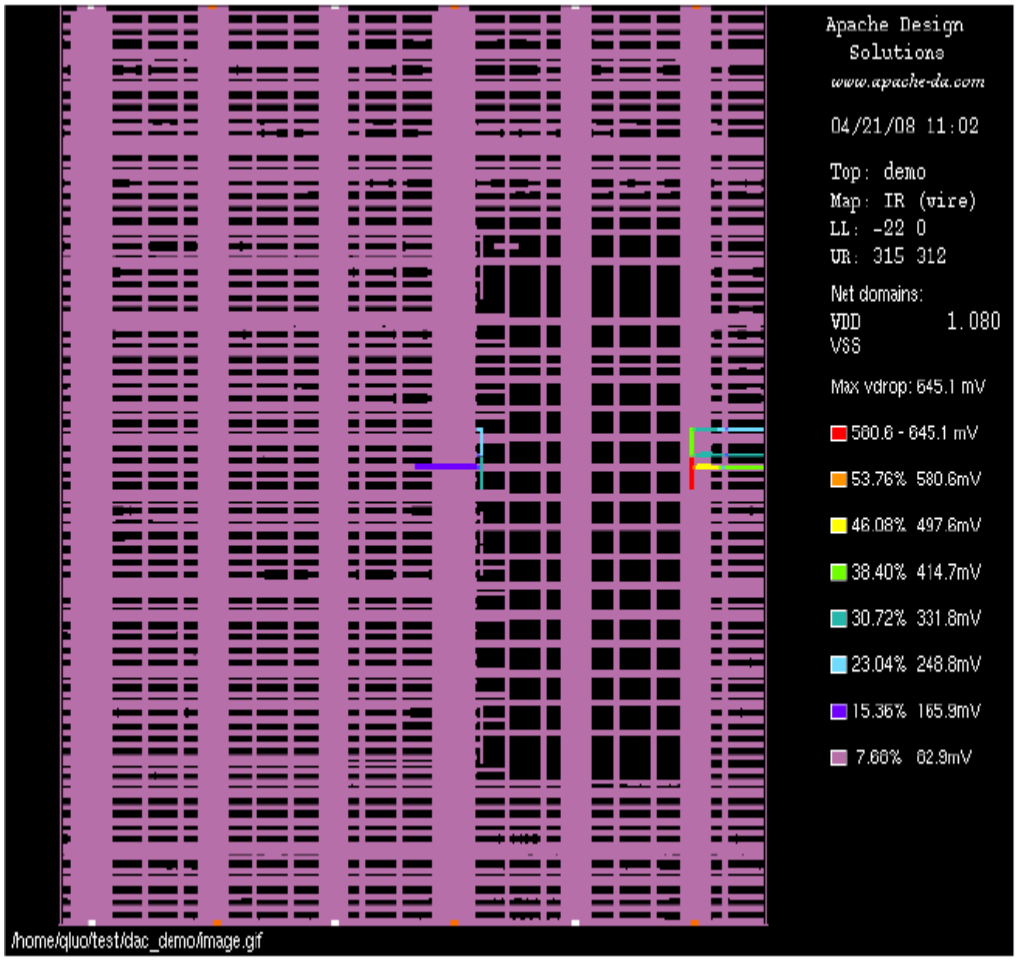


Figure D-1 Example dump of GIF file of Totem colormap

The command is:

```
dump gif -map <colorMapID>
    ? –background [<color name>| <rrggbb_num (Hex value)>]
    ?-o <outputFileName>? ?-nolegend?
```

where

-map <colorMapID> : specifies color map to save (and changes the view to that specified, if not presently displayed). <colorMapID> can be one of the following (case insensitive):

-background : controls the background color (default: black) when dumping GIF maps, using either the color name or the Hex number RGB representation. Examples:

```
dump gif -map SA -background red
dump gif -map SA -background #0000FF
```

(From 'View Results' buttons:)

SA: show all design features

IR: IR voltage analysis (static)

RES: total resistance

CUR: node current

DD: decap density

EM: electromagnetic analysis

TIR: transistor pin voltage drop

TRES: transistor pin resistance

TCUR: transistor pin current

TCAP: transistor pin capacitance

(From View menu commands:)

DIM: highlight disconnected instances
(**View -> Connectivity -> Highlight Disconnected Instance Map**)

DWM: highlight disconnected wires and vias
(**View -> Connectivity -> Highlight Disconnected Wire & Via Map**)

ITR: instance toggle rate
(**View -> Power Maps -> Toggle Map of Instances**)

TRD: toggle rate density (**View -> Power Maps -> Toggle Density Map**)

LPD: leakage power density
(**View -> Power Maps -> Leakage Power Density)**

PADC: pad current map (**View -> Power Maps -> Pad Current Map**)

PADV: pad voltage map (**View -> Power Maps -> Pad Voltage Map**)

TRM: total resistance (**View -> Resistance Maps**)

PRM: power (Vdd) resistance (**View -> Resistance Maps**)

GRM: ground (Vss) resistance (**View -> Resistance Maps**)

PIR: transistor pin voltage drop for static or dynamic analysis
(**View -> Transistor Pin Maps -> Voltage Drop Map**)

PRES: transistor pin resistance
(**View -> Transistor Pin Maps -> Resistance Map**)

PCUR: transistor pin maximum current
(**View -> Transistor Pin Maps -> Current Map**)

PMIN: transistor pin minimum voltage over dynamic simulation time

(**View -> Transistor Pin Maps -> Min voltage over simulation time**)

PAVG: transistor pin average voltage over dynamic simulation time

(**View -> Transistor Pin Maps -> Avg voltage over simulation time**)

PMAX: transistor pin maximum voltage over dynamic simulation time

(**View -> Transistor Pin Maps -> Max voltage over simulation time**)

maxTW: maximum Vdd-Vss value over time window
(**View -> Dynamic Instance DVD -> Max Vdd-Vss Over Time Window**)

minTW: minimum Vdd-Vss value over time window
(**View -> Dynamic Instance DVD -> Min Vdd-Vss Over Time Window**)

avgTW: average Vdd-Vss value over time window
(**View -> Dynamic Instance DVD -> Avg Vdd-Vss Over Time Window**))

minCyc: minimum Vdd-Vss value over clock cycle
(**View -> Dynamic Instance DVD**) **-> Min Vdd-Vss Over Whole Cycle**)

Decap: decap contributor (**View -> Decap Maps -> Decap Contributors**)

PDEV: pin-based device capacitance
(**View -> Transistor Pin Map -> PDEV**)

The following colormapIDs provide peak-to-peak DvD waveforms that have no GUI menu equivalents:

VDDP2P : peak-to-peak DvD waveform of Vdd = max Vdd(t) - min Vdd(t)

GNDP2P : peak-to-peak DvD waveform of Gnd = max Gnd(t) - min Gnd(t)

PGP2P: peak-to-peak DvD waveform of difference (Vdd-Gnd) = max (Vdd-Gnd)(t) - min (Vdd-Gnd)(t)

Examples of these peak-to-peak waveforms are shown in Figure D-2.

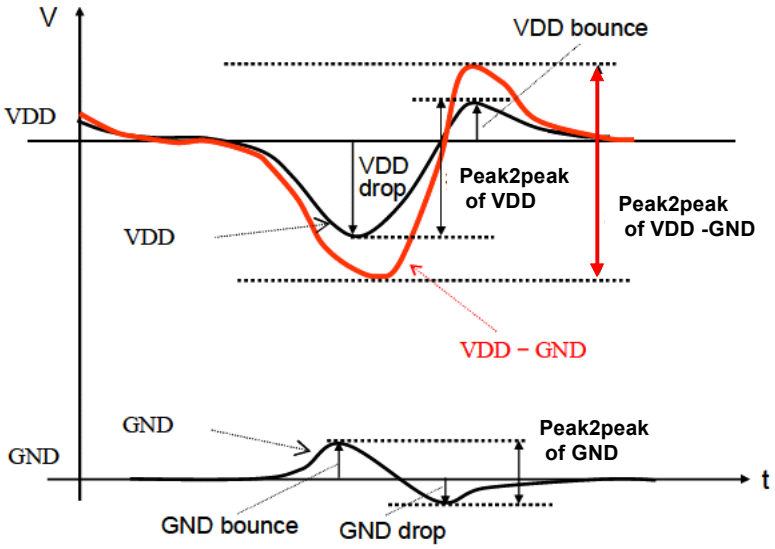


Figure D-2 Peak-to-peak DvD waveform definitions

DVP : Displays clamp differential voltage points map which maps the voltage difference between power and ground domain nodes. This map is a graphical display of “ESD Clamp Current/Voltage” list.

The following colormapIDs provide disconnected object maps that have no GUI menu equivalents:

DIM : Disconnected Instance Map

DWM : Disconnected Wire Map.

-o <outputFileName> : specifies the GIF output file name.

-nolegend : switches off the printing of logo / legend. Default - prints logo / legend

Note: see the example TCL script at the end of the chapter using the 'dump gif' command, page D-993**.**

b. `'dump gif -map ipcm -o <filename>`

Displays the instance peak current distribution in the design in a colormap display (in microAmps)

c. `'dump fao' <options>'`

Writes out Fix And Optimize-related TCL variable settings to screen or to a file, and either mesh or decap-related settings, or all FAO data (with no options).

```
dump fao ? [-mesh |-decap]? ?-o <tcl_file>?
```

where

-mesh | -decap : selects either mesh or decap-related settings to dump

-o <tcl_file> : specifies the name of the TCL file output.

d. The 'dump macro_pin_voltage' command reports voltages at the LEF macro pins, and the voltages at parent LEF pin locations of all instances of cells specified in GSR keyword GDS_CELLS (for digital place and route applications). This command can be executed after static/dynamic analysis using the syntax :

```
dump macro_pin_voltage ?-inst <inst_name>? ?-gds [0|1]?
?-o <filename>?
```

All arguments are optional. The basic command reports voltages for all macros in the design. The options are:

-inst : specifies the instance names for pin voltage reporting (default: all instances)

-gds [0| 1] : if set to 1, reports voltages on all GDS2DEF-created pins also (default: 0)

-o : specifies an output filename (default file: *.apache/.macropinvoltage*)

e. The 'dump memory_via' command can be used to report via information to the file specified after extraction, using the syntax:

```
dump -memory_via -o <file_name> -cells { <cell_list> }
```

where

-o <file_name> : specifies the output filename

-cells { <master_cell_list> } : specifies the master cell names to have instances listed

The format of the memory via information file is as follows:

```
<mem_master_name> <mem_inst1_name> <inst1_bbox> <total_inst_current>
<via_model1_name> <num_of_vias> <total_cut_area> <net_name>
<via_model2_name> ...
<mem_master_name> <memory_inst2_name> <inst2_bbox> ...
```

The above is repeated for each memory instance in the design as specified.

f. The 'dump node_count' command reports the node profile for a design, , using the syntax:

```
dump node_count -o <output_file>
```

Note that layers with less than 1% of the wires are ignored by the command. A sample node count output report follows:

```
Layer Based Profile(for the fullchip):

#Layer       Nodes
METAL1  0.237805 M
METAL2  0.026096 M
METAL3  0.147819 M
METAL4  0.193270 M
-------------------------
Total   0.604990 M

Total Number of Nodes           = 0.77 M
Die Area                        = 24.60 sqmm
Avg Number of Nodes per Sqmm    = 0.03 M/sqmm

Master Cell based profile for GDS2DEF cells: Summary
#Cell-Name    Num_insts    Avg_nodes_per_inst     Tot_nodes
   cell_50      1            0.109984 M          0.109984 M
   cell_52      1            0.087503 M          0.087503 M
   cell_30      1            0.001403 M          0.001403 M

Master Cell based profile for block DEF cells: Summary
#Cell-Name     Num_insts    Avg_nodes_per_inst    Tot_nodes
   cell_35      1             0.109648 M         0.109648 M
   cell_31      1             0.103521 M         0.103521 M

Cell: cell_30
Number of instances = 1
-------------------------------------------------------------
Layer Nodes_per_inst Total_nodes %of full-chip metal Node
count
METAL3  0.000856 M      0.000856 M      0.11
METAL4  0.000547 M      0.000547 M      0.07
-------------------------------------------------------------
Total   0.001403 M      0.001403 M      0.18
-------------------------------------------------------------
```

g. The ‘dump ptvoltage’ command displays, or writes out to a specified file, the PrimeTime TCL commands for each instance, for feedback to timing using PT. By default the average Vdd over the timing window (avgTW) is written for every instance. The syntax is:

```
dump ptvoltage -o <outFile> ? [ -aveTW | -maxTW | -minTW | -mincyc |
    -include_missing_TW [ minCyc | nominal ] ] ?
```

where

-o <outFile> : specifies name of output file

-aveTW : average Vdd over the timing window

-maxTW : maximum Vdd over the timing window

-minTW : minimum Vdd over the timing window

-mincyc : minimum Vdd over the clock cycle

-include_missing_TW : handles the case of missing timing windows, as follows:

- minCyc - the “minimum voltage over whole cycle” value is used for all TWs

- nominal - the nominal voltage for that cell (from the DB) is used.

If '-include_missing_TW' is not used, the command works as usual, which means instances with no timing windows are ignored (not included).

h. The ‘ dump res_network’ command creates a report on wires and vias in the design. A configuration file can be specified that contains the fields in the full dump report that are requested. The command syntax is as follows:

```
dump res_network -o <output_file> ? -compress ?
-exclude_region <region_file>?-box <llx> <lly> <urx> <ury>
?-nets <net1,net2,net3…>?-nets_file <file_name>?
-layers <layer1,layer2,layer3…> ? ?-conf <config_file>?
?-ignore_layers <layer1,layer2,layer3…>?
```

where

-o : specifies the output filename

-compress: saves the output in gzipped (*.gz) format

-nets : specifies the nets for which report needs to be created.

-nets_file : specifies a file that list out the net names for which the report have to be created.

-layers : specifies the layers for which report needs to be created.

-ignore_layers : specifies the layers to be ignored from 'dump res_network' output file.

-conf : the configuration file should contain a list of keywords representing the fields desired in the report for 'Wires', 'WireSegments' and 'Vias', as a subset of the full syntax, as follows:

WIRE $WIRE_ID $LAYER $NET_NAME $LLX $LLY $LRX $LRY $URX $URY $ULX $ULY $WIRE_WIDTH $RESISTANCE $CURRENT_ORIENT

WIRESEGMENT: $WIRESEG_ID $LAYER $NET_NAME $WIRE_WIDTH $STARTX $STARTY $ENDX $ENDY $AVG_I $AVG_ILIMIT $AVG_EM $RMS_I $RMS_ILIMIT $RMS_EM $PEAK_I $PEAK_ILIMIT $PEAK_EM $RESISTANCE $RES_ID $CURRENT_DIR

VIA $VIA_ID $LAYER $VIA_NAME $NET_NAME $CX $CY $CUT_NUM $CUT_WIDTH $CUT_HEIGHT $RESISTANCE $RES_ID $AVG_I $AVG_ILIMIT $AVG_EM $RMS_I $RMS_ILIMIT $RMS_EM $PEAK_I $PEAK_ILIMIT $PEAK_EM $CURRENT_DIR $VIA_RULE_NAME $LANDING_TYPE $COVERAGE_TYPE $CONNECTIONS $WIDTH_ABOVE $WIDTH_BELOW

The configuration file keywords are described below:

For wires:

$WIRE_ID - unique (internal) integer identifier for each wire

$LAYER - layer name that this wire is on

$NET_NAME - net name that contains the wire

$LLX $LLY

$LRX $LRY - coordinates defining the wire geometry

$URX $URY LL - lower left, LR - lower right

$ULX $ULY UL - upper left, UR - upper right

$WIRE_WIDTH - width of wire in um

$RESISTANCE - wire resistance in ohms

$CURRENT_ORIENT - wire orientation (based on current flow): h = horiz, v = vert

For wire segments:

$WIRESEG_ID - unique identifier for wire segment of the form X_Y, where X is the WIRE_ID for the wire containing this segment, and Y is the index for the wire segment, starting from 0 for each wire.

$LAYER - layer name that this wire segment is on

$NET_NAME - net name that contains the wire segment

$WIRE_WIDTH - width of the segment in um

$STARTX, $STARTY $ENDX, $ENDY - coordinates of the wire segment (spine)

$AVG_I - current through this segment during EM_AVG analysis

$AVG_ILIMIT - current limit for this segment for EM_AVG analysis

$AVG_EM - EM percent for EM_AVG analysis (I/ILIMIT)

$RMS_I - current through this segment during EM_RMS analysis

$RMS_ILIMIT - current limit for this segment for EM_RMS analysis

$RMS_EM - EM percent for EM_RMS analysis (I/ILIMIT)

$PEAK_I - current through this segment during EM_PEAK analysis

$PEAK_ILIMIT - current limit for this segment for EM_PEAK analysis

$PEAK_EM - EM percent for EM_PEAK analysis (I/ILIMIT)

$RESISTANCE - wire segment resistance in ohms

$RES_ID - unique (internal) integer id for this segment's resistor

$CURRENT_DIR - actual current direction through this wire segment: u = up, d = down, l = left, r = right

For Vias:

$VIA_ID - unique (internal) integer identifier for each via

$LAYER - cut layer name for this via

$VIA_NAME - via model name that applies to this via

$NET_NAME - net name that contains the via

$CX, $CY - X,Y location of the via's center point

$CUT_NUM - number of via cuts grouped together

$CUT_WIDTH - width of the cut rectangle for this via

$CUT_HEIGHT - height of the cut rectangle for this via

$RESISTANCE - via resistance in ohms

$RES_ID - unique (internal) integer id for this via's resistor

$AVG_I - current through this via during EM_AVG analysis

$AVG_ILIMIT - current limit for this via for EM_AVG analysis

$AVG_EM - EM percent for EM_AVG analysis (I/ILIMIT)

$RMS_I - current through this via during EM_RMS analysis

$RMS_ILIMIT - current limit for this via for EM_RMS analysis

$RMS_EM - EM percent for EM_RMS analysis (I/ILIMIT)

$PEAK_I - current through this via during EM_PEAK analysis

$PEAK_ILIMIT - current limit for this via for EM_PEAK analysis

$PEAK_EM - EM percent for EM_PEAK analysis (I/ILIMIT)

$CURRENT_DIR - actual current direction through this via: u = up, d = down

$VIA_RULE_NAME - via landing rule name (if any) associated with this via

$LANDING_TYPE- via landing type (if any) of this via

$COVERAGE_TYPE- via cover type (if any) of this via

$CONNECTIONS - outputs a list of wire segment id's that are attached to this via

$WIDTH_ABOVE - width of the wire attached to the top of this via

$WIDTH_BELOW - width of the wire attached to the bottom of this via

An example output file header would be as follows:

```
# For Wires
# <ID> <layer> <net> < ll(x y) lr(x y) ur(x y) ul(x y) > <width>
    <resistance><current_dir h/v >
# For Wire-segments
# <ID> <layer> <net> <start(x y) end(x y) (um)> <direction(l|r|u|d)>
    <current(A)> <em_limit(A)> <em(%)> <resistance(ohm)>
# For Vias
# <ID> <layer> <via_name> <net> <coord(x y) (um)> <cut_#>
    <cut_width(um)<cut_height(um)> <resistance(ohm)> <current_dir(u|d)>
    <current(A)> <em_limit(A)> <em%>  <rule-name> <Landing Type>
    <Wire-segid#1> <Wire-segid#2> ...  <Wire-segid#N>
```

i. The ‘dump sptemplate’ command writes out the state propagation constraint file template. By default primary input pins and clock roots are included in the file. The syntax is:

```
dump sptemplate ? -all ?
```

where

-all : specifies that register outputs are also included in the constraint file

j. The ‘dump mmx_pin_info’ command writes out transistor pin information within a selected x,y region. The syntax is:

```
dump mmx_pin_info ?-avg_volt_tw <tstart> <tend>? -o <output_file>
    ?-box <llx> <lly> <urx> <ury>? ?-report_disconnect? ?-wstpgarc ?
    ?-j <thread>? ?-inst_file <list>? ? -limit <rpt_entries>
```

where

-avg_volt_tw <tstart> <tend>: specifies the timing window start and end times (ns)

-o <outfile> : specifies the name of the output file. If no output file is specified, a summary is displayed in the GUI and written to the log file.

-box <llx>, <lly>, <urx>, <ury>: specifies the lower left and upper right corner coordinates for the region of interest.

-report_disconnect: dumps out information if the transistor pin is disconnected from P/G grid

-wstpgarc: generates subckt-based worst case Vdd-Vss reports in MMX-based transistor analysis

-j <threads> : number of threads for multi-CPU machines. Default is 1.

-inst_file : specifies a list of leaf instances and subckts whose Vdd-Vss values are to be found. If not given, it finds all instances subckts.

-limit : limits the number of entries in the output report

The inst_file contents format is:

```
<leaf_instance_name> <subckt_name_pattern>
```

For example:

```
adsU1 X112.X3.X0*
adsU1 X614.X3.X0*
```

An example of a subckt dump command output is:

```
#<InstName> <Subckt Name> <Power Domain> <Ground Domain>
```

```
            <Worst VDD-VSS> <Time_ns>  <VDD> <VSS> <VDD Node> <VSS Node>
adsU1 X112.X3.X0 VDD VSS 1.095690 1060.00000 1.138673 0.043021 7546 203793
adsU1 X112.X3.X0.X14 VDD VSS 1.03234 1060.0000 1.130435 0.09809 37273 201343
The output file format without the '-avg_volt_tw' option is as follows:
#Power(Current) Summary within region LL: (1000.0000 250.0000) UR:
(2000.0000 1100.0000)
#Domain Name Total No.Pin     Total Power / Effective Current
#-------------------------------------------------------------
#VDD          51118              6.1565e-03 W
#VSS          49004              -3.4196e-03 A

#<InstName:PinName> <Effective Current(A)> <PRatio> (<llx lly>) (<urx ury>)
<TransistorName>
adsU1:VDD.gds1 1.0321e-04 2103 (84.8250 62.6600) (85.0250 62.8600) X0.M5
adsU1:VDD.gds379 9.3990e-05 4263 (75.9250 60.4700) (76.1250 60.6700)
X0.X15.X31.M91
...
```

where '(<llx lly>)' and '(<urx ury>)' represent the lower left and upper right x,y corners of the pin geometry.

The output file format, without the '-avg_volt_tw' option, but with the '-report_disconnect' option, is as follows:

```
#<InstName:PinName> <Effective Current(A)> <PRatio> (<llx lly>) (<urx ury>)
<TransistorName> <Disconnected>
adsU1:VDD.gds1 1.0321e-04 2103 (84.8250 62.6600) (85.0250 62.8600) X0.M5 N
adsU1:VDD.gds379 9.3990e-05 4263 (75.9250 60.4700) (76.1250 60.6700)
X0.X15.X31.M91   N
...
```

The output file format using the '-avg_volt_tw' option is as follows:

```
#Time Unit: ps
#Time window start time: 0 ps
#Time window end time: 2000 ps
#<inst_name:pg_pin_name> <x_loc> <y_loc> <min_volt_tw@time>
<max_volt_tw@time> <avg_volt_tw> <xtor_name>
adsU1:VDD 82.8900 62.7600 1.7940@845 1.8000@0 1.7982 X0.M3
adsU1:VDD.gds1 84.9250 62.7600 1.7925@910 1.8000@0 1.7980 X0.M5
adsU1:VDD.gds2 11.8500 66.0000 1.7988@230 1.8000@0 1.7997 X0.X8.M3
adsU1:VDD.gds3 21.3250 18.7350 1.7960@350 1.8000@0 1.7990 X0.X9.X0.M22
...
```

where '<x_loc> <y_loc>' represent the center x,y coordinates of the pin.

In GUI mode, if the '-box' option is not specified, the region is based on the command 'condition set' (highest priority) and then the GUI selects the window region.

eco

eco [add [clamp_cell | clamp_instance |decap | pad | strap | stackvia | via switch]] |
[delete [clamp_instance | pad | strap | switch | via]] ? <options> ?

The ‘eco’ command defines changes to the design database (GUI-based operation only), which is equivalent to GUI ‘what-if’ operation.

a. ‘eco add clamp_cell’ adds or updates a clamp cell for ESD checking

```
eco add clamp_cell <cell_name> -width <W> -height <H>
    -vdd_pin <pinName_v1 pinName_v2 ...>
    -gnd_pin <pinName_g1 pinName_g2 ...>
```

where

-width <W>: specifies the cell width

-height <H>: specifies the cell height

-vdd_pin : specifies power pin name(s)

-gnd_pin: specifies ground pin name(s)

If the specified <cell_name> is previously given, the cell's properties (width, height, vdd/gnd_pin) are updated accordingly.

Note that an existing cell in the original design cannot be updated.

b. ‘eco add clamp_instance’ adds or updates a clamp instance for ESD checking.

```
eco add clamp_inst  <inst_name> -cell <cell_name>
    -pin <p1 p2...> -net <net1 net2...> -origin <x y>
    ?-orient [N |S |E |W |FN |FS |FE |FW ]?
```

where

-cell <cell_name>: specifies the master cell name of the instance

-pin : specifies the pin name(s) to be assigned net(s)

-net : specifies the net name of each pin

-origin <x y>: specifies the location of the instance

-orient : specifies the orientation of the instance relative to the master cell orientation: North, South, East, West, Flipped North, Flipped South, Flipped East, Flipped West. The default is N.

If the specified <instance_name> is previously given, the instance's properties (cell, pin, net, origin, orientation) are updated accordingly.

Note that the master cell of the added instance can be an existing cell or an ECO-created clamp cell. An existing instance in the original design cannot be updated.

c. ‘eco add decap’ adds decap cell as specified. Returns the added decap name. The syntax is:

```
eco add decap -metal <metalLayer> -decap <cellName>
    -power <x y> -ground <x y>
```

where

-metal <metalLayer> : specifies the metal layer name.

-decap <cellName> : specifies the decap cell name.

-power <x y> : specifies the decap insertion point for power.

-ground <x y> : specifies the decap insertion point for ground.

d. ‘eco add pad’ adds a single power/ground pad <metalLayer> at location x, y. Returns the added pad name. See also ‘fao add pad’ for adding a group of pads over an area. The syntax is:

```
eco add pad -metal <metalLayer> -type [power | ground] -x <x> -y <y>
```

where

-metal <metalLayer> : specifies the metal layer name.

-type [power | ground] : specifies the power or ground pad to add.

-x <x> : specifies the x location of the insertion in um.

-y <y> : specifies the y location of the insertion in um.

e. ‘`eco add strap`’ adds a strap (wire) as specified. Returns the ID of the added wire. The syntax is:

```
eco add strap -metal <metalLayer> -type [power | ground]
    ? -net <netName> ?
    -[horizontal | vertical] -start <x y> -end <x y>
    -width <width> ? -novias ?
    ? -toplayer <layerName>? ? -bottomlayer <layerName>?
```

where

-metal <metalLayer> : specifies the metal layer name.

-type [power | ground]: specifies the power or ground strap to add.

-net : specifies one power or ground net (-type then not required)

-[horizontal| vertical] : specifies the direction of the wire to add

-start : specifies the starting coordinates of the strap (lower left corner)

-end : specifies the end coordinates of the strap (upper right corner)

-width <width> : specifies the wire width

-novias : specifies no vias are placed associated with the strap

-toplayer <layerName>

-bottomlayer <layerName>

f. `eco add stackvia | via’` adds a via between layers as specified.

```
eco add [ stackvia | via ] -toplayer “<netname> <topLayerName>”
    -bottomlayer “<netname> <bottomLayerName>” ? -no_adjust
-x <x> -y <y> ?-viamodel <viaName>?
```

where

-toplayer “<netname> <topLayerName>” : specifies the net name and the top layer name for insertion.

-bottomlayer “<netname> <bottomLayerName>”: specifies the net name and the bottom layer name for insertion.

-viamodel <viaName>: specifies the via model.

-x : specifies the x location for insertion.

-y : specifies the y location for insertion.

-no_adjust: by default, adds via/stackvia at the center of the overlap; when this option is specified adds via/stackvia at a customized location based on -x/-y settings.

g. ‘`eco add via`’ adds a via with the specified characteristics. The syntax is:

```
eco add via -toplayer "<netname> <topLayerName>" -
bottomlayer "<netname> <bottomLayerName>" -x <x> -y <y>
-viamodel <viaName> -no_adjust
```

where

-topLayerName>/ -bottomlayer : specifies the layers the via connects and the associated net names.

-x/-y : specifies the coordinate location of the via

-viamodel : specifies the via model to be used

--no_adjust : specifies that more than one via can be placed between the specified layers.

h. ‘`eco add switch’` adds a switch instance with the specified characteristics. The syntax is:

```
eco add switch <instName> -master <cellName>
    -connect {<pin1 net1 pin2 net2 ...>} -x <x_loc> -y <y_loc>
    ? -count <xcount> <ycount> -pitch <x_pitch> <y_pitch> ?
    ? -orient [ N|S|E|W|FN|FS|FE|FW ]?
```

where

<instName> : specifies the instance name of switch (for placing an array of switches using ‘-pitch’ and ‘-count’ options, see the naming format below)

-master <cellName> : specifies the name of master switch cell

-connect {<pin1 net1 pin2 net2 ...>} : specifies the associated pin and net connections for switches to be added

-x <x_loc> -y <y_loc> : identifies the lower left corner x,y location of the single switch or the first switch in the array

-count <xcount> <ycount> : defines an array of identical switch instances placed adjacent to each other in the x and/or y direction. Switches in the specified array automatically are assigned names of the form ‘<instName>_a_b’, where a and b are a series of consecutive integers starting with the first switch named ‘<instName>_1_1’ at location <x_loc> <y_loc>.

-pitch <x_pitch> <y_pitch> : for an array of switches placed using the ‘-count’ option, specifies the x and y-direction pitch, which is the distance between the west edges (x direction) and south edges (y direction) of adjacent instances.
Note that a negative value for <x_pitch> causes the array to be placed to the left or west of the first instance at <x_loc> <y_loc>, and a negative value for <y_pitch> causes the array to be placed below or south of the first instance.

-orient [N|S|E|W|FN|FS|FE|FW] : specifies the desired switch instance orientation relative to the master cell orientation

i. ‘eco delete clamp_inst' deletes specified clamp instance(s)

```
eco delete clamp_inst { <inst_name> | -all -cell <cell_name>}
```

where

-inst_name: deletes instance specified

-all -cell : deletes all instances of the <cell_name> specified

Note that an instance in the original design cannot be deleted.

j. ‘`eco delete pad’` deletes a specified pad from the design. The syntax is:

```
eco delete pad <padName> ? -all ?
```

k. ‘`eco delete strap’` deletes a named strap (wire). The syntax is:

```
eco delete strap [<handle> | <metalLayer> -x <x> -y <y>]
```

where

<handle> : specifies the “handle” for the deleted strap

<metalLayer> : specifies the metal layer name for the strap deletion

-x <x> -y <y> : specifies the x,y coordinates for the strap to be deleted

l. ‘`eco delete via’` deletes a specified via or stackvia from the design. The syntax is:

```
eco delete via [<handle> | <metalLayer> -x <x> -y <y>]
```

where

[<handle> : specifies the “handle”

<metalLayer> : specifies the metal layer name for the via deletion

-x <x> -y <y> : specifies the x,y coordinates for the via to be deleted

m. ‘`eco delete switch`’ deletes the specified switch instance or set of switch instances. The syntax is:

```
eco delete switch <instName> ? -index <from_aF:bG> <to_aM:bN> ?
```

where

<instName> : specifies the name of the instance to be deleted

-index <from_aF:bG> <to_aM:bN> : specifies the range of identical switch instances to be deleted from an array, when the switches have been named with the format ‘<instName>_a_b’ using the ‘eco add switch -count’ command.

For example, the delete command

```
eco delete switch sw -index 1:2 2:4
```

would delete array switch instances ‘sw_1_2’, ‘sw_1_3’, ‘sw_1_4’, ‘sw_2_2’, ‘sw_2_3’, and ‘sw_2_4’.

export

---

export [db | eco | gridcheck | guiconf | res_calc | model] ? <options> ?

The ‘export’ command exports Totem files in different formats for use by particular tools.

a. ‘export db’ exports the Totem database to a specified hierarchical directory:

```
export db <dir1>/<dir2>/<DB_name>.db
```

b. The ‘export eco’ command exports a Totem ECO file with changes to the database, as a result of FAO modifications to wires, vias, and decap instances, for example. The syntax is:

```
export eco <eco_filename> ? -hier ?
    ?[ -cell {a b c ...} | -def ?-cell {a b c ...}? ?-def2pr?
     | ?-ploc <ploc_filename>? ? -gds_map <gds_layer>.map?
```

where

<eco_filename> : specifies the name of the exported ECO file for the top level design

-hier : specifies that an exported ECO decap file includes an extra attribute indicating the hierarchical name of the closest instance for each decap.

-def : in addition to creating the regular database ECO file, creates an additional DEF format file for the top level

-cell {a b c ... } : in addition to creating the regular top level ECO file, creates files of the type *[a | b | c |…].NORTH.eco* for each of the sub-block cells (a, b, c ...) specified
NOTE: If { a b c ... } includes the top design name, the specified <eco_filename> will not include any decap cells included in sub blocks of the design.

-def -cell {a b c ... } : in addition to creating the regular ECO file, and *[a | b | c|…].NORTH.eco* files for each of the sub-block cells, it also creates block DEF files of the type *[a | b | c |…].eco.def*

-def -def2pr : in the additional DEF file, Totem adds the attribute “+ SOURCE DIST” to each new decap instance line

-ploc <ploc_filename> : specifies a **.ploc* file to be generated resulting from an 'Add Pad' command, to provide the correct pad location, layer, pin name and net information. The package subckt information can then be manually added to the file.

-gds_map <gds_layer>.map : creates an extra tracing point file in GDS2DEF configuration file format, *adsRpt/gdsTracePt*, and writes the information in the Totem log file and the GUI log window. For the example ECO file below:

```
## output eco file
#Totem_Eco_20
DESIGN my_io
UNIT 2000
# UNITS DISTANCE MICRONS 1000 ;
# DIEAREA ( 342000 0 ) ( 722500 1095000 ) ;
# @ 146.182 632.321 << saves computation of real coords
ADD    pad   VGG_1  GND  M7  292365 1264642
# @ 331.253 623.508
ADD    pad   VGG_2  GND  M7  662506 1247017
...
```

the output *gdsmap* file has the form (where M7 is GDS layer 57):

```
## gdsmap file
...
GND_NETS {
    VGG {
         VGG_1 @ 57    488.182  623.508
#<-- 488.182 is 146.182, translated by 342
         VGG_2 @ 57    673.253  623.508
#<-- 673.253 is 331.253, translated by 342
    }
}
...
```

c. The 'export gridcheck' command creates a file containing the results of grid checking that can later be imported. The syntax is:

```
export gridcheck <filename>
```

d. The 'export guiconf' command specifies the export of a configuration file that saves all GUI color map settings that have been changed and applied. Note that dismissing a GUI dialog using the dialog box 'X' ("destroy") button returns the settings to their previous values, so they would *not* be exported by 'guiconf'. The syntax is:

```
export guiconf <output_filename>
```

where

<output_filename> : specifies the exported GUI settings configuration file

e. The 'export model' command generates mmx_view and cell_view CMM files in RAW format for SOC analysis. To generate a binary model, option '-include_binary' should be added. The 'export model' command can be called anytime in the flow. It is not mandatory to first run 'setup design' or any other step. However, you must specify a configuration file when using it before 'setup design'. The original usage flow also is supported. The syntax is:

```
export model ?-dir <outdir>? -mmx
?-config <macro_model_gen_cfg_file>? ?-mmx_view?
?-cell_view? ?-rawonly? ?-layout_only ?
?-l? ?-no_c? ?-mix? ?-tarz? ?-lmwait <wait_sec>?
```

```
?-include_binary ?-top_only ?-presim_detail_rpt
```

where

-dir <outdir>: specifies the output directory (default directory is "./"). The final CMM model is under directories:

<outdir>/<cell>_model/mmx_view/<cell>_cmm

<outdir>/<cell>_model/cell_view/<cell>_cmm

-config <macro_model_gen_cfg_file>: specifies the configuration file for creating the macro model. Get default configuration from *design.gsr* for the integrated GSR flow.

-mmx: model type is MMX cell

-mmx_view: generates mmx_view model

-cell_view: generates cell_view model

-rawonly: exports only raw models

-layout_only: generates CMM models from a GDS-only flow without netlist data

-l: inductance is extracted for the model

-no_c: capacitance is not extracted for the model

-mix: generates a mixed mode model with different master cell names for cell views and mmx views

-tarz: generates a tarz-type file for the CMM

-lmwait : allows a process to wait until a license is available, so that it does not exit due to lack of a license during the initialization process. With no time limit specified, the process is put into "sleep" mode indefinitely. When a wait limit is specified (seconds), if no license is obtained at the end of the specified time, Totem quits.

-include_binary: generates a binary model

-top_only : enables post-simulation CMM model generation for digital blocks.

-presim_detail_rpt : generates a detailed validation report that contains information on the contents of the macro model.The following information is included in the report:

- Type of CMM
- Layer-stack
- PG nets
- States
- Currents on different PG domains
- VCD compatibility
- general design stats

f. The 'export res_calc <filename> ' command exports the effective grid resistances computed from 'perform res_calc' to the specified file.

g. The 'export model' command generates a CMM model according to the options chosen and saves it to the output directory. The syntax is:

```
export model -mmx ?-config <macro_model_cfg_file>?
?-mmx_view? ?-cell_view? ?-dir <outdir>? -top_only?
```

where

-config <macro_model_cfg_file>: specifies the configuration file for creating the macro model. Default configuration is from "import gsr <tsr_style_config>

-mmx_view: generates 'mmx_view' model

-cell_view: generates ‘cell_view’ model

-dir <dir_name>/: specifies the output directory; default output directory is 'macro'
<dir_name>/<cell>_model/mmx_view/<cell>_cmm
<dir_name>/<cell>_model/cell_view/<cell>_cmm

The default output directory is named 'macro' if you do not specify a directory.

-top_only :enables post-simulation CMM model generation for digital blocks.

fao

fao add pad <options>

The 'fao add pad' command adds a set of power /ground pads over an entire mesh, or in a defined window. You must specify a window, a metal layer, a net, and also a pitch in both x and y directions by which to distribute pads evenly over the specified region. **Undo**/**Redo** commands can be used. The command line syntax is:

```
fao add pad -window {llx lly urx ury} -metal <layer>
    -net <net_name> -pitchX <microns> -pitchY <microns>
    -r <resistance> -l <inductance> -c <capacitance>
```

where

-window { }: defines lower left and upper right x,y coordinates of the rectangular area in which pads are added (by default, the entire mesh).

-metal: defines the routing layer (required). Only one metal layer is allowed, since every metal has its own placement.

-net: defines the net name (required)

-pitchX -pitchY: defines the incremental distance between pads, starting at lower left corner of the rectangle specified by '-window { }' (required) .

For example, for “-window {a b c d}”, starting at {a b}, Totem adds pads at {a b}, {a+pitchX b}, {a+2*pitchX b} ..., {a b+pitchY}, {a+pitchX b+pitchY}, {a+2*pitchX b+pitchY} ... , over the entire window.

-r -l -c: specifies pad RLC parameters (units the same as in the PLOC file)

generate

generate [simulationdeck | swvf] ? <options> ?

The ‘`generate`’ command generates specified files for review and/or future use.

a. ‘`generate simulationdeck`’ (for backannotation) generates the Spice deck for DvD backannotation to the timing flow. The syntax is:

```
generate simulationdeck -mmx -aplmmx_cfg <filename>
    -sim_time <time> ? -inst <inst_name>?
    ? -xtor_list <filename> ? ? -dir <output_dir>?
    ? -aplmmx_run_dir <run_dir_name>?
```

where

-mmx: specifies MMX SPICE simulation deck

-aplmmx_cfg <filename>: specifies APLMMX configuration file

-sim_time <time>: specifies total simulation time for DvD back-annotated SPICE simulation (ps)

-inst <inst_name>: specifies the MMX macro instance name (only required in custom design flow)

-xtor_list <filename>: specifies the file that lists the transistors for which you want to back-annotate DvD noise waveforms.

-dir <output_dir>: specifies the output directory to dump out SPICE deck for back-annotation simulation. Default is ‘aplmmx_ba’.

-aplmmx_run_dir <run_dir_name>: specifies the APLMMX run directory

Note: there must be at least one block in the design that is modeled with MMX methodology for the command to generate a functional deck.

b. the ‘generate swvf’ command generates a frequency-domain substrate noise color map for wires and vias. The syntax is:

```
generate swvf ? -layer {<layer_1> <layer_2> ... <layer_N>}?
? -freq <desired_freq>?  ?-freq_resolution <double>?
```

Default layer selection is Pwell and Nwell (same as in GUI interface). To display the color map after generation, use the command ‘show swvf’.

get

---

get [analysis_mode | build | cell | celltype | design | inst | instbynet | instofcell | master | net | orientation | pad | viamodel | ver] ?<options>?

You can query the database for object attributes, and different types of reports on results, such as voltage drop measurements, EM, and power data. The ‘get’ command allows querying the database to find object names based on pattern matching to

- search and find design elements
- execute scripts
- allow deeper access to the design

For details on the 'get' commands, use the TCL 'help' command, which lists all available objects and options. To see a list of top level 'get' objects, use the command:

```
help get
```

which displays the objects available in the log window. For more details, see section "Appendix D1 - TCL ‘get’ Command Reference", page D-a.

Lists of the following objects can be obtained by pattern matching: cell, celltype, inst, instbynet, instofcell, master, orientation, pad, ver.

The following objects have sub-options defining additional properties that can be requested: cell, inst, net, pad, design, viamodel, build. A list of properties available for each object can be obtained with a Help request of the form:

```
get cell -help
```

General ‘get’ command examples:

a. ‘`get cell <options>`’

Returns a TCL list of cell names that match the options specified,.

a1. ‘`get cell <pattern> [-glob|-regexp|-exact] -type macro`’

Returns a list of cells of type 'macro'.

a2. ‘`get cell <pattern> [ -glob | -regexp | -exact ] -design`’

Returns a list of cells referenced in the design. For example, for the regular expression ‘cell12_*’:

```
get cell cell12_* -glob
```

a3. ‘`get cell * -bbox <x1> <y1> <x2> <y2> -type <cell_type>`

Returns cell information based on specified '-type' and '-bbox' options, which when used together, allows you to get all of the cell names in a given bbox that are of a particular cell type.

b. ‘`get celltype`’ returns the cell type of a specific instance, such as inst241:

```
get celltype inst241
```

c. get design <options>

Returns specified design information

c1. ‘`get design -worst_inst_drop [-static | -dvd_type <type> ] ?-net <netName>? ? -limit <N>? ?-out_file <fname>`’

Returns the N worst static or DvD voltage drops for the design per instance, with an optional specification on the net to be reported. By default N=1.

c2. ‘`get design -layers`’

Returns all layers and vias in the design.

c3. ‘`get design -worst_em ?-net <netName>? ? -limit <N>?`’

Returns the N worst EM values for the design, or optionally for a specific net.

Output format:

```
<EM %> <X1> <Y1> <X2> <Y2> <layer> <netname>
```

c4. `get design -ipf_coverage -block <block_name>`
`-spef_coverage -block <block_name>`
`-sta_coverage -block <block_name>`
`-vcd_coverage -block <block_name>`

Returns design information for particular types of coverage with block-specific results.

d. ‘`get inst`’ `<options>`

Returns a TCL list of instance names that match the class specified. See section "Appendix D1 - TCL ‘get’ Command Reference", page D-a for details.

e. ‘`get instofcell`’ <options>

Returns a TCL list of instance names for the given master cell, such as for cell112_8:

```
get instofcell cell12_8
```

f. ‘`get master`’

Returns the master cell of the given instance, such as for inst241:

```
get master inst241
```

g. ‘`get pad <padName> -net`

Returns the name of the net connected to pad specified.

h. ‘`get switch <options>`’

Returns the switches matching the class specified.

h1. ‘`<switch_name> -int_voltage`‘

Returns the switch internal node voltage.

h2. ‘`get switch <switch_name> -voltage_drop` -out_file<fname>

Returns the voltage drop across the switch into the specified file.

h3. ‘`get switch <switch_name> -current`‘

Returns the current through the switch.

h4. ‘`get switch <switch_name> -net_pair`’

Returns the external and internal net-pair for a switch instance, in format VDD_NET1:VDD_NET2.

h5. ‘get switch <switch_name> -turn_on_time’

Returns the switch turn-on time.

gsr

gsr [get <GSR keyword> | [set | append] <GSR keyword> { {<keywordValues>} ... } | dump ? -o <filename> ?]

The ‘gsr’ command retrieves GSR variable values, sets GSR values, displays the contents, or sends the GSR contents to a file. The command only supports integer, double, Boolean, and string types.

a. ‘gsr get’ retrieves the present value of the specified GSR keyword. The syntax is:
```
gsr get <GSR keyword>
```

b. ‘gsr set’ allows setting the value of most GSR keywords by TCL command (can be set to null), and deletes all previous values, using the syntax:
```
gsr set <GSR keyword> <keywordValue>
```
The 'gsr set' command can be executed before setting up the design (‘setup design”). For GSR keywords that contain compound values, lists, and more complicated structures, the following proper TCL syntax is required (continuous string, no new lines):
```
gsr set <GSR_keyword> {{<keyword_option1> <value11> <value12>...}
    {<keyword_option2> <value21> <value22>... } ... }
```
For example:
```
gsr set VCD_FILE {{top file.vcd} {FRONT_PATH “testbench/”}
{SUBSTITUTE_PATH “”} {FRAME_SIZE 2000} {START_TIME 10000}
{END_TIME 20000} {TRUE_TIME 1}}
```
1. Example for input data-related keywords:
```
import gsr <>.gsr
gsr set LIB_FILES { .lib CUSTOM }
gsr set DEF_FILES {{block1.def block} {block2.def block} {top.def top}}
setup design
```
The specified input data then are automatically imported during the 'setup design' step.

2. Using 'gsr set' and 'gsr get' commands for specifying and retrieving equivalent power/ground nets. The syntax is:
```
gsr set VDD_NETS {{VDD <vdd_value> { <equiv_Net> } }
    {VDDQ <vddq_value> { <equiv_Net> }}
```
Example:
```
gsr set VDD_NETS {{VDD 1.1 {VDD1CORE }} {VDDQ 3.3{VDDQ1 } }
gsr get VDD_NETS
```
The output then would be:
```
VDD_NETS {
VDD 1.1 {
    VDD1CORE
    }
VDDQ 3.3 {
    VDDQ1
    }
}
```

c. ‘gsr append’ adds specified new values to the existing values of a GSR keyword (no existing values are changed), using the same syntax as ‘gsr set’, as follows:
```
gsr append <GSR keyword> <new_keyword_option> <new_values>
```

d. ‘`gsr dump`’ displays all intermediate level GSR keyword values on the screen, or sends all intermediate level GSR keywords to a specified file. The syntax is:

```
gsr dump ?-o <filename>?
```

where

<GSR keyword> : the specified keyword to operate on

<keywordValue> : specifies a new value for the keyword

<filename> : specifies the file in which to copy all GSR intermediate keywords and values

history

The 'history' command displays all previous commands typed on the command line during the session. The syntax is:

```
history
```

import

import [apl | avm | db | def | eco | esdcd | gsr | gridcheck | gsc | guiconf | keybinding | lef | lib | pad | power | res_calc | sdf | sta | tech | viamodel] ? <options> ? <inputFileName>

The ‘import’ command imports a number of types of files for use in Totem analysis, such as APL, AVM, DB, DEF, ECO, GSR, guiconf, LEF, LIB, PAD, POWER, and TECH. The general syntax for the command is:

```
import <file_type> <file_name>
```

Those import commands with more complicated syntax are described below.

a. The ‘import apl’ command imports an APL -generated file. The syntax is:

```
import apl ?-c? <inputFileName>
```

where

-c : specifies importing of an APL-generated device capacitance file.

b. The ‘import avm <configFilename>’ command runs the AVM utility and imports the results into the Totem database.

c. The ‘import db’ command imports a specified database with associated options. The syntax is:

```
import db <dir_name> ?-cache_mode [0|1]?
?-cache_dir <cacheTempDir>? ? -ignore_layout? ? -no_pfs?
```

where

<dir_name>: specifies the path and directory name of the database to import

-cache_mode [0 |1]: when set to 1 enables adaptive disk caching for ‘import db’. This only overrides the global setting of the GSR keyword CACHE_MODE for the ‘import db’ operation. Make sure that there is local disk space to use for disk caching.

-cache_dir <cacheTempDir>: specifies the directory where data is cached for ‘import db’. This only overrides the global setting of the GSR keyword CACHE_DIR for the ‘import db’ operation. For efficient operation, it should be on a local disk where Totem is running. Default: current working directory.

-ignore_layout : does not load layout data from the *DB/.MM* directory, but repeats post-simulation processing to re-create all IR/EM maps. This option can be used as a work-around for 'import db' when the layout stored in the *DB/.MM* directory has been corrupted.

-no_pfs : prevents an additional pfs license being checked out while importing DB of a PF-S run. It also excludes the PF-S data from importing and all functionalities that require a PFS license will be disabled after import.

NOTE

The compatibility of Totem databases from different releases is as follows:

- If the database version and Totem version are of the same major release, but from different minor releases or patches:
  a. A database generated by an older version of Totem can be loaded with a newer version of Totem, but new features in the newer versions of Totem will not be available.
  b. A database generated by a newer version of Totem cannot be loaded by an older version of Totem.
- If the database version and Totem version are from different major releases (for example, 14.x and 15.x), the database cannot be loaded across these versions.

Apache *recommends that the database version and the Totem version used to load the DB be the same. The "database version" is the version of the tool used to generate the database.*

c. The '`import def`' command imports a DEF file data into the design. The syntax is:

```
import def ?[-vddonly | -vssonly]? <inputFileName>
```

where

-vddonly | -vssonly : selects importation of Vdd or Vss data only

<imputFileName> : specifies DEF input file name

d. The 'import esdcd' command imports results from previously-run ESD CD checks to create color map displays. The syntax is:

```
import esdcd {<rule_name1> ... } ?[-exact|-glob|-regexp]?
? -outDir <dir_name>? ? -cacheEM [0|3]? ? -EMScale <value>?
? -EMRuleSet <name>? ? -saveAs <filename>?
```

where

<rule_name1> ... : specifies use of results from one rule, or merges results from several rules specified.

-exact/-glob/-regexp : provides filtering for rule names specified using pattern matching; default: 'exact'

-outDir: specifies the text report directory

-cacheEM : set to 3 to reuse the saved EM limit; default: 0 (no reuse)

-EMScale : sets scale factor for EM limit; default: 1.0

-EMRuleSet : specifies name of EM rule set to be used for EM checking

-saveAs <filename>" saves merged imported results, so that they can be loaded back quickly

e. The 'import gridcheck' command allows importing the results of a previously run gridcheck. The syntax is:

```
import gridcheck <filename>
```

f. The '`import keybinding <configFilename>`' command imports a file defining user-specified bindkeys. For a description of the keybinding file format, see section "Bindkey Functions", page B1-623.

g. The 'import pad' command Imports pad cell (*.pcell*), pad instance (*.pad*), or pad location (*.ploc*) files. The 'import pad' command is disabled after setup design is completed.

h. The ‘import res_calc <filename> ’ command imports effective grid resistances computed from 'perform res_calc' and then exported to a file.

i. The ‘import tech’ command allows you to import a third party (Virtuoso[tm]) technology file, and display a resource file and layer map file, using the syntax:

```
import_tech -tf <filename> -drf <filename> ?-vmap <filename>?
    ?-map <filename>?
```

where

-tf : specifies the Virtuoso technology filename

-drf : specifies the Virtuoso display resource filename

-vmap: specifies the Virtuoso layer map file (optional)

-map : specifies the Totem GDS layer map file if GDS_MAP_FILE is not defined in the GSR, or in the GDS2DB configuration file.

ircx2tech

ircx2tech <options>

The ‘ircx2tech’ command converts an input iRCX file to an appropriate Apache tech file with the syntax:

```
ircx2tech -i <input_file> -o <output_file> -v [min|typ|max]
    ?-m <layer_mapping_file>? ?-g captab ?
```

where

-i: specifies the input iRCX file

-o: specifies the name for the output tech file

-v: specifies a via resistance table with a different RC corner iRCX file

a. “-v min” for R_best/C_best corner

b. “-v max” for R_worst/C_worst corner

c. “-v typ” for typical corner

-m: specifies the name of the layer mapping file that contains the layer mapping table from the iRCX file to the tech file

-g captab: if specified, Totem calls *$APACHEROOT/bin/captab* to generate the corresponding capacitance lookup table file

marker

marker [add | addfile | clearall | delete] <values>

The ‘marker’ command adds or removes cross-hair marker(s) in the design, specified either as an x,y position or as an instance name, or a list of instance names.

a. ‘marker add -position’ generates a cross-hair marker with a specified color and size, and returns a marker name.

```
marker add -position <x> <y> ?-color <color>? ?-size <size>?
```

where

<x> <y> : specifies the x,y location for the new marker

-color <color> : specifies the color of the new marker. Default: yellow

-size <size> : specifies a number indicating the relative size of the new marker. Size increases with number value, with no maximum. Default: 3 (small) .

b. ‘marker add -instance’ generates a cross-hair marker with the specified instance name(s) used as marker name(s). Default marker size is 3 (small) and color is yellow.

```
marker add -instance <list of instances> ?-color <color>?
```

```
?-size <size>?
```

where

-instance <list of instances> : list of one or more instances to be marked

c. ‘marker addfile <filename>’ adds markers from a list in the specified file.

```
marker addfile <filename> [-instance | -position ]
  ?-color < >? ?-size < >?
```

where

-instance: the specified <filename> contains instance names, one per line

-position : the specified <filename> contains <x, y> coordinates for the markers, one location per line.

Note: the marker command puts a heavy demand on memory and CPU resources. Using more than about 10K markers makes drawing slow and could exhaust memory.

d. ‘marker [clearall | delete]’ clears all markers or deletes markers by name.

```
marker [ clearall | delete <marker name> ]
```

where

clearall: deletes all markers

delete <marker name> : specifies the name of the marker to be deleted (either the instance name, or the marker name returned when specified by position).

mesh

mesh [add | delete | fix | generate| optimize | snscalc | sub_grid| set_ width | vias] ? <options> ?

The ‘`mesh`’ commands perform various types of modifications to power grids, including adding, deleting, and modifying widths and spacing, to reduce voltage drop.

Please see Chapter 9, "Fixing and Optimizing Grid and Power Performance", for details about this command.

message

message [disable | enable | info | list] ? <options> ?

The ‘`message`’ command displays Error, Info and Warning message information.

a. ‘`message info`’ displays a description of the specified message. The syntax is:

```
message info <message_id>
```

where

<message_id> : the unique identifier of message to be displayed

b. ‘`message list`’ displays a list of all message IDs that are currently available. The syntax is:

```
message list
```

c. ‘`message disable`’ disables display of selected message IDs or a class of messages. The syntax is:

```
message disable [all | errors | info | warnings | <list of ids>]
```

where

all : specifies all messages are disabled

errors: specifies all Error messages are disabled

info: specifies all Info messages are disabled

warnings: specifies all Warning messages are disabled

<list of ids>: specifies particular message IDs are disabled

d. 'message enable' allows message display for the specified message IDs or a class of messages. The syntax is:

```
message enable [all | errors | info | warnings | <list of ids>]
```

where

all : specifies all messages are enabled

errors: specifies all Error messages are enabled

info: specifies all Info messages are enabled

warnings: specifies all Warning messages are enabled

<list of ids>: specifies particular message IDs are enabled

mmx

The 'mmx' command allows interactive manipulation of transistor current profiles within the Totem analysis environment without rerunning APLMMX characterization. You must specify (1) transistor name(s) and which state is to be modified in the current profile, (2) the desired current waveform (DC current value, PWL waveform, or current from a different current profile), and (3) operations on how to modify the existing current, or the newly-assigned current. Current can be scaled by a value, or the switching time can be offset, and/or the offset can be filled with a particular current value. The full syntax is:

```
mmx set_current
[ -name <xtor1 ...>| -file <filename> | -region <x1 y1 x2 y2>|
 -glob <pattern> | -regexp <pattern> |
 -regsub <top_pattern block_pattern> ]
[-spcurrent <current_profile> <gdsmmx_file> ?<state>? |
 -pwl <pwl_file> | -dc <dc_value>]
?-domain <domain>? ?-scale <value>? ?-offset <time_seconds>?
?-offsetfill <current_in_Amps>? ?-basespcurrent <state>?
[-save <filename> | -reload <filename>] ?-comment <comment>?
```

1. Transistor selection

Specifies which transistors' current profile is to be replaced by user-defined current source:

-name <xtor name>: specifies one or more transistor names directly.

Example:

```
mmx set_current -name { X384.X36.X7.M11  X284.X36.X7.M12  ... }
```

-file <filename> : specifies a file containing the transistor name.

-region <x1 y1 x2 y2>: select transistors inside the specified x,y coordinate region

Example:

```
mmx set_current -region { 0 15 20 30 }
```

-glob<pattern>: specifies a global pattern to select a group of transistors.

Example: mmx set_current -glob X112.X0.X0.X14*

This selects transistor X112.X0.X0.X14.M1, X112.X0.X0.X14.M2, … with name pattern X112.X0.X0.X14*.

-regexp<pattern>: specifies a regular expression pattern to match transistors.

-regsub <top-pattern block-pattern>: applies regular substitution to use a block-pattern to match transistors in a sub-hierarchy design and save the sub-pattern from matched transistors. Insert the matched sub-pattern into top-pattern to get full flattened transistor name in the design.

Note: -regsub can only go with -spcurrent <child.current child.gdsmmx state>.

Example:

```
-regsub {X112.X3.X0.X14.\1 (M[0-9]+)}
    -spcurrent {child.current  child.gdsmmx}
```

This command searches transistor name Mx (where x is a number) in the *child.gdsmmx* file and saves Mx as a sub-pattern. Then the command combines the top-pattern string “X112.X3.X0.X14.” with the sub-pattern to find the target transistor in the main design.

For example,

```
sub-block name     full flat name
M1          >>    X112.X3.X0.X14.M1
M3          >>    X112.X3.X0.X14.M3
M4          >>    X112.X3.X0.X14.M4
M10         >>    X112.X3.X0.X14.M10
```

2. User-defined current source

Specifies which type of current and what content is used to perform current replacement

-spcurrent <current_profile> <gdsmmx_file> ?<state>?: applies a current profile from another run. The selected transistor must have the same name in this specified GDSMMX file and main design. Or the ‘-regsub’ option should be used. If this current profile has multiple state data, a state name must be defined.

-pwl <pwl_file>: specifies a piece-wise-linear waveform to replace all selected transistors' current. The format of PWL input file is “<time> <value>” per line, as in the following example:

```
0 0
1e-11 5e-6
2e-11 5e-6
3e-11 5e-5
```

-dc<value>: specifies a DC value to replace all selected transistors' current.

Note: If no user-defined current source is given, operations listed in “Current modification operator” works on the original current profile.

3. Current modification operator

Applies simple operations to modify the current. If no user-defined current source is given, the operation applies to original current profile (*.apache/cell.current*) for selected transistors.

-scale <value>: scales the current of all selected transistors.

-offset <time_seconds>: delays the switching time in the second of the specified transistors.

-offsetfill <offset_time_current_value_A>: specifies the current value for the offset window defined in the ‘-offset’ option.

Example1:

```
mmx set_current -offset 2e-9 -offsetfill 1e-6
```

This example delays the switching time of the current by 2ns with fill in current value of 1uA between 0 and 2ns.

Example 2:

```
mmx set_current -scale 0.1
```

The example down-scales the whole current profile by a factor of 0.1.

4. File I/O interface

   These options allow you to output/save or reload the current profile.

   -save <filename>: saves all 'mmx set_current' commands performed before the save. Nothing is saved if you do not use the '-save' command option to store the snapshot. Without a filename, it overwrites the original current profile in *.apache/cell.current.*

   -reload <filename>: loads a previously saved current profile into memory.

5. Other 'mmx set_current' options

   -basespcurrent <state>: if the original current profile contains multi-state information, a state must be defined so that Totem knows the current from which state is to be modified.

   -domain <domain>: specifies a domain net name if the transistor connects to multiple domains.

   -comment <comment>: adds a comment to the command for future reference.

movie

movie [make | play] ? <options> ?

The 'movie' command sets up or plays an instance-based or transistor pin based "movie" of DvD performance, with the following options:

a. '`movie make`' saves data to create movie of dynamic voltage drop (DvD) changes sampled over the timing window. The syntax is:

```
movie make ?-transistor? ?-zoom? ?-time <t1> <t2>? ?-step <time_step>?
    ?-dir <path>?
```

   -transistor: creates a movie based on transistor-level pin voltages. Default is an instance-based DvD movie.

   -zoom : makes the movie view the current display zoom level

   -time <t1> <t2>: specifies a start and end time interval within the simulation time for a shorter movie (ps by default).

   -step <time_step>: specifies a longer time step for frame sampling, in multiples of the time step specified in the GSR keyword DYNAMIC_TIME_STEP (ps units by default)

b. '`movie play`' displays frame-by-frame in the GUI the movie specified in the '`movie make`' command. The syntax is:

```
movie play ?-exit? ?-dir <path>?
```

   -exit: removes the movie display window when the last frame is played.

   -dir <path>: specifies directory path for writing to or reading from the movie files.

perform

perform [analysis | clampcheck | cmmcheck | emcheck | esdcheck | extraction | gridcheck | pwrcalc | pwrmodel | res_calc] ? <options> ?

The '`perform`' command runs selected Totem analyses. The standard syntax is '`perform <cmd_name>`'. Commands that have more options are described below.

a. '`perform analysis`' executes various forms of Totem analysis, along with special options. The syntax is:

```
perform analysis [ -static | -signalEM | -dynamic | -vcd |
    -v(ector)less | -sir ] ? -simtime <simtime>?
    ? -mmx_per_xtor_derating ?
```

   for digital place and route analysis only

```
-vcd ?-t <start_time> <end_time>?
? -nx <nx> -ny <ny> -early ?
?-ipwr_only? ?-setup?
```

where

[-static | -signalEM | -dynamic | -vcd | -vectorless] : selects the type of analysis

-dynamic : runs mixed mode power analysis, with a combination of vectorless and VCD methods on different sections of the design. VCD blocks follow the switching scenario in VCD and the remainder follow the vectorless flow (switching according to the toggle rate). An enhanced Dynamic Simulation Engine provides up to 20% improvement in dynamic simulation run-time, using the option '-high_capacity 1'.

-vcd : VCD block follows the switching scenario in VCD, and the rest do not toggle to assure "pure vcd" analysis.

-vectorless (or "-dvd"}: entire design (both VCD blocks and vectorless blocks) follows the vectorless flow (switching according to the toggle rate). For regular multi-cycle vectorless analysis, Totem generates a switching scenario using the vectorless algorithm for the first cycle. For the second cycle, the same instances switching in the first cycle transition in the opposite direction. The third cycle is then the same as the first cycle and the fourth cycle is the same as the second cycle, and this behavior continues for each subsequent cycle. To have a different vectorless switching scenario in each cycle, use the '-mcycle' option. In mixed mode the whole design follows the vectorless flow (switching according to the toggle rate).

-sir: invokes the "save and reload" flow that saves the database and then reloads it again before running simulation in order to optimize memory use. Increases run time because of the extra export and import of the database. Can be used in static, dynamic and CPM flows. Multi-threading can be enabled using the GSR keyword 'MT_SR'.

-simtime <simtime> : specifies the simulation time in picoseconds

-vcd -t <start_time> <end_time>: specifies the simulation start and end times, and overrides both the START_TIME/END_TIME defined in the VCD_FILE GSR keyword and the most critical cycle selected by cycle selection.

-early : creates waveforms for demand current with accelerated simulation results by using the '-early' option and a partitioned chip (less aggressive node reduction). The demand current is captured for each partition in the files:

- *adsRpt/Dynamic/<design>.partition.ipwr*
- *adsRpt/Dynamic/<design>.partition.ignd*

-nx : specifies number. of partitions in the X-direction

-ny : specifies number of partitions in the Y-direction

-ipwr_only: more aggressive node reduction for quick ipwr waveform generation.

-mmx_per_xtor_derating: applies transistor-specific voltage derating factors based on the individual pin voltage drops at a given time point. You must also provide a voltage derating table or multiple simulation output files based on the derated voltages supplied in the APLMMX step, using the configuration file keywords APL_VOLTAGES and either SIM_OUTPUT_PREFIX or XTOR_VIRTUAL_DERATING_FILE. Note that if the design has multiple power and ground domains, you must specify pgarc data in a custom lib file with the GSR keyword LIB_FILES to enable the voltage derating function, along with specifying the APLMMX configuration file keywords:

b. the 'perform analysis' command has several options for substrate analysis, as described following.

b1. ‘perform analysis -inject current’ supports current sources with user-defined noise magnitudes for the substrate noise injection flow. Both sine and step inputs are supported, using the syntax:

```
perform analysis -inject_current -sine <freq> ?<mag>?
{<x> <y> <layer>|<PlocName>|ProbeName ... }
?-time2ss <presimTime>?
```

or

```
perform analysis -inject_current -step <mag>?
{<x> <y> <layer>| PlocName|ProbeName ...}
```

or

```
perform analysis -inject_current -wave <waveformA_file>
{<x> <y> <layer>|PlocName|ProbeName ...}  -wave
<waveformB_file> {<x> <y><layer>|PlocName|ProbeName ...}
...
```

Notes:

1. For -sine and -step options, the default is a 1A amplitude current injection.
2. For both sine and step, only one source can be declared, although copies of the same source can be attached at multiple locations.
3. For injection at plocs, the plocs are disconnected from the package by adding a 1MOhm resistor.
4. The -time2ss option is time to steady state in sec.

b2. ‘perform analysis -inject voltage’ supports voltage sources with user-defined noise magnitudes for the substrate noise injection flow, as well as a command for S21 scattering pattern analysis, using the syntax:

Standard voltage injection commands

```
perform analysis -inject_voltage -sine <freq> ?<mag>?
{<x> <y> <layer>|PlocName|ProbeName ...}
?-time2ss <presimTime>?
```

or

```
perform analysis -inject_voltage -step ?<mag>?
{<x> <y>  <layer>| PlocName|ProbeName ...}
```

or

```
perform analysis -inject_voltage -wave <waveformA_file>
{<x> <y> <layer>|PlocName|ProbeName ...}
-wave <waveformB_file> {<x> <y> <layer>|PlocName|ProbeName
...} ...
```

Notes:

1. For sine and step options, the default is a 1V amplitude voltage injection.
2. Only one sine or step source can be declared, although copies of the same source can be attached at multiple locations.
3. For injection at plocs, the plocs are disconnected from the package by adding a 1 MOhm resistor.

S21 analysis command

```
perform analysis -s21 <freq> <Ploc1> <Ploc2>
-time2ss <settlingTime>
```

Notes:

1. CSE connects the source and impedances at the ports automatically to compute s21 - - 40mA, 50Ohm at the input (Ploc1), and 50 Ohm at the output (Ploc2).

2. The connections to the ploc are left as it is. That is, CSE does not add a 1 MOhm resistor between the plocs and package.
3. Only plocs are supported for S21 analysis.

c. ‘perform clampcheck’ reports a list of clamps with the specified attributes.

```
perform clampcheck <net/bump_conn_options>
? -rule <rulefile 1> , <rulefile2>, ...
? -o <file>? ?-instConn?
? -bump <name_1>,<name_2> ?? -isolatedBump?
? -cell <cell_name>? ?-celltype <type>?
? -inst <inst_name>? ? -volt <voltage>? ?-net <net_name>?
? -netConn <net1> <net2> ...? ? -allNetConn ?
? -bumpConn <bump_name> ? ? -allBumpConn ? ? -loop?
? -loopLength <stage_num>? ? -b2bLoopLength <stage_num>?
? -roff <R_thresh> ? ? -rptDisConn ? ? -clamp <rule_file> ?
? -rule <rule_file> ? ? -esdStage {min,max} | <stage_num>?
? -detail? ?-append? ? -summary ?
```

where

<net/bump_conn_options> : indicates how net/bump connectivity is checked. The options are : -allNetConn, -allBumpConn, -netConn <Nets>, and -bumpConn <Nets>.

-rule: specifies the name(s) of the rule file(s) for checking. Multiple rulefiles can be specified separated by commas or using braces -rule {rulefile1 rulefile2 ...}

-o <file>: specifies the output filename. If this option is not specified the default output file for clamp pin coordinates is *adsRpt/ESD/clampcheck.rpt*.

-instConn: reports clamp instances that have missing connections to the P/G grid.

-bump : reports ESD pin pairs connected to bump(s)

-isolatedBump: reports bumps that do not connect to any clamp

-cell <cell_name>: reports information on specified clamp cell(s)

-celltype <type>: reports information on clamp cells of a particular clamp type

-inst <inst_name>: reports information on specified clamp instance(s).

-volt <voltage>: reports a list of clamps connected to a particular node voltage.

-net <net_name>: reports information on all clamp nodes connected to specified net(s), both directions.

-netConn: checks connectivity of each net pair (both directions), so for “-netConn {netX netY}”, the connectivity of both netX->netY and netY->netX is checked. If “-netConn netX” is specified, the connectivity from netX to all other nets and from all other nets to netX is checked. For each net pair reported, a symbol “->” or “<-” is added between the two nets to indicate the connectivity direction between them.

-allNetConn: reports clamp connectivity to all nets and domains, in both directions

-bumpConn: reports clamp connectivity to all bump connectivity from the specified bump

-allBumpConn: reports clamp connectivity of all bumps

-loop: lists the clamp instances of all B2B loops for queried net pairs.

-loopLength: -b2bLoopLength : set the EXACT stage number(s) to be reported for queried net pairs. Note that “-esdStage” sets the RANGE of stage numbers, while “-loopLength” sets the EXACT stage numbers. -loopLength can specify more than 2 numbers.

-roff: sets the resistance threshold value at which clamp devices are considered “OFF”. The default threshold value is 1e6 Ohms.

-rptDisConn: reports net-pairs with no clamps between, as well as clamps with only one pin and clamp pins not specified in ESD_PIN_PAIR

-clamp : verifies that the clamp pin location specified in the rule file is on the physical pin location of the clamp. Detailed messages are displayed in the *esd_info.rpt* file.

-esdStage: specifies either a specific stage or a range of clamp stages between which the net/domain pair checks have to be performed.

Example:

```
-perfom clampcheck -esdstage {1,3}
```

Here, net/domain pair check happens from stage 1 to stage 3.

-detail: reports detailed information on net/domain pair.

-append: appends new results to existing results in the output file, with no overwriting

-summary - generates a bump/clamp summary text report based on the rule file specified.

Output report format for bump summary:

```
# <Net> <Bump#> <Pass#> <Fail#> <OutRadius#> <Isolated#>
```

Output report format for clamp summary:

```
# <Net> <Pin#> <Connected#> <Unconnected#>
```

d. ‘`perform cmmcheck`’ provides information pertaining to a particular CMM database and CMM cells. The command can be invoked at any time. If the GSR is read in the '-cell' option can be used, otherwise, the full model path using the '-path' option must be specified to query CMM information. The syntax is:

```
perform cmmcheck ? -version?  ?-o <output_filename>?
       ?-path <cmm_db_path>? ?-statistics? ?-cell <cell_name>?
```

e. `‘perform emcheck’` can be used to perform power EM analysis for specific modes and nets from the command window, using the command:

```
perform emcheck ?-mode [AVG |RMS |PEAK |all ]?
?-net <net_file>?
```

where

-mode : specifies the type of emcheck to run

-net : specifies a file listing one or more nets to be checked. The file format accepts one net name per line; leading and trailing white spaces in the file are ignored.

Skip the -mode and -net options to perform analysis for all modes and all nets, respectively, in the design.

Outputs are reported in the file *adsRpt/SignalEM/<design_name>.em.worst*.

f. ‘`perform esdcheck`’ invokes the ESD checking command, using the syntax:

```
perform esdcheck  -rule {<file1> <file2> ...}
  -clamp {<file1> <file2> ...} ? -setupClamp?
? -thread <num_threads>? ? -statuscheck <time_interval>?
? -ruleType <rule_type1> <rule_type2> ...?
? -ruleName <rule_name> ?
? -optimize <mode>? ? -detail ? ? -append ?
? -setupClamp ? ? -shortNode{0|1|2}?
? -outDir <results_dir> ? ? -excel? ? -ignoreError ?
? -saveDBDir <path> ? ? -jobCount <num_jobs> ?
? -jobFile <job_defin_file> ?
?-from {<x> <y> <layer> <net>} -to {<x> <y> <layer> <net>}
```

```
[{-zapI <I> | -zapV <V> -zapR <R>} ]?
? -zapIRange <level_1> <level_2>....
? -zapVRange <level_1> <level_2>....
? -extendClampConn ? -slaveOnly
? -excludeRule <rule type>,<rule name> ? -excludeRuleFile
<file>
? -executeRule <type1,name1>..? -executeRuleFile
<typeNameFile(s)> ? -addNode ? -reportByPGArc
```

where

-rule <rules_files> : specifies the names of the rules files containing the rules to be checked. At least one rule must be specified. If the ESD check for a rule has an error, the default behavior is to stop execution. The '-ignoreError' option allows checking to continue past any errors, if possible, and attempt to finish all rules in the rule file.

-clamp <clamp_files> : specifies the clamp files. Clamp statements can be combined in the rule file or separately defined in a clamp file. See the Clamp file section for a description of the clamp file syntax and an example clamp file.

-setupClamp : when specified, the tcl command

```
pfs show clamp_iv <clampIVName> ?-plot? <options>
```

can be used to plot the I-V curve, and produce a text output file that can be plotted with xgraph or sv.

-thread <num_threads>: specifies the number of threads/processes to be used, to speed up the calculation. All available CPUs may be used to support the multiple threads unless '-mcore' is specified. Default=2.

-statusCheck: dumps out a progress report of the PathFinder run. By default the report is dumped in the file *adsRpt/ESD/esd_progress.rpt,* at the specified <time_interval>, in hours or fraction of an hour.

-ruleType <rule_type> : checks all rules that match the specified type. This allows flexibility to construct a large "template" file with many rules defined in it. Then when only -type is specified in the rule file, all rules that match the specified type are executed.

-ruleName <rule_name>: performs checks for the user-specified rule name. When only -ruleName' is specified, all rules that match the specified name are executed (regardless of the type). When both '-type' and '-name' are specified, the rules in the rule file that match both the specified rule type and rule name are executed.-optimize <mode>: as the mode specified increases from 0 to 3, more optimization is done, for greater performance gains.

Mode values 0 and 1 are only recommended to work around issues that have caused resistance computation to fail:

0: all optimization off

1: performs only basic netlist optimizations (up to 2X performance gain)

Mode values 2 and 3 are recommended for normal use.

2: performs aggressive netlist optimization for greater performance gain (up to 20X possible). This is the default.

3. In addition to optimization of mode 2, performs additional optimization for greater coverage (can double the performance as with 2). This setting would be effective only for C2I, C2M, and B2I resistance checks.

-detail : provides detailed reports on B2B and multi-stage checks, including data on x,y locations, netnames, metal layers and the cell names of the clamp instances, so you do not have to refer to any other files to get details.

-append : appends results to the output files, instead of overwriting the previous output file

-setupClamp: Generate clamp DB, typically used when user need not do extraction after modifying shorting regions

-shortNode: Node shorting option, same as GSR keyword, `ESD_SHORT_CLAMP_PIN in which`

0 (off): no clamp pin node shorting
1 (all): short all pin nodes
2 (loc): short all pin node locations as specified in clamp file.

-outDir <results_dir>: specifies the directory for the results files. Default: *adsRpt*.

-excel : creates a consolidated ESD report in Excel format for resistance and CD checks *(adsRpt/ESD/esd_excel.rpt)*, so that it can be easily sorted and processed using Excel, and also easily parsed by other flows downstream.

Output file with -excel option looks like:

From/To	A<0>	A<1>	A<2>	AVDD
A<0>	X	UNCONN	UNCONN	UNCONN
A<1>	UNCONN	X	UNCONN	UNCONN
A<2>	UNCONN	UNCONN	X	UNCONN
AVDD	UNCONN	UNCONN	UNCONN	X
AVSS	UNCONN	UNCONN	UNCONN	UNCONN
DQ<0>	4 5	4 5	4 5	5 5
DQ<1>	4 5	4 5	4 5	5 5
DQ<2>	4 5	4 5	4 5	5 5

Here, X indicates not-applicable (between same net-pairs) ; UNCONN indicates Unconnected ; <number1 number2> indicates the Min-ESD and Max-ESD Stages through which nets are connected

-ignoreError : where the <rules_file> contains multiple rules to be checked, if there are data issues or error conditions such that the ESD check for a rule has an error, the default behavior is to stop execution. '-ignoreError' allows checking to continue past any errors, if possible, and attempt to finish all rules in the rule file.

-saveDBDir: saves the DB into the specified directory. You can then execute esdcheck in the same run used for exporting the DB.

-jobCount: specifies the number of jobs to execute at a time when the rule file has multiple rules in it, or when an ESD current density check rule has many ‘from/to’ point pairs to be analyzed. This option can be used to partition the rule check into multiple sub-tasks, which are executed in parallel when the host machine has multiple CPUs. One job is performed by the parent PF session and the rest of the jobs are executed in different sessions in *adsESD1, adsESD2*, … as the working directory.

The recommended usage is to start the session by importing a previously-exported DB, then to run ESD checking with the -jobCount option specified.

This option provides a status report on CD checking, including rule name, rule type and the number of tests, in the *esd_info.rpt* file. Example file contents:

```
# ESD Job Assignment Report
```

```
#<JOB>  <WORKING_DIR> <RULE_NAME> <RULE_TYPE> <NUM_TESTS>
Job1    ./adsESD1       C2I_RULE        CLAMP2INST      1
Job1    ./adsESD1       G2S_RULE        BUMP2BUMP       1
Job1 ./adsESD1  SIGDC    CURRENT_DENSITY 16 (shotgun)
Job2    ./adsESD2       C2M_RULE        CLAMP2MACRO     1
Job2    ./adsESD2       S2P_RULE        BUMP2BUMP       1
Job2    ./adsESD2       GPDC            CURRENT_DENSITY 25
```

-jobFile : specifies the job definition rules file

-addNode : creates a node for accurate esdcheck. Options “-from {<x1 y1> <layerName1> ?<netName>? }” and “-to {<x2 y2> <layerName2> ?<netName>?}” must also be specified. A node is created at that location, provided there is no node within a 1 uM radius, and there are wires at the specified location.

Options for current density checking:

-from / -to : specifies the x,y coordinates, the layer and net for from and to zapping. Finds the closest nodes that match the specified points. Note that other specialized commands are also available for “-from/-to” checking:

-fromBump <bumName> -toBump <bumpName>

-fromClamp {<instName> <locId>} -toClamp {<instName> <locId>}

-fromInst {<instName> <netName>} -toInst {<instName> <netName>}

-fromInst {<instName> <pinName>} -toInst {<instName> <pinName>}

Note: For the specified instance, a node in the specified net connected to the pins of the instance is selected. More than one '”-from/-to” check can be specified, where all “-from” points are shorted and connected to the positive terminal of the zapping source, and all the “-to” points are connected to the ground terminal of the zapping source. All points to be shorted should be in the same net (or connected by power gating switches), and also physically connected in the layout. Otherwise, checking is not performed.

-zapI <I> : zapping current (default 1A)

-zapV <V> -zapR <R> : zapping voltage and zapping source resistance. (The default is to use '-zapI')

-zapIRange <level_1> <level_2>..: specifies multiple zap current levels for CD check. After checks for all levels are finished , an "ESD Zap Sweep Summary" will be generated for each rule that has used the zap range in the esd_summary.rpt .

-zapVRange <level_1> <level_2>..: specifies multiple zap voltage levels for CD check. After checks for all levels are finished , an "ESD Zap Sweep Summary" will be generated for each rule that has used the zap range in the esd_summary.rpt

-extendClampConn: uses the discharge paths through clamp devices, even for cases when from/to zap points are part of same net.

-slaveOnly: enables Pathfinder-DMP to run Resistance and CD checks only on slave machines. This will save the master machine memory.

-excludeRule <rule type>,<rule name> : excludes the defined rules from the rule file.

-excludeRuleFile <file>: Specifies a file which contains the list of rules to be excluded from the rule file.

-executeRule <type1,name1>: Defines the specific type name pairs that should be executed.

-executeRuleFile : Defines a file which contains specific type name pair list that has to be executed. Wildcard '*' can be used for "type" if the <name> is for all rule types and <name> is optional (can also use wildcard *),by which it covers all rules of this type.

-reportByPGArc : to only report pin pairs in pgarcs for the cell, if exist.

g. '`perform extraction`' builds network connectivity and extracts RLC values. Default - build power network and extract resistance. The syntax is:

```
perform extraction [-power | -ground] ?-l? ?-c?
```

where

-power : builds power network connectivity.

-ground : builds ground network connectivity.

-l : extracts inductance for specified nets.

-c : extracts capacitance for specified nets.

h. '`perform gridcheck`' generates a list of the relative power and ground circuit resistance for every instance (sorted by decreasing total relative resistance), or for standard cells and macro blocks separately, allowing an assessment of relative grid weakness for all nodes. The syntax is:

```
perform gridcheck ? -o <output_filename>? ?-limit <max_lines>?
    ? -perNet ? ? -perArc ? ? -box <bbox> ? ? -fullchip ?
    ? -stdcell [ ave | min | max | all | none ]?
    ? -macro [ ave | min | max | all | none ]? ? -package ?
    ? -lumpSwitch ? ? -fromSwitch? -printPin
    ? -excludeCellFile ? ? -excludeInstFile ? ?-excludeDecap?
    ? -byDistance ? ? -ignoreInterPloc ? -allPinType ?
```

where

-o <output_filename> : specifies output filename. Default name: *adsRpt/apache.gridcheck*

-ignore_floating : unconnected pin instances are not included in the output P/G Weakness report

-limit <max_lines> : if the '-perArc' option is not selected, higher resistance arcs are reported up to the limit and lower resistance arcs have fewer reports than the limit. If -perArc is specified, the maximum number of instances are reported for all P/G arcs. Default : 5000 per arc

-perNet : limits the number of report entries on a per-domain basis.

-perArc : gridcheck reports about the same number of instances for each P/G arc in the design, rather than reporting more P/G arcs with higher resistances.

-box : reports the P/G arc-based instance gridcheck resistance for all nodes in the specified box

-stdcell / -macro : selects the type of node resistance report, for standard cells or macro blocks

[ave | min | max | all | none]: 'ave' is the average resistance for all nodes in each instance selected, and is the default. For the 'min' and 'max' options, the node with the minimum or the maximum resistance value for the instance is reported, instead of the average of all nodes in the instance. The 'none' option is used to eliminate reporting on all standard cell or macro instances. For 'all', the largest 5000 node resistances in the design are reported.

-package : includes package resistance and connectivity in Spice netlist analysis

-lumpSwitch' : reports the lumped resistance of multi-finger switches, instead of single pin pair resistance

-fromSwitch : to perform gridcheck from Transistor on internal domain to the Power Switch.

-printPin : reports instance-specific power and ground pins in apache.gridcheck file.

-exlcudeCellFile , -exlucdeInstFile and -excludeDecap : to exclude cells/ instances/decaps during gridcheck.

-byDistance: reports the shortest physical path between two nodes instead of shortest electrical path.

-ignoreInterPloc: Ignore all the internal plocs and pick the external plocs. This feature can be enabled using the -set option, as follows:

```
perform min_res_path -set ignoreInterPloc on
```

-allPintype: inludes the non-P/G pins of the instances also in gridchek.

i. ‘`perform min_res_path`’ allows you to trace the minimum resistance path (SPT - “shortest path tracing”) between P/G pins of an instance or region and the pad connected to it, to identify weakly-connected instance, or between two specified nodes, including 45-degree routing. CMM instances are supported. Selects the node with the Worst Voltage Drop as the starting point for minimum resistance path tracing. Can be run prior to simulation if extraction has been completed. Note that in this case the IR drop column in the output report is blank, as IR analysis has not been performed. Reports voltage drop data on wires, vias and stacked vias in the report file *adsRpt/res_path.rpt*. The command also highlights the path in the GUI and creates a resistance bottleneck report that summarizes voltage drops and resistance along different segments of the path. The syntax is:

```
perform min_res_path -inst [<inst_name> |<region_name> ]
? -pin <pinA pinB ...> ? ? -allPin ?
? -pad {<pad1> <pad2> ... } ? ? -logicPin? ? -clear? ?
? -viaSwitch <switch_name>? ?-lumpSwitch ?  ? -addNode ?
? -from {<x1 y1> <layerName1> ?<netName>? }
  -to {<x2 y2> <layerName2> ?<netName>?}? ? -o <*.rpt>?
? -net <net_names> ? -allPinType ?-fromClamp ? ? -multipath
? -viaInst <inst> ? ? -fromSwitch ?
? -width ? ? -total_res ? ? -total_vol ?
? -simplified_report ? ? -append <filenam> ?
? -esdFail <esd_type> <N> ? -byDistance
? -nodeSel [minres |maxres | wstdrop ]?
? -set effR [On|Off]? ? -effR ?
? -set nodeSel [minres |maxres | wstdrop ]?
? -set fromClamp 1 ? -ignoreInterPloc
```

where

-inst : specifies the instance name or the block power assignment region name of interest. The minimum resistance paths are highlighted, connecting power pads and ground pads from the P/G pins of the specified instance or region

-pin: performs path tracing only for the specified pins of the instance. Other P/G pins are ignored in the GUI highlighting and text report.

-allPin : reports paths for all pins per specified instance. By default, a maximum of 10 pins are traced. If the instance has more than 10 pins, then the top 10 pins are picked based on the node selection specified (-nodeSel [minres|maxres|worstDrop]). To set a different limit, use the “-limit <num>” option.

-pad: specifies the pad or pads to which minimum resistance paths are found and highlighted from the P/G pins of the instance

-logicPin : reports the minimum resistance path to selected pin geometries of each of the logic pins. If two different pins are connected to the same net, it reports a path for each of the pins. The selection of pin geometry for the logic pin is determined by the “-nodeSel” option

-clear: clears all GUI highlights

-viaSwitch : traces minimum resistance paths through user-specified switch instance.

-fromSwitch : to perform min_res_path from Transistor on internal domain to the Power Switch.

-addNode : creates a node for accurate resistance calculation. Options “-from {<x1 y1> <layerName1> ?<netName>? }” and “-to {<x2 y2> <layerName2> ?<netName>?}” must also be specified. A node is created at that location, provided there is no node within a 1 uM radius, and there are wires at the specified location.

Please note the following points in using this option:

- Layer and net specifications are optional, but highly recommended.
- A node is created at the center of the wire, so it may not be the exact coordinate specified.
- You can use option “-searchDist <u>” to change the 1uM (default) allowable radius as desired.
- When a new node is created in this way, all previously recorded SPT/ gridcheck/min_res/ PF-S db data become invalid and are removed.

-lumpSwitch' : reports the lumped resistance of multi-finger switches, instead of single pin pair resistance

-effR: reports the effective resistance of the nodes. The capability can be turned on and off by using the “-set” option, as follows:

```
perform min_res_path -set effR [on|off]
```

The effective resistance value is in the comment column of each node (**View-> ESD Resistance List**). Once the effective resistance is computed, when you select the wire/via on an SPT path, the effective R is displayed, together with the shortest path resistance, as shown below:

```
Net: VSS
Wire: M2 [ Id = 358 ]
Width: 5 um
Length: 282 um
Coordinates: (1307.5, 0) (1312.5, 0) (1312.5, 282) (1307.5, 282)
Resistance:   3.59 Ohm   Metal Density:    0
Shortest path resistance at (1310.0000, 280.0000): 16.422104 Ohm
Effective grid resistance at (1310.0000, 280.0000): 1.435854 Ohm
```

-from/-to : allows tracing a resistance path from one node at or near <x1 y1> on layer <layerName1> to another node at or near <x2 y2> on layer <layerName2> (“point-to-point SPT”). Specifying the associated net names is optional. The minimum resistance path connecting nodes at <x1 y1> and

<x2 y2> is identified and highlighted. The output file gives the physical length of the calculated least-resistance path, the resistance value and the voltage drop for each segment of the path.

-o: redirects the output to the file specified, instead of the default file *adsRpt/res_path.rpt*

-net : defines the nets for which the minimum resistance path has to be traced.

-allPintype: inludes the non-P/G pins of the instances also in SPT tracing.

-fromClamp : traces from defined clamp nodes. To perform this, the clamp DB should be ready, using the “perform esdcheck -clamp <clampFile> -setupClamp” command executed before doing SPT tracing.

-multipath : traces and reports the top N shortest paths to all pads in the file *.adsRpt/multipath.rpt*. When the '-gui' option is used along with '-multipath', the top N shortest paths to all pads are displayed in the Multipath GUI dialog, which allows you to sort the results by the value of each column, and also select either a single line or multiple lines for display.

-viaInst <inst> : reports SPT paths to the bumps through the specified via instance. This option also can be used with the “-to < >” option.

-width : an additional column, Width, is displayed in the output report, showing the width of the metal wire segments.

-total_res: reports the total resistance of the minimum resistance path

-total_vol: reports the total voltage drop of the minimum resistance path

-simplified_report: specifies that a smaller simplified report file is to be generated besides the main report file. In the simplified file, each line represents one path, in the following format:

```
# instance masterCell pin net totalLength(um) totalResistance(Ohm)
```

The generated simplified file filename has a “_Simplified” suffix.

-append : the results of successive SPT runs are appended to the specified file; if it does not exist, Totem creates one. The default filename is *adsRpt/res_path.rpt*. If specified, this must be must be the last option in ‘perform min_res_path’ command.

-esdFail <esd_type> <N> - generates an SPT report for the top 'N' violations in ESD checking, by type (b2b, b2c, b2i, c2i, c2m). Default is 10.

-byDistance: reports the shortest physical path between two nodes instead of shortest electrical path.

-nodeSel : specifies which node (pin port) is the first on which to do path tracing inside an instance, where

minres - selects the node with the minimum grid check resistance as the first node (default)

maxres - selects the node with the maximum grid check resistance as the first node.

wstdrop - selects the node with worst voltage drop as the first node, if simulation is completed.

-effR : reports effective resistance of nodes.

-set effR [On | Off] : when On, sets the default value of effR for the session, calculating the effective resistance for all nodes (or until turned Off), particularly when using the GUI.

-set nodeSel <options> : when set, sets the default option value of nodeSel for the session, or until reset.

-set fromClamp 1 : turns on fromClamp SPT tracing . Then the following SPT executions are automatically traced from clamp nodes until it is turned off. This is designed primarily for SPT tracing from the GUI.

-ignoreInterPloc: Ignore all the internal plocs and pick the external plocs. This feature can be enabled using the -set option, as follows:

```
perform min_res_path -set ignoreInterPloc on
```

j. ‘`perform pwrcalc’` executes power calculation for generic (default), static, or dynamic analysis. The syntax is:

```
perform pwrcalc ?[-static | -dynamic]?
```

k. ‘`perform powermodel`’ creates a chip power model for system power integrity design. The syntax is:

```
perform powermodel [ -wirebond | <flip partitions>| -cdie | -static]?
? -parasitic ? ?-vcd ? ?<no model option-default>? ? -pincurrent ?
? -rleak ? ? -rleak_par? ?-solver mor? ?-plocname? ? -ind?
? -no_afs? ? -passive ? ? [ -noglobal_gnd | -global_gnd ] ?
? -repeat_current [ <start_time> | presim | best ]? -probe
? -internal_node -cell_file <cell_list_filename> ? ? -reportcap ?
? -o <output_filename> ? ? -reuse ? ? -io ?
? -esd ? -esd_subckt <netlist_file>? -esd_shorted_clamp_ok
? -port_grouping <group_cfg_file>?
```

where

-wirebond : specifies a wirebond package

<flip partitions> : -nx <num_x_ partitions> -ny <num_y_partitions>, specifies the number of partitions in the x and y directions. For -nx 1, -ny 1, a Cdie/Rdie report is generated in the *adsRpt/CPM/apache.Cdie* file.

-cdie : sets nx=1 and ny=1, and connects all power nets together and all ground nets together to obtain a single-port solution to obtain the equivalent Cdie and Rdie values for the chip. No current waveform is generated. A Cdie/Rdie report is generated in the *adsRpt/CPM/apache.Cdie* file.

Note that the -cdie option just provides faster run time by not calculating the current waveforms. For all other purposes it is equivalent to using ‘-nx1 -ny1’.

-static : creates static analysis chip power model, using DC conditions, a resistance-based circuit, and average current.

-parasitic: generates only the passive part of the CPM, without performing transient simulation, to generate the current signatures of the CPM ports, an extension of -cdie option for multi-partition CPMs. This can save time in transient simulation. However, the CPM model generated with -parasitic can only be used for DC and AC analysis (not usable for transient analysis).

-vcd : uses VCD file as basis for determining the worst case switching scenario

-pincurrent: specifies that CPM generate the model without current conservation (balanced current between Vdd and Vss) to achieve better correlation with Totem dynamic simulation results. In general, CPM enforces current conservation. However, in cases when Totem does not produce balanced VDD and VSS currents, this option can be used.

-rleak: causes the leak resistance to be added between ports on the VDD (power) net and the reference port on the VSS net.

-rleak_par: inserts leakage resistance between the VDD and VSS ports of each defined partition. Note that this option only works with partitioned CPM models. See the following section for more details on usage.

-solver mor: turns off the default ‘solver ac’ function, which is an accurate frequency-based linear solver AC solution with passivity enforcement.

-plocname: specifies pad/group names for CPM port names. If the '-wirebond' option is used, the subcircuit terminal names are taken from the pad names (if no grouping is specified), or the group names from the 6th column of the *.ploc* file. These group names are also known as SPICE node names, as this mechanism is used to connect a package using the keyword 'PACKAGE_SPICE_SUBCKT'. If this option is used with '-nx # -ny #' options, the node names are generated in the following form: PAR_0_0_VDD1, PAR_0_0_VSS2, ... For wirebond CPMs without any grouping, the subckt terminal name is the ploc name.

-ind: accounts for on-chip inductance, if the option '-l' of the Totem 'perform extraction' command has been used

Note: If you specify the '-l' option of the 'perform extraction' command, but not '-ind', inductance is ignored. Not specifying the '-l' option and using '-ind' is an error.

-no_afs : turns off the default AFS function. Adaptive Frequency Sweep for AC mode execution intelligently selects seven to nine frequencies (enough to reach convergence) to perform AC analysis, as opposed to 26 frequency samples that are performed by default in 'solver ac' mode. This option is recommended for design sizes exceeding 50 ports. Port count can be determined by computing N * M * P, where N = number of X partitions in the CPM (-nx option), M = number of Y partitions in the CPM (-ny option), and P = number of power and ground domains.

-passive : only effective with MOR function, which uses MOR to generate a passivity-enforced SPICE model, which can reduce accuracy. This mode can be useful if the SPICE simulation of the package/PCB CPM has convergence issues. The SPICE netlist is significantly smaller than using the default mode. But CPM is a compact model, so complexity is not a concern for cases with, for example, 100 bond pads, or up to 10x10 partitions for a flip-chip design.

-noglobal_gnd | -global_gnd: specifies the type of parasitic modeling in CPM, either (a) without Spice Node 0, using option '-noglobal_gnd' (the default), where there is a direct connection between the power and ground ports without going through Spice node 0, or (b) using Spice Node 0, using option '-global_gnd', in which the RLC parasitics from power and ground are connected to Spice global ground (node 0).

-repeat_current: specifies that the CPM current signature is repeated starting from the specified time point. Either a <start_time> in ns, the 'presim' time, or 'best' (chosen to cause the best continuity at the repeat point), can be chosen as the starting point of the repeating waveform. A warning is issued for incorrect values (such as a negative value or value greater than the maximum time of the PWL source). For non-zero values the closest time in the PWL definition is used. For example, if a '-repeat_current 1n' option is specified, and PWL time values ..., 900, 930, 960, 990, 1020, ... ps are defined, 'R=990 ps' is used. This is required for SPICE to accept the netlist. For the 'best' option to work well, the presim time and the transient simulation time need to be set to "n*T", where n is a positive integer, and T is the period of the clock frequency. With this option, the repeat time may be different for each individual PWL current source. Usage examples:

```
perform powermodel -nx 2 -ny 2 -repeat_current 0
```

which repeats starting from the beginning, t=0. Or,

```
perform powermodel -nx 2 -ny 2 -repeat_current 2n
```

which repeats starting from the time value in the PWL source definition closest to the 2ns time specified.

-probe: invokes the iCPM utility that enables the visibility of sensitive P/G connections in the design, and allows you to probe device locations inside the chip. You must set the PROBE_NODE_FILE GSR keyword, as described in section "iCPM- Internal Node Probing", page 8-255.

-internal_node: specifies additional ports located at P/ G pins on the same net that are to be shorted together to form one internal port. These nodes are named with the format '<instance name>_<netname>' in the CPM, such as “inst_1_VDD”.

Note that the options '-internal_node' and '-cell_file' are required to execute this feature, and the '-pincurrent' option should be used to keep the CPM currents at the correct value without further modifying the port currents, so that the sum of all port currents is zero. This feature also allows you to include/exclude the instance current profile and device capacitance from CPM creation, using the EXCLUDE option in the GSC file. To exclude instance current profiles and device capacitance, use the GSC syntax: '<instance name> EXCLUDE'.

-cell_file <cell_list_filename>: specifies a file containing a list of the instances whose internal P/G pins are to be exposed.

-reportcap: creates a log file report of all capacitance components included in the CPM generation. Example output:

```
Capacitance components -
     Intentional Decap - 0.000000e+00 pF
     Intrinsic   Decap - 2.404236e+02 pF
     Load        Decap - 1.945542e+02 pF
     Power grid  Decap - 3.260729e+00 pF
     Well        Decap - 0.000000e+00 pF
Total - 4.382385e+02 pF
```

-o <output_filename> : specifies output filename (default - *PowerModel.sp*, with the passive part in the file *PowerModel.sp.inc*)

-reuse : allows reuse of the generated current waveforms after one CPM run, if chip power models with different partitioning schemes are desired

-io : specifies that the I/O cells are to be included in the chip power model

-est : supports creating a CPM model for ESD devices. There are two methods provided, using PF-S IV curve definitions or Spice netlist/models for ESD devices. The ESD clamp filename must be specified with the GSR keyword:

```
ESD_CLAMP_FILES {
   <clamp_file_name>
}
```

Then execute ‘`perform powermodel -esd`’

Or, to include a Spice subckt file, run using the '`-esd_subckt`' option:

```
perform powermodel -esd -esd_subckt <netlist_file>
```

-esd_shorted_clamp_ok : enables inter cell shorting while creating output CECM netlist. By default, totem will prevent inter cell short creation.

-port_grouping <group_cfg_file>: to enable Power/Ground bump grouping on different layers and regions.

l. perform res_calc’ calculates the effective P/G grid resistance from all pads to selected instances, nets or locations, and provides an absolute resistance value in Ohms (whereas 'perform gridcheck' provides the minimum path resistance normalized to the maximum path resistance between points). The default invocation, without any options, creates a resistance report for the 1000 worst instances, as

indicated by quick estimation. The syntax is:

```
perform res_calc
 ? [-instance <name_list>]? ?[-instFile <filename>]? ? -addNode ?
 ? [-cell <name_list>]? ?-cellFile <cell_filename>?
 ? -nodeSel [minRes| maxRes| worstDrop]?  ? -box <llx lly urx ury>?
 ? -net <name_list> ? ?-layer <name_list>?
 ? -thread <num>? ? -mcore <num>? ? -allPoint? ? -loopmode ?
 ? -incremental? ?-append? ?-verbose? ?-limit <num>? ? -pernet ?
 ? -o <filename> ? ? -package ?  -fromSwitch ?
 ? -xtor <name_list>? ?-pin <name_list>? ?-allPin? ? -guardring ?
 ? -excludeCell <cellNames>? ? -excludeInst <instNames> ?
 ? -excludeCellFile <fileName> ? ? -excludeInstFile <fileName> ?
 ? -excludeDecap ? ? -gridcheckThreshold <threshold resistance>
 ? -fromLayer <layerName(s)> -fromInst <instName>
```

Note: the following options require a PathFinder license. See also section "Special Resistance Calculations for ESD Analysis", page 12-401**, for other ESD-specific commands.**

```
-shortAll
-from {<x1> <y1>} -from {<pt2>} ... -to {<pt3>} -to {<pt4>} ...
```

or

```
-to {<x> <y> ?<layer>? ?<net>? }
```

where

-instance : displays a report of the worst resistance location for the specified instance(s). The report contains worst case resistance for one location per domain per instance.

-instFile : same as the -instance option, except that you can specify a file containing a list of instances. The report contains worst case resistance for one location per domain per instance.

-addNode : creates a node for accurate resistance calculation. Options “-from {<x1 y1> <layerName1> ?<netName>? }” and “-to {<x2 y2> <layerName2> ?<netName>?}” , a node is created at that location, provided there is no node within a 1uM radius, and there are wires at the specified location.

Please note the following points in using this option:

- Layer and net specifications are optional, but highly recommended.
- A node is created at the center of the wire, so it may not be the exact coordinate specified. You can also use option “-searchDist <u>” to change the 1uM (default) radius as desired.
- When a new node is created in this way, all previously recorded SPT/ gridcheck/res_calc/ PF-S db data become invalid and are removed.

-cell : creates a resistance report of the worst resistance locations of all instance(s) for the specified cell masters. The report has one location per domain for each instance. <name_list> can be represented in two formats: {<name1> <name2> ... }, or <name1>,<name2>, ... (no spaces in this form)

-cellFile : specifies a text file that contains the cell names, one per line, for res_calc to compute the equivalent grid resistances.

-nodeSel : calculates and reports minimum resistance path, maximum resistance path or worst voltage drop path.

-box <llx lly urx ury> : creates a resistance report of the points within the box specified. By default, the report has a maximum 1000 locations within the defined box.

-net : specifies a list of nets for performing res_calc

-layer : reports the resistance values for specified layers.

-thread: specifies the number of threads, for speeding up the calculation

-allPoint: specifies that calculations are done for all points on specified instances

-loopmode : enables instance VDD + VSS resistance reporting. Note that when -loopmode is used, the value for '-limit' specifies the maximum number of instances, not nodes. A sample output report follows:

```
# Resistances from all pads to the listed points
# LOOP_R VDD_R GND_R   VDD(X Y LAYER NET)   GND(X Y LAYER NET) INSTANCE
80.9858 67.3984 13.5873 (4459.77 443.855 METAL3 VDD) (2095.97 561.905 METAL3
    VSS)   inst_129747/adsU1
4.41046 2.19395 2.21651 (4600.23 764.24 METAL1 VDD) 4600 767.93 METAL1 VSS)
    inst_509611
3.86578 1.86783 1.99796 (843.18 911.84 METAL1 VDD) 843.18 908.15 METAL1 VSS)
    inst_129995
2.59412 1.34763 1.24649 (2323.54 4074.99 METAL1 VDD (2321.08 4078.68 METAL1
    VSS)   inst_129228/inst_376373
2.58131 1.22812 1.35319 (2900.5 2230.53 METAL3 VDD) 2898.08 2204.01 METAL3
    VSS)   inst_129424/inst_92357
```

-incremental : checks the new res_calc database and computes the 'next N worst' instances, based on the gridcheck report.

-append: appends results to the output file.

-verbose: displays the full resistance report in the log window.

-limit <num>: specifies a maximum number of lines in the resistance report. Default is 1000. When -1 is specified, the program determines the limit based on the size of the design.

-pernet : limits the number of report entries on a per-domain basis.

-o <file>: saves the report to specified output file (default - *adsRpt/<design_name>.res_calc*). The general output format is shown below:

```
# Ohms Location <x y>    Layer  Net Pin Instance
2.58129 3316.37 3948.71  METAL1 VSS VSS instance1
```

-package : includes package resistance and connectivity in Spice netlist analysis

-fromSwitch : to report SPT from all power-switch gated-pins to a device terminal or specified location.

-xtor: calculates resistance for points on pins in transistors specified. Names should be separated by commas. (for MMX instances only)

-pin: calculates resistance for points on specified pins. Names should be separated by commas.

-allPin: calculates resistance for all pins on all MMX instances

-guardring: performs resistance calculation on the substrate guard ring

-excludeCell : specifies cell names to be excluded from resistance calculation

-excludeCellFile : specifies the filename that contains a list of cell names to be excluded

-excludeInst : specifies instances to be excluded from resistance calculation

-excludeInstFile : specifies the filename that contains a list of instance names to be excluded

-excludeDecap : specifies decap should be excluded from resistance calculation

-shortAll: shorts all points specified by a '-to option

-from {<ptn>}/ -to {<ptm> : performs resistance calculation from one or more nodes at or near <ptn> to one or more other nodes at or near <ptm>. The syntax for specifying <ptn> is { <xn> <yn> ?<layer>? ?<net>? }. Specifying the associated layer name or net name is optional.

-fromLayer <layerName(s)> -fromInst <instName> : -fromLayer <layerName(s)>' : works with the option '-fromInst <instName>'. This will allow user to specify layers for node selection on the specified instance.

-to { } (without '-from') : checks resistance from all pads to the specified point.

- gridcheckThreshold : to filter instances/pins above specified SPT resistance (in Ohms) and calculate effective resistance only for them.

pfs

---

pfs [add | delete | export | import | show] <options>

The 'pfs' command provides a number of functions to support ESD checking operations.

a. The 'pfs add clamp_pin' command creates shorted parallel clamp pin regions. Note that extraction must be run after adding or deleting shorted regions for the actions to take place.

```
pfs add clamp_pin <clamp_cell_name> ?-pin {<pin1> ... }?
? -layer {<layer1> ... }?  ?-region <x1> <y1> <x2> <y2>?
? -region <x3> <y3> <x4> <y4>? . . .
? -type <clampType>? ?-margin <distance(um)> ?
? -includeAll [1|2] ? ? -locID <location_ID>
? -pinpair {<pin_name1|locID1> <pin_name2|locID2>}
? -finX/-finY ?<num_fingers> ?-finAuto
? -par <partition_size>? ? -dist <um> ? > ? -multiPair ?
? -fwdR <forward_res> -bwdR <backward_res>?
? -perFwdR <per_pair_fwdR> -perBwdR <per_pair_bkwdR>?
? -ivname <iv_name>?
```

where

clamp_pin <cell_names> : specifies cell names for shorting

-pin <pin_name> : specifies pin names for shorting

-layer <layer_names> : specifies the layer name of the pins for shorting

-region <x1> <y1> <x2> <y2> : specifies the lower left and upper right corner x,y instance coordinates for the region for shorting. Multiple regions are also supported.

-locID <location ID> : specifies multiple regions in the same pin that are treated independently (in finger formation, shorting etc.)

-pinpair: to specify pin pair related attributes (-finX/Y, -fwdR/bwdR, -ivName wtc.) on a pair of pins (or locIDs)

-type <clampType>: To specify clamp type

-margin: expands the clamp bounding box by distance in um for node shorting considerations.

-includeAll: includes wire/via nodes in addition to pin nodes for shorting, where "1" means all wires/vias nodes inside the specified shorting bounding box are to be shorted, and "2" means all wires/vias nodes inside the expanded MARGIN shorting bounding box, but *not* inside the basic shorting bounding box, are to be shorted.

-finX/-finY <num_fingers>: enables automatic clamp finger pairing in horizontal or vertical direction for multiple finger devices. The fingers of the same pin

within the group are shorted together. Specifying a different number of fingers causes different finger pairing, as follows:

-finX/-finY : if <num_fingers> is not specified, each finger is defined by the PIN clamp file keyword. Each adjacent pin pair of different nets is recognized and defined by the ESD_PIN_PAIR clamp file keyword. When the clamp has multiple pin pairs(fingers), Pathfinder-S automatically calculate an appropriate scaling factor while generating clamp I-V curve using TCL command “pfs add clamp_pin -ivName” . It will include this scale value in the correct ESD_PIN_PAIR statement in the clamp file it generates in the format ESD_PIN_PAIR <pin1> <pin2> <ivName> <Scale>”. When Pathfinder-S uses the I-V curve, it divides all current values by the scale factor. User can also add scale factors in manually created clamp files.

-finX/-finY 1: each finger is defined by the PIN keyword. Each adjacent pin pair of different nets is recognized and defined by the ESD_PIN_PAIR keyword. Also, all nodes of the same finger are shorted together. When '-finX 1' or '-finY 1' option is used, the '-par' (number of partitions) value is automatically reset to 1 to avoid creating electronically redundant ESD_PIN_PAIRs.

-finX/-finY N (N > 1) : every N fingers are clustered as a group. All nodes of the same pin within the same group are shorted together. Pin pairs are formed between different pins of the same group. This speeds up ESD checking by reducing the number of clamp devices (ESD_PIN_PAIRs).

-finAuto : automatically determines the orientation of the clamp cells

-par <partition_size>: specifies partitioning of each finger into a specified number of pin segments for finer control of clamp pin modeling. By default each partitioned finger segment has at least one node for pin pairing.

-dist <um>: instead of controlling the value of ESD_PIN_PAIR with the '-par <count>' option, -dist allows you to create ESD_PIN_PAIRs on fingers per clamp instance based on a specified distance apart (spacing).

-multiPair: when “ESD_CLAMP_PIN_NODE_DISTANCE <um>” is set in the GSR, -multiPair creates ESD_PIN_PAIRs on all clamp fingers according to the node distance specified in “ESD_CLAMP_PIN_NODE_DISTANCE”, and you do not have to specify '-par/-dist' with a value when 'pfs add clamp_pin' is executed.

-fwdR <forward_res>: specifies the equivalent forward resistance (Ohms) of the group of pin pairs defined in the ESD_PIN_PAIR clamp file keyword (equivalent options are forwardR/-ron)

-bwdR <backward _res>: specifies the equivalent backward resistance of the group of pin pairs defined in the ESD_PIN_PAIR keyword (equivalent options are -backwardR/-revR/-reverseR/-roff)

-perFwdR /-perBwdR : specify forward and backward resistance (Ohms) for each ESD_PIN_PAIR generated in the cell template, instead of an equivalent distributed resistance as with “-fwdR” and “-bwdR” options.

-ivname<iv_name>: specifies the I-V model of the group of pin pairs defined by ESD_PIN_PAIR clamp file keyword

Note that the values of fwdR/bwdR are adjusted properly to all the pin pairs within the same region, so that the parallel combination of them equals the specified fwdR/bwdR.

-setupClamp: Generate clamp DB, typically used when user need not do extraction after modifying shorting regions

-shortNode{0|1|2}?: Node shorting option, same as GSR keyword, ESD_SHORT_CLAMP_PIN in which

0 (off): no clamp pin node shorting
1 (all): short all pin nodes
2 (loc): short all pin node locations as specified in clamp file.

b. The ‘pfs delete’ command deletes previous esdcheck results from the database, and also deletes previously created shorted clamp pin regions.

b1. To delete ESD results, the syntax is:

```
pfs delete ?-all? ?-type <rule_type>? ?-name <rule_name>?
```

where

-all : deletes all available ESD checking results

-type : deletes all ESD check results of the specified ESD type.

-name : deletes ESD check result of specified rule name. Note that the ‘-type’ option is required when the ‘-name’ option is specified.

Sample command to delete ESD results for rule ‘esd_b2b_rule’ of type b2b:

```
pfs delete -type b2b -name esd_b2b_rule
```

b2. To delete previously-created shorted clamp pin regions created with the ‘pfs add’ command, the syntax is:

```
pfs delete clamp_pin [ -all | -cell <cell_name> | -link ]
    ?-pin <pin_name? ?-pinPair {<pin_name1> <pin_name2>}?
```

where

-all : specifies that all shorting regions should be deleted

-cell : specifies the cell name for which shorting regions should be deleted

-link: to delete just the symbolic links in the directory adsRpt/ESD/ESD_DB.

-pin : to delete the specified pin setting in specified cell

-pinpair: to delete the specified pin pair setting in specified cell

c. The ‘pfs export’ command exports descriptions of shorting clamp region into the specified text file list of TCL commands that can be re-imported.

```
pfs export clamp_pin <output_file_name> ?-mergeDB?
    ? -esdCell <existing clamp_file>?
```

where

clamp_pin <output_file_name>: specifies the output filename

-mergeDB : merges an existing clamp DB with clamp pin settings into the specified output file

-esdCell’ : merges an <existing clamp file> with clamp pin settings into an output “template” file in a format that is similar to the clamp cell definition syntax, as well as include particular data options, if specified. The file syntax is:

```
pfs export clamp_pin <file_name> -esdCell ?-fwdR? ?<fwdR>?
    ?-bwdR <bwrdR> ? ? -ivname <I-V_model>?
```

where the file syntax keywords are:

-fwdR <forward_res>: specifies the equivalent forward resistance of the group of fingers defined in the ESD_PIN_PAIR keyword (equivalent options are forwardR/-ron)

-bwdR <backward _res>: specifies the equivalent backward resistance of the group of fingers defined in the ESD_PIN_PAIR keyword (equivalent options are -backwardR/-revR/-reverseR/-roff)

-ivname<iv_name>: specifies the I-V model of the group of pin pairs defined by ESD_PIN_PAIR

d. The 'pfs import' command is used to import the specified shorting clamp description file that has been previously exported using the 'pfs export' command.

```
pfs import clamp_pin <file_name>
```

e. The 'pfs show clamp_pin' command displays clamp pin shorting information.

```
pfs show clamp_pin ?-cell <cell_name>? ?-pin <pin_name>?
?-shorting? ?-layer <layer_name>? ? -pinPair ?
```

where

-cell, -pin, and -layer options select the object or group of objects displayed.

-shorting : specifies that all shorted clamp pin points matching the specified filtering options are displayed in white flight lines (after extraction). Otherwise, only the boundary of the shorting region is displayed.

-pinPair: displays paired-up pairs of clamp pins (yellow flight lines)

e1. The 'pfs show clamp_iv' command displays clamp I-V curves.

```
pfs show clamp_iv <clampIVName> ?-plot?
?-x min,max? ?-y min,max? ?-o <file>?
```

where

if '-o' is specified, an ASCII text file is dumped out that can be read by xgraph or sv.

if -o is specified and '-plot' is not used, no curve is plotted.

'-x' and '-y' : specify the ranges for X-axis (Volt) and Y-axis (Amp). The default for min,max is -10,10. Instead of '-x' and '-y', the following two options can be used

'-v' and '-i'

'-vol' and '-cur'

By default, the command plots the specified clamp I-V curve.

plot

---

plot [charge | current | rect | voltage | scatter | switching | line | analysis] <options> ?-o <outputFileName>? ?-sv ? ?-nograph ?

The 'plot' command generates graphical plots of simulation waveforms based on specified conditions. The definitions for the options that are available for several 'plot' commands are not repeated.

a. 'plot charge' creates a histogram of charge versus switching time. The syntax is:

```
plot charge ?-o <outputFileName>?
```

where

-o <outputFileName> : specifies output file name. Default - none

b. 'plot current' plots current waveforms for power, ground or individual nets or pads. The waveforms are extracted on the fly if not already created. Both Vdd and Vss currents are plotted. The output text file contains the working directory name and also the command used to create the text report in the header section. See the waveform viewer description under 'plot voltage' <options> following.

The general TCL command syntax is shown below; see the following examples for more specific usage and the option descriptions after the last one. The general syntax is:

```
plot current [-net ?-power|-ground? | -switch | -pad ]
    ? -virtual? ? -name <inst_name> ? ?-netname <net_name>?
```

```
? -inst_list <inst_list_file>?
? -ldo -name <LDO Instance name> ?
? -range <lowerNum> <upperNum>? ] ?-overlay?
?-o <output_file>? ?-sv ? ?-nograph?
? -region <x1 y1 x2 y2>?
?-probe -name <probe_name>? ?{<state1> ... <stateN>}?
```

where

-net -power |-ground : specifies plot of either total power or ground current

-switch : specifies plot of current for named switch instance

-pad : specifies plot of current for either all or of named pad instances

-virtual : plots current for specified low power virtual domain

-name : specifies the individual switch instance name, or pad name, for which to plot current.

-netname <net_name> provides the output sum of all switch currents associated with a specified net

-inst_list <inst_list_file> provides the output sum of all switch currents from the specified list

-ldo : displays output current waveforms of LDO instances for specified instances

-range <lowerNum> <upperNum> : specifies the current plots desired as ordered in the *pad.current* file

-overlay : adds a plot of total pad current to the total power or ground current plot

-sv : specifies that the waveform data in whatever native format is to be plotted using the Apache 'sv' program. By default the waveforms are extracted and rendered in xgraph or sv format. In general, waveform types supported by the 'sv' program are: **.tr0, *.ta0, *.ac0, *.sw0, *.hout, *.fsdb, *.wdb,* and **.pwl.*

-nograph : plot data is sent to the output file only, with no GUI display

-region : specifies corner coordinates for a rectangular region for which current is to be plotted, including presim switching current.

b1. plot current -net [-power | -ground] ?-o <file> ? ?-sv?

Plots the total current drawn from the power or ground nets.

b2. plot current -net [-power | -ground] -pad ?-o <file> ? ?-sv?

Plots the total current for all power/ground nets from the *pad.current* file.

b3. plot current -net [-power | -ground] -pad -overlay ?-sv?

The -overlay option adds the total pad current plot to the total power or ground current plot.

b4. plot current -net -name <net_name> -pad ?-o <file>? ?-sv?

Plots the current from all pads that belong to the specified net (from the *pad.current* file). The -pad option must be specified when -net and -name options are used.

b5. `plot current -region <x1 y1 x2 y2>`

Plots region-based current waveforms for all Vdd and Vss nets present in the specified region in the design. If the region is smaller than 50x50um, the command scales the region to 50x50um around the region specified. x1, y1 are the lower left coordinates, and x2, y2 are the upper right coordinates of the selected region. To plot the current for a specific domain in a region, add the following option:

```
plot current -region <x1 y1 x2 y2> -net -name <net_name>
```

b6. plot current -switch -name <switch_inst_name> ?-o <file> ? ?-sv?

Plots the current for the specified switch instance.

b7. plot current -pad ?-o <file> ?

Plots the current for all pads listed in the *pad.current* file. The maximum number of plots displayed is 30.

b8. plot current -pad -name <list_pad_names> ?-o <file> ? ?-sv?

Plots the individual current waveforms of pads specified from *pad.current*.

b9. plot current -pad -range <lowerNum> <upperNum> ?-o <file> ?

Plots the current of pads in the specified number range, as listed in the *pad.current* file. The maximum number of plots displayed is 30.

b10. 'plot current -pad -fft' displays an xgraph or sv plot of the pad current in the frequency domain.

Option descriptions:

-net -power |-ground : specifies plot of either total power or ground current

-switch : specifies plot of current for named switch instance

-pad : specifies plot of current for either all or of named pad instances

-range <lowerNum> <upperNum> : specifies the current plots desired as ordered in the *pad.current* file

-name <name>: specifies the individual net or switch name, or list of pads, for which to plot current.

-overlay : adds a plot of total pad current to the total power or ground current plot

-sv : specifies that the waveform data in whatever native format is to be plotted using the Apache 'sv' program. By default the waveforms are extracted and rendered in xgraph or sv format. In general, waveform types supported by the 'sv' program are: **.tr0, *.ta0, *.ac0, *.sw0, *.hout, *.fsdb, *.wdb,* and **.pwl.*

-nograph : plot data is sent to the output file only, with no GUI display

-fft: computes fast Fourier transform and plot frequency-magnitude result.

-start_time <time>, stop_time <time>: specifies start/stop time for FFT. The time unit can be sec, ms, us, ns, ps. Default is s.

-npts <number>: number of sampling points.

-fs <frequency>: sampling frequency. Units can be THz | GHz | MHz | KHz | Hz, default is Hz.

Note that -npts and -fs are used to calculate start and stop time using "stop_time-start_time = npts/fs". If neither start time or stop time are given, start time is assumed to be 0.

-window: specifies the use of a Hamming window w(k)=0.54-0.46*cos(2*Pi*k/(N-1)), k=0 N-1.+

b11. The command 'plot current -probe -name <probe_name> {<state1> ... <stateN>}' displays the current waveform in the Waveform Viewer at the defined probe locations for the states specified. For multi-cycle multi-state signal EM analysis, if the "-state" option is not specified, current waveforms for all states are displayed.

c. The 'plot line -vid' command allows drawing different lines in different views to highlight or display different issues within a multi-frame page/tab. The syntax is:

```
plot line -vid <frame_id> <plot options>
```

A "tab" or page view can have up to four frames, designated "A", B", "C", and "D". A multi-frame view then has up to 5 possible frame configurations, or relative positions for frames A, B, C, and D. A number of "get" functions are available to set up and manipulate the multi-frame displays from the Tcl command line, as described below. Responses to the commands are displayed in the log window.

- To get a unique frame ID for selected frames of a tab/page:
  `get_gui_vid -tab "<multi_page_name>" -frame [A|B|C|D]`
- To see all available tab names:
  `get_gui_tab_all`
- To get the current active tab name:
  `get_gui_tab_active`
- To get the tab configuration type for a specified frame:
  `get_gui_tab_type <frame_id>`
- To get the color map name from the frame ID:
  `get_gui_colormap <frame_id>`

d. the 'plot rect' command draws a specified rectangle on the design. The syntax is:

```
plot rect -position <ll_x> <ll_y> <ur_x> <ur_y>
    ?-fill ? ? -color <color_name> ?
```

where

-position : specifies the lower left and upper right x,y coordinates of the rectangle

-fill : fills the rectangle with same color as the rectangle lines

-color : specifies the color for the rectangle line, and the fill color, if specified (default white).

e. `'plot voltage' <options>`

Creates a Vdd, Gnd or Pad waveform plot for a specified instance or set of waveforms. The output text file contains the working directory name and also the command used to create the text report in the header section. The Totem integrated waveform viewer is the default viewer. A Snapshot menu button is available to take a snapshot of the waveform viewer for presentations, and a Search button to identify transistor pins in the signal browser, as shown in Figure D-3. Pattern matching in the search function is available to specify a partial device name and then click the **Enter** button. The Waveform Viewer automatically finds the proper matching entries and you then can cycle through multiple match entries by continuing to click the **Enter** button. Standalone waveform viewer mode also supports the *.hout* format as an input waveform file.

If the number of nodes is greater than 5M (default threshold), when the plot voltage command is executed the first time, the Vdd file is split into multiple Vdd files with 1M nodes in each file. From the second time the command is executed onwards, the 'plot voltage' command reads the split Vdd files. The 5M default threshold value can be adjusted using the GSR keyword 'SPLIT_VDD_EXTRACT_LP <value>'. The number of nodes in each split Vdd file can be adjusted using GSR keyword 'SPLIT_VDD_EXTRACT_LP_FSIZE <value>' (default 1M).

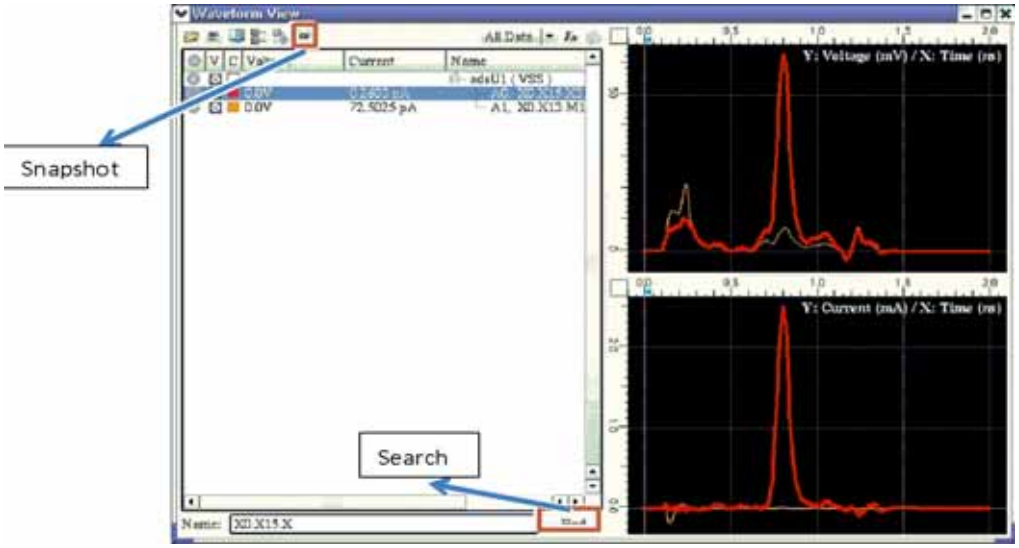


Figure D-3 Totem waveform viewer

For low power designs, use the -ext or -int options for switch instance voltages. A cross reference file named *cross_ref* is created to indicate which waveform data corresponds to which instance or pad, using actual net names. Note that there is no display when the '-file' option is used to request a large number of plots.

```
'plot voltage [ -name <inst_name> | -file <filename> ]
? [ -pad | -vdd | -gnd ]? ?[-ext |-int]?
? -virtual ? ? -ldo -name <LDO Instance name> ? ?-sub ?
?-netname <list of net names>? ?-pinname <list of pin names>?
? -o <output_file>? ? -nograph ?
```

where

-name <inst_name> : specifies the instance name or pad name for which the waveform is desired

-file <filename> : specifies a file that contains a list of pads or instances to have waveforms plotted, formatted one line per instance or pad name

-pad | -vdd | -gnd : specifies that PAD, VDD, and/or GND voltage waveforms are to be plotted

-ext | -int : for switch instances in low power circuits, selects which of the header or footer switch voltages are desired, internal or external.

-virtual: plots voltage for the specified internal low power voltage domain

-ldo : displays output voltage waveforms of LDO instances for specified instances

-sub : specifies substrate node plotting, which supports net names with hierarchical delimiters, "/".

-netname <list of net names>: specifies the instance net names for the waveforms desired.

-pinname <list of pin names>: specifies the instance pin names for the waveforms desired.

-o <output_file> : specifies either the output filename, or the directory name when used with the '-file' option. All the waveform data and cross file are dumped into that directory. The default directory is */wave/* . The plot naming

convention is <*output_file*>.<*Node*>.<*NetName*>, where <Node> is the node coordinates and layer separated by underscores. In the case of hierarchical net names, <NetName> is shortened and "/" is replaced by an underscore.

As an example, if the plot command is:

```
plot voltage -sub {4 5 NW} -o plotA
```

and the net of the node {*4 5 NW*} is *ua1/GHE/VDD2*, the name of the plot file written is

```
plotA.4_5_NW.u..E_VDD2
```

f. 'plot scatter'

Creates a scatter plot of switching instances on chip X,Y axes.

```
plot scatter ?-o <outputFileName>?
```

g. 'plot switching' ?-o <outputFileName>?

Creates a histogram of switching instances versus switching time.

h. 'plot analysis <options>'

Generates histogram results for all types of analyses. All GUI menu options for histogram analyses have a corresponding TCL command, as listed below:

```
plot analysis -type [ StaticIR | StaticEM | DynEM | DvD |
    signalem ] ? -emmode [ ave | rms | peak ] ?
?-layer [<layer_name> | ALL]? ?-net [<netname> | ALL]?
?-instance [avgTW | minTW | maxTW | minCyc |
instDrop | vddDrop | vssBounce]? ?-lowerLim <lower>?
?-upperLim <upper>? ?-o <filename>?
?[-binsize <size_mv> | -binnumber <number_bins>]? ?-nograph?
```

where

-type [StaticIR | StaticEM |DvD| DynEM |signalem |esd] : specifies the type of analysis histogram to be created, in units of mv for voltage and % of limit for EM.

-emmode [ave | rms | peak]: allows selection of the type of EM limit to plot.

-layer [<layer_name> | ALL] : for wire IR drop, specifies plot of specific metal layer name or plot for all metal layers

-net [<netname> | ALL] : for wire IR drop, specifies plotting Vdd drop or Vss bounce for a specific domain net name or ALL power nets.

-instance [avgTW | minTW | maxTW | minCyc | instDrop | vddDrop | vssBounce] : for instance voltage drop, specifies the parameter to use when generating the histogram:

avgTW : average Vdd-Vss difference over timing window

minTW : minimum Vdd-Vss difference over timing window

maxTW : maximum Vdd-Vss difference over timing window

minCyc : minimum Vdd-Vss difference over whole simulation

instDrop : Vdd minus Vss for static analysis

vddDrop : worst-case dynamic Vdd drop (minimum)

vssBounce : worst-case dynamic Vss bounce (maximum)

-lowerLim <lower> : specifies the lower limit of voltage shown in histogram.

-upperLim <upper> : specifies the upper limit of voltage shown in histogram.

-o <filename> : specifies output filename.

[-binsize <size> | -binnumber <number>] : specifies the step size, or the number of steps (in voltage) when generating the histogram.

-nograph : specifies no histogram will be displayed in GUI window.

h1. ‘plot analysis -type esd‘ command plots resistance histograms to support B2I, P2C, P2I and P2P checks.

```
plot analysis -type esd -ruleType <ruleType> -ruleName
<ruleName> ?-lowerLim <min>? ?-upperLim <max>? ?-binsize
<s>? ?-binnumber <n>? ?-o <output_file>? ?-nograph?
```

where

-ruleType : supports plots for rule types as follows: `BUMP2BUMP (b2b),BUMP2CLAMP (b2c), BUMP2INST(b2i), CLAMP2CLAMP(c2c),CLAMP2INST (c2i), CLAMP2MACRO(c2m), PIN2CLAMP (p2c), PIN2INST (p2i)`

-ruleName : results from user-specified rule name to be plotted

-o <output_file> : dumps out a text file report that can be used with xgraph or sv

print

---

print [charge | decap | dvd | instance | inst_data | memory | statistics | type] ? <options> ?

The ‘`print`’ command prints text-based reports for specified conditions.

a. ‘`print charge`’ prints charge statistics for each cell type. Apply ‘`condition set -xy`’. The syntax is:

```
print charge ?-o <outputFileName>?
```

where

-o : specifies output file. Default - output displayed in the terminal window where Totem was launched.

b. ‘`print decap`’ prints decap statistics for each cell type. Apply ‘`condition set -xy`’. The syntax is:

```
print decap ?-used? ? ? -netname <netname> ?
   ?-o <outputFileName>?
```

where

-used : excludes decap of floating pins for multi-vdd decap cells, and reports only those decaps or parts of decaps that are connected to analyzed nets.

-netname <netname> : specifies one net for which decap data is displayed

-o <outputFileName> : specifies output filename (no default file)

The command returns estimates of:

‘Non-switching Cap’ - the summation of cdev and C's in the SPEF and pin cap from all non-switching instances.

‘DevCap’ - the summation of the intrinsic cap from APL cdev values for all instances in the design.

‘LoadCap’ - the cap information in the SPEF files and pin cap from *.lib*.

‘Total PG Cap’ - from Totem analysis

‘Intentional Cap’ - cells tagged as “DECAP” in the DECAP_CELL GSR keyword.

Note that more accurate decap values are available by running CPM (Chapter 8, "Chip Power Models (CPM)".

c. ‘`print dvd`’ prints a DvD analysis report in N number sections of “timesteps” specified in the condition command. Default: 500ps. Report contains instances in the order of decreasing charge. Apply ‘`condition set -xy`’. The syntax is:

```
print dvd ?-limit [<number> | all]? -o <outputFileName>
```

where

- limit [<number> | all] : specifies the number of switching instances to be reported, or all instances. Default: 1000.

d. ‘`print instance`’ prints a list of instances and their switching status (preceded by 1 = switching, 0 = not switching). Apply ‘`condition set -xy -time -type`’.

```
print instance ?-o <outputFileName>?
```

e. ‘`print inst_data`’ prints information about the specified instance. The syntax is:

```
print inst_data <instance_name> ?-o <outputFileName>?
```

where

<instance_name> : specifies an instance for printing information.

f. ‘`print memory`’ prints a list of memory instances. Apply ‘`condition set -xy -time`’. The syntax is:

```
print memory ?-o <outputFileName>?
```

g. ‘`print statistics`’ prints a list of key statistics about the design, as in the following example:

```
Name of top cell:      GENERIC
Size of chip:          0 0 4920.62 5000.36

Number of power nets:  2
Number of ground nets: 1
Number of P/G pads:    80

Number of instances:   471853
Number of memory/ip:   6
Number of sub blocks:  0
Number of clock inst:  6022
Number of decaps:      133229

Number of nodes:       1018591
Number of resistors:   1149938
```

h. ‘`print type`’ prints a list of the number of instances of each cell type in the design. Apply ‘`condition set -xy -time`’.

```
print type ?-o <outputFileName>?
```

A sample ‘print type’ list:

```
Cell Type Statistics:
80878 of 476317 instances selected. 395439 ignored.
Rectangular region: Full chip
Type              Switch    Total    Switch %
-------------------------------------------
Combinational     6543      57900      11.30%
Sequential         784      16918       4.63%
Clock             3586       6060      59.17%
Special              0          0       0.00%
Memory/IP            0          0       0.00%
Total            10913      80878      13.49%
```

probe

probe [add | delete] ? <options>?

The ‘probe’ command allows selection and de-selection of particular nodes prior to extraction for which simulation output data are needed. No changes can be made to node selection after extraction; that is, if changes are made with the ‘probe’ command, extraction must be rerun. Note that probes also can be specified with the PROBE_NODE_FILE GSR keyword.

a. The ‘probe add’ command identifies locations for probing for which simulation output data is to be created. The syntax is:

```
probe add <x> <y> <layer_name> <probe_name>
```

For example:

```
probe add 1023.2 3457.1 Met1 nodeABC
```

To view current/voltage data for the probe locations, use the following commands:

```
plot current -probe -name  <probe_name>
plot voltage -probe -name  <probe_name>
```

b. The ‘probe delete’ command deletes specified previously-defined probes, or all previously defined probes. The syntax is:

```
probe delete [ <probe_name> | all ]
```

c. The ‘probe get’ command displays information requested about the specified probe by name:

- Displays the probe location with layer information (-location) .
- After extraction and simulation, displays current flowing through the named probe node (-current).
- In Static analysis, displays the current through and voltage at each probe node. For dynamic analysis, use the ‘plot’ commands.

```
probe get <probe_name> [ -location | -current ]
```

query

The ‘query’ command allows obtaining information on objects in the design.

a. The ‘query top’ command returns the bbox or name of the top level design. The syntax is:

```
query top [-bbox | -name]
```

b. The ‘query inst’ command returns the bbox of given instances in one of three forms:

- a TCL list of instances
- a pattern for name matching
- a filename that contains the instance names.

The syntax is:

```
query inst -bbox [<tcl-list> | <pattern> | <filename>]
?-o <file>?
```

report

The ‘report’ command creates various reports on Totem results.

report [ir | dvd | block_power_assignment/bpa | cmmpin | em_state | esdcheck | leafinst | power | res_calc | resistance | result] ? <options>?

a. ‘report ir/dvd’ commands provide reports of static and dynamic voltage drop values.

```
report [ ir | dvd] ?[ -routing | -instance ]? ? -wire ?
```

```
? -net [<net1> ... <netN> ] ?
? [-volt_bounce | -volt_drop | -neg_supply_bounce] ?
? -layer [ <layer1> ... <layerN> ]? ? -o <filepath> ?
? -threshold <volt_drop>? ?-limit <output_limit>?
? -precision <digits>?
? -bbox <x1> <y1> <x2> <y2> ?
```

where

ir : creates a static voltage drop report

dvd : creates a dynamic voltage drop report

-routing : generates a routing wire-based voltage drop report

-instance : generates an instance-based voltage drop report

-wire : generates voltage drop reports based on the largest drop for power and largest bounce for ground nets.

-net : specifies the nets to be reported. All nets are reported by default

-volt_bounce/-volt_drop/-neg_supply_bounce : reports voltage values for the selected nets. Note that combining power and ground nets at the same time is not allowed -- nets specified in a group should be either power nets or ground nets (see table below)

-layer : specifies the layers to be reported. All layers are reported by default.

-o : specifies the path for output file

-threshold : specifies a voltage drop threshold for reporting.
Note that '-threshold' is applicable only for the '-routing' option, and not with the '-instance' or ‘-limit’ options.

-limit : specifies the maximum number of output lines reported. When the option ‘-net’ is used, -limit refers to the maximum number of report lines per layer on which the net exists. Default is 1000 lines.

-precision : specifies the number of digits to report after the decimal point. Default: 3 digits, but some variation per type of report.

-bbox <x1> <y1> <x2> <y2>: reports IR/DvD results only for the specified bbox area

The output file has the following format:

```
#Report voltage information at each location (x, y)
# voltage  ideal_volt  net     x_y_location     layer_name
  0.0273   0.0000      VSS ( 811.480, 3446.870)  METAL1
  0.0275   0.0000      VSS ( 821.100, 3446.870)  METAL1
  0.0275   0.0000      VSS ( 824.780, 3446.870)  METAL1
  0.0276   0.0000      VSS ( 828.460, 3446.870)  METAL1
```

Option	Negative supply nets	Positive supply nets	Ground nets with 0V
-volt_drop	Reports nodes with voltages lower than ideal negative supply (max difference first)	Reports nodes with voltages lower than ideal supply voltage (max difference first) **Typical application**	Reports nodes with voltages lower than 0V (max difference first)
-volt_bounce	Reports nodes with voltages higher than ideal negative supply (max difference first)	Reports nodes with voltages higher than ideal negative supply (max difference first)	Reports nodes with voltages higher than 0 volt (max difference first) **Typical application**
-neg_supply_bounce	Reports nodes with voltages higher than ideal negative supply (max difference first) **Typical application**	Does not report any nodes	Does not report any nodes

Figure D-4 report voltage drop options

b. ‘report bpa’ produces a BPA summary report in the file *adsRpt/apache.blockPowerAssign*, which is the default file name for this command. With the '-node' option, it also reports nodes used for power/current assignment for the BPA instances.

```
report bpa/block_power_assignment ? -o <file>? ?-node?
```

c. ‘report cmmpin’ reports CMM pin static voltages, including boundary pin voltages, to insure that the interface of CMM blocks are at correct voltage. Static analysis must have been performed. A file is produced, *adsRpt/cmm_pin_voltages.rpt* by default, reporting the worst voltage for each pin of each CMM instance. The format of the report file is as follows.

```
## Report Format ##
<inst_name> <CMM_cell_name> <x_loc_of_pin> <y_loc_of_pin>
<layer_name> <net> <real_voltage> <ideal_voltage> <pin_name>
```

d. ‘report em_state’ loads and imports the results from the worst of the specified states from the GSC_FILE state definitions into the EM text/GUI reports, including single-state single-cycle or multi-state multi-cycle (MSMC) cases. To use this feature you must initially set GSR keyword ‘DYNAMIC_GENERATE_BRANCH_WAVEFORM 1’. If analysis is performed with this keyword turned on, you can use this command to switch current among states

```
report em_state ?-state { <state1> <state2> ... <stateN> }?
-weight_file <filename> ? -out_dir <outdir>
```

where

<state1> ... <stateN> are states defined in the GSC_FILE keyword

- weight_file : to specify state weight factor file for performing Iavg and Irms current scaling

-out_dir : when used, the avg, peak, and RMS output report files are copied to the specified output directory.

In the weight_file specified, user can specify the weight factor per state:

```
s1 w1
```

```
s2 w2
...
sN wN
```

The average and rms current of resistor branch will be scaled based on the weighted factor setting.

```
new_cur = cur_s1*w1 + cur_s2*w2, … + cur_sN*wN
```

d1. 'report em_state' without the '-state' option reports only the states that are currently used.

e. 'report esdcheck' specifies the type of reports to be created for the ESD checks. The command is:

```
report esdcheck -type <ruleType> ? -name <rule name>
? -b2cName <b2c_rule_name>? ?-[glob |-regexp ] ?
? -outDir <output_dir>? ? -excel?
? -reportB2C ? ? -reportC2B ?
? -arcR <arc_threshold>? ? -loopR <loop_threshold>?
? -parallelR <parallel_threshold>? ? -append?
? -cell { <cell1> ... <cellN> }?
? -failedOnly? ? -detail? ? -netPairVth <Vth_value> ?
? <net1>,...,<netN>? ? -peakVolt <peak_V?
?-diffVolt <diff_V>?
? -summary ? -sort [ name|em|I|R ]? ? ? -o <filename>?
```

where

-type: reports results for the specified type of ESD checks. '-type R' allows resistance checks from multiple point to multi-point ZAPS. Default is all types.

-name: specifies the name of the rule tested. If multiple checks have been specified within a single rule-- for example, two bump pairs are specified in the same rule-- PathFinder internally splits them into subrules, each with a single bump pair. Then it appends '(_\d+)+' to the base rule name. So when using 'report esdcheck', you can specify this derived name to look at the particular results desired.

-b2cName <b2c_rule_name>: user-specified B2C rule to be used for checking

-glob|-regexp: specifies the type of wildcard expressions used. PathFinder reports the results for all subrules that match the specification; 'glob' supports regular wildcard expressions and 'regexp' supports normal wild cards and also extended character strings. For example:

-outDir : specifies the report files directory. Default is *adsRpt/ESD*

-excel : creates a consolidated ESD report in Excel format for resistance and CD checks *(adsRpt/ESD/esd_excel.rpt)*, so that it can be easily sorted and processed using Excel, and also easily parsed by other flows downstream.

-reportB2C/-reportC2B: prepares reports on B2C/C2B CD checks in reports that have either bump-centric or clamp-centric results. By default, without either '-reportC2B' or '-reportB2C' options, 'report esdcheck' reports in the mode in which the esdcheck was performed.

-arcR: specifies the threshold for arc-R checking; required for B2C, C2C, C2I, C2M. The report format is:

```
# <Arc_R> <x,y loc> <layer> <domain> <pin> <des_inst>
  <clamp_inst> <clamp x,y location> <layer>
```

-loopR: specifies the threshold for loop-R checking; required for C2I

-parallelR: specifies the threshold for parallel-R checking; required for B2B

-append: appends results to existing results in the output directory.

-cell : reports CD results for specified cell names. For all cell names specified with the '-cell' option, the default output file is *adsRpt/ESD/esd_inst.rpt*.

-failedOnly: only reports failed results.

-detail : provides more detailed report results for B2B rules

-netPairVth: reports driver-receiver voltage pair threshold violations for current density (CD) checking rules. Optionally, one or more nets with the same VTH value can be specified, separated by commas.

-peakVolt: specifies the peak voltage threshold to be used for CD checks

-diffVolt : specifies the differential voltage threshold to be used for CD checks

-summary : prepares a summary esdcheck text report, primarily for CD checks, but some data is also presented for other rule types, if selected.

-sort : arranges the summary report items in the selected sorting order based on either rule name, worst EM % value, maximum clamp current, equivalent resistance between points. Default sorting order is by rule name.

-o : creates an output text report in the specified filename. By default, summary data is displayed to the screen only.

f. 'report leafinst' reports output adsU1 or adsCopy instance P/G pin voltages and geometries for GDS blocks in the 'report leafinst' output file. Reports not only pins of gdsmem, but also Black Box pins. The command syntax is:

```
report leafinst -pg_volt ?-geo? -o <filename>
   ? -f <input_file>?
```

where

-pg_volt: specifies pin voltages be reported. If '-pgvolt' only is used, the report is in the latest format, either with or without the -f option

-geo: specifies that pin geometries are to be reported. If '-geo' only is used, the report is in the original simplified format

-o <filename> : specifies the output filename

-f <input_file>: only reports instances/cells specified in this input file

Note: using both -pg_volt and -geo options is invalid.

The output report format is as follows:

```
<inst_name> <cell_name> <inst_x> <inst_y> <layer_name>
<domain> <eff_voltage> <source_voltage> <source_name>
```

where

<eff_voltage> is the actual voltage

<source_voltage> is the ideal voltage

A sample file is shown below:

```
core/mem/roms2/adsBoundary RODSH_ADSBOUNDARY 774.617 849.315 METAL4 VDD
    1.19579 1.2 VDD
core/mem/roms2/adsBoundary RODSH_ADSBOUNDARY 774.617 678.330 METAL4 VDD
    1.19561 1.2 VDD
core/mem/roms2/adsBoundary RODSH_ADSBOUNDARY 848.440 675.830 METAL4 VDD
    1.19562 1.2 VDD
core/mem/roms2/adsBoundary RODSH_ADSBOUNDARY 700.795 675.830 METAL4 VDD
    1.19549 1.2 VDD
```

g. 'report power' provides reports on design power data. The default is to report a summary of the whole design for all nets. The command syntax is:

```
report power ?-inst <inst_name>? ?-region <llx lly, urx, ury>?
?-region_name <region_name>? ?-xtorlist <xtorlist_file>?
?-net <net_name>? ?-mmx_only? ?-o <filename>? ?-all?
```

where

-inst: specifies the instance name (wildcards are supported)

-region <corner locations>: specifies the bounding box

-region_name: specifies the bounding box name for -region

-xtorlist: specifies a file containing device names (wildcards are supported)

-net: specifies the net names. Default is to report all nets.

-mmx_only: only reports power for MMX instances.

-o: dumps a summary and detailed report to <filename>

-all: generates report file *adsRpt/$designname.power.rpt*

h. 'report res_calc' produces a consolidated text report on the results of all res_calc operations performed. The syntax is:

```
report res_calc ?-o <file>? ?-loop?
```

where

-o <file>: specifies the output filename for res_calc results; the default filename is *adsRpt/<design>.res_calc*

-loop: reports the total of vdd+ gnd resistance for instances, as computed in the -loopMode option

i. 'report resistance' calculates and reports the resistance of a wire to verify the resistance calculation of Totem. The syntax is:

```
report resistance -layer <> -length <> -width <>
-density <> -spacel <> -spacer <> -colorw <> -colorl <>
-colorr <>
```

where

spacel and spacer are left and right wire spacing

colorw, colorl and colorr are the Mask Colors for the current wire, for left wire and right wire respectively. All dimensions are in microns.

Sample output:

```
report resistance -layer METAL5 -length 1526.24 -width 5
-density 0.278 -spacer 10 -spacel 0.1
Resistance: 22.931292
```

j. 'report result' creates a full set of Totem analysis reports and color maps as specified in the design constraint file. The syntax is:

```
report result ?-constraint <constraint_filename>?
?-o <output_file>?
```

where

-constraint <constraint_filename> : specifies constraint file name

-o <output_file> : specifies output filename

ring

The 'ring' command adds and deletes specified power grid rings from the design.

ring add <arguments>

See section "Mesh and ring commands - FAO functions to optimize and fix grids.", page 9-296, for details on the 'ring add' command.

ring delete <ring_name>

route fix

route fix [-from { x1 y1 x2 y2 <layer>} | -from_pt { xf yf <layer>}]

[-to {x1 y1 x2 y2 <layer>} | -to_pt { xt yt <layer>}] -width <float>

The ‘`route fix`’ command routes new power/ground wires and adds associated vias to the Totem design database to reduce excessive voltage drop.

See Chapter 9, "Fixing and Optimizing Grid and Power Performance", for details about using this command.

save

save design ?-o <dir_path>?

The ‘save’ command saves the design database for later use in a hierarchical design. The syntax is:

```
save <design_name> ?-o <dir_path>?
```

where

<design_name> : specifies the name of the design to save

-o <dir_path> : specifies the path to the directory in which to save the design

select

select [add | addfile | clear | clearall | get] ? <options> ?

The ‘`select`’ command selects and highlights objects in the GUI.

a. ‘`select add`’ adds the specified object to the selection.

Note: When a lot of objects are being selected, you can stop the selection process by pressing the keys CTRL + Break simultaneously, and the 'select add' operation stops.

The syntax is:

```
select add <object> ?-linewidth <width>? ?-color <color>?
   ?-name <groupname>? ?[-exact | -glob | -regexp]?
```

where

<object> : specifies a single instance or a TCL list of instance names. The names can include patterns (wildcard, expressions).

-linewidth <width> : specifies linewidth for the line used to highlight the selected object.

-color <color> : specifies the color used to highlight the selected object, from Xlib *rgb.txt* or their hex value.

-name <groupname> : adds a defined group of objects to highlight

[-exact | -glob | -regexp] : specifies object name matching criteria. Default - exact. ‘-glob’ allows use of wildcards, ‘-regexp’ allows use of regular expressions.

b. ‘`select addfile`’ adds objects specified in the file to the selection. The syntax is:

```
select addfile <FileName> ?-linewidth <width>? ?-color <color>?
```

where

<FileName> : specifies the filename containing a list of object.

c. ‘`select clear`’ removes the objects from the selection. The syntax is:

```
select clear <object> ?[-exact | -glob | -regexp]?
```

d. ‘`select clearall`’ removes all objects from the selection. The syntax is:

```
select clearall
```

e. ‘`select get`’ returns a list of object names that are currently selected:

```
select get
```

setup

---

setup [analysis_mode | .apache | apl | design | fsdb | package | pad | pss | switchres | wirebond | ldores] ? <options> ?

The ‘`setup`’ command sets up data and conditions required for performing Totem analysis.

a. ‘`setup analysis_mode`’ sets up data and conditions for the selected type of analysis prior to the ‘`perform`’ command, to save CPU and memory. To provide ESD-aware compatible CMMS (raw and binary), ‘setup analysis esd’ must be used for both model creation and top runs.The syntax is:

```
setup analysis_mode [static | dynamic |
     signalEM | esd ] ? -dmp <config_file>
```

where

-dmp : invokes distributed multi-processing for computation. User can also define a CUSTOM grid type in configuration file.

Example of config file for CUSTOM grid type:

```
GRID_TYPE CUSTOM
CUSTOM_SUBMIT <custom_command>
CUSTOM STATUS <script>
```

Totem will execute the custom command defined with CUSTOM_SUBMIT as "`<custom command> tmsemsim -config a -argua -argub -arguc ...`" through system call.

b. ‘`setup .apache <dir>`’ allows pointing the *.apache* directory to a user-specified local drive before loading the DB or the GSR file.

c. ‘`setup apl`’ generates setup data for an external APL run. The syntax is:

```
setup apl ?-dir <output_directory>?
```

where

-dir <output_directory> : specifies the directory for the APL setup file.

d. ‘`setup design`’ sets up a design database for analysis, after importing design files. The syntax is:

```
setup design ? <gsrFilename> ?
```

where

<gsrFilename> : name of GSR file

e. ‘`setup package`’ defines the package parasitic parameters R, C, and L for package equivalent circuits. RLC elements are considered a P/G loop, even if specified separately. The syntax is:

```
setup package [-c <c_val>] [-ignore | -power | -ground]
    [-r <r_val> | -l <l_val>]
```

where

-c <C_pF> : specifies equivalent package capacitance in picoFarads. Default: 0

-ignore : specifies package RLC parameters are to be ignored

-r <R_Ohms> : specifies equivalent package resistance in Ohms. Default: 0.

-l <L_pH> : specifies equivalent package inductance in picoHenrys. Default: 0

f. ‘`setup pad`’ defines the pad parasitics parameters. The syntax is:

```
setup pad [-ignore | -power | -ground] [-r <r_value> | -c <c_value> ]
```

where

-ignore : specifies pad parameters are to be ignored

-power : specifies the use of power RC parasitics.

-ground : specifies the use of ground RC parasitics.

-c <c_value> : specifies capacitance used for pad parasitics (pF). Default: 0.

-r <r_value> : specifies resistance used for pad parasitics (Ohms). Default: 0.

g. ‘`setup pss`’ defines the PLOC and package subcircuit files to be used in simulation. The syntax is:

```
setup pss -pad_file <filename> -subckt <pss_ckt_name>
```

where

-pad_file <filename> : specifies a PLOC pad definition file that must have a *.ploc* extension and be in PSS PLOC format.

-subckt <pss_ckt_name> : specifies the SPICE package subcircuit file

For example:

```
setup pss -pad_file test_pss.ploc -subckt pkgA.sp
```

h. ‘`setup switchres`’ defines or clears a switch resistance file specifying RON values, overriding the values of the switch model file:

```
setup switchres ?<file>? ?-clear? ?-Ron <Ohm>? ?-cmmOnly?
```

where

<file> : a file containing user-specific switch instance name and resistance value (equivalent to the GSR option SWITCH_RES_FILES)

-clear : to clear user-specific values

-Ron : specifies resistance value which is applied to all switches.

-cmmOnly : to apply -Ron to switch instances inside CMM only.

i. ‘`setup wirebond`’ defines wirebond or flip-chip parasitics. The syntax is:

```
setup wirebond [-ignore | -power | -ground]
    [-r <r_value> | -l <l_value> | -c <c_value>]
```

where

-ignore : specifies wirebond parameters are to be ignored

-power : specifies the use of power RLC parasitics.

-ground : specifies the use of ground RLC parasitics.

-c <c_value> : specifies capacitance used for wirebond parasitics (pF). Default: 0.

-l <l_value> : specifies inductance used for wirebond parasitics (pH). Default: 0.

-r <r_value> : specifies resistance used for wirebond parasitics (Ohms). Default: 0.

j. ‘`setup ldores`’ turns off (considers as open) or defines a resistance value for LDO cells during PathFinder functions.

```
setup ldores off | <R>
```

where

off : turns off or considers as open the LDO cells.

<R> : defines a resistance value for the LDO cells. Default value is 0.

Example in Tcl file:

```
setup ldores off
perform min_res_path
```

show

The ‘show’ command displays a colormap of specified Totem results in the GUI.

show <options>

The types of results that can be displayed by the ‘show’ command are the same as can be saved as a GIF by the ‘dump’ command. See the list of option names and descriptions in section "dump", page D-927.

snapshot

The ‘snapshot’ command saves a GIF screen shot of one of multiple frames, the waveform viewer, or the substrate technology layer viewer.

snapshot [-tab <name> | -waveform | -substrate] ? -o <filename> ?

where

-tab : specifies the multiple-frame tab name if there are multi-frame tabs displayed in main GUI window. If the tab name includes more than one space character, use “ “ or { } to group the words in the name.

-waveform : saves a snapshot of the Waveform Viewer

-substrate : saves a snapshot of the substrate technology layer viewer “Technology Layers (Substrate)”

-o : specifies the output GIF filename; default is *snapshot.gif*.

window

The ‘window’ command provides the ability to iconify and de-iconify the GUI window, the equivalent of the “-” button at the top right corner of the GUI window.

window [iconify | deiconify]

where

window iconify minimizes the GUI window

window deiconify maximizes the GUI window.

zoom

The ‘zoom’ command controls the GUI size display function, primarily for TCL script execution.

zoom [in | out | top | rect <x1 y1 x2 y2>]

where

in : increases the size of the objects in the GUI display by

out : decreases the size of the objects in the GUI display by

top : displays the full design

rect <x1 y1 x2 y2> : displays a rectangular portion of the design with corner coordinates x1,y1 and x2,y2

Running Totem in the TCL Script Mode

The following command runs Totem in TCL script mode by executing the commands in a specified TCL script command file. In this mode you do not have access to the GUI or to the interactive command line.

```
% totem -b <script_command_file> –lmwait <wait_time_in_sec>
```

The results from standard output can be redirected to a log file, as shown below.

```
% totem -b <script_command_file> >& <log_file>
```

The following commands run Totem in interactive command mode by executing the commands in a specified command file. In this method you have no access to the GUI, but can interactively issue TCL commands.

```
% Totem -i
% Totem > source <script_command_file>

% Totem -i
% Totem > type in tcl commands one by one

% Totem -i
% Totem > exit
```

The following command runs Totem from the GUI in various modes. Run Totem from the GUI by executing the commands in the command_file.

```
% Totem -f <command_file>
```

Run Totem from the GUI by using the “playback” of the commands in the script_command file.

```
% Totem
```

Click on menu: **Files -> Playback ->** <TCL_script_command_file>

Run Totem from the GUI by issuing the TCL command:

`‘source <TCL_script_command_file>’`

Use the window at the bottom of the GUI for entering TCL commands one at a time.

```
% Totem
```

In TCL command window type: `source <TCL_script_command_file>`

In TCL command window type TCL commands one by one.

Starting the GUI at a designated step in batch mode

You can start to run Totem in non-GUI (batch) mode initially, with a command prompt, and then display the GUI at a specified step for design examination. You must add the command 'gui ' in a command file after the command step where you want the GUI to be displayed, such as creating a command file *run.tcl*, as follows:

```
# Import data
import gsr GENERIC.gsr
setup design
# Calculate power
perform pwrcalc
# Power grid extraction
perform extraction -power -ground
# To display the GUI
gui
```

Then start Totem using:

```
totem -b run.tcl
```

When the ‘gui’ step is reached, the Totem GUI and the design are displayed. Now you can explore key aspects of the design and continue running commands from the GUI prompt.

TCL Script Execution Examples

The following TCL script command file performs DB exports at various stages of a normal Totem run.

```
import gsr user_data/design.gsr
setup design
export db Post_Setup_DB_Dir
perform pwrcalc
perform extraction -power -ground
export db Post_Extr_DB_Dir
setup pad -power -ground
setup wirebond -power -r <Res> -l <Induct> -c <Cap>
    -ground -r <Res> -l <Induct> -c <Cap>
setup package
perform analysis -static
export db Post_Simu_DB_Dir
```

DB directories *Post_Setup_DB_Dir, Post_Extr_DB_Dir*, and *Post_Simu_DB_Dir* are created. Each contains a snapshot of a Totem run at various stages of command execution.

The following script, using TCL commands, first loads the previously saved snapshot *Post_Setup_DB_Dir*, then continues with power calculation, extractions, and simulations.

```
import db Post_Setup_DB_Dir
perform pwrcalc
perform extraction -power -ground
setup pad -power -r 0.1 -l 0.1 -c 0.2
       -ground -r 0 -l 0 -c 0
setup wirebond -power -r 0.01 -l 0.1 -c 0.1
       -ground -r 0 -l 0 -c 0
setup package -r 0.002 -l 50 -c 5
perform analysis -static
```

The following execution script using TCL commands bypasses power calculation and extraction by loading the previously saved snapshot *Post_Extr_DB_Dir:*

```
import db Post_Extr_DB_Dir
setup pad -power -ground
setup wirebond -power -r <Res> -l <Induct> -c <Cap>
    -ground -r <Res> -l <Induct> -c <Cap>
setup package
perform analysis -static
```

Sample TCL Scripts

Static IR Drop Analysis Example

There are three ways to import design information into Totem:

- using the ‘`import gsr`’ command
- using the ‘`setup design`’ command
- using the `import gsr` command, and then specifying different design files to override those specified in the GSR file.

Following is an example of the three methods for importing design information for static voltage drop analysis using the TCL command format. Comment lines are preceded by "#" or "*" characters.

```
# importing data, method 1
import gsr design_static.gsr
setup design

# importing data, method 2
setup design design_static.gsr

# importing data, method 3 (not recommended)
import gsr design_static.gsr
import tech /work/6LM013um.tech
import def xyz.def
setup design
```

The TCL commands for running static voltage drop analysis are:

```
# begin static analysis
perform pwrcalc
perform extraction -power
setup pad -power -r 0.010000 -c 0.5
setup pad -ground -r 0.010000 -c 0.5
setup wirebond -power -r 0.245000 -l 1420 -c 5
setup wirebond -ground -r 0.245000 -l 1420 -c 5
setup package -r 0.002 -l 10 -c 3
perform analysis -static
export db design_static.results
```

In the above example, the pad cell *design.pcell* and pad location *design.ploc* files define the voltage and ground sources.

After power calculation is performed, the power results are automatically stored in the *adsPower* directory. If you have previously calculated power results, you will need to rename the existing *adsPower* directory, otherwise the new data will overwrite the existing information in it.

Power-net-only analysis and resistance extraction are defined by

```
perform extraction -power
```

The example ignores both capacitance and inductance extraction, as they are not required for static analysis.

The Totem database is exported to the *design_static.results* directory after static IR drop analysis is completed.

Dynamic Voltage Drop Analysis Example

Below is an example of a typical command file for DvD analysis using the TCL command format.

```
import gsr design_dynamic.gsr
import apl <cell>.current
import apl -c <cell>.cdev
setup design

#import power and begin dynamic analysis
```

```
import power adsPower_gold
perform extraction -power -ground -c
setup pad -power -r 0.010000 -c 0.5
setup pad -ground -r 0.010000 -c 0.5
setup wirebond -power -r 0.245000 -l 1420 -c 5
setup wirebond -ground -r 0.245000 -l 1420 -c 5
setup package -r 0.002 -l 10 -c 1
perform analysis [ -dynamic | -vectorless | -vcd ]
export db design_dynamic.results
```

The example shows importing of the pre-characterized decoupling capacitance file *<cell>.cdev*.

The power results are imported from the *adsPower_gold* directory. This assumes that the latest power results are already available in *adsPower_gold.* Optionally, you may re-calculate the power, in which case the power results are written to *adsPower* directory.

Simultaneous power and ground net analysis and resistance/capacitance extraction are defined by '`perform extraction -power -ground -c`'. The example ignores inductance extraction.

The Totem database is exported to the *design_dynamic.results* directory after dynamic voltage drop analysis is completed .

Automated Colormap Generation

The following TCL script creates six VDD voltage drop and six GND bounce color contour maps with values between 5% to 30% of nominal VDD value (max):

```
    set i 5
    set up 31
    while {$i < $up} {
    config colormap -percent -min 0 -max $i -wire
    config viewlayer -name metalonly -style fill
# or for Layer1: config viewlayer -name Layer1 -style fill
# for VDD
    config viewnet -name VDD -mode on
    config viewnet -name GND -mode off
    dump gif -map IR -o IR_VDD_$i.gif
# for GND
    config viewnet -name VDD -mode off
    config viewnet -name GND -mode on
    dump gif -map IR -o IR_GND_$i.gif
    incr i +5
}
```

Appendix E
Utility Programs

Introduction

This appendix describes key utility programs available in Totem:

- VCDTRANS: Generates a net toggle file from a Value Change Dump (VCD) file.
- VCDSCAN: Generates per-cycle power information from a VCD file for VCD-based dynamic voltage drop analysis.
- FSDBTRANS: Generates a net toggle file from a Fast Signal Database (FSDB) file.
- FSDBSCAN: Generates per cycle power information from a FSDB file for FSDB-based dynamic voltage drop analysis.
- `rhtech:` Generates a Totem technology file from standard *nxtgrd* or *itf* vendor technology files.
- `pt2timing.tcl :` PrimeTime TCL interface to generate the slew (transition times) and timing window for each instance.
- ACE: estimates the P/G pin intrinsic and intentional decap values for all types of memory and other complex IP blocks.
- SIM2IPROF: Generates current profiles from simulation outputs. Extracts “read”, “write”, and “standby” waveforms based on **.lib* formula.
- APLREADER: Converts data from a given current profile to standard output.
- APLMMX template: creates APLMMX configuration file templates for different modes automatically.
- clampviewer. Linux executable utility to preview ESD clamp I-V data.

Also, this appendix describes Totem GDS2DB functionality, which generates the design database natively (not an external utility).

vcdtrans

The `vcdtrans` utility generates the number of toggles for each net in a VCD file over the whole simulation time, or over frames of specified time. The input to the utility is a VCD file and the outputs are individual block toggle files called *<block>.toggle*. The recommended file extension for the toggle file is **.toggle*.

Syntax:

```
vcdtrans <input_file_name> -o <output_file_name> -c
    -s <start_time_in_ps> -e <end_time_in_ps>
    -w “<logical_module_name>” “<physical_module_name>”
```

where

<input_file_name> : specifies the VCD file.

-o <output_file_name> : specifies the toggle file.

-c : enables case-insensitive checking.

-s <start_time_in_ps> : specifies the start time, an integer with a default of t=0.

-e <end_time_in_ps> : specifies the end time, an integer with default of full simulation time

-w “<logical_module_name>” : defines the logical module name from VCD

“<physical_module_name>” : defines the physical module name from DEF.

`vcdtrans` can be run on a global VCD file to generate the global toggle file.

Note: If the specified time value of vctrans or vcdscan options -s and -e are smaller than 0.1, Totem assumes you want to use seconds as the time unit. Otherwise, ps is used as the time unit.

vcdscan

You can run VCD-based dynamic voltage drop analysis if you have a known-valid VCD file for your design. First, make sure that you have instance power calculation files generated by Totem, by default located in the *adsPower* directory. The `vcdscan` utility computes the peak power and per-frame power scenario. The default simulation frame size is 1/freq (1 cycle).

Syntax:

```
vcdscan <VCD_file_name> -f <cycle_time_in_ps>
    -w “<logical_module_name>” “<physical_module_name>”
    -msg <msg_file_name> -o <output_file_name>
    -d <input_file_dir_path> -cmd <command_file>
    -s <start_time> -e <end_time>
    -a <sim_frame_start_time> <sim_frame_end_time> -tt
```

where

<VCD_file_name> : specifies the VCD file from Verilog simulation.

-f <cycle_time_in_ps> : specifies the frame size or cycle time in picoseconds.

-s <start_time> -e <end_time> : defines the start and end times for a power calculation scan sequence, in ps

-a <sim_frame_start_time> <sim_frame_end_time> : defines the start and end time for the complete simulation frame, in ps

-w “<logical_module_name>” : specifies the logical module name from VCD

“<physical_module_name>” : specifies the physical module name from DEF.

-msg <message_file_name> : defines where the messages are saved. The default is print to screen.

-o <output_file_name> : defines where the output file containing frame-by-frame power data are saved. The default is <design>.pwr.

-d <input_file_dir_path> : defines the relative or absolute path to the input files. The default is the *adsPower* dir.

-cmd <command file> : defines a command file containing the vcdscan command options to be executed.

-tt : selects use of true timing results from vcd file for simulation

Example:

```
vcdscan VCD/design.VCD -f 3320 -msg vcdscan.msg
    -w "test/top" "" -o design.pwr
```

The output file *design.pwr* from the `vcdscan` utility contains the average power of each cycle, as well as the top five peak power cycles and their corresponding time steps.

NOTE:	Remember the peak power times (<FROM>, <TO>), the worst-case periods, as this information is used later in Totem during simulation.

To run VCD vector-based dynamic voltage drop analysis, invoke Totem by using the **Dynamic > VCD-based Dynamic IR-drop Analysis** command. The command prompts for simulation parameters, such as the peak power periods (<FROM>, <TO>) and the total simulation cycle time. Enter the simulation parameters and click **Start Simulation** to run VCD-based dynamic voltage drop analysis. If you already have peak power information from either intimate design knowledge or from a third-party tool, you can skip the `vcd` utilities and proceed directly to running **Dynamic > VCD-based Dynamic IR-drop Analysis** in Totem.

fsdbtrans

The `fsdbtrans` utility generates the number of toggles for each net in an FSDB file over the whole simulation time, or over frames of specified time. The input to the utility is a FSDB file and the outputs are individual block toggle files with names *<block>.toggle*. The recommended file extension for the toggle file is *.toggle*.

Syntax:

```
fsdbtrans <input_file_name> -o <output_file_name> -c
    -s <start_time_in_ps> -e <end_time_in_ps>
    -w “<logical_module_name>” “<physical_module_name>”
```

where

<input_file_name> : specifies the FSDB file.

-o <output_file_name> : specifies the toggle file.

-c : enables case-insensitive checking.

-s <start_time> : defines the start of scan time sequence in ps

-e <end_time> : defines the end of scan time sequence in ps

-w “<logical_module_name>” : specifies the logical module name from FSDB

“<physical_module_name>” : specifies the physical module name from DEF.

`fsdbtrans` can be run on a global FSDB file to generate the global toggle file.

fsdbscan

You can run FSDB-based dynamic voltage drop analysis if you have a known-valid FSDB file for your design. First, make sure that you have instance power calculation files generated by Totem, by default located in the *adsPower* directory. The `fsdbscan` utility computes the peak power and per-frame power scenario. The default simulation frame size is 1/freq (1 cycle).

Syntax:

```
fsdbscan <FSDB_file_name> -f <cycle_time_in_ps>
    -w “<logical_module_name>” “<physical_module_name>”
    -msg <msg_file_name> -o <output_file_name>
    -d <input_file_dir_path> -cmd <command_file>
    -s <start_time> -e <end_time>
    -a <sim_frame_start_time> <sim_frame_end_time> -tt
```

where

<FSDB_file_name> : specifies the FSDB file from Verilog simulation.

-f <cycle_time_in_ps> : specifies the frame size or cycle time in picoseconds.

-w “<logical_module_name>” : specifies the logical module name from FSDB

“<physical_module_name>” : specifies the physical module name from DEF.

-msg <message_file_name> : defines where the messages are saved. The default is printed to screen.

-o <output_file_name> : defines where the output file containing frame-by-frame power are saved. The default is <design>.pwr.

-d <adsPower_directory_path> : defines where the relative or absolute path is specified. The default is *adsPower*.

-cmd <command file> : defines a command file containing the fsdbscan command options to be executed.

-s <start_time> -e <end_time> : defines the start and end times for a power calculation scan sequence, in ps

-a <sim_frame_start_time> <sim_frame_end_time> : defines the start and end time for the complete simulation frame, in ps

-tt : selects use of true timing results from FSDB file for simulation

Example:

```
fsdbscan FSDB/design.FSDB -f 3320 -msg fsdbscan.msg
    -w "test/top" "" -o design.pwr
```

The output file *design.pwr* from the `vcdscan` utility contains the average power of each cycle as well as the peak power and its corresponding time steps.

NOTE: Remember the peak cycle times (<FROM>, <TO>), as this information is used later in Totem during simulation.

rhtech

The Totem Technology file specifies all required technology information in text format. The `rhtech` utility takes technology information provided in standard vendor *.*nxtgrd* or *.*itf* formats and translates the data into Totem readable tech file format. The syntax for invoking the `rhtech` utility from a UNIX command line is given below:

```
rhtech -i <input_file> [-o <output_file>]
[-m <layer_mapping_file>] [-e <EM_file>] [-t <temp_file>]
[-pe <foundry_specific_polynomial_em_file >]
```

where

-i <input_file> : specifies required vendor technology input file, in *nxtgrd* or *itf* format

-o <output_file> : specifies optional output file; default is '<*input_file-tech_name*>*.tech*'. Note that the output file contains several comment lines defining what input tech file generated the output, as shown in the example:

```
# Apache Totem Technology File
# generated by 'rhtech' program from file- 'in/tddmy09m7t.itf'
```

-m <layer_mapping_file> : specifies optional layer name mapping file. No default.

-e <EM_file> : specifies optional EM current density limit file; default is 1.0 .

-t <temp_file> : specifies optional temperature conditions file. No default.

-pe <foundry_specific_polynomial_EM_file > : to incorporate EM-Rule file section of GLOBAL_LIFE_TIME

The following provides more detail on input data to the utility:

-m : If needed, maps *.*nxtgrd* file layer names to the Tech file design layer names (not the same as layers specified in GDS2DB). Also specifies removal layers, such that if the ITF or *.*nxtgrd* file contains layers below metal1 such as poly, active and contact, these layers can be removed from the output file by adding them to the 'remove_layers' section of the map file. The format of the map file is as follows:

Map File Syntax

```
conducting_layers
    <Totem_layer_name> <input_techfile_layer_name>
via_layers
    <Totem_layer_name> <input_techfile_layer_name>
remove_layers
    <input_techfile_layer_name>
```

Example:

```
conducting_layers
    METAL1 metal1
    METAL2 metal2
via_layers
    VIA1    via1
remove_layers
    TEXT
    Poly
    Active
```

-e : Specifies the file providing the maximum acceptable EM current density. No EM limits are specified in the vendor technology files, so it must be specified here if required. The default value in tech file is 1.0 mA per micron wire width. The syntax of the specified EM file is:

```
<metal_layer_name> <EM_value> [<EM_adjust_value>]
<via_layer_name> <via_max_current>
```

where

<metal_layer_name>: tech file metal layer name

<EM_value>: 'EM' parameter value for specified metal layer in tech file (see section "METAL", page C-747)

<EM_adjust_value>: 'EM_adjust' option value for specified metal layer in tech file. If specified, the wire width used to calculate current density is biased as follows: calculated current density = true current in wire / (width of wire - EM_adjust).

<via_layer_name> : name of via layer

<via_max_current>: maximum allowable current for a via (mA) whose area is specified by the via Area keyword in the ITF file for that layer. The value is scaled for vias with a different area.

For example, if you have the following design rule, with an EM adjust of .01:

```
Metal                Imax (mA)
-------------------------------------------
Metal1              1.0 x (w-0.01)
Metal2              2.0 x (w-0.01)
...

Via     Imax (mA)
------------------------
```

```
Via1     0.189 per via
Via2     0.210 per via
```

then the *rhtech* EM input file should look like:

```
Metal1    1.0  0.01
Metal2    2.0  0.01
...
Via1      0.189
Via2      0.210
```

-t : Specifies the file providing the temperature at which the *nxtgrd* parameters are created, the final target Totem temperature, and the first and second order temperature coefficients. The syntax of the specified temperature file is:

```
<layer_name> <nominal_T> <Totem_actual_T> <Tc1> <Tc2>
```

where

<nominal_T> <Totem_actual_T>: nominal and Totem actual temperature conditions (ºC)

<Tc1> <Tc2> : first and second order temperature coefficients.

For example, if the input resistance parameters are at 25ºC, and the Totem analysis is to be performed at 125ºC, then in the temperature file you need to specify temperature data as follows for each layer:

```
<layer_name> 25 125 <Tc1> <Tc2>
...
```

Also note that for accuracy Totem can handle width-spacing-based sheet resistance data; `rhtech` can extract width-spacing based tables from the *nxtgrd* and *itf* files directly to the Apache technology file (as specified with the 'RPSQ_VS_WIDTH_AND_SPACING' keyword).

See section "Apache Technology File (*.tech)", page C-744, for details on the syntax and definitions of keywords in the technology file.

Native GDS2DB Handling

Certain data, such as memories, redistribution layers, flip-chip bump layer descriptions, and analog blocks, may only be available in GDSII format. This type of data must be converted to DEF/LEF format for integration into the Totem database. The `gds2db` utility generates a *.def* file from one GDSII file or a set of GDSII files. It is used to convert memory grids, flip-chip layers, extract a portion of the full-chip GDSII, and other applications where the block consumes negligible power.

Depending on the specifications, the program outputs a DEF file, or both DEF and LEF files, using the name of the cell, such as *<cellname>.def* and *<cellname>_adsgds.lef*. The created files contain the metal geometries of all the extracted nets. When these DEF/LEF files are used in Totem for analysis, the power distribution of these blocks are uniform and the current sinks are along the pins of the block.

If the GDS prboundary layer is missing for the cell, GDS2DB picks the cell bounding bbox from the cell geometries, and outputs it in LEF/DEF, as shown in Figure E-1.

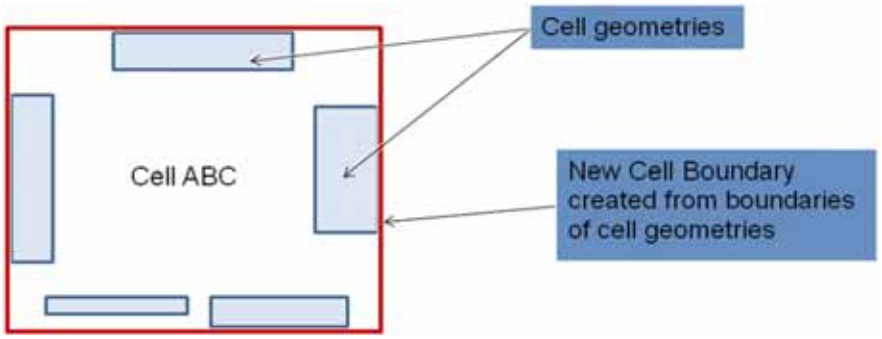


Figure E-1 Automatic cell bounding box determination

NOTE: Totem GDS2DB can process smooth 45-degree geometries for power/ground routing, including I/O rings/ RDL. Designs that specify 45° power routes are read in and handled smoothly in native 45° form. Other non-Manhattan shapes are not modeled.
GDS2DB also supports the MAG magnification keyword in the GDSII file that allows changing the size or dimensions of an instance relative to the master cell.

To perform GDS2DB setup, first create the GDS2DB configuration file and GDSII layer map file, as outlined following.

Creating the GDS2DB Configuration File

Create the GDSII configuration file *<gds_filename>.config.* There are four key sections in the GDSII configuration file:

- GDSII file: specifies a set of GDSII files for converting.
- Top cells definition: the name of the top cell structure(s) in GDSII.
- Nets definition: identifies the power, ground and signal nets that are labeled in text in the GDSII.
- Layer map definition: specifies the map file that contains information for mapping the GDSII layer numbers to the corresponding layers defined in LEF/DEF.

There are keywords for several optional function definitions in the GDSII configuration file:

- Input LEF
- Extraction layer geometries
- Memory core region definition
- Selective cell handling
- Auto pad location generation
- Other keywords

The keywords associated with each of the sections of the configuration file are described in section "GDS2DB Configuration File Keywords", page 3-100.

After the GDS2DB configuration file is properly set up, the database is created when Totem runs.

Special Applications

Cell Hierarchy Modeling

The “box cell” concept is useful in manually creating instance hierarchy inside the top cell to model sub-blocks for a top GDS cell in a hierarchical design. You may process a sub-block (hierarchical block) of the design separately and use the XX_BOX_CELLS configuration file keywords to create hierarchy. There are three types of box cell models:

- White box sub-block - creates a hierarchy and also all the sub-block geometries, including current sinks. Define using the keyword WHITE_BOX_CELLS.
- Black box sub-block - ignores the sub-block, including internal geometries. Define using the keyword BLACK_BOX_CELLS.
- Gray box sub-block - creates an instance of the sub-block without any geometries. Using Gray Box Cells, GDS2DB can handle large designs using manual hierarchy creation. Define using the keyword GRAY_BOX_CELLS.

See the detailed syntax and usage of these keywords in section "Selective Cell Hierarchy Handling", page 3-131, and section "Creating the GDS2DB Configuration File", page E-999, for a full description of the configuration file.

Several cases of using the Gray Box cells concept are described in the following section.

Case 1 - reducing memory use and run-time

This procedure resolves capacity issues, avoids extraction of geometries of sub-blocks, yet properly places bounding boxes of sub-blocks in the master. If the LEF file for the sub-block is already available from another tool, or by running GDS2DB for the sub-block first, use the LEF file to properly place the sub-blocks in DEF COMPONENTS section. The LEF must have the correct ORIGIN and SIZE for the sub-block. In this case put into the GDS2DB configuration file:

```
LEF_FILES {
 ./cellabc.lef
}
GRAY_BOX_CELLS {
 cellname 0
}
```

Case 2 - no LEF, but need bbox and pins at top level

This procedure is useful if you do not have a LEF file for the sub-block, but still want only the bounding box and pins of sub-block for top-level analysis. In this case put into the GDS2DB configuration file:

```
GRAY_BOX_CELLS {
 cellabc 0
}
CREATE_LEF_MACRO_FOR_BOX_CELLS 1
```

Internally, geometries are extracted for the sub-block to get the boundary box, so this has more memory and run-time use than in case 1.

Case 3 - no LEF, need bbox and pins, and retain P/G routing

This procedure is useful if you do not have a LEF file for the sub-block, but still want only the bounding box and pins of sub-block during top-level analysis. Also, you need to retain the P/G routing of block/top, which are only through the sub-block. In this case put into the GDS2DB configuration file:

```
GRAY_BOX_CELLS {
 cellabc 1
}
CREATE_LEF_MACRO_FOR_BOX_CELLS 1
```

Internally, geometries are extracted for the sub-block to get the boundary box. Tracing is enabled through the sub-block.

Case 4 - have LEF, need to retain P/G routing

LEF file for the sub-block is available, but need to retain the P/G routes of block/top, which are only through the sub-block. In this case put into the GDS2DB configuration file:

```
LEF_FILES {
 ./cellabc.lef
}
GRAY_BOX_CELLS {
 cellabc 1
}
```

Internally, geometries are extracted for the sub-block to enable tracing through the sub-block.

Case 5 - Using GDSII_FILE /GDS_CELLS to set up a Hierarchical Flow

GDSII_FILE and GDS_CELLS can be specified in the Totem GSR file, together with 'NATIVE_GDS_SUPPORT 2', to set up a hierarchical flow. The top-level should be part of the GDSII_FILE specification and the sub-blocks should be specified under the GDS_CELLS or DEF_FILES keywords. Logical connections between the sub-block and top-level are established automatically, based on sub-block DEF to top GDS pin overlap. Thus, if the power net at the top-level touches pin geometries of the sub-block defined in GRAY_BOX_CELLS, then the logical connection is established internally between the block and top level. Moreover, if the blocks specified under GRAY_BOX_CELLS do not have LEF/DEF specified under GDS_CELLS or DEF_FILES, the LEF file from GRAY_BOX_CELLS is automatically created with pins on the top two metal layers of the block (can be modified by PIN_CREATION_LAYERS). If there is a power/ground short between the block and top-level, for example, a VDD net in the sub-block touches a VSS net at the top (due to placement or other issues), logical connectivity is disabled and you must verify the block placement.

In 'NATIVE_GDS_SUPPORT 1' mode, this type of flow is also supported, but with only two levels of hierarchy, such as in a block-level run, no further sub-hierarchy should be defined using WHITE_BOX_CELLS and GRAY_BOX_CELLS.

Optimized geometry with Apache Physical Models

APACHE_PHYSICAL_MODEL

When set, reduces the node count for extraction and provides significant speed-up in network processing, with a minimal reduction in accuracy. To reduce unneeded geometries, below the specified APM layer only those geometries that are necessary to keep connectivity between the pins on the APM layer are retained. All geometries (wires/vias) are kept on or above the specified APM metal layer.

For current sources/sinks:

- pins are created on the specified APM metal layer.
- the bounding-box for a group of vias above the APM layer are formed, based on the intersection of wires at APM layer and the metal-layer above APM layer. Pins are created using this bounding-box, instead of creating a single pin below each via. Pratio is scaled, based on the number of vias used to create the pin.

- in memory core regions, pins are created with a 0 pratio value at the core-extraction-starting-layer. Core-extraction-starting layer should be the same or above APM layer.

For ESD analysis, APM supports expanded ESD clamp cell modeling by extracting detailed views inside clamps and a higher level of abstraction outside the ESD cells. If APACHE_PHYSICAL_MODEL is set and clamp-cells are auto-detected (using the ESD marker layer in the layermap), APM ESD flow is automatically invoked.

- for P/G nets, only geometries (wires/vias) on or above the specified metal layer are retained outside clamp instances, and all geometries are retained inside clamp instances. Also, no current sources/sinks are created outside clamp-instances, and current sources/sinks are created only inside clamp-instances.
- for signal nets, there is no change in modeling geometries and current sources/sinks.

This provides faster resistance and current density checks, without compromising accuracy.

If there are switch-cells in the design, during reduction of the wire-via network below APM layer, Totem retains the wires/vias that are necessary to maintain connectivity to switch-instance pins, to ensure that switches are not disconnected.

Default: none. *For GDS2DB only.*

Syntax:

```
APACHE_PHYSICAL_MODEL <APM_metal_layer>
```

Mini LVS Features

Layer Map Definition

The Layer Map file contains information regarding the metal layer numbers and text layer numbers associated with the metal layers, as well as via specifications. The format of the GDS2 layermap file allows multiple layer numbers to be specified, either as a layer number or text layer number. The layermap file syntax is

```
<Layer_name> <Layer_type> <Layer_num> <Text_layer_num> <Text_device_label>
    [ <Via_name> v <Layer_number> - ( <upper_margin> <lower_margin> ) ]
```

where

Layer_name : specifies the name of the layer

Layer_type: can be m = metal, v = via, c = metal cap, text, temp

Layer_num and Text_layer_num : can be specified by several layer-datatype pairs and layer numbers, separated by a ';' (semi-colon). Layer and datatype are separated by a ':' (colon). Text_layer_number can be ignored by inserting a dash '-'.

Text_device_label: text label for device

Via_name : specifies the via name. Note that multiple via layers may be specified between two metal layers.

<upper_margin> <lower_margin> : specifies the margin around the via on the upper layer and lower layer, respectively (default - 20 nm)

The following is an example of the layermap file format:

```
#Layer_name  Layer_type    Layer_num Text_lay_num   Text dev label
    METAL1         m          1:5;10:0       11:1;13
    VIA1           v          2;12           - (10 10)
    METAL2         m          13:0;15:50      -
    VIA2           v          3:12;3:43       -
    dio_txt        text       10:1;10:2       pw_diode_t;nw_diode_t
```

Logical Layer Geometry Definitions Using Boolean Expressions

Boolean expressions can be used to define logical layer operations in the GDS2DB layerMap file for layers such as 'diffusion', 'active', 'pplus', and 'nplus'. The syntax for defining logical layers is:

```
BOOLEAN <logic_layername> <layer_type> <Boolean_definition>
```

The general layermap syntax is:

```
<layer_name> <layer_type> <layer_number>:<datatype> <text_layer_number>
[ <Via_name> v <Layer_number> - ( <upper_margin> <lower_margin> ) ]
```

The following Boolean expressions can be used to define logical layer operations in the GDS2DB layerMap file.

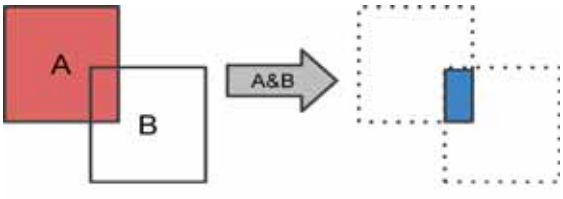


Figure E-2 (A&B) - intersection of the geometries of A and B

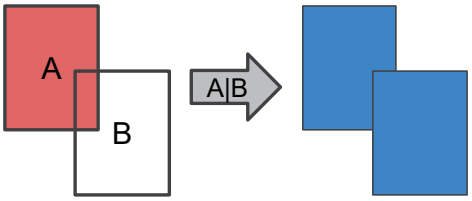


Figure E-3 (A|B) - either geometries of A or geometries of B

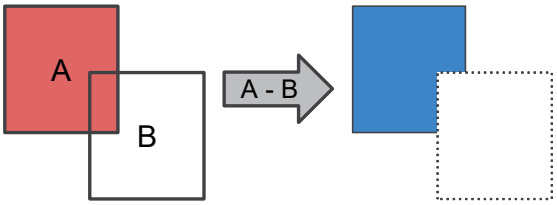


Figure E-4 (A-B) - inclusion of only the geometries of A that do not intersect with any of the geometries in B.

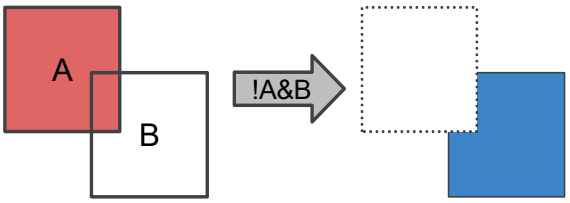


Figure E-5 (!A&B) - inclusion of only the geometries of B that do not intersect with any of the geometries in A. The same as B-A.

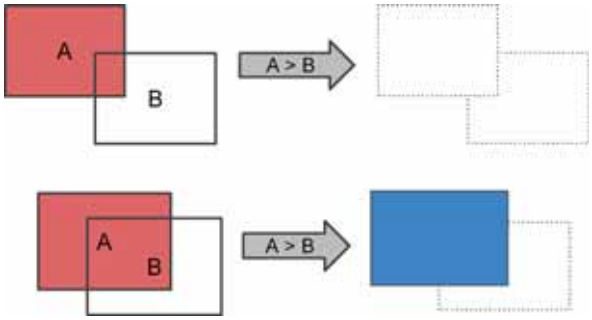


Figure E-6 (A>B) - inclusion of only the geometries of A whose centers lie within B. (A< B) performs the opposite operation.

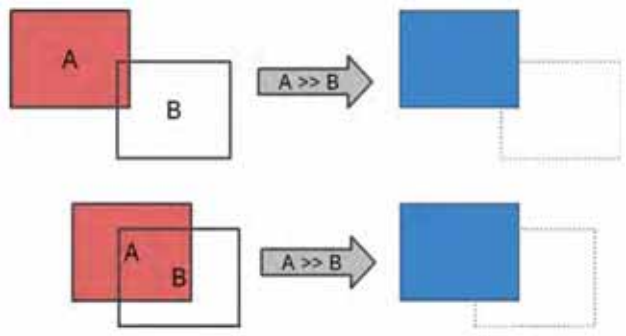


Figure E-7 (A>>B) - inclusion of only the geometries of A which touch B, irrespective of the centers. (A<<B) performs the opposite operation.

The operators described in the figures can be used to derive the following types of layers:

- Device (pdiff/ndiff/poly/cm)
- Contacts
- BumpVias
- Marker layers

Note that the BOOLEAN keyword works on a temp type layer only, and the resultant layer cannot be metal (m), via (v) or throughVia (tv). Nested temps also can be defined.

GDS2DB also does some special Boolean operations independent of the layerMap file, as follows:

- Metal-on-metal handling.
- Resistor handling (poly/diff/cm)
- Metal resistor handling

In the following layermap example, the device layers and the logical operator definitions are included:

```
nwell nw 21:0 -
poly poly 30:0 -
pplus temp 31:0 -
nplus temp 32:0 -
```

```
comp temp 22:0 -
cnt contact 33:0 -
m1 m 34:0 34:1
via1 v 35 -
m2 m 36:0 36:1
via2 v 38 -
m3 m 42:0 42:1
via3 v 40 -
m4 m 46:0 46:1
via4 v 41 -
m5 m 81:0 81:1
via5 v 82 -
m6 m 53:0 53:1
rv v 85 -
ap m 74:0 74:1
BOOLEAN ndiff pd comp&nplus
BOOLEAN pdiff nd comp&pplus
...
```

Note that it is assumed that the layer definitions are specified in ascending layer order. If the layers are specified in *descending* layer order, the layermap file specification in the GDS config file should be as follows:

```
GDS_LAYER_MAP <layer_map_file> DESC
```

Specific metal resistor support using the marker layer

By default GDS traces all metal layers. If you want to break GDS tracing, a metal resistor on the marker layer can be used. For specific metal-resistor situations in which two nets, for example netA and netB, are connected through a metal resistor, the metal resistor can be drawn in layout with the metal layer and a marker layer inside GDS. The metal resistor must be treated as “open”, so that netA and netB are not shorted. This type of metal resistor treatment is common to separate two nets in the layout. The layer map entry would look as follows:

```
<metal_resistor_name> metalres <resistor layer num> <metal layer num>
```

For this type of example, the GDS layer map file would be as follows:

```
m1 m 31:0 -  # normal metal definition
...
m3 m 33:0 -   # normal metal definition
mr1 metalres 110:11 31:0 # marker layer
mr2 metalres 110:12 32:0 # marker layer
mr3 metalres 110:13 33:0 # marker layer
```

FinFET Layer Support in GDS2DB

GDS2DB supports FinFET layer extraction by defining the FinFET marker layer in the layer map file, as shown in the example entry below:

```
RXFIN  findiff  2:42  -
```

Pins (single pins with multiple ports) are created on the overlap of cm, diffusion and findiff layers during GDS2DB modeling, so FinFET MOS device current is distributed to each fin. This feature is supported only in 'NATIVE_GDS_SUPPORT 2' mode.

Boundary Layer Definition

GDS2DB can create PR boundaries based on “b” type layer definitions in GDS. For example, in the GDS layer map file, you can define a “b” type entry as follows:

```
...
BDRY   b 0:0 -
```

To recognize this specification, in the GDS2DB configuration file you must set the keyword 'RETAIN_PRBOUNDARY_FOR_TOP_BBOX 1', so the PR boundary layer in top-cell is then used as the bounding-box for the top cell, even if there are geometries in the top cell extending outside the PR boundary.

To get a sub-block bounding box in the hierarchical flow, Totem honors sub-block prboundary geometry defined in the layer map. You must provide LEF for sub-blocks (generated with the prboundary defined in the layer map), which then is stitched up with the top-level seamlessly. The configuration file keyword is:

```
RETAIN_PRBOUNDARY_FOR_BOX_CELL_BBOX 1 (default 0)
```

Extracting multiple via layers between two metal layers

You can extract multiple via layers between two metal layers-- vias that are specified with different layer numbers in the GDS mapping file. They also need to be defined in the technology file, with their corresponding electrical properties, such as resistance. An example GDS mapping file would look as follows:

```
...
M8 m 18 -
VIA8_a v 50 -
VIA8_b v 51 -
AP m 19 -
```

Modeling through-via support

Native through-vias can be modeled as vias connecting two non-consecutive metal layers, using the keyword 'ENABLE_NATIVE_THROUGH_VIAS 1'. If this keyword is not specified, or is set to 0, through-vias are modeled as stacked-vias from bottom to top layer, instead of as a single through-via from a bottom to a top metal layer; in other words, all metal bounding boxes from upper layer to lower metal layers appear in the viamodel definition in this case. The format for specifying through-vias in the layer map file is as follows:

```
tv -
THROUGHVIA
```

Example layer map file:

```
TSV tv 10 -
BM m 11 -
VIA_DUM v 999 -
M1 m 31 131
MCAP c 2 - MCAPM2VIA
VIA1 v 51 -
M2 m 32 132

THROUGHVIA TSV BM M2
```

where

TSV: the through-via layer name. Via names (for the through-via geometries) in the output DEF file also start with this name.

BM: specifies the lower metal layer of the “TSV” via layer.

M2: specifies the upper metal layer of the “TSV” via layer.

Clamp Cell identification based on marker layer

ESD clamp cells can be automatically identified and extracted from GDS by specifying marker layers in the layer map. A sample layer map is shown below:

```
# <device_name>  ESD <cell_ID_layer> <pin_ID_layer1> <pin_ID_layer2>
CLAMP ESD 215:20 215:1 215:2
D1    ESD 215:21 215:1 215:2
D2    ESD 215:22 215:1 215:2
D3    ESD 215:23 215:1 215:2
D4    ESD 215:24 215:1 215:2
D5    ESD 215:26 215:1 215:2
D6    ESD 215:27 215:1 215:2
```

The presence of a marker layer inside the cells causes GDS2DB to classify them as ESD devices and output them in LEF/DEF format.

GDS2DB support for MOM device handling

The procedure for handling MOM (Metal on Metal) devices is as follows:

1. You must define “MOM” devices in the APLMMX configuration file keyword “DEVICE_MODEL_CONFIG <config_file>”. The format of the file is:

```
<model_type> <model_name> <marker_name> <pin_name1> (<pin_layer_name1>)
    <pin_name2> (<pin_layer_name2>) ...
```

2. APLMMX puts MOM device data into *<cell>.gdsmmx* files for use by GDS2DB.
3. You must specify a marker layer for the MOM devices in the GDS layer map.

   For MOM cap marker layers provided in the GDS layer map file for GDS2DB, all geometries overlapping MOM markers of cells containing the MOM marker layer are ignored, to avoid double counting the parasitic coupling effect inside MOM devices. MOM layers should be defined in GDS layer map file as shown below.

   ```
   <MOM layername> mom <layer> -
   ```

   For example,

   ```
   md  mom  150:1  -
   ```

   In the design shown in Figure E-8, hierarchy A contains a MOM marker layer. Hence geometries from A and B1 overlapping the MOM marker are ignored. None of the geometries of other cells are ignored.

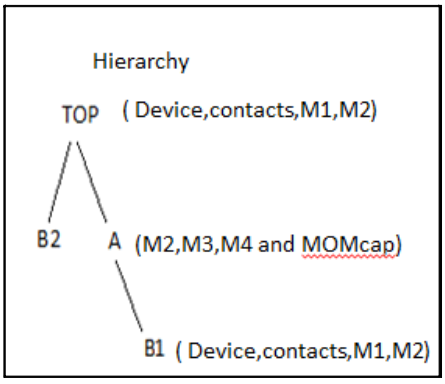


Figure E-8 MOM cap handling

4. If a MOM device does not overlap a MOM marker layer, a threshold value can be specified to help identify which MOM cell it belongs to, by using the following GDS2DB configuration file keyword syntax:

   ```
   MOM_SNAP_THRESHOLD_DISTANCE <threshold_um>
   ```

5. Pins are created on the corresponding nets/layers by GDS2DB.

Bump Via Support

Bump vias are special vias between two metal layers in addition to the regular vias between these metal layers, as shown in Figure E-9. Bump vias have the following properties:

- their electrical properties differ from normal vias between the same metal layers, so they need to be modeled as a separate layer.
- they are generally defined as logical operations between two other GDS layers.

As with all layer specifications, the Bump Via layer has to be specified in the GDS layer map file, and similarity with “THROUGHVIA” support is maintained. The format for specifying bump vias in the GDS map file is as follows:

```
BUMPVIA <bump_via_layer_name> <logical_layer_operation>
      <associated_via_layer_name>
```

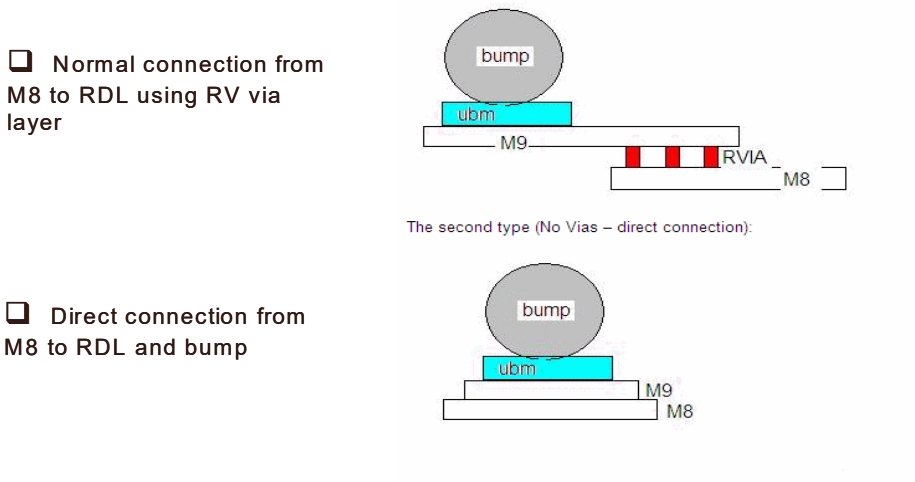


Figure E-9 Bump via modeling

Sample layer map file with bump vias would look like:

```
M8 m 38 138
RV v 85 -
M9 m 74 126
CB2 temp 86 -
CBD temp 169 -
BUMPVIA BV (CB2&CBD) RV
```

where

BV: the bump via layer name. The via names (for the bump via geometries) in the output DEF file also start with this name.

(CB2&CBD): specifies the logical layer operation that defines the bump via geometries. Any valid regular expression is supported in the GDS2DB logical layer operations for device layers. As is the case of device layer logical layer operations, all layers that are part of the expression MUST already be defined as “temp” layers in the layer map file.

RV: specifies the associated via layer name--that is, the regular via layer name at whose level the Bump Via is created. So the bump via has the same upper layer and lower layer as the ‘RV’ via layer.

pt2timing

Totem's PrimeTime TCL interface *pt2timing* is used to obtain timing information such as slew (transition times) and timing windows. This information is used by Totem to improve the accuracy of both static IR and Vectorless Dynamic voltage drop analysis. For static IR drop analysis, the slew information is used to calculate the average power. In addition, for dynamic voltage drop analysis, the timing window information is used to calculate the peak power and voltage drop.

The following steps are performed in the PrimeTime environment:

1. Invoke PrimeTime.
   ```
   pt_shell
   ```
2. Read in the design.

   If a fully annotated PrimeTime database is available, use the `read_db` command:
   ```
   pt_shell> read_db <path>/<design_db_name>
   ```
   If a fully-annotated database is not available, then the Verilog netlist and library must be read using the `read_verilog` command:
   ```
   pt_shell> read_verilog <path>/<verilog_netlist>
   ```
3. Link the design.

   Ensure that all link paths are set correctly before linking the design.
   ```
   pt_shell> link_design
   ```
4. Read in the parasitic information.

   The parasitic information yields accurate timing information in the design:
   ```
   pt_shell> read_parasitics <path>/<parasitic_netlist>
   ```
5. Set up case analysis and constraints.

   The appropriate case analysis and constraints must be set up prior to calculating timing information. Case analysis is required to get the correct operating conditions for clock gating or muxing. Source the appropriate PrimeTime constraint files for your design.

   Note that STA tools such as PrimeTime use BC/WC analysis as default, where the worst-case corner conditions are used for the Setup time check and the best-case corner conditions are used for the Hold time check on a clock path. If the worst pin timing is used for min_arrival and the best pin timing is used for max_arrival, then the min_arrival can be greater than max_arrival. To avoid this situation, make sure that the following conditions are met prior to running PrimeTime timing analysis:
   - the SPEF file is generated for worst-case corner conditions
   - the *.lib* and *.db* files contain worst-case corner conditions
   - For PrimeTime script, use `set_analysis_type single` and `set_operating_condition` or `set_min_library` to be ‘worst’.
   - Use a propagated clock.

6. After loading the design in Primetime, source the *pt2timing.tcl* script. The script extracts the slew (transition times) and timing window information. The default units for these values are nanoseconds (e-9). Note that if ILMs (Interface Logic Models) are used for the full-chip timing, then the ILM must be disabled prior to running the *pt2timing.tcl* script.
7. Write out the timing file using the command 'getSTA'. A typical PT command file for generating the STA file looks as follows:

```
set search_path ". ./dbs/"
set link_path "* lib1.db"
read_verilog generic.v
current_design generic
read_parasitics generic.spef
read_sdc generic.sdc
update_timing
source <path>/pt2timing.tcl
getSTA *
```

   Note that by default no explicit timing updates are performed prior to timing window generation, which can reduce runtime by up to 25%. A full timing update can be invoked prior to timing window generation by using the command 'getSTA * -full'. Incremental timing updates can be invoked prior to timing window generation by using the command 'getSTA * -inc'.
8. Primetime exits after generating the timing file. The output file is saved as *<design>.timing* in the working directory.
9. The *pt2timing* script internally sets false paths on all clock domains before generating the timing windows. It has no impact on timing window accuracy, but greatly improves timing file generation runtime. So, after executing getSTA script it is not possible to report timing and slack. If you do not want to exit from Primetime after timing window generation, you can use '-noexit' switch

```
getSTA * -noexit
```

10. If you are running *pt2timing* on blocks inside the top level design, you should use the –b switch. This generates timing windows for primary input/output ports also.

```
getSTA * -b
```

ACE Decap and ESR Characterization

ACE estimates the P/G pin intrinsic and intentional decap values for all types of memory and other complex IP blocks. It uses a unique tracing algorithm to find the P/G network in a cell, and calls SPICE to characterize representative MOSFET's gate capacitance, and generates various types of capacitance and effective resistance values for the cell in APL format, as well as an estimate of leakage current. ACE also traces R elements in power/ground nets using DSPF information (*|NET, *|I ...) to identify relevant capacitance. You can then inspect, merge, and import ACE results into the Totem database to perform DvD analysis.

ACE can recognize header/footer switches embedded inside a memory or a functional block. This is especially useful for low-power design, since many cells come with the switches built-in. Since ACE uses a net tracing algorithm, it runs extremely fast. For a half-million MOSFET cell, the typical run time is approximately a minute on a Linux box.

Identifying tied-off devices as decaps

ACE automatically identifies tied-off devices as decaps, including the following configurations of device connections:

For tied-off conditions, treat as decaps.

a. nmos:
   - source/gate ties to ground, drain to power.
   - drain/gate ties to ground, source to power.

b. pmos:
   - source/gate ties to power, drain to ground.
   - drain/gate ties to power, source to ground.

For the following combinations, assign zero current, but do not treat as decap:

a. nmos:
   - source/gate ties to ground, drain to signal.
   - drain/gate ties to ground, source to signal.

b. pmos:
   - source/gate ties to power, drain to signal.
   - drain/gate ties to power, source to signal.

ACE Configuration File

ACE requires an input configuration file, using the keywords described on the following pages (names are not case sensitive). The required keywords are:

- DEVICE_MODEL_LIBRARY <device_model_library_file>
- EXTERNAL_POWER_NET
- EXTERNAL_GROUND_NET
- SUBCKT
- MODELFILE

  Note that new nanometer device model libraries are often too complicated to be specified using MODELFILE. The libraries also can be encapsulated using the INCLUDE keyword instead of MODELFILE.
- VDDVALUE

Other configuration file keywords are optional, and are described in the following section.

Ace_HSpice <path/binary>

Specifies that HSpice is used as the characterization simulator. You only need to specify this keyword when HSpice is needed. The default simulator is NSpice.

Ace_Eldo <path/binary>

Specifies that ELDO is used as the characterization simulator. You only need to specify this keyword when ELDO is needed. The default simulator is NSpice.

Ace_Spectre <path/binary>

Specifies that Spectre is used as the characterization simulator. You only need to specify this keyword when Spectre is needed. The default simulator is NSpice.

Decap_Subckt

Defines sub-circuit-based intentional decap instances in the Spice netlist. The syntax and usage is described following.

```
DECAP_SUBCKT {
    <decap_subckt_name1> <port_type_list1> param <param_list1>
    <decap_subckt_name2> <port_type_list2> param <param_list2>
    ...
    <decap_subckt_nameN> <port_type_listN> param <param_list3>
}
```

where

<decap_subckt_nameN>: describes the subcircuit name of the intentional decap instance

<port_type_listN>: defines the type of subckt port connection, either power, ground, or signal net, designated as 'power', 'ground', or 'na' in the port type list ('na' designates a signal net connection)

Note that there must be one item in the <port_type_list> for each pin name.

param <param_list>: defines the parameters that determine the size of the intentional decap instance. For example, in a Spice netlist as follows:

```
XDECAP1   PLUS   MINUS   NMOSCAP   lr=6.3u   wr=3.92u   m=4 ...
XDECAP2   PLUS   MINUS   NMOSCAP   lr=3.3u   wr=1.92u   m=8 ...
...
```

Encrypted_Files <encrypted_model_file>

When specified, supports encrypted model files in flows that use the SPECTRE simulator. Must have the ACE ModelFile configuration file keyword defined also, with the same filename.

External_Power_Net <ext_power_pin> ...

Defines the external power pin(s) of the cell. Different power pins represent different power domains. The power pin names must be Spice pin names declared in the Spice subckt file. You must provide power pin names using this keyword for ACE to trace the power nets inside the Spice subckts. See also Internal_Power_Net and VP_Pairing keyword usage.

External_Ground_Net <ext_ground_pin> ...

Defines external ground pin(s) of a cell. Different ground pins represent different ground domains. The ground pin names must be Spice pin names declared in the Spice subckt file. You must provide ground pin names using this keyword for ACE to trace the ground nets inside the Spice subckts. See also Internal_Ground_Net and VP_Pairing keyword usage.

extract_diodecap

When set, ACE extracts capacitance of wrapped diodes.

Syntax:

```
extract_diodecap [1 | 0 ]
```

FINFET_UNIT_WIDTH <value_meters>

Identifies unique lengths for transistors for cdev characterization. For FinFETs, transistors are defined by the number of fins (NFIN) and length (L), instead of width and length. In 16nm technology ACE only traces the unique size transistors with different Ls and you must specify the fin width using this keyword.

FLIP_WELL

Using this keyword ACE can handle cells made from FDSOI technology, or the flip-well process. FLIP_WELL must be set to 1 to enable this feature.

Global <node_name> [<node_name> ...]

Defines global nodes (nodes that are connected in all hierarchical levels).

Ignore_Decap_Subckt_Checking [0 | 1]

Allows you to control of whether ACE errors out or just prints a warning message when a decap subckt is not instantiated. Set the keyword to 1 to prevent ACE from erroring out. Default is 0.

Ignore_Subckt

Specifies capacitance contributions to be excluded from the specified subckt(s) and their descendents. This is useful when you want to extract, for example, only the capacitance contributed by the top level elements, and ignore subckt contributions. The syntax is:

```
Ignore_subckt {
    <subckt_name>
    ...
    }
```

IGNORE_UNUSED_PG [0 | 1]

ACE in the APLMMX signal EM flow bypasses unused/undefined P/G nets. APLMMX automatically sets this keyword when running ACE for vectorless signal EM device characterization. By default - 0 (off).

Include <file>

Used to include needed Spice files, such as a subckt definition, or Spice model file.

Internal_Power_Net <internal_power_pin> ...
Internal_Ground_Net <internal_gnd_pin> ...

Specifies separate internal P/G nets for header or footer switches in the ACE configuration file. So for switch cases, internal nets have different entries in the *<>.mcap* file. ACE handles hierarchical internal nets by using hierarchical power/ground pin names, with "**.**" as a hierarchy divider. In this way you can include all RC elements related to this net in ACE characterization, but this can be used only with a hierarchical subcircuit netlist. Also, Cdev characterization for internal nets can be performed using these keywords. The switch cap between external and internal nets is ignored. The output Cdev contains entries for both external and internal nets. Use also the keyword VP_PAIRING, as follows.

```
EXTERNAL_POWER_NET <external_power_pin/net> ...
EXTERNAL_GROUND_NET <external_gnd_pin/net> ...
INTERNAL_POWER_NET <internal_power_pin/net> ...
INTERNAL_GROUND_NET <internal_gnd_pin/net> ...
VP_PAIRING   <internal_net> <external_net>
? SWITCH_SUBCKT [<switch_subckt_name> |
       <switch_filename>]?
```

An example subcircuit appears as follows:

```
.SUBCKT BLOCK in out vdd vss  nd5
 Xswitch ctrl vdd net_int SWITCH
```

```
 Xinv net_int vss in out INV
.ends

.SUBCKT testckt in1 in2 out1 out2 vdd vss
 Xblock1  in1 out1 vdd vss  nd5  BLOCK
 Xblock2  in2 out2 vdd vss  nd5  BLOCK
.ENDS
```

Sample ACE configuration file settings are shown following:

```
EXTERN_POWER_NETS vdd
INTERNAL_POWER_NET  Xblock1.net_int Xblock2.net_int
VP_PAIRING (Xblock1.net_int  vdd) (Xblock2.net_int  vdd)
EXTERN_GROUND_NETS  vss
VDDVALUE vdd 0.85
switch_sub SWITCH
```

The output *mcap* file then contains Cdev entries for both external and internal nets. An example aplreader output would be:

```
Info: Reading cdev file- 'testckt.mcap'
Info: cell= testckt
Info: pin= vdd cap= 0.389455 pf, res= 2.56772 ohm, leak= 20 uA
Info: pin=Xblock1.net_int cap= 13.866 pf, res= 0.072121 ohm, leak= 40 uA
Info: pin=Xblock2.net_int cap= 30.1777 pf, res= 0.03313 ohm, leak= 60 uA
Info: pin= vss cap= 16.2794 pf, res= 0.0615381 ohm, leak= 120 uA
```

K_GROUND_CAP/K_FLOAT_CAP

Using these keywords ACE can extract signal net capacitance from the DSPF netlist, and adds its value in the cdev file generated (ACE classifies parasitic capacitance into three types: P/G parasitic cap, grounded cap, and floating cap.)

K_GROUND_CAP defines the fractional contribution of grounded signal cap to P/G parasitic cap, and K_FLOAT_CAP defines the fractional contribution of floating signal cap to P/G parasitic cap. Their default value is 0.5, and their allowable range is 0 to 1.0.

```
K_GROUND_CAP <value>
K_FLOAT_CAP <value>
```

Leakage_i

Specifies per-arc leakage current in Amps. Required when the low power '-pwc' option is used in ACE, with the syntax:

```
Leakage_i {
    <vdd_pin1> <gnd_pin1> <leakage_A>
    ...
    }
```

Metal_Resistor/ Metal_Resistor_File

These keywords allow ACE to trace through resistors defined as subcircuits and include transistors that are connected beyond the subcircuit resistors, which then enables cap characterization of MOS transistors connected after the resistors. To perform resistor trace-through, the subcircuit names must be defined using the syntax:

```
METAL_RESISTOR <model_name1> <model_name2> ...
```

or

```
METAL_RESISTOR_FILE <path to metal resistor file>
```

The format of the contents of the specified METAL_RESISTOR_FILE is as follows:

```
<Resistor_model_name1>
<Resistor_model_name2>
<Resistor_model_name3>
...
```

ModelFile <file> <corner>

Specifies Spice device model library and process corner case.

MOS_Report [0 | 1]

When set to 1, produces a *<cellname>.mos.report* file, as shown in the following example of a MOS report for a memory:

```
Mos Name W L Count Domain
psvtlp 0.875 0.06 12 vdda
nsvtlp 0.46 0.06 14 vss
psvtlp 2.5 0.06 12 vdda
```

NOMINAL_VDD <vdd_domain1> <v1> ?<vdd_domain2> <v2>?

Specifies the nominal Vdd value for each Vdd domain when they are different from the value specified by the keyword VDD_VALUE, which is user-specified in the APLMMX configuration file, or in the ACE input file. If 'NOMINAL_VDD' is not specified when SIGNAL_PARASITIC_C is turned on, a warning is displayed and the nominal Vdd value is assumed to be the same as specified by VDD_VALUE.

NMOS_Model_Name <model_name>

Defines the NMOS model name if a device model file is not specified.

PMOS_Model_Name <model_name>

Defines the PMOS model name if a device model file is not specified.

Option <Spice_option>

Specifies any standard Spice option to be passed to the Spice deck for ACE decap characterization. Multiple OPTION definitions are allowed on separate lines. Example: OPTION scale=1e-6

Option parhier=[local | global]

Specifies the priority of hierarchical parameter. The default value is ‘global’.

PREPROCESS_SUBCKT_FILE [0| 1]

When turned on, handles Spectre netlists from Virtuoso, which contain the test bench setup, such that statements in the test bench and any other “include” files that are not relevant for capacitance characterization are automatically skipped, reducing ACE runtime.

Primary_Power_Net <pin_name>
Primary_Ground_Net <pin_name>

Supports PWL cap data for low power analysis using the ACE '-pwc' option. When using the '-pwc' option, you must specify leakage values in the ACE configuration file using keyword 'Leakage_i' to get an accurate value. If you specify the primary pin names, ACE uses the specified arc to perform post-processing and generates

pwcap data. If you do not specify the primary pin name, ACE by default uses the first P/G pins listed as the primary P/G pins.

R_TRACE_METHOD [0|1]

By default (0), automatically determines the correct DC connection between ports. .

SIGCAP_FACTOR <value>

Specifies the fraction of the signal net capacitance to be included in P/G pin cap (<value> between 0 and 1). The default value is 0.5.

SIGNAL_PARASITIC_C [0 | 1]

When SIGNAL_PARASITIC_C is turned On, signal net Cload is added to P/G net intrinsic capacitance. The default is 0, off.

Simulator_Command_Option <option>

Supports use of NSpice, HSpice, Eldo, and Spectre tools. Adding this parameter to the ACE configuration file appends specified user options to the Spice simulation command line.

Subckt <file>

Defines the Spice subckt netlist for the cell. The P/G pin names in the netlist must be consistent with the pin names specified in External_Power/Ground_Net.

SweepValue <v1> <v2> ...

Specifies voltage sweep values for PWCAP data generation when the ACE ‘-pwc’ option is specified. If you do not specify voltage sweep values, ACE by default uses 11 standard Vdd values to do scaling (10%, 20%, ..., 100%, and 110% of Vdd).

Switch_Subckt [<switch_subckt_name> | <switch_subckt_filename>]

Defines the name of a header/footer switch subckt or a Spice-format switch subcircuit filename. Note that the subckt(s) definition must be encapsulated in file(s) specified by SUBCKT or INCLUDE constructs. For low power cells that have the switch built-in, you must declare the switch subckt using this keyword. For cells that do not have header/footer switches, you do not need to specify this keyword.

Temperature <value>

Specifies the temperature value for MOSFET characterization.

Toggle_Rate_Assignment

Assigns different toggle rates to different sub-circuits inside a block or IP, in addition to the top level. To use this feature, add the following in the ACE configuration file:

```
TOGGLE_RATE_ASSIGNMENT {
<subcircuit_1> <toggle_rate1>
<subcircuit_2> <toggle_rate2>
...
}
```

VP_Mapping (<LEF_pin> <Spice_pin>) [...]

Defines the P/G pin mapping between LEF and Spice netlists. If the P/G pin names are different in LEF and Spice, you need to use this keyword to define the mapping

output result file. ACE uses Spice pins during characterization, and uses the LEF pin name in the output result file.

VddValue <vdd_domain> <voltage> [<vdd_domain> <voltage> ...]

Specifies Vdd domain name(s) and corresponding value(s). These values are used during MOSFET characterization. The Vdd domain names must be consistent with those specified in External_Power_Net.

ACE supports multiple nominal Vdd characterization using the following syntax:

```
VDDVALUE <vdd_pin1 value1a> [<vdd_pin2 value1b>…]
VDDVALUE <vdd_pin1 value2b> [<vdd_pin2 value2b>…]
```

In this syntax <*value1a> and <*value1b> represent different voltage corners of multiple nominal characterization.

VP_Pairing (<internal_net1> <external_net1>) (...)

Defines P/G pairs for internal and external nets. If there are any switches present in the cell, you must specify the P/G pairs using this keyword. Otherwise this keyword is not necessary. See also use of Internal/External_Power_Net and Internal/External_Ground_Net keywords.

VTH_FACTOR <value>

The scaling factor VTH_FACTOR specifies whether signal capacitance is to be added to Vdd pin intrinsic capacitance. When SIGNAL_PARASITIC_C is set to 1, for each Vdd domain, if VDD_VALUE > VTH_FACTOR*NOMINAL_VDD, the signal net Cload for that domain is added to P/G pin capacitance accordingly. The VTH_FACTOR <value> must be a floating point number between 0 and 1. Default is 0.5.

wrapsub <subckt_name> [...]

When a subckt name is specified using this keyword, ACE looks for 'm' instead of 'x' as the leading character in the element line, for instances in the HSIM-wrapper processed netlist. This rule only applies to specified subckts. Other subckts still follow the normal SPICE rule, which uses 'x' as the leading character for subckt instances.

XMOS_INST_PARAM [ON |OFF]

When set, parses all instance parameters of the first XMOS instance in a table of unique size listings. The default value is off.

Running ACE Characterization

The command for invoking ACE from a UNIX command line is:

```
ace [-d] [-o <outfile>] [-toggle_rate <On_fraction> ] [-pwc ]
    <cellname>.smin
```

where

-o <outfile>: specifies output filename

-toggle_rate <On_fraction>: controls the fraction of On time for intrinsic MOSFETs, and thus their effective capacitance (ESC) value. The <On_fraction> should be between 0 and 1. A larger toggle_rate results in a smaller intrinsic ESC value, since it means more transistors are switching and acting less like capacitance.

-pwc: specifies generation of PWL capacitance data for low power analysis.

Output Result Files

ACE generates three files in the APL result directory:

- *<cellname>.mcap* - APL's CDEV file format (pin-based capacitance) to be imported into Totem
- *<cellname>.ace.mmx* - more detailed pin capacitance data (intentional, intrinsic, parasitic) to be imported into MMX
- *<cellname>.ace.decap* - detailed intentional decap information, including a list of each unique sized MOS decap, its sizes (W, L), model name, decap value, and the P/G pins it connects to. Another list is the total decap contribution to each P/G arc, with each decap cell's instance count.

sim2iprof

The sim2iprof utility uses third-party simulation output files, such as *.fsdb, .ta0, .tr0-.trX, .out, .wdb* or *.pwl* format files as inputs to obtain Read/Write/Standby mode data for memories, and generates current profiles in *a <cell>.current* file for Totem power analysis. Hence the utility supports NSPICE, HSPICE, NanoSim, Eldo and HSIM outputs in one or more of the previously noted formats.

You can use one of three sim2iprof configuration file keywords to define memory timing:

- MCF_FILE - the file *.apache/apache.mcf,* which is supplied by user, defines the clock and data timing. When the MCF_FILE keyword and this file is specified, the tool automatically parses the *.lib* formula and generates the output.

- SIM_FUNC - use SIM_FUNC to describe the Boolean timing equation for READ, WRITE, and STANDBY functions explicitly, together with the clock name. The equations are available in the corresponding *.lib* file,
- SIM_TIME - you can specify directly capture times for READ, WRITE, STANDBY-H, and STANDBY-L if you have knowledge about the waveforms. You must specify the exact starting times for each Read/ Write/ Standby cycle.

The Vdd current waveform is extracted and converted to Totem *<cell>.current* format. To get decap information, you can specify the '-avm' option and an avm config file, which runs the AVM utility (see the *RedHawk User Manual)* to obtain decap values, equivalent power circuit resistance and capacitance, and leakage power data, and write the data to the *<cell>.cdev* file. Alternatively, if you have already obtained decap data by running other tools, such as ACE, you can directly enter the information in the config file, and 'sim2iprof 'then writes them into the *<cell>.cdev* file.

Running sim2iprof

To run the 'sim2iprof' utility, use the following invocation:

```
sim2iprof <config_file> ? -log <log_file> ?
    ?-o <current_output_file>? ?-avm <avm_config>?
```

where

<config_file> : specifies the name of the configuration file

-log <log_file> : allows specifying a log file for each SIM2IPROF run

-o <current_output_file> : specifies optional output filename. The default name is *<cell>.current* when the keyword SIM_FILE is used in the config file, and *<cellname>.spiprof* when the keyword CELL is used.

-avm <avm_config> : specifies optional AVM configuration filename to allow AVM to generate decap data. The default is the log file, *sim2iprof.log,* which contains Information, Warning and Error messages.

NOTE: The specified simulation result file used as a data source must contain both P/G pin current waveforms and related signal voltage waveform data.

sim2iprof Configuration File

Required keywords for `sim2iprof` are either SIM_FILE or CELL, as well as VDD_PIN and one of MCF_FILE, SIM_FUNC, or SIM_TIME keywords. Following is a summary of the required and optional configuration file keywords and options used to specify the inputs for current profile generation.

Note: If CUSTOM_STATE_FILE is specified in the config file, then the keywords CUSTOM_STATE_SIM_TIME, CUSTOM_STATE_SIM_FUNC, SIM_TIME, and SIM_FUNC are ignored.
Among keywords CUSTOM_STATE_SIM_TIME, CUSTOM_STATE_SIM_FUNC, SIM_TIME, and SIM_FUNC, if you specify more than one of them, the first one in the file is used, and the rest are ignored.

Note: SIM_FILE reads Vdd value from simulation result file (that is, the ".vdd line in the .hout file), which may not reflect the true Vdd value used in the simulation, and can cause one simulation result file to be dropped if the Vdd values are the same among two simulation result files. Therefore, for multiple simulation result files with different Vdd values, using the FILENAME option of the keyword CELL is recommended, since then you can directly set the Vdd values for each result file.

Following is a summary of SIM2IPOROF configuration file keywords and the options used to specify the inputs for current profile generation, in alphabetical order:

ACCURATE_MODE [0 | 1]

To achieve greater accuracy during characterization of PWL inputs, set this keyword to 1. The default value is 0. However, file size is increased in ACCURATE mode, so the size of the output file for big test cases can be reduced by using the default value.

ALIGN_VSS_CURRENT [0| 1]

Controls the window range for VSS_PIN. For the default of 1, VSS_PIN uses the same window as VDD_PIN. You should set this keyword to 0 when the window range for VSS_PIN is not the same as the VDD_PIN window range.

BOOLEAN_FLAG "<int_flag>"

To support multi-cycle memories.you must define internal flags within the configuration file with this keyword and also the "MODIFY_BOOLEAN_FLAG" keyword when characterizing APLs. See the example in the MODIFY_BOOLEAN_FLAG description.

CDEV_FILE <filename>

Specifies the device capacitance filename. SIM2IPROF reads this file and generates a new cdev file *<cellname>.cdev*. If the CDEV_FILE name specified is *<cellname>.cdev,* a random number is added to the filename, such as *<cellname>_XXXX.cdev.* If neither CDEV_FILE nor the CDEV section in CELL is specified in the config file, the

<cellname>.cdev file is not created, and the extracted leakage data is only printed in the log file. A warning is issued that the leakage data is lost.

CDEV

Sim2iprof supports multiple nominal voltage cdev characterization as specified in this keyword for the same multiple nominal voltage corner specified for sim2iprof input files.

Syntax:

```
CDEV <vdd_pin1>=<Volt_a1> <vdd_pin2>=<Volt_b1> {
   <vdd_gnd_arc1> {
     C0 = <value>
      C1 = <value>
      R0 = <value>
      R1 = <value>
      leak0 = <value>
      leak1 = <value>
    }
    <vdd_gnd_arc2> {
    ...
    }
}
CDEV <vdd_pin1>=<Volt_a1> <vdd_pin2>=<Volt_b1> {
...
...
}
```

CELL <cellname> { <cell_parameters> }

For each cell, specifies the result file, Vdd values, transition times, loads, and leakage values. ***Either*** SIM_FILE or CELL is required; both cannot be used in the same config file. However, using the CELL keyword is recommended, since it provides more data.

For the CELL keyword, two APL files are created in the current directory:

- *<cellname>.spiprof* - cell switching current file
- *<cellname>.cdev* - cell cdev file (if CDEV is specified)

If multiple Vdd values are specified (norminal value and degraded values), the switching waveforms are sorted in descending order of Vdd value.

```
CELL <cellname> {
    FILENAME {
    <sim_result_file_1> <vdd_pin1>=<Volt_a> <vdd_pin2>=<Volt_b > ...
    <sim_result_file_2> ...
    ...
    }
    SLEW {
    <trans_time_sec>
    }
    LOAD {
    <load_cap_F>
    }
    CDEV {
    <vdd_pin> <gnd_pin> {
    C0 = <cap_low_F>
    C1 = <cap_hi_F>
```

```
    R0 = <R_low_Ohms>
    R1 = <R_hi_Ohms>
    ? LEAK0 = <leak_low_A>
     LEAK1 = <leak_hi_A> ?
    }
    ...
    }
}
```

where

<sim_result_file_n>: specifies one simulation result file per line, which can be in **.fsdb, *.hout, *.tr0-*.trX*, **.ta0-*.taX, wdb*, or **.pwl* format. Each file contains simulation results for a different set of Vdd values. At least one <sim_result_file> must be specified. The Vdd pin order should be consistent with that specified using the keyword VDD_PIN. If no Vdd value can be read from simulation results, and no Vdd value is specified, it is assumed to be 1.0 V. Supports separate FSDB files for different states (Read, Write, Standby) for memories.

For a **.pwl* file, the file format for defining pin voltage/current is as follows:

```
<Pin_name> PWL (
+ <time_1> <value_1>
+ <time_2> <value_2>
+ ...
)
```

where <Pin_name> is the pin name for a signal, and <time_n> <value_n> are the associated time and voltage or current values, in seconds and Volts or Amps, respectively. The <Pin_name> needs to match the pin name specified in the configuration file.
Note also that if current is to be specified for a pin, the pin name **must have an initial "I" to be correctly identified** and have current units.

Example:

```
Ivdd PWL (
+ 0.0 0.0
+ 1.0e-9 1e-3
+ 5.0e-9 1e-3
+ 6.0e-9 1e-6
+ 1.0e-8 1e-6
)
```

Note the use of "+" as a continuation character.

<vdd_pin1>=<Volt_a> <vdd_pin2>=<Volt_b>: optionally, you can also specify each Vdd domain's nominal voltage value.

SLEW <trans_time> : input transition time (sec)

LOAD <load_cap_F> : output capacitance (Farad)

Note: normally for SIM2IPROF applications, which are capturing switching currents for large macros such as memory and I/Os, there is no need to specify SLEW and LOAD values, since they have little impact on the overall switching current. However, these two keywords can be used to specify your own sample values for a macro.

CDEV : specifies pwr/gnd arcs, device capacitance, "ESR" equivalent power resistance, and leakage current, which is subtracted from switching current. If CDEV_FILE, LEAKAGE_VALUE and the CDEV section in CELL are all specified, CDEV section has higher priority. If the CDEV section are

specified, but leakage in the CDEV section is not specified, SIM2IPROF performs extraction from the simulation results and puts that data into the *.cdev* file. If neither the CDEV section nor CDEV_FILE is specified in the config file, the *<cellname>.cdev* file is not created, and the extracted leakage data is only printed in the log file. A warning is issued that the leakage data is lost.

<vdd_pin> <gnd_pin> : specifies power/ground arcs, one per line for multiple Vdd/Vss pins. Note that if you specify Vdd pin values, you must do it for *ALL* Vdd domains/pins. A partial pin list generates an error. If no Vdd values are specified, all are taken from simulation. If there are multiple Vdd values (derated Vdd values), add them in new lines within the FILENAME section.

C0 = <cap_low_F> : equivalent power circuit capacitance (Farad), cell output low

C1 = <cap_hi_F> : equivalent power circuit capacitance (Farad), cell output high

R0 = <R_low_Ohms> : equivalent power circuit resistance, cell output low

R1 = <R_hi_Ohms> : equivalent power circuit resistance, cell output high

LEAK0 = <leak_low_A> : leakage current (Amps), cell output low (optional)

LEAK1 = <leak_hi_A> : leakage current (Amps), cell output high (optional)

COMPOSITE_STATE

Supports summation and scaling of individual current profiles, which can generate multi-state current profiles for user-defined input/output pin scenarios for clock toggle, data pin toggle, and clock and output toggle. Multi-bit sequential cell handling also can include individual edge-triggered events, where all input/output pins do not have to trigger off the same event. When sim2iprof detects a composite switching state name defined in the COMPOSITE_STATE keyword, it calculates the state algebraic expression. The Boolean expression and active input pin are optional. The syntax is:

```
COMPOSITE_STATE {
    <state_name> “<Boolean_eq>” “<active_input_pin>”
    “<state_algebraic_expression>”
...
}
```

where

<state_name>: specifies the user-defined state name for each composite state

<Boolean_eq>: specifies the Boolean expression to be used in VCD mode analysis to determine when the specific state is On (active).

<active_input_pin> : specifies the input pin that switches

<state_algebraic_expression>: defines a linear combination of multi-state currents using Boolean definitions. Note that a composite state cannot be used to define another composite state.

Example 1:

```
COMPOSITE_STATE {
  OUT1 "a&b" "clk" "10 * RISE_OUT"
  OUT2 "" "" "RISE_OUT * 3"
}
```

Example 2:

```
COMPOSITE_STATE {
   DATA_IN_TOGGLE_POS “data1” “data1” “DATA_IN_RISE”
   DATA_OUT_TOGGLE_POS “data_out_pin” “data_out_pin” “REF_WAVEFORM1 - CLOCK”
```

```
    CLOCK_TRIG_POS_VCD “(en1|en2)&clk” “clk” “CLOCL_RISE_TOGGLE”
    CLOCK_TRIG_NEG_VCD “(en1|en2)&!clk” “!clk” “CLOCK_FALL_TOGGLE”
}
```

Note that all base states specified in COMPOSITE_STATE must be defined in either CUSTOM_STATE_SIM_TIME or CUSTOM_STATE_SIM_FUNC, such as:

```
CUSTOM_STATE_SIM_TIME {
    # Base state definitions
    # <state_name> <Boolean_express> <active_input_pin> <start_time>
        <end_time>
CLOCK_TRIGGER  “” clk 5.0e-9 6.0e-9
INPUT_TOGGLE   “” “” 8.0e-9 9.0e-9
OUTPUT_TOGGLE  “” clk 5.0e-9 6.0e-9
}
COMPOSITE_STATE {
 # composite switching state definitions
 # <state_name> <Boolean_eq> <active_input_pin>
        <state_algebraic_expression>
 OUTPUT_STATES1 “”       “” “1*OUTPUT_TOGGLE”
 OUTPUT_STATES2 “a & b”    clk “2*OUTPUT_TOGGLE -
 1*CLOCK_TRIGGER”
 OUTPUT_STATES3 “”     “” “32*OUTPUT_TOGGLE -
 31*CLOCK_TRIGGER”
 STANDBY_STATE1 “c & d”      clk “1*INPUT_TOGGLE”
 }
```

Also, a composite state cannot be used by another composite state. For example, specifying the following is **NOT** allowed:

```
COMPOSITE_STATE {
    OUTPUT_STATES1 “ ” “ ” “1*OUTPUT_TOGGLE”
    OUTPUT_STATES2 “ ” “ ” “2*OUTPUT_TOGGLE”
}
```

CURRENT_INVERT_FLAG [0 | 1]

Controls inversion of the current waveform by SIM2IPROF, depending on the keyword setting and total charge and the peak of the waveform. The keyword usage is as follows:

- When the keyword is not specified, the default behavior is SIM2IPROF inverts the current if the total charge and the peak are both negative.
- When 'CURRENT_INVERT_FLAG 0' : no inverting occurs. The output waveform is consistent with the FSDB data.
- When 'CURRENT_INVERT_FLAG 1' : all current waveforms are inverted.

CURRENT_SCALING_FACTOR <I_factor>

Multiplies all calculated output current values in *<cell>.current* by the <I_factor>. Default: 1.0.

Where different current scaling is needed for each custom state, use a separate configuration file for each custom state and a different CURRENT_SCALING_FACTOR in each config file. For example, on the command line specify:

```
sim2iprof  state1.config state2.config
```

In this way the configuration files provide individual current scaling for each custom state.

CUSTOM_STATE_FILE { <filename> }

Specifies a file containing cell state and timing information, such as Boolean timing equation for each custom state. The format of the state file is as follows:

```
CELL <mem_cell> {
<state_name> "<Boolean_eq>" "<active_pin_names>" ?<output_pin>?
    ...
    }
```

where

<mem_cell>: specifies the name of memory cell

<state_name>: required user state name composed of a string of letters, numbers, and/or '_' used to define state of a multi-state instance in the GSC.

Note that there are two reserved state names, as follows:

STANDBY_TRIG: captures i(Vdd) and i(Vss) when only the clock pin is switching in standby mode, from low to high for positive-triggered cells and from high to low for negative-triggered cells.

STANDBY_NTRIG: captures i(Vdd) and i(Vss) when only the clock pin is switching in standby mode, from high to low for positive-triggered cells and from low to high for negative-triggered cells.

<Boolean_eq>: required Boolean expression defining the trigger condition for the cell. For example, "A&B&!C" refers to A=1, B=1, and C=0. A state definition is a string composed of pin names and Boolean operators: ! (negation), & (and), and space (optional). Each pin name should appear only once to avoid ambiguity. Also, if <Boolean_eq> contains spaces, such as "a & b & c", then the equation must be inside double quotes. "CLK" refers to positive edge trigger and "!CLK "refers to negative edge trigger.

Note: The Boolean equation must be conditioned by the trigger expression, with a character limit of 1024 characters (the trigger expression can have non-clock inputs).

<active_pin_names>: required list of pin(s) that are switching for APL characterization. For example, for a defined state of "A&B&!C" and active pin "B" means that A=1, B switches from 0 to 1, and C=0. Multiple active pins are allowed if the pins switch simultaneously, and each should be in double quotes. If there is only one active pin, double quotes are not required.

<output pin> : optional definition of pin used by APL to monitor output switching. For single output pin cells, there is no need to specify the output pin in CUSTOM_STATE_FILE. For multiple-output cells, if <output_pin> is not given, APL arbitrarily chooses an output pin, which may or may not switch with the active pin(s).

NOTE: When creating multiple-state models, you must capture current profiles for reserved states 'standby_trig' and 'standby_ntrig'. During dynamic analysis (vectorless/VCD), if a memory *does not switch* in one of the other custom states (read/write/compare), they are assumed to be in standby mode and a current profile corresponding to a standby state is used. If standby states are not captured, Totem treats the instances not switching in other custom states as decaps, and they do not appear in the output *adsRpt/*dvd** reports or in instance DvD colormaps.

CUSTOM_STATE_SIM_FUNC

```
CUSTOM_STATE_SIM_FUNC {
<state_name> <Boolean_eq> <clk_name> ?<alias_state_name>?
....
}
```

Specifies custom functional states for simulation, where

<state_name> : user name for custom state

<Boolean_eq> : Boolean equation defining switching

<clk_name>: specifies the clock name

<alias_state_name>: specifies an acceptable alternative Boolean equation and current profile if there are no current profiles in the simulation output data for the particular state.

CUSTOM_STATE_SIM_TIME

```
CUSTOM_STATE_ SIM_TIME {
<state_name> <Boolean_eq> <active_input_pin> <start_time>
    <end_time>
VDD_PIN <pin_name> <start_time> <end_time> <iscale=<value>
[VDD_PIN <pin_name> <start_time> <end_time>
<iscale=<value>...]
VSS_PIN <pin_name> <start_time> <end_time> <iscale=<value>
[VSS_PIN <pin_name> <start_time> <end_time>
<iscale=<value>...]
        ...
 }
```

Supports custom multiple-state Boolean expressions in LIB files. Each line is a state definition, where

<state_name>: user name for custom state

<Boolean_eq> and <active_input_pin> : used to report in cell current file only.

<start_time>: start of capture times for the state.

<end_time> : optional end time for state.

If specified, the duration is computed as (<end_time> - <start_time>), or else the value specified in the DURATION keyword is used.

For example:

```
CUSTOM_STATE_SIM_TIME {
   M0 cena&cenb clka 22.4e-9 25.2e-9
   M1 cena      clka 2.4e-9
   M2 !cena     clka 7.4e-9
   M3 cena&cenc clka 12.4e-9
   M4 cenb      clka 17.4e-9
  }
  DURATION 5e-9
```

VDD_PIN / VSS_PIN <pin_name> <start_time> <end_time> <iscale>: defines the capture window per power/ground pin, for multiple power/ground designs. If specified, aplmmx will combine the peak current of each PG pin captured within the respective timing windows into a single merged state. Option <iscale > scales the pin current to specified value.

Example:

```
CUSTOM_STATE_SIM_TIME {
    TOOGLE_100 "E0" "C0" 250.1ns
```

```
    TOOGLE_101 "E1" "C1" 251.1ns 270.1ns
    TOOGLE_102 "E2" "C2" VDD_PIN vVDDL 252.2ns 271.2ns
iscale=1.0 VDD_PIN vVDDP 262.2ns 281.2ns VDD_PIN vVDDA
272.2ns VSS_PIN vVSSA 292.2ns 311.2ns iscale=1.0
    }
```

CYCLE_SELECTION [peak | avg]

Specifies cycle selection criteria when there are multiple cycles that meet SIM_FUNC formula definition. ‘peak’ selects the cycle with the highest peak current value and ‘avg’ selects cycle that is the summation of current values within the cycle. Optional - default =peak.

CYCLE_SKIP <cycle0> < cycle1> ...

Specifies skipping extraction of undesired cycles, such as an initial spike cycle. For example, the first clock cycle may include a large amount of noise that is not to be considered. Cycle count starts from 0. Optional.

DATAVERSION [7v1 | pre7v1]

Because of a different format prior to Totem 7.1, specifies the data version of the switching current file.

DELAY_TIME <delay_time_in_sec>

Specifies delay after the triggering edge of clock to start capture period. Optional - default =0.

DURATION <time_sec>

Specifies all transactions (READ/WRITE/STANDBY) that are normally defined at the point at which the current drops below 10% of maximum value. If specified, *sim2iprof* uses DURATION value rather than 10% drop point to determine the end time and the actual duration.

ERROR_MISSING_STANDBY_STATE [0|1]

When set, causes sim2iprof to error out if standby states are missed in the configuration file. The default is 0, in which case a warning is issued and processing continues.

IGNORE_INPUT_CDEV_ERROR

Overrides different cell names in the *mcap* and *cdev* files. If set to 1 and cell names are different in the *mcap* and *cdev* files, cell names in *mcap* are replaced by *cdev* names in the configuration file.

IPROF_SAMPLING_MODE

```
IPROF_SAMPLING_MODE
[ <number_samples> | Default (50) |
    Extended (100) |
    Accurate (400) ]
```

Specifies the number of sampling points for generating APL current profiles. More sampling points provide finer waveform resolution in the final *<cell>.current* file.

IGNORE_UNRECOGNIZED_KEYWORD [0 | 1]

Controls the response to errors found in checking syntax and spelling of configuration file keywords. By default (1) unrecognized keywords are identified in a Warning message, but processing continues. If IGNORE_UNRECOGNIZED_KEYWORD is set to 0, SIM2IPROF errors out for any unrecognized keyword. Be sure to spell the keyword correctly when using it to insure that it is in effect.

LEAKAGE [1 | 0]

Specifies whether leakage currents are subtracted from switching current values or all leakage currents are set to zero. If 'LEAKAGE 0' is specified, sim2iprof turns off leakage subtraction and sets all pin leakage currents to 0. Default value is '1': leakage currents are considered.

LEAKAGE_VALUE

```
LEAKAGE_VALUE {
    <vdd_pin> <gnd_pin> {
      LEAKAGE0 = <val>
      LEAKAGE1 = <val>
    }
  }
```

Specifies pin names <vdd_pin> and <gnd_pin>, and leakage values in Amps for output low and output high conditions LEAKAGE0 and LEAKAGE1, respectively. If both CDEV_FILE and LEAKAGE_VALUE are given, SIM2IPROF uses values from LEAKAGE_VALUE when creating *<cellname>.cdev*. If only LEAKAGE_VALUE is specified, this leakage is subtracted from switching current. A warning is issued that the leakage data is lost. Supports multiple nominal voltage cdev characterization.

MCF_FILE <mcf_file>

Specifies the READ/WRITE/STANDBY Boolean timing formula. The MCF file *.apache/apache.mcf* is generated *w*hen performing power calculation in Totem. If MCF_FILE is used, SIM_FUNC and SIM_TIME cannot be used.

MINTIMESTEP

SIM2IPLOT can handle corner cases with large signal waveforms and simulation times greater than 1 us, and also small time steps on the order of 0.1 ps, while providing accurate results. Use this keyword to configure the desired time step or resolution. Data points for which the time step is less than the specified value are skipped.

Syntax:

```
MINTIMESTEP <time_in_seconds>
```

MODIFY_BOOLEAN_FLAG <state> “<int_flag_name>”

To support multi-cycle memories.you must define internal flags within the configuration file with this keyword and also the BOOLEAN_FLAG keyword when characterizing APLs. For example:

```
BOOLEAN_FLAG "!int_flag"
MODIFY_BOOLEAN_FLAG S1 "int_flag"
MODIFY_BOOLEAN_FLAG S3 "!int_flag"
CUSTOM_STATE_SIM_TIME {
  S1 "CLK & IN1 & !int_flag" CLK 0 1e-9
```

```
  S2 "CLK & IN2 & int_flag" CLK 1e-9 2e-9
  S3 "CLK & IN3 & int_flag" CLK 1e-9 2e-9
}
```

In above example the initial status of the internal flag int_flag is false "!". As soon as the Boolean condition for state S1 is met and the CLK triggering this event is accomplished, the internal flag status changes to true. Now states S2 and S3 can become true, depending on the additional inputs CLK, IN2 and IN3. If the condition for state S3 is met and the CLK triggering this event accomplished, the internal flag status changes back to false.

NODE_PLOT <node_name1> <node_name2> ...

Specifies signal names to extract X,Y coordinates, which can be viewed by the waveform visualization tool (i.e., xgraph). Each specified node generates a *node_name.xy* file. Note: Vdd current value is stored as a negative value. Scaling factors must be properly specified when viewing voltage and current waveforms together. Optional.

PIN_MAP

```
PIN_MAP {<pin_name_.lib file> <pin_name_result file>}
```

Maps the pin names if those used in SIM_FUNC (from *.lib*) are not the same as those used in the simulation result file. Optional.

PROCESS [SS | TT | FF]

Specifies a process name (optional). Default : XX (interpreted as no process selection).

RATIO <percentage>

Specifies a ratio of the peak value for determining the end of the capture window. The last occurrence of the ratio value in the event is considered the end of the capture. Can be used to filter out noise. Optional - default =0.1 (10%).

SCALE_LEAKAGE [0 |1]

If 'CURRENT_SCALING_FACTOR' is specified and SCALE_LEAKAGE is set to 1 (default), both switching current and leakage current are scaled. If 'SCALE_LEAKAGE 0' is set, then leakage current is not scaled.

SETUP_TIME <setup_time_in_sec>

Specifies the setup time before the triggering edge of the clock to start capture period. Optional - default =0.

SIM_FILE

```
SIM_FILE { <cellname> <filename> ? <trans_time_ps>?
?<load_fF>?
... }
```

Specifies the list of cell and file names, optional rise and fall transition times in ps, and unit load capacitance in fF units.

Either SIM_FILE or CELL is required; both cannot be used in the same config file.

SIM_FUNC

```
SIM_FUNC {
```

```
READ <Boolean_eq> <clk_Name>
WRITE <Boolean_eq> <clk_Name>
STANDBY <Boolean_eq> <clk_Name>
}
```

Specifies the Boolean timing equation for start of READ/ WRITE/ STANDBY signals from *.lib* file, and the associated clock name. The time at which signals fall to 10% of peak value is “end” time. If SIM_FUNC is used, then MCF_FILE and SIM_TIME keywords cannot be used. Note “!” identifies a negative value.

SIM_TIME

```
SIM_TIME {
READ <Start_time>
WRITE <Start_time>
STANDBY-H <Start_time>
STANDBY-L <Start_time>
}
```

Specifies the starting times for READ/ WRITE/ STANDBY-H / STANDBY-L signals, in seconds. Can be used instead of SIM_FUNC or MCF_FILE.

Note: For I/O blocks, because of the difficulty in specifying STANDBY-H and STANDBY-L values from SPICE simulation, if the Standby values are not specified, sim2iprof assigns zero current by default for those states.

SLEW_THRESHOLD <low_V> <hi_V>

Specifies low and high fraction of Vdd for determining slew value.

SPICE2LEF_PIN_MAPPING

```
SPICE2LEF_PIN_MAPPING {
    <subckt_pin_name>  <LEF_pin_name>
    ...
}
```

Maps pin names between simulation result files and LEF data. When writing pin names to *<cell>.current* and *<cell>.cdev* files, LEF pin names are used instead of SPICE pin names.

TEMPERATURE <degree_C>

Specifies operating temp for current profile generation.

VDD_PIN <vdd_pin_name> ...

Specifies Vdd pin names that have a current waveform, including multiple power domain pins. **Required**.

VSS_PIN <vss_pin_name> ...

Specifies Vss pin names that have a current waveform, including multiple power domain pins. **Required.**

WORST_IPEAK

By default *sim2iprof* picks the time window with the worst peak current value if multiple time windows satisfy the Boolean expression. When you specify 'WORST_IPEAK 0' *sim2iprof* picks the *first* time window that satisfies the Boolean expression.

NOTE: APL's switching current data (the <*cell*>*.current* file) should not contain any leakage data, to avoid double reporting of leakage in both <*cell*>*.current* and <*cell*>*.cdev.* If you specify leakage in the config file in the CDEV section, that leakage data is written into the <*cell*>*.cdev* file. If you do not specify leakage in the CDEV section in the config file, *sim2iprof* performs extraction from the simulation results and puts that data into the <*cell*>*.cdev* file. If the CDEV section is not specified in the config file, the <*cell*>*.cdev* file is not created, and the extracted leakage data is only printed in the log file. In all cases, leakage data is removed from the <*cell*>*.current* file.

Configuration File Example

The following is an example of a sim2iprof configuration file using the ‘CELL’ keyword. The file includes keywords typically used to automatically generate optimal results. Other optional keywords may be used to tune the results to meet special needs.

```
#-- sim2iprof configuration file generated by
sim2iprof_setup.pl on Thu Feb 1 15:23:41 PST 2007

CELL (sram128x32) {
FILENAME {
# <filename> [<vdd1>=<vdd1_v2> <vdd2>=<vdd2_v2>...]
dram.s0.fsdb vdd=1.15 vddo=1.05
dram.s1.fsdb vdd=1.1 vddo=1.0
}
SLEW {
    11ps
}
LOAD {
    15fF
}
CDEV {
    vpwr vgnd {
    C0 = 1.0p
    C1 = 2.0p
    R0 = 400.21
    R1 = 395
    LEAK0 = 1.0e-6
    LEAK1 = 4.2uA
}
ivdd vgnd {
    C0 = 3.0p
    C1 = 5.0p
    R0 = 621
    R1 = 385
    LEAK0 = 2.0e-6
```

```
    LEAK1 = 4.2uA
}
} // end of CDEV
} // end of cell

SIM_TIME {
READ 1e-12
WRITE 1e-12
STANDBY-H 1e-12
STANDBY-L 2e-12
}
VDD_PIN vpwr ivdd
VSS_PIN vgnd
DURATION 500e-12
```

Some of the key *sim2iprof* functions and terminology are illustrated in Figure E-10.

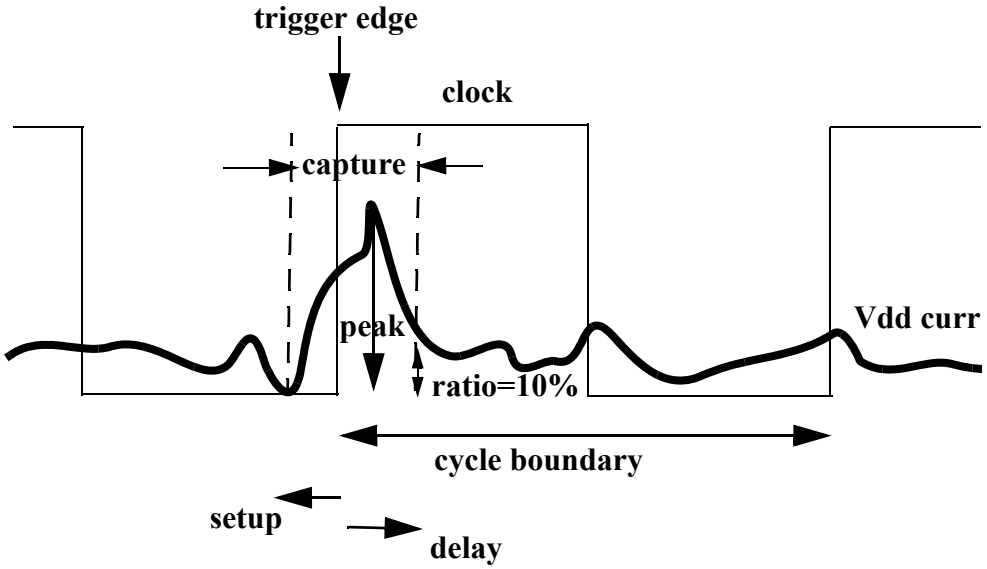


Figure E-10 Sample sim2iprof waveform and terminology

Output

By default, *sim2iprof* generates a *<cell>.current* file and a *sim2iprof.log* file. The *sim2iprof.log* reports the time window, peak current, area, and leakage for pins, and also the voltage with which the extracted values are associated. A sample output report is given below.

```
Info: Printing 'nb_write' of pin 'vvdd_wb' summary at vvdd_wb=1.21 ...
Info:   window range = (196.12, 200.12) ns
Info:   peak value   = 55769.6 uA
Info:   area value   = 25.7529 pc
Info:   leak value   = 0 uA
Info: Printing 'nb_write' of pin 'vvss_wb' summary at vvdd_wb=1.21 ...
Info:   window range = (196.12, 200.12) ns
Info:   peak value   = 91668.8 uA
Info:   area value   = 38.6493 pc
```

```
Info:    leak value    = 0 uA
```

Check the log file for Error or Warning messages if it does not produce the expected results. Totem uses the *<cell>.current* file for further analysis.

aplreader

The *aplreader* utility extracts data from the APL waveform files such as *<cell>.current, vmemory.current* or **.cdev,* and displays the information in the terminal window where it is invoked.

Running aplreader

To run the `aplreader` utility, use the following Linux/Solaris invocation:

```
aplreader <input_dir/file> [-l <cell_list_file>]
```

where

<input_dir/file> : specifies the input file directory and filename

-l <cell_list_file> : specifies an optional cell list file for which only cells in the file are processed

aplreader Output

Current Outputs

The `aplreader` utility writes to the display the following current-related parameters:

- “cell”: name of cell
- “C1”, ”C2” - load capacitances in Farads
- domain name and nominal voltage in Volts
- “R” - load resistance in Ohms
- “Slew1”, “Slew2” - input transition times in seconds
- “state” - state name
- “vector” - Boolean definition of vector for the state
- “active input”, “active output” - names of active input and active output pins
- “pin” - power/ground pin name
- “peak” - peak cell current in Amperes
- “area” - charge under the waveform in Coulombs
- “width” - duration of waveform in seconds

The format of the *aplreader* output for current waveform files, **.current*, is as follows:

```
-----------------------------
cell : <cellname>
vdd1 = <vdd value 1> ; vdd2 = <vdd value 2>; ... vddN = <vdd value
N) ; C1 = <Cload1 value> ; R = <load resistance> ; C2 = <Cload2
value>; Slew1 = <rising transition time>; Slew2 = <falling
transition time> ;
state = <state name> ; vector = <Boolean_eq> ; active_input =
<active_input name> ; active_output = <active_output name> ;
pin            peak               area          width
<pin name> <peak I - A> <Total charge - C> <Wave duration - S>
```

```
(... repeat previous syntax for all Vdd and Vss pins)

(... repeat previous syntax for all toggles)

(... repeat previous syntax for all Vdd values)

(... repeat previous syntax for all cells)
---------------------------------------
```

aplreader Current Output Examples

The following is a sample *aplreader* output for a standard cell APL characterization:

```
----------------------------------------
cell : NAND2
vdd1 = 1.3915 V ; C1 = 0 F ; R = 0 Ohm ; C2 = 2.80552e-15 F ; Slew1 = 1.7046e-
11 S ; Slew2 = 1.0228e-11 S ;

state = output_rise ; vector = a&b ; active_input = a ; active_output = z ;
     pin              peak            area           width
     vddfx            9.00e-05  A     5.58e-15 C     1.44e-10 S
     vssfx            9.40e-05  A     5.72e-15 C     1.44e-10 S
state = output_fall ; vector = a&b ; active_input = a ; active_output = z ;
     pin              peak            area           width
     vddfx            8.10e-05  A     1.94e-15 C     9.60e-11 S
     vssfx            3.80e-05  A     1.31e-15 C     9.50e-11 S
----------------------------------------
```

The following is a sample *aplreader* output for a memory cell characterized by *sim2iprof* and *avm:*

```
----------------------------------------
cell : memory_a
vdd1 = 1.188 ; vdd2 = 1.08 ; C1 = 3.9e-14 ; R = 180.9 ;
    C2 = 1.8e-13 ; Slew1 = 1.2e-10 ; Slew2 = 9.1e-11 ;

state = READ ; vector = ram_en*re ; active_input = clk ;
active_output = N/A ;
pin     peak      area        width
vdd1    4.6e-2 A  3.7e-12 C  1.7e-9 S
vdd2    3.6e-3 A  5.3e-12 C  1.7e-9 S
vss     6.8e-2 A  6.3e-12 C  5.7e-10 S

state = WRITE ; vector = ram_en*we ; active_input = clk ;
active_output = N/A ;
pin     peak      area       width
vdd1    2.3e-2 A  2.3e-12 C 1.7e-9 S
vdd2    2.6e-4 A  2.2e-12 C 1.7e-9 S
vss     4.9e-3 A  9.2e-13 C 6.3e-10 S

state = STANDBY ; vector = ram_en*!re*!we ; active_input = clk ;
active_output = N/A ;
pin     peak      area       width
```

```
vdd1    3.5e-5 A  1.7e-13 C  1.7e-9 S
vdd2    2.6e-5 A  4.7e-13 C  1.7e-9 S
vss     4.9e-5 A  9.9e-13 C  6.3e-10 S
##--NOTE: use “output_rise” for C01, “output_fall” for C10,
“output_nochange_input_trigger” for C00,
“output_nochange_input_nontrigger” for C11.
----------------------------------------
```

Equivalent Device Capacitance and Resistance Outputs

APL utilities (aplchk, aplreader, aplmerge) support both 10.1 version format and 9.2 and earlier version formats for cdev/pwcdev data. The 10.1 cdev/pwcdev models are transparent to Totem, and can be imported into 10.1 in the same way as in 9.2. However, cdev/pwcdev data generated in 10.1 cannot be used by 9.2 or earlier versions. If 'DECAP_MODEL 1' is specified in the APL configuration file, cdev/pwcdev characterization uses the 9.2 flow and generates 9.2 format cdev/pwcdev data.

aplreader Device C and R Output Examples - version 10.1 and later

Sample *aplreader* output for equivalent device capacitance and resistance characterization:

```
Temperature = 125.00000 C; State = ADS_DEFAULT_STATE_LOW; vector = i;
    active_input = i; active_output = o;
    VDD = 1.200000 V; VSS = 0.000000 V;
            pin = VDD, esc = 1.67321e-15 F, esr = 630.892 ohm
            pin = VSS, esc = 1.67321e-15 F, esr = 630.892 ohm
            pin = VDD, leak = 5.16966e-10 A
            pin = VSS, leak = 5.16966e-10 A
Temperature = 125.00000 C; State = ADS_DEFAULT_STATE_HIGH; vector = i;
    active_input = i; active_output = o;
   VDD = 1.200000 V; VSS = 0.000000 V;
            pin = VDD, esc = 1.80674e-15 F, esr = 797.314 ohm
            pin = VSS, esc = 1.80674e-15 F, esr = 797.314 ohm
            pin = VDD, leak = 4.01521e-10 A
            pin = VSS, leak = 4.00058e-10 A
```

aplreader Device C and R Output Examples - versions prior to 10.1

The format of *aplreader* output for equivalent device capacitance and resistance characterization files, **.cdev*, as follows for Totem versions prior to 10.1:

```
Info: cell=<cellname>
Info: pin= <pin name> cap = <capacitance>, res= <resistance>,
leak= <leakage_current>
(... repeat previous syntax for all cells)
```

Sample *aplreader* output for equivalent device capacitance and resistance characterization:

```
----------------- single-rail cell --------------------
Info: cell=BUF
Info:   pin= VDD1 cap= 0.0107297 pf, res= 773.13 ohm, leak= 0.0792457 uA -
    average R/C/Leakage for output high and output low for vdd pin


Info:   pin= VSS cap= 0.0197033 pf, res= 795.543 ohm, leak= 0.206591 uA -
    average R/C/Leakage for output high and output low for vss pin
```

```
---------------- multi-rail cell --------------------
Info: cell=DFF
Info:   pin= VDD1 cap= 0.0982371 pf, res= 3649.82 ohm, leak= 0.584086 uA -
    average R/C/Leakage for output high and output low for vdd1 pin
Info:   pin= VDD2 cap= 0.0090145 pf, res= 3136.66 ohm, leak= 0.127448 uA -
    average R/C/Leakage for output high and output low for vdd2 pin
Info:   pin= VSS cap= 0.0995312 pf, res= 2998.8 ohm, leak= 0.711534 uA -
    average R/C/Leakage for output high and output low for vss pin
-----------------------------------
```

Piecewise Linear Capacitance and Resistance Outputs

The format of aplreader output for power-up equivalent capacitance and resistance characterization files, **.pwcdev*, is as follows:

```
Info: cell=<cellname>
Temperature =  ; State =  ; vector =  ; active_input =  ;
active_output =  ;
vdd = value;  vss = value;
arc=<P/G domain>,   esc = <capacitance>, esr= <resistance>,
pin= p/g pin; leak = <leakage_current>
(... repeat for all Vdd values up to three. If there are more than
three Vdd values, aplreader takes the smallest, middle, and
largest Vdd values)
(... repeat for all states)
(... repeat previous syntax for all cells)
```

aplreader PWL Device C and R Output Example

The following is a sample aplreader output for a power-up equivalent capacitance and resistance characterization:

```
-----------------------------------
Info: cell=test_cell
 Temperature = 125.000000 C; State = ADS_DEFAULT_STATE_LOW; vector = A;
    active_input = A; active_output = X;
        VDD = 0.090000 V;       VSS = 0.000000 V;
                arc = (VDD VSS), esc = 4.04313e-15 F, esr = 761.765 ohm
                pin = VDD, leak = 1.33983e-08 A
        VDD = 0.540000 V;       VSS = 0.000000 V;
                arc = (VDD VSS), esc = 8.11297e-15 F, esr = 1138.18 ohm
                pin = VDD, leak = 5.60882e-08 A
        VDD = 0.990000 V;       VSS = 0.000000 V;
                arc = (VDD VSS), esc = 1.01403e-14 F, esr = 161.96 ohm
                pin = VDD, leak = 2.91051e-07 A
 Temperature = 125.000000 C; State = ADS_DEFAULT_STATE_HIGH; vector = A;
    active_input = A; active_output = X;
        VDD = 0.090000 V;       VSS = 0.000000 V;
                arc = (VDD VSS), esc = 4.04353e-15 F, esr = 1370.52 ohm
                pin = VDD, leak = 1.33983e-08 A
        VDD = 0.540000 V;       VSS = 0.000000 V;
                arc = (VDD VSS), esc = 1.03437e-14 F, esr = 459.626 ohm
                pin = VDD, leak = 6.42139e-08 A
        VDD = 0.990000 V;       VSS = 0.000000 V;
```

```
                arc = (VDD VSS), esc = 1.33618e-14 F, esr = 124.924 ohm
                pin = VDD, leak = 1.85495e-07 A
---------------------------------
```

aplcdev2pwc

The aplcdev2pwc utility is invoked from a Linux/Solaris command line to check the validity of cdev data, and it then converts the cdev data to pwcdev data. The input cdev file can be any version, but the output pwcdev file can only be a 5v3 version or older.

When converting cdev data to pwcap data, the Cap/Res/Leakage values for nominal VDD value cdev data are scaled with following formulas:

C_pwcap(VDD) = C_cdev * (VDD / nominal_VDD) ^ cf;

R_pwcap(VDD) = R_cdev * (VDD / nominal_VDD) ^ rf;

Leak_pwcap(VDD) = Leak_cdev * (VDD / nominal_VDD) ^ lf;

The syntax of the command is:

```
aplcdev2pwc [-cf <cap_factor>] [-rf <res_factor>] [-lf
<leakage_factor>] [-o <output_pwcdev_file>]
<input_APL_config_file> <input_cdev_file>
```

where

-cf: specifies the capacitance factor (default 0.5)

-rf: specifies the resistance factor (default -0.5)

-lf: specifies the leakage current factor (default 1.5)

-o: specifies the output pwcdev filename

Example invocation:

```
aplcdev2pwc apl.ABCconfig cellMNO.cdev -o cell.pwcdev
```

aplcdev2pwc Configuration File

The input configuration file is has many of the same keywords as that for APL cdev/pwcap characterization. Only the following keywords are needed in this conversion:

PROCESS <process name> - specifies a process type, such as TT, SS, or FF

VDD <value> - specifies the voltage

VDD_PIN_NAME <name> - specifies the VDD pin name

GND_PIN_NAME <name> - specifies the ground pin name

PRIMARY_VDD_PIN <name> - in low power designs, specifies the associated power pin name

PRIMARY_GND_PIN <name> - in low power designs, specifies the associated ground pin name

SWEEPVALUE <value1> [<value2> ...] - specifies a list of voltage samples in pwcdev

LEAKAGE_SCALING <leakage_scaling_factor_for_vdd1> [<leakage_scaling_factor_for_vdd2> ...] - specifies custom scaling factors for leakage for each voltage in SWEEPVALUE. For example:

```
SWEEPVALUE 1.0 0.9 0.8 0.7
LEAKAGE_SCALING 2 1.9 1.8 1.7
```

In above example, leakage is scaled by a factor 2 for 1.0V, 1.9 for 0.9V and so on. If LEAKAGE_SCALING is specified, SWEEPVALUE should also be there, otherwise the process errors out. There also should be an equal number of values.

APLMMX Template Utility

This utility creates APLMMX configuration file templates for different modes automatically. The syntax is:

```
aplmmx_template -sig ?[-vbased|-vless]? ?–lvs? ?-o <filename>?
aplmmx_template -pwr ?[-vbased|-static]
[idsat|dev_cur|rollup|gds_only]? ?–lvs?
    ? -o <filename> ?
```

where

-sig : creates a signalEM analysis template

-pwr : creates a power analysis template

-vbased : creates a template for vector-based mode

-vless : creates a template for signalEM vectorless mode

-static : creates a template for power vectorless mode. Must specify an argument, idsat, dev_cur, rollup, or gds_only

-o <filename> : specifies template file name

-lvs : creates a template for LVS flow

Arguments within “? ?” characters are optional.

clampviewer

Linux executable utility `'$APACHEROOT/bin/clampviewer'` previews ESD clamp I-V data. The 'clampviewer' command takes the PF-S clamp file as input, and if there is any clamp I-V data defined in the file, it xgraph plots the I-Vs, similar to the tcl command `'pfs show clamp_iv <ivName>'`. The syntax is:

```
clampviewer <IV_FileName> [<IV_Name>] [<options>]
```

The available options are :

IV_Name : name(s) of I-V plots specified in the file to plot (for example, 'iv1, iv2'), By default all I-Vs in the file are plotted.

-h : displays specified help message

-i <min,max> : plot range for current (default: [-10,10])

-v <min,max> : plot range for voltage (default: [-10,10])

-marker : draw marker

-oneview : put all I-Vs into one plot

-tlp : also plot TLP data (when I-V is derived from TLP_FILE)

-fit <value> : TLP fitting factor (default: 1.0)

An example I-V output is shown in Figure E-11.

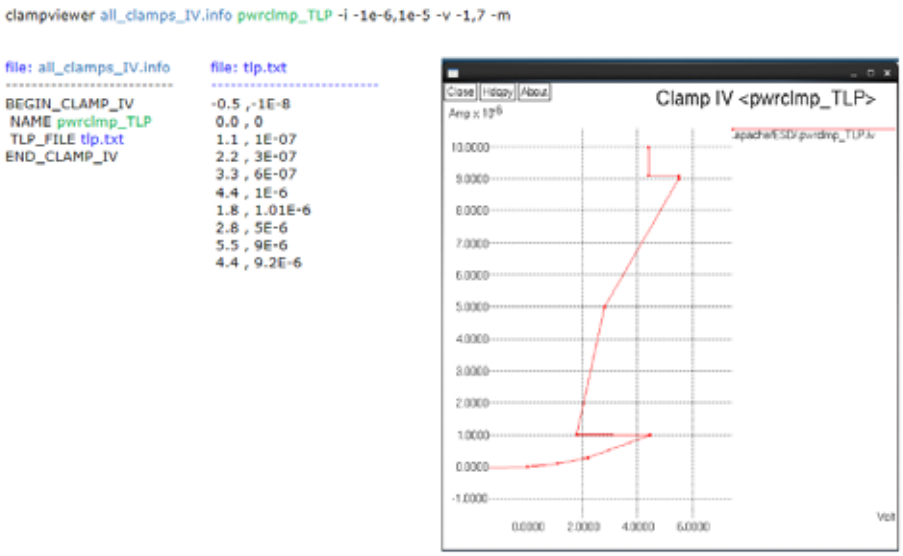


Figure E-11 clampviewer IV plot

Appendix F
Third-Party Software Licenses

This product contains licensed software which requires reproduction of the following notices:

THE ACCOMPANYING PROGRAM IS PROVIDED UNDER THE TERMS OF THIS COMMON PUBLIC LICENSE ("AGREEMENT"). ANY USE, REPRODUCTION OR DISTRIBUTION OF THE PROGRAM CONSTITUTES RECIPIENT'S ACCEPTANCE OF THIS AGREEMENT.

1. DEFINITIONS

"Contribution" means:

a) in the case of the initial Contributor, the initial code and documentation distributed under this Agreement, and

b) in the case of each subsequent Contributor:

i) changes to the Program, and

ii) additions to the Program;

where such changes and/or additions to the Program originate from and are distributed by that particular Contributor. A Contribution 'originates' from a Contributor if it was added to the Program by such Contributor itself or anyone acting on such Contributor's behalf. Contributions do not include additions to the Program which: (i) are separate modules of software distributed in conjunction with the Program under their own license agreement, and (ii) are not derivative works of the Program.

"Contributor" means any person or entity that distributes the Program.

"Licensed Patents " mean patent claims licensable by a Contributor which are necessarily infringed by the use or sale of its Contribution alone or when combined with the Program.

"Program" means the Contributions distributed in accordance with this Agreement.

"Recipient" means anyone who receives the Program under this Agreement, including all Contributors.

2. GRANT OF RIGHTS

a) Subject to the terms of this Agreement, each Contributor hereby grants Recipient a non-exclusive, worldwide, royalty-free copyright license to reproduce, prepare derivative works of, publicly display, publicly perform, distribute and sublicense the Contribution of such Contributor, if any, and such derivative works, in source code and object code form.

b) Subject to the terms of this Agreement, each Contributor hereby grants Recipient a non-exclusive, worldwide, royalty-free patent license under Licensed Patents to make, use, sell, offer to sell, import and otherwise transfer the Contribution of such Contributor, if any, in source code and object code form. This patent license shall apply to the combination of the Contribution and the Program if, at the time the Contribution is added by the Contributor, such addition of the Contribution causes such combination to be covered by the Licensed Patents. The patent license shall not apply to any other combinations which include the Contribution. No hardware per se is licensed hereunder.

c) Recipient understands that although each Contributor grants the licenses to its Contributions set forth herein, no assurances are provided by any Contributor that the Program does not infringe the patent or other intellectual property rights of any other

entity. Each Contributor disclaims any liability to Recipient for claims brought by any other entity based on infringement of intellectual property rights or otherwise. As a condition to exercising the rights and licenses granted hereunder, each Recipient hereby assumes sole responsibility to secure any other intellectual property rights needed, if any. For example, if a third party patent license is required to allow Recipient to distribute the Program, it is Recipient's responsibility to acquire that license before distributing the Program.

d) Each Contributor represents that to its knowledge it has sufficient copyright rights in its Contribution, if any, to grant the copyright license set forth in this Agreement.

3. REQUIREMENTS

A Contributor may choose to distribute the Program in object code form under its own license agreement, provided that:

a) it complies with the terms and conditions of this Agreement; and

b) its license agreement:

i) effectively disclaims on behalf of all Contributors all warranties and conditions, express and implied, including warranties or conditions of title and non-infringement, and implied warranties or conditions of merchantability and fitness for a particular purpose;

ii) effectively excludes on behalf of all Contributors all liability for damages, including direct, indirect, special, incidental and consequential damages, such as lost profits;

iii) states that any provisions which differ from this Agreement are offered by that Contributor alone and not by any other party; and

iv) states that source code for the Program is available from such Contributor, and informs licensees how to obtain it in a reasonable manner on or through a medium customarily used for software exchange.

When the Program is made available in source code form:

a) it must be made available under this Agreement; and

b) a copy of this Agreement must be included with each copy of the Program.

Contributors may not remove or alter any copyright notices contained within the Program.

Each Contributor must identify itself as the originator of its Contribution, if any, in a manner that reasonably allows subsequent Recipients to identify the originator of the Contribution.

4. COMMERCIAL DISTRIBUTION

Commercial distributors of software may accept certain responsibilities with respect to end users, business partners and the like. While this license is intended to facilitate the commercial use of the Program, the Contributor who includes the Program in a commercial product offering should do so in a manner which does not create potential liability for other Contributors. Therefore, if a Contributor includes the Program in a commercial product offering, such Contributor ("Commercial Contributor") hereby agrees to defend and indemnify every other Contributor ("Indemnified Contributor") against any losses, damages and costs (collectively "Losses") arising from claims, lawsuits and other legal actions brought by a third party against the Indemnified Contributor to the extent caused by the acts or omissions of such Commercial Contributor in connection with its distribution of the Program in a commercial product offering. The obligations in this section do not apply to any claims or Losses relating to any actual or alleged intellectual property infringement. In order to qualify, an Indemnified Contributor must: a) promptly notify the Commercial Contributor in writing of such claim, and b) allow the Commercial Contributor to control, and cooperate with the Commercial Contributor in, the defense and any related settlement negotiations. The Indemnified Contributor may participate in any such claim at its own expense.

For example, a Contributor might include the Program in a commercial product offering, Product X. That Contributor is then a Commercial Contributor. If that Commercial Contributor then makes performance claims, or offers warranties related to Product X, those performance claims and warranties are such Commercial Contributor's responsibility alone. Under this section, the Commercial Contributor would have to defend claims against the other Contributors related to those performance claims and warranties, and if a court requires any other Contributor to pay any damages as a result, the Commercial Contributor must pay those damages.

5. NO WARRANTY

EXCEPT AS EXPRESSLY SET FORTH IN THIS AGREEMENT, THE PROGRAM IS PROVIDED ON AN "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, EITHER EXPRESS OR IMPLIED INCLUDING, WITHOUT LIMITATION, ANY WARRANTIES OR CONDITIONS OF TITLE, NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Each Recipient is solely responsible for determining the appropriateness of using and distributing the Program and assumes all risks associated with its exercise of rights under this Agreement, including but not limited to the risks and costs of program errors, compliance with applicable laws, damage to or loss of data, programs or equipment, and unavailability or interruption of operations.

6. DISCLAIMER OF LIABILITY

EXCEPT AS EXPRESSLY SET FORTH IN THIS AGREEMENT, NEITHER RECIPIENT NOR ANY CONTRIBUTORS SHALL HAVE ANY LIABILITY FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING WITHOUT LIMITATION LOST PROFITS), HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OR DISTRIBUTION OF THE PROGRAM OR THE EXERCISE OF ANY RIGHTS GRANTED HEREUNDER, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

7. GENERAL

If any provision of this Agreement is invalid or unenforceable under applicable law, it shall not affect the validity or enforceability of the remainder of the terms of this Agreement, and without further action by the parties hereto, such provision shall be reformed to the minimum extent necessary to make such provision valid and enforceable.

If Recipient institutes patent litigation against a Contributor with respect to a patent applicable to software (including a cross-claim or counterclaim in a lawsuit), then any patent licenses granted by that Contributor to such Recipient under this Agreement shall terminate as of the date such litigation is filed. In addition, if Recipient institutes patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Program itself (excluding combinations of the Program with other software or hardware) infringes such Recipient's patent(s), then such Recipient's rights granted under Section 2(b) shall terminate as of the date such litigation is filed.

All Recipient's rights under this Agreement shall terminate if it fails to comply with any of the material terms or conditions of this Agreement and does not cure such failure in a reasonable period of time after becoming aware of such noncompliance. If all Recipient's rights under this Agreement terminate, Recipient agrees to cease use and distribution of the Program as soon as reasonably practicable. However, Recipient's obligations under this Agreement and any licenses granted by Recipient relating to the Program shall continue and survive.

Everyone is permitted to copy and distribute copies of this Agreement, but in order to avoid inconsistency the Agreement is copyrighted and may only be modified in the following manner. The Agreement Steward reserves the right to publish new versions

(including revisions) of this Agreement from time to time. No one other than the Agreement Steward has the right to modify this Agreement. IBM is the initial Agreement Steward. IBM may assign the responsibility to serve as the Agreement Steward to a suitable separate entity. Each new version of the Agreement will be given a distinguishing version number. The Program (including Contributions) may always be distributed subject to the version of the Agreement under which it was received. In addition, after a new version of the Agreement is published, Contributor may elect to distribute the Program (including its Contributions) under the new version. Except as expressly stated in Sections 2(a) and 2(b) above, Recipient receives no rights or licenses to the intellectual property of any Contributor under this Agreement, whether expressly, by implication, estoppel or otherwise. All rights in the Program not expressly granted under this Agreement are reserved.

This Agreement is governed by the laws of the State of New York and the intellectual property laws of the United States of America. No party to this Agreement will bring a legal action under this Agreement more than one year after the cause of action arose. Each party waives its rights to a jury trial in any resulting litigation.

The MIT License (MIT)

Copyright (c) 2017 ZHU Kaidi

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Permission is hereby granted, free of charge, to any person or organization obtaining a copy of the software and accompanying documentation covered by this license (the "Software") to use, reproduce, display, distribute, execute, and transmit the Software, and to prepare derivative works of the Software, and to permit third-parties to whom the Software is furnished to do so, all subject to the following:

The copyright notices in the Software and this entire statement, including the above license grant, this restriction and the following disclaimer, must be included in all copies of the Software, in whole or in part, and all derivative works of the Software, unless such copies or derivative works are solely in the form of machine-executable object code generated by a source language processor.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-

INFRINGEMENT. IN NO EVENT SHALL THE COPYRIGHT HOLDERS OR ANYONE DISTRIBUTING THE SOFTWARE BE LIABLE FOR ANY DAMAGES OR OTHER LIABILITY, WHETHER IN CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Copyright (C) 1996-2002 by Jorn Lind-Nielsen All rights reserved

Permission is hereby granted, without written agreement and without license or royalty fees, to use, reproduce, prepare derivative works, distribute, and display this software and its documentation for any purpose, provided that (1) the above copyright notice and the following two paragraphs appear in all copies of the source code and (2) redistributions, including without limitation binaries, reproduce these notices in the supporting documentation. Substantial modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated in all files where they apply.

IN NO EVENT SHALL JORN LIND-NIELSEN, OR DISTRIBUTORS OF THIS SOFTWARE BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THE AUTHORS OR ANY OF THE ABOVE PARTIES HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

JORN LIND-NIELSEN SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

The GAlib source code is not in the public domain, but it is available at no cost. As a work developed by Matthew Wall using MIT resources and MIT funding, the original GAlib source code copyright is owned by the Massachusetts Institute of Technology. Later portions were added by Matthew Wall without MIT funding. All rights are reserved.

GAlib License Agreement

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

You may copy and distribute the source code and/or library/executable code for GAlib in any medium provided that you conspicuously and appropriately give credit to the author and keep intact all copyright and disclaimer notices in the library.

Any publications of work based upon experiments that use GAlib must include a suitable acknowledgement of GAlib. A suggested acknowledgement is: "The software for this work used the GAlib genetic algorithm package, written by Matthew Wall at the Massachusetts Institute of Technology."

The author of GAlib and MIT assume absolutely no responsibility for the use or misuse of GAlib. In no event shall the author of GAlib or MIT be liable for any damages resulting from use or performance of GAlib.

The GIFLIB distribution is Copyright (c) 1997 Eric S. Raymond

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Copyright (c) 1992-2013 The University of Tennessee and The University of Tennessee Research Foundation. All rights reserved. Copyright (c) 2000-2013 The University of California Berkeley. All rights reserved. Copyright (c) 2006-2013 The University of Colorado Denver. All rights reserved.

$COPYRIGHT$

Additional copyrights may follow

$HEADER$

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.

- Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

The copyright holders provide no reassurances that the source code provided does not infringe any patent, copyright, or any other intellectual property rights of third parties. The copyright holders disclaim any liability to any recipient for claims brought against recipient by any third party for infringement of that parties intellectual property rights.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR

BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright (c) 2003-2009 Tim Kientzle All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met: 1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer in this position and unchanged. 2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE AUTHOR(S) ``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR(S) BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright (C) 2002-2011 Carsten Haitzler and various contributors (see AUTHORS)

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright 1990 - 1997 by AT&T, Lucent Technologies and Bellcore.

Permission to use, copy, modify, and distribute this software and its documentation for any purpose and without fee is hereby granted, provided that the above copyright notice appear in all copies and that both that the copyright notice and this permission notice and warranty disclaimer appear in supporting documentation, and that the names of AT&T, Bell Laboratories, Lucent or Bellcore or any of their entities not be used in advertising or publicity pertaining to distribution of the software without specific, written prior permission.

AT&T, Lucent and Bellcore disclaim all warranties with regard to this software, including all implied warranties of merchantability and fitness. In no event shall AT&T, Lucent or Bellcore be liable for any special, indirect or consequential damages or any damages whatsoever resulting from loss of use, data or profits, whether in an action of contract, negligence or other tortious action, arising out of or in connection with the use or performance of this software.

LIbigl is licensed under the Mozilla Public License (MPL) Version 2.0, the text of which can be found at: https://www.mozilla.org/media/MPL/2.0/index.815ca599c9df.txt. Please contact open.source@ansys.com for a copy of the Libigl source code.

This software is based in part on the work of the Independent JPEG Group.

Copyright 1995-2013, Regents of the University of Minnesota

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License.

You may obtain a copy of the License at

http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Copyright (C) 2002 Michael Ringgaard. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

3. Neither the name of the project nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright (c) 2005 Nominet UK (www.nic.uk)

All rights reserved.

Contributors: Ben Laurie, Rachel Willmer. The Contributors have asserted their moral rights under the UK Copyright Design and Patents Act 1988 to be recorded as the authors of this copyright work.

This is a BSD-style Open Source licence.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

3. The name of Nominet UK or the contributors may not be used to endorse or promote products derived from this software without specific prior written permission;

and provided that the user accepts the terms of the following disclaimer:

THIS SOFTWARE IS PROVIDED BY NOMINET UK AND CONTRIBUTORS ``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,

THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL NOMINET UK OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright (c) 1998-2008 The OpenSSL Project. All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

3. All advertising materials mentioning features or use of this software must display the following acknowledgment: "This product includes software developed by the OpenSSL Project for use in the OpenSSL Toolkit. (http://www.openssl.org/)"

4. The names "OpenSSL Toolkit" and "OpenSSL Project" must not be used to endorse or promote products derived from this software without prior written permission. For written permission, please contact openssl-core@openssl.org.

5. Products derived from this software may not be called "OpenSSL" nor may "OpenSSL" appear in their names without prior written permission of the OpenSSL Project.

6. Redistributions of any form whatsoever must retain the following acknowledgment: "This product includes software developed by the OpenSSL Project for use in the OpenSSL Toolkit (http://www.openssl.org/)"

THIS SOFTWARE IS PROVIDED BY THE OpenSSL PROJECT ``AS IS'' AND ANY EXPRESSED OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE OpenSSL PROJECT OR ITS CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT

(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
==========

This product includes cryptographic software written by Eric Young (eay@cryptsoft.com). This product includes software written by Tim Hudson (tjh@cryptsoft.com).

Original SSLeay License ----------

Copyright (C) 1995-1998 Eric Young (eay@cryptsoft.com) All rights reserved.

This package is an SSL implementation written by Eric Young (eay@cryptsoft.com). The implementation was written so as to conform with Netscapes SSL.

This library is free for commercial and non-commercial use as long as the following conditions are aheared to. The following conditions apply to all code found in this distribution, be it the RC4, RSA, lhash, DES, etc., code; not just the SSL code. The SSL documentation included with this distribution is covered by the same copyright terms except that the holder is Tim Hudson (tjh@cryptsoft.com).

Copyright remains Eric Young's, and as such any Copyright notices in the code are not to be removed. If this package is used in a product, Eric Young should be given attribution as the author of the parts of the library used. This can be in the form of a textual message at program startup or in documentation (online or textual) provided with the package.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met: 1. Redistributions of source code must retain the copyright notice, this list of conditions and the following disclaimer. 2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution. 3. All advertising materials mentioning features or use of this software must display the following acknowledgement: "This product includes cryptographic software written by Eric Young (eay@cryptsoft.com)" The word 'cryptographic' can be left out if the rouines from the library being used are not cryptographic related :-). 4. If you include any Windows specific code (or a derivative thereof) from the apps directory (application code) you must include an acknowledgement: "This product includes software written by Tim Hudson (tjh@cryptsoft.com)"

THIS SOFTWARE IS PROVIDED BY ERIC YOUNG ``AS IS'' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE

USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

The licence and distribution terms for any publically available version or derivative of this code cannot be changed. i.e. this code cannot simply be copied and put under another distribution licence [including the GNU Public Licence.]

Redistribution and use in source and binary forms, with or without

modification, are permitted provided that the following conditions are met:

Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

Neither the name of the University of Cambridge nor the name of Google Inc. nor the names of their contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THEIMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSEARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BELIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, ORCONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OFSUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESSINTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER INCONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THEPOSSIBILITY OF SUCH DAMAGE.

Copyright (c) 2009, Krasnoshchekov Petr

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

Neither the name of the <organization> nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY Krasnoshchekov Petr "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL Krasnoshchekov Petr BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright [Copyright (c) 1994-1996 The Regents of the Univ. of California. All rights reserved.

Permission is hereby granted, without written agreement and without license or royalty fees, to use, copy, modify, and distribute this software and its documentation for any purpose, provided that the above copyright notice and the following two paragraphs appear in all copies of this software.

IN NO EVENT SHALL THE UNIVERSITY OF CALIFORNIA BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE AND ITS DOCUMENTATION, EVEN IF THE UNIVERSITY OF CALIFORNIA HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE UNIVERSITY OF CALIFORNIA SPECIFICALLY DISCLAIMS ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE SOFTWARE PROVIDED HEREUNDER IS ON AN "AS IS" BASIS, AND THE UNIVERSITY OF CALIFORNIA HAS NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.]

This program is based in part on the work of the Qwt project (http://qwt.sf.net).

Copyright (c) 1985,86,87,88 by Kenneth S. Kundert and the University of California.

Permission to use, copy, modify, and distribute this software and its documentation for any purpose and without fee is hereby granted, provided that the copyright notices appear in all copies and supporting documentation and that the authors and the University of California are properly credited. The authors and the University of California make no representations as to the suitability of this software for any purpose. It is provided `as is', without express or implied warranty.

$Date: 88/06/18 11:13:29 $ $Revision: 1.2 $

Copyright (c) 2003, The Regents of the University of California, through Lawrence Berkeley National Laboratory (subject to receipt of any required approvals from U.S. Dept. of Energy)

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

(1) Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer. (2) Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution. (3) Neither the name of Lawrence Berkeley National Laboratory, U.S. Dept. of Energy nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright (C) 1996, Makoto Matsumoto All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

3. The names of its contributors may not be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Copyright, and License:

Copyright (c) 2005 by Timothy A. Davis, University of Florida, davis@cise.ufl.edu. All Rights Reserved.

UMFPACK License:

Your use or distribution of UMFPACK or any derivative code implies that you agree to this License.

THIS MATERIAL IS PROVIDED AS IS, WITH ABSOLUTELY NO WARRANTY EXPRESSED OR IMPLIED. ANY USE IS AT YOUR OWN RISK.

Permission is hereby granted to use or copy this program, provided that the Copyright, this License, and the Availability of the original version is retained on all copies. User documentation of any code that uses UMFPACK or any modified version of UMFPACK code must cite the Copyright, this License, the Availability note, and "Used by permission." Permission to modify the code and to distribute modified code is granted, provided the Copyright, this License, and the Availability note are retained, and a notice that the code was modified is included. This software was developed with support from the National Science Foundation, and is provided to you free of charge.

Most files in this release are marked with the copyrights of the organizations who have edited them. The copyrights below are in no particular order and generally reflect members of the Open MPI core team who have contributed code to this release. The copyrights for code used under license from other parties are included in the corresponding files.

Copyright (c) 2004-2010 The Trustees of Indiana University and Indiana University Research and Technology Corporation. All rights reserved.

Copyright (c) 2004-2010 The University of Tennessee and The University of Tennessee Research Foundation. All rights reserved.

Copyright (c) 2004-2010 High Performance Computing Center Stuttgart, University of Stuttgart. All rights reserved.

Copyright (c) 2004-2008 The Regents of the University of California. All rights reserved.

Copyright (c) 2006-2010 Los Alamos National Security, LLC. All rights reserved.

Copyright (c) 2006-2010 Cisco Systems, Inc. All rights reserved.

Copyright (c) 2006-2010 Voltaire, Inc. All rights reserved.

Copyright (c) 2006-2011 Sandia National Laboratories. All rights reserved.

Copyright (c) 2006-2010 Sun Microsystems, Inc. All rights reserved. Use is subject to license terms.

Copyright (c) 2006-2010 The University of Houston. All rights reserved.

Copyright (c) 2006-2009 Myricom, Inc. All rights reserved.

Copyright (c) 2007-2008 UT-Battelle, LLC. All rights reserved.

Copyright (c) 2007-2010 IBM Corporation. All rights reserved.

Copyright (c) 1998-2005 Forschungszentrum Juelich, Juelich Supercomputing Centre, Federal Republic of Germany

Copyright (c) 2005-2008 ZIH, TU Dresden, Federal Republic of Germany

Copyright (c) 2007 Evergrid, Inc. All rights reserved.

Copyright (c) 2008 Chelsio, Inc. All rights reserved.

Copyright (c) 2008-2009 Institut National de Recherche en Informatique. All rights reserved.

Copyright (c) 2007 Lawrence Livermore National Security, LLC. All rights reserved.

Copyright (c) 2007-2009 Mellanox Technologies. All rights reserved.

Copyright (c) 2006-2010 QLogic Corporation. All rights reserved.

Copyright (c) 2008-2010 Oak Ridge National Labs. All rights reserved.

Copyright (c) 2006-2010 Oracle and/or its affiliates. All rights reserved.

Copyright (c) 2009 Bull SAS. All rights reserved.

Copyright (c) 2010 ARM ltd. All rights reserved.

Copyright (c) 2010-2011 Alex Brick <bricka (a] ccs.neu.edu>. All rights reserved.

Copyright (c) 2012 The University of Wisconsin-La Crosse. All rights reserved.

$COPYRIGHT$

Additional copyrights may follow

$HEADER$

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.

Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

The copyright holders provide no reassurances that the source code provided does not infringe any patent, copyright, or any other intellectual property rights of third parties. The copyright holders disclaim any liability to any recipient for claims brought against recipient by any third party for infringement of that parties intellectual property rights.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS “AS IS” AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, and other parties. The following terms apply to all files associated with the software, unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN “AS IS” BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only “Restricted Rights” in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as “Commercial Computer Software” and the Government shall have only “Restricted Rights” as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

This file is subject to the following copyright notice, which is different from the notice used elsewhere in Tcl, but roughly equivalent in meaning.

Copyright (c) 1992,1993,1995,1996, Jens-Uwe Mager, Helios Software GmbH

Not derived from licensed software.

Permission is granted to freely use, copy, modify, and redistribute this software, provided that the author is not construed to be liable for any results of using the software, alterations are clearly marked as such, and this notice is not modified.

Copyright (c) 1987-1994 The Regents of the University of California.

Copyright (c) 1994-1995 Sun Microsystems, Inc.

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN “AS IS” BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

RESTRICTED RIGHTS: Use, duplication or disclosure by the government is subject to the restrictions as set forth in subparagraph (c) (1) (ii) of the Rights in Technical Data and Computer Software Clause as DFARS 252.227-7013 and FAR 52.227-19.

Copyright (c) 1993-1998 Lucent Technologies, Inc.

Permission to use, copy, modify, and distribute this software and its documentation for any purpose and without fee is hereby granted, provided that the above copyright notice appear in all copies and that both that the copyright notice and warranty disclaimer appear in supporting documentation, and that the names of Lucent Technologies any of their entities not be used in advertising or publicity pertaining to distribution of the software without specific, written prior permission.

Lucent disclaims all warranties with regard to this software, including all implied warranties of merchantability and fitness. In no event shall Lucent be liable for any special, indirect or consequential damages or any damages whatsoever resulting from loss of use, data or profits, whether in an action of contract, negligence or other tortuous action, arising out of or in connection with the use or performance of this software.

Gifsicle is copyright (C) 1997-2017 Eddie Kohler and licensed under GNU General Public License (GPL) Version 2. Please contact open.source@ansys.com for a copy of the Gifsicle source code.

G Wave is copyright (C) Steve Tell and licensed under GNU General Public License (GPL) Version 2. Please contact open.source@ansys.com for a copy of the GNU Readline source code.

GNU Readline is copyright (C) Free Software Foundation and licensed under GNU General Public License (GPL) Version 2. Please contact open.source@ansys.com for a copy of the GNU Readline source code.

GNU GENERAL PUBLIC LICENSE

Version 2, June 1991

Copyright (C) 1989, 1991 Free Software Foundation, Inc.

51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users. This General Public License applies to most of the Free Software Foundation's software and to any other program whose authors commit to using it. (Some other Free Software Foundation software is covered by the GNU Lesser General Public License instead.) You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too,

receive or can get the source code. And you must show them these terms so they know their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that redistributors of a free program will individually obtain patent licenses, in effect making the program proprietary. To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License. The “Program”, below, refers to any such program or work, and a “work based on the Program” means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term “modification”.) Each licensee is addressed as “you”.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

1. You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and give any other recipients of the Program a copy of this License along with the Program.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

a) You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.

b) You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License.

c) If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this License.

(Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above, provided that you also do one of the following:

a) Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

b) Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,

c) Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.

4. You may not copy, modify, sub license, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sub license or distribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

5. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative

works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

7. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Program at all. For example, if a patent license would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Program under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

9. The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and “any later version”, you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you may choose any version ever published by the Free Software Foundation.

10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

11. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS

CSPARSE is Copyright (C) 2006, Timothy Davis and licensed under GNU Library General Public License (LGPL) Version 2.1. Please contact open.source@ansys.com for a copy of the CSPARSE source code.

GDK, GTK+, and GLib are Copyright (C) The GNOME Project and licensed under GNU Library General Public License (LGPL) Version 2.1. Please contact open.source@ansys.com for a copy of the GDK or GLib source code.

Libart is Copyright (C) Raph Levien and licensed under GNU Library General Public License (LGPL) Version 2.1. Please contact open.source@ansys.com for a copy of the Libart source code.

The Qt GUI Toolkit is Copyright (C) 2015 The Qt Company Ltd. and licensed under GNU Library General Public License (LGPL) Version 2.1. Please contact open.source@ansys.com for a copy of the Qt GUI Toolkit source code.

GNU LESSER GENERAL PUBLIC LICENSE

Version 2.1, February 1999

Copyright (C) 1991, 1999 Free Software Foundation, Inc.

51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

[This is the first released version of the Lesser GPL. It also counts as the successor of the GNU Library Public License, version 2, hence the version number 2.1.]

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licenses are intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users.

This license, the Lesser General Public License, applies to some specially designated software packages--typically libraries--of the Free Software Foundation and other authors who decide to use it. You can use it too, but we suggest you first think carefully about whether this license or the ordinary General Public License is the better strategy to use in any particular case, based on the explanations below.

When we speak of free software, we are referring to freedom of use, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish); that you receive source code or can get it if you want it; that you can change the software and use pieces of it in new free programs; and that you are informed that you can do these things.

To protect your rights, we need to make restrictions that forbid distributors to deny you these rights or to ask you to surrender these rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link other code with the library, you must provide complete object files to the recipients, so that they can relink them with the library after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

We protect your rights with a two-step method: (1) we copyright the library, and (2) we offer you this license, which gives you legal permission to copy, distribute and/or modify the library.

To protect each distributor, we want to make it very clear that there is no warranty for the free library. Also, if the library is modified by someone else and passed on, the recipients should know that what they have is not the original version, so that the original author's reputation will not be affected by problems that might be introduced by others.

Finally, software patents pose a constant threat to the existence of any free program. We wish to make sure that a company cannot effectively restrict the users of a free program by obtaining a restrictive license from a patent holder. Therefore, we insist that any patent license obtained for a version of the library must be consistent with the full freedom of use specified in this license.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License. This license, the GNU Lesser General Public License, applies to certain designated libraries, and is quite different from the ordinary General Public License. We use this license for certain libraries in order to permit linking those libraries into non-free programs.

When a program is linked with a library, whether statically or using a shared library, the combination of the two is legally speaking a combined work, a derivative of the original library. The ordinary General Public License therefore permits such linking only if the

entire combination fits its criteria of freedom. The Lesser General Public License permits more lax criteria for linking other code with the library.

We call this license the “Lesser” General Public License because it does Less to protect the user's freedom than the ordinary General Public License. It also provides other free software developers Less of an advantage over competing non-free programs. These disadvantages are the reason we use the ordinary General Public License for many

libraries. However, the Lesser license provides advantages in certain special circumstances.

For example, on rare occasions, there may be a special need to encourage the widest possible use of a certain library, so that it becomes a de-facto standard. To achieve this, non-free programs must be allowed to use the library. A more frequent case is that a free library does the same job as widely used non-free libraries. In this case, there is little to gain by limiting the free library to free software only, so we use the Lesser General Public License.

In other cases, permission to use a particular library in non-free programs enables a greater number of people to use a large body of free software. For example, permission to use the GNU C Library in non-free programs enables many more people to use the whole GNU operating system, as well as its variant, the GNU/Linux operating system.

Although the Lesser General Public License is Less protective of the users' freedom, it does ensure that the user of a program that is linked with the Library has the freedom and the wherewithal to run that program using a modified version of the Library.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a “work based on the library” and a “work that uses the library”. The former contains code derived from the library, whereas the latter must be combined with the library in order to run.

GNU LESSER GENERAL PUBLIC LICENSE TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Lesser General Public License (also called “this License”). Each licensee is addressed as “you”.

A “library” means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The “Library”, below, refers to any such software library or work which has been distributed under these terms. A “work based on the Library” means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term “modification”.)

“Source code” for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

a) The modified work must itself be a software library.

b) You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.

c) You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.

d) If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful. (For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices. Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy. This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the

requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a “work that uses the Library”. Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

However, linking a “work that uses the Library” with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a “work that uses the library”. The executable is therefore covered by this License.

Section 6 states terms for distribution of such executables.

When a “work that uses the Library” uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law. If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.) Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also combine or link a “work that uses the Library” with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

a) Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable “work that uses the Library”, as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)

b) Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (1) uses at run time a copy of the library already present on the user's computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.

c) Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.

d) If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.

e) Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the “work that uses the Library” must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.

b) Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

8. You may not copy, modify, sub license, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sub license, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties with this License.

11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

13. The Free Software Foundation may publish revised and/or new versions of the Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and “any later version”, you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

15. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY “AS IS” WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS